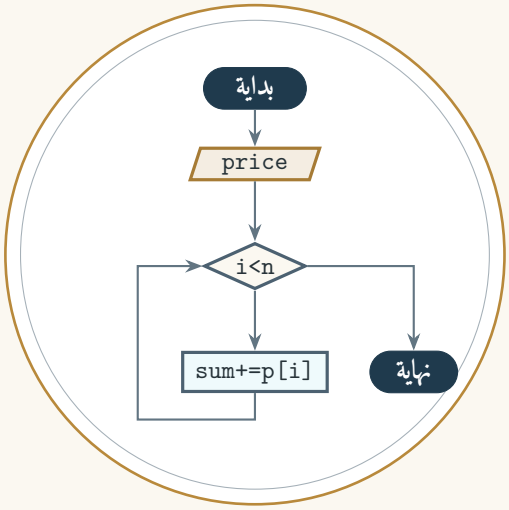
سلسلة أساسيات البرمجة

الخوارزميات ولغة C
كتاب التمارين المحلولة

650 تمرينًا محلولًا بالتفصيل من التجارة الإلكترونية وعلم البيانات



13	650	3
فصلًا	تمرينًا	مستويات

د. بن عبد الرزاق يوسف

طلبة السنة الأولى • 2026

الخوارزميات ولغة C: كتاب التمارين المحلولة

تأليف: د. بن عبد الرزاق يوسف
الطبعة: الأولى، 2026
ردمك (ISBN): .....
الإيداع القانوني: .....

© 2026 د. بن عبد الرزاق يوسف. جميع الحقوق محفوظة للمؤلف. لا يجوز نسخ أي جزء من هذا الكتاب أو نشره أو تخزينه أو نقله بأي وسيلة كانت، إلكترونية أو ميكانيكية، بالتصوير أو التسجيل أو غير ذلك، دون إذن كتابي مسبق من المؤلف، باستثناء الاقتباسات القصيرة لأغراض التعليم والنقد مع ذكر المصدر.

جميع البرامج الواردة في الكتاب تُرجمت بالمترجم gcc وفق المعيار C99 ونُفّذت، والنتائج المعروضة هي نتائج التنفيذ الحقيقية. أسماء المنتجات والمتاجر في التمارين خيالية.

المحتويات

مقدمة xvii

1 مدخل إلى الخوارزميات: المتغيرات والعمليات 1

1.1 السعر مع الرسم على القيمة المضافة . . . 2
1.2 مبلغ سطر في الفاتورة . . . 3
1.3 متوسط تقييمات منتج . . . 4
1.4 تحويل العملة . . . 5
1.5 الوزن الإجمالي للشحنة . . . 6
1.6 التخفيض الموسمي . . . 7
1.7 الربح وهامش الربح . . . 8
1.8 تبديل سعرين . . . 9
1.9 مدة جلسة التصفح . . . 10
1.10 معدل التحويل . . . 11
1.11 عدد علب التغليف . . . 13
1.12 المسافة بين زبونين . . . 14
1.13 المتوسط والتباين . . . 15
1.14 التطبيع بين 0 و1 . . . 16
1.15 الدرجة المعيارية . . . 17
1.16 متوسط النجوم الموزون . . . 18
1.17 نمو رقم الأعمال . . . 19
1.18 تفكيك مبلغ إلى أوراق نقدية . . . 20
1.19 ساعة وصول الطلبية . . . 22
1.20 مدة الاشتراك بالأسابيع . . . 23
1.21 عكس رمز المنتج . . . 24
1.22 مجموع التحقق لرقم الطلبية . . . 25
1.23 تقريب السعر إلى منزلتين . . . 26
1.24 مؤشرات الحملة الإعلانية . . . 27
1.25 القيمة الدائمة للزبون . . . 29
1.26 متوسط قيمة الطلبية . . . 30
1.27 أيام تغطية المخزون . . . 31
1.28 التنبؤ بنموذج خطي . . . 32
1.29 مستقيم يمر بنقطتين . . . 33
1.30 حصص الأصناف من رقم الأعمال . . . 34
1.31 الطابع الزمني إلى ساعة . . . 35
1.32 فاتورة كاملة . . . 36
1.33 مقارنة سعر الكيلوغرام . . . 38
1.34 نقاط الوفاء . . . 39
1.35 متوسط مربع الخطأ . . . 40
1.36 السعر بعد التضخم . . . 41

1.37 تبديل دون متغير مساعد 43
1.38 المبلغ بالسنتيم 44
1.39 رموز الأصناف والحروف 45
1.40 أسبوع ويوم التسليم 46
1.41 الأكبر بين سعرين دون شرط 48
1.42 الانحراف المعياري بطريقة نيوتن 49
1.43 التغاير بين السعر والمبيعات 50
1.44 انحدار خطي بالمربعات الصغرى 52
1.45 مدة المشاهدة الكلية 53
1.46 خصائص المنتج بالبتات 54
1.47 المتوسط المتحرك الأسي 55
1.48 رقم التحقق لرمز EAN-8 57
1.49 مرونة الطلب السعرية 58
1.50 تعبئة حاوية الشحن 59

2 البنيات الشرطية 63
2.1 الشحن المجاني 64
2.2 توفر المنتج في المخزون 65
2.3 رقم طلبية زوجي أم فردي 66
2.4 الأرخص بين عرضين 67
2.5 التحقق من التقييم 68
2.6 ربح أم خسارة 69
2.7 رمز القسيمة 70
2.8 بيع منتج محدد بالسن 71
2.9 طريقة الدفع 72
2.10 نجاح تجربة A/B 74
2.11 تخفيض حسب الشرائح 76
2.12 تكلفة الشحن حسب الوزن 77
2.13 أفضل تقييم بين ثلاثة منتجات 79
2.14 ترتيب ثلاثة أسعار 81
2.15 السنة الكبيسة 83
2.16 عدد أيام الشهر 85
2.17 تصنيف الزبائن حسب الإنفاق 86
2.18 الرسم حسب صنف المنتج 88
2.19 كشف القيم الشاذة 89
2.20 هيمنة منتج على آخر 91
2.21 التقييم إلى نجوم 92
2.22 نقطة التعادل 94
2.23 صحة تاريخ الطلبية 96
2.24 اليوم الموالي 98
2.25 خلية مصفوفة الالتباس 100
2.26 قاعدة توصية 101
2.27 منطقة التوصيل 103
2.28 عرض 3 بثمن 2 104

2.29 تداخل فترتي ترويج 105
2.30 آلة حاسبة للأسعار 107
2.31 تأجيل التوصيل في عطلة نهاية الأسبوع 108
2.32 صلاحية البطاقة البنكية 110
2.33 هدية حسب حجم السلة 111
2.34 تطور مؤشر الأداء 113
2.35 السعر الأمثل 115
2.36 الوسيط بين ثلاث قيم 116
2.37 تسعير حسب ساعة الطلب 117
2.38 تصنيف المشاعر 119
2.39 كشف الاحتيال بالقواعد 120
2.40 الفائز في تجربة A/B 122
2.41 يوم الأسبوع لتاريخ ما 125
2.42 شجرة قرار للتنبؤ بمغادرة الزبون 126
2.43 ترتيب أربعة أسعار بشبكة مقارنات 127
2.44 الضريبة التصاعدية على دخل البائع 129
2.45 عدد الأيام بين تاريخين 131
2.46 اختيار شركة الشحن الأرخص 134
2.47 هل العنوان داخل منطقة التوصيل؟ 135
2.48 أقرب جار لتصنيف زبون جديد 137
2.49 محرك التسعير الكامل 139
2.50 توازن العرض والطلب 141

3 الحلقات التكرارية **145**

3.1 مجموع أسعار السلة 147
3.2 عدد الطلبيات الكبيرة 148
3.3 متوسط التقييمات 150
3.4 جدول الأسعار حسب الكمية 152
3.5 العد التنازلي للتخفيضات 153
3.6 الإدخال حتى الصفر 155
3.7 أغلى منتج 156
3.8 أرقام الطلبيات الزوجية 158
3.9 الادخار لشراء حاسوب 160
3.10 إعادة طلب تقييم صالح 162
3.11 الأصغر والأكبر والمدى 164
3.12 المتوسط والتباين في مرور واحد 165
3.13 الجذر التربيعي بطريقة نيوتن 167
3.14 ترتيبات عرض المنتجات 169
3.15 عدد الحزم الممكنة 171
3.16 أكبر قاسم مشترك لأحجام العبوات 173
3.17 تزامن دورات التموين 174
3.18 هل رقم المنتج أولي؟ 176
3.19 أرقام رقم الطلبية 177
3.20 رمز تخفيض متناظر 179

3.21 نمو المستخدمين بمتتالية فيبوناتشي 181
3.22 الفائدة المركبة على عدة سنوات 183
3.23 جدول تسديد الشراء بالتقسيط 185
3.24 أطول فترة نمو متواصل 186
3.25 المتوسط المتحرك لثلاثة أيام 188
3.26 مدرج تكراري للتقييمات 190
3.27 رقم المنتج بالنظام الثنائي 192
3.28 شبكة التخفيضات 194
3.29 عد الزبائن في كل شريحة 196
3.30 محاكاة نفاد المخزون 198
3.31 التنعيم الأسي لسلسلة زمنية 200
3.32 الانحدار الخطي على n نقطة 202
3.33 معامل الارتباط 204
3.34 ثاني أغلى منتج 206
3.35 هرم مستويات الوفاء 208
3.36 سعر التوازن بالتنصيف 210
3.37 الفائدة المستمرة بسلسلة تايلور 212
3.38 القفزات المفاجئة في الزيارات 214
3.39 تخمين السعر بالبحث الثنائي 216
3.40 يوم بلوغ هدف رقم الأعمال 218
3.41 الأرقام الأولية حتى N 221
3.42 التجميع بـ K-Means في بعد واحد 222
3.43 محاكاة مونت كارلو بمولد عشوائي يدوي 224
3.44 الانحدار بالنزول التدريجي 226
3.45 التحقق من رقم البطاقة بخوارزمية Luhn 228
3.46 مثلث باسكال لعدد الحزم 230
3.47 التحويل إلى النظام الست عشري 232
3.48 الجذر الصحيح بالبحث الثنائي 234
3.49 محاكاة سياسة التموين (s, S) 236
3.50 أفضل يوم للشراء والبيع 238

4 الجداول أحادية البعد 241
4.1 عرض كتالوج الأسعار 242
4.2 مبيعات الأسبوع 243
4.3 أكثر منتج مبيعاً 245
4.4 المنتجات النافدة 247
4.5 رفع كل الأسعار 249
4.6 آخر الطلبيات أولًا 251
4.7 البحث عن منتج 253
4.8 نسخ المنتجات الفاخرة 255
4.9 نسبة التقييمات الإيجابية 257
4.10 الفاتورة من جدولين 259
4.11 عكس الجدول في مكانه 262
4.12 تدوير جدول المناوبات 263

4.13 إدراج منتج جديد في موضع 265
4.14 حذف منتج متوقف 267
4.15 الترتيب بالفقاعات 269
4.16 الترتيب بالانتقاء 271
4.17 الترتيب بالإدراج 273
4.18 البحث الثنائي في كتالوج مرتب 275
4.19 وسيط أسعار السوق 277
4.20 المنوال بجدول عدادات 279
4.21 توزيع الطلبيات حسب الساعة 281
4.22 حذف المكرر من جدول مرتب 283
4.23 دمج قائمتي أسعار مرتبتين 285
4.24 المجاميع التراكمية واستعلام الفترات 287
4.25 المتوسط المتحرك بنافذة k 289
4.26 تطبيع جدول الأسعار 291
4.27 التوحيد المعياري للبيانات 293
4.28 المئين التسعون لزمن التوصيل 295
4.29 منتجان بقيمة بطاقة الهدية 296
4.30 التشابه بين زبونين 298
4.31 أكبر هبوط يومي 300
4.32 القيم الشاذة بطريقة المدى الربيعي 302
4.33 نقل المنتجات النافدة إلى الآخر 304
4.34 غربال إراتوستينس 306
4.35 عدد المنتجات المختلفة في سلة 308
4.36 مطابقة جردين 310
4.37 البتات في جدول 312
4.38 جمع أعداد ضخمة 314
4.39 أطول فترة صعود في جدول 316
4.40 الدرجة الموزونة للمنتج 318
4.41 أفضل فترة ربح متتالية (خوارزمية كادان) 321
4.42 الترتيب السريع دون عودية 322
4.43 الترتيب بالدمج من الأسفل إلى الأعلى 324
4.44 الترتيب بالعد لمليون تقييم 326
4.45 تصنيف زبون بـ k أقرب جيران 328
4.46 تجميع الزبائن إلى ثلاث فئات 330
4.47 اختيار الحملات الإعلانية بميزانية محدودة 332
4.48 المنتجات التي تُشترى معًا 334
4.49 معامل جيني لتركز الإنفاق 336
4.50 أطول فترة ضمن الميزانية 338

5 الجداول ثنائية البعد 341
5.1 جدول مبيعات المتاجر 342
5.2 المجموع الكلي للمبيعات 343
5.3 مجموع كل متجر 345
5.4 مجموع كل يوم 347

5.5 أفضل خانة في المصفوفة . . . 349
5.6 الرفوف الفارغة في المستودع . . . 351
5.7 تعديل كل الأسعار في المصفوفة . . . 353
5.8 قراءة مصفوفة تقييمات . . . 355
5.9 القطر الرئيسي لمصفوفة التحويلات . . . 357
5.10 البحث عن منتج في الرفوف . . . 359
5.11 منقول المصفوفة . . . 362
5.12 جمع مبيعات شهرين . . . 363
5.13 جداء المصفوفات: كلفة الطلبيات عند الموردين . . . 365
5.14 فاتورة كل زبون بجداء مصفوفة في شعاع . . . 367
5.15 أفضل متجر في كل يوم . . . 369
5.16 متوسط تقييم كل منتج دون الخانات الفارغة . . . 371
5.17 تطبيع أعمدة جدول الخصائص . . . 373
5.18 مصفوفة الارتباط . . . 375
5.19 مصفوفة المسافات بين الزبائن . . . 377
5.20 مصفوفة الالتباس والدقة . . . 379
5.21 الدقة والاستدعاء لكل صنف . . . 381
5.22 تناظر مصفوفة الشراء المشترك . . . 383
5.23 حساب الشراء المشترك من السلات . . . 385
5.24 أقرب رف فارغ . . . 387
5.25 خريطة حرارية للطلبيات . . . 389
5.26 جدول محوري من السجلات . . . 391
5.27 مجاميع المستطيلات بالمجاميع التراكمية الثنائية . . . 393
5.28 تدوير صورة المنتج . . . 395
5.29 تصغير صورة إلى مصغّرة . . . 397
5.30 تنعيم صورة بمرشح المتوسط . . . 399
5.31 سلسلة ماركوف لحالات الزبائن . . . 401
5.32 جدول المسافات المثلثي . . . 403
5.33 المستودع الأرخص لكل زبون . . . 405
5.34 كثافة مصفوفة متفرقة . . . 407
5.35 عدد مسارات الروبوت في المستودع . . . 409
5.36 أرخص مسار للتوصيل في المدينة . . . 411
5.37 أسعار الوحدة من ثلاث فواتير (قاعدة كرامر) . . . 413
5.38 ترتيب المتاجر حسب مجموع مبيعاتها . . . 415
5.39 حجز فترات توصيل متتالية . . . 417
5.40 قوة مصفوفة الانتقال . . . 419
5.41 حل جملة معادلات بطريقة غاوس . . . 422
5.42 مقلوب مصفوفة بطريقة غاوس-جوردان . . . 423
5.43 ترتيب صفحات المنتجات بخوارزمية PageRank . . . 425
5.44 أقصر المسافات بين كل المستودعات (فلويد-وارشال) . . . 427
5.45 أقصر طريق للتوصيل (ديكسترا) . . . 429
5.46 الانحدار الخطي المتعدد بالمعادلات الطبيعية . . . 431
5.47 تجميع الزبائن في بعدين . . . 433
5.48 التنبؤ بتقييم مفقود (الترشيح التعاوني) . . . 435

5.49 مناطق التوصيل المتصلة (ملء المناطق) 437
5.50 أطول مسار تصفح مشترك 440

6 السلاسل النصية **443**
6.1 طول اسم المنتج 444
6.2 عدد كلمات التعليق 445
6.3 رمز المنتج بالحروف الكبيرة 446
6.4 عرض الاسم معكوسًا 448
6.5 عدد الأرقام في رمز ترويجي 449
6.6 نسخ سلسلة يدويًا 451
6.7 الاسم الكامل للزبون 452
6.8 مطابقة كلمة المرور 454
6.9 علامات التعجب في التعليقات 455
6.10 تحقق بسيط من البريد الإلكتروني 457
6.11 اسم متناظر 459
6.12 أطول كلمة في الوصف 460
6.13 تنسيق أسماء المنتجات 462
6.14 حذف الفراغات الزائدة 464
6.15 تحويل نص إلى عدد صحيح 466
6.16 تحويل عدد صحيح إلى نص 468
6.17 قراءة سعر عشري من نص 470
6.18 رابط صديق لمحركات البحث 472
6.19 البحث عن كلمة مفتاحية 474
6.20 حجب الكلمات المسيئة 476
6.21 تكرار الحروف في الأوصاف 478
6.22 أسماء متجانسة الحروف 480
6.23 تشفير رمز القسيمة بإزاحة قيصر 482
6.24 قوة كلمة المرور 484
6.25 التحقق من رقم الهاتف 485
6.26 تفكيك سطر CSV 487
6.27 ترتيب أسماء المنتجات أبجديًا 489
6.28 ضغط الرموز بالترميز الطولي 491
6.29 إخفاء رقم البطاقة 493
6.30 تنسيق المبالغ بفواصل الآلاف 495
6.31 قراءة تاريخ بصيغة ISO 497
6.32 استخراج الوسوم من التعليقات 499
6.33 تحليل المشاعر بالكلمات المفتاحية 501
6.34 الكلمات المختلفة في تعليق 503
6.35 أطول بادئة مشتركة لرموز المنتجات 505
6.36 توازن الأقواس في ملف إعدادات 507
6.37 لون المنتج من الشفرة الست عشرية 509
6.38 سهولة قراءة وصف المنتج 511
6.39 مسافة هامينغ بين رمزين 513
6.40 تحليل معاملات رابط البحث 515

6.41 مسافة التحرير بين كلمتي بحث 518
6.42 البحث السريع عن نمط بخوارزمية KMP 520
6.43 أطول جزء مشترك بين وصفين 521
6.44 أكثر الكلمات تكرارًا في التعليقات 523
6.45 حساب عبارة سعر مكتوبة كنص 525
6.46 مطابقة رموز المنتجات بالأحرف البديلة 527
6.47 أطول اسم متناظر داخل نص 529
6.48 ترميز Base64 لمفتاح الواجهة البرمجية 531
6.49 تنسيق وصف المنتج بعرض ثابت 533
6.50 ضغط النص بالقاموس على طريقة LZW 535

7 الدوال 539
7.1 دالة السعر مع الرسم على القيمة المضافة 540
7.2 دالة الأكبر بين عرضين 541
7.3 دالة تحويل العملة 542
7.4 دالة توفر المنتج في المخزون 543
7.5 دالة رسم خط فاصل في الفاتورة 545
7.6 دالة متوسط ثلاثة تقييمات 546
7.7 دالة تكلفة التوصيل حسب المسافة 546
7.8 دالة المربع في حساب الخطأ 547
7.9 دالة السنة الكبيسة 549
7.10 دالة النسبة المئوية للتغير 550
7.11 دالة الجذر التربيعي بطريقة نيوتن 552
7.12 دالة القوة للفائدة المركبة 553
7.13 دالة مجموع جدول ومتوسطه 554
7.14 دالة البحث عن منتج بالرمز 555
7.15 دالة ترتيب الأسعار 557
7.16 دالة طول السلسلة ودالة المقارنة 558
7.17 دالة القاسم المشترك الأكبر لتوزيع العلب 560
7.18 دالة العدد الأولي لأرقام القسائم 561
7.19 دالة تحويل المدة إلى ساعات ودقائق 563
7.20 دالة الخصم حسب فئة الزبون 565
7.21 دالة التحقق من رقم البطاقة بخوارزمية لون 567
7.22 دالة التطبيع بين الصفر والواحد 569
7.23 دالة عدّ المنتجات تحت عتبة 571
7.24 دالة الأس الأسي بسلسلة تايلور 573
7.25 دالة التقريب إلى أقرب مضاعف 574
7.26 دالة رقم الفاتورة المتناظر 576
7.27 دالة رقم التحقق لرمز المنتج 577
7.28 دالة عرض العدد بالنظام الثنائي 579
7.29 دالة الوسيط لأسعار السوق 581
7.30 دالة المنوال للمقاس الأكثر طلبًا 583
7.31 دالة معامل الارتباط بين الإعلان والمبيعات 585
7.32 دوال الفاتورة المركبة 586

7.33 دالة القسط الشهري للشراء بالتقسيط . . . 588
7.34 دالة رسم مخطط الأعمدة النصي . . . 589
7.35 دالة البحث الثنائي في الكتالوج . . . 591
7.36 دالة تحويل درجة الحرارة لمخزن التبريد . . . 593
7.37 دالة تقييم المنتج بالنجوم . . . 595
7.38 دالة الوقت المتوقع للتوصيل . . . 597
7.39 دالة المتوسط المتحرك للمبيعات . . . 598
7.40 دالة عمر الزبون وشريحته . . . 600
7.41 الانحدار الخطي بالدوال . . . 603
7.42 الترتيب بالدمج التصاعدي بالدوال . . . 605
7.43 تجميع الزبائن بخوارزمية K-means أحادية البعد . . . 606
7.44 اختيار المنتجات الترويجية بمسألة حقيبة الظهر . . . 608
7.45 سلسلة ماركوف لحالات الزبائن . . . 610
7.46 سعر التعادل بطريقة التنصيف . . . 612
7.47 التكامل العددي لمنحنى الطلب . . . 614
7.48 غربال إراتوستينس وتحليل الأعداد . . . 616
7.49 دوال التواريخ لحساب موعد التسليم . . . 618
7.50 تصنيف الزبائن بأقرب الجيران . . . 620

8 العودية 623

8.1 عدد طرق ترتيب المنتجات في الواجهة . . . 625
8.2 مجموع أرقام رقم الطلبية عوديًا . . . 626
8.3 القوة عوديًا لنمو المتابعين . . . 627
8.4 العد التنازلي لنهاية العرض . . . 628
8.5 مجموع سلة المشتريات عوديًا . . . 629
8.6 عرض رمز المنتج معكوسًا عوديًا . . . 630
8.7 أغلى منتج عوديًا . . . 631
8.8 متتالية فيبوناتشي لنمو المشتركين . . . 632
8.9 القاسم المشترك الأكبر عوديًا . . . 633
8.10 عدّ تكرار تقييم معين عوديًا . . . 634
8.11 القوة السريعة بالتربيع . . . 636
8.12 البحث الثنائي العودي في الأسعار . . . 637
8.13 الرمز المتناظر عوديًا . . . 638
8.14 التحويل إلى الثنائي عوديًا . . . 639
8.15 لون المنتج بالست عشري عوديًا . . . 641
8.16 فواصل الآلاف في المبالغ عوديًا . . . 642
8.17 الجذر الرقمي لرمز التحقق . . . 644
8.18 طول متتالية كولاتز . . . 645
8.19 أبراج هانوي لنقل الصناديق . . . 647
8.20 فيبوناتشي بالتذكر وعدد الاستدعاءات . . . 649
8.21 عدد الحزم الممكنة بمثلث باسكال . . . 650
8.22 كل الحزم الممكنة من أربعة منتجات . . . 652
8.23 أقصر جولة توصيل بالتباديل . . . 654
8.24 عدد طرق دفع مبلغ بالقطع النقدية . . . 655

8.25 أقل عدد من القطع لإرجاع الباقي 656
8.26 عدد مسارات التوصيل في شبكة الشوارع 657
8.27 أصغر وأكبر سعر بالتقسيم 659
8.28 صرف بطاقة الهدية بالضبط 660
8.29 تجزئة الكمية إلى دفعات 661
8.30 أعداد كاتالان وأشجار القرار 663
8.31 مساحة المنطقة الحرة في المخزن 664
8.32 مسار الروبوت في متاهة المخزن 666
8.33 مبيعات شجرة الأقسام 668
8.34 الجذر التربيعي بالتنصيف العودي 669
8.35 قلب السلسلة في مكانها عوديًا 670
8.36 حساب كثير الحدود بطريقة هورنر العودية 672
8.37 دمج قائمتين مرتبتين عوديًا 674
8.38 رموز ثنائية دون واحدين متتاليين 675
8.39 اختيار ثلاثة منتجات ضمن الميزانية 676
8.40 عمق التداخل في ملف إعدادات 678
8.41 الترتيب بالدمج وعدد الانقلابات 680
8.42 الترتيب السريع للأسعار 681
8.43 توزيع الأكشاك دون تهديد متبادل 683
8.44 حل لغز السودوكو الترويجي 685
8.45 محلل صيغ الأسعار بالأقواس 687
8.46 أطول اتجاه صاعد في الأسعار 689
8.47 الاختيار السريع لوسيط الأسعار 691
8.48 أقرب مستودعين بطريقة فرّق تسد 693
8.49 جولة التوصيل المثلى بالتفرع والتحديد 695
8.50 تقطيع الوسم إلى كلمات من القاموس 697

9 الهياكل 701

9.1 هيكل المنتج 702
9.2 قراءة بيانات زبون 703
9.3 مقارنة تاريخي طلبيتين 704
9.4 المسافة بين المتجر والزبون 705
9.5 مبلغ الطلبية من سطورها 706
9.6 قيمة المخزون الإجمالية 708
9.7 دالة تعيد هيكلًا معدلًا 710
9.8 متوسط تقييمات منتج 711
9.9 جمع مدتين بهيكل الوقت 713
9.10 نسخ الهياكل بالإسناد 714
9.11 أرخص منتج في كل فئة 716
9.12 ترتيب المنتجات حسب السعر 717
9.13 ترتيب الزبائن أبجديًا 718
9.14 البحث عن منتج برمزه النصي 720
9.15 الطلبية بهياكل متداخلة 722
9.16 تقرير إعادة التموين 723

9.17 حساب الكسور لتقسيم الحصص 725
9.18 المبالغ بالدينار والسنتيم دون أخطاء تقريب 726
9.19 هل يدخل الصندوق في صندوق الشحن 728
9.20 دالة تعيد ملخصًا إحصائيًا 730
9.21 المبيعات حسب الفئة 732
9.22 صلاحية قسيمة التخفيض 734
9.23 عمولات فريق المبيعات 736
9.24 أفضل ثلاثة زبائن 738
9.25 دمج نوافذ التوصيل المتداخلة 740
9.26 موقع المستودع المركزي 741
9.27 حالات الطلبية بالتعداد 743
9.28 تاريخ الاستحقاق بإضافة أيام 745
9.29 إدارة سلة المشتريات 747
9.30 توزيع قيم السلات على فئات 749
9.31 معدل الارتداد ومدة الجلسات 751
9.32 تكلفة الشحن حسب المنطقة والوزن 753
9.33 تقييم الزبائن بطريقة RFM 755
9.34 منطقة تغطية التوصيلات 757
9.35 تصفية الكتالوج بعدة معايير 758
9.36 سجل معاملات المحفظة الإلكترونية 760
9.37 اختيار المورد بالعلامة المرجحة 762
9.38 أكبر انخفاض في تاريخ السعر 764
9.39 مقارنة إصدارات تطبيق المتجر 766
9.40 نتائج اختبار A/B لصفحة المنتج 768
9.41 قاعدة بيانات مصغرة بالربط بين الجداول 771
9.42 طابور الأولوية للطلبيات بكومة ثنائية 773
9.43 التراجع في السلة بمكدس 775
9.44 طابور تذاكر الدعم الدائري 777
9.45 أقصر طريق توصيل بخوارزمية ديكسترا 779
9.46 جذع شجرة قرار بمؤشر جيني 781
9.47 جدول تجزئة لرموز المنتجات 783
9.48 جدولة أكبر عدد من العروض الترويجية 785
9.49 الأعداد الكبيرة لرقم الأعمال التراكمي 787
9.50 انحدار متعدد بحل المعادلات الطبيعية 790

10 المؤشرات والحجز الديناميكي **793**
10.1 عنوان السعر وقيمته 794
10.2 تبديل سعرين بالمؤشرات 795
10.3 إنقاص المخزون عبر مؤشر 796
10.4 المرور على جدول بحساب المؤشرات 797
10.5 دالة تعيد الأصغر والأكبر 798
10.6 جدول ديناميكي بحجم يحدده المستخدم 799
10.7 مؤشر على هيكل والسهم 801
10.8 طول السلسلة بالمؤشرات 802

10.9 حجز هيكل منتج ديناميكيًا 803
10.10 عدد العناصر بين مؤشرين 805
10.11 جدول يكبر تلقائيًا 807
10.12 نسخة ديناميكية من اسم المنتج 808
10.13 الاسم الكامل في ذاكرة ديناميكية 810
10.14 قلب الجدول بمؤشرين متقابلين 812
10.15 حذف المكررات في مكانها 813
10.16 تقطيع الوصف إلى كلمات بالمؤشرات 815
10.17 ترتيب مؤشرات المنتجات لا المنتجات 817
10.18 مصفوفة مبيعات ديناميكية 819
10.19 استراتيجية الخصم بمؤشر دالة 821
10.20 ترتيب عام بدالة مقارنة 823
10.21 دالة تحجز جدولًا للمستدعي 825
10.22 تصفية المنتجات في جدول جديد 826
10.23 دمج قائمتين في ذاكرة جديدة 828
10.24 منقول مصفوفة التقييمات ديناميكيًا 829
10.25 تفكيك سطر CSV في مكانه 831
10.26 إحصاءات متجددة عبر مؤشر على هيكل 833
10.27 المخزن الدائري لآخر الأسعار 835
10.28 طلبيات الزبائن بأسطر متفاوتة الطول 837
10.29 البحث عن كلمة يعيد مؤشرًا 838
10.30 حذف الفراغات الطرفية بالمؤشرات 840
10.31 البحث الثنائي يعيد مؤشرًا 842
10.32 نسخة عميقة من الكتالوج 844
10.33 تطبيق دالة على كل عناصر الجدول 846
10.34 تبديل عام بمؤشر void 847
10.35 مكدس ديناميكي يكبر عند الحاجة 849
10.36 جداء مصفوفتين ديناميكيتين 851
10.37 المجاميع التراكمية للاستعلامات السريعة 853
10.38 كشف تسرب الذاكرة بعدّاد 855
10.39 مدرج تكراري بعدد فئات متغير 857
10.40 إدراج اسم في قائمة مرتبة ديناميكية 859
10.41 الجدول الديناميكي الكامل 862
10.42 مصفوفة التقييمات المتناثرة CSR 864
10.43 منشئ النصوص لتوليد الفاتورة 865
10.44 جدول تجزئة بالسلاسل لعد الكلمات 867
10.45 الترتيب بالكومة لمؤشرات المنتجات 869
10.46 مصفوفة التشابه بين المنتجات 871
10.47 أطول تسلسل مشترك بين سجلي تصفح 873
10.48 الترتيب بالأساس لأرقام الطلبيات 875
10.49 محاكاة مدير الذاكرة بالملاءمة الأولى 877
10.50 تجميع نقاط التوصيل بخوارزمية K-means 879

11 الملفات **883**

11.1 كتابة وصل الشراء في ملف 884
11.2 قراءة قائمة المنتجات بـ fscanf 885
11.3 عدّ أسطر سجل الطلبيات 887
11.4 إضافة طلبيات إلى نهاية الملف 888
11.5 نسخة احتياطية من ملف الزبائن 890
11.6 عدد كلمات تعليقات الزبائن 891
11.7 التعامل مع ملف غير موجود 893
11.8 متوسط قراءات الحرارة من ملف 894
11.9 قراءة الأسطر بـ fgets وترقيمها 896
11.10 تصدير الطلبيات بصيغة CSV 897
11.11 الإيراد لكل منتج من ملف CSV 900
11.12 استخراج الأخطاء من سجل الخادم 901
11.13 دمج ملفين مرتبين للطلبيات 903
11.14 حفظ الكتالوج في ملف ثنائي 905
11.15 تعديل سجل مباشرة بـ fseek 907
11.16 حجم الملف بـ fseek و ftell 908
11.17 الكلمات الأكثر تكرارًا في ملف التعليقات 910
11.18 تحديث المخزون من ملف الحركات 912
11.19 قراءة ملف الإعدادات 914
11.20 تقسيم السجل إلى ملفات شهرية 916
11.21 استبدال اسم منتج في ملف الوصف 918
11.22 إحصاءات أعمدة ملف بيانات 920
11.23 تصدير الكتالوج بصيغة JSON 922
11.24 كشف البريد المكرر في ملف الزبائن 924
11.25 مدرج التقييمات من ملف 926
11.26 إعادة بناء الرصيد من دفتر العمليات 928
11.27 تشفير ملف الزبائن بمفتاح XOR 930
11.28 أطول سطر في ملف الأوصاف 931
11.29 آخر أسطر السجل بمخزن دائري 933
11.30 مقارنة نسختين من قائمة الأسعار 935
11.31 تقرير بأعمدة ثابتة العرض من CSV 937
11.32 البحث في السجل دون اعتبار حجم الحروف 939
11.33 قراءة مصفوفة من ملف وكتابة منقولتها 941
11.34 عدد الزوار المختلفين في سجل الدخول 943
11.35 المجموع اليومي من سجل مؤرخ 945
11.36 حفظ جلسة السلة واستعادتها 947
11.37 ضغط ملف بالترميز الطولي وفك ضغطه 949
11.38 عدّ الرموز المتتالية في ملف مرتب 951
11.39 تنظيف ملف بيانات ناقص 953
11.40 فهرس الأسطر للوصول المباشر 955
11.41 الترتيب الخارجي لملف كبير 958
11.42 ملف ثنائي مفهرس والبحث بالقفز 960
11.43 كشف تلف الملفات بمجموع CRC-32 962
11.44 مصنف بايز للتعليقات المزيفة 964

11.45 تقسيم سجل النقرات إلى جلسات 967
11.46 مطابقة كشف البنك مع الطلبيات 970
11.47 ترميز ملف بـ Base64 وفك ترميزه 971
11.48 نموذج الانحدار من ملف وكتابة التنبؤات 973
11.49 قاعدة بيانات المنتجات بالحذف المنطقي والضغط 975
11.50 فهرس معكوس للبحث في الكتالوج 978

12 القوائم المتصلة **981**
12.1 أول قائمة متصلة للمنتجات 982
12.2 الإدراج في رأس القائمة 983
12.3 عدد العقد ومجموع الأسعار 984
12.4 البحث عن منتج في القائمة 986
12.5 طابور الطلبيات بمؤشر الذيل 987
12.6 تحرير القائمة كاملة 989
12.7 أغلى منتج في القائمة 991
12.8 سحب الطلبية من رأس القائمة 993
12.9 عرض القائمة معكوسة عوديًا 995
12.10 بناء قائمة من جدول 996
12.11 الإدراج المرتب حسب السعر 998
12.12 حذف منتج برمزه 999
12.13 قلب القائمة في مكانها 1001
12.14 دمج قائمتين مرتبتين بالروابط 1003
12.15 حذف المكررات من قائمة مرتبة 1005
12.16 العقدة الوسطى بمؤشرين 1007
12.17 كشف الحلقة في سلسلة التحويلات 1009
12.18 العنصر رقم k من النهاية 1011
12.19 فصل الطلبيات حسب نوع التوصيل 1013
12.20 سجل التصفح بقائمة مزدوجة 1015
12.21 توزيع دائري للطلبيات على السائقين 1017
12.22 مكدس التراجع بقائمة متصلة 1019
12.23 طابور معالجة الطلبيات بقائمة 1021
12.24 جمع كثيرات حدود التكلفة بالقوائم 1023
12.25 جمع أعداد ضخمة بقوائم الأرقام 1025
12.26 الترتيب بالإدراج على قائمة متصلة 1027
12.27 حذف المنتجات منخفضة التقييم 1029
12.28 الزبائن المشتركون بين حملتين 1031
12.29 تقسيم القائمة حول عتبة المخزون 1033
12.30 المنتجات المشاهدة مؤخرًا 1035
12.31 قائمة أقسام تحتوي على قوائم منتجات 1037
12.32 الجداء السلمي لمتجهات متناثرة 1039
12.33 تدوير قائمة العروض 1041
12.34 هل رمز المنتج متناظر في قائمة 1043
12.35 تبديل العقد زوجًا بزوج 1045
12.36 نسخة مستقلة من قائمة الطلبيات 1046

12.37 تكرار الكلمات في قائمة مرتبة 1048
12.38 حذف آخر ظهور لمنتج في السلة 1050
12.39 اتحاد قائمتي مشتركي النشرة 1052
12.40 الحذف من القائمة المزدوجة 1054
12.41 الترتيب بالدمج للقوائم المتصلة 1057
12.42 ذاكرة تخزين مؤقت LRU لصفحات المنتجات 1058
12.43 سحب الفائز بقائمة دائرية 1060
12.44 جداء كثيري حدود بالقوائم 1062
12.45 شبكة الإحالات بقوائم الجوار والبحث بالعرض 1064
12.46 مجمع عقد يعيد استعمال الذاكرة 1066
12.47 نسخ قائمة بروابط التوصية 1068
12.48 تسطيح شجرة الأقسام متعددة المستويات 1071
12.49 إعادة ترتيب العرض بالتناوب 1072
12.50 ترتيب مهام تجهيز الطلبية طوبولوجيًا 1074

13 الأشجار الثنائية 1077
13.1 أول شجرة ثنائية للأقسام 1079
13.2 طرق المرور الثلاث 1080
13.3 عدد عقد الشجرة 1081
13.4 ارتفاع شجرة الأقسام 1082
13.5 مجموع الأسعار في الشجرة 1083
13.6 إدراج الأسعار في شجرة بحث ثنائية 1084
13.7 البحث عن رمز في شجرة البحث 1086
13.8 أرخص وأغلى منتج في شجرة البحث 1088
13.9 عدد الأوراق في شجرة الأقسام 1090
13.10 تحرير الشجرة بالمرور البعدي 1091
13.11 حذف منتج من شجرة البحث 1094
13.12 المرور بالعرض مستوى بمستوى 1095
13.13 رسم الشجرة أفقيًا 1097
13.14 عدد المنتجات في مجال سعري 1099
13.15 السعر رقم k في الترتيب 1100
13.16 أغلى منتج ضمن الميزانية 1102
13.17 التحقق من صحة شجرة البحث 1104
13.18 أقرب سلف مشترك لسعرين 1105
13.19 الشجرة المرآة 1107
13.20 هل الشجرة متوازنة 1108
13.21 مسار حزمة بمجموع محدد 1110
13.22 كل قواعد شجرة القرار 1111
13.23 شجرة متوازنة من قائمة مرتبة 1113
13.24 شجرة العبارة لحساب السعر 1115
13.25 شجرة بحث لأسماء المنتجات 1116
13.26 أعرض مستوى في شجرة الأقسام 1118
13.27 قطر شجرة شبكة التوزيع 1120
13.28 مجموع المبيعات في كل مستوى 1121

13.29 تكرار الكلمات بشجرة بحث 1123
13.30 حفظ الشجرة في نص واستعادتها 1125
13.31 مقارنة نسختين من شجرة الأقسام 1127
13.32 السعر التالي في الترتيب 1128
13.33 مجموع المبيعات في مجال تواريخ 1130
13.34 إعادة بناء الشجرة من مرورين 1131
13.35 تصنيف زبون بشجرة قرار 1133
13.36 مسار التنقل من الجذر إلى القسم 1135
13.37 تحويل شجرة البحث إلى قائمة مرتبة 1137
13.38 دمج كتالوجين مرتبين من شجرتين 1139
13.39 المرور الوسطي التكراري بمكدس 1140
13.40 أقرب قسم نهائي من الجذر 1142
13.41 فهرس أسعار متوازن ذاتيًا AVL 1145
13.42 ترميز هوفمان لرموز الحالات 1147
13.43 الإكمال التلقائي لأسماء المنتجات بشجرة البادئات 1149
13.44 منتجان مجموع سعريهما يساوي القسيمة 1152
13.45 شجرة المقاطع لأدنى سعر في فترة 1154
13.46 تعلم شجرة قرار عوديًا 1156
13.47 أقرب مستودع بشجرة k-d 1158
13.48 مرور موريس دون مكدس 1160
13.49 شجرة فينويك للمبيعات التراكمية 1162
13.50 بناء شجرة العبارة من صيغة نصية 1164

المراجع **1167**

مقدمة

إعداد: د. بن عبد الرزاق يوسف — الفئة: طلبة السنة الأولى

يجمع هذا الكتاب **650 تمرينًا محلولًا** في أساسيات الخوارزميات ولغة C، موزعة على الفصول الثلاثة عشر للمقياس. كل التمارين مستوحاة من مجالين قريبين من الواقع المهني: **التجارة الإلكترونية** (سلات، طلبيات، مخزون، فواتير، توصيل) و**علم البيانات** (متوسطات، تباين، تطبيع، انحدار، تصنيف، تجميع).

بنية كل تمرين

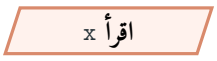


- **نص التمرين**: المسألة كما يطرحها متجر أو محلل بيانات.
- **خطوات الحل**: الفكرة الخوارزمية مرتبة خطوة بخطوة قبل أي كود.
- **البرنامج**: برنامج C كامل، تعليقاته **باللون الأحمر** تشرح كل سطر مهم.
- **نتيجة التنفيذ**: المخرجات **الحقيقية** للبرنامج (كل البرامج تُرجمت ونُفّذت).
- **المخطط الانسيابي** (Organigramme): التمثيل البياني للخوارزمية.

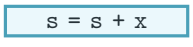
رموز المخطط الإنسيابي



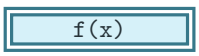
بداية الخوارزمية أو نهايتها.



عملية إدخال (قراءة) أو إخراج (كتابة).



معالجة: إسناد أو حساب.



استدعاء دالة (إجراء معرَّف في مكان آخر).



اختبار شرط: مخرجان **نعم** و**لا**.

اتجاه سير التنفيذ.

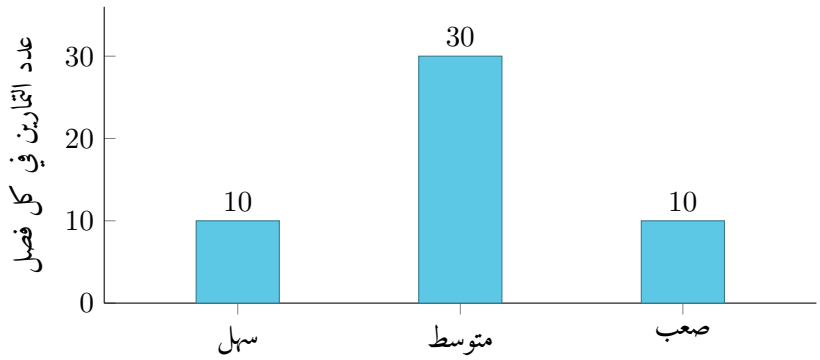
قاعدة الكتاب: لا دوال جاهزة

ممنوع استعمال الدوال المعرفة مسبقًا

الهدف هو تعلم الخوارزميات، لذلك نكتب بأنفسنا كل ما توفره المكتبات عادة: طول سلسلة، مقارنة سلسلتين، الجذر التربيعي، القيمة المطلقة، الترتيب، البحث، التحويل بين الأعداد والنصوص...

الصنف	الدوال
✔ مسموح	الإدخال والإخراج: `printf`، `scanf`، `getchar`، `putchar`
✔ مسموح (الفصول 10–13)	الذاكرة والملفات: `malloc`، `free`، `fopen`، `fclose`، `fprintf`، `fscanf`، `fgetc`، `fputc`، `fgets`، `fputs`، `fread`، `fwrite`، `fseek`، `ftell`، `rewind`
✘ ممنوع	`strlen`، `strcpy`، `strcmp`، `sqrt`، `pow`، `abs`، `qsort`، `atoi`، `toupper`، `rand`، ...

مستويات الصعوبة



- سهل ★ الفصل. من واحد لمفهوم مباشر تطبيق
- متوسط ★★ خوارزميًا. تفكيرًا وتتطلب مفاهيم عدة تجمع حقيقية مسألة
- صعب ★★★ يدويًا. بالكامل تُبنى تحسين...) تجميع، بحث، (ترتيب، متقدمة خوارزمية

ترجمة البرامج وتنفيذها

كل برنامج موجود في المجلد `code/chNN/exMM.c` مع نتيجة تنفيذه `exMM.out`:

```
gcc -std=c99 -Wall -Wextra code/ch04/ex12.c -o ex12
./ex12 < code/ch04/ex12.in
```

الفصل 1

مدخل إلى الخوارزميات: المتغيرات والعمليات

تعريفات

- **الخوارزمية**: سلسلة منتهية ومرتبة من التعليمات تحل مسألة: مدخلات ← معالجة ← مخرجات.
- **المتغير**: خانة في الذاكرة لها اسم ونوع وقيمة يمكن أن تتغير.
- **الثابت**: قيمة لا تتغير أثناء التنفيذ (`#define TVA 0.19` أو `const`).
- **التعبير**: تركيب من متغيرات وثوابت وعوامل يُحسب ليعطي قيمة.



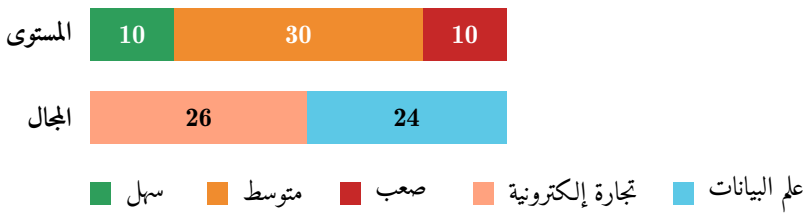
النوع	الصيغة	الاستعمال في التمارين
`int`	%d	الكميات، عدد الطلبيات، الثواني
`long`	%ld	الأعداد الكبيرة (رموز المنتجات، الأجام)
`double`	%f / %lf	الأسعار، النسب، المتوسطات
`char`	%c	رموز الأصناف

القسمة في C

`2 / 7` تعطي `3` (قسمة صحيحة) و`2 % 7` تعطي `1`، أما `2.0 / 7` فتعطي `3.5`.

للاستزادة: [1, 2, 9]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 1.1 السعر مع الرسم على القيمة المضافة

سهل تجارة إلكترونية

يبيع متجر إلكتروني منتجًا بسعر دون رسوم HT. اكتب برنامجًا يقرأ هذا السعر ثم يحسب قيمة الرسم على القيمة المضافة بنسبة 19% والسعر النهائي TTC.

خطوات الحل

1) نعرّف ثابتًا TVA = 0.19.
2) نقرأ السعر ht.
3) نحسب الرسم tax = ht * TVA.
4) نحسب السعر النهائي ttc = ht + tax ثم نعرض النتيجتين.

البرنامج بلغة C

```c
#include <stdio.h>

#define TVA 0.19                    // نسبة الرسم 19%

int main(void) {
    double ht, tax, ttc;            // السعر دون رسوم، الرسم، السعر النهائي
    printf("Price HT: ");
    scanf("%lf", &ht);              // قراءة السعر دون رسوم
    tax = ht * TVA;                 // قيمة الرسم
    ttc = ht + tax;                 // السعر النهائي
    printf("\nTax = %.2f DA\n", tax);
    printf("TTC = %.2f DA\n", ttc);
    return 0;
}
```

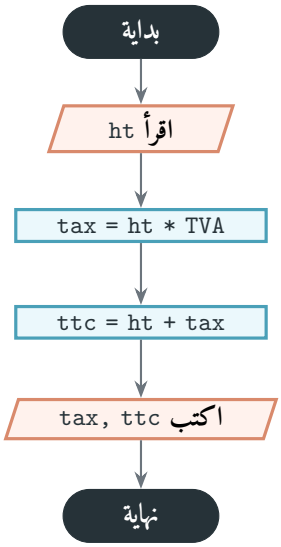
المدخلات

```
2500
```

نتيجة التنفيذ

```
Price HT:
Tax = 475.00 DA
TTC = 2975.00 DA
```

المخطط الانسيابي (Organigramme)



تمرين 1.2 مبلغ سطر في الفاتورة

تجارة إلكترونية سهل

اشترى زبون عدة وحدات من نفس المنتج. اكتب برنامجًا يقرأ **سعر الوحدة** و**الكمية** ويعرض مبلغ هذا السطر من الفاتورة.

خطوات الحل

1) نقرأ سعر الوحدة `price` (عدد حقيقي) والكمية `qty` (عدد صحيح).
2) نضرب السعر في الكمية: `amount = price * qty`.
3) نعرض المبلغ بمنزلتين عشريتين.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, amount;          // سعر الوحدة ومبلغ السطر
    int qty;                       // الكمية عدد صحيح
    scanf("%lf %d", &price, &qty);
    amount = price * qty;          // المبلغ = السعر × الكمية
    printf("%d x %.2f = %.2f DA\n", qty, price, amount);
    return 0;
}
```

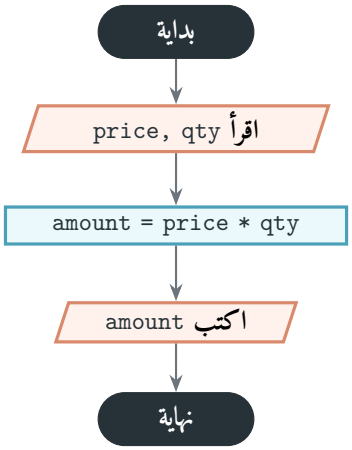
المدخلات

```
349.90 3
```

نتيجة التنفيذ

```
3 x 349.90 = 1049.70 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.3 متوسط تقييمات منتج

علم البيانات سهل

ترك ثلاثة زبائن تقييمات لمنتج (من 1 إلى 5). اكتب برنامجًا يقرأ التقييمات الثلاثة ويحسب **متوسطها** كعدد حقيقي.

خطوات الحل

1) نقرأ التقييمات `r1` و`r2` و`r3` كأعداد صحيحة.
2) نحسب المجموع ثم نقسمه على `3.0` (وليس `3`) لنحصل على قسمة حقيقية.
3) نعرض المتوسط.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int r1, r2, r3;              // التقييمات الثلاثة
    double avg;
    scanf("%d %d %d", &r1, &r2, &r3);
    // القسمة على 3.0 تجعل النتيجة حقيقية
    avg = (r1 + r2 + r3) / 3.0;
    printf("Average rating = %.2f / 5\n", avg);
    return 0;
}
```

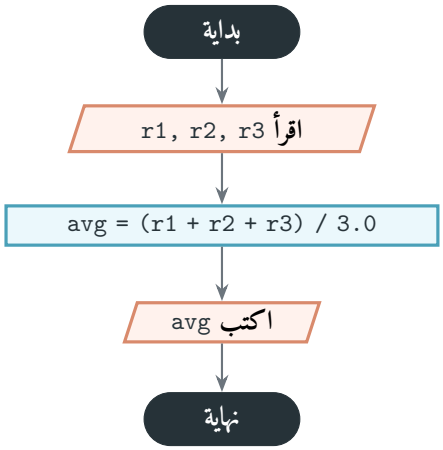
المدخلات

```
4 5 3
```

نتيجة التنفيذ

```
Average rating = 4.00 / 5
```

المخطط الإنسيابي (Organigramme)



تمرين 1.4 تحويل العملة

تجارة إلكترونية سهل

يعرض متجر جزائري أسعاره بالدينار، ويريد عرضها أيضًا باليورو. سعر الصرف ثابت: 1 يورو = 146.5 دج. اكتب برنامجًا يحول مبلغًا مقروءًا من الدينار إلى اليورو.

خطوات الحل

1) نعرّف الثابت `RATE = 146.5`.
2) نقرأ المبلغ بالدينار `da`.
3) نقسم على سعر الصرف: `eur = da / RATE`.

البرنامج بلغة C

```c
#include <stdio.h>

#define RATE 146.5                  // سعر صرف اليورو بالدينار

int main(void) {
    double da, eur;
    scanf("%lf", &da);              // المبلغ بالدينار
    eur = da / RATE;                // التحويل إلى اليورو
    printf("%.2f DA = %.2f EUR\n", da, eur);
    return 0;
}
```

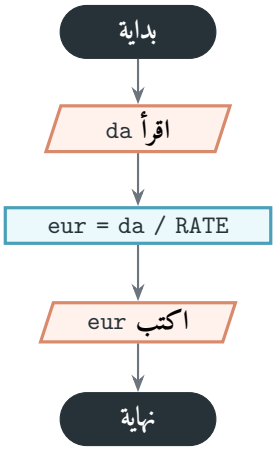
المدخلات

```
12000
```

نتيجة التنفيذ

```
12000.00 DA = 81.91 EUR
```

المخطط الإنسيابي (Organigramme)



تمرين 1.5 الوزن الإجمالي للشحنة

تجارة إلكترونية سهل

تتكون شحنة من ثلاثة طرود. اكتب برنامجًا يقرأ أوزانها بالكيلوغرام ويعرض **الوزن الإجمالي** و**متوسط** وزن الطرد.

خطوات الحل

1) نقرأ الأوزان الثلاثة `w1` و`w2` و`w3`.
2) نحسب المجموع `total`.
3) نحسب المتوسط `total / 3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double w1, w2, w3, total, avg;
    scanf("%lf %lf %lf", &w1, &w2, &w3);
    total = w1 + w2 + w3;          // الوزن الإجمالي
    avg = total / 3;               // total حقيقي، فالقسمة حقيقية
    printf("Total weight = %.2f kg\n", total);
    printf("Average weight = %.2f kg\n", avg);
    return 0;
}
```

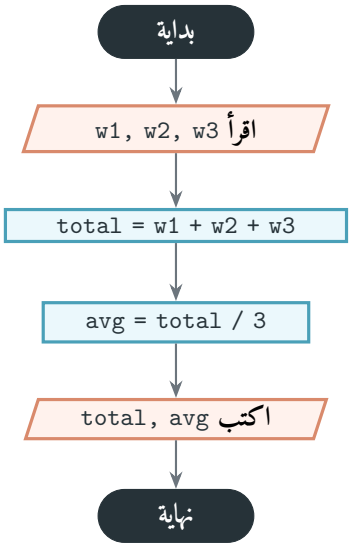
المدخلات

```
2.4 0.8 5.1
```

نتيجة التنفيذ

```
Total weight = 8.30 kg
Average weight = 2.77 kg
```

المخطط الإنسيابي (Organigramme)



تمرين 1.6 التخفيض الموسمي

تجارة إلكترونية سهل

في فترة التخفيضات، يُطبَّق على كل منتج نسبة تخفيض. اكتب برنامجًا يقرأ السعر الأصلي ونسبة التخفيض (بالمئة) ويعرض قيمة التخفيض والسعر الجديد.

خطوات الحل

1) نقرأ السعر `price` والنسبة `rate`.
2) قيمة التخفيض: `disc = price * rate / 100`.
3) السعر الجديد: `newPrice = price - disc`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, rate, disc, newPrice;
    scanf("%lf %lf", &price, &rate);
    disc = price * rate / 100;     // قيمة التخفيض
    newPrice = price - disc;       // السعر بعد التخفيض
    printf("Discount = %.2f DA\n", disc);
    printf("New price = %.2f DA\n", newPrice);
    return 0;
}
```

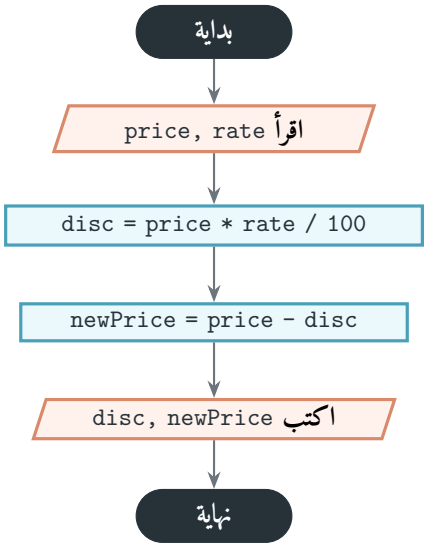
المدخلات

```
8900 30
```

نتيجة التنفيذ

```
Discount = 2670.00 DA
New price = 6230.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.7 الربح وهامش الربح

تجارة إلكترونية سهل

يشتري بائع منتجًا بسعر `buy` ويبيعه بسعر `sell`. اكتب برنامجًا يحسب **الربح** و**هامش الربح** كنسبة مئوية من سعر البيع.

خطوات الحل

1) نقرأ سعر الشراء وسعر البيع.
2) الربح: `profit = sell - buy`.
3) الهامش: `margin = profit / sell * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double buy, sell, profit, margin;
    scanf("%lf %lf", &buy, &sell);
    profit = sell - buy;              // الربح لكل وحدة
    margin = profit / sell * 100;     // الهامش بالمئة
    printf("Profit = %.2f DA\n", profit);
    printf("Margin = %.2f %%\n", margin);   // %% تطبع الرمز %
    return 0;
}
```

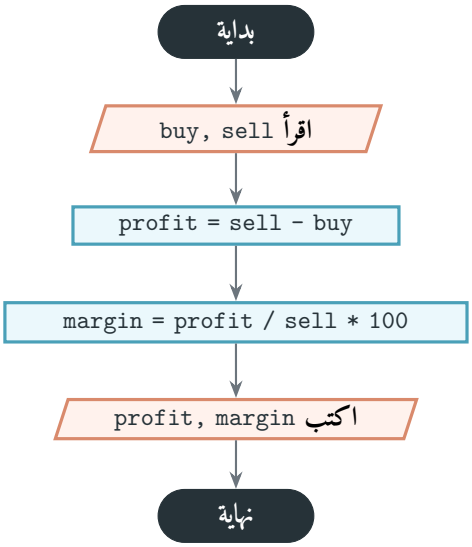
المدخلات

```
1500 2100
```

نتيجة التنفيذ

```
Profit = 600.00 DA
Margin = 28.57 %
```

المخطط الانسيابي (Organigramme)



تمرين 1.8 تبديل سعرين

تجارة إلكترونية سهل

أدخل موظف سعري منتجين بالعكس. اكتب برنامجًا يقرأ السعرين priceA وpriceB ثم يبدل قيمتيهما باستعمال متغير مساعد، ويعرضهما قبل التبديل وبعده.

خطوات الحل

1) نقرأ priceA وpriceB.
2) نحفظ priceA في متغير مساعد tmp.
3) نضع priceB في priceA ثم tmp في priceB.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int priceA, priceB, tmp;
    scanf("%d %d", &priceA, &priceB);
    printf("Before: priceA = %d, priceB = %d\n", priceA, priceB);
    tmp = priceA;          // نحفظ priceA قبل أن نفقدها
    priceA = priceB;            // priceA تأخذ قيمة priceB
    priceB = tmp;          // priceB تأخذ القيمة القديمة لـ priceA
    printf("After:  priceA = %d, priceB = %d\n", priceA, priceB);
    return 0;
}
```

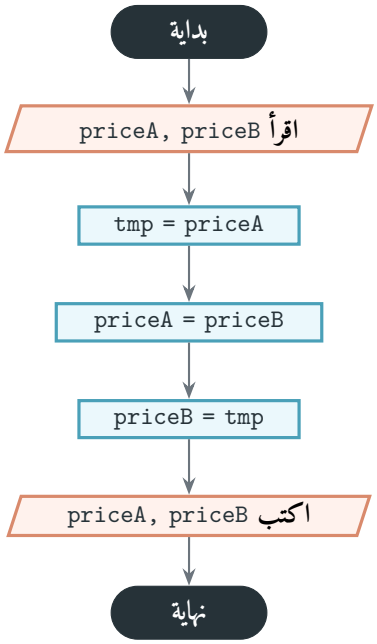
المدخلات

```
450 1200
```

نتيجة التنفيذ

```
Before: priceA = 450, priceB = 1200
After:  priceA = 1200, priceB = 450
```

المخطط الإنسيابي (Organigramme)



تمرين 1.9 مدة جلسة التصفح

علم البيانات سهل

يسجل موقع التجارة الإلكترونية مدة كل جلسة بالثواني. اكتب برنامجًا يحول المدة إلى **دقائق وثوانٍ** باستعمال القسمة الصحيحة `/` وباقي القسمة `%`.

خطوات الحل

1) نقرأ المدة `seconds` بالثواني.
2) عدد الدقائق: `minutes = seconds / 60` (قسمة صحيحة).
3) الثواني المتبقية: `rest = seconds % 60`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int seconds, minutes, rest;
    scanf("%d", &seconds);              // المدة بالثواني
    minutes = seconds / 60;                  // قسمة صحيحة: عدد الدقائق الكاملة
    rest = seconds % 60;                   // الباقي: الثواني المتبقية
    printf("%d s = %d min %d s\n", seconds, minutes, rest);
    return 0;
}
```

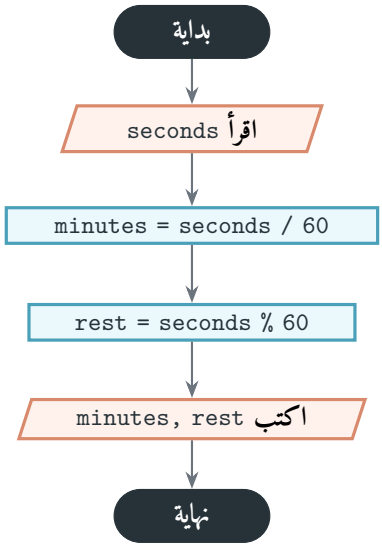
المدخلات

```
437
```

نتيجة التنفيذ

```
437 s = 7 min 17 s
```

المخطط الانسيابي (Organigramme)



تمرين 1.10 معدل التحويل

علم البيانات سهل

معدل التحويل هو نسبة الزوار الذين قاموا بالشراء. اكتب برنامجًا يقرأ عدد الزوار وعدد الطلبيات ويعرض معدل التحويل بالمئة.

خطوات الحل

1) نقرأ `visitors` و`orders` كأعداد صحيحة.
2) نحول أحدهما إلى `double` قبل القسمة لتجنب القسمة الصحيحة.
3) `rate = orders / visitors * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int visitors, orders;
    double rate;
    scanf("%d %d", &visitors, &orders);
    // التحويل (double) يمنع القسمة الصحيحة التي تعطي 0
    rate = (double)orders / visitors * 100;
    printf("Conversion rate = %.2f %%\n", rate);
    return 0;
}
```

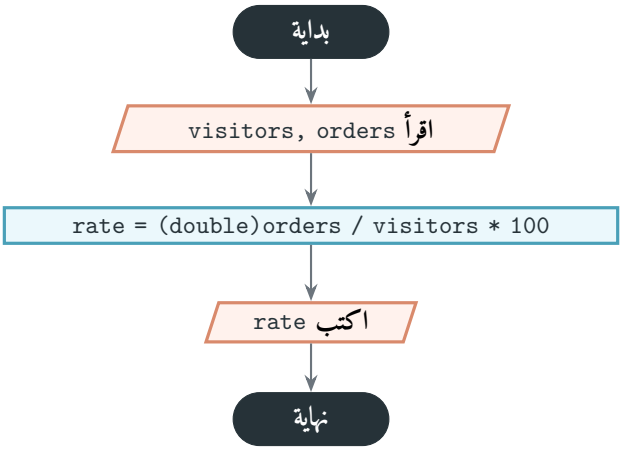
المدخلات

```
5400 162
```

نتيجة التنفيذ

```
Conversion rate = 3.00 %
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 1.11 عدد علب التغليف

تجارة إلكترونية ★★ متوسط

يجب تغليف `items` منتجًا في علب تتسع كل واحدة لـ `capacity` منتجات. اكتب برنامجًا يحسب **عدد العلب اللازمة** وعدد المنتجات في **العلبة الأخيرة**، دون استعمال أي شرط.

خطوات الحل

1) عدد العلب هو القسمة الصحيحة مقربة إلى الأعلى: `boxes = (items + capacity - 1) / capacity`.
2) المنتجات الموجودة في العلب الممتلئة: `(boxes - 1) * capacity`.
3) العلبة الأخيرة: `last = items - (boxes - 1) * capacity`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int items, capacity, boxes, last;
    scanf("%d %d", &items, &capacity);
    // إضافة capacity-1 قبل القسمة تقرب النتيجة إلى الأعلى
    boxes = (items + capacity - 1) / capacity;
    // ما تبقى بعد ملء كل العلب ما عدا الأخيرة
    last = items - (boxes - 1) * capacity;
    printf("Boxes needed = %d\n", boxes);
    printf("Items in last box = %d\n", last);
    return 0;
}
```

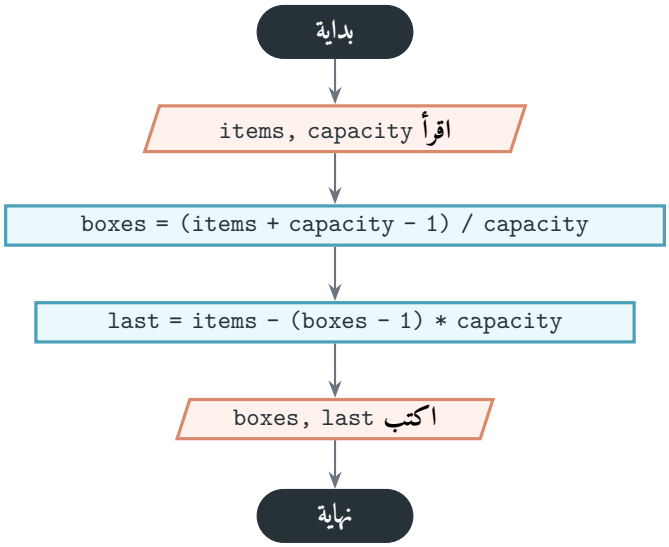
المدخلات

```
47 12
```

نتيجة التنفيذ

```
Boxes needed = 4
Items in last box = 11
```

المخطط الإنسيابي (Organigramme)



تمرين 1.12 المسافة بين زبونين

علم البيانات متوسط

نمثل كل زبون بخاصيتين: عدد الطلبيات `x` ومتوسط السلة بالآلاف `y`. اكتب برنامجًا يحسب **مربع المسافة الإقليدية** بين زبونين (دون جذر)، ويعرض أيضًا الفرق في عدد الطلبيات.

خطوات الحل

1) نقرأ إحداثيات الزبونين `(x1, y1)` و`(x2, y2)`.
2) نحسب الفرقين `dx = x1 - x2` و`dy = y1 - y2`.
3) مربع المسافة: `d2 = dx*dx + dy*dy`.
4) مسافة مانهاتن للخاصية الأولى هي `dx` مباشرة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, dx, dy, d2;
    scanf("%lf %lf", &x1, &y1);   // الزبون الأول
    scanf("%lf %lf", &x2, &y2);   // الزبون الثاني
    dx = x1 - x2;                 // الفرق في عدد الطلبيات
    dy = y1 - y2;                 // الفرق في متوسط السلة
    d2 = dx * dx + dy * dy;       // مربع المسافة دون جذر
    printf("Squared distance = %.2f\n", d2);
    printf("Orders gap = %.0f\n", dx);
    return 0;
}
```

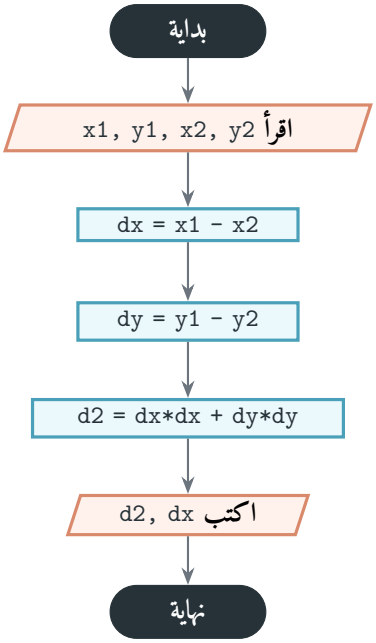
المدخلات

```
12 4.5
7 2.5
```

نتيجة التنفيذ

```
Squared distance = 29.00
Orders gap = 5
```

المخطط الانسيابي (Organigramme)



تمرين 1.13 المتوسط والتباين

علم البيانات متوسط

لدينا مبيعات أربعة أيام. اكتب برنامجًا يحسب **المتوسط** و**التباين** (متوسط مربعات الانحرافات عن المتوسط).

خطوات الحل

1) نقرأ القيم الأربع ونحسب المتوسط `mean`.
2) نحسب انحراف كل قيمة: `ai - mean`.
3) التباين: مجموع مربعات الانحرافات مقسوم على `4`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double day1, day2, day3, day4, mean, variance;
    scanf("%lf %lf %lf %lf", &day1, &day2, &day3, &day4);
    mean = (day1 + day2 + day3 + day4) / 4;          // المتوسط
    // التباين: متوسط مربعات الانحرافات
    variance = ((day1 - mean) * (day1 - mean) + (day2 - mean) * (day2 - mean)
       + (day3 - mean) * (day3 - mean) + (day4 - mean) * (day4 - mean)) / 4;
    printf("Mean = %.2f\n", mean);
    printf("Variance = %.2f\n", variance);
    return 0;
}
```

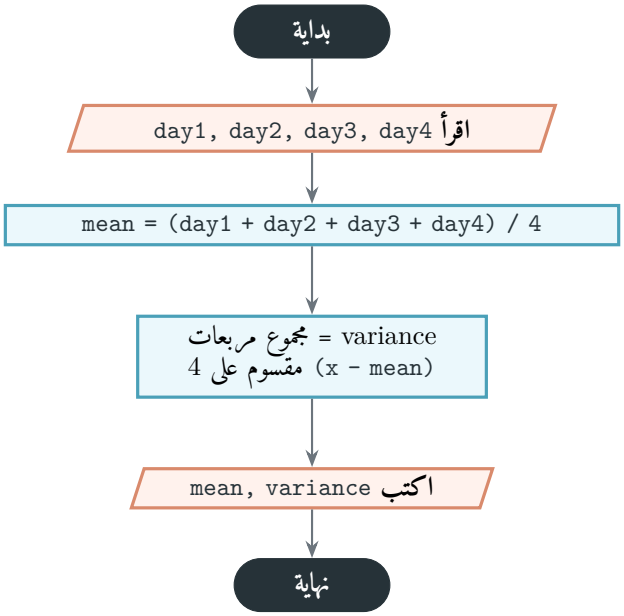
المدخلات

```
120 95 140 105
```

نتيجة التنفيذ

```
Mean = 115.00
Variance = 287.50
```

المخطط الإنسيابي (Organigramme)



تمرين 1.14 التطبيع بين 0 و1

علم البيانات ★★ متوسط

قبل تدريب نموذج، نحوّل الأسعار إلى المجال `[0, 1]` بطريقة **Min-Max**. اكتب برنامجًا يقرأ أصغر سعر وأكبر سعر وسعرًا `price` ويعرض قيمته المطبّعة.

خطوات الحل

1) نقرأ `min` و`max` و`price`.
2) الصيغة: `xn = (price - min) / (max - min)`.
3) نعرض `xn` بثلاث منازل عشرية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double min, max, price, xn;
    scanf("%lf %lf %lf", &min, &max, &price);
    // نطرح الأصغر ثم نقسم على المدى
    xn = (price - min) / (max - min);
    printf("Normalized value = %.3f\n", xn);
    return 0;
}
```

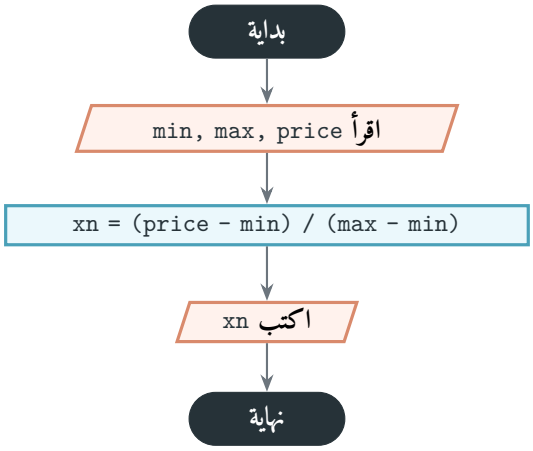
المدخلات

```
200 5200 1450
```

نتيجة التنفيذ

```
Normalized value = 0.250
```

المخطط الإنسيابي (Organigramme)



تمرين 1.15 الدرجة المعيارية

علم البيانات ★★ متوسط

لاكتشاف الطلبيات غير العادية نحسب **الدرجة المعيارية** `zscore = (amount - mean) / std`. اكتب برنامجًا يقرأ مبلغ طلبية ومتوسط المبالغ وانحرافها المعياري، ويعرض `zscore` ثم المبلغ المقابل لـ `zscore = 2`.

خطوات الحل

1) نقرأ `amount` و`mean` و`std`.
2) `zscore = (amount - mean) / std`.
3) المبلغ الموافق لدرجة معيارية 2: `x2 = mean + 2 * std`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, mean, std, zscore, x2;
    scanf("%lf %lf %lf", &amount, &mean, &std);
    zscore = (amount - mean) / std;          // كم انحرافًا معياريًا يبعد amount عن المتوسط
    x2 = mean + 2 * std;           // العتبة الموافقة لـ zscore = 2
    printf("z-score = %.2f\n", zscore);
    printf("Threshold (z = 2) = %.2f DA\n", x2);
    return 0;
}
```

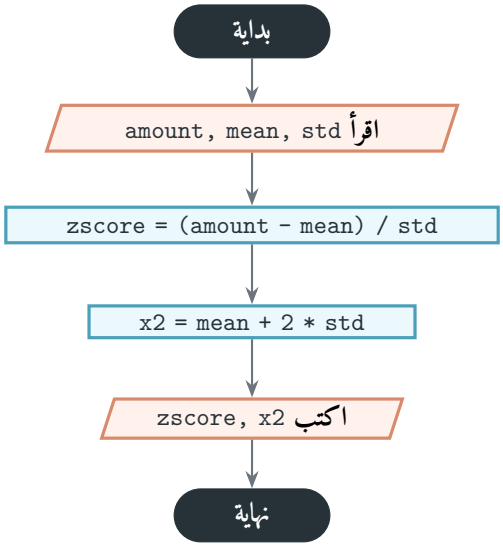
المدخلات

```
9800 4200 1900
```

نتيجة التنفيذ

```
z-score = 2.95
Threshold (z = 2) = 8000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.16 متوسط النجوم الموزونة

علم البيانات ★★ متوسط

تعرض صفحة المنتج عدد التقييمات لكل نجمة (من نجمة إلى خمس نجوم). اكتب برنامجًا يقرأ الأعداد الخمسة ويحسب **المتوسط الموزون** وعدد التقييمات الكلي.

خطوات الحل

1) نقرأ `c1` إلى `c5`.
2) العدد الكلي: `nbReviews = c1 + c2 + c3 + c4 + c5`.
3) المجموع الموزون:

$$\texttt{weighted} = 1{\cdot}c1 + 2{\cdot}c2 + 3{\cdot}c3 + 4{\cdot}c4 + 5{\cdot}c5$$

.

4) المتوسط: `weighted / nbReviews` بقسمة حقيقية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int c1, c2, c3, c4, c5, nbReviews, weighted;
    double avg;
    scanf("%d %d %d %d %d", &c1, &c2, &c3, &c4, &c5);
    nbReviews = c1 + c2 + c3 + c4 + c5;                  // عدد التقييمات
    weighted = 1 * c1 + 2 * c2 + 3 * c3 + 4 * c4 + 5 * c5; // كل نجمة × عددها
    avg = (double)weighted / nbReviews;                  // متوسط موزون
    printf("Reviews = %d\n", nbReviews);
    printf("Weighted average = %.2f stars\n", avg);
    return 0;
}
```

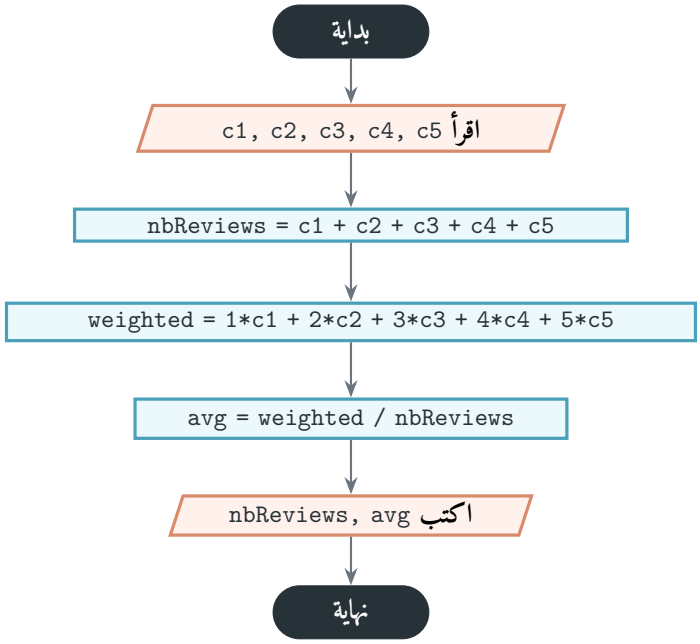
المدخلات

```
3 5 12 40 60
```

نتيجة التنفيذ

```
Reviews = 120
Weighted average = 4.24 stars
```

المخطط الإنسيابي (Organigramme)



تمرين 1.17 نمو رقم الأعمال

علم البيانات ★★ متوسط

رقم أعمال متجر هذا الشهر `revenue`، ويتوقع نموًا شهريًا ثابتًا بنسبة `growth` بالمئة. اكتب برنامجًا يحسب رقم الأعمال المتوقع بعد 1 و2 و3 أشهر، دون حلقات ودون دالة `pow`.

خطوات الحل

1) نحسب المعامل `factor = 1 + growth / 100`.
2) بعد شهر: `r1 = revenue * factor`، بعد شهرين: `r2 = r1 * factor`، بعد ثلاثة: `r3 = r2 * factor`.
3) نعرض القيم الثلاث.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double revenue, growth, factor, r1, r2, r3;
    scanf("%lf %lf", &revenue, &growth);
    factor = 1 + growth / 100;              // معامل النمو الشهري
    r1 = revenue * factor;                  // بعد شهر
    r2 = r1 * factor;                       // بعد شهرين = revenue × factor × factor
    r3 = r2 * factor;                       // بعد ثلاثة أشهر
    printf("Month 1: %.2f\n", r1);
```

```
    printf("Month 2: %.2f\n", r2);
    printf("Month 3: %.2f\n", r3);
    return 0;
}
```

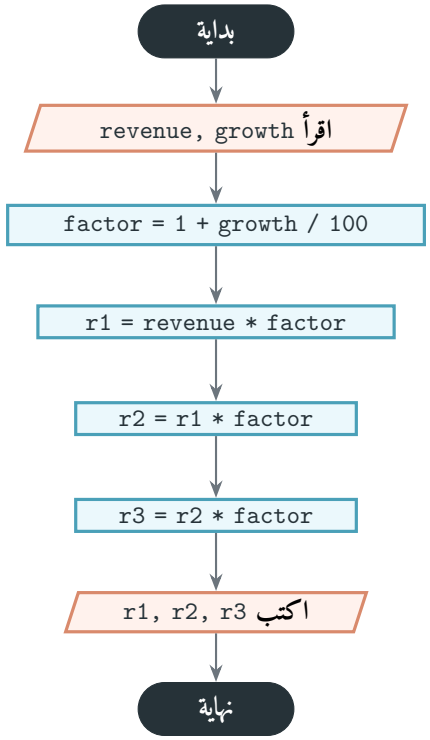
المدخلات

```
250000 8
```

نتيجة التنفيذ

```
Month 1: 270000.00
Month 2: 291600.00
Month 3: 314928.00
```

المخطط الإنسيابي (Organigramme)



تمرين 1.18 تفكيك مبلغ إلى أوراق نقدية

تجارة إلكترونية ★★ متوسط

عند الدفع نقدًا عند الاستلام، يريد عامل التوصيل تحضير المبلغ بأقل عدد من الأوراق: 2000 و1000 و500 و200، والباقي قطع نقدية. اكتب برنامجًا يفكك المبلغ.

خطوات الحل

1) عدد أوراق 2000: `notes2000 = money / 2000` ثم نأخذ الباقي `money = money % 2000`.
2) نكرر نفس الفكرة مع 1000 ثم 500 ثم 200.
3) ما بقي في `money` هو مبلغ القطع النقدية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int money, notes2000, notes1000, notes500, notes200;
    scanf("%d", &money);                 // المبلغ بالدينار
    notes2000 = money / 2000; money = money % 2000;   // أوراق 2000 ثم ما تبقى
    notes1000 = money / 1000; money = money % 1000;   // أوراق 1000
    notes500 = money / 500;  money = money % 500;     // أوراق 500
    notes200 = money / 200;  money = money % 200;     // أوراق 200
    printf("2000 DA x %d\n1000 DA x %d\n", notes2000, notes1000);
    printf("500 DA x %d\n200 DA x %d\n", notes500, notes200);
    printf("Coins: %d DA\n", money);  // الباقي قطع نقدية
    return 0;
}
```

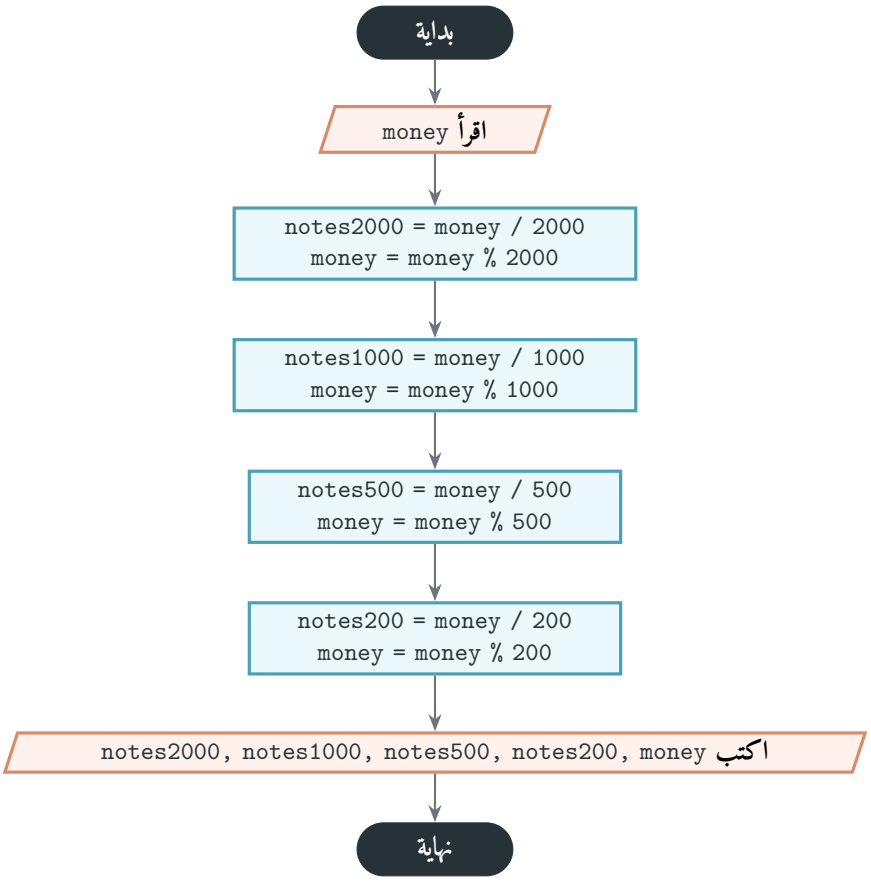
المدخلات

```
7900
```

نتيجة التنفيذ

```
2000 DA x 3
1000 DA x 1
500 DA x 1
200 DA x 2
Coins: 0 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.19 ساعة وصول الطلبية

تجارة إلكترونية ★★ متوسط

تُسجَّل طلبية في الساعة `hour:minute` ويستغرق توصيلها `delay` دقيقة. اكتب برنامجًا يحسب **ساعة الوصول** بصيغة 24 ساعة، حتى لو تجاوزت منتصف الليل.

خطوات الحل

1) نحول وقت الطلب إلى دقائق منذ منتصف الليل: `total = hour * 60 + minute + delay`.
2) ساعة الوصول: `(total / 60) % 24`.
3) الدقائق: `total % 60`.
4) نستعمل `%02d` لعرض صفر على اليسار.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hour, minute, delay, total, ah, am;
    scanf("%d %d %d", &hour, &minute, &delay);
    total = hour * 60 + minute + delay;        // الدقائق منذ منتصف الليل
    ah = (total / 60) % 24;        // 24 % تعيد الساعة بعد 23 إلى 0
    am = total % 60;               // الدقائق
    printf("Arrival time: %02d:%02d\n", ah, am);
    return 0;
}
```

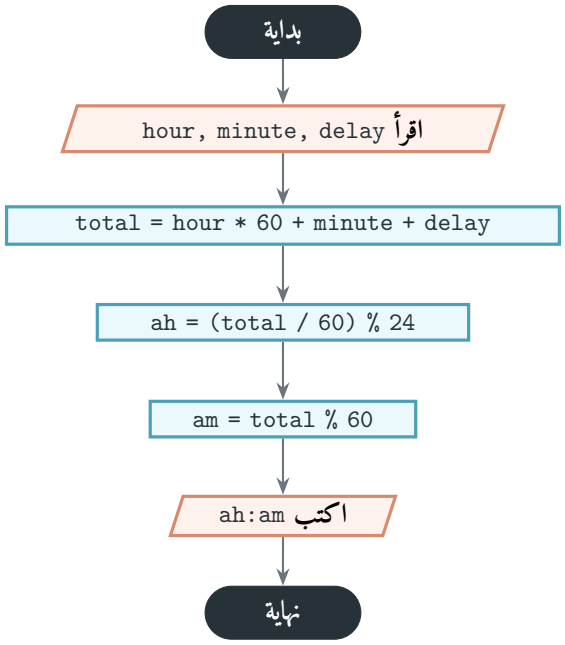
المدخلات

```
22 35 150
```

نتيجة التنفيذ

```
Arrival time: 01:05
```

المخطط الإنسيابي (Organigramme)



تمرين 1.20 مدة الإشتراك بالأسابيع

تجارة إلكترونية ★★ متوسط

يشترك زبون في خدمة توصيل مجاني لمدة days يومًا. اكتب برنامجًا يعرض هذه المدة بالأسابيع والأيام، وعدد الأشهر التقريبية (شهر = 30 يومًا) مع الأيام المتبقية.

خطوات الحل

1) الأسابيع: weeks = days / 7 والأيام: days % 7.
2) الأشهر: mo = days / 30 والأيام: days % 30.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int days, weeks, rw, mo, rm;
    scanf("%d", &days);
    weeks = days / 7;   rw = days % 7;       // أسابيع كاملة + أيام
    mo = days / 30; rm = days % 30;      // أشهر تقريبية + أيام
    printf("%d days = %d weeks + %d days\n", days, weeks, rw);
    printf("%d days = %d months + %d days\n", days, mo, rm);
    return 0;
}
```

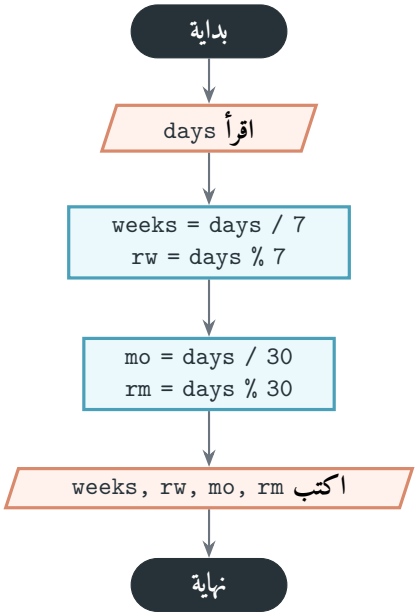
المدخلات

```
200
```

نتيجة التنفيذ

```
200 days = 28 weeks + 4 days
200 days = 6 months + 20 days
```

المخطط الإنسيابي (Organigramme)



تمرين 1.21 عكس رمز المنتج

تجارة إلكترونية متوسط

لكل منتج رمز من **ثلاثة أرقام**. لإنشاء رمز سري يعكس النظام ترتيب الأرقام (مثلًا 472 يصبح 274). اكتب برنامجًا يقوم بذلك باستعمال / و% فقط.

خطوات الحل

1) رقم الآحاد: `units = code % 10`.
2) رقم العشرات: `tens = (code / 10) % 10`.
3) رقم المئات: `hundreds = code / 100`.
4) الرمز المعكوس:

```
reversed = units * 100 + tens * 10 + hundreds
```

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int code, units, tens, hundreds, reversed;
    scanf("%d", &code);
    units = code % 10;              // رقم الآحاد
    tens = (code / 10) % 10;        // نحذف الآحاد ثم نأخذ الرقم الأخير
    hundreds = code / 100;          // رقم المئات
    reversed = units * 100 + tens * 10 + hundreds; // نعيد التركيب بالترتيب المعاكس
    printf("Code %d -> %d\n", code, reversed);
    return 0;
}
```

```
12 }
```

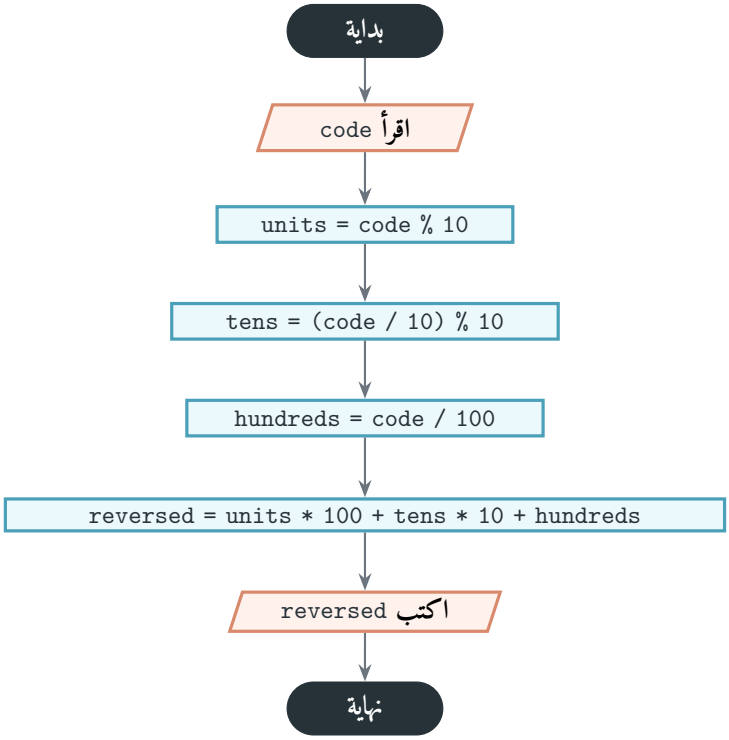
المدخلات

```
472
```

نتيجة التنفيذ

```
Code 472 -> 274
```

المخطط الإنسيابي (Organigramme)



تمرين 1.22 مجموع التحقق لرقم الطلبية

تجارة إلكترونية متوسط

رقم كل طلبية مكون من أربعة أرقام. لاكتشاف أخطاء الإدخال نحسب مجموع أرقامه ثم رقم التحقق S % 10. اكتب برنامجًا يقوم بذلك.

خطوات الحل

1) نستخرج الأرقام الأربعة بالقسمة على 1000 و100 و10 وبباقي القسمة على 10.
2) نجمعها في digitSum.
3) رقم التحقق هو digitSum % 10.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int order, d1, d2, d3, d4, digitSum;
```

```c
    scanf("%d", &order);
    d1 = order / 1000;          // رقم الآلاف
    d2 = (order / 100) % 10;    // رقم المئات
    d3 = (order / 10) % 10;     // رقم العشرات
    d4 = order % 10;            // رقم الآحاد
    digitSum = d1 + d2 + d3 + d4;    // مجموع الأرقام
    printf("Digit sum = %d\n", digitSum);
    printf("Check digit = %d\n", digitSum % 10);
    return 0;
}
```

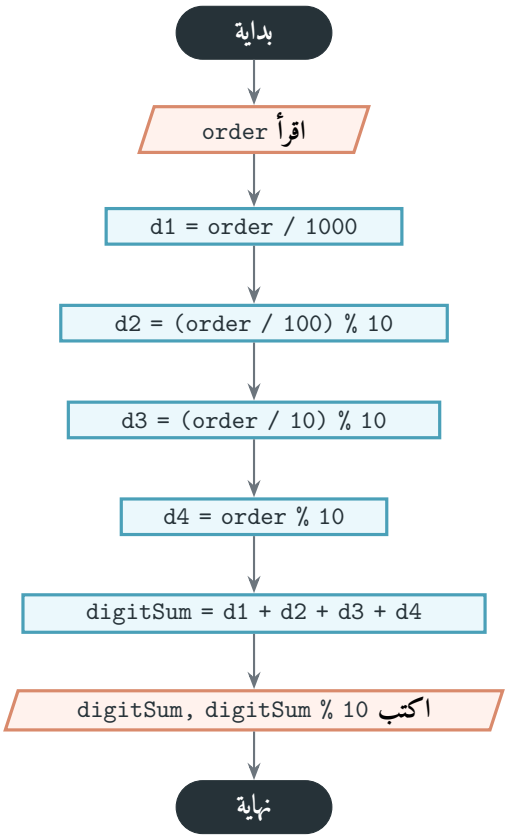
المدخلات

```
5837
```

نتيجة التنفيذ

```
Digit sum = 23
Check digit = 3
```

المخطط الإنسيابي (Organigramme)



تمرين 1.23 تقريب السعر إلى منزلتين

تجارة إلكترونية ★★ متوسط

ينتج حساب الضرائب أحيانًا أسعارًا مثل 1234.5678. اكتب برنامجًا يقرب سعرًا إلى **منزلتين عشريتين** دون دالة round، ثم إلى **أقرب 10 دنانير**.

خطوات الحل

1) نضرب في 100 ونضيف 0.5 ثم نحول إلى `int` (يحذف الجزء العشري).
2) نقسم على `100.0` لنرجع إلى الدينار.
3) لأقرب 10: `((int)(price / 10 + 0.5)) * 10`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, r2;
    int r10;
    scanf("%lf", &price);
    // إضافة 0.5 ثم الحذف بالتحويل إلى int = تقريب لأقرب عدد صحيح
    r2 = (int)(price * 100 + 0.5) / 100.0;
    r10 = (int)(price / 10 + 0.5) * 10;      // تقريب لأقرب 10 دنانير
    printf("Rounded (2 decimals) = %.4f\n", r2);
    printf("Rounded (10 DA) = %d\n", r10);
    return 0;
}
```

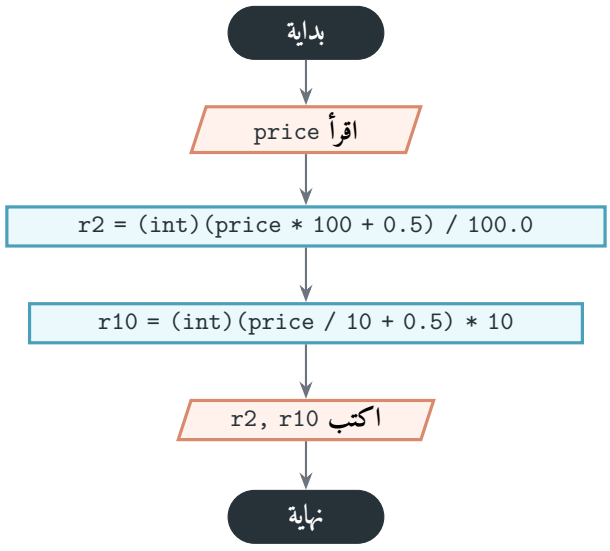
المدخلات

```
1234.5678
```

نتيجة التنفيذ

```
Rounded (2 decimals) = 1234.5700
Rounded (10 DA) = 1230
```

المخطط الانسيابي (Organigramme)



تمرين 1.24 مؤشرات الحملة الإعلانية

علم البيانات ★★ متوسط

حملة إعلانية حققت عددًا من **مرات الظهور** والنقرات وكلفت مبلغًا معينًا. اكتب برنامجًا يحسب نسبة النقر CTR وتكلفة النقرة CPC وتكلفة الألف ظهور CPM.

خطوات الحل

1) `CTR = clicks / impressions * 100`.
2) `CPC = cost / clicks`.
3) `CPM = cost / impressions * 1000`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double imp, clicks, cost, ctr, cpc, cpm;
    scanf("%lf %lf %lf", &imp, &clicks, &cost);
    ctr = clicks / imp * 100;      // نسبة النقر بالمئة
    cpc = cost / clicks;           // تكلفة النقرة الواحدة
    cpm = cost / imp * 1000;       // تكلفة كل ألف ظهور
    printf("CTR = %.2f %%\n", ctr);
    printf("CPC = %.2f DA\n", cpc);
    printf("CPM = %.2f DA\n", cpm);
    return 0;
}
```

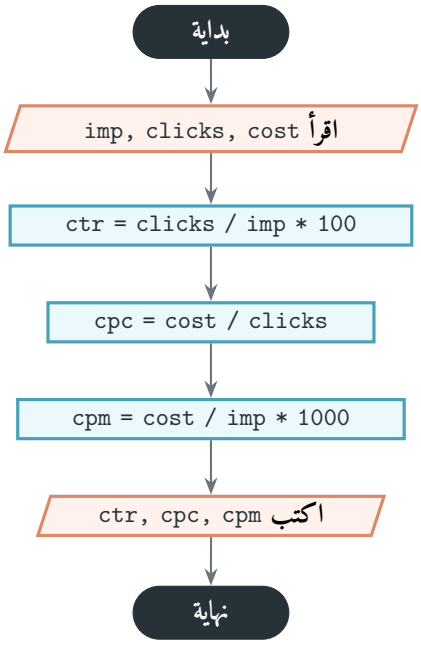
المدخلات

```
120000 3000 45000
```

نتيجة التنفيذ

```
CTR = 2.50 %
CPC = 15.00 DA
CPM = 375.00 DA
```

المخطط الانسيابي (Organigramme)



تمرين 1.25 القيمة الدائمة للزبون

علم البيانات ★★ متوسط

القيمة الدائمة للزبون CLV تقدر بـ: متوسط السلة × عدد الطلبيات في السنة × عدد سنوات الوفاء × هامش الربح. اكتب برنامجًا يحسبها.

خطوات الحل

1) نقرأ `basket` و`freq` و`years` و`margin`.
2) رقم الأعمال المتوقع: `rev = basket * freq * years`.
3) `clv = rev * margin`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double basket, margin, rev, clv;
    int freq, years;
    scanf("%lf %d %d %lf", &basket, &freq, &years, &margin);
    rev = basket * freq * years;  // رقم الأعمال طوال مدة الوفاء
    clv = rev * margin;           // الربح الذي يحققه الزبون
    printf("Expected revenue = %.2f DA\n", rev);
    printf("CLV = %.2f DA\n", clv);
    return 0;
}
```

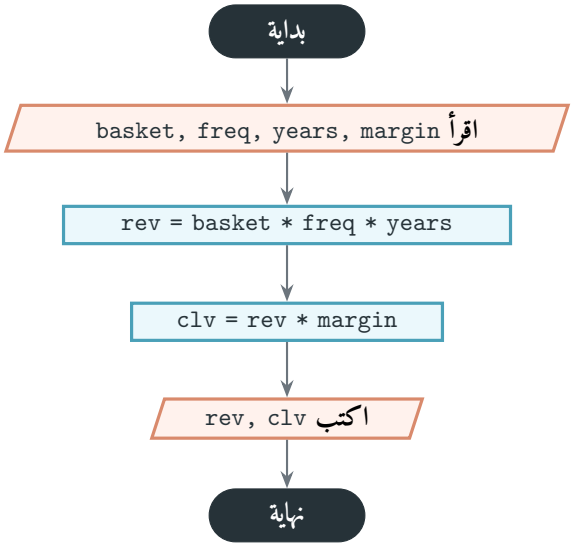
المدخلات

```
3200 6 4 0.25
```

نتيجة التنفيذ

```
Expected revenue = 76800.00 DA
CLV = 19200.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.26 متوسط قيمة الطلبية

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ رقم الأعمال (عدد صحيح) وعدد الطلبيات، ثم يعرض متوسط قيمة الطلبية مرة **بقسمة صحيحة** ومرة **بقسمة حقيقية**، ليوضح الفرق بينهما.

خطوات الحل

1) القسمة بين عددين صحيحين تحذف الجزء العشري: `rev / nbOrders`.
2) للحصول على قسمة حقيقية نحول: `(double)rev / nbOrders`.
3) الفرق بين النتيجتين هو الجزء الضائع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rev, nbOrders, quotient;
    double avg;
    scanf("%d %d", &rev, &nbOrders);
    quotient = rev / nbOrders;                  // قسمة صحيحة: الجزء العشري يضيع
    avg = (double)rev / nbOrders;               // قسمة حقيقية
    printf("Integer division = %d\n", quotient);
    printf("Real division    = %.4f\n", avg);
    printf("Lost part        = %.4f\n", avg - quotient);
    return 0;
}
```

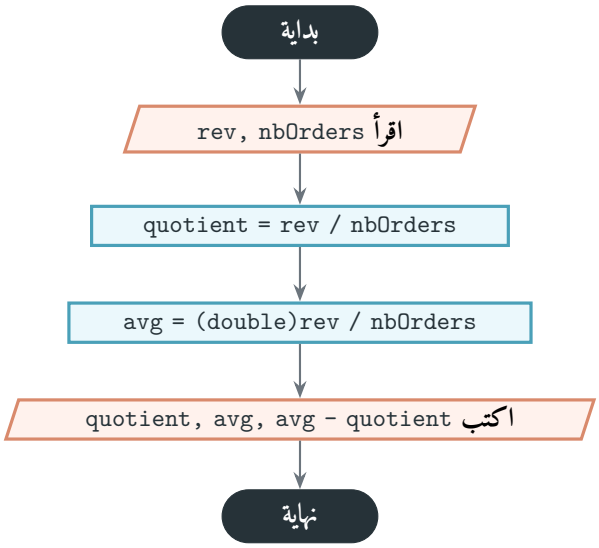
المدخلات

```
98765 23
```

نتيجة التنفيذ

```
Integer division = 4294
Real division    = 4294.1304
Lost part        = 0.1304
```

المخطط الإنسيابي (Organigramme)



تمرين 1.27 أيام تغطية المخزون

تجارة إلكترونية متوسط

مخزون منتج في بداية الأسبوع `stock`. بيعت منه `sold` وحدة ووصلت شحنة من `rec` وحدة. اكتب برنامجًا يحسب المخزون الجديد وعدد الأيام **الكاملة** التي يغطيها إذا كان البيع اليومي `daily`.

خطوات الحل

1) المخزون الجديد: `stock = stock - sold + rec`.
2) أيام التغطية: `days = stock / daily` (قسمة صحيحة).
3) الوحدات المتبقية بعد آخر يوم كامل: `stock % daily`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stock, sold, rec, daily, days, rest;
    scanf("%d %d %d %d", &stock, &sold, &rec, &daily);
    stock = stock - sold + rec;          // تحديث المخزون
    days = stock / daily;                // عدد الأيام الكاملة
    rest = stock % daily;                // ما يبقى بعدها
    printf("New stock = %d\n", stock);
    printf("Covers %d full days (%d left)\n", days, rest);
    return 0;
}
```

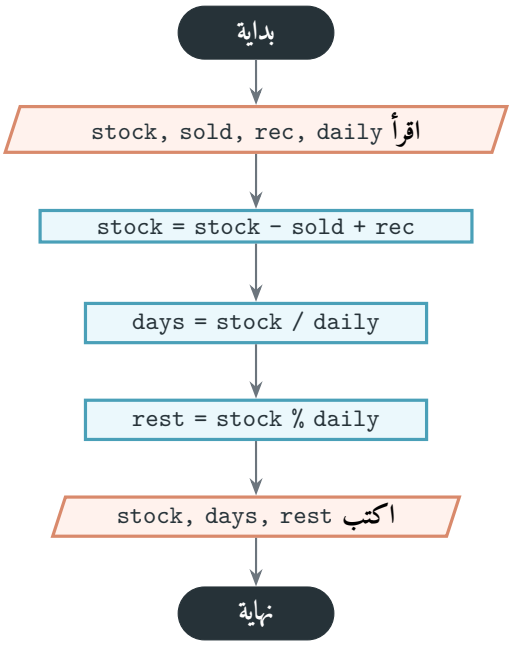
المدخلات

```
340 215 500 45
```

نتيجة التنفيذ

```
New stock = 625
Covers 13 full days (40 left)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.28 التنبؤ بنموذج خطي

علم البيانات ★★ متوسط

درّب محلل بيانات نموذجًا خطيًا يتنبأ بالمبيعات `y` بدلالة ميزانية الإعلان `x` (بالآلاف): `y = slope*x + intercept`.
اكتب برنامجًا يقرأ `slope` و`intercept` وميزانيتين ويعرض التنبؤين والفرق بينهما.

خطوات الحل

1) نقرأ معاملي النموذج `slope` و`intercept`.
2) نقرأ الميزانيتين `x1` و`x2`.
3) نحسب `y1 = slope*x1 + intercept` و`y2 = slope*x2 + intercept` والفرق `y2 - y1`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double slope, intercept, x1, x2, y1, y2;
    scanf("%lf %lf", &slope, &intercept);      // معاملا النموذج
    scanf("%lf %lf", &x1, &x2);    // ميزانيتان
    y1 = slope * x1 + intercept;               // التنبؤ الأول
    y2 = slope * x2 + intercept;               // التنبؤ الثاني
    printf("Prediction 1 = %.1f\n", y1);
    printf("Prediction 2 = %.1f\n", y2);
```

```c
    printf("Gain = %.1f\n", y2 - y1);   // = slope × (x2 - x1)
    return 0;
}
```

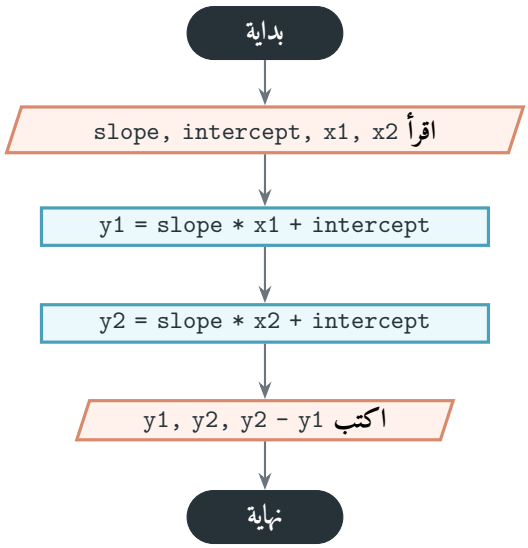
المدخلات

```
3.2 150
20 35
```

نتيجة التنفيذ

```
Prediction 1 = 214.0
Prediction 2 = 262.0
Gain = 48.0
```

المخطط الانسيابي (Organigramme)



تمرين 1.29 مستقيم يمر بنقطتين

علم البيانات ★★ متوسط

لاحظنا أن سعر 100 دج يحقق 900 مبيعة، وسعر 150 دج يحقق 650 مبيعة. اكتب برنامجًا يقرأ نقطتين `(x1, y1)` و`(x2, y2)` ويحسب **الميل** و**الجزء المقطوع** للمستقيم المار بهما، ثم يتنبأ بالمبيعات عند سعر 120.

خطوات الحل

1) الميل: `slope = (y2 - y1) / (x2 - x1)`.
2) الجزء المقطوع: `intercept = y1 - slope * x1`.
3) التنبؤ: `sales = slope * 120 + intercept`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, slope, intercept, sales;
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);
    slope = (y2 - y1) / (x2 - x1);     // الميل: تغير sales لكل وحدة من x
```

```c
    intercept = y1 - slope * x1;              // من intercept + slope*x1 = y1
    sales = slope * 120 + intercept;              // التنبؤ عند السعر 120
    printf("Slope a = %.2f\n", slope);
    printf("Intercept b = %.2f\n", intercept);
    printf("Sales at 120 DA = %.0f\n", sales);
    return 0;
}
```

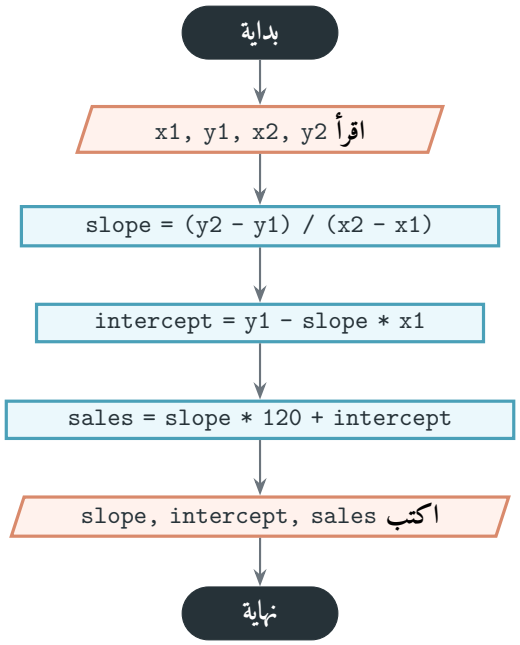
المدخلات

```
100 900
150 650
```

نتيجة التنفيذ

```
Slope a = -5.00
Intercept b = 1400.00
Sales at 120 DA = 800
```

المخطط الإنسيابي (Organigramme)



تمرين 1.30 حصص الأصناف من رقم الأعمال

علم البيانات ★★ متوسط

حقق متجر رقم أعمال في ثلاثة أصناف: إلكترونيات، ملابس، كتب. اكتب برنامجًا يحسب حصة كل صنف بالمئة، ويتحقق أن مجموع الحصص 100%.

خطوات الحل

1) المجموع: `total = electronics + clothing + books`.
2) حصة كل صنف: `/ total * 100` .
3) نعرض مجموع الحصص للتحقق.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double electronics, clothing, books, total, pe, pc, pb;
    scanf("%lf %lf %lf", &electronics, &clothing, &books);
    total = electronics + clothing + books;              // رقم الأعمال الكلي
    pe = electronics / total * 100;                      // حصة الإلكترونيات
    pc = clothing / total * 100;                         // حصة الملابس
    pb = books / total * 100;                            // حصة الكتب
    printf("Electronics: %.1f %%\n", pe);
    printf("Clothing:    %.1f %%\n", pc);
    printf("Books:       %.1f %%\n", pb);
    printf("Check sum:   %.1f %%\n", pe + pc + pb);
    return 0;
}
```

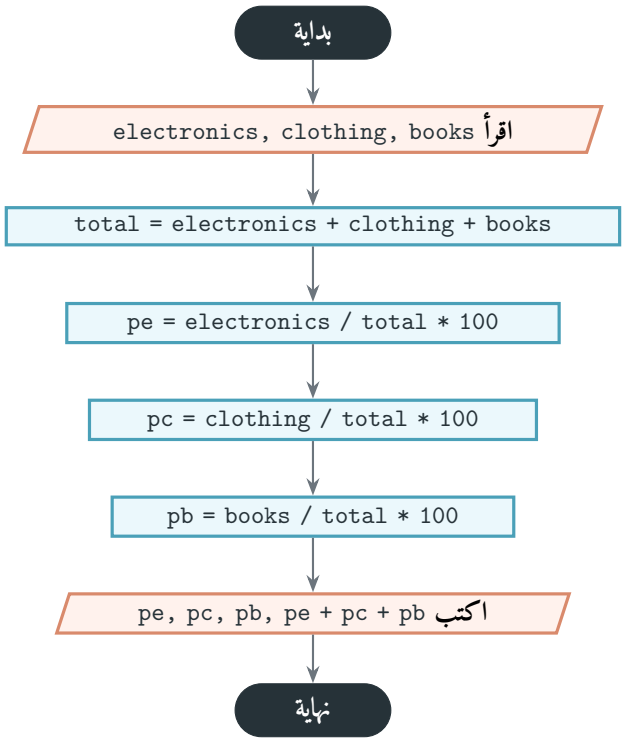
المدخلات

```
540000 310000 150000
```

نتيجة التنفيذ

```
Electronics: 54.0 %
Clothing:    31.0 %
Books:       15.0 %
Check sum:   100.0 %
```

المخطط الإنسيابي (Organigramme)



تمرين 1.31 الطابع الزمني إلى ساعة

علم البيانات ★★ متوسط

تسجل قاعدة البيانات وقت كل نقرة بعدد الثواني منذ منتصف الليل. اكتب برنامجًا يحول هذا العدد إلى الصيغة `hh:mm:ss`.

خطوات الحل

1) الساعات: `hours = timestamp / 3600`.
2) الدقائق: `minutes = (timestamp % 3600) / 60`.
3) الثواني: `seconds = timestamp % 60`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int timestamp, hours, minutes, seconds;
    scanf("%d", &timestamp);              // ثوانٍ منذ منتصف الليل
    hours = timestamp / 3600;                  // 3600 ثانية في الساعة
    minutes = (timestamp % 3600) / 60;           // ما تبقى من الساعة بالدقائق
    seconds = timestamp % 60;                    // الثواني المتبقية
    printf("%d s -> %02d:%02d:%02d\n", timestamp, hours, minutes, seconds);
    return 0;
}
```

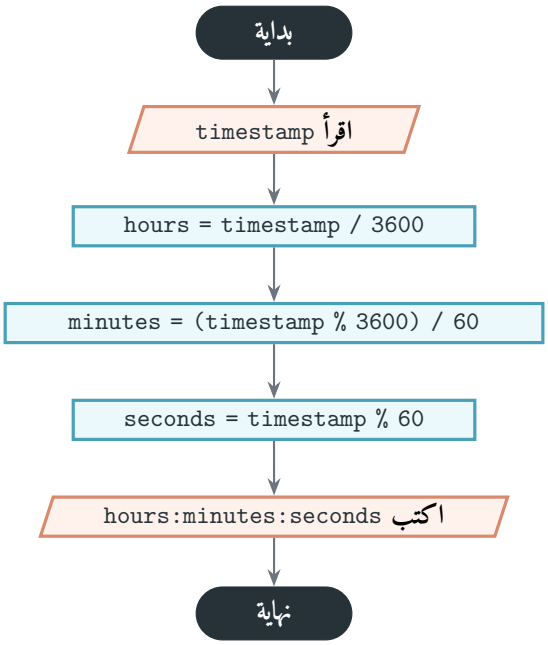
المدخلات

```
53917
```

نتيجة التنفيذ

```
53917 s -> 14:58:37
```

المخطط الإنسيابي (Organigramme)



تمرين 1.32 فاتورة كاملة — تجارة إلكترونية — متوسط

تحتوي سلة على ثلاثة منتجات (سعر وكمية لكل منها). يضاف الرسم 19% ثم مصاريف شحن ثابتة 600 دج. اكتب برنامجًا يعرض المجموع دون رسم، الرسم، والمبلغ النهائي.

خطوات الحل

1) نقرأ السعر والكمية لكل منتج.
2) `ht = p1*q1 + p2*q2 + p3*q3`.
3) `tax = ht * TVA` ثم `total = ht + tax + SHIP`.

البرنامج بلغة C

```c
#include <stdio.h>

#define TVA  0.19                 // نسبة الرسم
#define SHIP 600.0                // مصاريف الشحن الثابتة

int main(void) {
    double p1, p2, p3, ht, tax, total;
    int q1, q2, q3;
    scanf("%lf %d", &p1, &q1);    // المنتج الأول
    scanf("%lf %d", &p2, &q2);    // المنتج الثاني
    scanf("%lf %d", &p3, &q3);    // المنتج الثالث
    ht = p1 * q1 + p2 * q2 + p3 * q3;   // مجموع الأسطر
    tax = ht * TVA;
    total = ht + tax + SHIP;
    printf("Subtotal HT = %10.2f\n", ht);
    printf("Tax 19%%     = %10.2f\n", tax);
    printf("Shipping    = %10.2f\n", SHIP);
    printf("TOTAL       = %10.2f DA\n", total);
    return 0;
}
```

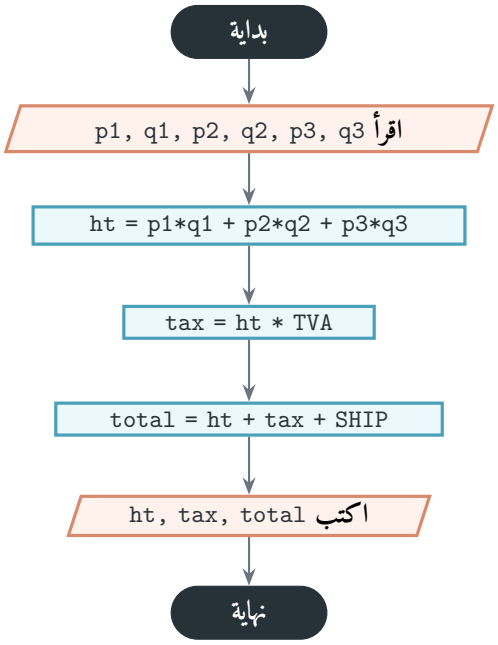
المدخلات

```
1200 2
450 4
3999 1
```

نتيجة التنفيذ

```
Subtotal HT =    8199.00
Tax 19%     =    1557.81
Shipping    =     600.00
TOTAL       =   10356.81 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.33 مقارنة سعر الكيلوغرام

تجارة إلكترونية ★★ متوسط

يعرض المتجر نفس القهوة في عبوتين: 250 غ بسعر p1 و1 كغ بسعر p2. اكتب برنامجًا يحسب **سعر الكيلوغرام** لكل عبوة والفرق بينهما والتوفير بالمئة عند شراء العبوة الكبيرة.

خطوات الحل

1) سعر الكيلوغرام: `price / grams * 1000`.
2) الفرق: `k1 - k2`.
3) التوفير بالمئة: `(k1 - k2) / k1 * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double g1, p1, g2, p2, k1, k2, save;
    scanf("%lf %lf", &g1, &p1);   // الوزن بالغرام والسعر للعبوة 1
    scanf("%lf %lf", &g2, &p2);   // العبوة 2
    k1 = p1 / g1 * 1000;          // سعر الكيلوغرام للعبوة 1
    k2 = p2 / g2 * 1000;          // سعر الكيلوغرام للعبوة 2
    save = (k1 - k2) / k1 * 100;  // التوفير النسبي
    printf("Pack 1: %.2f DA/kg\n", k1);
    printf("Pack 2: %.2f DA/kg\n", k2);
    printf("Difference: %.2f DA/kg (%.1f %% saved)\n", k1 - k2, save);
    return 0;
}
```

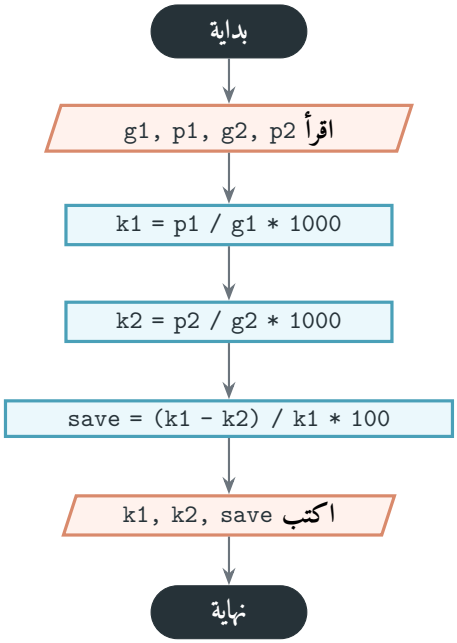
المدخلات

```
250 380
1000 1350
```

نتيجة التنفيذ

```
Pack 1: 1520.00 DA/kg
Pack 2: 1350.00 DA/kg
Difference: 170.00 DA/kg (11.2 % saved)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.34 نقاط الوفاء

تجارة إلكترونية متوسط

يحصل الزبون على **نقطة وفاء** عن كل 100 دج مشتريات كاملة، وكل 50 نقطة تساوي قسيمة بـ 500 دج. اكتب برنامجًا يحسب عدد النقاط والمبلغ الناقص للنقطة التالية وعدد القسائم.

خطوات الحل

1) النقاط: `pts = amount / 100`.
2) الناقص للنقطة التالية: `100 - amount % 100`.
3) القسائم: `pts / 50` وقيمتها `× 500`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int amount, pts, missing, vouchers;
    scanf("%d", &amount);
    pts = amount / 100;              // نقطة لكل 100 دج كاملة
    missing = 100 - amount % 100;    // ما ينقص للوصول إلى المئة التالية
    vouchers = pts / 50;             // قسيمة لكل 50 نقطة
```

```c
    printf("Points = %d\n", pts);
    printf("Next point in %d DA\n", missing);
    printf("Vouchers = %d (%d DA)\n", vouchers, vouchers * 500);
    return 0;
}
```

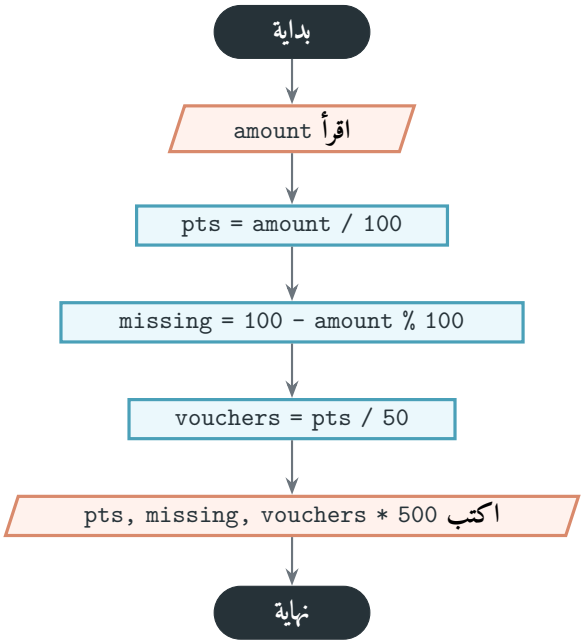
المدخلات

```
12370
```

نتيجة التنفيذ

```
Points = 123
Next point in 30 DA
Vouchers = 2 (1000 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 1.35 متوسط مربع الخطأ

علم البيانات متوسط

تنبأ نموذج بمبيعات ثلاثة أيام. اكتب برنامجًا يقرأ لكل يوم القيمة الحقيقية y والمتوقعة p، ويحسب الأخطاء ومتوسط مربع الخطأ MSE.

خطوات الحل

1) الخطأ لكل يوم: e = y - p.
2) نجمع مربعات الأخطاء.
3) MSE = / 3.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double y1, p1, y2, p2, y3, p3, e1, e2, e3, mse;
    scanf("%lf %lf", &y1, &p1);
    scanf("%lf %lf", &y2, &p2);
    scanf("%lf %lf", &y3, &p3);
    e1 = y1 - p1;                 // خطأ اليوم 1
    e2 = y2 - p2;                 // خطأ اليوم 2
    e3 = y3 - p3;                 // خطأ اليوم 3
    // التربيع يجعل كل الأخطاء موجبة ويعاقب الأخطاء الكبيرة
    mse = (e1 * e1 + e2 * e2 + e3 * e3) / 3;
    printf("Errors: %.1f %.1f %.1f\n", e1, e2, e3);
    printf("MSE = %.2f\n", mse);
    return 0;
}
```

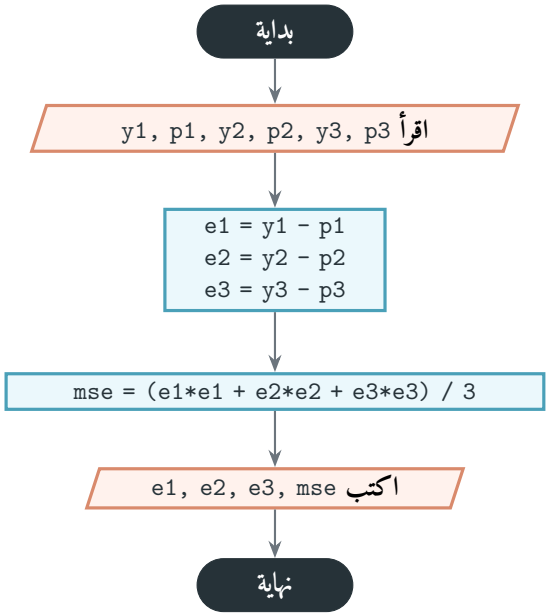
المدخلات

```
100 92
80 85
120 111
```

نتيجة التنفيذ

```
Errors: 8.0 -5.0 9.0
MSE = 56.67
```

المخطط الانسيابي (Organigramme)



تمرين 1.36 السعر بعد التضخم

علم البيانات ★★ متوسط

سعر منتج اليوم `price`، ونسبة التضخم السنوية `inflation` بالمئة. اكتب برنامجًا يحسب سعره المتوقع بعد سنتين، وكذلك

القيمة الحقيقية لمبلغ 10000 دج بعد سنتين (أي قسمته على معامل التضخم مرتين).

خطوات الحل

1) المعامل: `factor = 1 + inflation / 100`.
2) السعر بعد سنتين: `price * factor * factor`.
3) القيمة الحقيقية: `10000 / (factor * factor)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, inflation, factor, p2, real;
    scanf("%lf %lf", &price, &inflation);
    factor = 1 + inflation / 100;              // معامل التضخم السنوي
    p2 = price * factor * factor;              // سنتان = ضرب مرتين
    real = 10000 / (factor * factor);          // القدرة الشرائية لـ 10000 دج
    printf("Price in 2 years = %.2f DA\n", p2);
    printf("Real value of 10000 DA = %.2f DA\n", real);
    return 0;
}
```

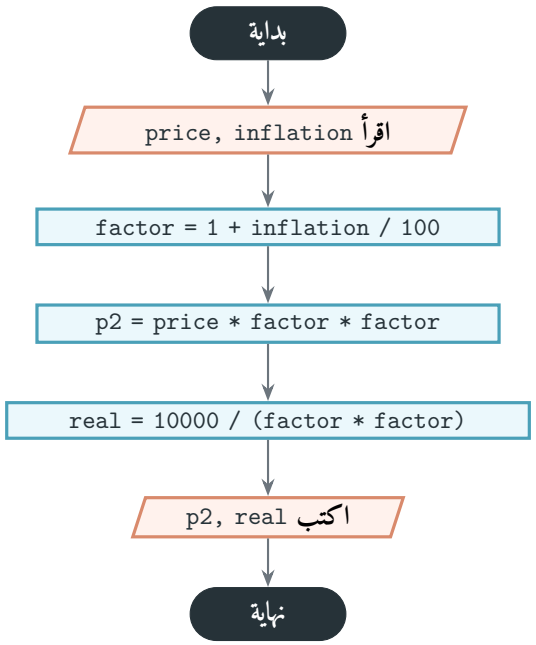
المدخلات

```
4500 9
```

نتيجة التنفيذ

```
Price in 2 years = 5346.45 DA
Real value of 10000 DA = 8416.80 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.37 تبديل دون متغير مساعد

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يبدل كميتي مخزون في مستودعين `stockA` و`stockB` **دون استعمال متغير ثالث**، باستعمال الجمع والطرح فقط.

خطوات الحل

1) `stockA = stockA + stockB` (stockA تحتوي المجموع).
2) `stockB = stockA - stockB` (stockB تأخذ القيمة القديمة لـ stockA).
3) `stockA = stockA - stockB` (stockA تأخذ القيمة القديمة لـ stockB).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stockA, stockB;
    scanf("%d %d", &stockA, &stockB);
    printf("Before: stockA = %d, stockB = %d\n", stockA, stockB);
    stockA = stockA + stockB;        // stockA = a0 + b0
    stockB = stockA - stockB;        // stockB = a0 + b0 - b0 = a0
    stockA = stockA - stockB;        // stockA = a0 + b0 - a0 = b0
    printf("After:  stockA = %d, stockB = %d\n", stockA, stockB);
    return 0;
}
```

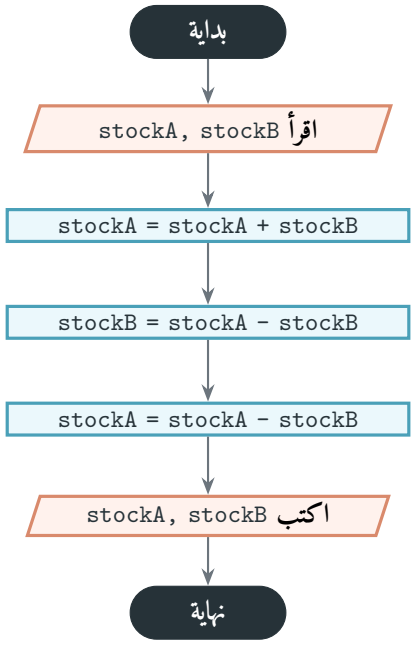
المدخلات

```
75 320
```

نتيجة التنفيذ

```
Before: stockA = 75, stockB = 320
After:  stockA = 320, stockB = 75
```

المخطط الإنسيابي (Organigramme)



تمرين 1.38 المبلغ بالسنتيم

تجارة إلكترونية متوسط

لتجنب أخطاء الأعداد الحقيقية، يخزن نظام الدفع المبالغ بالسنتيم كأعداد صحيحة. اكتب برنامجًا يقرأ مبلغين بالسنتيم ويعرض مجموعهما بالصيغة DA.cc.

خطوات الحل

1) نجمع المبلغين كأعداد صحيحة: totalCents = centsA + centsB.
2) الدينار: totalCents / 100، السنتيم: totalCents % 100.
3) نعرض السنتيم برقمين %02d.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int centsA, centsB, totalCents;
    scanf("%d %d", &centsA, &centsB);        // مبلغان بالسنتيم
    totalCents = centsA + centsB;                    // جمع دقيق بلا أخطاء تقريب
    // 02d% تضمن ظهور الصفر: 5 سنتيم تكتب 05
    printf("Total = %d.%02d DA\n", totalCents / 100, totalCents % 100);
    return 0;
}
```

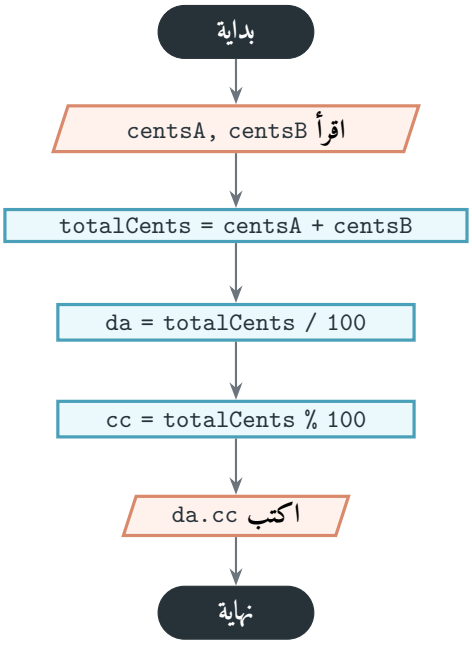
المدخلات

```
159990 45025
```

نتيجة التنفيذ

```
Total = 2050.15 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.39 رموز الأصناف والحروف

تجارة إلكترونية ★★ متوسط

يرمز لكل صنف بحرف كبير (A، B، ...). اكتب برنامجًا يقرأ رمز صنف ويعرض **شفرته** ASCII، والحرف **التالي**، و**ترتيب** الصنف (A هو 1)، والحرف الصغير الموافق دون دالة `tolower`.

خطوات الحل

1) الحرف في C عدد صحيح، فنعرضه بـ `%d` لنحصل على شفرته.
2) التالي: `category + 1`، الترتيب: `category - 'A' + 1`.
3) الحرف الصغير: `category + ('a' - 'A')` لأن الفرق ثابت (32).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char category, next, low;
    int rank;
    scanf(" %c", &category);
    next = category + 1;                // الحرف الموالي في جدول ASCII
    rank = category - 'A' + 1;          // المسافة عن A
    low = category + ('a' - 'A');       // الفرق بين الصغير والكبير ثابت
    printf("Code ASCII = %d\n", category);
    printf("Next category = %c\n", next);
    printf("Rank = %d\n", rank);
    printf("Lowercase = %c\n", low);
    return 0;
}
```

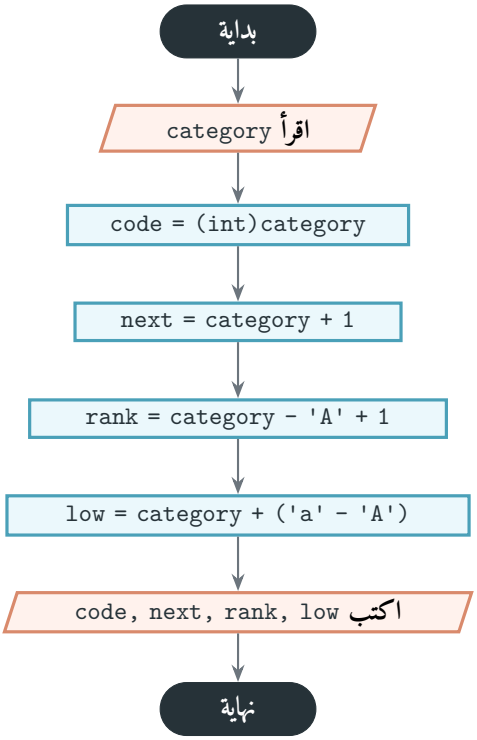
المدخلات

```
G
```

نتيجة التنفيذ

```
Code ASCII = 71
Next category = H
Rank = 7
Lowercase = g
```

المخطط الإنسيابي (Organigramme)



تمرين 1.40 أسبوع ويوم التسليم

تجارة إلكترونية ★★ متوسط

يُرقَّم كل يوم من السنة من 1 إلى 365، واليوم 1 يوم أحد. تُطلب سلعة في اليوم `orderDay` وتُسلَّم بعد `delay` يومًا. اكتب برنامجًا يعرض رقم يوم التسليم ورقم أسبوعه ورقم يومه في الأسبوع (0 = أحد).

خطوات الحل

1) يوم التسليم: `deliveryDay = orderDay + delay`.
2) الأسبوع: `(deliveryDay - 1) / 7 + 1`.
3) يوم الأسبوع: `(deliveryDay - 1) % 7`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orderDay, delay, deliveryDay, week, wd;
    scanf("%d %d", &orderDay, &delay);
```

```c
    deliveryDay = orderDay + delay;                    // رقم يوم التسليم في السنة
    week = (deliveryDay - 1) / 7 + 1;        // نطرح 1 لأن الترقيم يبدأ من 1
    wd = (deliveryDay - 1) % 7;              // 0 أحد، 1 إثنين، ...
    printf("Delivery day = %d\n", deliveryDay);
    printf("Week = %d, weekday = %d\n", week, wd);
    return 0;
}
```

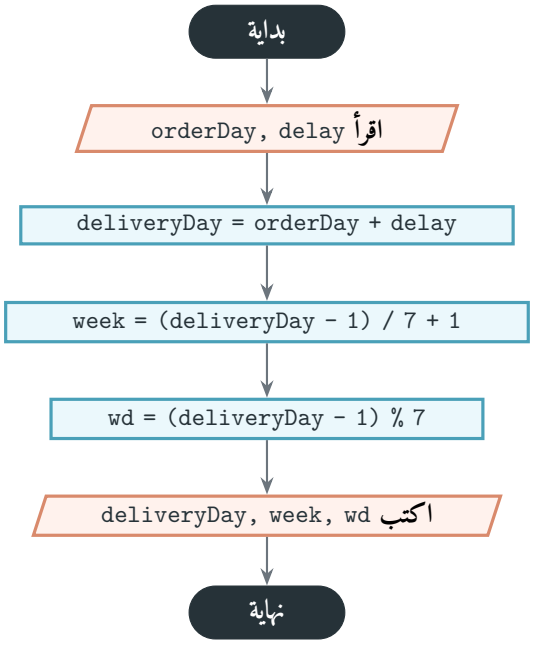
المدخلات

```
58 17
```

نتيجة التنفيذ

```
Delivery day = 75
Week = 11, weekday = 4
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 1.41 الأكبر بين سعرين دون شرط

تجارة إلكترونية صعب

اكتب برنامجًا يحدد أكبر وأصغر سعرين صحيحين دون استعمال `if` ولا العامل الشرطي. استعمل الصيغة `max = (priceA + priceB + |priceA - priceB|) / 2` واحسب القيمة المطلقة بعمليات على البتات.

خطوات الحل

1) `diff = priceA - priceB`.
2) قناع الإشارة: `mask = diff >> 31` يساوي 0 إذا كان `diff` موجبًا و-1 إذا كان سالبًا.
3) القيمة المطلقة: `ad = (diff ^ mask) - mask`.
4) `max = (priceA + priceB + ad) / 2` و`min = (priceA + priceB - ad) / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int priceA, priceB, diff, mask, ad, max, min;
    scanf("%d %d", &priceA, &priceB);
    diff = priceA - priceB;
    mask = diff >> 31;              // كل البتات 1 إذا كان diff سالبًا، وإلا 0
    ad = (diff ^ mask) - mask;      // إذا كان سالبًا: عكس البتات ثم +1 = -diff
    max = (priceA + priceB + ad) / 2;       // الأكبر
    min = (priceA + priceB - ad) / 2;       // الأصغر
    printf("|a - b| = %d\n", ad);
    printf("Max = %d, Min = %d\n", max, min);
    return 0;
}
```

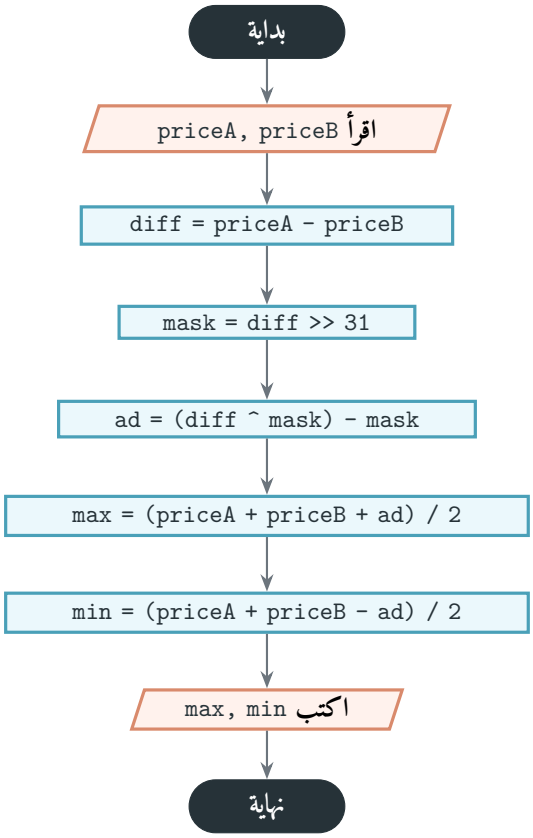
المدخلات

```
1750 2380
```

نتيجة التنفيذ

```
|a - b| = 630
Max = 2380, Min = 1750
```

المخطط الإنسيابي (Organigramme)



تمرين 1.42 الإنحراف المعياري بطريقة نيوتن

علم البيانات صعب ★★★

لدينا تقييمات ثلاثة منتجات. اكتب برنامجًا يحسب **التباين** ثم **الانحراف المعياري** دون دالة `sqrt`، باستعمال 6 تكرارات مكتوبة يدويًا لطريقة نيوتن: `root = (root + variance / root) / 2`.

خطوات الحل

1) نحسب المتوسط `mean` ثم التباين `variance`.
2) نبدأ بتقدير أولي `root = variance` (أو 1 إذا كان صغيرًا).
3) نطبق صيغة نيوتن ست مرات، فيقترب `root` بسرعة من الجذر.
4) نتحقق بحساب `root * root`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double rating1, rating2, rating3, mean, variance, root;
    scanf("%lf %lf %lf", &rating1, &rating2, &rating3);
    mean = (rating1 + rating2 + rating3) / 3;                       // المتوسط
    variance = ((rating1 - mean) * (rating1 - mean) + (rating2 - mean) * (rating2 - mean)
       + (rating3 - mean) * (rating3 - mean)) / 3;                  // التباين
    root = variance + 1;                                            // تقدير أولي أكبر من الجذر
    // كل تكرار يضاعف تقريبًا عدد الأرقام الصحيحة
```

```c
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    root = (root + variance / root) / 2;
    printf("Variance = %.6f\n", variance);
    printf("Std dev  = %.6f\n", root);
    printf("Check r*r = %.6f\n", root * root);      // يجب أن يساوي التباين
    return 0;
}
```

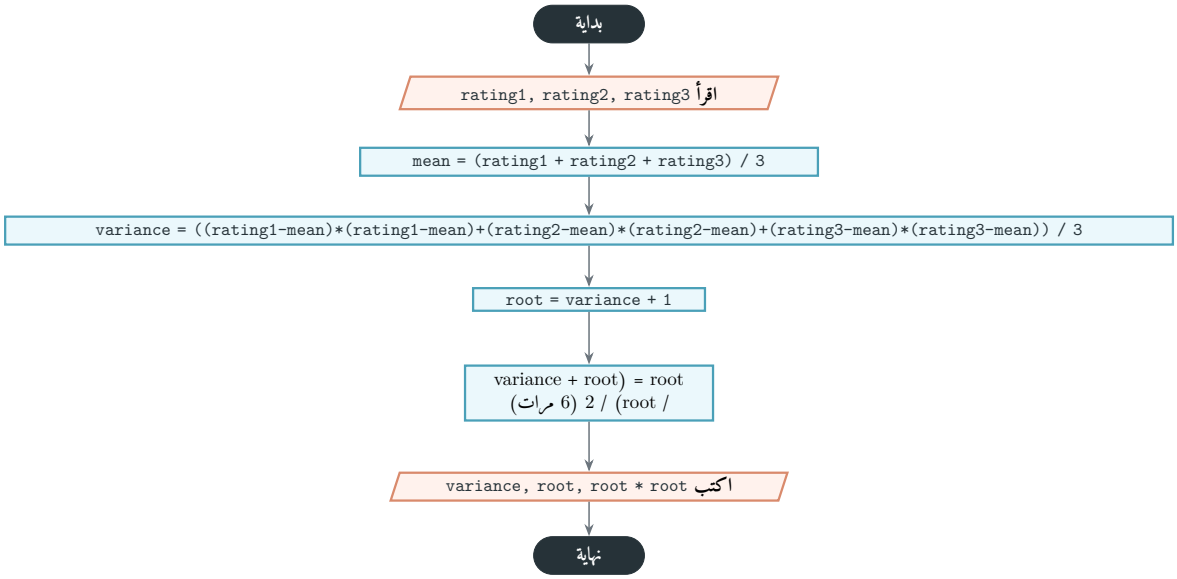
المدخلات

```
3.5 4.8 4.1
```

نتيجة التنفيذ

```
Variance = 0.282222
Std dev  = 0.531246
Check r*r = 0.282222
```

المخطط الإنسيابي (Organigramme)



تمرين 1.43 التغاير بين السعر والمبيعات

علم البيانات صعب ★★★

لدينا ثلاث ملاحظات (سعر، مبيعات). اكتب برنامجًا يحسب **التغاير** `cov` بين المتغيرين ويستنتج اتجاه العلاقة بطباعة إشارة التغاير، دون شرط: نطبع `cov > 0` كعدد (1 أو 0).

خطوات الحل

1) نحسب المتوسطين `mx` و`my`.
2) التغاير: متوسط الجداءات `(xi - mx) * (yi - my)`.
3) في C يعطي التعبير المنطقي `cov > 0` القيمة 1 أو 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, x3, y3, mx, my, cov;
    int pos;
    scanf("%lf %lf %lf %lf %lf %lf", &x1, &y1, &x2, &y2, &x3, &y3);
    mx = (x1 + x2 + x3) / 3;       // متوسط الأسعار
    my = (y1 + y2 + y3) / 3;       // متوسط المبيعات
    // التغاير: هل يتحرك المتغيران في نفس الاتجاه؟
    cov = ((x1 - mx) * (y1 - my) + (x2 - mx) * (y2 - my)
         + (x3 - mx) * (y3 - my)) / 3;
    pos = cov > 0;                 // تعبير منطقي: 1 صحيح، 0 خطأ
    printf("Mean price = %.1f, mean sales = %.1f\n", mx, my);
    printf("Covariance = %.2f\n", cov);
    printf("Positive relation? %d\n", pos);
    return 0;
}
```

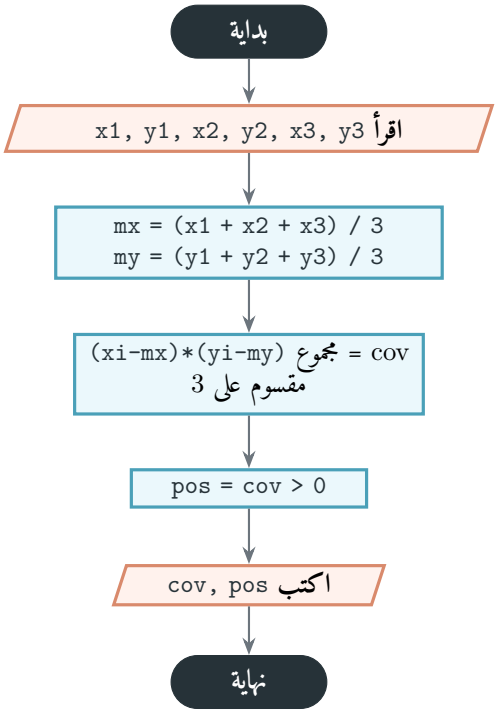
المدخلات

```
100 520
150 410
200 260
```

نتيجة التنفيذ

```
Mean price = 150.0, mean sales = 396.7
Covariance = -4333.33
Positive relation? 0
```

المخطط الإنسيابي (Organigramme)



تمرين 1.44 انحدار خطي بالمربعات الصغرى

علم البيانات صعب ★★★

اكتب برنامجًا يحسب معاملي **مستقيم الانحدار** `y = slope*x + intercept` بطريقة المربعات الصغرى لثلاث نقاط (ميزانية إعلان، مبيعات):

$$\text{slope} = \frac{n\sum xy - \sum x \sum y}{n\sum x^2 - (\sum x)^2}$$

و`intercept = (Sy - slope*Sx) / nbPoints`.

خطوات الحل

1) نحسب المجاميع `Sx` و`Sy` و`Sxy` و`Sxx`.
2) نطبق صيغة الميل `slope` ثم الجزء المقطوع `intercept`.
3) نتحقق بالتنبؤ عند `x = 3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double x1, y1, x2, y2, x3, y3, Sx, Sy, Sxy, Sxx, slope, intercept;
    const int nbPoints = 3;                    // عدد النقاط
    scanf("%lf %lf %lf %lf %lf %lf", &x1, &y1, &x2, &y2, &x3, &y3);
    Sx = x1 + x2 + x3;                         // مجموع x
    Sy = y1 + y2 + y3;                         // مجموع y
    Sxy = x1 * y1 + x2 * y2 + x3 * y3;         // مجموع الجداءات
    Sxx = x1 * x1 + x2 * x2 + x3 * x3;         // مجموع المربعات
    slope = (nbPoints * Sxy - Sx * Sy) / (nbPoints * Sxx - Sx * Sx);   // الميل
    intercept = (Sy - slope * Sx) / nbPoints;                          // الجزء المقطوع
    printf("y = %.4f x + %.4f\n", slope, intercept);
    printf("Prediction at x = 3: %.4f\n", slope * 3 + intercept);
    return 0;
}
```

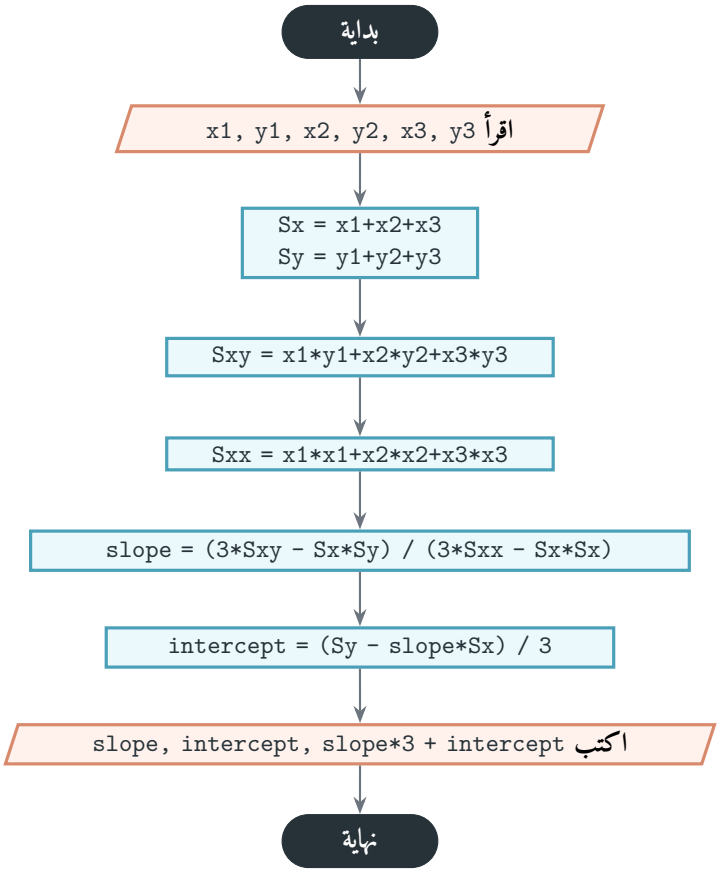
المدخلات

```
1 3
2 5
4 9
```

نتيجة التنفيذ

```
y = 2.0000 x + 1.0000
Prediction at x = 3: 7.0000
```

المخطط الإنسيابي (Organigramme)



تمرين 1.45 مدة المشاهدة الكلية

علم البيانات صعب ★★★

تجمع منصة فيديوهات المنتجات مدة المشاهدة الكلية بالثواني (عدد كبير). اكتب برنامجًا يحولها إلى **أيام وساعات ودقائق وثوانٍ**، ويحسب متوسط المشاهدة لكل زائر بالدقائق والثواني. استعمل النوع `long`.

خطوات الحل

1) الأيام: `totalSec / 86400`، ثم `totalSec % 86400`.
2) الساعات من الباقي: `/ 3600`، ثم الدقائق `/ 60`، ثم الثواني `% 60`.
3) متوسط الزائر: `avg = totalSec / visitors` ثم نفككه إلى دقائق وثوانٍ.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long totalSec, rest, days, hours, minutes, seconds, visitors, avg;
    scanf("%ld %ld", &totalSec, &visitors);      // الثواني الكلية، عدد الزوار
    days = totalSec / 86400;  rest = totalSec % 86400; // 86400 ثانية في اليوم
    hours = rest / 3600;    rest = rest % 3600;  // الساعات من الباقي
    minutes = rest / 60;      seconds = rest % 60;     // الدقائق والثواني
    avg = totalSec / visitors;                   // متوسط الزائر بالثواني
    printf("Total: %ld d %ld h %ld min %ld s\n", days, hours, minutes, seconds);
```

```
    printf("Per visitor: %ld min %ld s\n", avg / 60, avg % 60);
    return 0;
}
```

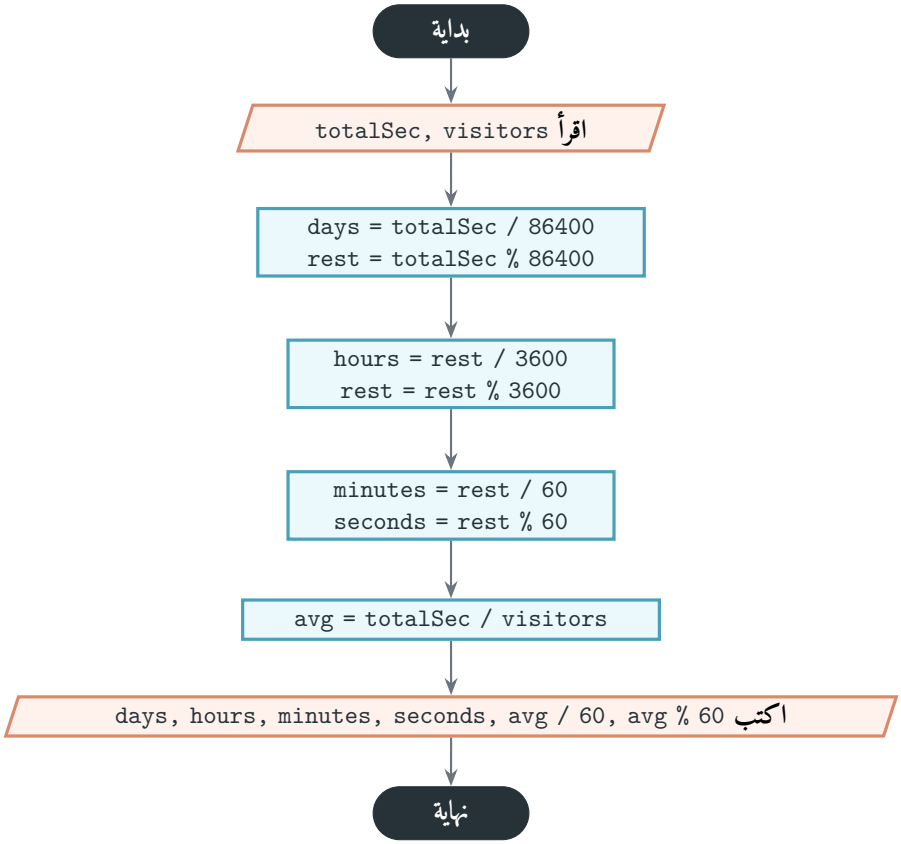
المدخلات

```
987654 1520
```

نتيجة التنفيذ

```
Total: 11 d 10 h 20 min 54 s
Per visitor: 10 min 49 s
```

المخطط الإنسيابي (Organigramme)



تمرين 1.46 خصائص المنتج بالبتات

تجارة إلكترونية صعب

تخزن خصائص منتج في عدد صحيح بين 0 و15، كل بت يمثل خاصية: البت 0 = متوفر، البت 1 = مخفض، البت 2 = شحن مجاني، البت 3 = جديد. اكتب برنامجًا يستخرج قيمة كل خاصية (0 أو 1) ويعرض التمثيل الثنائي.

خطوات الحل

1) البت `k` هو `(flags / 2^k) % 2`، أو `(flags >> k) & 1`.
2) نستخرج البتات الأربعة.
3) التمثيل الثنائي يُكتب من البت 3 إلى البت 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int flags, b0, b1, b2, b3;
    scanf("%d", &flags);
    b0 = flags & 1;              // البت 0: متوفر
    b1 = (flags >> 1) & 1;       // إزاحة لليمين ثم نأخذ آخر بت
    b2 = (flags >> 2) & 1;       // البت 2: شحن مجاني
    b3 = (flags >> 3) & 1;       // البت 3: جديد
    printf("Binary: %d%d%d%d\n", b3, b2, b1, b0);
    printf("In stock: %d\nOn sale: %d\n", b0, b1);
    printf("Free shipping: %d\nNew: %d\n", b2, b3);
    return 0;
}
```

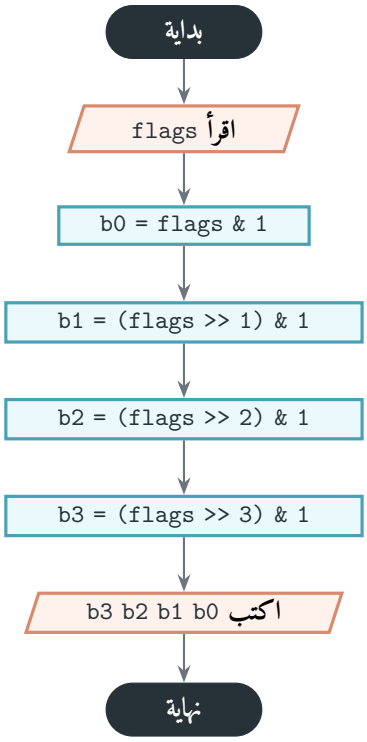
المدخلات

```
13
```

نتيجة التنفيذ

```
Binary: 1101
In stock: 1
On sale: 0
Free shipping: 1
New: 1
```

المخطط الانسيابي (Organigramme)



تمرين 1.47 المتوسط المتحرك الأسي علم البيانات صعب

لتنعيم سلسلة أسعار نستعمل **المتوسط المتحرك الأسي**: `ema = alpha * p + (1 - alpha) * ema`، مع `ema` يبدأ بالسعر الأول. اكتب برنامجًا يطبقه على أربعة أسعار ويعرض القيمة بعد كل سعر.

خطوات الحل

1) نقرأ `alpha` ثم الأسعار الأربعة.
2) `ema = p1`.
3) نحدّث `ema` بكل سعر جديد بالصيغة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double alpha, p1, p2, p3, p4, ema;
    scanf("%lf", &alpha);                  // معامل التنعيم بين 0 و1
    scanf("%lf %lf %lf %lf", &p1, &p2, &p3, &p4);
    ema = p1;                              // البداية: أول سعر
    printf("t1: %.2f\n", ema);
    ema = alpha * p2 + (1 - alpha) * ema;  // وزن alpha للسعر الجديد
    printf("t2: %.2f\n", ema);
    ema = alpha * p3 + (1 - alpha) * ema;
    printf("t3: %.2f\n", ema);
    ema = alpha * p4 + (1 - alpha) * ema;
    printf("t4: %.2f\n", ema);
    return 0;
}
```

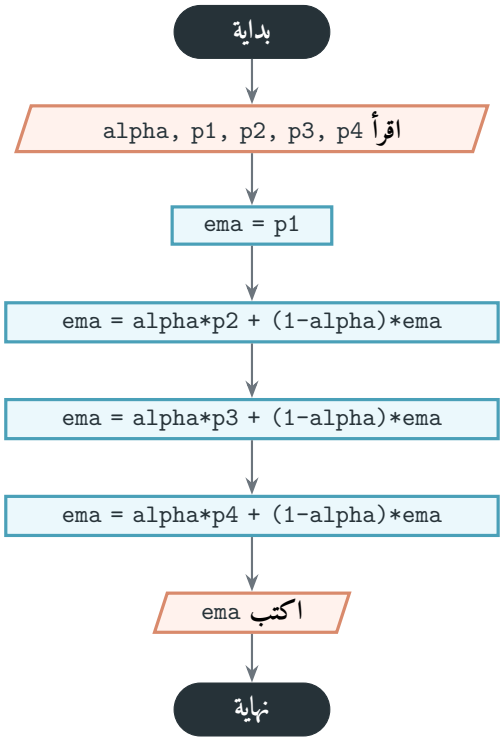
المدخلات

```
0.5
100 120 90 130
```

نتيجة التنفيذ

```
t1: 100.00
t2: 110.00
t3: 100.00
t4: 115.00
```

المخطط الإنسيابي (Organigramme)



تمرين 1.48 رقم التحقق لرمز EAN-8

تجارة إلكترونية صعب

رمز الأعمدة **EAN-8** يتكون من 7 أرقام ورقم تحقق. نضرب الأرقام في المواضع الفردية (1، 3، 5، 7) في 3 ونجمعها مع أرقام المواضع الزوجية، ورقم التحقق `(10 - weightedSum % 10) % 10`. اكتب برنامجًا يحسبه لعدد من 7 أرقام.

خطوات الحل

1) نستخرج الأرقام السبعة من اليسار إلى اليمين بالقسمة على قوى 10.

2) `weightedSum = 3*(d1 + d3 + d5 + d7) + (d2 + d4 + d6)`

.

3) `check = (10 - weightedSum % 10) % 10`، و% 10 الأخيرة تحول 10 إلى 0.

4) الرمز الكامل: `code * 10 + check`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long code;
    int d1, d2, d3, d4, d5, d6, d7, weightedSum, check;
    scanf("%ld", &code);
    d1 = code / 1000000;              // الرقم الأول من اليسار
    d2 = (code / 100000) % 10;
    d3 = (code / 10000) % 10;
    d4 = (code / 1000) % 10;
    d5 = (code / 100) % 10;
    d6 = (code / 10) % 10;
```

```c
    d7 = code % 10;                  // الرقم السابع
    weightedSum = 3 * (d1 + d3 + d5 + d7) + (d2 + d4 + d6);   // المجموع الموزون
    check = (10 - weightedSum % 10) % 10;   // ما يكمل weightedSum إلى مضاعف 10
    printf("Weighted sum = %d\n", weightedSum);
    printf("Check digit = %d\n", check);
    printf("EAN-8 = %ld\n", code * 10 + check);
    return 0;
}
```

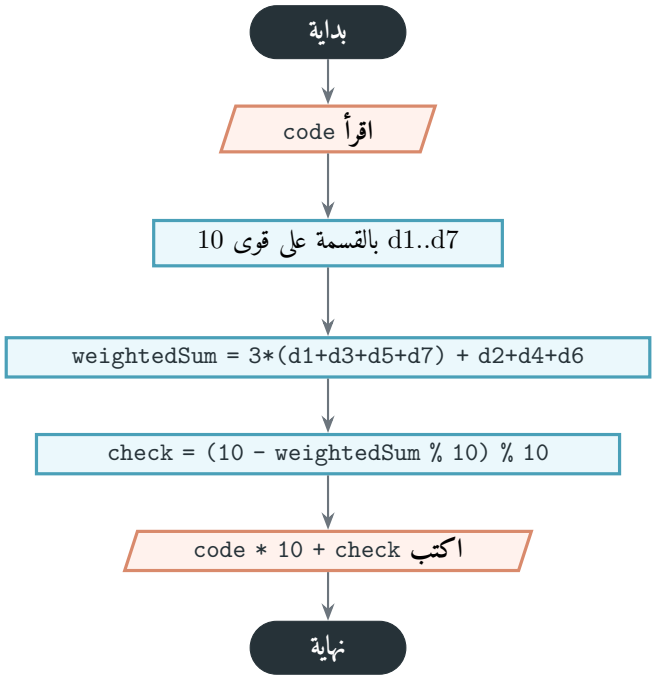
المدخلات

```
9638507
```

نتيجة التنفيذ

```
Weighted sum = 86
Check digit = 4
EAN-8 = 96385074
```

المخطط الإنسيابي (Organigramme)



تمرين 1.49 مرونة الطلب السعرية

علم البيانات صعب ★★★★

تقيس **مرونة الطلب** تأثر المبيعات بتغير السعر. بطريقة نقطة المنتصف:

```
elasticity = ((Q2 - Q1) / ((Q1 + Q2) / 2)) / ((P2 - P1) / ((P1 + P2) / 2))
```

. اكتب برنامجًا يحسبها، ويحسب رقم الأعمال قبل التغيير وبعده.

خطوات الحل

1) نقرأ `(P1, Q1)` و`(P2, Q2)`.
2) التغير النسبي للكمية: `dq = (Q2 - Q1) / ((Q1 + Q2) / 2)`.
3) التغير النسبي للسعر: `dp` بنفس الطريقة.

4) `elasticity = dq / dp`، ورقم الأعمال `P * Q`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double P1, Q1, P2, Q2, dq, dp, elasticity;
    scanf("%lf %lf %lf %lf", &P1, &Q1, &P2, &Q2);
    dq = (Q2 - Q1) / ((Q1 + Q2) / 2);      // التغير النسبي للكمية
    dp = (P2 - P1) / ((P1 + P2) / 2);      // التغير النسبي للسعر
    elasticity = dq / dp;                  // المرونة
    printf("Elasticity = %.3f\n", elasticity);
    printf("Revenue before = %.0f DA\n", P1 * Q1);
    printf("Revenue after  = %.0f DA\n", P2 * Q2);
    return 0;
}
```

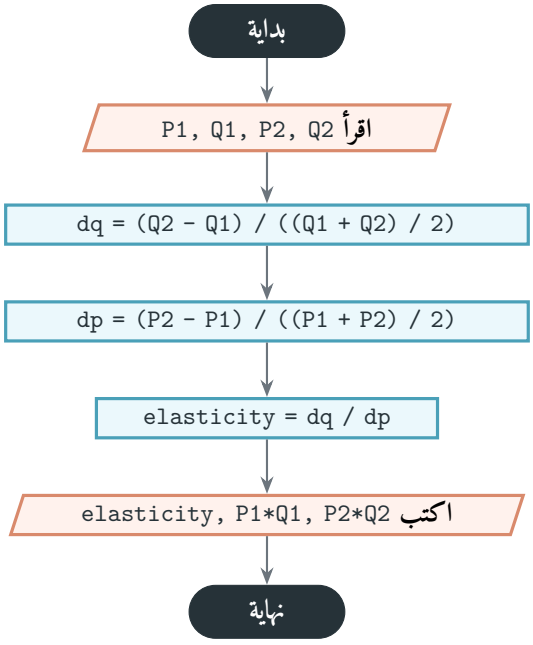
المدخلات

```
2000 500
2400 380
```

نتيجة التنفيذ

```
Elasticity = -1.500
Revenue before = 1000000 DA
Revenue after  = 912000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 1.50 تعبئة حاوية الشحن — تجارة إلكترونية — صعب ★★★

حاوية شحن أبعادها `contLength × contWidth × contHeight` (سم) ونريد ملأها بعلب أبعادها `boxLength × boxWidth × boxHeight` مرتبة في نفس الاتجاه. اكتب برنامجًا يحسب عدد العلب في كل بعد، والعدد الكلي، ونسبة **الحجم المستغل**.

خطوات الحل

1) عدد العلب في كل بعد: `contLength / boxLength` و`contWidth / boxWidth` و`contHeight / boxHeight` (قسمة صحيحة).
2) العدد الكلي: جداء الأعداد الثلاثة.
3) الحجم المستغل: `nbBoxes * boxLength * boxWidth * boxHeight`، والنسبة بالمئة من `contLength * contWidth * contHeight`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long contLength, contWidth, contHeight, boxLength, boxWidth, boxHeight, nx, ny, nz, nbBoxes, used;
    double pct;
    scanf("%ld %ld %ld", &contLength, &contWidth, &contHeight);     // أبعاد الحاوية
    scanf("%ld %ld %ld", &boxLength, &boxWidth, &boxHeight);     // أبعاد العلبة
    nx = contLength / boxLength;                    // علب على الطول
    ny = contWidth / boxWidth;                    // علب على العرض
    nz = contHeight / boxHeight;                    // طبقات على الارتفاع
    nbBoxes = nx * ny * nz;             // العدد الكلي
    used = nbBoxes * boxLength * boxWidth * boxHeight;         // الحجم المشغول بالعلب
    pct = (double)used / (contLength * contWidth * contHeight) * 100;
    printf("Grid: %ld x %ld x %ld\n", nx, ny, nz);
    printf("Boxes = %ld\n", nbBoxes);
    printf("Volume used = %.2f %%\n", pct);
    return 0;
}
```

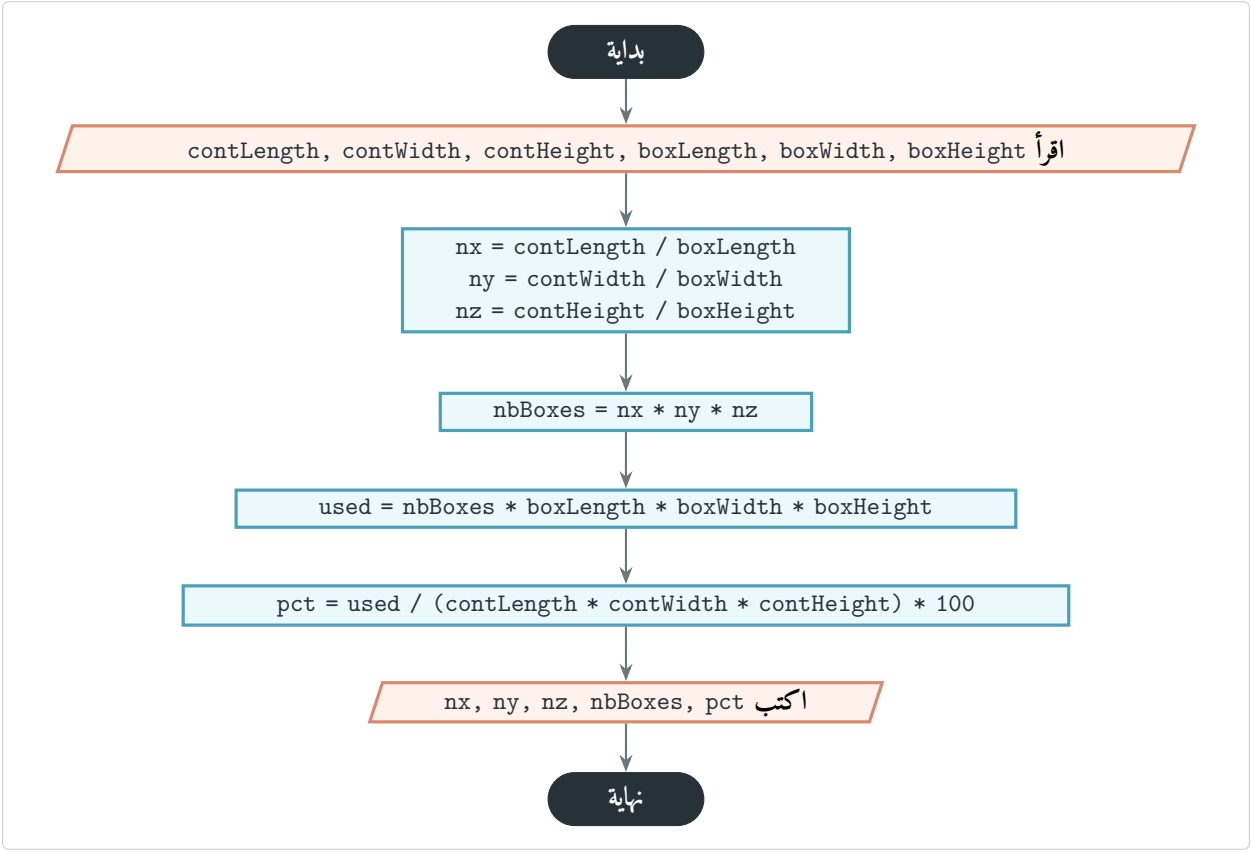
المدخلات

```
600 240 250
40 30 50
```

نتيجة التنفيذ

```
Grid: 15 x 8 x 5
Boxes = 600
Volume used = 100.00 %
```

المخطط الإنسيابي (Organigramme)

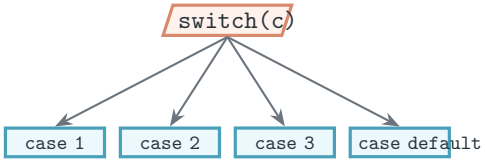


الفصل 2

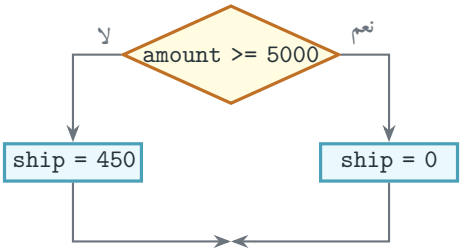
البنيات الشرطية

تعريفات

- **البنية الشرطية**: تنفذ مجموعة تعليمات **فقط إذا** تحقق شرط (`if`).
- **البديل**: `else` ينفذ عندما يكون الشرط خاطئًا، و`else if` يسمح بسلسلة من الحالات.
- **الاختيار المتعدد**: `switch` يقارن متغيرًا صحيحًا أو حرفًا بقيم ثابتة (`case`) و`break` ينهي الحالة.
- **الشرط**: تعبير منطقي قيمته 1 (صحيح) أو 0 (خطأ)، يُبنى بـ `>= > <= < != ==` ويُركب بـ `&& || !`.



`switch`



`if ... else`

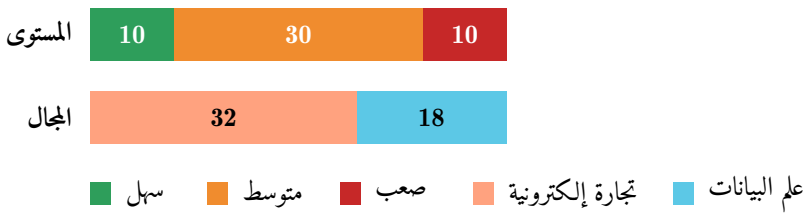
A	B	A && B	A || B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

أخطاء شائعة

`if (x = 5)` إسناد وليس مقارنة (اكتب ==)، ونسيان `break` في `switch` يجعل التنفيذ ينزل إلى الحالة الموالية.

للاستزادة: [1, 2, 3]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 2.1 الشحن المجاني

تجارة إلكترونية سهل

يقدم متجر **الشحن المجاني** للطلبيات التي يبلغ مبلغها 5000 دج أو أكثر، وإلا تضاف مصاريف شحن 450 دج. اكتب برنامجًا يقرأ مبلغ السلة ويعرض المبلغ النهائي.

خطوات الحل

1) نقرأ المبلغ `amount`.
2) إذا كان `amount >= 5000` فالشحن 0، وإلا 450.
3) نعرض الشحن والمبلغ النهائي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, ship;
    scanf("%lf", &amount);
    if (amount >= 5000)
        ship = 0;                   // الشحن مجاني
    else
        ship = 450;                 // مصاريف الشحن العادية
    printf("Shipping = %.2f DA\n", ship);
    printf("To pay = %.2f DA\n", amount + ship);
    return 0;
}
```

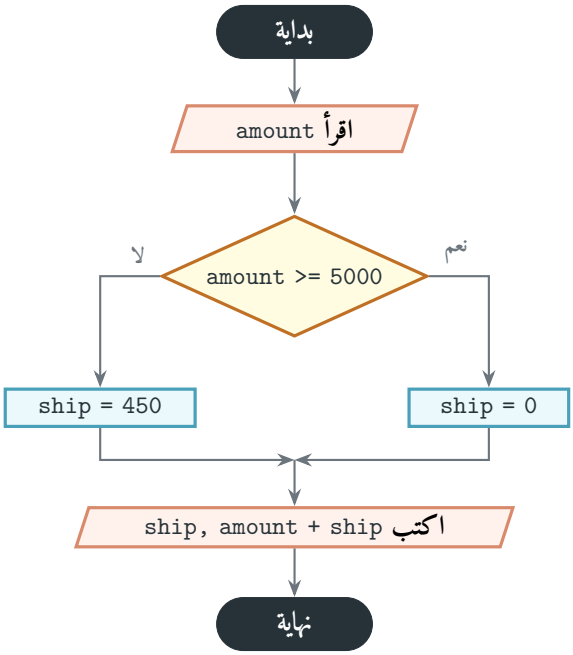
المدخلات

```
4200
```

نتيجة التنفيذ

```
Shipping = 450.00 DA
To pay = 4650.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.2 توفر المنتج في المخزون

تجارة إلكترونية سهل

يطلب زبون كمية quantity من منتج مخزونه stock. اكتب برنامجًا يعرض "Order accepted" إذا كان المخزون كافيًا مع المخزون المتبقي، وإلا يعرض الكمية الناقصة.

خطوات الحل

1) نقرأ المخزون stock والكمية المطلوبة quantity.
2) إذا كان quantity <= stock: نقبل الطلب ونعرض stock - quantity.
3) وإلا: نعرض النقص quantity - stock.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int stock, quantity;
    scanf("%d %d", &stock, &quantity);        // المخزون والكمية المطلوبة
    if (quantity <= stock) {
        printf("Order accepted\n");
        printf("Remaining stock = %d\n", stock - quantity);
    } else {
        // المخزون غير كافٍ
        printf("Not enough stock: missing %d units\n", quantity - stock);
    }
    return 0;
}
```

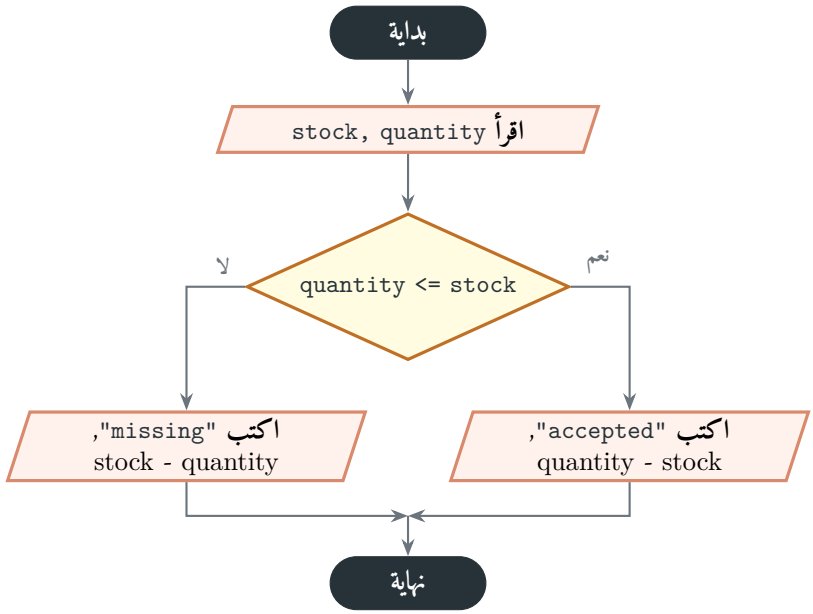
المدخلات

```
12 8
```

نتيجة التنفيذ

```
Order accepted
Remaining stock = 4
```

المخطط الإنسيابي (Organigramme)



تمرين 2.3 رقم طلبية زوجي أم فردي

تجارة إلكترونية سهل

لتوزيع العمل بين مستودعين، تُرسل الطلبيات ذات الرقم **الزوجي** إلى المستودع A والفردية إلى المستودع B. اكتب برنامجًا يحدد المستودع.

خطوات الحل

1) نقرأ رقم الطلبية `orderId`.
2) العدد زوجي إذا كان `orderId % 2 == 0`.
3) نعرض اسم المستودع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orderId;
    scanf("%d", &orderId);
    if (orderId % 2 == 0)                // باقي القسمة على 2 يساوي 0: زوجي
        printf("Order %d -> warehouse A\n", orderId);
    else
        printf("Order %d -> warehouse B\n", orderId);
    return 0;
}
```

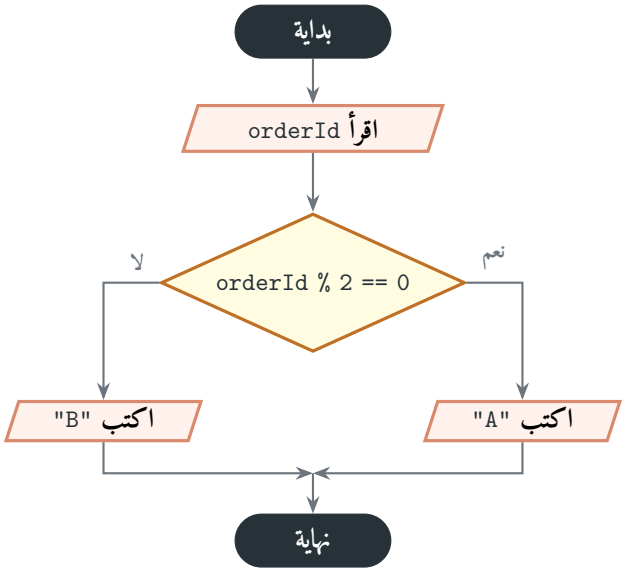
المدخلات

```
10457
```

نتيجة التنفيذ

```
Order 10457 -> warehouse B
```

المخطط الانسيابي (Organigramme)



تمرين 2.4 الأرخص بين عرضين

تجارة إلكترونية سهل

يعرض بائعان نفس المنتج بسعرين p1 وp2. اكتب برنامجًا يحدد البائع الأرخص والفرق بين السعرين، مع حالة التساوي.

خطوات الحل

1) نقرأ السعرين.
2) نقارن بـ if / else if / else لمعالجة الحالات الثلاث.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double p1, p2;
    scanf("%lf %lf", &p1, &p2);
    if (p1 < p2)
        printf("Seller 1 is cheaper by %.2f DA\n", p2 - p1);
    else if (p2 < p1)              // نختبر الحالة الثانية فقط إذا فشلت الأولى
        printf("Seller 2 is cheaper by %.2f DA\n", p1 - p2);
    else
        printf("Same price\n");    // لم يبق إلا التساوي
    return 0;
}
```

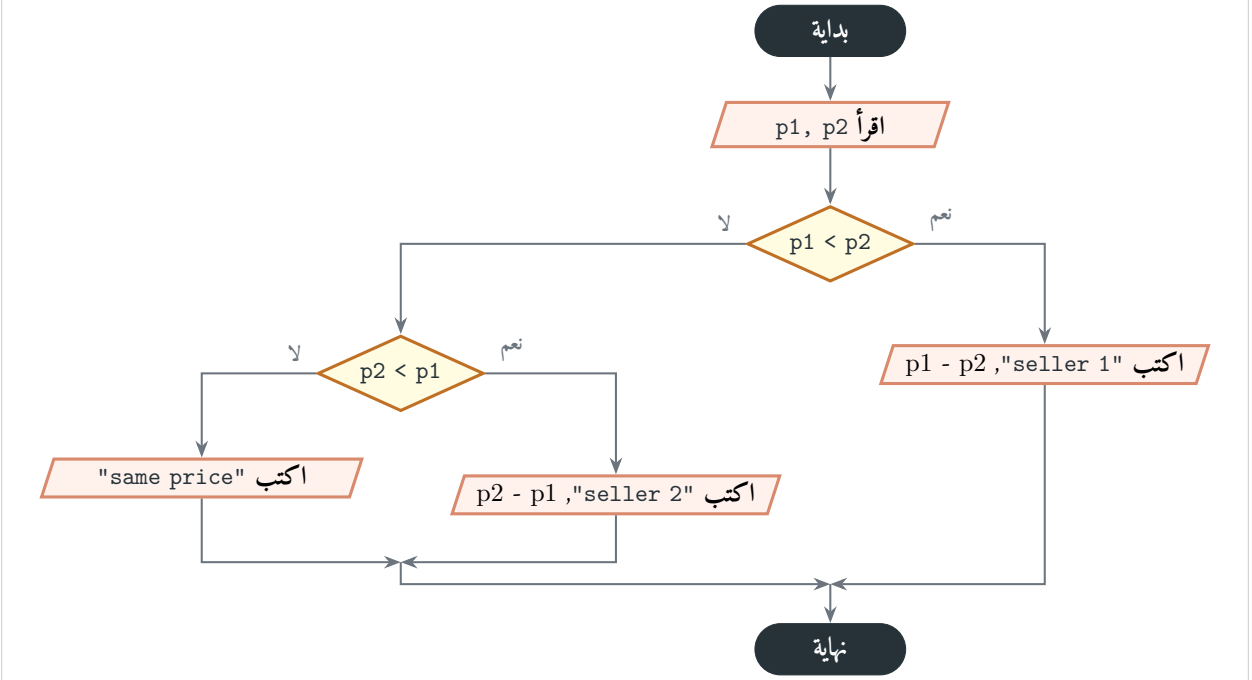
المدخلات

```
15400 14950
```

نتيجة التنفيذ

```
Seller 2 is cheaper by 450.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.5 التحقق من التقييم

علم البيانات سهل

قبل حفظ تقييم في قاعدة البيانات، يجب التأكد أنه بين 1 و5. اكتب برنامجًا يقرأ تقييمًا ويعرض صالح أو غير صالح.

خطوات الحل

1) نقرأ التقييم `rating`.
2) الشرط المركب: `rating >= 1 && rating <= 5`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rating;
    scanf("%d", &rating);
    // الشرطان معًا يجب أن يتحققا : &&
    if (rating >= 1 && rating <= 5)
        printf("Rating %d: valid\n", rating);
    else
        printf("Rating %d: invalid (must be 1..5)\n", rating);
    return 0;
}
```

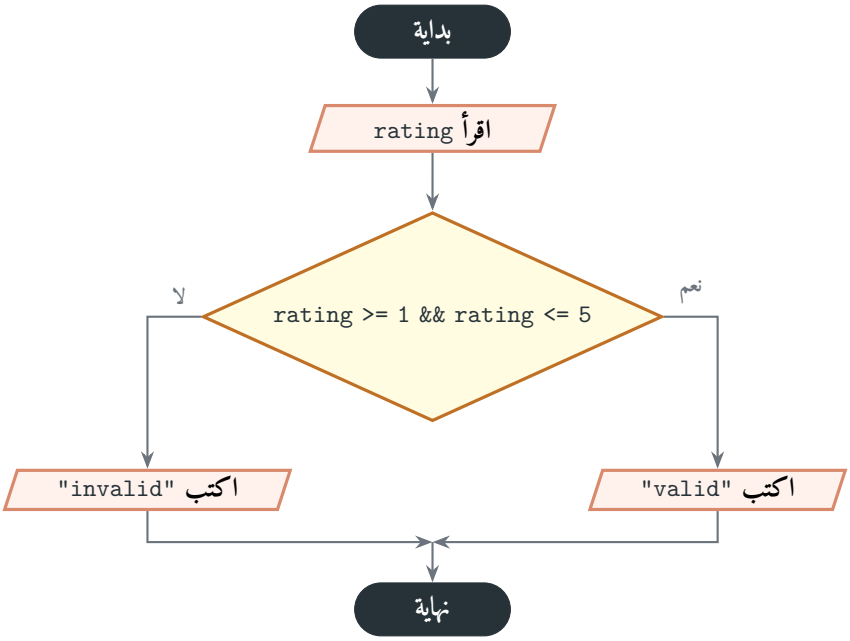
المدخلات

```
6
```

نتيجة التنفيذ

```
Rating 6: invalid (must be 1..5)
```

المخطط الانسيابي (Organigramme)



تمرين 2.6 ربح أم خسارة

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ رقم الأعمال الشهري والتكاليف، ويعرض ربح أو خسارة أو توازن مع القيمة.

خطوات الحل

1) نحسب `result = revenue - cost`.
2) إذا كان `result > 0` ربح، إذا كان `result < 0` خسارة، وإلا توازن.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double revenue, cost, result;
    scanf("%lf %lf", &revenue, &cost);
    result = revenue - cost;              // النتيجة الصافية
    if (result > 0)
        printf("Profit: %.2f DA\n", result);
    else if (result < 0)
        printf("Loss: %.2f DA\n", -result); // نعرض الخسارة كعدد موجب
    else
        printf("Break-even\n");
    return 0;
```

```
14 }
```

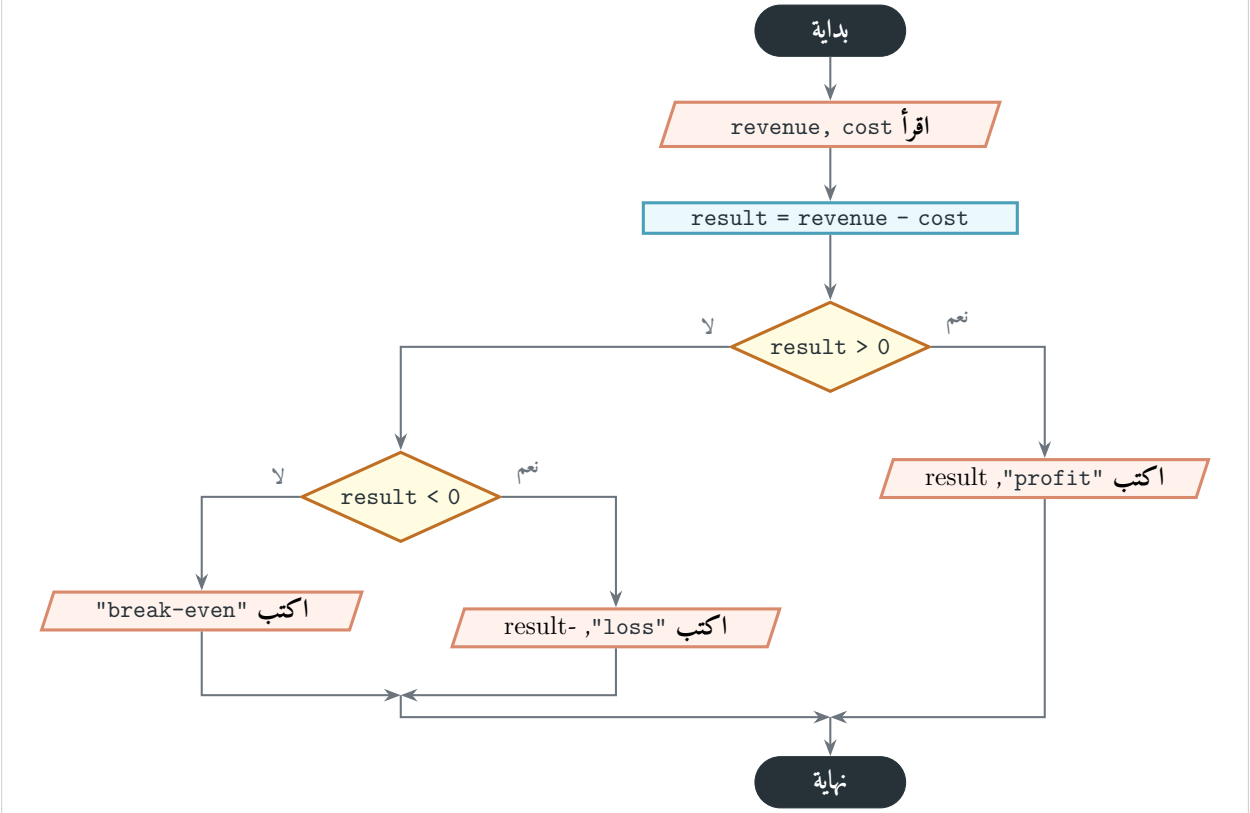
المدخلات

```
380000 412500
```

نتيجة التنفيذ

```
Loss: 32500.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.7 رمز القسيمة

تجارة إلكترونية سهل

القسيمة الصالحة لهذا الشهر رمزها **2026**، وتمنح تخفيضًا بـ 15%. اكتب برنامجًا يقرأ مبلغ السلة والرمز الذي أدخله الزبون، ويطبق التخفيض إن كان الرمز صحيحًا.

خطوات الحل

1) نقرأ المبلغ والرمز.
2) إذا كان `code == CODE` نضرب المبلغ في `0.85`.
3) نعرض المبلغ النهائي.

البرنامج بلغة C

```c
#include <stdio.h>

#define CODE 2026                   // رمز القسيمة الصالح

int main(void) {
    double amount;
    int code;
    scanf("%lf %d", &amount, &code);
    if (code == CODE) {             // == للمقارنة، و = للإسناد
        amount = amount * 0.85;     // تخفيض 15%
        printf("Coupon accepted\n");
    }
    printf("To pay = %.2f DA\n", amount);
    return 0;
}
```

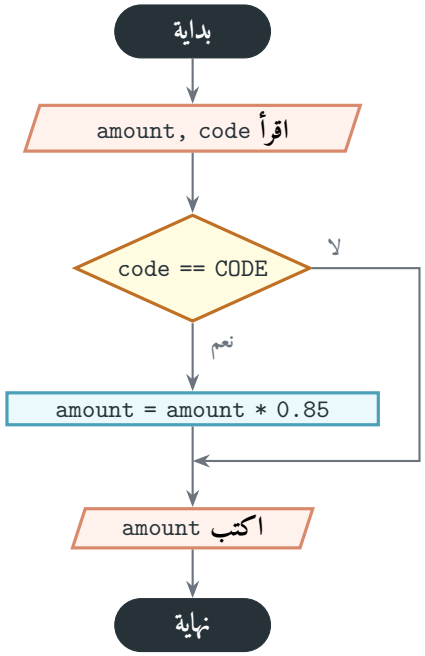
المدخلات

```
6400 2026
```

نتيجة التنفيذ

```
Coupon accepted
To pay = 5440.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.8 بيع منتج محدد بالسن

تجارة إلكترونية سهل

بعض المنتجات (مثل ألعاب الفيديو المصنفة) لا تباع لمن هم دون 16 سنة. اكتب برنامجًا يقرأ سنة ميلاد الزبون والسنة الحالية، ويحدد إن كان يُسمح له بالشراء.

خطوات الحل

1) السن: `age = year - birth`.
2) إذا كان `age >= 16` نسمح، وإلا نرفض ونعرض عدد السنوات الباقية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int birth, year, age;
    scanf("%d %d", &birth, &year);
    age = year - birth;                 // السن التقريبية
    if (age >= 16)
        printf("Age %d: purchase allowed\n", age);
    else
        printf("Age %d: refused (%d years left)\n", age, 16 - age);
    return 0;
}
```

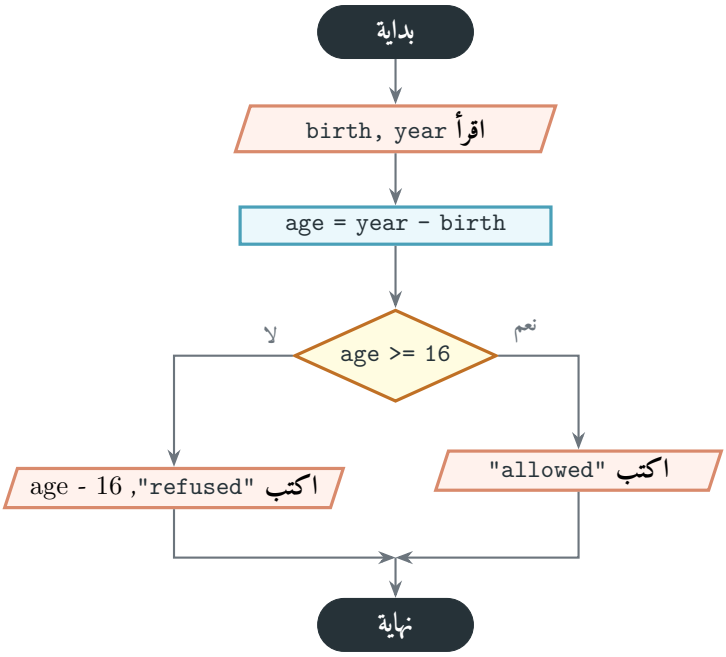
المدخلات

```
2011 2026
```

نتيجة التنفيذ

```
Age 15: refused (1 years left)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.9 طريقة الدفع

تجارة إلكترونية سهل

يختار الزبون طريقة الدفع برقم: 1 بطاقة بنكية، 2 الدفع عند الاستلام، 3 حساب بريدي جاري. اكتب برنامجًا يعرض اسم الطريقة والمصاريف الإضافية (0، 200، 50 دج) باستعمال `switch`.

خطوات الحل

1) نقرأ الاختيار choice.
2) switch (choice) مع حالة لكل رقم وbreak في نهايتها.
3) الحالة default تعالج الأرقام غير الصحيحة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int choice;
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Bank card, fee = 0 DA\n");
            break;                  // يمنع تنفيذ الحالات الموالية break
        case 2:
            printf("Cash on delivery, fee = 200 DA\n");
            break;
        case 3:
            printf("CCP account, fee = 50 DA\n");
            break;
        default:                    // أي قيمة أخرى
            printf("Invalid choice\n");
    }
    return 0;
}
```

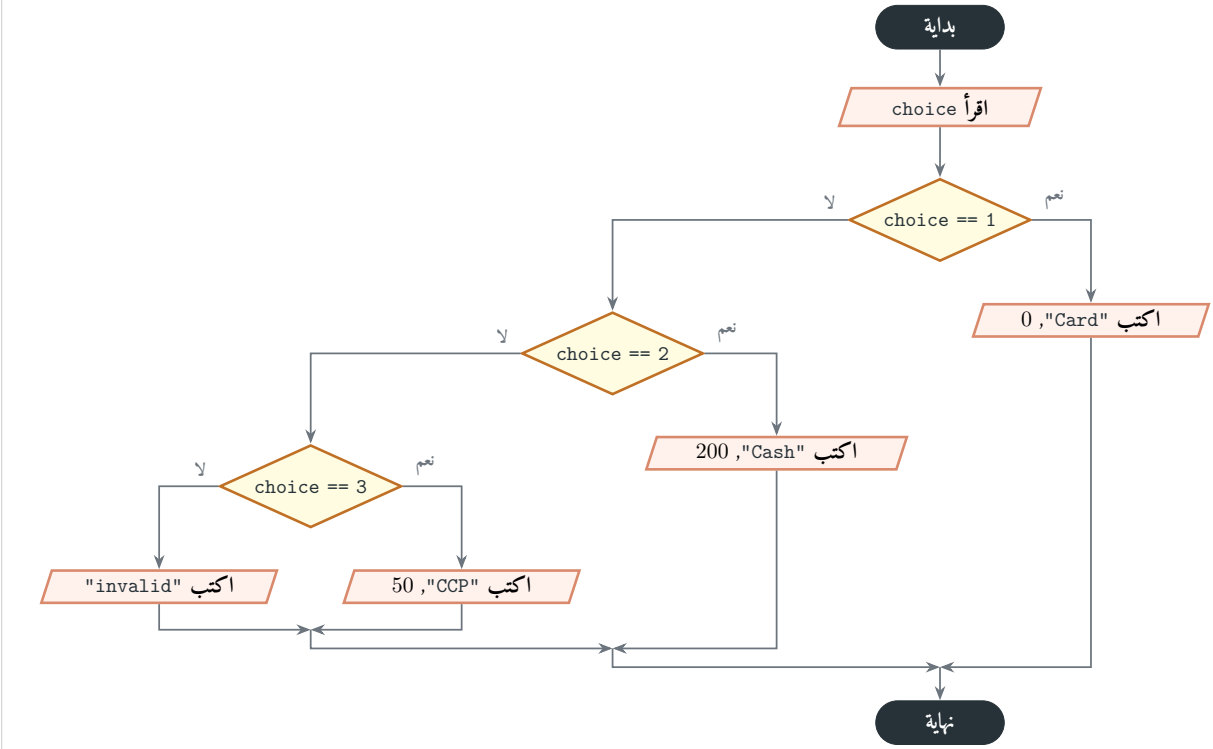
المدخلات

```
2
```

نتيجة التنفيذ

```
Cash on delivery, fee = 200 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.10 نجاح تجربة A/B

علم البيانات سهل

في تجربة A/B نعتبر النسخة الجديدة ناجحة إذا تجاوز معدل تحويلها 3.5%. اكتب برنامجًا يقرأ عدد الزوار وعدد المشتريات للنسخة الجديدة ويحكم عليها.

خطوات الحل

1) نحسب المعدل `rate = buys / visitors * 100`.
2) نقارنه بالعتبة `3.5`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double visitors, buys, rate;
    scanf("%lf %lf", &visitors, &buys);
    rate = buys / visitors * 100;      // معدل التحويل بالمئة
    printf("Conversion = %.2f %%\n", rate);
    if (rate > 3.5)
        printf("New version: SUCCESS\n");
    else
        printf("New version: FAILURE\n");
    return 0;
}
```

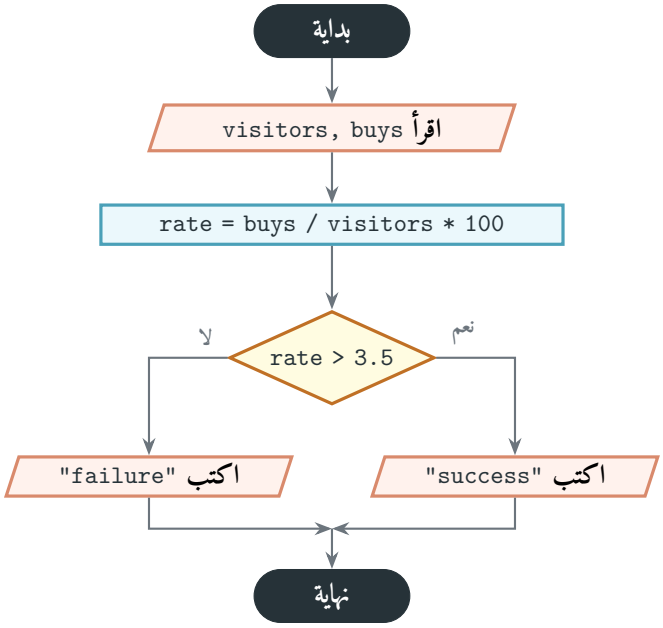
المدخلات

```
2400 91
```

نتيجة التنفيذ

```
Conversion = 3.79 %
New version: SUCCESS
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 2.11 تخفيض حسب الشرائح

تجارة إلكترونية ★★ متوسط

يطبق متجر تخفيضًا حسب مبلغ السلة: أقل من 2000 دج لا تخفيض، من 2000 إلى أقل من 10000 تخفيض 5%، من 10000 إلى أقل من 50000 تخفيض 10%، و50000 فأكثر تخفيض 15%. اكتب برنامجًا يحسب المبلغ النهائي.

خطوات الحل

1) نرتب الشروط من الأكبر إلى الأصغر في سلسلة `if / else if`.
2) كل فرع يحدد النسبة `rate`.
3) المبلغ النهائي: `amount * (1 - rate / 100)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, rate, final;
    scanf("%lf", &amount);
    // نبدأ بأعلى شريحة: كل else if تعني أن الشروط السابقة فشلت
    if (amount >= 50000)
        rate = 15;
    else if (amount >= 10000)
        rate = 10;
    else if (amount >= 2000)
        rate = 5;
    else
        rate = 0;
    final = amount * (1 - rate / 100);
    printf("Discount rate = %.0f %%\n", rate);
    printf("Final amount = %.2f DA\n", final);
    return 0;
}
```

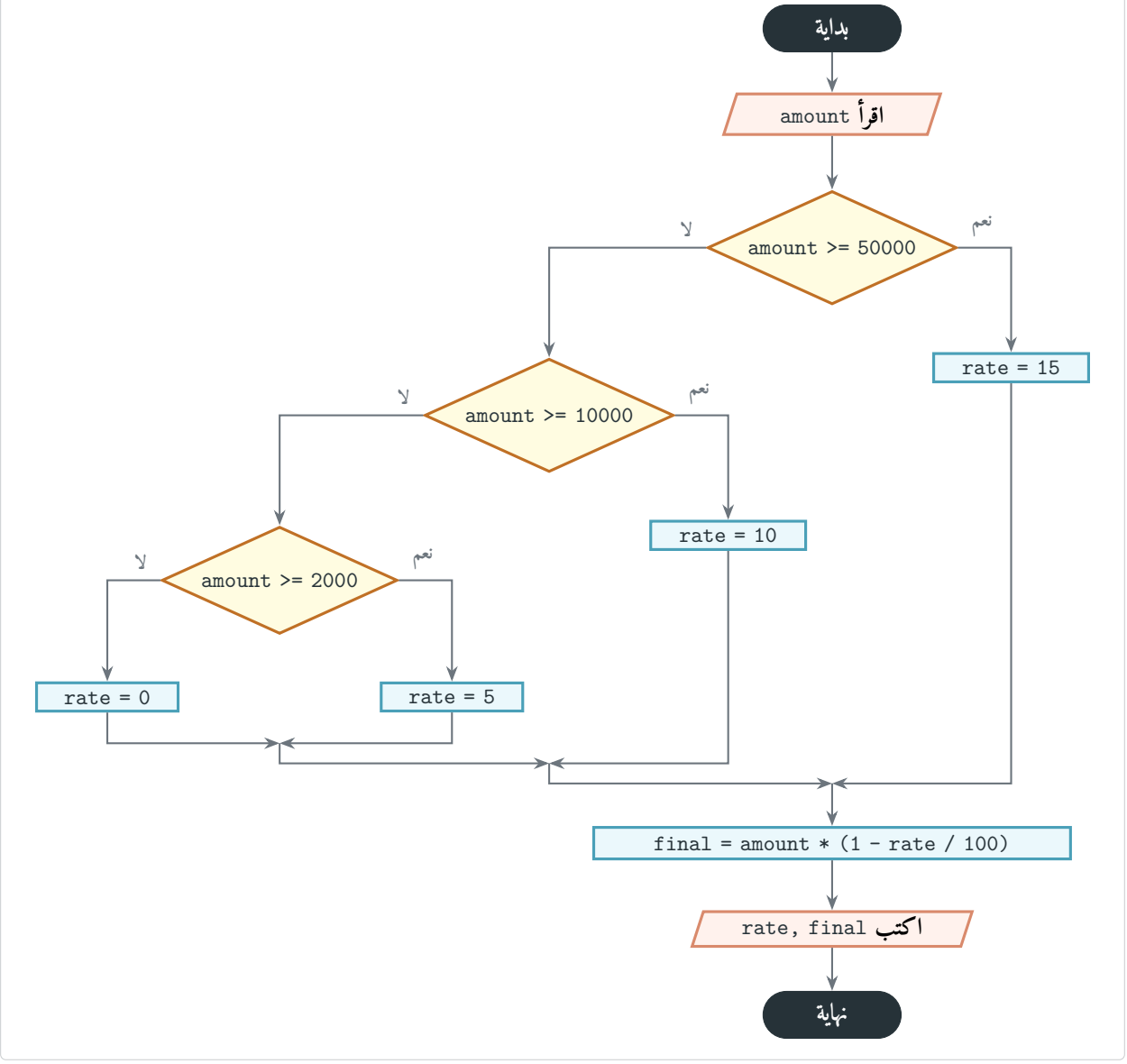
المدخلات

```
23500
```

نتيجة التنفيذ

```
Discount rate = 10 %
Final amount = 21150.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.12 تكلفة الشحن حسب الوزن

تجارة إلكترونية ★★ متوسط

تحسب شركة التوصيل التكلفة حسب الوزن: حتى 1 كغ: 400 دج، حتى 5 كغ: 700 دج، حتى 20 كغ: 1200 دج، وفوق ذلك 1200 دج + 80 دج لكل كيلوغرام إضافي. وتضاعف التكلفة إذا كان التوصيل سريعًا. اكتب البرنامج.

خطوات الحل

1) نقرأ الوزن `weight` ونوع التوصيل `fast` (1 سريع، 0 عادي).
2) نحدد التكلفة حسب الشريحة.
3) في الشريحة الأخيرة: `1200 + (weight - 20) * 80`.
4) إذا كان `fast == 1` نضاعف التكلفة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double weight, cost;
    int fast;
    scanf("%lf %d", &weight, &fast);        // الوزن ونوع التوصيل
    if (weight <= 1)
        cost = 400;
    else if (weight <= 5)
        cost = 700;
    else if (weight <= 20)
        cost = 1200;
    else
        cost = 1200 + (weight - 20) * 80;  // 80 دج لكل كغ فوق 20
    if (fast == 1)
        cost = cost * 2;                    // التوصيل السريع مضاعف
    printf("Shipping cost = %.2f DA\n", cost);
    return 0;
}
```

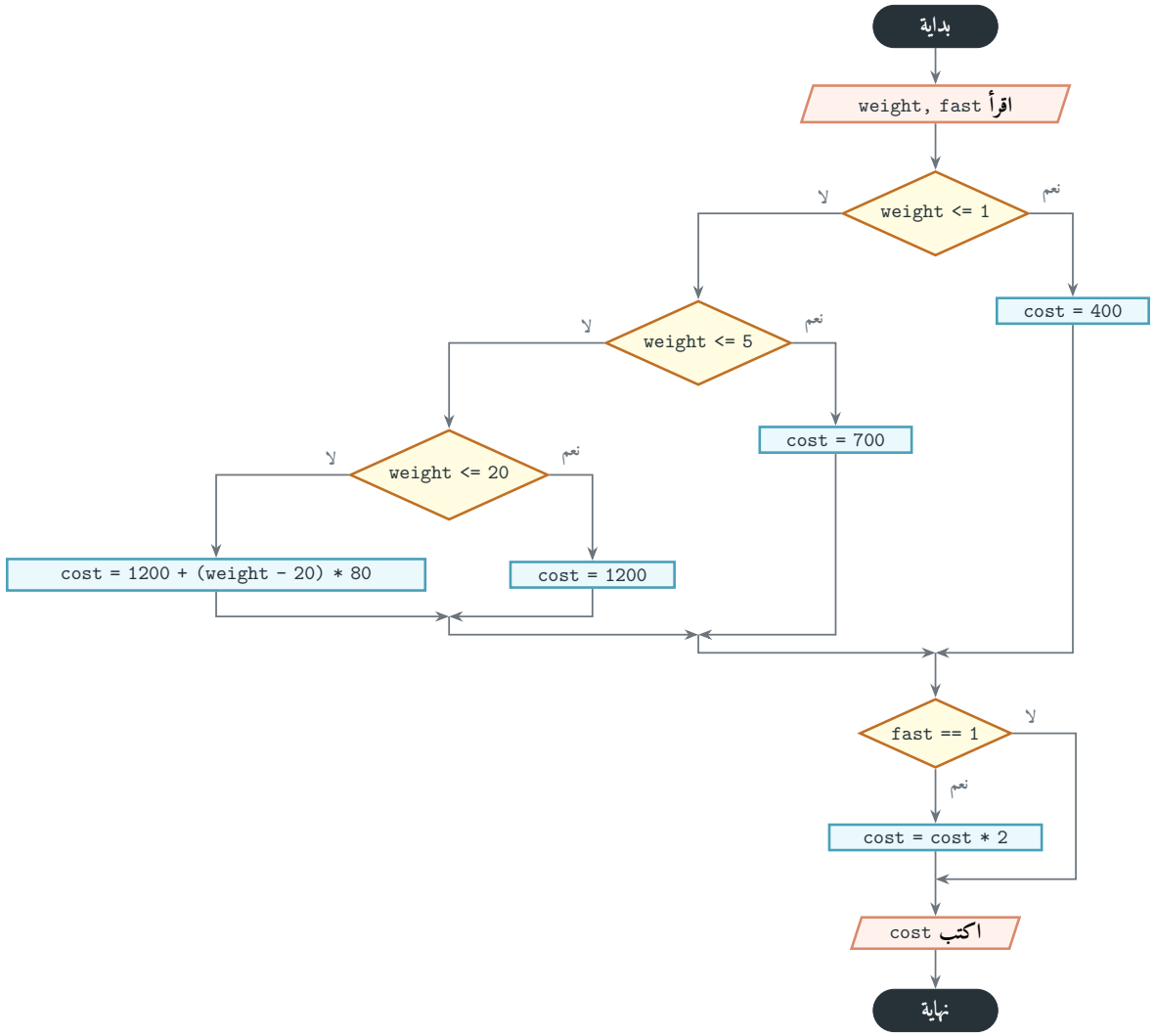
المدخلات

```
27.5 1
```

نتيجة التنفيذ

```
Shipping cost = 3600.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.13 أفضل تقييم بين ثلاثة منتجات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ متوسطات تقييم ثلاثة منتجات ويعرض رقم المنتج الأفضل وتقييمه. في حالة التساوي نختار المنتج ذا الرقم الأصغر.

خطوات الحل

1) نفترض أن الأول هو الأفضل: `best = 1` و`max = ratingA`.
2) نقارن `ratingB` بـ `max`: إذا كان أكبر تمامًا نحدّث.
3) نفعل نفس الشيء مع `ratingC`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double ratingA, ratingB, ratingC, max;
    int best;
```

```
    scanf("%lf %lf %lf", &ratingA, &ratingB, &ratingC);
    max = ratingA;                      // نفترض أن الأول هو الأفضل
    best = 1;
    if (ratingB > max) {                // > تمامًا: عند التساوي نحتفظ بالأصغر رقمًا
        max = ratingB;
        best = 2;
    }
    if (ratingC > max) {
        max = ratingC;
        best = 3;
    }
    printf("Best product: #%d (%.1f stars)\n", best, max);
    return 0;
}
```

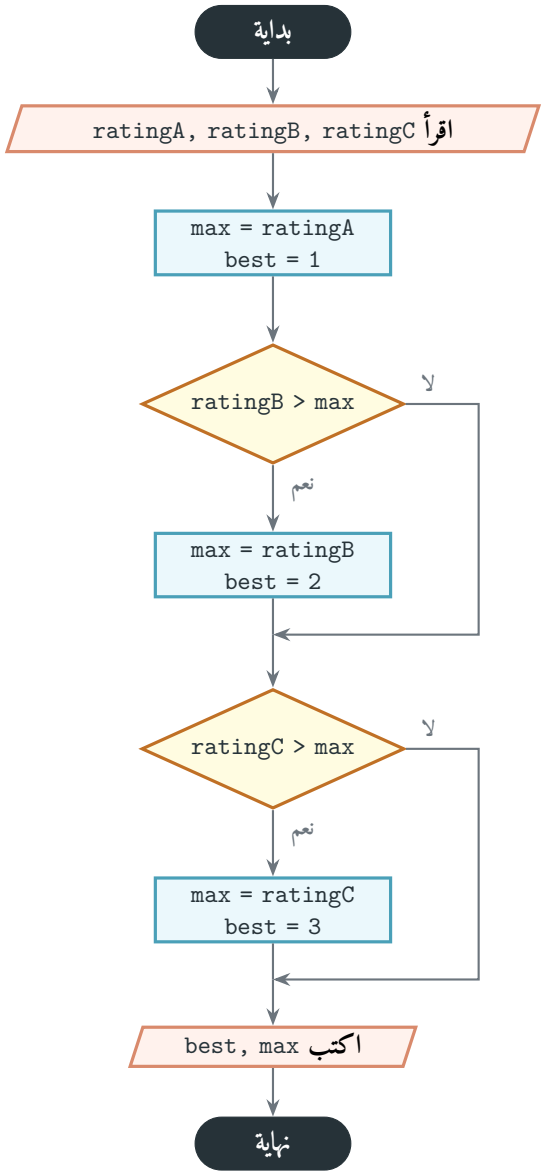
المدخلات

```
4.3 4.7 4.7
```

نتيجة التنفيذ

```
Best product: #2 (4.7 stars)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.14 ترتيب ثلاثة أسعار

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يقرأ ثلاثة أسعار ويعرضها **مرتبة تصاعديًا** باستعمال ثلاث مقارنات وتبديلات فقط.

خطوات الحل

1) إذا كان `price1 > price2` نبدل `price1` و`price2`.
2) إذا كان `price2 > price3` نبدل `price2` و`price3` (الآن `price3` هو الأكبر).
3) إذا كان `price1 > price2` نبدل مرة أخرى.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```
    double price1, price2, price3, tmp;
    scanf("%lf %lf %lf", &price1, &price2, &price3);
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // price2 => price1 الآن
    if (price2 > price3) { tmp = price2; price2 = price3; price3 = tmp; }   // price3 الأكبر صعد إلى
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // نرتب الاثنين الأولين من جديد
    printf("Sorted: %.0f %.0f %.0f\n", price1, price2, price3);
    return 0;
}
```

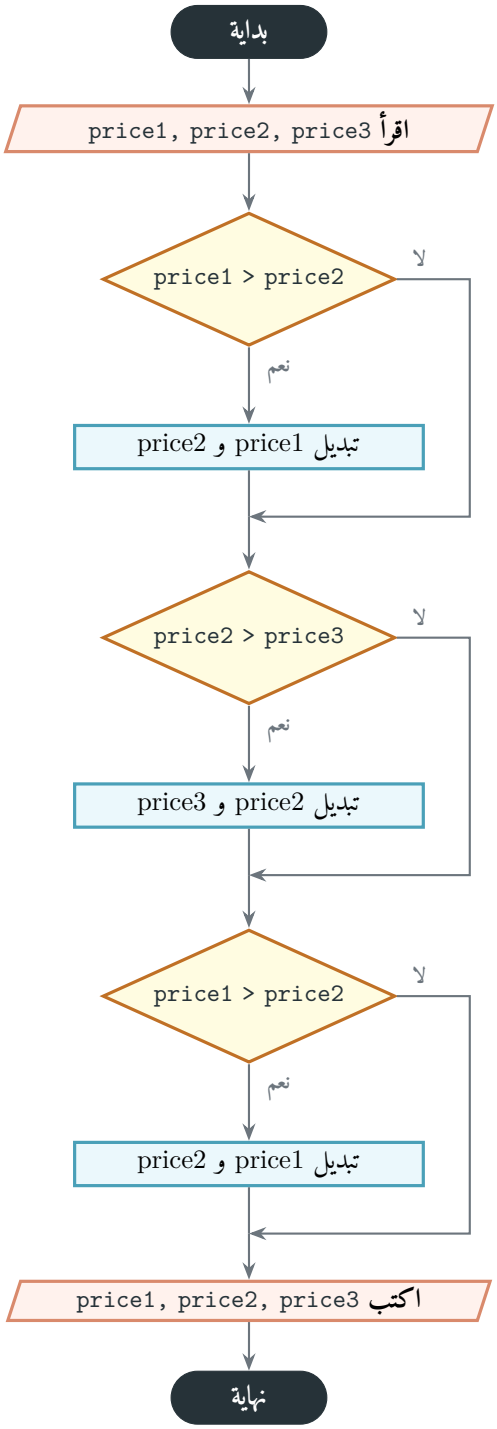
المدخلات

```
950 320 610
```

نتيجة التنفيذ

```
Sorted: 320 610 950
```

المخطط الإنسيابي (Organigramme)



تمرين 2.15 السنة الكبيسة

تجارة إلكترونية ★★ متوسط

لحساب تواريخ التسليم في فبراير نحتاج معرفة إن كانت السنة **كبيسة**: تقبل القسمة على 4 ولا تقبل القسمة على 100، أو تقبل القسمة على 400. اكتب برنامجًا يحدد ذلك ويعرض عدد أيام فبراير.

خطوات الحل

1) الشرط:

```
(year % 4 == 0 && year % 100 != 0) || year % 400 == 0
```

.

2) إذا تحقق: فبراير 29 يومًا، وإلا 28.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int year, feb;
    scanf("%d", &year);
    // يكفي أن يتحقق أحد الطرفين : ||
    if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
        feb = 29;
    else
        feb = 28;
    printf("%d: %s year, February has %d days\n",
           year, feb == 29 ? "leap" : "common", feb);
    return 0;
}
```

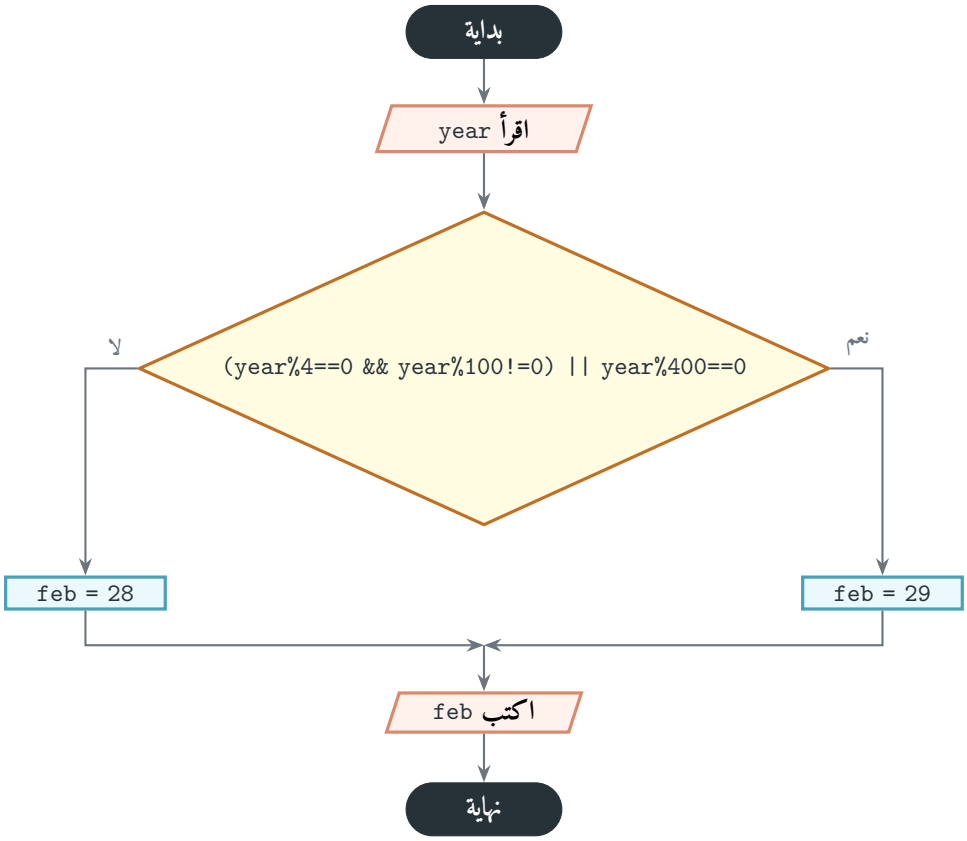
المدخلات

```
2028
```

نتيجة التنفيذ

```
2028: leap year, February has 29 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.16 عدد أيام الشهر — تجارة إلكترونية — متوسط

لحساب مدة اشتراك شهري، اكتب برنامجًا يقرأ رقم الشهر والسنة ويعرض **عدد أيامه** باستعمال `switch` مع تجميع الحالات المتشابهة.

خطوات الحل

1) الأشهر 1، 3، 5، 7، 8، 10، 12 فيها 31 يومًا: حالات متتالية دون `break`.
2) الأشهر 4، 6، 9، 11 فيها 30 يومًا.
3) الشهر 2 يعتمد على السنة الكبيسة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int month, year, days;
    scanf("%d %d", &month, &year);
    switch (month) {
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:      // عدة حالات تشترك في نفس الكود
            days = 31;
            break;
        case 4: case 6: case 9: case 11:
            days = 30;
```

```c
            break;
        case 2:
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                days = 29;              // سنة كبيسة
            else
                days = 28;
            break;
        default:
            days = 0;                   // شهر غير صالح
    }
    if (days == 0)
        printf("Invalid month\n");
    else
        printf("Month %d/%d has %d days\n", month, year, days);
    return 0;
}
```

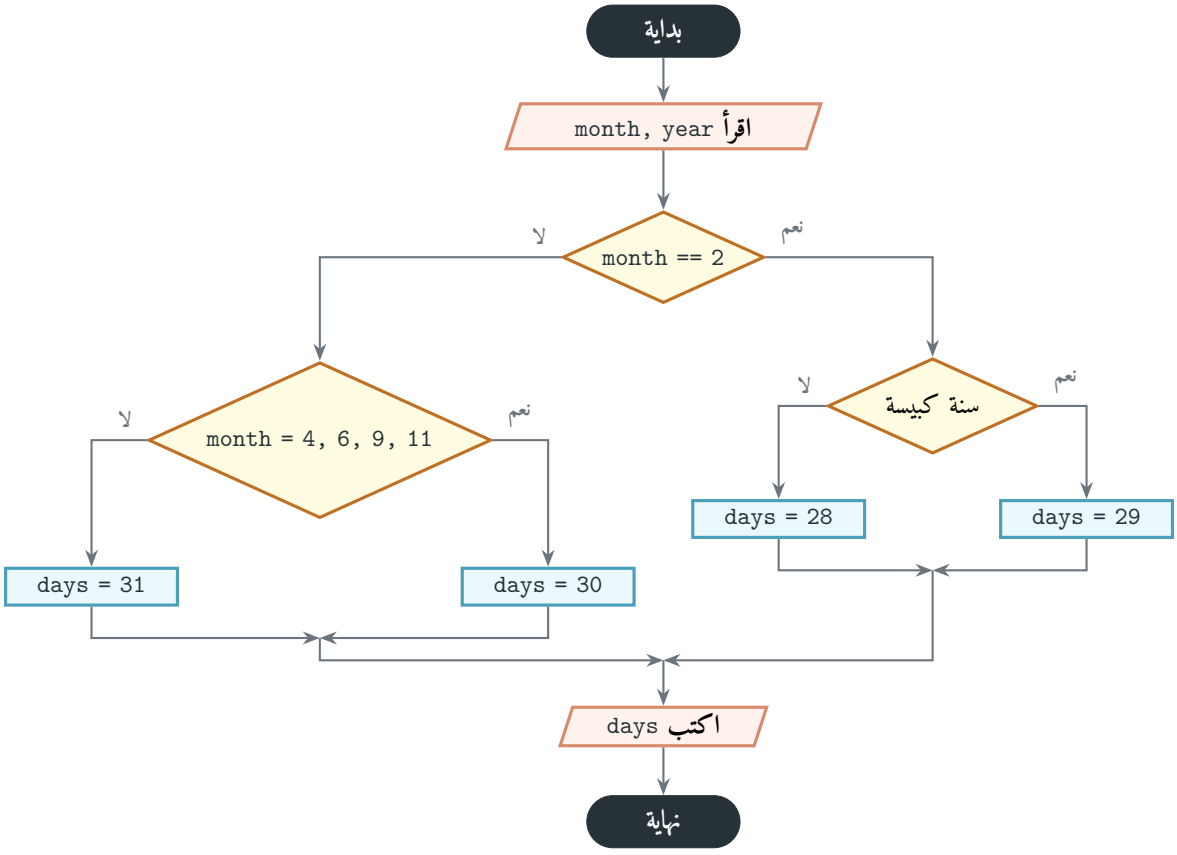
المدخلات

```
4 2026
```

نتيجة التنفيذ

```
Month 4/2026 has 30 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.17 تصنيف الزبائن حسب الإنفاق

علم البيانات

متوسط

يصنف قسم التسويق الزبائن حسب إنفاقهم السنوي وعدد طلبياتهم: **VIP** إذا تجاوز الإنفاق 200000 وعدد الطلبيات 20، **وفي** إذا تجاوز الإنفاق 50000 أو الطلبيات 10، **عادي** إذا كانت له طلبية واحدة على الأقل، وإلا **غير نشط**.

خطوات الحل

1) نقرأ الإنفاق `spend` وعدد الطلبيات `nbOrders`.
2) نختبر الشروط من الأقوى إلى الأضعف.
3) نستعمل `&&` للفئة الأولى و`||` للثانية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double spend;
    int nbOrders;
    scanf("%lf %d", &spend, &nbOrders);
    if (spend > 200000 && nbOrders > 20)          // الشرطان معًا
        printf("Segment: VIP\n");
    else if (spend > 50000 || nbOrders > 10)      // يكفي أحدهما
        printf("Segment: Loyal\n");
    else if (nbOrders >= 1)
        printf("Segment: Regular\n");
    else
        printf("Segment: Inactive\n");
    return 0;
}
```

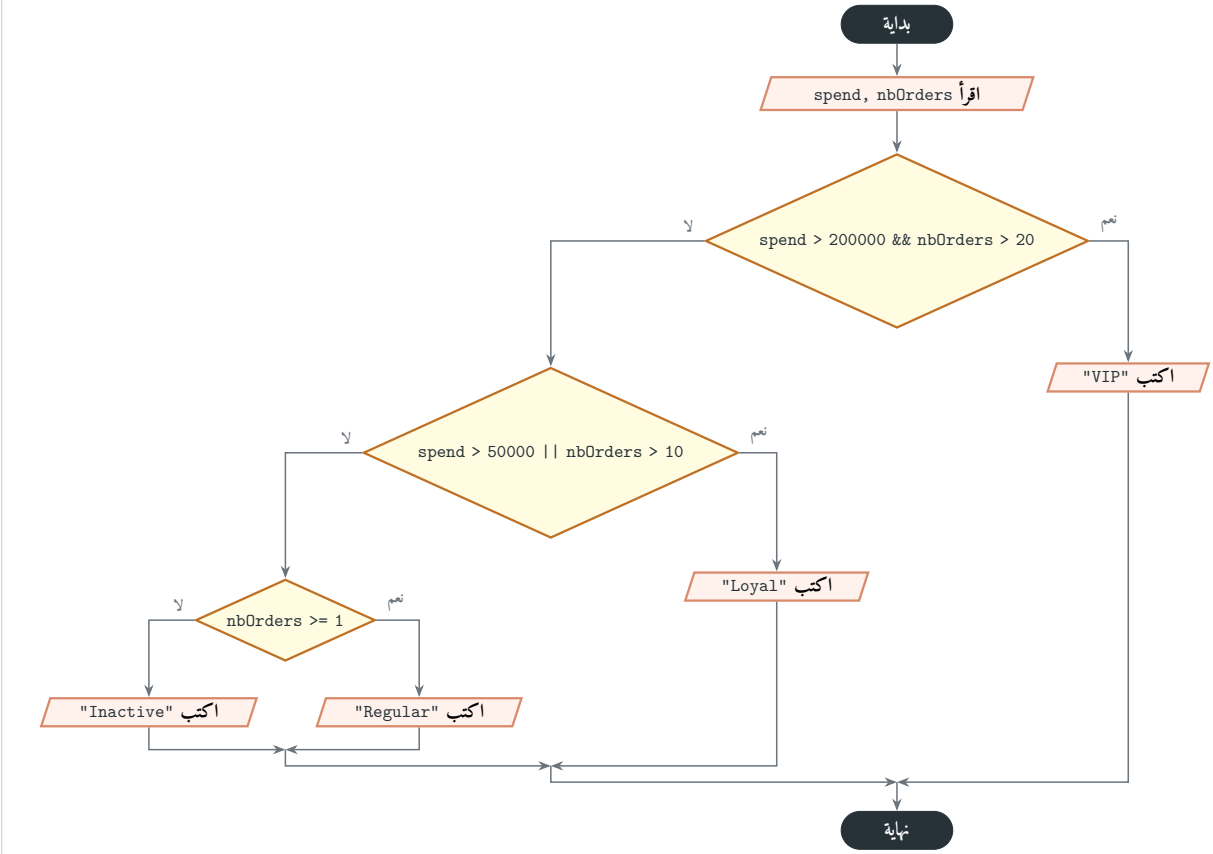
المدخلات

```
85000 7
```

نتيجة التنفيذ

```
Segment: Loyal
```

المخطط الإنسيابي (Organigramme)



تمرين 2.18 الرسم حسب صنف المنتج

تجارة إلكترونية متوسط

تختلف نسبة الرسم حسب صنف المنتج: `'A'` مواد غذائية أساسية 0%، `'B'` أدوية 9%، `'C'` بقية المنتجات 19%. اكتب برنامجًا يقرأ الصنف والسعر ويعرض السعر مع الرسم، ويقبل الحرف كبيرًا أو صغيرًا.

خطوات الحل

1) نقرأ الحرف `cat` والسعر.
2) في `switch` نضع `case 'A': case 'a':` معًا.
3) نحسب `price * (1 + taxRate)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char cat;
    double price, taxRate;
    scanf(" %c %lf", &cat, &price);
    switch (cat) {
        case 'A': case 'a': taxRate = 0;    break;   // مواد أساسية
        case 'B': case 'b': taxRate = 0.09; break;   // أدوية
        case 'C': case 'c': taxRate = 0.19; break;   // بقية المنتجات
```

```c
        default:            taxRate = -1;           // صنف غير معروف
    }
    if (taxRate >= 0)
        printf("Category %c: %.2f DA with tax\n", cat, price * (1 + taxRate));
    else
        printf("Unknown category\n");
    return 0;
}
```

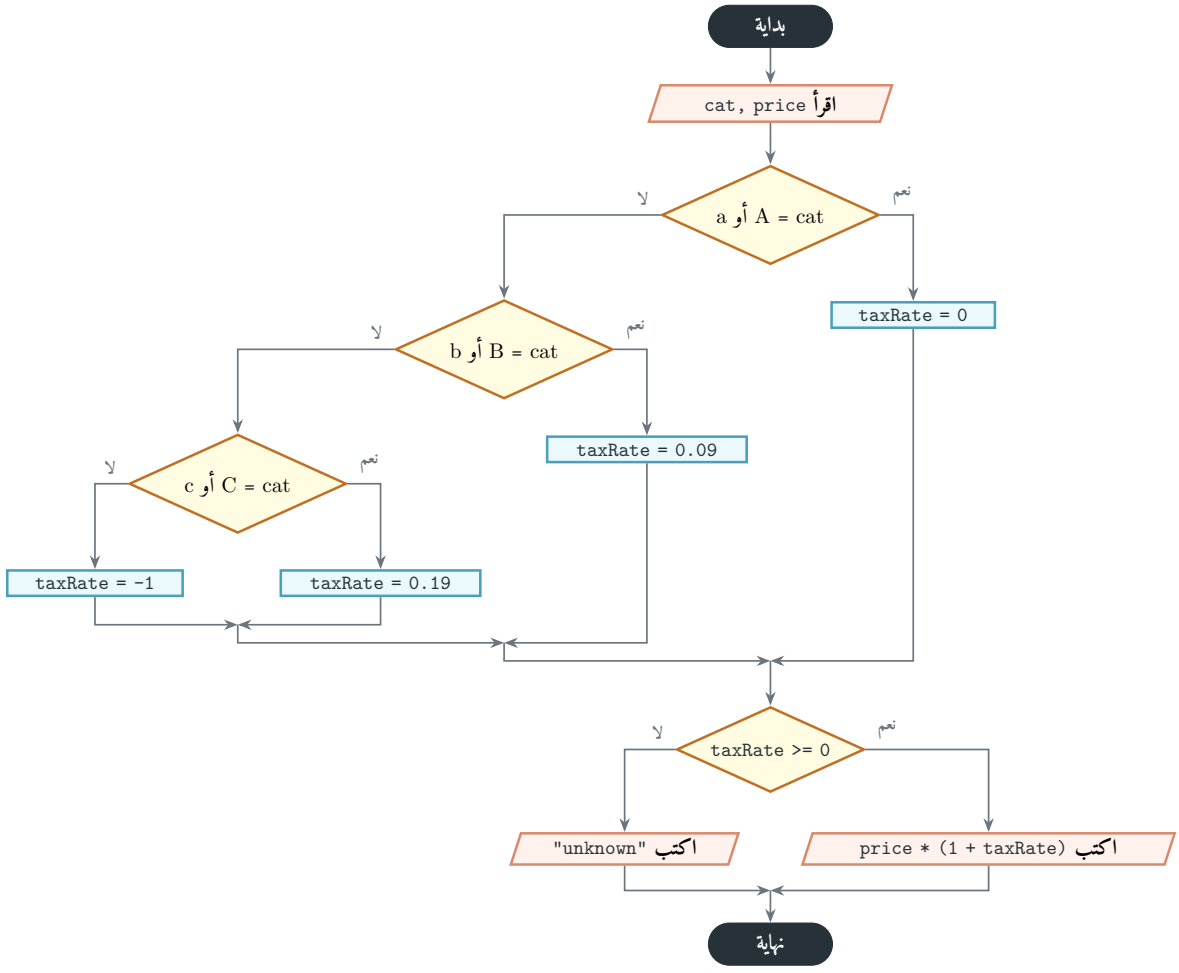
المدخلات

```
c 12000
```

نتيجة التنفيذ

```
Category c: 14280.00 DA with tax
```

المخطط الإنسيابي (Organigramme)



تمرين 2.19 كشف القيم الشاذة

علم البيانات متوسط

نعتبر مبلغ طلبية **شاذًا** إذا كانت درجته المعيارية أكبر من 3 أو أصغر من -3، **مشبوهًا** إذا كانت قيمتها المطلقة بين 2 و3، **وعاديًا** في غير ذلك. اكتب برنامجًا يقرأ المبلغ والمتوسط والانحراف المعياري ويصنفه.

خطوات الحل

1) نحسب `zscore = (amount - mean) / std`.
2) نحسب القيمة المطلقة يدويًا: إذا كان `zscore < 0` فـ `az = -zscore`.
3) نقارن `az` بالعتبتين 3 و2.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount, mean, std, zscore, az;
    scanf("%lf %lf %lf", &amount, &mean, &std);
    zscore = (amount - mean) / std;          // الدرجة المعيارية
    az = zscore;
    if (az < 0)                    // القيمة المطلقة دون دالة fabs
        az = -az;
    printf("z = %.2f -> ", zscore);
    if (az > 3)
        printf("outlier\n");
    else if (az >= 2)
        printf("suspicious\n");
    else
        printf("normal\n");
    return 0;
}
```

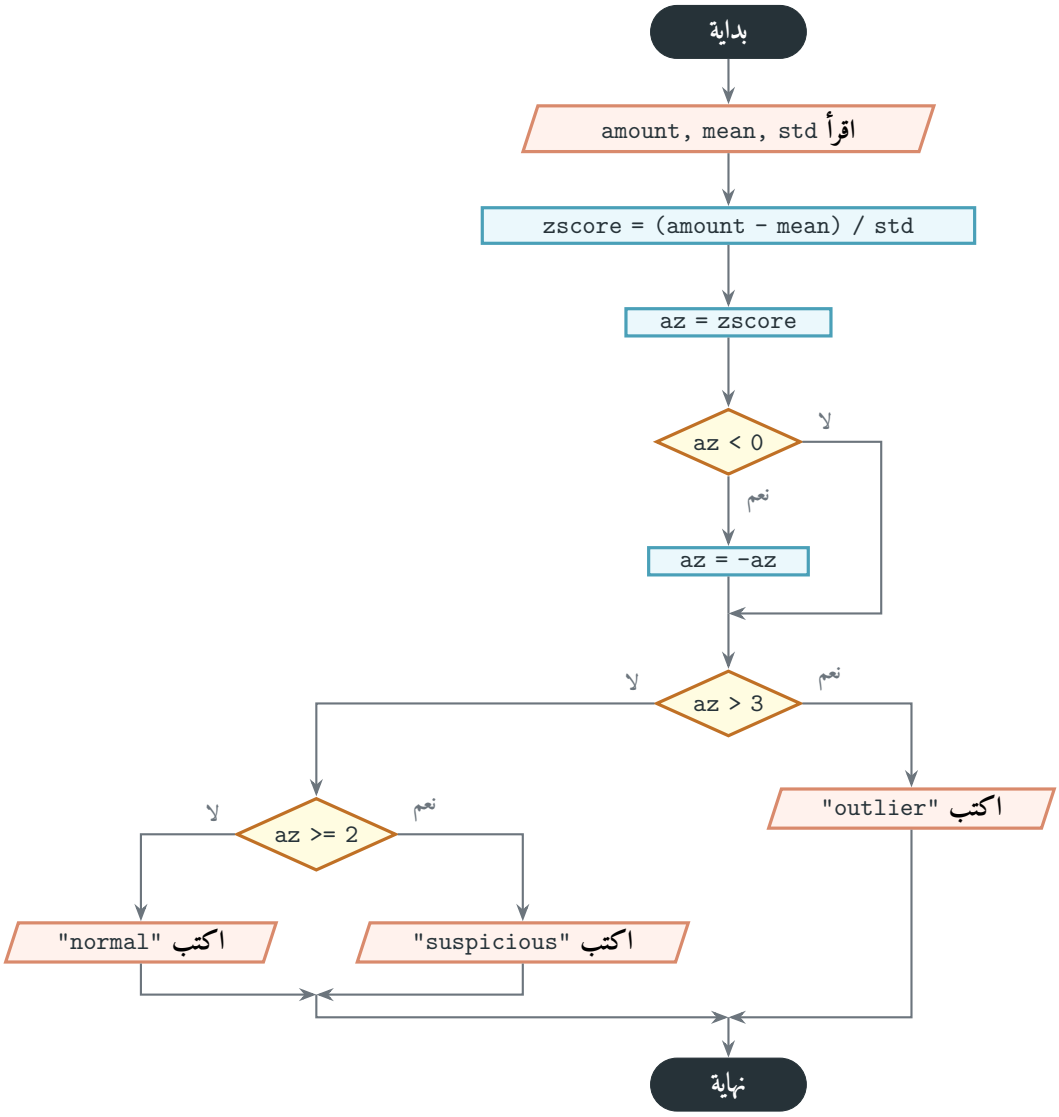
المدخلات

```
15500 4800 3100
```

نتيجة التنفيذ

```
z = 3.45 -> outlier
```

المخطط الإنسيابي (Organigramme)



تمرين 2.20 هيمنة منتج على آخر

علم البيانات ★★ متوسط

نقول إن منتجًا يهيمن على آخر إذا كان سعره أقل أو يساويه وتقييمه أكبر أو يساويه، مع تفوق في أحدهما على الأقل. اكتب برنامجًا يقرأ (سعر، تقييم) لمنتجين ويحدد علاقة الهيمنة بينهما.

خطوات الحل

1) `d1`: المنتج 1 يهيمن إذا كان

```
p1 <= p2 && r1 >= r2 && (p1 < p2 || r1 > r2)
```

.

2) `d2` بنفس الطريقة بتبديل الأدوار.

3) إذا لم يهيمن أي منهما فالمنتجان غير قابلين للمقارنة (أو متساويان).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double p1, r1, p2, r2;
    scanf("%lf %lf %lf %lf", &p1, &r1, &p2, &r2);
    // ليس أسوأ في أي معيار، وأفضل في معيار واحد على الأقل
    if (p1 <= p2 && r1 >= r2 && (p1 < p2 || r1 > r2))
        printf("Product 1 dominates product 2\n");
    else if (p2 <= p1 && r2 >= r1 && (p2 < p1 || r2 > r1))
        printf("Product 2 dominates product 1\n");
    else
        printf("No dominance (trade-off)\n");
    return 0;
}
```

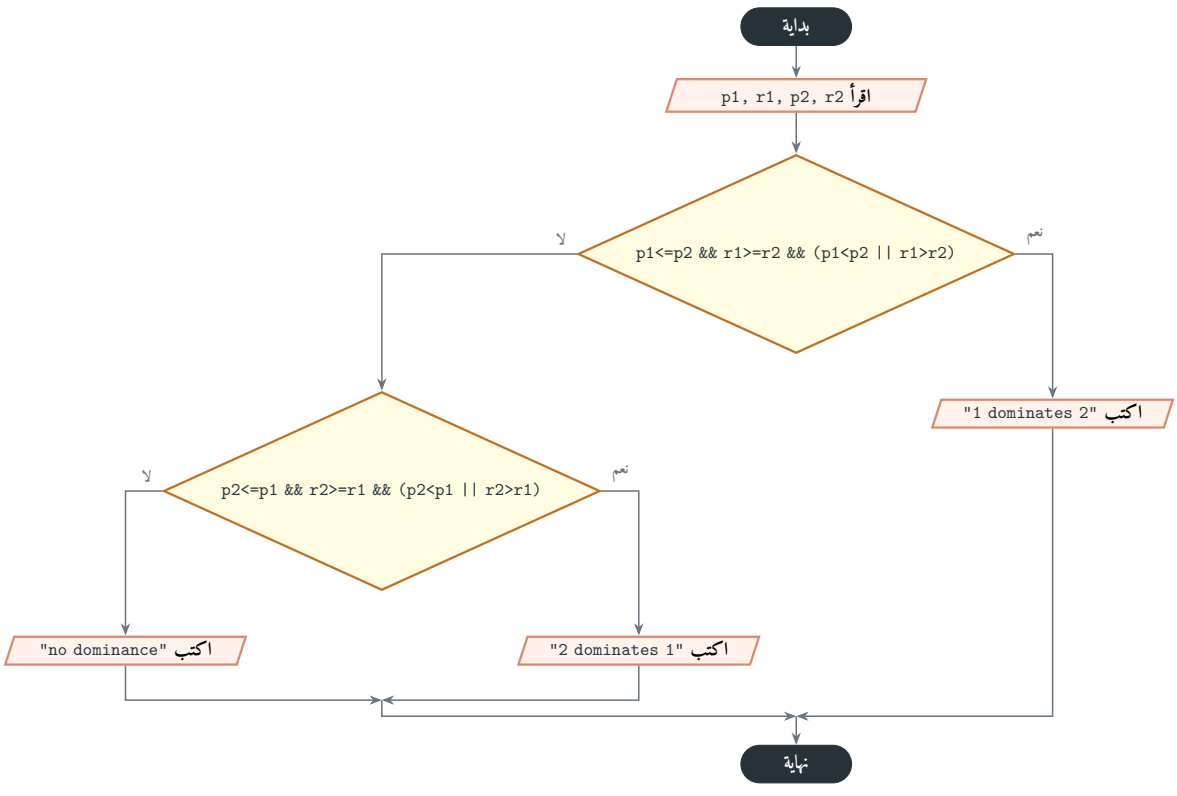
المدخلات

```
3200 4.5
3500 4.1
```

نتيجة التنفيذ

```
Product 1 dominates product 2
```

المخطط الإنسيابي (Organigramme)



تمرين 2.21 التقييم إلى نجوم

علم البيانات ★★ متوسط

يُعرض متوسط التقييم (عدد حقيقي بين 0 و5) على شكل نجوم: نقربه لأقرب نصف نجمة. اكتب برنامجًا يقرأ المتوسط ويعرض

عدد النجوم الكاملة وهل توجد نصف نجمة، ثم وصفًا: ممتاز (4.5 فأكثر)، جيد (3.5 فأكثر)، متوسط (2.5 فأكثر)، ضعيف.

خطوات الحل

1) نقرب لأقرب نصف: `halves = (int)(avg * 2 + 0.5)` (عدد الأنصاف).
2) النجوم الكاملة: `halves / 2`، ونصف نجمة إذا كان `halves % 2 == 1`.
3) الوصف بسلسلة `if / else if` على القيمة المقربة `halves / 2.0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double avg, rounded;
    int halves, full, half;
    scanf("%lf", &avg);
    halves = (int)(avg * 2 + 0.5);      // عدد أنصاف النجوم بعد التقريب
    full = halves / 2;                  // النجوم الكاملة
    half = halves % 2;                  // 1 إذا بقي نصف نجمة
    rounded = halves / 2.0;                     // القيمة المقربة
    printf("%.2f -> %d full star(s)", avg, full);
    if (half == 1)
        printf(" + half star");
    printf(" (%.1f)\n", rounded);
    if (rounded >= 4.5)       printf("Excellent\n");
    else if (rounded >= 3.5)  printf("Good\n");
    else if (rounded >= 2.5)  printf("Average\n");
    else                      printf("Poor\n");
    return 0;
}
```

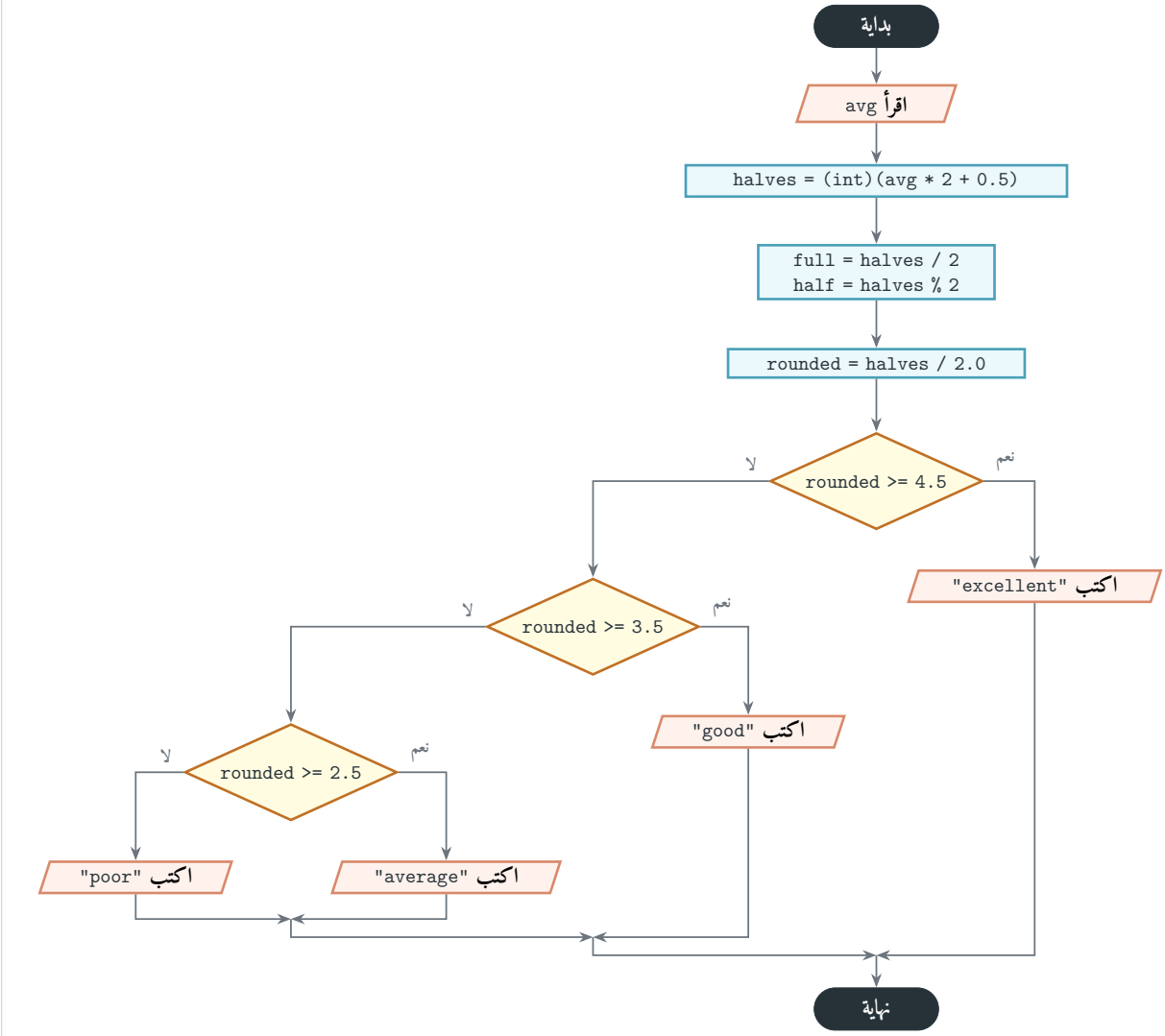
المدخلات

```
3.83
```

نتيجة التنفيذ

```
3.83 -> 4 full star(s) (4.0)
Good
```

المخطط الإنسيابي (Organigramme)



تمرين 2.22 نقطة التعادل

متوسط ★★ تجارة إلكترونية

بائع لديه تكاليف ثابتة `fixedCost` (كراء، إعلانات)، وتكلفة متغيرة `varCost` لكل وحدة، وسعر بيع `price`. **نقطة التعادل** هي أصغر عدد وحدات يغطي التكاليف: `fixedCost / (price - varCost)` مقربًا إلى الأعلى. اكتب برنامجًا يحسبها، مع معالجة الحالة التي يكون فيها السعر غير كافٍ.

خطوات الحل

1) الهامش للوحدة: `margin = price - varCost`.
2) إذا كان `margin <= 0` فلا يمكن تحقيق ربح أبدًا.
3) وإلا: `units = fixedCost / margin`، وإذا كان هناك باقٍ نضيف 1.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```
    long fixedCost, varCost, price, margin, units;
    scanf("%ld %ld %ld", &fixedCost, &varCost, &price);
    margin = price - varCost;                    // ما تربحه في كل وحدة
    if (margin <= 0) {
        printf("Price too low: never profitable\n");
    } else {
        units = fixedCost / margin;                // قسمة صحيحة
        if (fixedCost % margin != 0)
            units = units + 1;                  // تقريب إلى الأعلى
        printf("Margin per unit = %ld DA\n", margin);
        printf("Break-even = %ld units\n", units);
    }
    return 0;
}
```

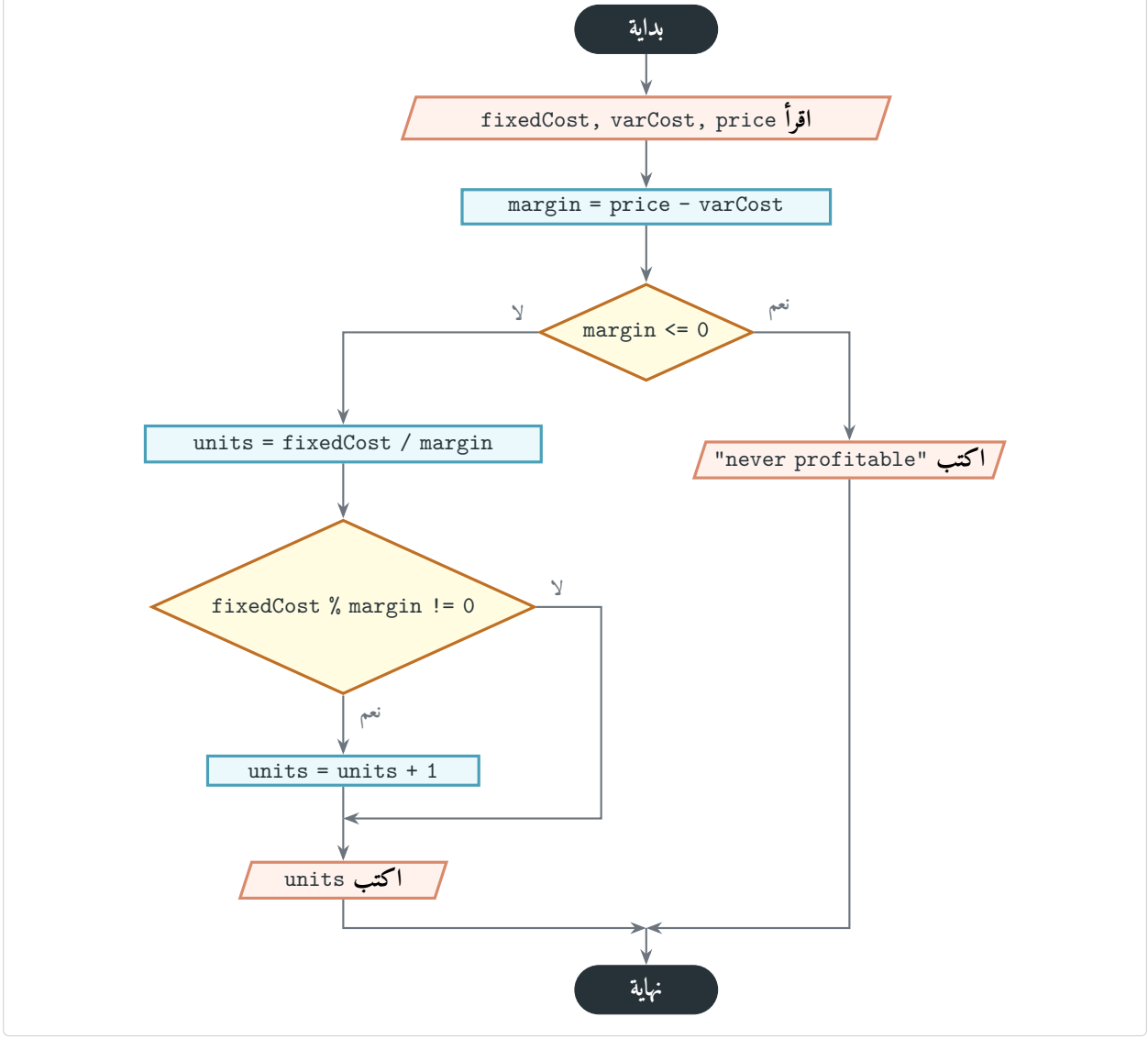
المدخلات

```
120000 650 1100
```

نتيجة التنفيذ

```
Margin per unit = 450 DA
Break-even = 267 units
```

المخطط الإنسيابي (Organigramme)



تمرين 2.23 صحة تاريخ الطلبية

تجارة إلكترونية ★★ متوسط

يدخل الزبون تاريخ استلام مرغوب `jj mm aaaa`. اكتب برنامجًا يتحقق من **صحة التاريخ** (الشهر بين 1 و12، واليوم بين 1 وعدد أيام الشهر مع مراعاة السنة الكبيسة).

خطوات الحل

1) نتحقق أولًا من الشهر.
2) نحسب عدد أيام الشهر `max` (31 أو 30 أو 28/29).
3) التاريخ صحيح إذا كان `1 <= day <= max`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```c
    int day, month, year, max;
    scanf("%d %d %d", &day, &month, &year);
    if (month < 1 || month > 12) {
        printf("Invalid month\n");
    } else {
        if (month == 2) {
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                max = 29;
            else
                max = 28;
        } else if (month == 4 || month == 6 || month == 9 || month == 11) {
            max = 30;
        } else {
            max = 31;
        }
        if (day >= 1 && day <= max)        // اليوم داخل حدود الشهر
            printf("%02d/%02d/%d is valid\n", day, month, year);
        else
            printf("Invalid day: month %d has %d days\n", month, max);
    }
    return 0;
}
```

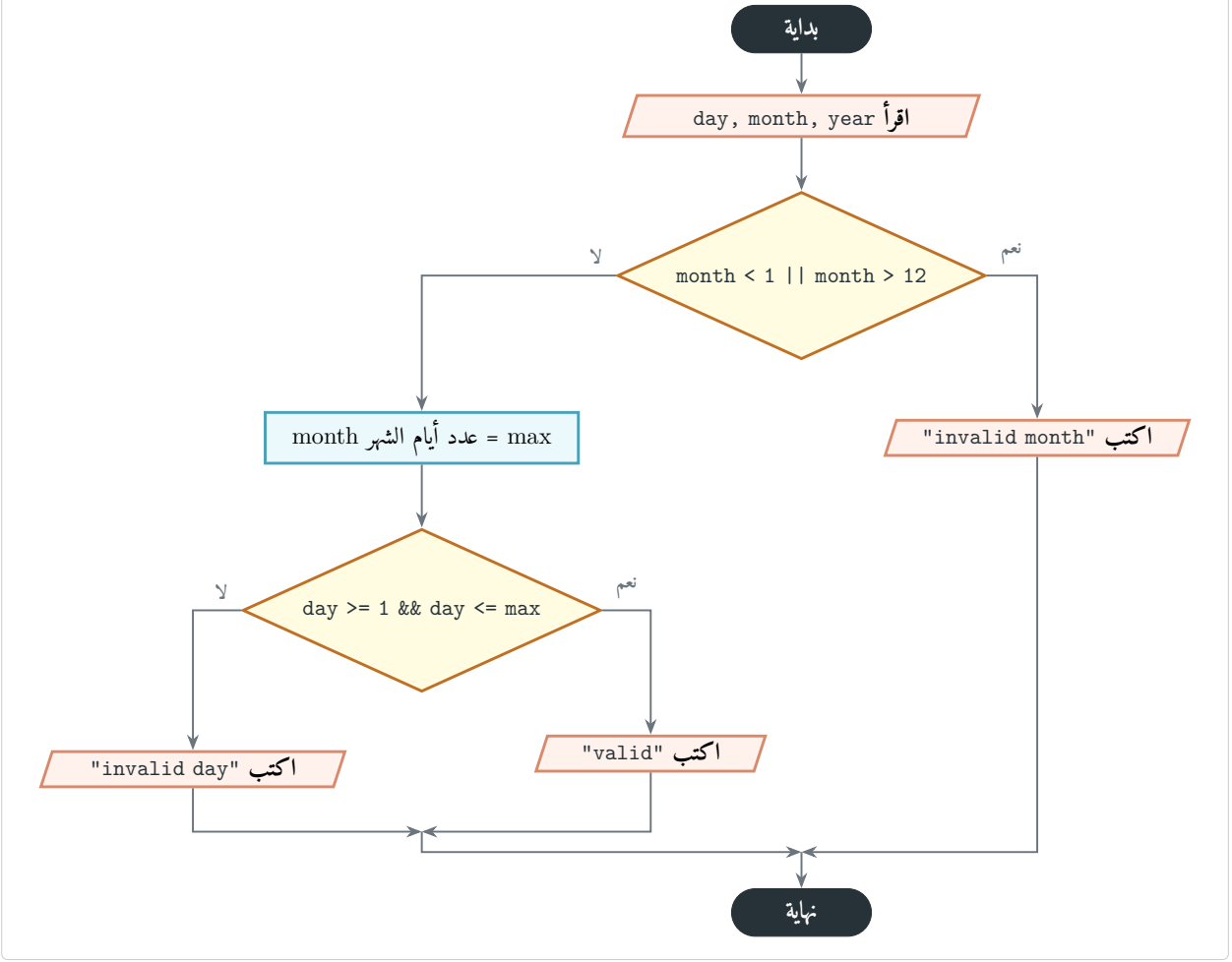
المدخلات

```
29 2 2027
```

نتيجة التنفيذ

```
Invalid day: month 2 has 28 days
```

المخطط الإنسيابي (Organigramme)



تمرين 2.24 اليوم الموالي

تجارة إلكترونية متوسط

التوصيل يتم دائمًا في اليوم الموالي للطلب. اكتب برنامجًا يقرأ تاريخ الطلب (صحيحًا) ويعرض تاريخ التوصيل، مع الانتقال إلى الشهر والسنة التالية عند الحاجة.

خطوات الحل

1) نحسب عدد أيام الشهر `max`.
2) إذا كان `day < max` نزيد اليوم فقط.
3) وإلا: اليوم 1، والشهر يزيد، وإذا كان الشهر 12 يصبح 1 وتزيد السنة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, month, year, max;
    scanf("%d %d %d", &day, &month, &year);
    if (month == 2)
        max = ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) ? 29 : 28;
    else if (month == 4 || month == 6 || month == 9 || month == 11)
        max = 30;
```

```c
    else
        max = 31;
    if (day < max) {
        day = day + 1;                  // نفس الشهر
    } else {
        day = 1;                        // بداية شهر جديد
        if (month == 12) {
            month = 1;                  // بداية سنة جديدة
            year = year + 1;
        } else {
            month = month + 1;
        }
    }
    printf("Delivery date: %02d/%02d/%d\n", day, month, year);
    return 0;
}
```

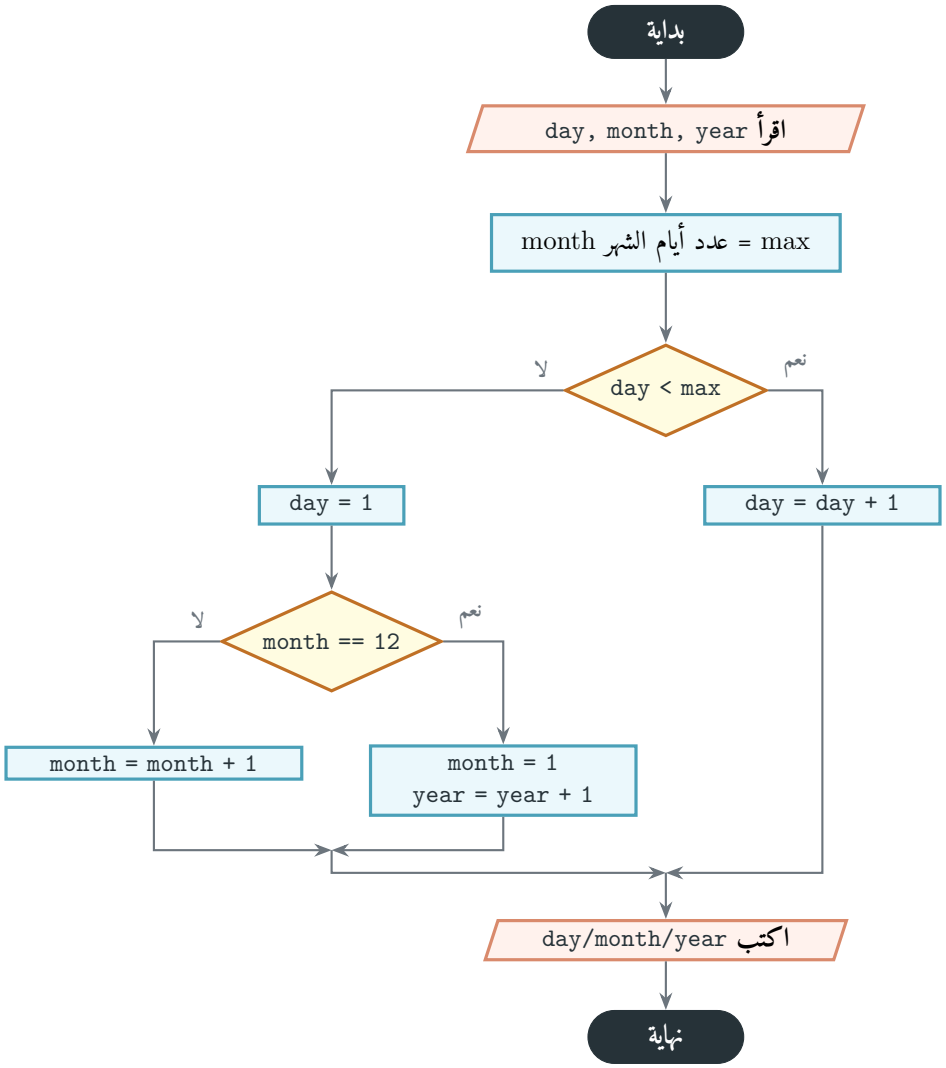
⌨ المدخلات

```
31 12 2026
```

>_ نتيجة التنفيذ

```
Delivery date: 01/01/2027
```

المخطط الإنسيابي (Organigramme)



تمرين 2.25 خلية مصفوفة الإلتباس

علم البيانات متوسط

نموذج يتنبأ هل سيلغي الزبون طلبيته (1) أم لا (0). اكتب برنامجًا يقرأ القيمة **الحقيقية** والقيمة **المتنبأ بها** ويحدد نوع النتيجة: TP أو TN أو FP أو FN مع شرح.

خطوات الحل

1) إذا تساوت القيمتان فالتنبؤ صحيح: TP إذا كانت 1 وTN إذا كانت 0.
2) وإلا فهو خطأ: FP إذا تنبأ بـ 1، وFN إذا تنبأ بـ 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int actual, pred;
    scanf("%d %d", &actual, &pred);
    if (actual == pred) {            // تنبؤ صحيح
```

```c
        if (pred == 1)
            printf("TP: cancellation correctly detected\n");
        else
            printf("TN: normal order correctly detected\n");
    } else {                            // تنبؤ خاطئ
        if (pred == 1)
            printf("FP: false alarm\n");
        else
            printf("FN: missed cancellation\n");
    }
    return 0;
}
```

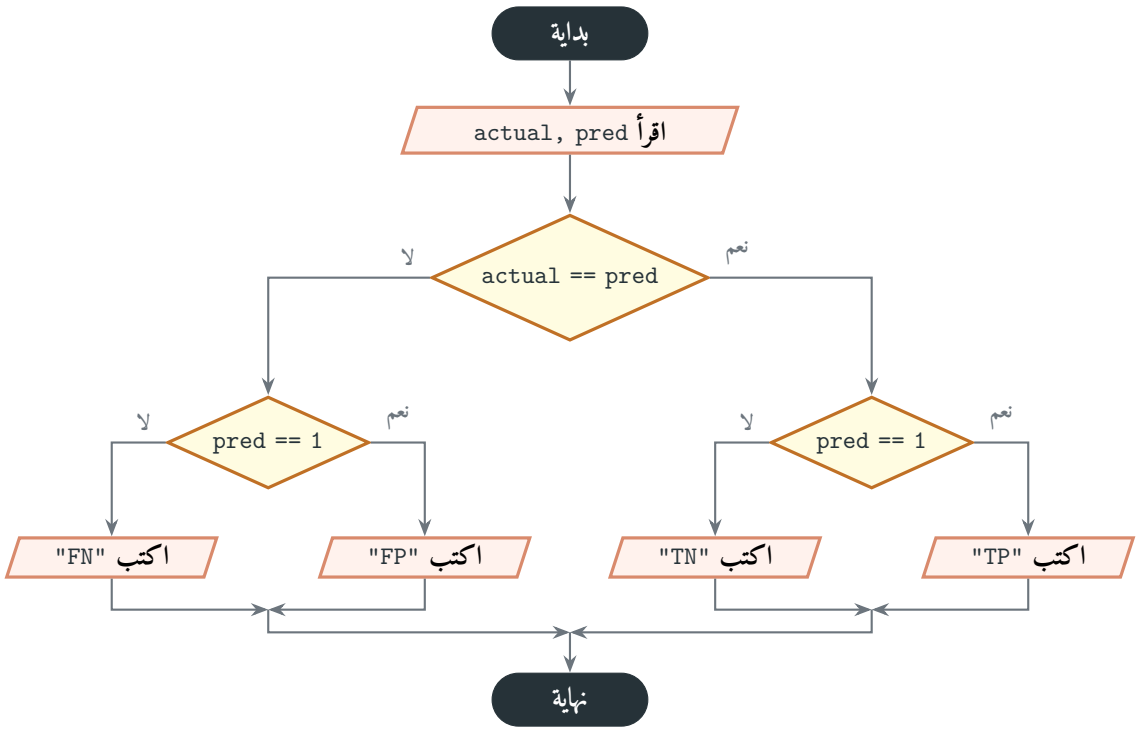
المدخلات

```
0 1
```

نتيجة التنفيذ

```
FP: false alarm
```

المخطط الإنسيابي (Organigramme)



تمرين 2.26 قاعدة توصية

علم البيانات ★★ متوسط

استخرجنا من بيانات المشتريات القاعدة: من اشترى **هاتفًا** ولم يشترِ **غلافًا** نقترح عليه غلافًا؛ من اشترى غلافًا وشاحنًا معًا نقترح سماعات؛ وإلا لا توصية. اكتب برنامجًا يقرأ ثلاث قيم (0 أو 1) للهاتف والغلاف والشاحن ويعرض التوصية.

خطوات الحل

1) نقرأ phone وcover وcharger.
2) القاعدة الأولى: phone && !cover.
3) القاعدة الثانية: cover && charger.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int phone, cover, charger;
    scanf("%d %d %d", &phone, &cover, &charger);
    // 0 يساوي cover إذا كان cover صحيح! تعني النفي: !
    if (phone && !cover)
        printf("Recommend: phone cover\n");
    else if (cover && charger)
        printf("Recommend: earphones\n");
    else
        printf("No recommendation\n");
    return 0;
}
```

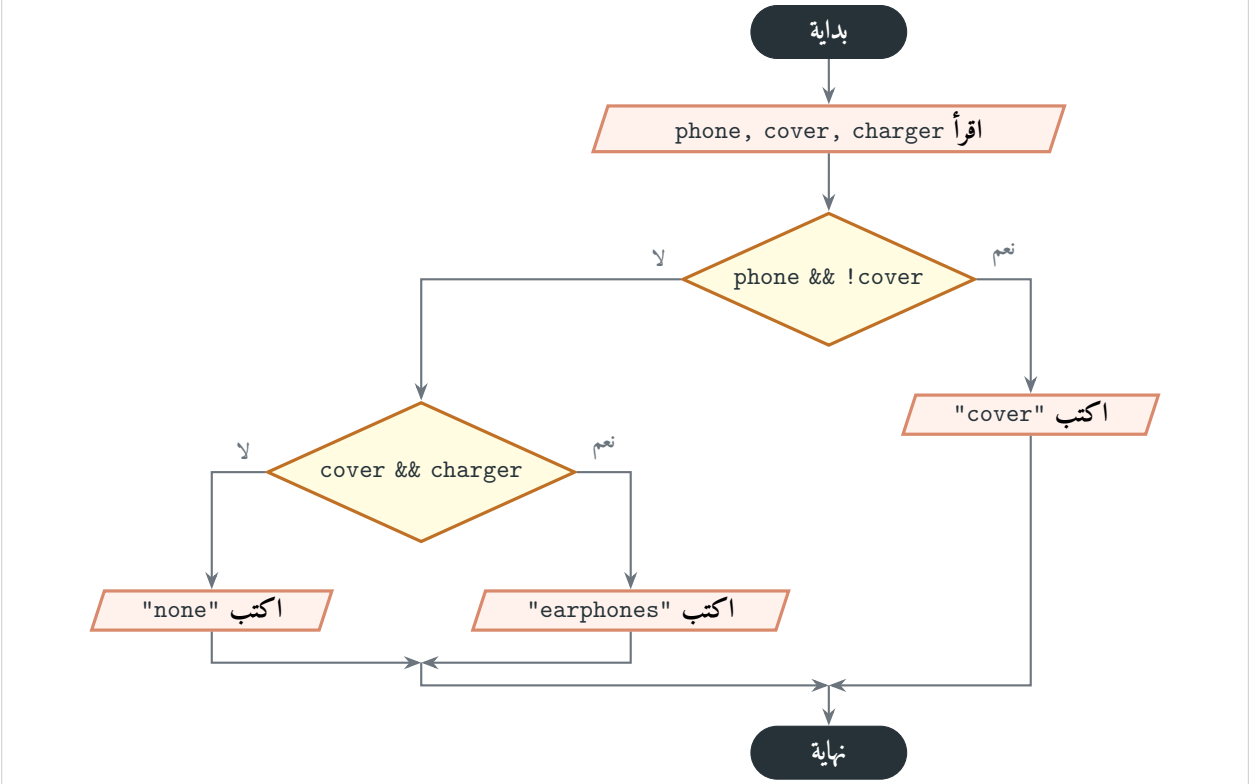
المدخلات

```
1 0 1
```

نتيجة التنفيذ

```
Recommend: phone cover
```

المخطط الإنسيابي (Organigramme)



تمرين 2.27 منطقة التوصيل

تجارة إلكترونية ★★ متوسط

تقسم شركة التوصيل الولايات حسب رقمها: الولاية 16 (الجزائر) منطقة 0 بـ 300 دج، الولايات 9 و35 و42 منطقة 1 بـ 500 دج، الولايات من 1 إلى 30 الباقية منطقة 2 بـ 700 دج، ومن 31 إلى 58 منطقة 3 بـ 900 دج. اكتب برنامجًا يحدد المنطقة والسعر.

خطوات الحل

1) نتحقق أولًا من صحة الرقم (1 إلى 58).
2) نختبر الحالات الخاصة قبل المجالات العامة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int wilaya, zone, price;
    scanf("%d", &wilaya);
    if (wilaya < 1 || wilaya > 58) {
        printf("Invalid wilaya code\n");
    } else {
        // الحالات الخاصة أولًا، ثم المجالات العامة
        if (wilaya == 16) {
            zone = 0; price = 300;
        } else if (wilaya == 9 || wilaya == 35 || wilaya == 42) {
            zone = 1; price = 500;
        } else if (wilaya <= 30) {
            zone = 2; price = 700;
        } else {
            zone = 3; price = 900;
        }
        printf("Wilaya %d: zone %d, %d DA\n", wilaya, zone, price);
    }
    return 0;
}
```

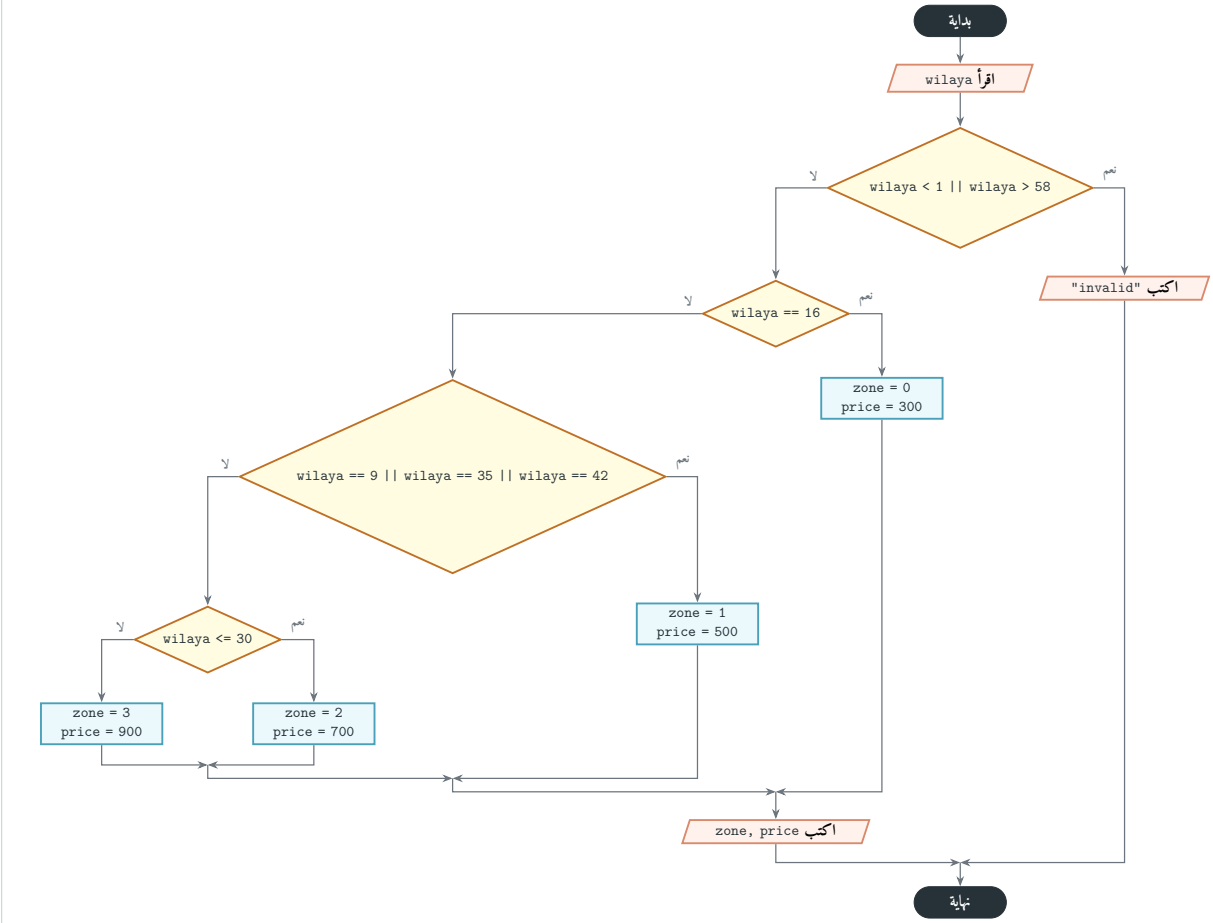
المدخلات

```
35
```

نتيجة التنفيذ

```
Wilaya 35: zone 1, 500 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.28 عرض 3 بثمن 2

تجارة إلكترونية ★★ متوسط

عرض ترويجي: كل **3 وحدات** من نفس المنتج يدفع الزبون ثمن 2 فقط. وإذا اشترى 10 وحدات أو أكثر يحصل أيضًا على تخفيض إضافي 5% على المبلغ. اكتب برنامجًا يحسب المبلغ والتوفير.

خطوات الحل

1) الوحدات المجانية: `free = quantity / 3`.
2) المبلغ: `(quantity - free) * price`.
3) إذا كان `quantity >= 10` نطبق 5% إضافية.
4) التوفير: `total - quantity * price`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int quantity, free;
    double price, total;
    scanf("%d %lf", &quantity, &price);
    free = quantity / 3;                    // وحدة مجانية لكل 3
```

```c
    total = (quantity - free) * price;
    if (quantity >= 10)
        total = total * 0.95;          // تخفيض إضافي للكميات الكبيرة
    printf("Free units = %d\n", free);
    printf("Total = %.2f DA\n", total);
    printf("Saved = %.2f DA\n", quantity * price - total);
    return 0;
}
```

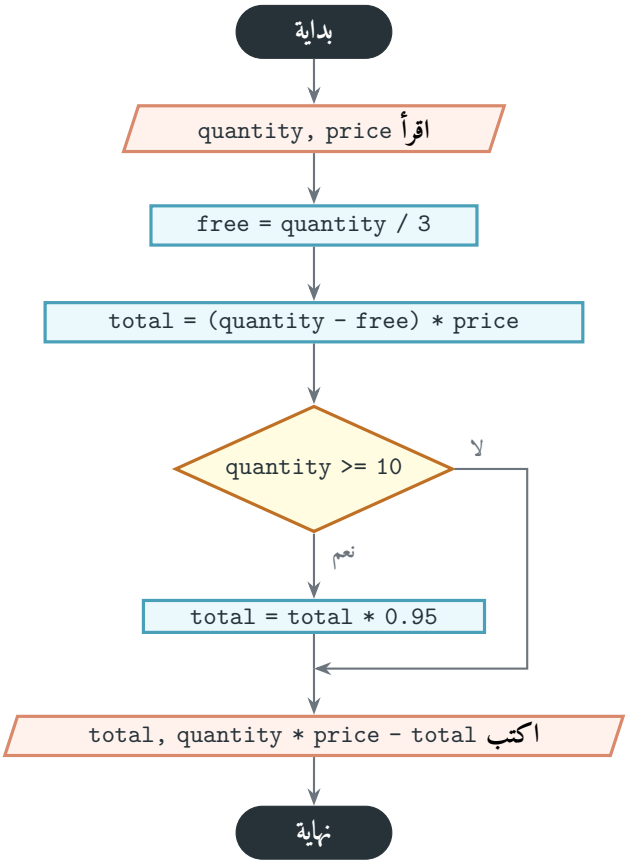
المدخلات

```
11 850
```

نتيجة التنفيذ

```
Free units = 3
Total = 6460.00 DA
Saved = 2890.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.29 تداخل فترتي ترويج

تجارة إلكترونية ★★ متوسط

لدينا حملتان ترويجيتان، لكل منهما يوم بداية ويوم نهاية (أرقام أيام السنة). اكتب برنامجًا يحدد إن كانت الحملتان **متداخلتين**، ويعرض فترة التداخل وطولها.

خطوات الحل

1) بداية التداخل: الأكبر بين البدايتين `overlapStart`.
2) نهاية التداخل: الأصغر بين النهايتين `overlapEnd`.
3) التداخل موجود إذا كان `overlapStart <= overlapEnd`، وطوله `overlapEnd - overlapStart + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int a1, b1, a2, b2, overlapStart, overlapEnd;
    scanf("%d %d %d %d", &a1, &b1, &a2, &b2);
    overlapStart = (a1 > a2) ? a1 : a2;       // البداية الأكبر
    overlapEnd = (b1 < b2) ? b1 : b2;       // النهاية الأصغر
    if (overlapStart <= overlapEnd)
        printf("Overlap: days %d to %d (%d days)\n", overlapStart, overlapEnd, overlapEnd - overlapStart + 1);
    else
        printf("No overlap\n");
    return 0;
}
```

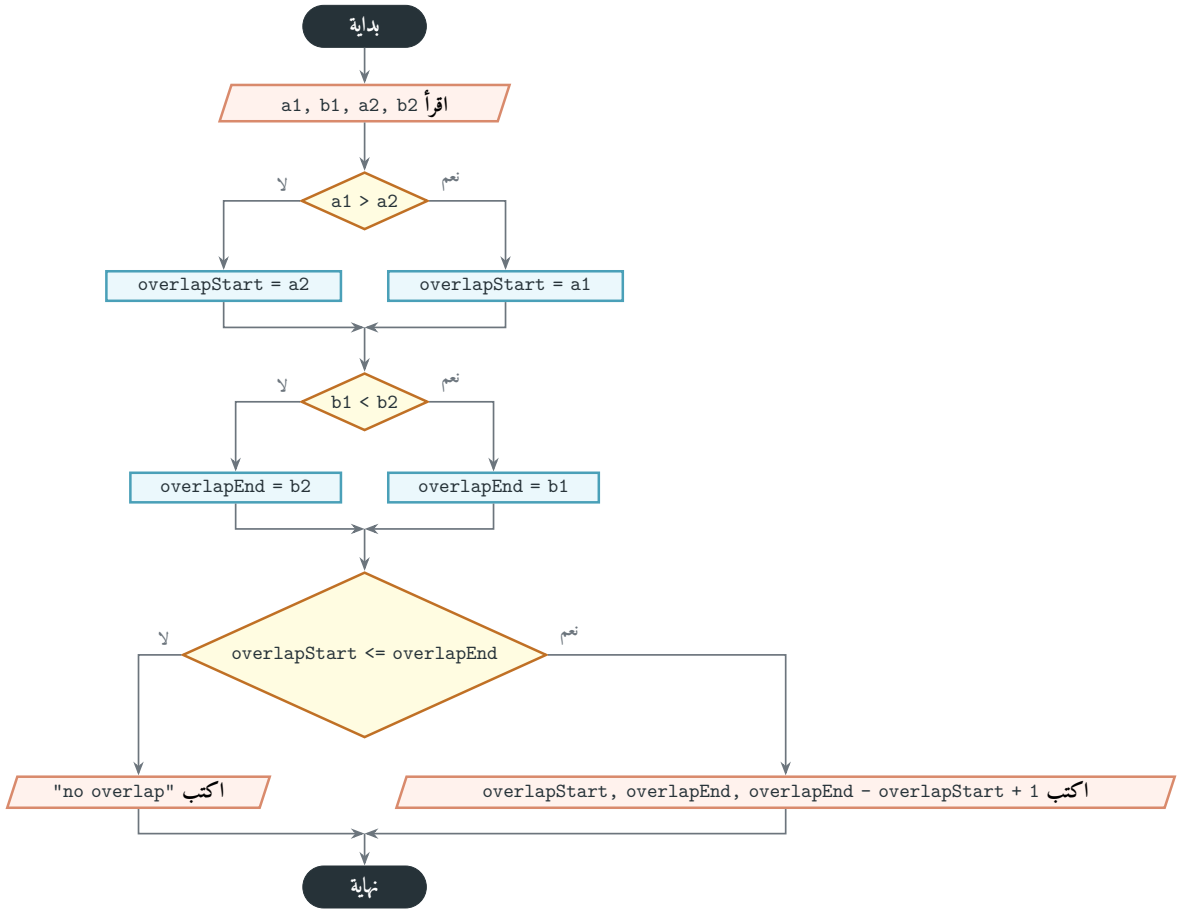
المدخلات

```
40 75
60 90
```

نتيجة التنفيذ

```
Overlap: days 60 to 75 (16 days)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.30 آلة حاسبة للأسعار

تجارة إلكترونية ★★ متوسط

يستعمل موظف المخزن آلة حاسبة بسيطة: يقرأ عددين وعاملًا (+ أو - أو * أو / أو % للنسبة المئوية: a% من b). اكتب البرنامج باستعمال `switch` مع منع القسمة على صفر.

خطوات الحل

1) نقرأ `left` ثم العامل `op` ثم `right`.
2) لكل عامل حالة في `switch`.
3) في حالة `/` نتحقق أن `right != 0`.
4) `%` تعطي `left * right / 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double left, right, result = 0;
    char op;
    int ok = 1;                         // 1 إذا كانت العملية صالحة
    scanf("%lf %c %lf", &left, &op, &right);
    switch (op) {
```

```c
        case '+': result = left + right; break;
        case '-': result = left - right; break;
        case '*': result = left * right; break;
        case '/':
            if (right == 0) ok = 0;        // لا نقسم على صفر
            else result = left / right;
            break;
        case '%': result = left * right / 100; break;   // right بالمئة من left
        default: ok = 0;
    }
    if (ok)
        printf("%.2f %c %.2f = %.2f\n", left, op, right, result);
    else
        printf("Error: invalid operation\n");
    return 0;
}
```

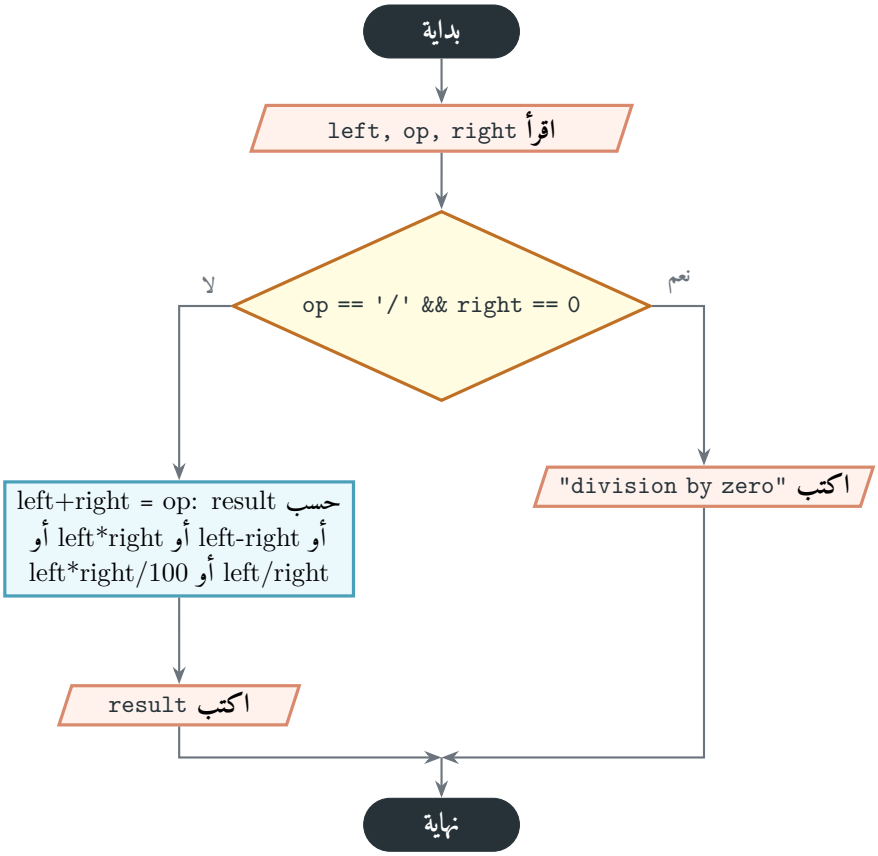
المدخلات

```
15 % 8400
```

نتيجة التنفيذ

```
15.00 % 8400.00 = 1260.00
```

المخطط الإنسيابي (Organigramme)



تمرين 2.31 تأجيل التوصيل في عطلة نهاية الأسبوع — تجارة إلكترونية — ★★ متوسط

لا توصيل يومي **الجمعة والسبت**. أيام الأسبوع مرقمة من 0 (الأحد) إلى 6 (السبت). اكتب برنامجًا يقرأ يوم الطلب ومدة التحضير بالأيام، ويحدد يوم التوصيل؛ إذا وقع في الجمعة أو السبت يؤجَّل إلى الأحد.

خطوات الحل

1) يوم التوصيل: `deliveryDay = (day + prep) % 7`.
2) إذا كان `deliveryDay == 5` نضيف يومين، وإذا كان `deliveryDay == 6` نضيف يومًا، ثم `% 7`.
3) نعرض اسم اليوم باستعمال `switch`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, prep, deliveryDay;
    scanf("%d %d", &day, &prep);
    deliveryDay = (day + prep) % 7;              // اليوم الطبيعي للتوصيل
    if (deliveryDay == 5 || deliveryDay == 6) {  // الجمعة أو السبت
        printf("Weekend: postponed to Sunday\n");
        deliveryDay = 0;
    }
    printf("Delivery day: ");
    switch (deliveryDay) {
        case 0: printf("Sunday\n"); break;
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
    }
    return 0;
}
```

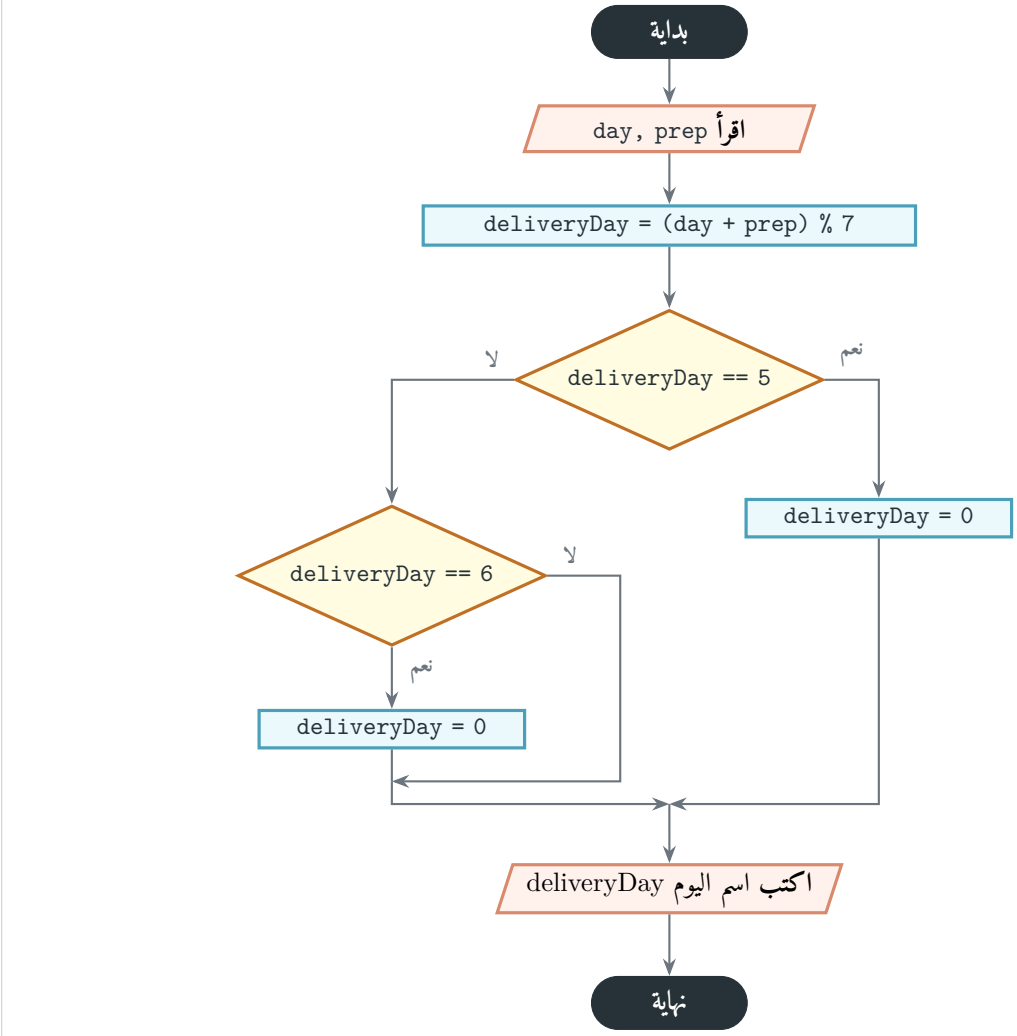
المدخلات

```
3 2
```

نتيجة التنفيذ

```
Weekend: postponed to Sunday
Delivery day: Sunday
```

المخطط الإنسيابي (Organigramme)



تمرين 2.32 صلاحية البطاقة البنكية

تجارة إلكترونية متوسط

قبل قبول الدفع تحقق أن البطاقة **غير منتهية الصلاحية**: تاريخ انتهائها (شهر، سنة) يجب أن يكون بعد التاريخ الحالي أو في نفس الشهر. اكتب برنامجًا يقرأ التاريخين ويحكم، ويعرض عدد الأشهر المتبقية.

خطوات الحل

1) نحول كل تاريخ إلى عدد أشهر: `y * 12 + m`.
2) البطاقة صالحة إذا كان `exp >= now`.
3) الأشهر المتبقية: `exp - now`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int em, ey, cm, cy, exp, now;
    scanf("%d %d", &em, &ey);          // تاريخ الانتهاء
```

```c
    scanf("%d %d", &cm, &cy);          // التاريخ الحالي
    // تحويل التاريخ إلى عدد أشهر يسهل المقارنة
    exp = ey * 12 + em;
    now = cy * 12 + cm;
    if (exp >= now)
        printf("Card valid (%d months left)\n", exp - now);
    else
        printf("Card expired\n");
    return 0;
}
```

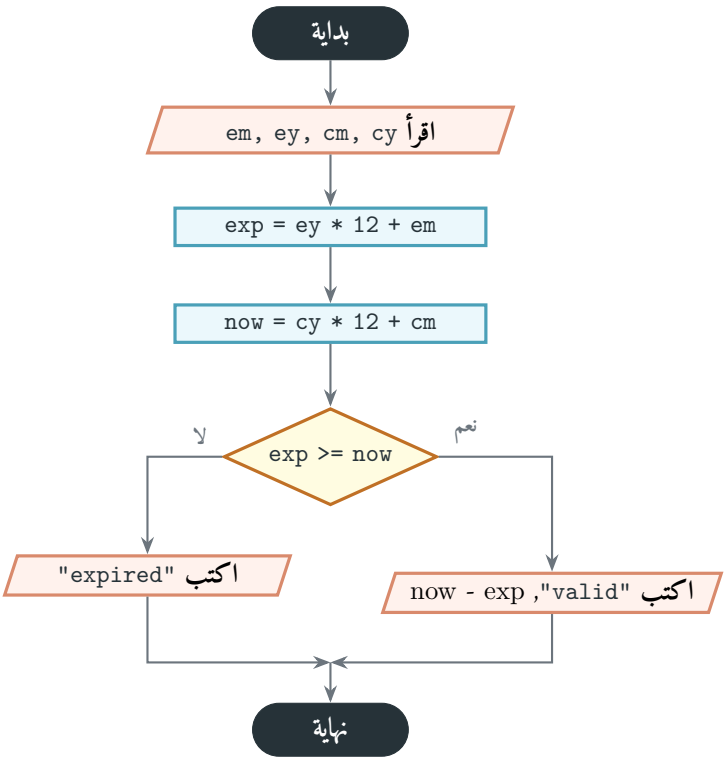
المدخلات

```
3 2027
11 2026
```

نتيجة التنفيذ

```
Card valid (4 months left)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.33 هدية حسب حجم السلة

تجارة إلكترونية ★★ متوسط

تصنف السلات حسب عدد القطع: صغيرة (1-3)، متوسطة (4-9)، كبيرة (10 فأكثر). الهدية: لا شيء للصغيرة، كيس قماشي للمتوسطة، وللكبيرة كيس + قسيمة 500 دج إذا تجاوز المبلغ 20000 دج. اكتب برنامجًا يعرض الصنف والهدية.

خطوات الحل

1) نحدد الصنف بالمجالات.

2) في صنف الكبيرة نضيف شرطًا داخليًا على المبلغ (شرط متداخل).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbItems;
    double amount;
    scanf("%d %lf", &nbItems, &amount);
    if (nbItems >= 10) {
        printf("Large basket: tote bag");
        if (amount > 20000)           // شرط متداخل
            printf(" + 500 DA voucher");
        printf("\n");
    } else if (nbItems >= 4) {
        printf("Medium basket: tote bag\n");
    } else {
        printf("Small basket: no gift\n");
    }
    return 0;
}
```

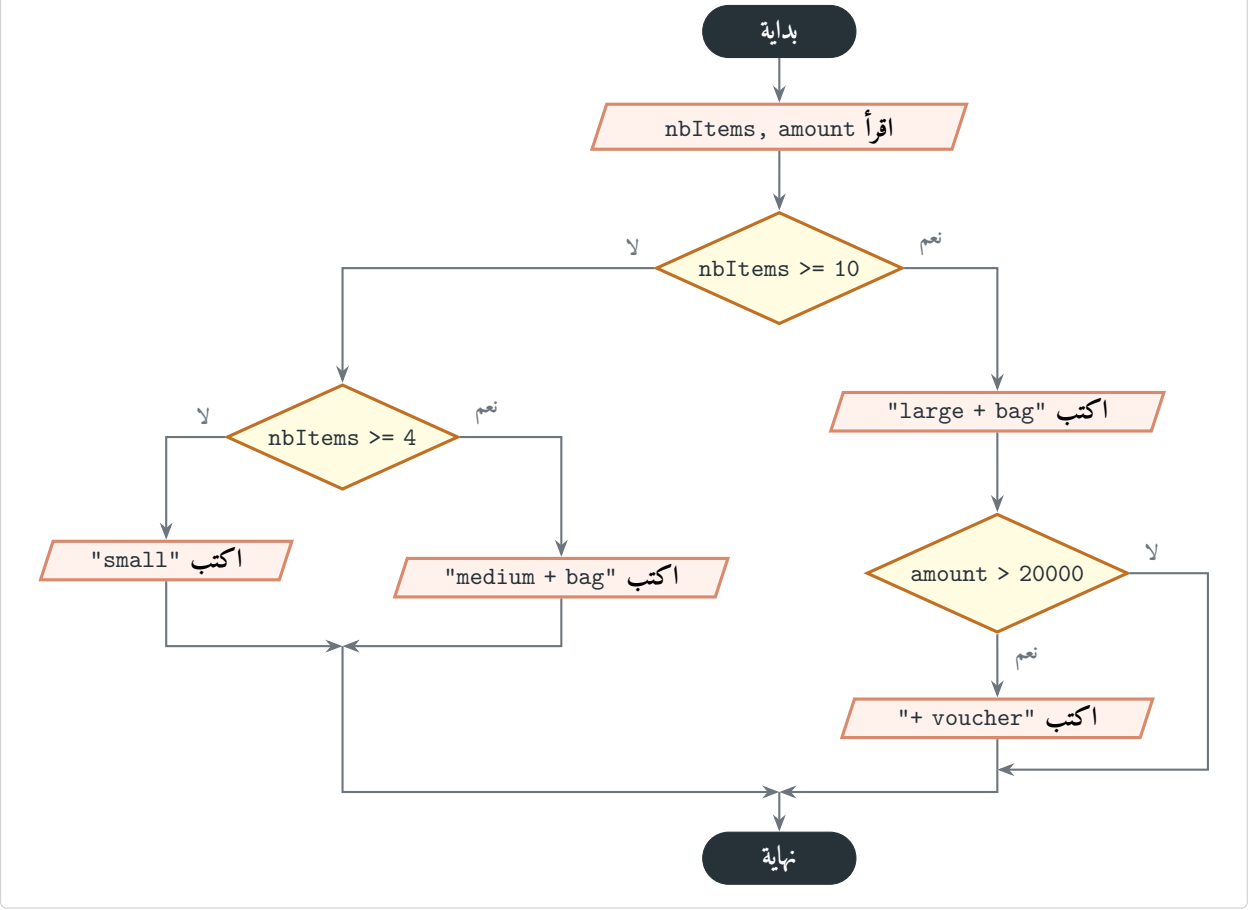
المدخلات

```
12 18500
```

نتيجة التنفيذ

```
Large basket: tote bag
```

المخطط الإنسيابي (Organigramme)



تمرين 2.34 تطور مؤشر الأداء

علم البيانات ★★ متوسط

نقارن قيمة مؤشر (مثل عدد الطلبيات) بين أسبوعين. نعتبر التغير **مستقرًا** إذا كانت النسبة بين -2% و+2%، وإلا **ارتفاعًا** أو **انخفاضًا**، ونعتبره **حادًا** إذا تجاوز 20% بالقيمة المطلقة. اكتب البرنامج.

خطوات الحل

1) نسبة التغير: `change = (now - before) / before * 100`.
2) القيمة المطلقة `ac` يدويًا.
3) إذا كان `ac <= 2` مستقر، وإلا نحدد الاتجاه، ثم نختبر `ac > 20`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double before, now, change, ac;
    scanf("%lf %lf", &before, &now);
    change = (now - before) / before * 100;     // التغير بالمئة
    ac = (change < 0) ? -change : change;                // القيمة المطلقة
    printf("Change = %.1f %% : ", change);
    if (ac <= 2) {
        printf("stable\n");
```

```c
    } else {
        if (change > 0) printf("increase");
        else       printf("decrease");
        if (ac > 20)
            printf(" (sharp!)");          // تنبيه للتغير الكبير
        printf("\n");
    }
    return 0;
}
```

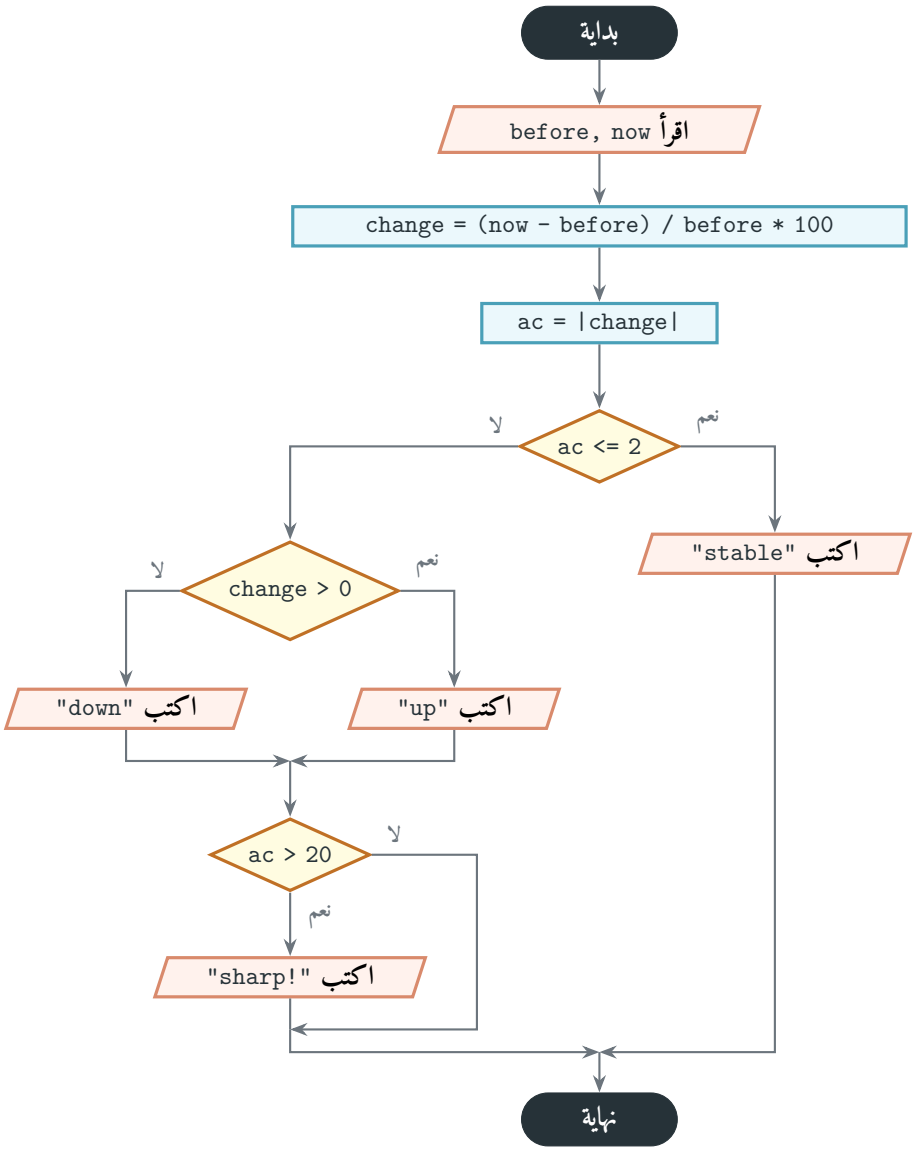
المدخلات

```
1250 980
```

نتيجة التنفيذ

```
Change = -21.6 % : decrease (sharp!)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.35 السعر الأمثل

علم البيانات متوسط

قدّر محلل أن الطلب يتبع Q = demandMax - slope*price. رقم الأعمال
revenue = price * (demandMax - slope*price)
يبلغ أقصاه عند price* = demandMax / (2b). لكن السعر يجب أن يبقى بين pmin وpmax. اكتب برنامجًا يحسب السعر الأمثل المسموح ورقم الأعمال الموافق.

خطوات الحل

1) price = demandMax / (2 * slope).
2) إذا كان price < pmin نأخذ pmin، وإذا كان price > pmax نأخذ pmax.
3) revenue = price * (demandMax - slope * price)

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double demandMax, slope, pmin, pmax, price, revenue;
    scanf("%lf %lf %lf %lf", &demandMax, &slope, &pmin, &pmax);
    price = demandMax / (2 * slope);                // قمة القطع المكافئ
    printf("Unconstrained optimum = %.2f DA\n", price);
    if (price < pmin)
        price = pmin;                               // نحصر السعر في المجال المسموح
    else if (price > pmax)
        price = pmax;
    revenue = price * (demandMax - slope * price);
    printf("Chosen price = %.2f DA\n", price);
    printf("Revenue = %.2f DA\n", revenue);
    return 0;
}
```

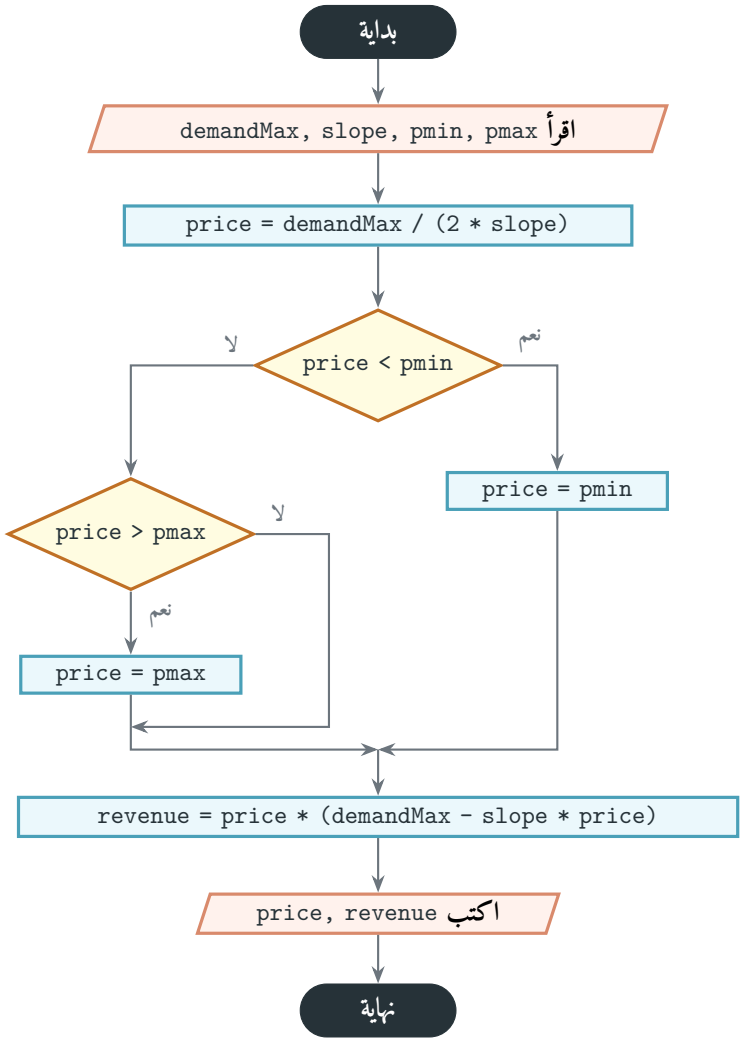
المدخلات

```
5000 2 1500 3000
```

نتيجة التنفيذ

```
Unconstrained optimum = 1250.00 DA
Chosen price = 1500.00 DA
Revenue = 3000000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.36 الوسيط بين ثلاث قيم

علم البيانات ★★ متوسط

الوسيط أقل تأثرًا بالقيم الشاذة من المتوسط. اكتب برنامجًا يقرأ ثلاثة مبالغ طلبيات ويعرض **وسيطها** (القيمة الوسطى) ومتوسطها للمقارنة، دون ترتيب كامل.

خطوات الحل

1) `amount1` هو الوسيط إذا كان بين `amount2` و`amount3`:

```
(amount2 <= amount1 && amount1 <= amount3) || (amount3 <= amount1 && amount1 <= amount2)
```

.

2) نفس الاختبار لـ `amount2`، وإلا فالوسيط `amount3`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount1, amount2, amount3, med;
    scanf("%lf %lf %lf", &amount1, &amount2, &amount3);
    if ((amount2 <= amount1 && amount1 <= amount3) || (amount3 <= amount1 && amount1 <= amount2))
```

```
        med = amount1;                    // amount1 بين الاثنين الآخرين
    else if ((amount1 <= amount2 && amount2 <= amount3) || (amount3 <= amount2 && amount2 <= amount1))
        med = amount2;
    else
        med = amount3;
    printf("Median = %.2f\n", med);
    printf("Mean   = %.2f (pulled by the outlier)\n", (amount1 + amount2 + amount3) / 3);
    return 0;
}
```

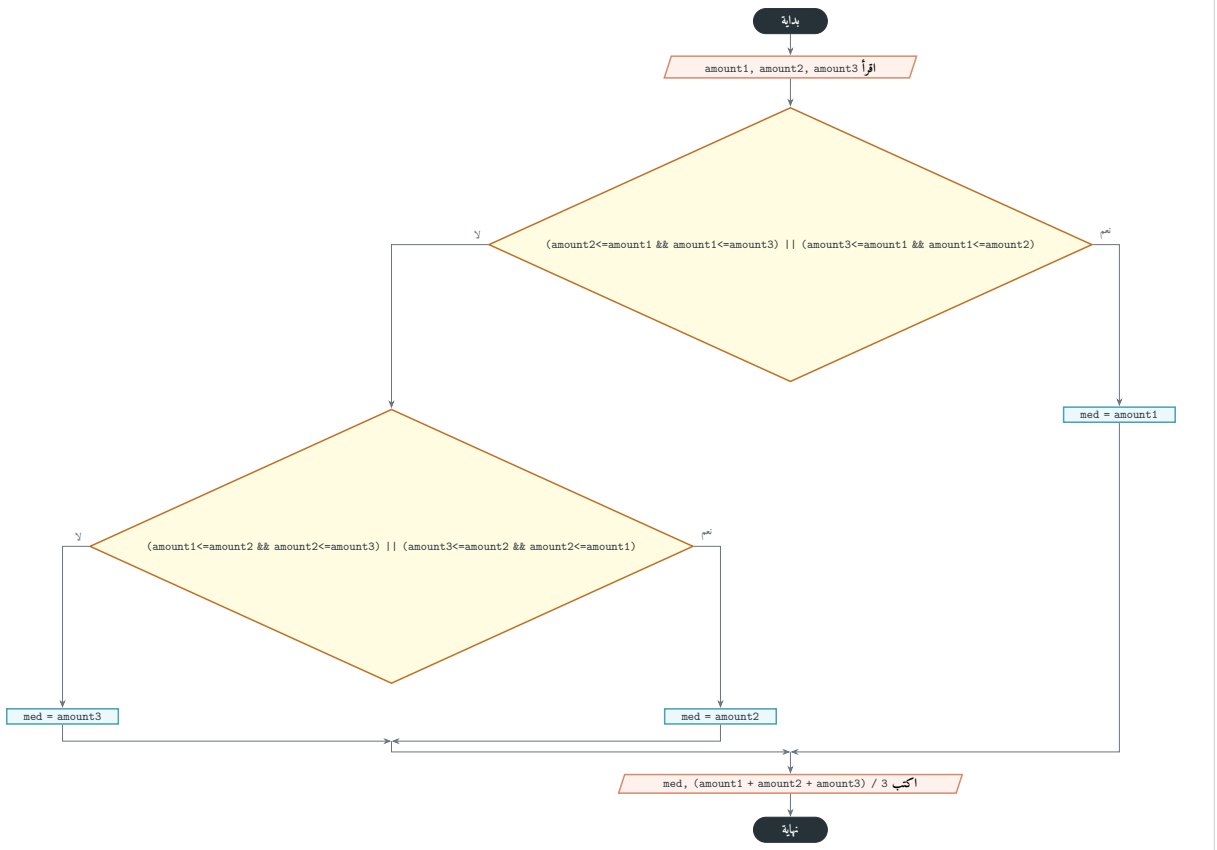
المدخلات

```
1200 98000 1500
```

نتيجة التنفيذ

```
Median = 1500.00
Mean   = 33566.67 (pulled by the outlier)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.37 تسعير حسب ساعة الطلب

تجارة إلكترونية ★★ متوسط

تطبق منصة توصيل الوجبات أسعارًا حسب الساعة: من 11 إلى 13 ومن 19 إلى 21 (ساعات الذروة) زيادة 20%، من 15 إلى 17 (ساعة سعيدة) تخفيض 25%، ومن 0 إلى 6 زيادة ليلية 30%. اكتب برنامجًا يقرأ الساعة والسعر.

خطوات الحل

1) نحدد المعامل `factor` حسب المجال.

2) مجالا الذروة يُجمعان بـ `||`.
3) السعر النهائي `price * factor`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hour;
    double price, factor;
    scanf("%d %lf", &hour, &price);
    if ((hour >= 11 && hour <= 13) || (hour >= 19 && hour <= 21))
        factor = 1.2;                       // ساعات الذروة
    else if (hour >= 15 && hour <= 17)
        factor = 0.75;                      // الساعة السعيدة
    else if (hour >= 0 && hour <= 6)
        factor = 1.3;                       // الليل
    else
        factor = 1.0;                       // السعر العادي
    printf("Hour %d: factor %.2f -> %.2f DA\n", hour, factor, price * factor);
    return 0;
}
```

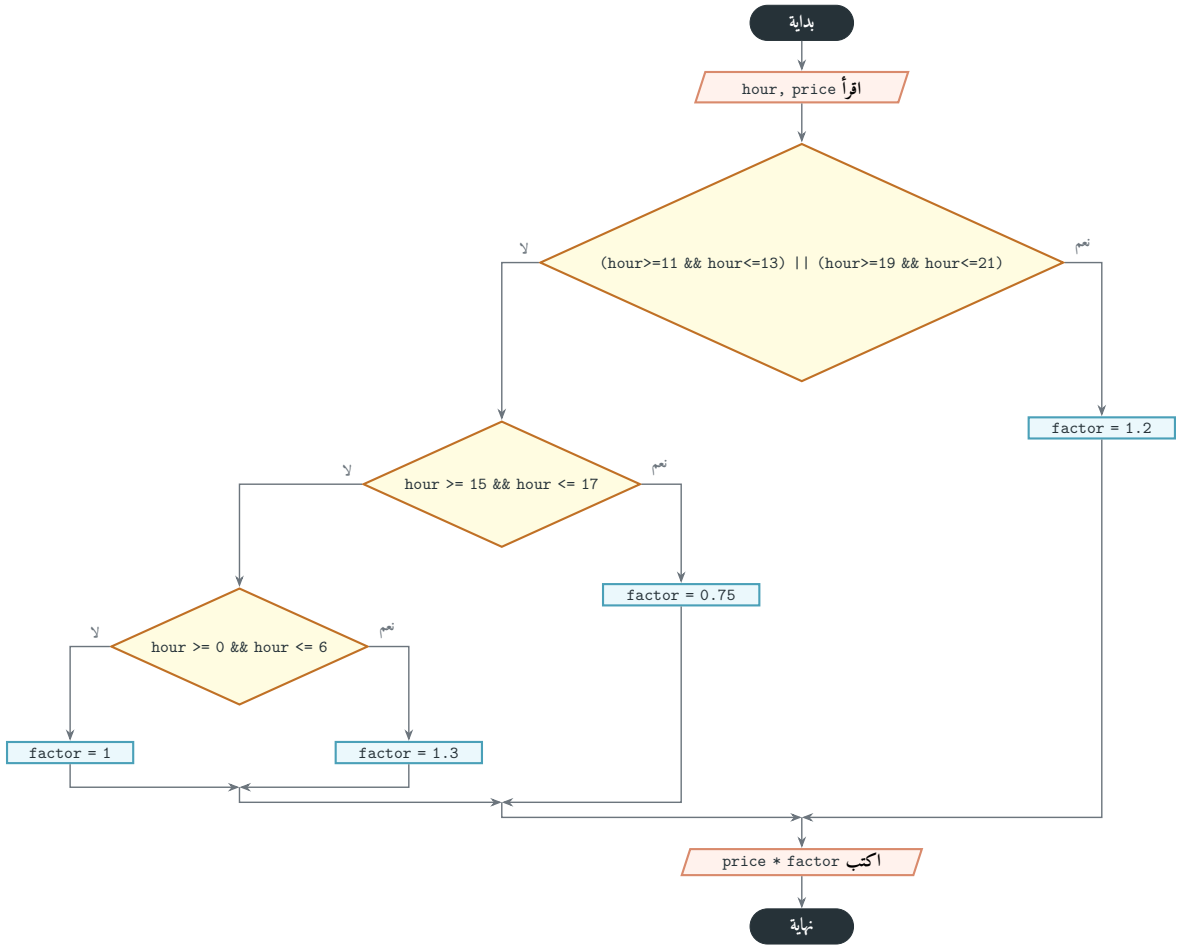
المدخلات

```
20 1400
```

نتيجة التنفيذ

```
Hour 20: factor 1.20 -> 1680.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.38 تصنيف المشاعر

علم البيانات ★★ متوسط

يعطي نموذج تحليل المشاعر لكل تعليق درجة بين -1 و1 ودرجة ثقة بين 0 و1. اكتب برنامجًا يصنف التعليق: إذا كانت الثقة أقل من 0.6 فهو **غير مؤكد**، وإلا: إيجابي (درجة > 0.25)، سلبي (< -0.25)، أو محايد.

خطوات الحل

1) نختبر الثقة أولًا: إنها شرط مسبق.
2) ثم نصنف حسب الدرجة بثلاث حالات.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double score, conf;
    scanf("%lf %lf", &score, &conf);
    if (conf < 0.6) {
        printf("Uncertain (confidence %.2f)\n", conf);
    } else if (score > 0.25) {
        printf("Positive review\n");
    } else if (score < -0.25) {
```

```
        printf("Negative review\n");      // يرسل إلى خدمة الزبائن
    } else {
        printf("Neutral review\n");
    }
    return 0;
}
```

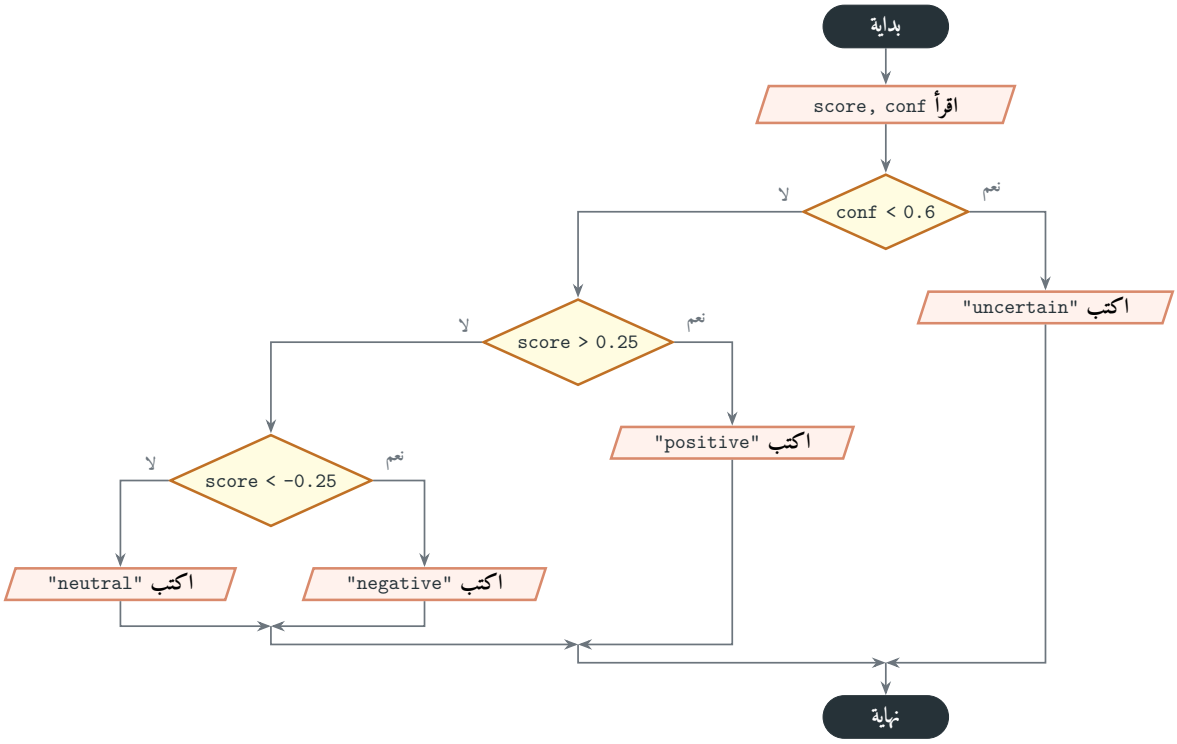
المدخلات

```
-0.62 0.81
```

نتيجة التنفيذ

```
Negative review
```

المخطط الإنسيابي (Organigramme)



تمرين 2.39 كشف الاحتيال بالقواعد

علم البيانات متوسط

نعطي كل عملية دفع **درجة خطر**: +40 إذا تجاوز المبلغ 100000 دج، +30 إذا كان البلد **مختلفًا** عن بلد الحساب، +20 إذا كانت الساعة بين 0 و5، +25 إذا كان الحساب عمره أقل من 7 أيام. العملية **مرفوضة** إذا بلغت الدرجة 70، و**للمراجعة** من 40. اكتب البرنامج.

خطوات الحل

1) نقرأ المبلغ، `foreign` (0 أو 1)، الساعة، وعمر الحساب.
2) كل قاعدة شرط مستقل `if` يضيف نقاطًا (ليست `else if`).
3) القرار النهائي حسب المجموع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double amount;
    int foreign, hour, age, risk = 0;
    scanf("%lf %d %d %d", &amount, &foreign, &hour, &age);
    // شروط مستقلة: كل قاعدة تضيف نقاطها
    if (amount > 100000)          risk = risk + 40;
    if (foreign == 1)             risk = risk + 30;
    if (hour >= 0 && hour <= 5)   risk = risk + 20;
    if (age < 7)                  risk = risk + 25;
    printf("Risk score = %d -> ", risk);
    if (risk >= 70)
        printf("REJECTED\n");
    else if (risk >= 40)
        printf("MANUAL REVIEW\n");
    else
        printf("ACCEPTED\n");
    return 0;
}
```

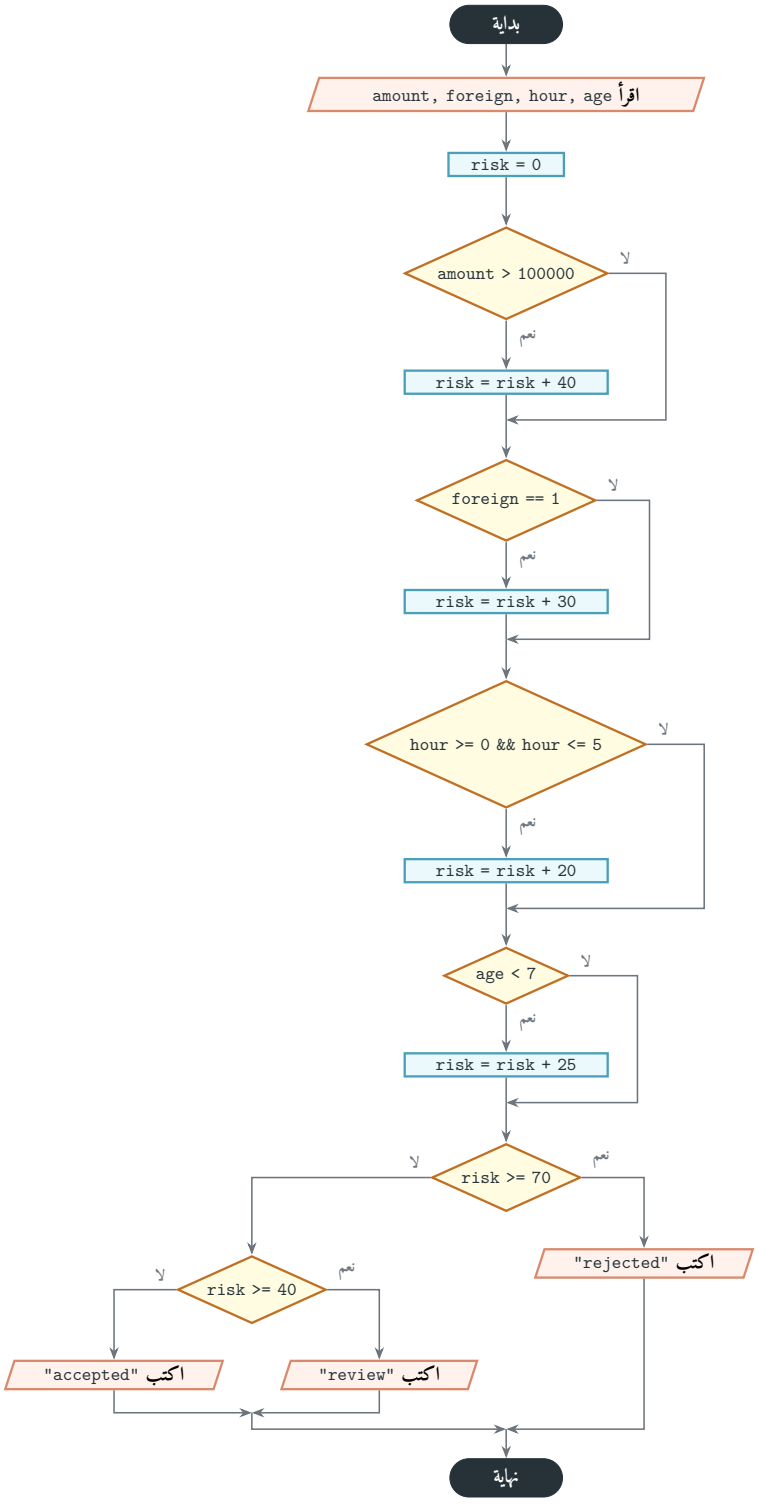
المدخلات

```
125000 1 3 30
```

نتيجة التنفيذ

```
Risk score = 90 -> REJECTED
```

المخطط الإنسيابي (Organigramme)



تمرين 2.40 الفائز في تجربة A/B

علم البيانات ★★ متوسط

نسختان A وB من صفحة منتج. اكتب برنامجًا يقرأ الزوار والمشتريات لكل نسخة، ويعلن الفائز إذا كان الفرق في معدل التحويل **0.5 نقطة على الأقل** وكان لكل نسخة 1000 زائر على الأقل؛ وإلا "غير حاسم".

خطوات الحل

1) نحسب المعدلين ra وrb.
2) شرط حجم العينة: na >= 1000 && nb >= 1000.
3) الفرق gap = ra - rb، ونقارن gap >= 0.5 أو gap <= -0.5.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double na, ca, nb, cb, ra, rb, gap;
    scanf("%lf %lf %lf %lf", &na, &ca, &nb, &cb);
    ra = ca / na * 100;                    // معدل تحويل النسخة A
    rb = cb / nb * 100;                    // معدل تحويل النسخة B
    printf("A: %.2f %%  B: %.2f %%\n", ra, rb);
    if (na < 1000 || nb < 1000) {
        printf("Sample too small\n");      // لا نحكم على عينة صغيرة
    } else {
        gap = ra - rb;
        if (gap >= 0.5)       printf("Winner: A\n");
        else if (gap <= -0.5) printf("Winner: B\n");
        else                  printf("Inconclusive\n");
    }
    return 0;
}
```

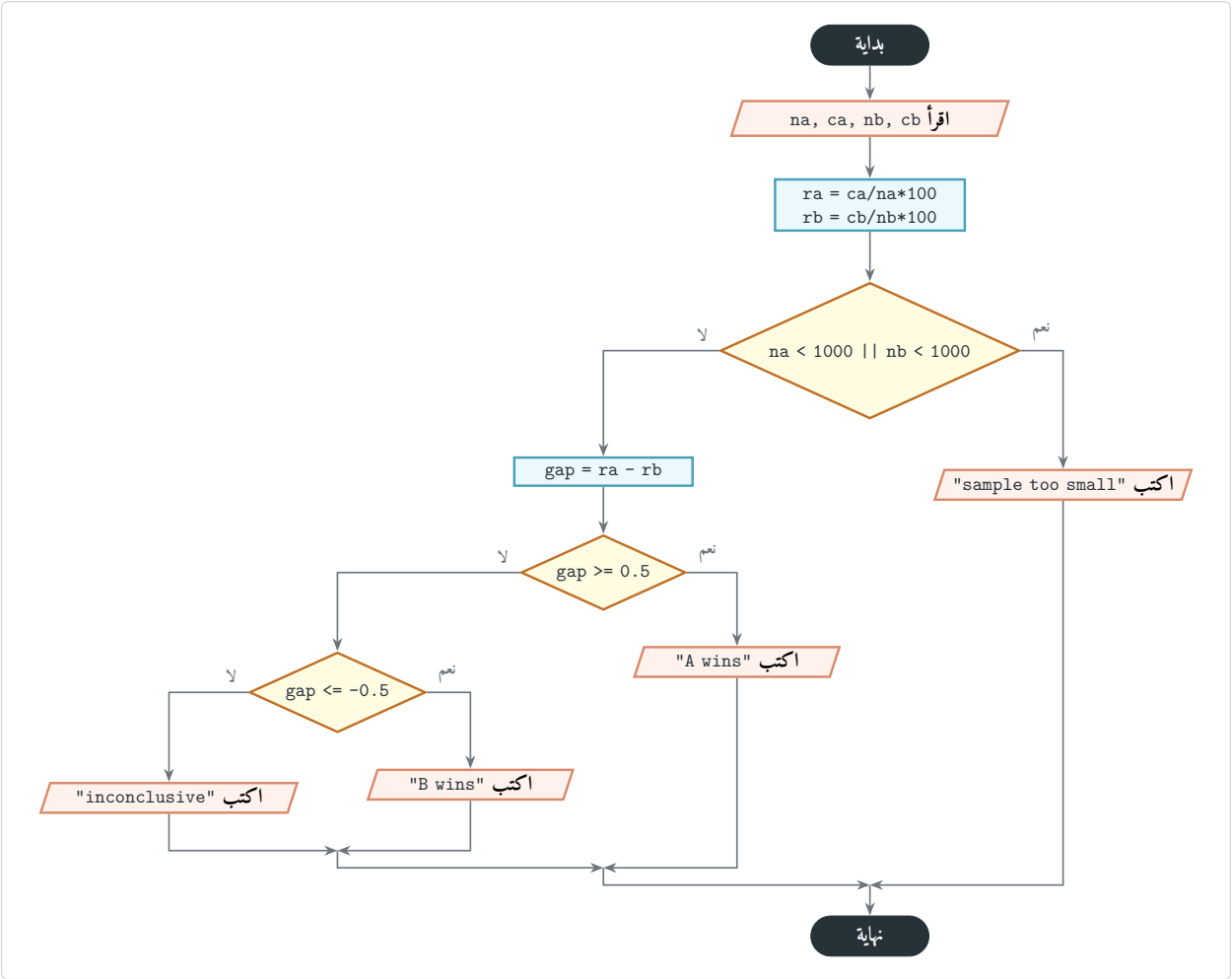
المدخلات

```
1500 60
1450 41
```

نتيجة التنفيذ

```
A: 4.00 %  B: 2.83 %
Winner: A
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 2.41 يوم الأسبوع لتاريخ ما

تجارة إلكترونية ★★★ صعب

لتخطيط الحملات نحتاج يوم الأسبوع لأي تاريخ. اكتب برنامجًا يطبق **صيغة زيلر**: نعتبر جانفي وفيفري الشهرين 13 و14 من السنة السابقة، ثم

`weekday = (day + 13(month+1)/5 + yearInCentury + yearInCentury/4 + century/4 + 5J) % 7`

حيث `yearInCentury = year % 100` و`century = year / 100`، و`weekday = 0` يعني السبت.

خطوات الحل

1) إذا كان `month < 3`: `month = month + 12` و`year = year - 1`.
2) نحسب `yearInCentury` و`century` ثم `weekday` بقسمات صحيحة.
3) نحول `weekday` إلى اسم يوم بـ `switch` (0 سبت، 1 أحد، ...).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int day, month, year, yearInCentury, century, weekday;
    scanf("%d %d %d", &day, &month, &year);
    printf("%02d/%02d/%d is a ", day, month, year);
    if (month < 3) {              // جانفي وفيفري: شهرا 13 و14 من السنة السابقة
        month = month + 12;
        year = year - 1;
    }
    yearInCentury = year % 100;              // السنة داخل القرن
    century = year / 100;                    // القرن
    weekday = (day + 13 * (month + 1) / 5 + yearInCentury + yearInCentury / 4 + century / 4 + 5 * century) % 7;
    switch (weekday) {
        case 0: printf("Saturday\n"); break;
        case 1: printf("Sunday\n"); break;
        case 2: printf("Monday\n"); break;
        case 3: printf("Tuesday\n"); break;
        case 4: printf("Wednesday\n"); break;
        case 5: printf("Thursday\n"); break;
        default: printf("Friday\n");
    }
    return 0;
}
```

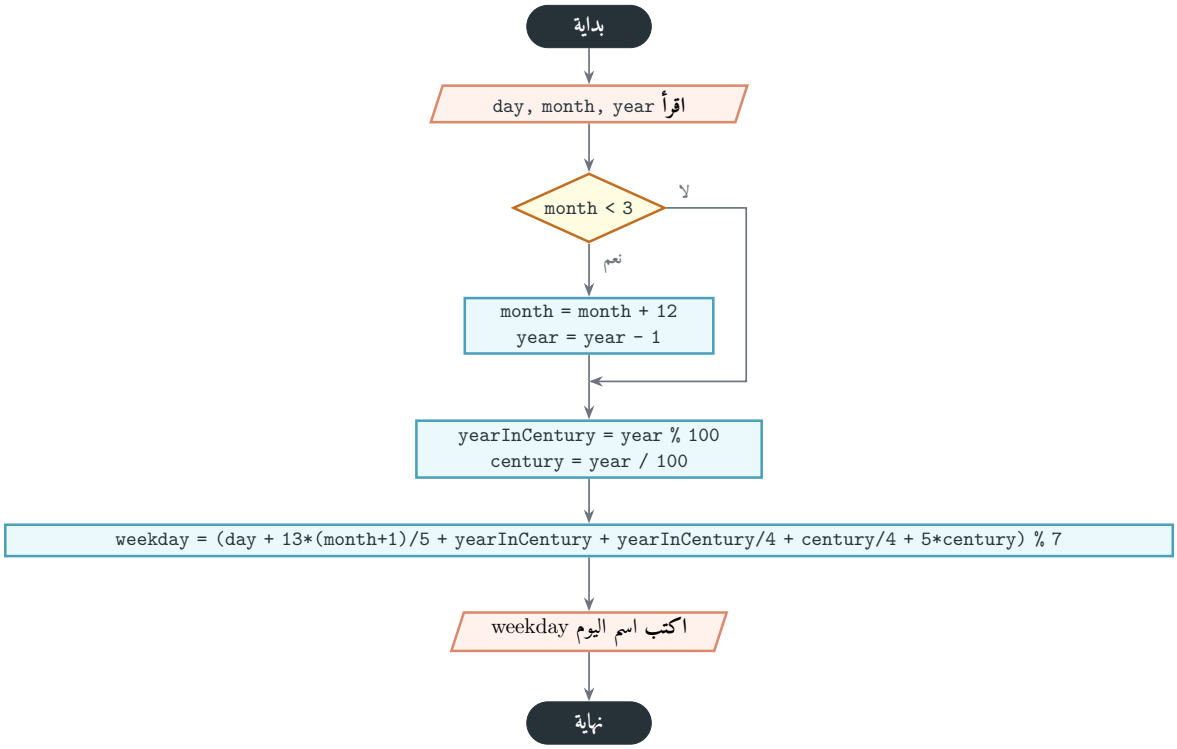
المدخلات

```
1 11 2026
```

نتيجة التنفيذ

```
01/11/2026 is a Sunday
```

المخطط الإنسيابي (Organigramme)



تمرين 2.42 شجرة قرار للتنبؤ بمغادرة الزبون

علم البيانات صعب

درّبنا **شجرة قرار** للتنبؤ بمغادرة الزبون (churn). القواعد: إذا مرّ أكثر من 60 يومًا على آخر طلبية: إذا كان عدد الشكاوى ≥ 2 فـ "مغادر" (احتمال 0.85)، وإلا إذا كان الإنفاق < 10000 فـ "مغادر" (0.65) وإلا "باقٍ" (0.40). إذا مر 60 يومًا أو أقل: إذا كان الإنفاق > 50000 فـ "باقٍ" (0.05)، وإلا إذا كانت الشكاوى ≥ 3 فـ "مغادر" (0.55) وإلا "باقٍ" (0.15). اكتب البرنامج.

خطوات الحل

1) نقرأ الأيام منذ آخر طلبية، عدد الشكاوى، والإنفاق.
2) نترجم كل عقدة في الشجرة إلى `if` وكل فرع إلى `else`.
3) كل ورقة تعطي الصنف والاحتمال.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int days, compl;
    double spend, proba;
    scanf("%d %d %lf", &days, &compl, &spend);
    if (days > 60) {                    // العقدة الجذر
        if (compl >= 2)
            proba = 0.85;
        else if (spend < 10000)
            proba = 0.65;
```

```c
        else
            proba = 0.40;
    } else {
        if (spend > 50000)
            proba = 0.05;
        else if (compl >= 3)
            proba = 0.55;
        else
            proba = 0.15;
    }
    // نصنف الزبون مغادرًا إذا تجاوز الاحتمال 0.5
    printf("Churn probability = %.2f\n", proba);
    if (proba >= 0.5)
        printf("Prediction: will churn -> send offer\n");
    else
        printf("Prediction: will stay\n");
    return 0;
}
```

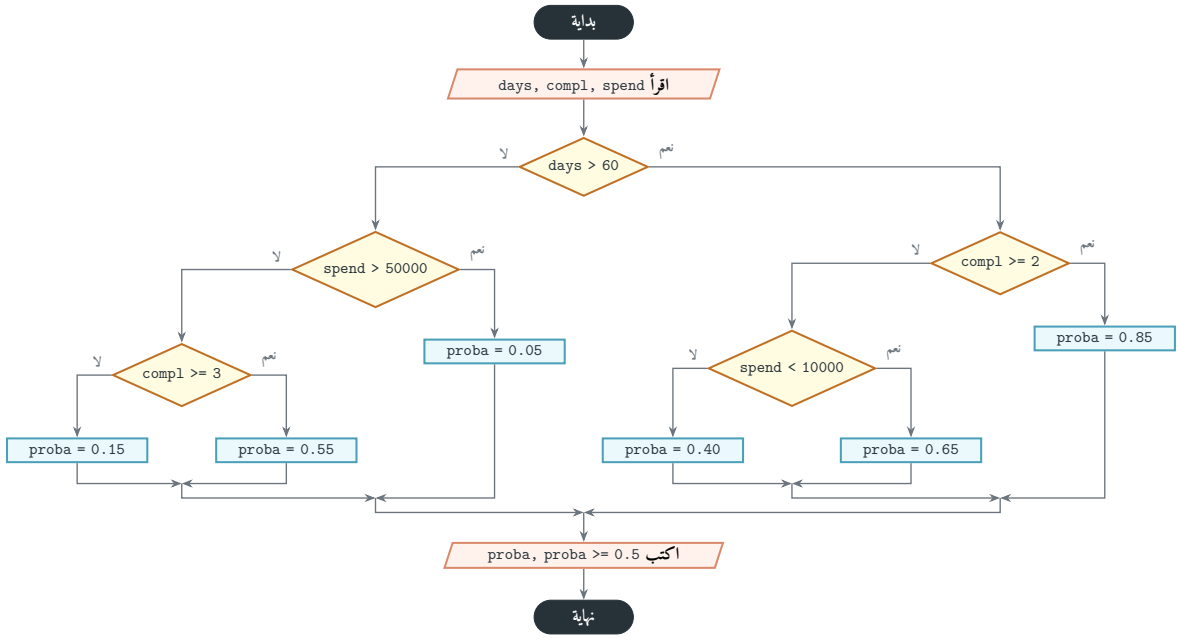
المدخلات

```
75 1 15000
```

نتيجة التنفيذ

```
Churn probability = 0.40
Prediction: will stay
```

المخطط الإنسيابي (Organigramme)



تمرين 2.43 ترتيب أربعة أسعار بشبكة مقارنات

تجارة إلكترونية صعب

اكتب برنامجًا يرتب أربعة أسعار تصاعديًا باستعمال **شبكة ترتيب** ثابتة من 5 مقارنات وتبديلات فقط: (price1,price2)، (price3,price4)، (price1,price3)، (price2,price4)، (price2,price3). ثم اعرض أيضًا الوسيط

(متوسط القيمتين الوسطيتين).

خطوات الحل

1) كل مقارنة `(x,y)`: إذا كان `x > y` نبدلهما.
2) بعد `(price1,price2)` و`(price3,price4)`: كل زوج مرتب.
3) `(price1,price3)` يضع الأصغر في `price1`، و`(price2,price4)` يضع الأكبر في `price4`.
4) `(price2,price3)` يرتب القيمتين الوسطيتين.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price1, price2, price3, price4, tmp;
    scanf("%lf %lf %lf %lf", &price1, &price2, &price3, &price4);
    if (price1 > price2) { tmp = price1; price1 = price2; price2 = tmp; }   // رتب الزوج الأول
    if (price3 > price4) { tmp = price3; price3 = price4; price4 = tmp; }   // رتب الزوج الثاني
    if (price1 > price3) { tmp = price1; price1 = price3; price3 = tmp; }   // أصبح الأصغر price1
    if (price2 > price4) { tmp = price2; price2 = price4; price4 = tmp; }   // أصبح الأكبر price4
    if (price2 > price3) { tmp = price2; price2 = price3; price3 = tmp; }   // رتب الوسطين
    printf("Sorted: %.0f %.0f %.0f %.0f\n", price1, price2, price3, price4);
    printf("Median = %.1f\n", (price2 + price3) / 2);
    return 0;
}
```

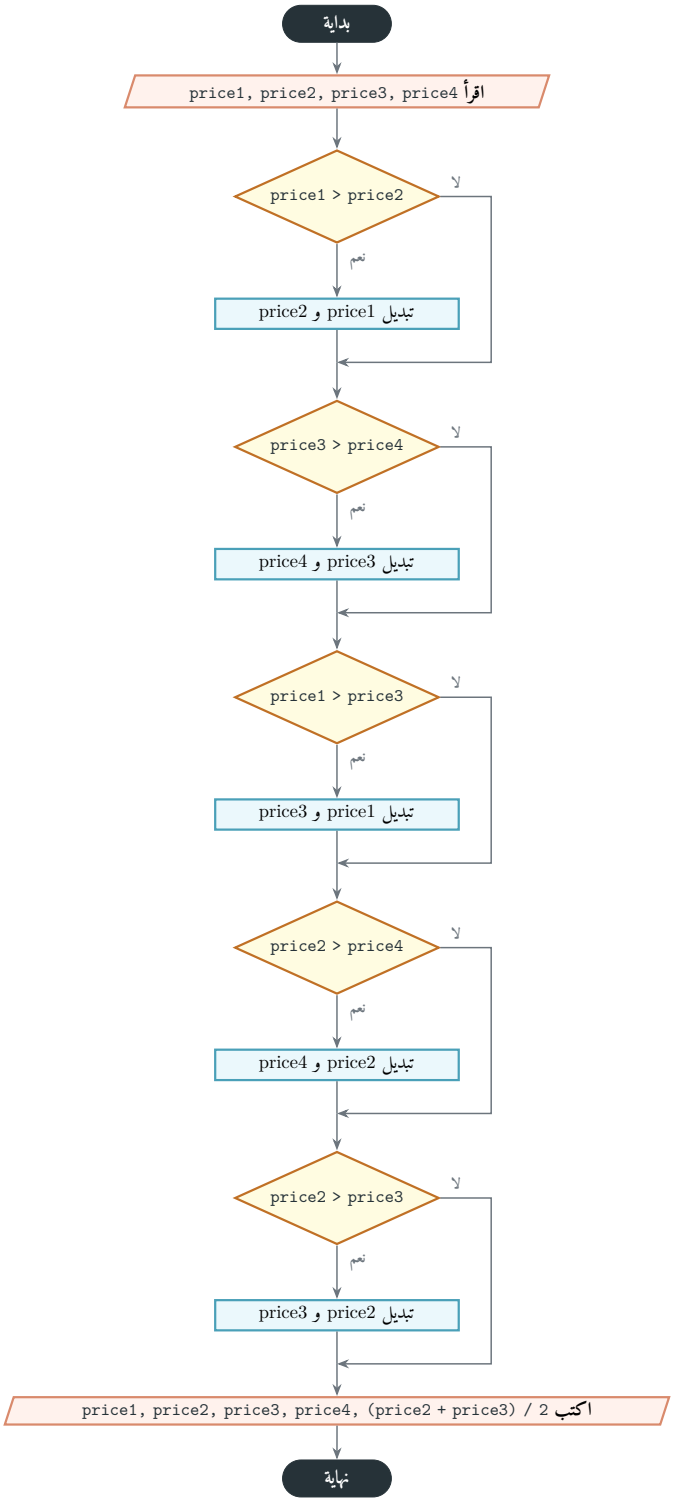
المدخلات

```
780 120 455 300
```

نتيجة التنفيذ

```
Sorted: 120 300 455 780
Median = 377.5
```

المخطط الإنسيابي (Organigramme)



تمرين 2.44 الضريبة التصاعدية على دخل البائع

تجارة إلكترونية صعب

يدفع البائع على المنصة ضريبة **تصاعدية** على دخله السنوي: 0% حتى 240000، 23% للجزء بين 240000 و480000، 27% للجزء بين 480000 و960000، 30% للجزء بين 960000 و1920000، و33% لما فوق ذلك. اكتب برنامجًا يحسب الضريبة والنسبة الفعلية.

خطوات الحل

1) كل شريحة تُطبق نسبتها على الجزء الواقع داخلها فقط.
2) نبدأ من الشريحة العليا: إذا كان `rest > 1920000` نضيف `(rest - 1920000) * 0.33` ونضع `rest = 1920000`.
3) نكرر نزولًا حتى الشريحة الأولى.
4) النسبة الفعلية: `tax / income * 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double income, rest, tax = 0;
    scanf("%lf", &income);
    rest = income;                          // الجزء الذي لم يُضرب بعد
    if (rest > 1920000) {                   // الجزء فوق 1920000
        tax = tax + (rest - 1920000) * 0.33;
        rest = 1920000;
    }
    if (rest > 960000) {
        tax = tax + (rest - 960000) * 0.30;
        rest = 960000;
    }
    if (rest > 480000) {
        tax = tax + (rest - 480000) * 0.27;
        rest = 480000;
    }
    if (rest > 240000)                      // الشريحة الأولى معفاة
        tax = tax + (rest - 240000) * 0.23;
    printf("Tax = %.2f DA\n", tax);
    if (income > 0)
        printf("Effective rate = %.2f %%\n", tax / income * 100);
    return 0;
}
```

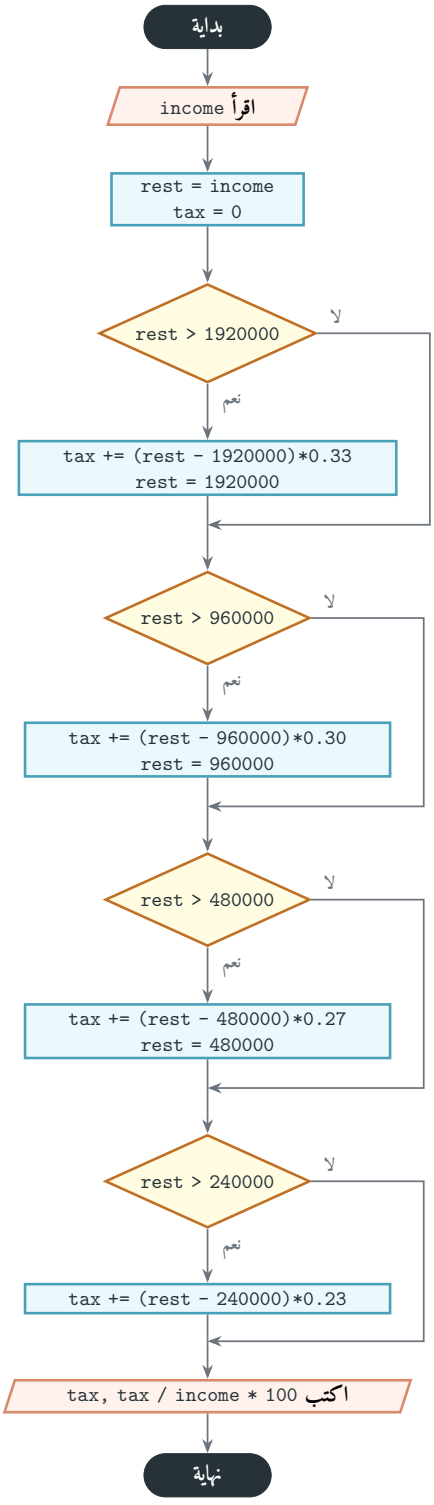
المدخلات

```
1350000
```

نتيجة التنفيذ

```
Tax = 301800.00 DA
Effective rate = 22.36 %
```

المخطط الإنسيابي (Organigramme)



تمرين 2.45 عدد الأيام بين تاريخين

تجارة إلكترونية صعب

لحساب مدة بقاء طلبية في المستودع، اكتب برنامجًا يقرأ تاريخين من **نفس السنة** ويحسب عدد الأيام بينهما. نحسب رقم كل تاريخ في السنة باستعمال `switch` دون `break` لتجميع أيام الأشهر السابقة.

خطوات الحل

1) رقم اليوم في السنة: `n = d +`.
2) `switch (m)` يبدأ من الشهر السابق وينزل (fall-through) جامعًا أيام كل شهر.
3) نضيف يومًا إذا كانت السنة كبيسة والشهر بعد فيفري.
4) الفرق `n2 - n1`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int year, d1, m1, d2, m2, n1, n2, leap, diff;
    scanf("%d %d %d %d %d", &year, &d1, &m1, &d2, &m2);
    leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;

    n1 = d1;
    switch (m1 - 1) {          // دون break: نجمع كل الأشهر السابقة
        case 11: n1 += 30;     // نوفمبر
        // fall through
        case 10: n1 += 31;
        // fall through
        case 9:  n1 += 30;
        // fall through
        case 8:  n1 += 31;
        // fall through
        case 7:  n1 += 31;
        // fall through
        case 6:  n1 += 30;
        // fall through
        case 5:  n1 += 31;
        // fall through
        case 4:  n1 += 30;
        // fall through
        case 3:  n1 += 31;
        // fall through
        case 2:  n1 += 28 + leap;   // فيفري
        // fall through
        case 1:  n1 += 31;          // جانفي
    }
    n2 = d2;
    switch (m2 - 1) {          // نفس الطريقة للتاريخ الثاني
        case 11: n2 += 30;
        // fall through
        case 10: n2 += 31;
        // fall through
        case 9:  n2 += 30;
        // fall through
        case 8:  n2 += 31;
        // fall through
        case 7:  n2 += 31;
        // fall through
        case 6:  n2 += 30;
        // fall through
        case 5:  n2 += 31;
        // fall through
```

```c
        case 4:  n2 += 30;
        // fall through
        case 3:  n2 += 31;
        // fall through
        case 2:  n2 += 28 + leap;
        // fall through
        case 1:  n2 += 31;
    }
    diff = n2 - n1;
    if (diff < 0)
        diff = -diff;
    printf("Day numbers: %d and %d\n", n1, n2);
    printf("Days between = %d\n", diff);
    return 0;
}
```

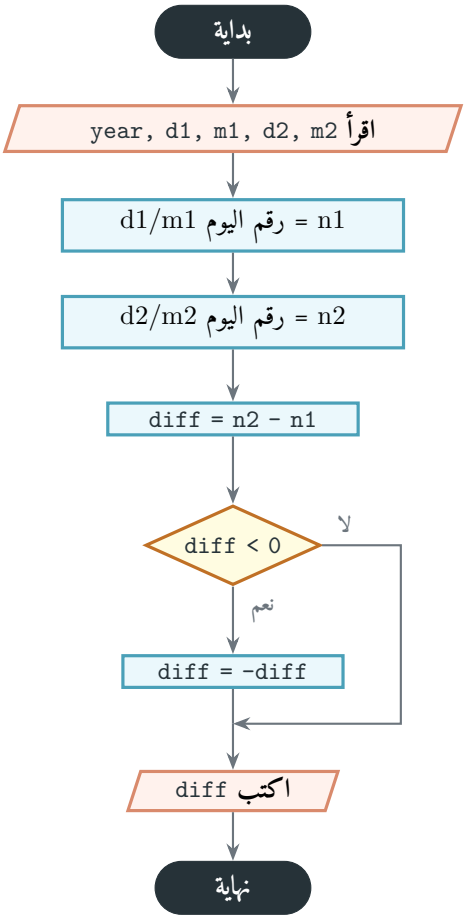
المدخلات

```
2026
15 1
3 4
```

نتيجة التنفيذ

```
Day numbers: 15 and 93
Days between = 78
```

المخطط الإنسيابي (Organigramme)



تمرين 2.46 اختيار شركة الشحن الأرخص

تجارة إلكترونية صعب

ثلاث شركات شحن بتسعيرات مختلفة لطرد وزنه weight كغ ومسافته km: الشركة 1: 300 + 25*weight + 2*km؛ الشركة 2: 500 + 15*weight ولا تقبل أكثر من 30 كغ؛ الشركة 3: 0.9 * (200 + 40*weight) إذا كانت المسافة أقل من 100 كم، وإلا 200 + 40*weight + 3*km. اكتب برنامجًا يحسب التكاليف ويختار الأرخص بين الشركات المتاحة.

خطوات الحل

1) نحسب c1.
2) c2 فقط إذا كان weight <= 30، وإلا نعتبرها غير متاحة (ok2 = 0).
3) c3 حسب المسافة.
4) نختار الأرخص مع تجاهل الشركة غير المتاحة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double weight, km, c1, c2, c3, min;
    int ok2, best;
    scanf("%lf %lf", &weight, &km);
    c1 = 300 + 25 * weight + 2 * km;
    ok2 = (weight <= 30);                  // الشركة 2 لها حد للوزن
    c2 = 500 + 15 * weight;
    if (km < 100)
        c3 = 0.9 * (200 + 40 * weight);    // تخفيض للمسافات القصيرة
    else
        c3 = 200 + 40 * weight + 3 * km;
    printf("Carrier 1: %.2f DA\n", c1);
    if (ok2) printf("Carrier 2: %.2f DA\n", c2);
    else     printf("Carrier 2: not available\n");
    printf("Carrier 3: %.2f DA\n", c3);
    best = 1;  min = c1;                   // نبدأ بالشركة 1
    if (ok2 && c2 < min) { best = 2; min = c2; }
    if (c3 < min)        { best = 3; min = c3; }
    printf("Cheapest: carrier %d (%.2f DA)\n", best, min);
    return 0;
}
```

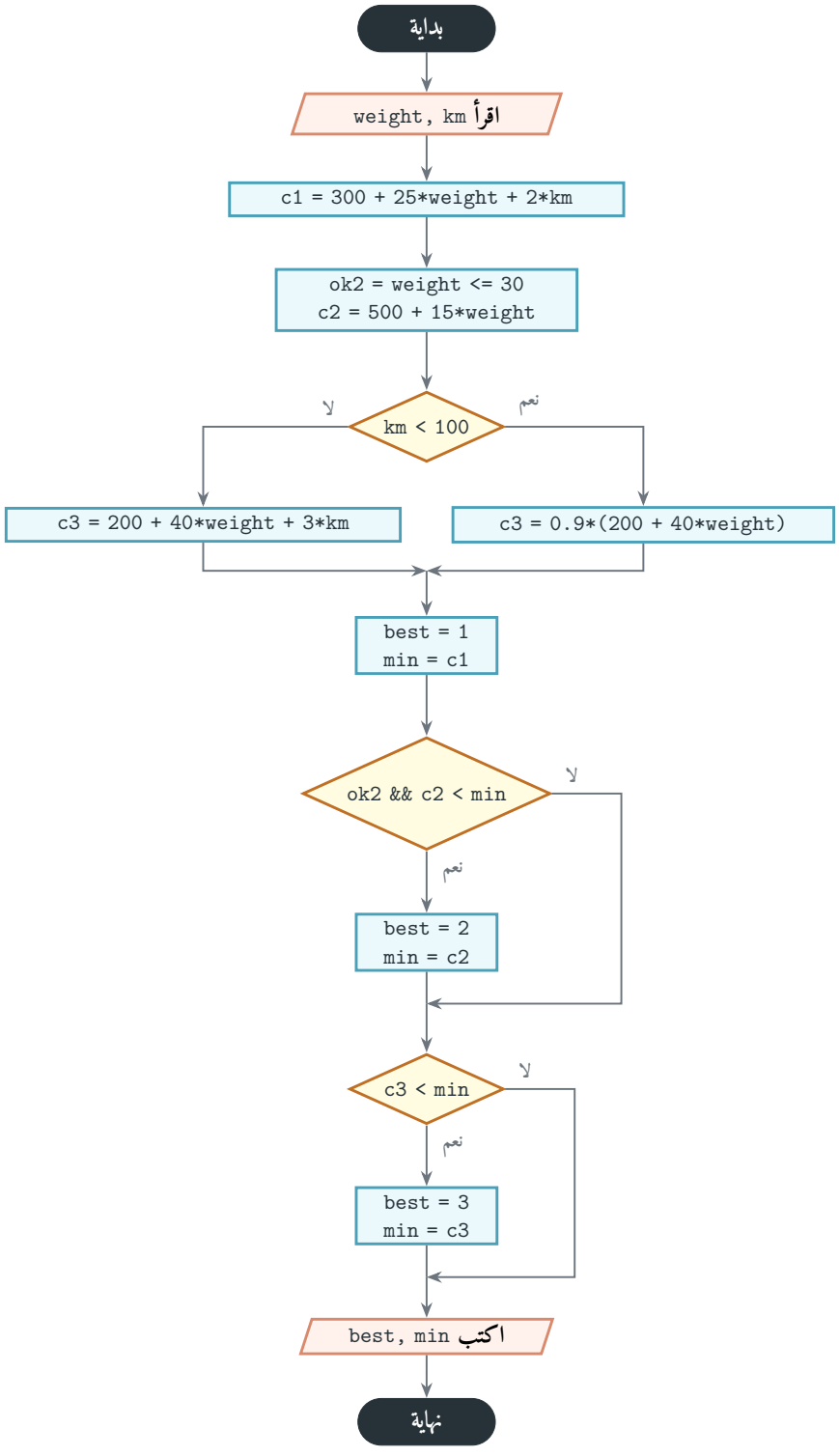
المدخلات

```
12 350
```

نتيجة التنفيذ

```
Carrier 1: 1300.00 DA
Carrier 2: 680.00 DA
Carrier 3: 1730.00 DA
Cheapest: carrier 2 (680.00 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 2.47 هل العنوانُ داخل منطقة التوصيل؟ تجارة إلكترونية صعب

يخدم مستودع منطقتين: **دائرة** مركزها المستودع `(cx, cy)` ونصف قطرها `radius` كم، و**مستطيل** صناعي `[x1, x2] × [y1, y2]`. اكتب برنامجًا يقرأ إحداثيات الزبون ويحدد المنطقة التي يقع فيها (أو كلتيهما أو لا شيء)، دون استعمال الجذر التربيعي.

خطوات الحل

1) داخل الدائرة: `(custX-cx)² + (custY-cy)² <= radius²` (نقارن المربعات لتجنب الجذر).
2) داخل المستطيل: `x1 <= custX <= x2` و`y1 <= custY <= y2`.
3) نجمع النتيجتين لتحديد الحالات الأربع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double cx, cy, radius, x1, x2, y1, y2, custX, custY, d2;
    int inC, inR;
    scanf("%lf %lf %lf", &cx, &cy, &radius);           // الدائرة
    scanf("%lf %lf %lf %lf", &x1, &x2, &y1, &y2); // المستطيل
    scanf("%lf %lf", &custX, &custY);                    // الزبون
    d2 = (custX - cx) * (custX - cx) + (custY - cy) * (custY - cy);
    inC = (d2 <= radius * radius);              // مقارنة المربعات بدل الجذر
    inR = (custX >= x1 && custX <= x2 && custY >= y1 && custY <= y2);
    printf("Squared distance = %.1f (limit %.1f)\n", d2, radius * radius);
    if (inC && inR)
        printf("Inside both zones\n");
    else if (inC)
        printf("Inside the circle only\n");
    else if (inR)
        printf("Inside the industrial rectangle only\n");
    else
        printf("Out of delivery zone\n");
    return 0;
}
```

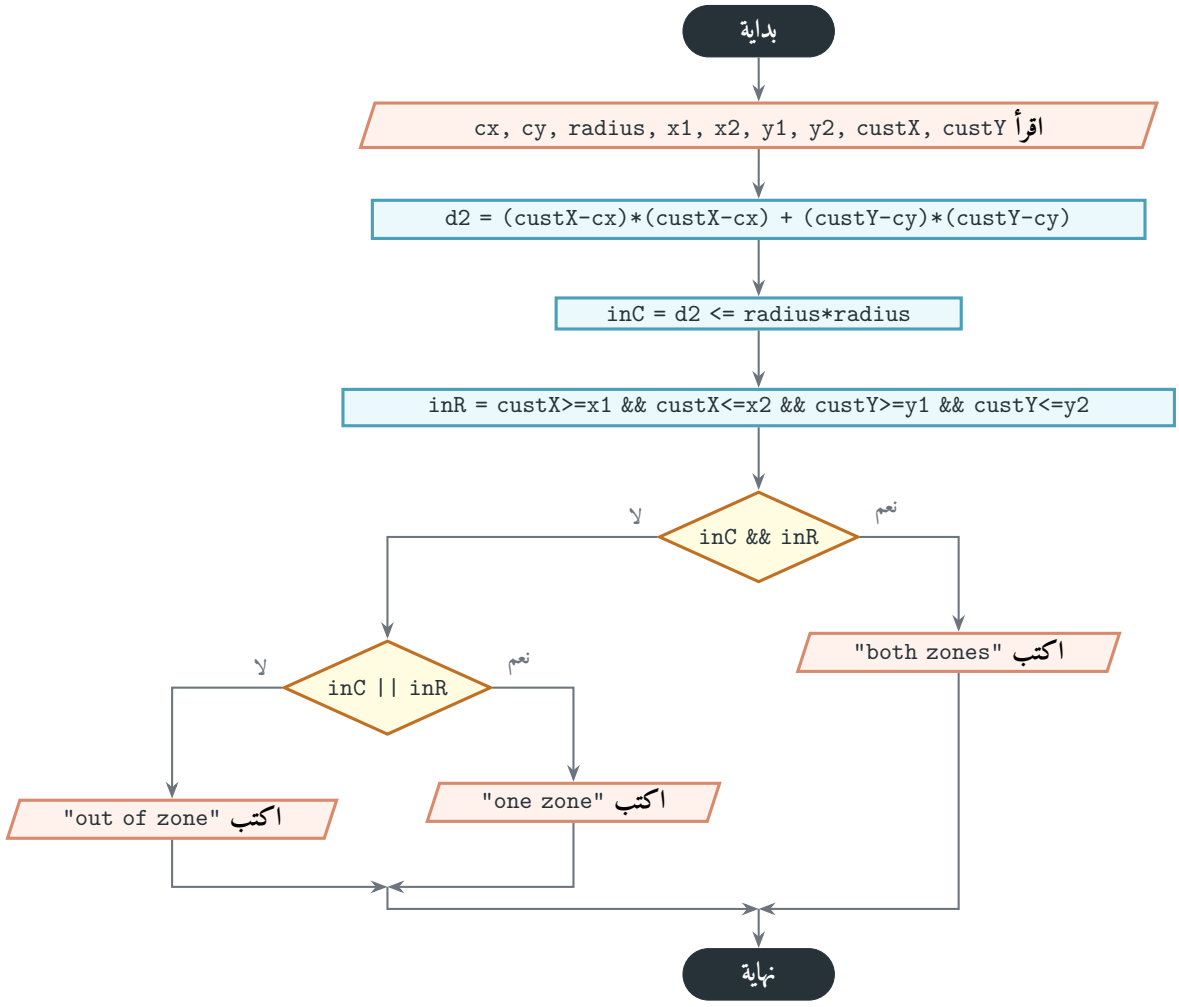
المدخلات

```
0 0 10
12 20 -3 4
13 2
```

نتيجة التنفيذ

```
Squared distance = 173.0 (limit 100.0)
Inside the industrial rectangle only
```

المخطط الإنسيابي (Organigramme)



تمرين 2.48 أقرب جار لتصنيف زبون جديد

علم البيانات صعب ★★★

لدينا أربعة زبائن معروفين، لكل منهم (عدد الزيارات، متوسط السلة بالآلاف) وصنف: الأول والثاني "مقتصد" (0)، الثالث والرابع "مبذر" (1). البيانات ثابتة في البرنامج: `(2,1.5)`، `(4,2)`، `(9,7.5)`، `(11,6)`. اكتب برنامجًا يقرأ زبونًا جديدًا ويصنفه بطريقة **أقرب جار** (1-NN) باستعمال مربع المسافة.

خطوات الحل

1) نحسب مربع المسافة إلى كل زبون من الأربعة.
2) نحتفظ بأصغر مسافة `best` وصنفها `cls`، ونحدّثهما عند كل مسافة أصغر.
3) نعرض الصنف المتنبأ به.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double visits, basket, d1, d2, d3, d4, best;
```

```c
    int cls, who;
    scanf("%lf %lf", &visits, &basket);
    // مربع المسافة إلى كل زبون معروف (بيانات التدريب)
    d1 = (visits - 2) * (visits - 2) + (basket - 1.5) * (basket - 1.5);
    d2 = (visits - 4) * (visits - 4) + (basket - 2) * (basket - 2);
    d3 = (visits - 9) * (visits - 9) + (basket - 7.5) * (basket - 7.5);
    d4 = (visits - 11) * (visits - 11) + (basket - 6) * (basket - 6);
    printf("d2: %.2f %.2f %.2f %.2f\n", d1, d2, d3, d4);
    best = d1; cls = 0; who = 1;          // نبدأ بالجار الأول
    if (d2 < best) { best = d2; cls = 0; who = 2; }
    if (d3 < best) { best = d3; cls = 1; who = 3; }
    if (d4 < best) { best = d4; cls = 1; who = 4; }
    printf("Nearest neighbour: customer %d\n", who);
    if (cls == 0) printf("Predicted class: thrifty\n");
    else          printf("Predicted class: big spender\n");
    return 0;
}
```

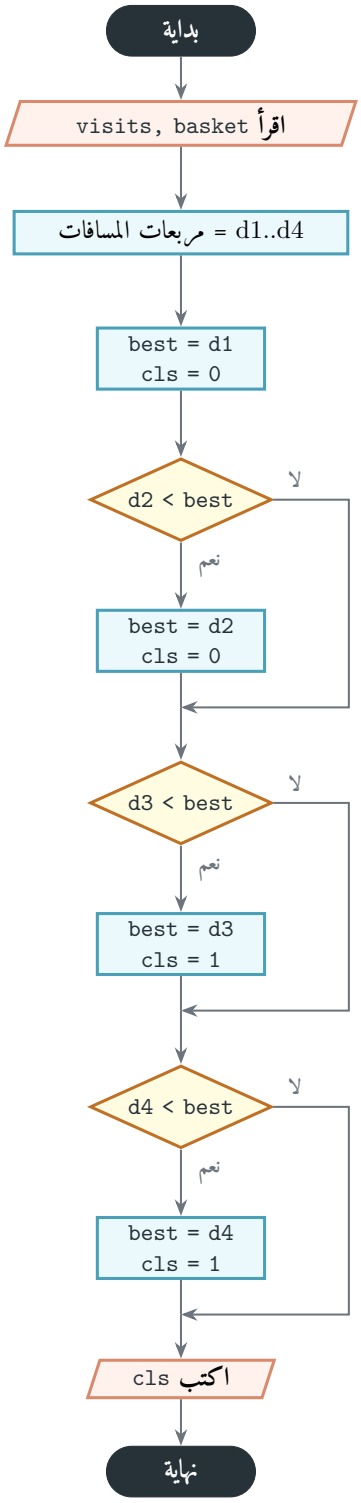
المدخلات

```
7 5
```

نتيجة التنفيذ

```
d2: 37.25 18.00 10.25 17.00
Nearest neighbour: customer 3
Predicted class: big spender
```

المخطط الإنسيابي (Organigramme)



تمرين 2.49 محرك التسعير الكامل

تجارة إلكترونية

صعب ★★★

اكتب برنامج **محرك تسعير** يقرأ: مبلغ السلة، مستوى الوفاء (0 عادي، 1 فضي، 2 ذهبي)، رمز قسيمة (0 لا شيء، 1 = -10%، 2 = -1000 دج إذا تجاوزت السلة 8000)، والولاية (16 أو غيرها). القواعد: الوفاء يعطي 0% أو 3% أو 7%؛ **مجموع التخفيضات لا يتجاوز 30%** من السلة؛ ثم الرسم 19%؛ ثم الشحن: مجاني للذهبي أو إذا تجاوز المبلغ 10000، وإلا 300 للولاية 16 و700

لغيرها.

خطوات الحل

1) نحسب تخفيض الوفاء dl بـ switch.
2) نحسب تخفيض القسيمة dc مع شرطها.
3) نحصر المجموع: إذا كان dl + dc > 0.3 * cart نجعله 0.3 * cart.
4) نطبق الرسم ثم نحدد الشحن.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double cart, dl, dc, disc, net, ship;
    int level, coupon, wilaya;
    scanf("%lf %d %d %d", &cart, &level, &coupon, &wilaya);
    switch (level) {                        // تخفيض الوفاء
        case 1:  dl = cart * 0.03; break; // فضي
        case 2:  dl = cart * 0.07; break; // ذهبي
        default: dl = 0;
    }
    dc = 0;                                 // تخفيض القسيمة
    if (coupon == 1)
        dc = cart * 0.10;
    else if (coupon == 2 && cart > 8000)
        dc = 1000;
    disc = dl + dc;
    if (disc > 0.3 * cart)                  // سقف التخفيضات 30%
        disc = 0.3 * cart;
    net = (cart - disc) * 1.19;             // الرسم بعد التخفيض
    if (level == 2 || net > 10000)
        ship = 0;
    else if (wilaya == 16)
        ship = 300;
    else
        ship = 700;
    printf("Discounts = %.2f DA\n", disc);
    printf("Net with tax = %.2f DA\n", net);
    printf("Shipping = %.2f DA\n", ship);
    printf("TOTAL = %.2f DA\n", net + ship);
    return 0;
}
```

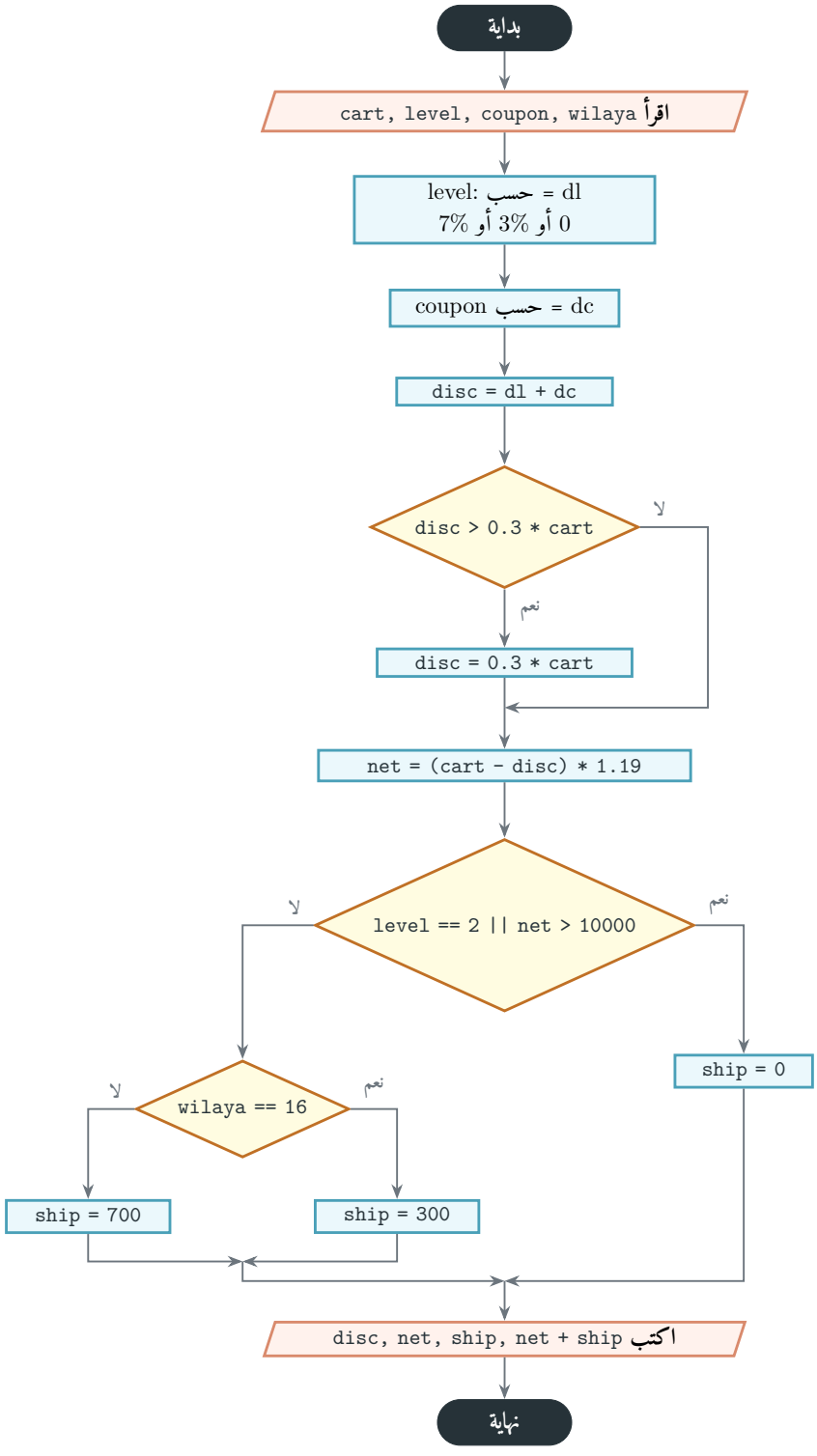
المدخلات

```
12500 2 1 31
```

نتيجة التنفيذ

```
Discounts = 2125.00 DA
Net with tax = 12346.25 DA
Shipping = 0.00 DA
TOTAL = 12346.25 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 2.50 توازن العرض والطلب

علم البيانات صعب ★★★

في سوق منتج: الطلب `Qd = demandBase - demandSlope*price` والعرض `Qs = supplyBase + supplySlope*price`. سعر التوازن يحقق `Qd = Qs` أي

```
price* = (demandBase - supplyBase) / (demandSlope + supplySlope)
```

. تفرض الدولة أحيانًا سعرًا أدنى `floor`. اكتب برنامجًا يحسب التوازن، وإذا كان السعر الأدنى أعلى من `price*` يُباع بالسعر

الأدنى ويظهر فائض Qs - Qd. تحقق أيضًا أن التوازن ممكن (price* > 0 و Q* > 0).

خطوات الحل

1) $$price = (demandBase - supplyBase) / (demandSlope + supplySlope)$$

و

$$quantity = demandBase - demandSlope * price$$

.

2) إذا كان price <= 0 أو quantity <= 0: لا يوجد توازن ذو معنى.
3) إذا كان floor > price: نحسب Qd وQs عند floor والفائض.
4) وإلا السعر الأدنى غير مؤثر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double demandBase, demandSlope, supplyBase, supplySlope, floor_p, price, quantity, qd, qs;
    scanf("%lf %lf %lf %lf", &demandBase, &demandSlope, &supplyBase, &supplySlope);
    scanf("%lf", &floor_p);                 // السعر الأدنى المفروض
    price = (demandBase - supplyBase) / (demandSlope + supplySlope);                //
      supplySlope*price + supplyBase = demandSlope*price - demandBase من
    quantity = demandBase - demandSlope * price;
    if (price <= 0 || quantity <= 0) {
        printf("No meaningful equilibrium\n");
    } else {
        printf("Equilibrium: p* = %.2f DA, q* = %.2f units\n", price, quantity);
        if (floor_p > price) {                  // السعر الأدنى مؤثر
            qd = demandBase - demandSlope * floor_p;          // الطلب ينخفض
            qs = supplyBase + supplySlope * floor_p;          // العرض يرتفع
            printf("At floor %.2f: demand %.2f, supply %.2f\n", floor_p, qd, qs);
            printf("Surplus = %.2f units\n", qs - qd);
        } else {
            printf("Price floor not binding\n");
        }
    }
    return 0;
}
```

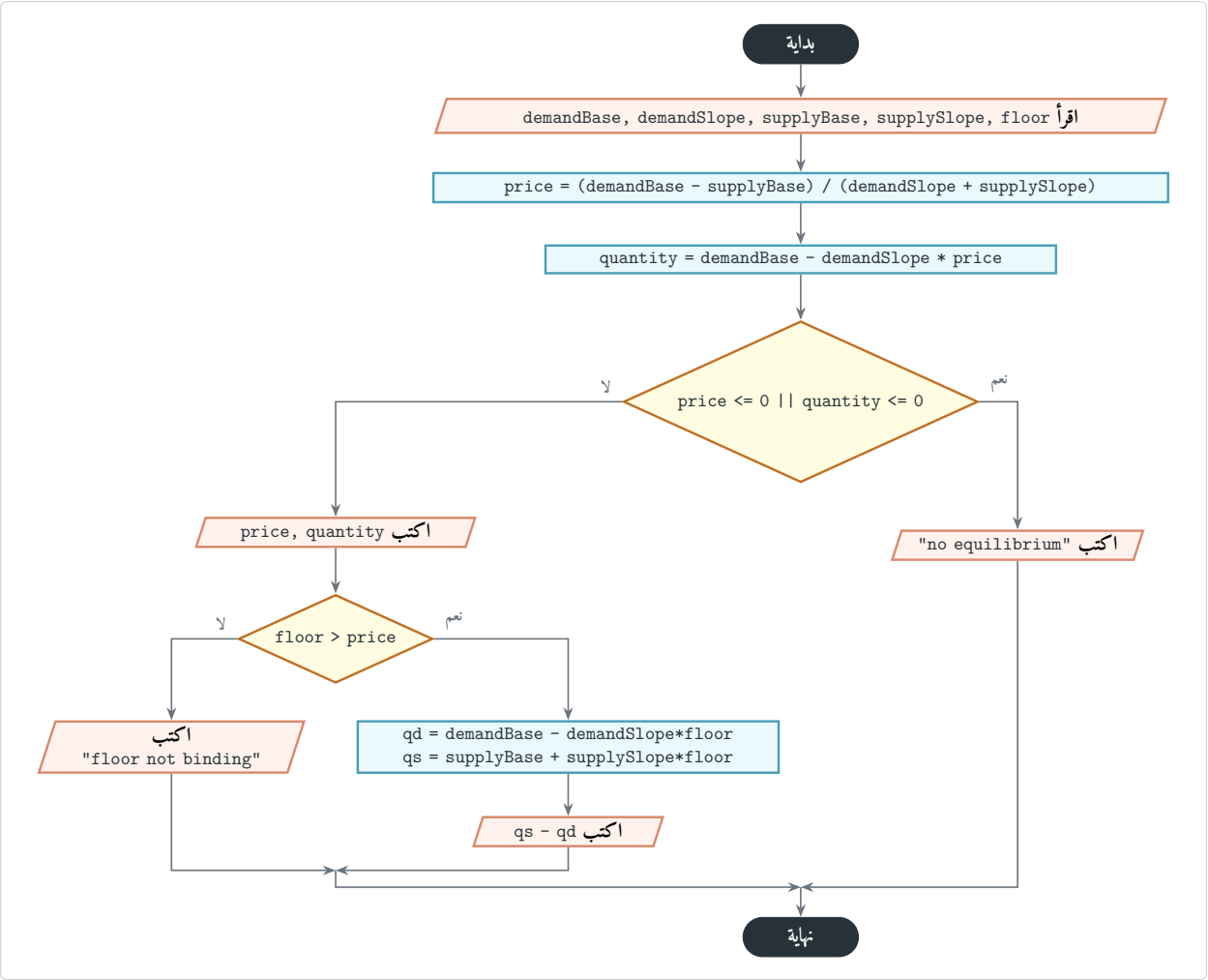
المدخلات

```
1000 4 100 2
180
```

نتيجة التنفيذ

```
Equilibrium: p* = 150.00 DA, q* = 400.00 units
At floor 180.00: demand 280.00, supply 460.00
Surplus = 180.00 units
```

المخطط الإنسيابي (Organigramme)

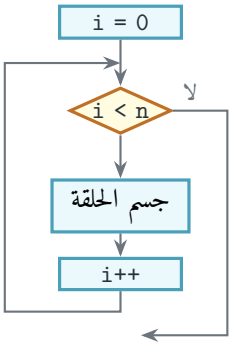


الفصل 3

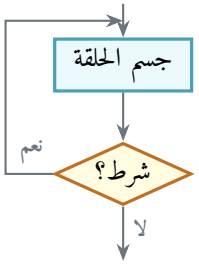
الحلقات التكرارية

تعريفات

- **الحلقة التكرارية**: تعيد تنفيذ مجموعة تعليمات ما دام شرط ما محققًا.
- `for`: عندما يكون **عدد التكرارات معروفًا** مسبقًا (تهيئة، شرط، تحديث).
- `while`: الشرط يُختبر **قبل** كل دورة، فقد لا تُنفذ الحلقة أبدًا.
- `do ... while`: الشرط يُختبر **بعد** الدورة، فتُنفذ **مرة واحدة على الأقل**.
- **المجمِّع والعداد**: متغير يُهيأ قبل الحلقة (`sum = 0`، `count = 0`) ويُحدَّث داخلها.



`for / while`



`do ... while`

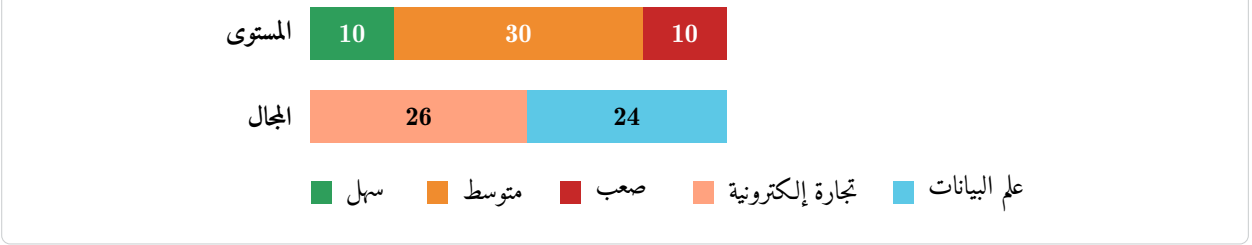
الحلقة	متى نستعملها؟	مثال من التمارين
`for`	عدد معروف من التكرارات	المرور على n سعراً
`while`	التوقف عند تحقق شرط	الادخار حتى بلوغ السعر
`do while`	تنفيذ مرة على الأقل	إعادة طلب إدخال صالح

الحلقة اللانهائية

إذا لم يتغير متغير الشرط داخل الحلقة فلن تتوقف أبدًا. تأكد دائمًا أن كل دورة **تقرّب** الحلقة من نهايتها.

للاستزادة: [1, 2, 6]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 3.1 مجموع أسعار السلة

سهل ⭐ تجارة إلكترونية

اكتب برنامجًا يقرأ عدد المنتجات `nbItems` ثم أسعارها، ويعرض المجموع باستعمال حلقة `for`.

خطوات الحل

1) نقرأ `nbItems` ونهيئ `sum = 0`.
2) حلقة `for` من `item = 1` إلى `nbItems`: نقرأ سعرًا ونضيفه.
3) نعرض المجموع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbItems, item;
    double price, sum = 0;          // المجمّع يبدأ من الصفر
    scanf("%d", &nbItems);
    for (item = 1; item <= nbItems; item++) {   // nbItems تكرارًا بالضبط
        scanf("%lf", &price);
        sum = sum + price;          // نضيف السعر الحالي
    }
    printf("Basket total = %.2f DA\n", sum);
    return 0;
}
```

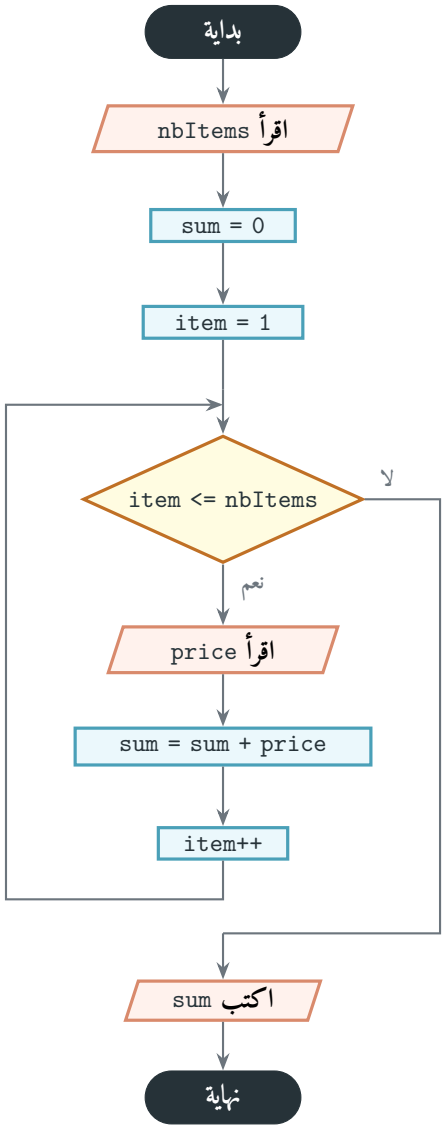
المدخلات

```
5
1200 450 3999 89.5 700
```

نتيجة التنفيذ

```
Basket total = 6438.50 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.2 عدد الطلبيات الكبيرة

علم البيانات

سهل

اكتب برنامجًا يقرأ `nbOrders` مبالغ طلبيات ويعد كم منها يتجاوز **10000 دج**، ثم يعرض نسبتها.

خطوات الحل

1) عداد `count = 0`.
2) لكل مبلغ: إذا كان `10000 <` نزيد العداد.
3) النسبة: `count * 100.0 / nbOrders`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbOrders, index, count = 0;
    double amount;
```

```c
    scanf("%d", &nbOrders);
    for (index = 0; index < nbOrders; index++) {
        scanf("%lf", &amount);
        if (amount > 10000)
            count++;              // طلبية كبيرة: نزيد العداد
    }
    printf("Large orders: %d / %d (%.1f %%)\n", count, nbOrders, count * 100.0 / nbOrders);
    return 0;
}
```

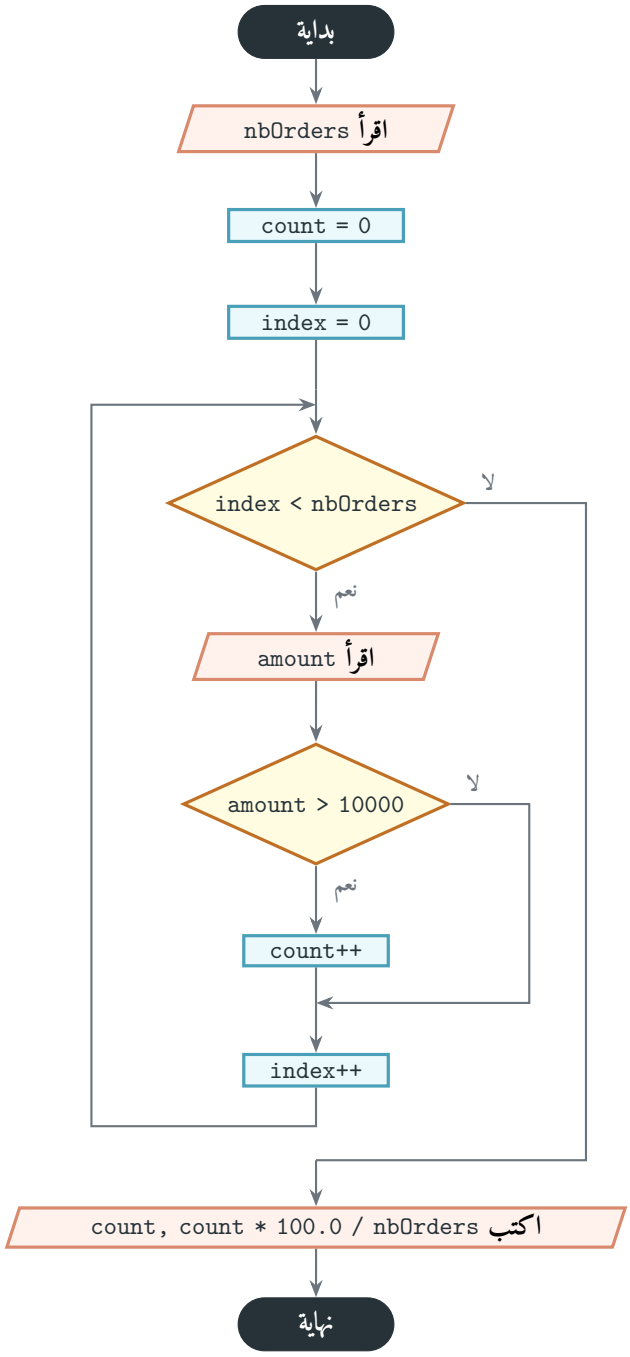
المدخلات

```
8
2500 12000 800 15600 9999 10001 4300 22000
```

نتيجة التنفيذ

```
Large orders: 4 / 8 (50.0 %)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.3 متوسط التقييمات

علم البيانات

سهل

اكتب برنامجًا يقرأ عدد التقييمات `nbReviews` ثم التقييمات (أعداد صحيحة من 1 إلى 5) ويحسب **المتوسط**.

خطوات الحل

1) نجمع التقييمات في `sum`.
2) المتوسط `(double)sum / nbReviews`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbReviews, index, rating, sum = 0;
    scanf("%d", &nbReviews);
    for (index = 0; index < nbReviews; index++) {
        scanf("%d", &rating);
        sum += rating;                  // rating + sum = sum تعني rating =+ sum
    }
    printf("Average = %.2f\n", (double)sum / nbReviews);
    return 0;
}
```

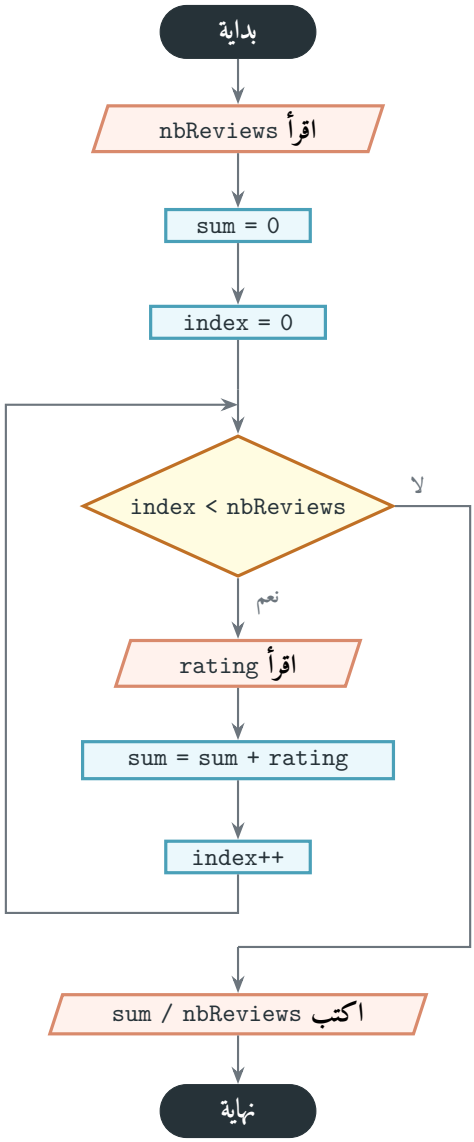
المدخلات

```
7
5 4 4 3 5 2 4
```

نتيجة التنفيذ

```
Average = 3.86
```

المخطط الإنسيابي (Organigramme)



تمرين 3.4 جدول الأسعار حسب الكمية

تجارة إلكترونية سهل

يعرض المتجر جدولًا يبين ثمن 1 إلى 10 وحدات من منتج سعره مقروء. اكتب البرنامج.

خطوات الحل

1) حلقة `for` على الكمية `qty` من 1 إلى 10.
2) نعرض `qty` و`qty * price` في كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int qty;
    double price;
```

```c
    scanf("%lf", &price);
    printf("Qty | Price\n");
    for (qty = 1; qty <= 10; qty++)                    // سطر لكل كمية
        printf("%3d | %8.2f\n", qty, qty * price);
    return 0;
}
```

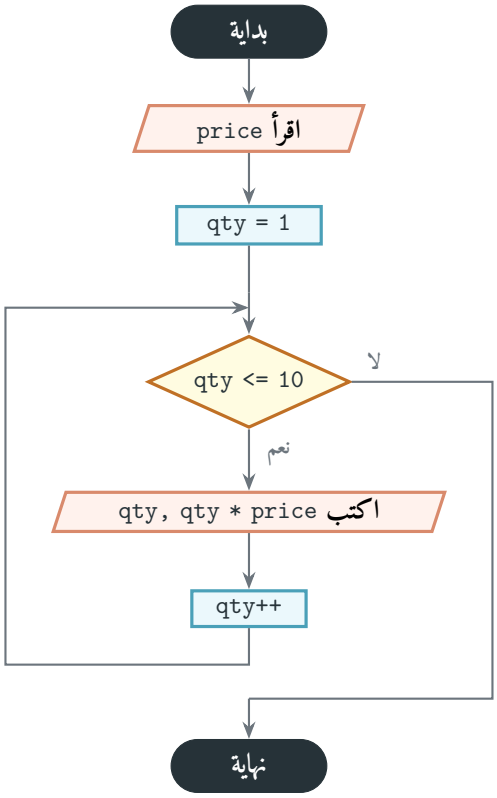
المدخلات

```
275
```

نتيجة التنفيذ

```
Qty | Price
  1 |   275.00
  2 |   550.00
  3 |   825.00
  4 |  1100.00
  5 |  1375.00
  6 |  1650.00
  7 |  1925.00
  8 |  2200.00
  9 |  2475.00
 10 |  2750.00
```

المخطط الإنسيابي (Organigramme)



تمرين 3.5 العد التنازلي للتخفيضات

تجارة إلكترونية ⭐ سهل

تبدأ تخفيضات "الجمعة السوداء" بعد hours ساعة. اكتب برنامجًا يعرض **عدًا تنازليًا** بخطوة 6 ساعات باستعمال حلقة while حتى يصل إلى 0 أو أقل.

خطوات الحل

1) ما دام hours > 0 نعرض hours ثم نطرح 6.
2) عند الخروج من الحلقة نعرض رسالة البداية.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int hours;
    scanf("%d", &hours);
    while (hours > 0) {                    // الشرط يُختبر قبل كل دورة
        printf("%d hours left\n", hours);
        hours = hours - 6;
    }
    printf("Black Friday sale started!\n");
    return 0;
}
```

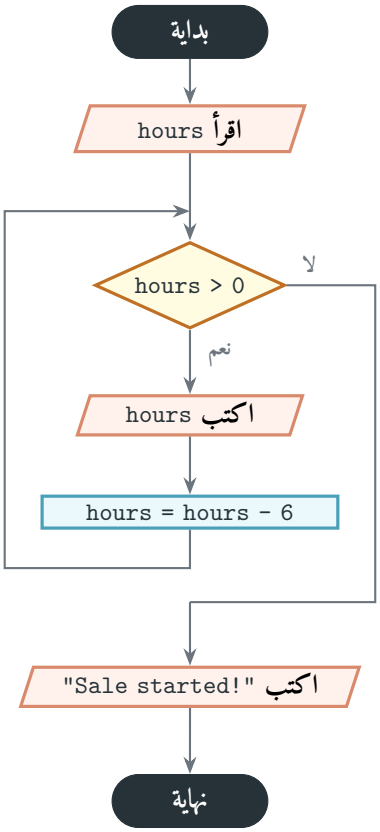
المدخلات

```
27
```

نتيجة التنفيذ

```
27 hours left
21 hours left
15 hours left
9 hours left
3 hours left
Black Friday sale started!
```

المخطط الإنسيابي (Organigramme)



تمرين 3.6 الإدخال حتى الصفر

سهل تجارة إلكترونية

يُدخل أمين الصندوق أسعار المنتجات واحدًا واحدًا ولا يعرف عددها مسبقًا، ويُنهي بإدخال **0**. اكتب برنامجًا يحسب المجموع وعدد المنتجات.

خطوات الحل

1) نقرأ السعر الأول قبل الحلقة.
2) ما دام `price != 0`: نضيفه ونزيد العداد ثم نقرأ السعر التالي.
3) القيمة 0 **قيمة حارسة** تنهي الإدخال.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, sum = 0;
    int nbItems = 0;
    scanf("%lf", &price);            // القراءة الأولى قبل الحلقة
    while (price != 0) {             // 0 قيمة حارسة: نهاية الإدخال
        sum += price;
        nbItems++;
        scanf("%lf", &price);        // نقرأ السعر التالي
    }
```

```
12      printf("%d items, total = %.2f DA\n", nbItems, sum);
13      return 0;
14  }
```

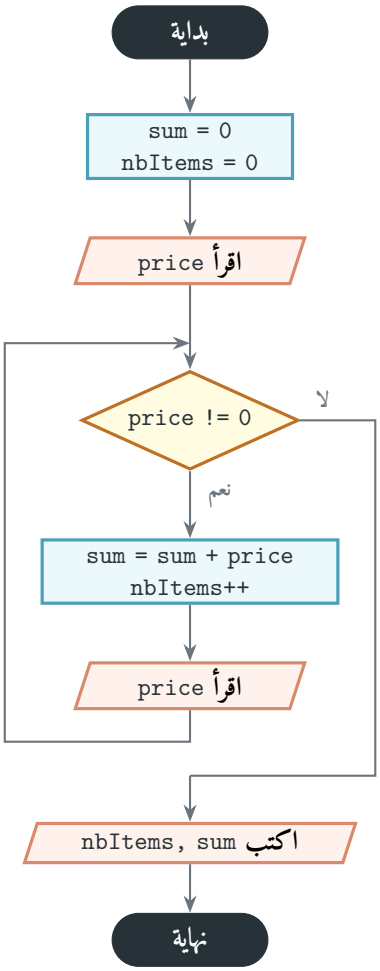
المدخلات

```
350 1200 75 990 0
```

نتيجة التنفيذ

```
4 items, total = 2615.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.7 أغلى منتج

تجارة إلكترونية

سهل

اكتب برنامجًا يقرأ nbPrices أسعار ويعرض أكبرها ورقم ترتيبه.

خطوات الحل

1) نقرأ السعر الأول ونعتبره الأكبر: max = price و pos = 1.
2) لكل سعر موالٍ: إذا كان أكبر من max نحدّث max وpos.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPrices, index, pos;
    double price, max;
    scanf("%d %lf", &nbPrices, &price);
    max = price;                        // أول سعر هو الأكبر مبدئيًا
    pos = 1;
    for (index = 2; index <= nbPrices; index++) {
        scanf("%lf", &price);
        if (price > max) {              // وجدنا سعرًا أكبر
            max = price;
            pos = index;
        }
    }
    printf("Most expensive: %.2f DA (item #%d)\n", max, pos);
    return 0;
}
```

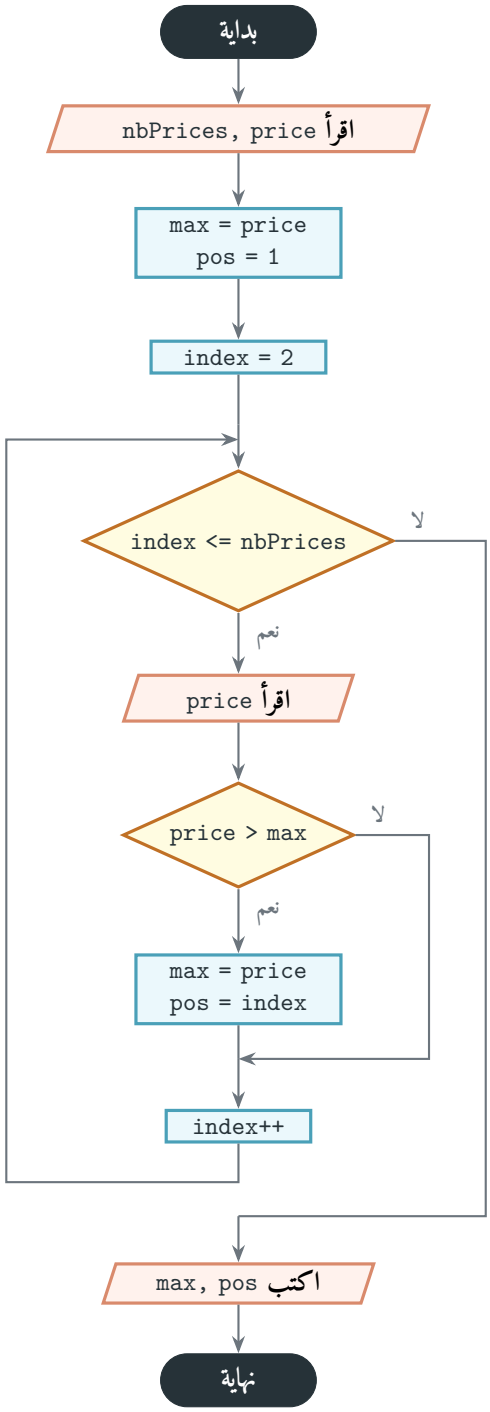
المدخلات

```
6
450 1290 300 1290 870 99
```

نتيجة التنفيذ

```
Most expensive: 1290.00 DA (item #2)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.8 أرقام الطلبيات الزوجية

تجارة إلكترونية سهل

اكتب برنامجًا يعرض كل أرقام الطلبيات **الزوجية** بين `first` و`last` (ضمنًا) ويعد عددها.

خطوات الحل

1) حلقة من `first` إلى `last`.
2) نعرض الرقم إذا كان `orderId % 2 == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int first, last, orderId, count = 0;
    scanf("%d %d", &first, &last);
    for (orderId = first; orderId <= last; orderId++) {
        if (orderId % 2 == 0) {         // رقم زوجي
            printf("%d ", orderId);
            count++;
        }
    }
    printf("\nEven order numbers: %d\n", count);
    return 0;
}
```

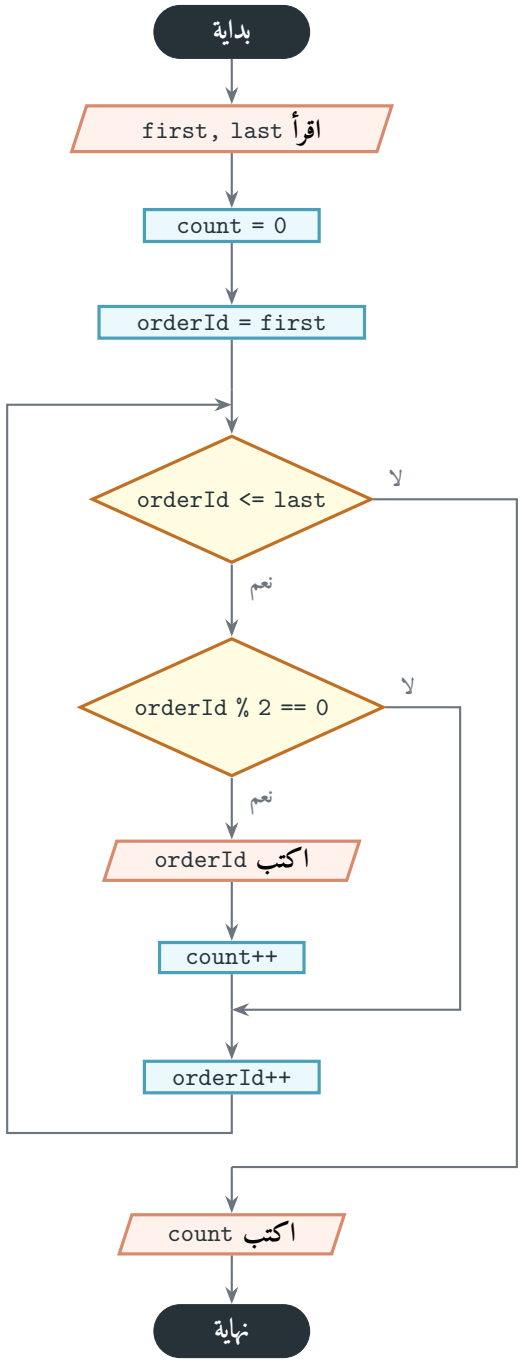
المدخلات

```
1001 1015
```

نتيجة التنفيذ

```
1002 1004 1006 1008 1010 1012 1014
Even order numbers: 7
```

المخطط الإنسيابي (Organigramme)



تمرين 3.9 الإدخار لشراء حاسوب

تجارة إلكترونية سهل

يريد طالب شراء حاسوب سعره `price`. يدّخر كل شهر مبلغًا `monthly`. اكتب برنامجًا يحسب **عدد الأشهر** اللازمة باستعمال `while`.

خطوات الحل

1) `saved = 0` و`months = 0`.
2) ما دام `saved < price`: نضيف `monthly` ونزيد الأشهر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double price, monthly, saved = 0;
    int months = 0;
    scanf("%lf %lf", &price, &monthly);
    while (saved < price) {        // لا نعرف عدد التكرارات مسبقًا
        saved += monthly;
        months++;
    }
    printf("Months needed: %d (saved %.2f DA)\n", months, saved);
    return 0;
}
```

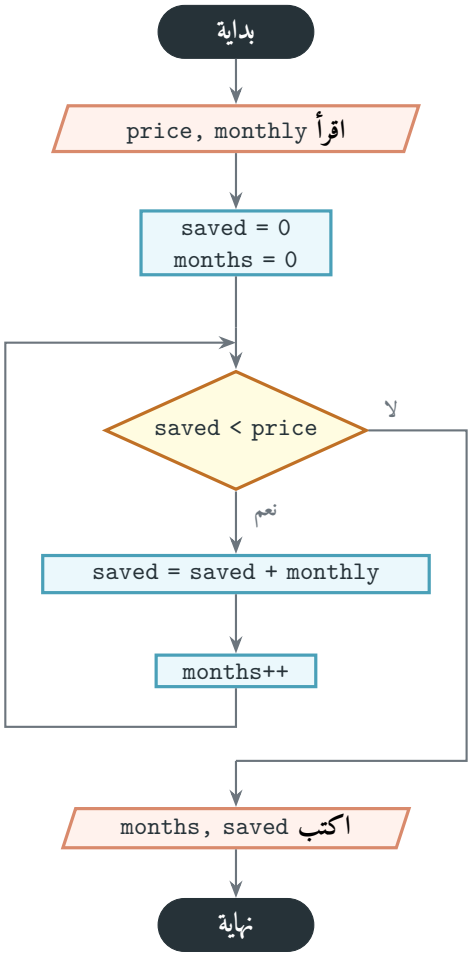
المدخلات

```
95000 12500
```

نتيجة التنفيذ

```
Months needed: 8 (saved 100000.00 DA)
```

المخطط الانسيابي (Organigramme)



تمرين 3.10 إعادة طلب تقييم صالح

علم البيانات سهل

اكتب برنامجًا يطلب من الزبون تقييمًا بين 1 و5، ويعيد الطلب ما دام الإدخال غير صالح، باستعمال `do ... while`.

خطوات الحل

1) `do`: نقرأ التقييم.
2) `while (rating < 1 || rating > 5)`: نعيد إذا كان خارج المجال.
3) الحلقة تنفذ مرة واحدة على الأقل.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int rating, tries = 0;
    do {
        printf("Rating (1-5): ");
        scanf("%d", &rating);
        tries++;
        if (rating < 1 || rating > 5)
            printf("invalid!\n");
    } while (rating < 1 || rating > 5);      // نعيد ما دام الإدخال خاطئًا
    printf("Accepted rating %d after %d tries\n", rating, tries);
    return 0;
}
```

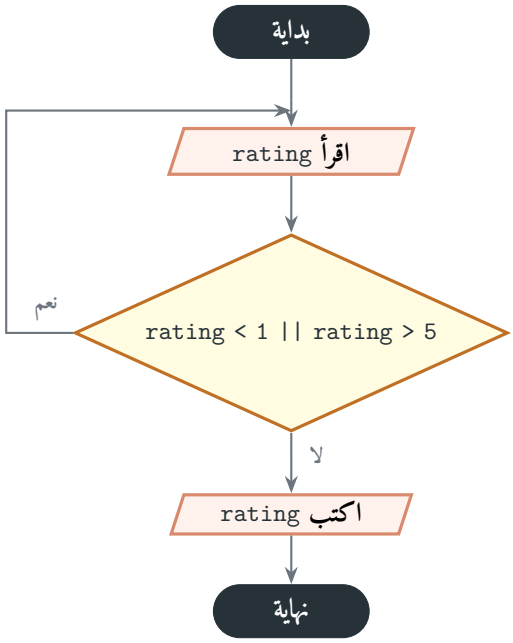
المدخلات

```
7 0 4
```

نتيجة التنفيذ

```
Rating (1-5): invalid!
Rating (1-5): invalid!
Rating (1-5): Accepted rating 4 after 3 tries
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 3.11 الأصغر والأكبر والمدى

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ أوقات تحميل `nbPages` صفحات (بالملي ثانية) ويعرض **الأصغر** و**الأكبر** و**المدى** (الفرق بينهما) والمتوسط، في مرور واحد على البيانات.

خطوات الحل

1) القيمة الأولى تهيّئ `min` و`max` و`sum`.
2) في كل تكرار نحدّث الثلاثة.
3) المدى: `max - min`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPages, index;
    double loadTime, min, max, sum;
    scanf("%d %lf", &nbPages, &loadTime);
    min = max = sum = loadTime;          // التهيئة بالقيمة الأولى
    for (index = 2; index <= nbPages; index++) {
        scanf("%lf", &loadTime);
        sum += loadTime;
        if (loadTime < min) min = loadTime;     // تحديث الأصغر
        if (loadTime > max) max = loadTime;     // تحديث الأكبر
    }
    printf("Min = %.0f ms, Max = %.0f ms\n", min, max);
    printf("Range = %.0f ms, Mean = %.1f ms\n", max - min, sum / nbPages);
    return 0;
}
```

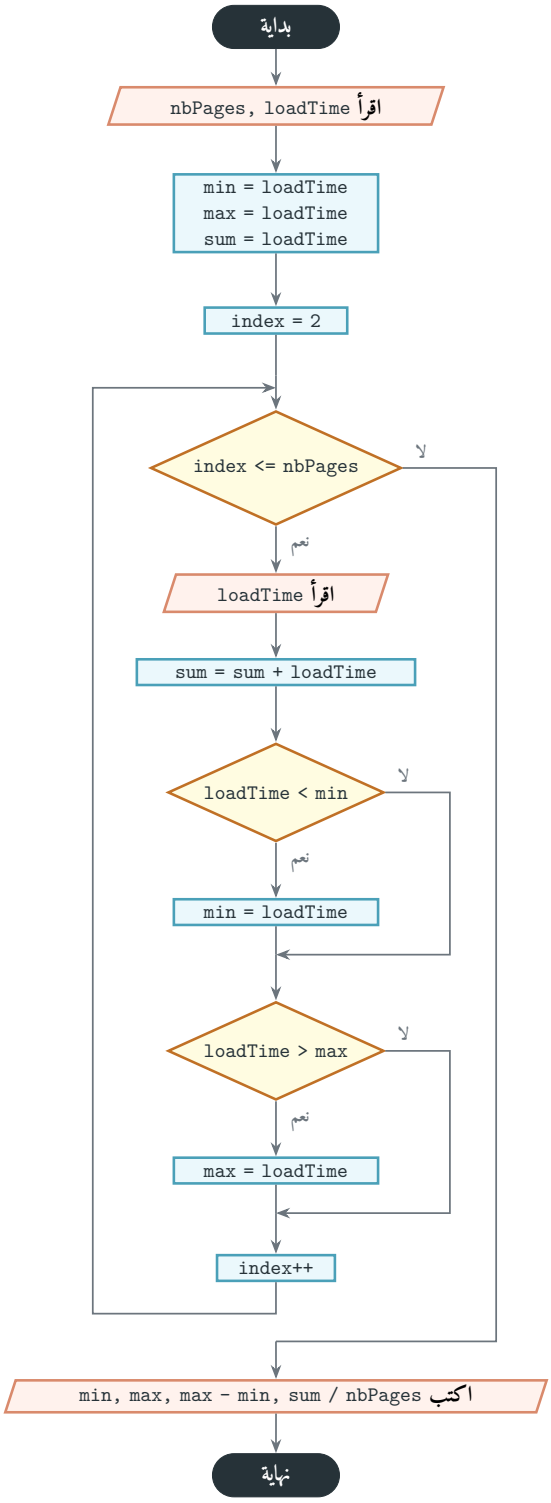
المدخلات

```
7
320 280 910 450 300 275 610
```

نتيجة التنفيذ

```
Min = 275 ms, Max = 910 ms
Range = 635 ms, Mean = 449.3 ms
```

المخطط الإنسيابي (Organigramme)



تمرين 3.12 المتوسط والتباين في مرور واحد

علم البيانات ★★ متوسط

لحساب التباين دون تخزين البيانات نستعمل الصيغة: $var = (\Sigma sales^2)/nbDays - mean^2$. اكتب برنامجًا يقرأ nbDays قيم مبيعات يومية ويحسب المتوسط والتباين في مرور واحد.

خطوات الحل

1) نجمع `sum = Σsales` و`s2 = Σsales²` في نفس الحلقة.
2) `mean = sum / nbDays`.
3) `var = s2 / nbDays - mean * mean`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day;
    double sales, sum = 0, s2 = 0, mean, var;
    scanf("%d", &nbDays);
    for (day = 0; day < nbDays; day++) {
        scanf("%lf", &sales);
        sum += sales;                   // مجموع القيم
        s2 += sales * sales;              // مجموع المربعات
    }
    mean = sum / nbDays;
    var = s2 / nbDays - mean * mean;    // متوسط المربعات ناقص مربع المتوسط
    printf("Mean = %.2f\n", mean);
    printf("Variance = %.2f\n", var);
    return 0;
}
```

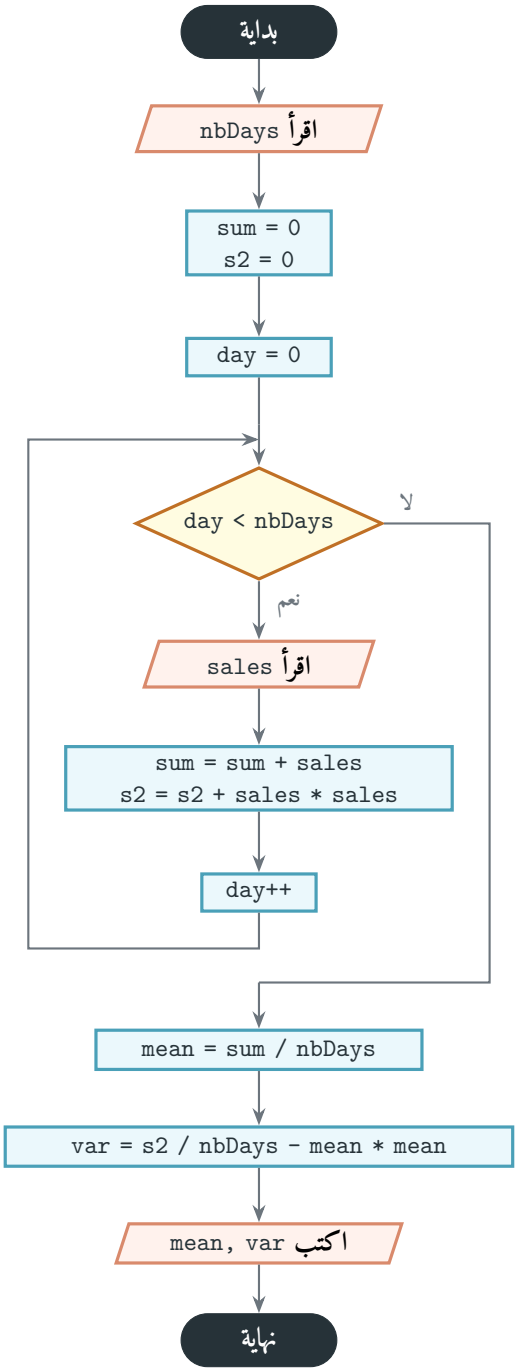
المدخلات

```
6
120 135 98 160 142 115
```

نتيجة التنفيذ

```
Mean = 128.33
Variance = 400.22
```

المخطط الإنسيابي (Organigramme)



تمرين 3.13 الجذر التربيعي بطريقة نيوتن

علم البيانات ★★ متوسط

نحتاج الانحراف المعياري، أي **جذر** التباين، دون `sqrt`. اكتب برنامجًا يحسب الجذر بطريقة نيوتن `root = (root + value / root) / 2` ويكرر حتى يصبح الفرق بين تقديرين متتاليين أقل من `1e-9`، ويعرض عدد التكرارات.

خطوات الحل

1) نبدأ بـ `root = value` (أو 1 إذا كان `value < 1`).

2) do: نحفظ old = root ثم نحسب root الجديد.

3) نكرر ما دام |root - old| > 1e-9.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double value, root, old, diff;
    int it = 0;
    scanf("%lf", &value);
    root = (value > 1) ? value : 1;           // تقدير أولي موجب
    do {
        old = root;
        root = (root + value / root) / 2;      // متوسط root و value/root
        it++;
        diff = root - old;
        if (diff < 0) diff = -diff;   // القيمة المطلقة
    } while (diff > 1e-9);
    printf("sqrt(%.2f) = %.9f\n", value, root);
    printf("Iterations: %d\n", it);
    return 0;
}
```

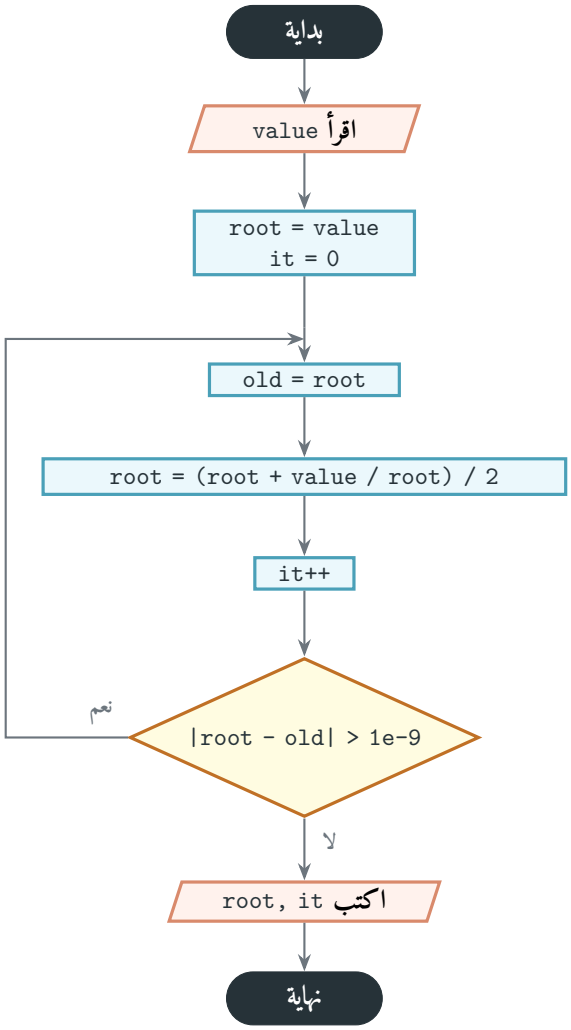
المدخلات

```
547.25
```

نتيجة التنفيذ

```
sqrt(547.25) = 23.393375131
Iterations: 10
```

المخطط الإنسيابي (Organigramme)



تمرين 3.14 ترتيبات عرض المنتجات

تجارة إلكترونية ★★ متوسط

في واجهة الصفحة الرئيسية `slots` خانات ونريد اختيار وترتيب منتجات من بين `nbProducts`. عدد **الترتيبات** هو `nbProducts! / (nbProducts-slots)! = nbProducts × (nbProducts-1) × ... × (nbProducts-slots+1)`. اكتب برنامجًا يحسبه بحلقة، ويحسب أيضًا `nbProducts!`.

خطوات الحل

1) `arr = 1`، ثم نضرب في `nbProducts`، `nbProducts-1`، ... `slots` مرات.
2) `fact = 1` ثم نضرب في 1، 2، ...، `nbProducts`.
3) نستعمل `long long` لأن الأعداد تكبر بسرعة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbProducts, slots, step;
    long long arr = 1, fact = 1;  // لتجنب تجاوز السعة long long
    scanf("%d %d", &nbProducts, &slots);
    for (step = 0; step < slots; step++)
```

```
8           arr *= (nbProducts - step);           // (عوامل slots) ... × (nbProducts-1) × nbProducts
9       for (step = 2; step <= nbProducts; step++)
10          fact *= step;                 // nbProducts!
11      printf("Arrangements A(%d,%d) = %lld\n", nbProducts, slots, arr);
12      printf("%d! = %lld\n", nbProducts, fact);
13      return 0;
14  }
```

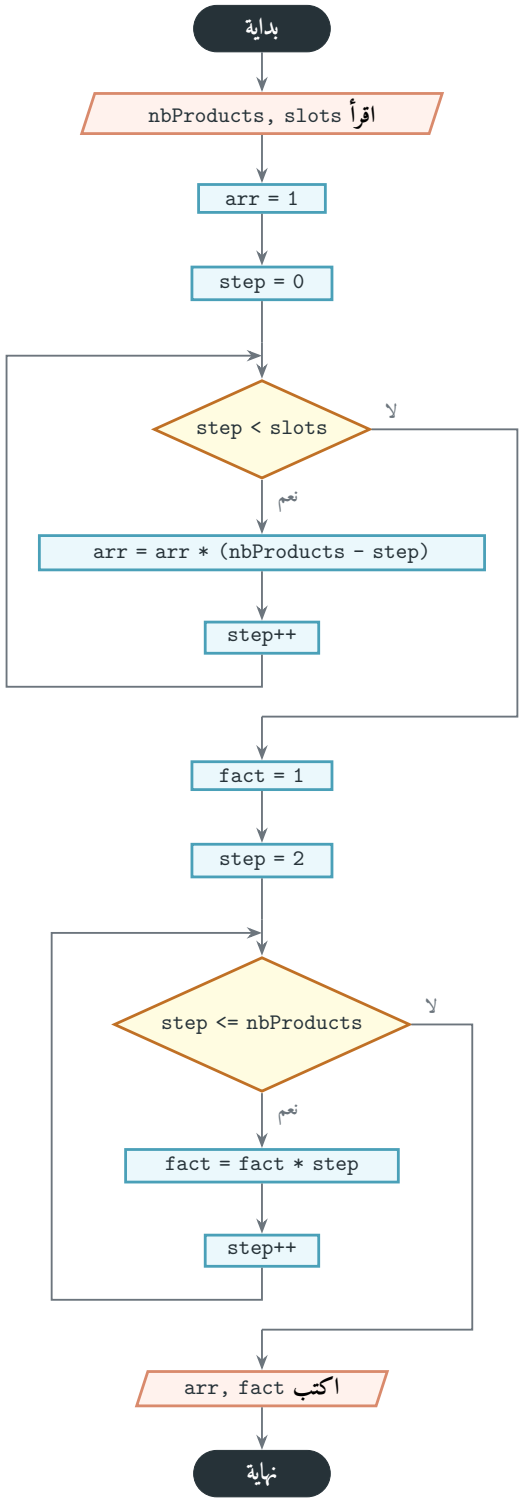
المدخلات

```
10 4
```

نتيجة التنفيذ

```
Arrangements A(10,4) = 5040
10! = 3628800
```

المخطط الإنسيابي (Organigramme)



تمرين 3.15 عدد الحزم الممكنة

تجارة إلكترونية ★★ متوسط

يريد المتجر تكوين حزم من `size` منتجات مختلفة من بين `nbProducts` (الترتيب غير مهم). عدد التوفيقات `C(nbProducts,size)`. اكتب برنامجًا يحسبه بحلقة واحدة دون حساب العامليات الكاملة: `C = C * (nbProducts - size + step) / step` لكل `step` من 1 إلى `size`.

خطوات الحل

1) `bundles = 1`.
2) لكل `step` من 1 إلى `size`: نضرب أولًا ثم نقسم؛ القسمة دائمًا صحيحة بالضبط.
3) هذه الطريقة تتجنب الأعداد الضخمة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbProducts, size, step;
    long long bundles = 1;
    scanf("%d %d", &nbProducts, &size);
    for (step = 1; step <= size; step++) {
        // بعد هذه الخطوة bundles = C(nbProducts-size+step, step)، عدد صحيح دائمًا
        bundles = bundles * (nbProducts - size + step) / step;
    }
    printf("Bundles C(%d,%d) = %lld\n", nbProducts, size, bundles);
    return 0;
}
```

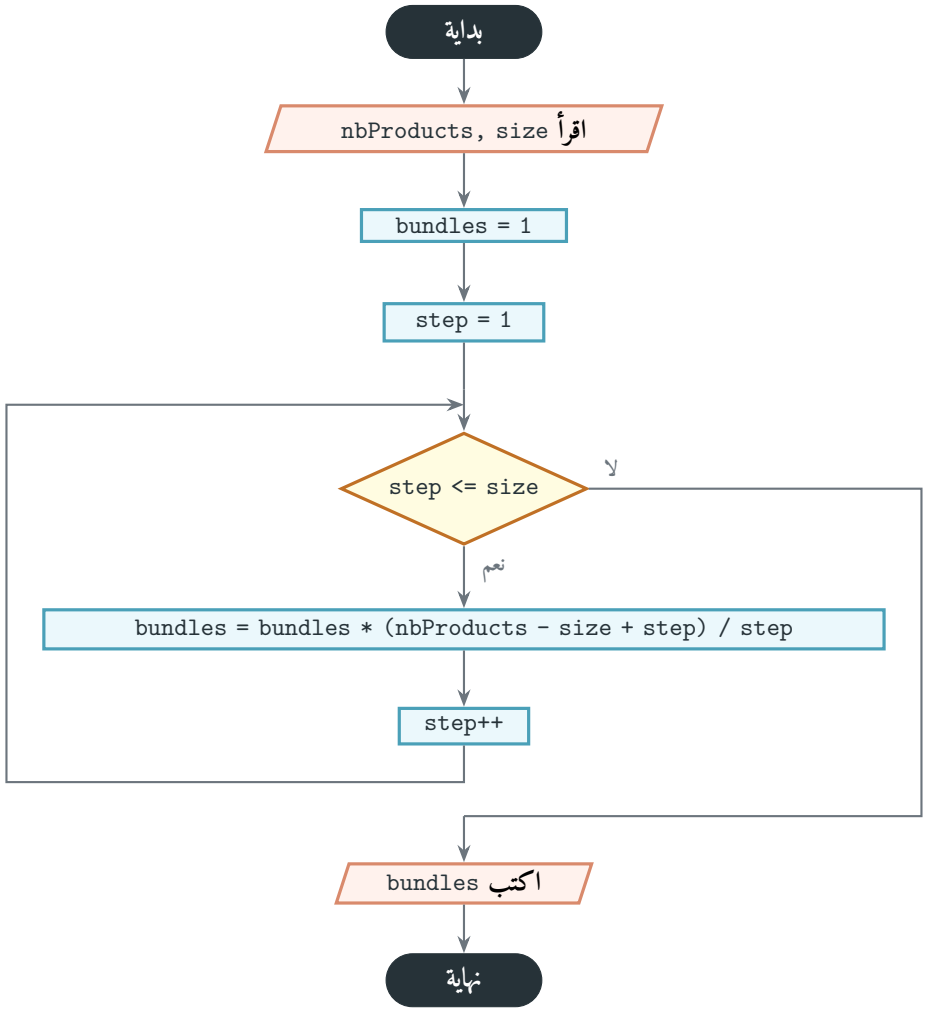
المدخلات

```
20 5
```

نتيجة التنفيذ

```
Bundles C(20,5) = 15504
```

المخطط الإنسيابي (Organigramme)



تمرين 3.16 أكبر قاسم مشترك لأحجام العبوات

تجارة إلكترونية ★★ متوسط

يصل منتج في كراتين من packA وحدة ومنتج آخر في كراتين من packB وحدة. لتكوين رفوف متساوية دون باقٍ نحتاج **أكبر قاسم مشترك**. اكتب برنامجًا يحسبه بخوارزمية **إقليدس**.

خطوات الحل

1) ما دام packB != 0: rest = packA % packB ثم packA = packB و packB = rest.
2) عند التوقف، packA هو القاسم المشترك الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int packA, packB, rest, a0, b0;
    scanf("%d %d", &packA, &packB);
    a0 = packA; b0 = packB;              // نحفظ القيم الأصلية للعرض
    while (packB != 0) {
        rest = packA % packB;            // pgcd(packA, packB) = pgcd(packB, packA % packB)
        packA = packB;
```

```c
        packB = rest;
    }
    printf("GCD(%d, %d) = %d\n", a0, b0, packA);
    printf("Shelves of %d units: %d + %d\n", packA, a0 / packA, b0 / packA);
    return 0;
}
```

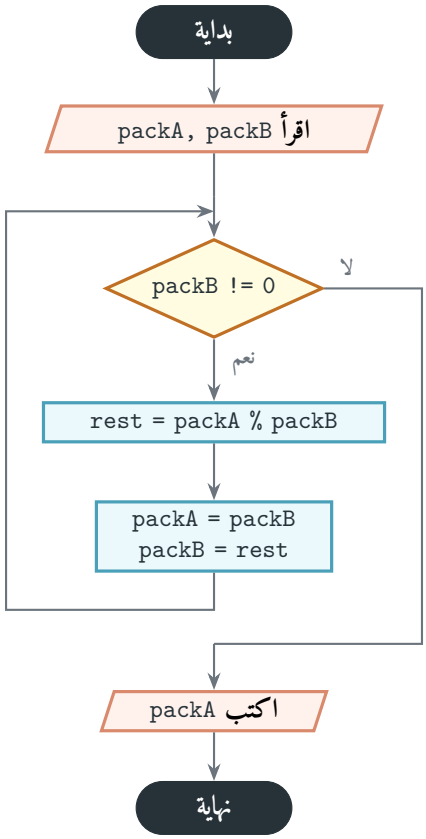
المدخلات

```
84 120
```

نتيجة التنفيذ

```
GCD(84, 120) = 12
Shelves of 12 units: 7 + 10
```

المخطط الإنسيابي (Organigramme)



تمرين 3.17 تزامن دورات التموين

تجارة إلكترونية ★★ متوسط

يُموَّن منتج كل cycleA أيام ومنتج آخر كل cycleB أيام، وقد مُوِّنا معًا اليوم. بعد كم يومًا يُموَّنان معًا من جديد؟ اكتب برنامجًا يحسب المضاعف المشترك الأصغر lcm = cycleA / gcd * cycleB.

خطوات الحل

1) نحسب gcd(cycleA, cycleB) بخوارزمية إقليدس على نسختين من المتغيرين.
2) lcm = cycleA / g * cycleB (نقسم أولًا لتجنب تجاوز السعة).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int cycleA, cycleB, gcdA, gcdB, rest, lcm;
    scanf("%d %d", &cycleA, &cycleB);
    gcdA = cycleA; gcdB = cycleB;                // نسختان لكي لا نفقد cycleA و cycleB
    while (gcdB != 0) {
        rest = gcdA % gcdB;
        gcdA = gcdB;
        gcdB = rest;
    }                                // gcdA = pgcd(cycleA, cycleB)
    lcm = cycleA / gcdA * cycleB;
    printf("Both restocked again after %d days\n", lcm);
    return 0;
}
```

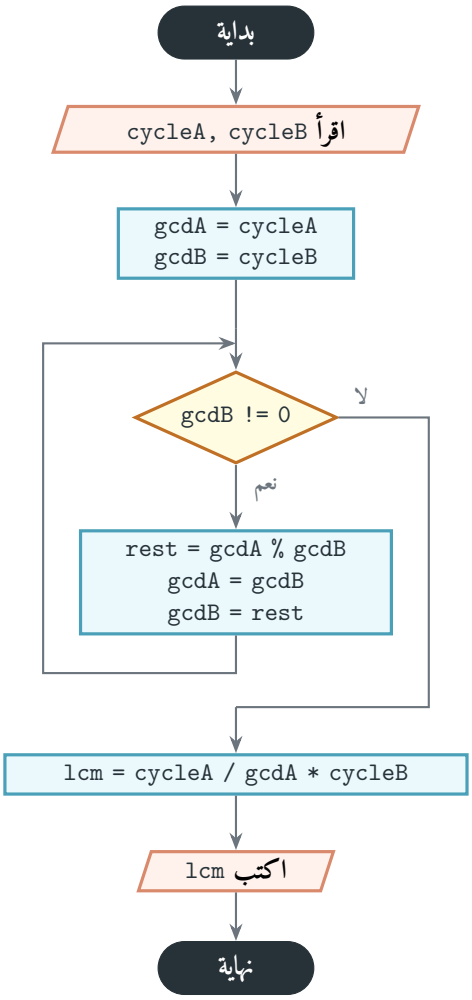
المدخلات

```
6 15
```

نتيجة التنفيذ

```
Both restocked again after 30 days
```

المخطط الإنسيابي (Organigramme)



تمرين 3.18 هل رقم المنتج أولي؟

تجارة إلكترونية متوسط

لتوزيع المنتجات على خوادم التخزين نستعمل أرقامًا أولية. اكتب برنامجًا يتحقق إن كان عدد ما أوليًا، باختبار القواسم من 2 حتى `divisor * divisor <= number` فقط.

خطوات الحل

1) إذا كان `number < 2` فهو ليس أوليًا.
2) نجرب القواسم `divisor` ما دام `divisor * divisor <= number`؛ نتوقف عند أول قاسم.
3) إذا لم نجد قاسمًا فهو أولي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int number, divisor, prime;
    scanf("%d", &number);
    prime = (number >= 2);              // 0 و 1 ليسا أوليين
    // يكفي البحث حتى الجذر: إذا كان number = a × b فأحدهما <= الجذر
    for (divisor = 2; prime && divisor * divisor <= number; divisor++) {
        if (number % divisor == 0)
            prime = 0;                  // وجدنا قاسمًا
    }
    if (prime)
        printf("%d is prime\n", number);
    else
        printf("%d is not prime\n", number);
    return 0;
}
```

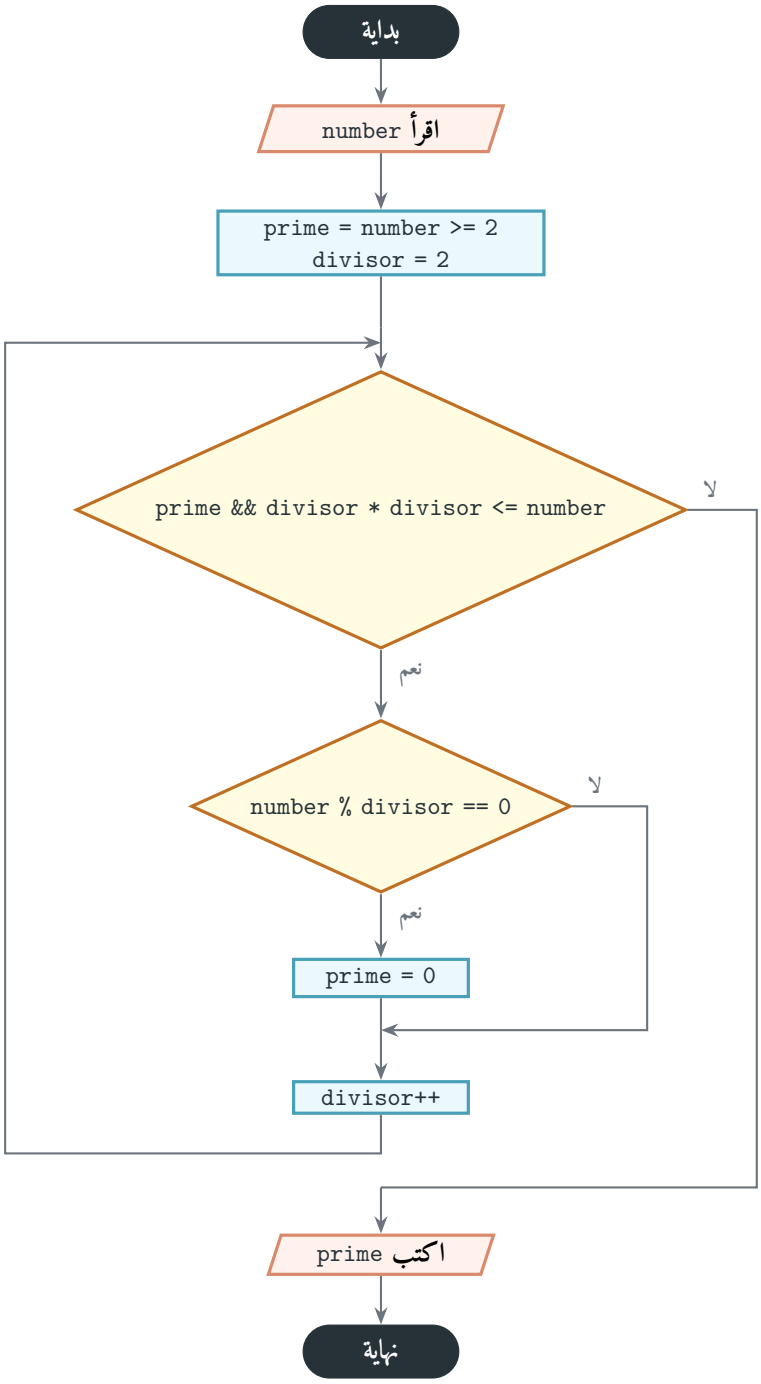
المدخلات

```
7919
```

نتيجة التنفيذ

```
7919 is prime
```

المخطط الإنسيابي (Organigramme)



تمرين 3.19 أرقام رقم الطلبية

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يقرأ رقم طلبية (عدد صحيح موجب طويل) ويحسب **عدد أرقامه** و**مجموعها** و**أكبر رقم** فيه، بحلقة تقسم على 10.

خطوات الحل

1) ما دام `number > 0`: الرقم الأخير `digit = number % 10` ثم `number = number / 10`.
2) نزيد العداد ونضيف `digit` للمجموع ونحدّث الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long number;
    int digit, count = 0, sum = 0, big = 0;
    scanf("%ld", &number);
    do {                            // do-while: الرقم 0 له رقم واحد
        digit = number % 10;                // آخر رقم
        count++;
        sum += digit;
        if (digit > big) big = digit;
        number = number / 10;               // نحذف آخر رقم
    } while (number > 0);
    printf("Digits = %d, sum = %d, largest = %d\n", count, sum, big);
    return 0;
}
```

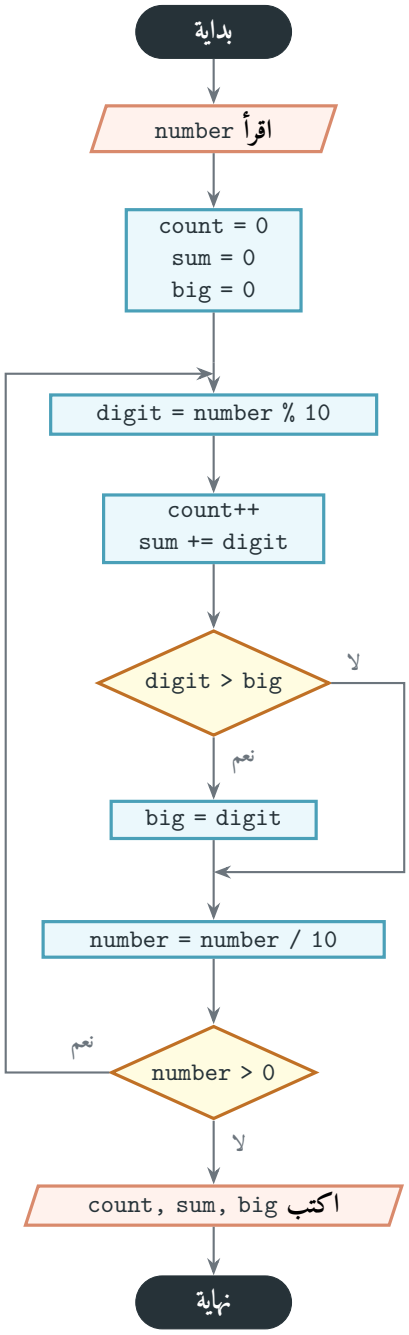
المدخلات

```
90417253
```

نتيجة التنفيذ

```
Digits = 8, sum = 31, largest = 9
```

المخطط الإنسيابي (Organigramme)



تمرين 3.20 رمز تخفيض متناظر

تجارة إلكترونية ★★ متوسط

رموز التخفيض الخاصة هي أعداد متناظرة (تُقرأ بنفس الطريقة من اليسار واليمين مثل 12321). اكتب برنامجًا يعكس العدد بحلقة ويتحقق إن كان متناظرًا.

خطوات الحل

1) `rev = 0`، وما دام `code > 0`: `rev = rev * 10 + code % 10` ثم `code /= 10`.
2) نقارن `rev` بالعدد الأصلي.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long code, work, rev = 0;
    scanf("%ld", &code);
    work = code;                          // نعمل على نسخة
    while (work > 0) {
        rev = rev * 10 + work % 10;  // نلصق آخر رقم في نهاية rev
        work = work / 10;
    }
    printf("Reversed: %ld\n", rev);
    if (rev == code)
        printf("%ld is a valid palindrome code\n", code);
    else
        printf("%ld is not a palindrome\n", code);
    return 0;
}
```

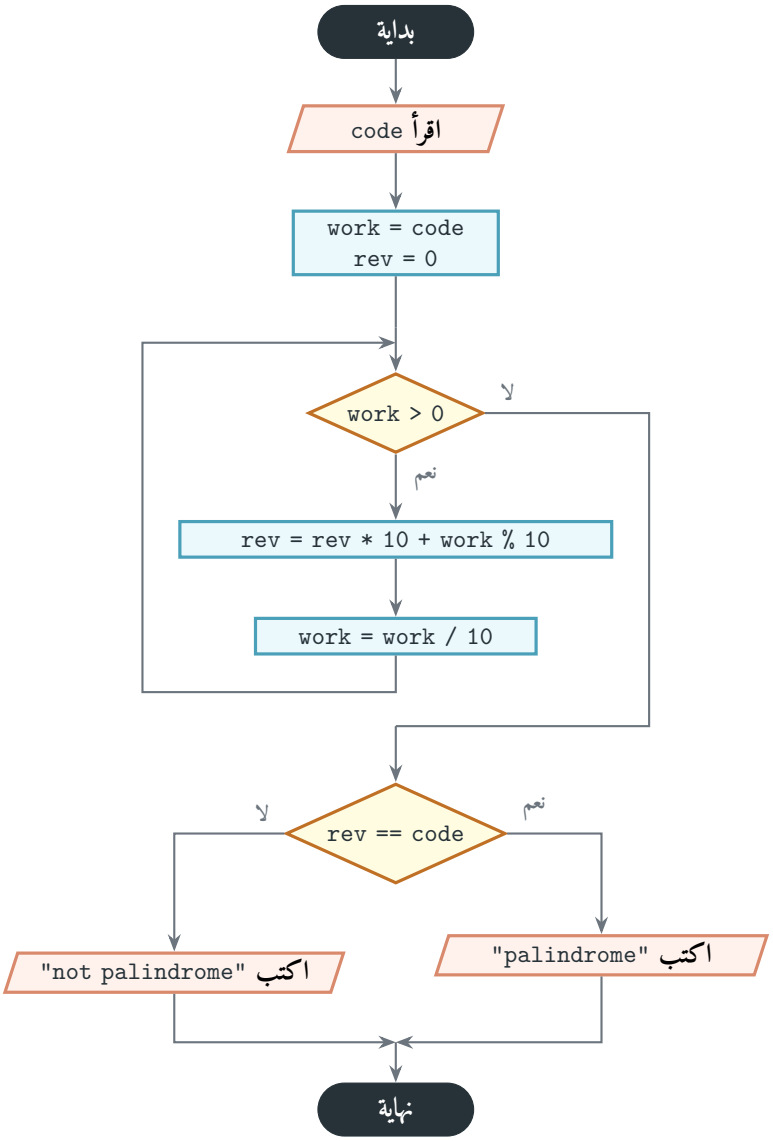
المدخلات

```
45654
```

نتيجة التنفيذ

```
Reversed: 45654
45654 is a valid palindrome code
```

المخطط الإنسيابي (Organigramme)



تمرين 3.21 نمو المستخدمين بمتتالية فيبوناتشي

علم البيانات ★★ متوسط

في نموذج نمو مبسط، عدد المستخدمين الجدد كل شهر يساوي مجموع الشهرين السابقين (متتالية **فيبوناتشي**)، بدءًا من 1 و1. اكتب برنامجًا يعرض الأشهر حتى يتجاوز العدد الشهري `target`، مع المجموع التراكمي.

خطوات الحل

1) `prev = 1`، `current = 1` (الشهران الأولان).
2) في كل دورة: `next = prev + current` ثم نزيح `prev = current` و`current = next`.
3) نتوقف عندما يتجاوز العدد الهدف.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```
    long prev = 1, current = 1, next, total = 2, target;
    int month = 2;
    scanf("%ld", &target);
    printf("Month 1: 1\nMonth 2: 1\n");
    while (current <= target) {
        next = prev + current;                // الشهر الجديد = مجموع الشهرين السابقين
        prev = current;                       // نزيح النافذة
        current = next;
        total += next;
        month++;
        printf("Month %d: %ld\n", month, next);
    }
    printf("Total users after %d months: %ld\n", month, total);
    return 0;
}
```

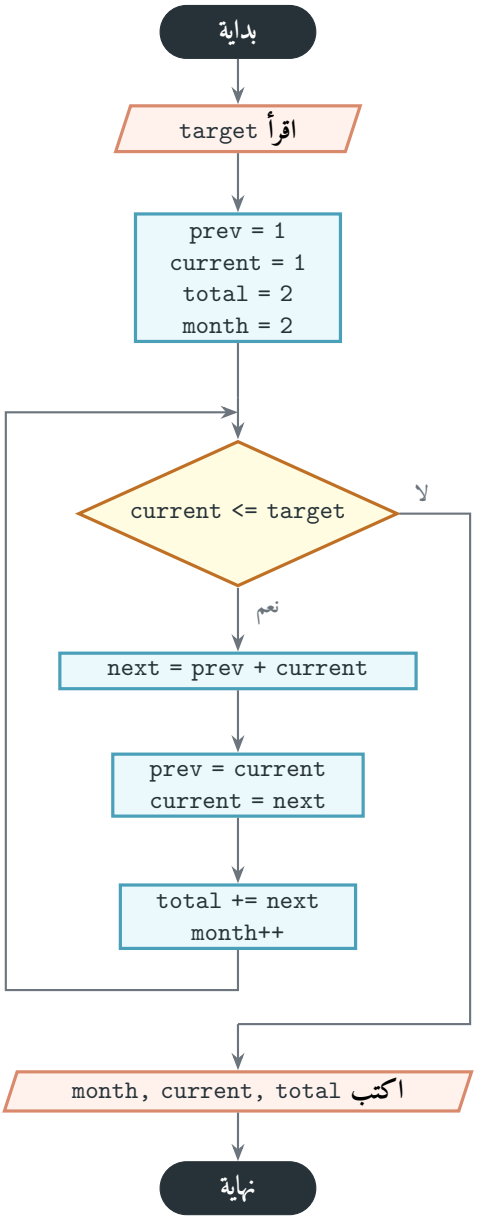
المدخلات

```
500
```

نتيجة التنفيذ

```
Month 1: 1
Month 2: 1
Month 3: 2
Month 4: 3
Month 5: 5
Month 6: 8
Month 7: 13
Month 8: 21
Month 9: 34
Month 10: 55
Month 11: 89
Month 12: 144
Month 13: 233
Month 14: 377
Month 15: 610
Total users after 15 months: 1596
```

المخطط الإنسيابي (Organigramme)



تمرين 3.22 الفائدة المركبة على عدة سنوات

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب قيمة استثمار `capital` بعد `years` سنوات بنسبة فائدة سنوية `rate`%، ويعرض القيمة في نهاية كل سنة، دون `pow`.

خطوات الحل

1) `factor = 1 + rate / 100`.
2) حلقة على السنوات: `capital = capital * factor` ثم العرض.

البرنامج بلغة C

```c
#include <stdio.h>

```

```c
int main(void) {
    double capital, rate, factor;
    int years, year;
    scanf("%lf %lf %d", &capital, &rate, &years);
    factor = 1 + rate / 100;
    for (year = 1; year <= years; year++) {
        capital = capital * factor;                // القوة تُبنى بالضرب المتكرر
        printf("Year %d: %.2f DA\n", year, capital);
    }
    return 0;
}
```

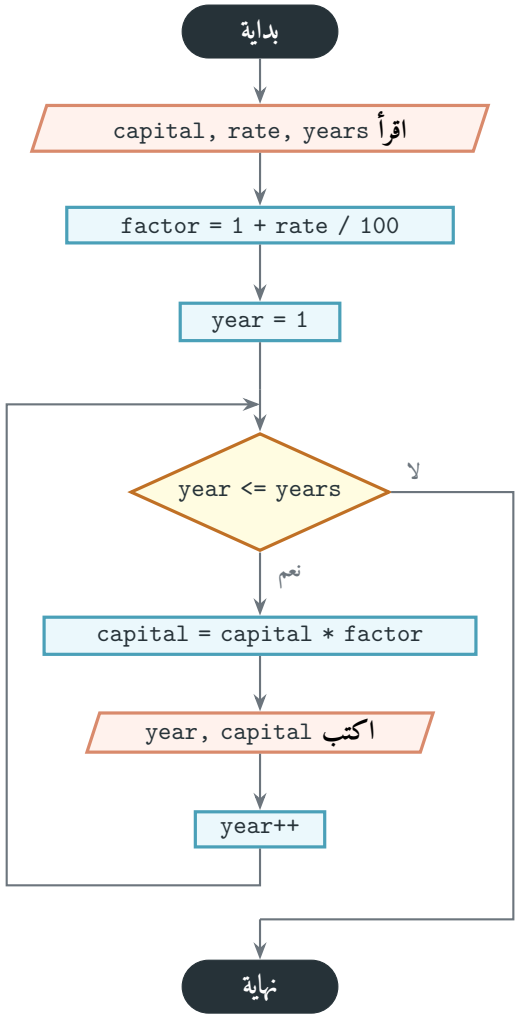
المدخلات

```
100000 6 5
```

نتيجة التنفيذ

```
Year 1: 106000.00 DA
Year 2: 112360.00 DA
Year 3: 119101.60 DA
Year 4: 126247.70 DA
Year 5: 133822.56 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.23 جدول تسديد الشراء بالتقسيط

تجارة إلكترونية ★★ متوسط

اشترى زبون هاتفًا بالتقسيط: المبلغ balance، فائدة شهرية rate% على الباقي، وقسط شهري ثابت installment. اكتب برنامجًا يعرض لكل شهر الفائدة والباقي، حتى السداد الكامل، مع مجموع الفوائد.

خطوات الحل

1) ما دام الباقي balance > 0: الفائدة interest = balance * rate / 100.
2) balance = balance + interest - installment؛ إذا أصبح سالبًا فالقسط الأخير أصغر.
3) نجمع الفوائد.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double balance, rate, installment, interest, totI = 0;
    int month = 0;
    scanf("%lf %lf %lf", &balance, &rate, &installment);
    printf("Month | Interest | Remaining\n");
    while (balance > 0) {
        interest = balance * rate / 100;          // فائدة الشهر على الباقي
        totI += interest;
        balance = balance + interest - installment;          // نضيف الفائدة ونطرح القسط
        month++;
        if (balance < 0) balance = 0;          // القسط الأخير يسدد ما تبقى فقط
        printf("%5d | %8.2f | %9.2f\n", month, interest, balance);
    }
    printf("Total interest = %.2f DA\n", totI);
    return 0;
}
```

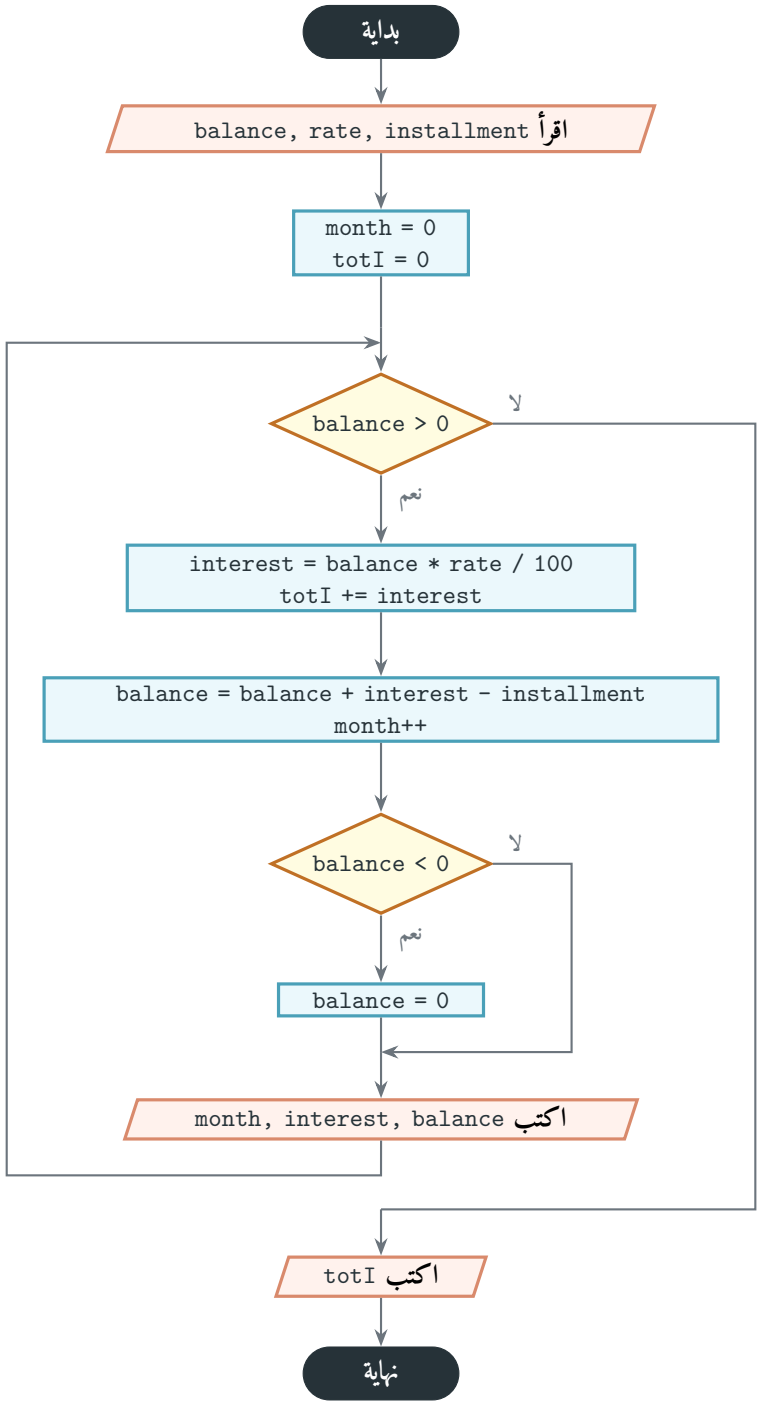
المدخلات

```
60000 1.5 11000
```

نتيجة التنفيذ

```
Month | Interest | Remaining
    1 |   900.00 |  49900.00
    2 |   748.50 |  39648.50
    3 |   594.73 |  29243.23
    4 |   438.65 |  18681.88
    5 |   280.23 |   7962.10
    6 |   119.43 |      0.00
Total interest = 3081.54 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.24 أطول فترة نمو متواصل

علم البيانات

متوسط

اكتب برنامجًا يقرأ مبيعات `nbDays` يومًا متتالية ويحدد **أطول سلسلة** من الأيام التي ارتفعت فيها المبيعات مقارنة باليوم السابق، ويوم نهايتها.

خطوات الحل

1) نحتفظ بالقيمة السابقة `prev`.

2) إذا كان `sales > prev` نزيد طول السلسلة الحالية `cur`، وإلا نعيدها إلى 0.

3) نحدّث الأطول `best` ويوم نهايته.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day, cur = 0, best = 0, end = 1;
    double prev, sales;
    scanf("%d %lf", &nbDays, &prev);
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &sales);
        if (sales > prev)
            cur++;                  // السلسلة تستمر
        else
            cur = 0;                // السلسلة انقطعت
        if (cur > best) {
            best = cur;
            end = day;
        }
        prev = sales;                   // اليوم الحالي يصبح السابق
    }
    printf("Longest growth streak: %d days (ending day %d)\n", best, end);
    return 0;
}
```

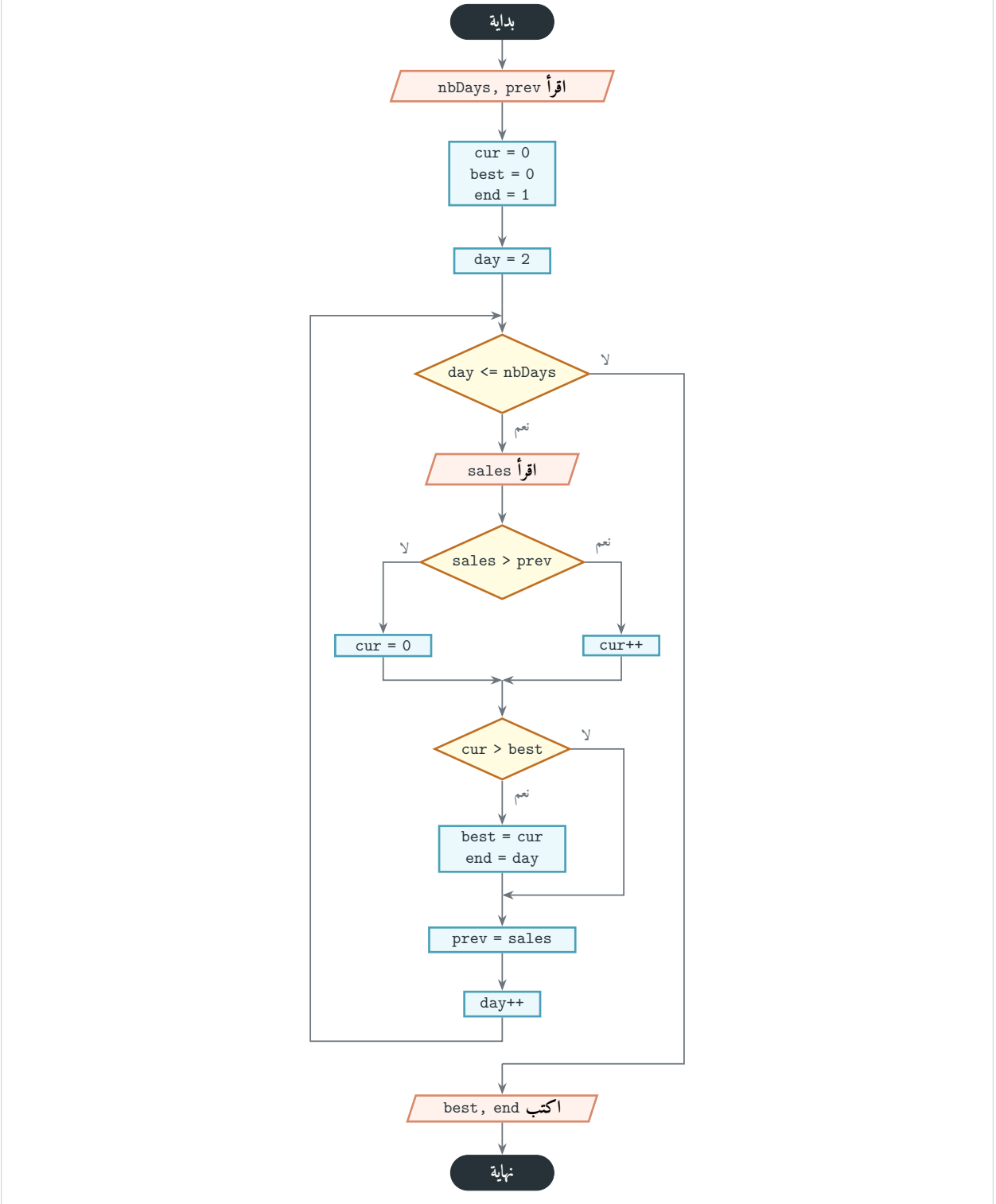
المدخلات

```
10
50 55 60 58 61 64 70 75 72 80
```

نتيجة التنفيذ

```
Longest growth streak: 4 days (ending day 8)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.25 المتوسط المتحرك لثلاثة أيام

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ مبيعات `nbDays` يومًا ويعرض ابتداءً من اليوم الثالث **المتوسط المتحرك** لآخر 3 أيام، دون جداول: نحتفظ بآخر قيمتين في متغيرين.

خطوات الحل

1) نقرأ أول قيمتين في `p2` (قبل الأخيرة) و`p1` (الأخيرة).
2) لكل قيمة جديدة `sales`: المتوسط `(p2 + p1 + sales) / 3`.
3) نزيح النافذة: `p2 = p1` ثم `p1 = sales`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day;
    double p2, p1, sales;
    scanf("%d %lf %lf", &nbDays, &p2, &p1);     // أول يومين
    for (day = 3; day <= nbDays; day++) {
        scanf("%lf", &sales);
        printf("Day %d: MA3 = %.2f\n", day, (p2 + p1 + sales) / 3);
        p2 = p1;                    // إزاحة النافذة بمقدار يوم
        p1 = sales;
    }
    return 0;
}
```

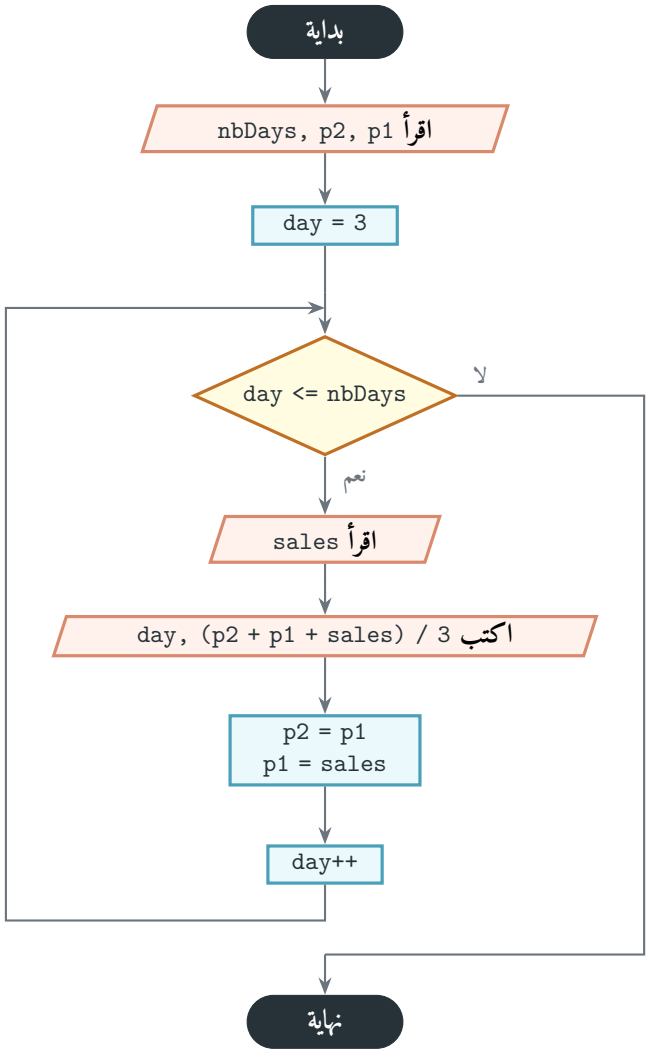
المدخلات

```
7
100 130 115 160 140 170 150
```

نتيجة التنفيذ

```
Day 3: MA3 = 115.00
Day 4: MA3 = 135.00
Day 5: MA3 = 138.33
Day 6: MA3 = 156.67
Day 7: MA3 = 153.33
```

المخطط الإنسيابي (Organigramme)



تمرين 3.26 مدرج تكراري للتقييمات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ `nbReviews` تقييمات (1 إلى 5) ويعد تكرار كل قيمة في خمسة عدادات، ثم يرسم **مدرجًا تكراريًا** نصيًا بالرمز * لكل نجمة.

خطوات الحل

1) خمسة عدادات `c1` إلى `c5` نزيدها بـ `switch`.
2) لرسم السطر `stars` نستعمل حلقة داخلية تطبع * عددًا من المرات يساوي العداد.
3) لتجنب تكرار الكود نختار العداد في كل دورة بـ `switch`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbReviews, index, rating, stars, star, bar;
    int c1 = 0, c2 = 0, c3 = 0, c4 = 0, c5 = 0;
```

```c
    scanf("%d", &nbReviews);
    for (index = 0; index < nbReviews; index++) {
        scanf("%d", &rating);
        switch (rating) {              // نزيد عداد القيمة المقروءة
            case 1: c1++; break;
            case 2: c2++; break;
            case 3: c3++; break;
            case 4: c4++; break;
            case 5: c5++; break;
        }
    }
    for (stars = 5; stars >= 1; stars--) {    // من 5 نجوم إلى نجمة واحدة
        switch (stars) {
            case 5: bar = c5; break;
            case 4: bar = c4; break;
            case 3: bar = c3; break;
            case 2: bar = c2; break;
            default: bar = c1;
        }
        printf("%d stars | ", stars);
        for (star = 0; star < bar; star++)   // حلقة داخلية: طول العمود
            printf("*");
        printf(" (%d)\n", bar);
    }
    return 0;
}
```

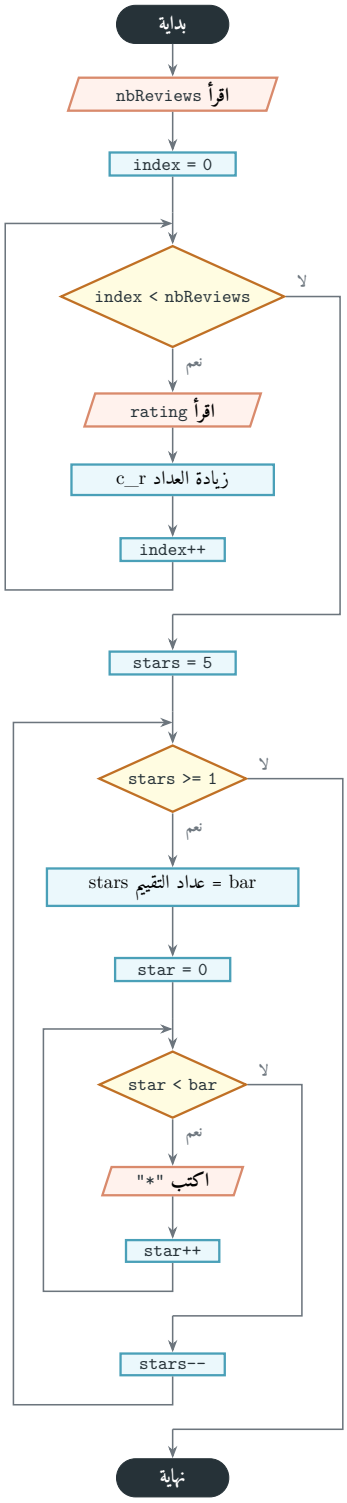
المدخلات

```
15
5 4 5 3 5 4 4 2 5 1 4 5 3 4 5
```

نتيجة التنفيذ

```
5 stars | ****** (6)
4 stars | ***** (5)
3 stars | ** (2)
2 stars | * (1)
1 stars | * (1)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.27 رقم المنتج بالنظام الثنائي

تجارة إلكترونية

متوسط

تخزن بطاقات RFID أرقام المنتجات بالنظام **الثنائي**. اكتب برنامجًا يحول عددًا عشريًا إلى الثنائي دون جداول: نبني الناتج كعدد يُقرأ بالأرقام 0 و1، ونعد البتات التي قيمتها 1.

خطوات الحل

1) ما دام number > 0: البت bit = number % 2 ثم number = number / 2.
2) البتات تخرج من اليمين إلى اليسار، فنضعها في الموضع place (1، 10، 100، ...): bin = bin + bit * place.
3) نضاعف place في 10 في كل دورة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int number, n0, bit, ones = 0;
    long long bin = 0, place = 1;
    scanf("%d", &number);
    n0 = number;
    while (number > 0) {
        bit = number % 2;              // البت الأقل وزنًا
        ones += bit;
        bin = bin + bit * place;       // نضعه في موضعه العشري
        place = place * 10;
        number = number / 2;           // ننتقل إلى البت الموالي
    }
    printf("%d in binary = %lld (%d ones)\n", n0, bin, ones);
    return 0;
}
```

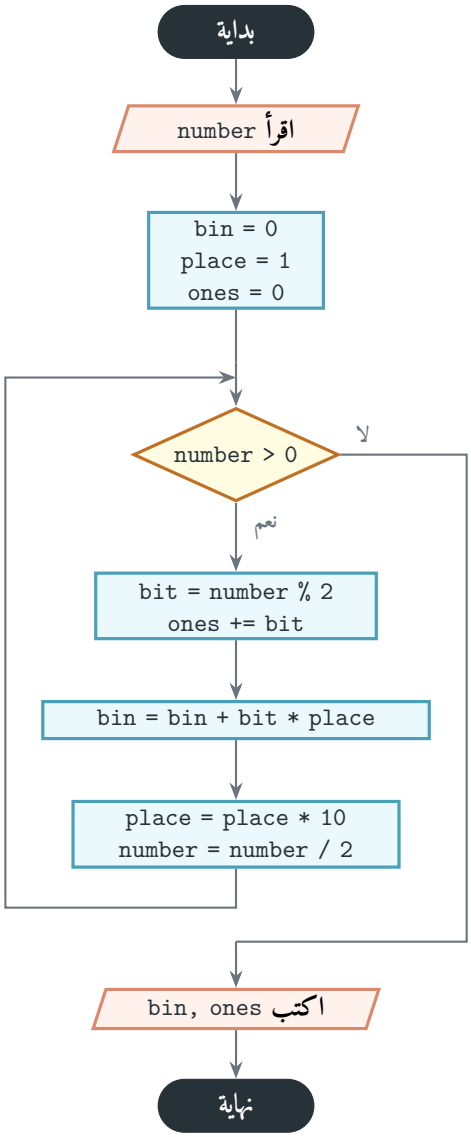
المدخلات

```
202
```

نتيجة التنفيذ

```
202 in binary = 11001010 (4 ones)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.28 شبكة التخفيضات

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يعرض جدولًا ثنائيًا: الأسطر كميات من 1 إلى 5، والأعمدة نسب تخفيض 0% و10% و20% و30%، وكل خانة تحتوي ثمن الكمية بعد التخفيض لسعر وحدة مقروء. استعمل **حلقتين متداخلتين**.

خطوات الحل

1) الحلقة الخارجية على الكمية `qty`.
2) الحلقة الداخلية على النسبة `discount` من 0 إلى 30 بخطوة 10.
3) الخانة: `qty * price * (100 - discount) / 100`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
```

```c
    double price;
    int qty, discount;
    scanf("%lf", &price);
    printf("qty |     0%%    10%%    20%%    30%%\n");
    for (qty = 1; qty <= 5; qty++) {            // سطر لكل كمية
        printf("%3d |", qty);
        for (discount = 0; discount <= 30; discount += 10)     // عمود لكل نسبة
            printf(" %6.0f", qty * price * (100 - discount) / 100);
        printf("\n");
    }
    return 0;
}
```

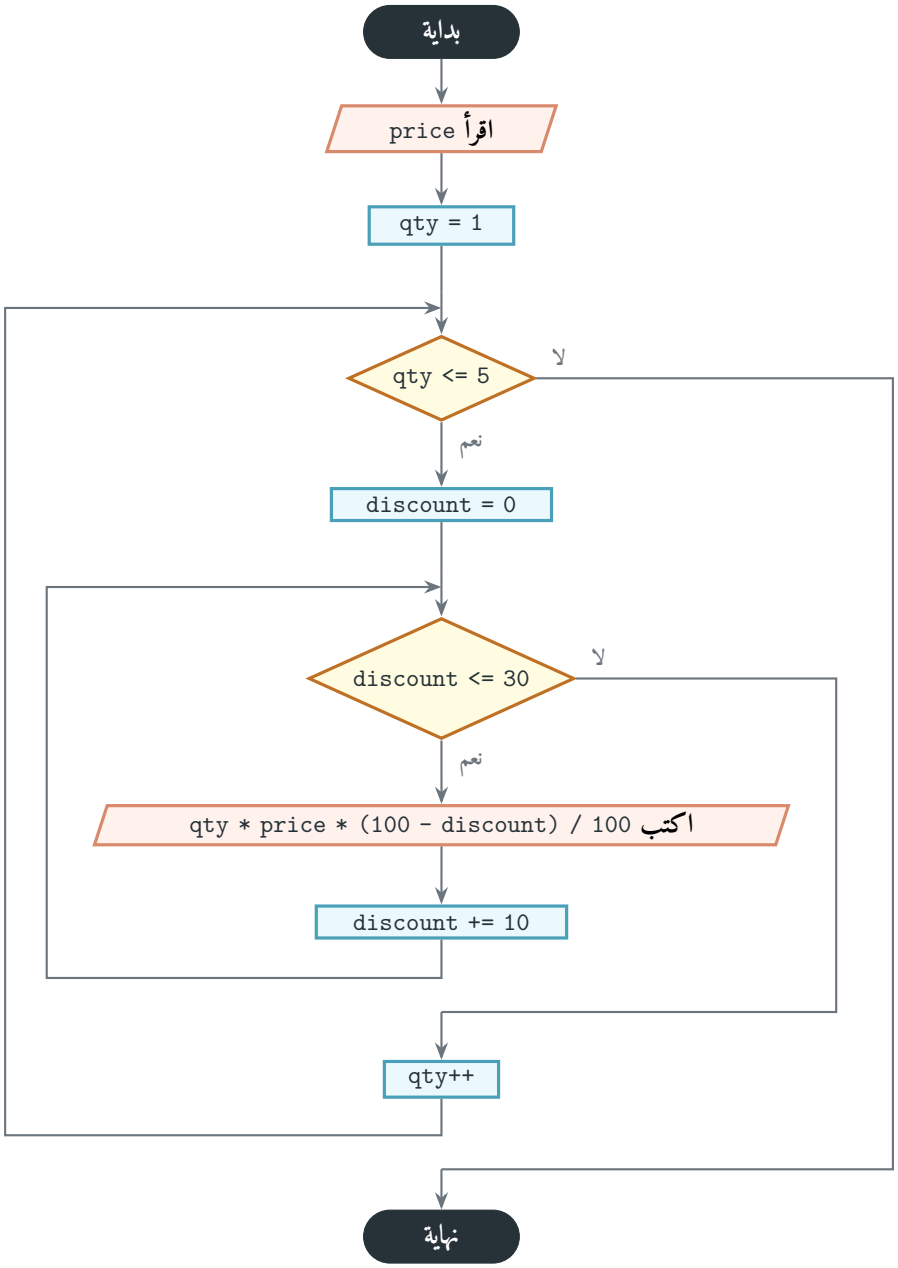
المدخلات

```
1500
```

نتيجة التنفيذ

```
qty |     0%    10%    20%    30%
  1 |   1500   1350   1200   1050
  2 |   3000   2700   2400   2100
  3 |   4500   4050   3600   3150
  4 |   6000   5400   4800   4200
  5 |   7500   6750   6000   5250
```

المخطط الإنسيابي (Organigramme)



تمرين 3.29 عد الزبائن في كل شريحة

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ الإنفاق السنوي لـ `nbCustomers` زبونًا ويعد الزبائن في كل شريحة: أقل من 20000، من 20000 إلى أقل من 100000، و100000 فأكثر، مع متوسط إنفاق كل شريحة.

خطوات الحل

1) ثلاثة عدادات وثلاثة مجاميع.
2) في كل دورة نحدد الشريحة بـ `if / else if`.
3) المتوسط لكل شريحة إذا كان عددها غير صفري.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbCustomers, index, n1 = 0, n2 = 0, n3 = 0;
    double spend, s1 = 0, s2 = 0, s3 = 0;
    scanf("%d", &nbCustomers);
    for (index = 0; index < nbCustomers; index++) {
        scanf("%lf", &spend);
        if (spend < 20000)       { n1++; s1 += spend; }   // صغار
        else if (spend < 100000) { n2++; s2 += spend; }   // متوسطون
        else                     { n3++; s3 += spend; }   // كبار
    }
    printf("Low:    %d customers", n1);
    if (n1 > 0) printf(", avg %.0f", s1 / n1);   // تجنب القسمة على 0
    printf("\nMedium: %d customers", n2);
    if (n2 > 0) printf(", avg %.0f", s2 / n2);
    printf("\nHigh:   %d customers", n3);
    if (n3 > 0) printf(", avg %.0f", s3 / n3);
    printf("\n");
    return 0;
}
```

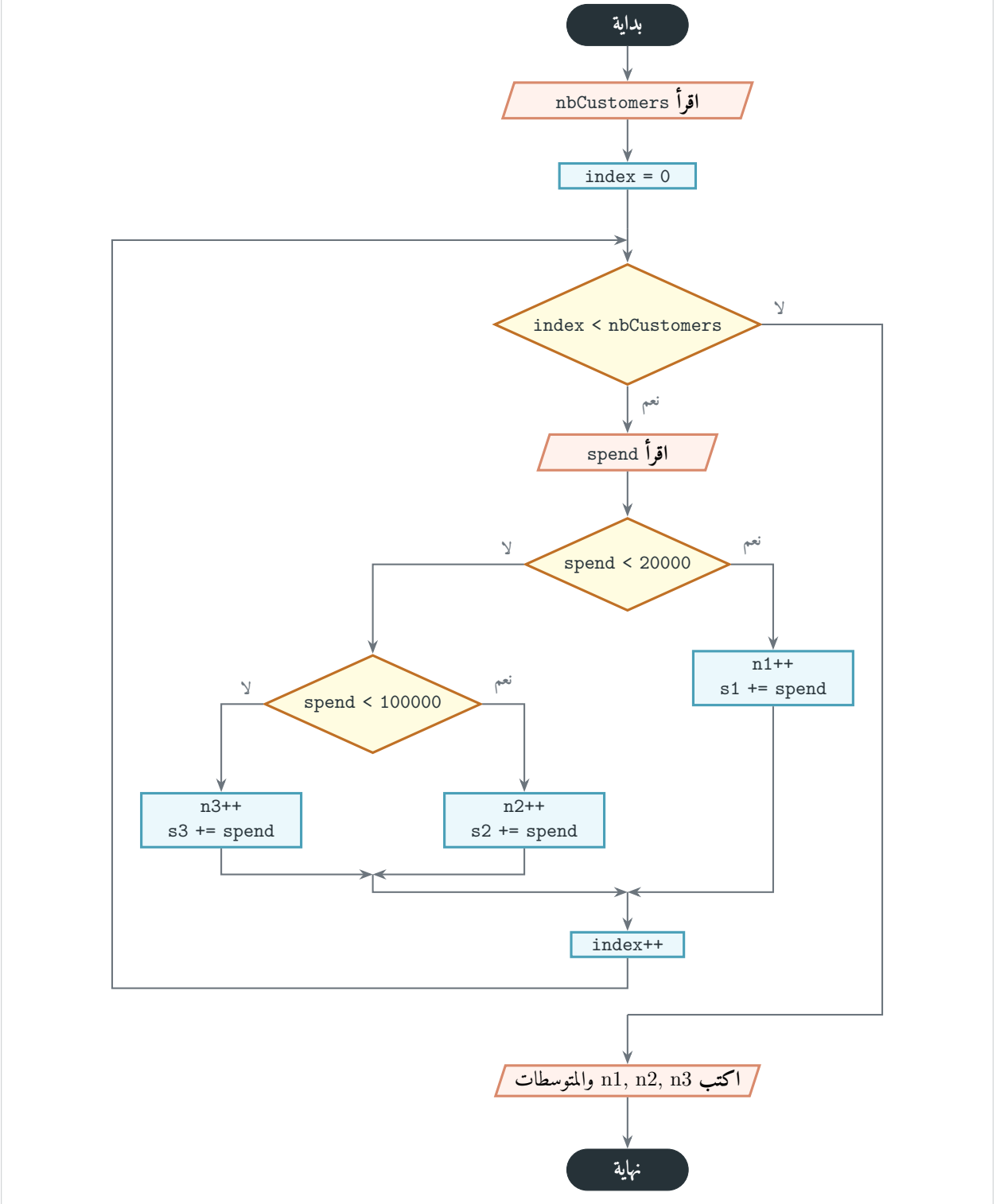
المدخلات

```
9
15000 120000 45000 8000 99000 250000 30000 61000 19999
```

نتيجة التنفيذ

```
Low:    3 customers, avg 14333
Medium: 4 customers, avg 58750
High:   2 customers, avg 185000
```

المخطط الإنسيابي (Organigramme)



تمرين 3.30 محاكاة نفاد المخزون

تجارة إلكترونية

متوسط

مخزون منتج `stock` وحدة. الطلب اليومي يبدأ بـ `initDemand` وحدة ويرتفع بنسبة `growth`% كل يوم (منتج رائج). اكتب برنامجًا يحاكي الأيام حتى **نفاد المخزون**، ويعرض يوم النفاد والطلب غير الملبى في آخر يوم.

خطوات الحل

1) ما دام `stock > 0`: نزيد اليوم، الطلب اليوم `dem` (مقرب لعدد صحيح).
2) إذا كان `dem >= stock` فالمخزون ينفد اليوم والناقص `dem - stock`.
3) وإلا `stock = stock - dem` ثم `dem = dem * (1 + growth / 100)`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double stock, initDemand, growth, dem;
    int day = 0;
    scanf("%lf %lf %lf", &stock, &initDemand, &growth);
    dem = initDemand;
    while (stock > 0) {
        day++;
        dem = (int)(dem + 0.5);   // الطلب عدد صحيح من الوحدات
        if (dem >= stock) {           // آخر يوم: لا يكفي المخزون
            printf("Stock out on day %d (unmet demand: %.0f)\n", day, dem - stock);
            stock = 0;
        } else {
            stock = stock - dem;
            printf("Day %2d: sold %3.0f, left %4.0f\n", day, dem, stock);
            dem = dem * (1 + growth / 100);   // الطلب يرتفع
        }
    }
    return 0;
}
```

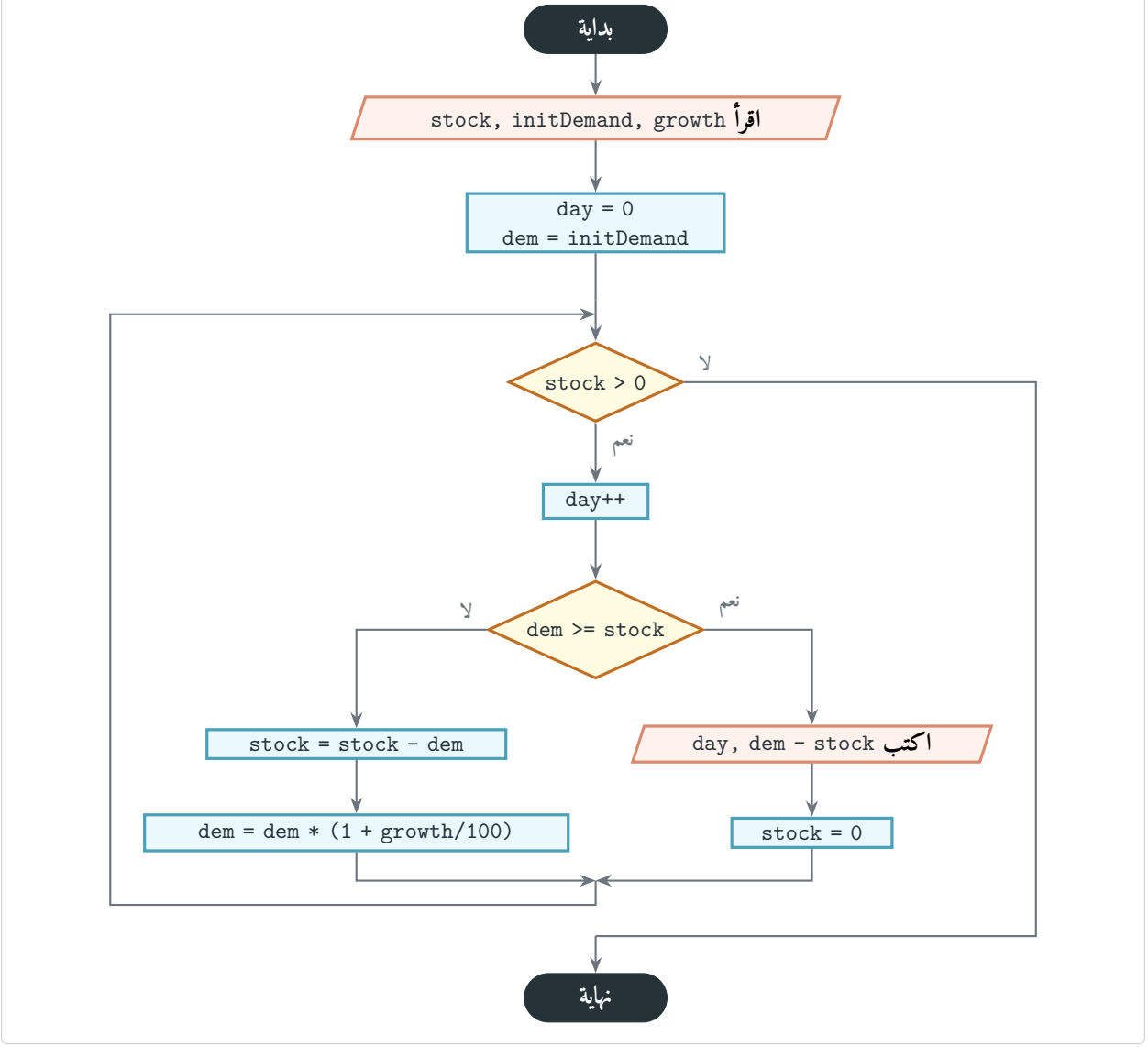
المدخلات

```
1000 40 10
```

نتيجة التنفيذ

```
Day  1: sold  40, left  960
Day  2: sold  44, left  916
Day  3: sold  48, left  868
Day  4: sold  53, left  815
Day  5: sold  58, left  757
Day  6: sold  64, left  693
Day  7: sold  70, left  623
Day  8: sold  77, left  546
Day  9: sold  85, left  461
Day 10: sold  94, left  367
Day 11: sold 103, left  264
Day 12: sold 113, left  151
Day 13: sold 124, left   27
Stock out on day 14 (unmet demand: 109)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.31 التنعيم الأسي لسلسلة زمنية

علم البيانات

متوسط

اكتب برنامجًا يقرأ معامل التنعيم `alpha` وعدد الأيام `nbDays` ثم المبيعات، ويعرض لكل يوم القيمة المنعّمة

```
smooth = alpha * sales + (1 - alpha) * smooth
```

والتنبؤ لليوم الموالي.

خطوات الحل

1) القيمة الأولى تهيئ `smooth`.
2) لكل قيمة موالية نطبق الصيغة.
3) التنبؤ لليوم `nbDays + 1` هو آخر قيمة `smooth`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double alpha, sales, smooth;
    int nbDays, day;
    scanf("%lf %d %lf", &alpha, &nbDays, &sales);
    smooth = sales;                                  // نبدأ بالملاحظة الأولى
    printf("Day 1: %.2f\n", smooth);
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &sales);
        smooth = alpha * sales + (1 - alpha) * smooth;  // مزيج من الجديد والقديم
        printf("Day %d: %.2f\n", day, smooth);
    }
    printf("Forecast day %d: %.2f\n", nbDays + 1, smooth);
    return 0;
}
```

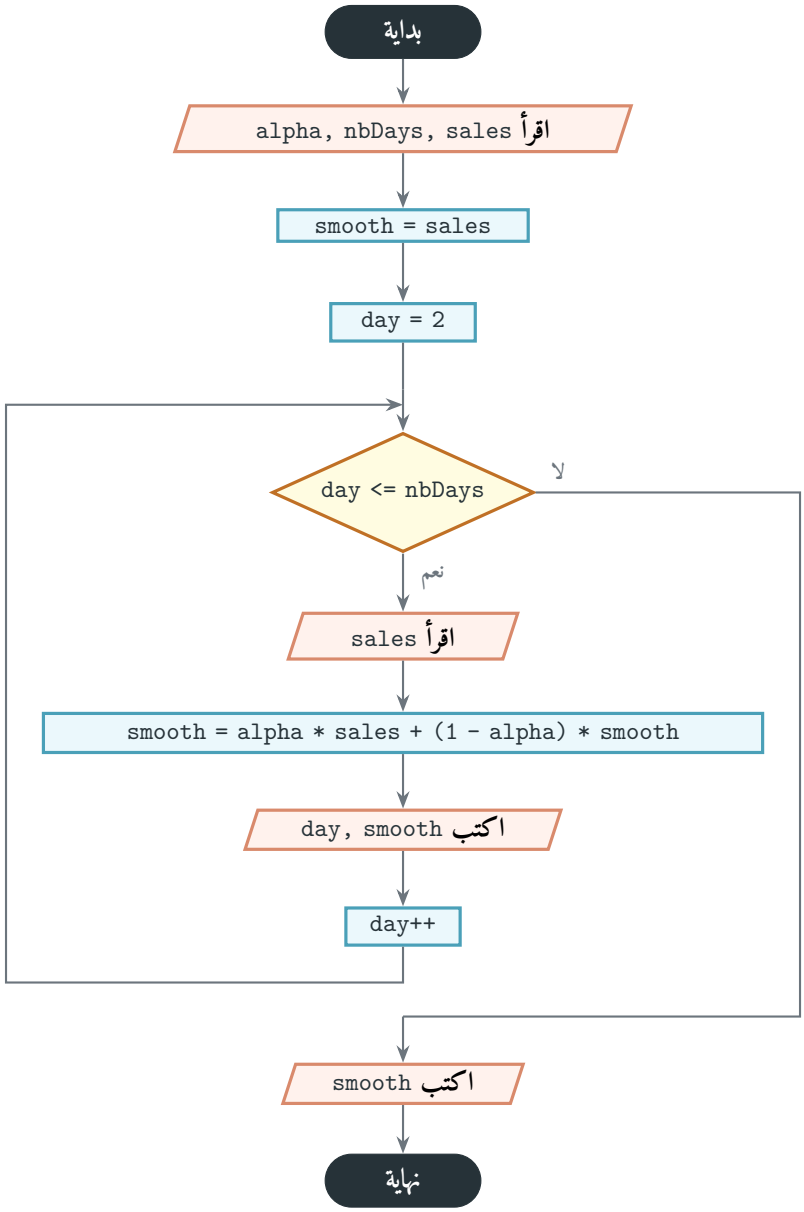
المدخلات

```
0.3 6
200 260 210 300 280 330
```

نتيجة التنفيذ

```
Day 1: 200.00
Day 2: 218.00
Day 3: 215.60
Day 4: 240.92
Day 5: 252.64
Day 6: 275.85
Forecast day 7: 275.85
```

المخطط الإنسيابي (Organigramme)



تمرين 3.32 الإنحدار الخطي على n نقطة

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ nbPoints نقطة (ميزانية الإعلان بالآلاف، المبيعات) ويحسب مستقيم **المربعات الصغرى** sales = slope budget + intercept في مرور واحد بجمع Σbudget وΣsales وΣxy و$\Sigma budget^2$.

خطوات الحل

1) نجمع المجاميع الأربعة في حلقة واحدة.

2) $slope = (nbPoints\ \Sigma xy - \Sigma budget\ \Sigma sales) / (nbPoints\ \Sigma budget^2 - (\Sigma budget)^2)$

.

3) $intercept = (\Sigma sales - slope\ \Sigma budget) / nbPoints$

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPoints, index;
    double budget, sales, sx = 0, sy = 0, sxy = 0, sxx = 0, slope, intercept;
    scanf("%d", &nbPoints);
    for (index = 0; index < nbPoints; index++) {
        scanf("%lf %lf", &budget, &sales);
        sx += budget;  sy += sales;        // المجاميع البسيطة
        sxy += budget * sales;             // مجموع الجداءات
        sxx += budget * budget;             // مجموع المربعات
    }
    slope = (nbPoints * sxy - sx * sy) / (nbPoints * sxx - sx * sx);
    intercept = (sy - slope * sx) / nbPoints;
    printf("Sales = %.3f * budget + %.3f\n", slope, intercept);
    printf("Forecast for budget 8: %.1f\n", slope * 8 + intercept);
    return 0;
}
```

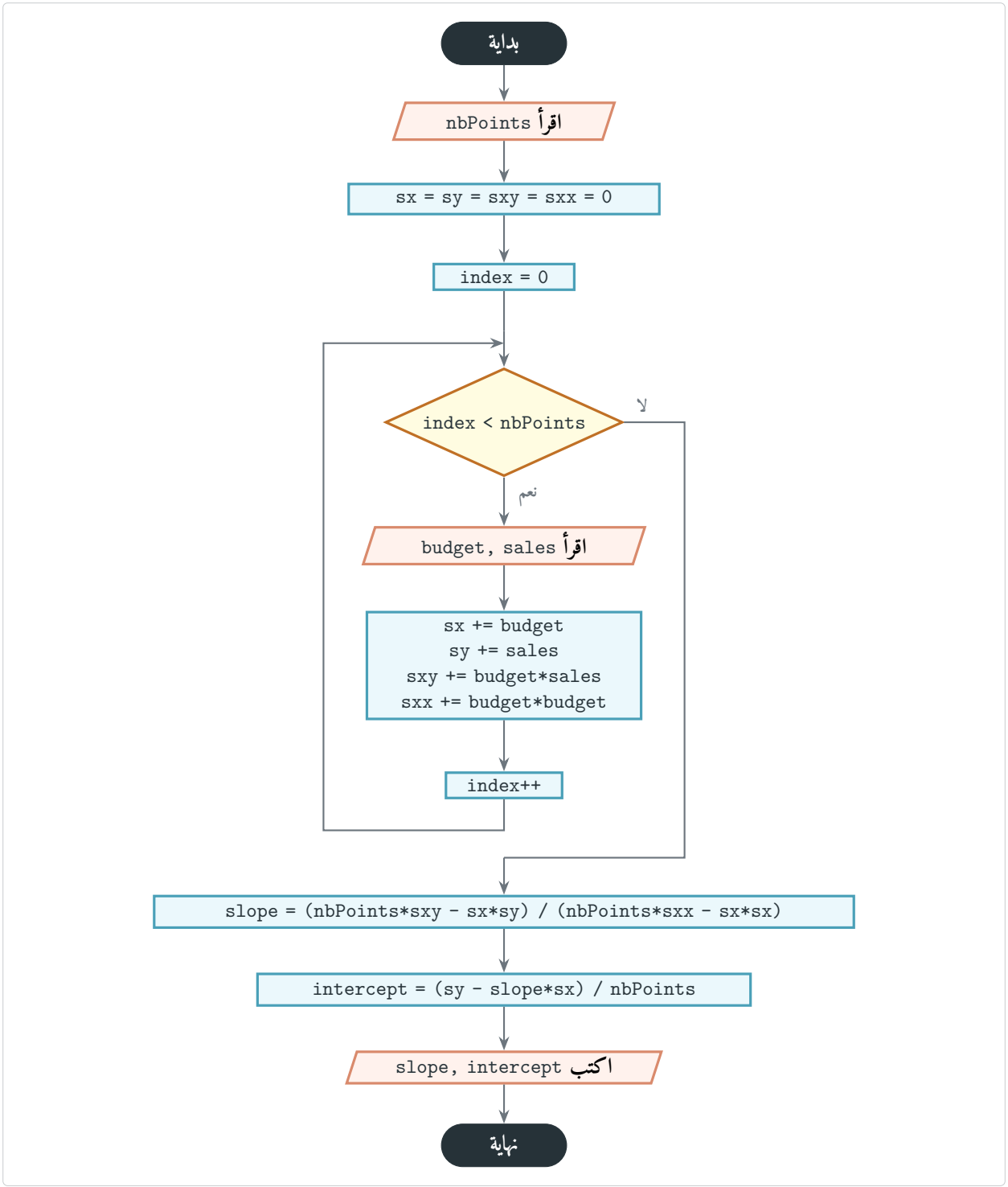
المدخلات

```
6
1 52
2 61
3 74
4 79
5 93
6 101
```

نتيجة التنفيذ

```
Sales = 9.886 * budget + 42.067
Forecast for budget 8: 121.2
```

المخطط الإنسيابي (Organigramme)



تمرين 3.33 معامل الإرتباط

علم البيانات

متوسط

اكتب برنامجًا يحسب **معامل ارتباط بيرسون** بين الوقت المقضي في الموقع (دقائق) ومبلغ الشراء لـ nbCustomers زبون:

$$r = \frac{n\sum xy - \sum x \sum y}{\sqrt{\left(n\sum x^2 - \left(\sum x\right)^2\right)\left(n\sum y^2 - \left(\sum y\right)^2\right)}}$$

، مع حساب الجذر بطريقة نيوتن.

خطوات الحل

1) نجمع Σminutes، Σamount، Σxy، $\Sigma\text{minutes}^2$، Σamount^2.
2) البسط num والمقدار تحت الجذر den2.
3) الجذر بحلقة نيوتن (40 تكرارًا كافية).
4) r = num / den.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbCustomers, index;
    double minutes, amount, sx = 0, sy = 0, sxy = 0, sxx = 0, syy = 0;
    double num, den2, den;
    scanf("%d", &nbCustomers);
    for (index = 0; index < nbCustomers; index++) {
        scanf("%lf %lf", &minutes, &amount);
        sx += minutes; sy += amount; sxy += minutes * amount;
        sxx += minutes * minutes; syy += amount * amount;
    }
    num = nbCustomers * sxy - sx * sy;
    den2 = (nbCustomers * sxx - sx * sx) * (nbCustomers * syy - sy * sy);
    den = den2;                          // الجذر بطريقة نيوتن
    for (index = 0; index < 40; index++)
        den = (den + den2 / den) / 2;
    printf("Pearson r = %.4f\n", num / den);
    return 0;
}
```

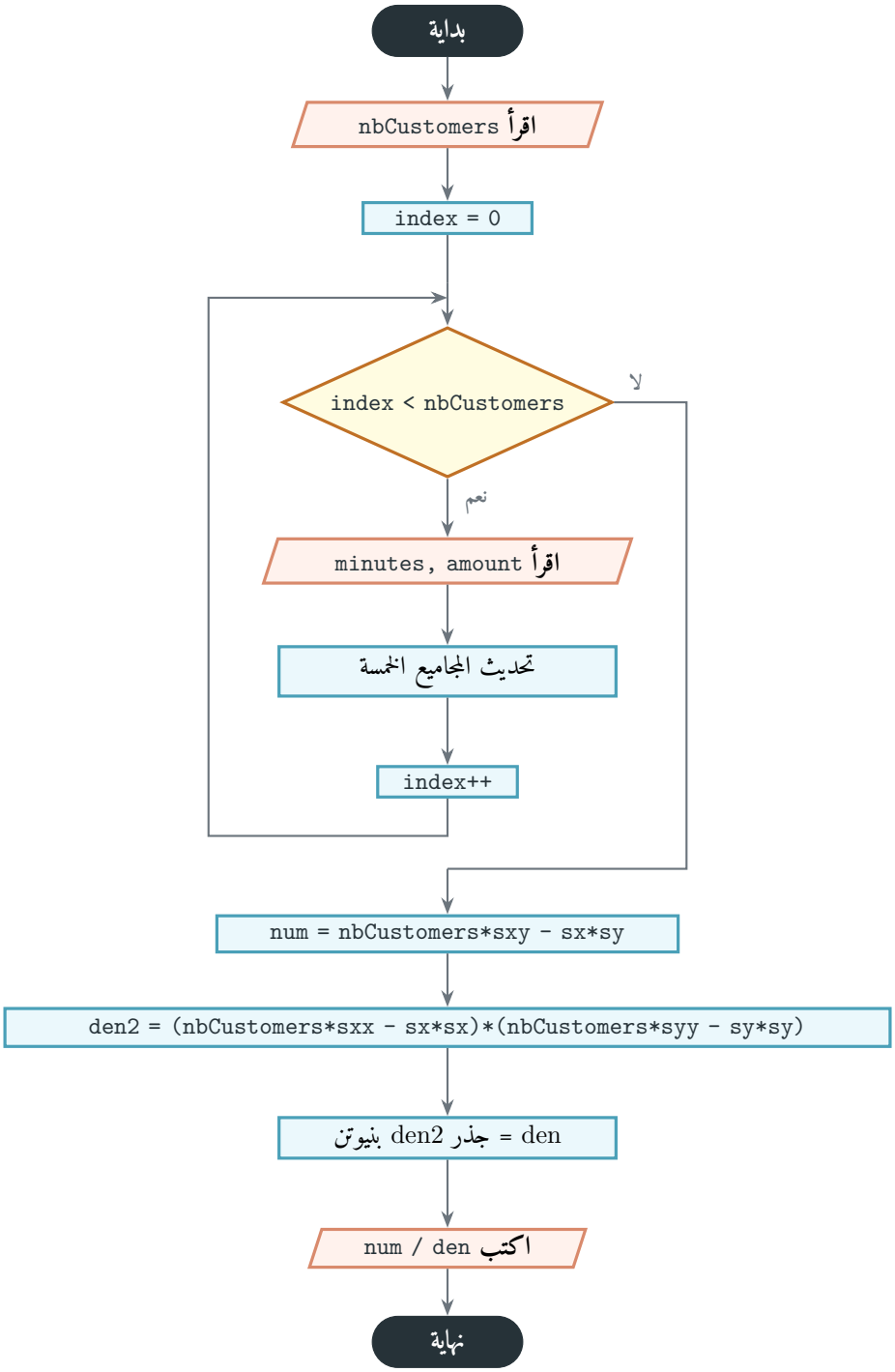
المدخلات

```
6
3 500
8 2100
5 1200
12 3900
2 300
9 2600
```

نتيجة التنفيذ

```
Pearson r = 0.9950
```

المخطط الإنسيابي (Organigramme)



تمرين 3.34 ثاني أغلى منتج

تجارة إلكترونية

متوسط

اكتب برنامجًا يقرأ `nbPrices` أسعار **مختلفة** ويجد **ثاني أكبر** سعر في مرور واحد، دون ترتيب ودون جداول.

خطوات الحل

1) نحتفظ بـ `first` (الأكبر) و`second` (الثاني)، مهيأين بقيمة صغيرة جدًا.

2) إذا كان `price > first` الأكبر القديم يصبح الثاني و`first = price`.

3) وإلا إذا كان `price > second` :`second = price`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbPrices, index;
    double price, first = -1, second = -1;     // الأسعار موجبة
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++) {
        scanf("%lf", &price);
        if (price > first) {
            second = first;        // الأكبر القديم ينزل إلى المرتبة الثانية
            first = price;
        } else if (price > second) {
            second = price;
        }
    }
    printf("Highest = %.0f, second = %.0f\n", first, second);
    return 0;
}
```

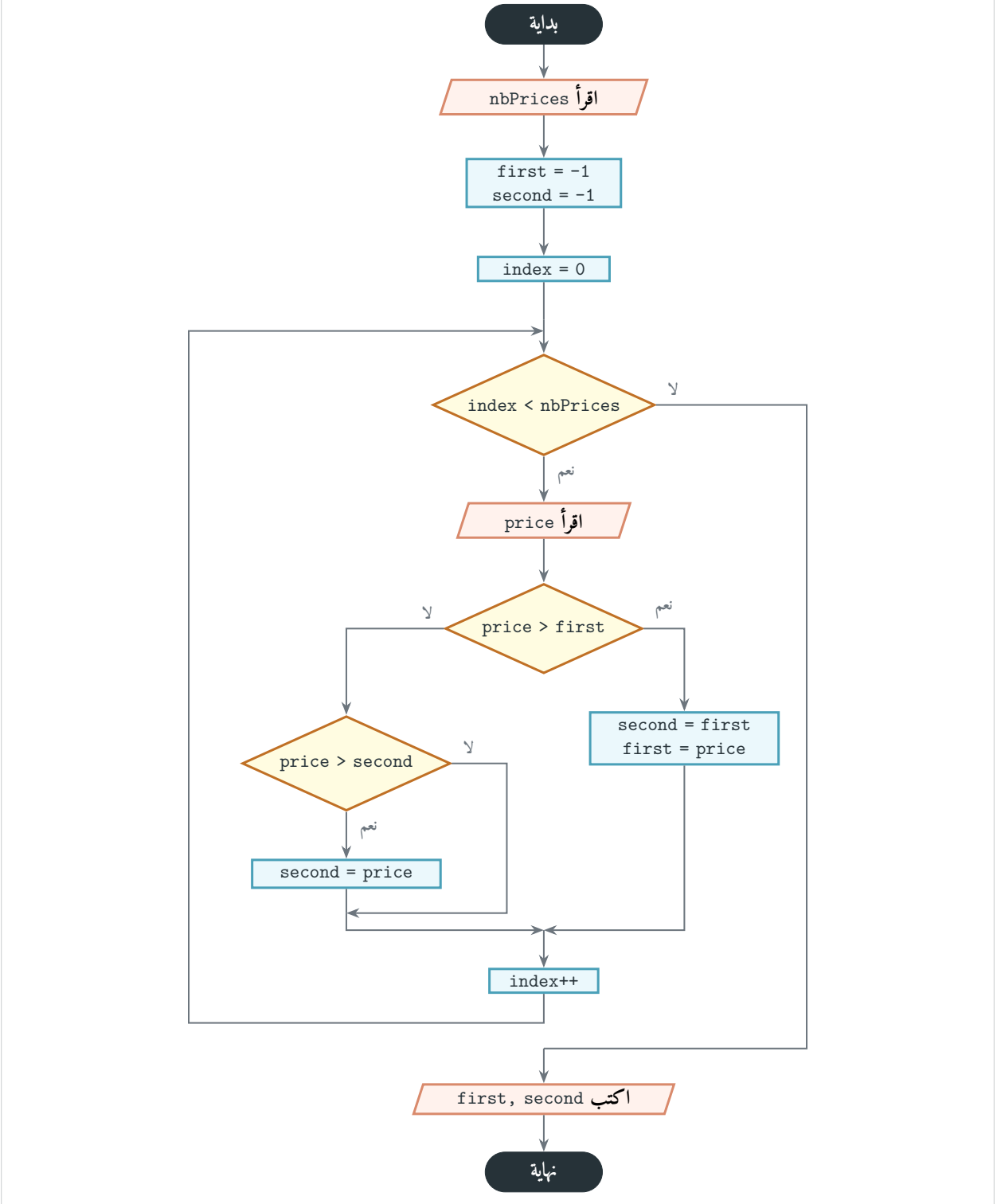
المدخلات

```
7
450 1290 300 2200 870 1990 99
```

نتيجة التنفيذ

```
Highest = 2200, second = 1990
```

المخطط الإنسيابي (Organigramme)



تمرين 3.35 هرم مستويات الوفاء

تجارة إلكترونية

متوسط

اكتب برنامجًا يرسم هرمًا من `levels` أسطر يمثل مستويات الوفاء: السطر `row` يحتوي `levels - row` فراغًا ثم `2i - 1` نجمة، ثم يعرض رقم المستوى.

خطوات الحل

1) حلقة خارجية على الأسطر `row` من 1 إلى `levels`.
2) حلقة داخلية أولى للفراغات، وثانية للنجوم.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int levels, row, col;
    scanf("%d", &levels);
    for (row = 1; row <= levels; row++) {
        for (col = 1; col <= levels - row; col++)      // فراغات للتوسيط
            printf(" ");
        for (col = 1; col <= 2 * row - 1; col++)  // عدد فردي من النجوم
            printf("*");
        printf("  level %d\n", levels - row + 1);
    }
    return 0;
}
```

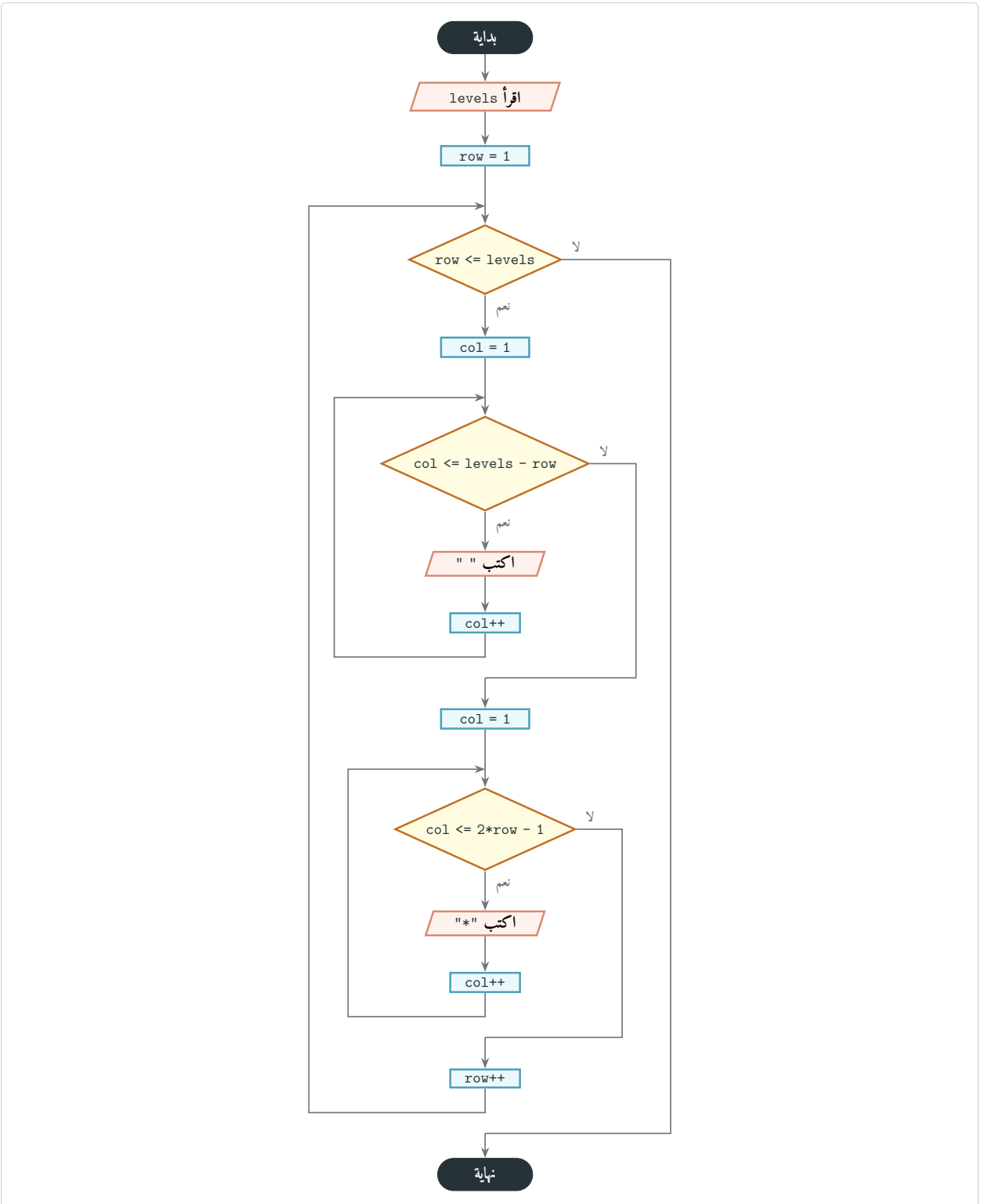
المدخلات

```
5
```

نتيجة التنفيذ

```
    *  level 5
   ***  level 4
  *****  level 3
 *******  level 2
*********  level 1
```

المخطط الإنسيابي (Organigramme)



تمرين 3.36 سعر التوازن بالتنصيف

علم البيانات ★★ متوسط

الطلب على منتج `D(p) = 5000 / p` (غير خطي) والعرض `S(p) = 2p - 20`. اكتب برنامجًا يجد سعر التوازن (حيث `D(p) - S(p) = 0`) بطريقة **التنصيف** في المجال `[lo, hi]` بدقة 0.001.

خطوات الحل

1) `gap(p) = 5000 / p - (2p - 20)` موجبة عند `lo` وسالبة عند `hi`.
2) ما دام `hi - lo > 0.001`: `mid = (lo + hi) / 2`.
3) إذا كان `gap(mid) > 0` فالجذر على اليمين: `lo = mid`، وإلا `hi = mid`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double lo, hi, mid, gap;
    int it = 0;
    scanf("%lf %lf", &lo, &hi);
    while (hi - lo > 0.001) {
        mid = (lo + hi) / 2;
        gap = 5000 / mid - (2 * mid - 20);  // الطلب ناقص العرض
        if (gap > 0)
            lo = mid;                       // الطلب أكبر: السعر يجب أن يرتفع
        else
            hi = mid;
        it++;                               // كل دورة تقسم المجال على 2
    }
    mid = (lo + hi) / 2;
    printf("Equilibrium price = %.3f DA (%d iterations)\n", mid, it);
    printf("Quantity = %.2f\n", 5000 / mid);
    return 0;
}
```

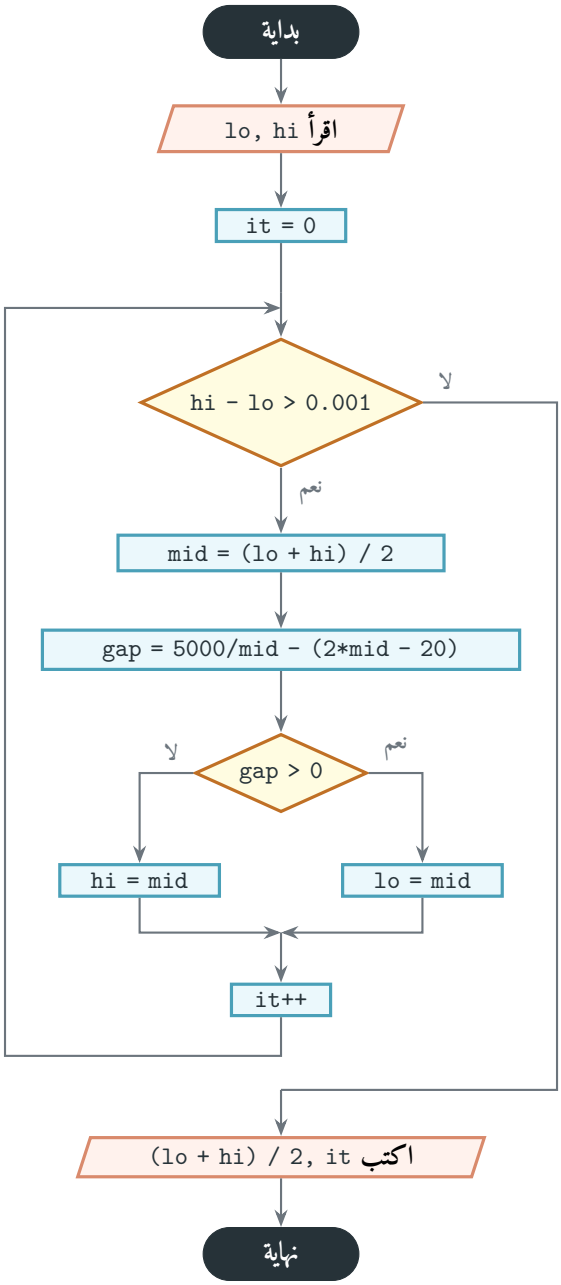
المدخلات

```
10 100
```

نتيجة التنفيذ

```
Equilibrium price = 55.250 DA (17 iterations)
Quantity = 90.50
```

المخطط الإنسيابي (Organigramme)



تمرين 3.37 الفائدة المستمرة بسلسلة تايلور

علم البيانات

متوسط

مع الفائدة المستمرة يصبح المبلغ `capital × e^(rt)`. اكتب برنامجًا يحسب `e^exponent` بجمع حدود سلسلة تايلور
`1 + exponent + exponent²/2! + exponent³/3! + ...`
حتى يصبح الحد أصغر من `1e-12`، دون حساب العامليات منفصلة.

خطوات الحل

1) `exponent = rate * years`، `term = 1`، `sum = 1`، `term_k = 0`.
2) الحد التالي من السابق: `term = term * exponent / term_k`.
3) نتوقف عندما يصبح `term < 1e-12`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double capital, rate, years, exponent, term = 1, sum = 1;
    int term_k = 0;
    scanf("%lf %lf %lf", &capital, &rate, &years);
    exponent = rate * years;
    do {
        term_k++;
        term = term * exponent / term_k;      // من الحد السابق !term_k / term_k^exponent
        sum += term;
    } while (term > 1e-12);
    printf("e^%.2f = %.10f (%d terms)\n", exponent, sum, term_k);
    printf("Continuous: %.2f DA\n", capital * sum);
    return 0;
}
```

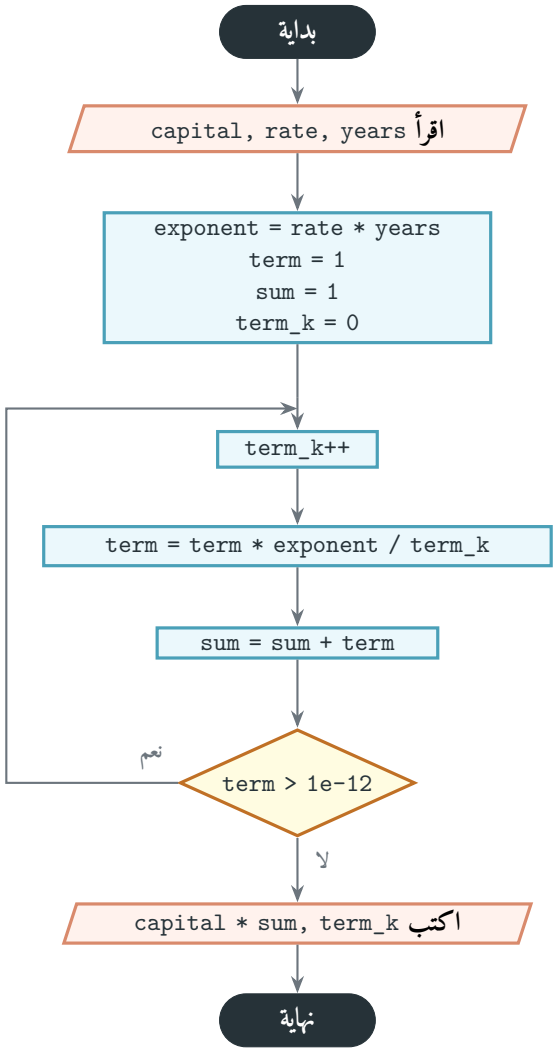
المدخلات

```
50000 0.05 10
```

نتيجة التنفيذ

```
e^0.50 = 1.6487212707 (12 terms)
Continuous: 82436.06 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.38 القفزات المفاجئة في الزيارات

علم البيانات ★★ متوسط

اكتب برنامجًا يقرأ عدد الزيارات لـ `nbHours` ساعة ويكشف **القفزات**: كل ساعة تختلف زياراتها عن الساعة السابقة بأكثر من 50%. نعرض رقم الساعة ونسبة التغير.

خطوات الحل

1) نحتفظ بالقيمة السابقة.
2) `change = (visits - prev) / prev * 100` ثم قيمتها المطلقة.
3) إذا كانت `> 50` نعرض تنبيهًا ونعد القفزات.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbHours, hour, jumps = 0;
    double prev, visits, change, ac;
```

```c
    scanf("%d %lf", &nbHours, &prev);
    for (hour = 2; hour <= nbHours; hour++) {
        scanf("%lf", &visits);
        change = (visits - prev) / prev * 100;  // التغير النسبي
        ac = (change < 0) ? -change : change;
        if (ac > 50) {
            printf("Hour %d: jump of %+.1f %%\n", hour, change);
            jumps++;
        }
        prev = visits;
    }
    printf("Total jumps: %d\n", jumps);
    return 0;
}
```

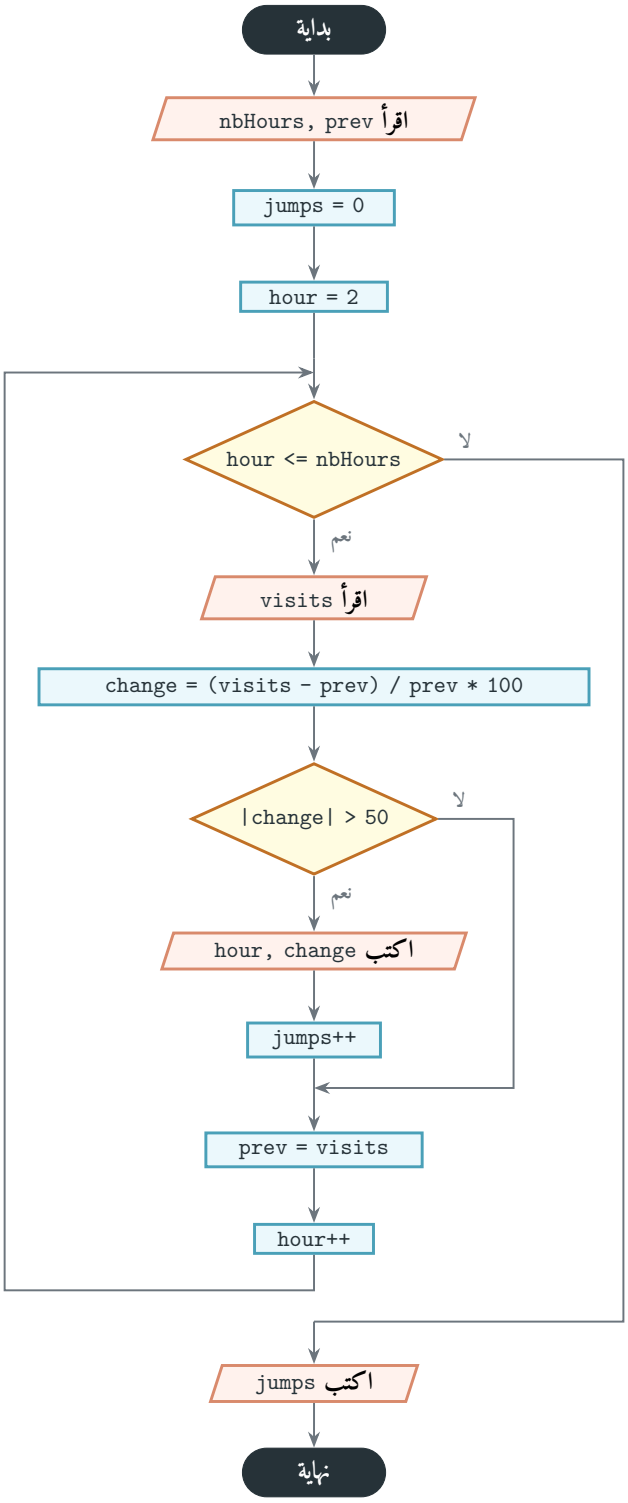
المدخلات

```
8
400 420 900 880 300 310 650 640
```

نتيجة التنفيذ

```
Hour 3: jump of +114.3 %
Hour 5: jump of -65.9 %
Hour 7: jump of +109.7 %
Total jumps: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 3.39 تخمين السعر بالبحث الثنائي

تجارة إلكترونية

متوسط

في لعبة ترويجية يخفي المتجر سعراً بين 1 و100000. اكتب برنامجًا يقرأ السعر المخفي ثم يجعل الحاسوب **يخمنه** بالبحث الثنائي: في كل محاولة يقترح المنتصف ويُقال له "أكبر" أو "أصغر". اعرض المحاولات.

خطوات الحل

1) `lo = 1`، `hi = 100000`.
2) `guess = (lo + hi) / 2`؛ إذا كان أصغر من السعر: `lo = guess + 1`، إذا كان أكبر: `hi = guess - 1`.
3) نتوقف عند الإصابة؛ عدد المحاولات لا يتجاوز 17.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int secret, lo = 1, hi = 100000, guess, tries = 0;
    scanf("%d", &secret);
    do {
        guess = (lo + hi) / 2;     // نجرب منتصف المجال
        tries++;
        printf("Try %2d: %6d -> ", tries, guess);
        if (guess < secret) {
            printf("higher\n");
            lo = guess + 1;        // نتخلص من النصف الأيسر
        } else if (guess > secret) {
            printf("lower\n");
            hi = guess - 1;        // نتخلص من النصف الأيمن
        } else {
            printf("found!\n");
        }
    } while (guess != secret);
    printf("Found in %d tries\n", tries);
    return 0;
}
```

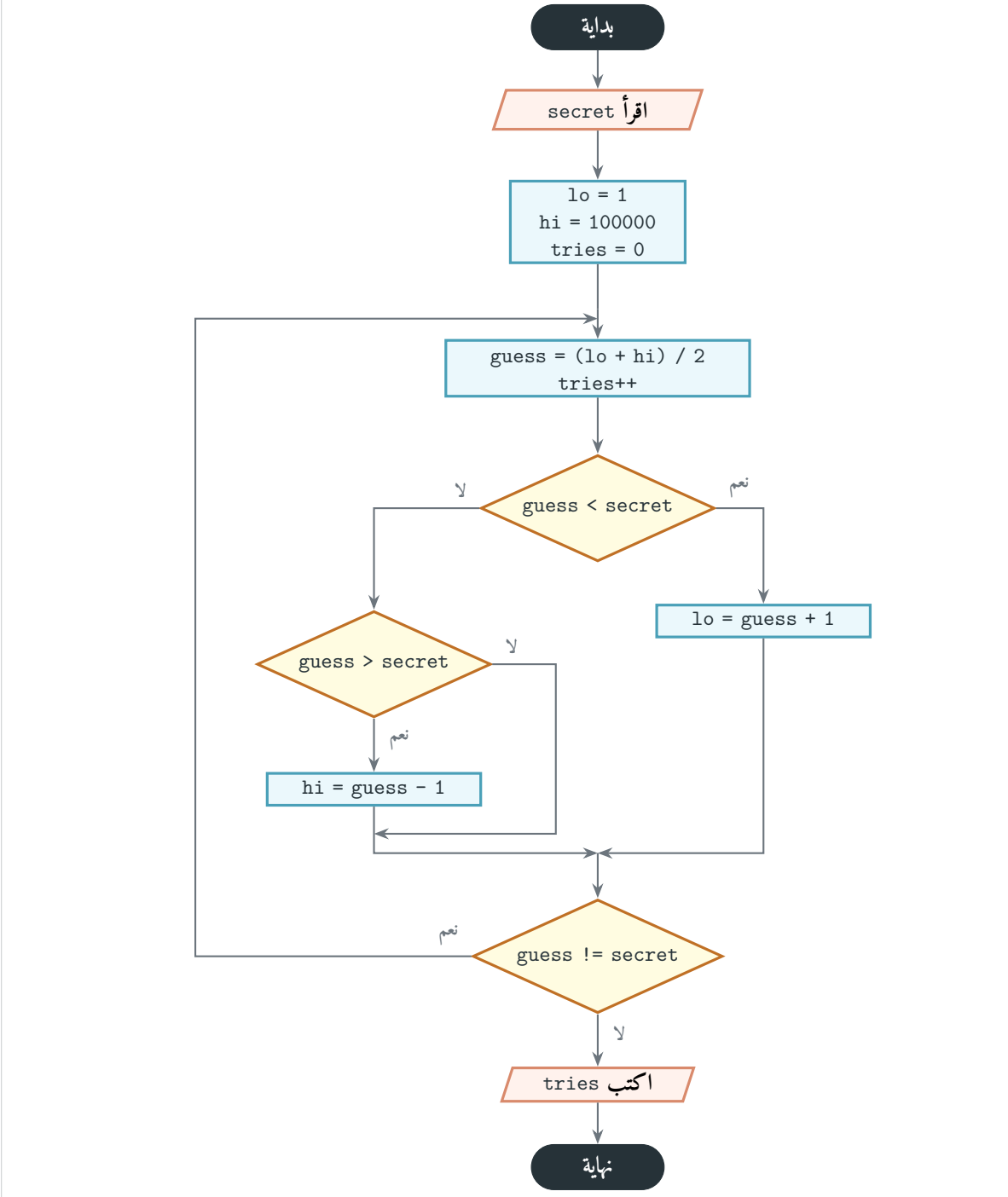
المدخلات

```
73421
```

نتيجة التنفيذ

```
Try  1:  50000 -> higher
Try  2:  75000 -> lower
Try  3:  62500 -> higher
Try  4:  68750 -> higher
Try  5:  71875 -> higher
Try  6:  73437 -> lower
Try  7:  72656 -> higher
Try  8:  73046 -> higher
Try  9:  73241 -> higher
Try 10:  73339 -> higher
Try 11:  73388 -> higher
Try 12:  73412 -> higher
Try 13:  73424 -> lower
Try 14:  73418 -> higher
Try 15:  73421 -> found!
Found in 15 tries
```

المخطط الإنسيابي (Organigramme)



تمرين 3.40 يوم بلوغ هدف رقم الأعمال

علم البيانات ★★ متوسط

هدف الشهر رقم أعمال `target`. اكتب برنامجًا يقرأ مبيعات الأيام (30 يومًا كحد أقصى) ويتوقف فور بلوغ الهدف باستعمال `break`، ويعرض اليوم؛ وإذا لم يُبلغ الهدف يعرض الناقص.

خطوات الحل

1) نجمع المبيعات في `cum`.

2) إذا كان `cum >= target` نحفظ اليوم ونخرج بـ `break`.

3) بعد الحلقة نميز الحالتين بالمتغير `reached`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double target, sales, cum = 0;
    int maxDays, day, reached = 0;
    scanf("%lf %d", &target, &maxDays);
    for (day = 1; day <= maxDays; day++) {
        if (scanf("%lf", &sales) != 1)  // نهاية البيانات المتوفرة
            break;
        cum += sales;
        if (cum >= target) {
            reached = day;
            break;                      // نخرج من الحلقة فورًا
        }
    }
    if (reached)
        printf("Target reached on day %d (%.0f DA)\n", reached, cum);
    else
        printf("Target missed by %.0f DA\n", target - cum);
    return 0;
}
```

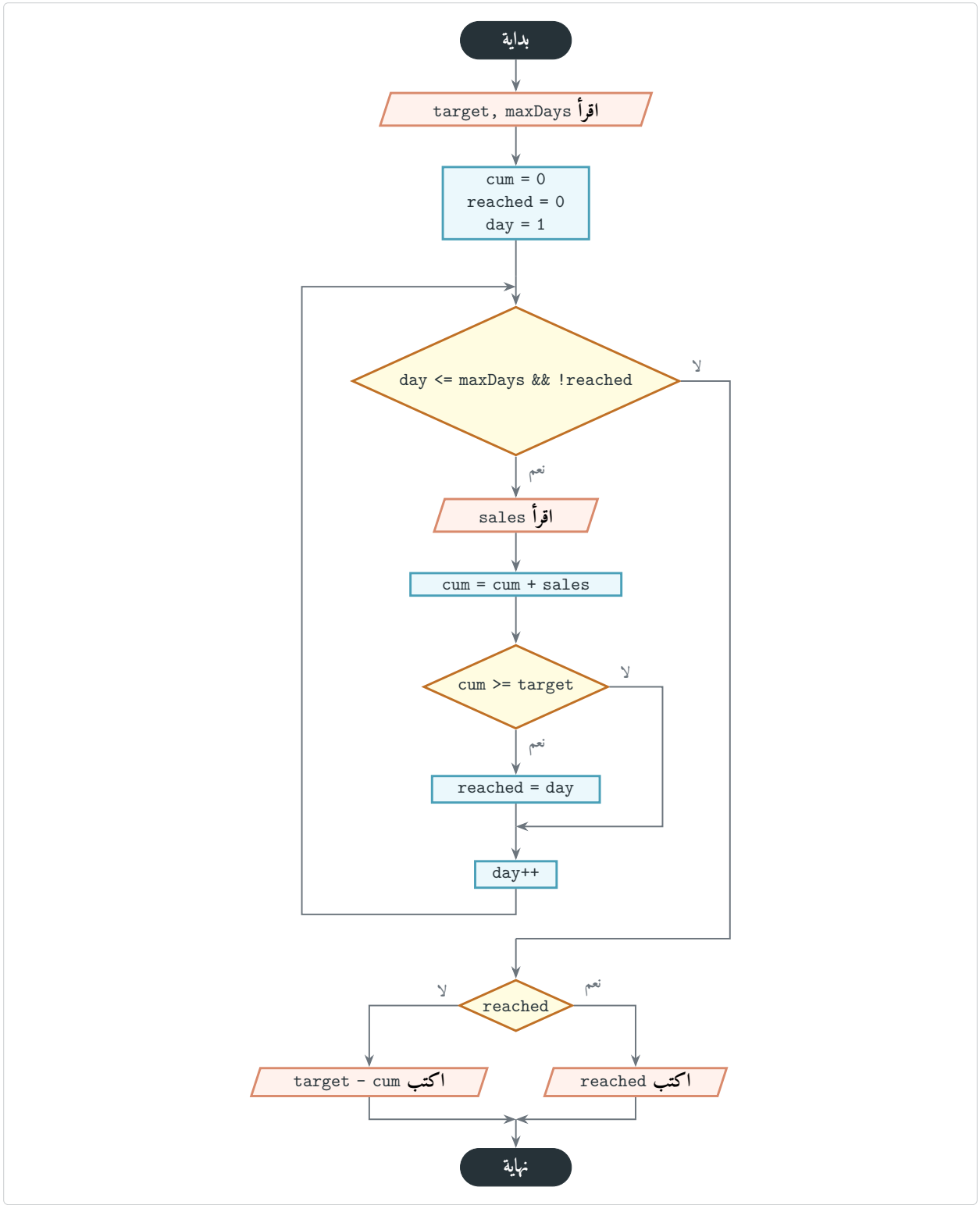
المدخلات

```
100000 30
8500 12000 9900 15000 7000 11000 13500 9000 14000 6000 3000 2000
```

نتيجة التنفيذ

```
Target reached on day 10 (105900 DA)
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 3.41 الأرقام الأولية حتى N

تجارة إلكترونية
صعب

اكتب برنامجًا يعرض كل الأعداد الأولية الأصغر من أو تساوي `limit` (لاستعمالها كأرقام خوادم)، ويعد عددها ومجموعها، باستعمال حلقتين متداخلتين واختبار القواسم الفردية فقط.

خطوات الحل

1) العدد 2 أولي، ثم نختبر الأعداد الفردية فقط `number = 3, 5, 7, ...`.
2) لكل `number`: حلقة داخلية على القواسم الفردية `divisor` ما دام `divisor * divisor <= number`.
3) نتوقف عند أول قاسم (`isp = 0`).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int limit, number, divisor, isp, count = 0, sum = 0;
    scanf("%d", &limit);
    if (limit >= 2) {                    // 2 هو العدد الأولي الزوجي الوحيد
        printf("2 ");
        count = 1; sum = 2;
    }
    for (number = 3; number <= limit; number += 2) {     // الأعداد الفردية فقط
        isp = 1;
        for (divisor = 3; isp && divisor * divisor <= number; divisor += 2)
            if (number % divisor == 0)
                isp = 0;                 // له قاسم: ليس أوليًا
        if (isp) {
            printf("%d ", number);
            count++;
            sum += number;
        }
    }
    printf("\n%d primes, sum = %d\n", count, sum);
    return 0;
}
```

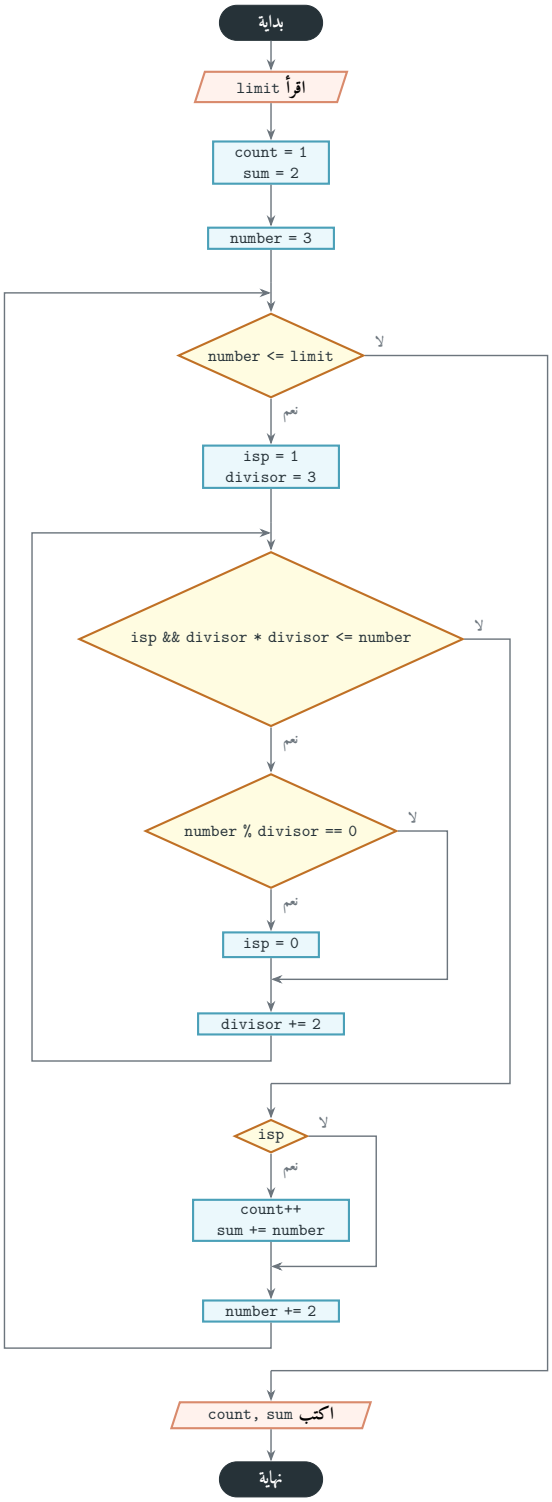
المدخلات

```
100
```

نتيجة التنفيذ

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
25 primes, sum = 1060
```

المخطط الإنسيابي (Organigramme)



تمرين 3.42 التجميع بـ K-Means في بعد واحد

علم البيانات صعب ★★★★

نريد تقسيم 20 زبونًا إلى **مجموعتين** حسب إنفاقهم بخوارزمية **K-Means**. الإنفاق مولَّد بصيغة:

```
spend(customer) = (customer * 37) % 50 + (customer % 2) * 60
```

لـ `customer` من 0 إلى 19 (لا نخزنها، نعيد حسابها). نبدأ بمركزين `c1` و`c2` مقروءين ونكرر: نسند كل نقطة لأقرب مركز ثم نحسب المراكز الجديدة كمتوسطات، حتى الاستقرار.

خطوات الحل

1) في كل تكرار: نمر على النقاط، إذا كانت `|spend - c1| <= |spend - c2|` تنتمي للمجموعة 1.
2) نجمع مجموع وعدد كل مجموعة، ثم المركز الجديد = المتوسط.
3) نتوقف عندما لا يتغير المركزان (أو بعد 20 تكرارًا).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double c1, c2, m1, m2, s1, s2, spend, d1, d2;
    int customer, n1, n2, it = 0, changed = 1;
    scanf("%lf %lf", &c1, &c2);           // المركزان الابتدائيان
    while (changed && it < 20) {
        s1 = s2 = 0; n1 = n2 = 0;
        for (customer = 0; customer < 20; customer++) {
            spend = (customer * 37) % 50 + (customer % 2) * 60;   // إنفاق الزبون customer
            d1 = spend - c1; if (d1 < 0) d1 = -d1;  // المسافة إلى c1
            d2 = spend - c2; if (d2 < 0) d2 = -d2;  // المسافة إلى c2
            if (d1 <= d2) { s1 += spend; n1++; }    // أقرب إلى c1
            else          { s2 += spend; n2++; }
        }
        m1 = (n1 > 0) ? s1 / n1 : c1;   // المركز الجديد = المتوسط
        m2 = (n2 > 0) ? s2 / n2 : c2;
        changed = (m1 != c1 || m2 != c2);
        c1 = m1; c2 = m2;
        it++;
        printf("Iter %d: c1 = %.2f (%d), c2 = %.2f (%d)\n", it, c1, n1, c2, n2);
    }
    printf("Converged after %d iterations\n", it);
    return 0;
}
```

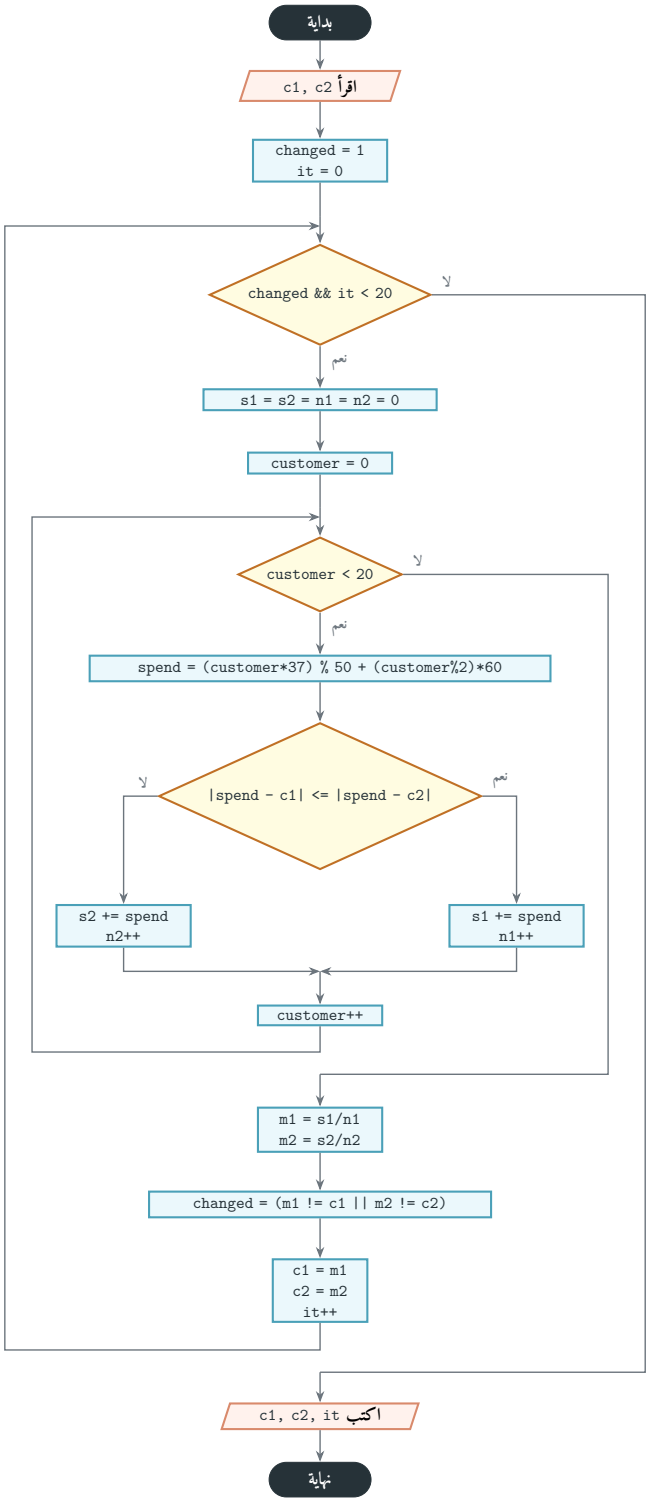
المدخلات

```
10 20
```

نتيجة التنفيذ

```
Iter 1: c1 = 0.00 (1), c2 = 56.84 (19)
Iter 2: c1 = 16.67 (6), c2 = 70.00 (14)
Iter 3: c1 = 20.29 (7), c2 = 72.15 (13)
Iter 4: c1 = 25.78 (9), c2 = 77.09 (11)
Iter 5: c1 = 28.00 (10), c2 = 80.00 (10)
Iter 6: c1 = 28.00 (10), c2 = 80.00 (10)
Converged after 6 iterations
```

المخطط الإنسيابي (Organigramme)



تمرين 3.43 محاكاة مونت كارلو بمولد عشوائي يدوي — علم البيانات — صعب ★★★

دون الدالة `rand`، نبني **مولد أعداد شبه عشوائية خطيًا** (LCG):

```
seed = (seed * 1103515245 + 12345) % 2147483648
```

و `uniform = seed / 2147483648.0`، عدد بين 0 و1. كل زبون يشتري 1 إلى 5 قطع

(1 + (int)(uniform * 5)) وكل قطعة سعرها بين 500 و3000. اكتب برنامجًا يحاكي nbBaskets سلة ويقدر احتمال أن تتجاوز السلة 6000 دج ومتوسط السلة.

خطوات الحل

1) نستعمل unsigned long long للمولد لتجنب تجاوز السعة.
2) لكل سلة: نولد عدد القطع، ثم لكل قطعة نولد سعراً 500 + uniform * 2500.
3) نعد السلات التي تتجاوز 6000 ونجمع المبالغ.
4) الاحتمال ≈ count / nbBaskets.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long seed;
    int nbBaskets, basketNo, item, nbItems, big = 0;
    double uniform, basket, total = 0;
    scanf("%llu %d", &seed, &nbBaskets);
    for (basketNo = 0; basketNo < nbBaskets; basketNo++) {
        seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
        uniform = seed / 2147483648.0;          // عدد شبه عشوائي في [0, 1)
        nbItems = 1 + (int)(uniform * 5);       // عدد القطع: 1 إلى 5
        basket = 0;
        for (item = 0; item < nbItems; item++) {
            seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
            uniform = seed / 2147483648.0;
            basket += 500 + uniform * 2500;     // سعر بين 500 و 3000
        }
        total += basket;
        if (basket > 6000)
            big++;
    }
    printf("P(basket > 6000) ~ %.4f\n", (double)big / nbBaskets);
    printf("Mean basket ~ %.2f DA\n", total / nbBaskets);
    return 0;
}
```

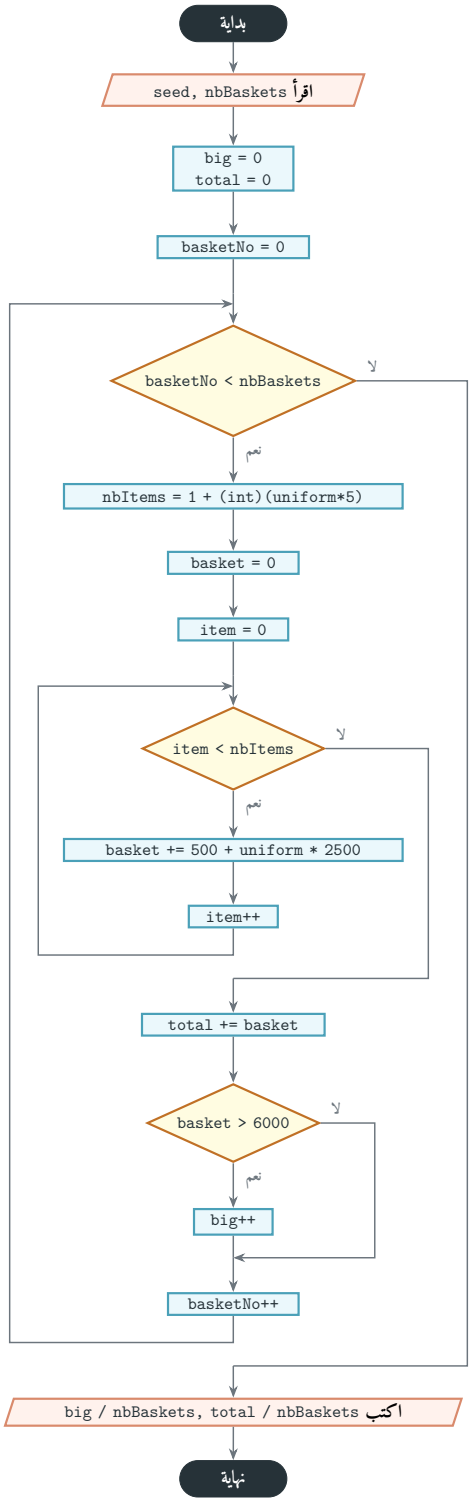
المدخلات

```
2026 20000
```

نتيجة التنفيذ

```
P(basket > 6000) ~ 0.3942
Mean basket ~ 5239.69 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.44 الإنحدار بالنزول التدريجي

علم البيانات صعب ★★★★

بدل الصيغة المغلقة، ندرب نموذج `target = weight feature + bias` بخوارزمية **النزول التدريجي** (Gradient Descent). البيانات: `feature(index) = index` و

```
target(index) = 3i + 7 + ((index * 7) % 5 - 2)
```

لـ `index` من 0 إلى 9. في كل حقبة نحسب المشتقتين `dw = (2/nbPoints) Σ (ŷ - target) feature`

اكتب برنامجًا يدرب النموذج ويعرض الخطأ. db = (2/nbPoints) Σ (ŷ - target) ثم weight -= lr * dw و bias -= lr * db.

خطوات الحل

1) نهيئ weight = 0 و bias = 0.
2) لكل حقبة: نمر على النقاط ونجمع التدرجين والخطأ التربيعي.
3) نحدّث المعاملين بمعدل التعلم lr.
4) نعرض التقدم كل 500 حقبة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double lr, weight = 0, bias = 0, dw, db, mse, feature, target, err;
    int epochs, epoch, index, nbPoints = 10;
    scanf("%lf %d", &lr, &epochs);
    for (epoch = 1; epoch <= epochs; epoch++) {
        dw = db = mse = 0;
        for (index = 0; index < nbPoints; index++) {
            feature = index;
            target = 3 * index + 7 + ((index * 7) % 5 - 2);  // بيانات مع ضجيج
            err = weight * feature + bias - target;          // خطأ التنبؤ
            dw += err * feature;                             // مساهمة النقطة في التدرج
            db += err;
            mse += err * err;
        }
        weight -= lr * 2 * dw / nbPoints;     // خطوة في عكس اتجاه التدرج
        bias -= lr * 2 * db / nbPoints;
        if (epoch % 500 == 0)
            printf("Epoch %4d: weight = %.4f, bias = %.4f, MSE = %.4f\n", epoch, weight, bias, mse /
      ↪ nbPoints);
    }
    return 0;
}
```

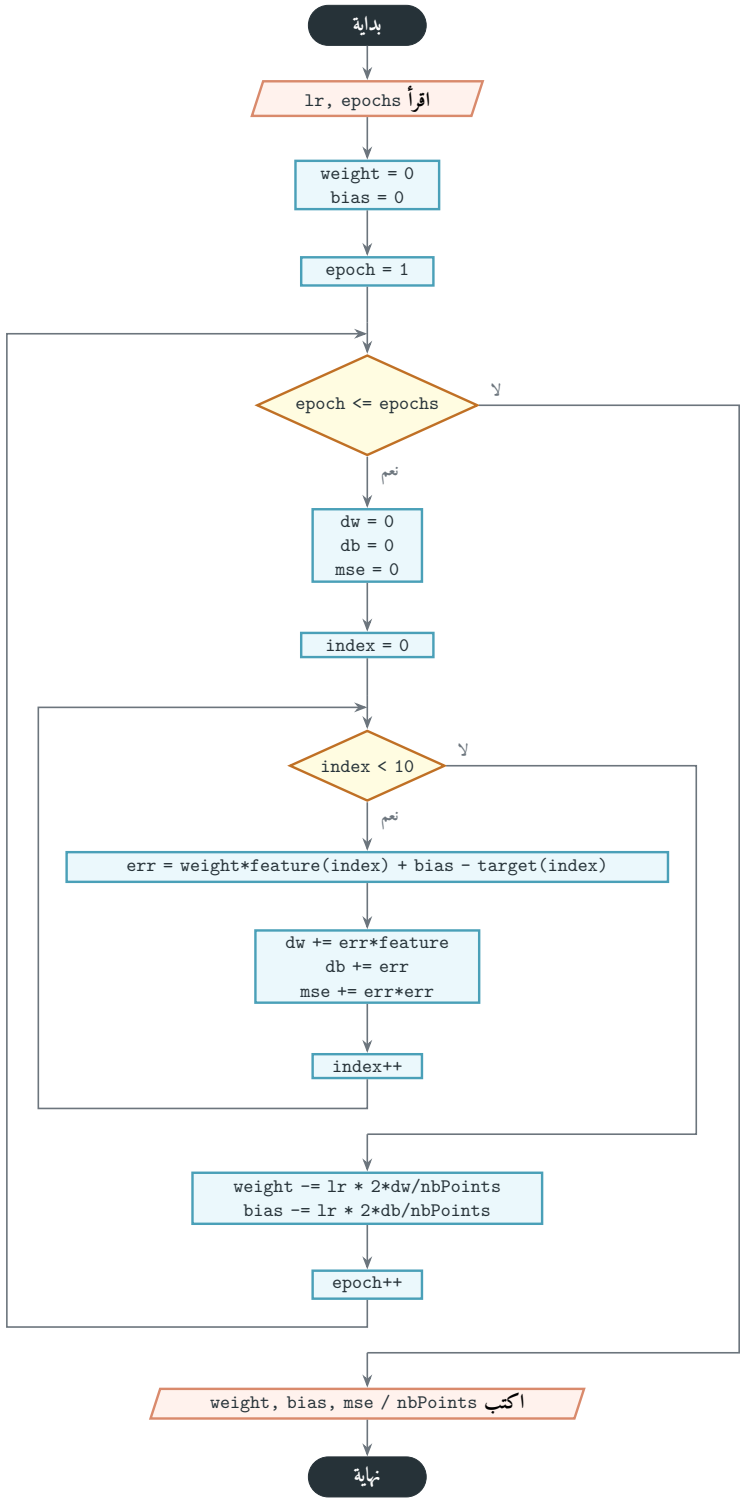
المدخلات

```
0.01 3000
```

نتيجة التنفيذ

```
Epoch  500: weight = 3.1758, bias = 6.1123, MSE = 1.9131
Epoch 1000: weight = 3.1244, bias = 6.4344, MSE = 1.8789
Epoch 1500: weight = 3.1214, bias = 6.4534, MSE = 1.8788
Epoch 2000: weight = 3.1212, bias = 6.4545, MSE = 1.8788
Epoch 2500: weight = 3.1212, bias = 6.4545, MSE = 1.8788
Epoch 3000: weight = 3.1212, bias = 6.4545, MSE = 1.8788
```

المخطط الإنسيابي (Organigramme)



تمرين 3.45 التحقق من رقم البطاقة بخوارزمية Luhn

تجارة إلكترونية

صعب ★★★★

قبل إرسال الدفع للبنك نتحقق من رقم البطاقة بخوارزمية **Luhn**: من اليمين، نضاعف رقمًا من كل اثنين (الثاني، الرابع، ...)، وإذا تجاوز الناتج 9 نطرح 9، ثم نجمع كل الأرقام؛ الرقم صالح إذا كان المجموع يقبل القسمة على 10. اكتب البرنامج لرقم من 16 خانة.

خطوات الحل

1) نقرأ الرقم في `unsigned long long`.
2) حلقة: `digit = number % 10` ثم `number /= 10`، مع موضع `pos` يبدأ من 1 من اليمين.
3) إذا كان `pos` زوجيًا: `digit = 2d` ثم `digit -= 9` إذا تجاوز 9.
4) نجمع، ونتحقق من `sum % 10 == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long number;
    int digit, sum = 0, pos = 1;
    scanf("%llu", &number);
    while (number > 0) {
        digit = number % 10;                // نقرأ الأرقام من اليمين
        number = number / 10;
        if (pos % 2 == 0) {                 // رقم من كل اثنين
            digit = 2 * digit;
            if (digit > 9)
                digit = digit - 9;          // = مجموع رقمي الناتج
        }
        sum += digit;
        pos++;
    }
    printf("Luhn sum = %d\n", sum);
    if (sum % 10 == 0)
        printf("Card number is valid\n");
    else
        printf("Card number is INVALID\n");
    return 0;
}
```

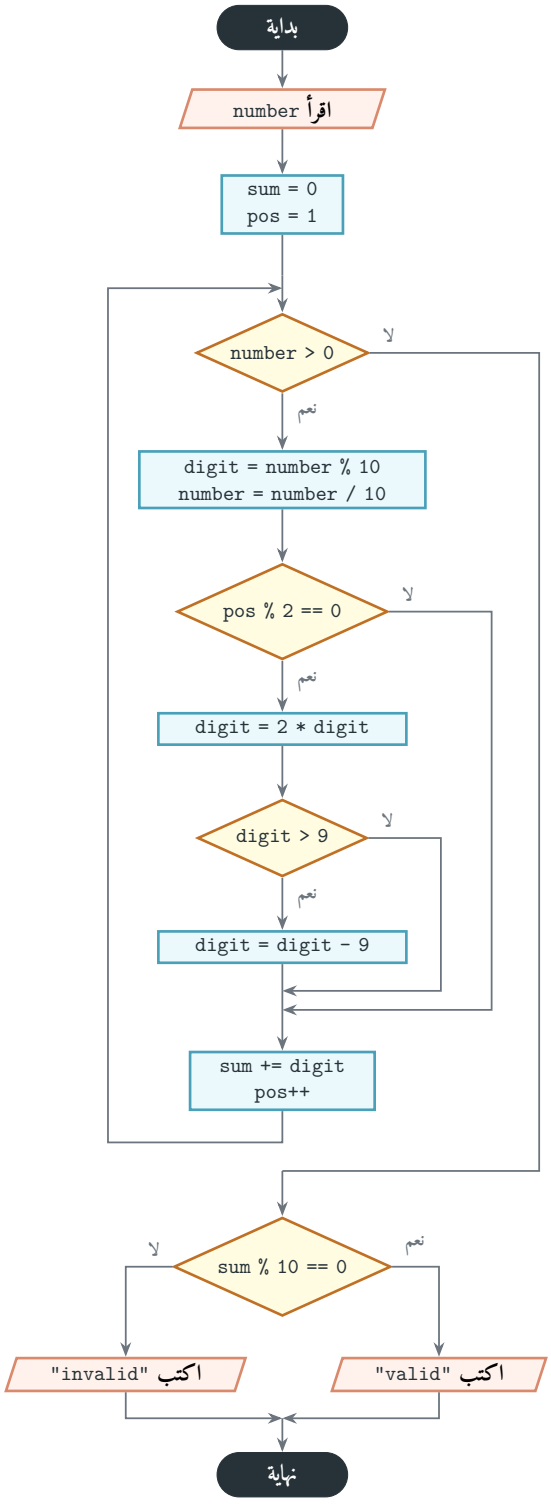
المدخلات

```
4539578763621486
```

نتيجة التنفيذ

```
Luhn sum = 80
Card number is valid
```

المخطط الإنسيابي (Organigramme)



تمرين 3.46 مثلث باسكال لعدد الحزم

تجارة إلكترونية

صعب ★★★★

يعطي **مثلث باسكال** في السطر `row` والعمود `col` عدد الحزم `C(row,col)`. اكتب برنامجًا يعرض الأسطر من 0 إلى `maxRow`، ويحسب كل عنصر من سابقه في نفس السطر:

```
C(row,col) = C(row,col-1) * (row - col + 1) / col
```

، ثم يعرض مجموع كل سطر (يساوي `2^row`).

خطوات الحل

1) حلقة خارجية على السطر `row`.
2) في كل سطر: `coef = 1`، وحلقة داخلية تحدّث `coef` وتطبعه.
3) نجمع عناصر السطر للتحقق.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int maxRow, row, col;
    long coef, rowSum;
    scanf("%d", &maxRow);
    for (row = 0; row <= maxRow; row++) {
        coef = 1; rowSum = 1;
        printf("n=%2d: 1", row);
        for (col = 1; col <= row; col++) {
            coef = coef * (row - col + 1) / col;  // العنصر من سابقه، القسمة دقيقة
            rowSum += coef;
            printf(" %ld", coef);
        }
        printf("  (sum %ld)\n", rowSum);   // 2 = أس row
    }
    return 0;
}
```

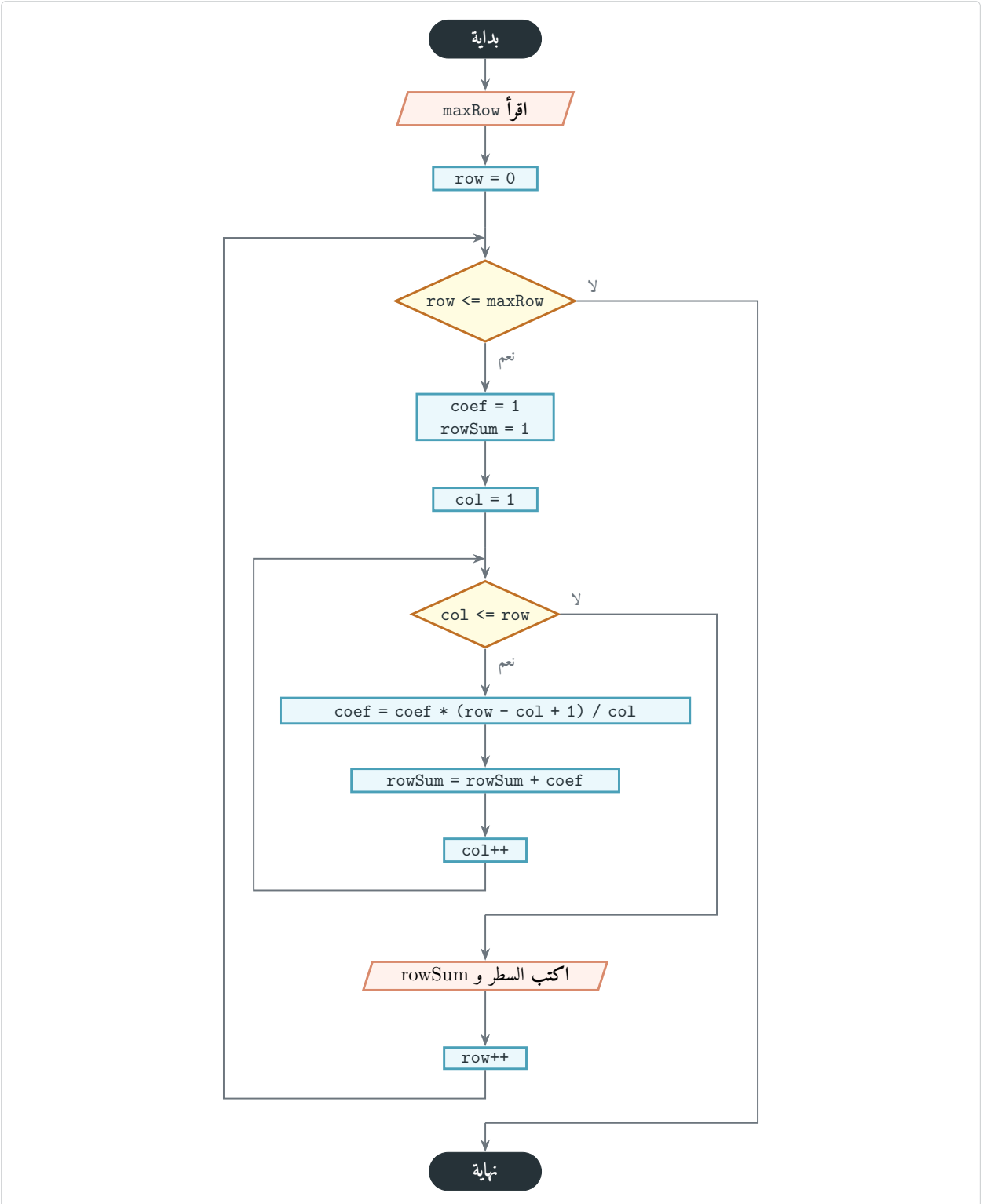
المدخلات

```
10
```

نتيجة التنفيذ

```
n= 0: 1  (sum 1)
n= 1: 1 1  (sum 2)
n= 2: 1 2 1  (sum 4)
n= 3: 1 3 3 1  (sum 8)
n= 4: 1 4 6 4 1  (sum 16)
n= 5: 1 5 10 10 5 1  (sum 32)
n= 6: 1 6 15 20 15 6 1  (sum 64)
n= 7: 1 7 21 35 35 21 7 1  (sum 128)
n= 8: 1 8 28 56 70 56 28 8 1  (sum 256)
n= 9: 1 9 36 84 126 126 84 36 9 1  (sum 512)
n=10: 1 10 45 120 210 252 210 120 45 10 1  (sum 1024)
```

المخطط الإنسيابي (Organigramme)



تمرين 3.47 التحويل إلى النظام الست عشري

تجارة إلكترونية

صعب

تُعرض أرقام الشحنات بالنظام **الست عشري** (0-9 ثم A-F). اكتب برنامجًا يحول عددًا موجبًا إلى الست عشري دون جداول ودون `printf("%X")`: نحدد أولًا أكبر قوة للعدد 16 لا تتجاوز العدد، ثم نستخرج الأرقام من اليسار إلى اليمين.

خطوات الحل

1) `power = 1`، وما دام `power * 16 <= number`: `power *= 16`.
2) ما دام `power > 0`: الرقم `digit = number / power`، ثم `number %= power` و`power /= 16`.
3) نطبع `'0' + digit` إذا كان `digit < 10`، وإلا `'A' + digit - 10`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    unsigned long long number, power = 1;
    int digit;
    scanf("%llu", &number);
    printf("%llu = 0x", number);
    while (power * 16 <= number)           // أكبر قوة لـ 16 لا تتجاوز number
        power *= 16;
    while (power > 0) {
        digit = number / power;                // الرقم الحالي من اليسار
        number = number % power;
        if (digit < 10)
            putchar('0' + digit);      // 0..9
        else
            putchar('A' + digit - 10);// A..F
        power /= 16;
    }
    printf("\n");
    return 0;
}
```

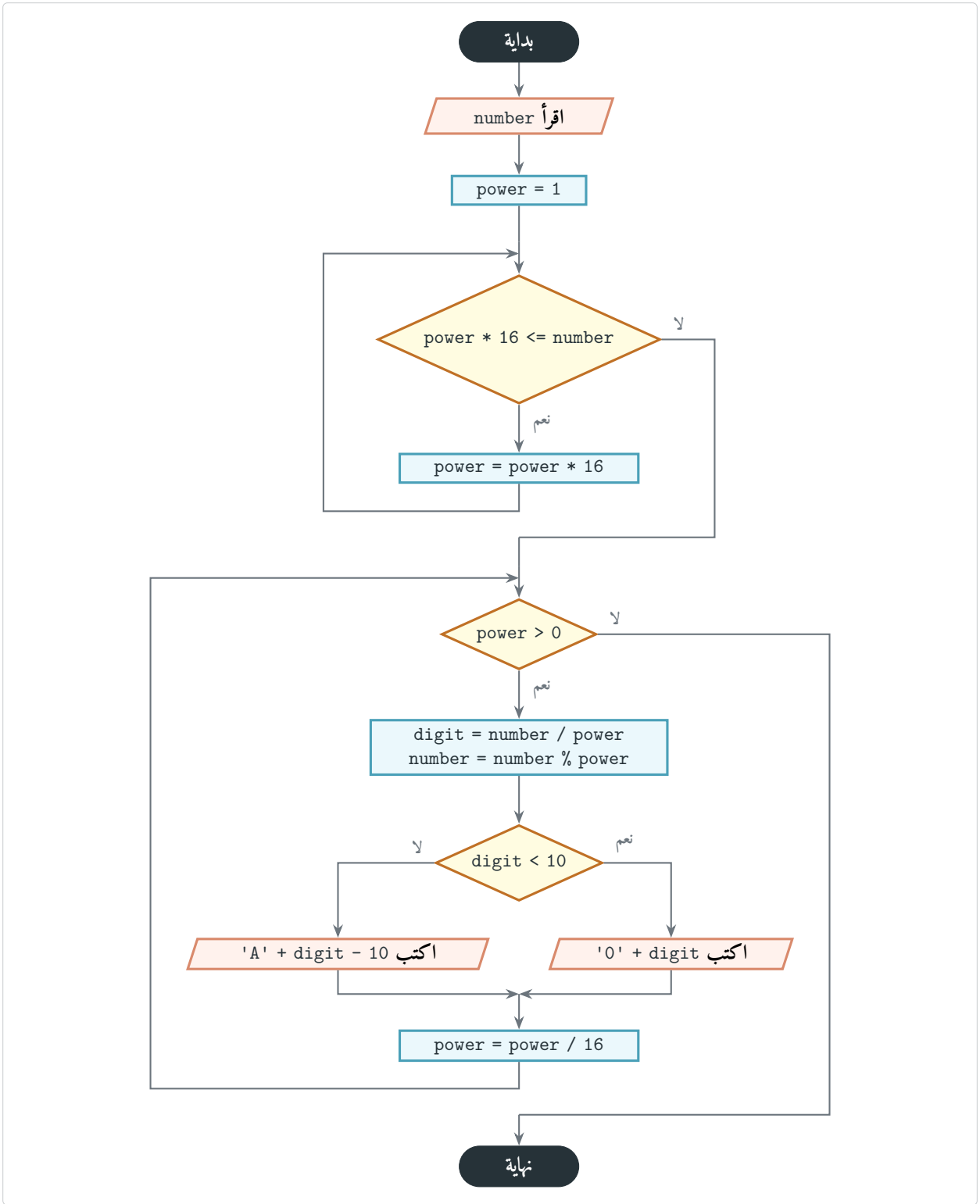
المدخلات

```
3054198966
```

نتيجة التنفيذ

```
3054198966 = 0xB60B60B6
```

المخطط الإنسيابي (Organigramme)



تمرين 3.48 الجذر الصحيح بالبحث الثنائي

علم البيانات ★★★★ صعب

لتنظيم صور المنتجات في **شبكة مربعة**، نحتاج أكبر عدد صحيح `r` بحيث `r² <= number`. اكتب برنامجًا يجده بالبحث الثنائي (دون أعداد حقيقية)، ثم يحدد إن كان `number` مربعًا كاملًا، وأبعاد الشبكة اللازمة `r × c`.

خطوات الحل

1) `lo = 0`، `hi = number` (أو مليارين كحد أقصى).
2) `mid = lo + (hi - lo) / 2`؛ إذا كان `mid * mid <= number` فـ `ans = mid` و`lo = mid + 1`، وإلا `hi = mid - 1`.
3) عدد الأعمدة: `c = (number + r - 1) / r`، وإذا لم يكن مربعًا كاملًا نستعمل `r + 1` أسطر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long long number, lo, hi, mid, ans = 0;
    scanf("%lld", &number);
    lo = 0;
    hi = (number < 3000000000LL) ? number : 3000000000LL;  // حد أعلى آمن للمربع
    while (lo <= hi) {
        mid = lo + (hi - lo) / 2;      // يتجنب تجاوز السعة عند الجمع
        if (mid * mid <= number) {
            ans = mid;                 // مرشح صالح، نبحث عن أكبر
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    printf("isqrt(%lld) = %lld\n", number, ans);
    if (ans * ans == number)
        printf("Perfect square: grid %lld x %lld\n", ans, ans);
    else
        printf("Not a square: grid %lld x %lld\n", ans + 1, (number + ans) / (ans + 1));
    return 0;
}
```

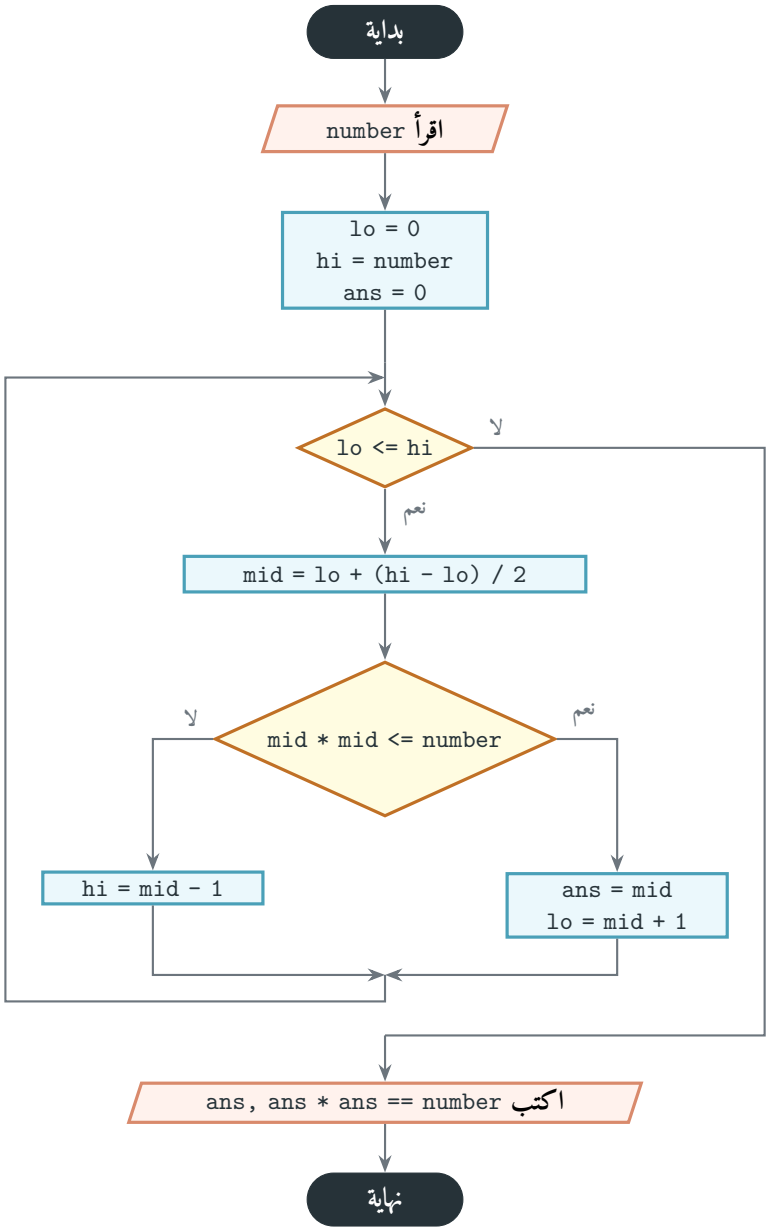
المدخلات

```
1000000007
```

نتيجة التنفيذ

```
isqrt(1000000007) = 31622
Not a square: grid 31623 x 31623
```

المخطط الإنسيابي (Organigramme)



تمرين 3.49 محاكاة سياسة التموين (s, S)

تجارة إلكترونية صعب ★★★

سياسة (s, S): في نهاية كل يوم، إذا نزل المخزون تحت `minLevel` نطلب ما يرفعه إلى `maxLevel`، ويصل الطلب بعد يومين. الطلب اليومي عشوائي بين 0 و39 بمولد LCG (`seed = (seed * 75) % 65537`, `demand = seed % 40`). التكاليف: 2 دج تخزين لكل وحدة في اليوم، 50 دج لكل وحدة ناقصة، و500 دج لكل طلبية. اكتب برنامجًا يحاكي 60 يومًا ويعرض التكاليف وعدد أيام النفاد.

خطوات الحل

1) متغيرات: `stock`، كمية الطلب المعلق `pending`، وأيام الانتظار `wait`.
2) كل يوم: نستقبل الطلب إذا حان وقته، نولد الطلب، نبيع ما نستطيع ونحسب النقص.
3) تكلفة التخزين على المخزون المتبقي.

4) إذا كان `stock < minLevel` ولا يوجد طلب معلق: نطلب `maxLevel - stock`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long minLevel, maxLevel, seed, stock, pending = 0, demand;
    int day, wait = 0, outdays = 0, orders = 0;
    double hold = 0, shortc = 0, orderc = 0;
    scanf("%ld %ld %ld", &minLevel, &maxLevel, &seed);
    stock = maxLevel;
    for (day = 1; day <= 60; day++) {
        if (pending > 0) {            // طلبية في الطريق
            wait--;
            if (wait == 0) {          // وصلت
                stock += pending;
                pending = 0;
            }
        }
        seed = (seed * 75) % 65537;   // مولد عشوائي يدوي
        demand = seed % 40;                // طلب اليوم 0..39
        if (demand > stock) {              // نفاد: الطلب أكبر من المخزون
            shortc += 50.0 * (demand - stock);
            stock = 0;
            outdays++;
        } else {
            stock -= demand;
        }
        hold += 2.0 * stock;          // تكلفة التخزين اليومية
        if (stock < minLevel && pending == 0) {
            pending = maxLevel - stock;      // نطلب ما يكمل إلى maxLevel
            wait = 2;
            orders++;
            orderc += 500;
        }
    }
    printf("Orders placed: %d\n", orders);
    printf("Stock-out days: %d\n", outdays);
    printf("Holding = %.0f, shortage = %.0f, ordering = %.0f\n", hold, shortc, orderc);
    printf("Total cost = %.0f DA\n", hold + shortc + orderc);
    return 0;
}
```

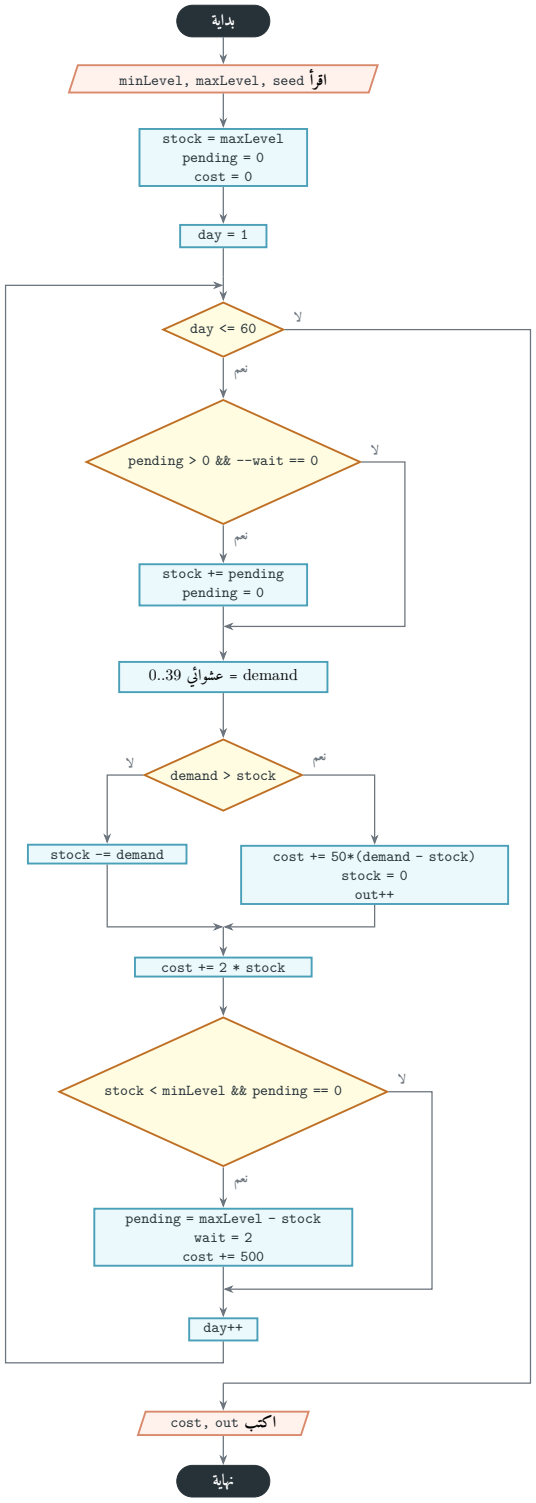
المدخلات

```
60 150 7
```

نتيجة التنفيذ

```
Orders placed: 10
Stock-out days: 0
Holding = 9132, shortage = 0, ordering = 5000
Total cost = 14132 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 3.50 أفضل يوم للشراء والبيع

علم البيانات صعب ★★★★

يتابع تاجر سعر منتج يوميًا لمدة `nbDays` يومًا ويريد **الشراء مرة والبيع مرة** لاحقًا لتحقيق أكبر ربح. اكتب برنامجًا يحل المسألة في **مرور واحد**: نحتفظ بأدنى سعر رأيناه حتى الآن وبيوم حدوثه.

خطوات الحل

1) `minP` = أدنى سعر حتى اليوم، و`minDay` يومه.
2) كل يوم: الربح الممكن إذا بعنا اليوم هو `price - minP`.
3) إذا كان أكبر من `best` نحفظه مع يومي الشراء والبيع.
4) ثم نحدّث `minP` إذا كان سعر اليوم أصغر.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int nbDays, day, minDay = 1, buy = 0, sell = 0;
    double price, minP, best = 0;
    scanf("%d %lf", &nbDays, &price);
    minP = price;
    for (day = 2; day <= nbDays; day++) {
        scanf("%lf", &price);
        if (price - minP > best) {    // البيع اليوم أفضل من كل ما سبق
            best = price - minP;
            buy = minDay;
            sell = day;
        }
        if (price < minP) {           // سعر شراء أفضل للأيام القادمة
            minP = price;
            minDay = day;
        }
    }
    if (best > 0)
        printf("Buy day %d, sell day %d: profit %.0f DA\n", buy, sell, best);
    else
        printf("No profitable trade\n");
    return 0;
}
```

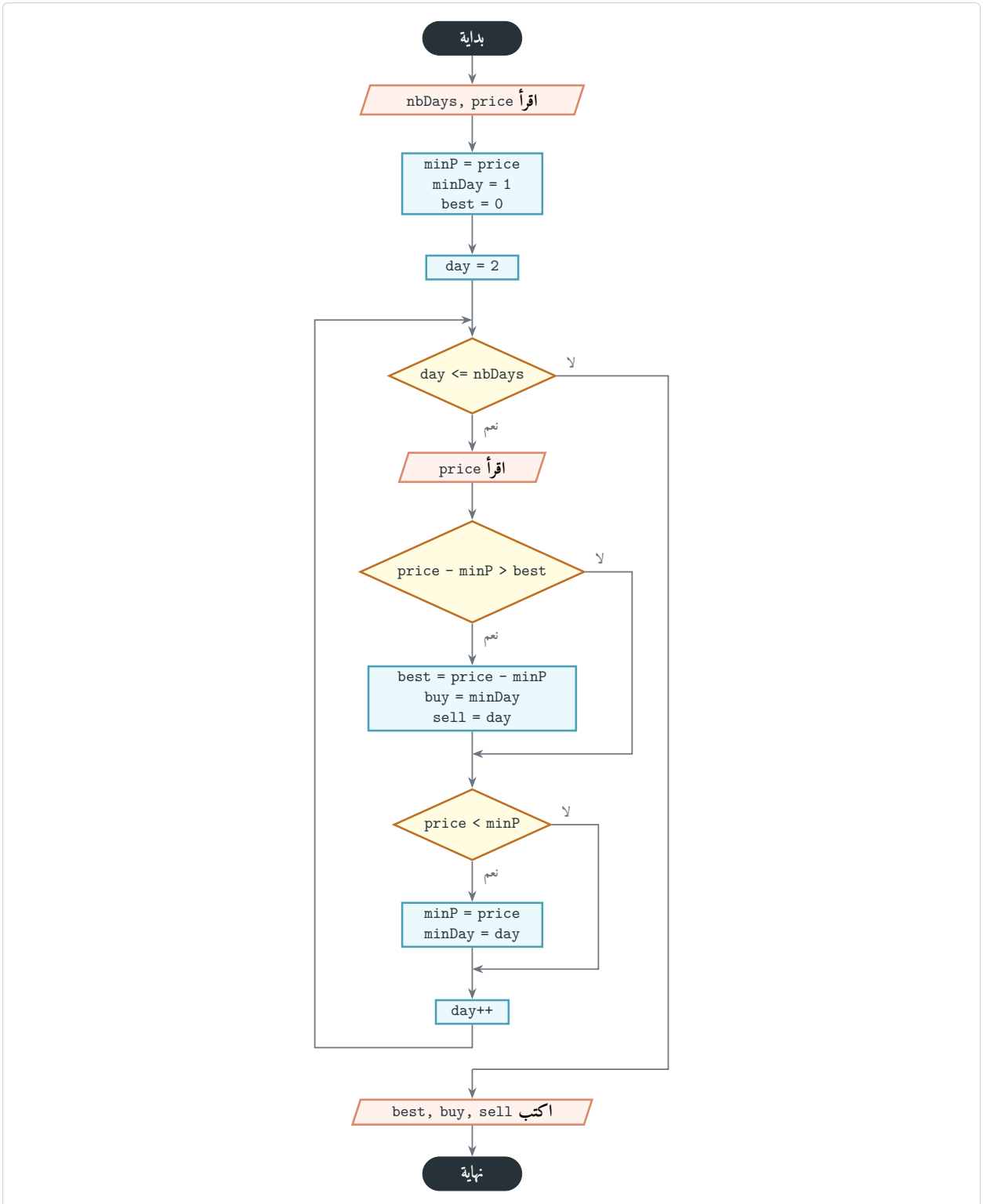
المدخلات

```
10
320 280 300 250 310 390 360 240 330 370
```

نتيجة التنفيذ

```
Buy day 4, sell day 6: profit 140 DA
```

المخطط الإنسيابي (Organigramme)



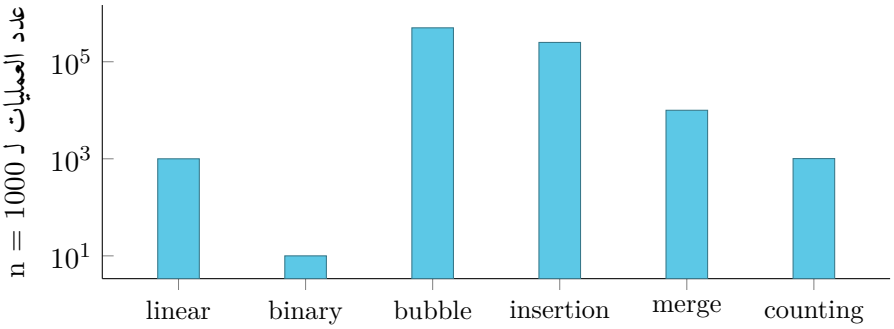
الفصل 4

الجداول أحادية البعد

تعريفات

- **الجدول**: مجموعة عناصر من نفس النوع متجاورة في الذاكرة، لها اسم واحد، ونصل إلى كل عنصر بفهرسه.
- الفهرس في C يبدأ من 0 وينتهي عند `SIZE - 1`.
- **التصريح**: `double prices[50];` **والتهيئة**: `int stock[4] = {5, 0, 12, 7};`
- **الحجم المستعمل** `nbItems` قد يكون أصغر من **السعة** `MAX`.





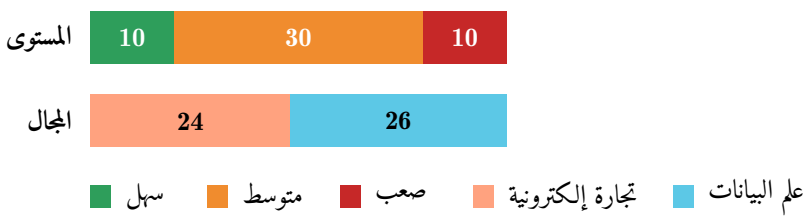
مقارنة تقريبية لتكلفة خوارزميات البحث والترتيب المستعملة في هذا الفصل (سلم لوغاريتمي).

تجاوز حدود الجدول

C لا يتحقق من الفهرس: الكتابة في `prices[50]` لجدول من 50 خانة تفسد الذاكرة دون أي رسالة خطأ.

للاستزادة: [7, 8, 5, 13]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 4.1 عرض كتالوج الأسعار

تجارة إلكترونية | سهل

اكتب برنامجًا يقرأ أسعار nbPrices منتجات (nbPrices <= 50) في جدول، ثم يعرض كل سعر مع فهرسه.

خطوات الحل

1) نصرح بجدول double prices[50].
2) حلقة أولى للقراءة: scanf("%lf", &prices[index]).
3) حلقة ثانية للعرض.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 50                    // السعة القصوى للجدول

int main(void) {
    double prices[MAX];           // جدول من 50 عددًا حقيقيًا
    int nbPrices, index;
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++)
        scanf("%lf", &prices[index]);      // index عنوان الخانة prices[index]&
    for (index = 0; index < nbPrices; index++)
        printf("T[%d] = %.2f DA\n", index, prices[index]);
    return 0;
}
```

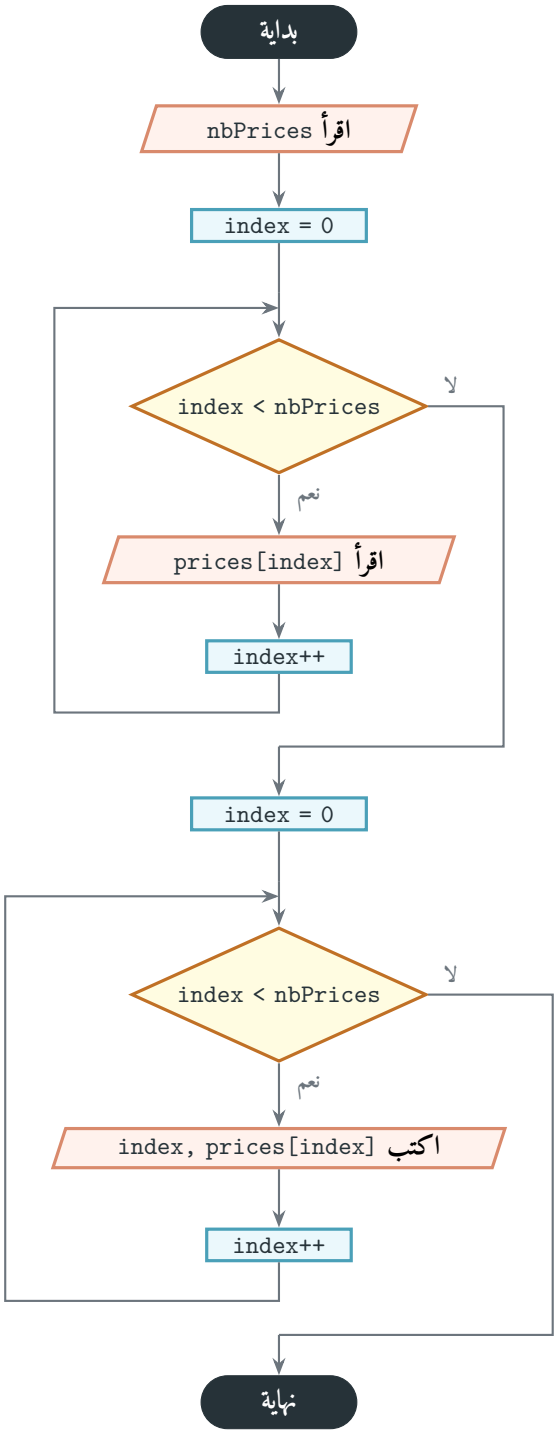
المدخلات

```
5
1200 450 3999 89.5 700
```

نتيجة التنفيذ

```
T[0] = 1200.00 DA
T[1] = 450.00 DA
T[2] = 3999.00 DA
T[3] = 89.50 DA
T[4] = 700.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.2 مبيعات الأسبوع

علم البيانات | سهل

مبيعات أسبوع مخزنة في جدول مهيأ مباشرة في البرنامج. اكتب برنامجًا يحسب **المجموع** و**المتوسط** وعدد الأيام التي تجاوزت المتوسط.

خطوات الحل

1) نهيئ الجدول `{...}` بسبع قيم.

2) حلقة أولى للمجموع، ثم المتوسط.

3) حلقة ثانية للعد: نحتاج المتوسط أولًا، لذلك نمر على الجدول مرتين.

البرنامج بلغة C

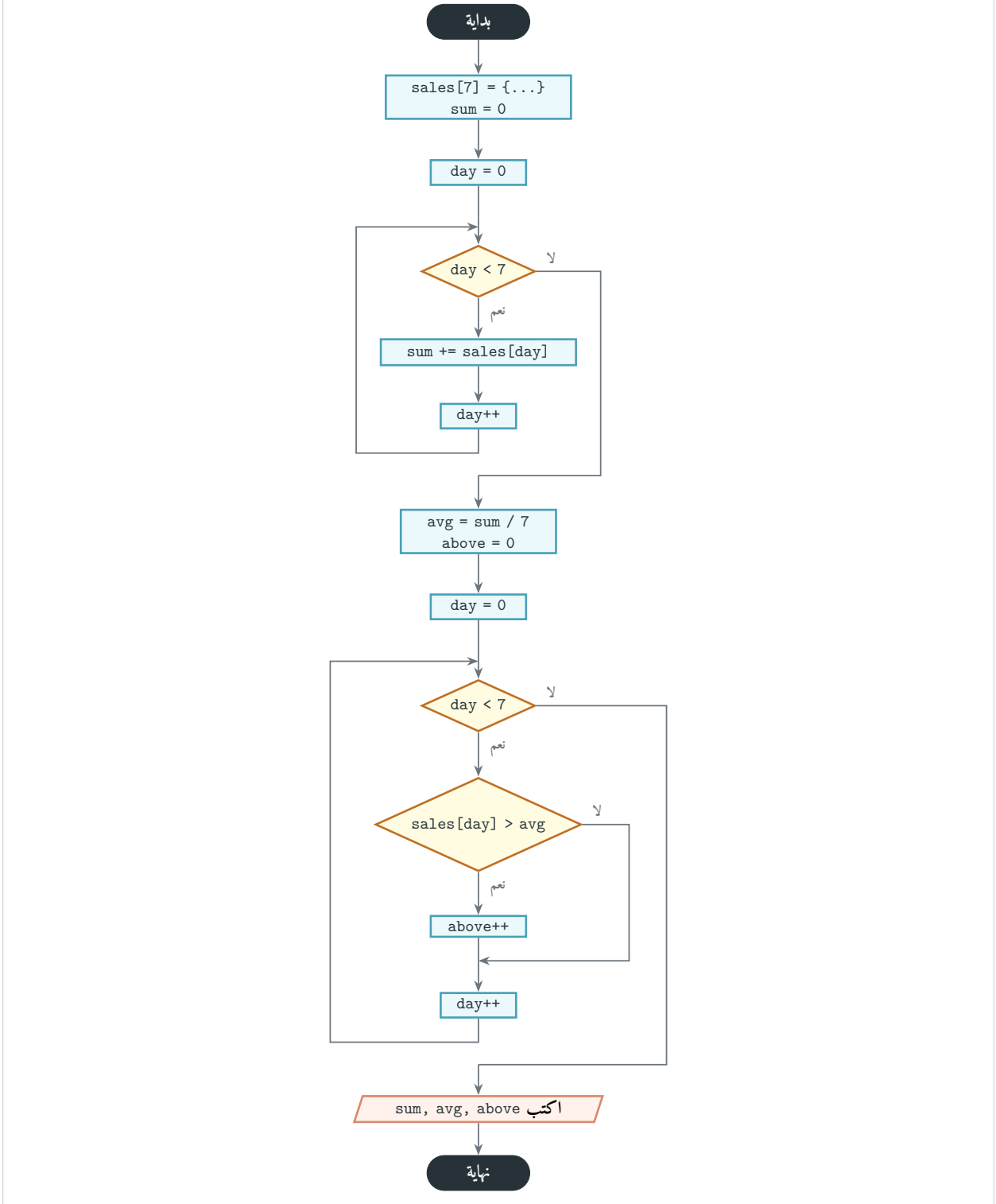
```c
#include <stdio.h>

int main(void) {
    double sales[7] = {120, 95, 140, 180, 160, 210, 75};   // مبيعات 7 أيام
    double sum = 0, avg;
    int day, above = 0;
    for (day = 0; day < 7; day++)
        sum += sales[day];
    avg = sum / 7;
    // مرور ثانٍ: لم يكن المتوسط معروفًا في المرور الأول
    for (day = 0; day < 7; day++)
        if (sales[day] > avg)
            above++;
    printf("Total = %.0f, average = %.2f\n", sum, avg);
    printf("Days above average: %d\n", above);
    return 0;
}
```

نتيجة التنفيذ

```
Total = 980, average = 140.00
Days above average: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 4.3 أكثر منتج مبيعًا

تجارة إلكترونية

سهل

يحتوي الجدول `sold` عدد الوحدات المباعة لكل منتج (الفهرس هو رقم المنتج). اكتب برنامجًا يجد **أكثر منتج مبيعًا** وفهرسه.

خطوات الحل

1) `best = 0` (فهرس الأكبر مبدئيًا).
2) لكل `index` من 1: إذا كان `sold[index] > sold[best]` فـ `best = index`.
3) نحفظ الفهرس فقط، فالقيمة هي `sold[best]`.

البرنامج بلغة C

```c
#include <stdio.h>

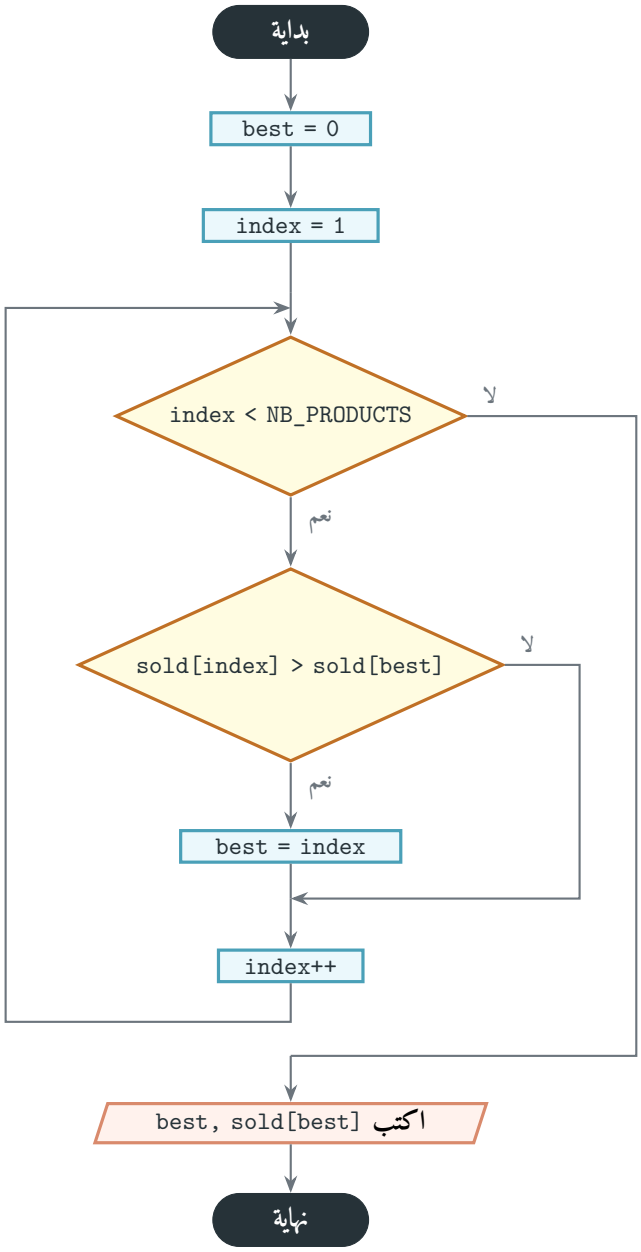
#define NB_PRODUCTS 8

int main(void) {
    int sold[NB_PRODUCTS] = {34, 120, 56, 98, 143, 12, 77, 143};
    int index, best = 0;              // نحفظ الفهرس، لا القيمة
    for (index = 1; index < NB_PRODUCTS; index++)
        if (sold[index] > sold[best])
            best = index;
    printf("Best seller: product #%d with %d units\n", best, sold[best]);
    return 0;
}
```

نتيجة التنفيذ

```
Best seller: product #4 with 143 units
```

المخطط الإنسيابي (Organigramme)



تمرين 4.4 المنتجات النافدة

تجارة إلكترونية

سهل

اكتب برنامجًا يعرض أرقام المنتجات التي مخزونها **0** من جدول المخزون، ويعد عددها.

خطوات الحل

1) نمر على الجدول.
2) إذا كان `stock[index] == 0` نعرض `index` ونزيد العداد.

البرنامج بلغة C

```c
#include <stdio.h>

```

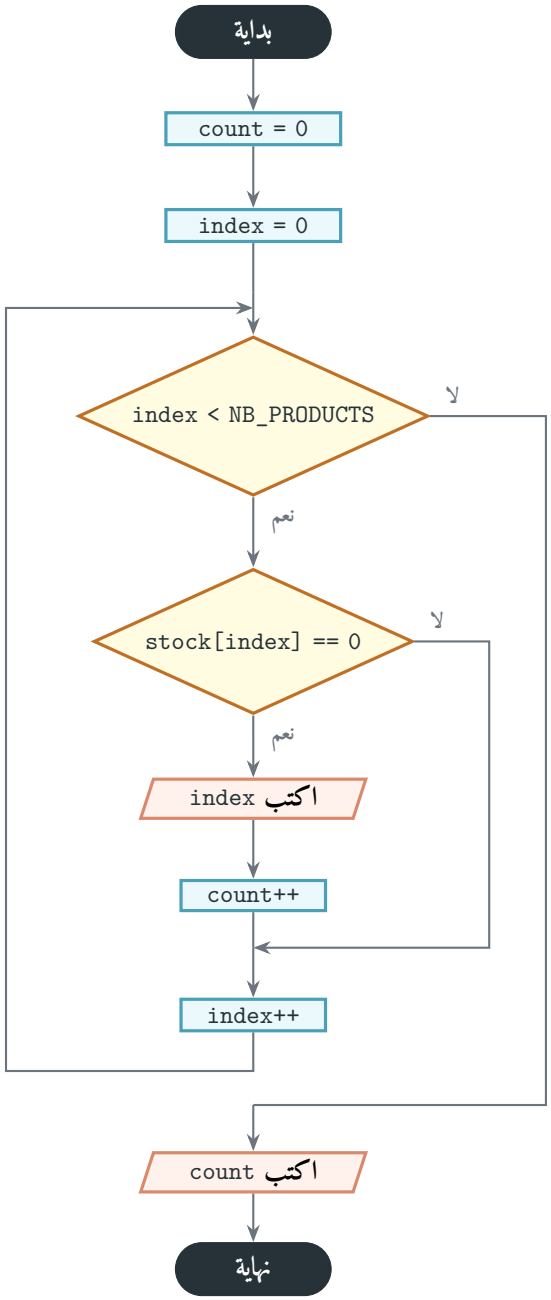
```c
#define NB_PRODUCTS 10

int main(void) {
    int stock[NB_PRODUCTS] = {5, 0, 12, 0, 7, 3, 0, 22, 1, 9};
    int index, count = 0;
    printf("Out of stock:");
    for (index = 0; index < NB_PRODUCTS; index++) {
        if (stock[index] == 0) {      // منتج نافد
            printf(" #%d", index);
            count++;
        }
    }
    printf("\n%d products to reorder\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
Out of stock: #1 #3 #6
3 products to reorder
```

المخطط الإنسيابي (Organigramme)



تمرين 4.5 رفع كل الأسعار

تجارة إلكترونية

سهل

بسبب ارتفاع تكاليف النقل، يرفع المتجر **كل** الأسعار بنسبة `percent`%. اكتب برنامجًا يعدل الجدول **في مكانه** ثم يعرضه.

خطوات الحل

1) نقرأ النسبة `percent`.
2) لكل خانة:

```
prices[index] = prices[index] * (1 + percent / 100)
```

.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_PRICES 5

int main(void) {
    double prices[NB_PRICES] = {1000, 2500, 499, 12000, 75};
    double percent;
    int index;
    scanf("%lf", &percent);
    for (index = 0; index < NB_PRICES; index++)
        prices[index] = prices[index] * (1 + percent / 100);  // تعديل الخانة في مكانها
    for (index = 0; index < NB_PRICES; index++)
        printf("%.2f ", prices[index]);
    printf("\n");
    return 0;
}
```

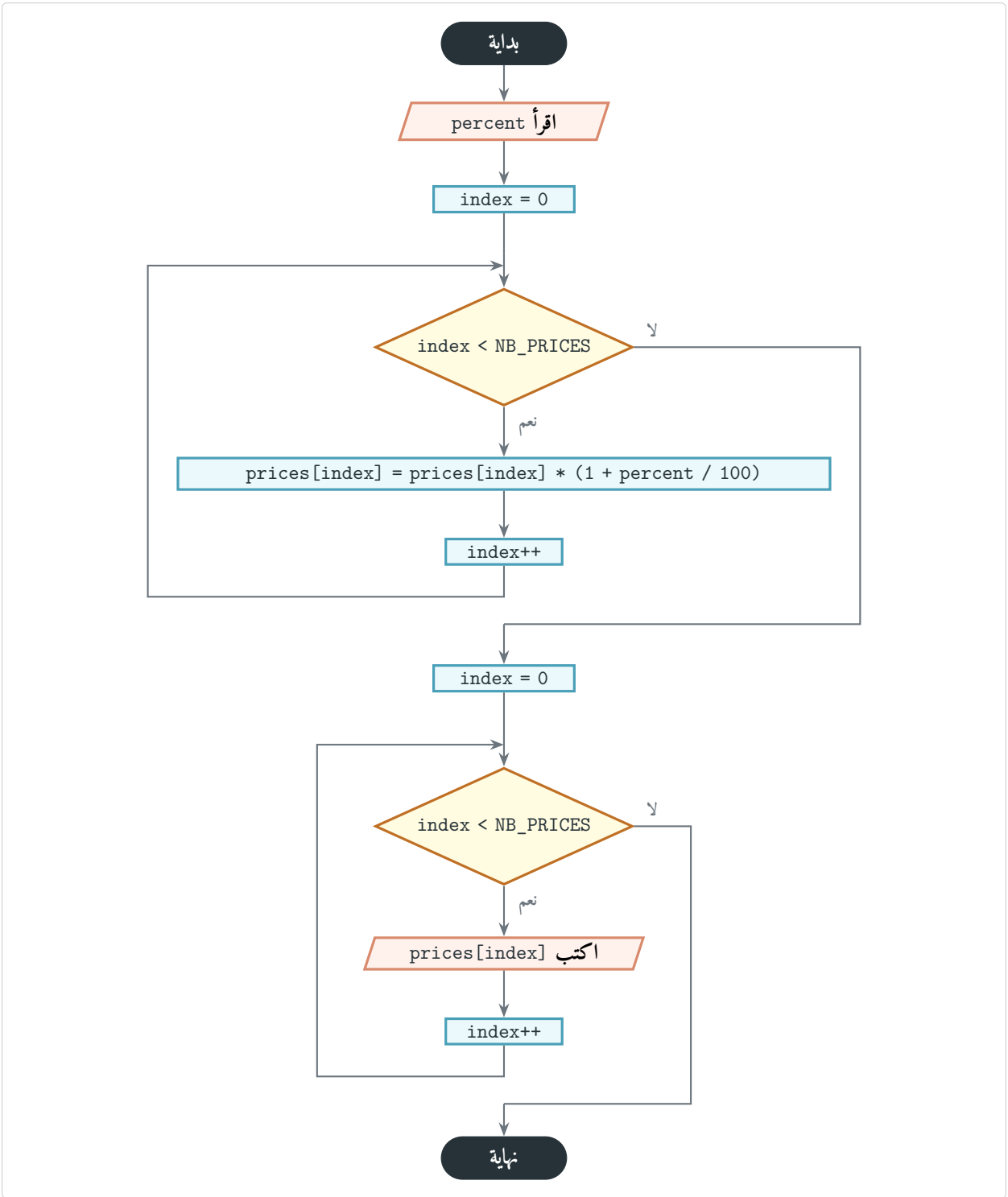
المدخلات

```
8
```

نتيجة التنفيذ

```
1080.00 2700.00 538.92 12960.00 81.00
```

المخطط الإنسيابي (Organigramme)



تمرين 4.6 آخر الطلبيات أولًا

تجارة إلكترونية

سهل

تحفظ لوحة التحكم أرقام الطلبيات بترتيب وصولها. اكتب برنامجًا يقرأ `nbOrders` رقمًا ويعرضها **من الأحدث إلى الأقدم**.

خطوات الحل

1) نقرأ الأرقام في الجدول.

2) نعرضها بحلقة تنازلية من `nbOrders - 1` إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int orders[100], nbOrders, index;
    scanf("%d", &nbOrders);
    for (index = 0; index < nbOrders; index++)
        scanf("%d", &orders[index]);
    printf("Latest first:");
    for (index = nbOrders - 1; index >= 0; index--)  // من الفهرس الأخير إلى 0
        printf(" %d", orders[index]);
    printf("\n");
    return 0;
}
```

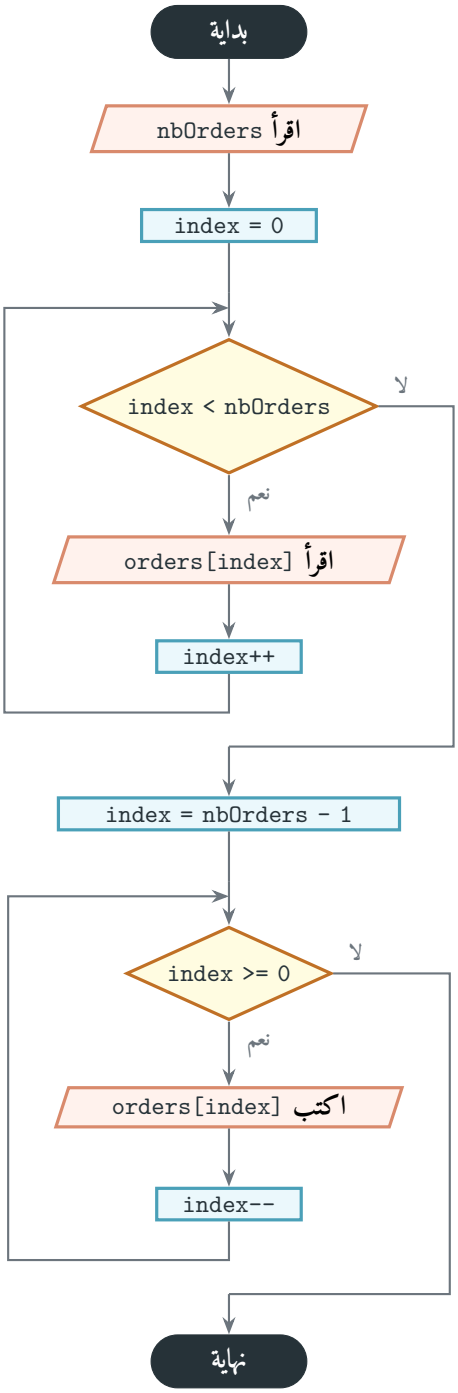
المدخلات

```
6
1001 1002 1005 1009 1010 1014
```

نتيجة التنفيذ

```
Latest first: 1014 1010 1009 1005 1002 1001
```

المخطط الإنسيابي (Organigramme)



تمرين 4.7 البحث عن منتج

تجارة إلكترونية سهل

اكتب برنامجًا يبحث عن رقم منتج في جدول الأرقام بحثًا تسلسليًا، ويعرض موضعه أو رسالة "غير موجود".

خطوات الحل

1) `pos = -1` يعني "لم نجده بعد".
2) تمر على الجدول ما دام `pos == -1`؛ عند التساوي نحفظ الموضع.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_IDS 8

int main(void) {
    int ids[NB_IDS] = {1203, 4410, 7781, 5512, 3001, 9045, 2210, 6600};
    int target, index, pos = -1;           // غير موجود :1-
    scanf("%d", &target);
    for (index = 0; index < NB_IDS && pos == -1; index++)  // نتوقف عند أول تطابق
        if (ids[index] == target)
            pos = index;
    if (pos >= 0)
        printf("Product %d found at index %d\n", target, pos);
    else
        printf("Product %d not found\n", target);
    return 0;
}
```

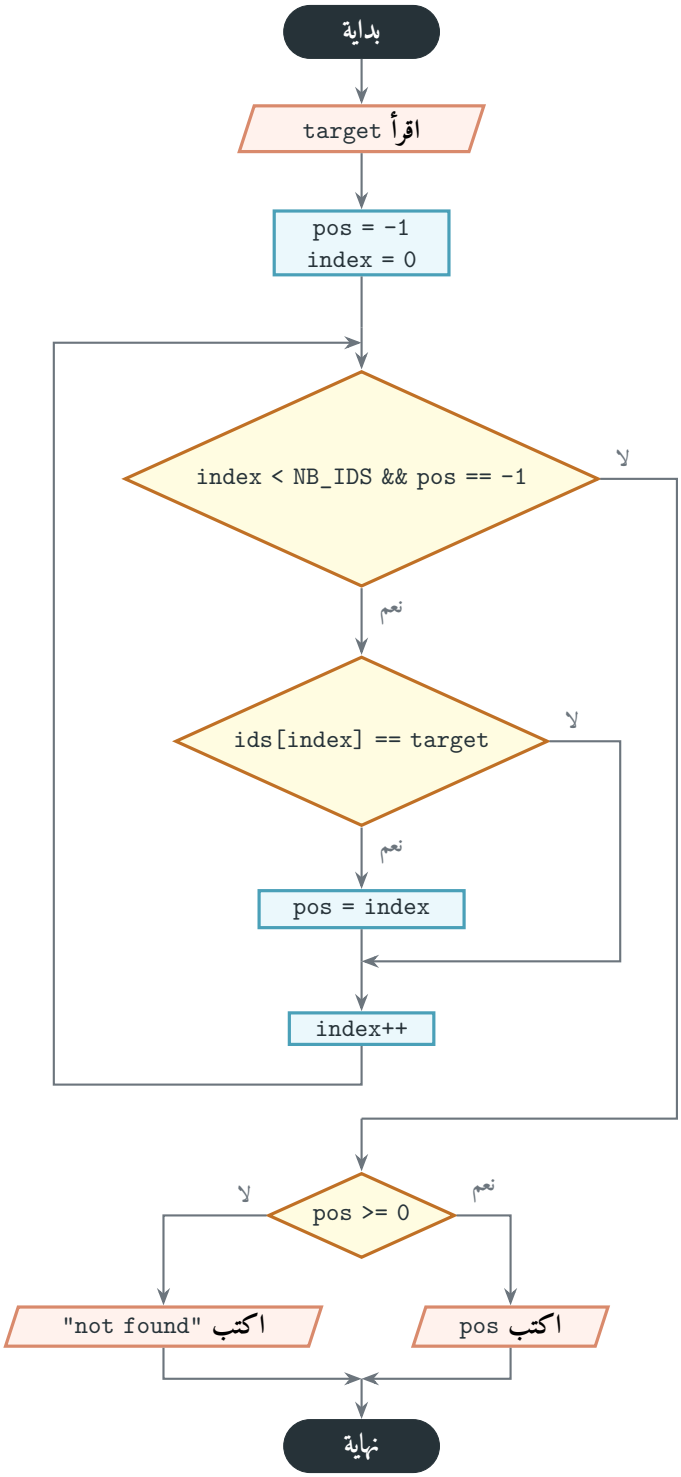
المدخلات

```
5512
```

نتيجة التنفيذ

```
Product 5512 found at index 3
```

المخطط الإنسيابي (Organigramme)



تمرين 4.8 نسخ المنتجات الفاخرة

تجارة إلكترونية

سهل

اكتب برنامجًا ينسخ الأسعار التي تتجاوز 5000 دج من جدول `prices` إلى جدول جديد `premium` (قسم المنتجات الفاخرة)، ثم يعرض `premium`.

خطوات الحل

1) `nbPremium = 0` عدد العناصر في `premium`.
2) إذا كان `prices[index] > 5000`: `premium[nbPremium] = prices[index]` ثم `nbPremium++`.
3) `nbPremium` في النهاية هو حجم `premium`.

البرنامج بلغة C

```c
#include <stdio.h>

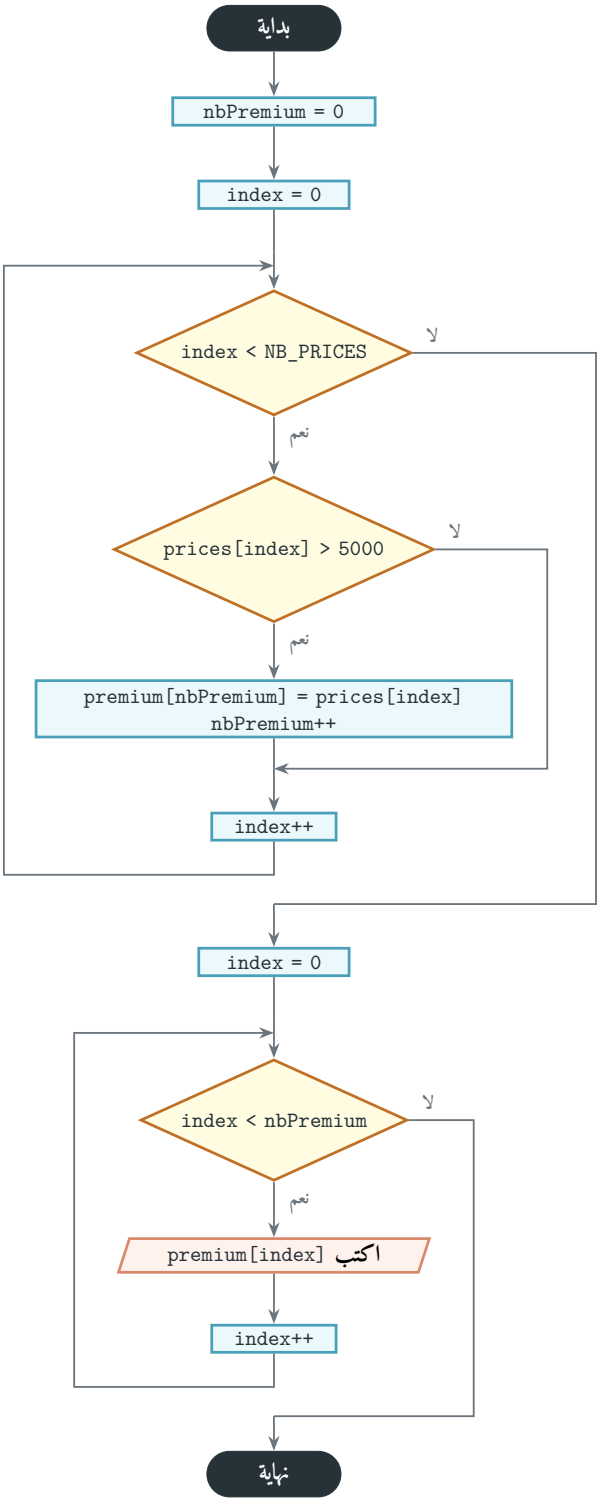
#define NB_PRICES 8

int main(void) {
    double prices[NB_PRICES] = {1200, 8900, 450, 15000, 5000, 7200, 300, 22000};
    double premium[NB_PRICES];                  // في أسوأ الحالات كل العناصر
    int index, nbPremium = 0;                   // nbPremium: عدد العناصر المنسوخة
    for (index = 0; index < NB_PRICES; index++)
        if (prices[index] > 5000)
            premium[nbPremium++] = prices[index];        // نسخ ثم نزيد nbPremium
    printf("%d premium products:", nbPremium);
    for (index = 0; index < nbPremium; index++)
        printf(" %.0f", premium[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
4 premium products: 8900 15000 7200 22000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.9 نسبة التقييمات الإيجابية

علم البيانات سهل

اكتب برنامجًا يحسب نسبة التقييمات **الإيجابية** (4 أو 5 نجوم) في جدول من التقييمات.

خطوات الحل

1) نعد العناصر التي تحقق `ratings[index] >= 4`.

2) النسبة: `pos * 100.0 / NB_REVIEWS`.

البرنامج بلغة C

```c
#include <stdio.h>

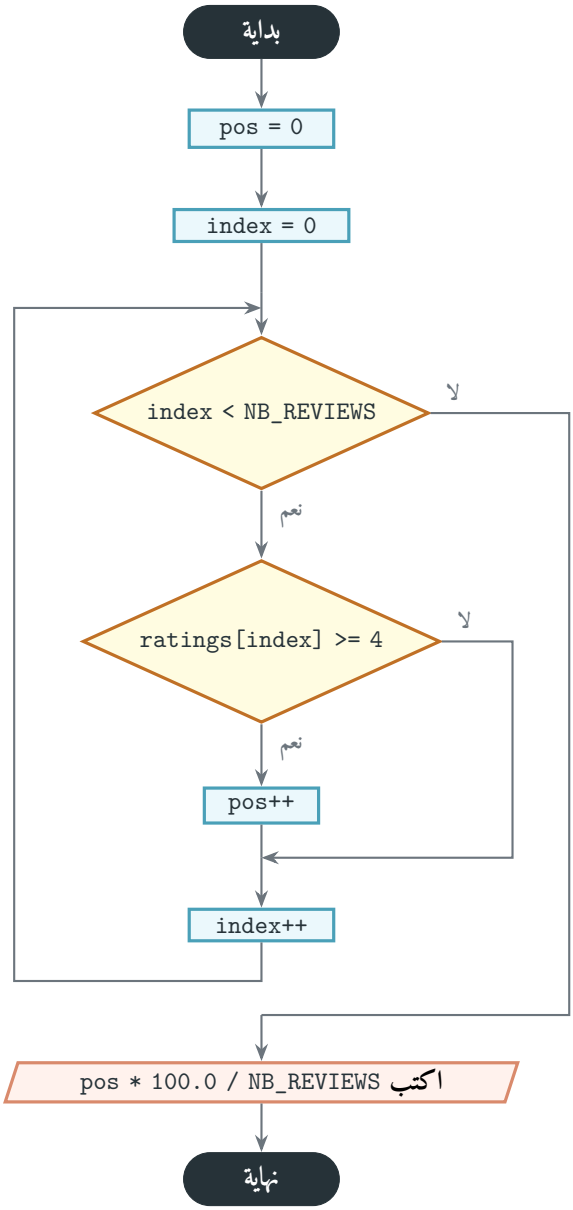
#define NB_REVIEWS 12

int main(void) {
    int ratings[NB_REVIEWS] = {5, 4, 3, 5, 2, 4, 4, 1, 5, 3, 4, 5};
    int index, pos = 0;
    for (index = 0; index < NB_REVIEWS; index++)
        if (ratings[index] >= 4)
            pos++;
    printf("Positive reviews: %d / %d = %.1f %%\n", pos, NB_REVIEWS, pos * 100.0 / NB_REVIEWS);
    return 0;
}
```

نتيجة التنفيذ

```
Positive reviews: 8 / 12 = 66.7 %
```

المخطط الإنسيابي (Organigramme)



تمرين 4.10 الفاتورة من جدولين

تجارة إلكترونية

سهل

جدول `price` يحتوي أسعار المنتجات وجدول `qty` الكميات المطلوبة (نفس الفهرس = نفس المنتج). اكتب برنامجًا يعرض مبلغ كل سطر والمجموع.

خطوات الحل

1) لكل `index`: `line = price[index] * qty[index]`.
2) تجاهل الأسطر ذات الكمية 0.
3) نجمع المبالغ.

البرنامج بلغة C

```c
#include <stdio.h>

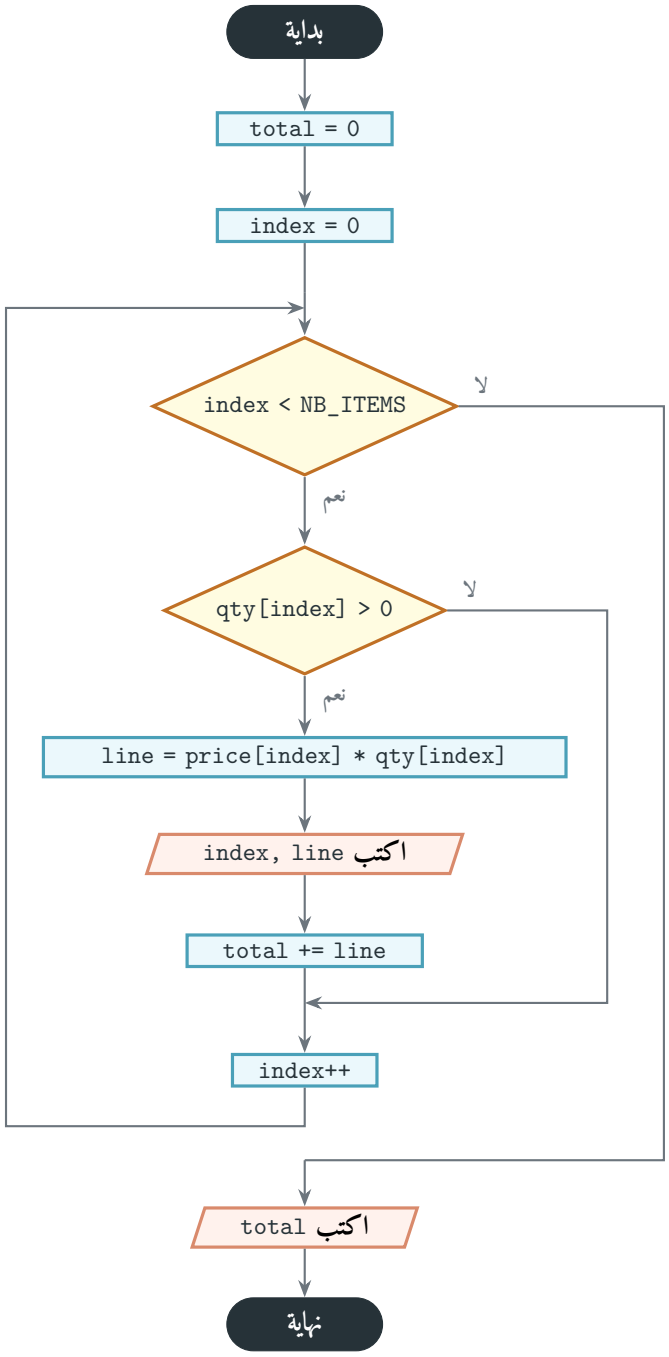
#define NB_ITEMS 5

int main(void) {
    double price[NB_ITEMS] = {350, 1200, 80, 4500, 999};
    int qty[NB_ITEMS] = {2, 0, 10, 1, 3};    // جدولان متوازيان
    double total = 0, line;
    int index;
    for (index = 0; index < NB_ITEMS; index++) {
        if (qty[index] > 0) {
            line = price[index] * qty[index];
            printf("Item %d: %d x %.2f = %.2f\n", index, qty[index], price[index], line);
            total += line;
        }
    }
    printf("TOTAL = %.2f DA\n", total);
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 2 x 350.00 = 700.00
Item 2: 10 x 80.00 = 800.00
Item 3: 1 x 4500.00 = 4500.00
Item 4: 3 x 999.00 = 2997.00
TOTAL = 8997.00 DA
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 4.11 عكس الجدول في مكانه

علم البيانات ★★ متوسط

لعرض سلسلة زمنية من الأحدث إلى الأقدم، اكتب برنامجًا **يعكس** جدولًا في مكانه (دون جدول ثانٍ) بتبديل العناصر المتناظرة.

خطوات الحل

1) `left` من البداية و`right` من النهاية.
2) ما دام `left < right`: نبدل `series[left]` و`series[right]` ثم `left++` و`right--`.

البرنامج بلغة C

```c
#include <stdio.h>

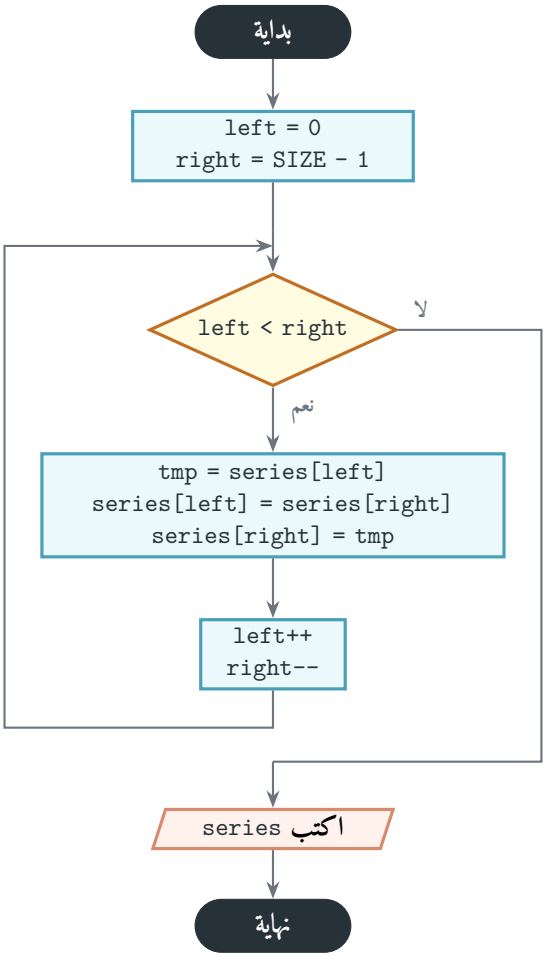
#define SIZE 7

int main(void) {
    int series[SIZE] = {10, 20, 30, 40, 50, 60, 70};
    int left = 0, right = SIZE - 1, tmp;
    while (left < right) {                // نتوقف عند المنتصف
        tmp = series[left];               // تبديل الطرفين
        series[left] = series[right];
        series[right] = tmp;
        left++;
        right--;
    }
    for (left = 0; left < SIZE; left++)
        printf("%d ", series[left]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
70 60 50 40 30 20 10
```

المخطط الإنسيابي (Organigramme)



تمرين 4.12 تدوير جدول المناوبات

تجارة إلكترونية ★★ متوسط

جدول يحتوي أرقام عمال التوصيل حسب أيام الأسبوع. كل أسبوع **يدور** الجدول بخطوة `steps` نحو اليسار (العامل الأول يصبح في الآخر). اكتب البرنامج.

خطوات الحل

1) تدوير بخطوة واحدة: نحفظ `shifts[0]`، نزيح الباقي نحو اليسار، ونضع المحفوظ في الآخر.
2) نكرر ذلك `steps` مرة (أو `steps % NB_DAYS` مرة).

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 7

int main(void) {
    int shifts[NB_DAYS] = {11, 12, 13, 14, 15, 16, 17};
    int steps, step, index, first;
    scanf("%d", &steps);
    for (step = 0; step < steps % NB_DAYS; step++) {      // تدوير NB_DAYS مرة يعيد الجدول كما كان
        first = shifts[0];
        for (index = 0; index < NB_DAYS - 1; index++)
            shifts[index] = shifts[index + 1];            // إزاحة نحو اليسار
```

```
13          shifts[NB_DAYS - 1] = first;
14      }
15      printf("Rotated:");
16      for (index = 0; index < NB_DAYS; index++)
17          printf(" %d", shifts[index]);
18      printf("\n");
19      return 0;
20  }
```

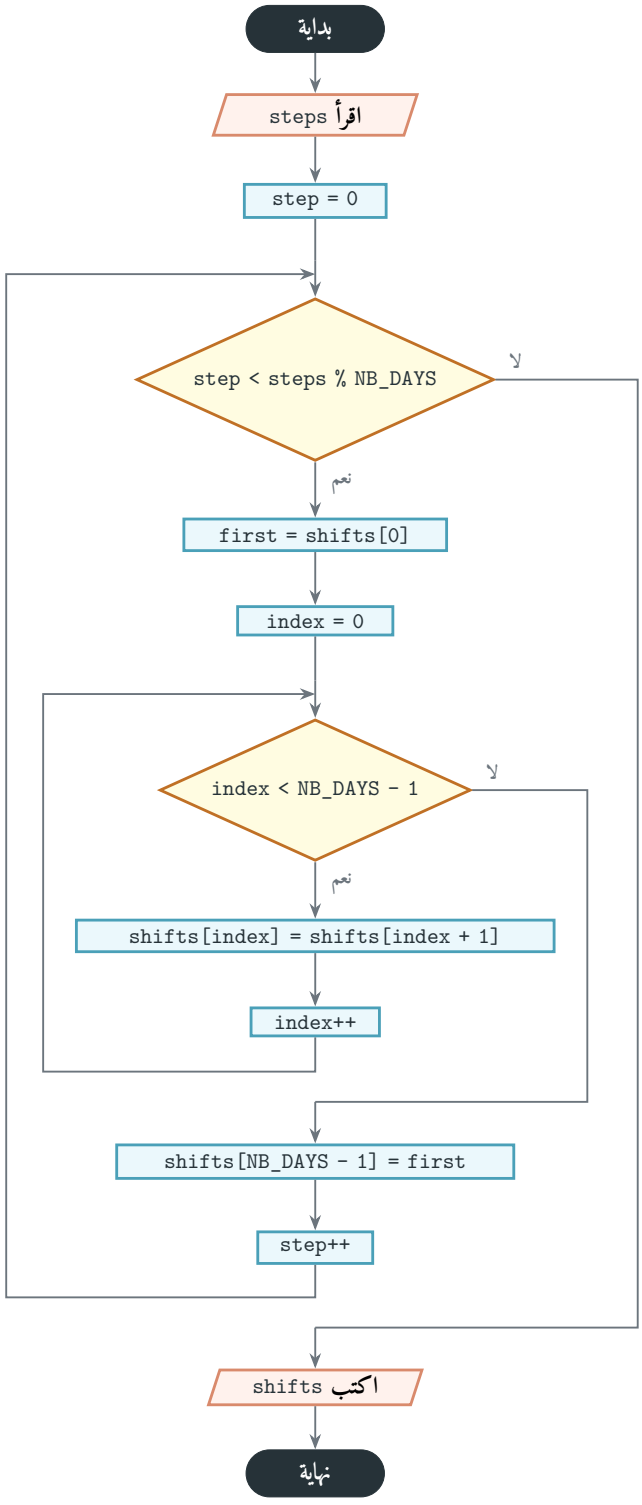
المدخلات

```
2
```

نتيجة التنفيذ

```
Rotated: 13 14 15 16 17 11 12
```

المخطط الإنسيابي (Organigramme)



تمرين 4.13 إدراج منتج جديد في موضع

تجارة إلكترونية ★★ متوسط

قائمة المنتجات المميزة مخزنة في جدول. اكتب برنامجًا يدرج منتجًا جديدًا في الموضع pos مع إزاحة العناصر الموالية نحو اليمين.

خطوات الحل

1) نتحقق أن `0 <= pos <= size` وأن الجدول غير ممتلئ.
2) نزيح من الآخر: `featured[index] = featured[index - 1]` لـ `index` من `size` نزولًا إلى `pos + 1`.
3) `featured[pos] = newId` ثم `size++`.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 10

int main(void) {
    int featured[MAX] = {101, 205, 330, 412, 560};
    int size = 5, pos, newId, index;          // size: عدد العناصر المستعملة
    scanf("%d %d", &pos, &newId);
    if (pos < 0 || pos > size || size == MAX) {
        printf("Insertion impossible\n");
        return 0;
    }
    // نبدأ من الآخر لكي لا نمسح أي عنصر
    for (index = size; index > pos; index--)
        featured[index] = featured[index - 1];
    featured[pos] = newId;
    size++;
    for (index = 0; index < size; index++)
        printf("%d ", featured[index]);
    printf("\n");
    return 0;
}
```

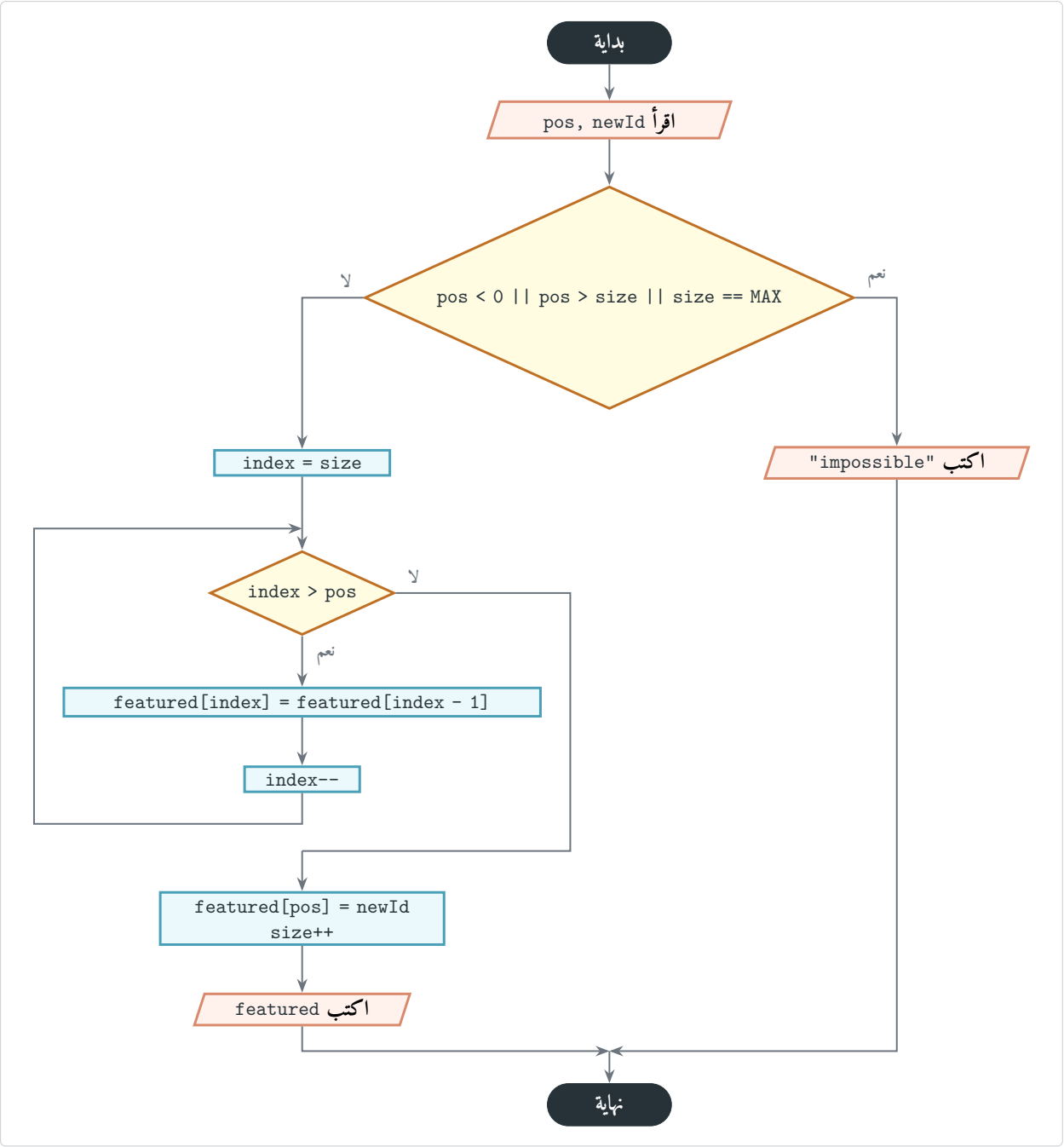
المدخلات

```
2 777
```

نتيجة التنفيذ

```
101 205 777 330 412 560
```

المخطط الإنسيابي (Organigramme)



تمرين 4.14 حذف منتج متوقف

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يحذف من الجدول المنتج ذي الرقم `removedId` (إن وُجد) بإزاحة العناصر الموالية نحو اليسار، ثم يعرض الجدول الجديد.

خطوات الحل

1) نبحث عن موضع `removedId`.
2) إذا وُجد في `pos`: `products[index] = products[index + 1]` لـ `index` من `pos` إلى `size - 2`، ثم `size--`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int products[10] = {101, 205, 330, 412, 560, 618};
    int size = 6, removedId, index, pos = -1;
    scanf("%d", &removedId);
    for (index = 0; index < size; index++)          // البحث عن الموضع
        if (products[index] == removedId) { pos = index; break; }
    if (pos == -1) {
        printf("%d not found\n", removedId);
    } else {
        for (index = pos; index < size - 1; index++)
            products[index] = products[index + 1];          // نسد الفراغ
        size--;                                 // الجدول فقد عنصرًا
        for (index = 0; index < size; index++)
            printf("%d ", products[index]);
        printf("\n");
    }
    return 0;
}
```

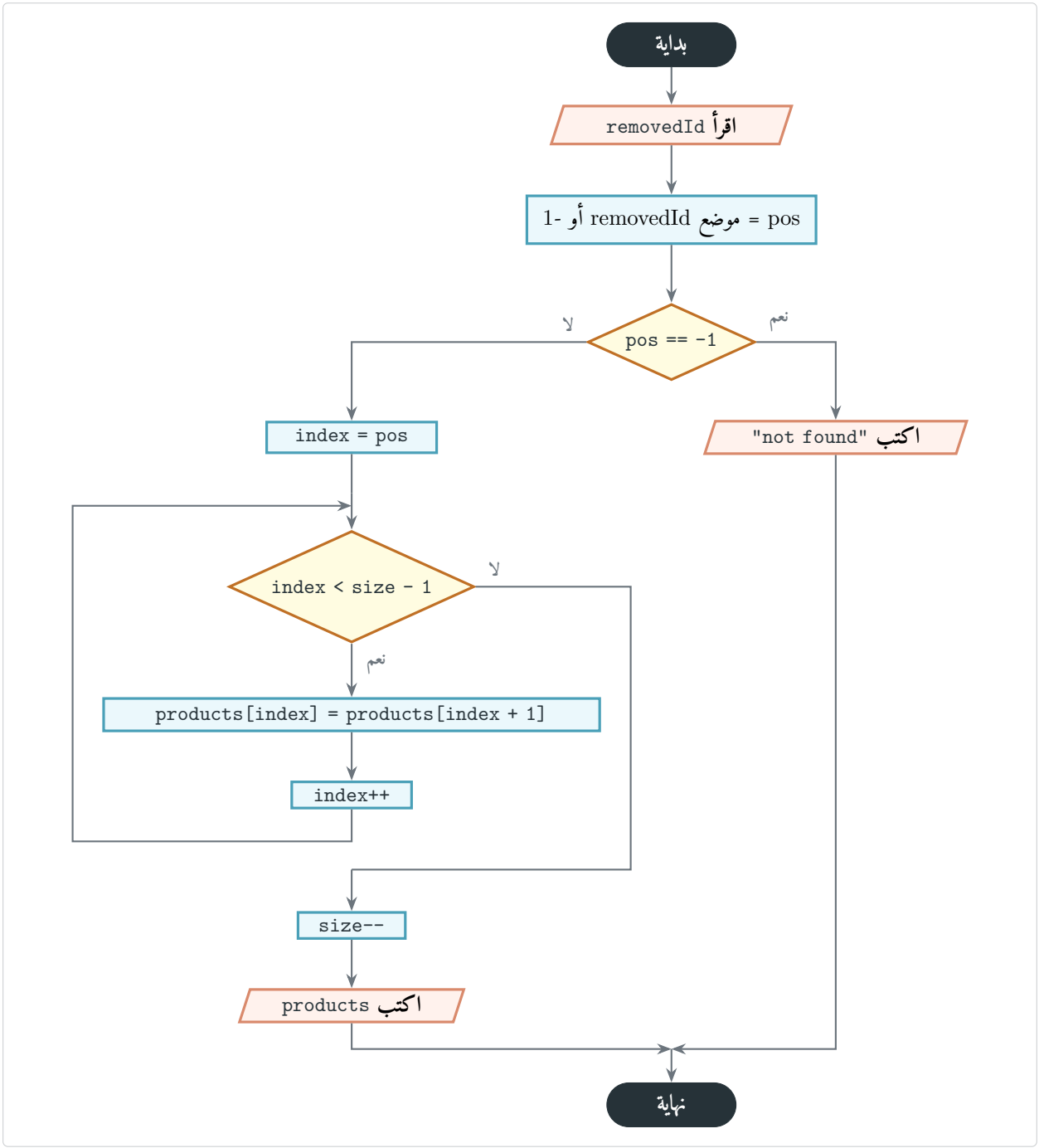
المدخلات

```
330
```

نتيجة التنفيذ

```
101 205 412 560 618
```

المخطط الإنسيابي (Organigramme)



تمرين 4.15 الترتيب بالفقاعات

تجارة إلكترونية

متوسط

اكتب برنامجًا يرتب أسعار المنتجات تصاعديًا بخوارزمية **الفقاعات**، مع التوقف المبكر إذا لم يحدث أي تبديل في دورة كاملة.

خطوات الحل

1) في كل دورة نقارن كل عنصرين متجاورين ونبدلهما إذا كانا غير مرتبين.
2) بعد الدورة الأولى يصل الأكبر إلى الآخر.
3) `swapped = 0` في بداية الدورة؛ إذا بقي 0 فالجدول مرتب.

البرنامج بلغة C

```c
#include <stdio.h>

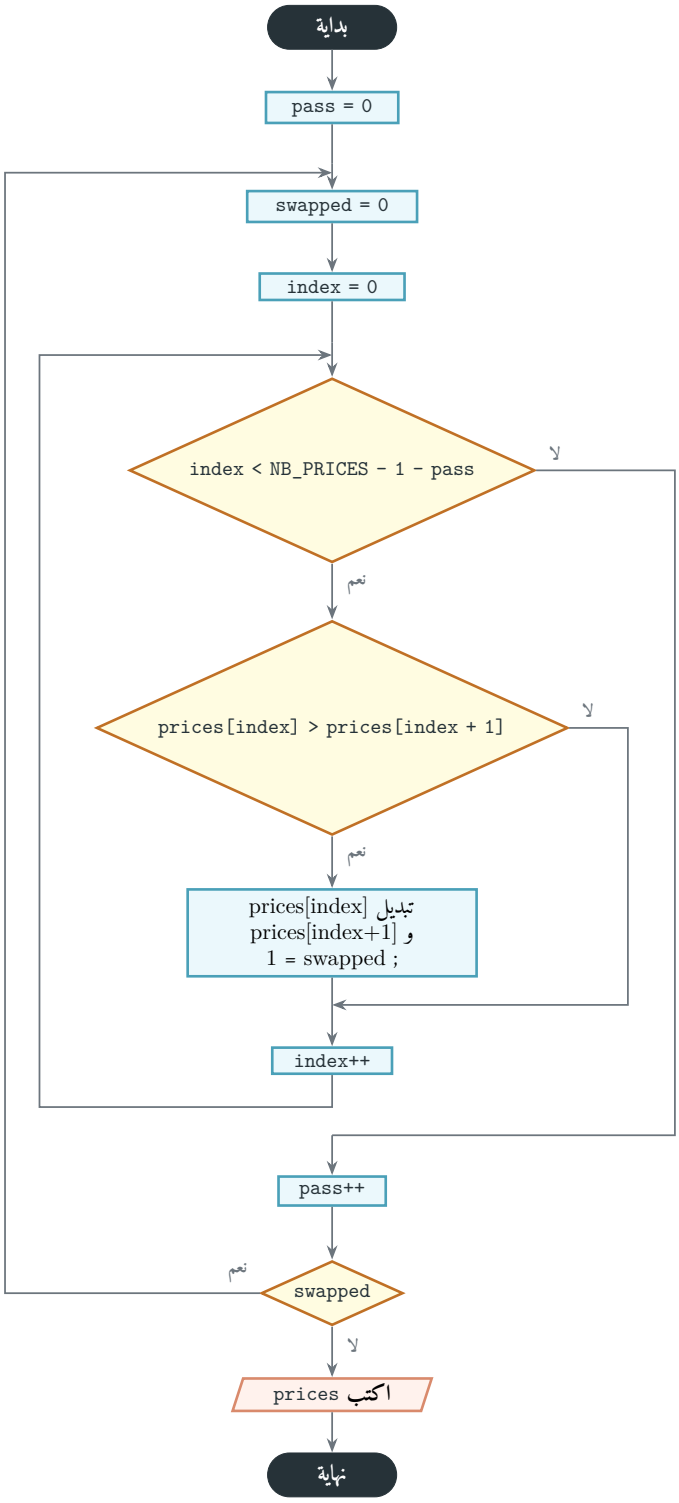
#define NB_PRICES 8

int main(void) {
    double prices[NB_PRICES] = {890, 120, 450, 3000, 75, 640, 1200, 300};
    double tmp;
    int index, pass = 0, swapped;
    do {
        swapped = 0;
        // آخر pass عناصر مرتبة سلفًا
        for (index = 0; index < NB_PRICES - 1 - pass; index++) {
            if (prices[index] > prices[index + 1]) {
                tmp = prices[index]; prices[index] = prices[index + 1]; prices[index + 1] = tmp;
                swapped = 1;
            }
        }
        pass++;
    } while (swapped);            // لا تبديل = الجدول مرتب
    printf("Sorted in %d passes:", pass);
    for (index = 0; index < NB_PRICES; index++)
        printf(" %.0f", prices[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Sorted in 6 passes: 75 120 300 450 640 890 1200 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.16 الترتيب بالإنتقاء

علم البيانات ★★ متوسط

اكتب برنامجًا يرتب عدد زيارات صفحات الموقع تنازليًا بخوارزمية الانتقاء: في كل خطوة نبحث عن أكبر عنصر في الجزء غير المرتب ونضعه في بدايته.

خطوات الحل

1) لكل index من 0 إلى NB_PAGES - 2: نبحث عن فهرس الأكبر maxPos بين index و NB_PAGES - 1.
2) نبدل visits[index] و visits[maxPos].
3) عدد التبديلات لا يتجاوز NB_PAGES - 1.

البرنامج بلغة C

```c
#include <stdio.h>

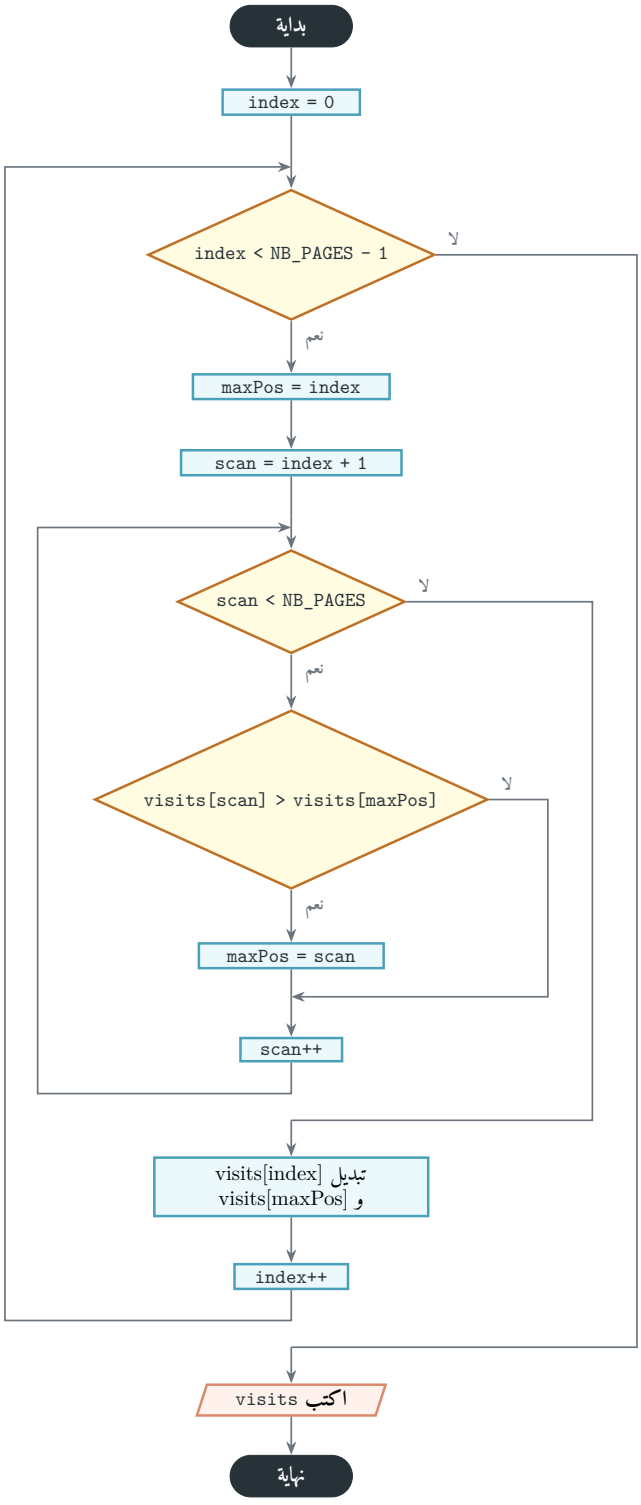
#define NB_PAGES 8

int main(void) {
    int visits[NB_PAGES] = {340, 1200, 85, 960, 1200, 410, 77, 2300};
    int index, scan, maxPos, tmp;
    for (index = 0; index < NB_PAGES - 1; index++) {
        maxPos = index;                    // فهرس الأكبر في visits[index..N-1]
        for (scan = index + 1; scan < NB_PAGES; scan++)
            if (visits[scan] > visits[maxPos])
                maxPos = scan;
        tmp = visits[index]; visits[index] = visits[maxPos]; visits[maxPos] = tmp;  // نضعه في الموضع index
    }
    printf("Pages by visits:");
    for (index = 0; index < NB_PAGES; index++)
        printf(" %d", visits[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Pages by visits: 2300 1200 1200 960 410 340 85 77
```

المخطط الإنسيابي (Organigramme)



تمرين 4.17 الترتيب بالإدراج

تجارة إلكترونية ★★ متوسط

تصل الطلبيات بأوقات غير مرتبة (بالدقائق). اكتب برنامجًا يرتبها بخوارزمية **الإدراج**: نأخذ كل عنصر وندرجه في مكانه في الجزء المرتب على يساره.

خطوات الحل

1) لكل index من 1: key = times[index].
2) نزيح العناصر الأكبر من key نحو اليمين: scan من index - 1 نزولًا.
3) نضع key في times[scan + 1].

البرنامج بلغة C

```c
#include <stdio.h>

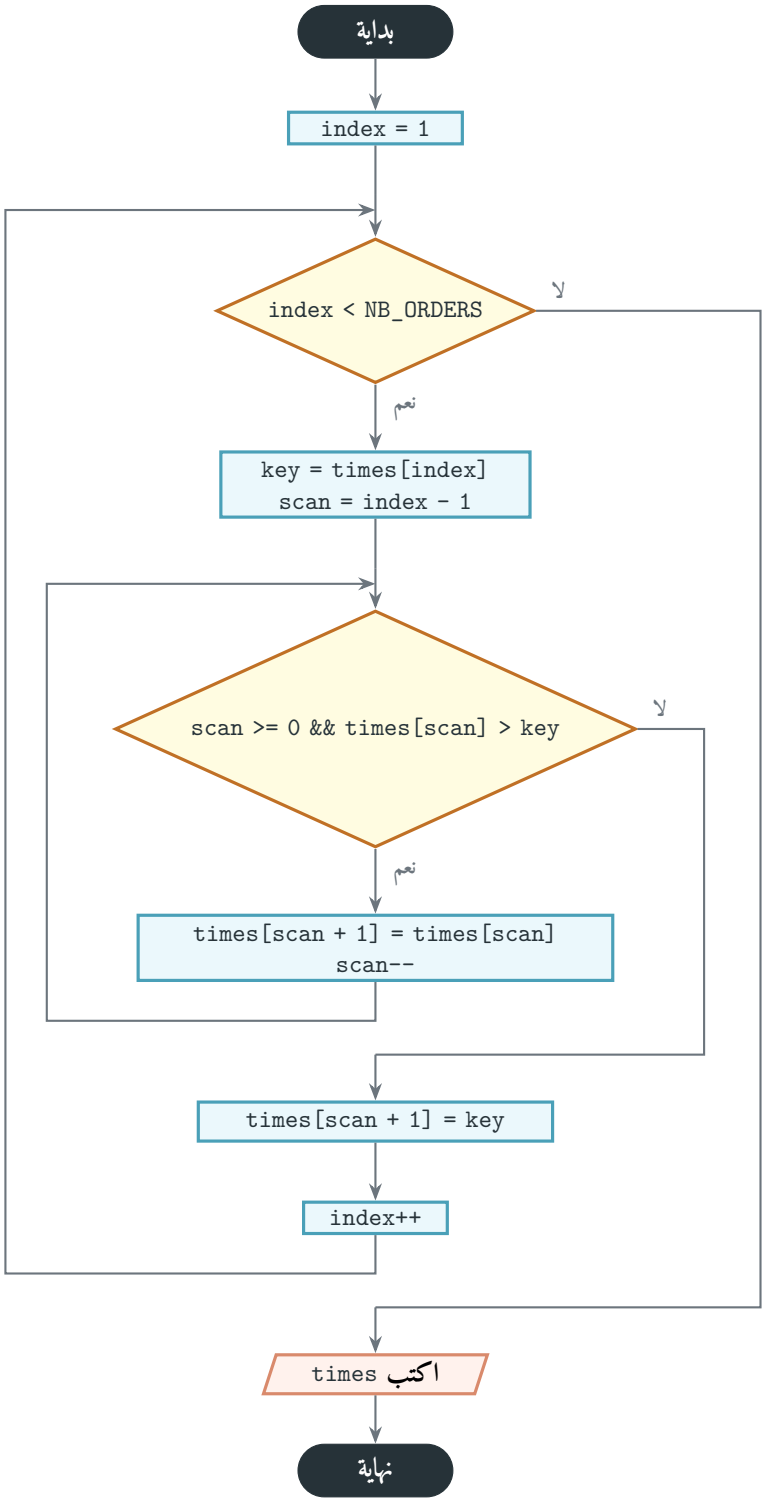
#define NB_ORDERS 8

int main(void) {
    int times[NB_ORDERS] = {615, 540, 720, 480, 600, 545, 900, 510};
    int index, scan, key;
    for (index = 1; index < NB_ORDERS; index++) {
        key = times[index];                 // العنصر المراد إدراجه
        scan = index - 1;
        while (scan >= 0 && times[scan] > key) {
            times[scan + 1] = times[scan];      // نزيح الأكبر نحو اليمين
            scan--;
        }
        times[scan + 1] = key;              // الفراغ الذي بقي هو مكانه
    }
    for (index = 0; index < NB_ORDERS; index++)
        printf("%02d:%02d ", times[index] / 60, times[index] % 60);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
08:00 08:30 09:00 09:05 10:00 10:15 12:00 15:00
```

المخطط الإنسيابي (Organigramme)



تمرين 4.18 البحث الثنائي في كتالوج مرتب

تجارة إلكترونية

متوسط

أرقام المنتجات في الكتالوج **مرتبة تصاعديًا**. اكتب برنامجًا يبحث عن رقم **بحثًا ثنائيًا** ويعرض عدد المقارنات.

خطوات الحل

1) `lo = 0`، `hi = NB_IDS - 1`.
2) `mid = (lo + hi) / 2`: إذا كان `catalog[mid] == target` وجدناه.
3) إذا كان `catalog[mid] < target` نبحث يميناً `lo = mid + 1`، وإلا يساراً `hi = mid - 1`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_IDS 12

int main(void) {
    int catalog[NB_IDS] = {1001, 1203, 2210, 3001, 3550, 4410, 5512,
                6600, 7781, 8120, 9045, 9900};      // مرتب
    int target, lo = 0, hi = NB_IDS - 1, mid, pos = -1, cmp = 0;
    scanf("%d", &target);
    while (lo <= hi && pos == -1) {
        mid = (lo + hi) / 2;
        cmp++;
        if (catalog[mid] == target)
            pos = mid;
        else if (catalog[mid] < target)
            lo = mid + 1;          // target في النصف الأيمن
        else
            hi = mid - 1;          // target في النصف الأيسر
    }
    if (pos >= 0)
        printf("Found at index %d after %d comparisons\n", pos, cmp);
    else
        printf("Not found (%d comparisons)\n", cmp);
    return 0;
}
```

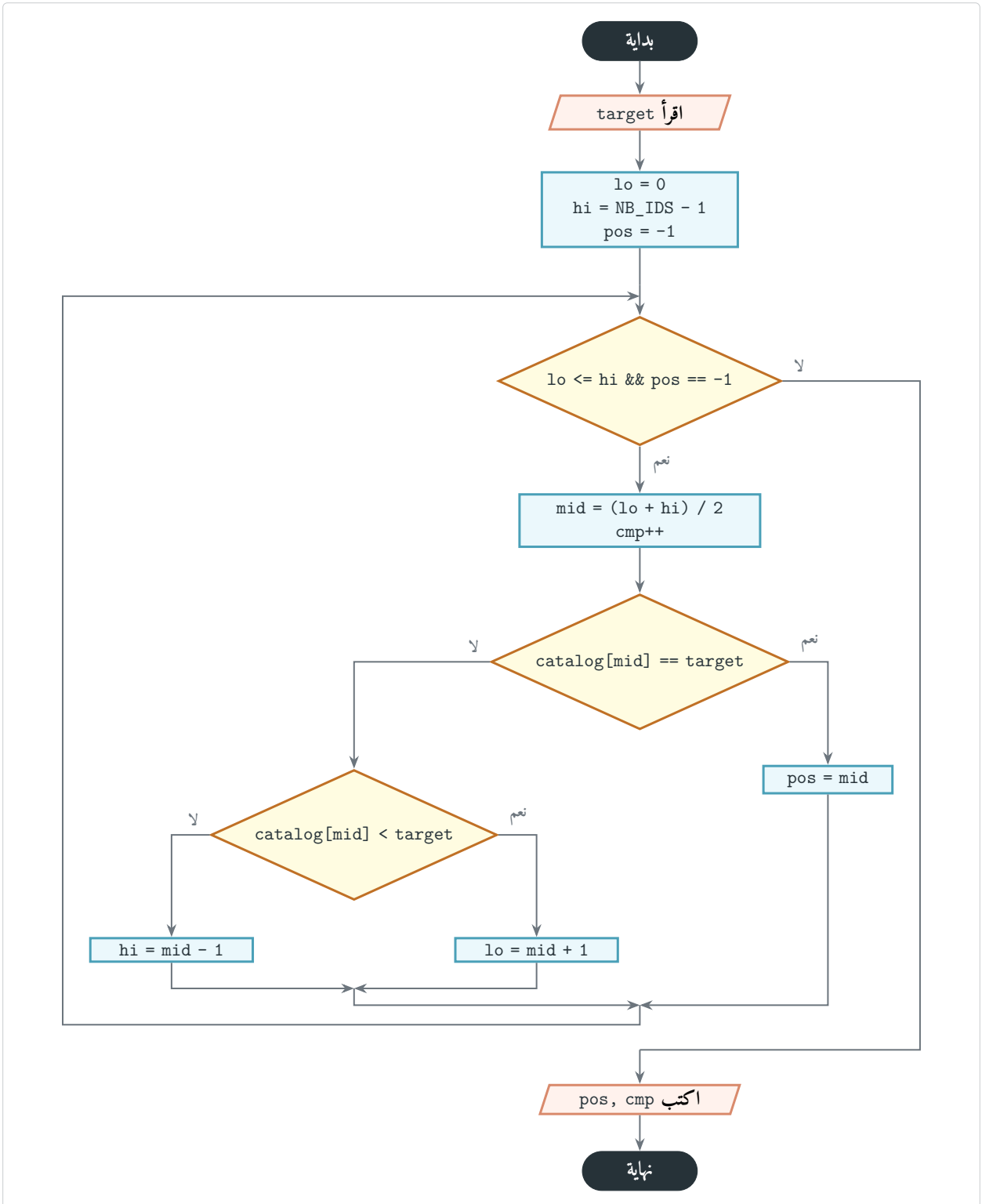
المدخلات

```
7781
```

نتيجة التنفيذ

```
Found at index 8 after 2 comparisons
```

المخطط الإنسيابي (Organigramme)



تمرين 4.19 وسيط أسعار السوق

علم البيانات ★★ متوسط

لتحديد سعر عادل لمنتج نجمع أسعار المنافسين ونأخذ **الوسيط**. اكتب برنامجًا يقرأ `nbPrices` أسعار، يرتبها، ثم يحسب الوسيط (العنصر الأوسط، أو متوسط العنصرين الأوسطين إذا كان `nbPrices` زوجيًا).

خطوات الحل

1) نقرأ الأسعار ونرتبها بالإدراج.
2) إذا كان nbPrices فردياً: prices[nbPrices / 2].
3) وإلا:

```
(prices[nbPrices/2 - 1] + prices[nbPrices/2]) / 2
```

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double prices[100], key, med;
    int nbPrices, index, scan;
    scanf("%d", &nbPrices);
    for (index = 0; index < nbPrices; index++)
        scanf("%lf", &prices[index]);
    for (index = 1; index < nbPrices; index++) {          // ترتيب بالإدراج
        key = prices[index];
        for (scan = index - 1; scan >= 0 && prices[scan] > key; scan--)
            prices[scan + 1] = prices[scan];
        prices[scan + 1] = key;
    }
    if (nbPrices % 2 == 1)
        med = prices[nbPrices / 2];                   // العنصر الأوسط
    else
        med = (prices[nbPrices / 2 - 1] + prices[nbPrices / 2]) / 2;
    printf("Median price = %.2f DA\n", med);
    return 0;
}
```

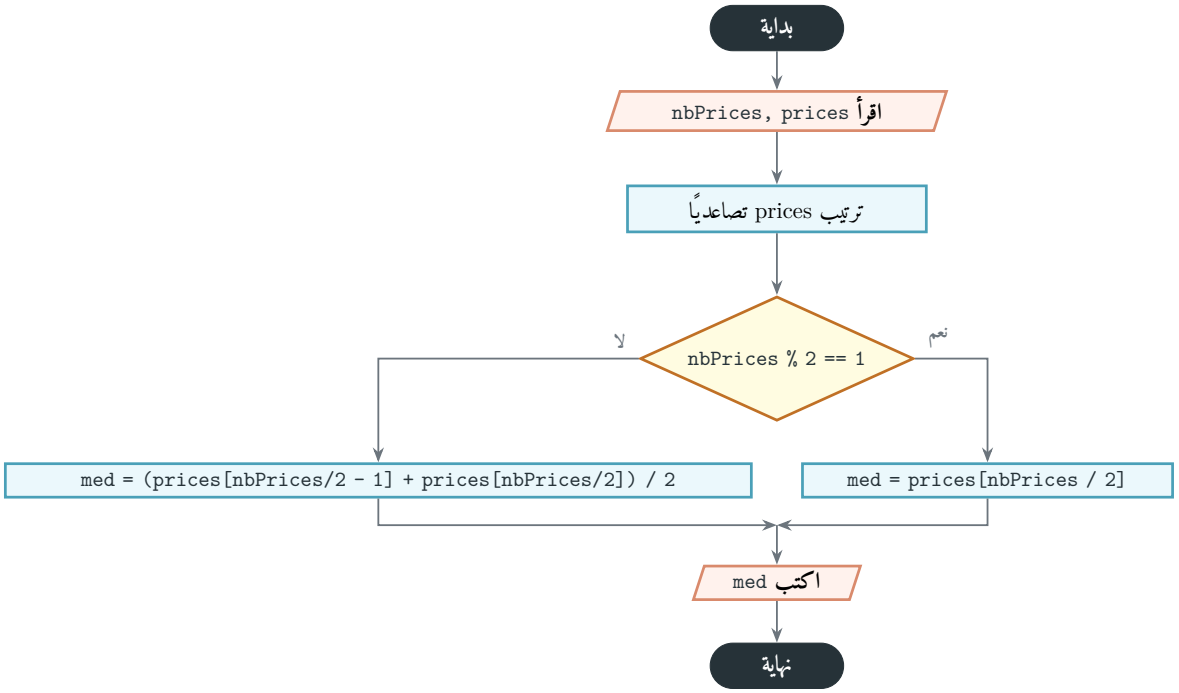
المدخلات

```
8
2500 2390 2700 9900 2450 2600 2550 1200
```

نتيجة التنفيذ

```
Median price = 2525.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.20 المنوال بجدول عدادات

علم البيانات ★★ متوسط

اكتب برنامجًا يجد **المنوال** (القيمة الأكثر تكرارًا) في جدول تقييمات من 1 إلى 5، باستعمال **جدول عدادات** `cnt[6]` حيث `cnt[value]` عدد مرات ظهور `value`.

خطوات الحل

1) نهيئ `cnt` بأصفار: `int cnt[6] = {0};`.
2) لكل تقييم `ratings`: `cnt[ratings]++` (القيمة نفسها فهرس).
3) المنوال هو الفهرس ذو أكبر عداد.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_REVIEWS 15

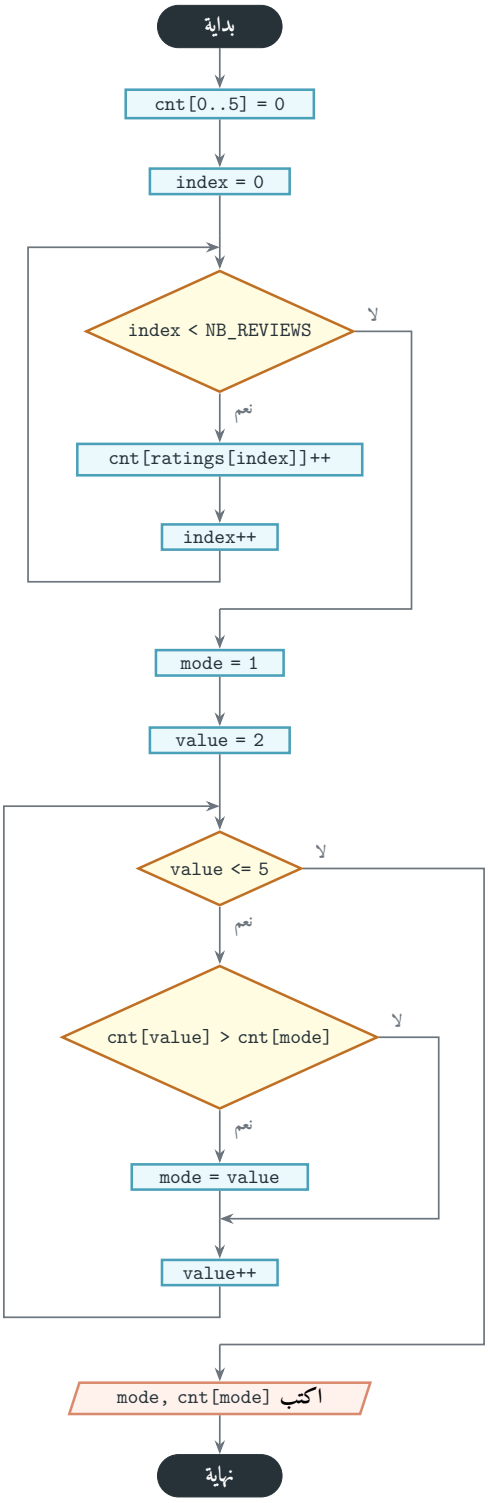
int main(void) {
    int ratings[NB_REVIEWS] = {4, 5, 3, 4, 4, 2, 5, 4, 1, 3, 4, 5, 5, 4, 3};
    int cnt[6] = {0};              // كل الخانات تبدأ من 0
    int index, value, mode = 1;
    for (index = 0; index < NB_REVIEWS; index++)
        cnt[ratings[index]]++;             // التقييم نفسه هو الفهرس
    for (value = 2; value <= 5; value++)
        if (cnt[value] > cnt[mode])
            mode = value;
    for (value = 1; value <= 5; value++)
        printf("%d stars: %d\n", value, cnt[value]);
    printf("Mode = %d (%d times)\n", mode, cnt[mode]);
```

```
17      return 0;
18  }
```

نتيجة التنفيذ

```
1 stars: 1
2 stars: 1
3 stars: 3
4 stars: 6
5 stars: 4
Mode = 4 (6 times)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.21 توزيع الطلبيات حسب الساعة

علم البيانات ★★ متوسط

لدينا ساعات 20 طلبية (من 0 إلى 23). اكتب برنامجًا يحسب عدد الطلبيات في كل ساعة بجدول `cnt[24]`، ويعرض الساعات غير الفارغة مع شريط نجوم، ثم **ساعة الذروة**.

خطوات الحل

1) `cnt[hour]++` لكل طلبية.
2) نعرض فقط الساعات التي `cnt[hour] > 0`.
3) ساعة الذروة: فهرس أكبر عداد.

البرنامج بلغة C

```c
#include <stdio.h>

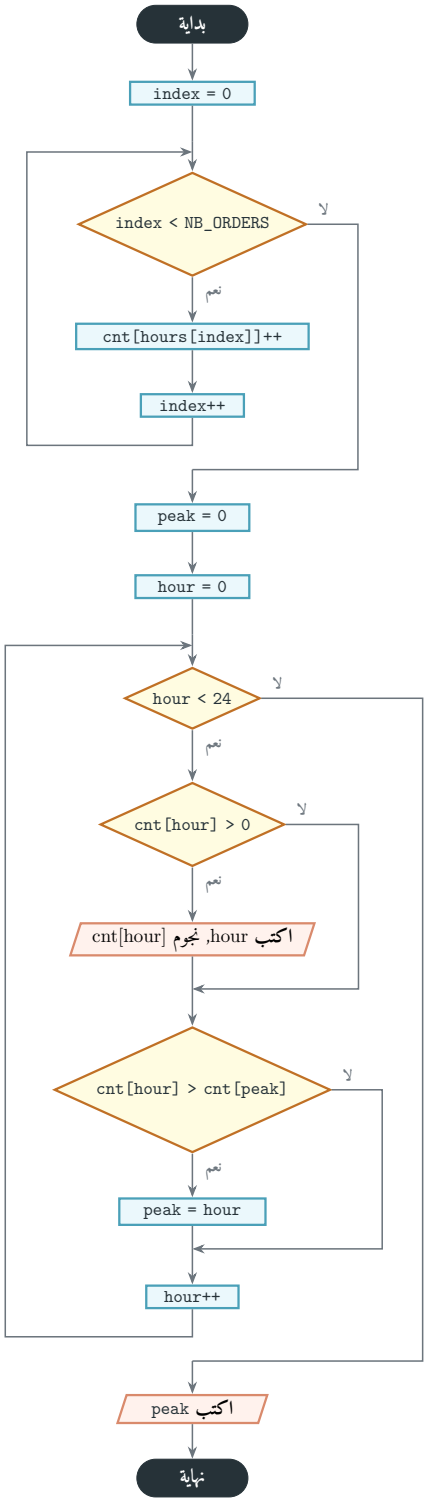
#define NB_ORDERS 20

int main(void) {
    int hours[NB_ORDERS] = {9, 12, 13, 20, 21, 12, 20, 22, 8, 13,
                    20, 21, 12, 19, 20, 23, 13, 20, 10, 21};
    int cnt[24] = {0};
    int index, hour, bar, peak = 0;
    for (index = 0; index < NB_ORDERS; index++)
        cnt[hours[index]]++;
    for (hour = 0; hour < 24; hour++) {
        if (cnt[hour] > 0) {
            printf("%02dh | ", hour);
            for (bar = 0; bar < cnt[hour]; bar++)
                printf("#");
            printf(" %d\n", cnt[hour]);
        }
        if (cnt[hour] > cnt[peak])
            peak = hour;
    }
    printf("Peak hour: %02dh\n", peak);
    return 0;
}
```

نتيجة التنفيذ

```
08h | # 1
09h | # 1
10h | # 1
12h | ### 3
13h | ### 3
19h | # 1
20h | ##### 5
21h | ### 3
22h | # 1
23h | # 1
Peak hour: 20h
```

المخطط الإنسيابي (Organigramme)



تمرين 4.22 حذف المكرر من جدول مرتب

علم البيانات ★★ متوسط

في سجل مرتب لأرقام الزبائن الذين زاروا الموقع تتكرر بعض الأرقام. اكتب برنامجًا يحذف **المكرر** في مكانه ويعرض عدد الزبائن المختلفين.

خطوات الحل

1) بما أن الجدول مرتب، المكرر متجاور.
2) `nbUnique = 1` (العنصر الأول محفوظ دائمًا).
3) لكل `index` من 1: إذا كان `visitors[index] != visitors[nbUnique - 1]` نكتب `visitors[nbUnique] = visitors[index]` و`nbUnique++`.

البرنامج بلغة C

```c
#include <stdio.h>

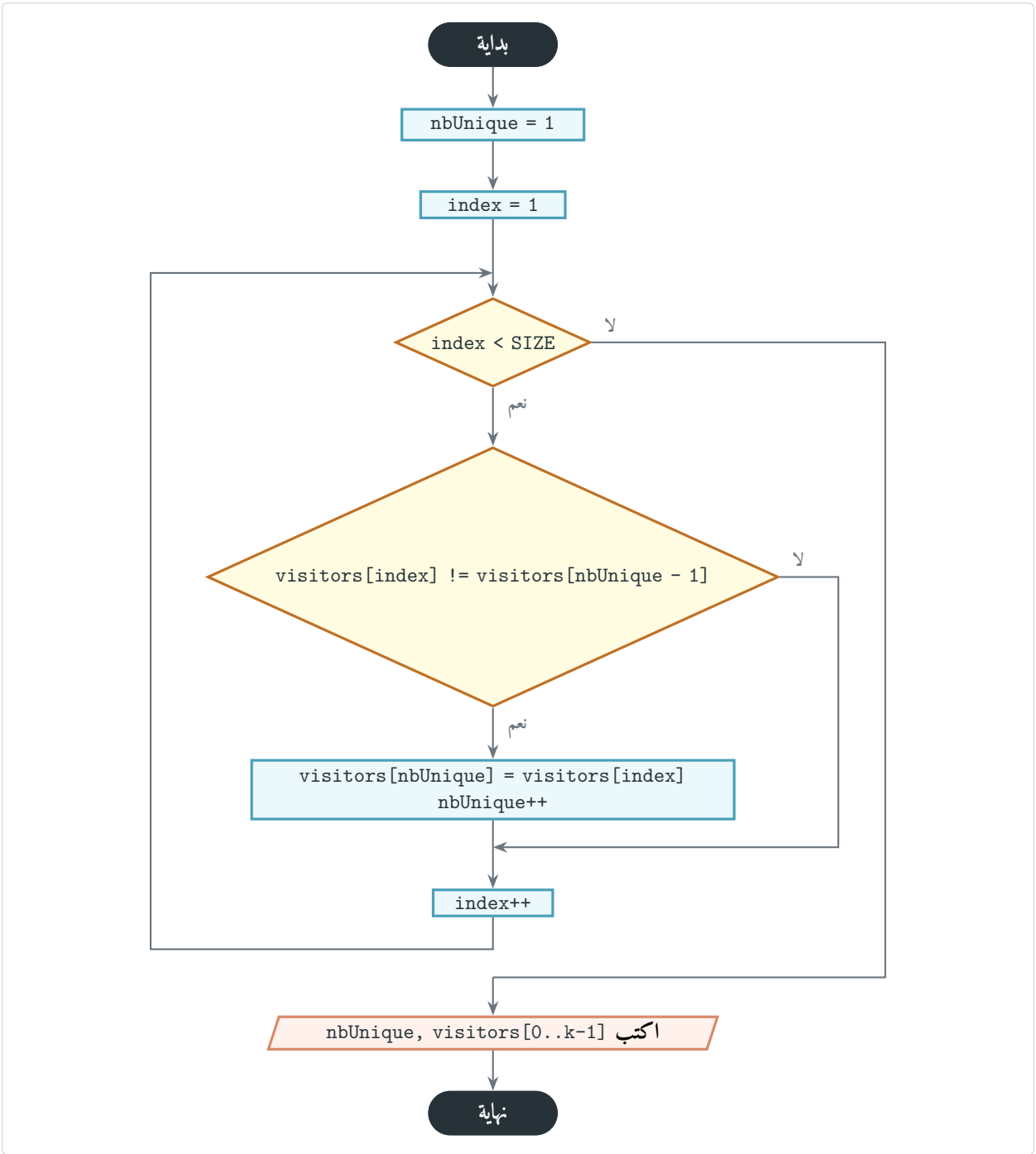
#define SIZE 12

int main(void) {
    int visitors[SIZE] = {101, 101, 105, 107, 107, 107, 110, 112, 112, 115, 115, 120};
    int index, nbUnique = 1;                // nbUnique: حجم الجزء دون تكرار
    for (index = 1; index < SIZE; index++)
        if (visitors[index] != visitors[nbUnique - 1])      // مختلف عن آخر عنصر محفوظ
            visitors[nbUnique++] = visitors[index];
    printf("%d unique customers:", nbUnique);
    for (index = 0; index < nbUnique; index++)
        printf(" %d", visitors[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
7 unique customers: 101 105 107 110 112 115 120
```

المخطط الإنسيابي (Organigramme)



تمرين 4.23 دمج قائمتي أسعار مرتبتين

تجارة إلكترونية ★★ متوسط

لدى مستودعين قائمتا أسعار **مرتبتان**. اكتب برنامجًا **يدمجهما** في قائمة واحدة مرتبة في مرور واحد (دون إعادة ترتيب).

خطوات الحل

1) مؤشران `posA` على `listA` و`posB` على `listB`.
2) نأخذ الأصغر بين `listA[posA]` و`listB[posB]` ونضعه في `merged[posC]`.
3) عند نفاد إحدى القائمتين ننسخ بقية الأخرى.

البرنامج بلغة C

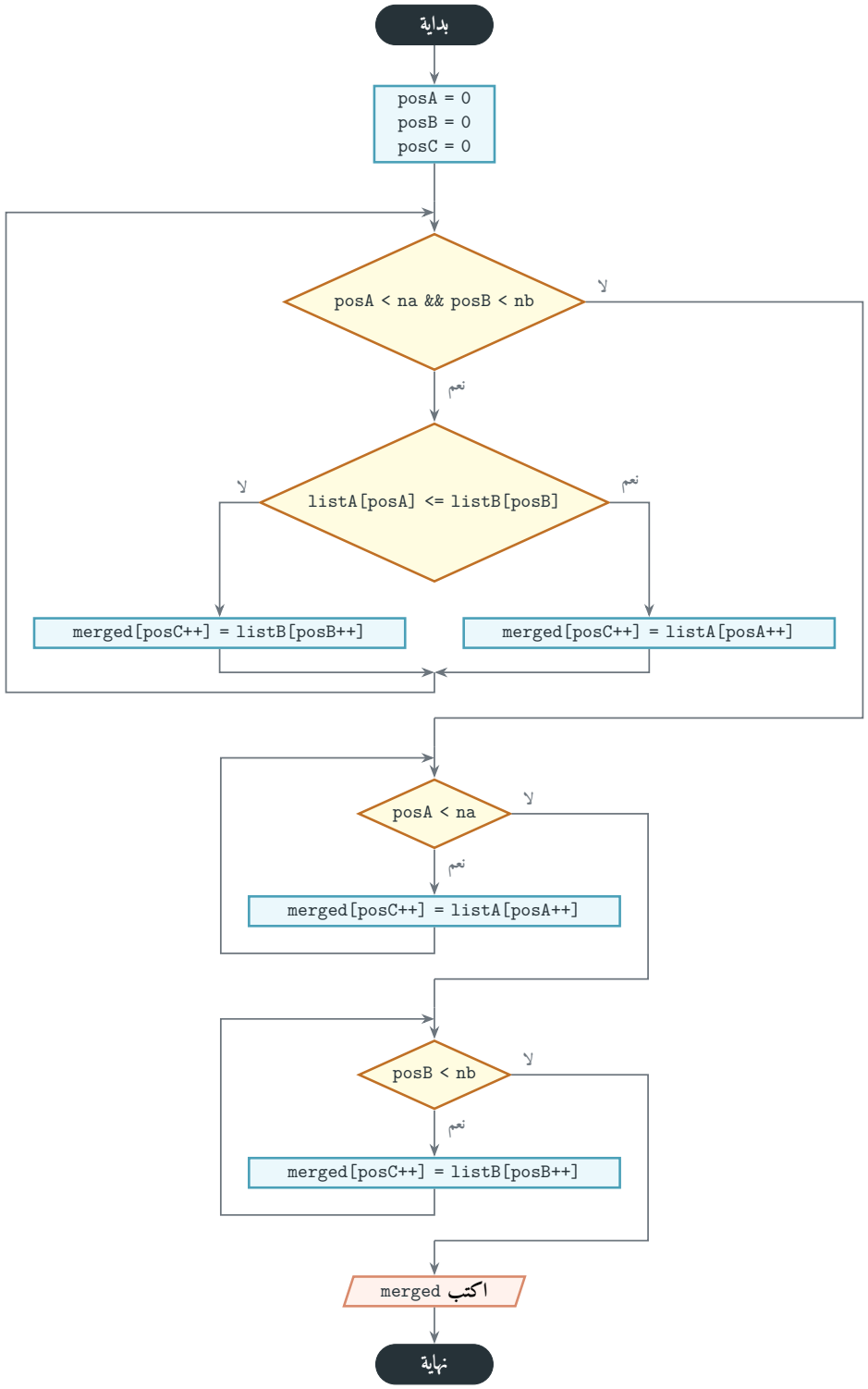
```c
#include <stdio.h>

int main(void) {
    int listA[5] = {120, 450, 800, 1500, 3000};
    int listB[4] = {99, 500, 820, 2500};
    int merged[9], na = 5, nb = 4, posA = 0, posB = 0, posC = 0;
    while (posA < na && posB < nb) {
        if (listA[posA] <= listB[posB])
            merged[posC++] = listA[posA++];      // الأصغر من listA
        else
            merged[posC++] = listB[posB++];      // الأصغر من listB
    }
    while (posA < na) merged[posC++] = listA[posA++];   // بقية listA
    while (posB < nb) merged[posC++] = listB[posB++];   // بقية listB
    for (posA = 0; posA < posC; posA++)
        printf("%d ", merged[posA]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
99 120 450 500 800 820 1500 2500 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.24 المجاميع التراكمية واستعلام الفترات

علم البيانات ★★ متوسط

تطلب لوحة التحكم مرارًا مجموع المبيعات بين يومين from وto. بدل جمع الفترة في كل مرة، نحسب مرة واحدة جدول **المجاميع التراكمية** prefix[d] = sales[0] + ... + sales[d-1]، فيصبح مجموع الفترة prefix[to + 1] - prefix[from]. اكتب البرنامج.

خطوات الحل

1) `prefix[0] = 0` ثم `prefix[day + 1] = prefix[day] + sales[day]`.
2) لكل استعلام: `prefix[to + 1] - prefix[from]` في عملية واحدة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 10

int main(void) {
    int sales[NB_DAYS] = {120, 95, 140, 180, 160, 210, 75, 130, 150, 90};
    int prefix[NB_DAYS + 1];          // خانة إضافية للمجموع الفارغ
    int day, query, from, to;
    prefix[0] = 0;
    for (day = 0; day < NB_DAYS; day++)
        prefix[day + 1] = prefix[day] + sales[day];   // تراكم
    for (query = 0; query < 2; query++) {
        scanf("%d %d", &from, &to);
        // مجموع الأيام from..to دون حلقة
        printf("Sales days %d..%d = %d\n", from, to, prefix[to + 1] - prefix[from]);
    }
    return 0;
}
```

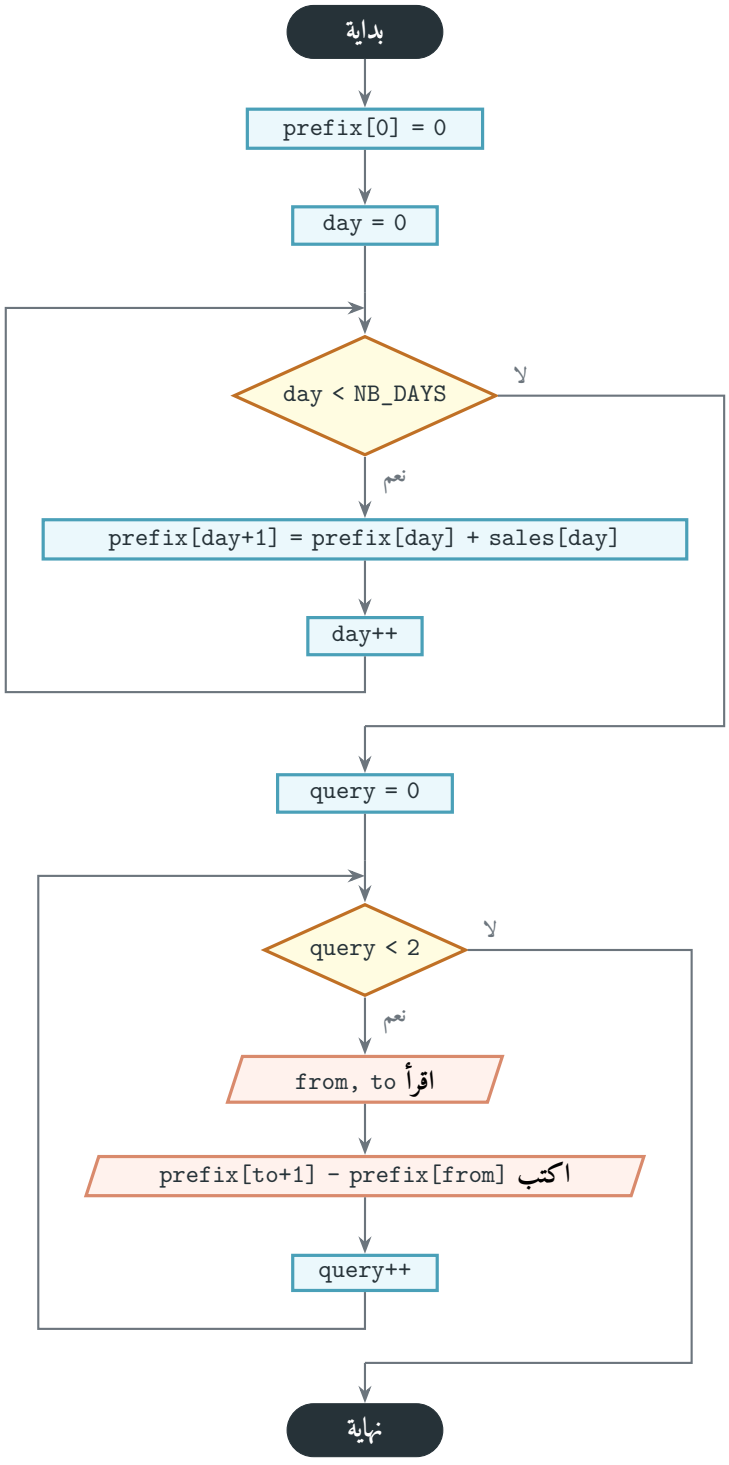
المدخلات

```
2 5
0 9
```

نتيجة التنفيذ

```
Sales days 2..5 = 690
Sales days 0..9 = 1350
```

المخطط الإنسيابي (Organigramme)



تمرين 4.25 المتوسط المتحرك بنافذة k

علم البيانات

متوسط

اكتب برنامجًا يحسب **المتوسط المتحرك** لمبيعات يومية بنافذة طولها `window` يومًا. لتجنب إعادة الجمع، ننزلق بالنافذة: نضيف اليوم الجديد ونطرح اليوم الذي خرج منها.

خطوات الحل

1) نحسب مجموع أول `window` يومًا.
2) لكل يوم `day >= window`:

```
windowSum += sales[day] - sales[day - window]
```

.
3) المتوسط `windowSum / window`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_DAYS 10

int main(void) {
    double sales[NB_DAYS] = {100, 130, 115, 160, 140, 170, 150, 190, 210, 180};
    double windowSum = 0;
    int window, day;
    scanf("%d", &window);
    for (day = 0; day < window; day++)
        windowSum += sales[day];          // النافذة الأولى
    printf("Days 1-%d: %.2f\n", window, windowSum / window);
    for (day = window; day < NB_DAYS; day++) {
        // اليوم الجديد يدخل، واليوم الأقدم يخرج
        windowSum += sales[day] - sales[day - window];
        printf("Days %d-%d: %.2f\n", day - window + 2, day + 1, windowSum / window);
    }
    return 0;
}
```

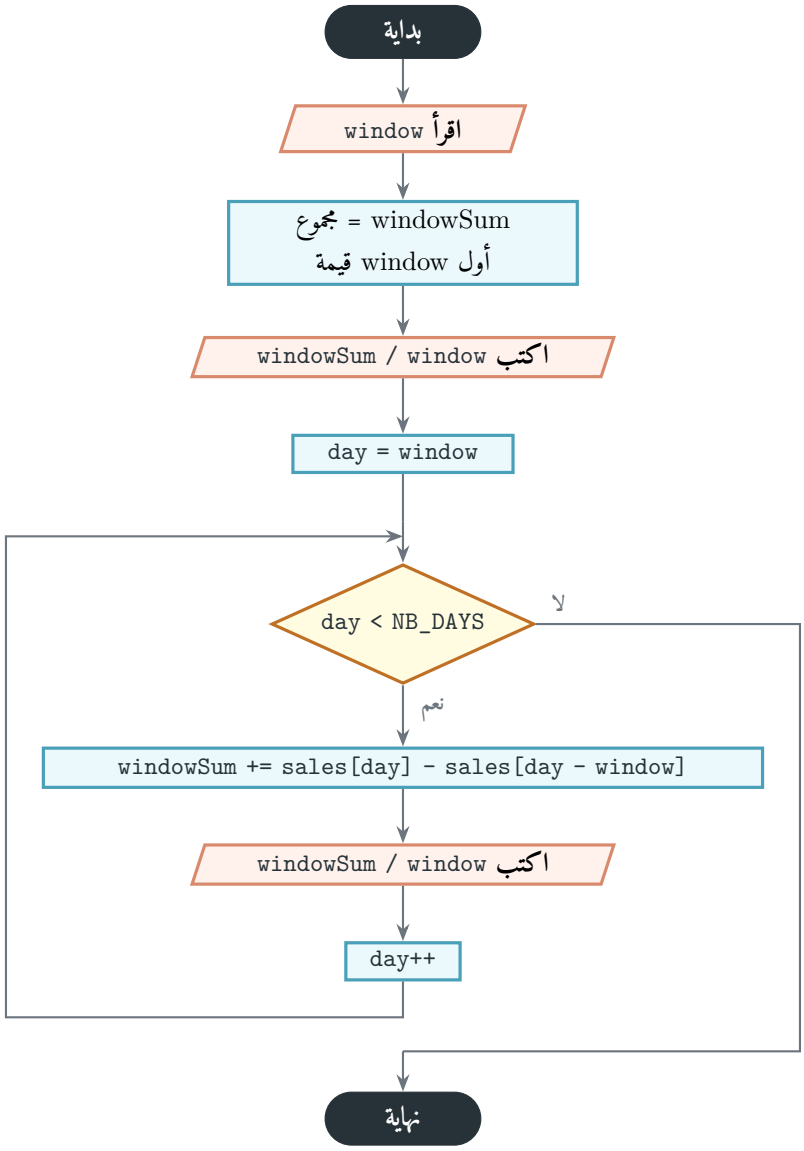
المدخلات

```
4
```

نتيجة التنفيذ

```
Days 1-4: 126.25
Days 2-5: 136.25
Days 3-6: 146.25
Days 4-7: 155.00
Days 5-8: 162.50
Days 6-9: 180.00
Days 7-10: 182.50
```

المخطط الإنسيابي (Organigramme)



تمرين 4.26 تطبيع جدول الأسعار

علم البيانات ★★ متوسط

اكتب برنامجًا يحول كل الأسعار في جدول إلى المجال [0, 1] بطريقة **Min-Max**، بعد إيجاد الأصغر والأكبر في مرور أول.

خطوات الحل

1) المرور الأول: `minPrice` و`maxPrice`.
2) المرور الثاني:

```
scaled[index] = (prices[index] - minPrice) / (maxPrice - minPrice)
```

.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_PRICES 7
```

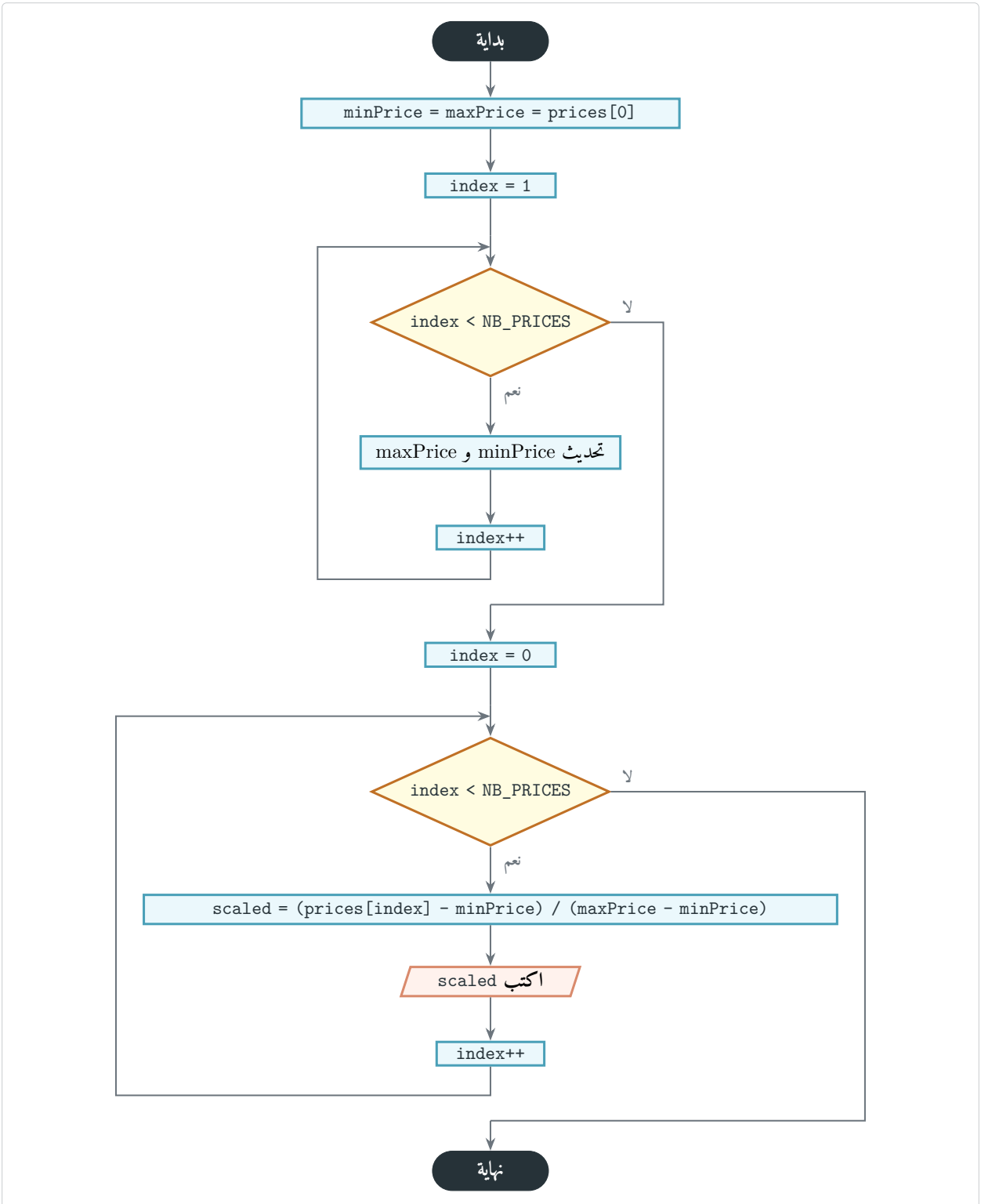
```c

int main(void) {
    double prices[NB_PRICES] = {1200, 450, 3990, 89, 700, 2500, 1600};
    double minPrice = prices[0], maxPrice = prices[0];
    int index;
    for (index = 1; index < NB_PRICES; index++) {
        if (prices[index] < minPrice) minPrice = prices[index];
        if (prices[index] > maxPrice) maxPrice = prices[index];
    }
    for (index = 0; index < NB_PRICES; index++) {
        // 0 للأرخص و 1 للأغلى
        double scaled = (prices[index] - minPrice) / (maxPrice - minPrice);
        printf("%7.0f -> %.3f\n", prices[index], scaled);
    }
    return 0;
}
```

نتيجة التنفيذ

```
   1200 -> 0.285
    450 -> 0.093
   3990 -> 1.000
     89 -> 0.000
    700 -> 0.157
   2500 -> 0.618
   1600 -> 0.387
```

المخطط الإنسيابي (Organigramme)



تمرين 4.27 التوحيد المعياري للبيانات

علم البيانات ★★ متوسط

قبل تدريب نموذج نحول أعمار الزبائن إلى **درجات معيارية** `(x - mean) / std`. اكتب برنامجًا يحسب المتوسط والانحراف المعياري (الجذر بطريقة نيوتن) ثم الجدول المعياري، ويتحقق أن متوسطه ≈ 0.

خطوات الحل

1) المتوسط في مرور أول، ثم التباين في مرور ثانٍ.
2) الجذر بطريقة نيوتن.
3) `standard[index] = (ages[index] - mean) / std`

.

البرنامج بلغة C

```c
#include <stdio.h>

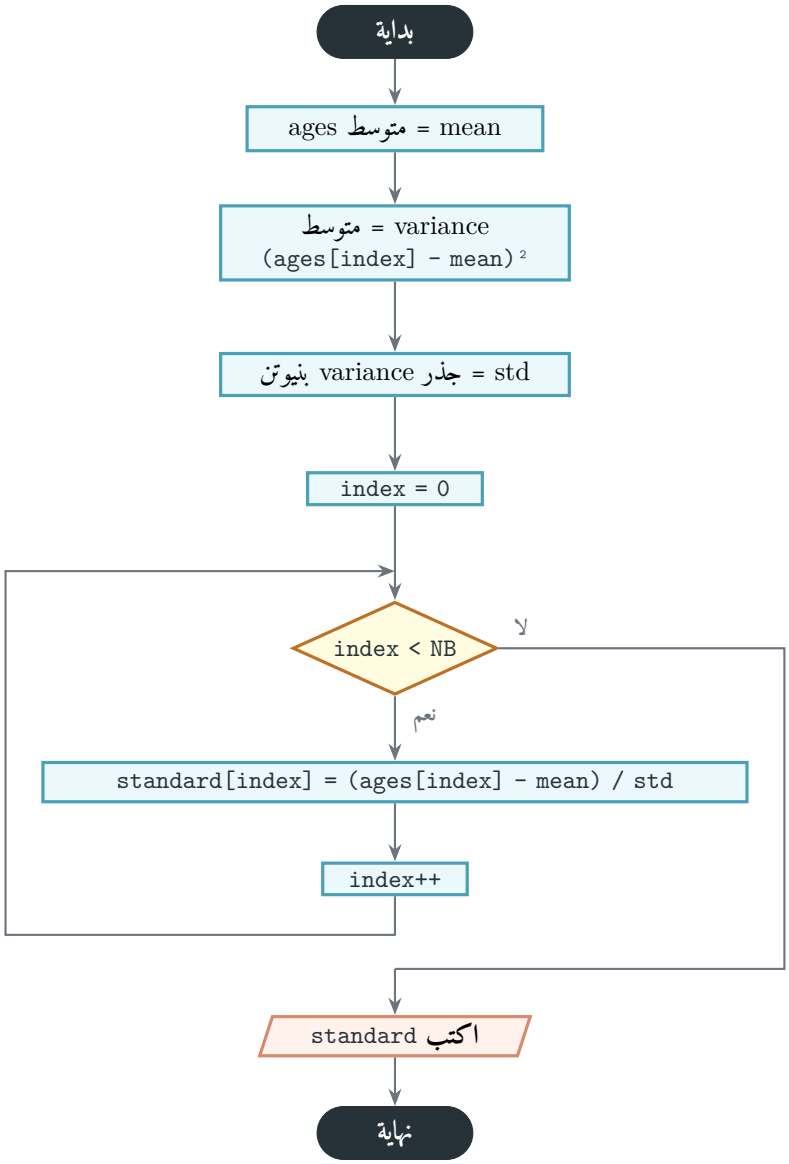
#define NB 8

int main(void) {
    double ages[NB] = {22, 35, 41, 28, 19, 55, 33, 47};
    double standard[NB], mean = 0, variance = 0, std, check = 0;
    int index, step;
    for (index = 0; index < NB; index++)
        mean += ages[index];
    mean /= NB;
    for (index = 0; index < NB; index++)
        variance += (ages[index] - mean) * (ages[index] - mean);
    variance /= NB;
    std = variance;                         // الجذر بطريقة نيوتن
    for (step = 0; step < 30; step++)
        std = (std + variance / std) / 2;
    for (index = 0; index < NB; index++) {
        standard[index] = (ages[index] - mean) / std;
        check += standard[index];
        printf("%4.0f -> %+.3f\n", ages[index], standard[index]);
    }
    printf("mean = %.2f, std = %.3f, new mean = %.6f\n", mean, std, check / NB);
    return 0;
}
```

نتيجة التنفيذ

```
  22 -> -1.130
  35 -> +0.000
  41 -> +0.522
  28 -> -0.609
  19 -> -1.391
  55 -> +1.739
  33 -> -0.174
  47 -> +1.043
mean = 35.00, std = 11.500, new mean = -0.000000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.28 المئين التسعون لزمن التوصيل

علم البيانات ★★ متوسط

تَعِد شركة التوصيل بأن 90% من الطلبيات تصل في أقل من مدة معينة. اكتب برنامجًا يرتب أزمنة التوصيل (بالساعات) ويحسب المئين 90 بطريقة الرتبة الأقرب: العنصر ذو الرتبة `ceil(0.9 × n)`.

خطوات الحل

1) نرتب الجدول (ترتيب بالإدراج).
2) الرتبة: `rank = (90 * n + 99) / 100` (تقريب للأعلى بأعداد صحيحة).
3) المئين هو `times[rank - 1]`.

البرنامج بلغة C

```c
#include <stdio.h>

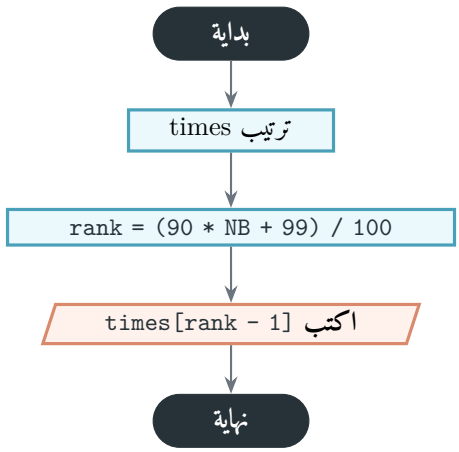
#define NB 15

int main(void) {
    int times[NB] = {24, 48, 30, 26, 72, 36, 28, 50, 25, 96, 30, 44, 27, 33, 60};
    int index, scan, key, rank;
    for (index = 1; index < NB; index++) {        // ترتيب بالإدراج
        key = times[index];
        for (scan = index - 1; scan >= 0 && times[scan] > key; scan--)
            times[scan + 1] = times[scan];
        times[scan + 1] = key;
    }
    rank = (90 * NB + 99) / 100;      // ceil(n × 0.9) بأعداد صحيحة
    printf("Sorted:");
    for (index = 0; index < NB; index++)
        printf(" %d", times[index]);
    printf("\nP90 = %d hours (rank %d of %d)\n", times[rank - 1], rank, NB);
    return 0;
}
```

نتيجة التنفيذ

```
Sorted: 24 25 26 27 28 30 30 33 36 44 48 50 60 72 96
P90 = 72 hours (rank 14 of 15)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.29 منتجان بقيمة بطاقة الهدية

تجارة إلكترونية ★★ متوسط

يملك زبون **بطاقة هدية** بمبلغ `budget` ويريد شراء **منتجين** مجموع سعريهما يساوي المبلغ بالضبط. الأسعار مرتبة. اكتب برنامجًا يجد زوجًا بطريقة **المؤشرين** في مرور واحد.

خطوات الحل

1) `left = 0` و`right = NB - 1`.
2) إذا كان المجموع أصغر من المطلوب نزيد `left`، وإذا كان أكبر ننقص `right`.
3) نتوقف عند التساوي أو عندما يلتقي المؤشران.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 9

int main(void) {
    int prices[NB] = {450, 900, 1200, 1800, 2100, 2600, 3200, 3800, 4500};
    int budget, left = 0, right = NB - 1, sum, found = 0;
    scanf("%d", &budget);
    while (left < right && !found) {
        sum = prices[left] + prices[right];
        if (sum == budget)
            found = 1;
        else if (sum < budget)
            left++;                 // نحتاج مجموعًا أكبر
        else
            right--;                // نحتاج مجموعًا أصغر
    }
    if (found)
        printf("%d + %d = %d DA\n", prices[left], prices[right], budget);
    else
        printf("No pair matches %d DA\n", budget);
    return 0;
}
```

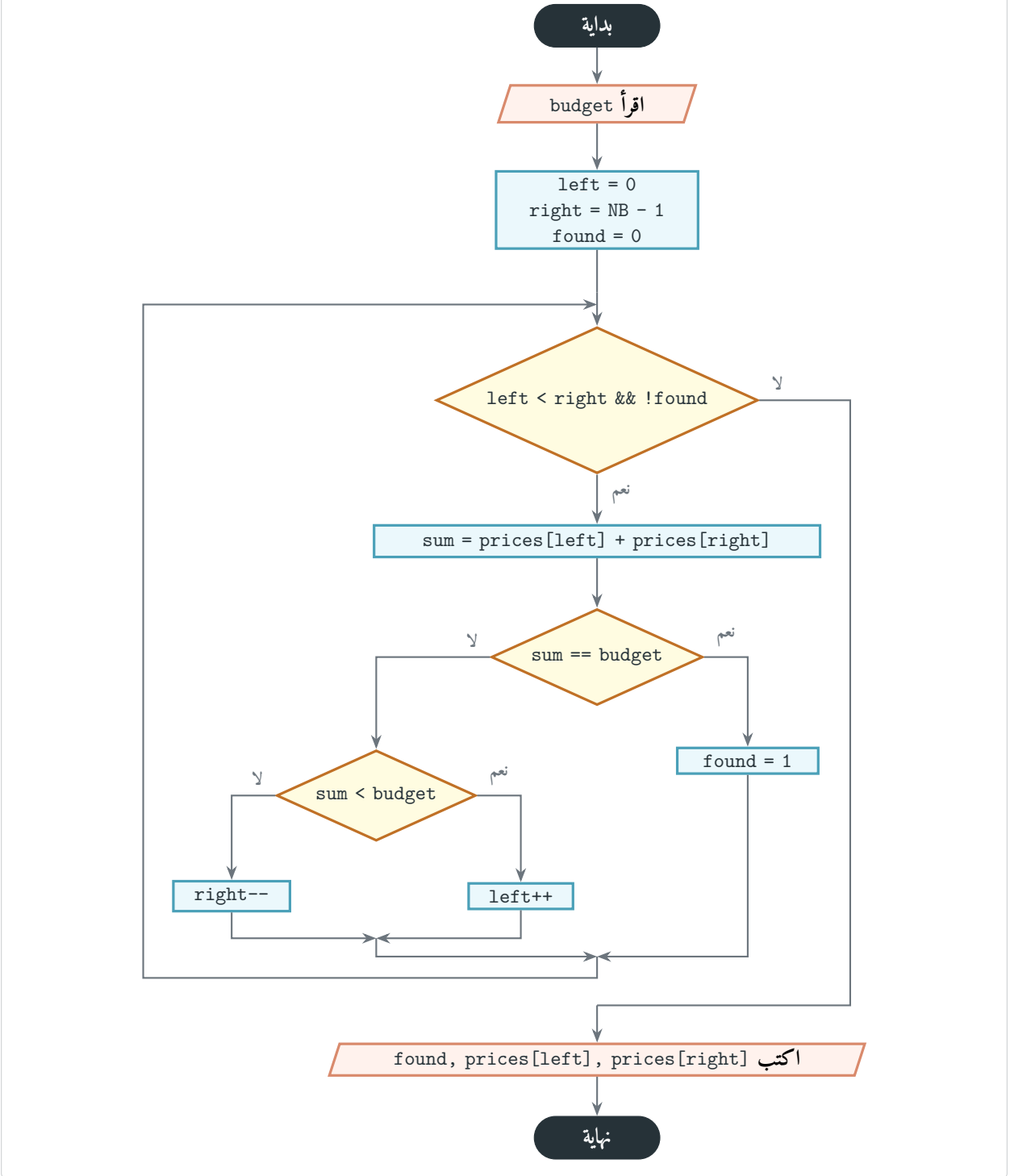
المدخلات

```
5000
```

نتيجة التنفيذ

```
1200 + 3800 = 5000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.30 التشابه بين زبونين

علم البيانات

متوسط

نمثل كل زبون بجدول عدد مشترياته في 6 أصناف. اكتب برنامجًا يحسب **تشابه جيب التمام** (`cos = (u·v) / (|u| |v|)`) بين زبونين (الجذر بطريقة نيوتن). قيمة قريبة من 1 تعني ذوقًا متشابهًا.

خطوات الحل

1) الجداء السلمي: $dot = \Sigma\ first[k] * second[k]$.
2) المعيار: $normFirst = \sqrt{\Sigma\ first[k]^2}$ ونفس الشيء للثاني.
3) $cosine = dot / (normFirst * normSecond)$.

البرنامج بلغة C

```c
#include <stdio.h>

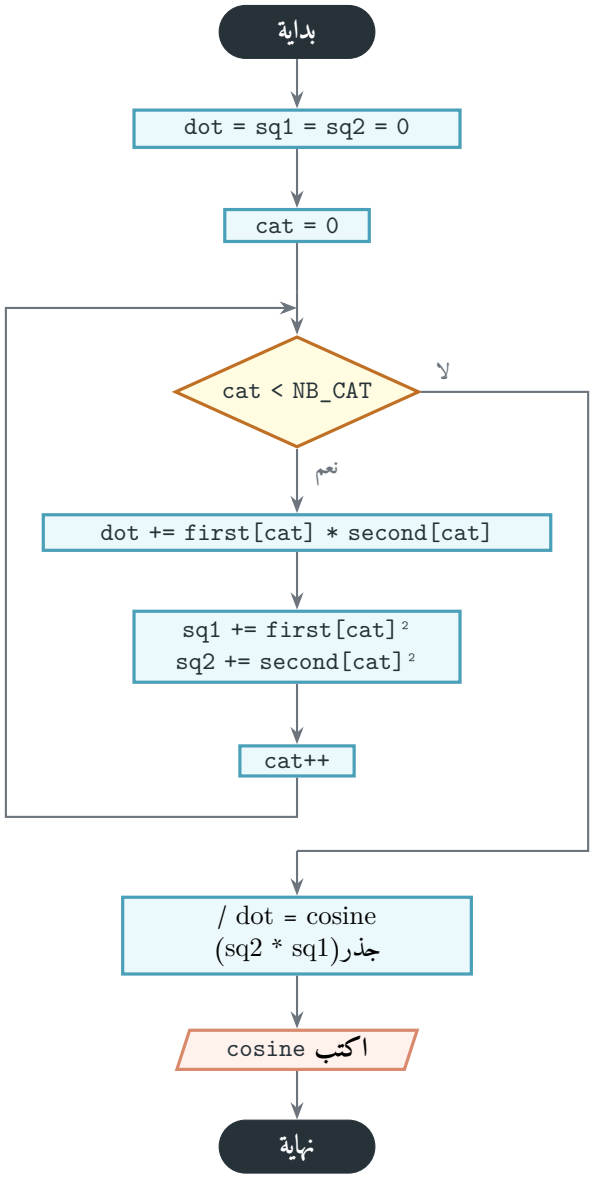
#define NB_CAT 6

int main(void) {
    double first[NB_CAT]  = {5, 0, 2, 8, 1, 0};   // مشتريات الزبون الأول
    double second[NB_CAT] = {4, 1, 3, 6, 0, 0};   // الزبون الثاني
    double dot = 0, sq1 = 0, sq2 = 0, product, root;
    int cat, step;
    for (cat = 0; cat < NB_CAT; cat++) {
        dot += first[cat] * second[cat];      // الجداء السلمي
        sq1 += first[cat] * first[cat];
        sq2 += second[cat] * second[cat];
    }
    product = sq1 * sq2;
    root = product;                           // جذر product بطريقة نيوتن
    for (step = 0; step < 40; step++)
        root = (root + product / root) / 2;
    printf("Dot = %.0f, norms product = %.4f\n", dot, root);
    printf("Cosine similarity = %.4f\n", dot / root);
    return 0;
}
```

نتيجة التنفيذ

```
Dot = 74, norms product = 76.3413
Cosine similarity = 0.9693
```

المخطط الإنسيابي (Organigramme)



تمرين 4.31 أكبر هبوط يومي

علم البيانات ★★ متوسط

اكتب برنامجًا يبني جدول **التغيرات اليومية** `diff[day] = sales[day + 1] - sales[day]` ثم يجد **أكبر هبوط** (أصغر قيمة سالبة) واليوم الذي حدث فيه.

خطوات الحل

1) جدول التغيرات أقصر بخانة واحدة.
2) نبحث عن أصغر عنصر في `diff`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 9
```

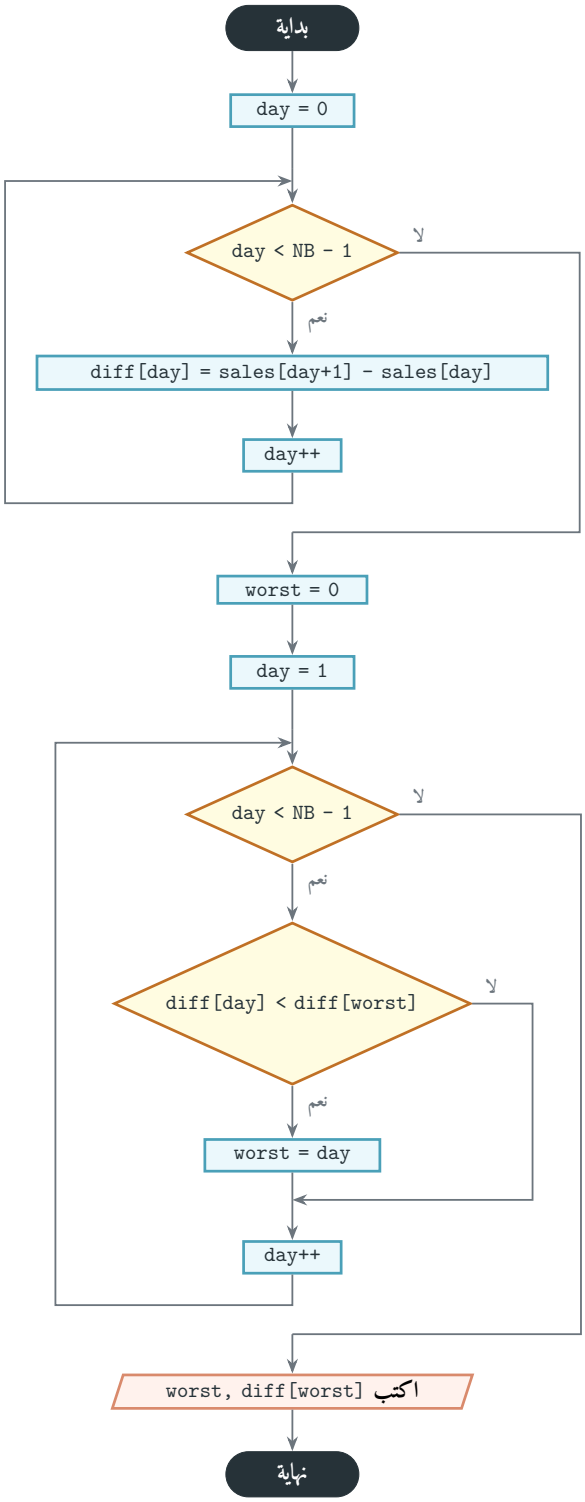
```c

int main(void) {
    int sales[NB] = {200, 230, 210, 260, 180, 190, 240, 150, 170};
    int diff[NB - 1], day, worst = 0;
    for (day = 0; day < NB - 1; day++) {
        diff[day] = sales[day + 1] - sales[day];  // تغير من يوم لآخر
        printf("%+d ", diff[day]);
    }
    for (day = 1; day < NB - 1; day++)
        if (diff[day] < diff[worst])
            worst = day;
    printf("\nBiggest drop: %d (from day %d to day %d)\n",
           diff[worst], worst + 1, worst + 2);
    return 0;
}
```

نتيجة التنفيذ

```
+30 -20 +50 -80 +10 +50 -90 +20
Biggest drop: -90 (from day 7 to day 8)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.32 القيم الشاذة بطريقة المدى الربيعي

علم البيانات ★★ متوسط

اكتب برنامجًا يكشف المبالغ **الشاذة** بطريقة **IQR**: نرتب البيانات، نأخذ الربيع الأول `Q1` (الرتبة $n/4$) والثالث `Q3` (الرتبة $3n/4$)، ثم كل قيمة خارج `[Q1 - 1.5 IQR, Q3 + 1.5 IQR]` شاذة.

خطوات الحل

1) ترتيب الجدول.
2) `q1 = values[NB / 4]` و`q3 = values[3 * NB / 4]` (تقريب بسيط).
3) `iqr = q3 - q1`، الحدان `low` و`high`.
4) نعرض القيم خارج الحدين.

البرنامج بلغة C

```c
#include <stdio.h>

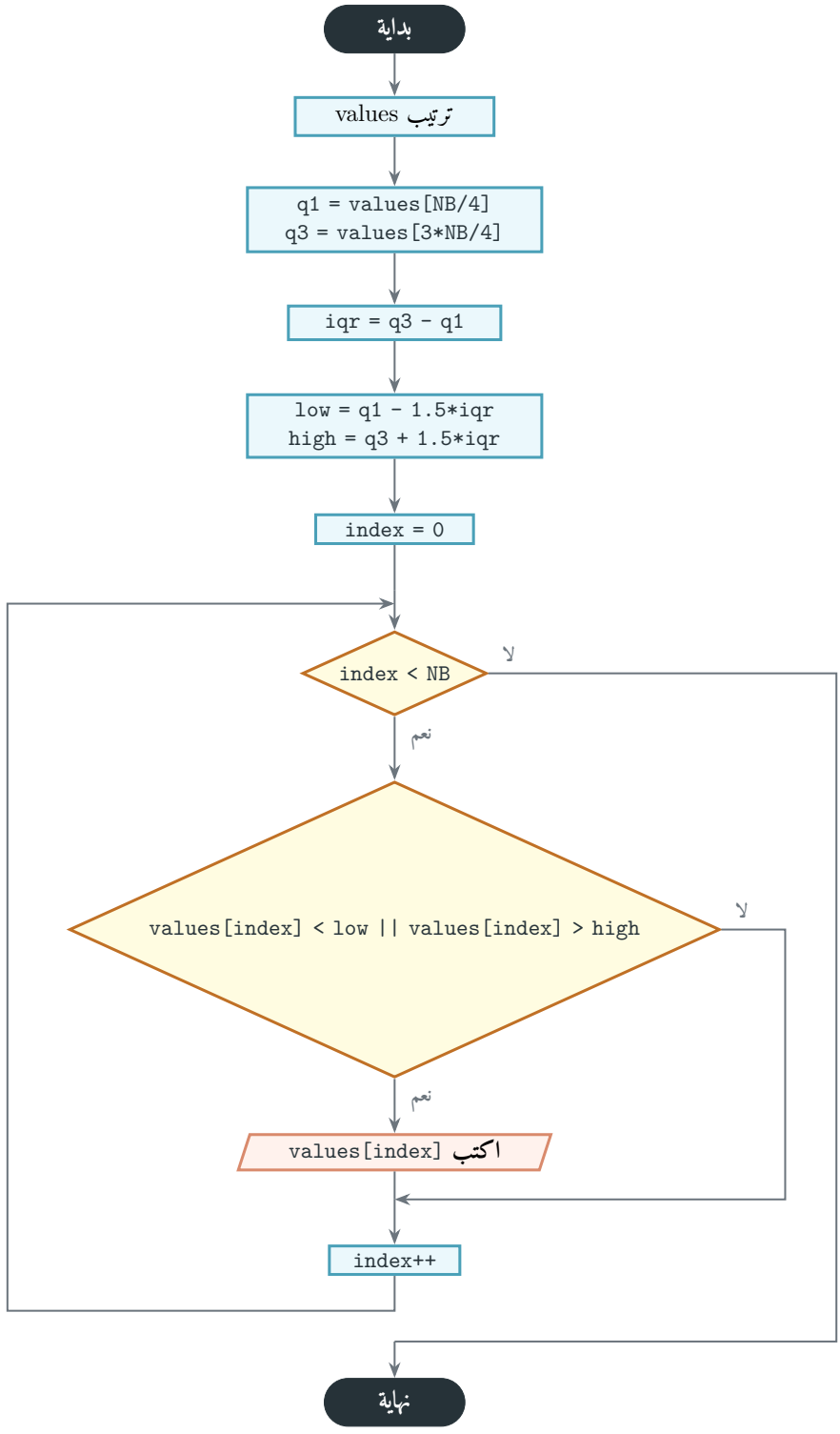
#define NB 12

int main(void) {
    double values[NB] = {3200, 2900, 3500, 41000, 3100, 2700,
                         3300, 150, 3600, 2800, 3400, 3000};
    double key, q1, q3, iqr, low, high;
    int index, scan;
    for (index = 1; index < NB; index++) {        // ترتيب بالإدراج
        key = values[index];
        for (scan = index - 1; scan >= 0 && values[scan] > key; scan--)
            values[scan + 1] = values[scan];
        values[scan + 1] = key;
    }
    q1 = values[NB / 4];                  // الربيع الأول
    q3 = values[3 * NB / 4];              // الربيع الثالث
    iqr = q3 - q1;
    low = q1 - 1.5 * iqr;
    high = q3 + 1.5 * iqr;
    printf("Q1 = %.0f, Q3 = %.0f, bounds = [%.0f, %.0f]\n", q1, q3, low, high);
    for (index = 0; index < NB; index++)
        if (values[index] < low || values[index] > high)
            printf("Outlier: %.0f\n", values[index]);
    return 0;
}
```

نتيجة التنفيذ

```
Q1 = 2900, Q3 = 3500, bounds = [2000, 4400]
Outlier: 150
Outlier: 41000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.33 نقل المنتجات النافدة إلى الآخر

تجارة إلكترونية ★★ متوسط

في صفحة النتائج نريد عرض المنتجات **المتوفرة أولًا** ثم النافدة (مخزون 0)، مع **الحفاظ على الترتيب الأصلي** داخل كل مجموعة.
اكتب برنامجًا يعيد ترتيب جدولي الأرقام والمخزون في مكانهما.

خطوات الحل

1) مؤشر الكتابة `write = 0`.
2) نمر على الجدول: كل منتج متوفر ننقله إلى `write` ثم `write++`.
3) نحتاج جدولًا مؤقتًا للنافدة لنضعها بعد ذلك بنفس ترتيبها.

البرنامج بلغة C

```c
#include <stdio.h>

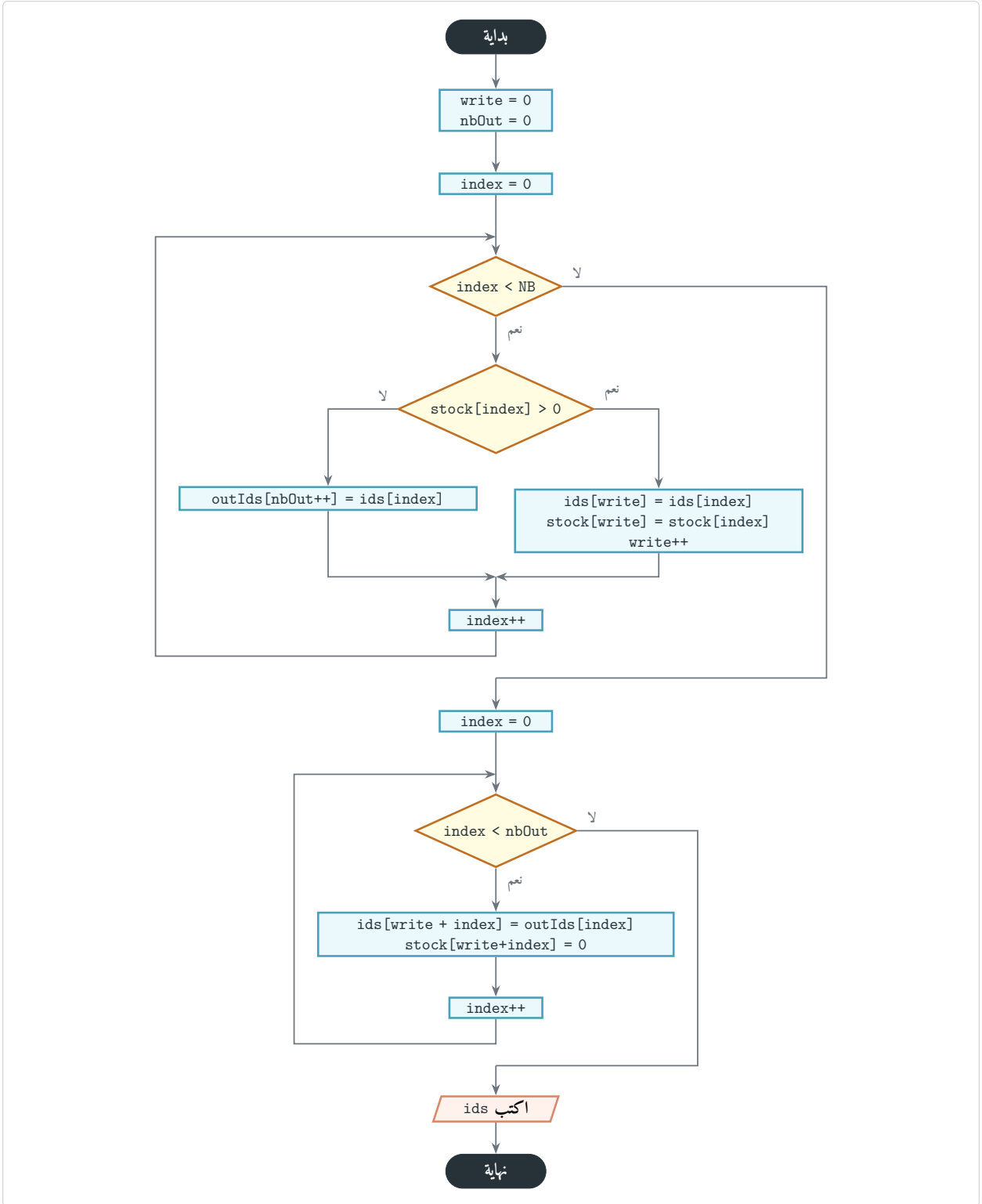
#define NB 8

int main(void) {
    int ids[NB]   = {11, 12, 13, 14, 15, 16, 17, 18};
    int stock[NB] = { 4,  0,  9,  0,  2,  7,  0,  1};
    int outIds[NB], write = 0, nbOut = 0, index;
    for (index = 0; index < NB; index++) {
        if (stock[index] > 0) {
            ids[write] = ids[index];      // دائمًا: لا نمسح شيئًا index => write
            stock[write] = stock[index];
            write++;
        } else {
            outIds[nbOut++] = ids[index]; // نحفظ النافدة جانبًا
        }
    }
    for (index = 0; index < nbOut; index++) {
        ids[write + index] = outIds[index];
        stock[write + index] = 0;
    }
    for (index = 0; index < NB; index++)
        printf("#%d(%d) ", ids[index], stock[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
#11(4) #13(9) #15(2) #16(7) #18(1) #12(0) #14(0) #17(0)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.34 غربال إراتوستينس

تجارة إلكترونية

متوسط

لإنشاء أرقام أولية لجداول التجزئة في قاعدة البيانات، اكتب برنامجًا يطبق **غربال إراتوستينس** حتى LIMIT: جدول `isPrime` نشطب فيه مضاعفات كل عدد أولي.

خطوات الحل

1) نهيّئ `isPrime[k] = 1` لكل `k >= 2`.
2) لكل `number` من 2 ما دام `number * number <= LIMIT`: إذا كان أوليًا نشطب مضاعفاته بدءًا من `number * number`.
3) نعرض الأعداد التي بقيت.

البرنامج بلغة C

```c
#include <stdio.h>

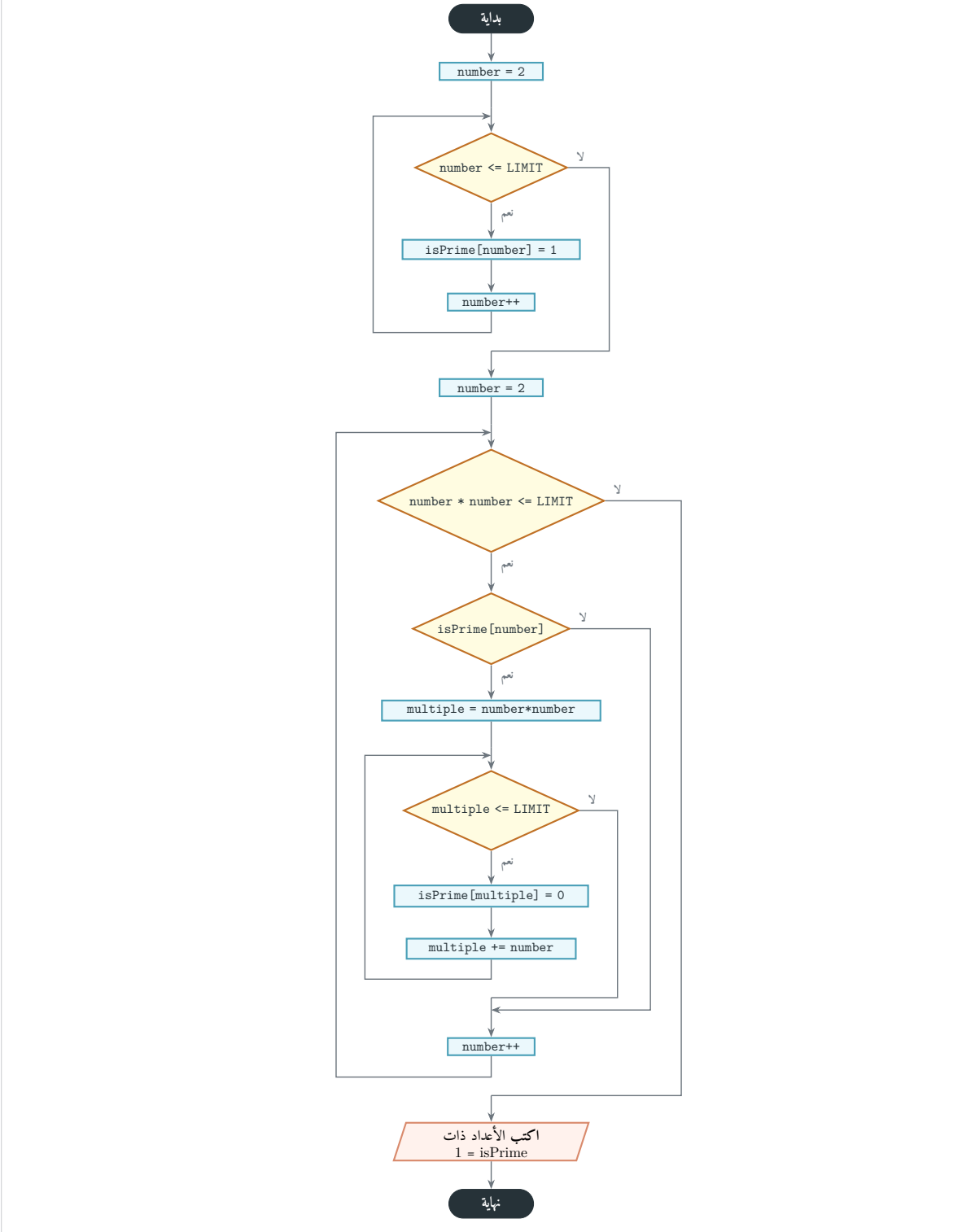
#define LIMIT 120

int main(void) {
    int isPrime[LIMIT + 1];
    int number, multiple, count = 0;
    for (number = 2; number <= LIMIT; number++)
        isPrime[number] = 1;              // كل عدد مرشح مبدئيًا
    for (number = 2; number * number <= LIMIT; number++)
        if (isPrime[number])
            // المضاعفات الأصغر من number² شُطبت سابقًا
            for (multiple = number * number; multiple <= LIMIT; multiple += number)
                isPrime[multiple] = 0;
    for (number = 2; number <= LIMIT; number++)
        if (isPrime[number]) {
            printf("%d ", number);
            count++;
        }
    printf("\n%d primes up to %d\n", count, LIMIT);
    return 0;
}
```

نتيجة التنفيذ

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109
    113
30 primes up to 120
```

المخطط الإنسيابي (Organigramme)



تمرين 4.35 عدد المنتجات المختلفة في سلة

تجارة إلكترونية

متوسط

سجل المشتريات غير مرتب ويحتوي أرقام منتجات متكررة. اكتب برنامجًا يعد المنتجات **المختلفة** ويعرض كل منتج مع عدد مرات شرائه، دون ترتيب الجدول.

خطوات الحل

1) لكل عنصر `index`: تحقق هل ظهر **قبله** (حلقة داخلية على `0..index-1`).
2) إذا لم يظهر فهو جديد: نعده ونحسب تكراره في بقية الجدول.

البرنامج بلغة C

```c
#include <stdio.h>

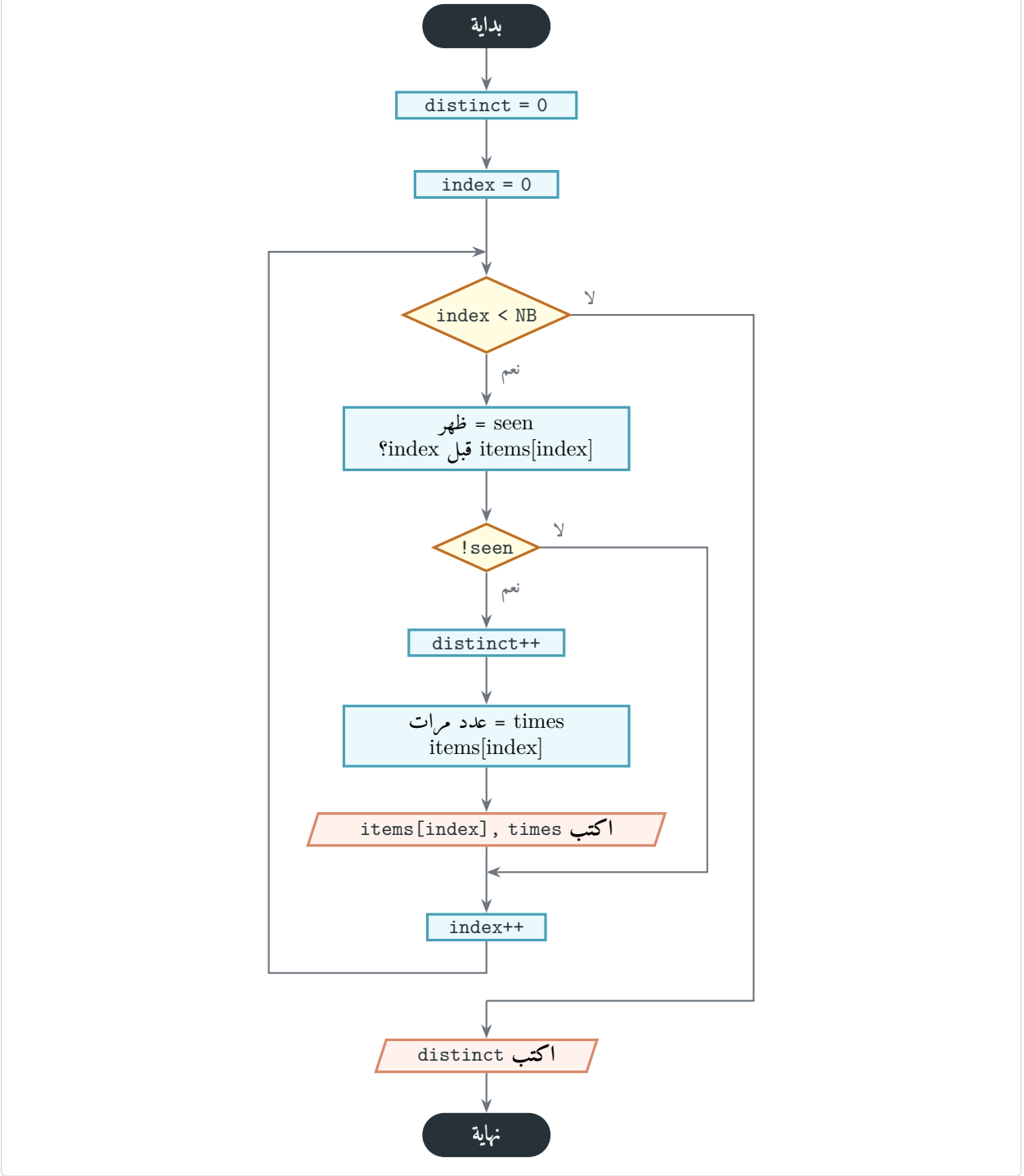
#define NB 12

int main(void) {
    int items[NB] = {204, 117, 204, 350, 117, 204, 980, 350, 411, 117, 204, 980};
    int index, scan, seen, times, distinct = 0;
    for (index = 0; index < NB; index++) {
        seen = 0;
        for (scan = 0; scan < index && !seen; scan++)
            if (items[scan] == items[index])
                seen = 1;                // عولج سابقًا
        if (!seen) {
            distinct++;
            times = 0;
            for (scan = index; scan < NB; scan++)
                if (items[scan] == items[index])
                    times++;
            printf("Product %d bought %d time(s)\n", items[index], times);
        }
    }
    printf("Distinct products: %d\n", distinct);
    return 0;
}
```

نتيجة التنفيذ

```
Product 204 bought 4 time(s)
Product 117 bought 3 time(s)
Product 350 bought 2 time(s)
Product 980 bought 2 time(s)
Product 411 bought 1 time(s)
Distinct products: 5
```

المخطط الإنسيابي (Organigramme)



تمرين 4.36 مطابقة جردين

تجارة إلكترونية

متوسط

بعد الجرد السنوي نقارن جدول المخزون **في النظام** مع جدول **العد الفعلي** لنفس المنتجات. اكتب برنامجًا يعرض المنتجات المختلفة مع الفرق، ويحكم إن كان الجردان متطابقين.

خطوات الحل

1) نمر على الجدولين بنفس الفهرس.
2) إذا كان `system[index] != counted[index]` نعرض الفرق ونزيد عداد الاختلافات.

البرنامج بلغة C

```c
#include <stdio.h>

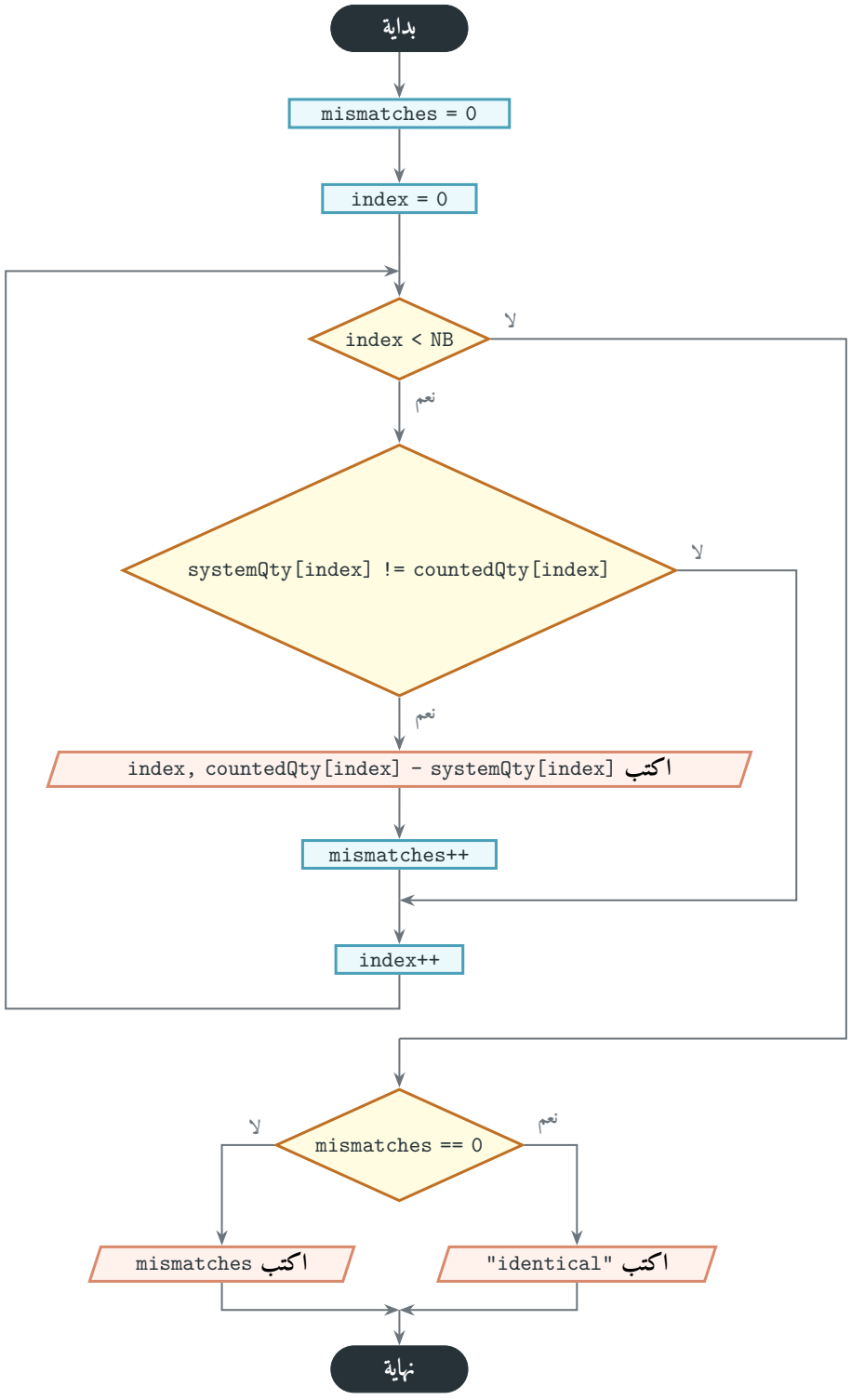
#define NB 8

int main(void) {
    int systemQty[NB]  = {40, 12, 7, 0, 25, 60, 3, 18};
    int countedQty[NB] = {40, 11, 7, 0, 27, 60, 3, 15};
    int index, mismatches = 0;
    for (index = 0; index < NB; index++) {
        if (systemQty[index] != countedQty[index]) {
            printf("Product %d: system %d, counted %d (%+d)\n", index,
                   systemQty[index], countedQty[index],
                   countedQty[index] - systemQty[index]);
            mismatches++;
        }
    }
    if (mismatches == 0)
        printf("Inventories are identical\n");
    else
        printf("%d mismatch(es) to investigate\n", mismatches);
    return 0;
}
```

نتيجة التنفيذ

```
Product 1: system 12, counted 11 (-1)
Product 4: system 25, counted 27 (+2)
Product 7: system 18, counted 15 (-3)
3 mismatch(es) to investigate
```

المخطط الإنسيابي (Organigramme)



تمرين 4.37 البتات في جدول

تجارة إلكترونية

متوسط

اكتب برنامجًا يحول رقم شحنة إلى **الثنائي** بتخزين البتات في جدول (البت الأقل وزنًا أولًا)، ثم يعرضها بالترتيب الصحيح، ويعرض أيضًا عدد الخانات.

خطوات الحل

1) ما دام number > 0: bits[nbBits++] = number % 2 و number /= 2.

2) العرض من nbBits - 1 نزولًا إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int bits[32], nbBits = 0, number, pos;
    scanf("%d", &number);
    printf("%d = ", number);
    do {
        bits[nbBits++] = number % 2;  // البتات من اليمين إلى اليسار
        number = number / 2;
    } while (number > 0);
    for (pos = nbBits - 1; pos >= 0; pos--)    // العرض بالترتيب المعاكس
        printf("%d", bits[pos]);
    printf(" (%d bits)\n", nbBits);
    return 0;
}
```

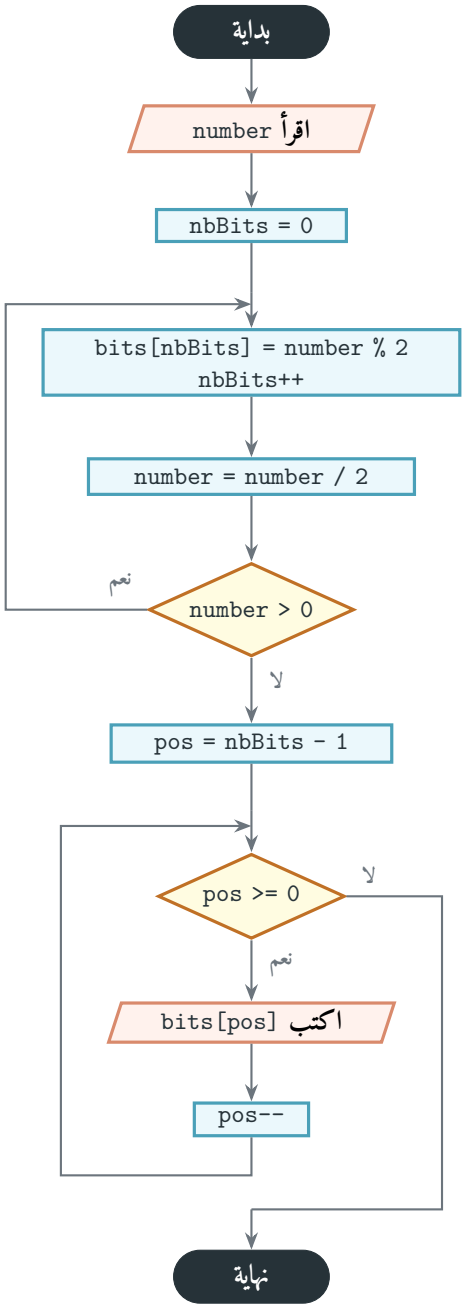
المدخلات

```
1453
```

نتيجة التنفيذ

```
1453 = 10110101101 (11 bits)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.38 جمع أعداد ضخمة

علم البيانات ★★ متوسط

رقم الأعمال السنوي لمنصة كبيرة يتجاوز سعة `long long`. نخزن العدد في **جدول من الأرقام** (الآحاد في الخانة 0). اكتب برنامجًا يجمع عددين من 25 رقمًا بخوارزمية الجمع المدرسية مع الاحتفاظ.

خطوات الحل

1) نقرأ كل عدد حرفًا بحرف بـ `scanf(" %c")` ونحول `ch - '0'`.
2) نخزن معكوسًا: الحرف الأول هو أعلى رقم.

3) لكل موضع: `sum = numA[pos] + numB[pos] + carry`، الرقم `sum % 10` والاحتفاظ `sum / 10`.

البرنامج بلغة C

```c
#include <stdio.h>

#define DIGITS 25

int main(void) {
    int numA[DIGITS], numB[DIGITS], result[DIGITS + 1];
    int pos, sum, carry = 0;
    char digit;
    for (pos = DIGITS - 1; pos >= 0; pos--) {      // الآحاد في الخانة 0
        scanf(" %c", &digit);
        numA[pos] = digit - '0';       // تحويل الحرف إلى رقم
    }
    for (pos = DIGITS - 1; pos >= 0; pos--) {
        scanf(" %c", &digit);
        numB[pos] = digit - '0';
    }
    for (pos = 0; pos < DIGITS; pos++) {
        sum = numA[pos] + numB[pos] + carry;
        result[pos] = sum % 10;        // الرقم المكتوب
        carry = sum / 10;              // الاحتفاظ للموضع التالي
    }
    result[DIGITS] = carry;
    pos = (carry > 0) ? DIGITS : DIGITS - 1;    // نتجاهل الصفر البادئ
    printf("Sum = ");
    for (; pos >= 0; pos--)
        printf("%d", result[pos]);
    printf("\n");
    return 0;
}
```

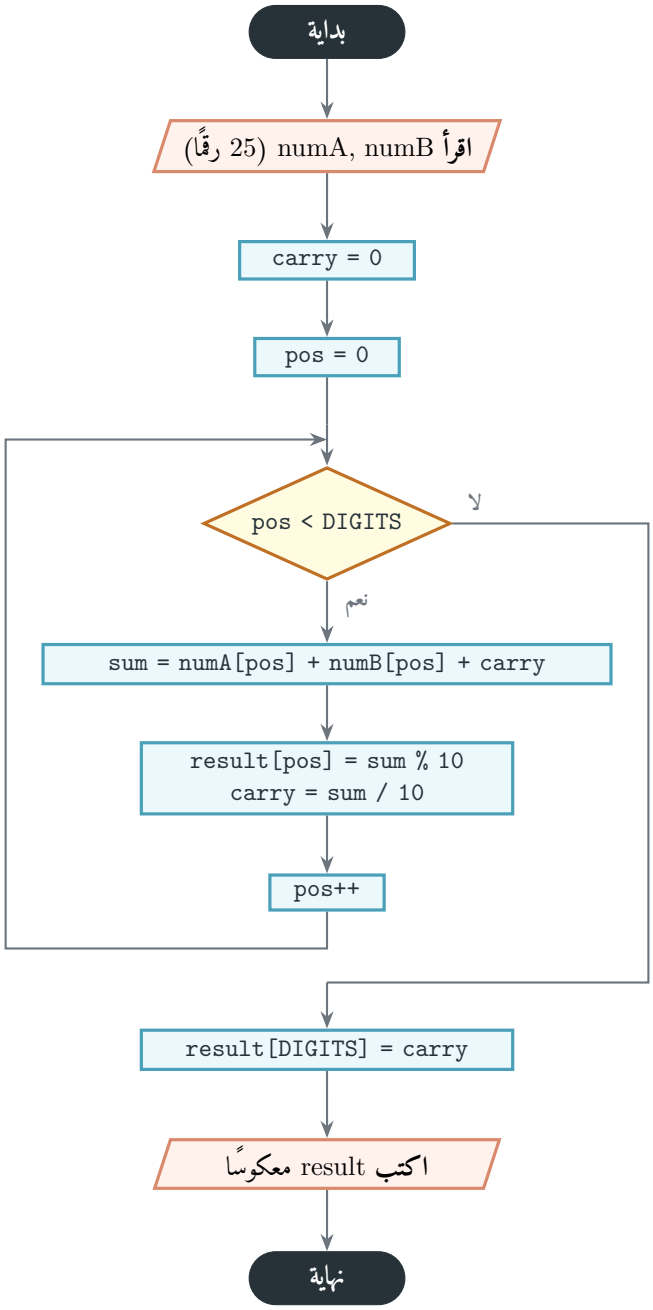
المدخلات

```
98765432109876543210987 65
12345678901234567890123 45
```

نتيجة التنفيذ

```
Sum = 111111111011111111110111110
```

المخطط الإنسيابي (Organigramme)



تمرين 4.39 أطول فترة صعود في جدول

علم البيانات

متوسط

اكتب برنامجًا يجد في جدول أسعار سهم شركة تجارة إلكترونية **أطول فترة متصلة** من الارتفاع المتتالي، ويعرض فهرسي بدايتها ونهايتها وقيمها.

خطوات الحل

1) `start` بداية الفترة الحالية.
2) إذا لم يكن `price[index] > price[index - 1]` تبدأ فترة جديدة عند `index`.
3) إذا كان طول الفترة الحالية `index - start + 1` أكبر من الأفضل نحفظها.

البرنامج بلغة C

```c
#include <stdio.h>

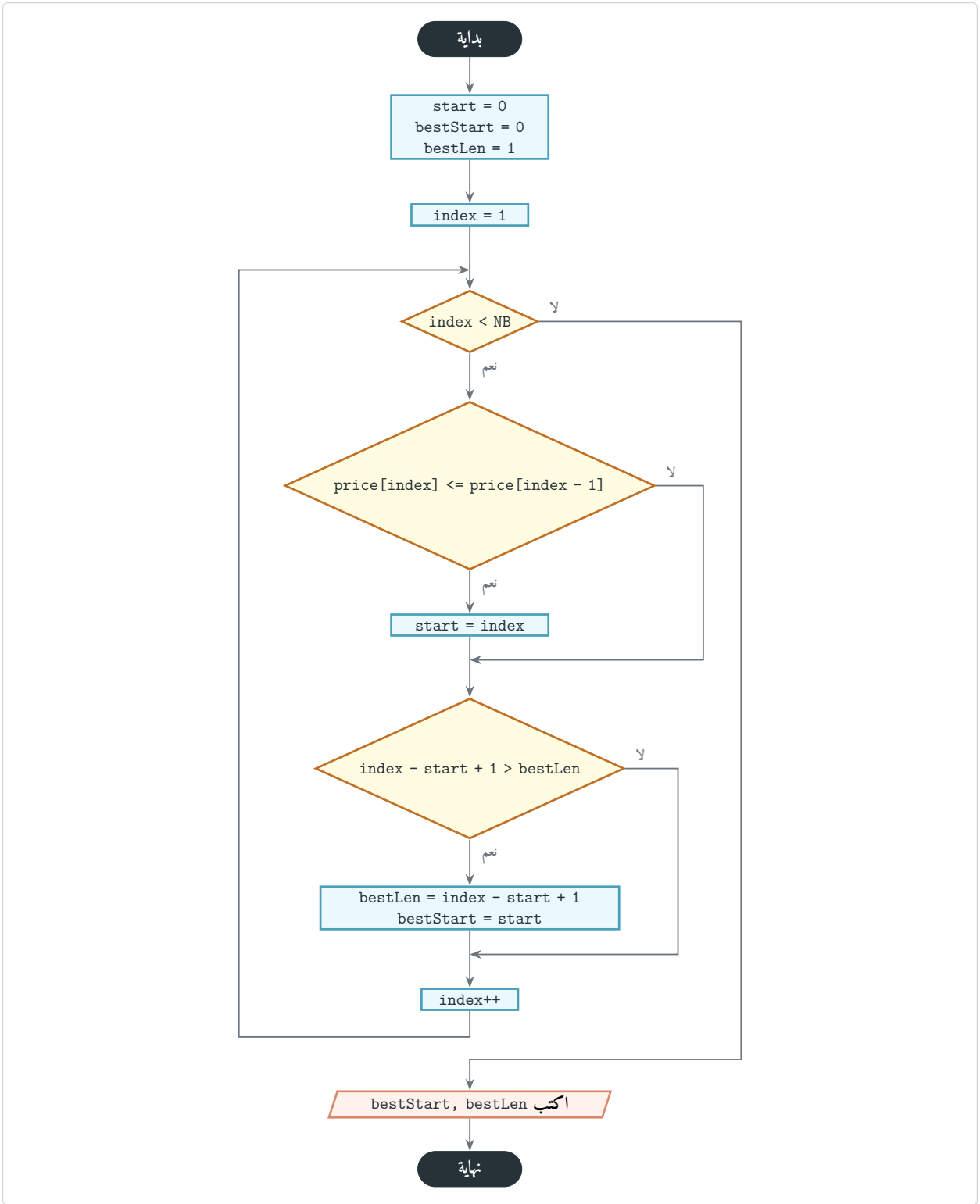
#define NB 12

int main(void) {
    double price[NB] = {50, 52, 51, 53, 55, 58, 60, 57, 59, 61, 62, 60};
    int index, start = 0, bestStart = 0, bestLen = 1;
    for (index = 1; index < NB; index++) {
        if (price[index] <= price[index - 1])
            start = index;                    // الصعود انقطع
        if (index - start + 1 > bestLen) {
            bestLen = index - start + 1;
            bestStart = start;
        }
    }
    printf("Longest rise: index %d to %d (%d values):",
           bestStart, bestStart + bestLen - 1, bestLen);
    for (index = bestStart; index < bestStart + bestLen; index++)
        printf(" %.0f", price[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Longest rise: index 2 to 6 (5 values): 51 53 55 58 60
```

المخطط الإنسيابي (Organigramme)



تمرين 4.40 الدرجة الموزونة للمنتج

علم البيانات ★★ متوسط

لترتيب نتائج البحث نعطي كل منتج **درجة** = الجداء السلمي بين جدول خصائصه (التقييم، المبيعات المطبعة، الهامش، التوفر) وجدول **الأوزان** `{0.4, 0.3, 0.2, 0.1}`. اكتب برنامجًا يحسب درجات 4 منتجات ويعرض الأفضل.

خطوات الحل

1) الخصائص في جدول أحادي واحد: المنتج `product` خصائصه من `product * 4` إلى `product * 4 + 3`.
2) الدرجة:

$$\Sigma \text{ weights[feat]} * \text{features[product * 4 + feat]}$$

.
3) نحتفظ بالأفضل.

البرنامج بلغة C

```c
#include <stdio.h>

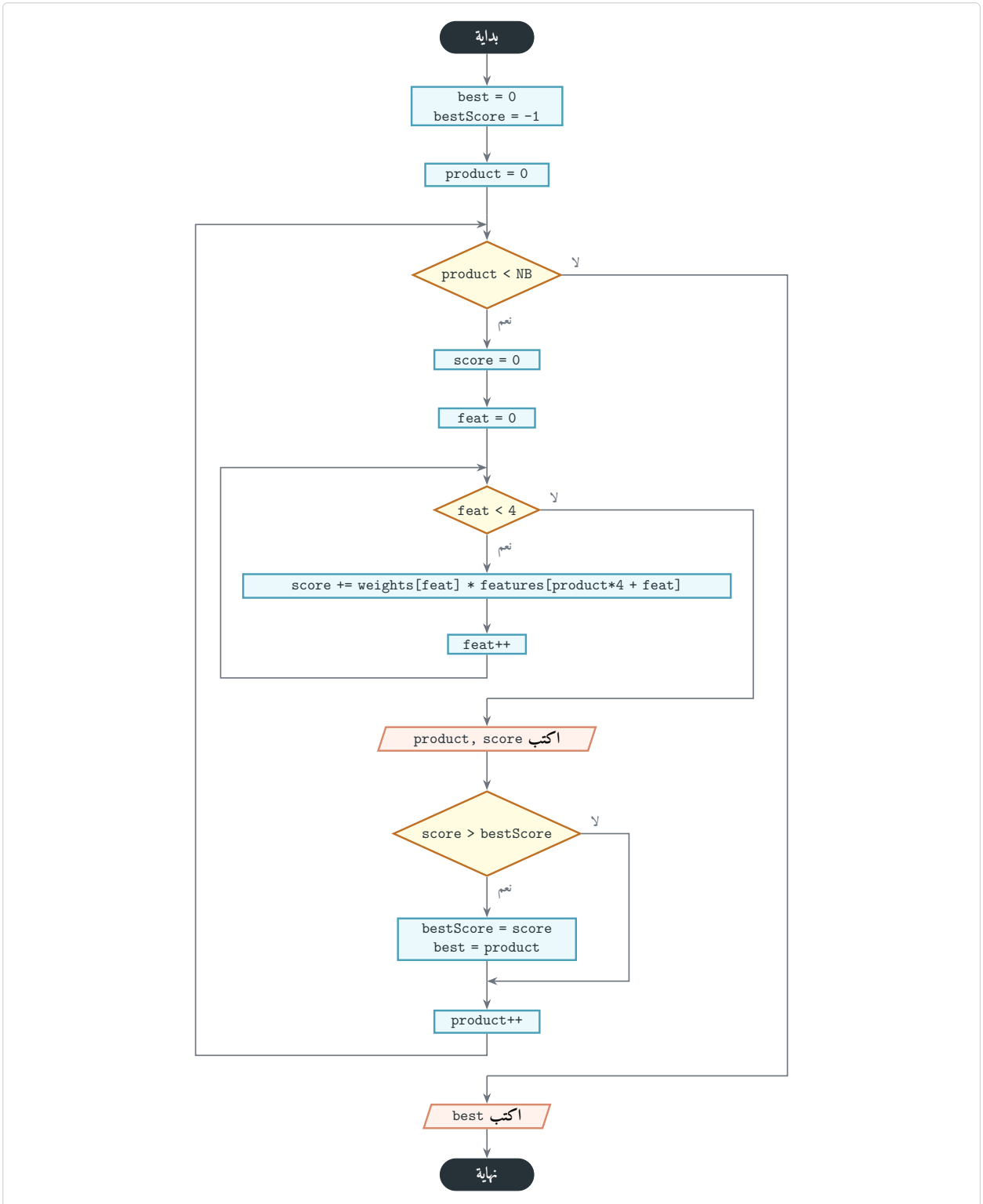
#define NB 4
#define NB_FEAT 4

int main(void) {
    double weights[NB_FEAT] = {0.4, 0.3, 0.2, 0.1};
    // كل 4 قيم متتالية = منتج: تقييم، مبيعات، هامش، توفر (بين 0 و 1)
    double features[NB * NB_FEAT] = {0.90, 0.40, 0.30, 1.0,
                                     0.70, 0.95, 0.50, 1.0,
                                     0.85, 0.60, 0.80, 0.0,
                                     0.60, 0.30, 0.90, 1.0};
    double score, bestScore = -1;
    int product, feat, best = 0;
    for (product = 0; product < NB; product++) {
        score = 0;
        for (feat = 0; feat < NB_FEAT; feat++)
            score += weights[feat] * features[product * NB_FEAT + feat];
        printf("Product %d: score %.3f\n", product, score);
        if (score > bestScore) {
            bestScore = score;
            best = product;
        }
    }
    printf("Top result: product %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Product 0: score 0.640
Product 1: score 0.765
Product 2: score 0.680
Product 3: score 0.610
Top result: product 1
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 4.41 أفضل فترة ربح متتالية (خوارزمية كادان)

علم البيانات ★★★ صعب

جدول `profit` يحتوي الربح الصافي اليومي لحملة إعلانية (قد يكون سالبًا). اكتب برنامجًا يجد **الفترة المتصلة** ذات **أكبر مجموع** بخوارزمية **كادان** في مرور واحد، مع يومي بدايتها ونهايتها.

خطوات الحل

1) `current` أفضل مجموع لفترة تنتهي عند اليوم الحالي.
2) إذا كان `current < 0` فمن الأفضل البدء من جديد: `current = profit[day]` و`start = day`.
3) وإلا: `current += profit[day]`.
4) إذا كان `current > best` نحفظ الفترة.

البرنامج بلغة C

```c
#include <stdio.h>

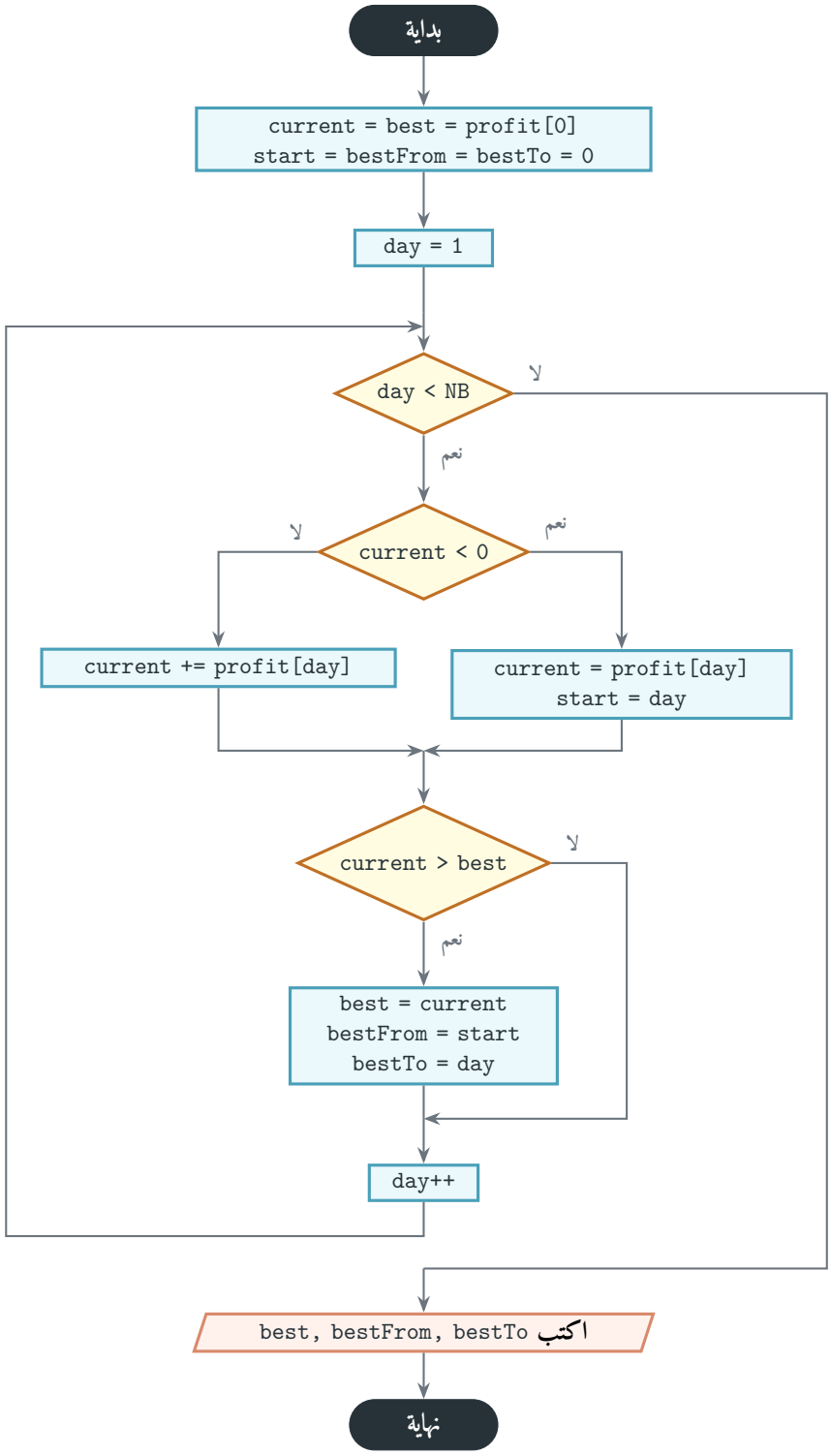
#define NB 12

int main(void) {
    int profit[NB] = {-200, 150, 300, -120, 400, -500, 100, 250, -50, 300, -400, 80};
    int current = profit[0], best = profit[0];
    int start = 0, bestFrom = 0, bestTo = 0, day;
    for (day = 1; day < NB; day++) {
        if (current < 0) {          // ماضٍ سالب يضر: نبدأ فترة جديدة
            current = profit[day];
            start = day;
        } else {
            current += profit[day];   // نمدد الفترة الحالية
        }
        if (current > best) {
            best = current;
            bestFrom = start;
            bestTo = day;
        }
    }
    printf("Best period: day %d to day %d, profit = %d DA\n",
           bestFrom + 1, bestTo + 1, best);
    return 0;
}
```

نتيجة التنفيذ

```
Best period: day 2 to day 10, profit = 830 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 4.42 الترتيب السريع دون عودية

تجارة إلكترونية

صعب ★★★★

اكتب برنامجًا يرتب أسعار 12 منتجًا بخوارزمية **الترتيب السريع** (Quick Sort) دون استعمال الدوال العودية: نستعمل **مكدسًا يدويًا** (جدولين `stackLow` و`stackHigh`) لحفظ الأجزاء التي تنتظر الترتيب. التقسيم بطريقة Lomuto مع آخر عنصر كمحور.

خطوات الحل

1) ندفع المجال `[0, NB-1]` في المكدس.
2) ما دام المكدس غير فارغ: نسحب مجالًا `[low, high]`.
3) التقسيم: كل عنصر أصغر من المحور ينقل إلى اليسار، ثم يوضع المحور في مكانه `pivotPos`.
4) ندفع الجزأين `[low, pivotPos-1]` و`[pivotPos+1, high]` إذا كان طولهما أكبر من 1.

البرنامج بلغة C

```c
#include <stdio.h>

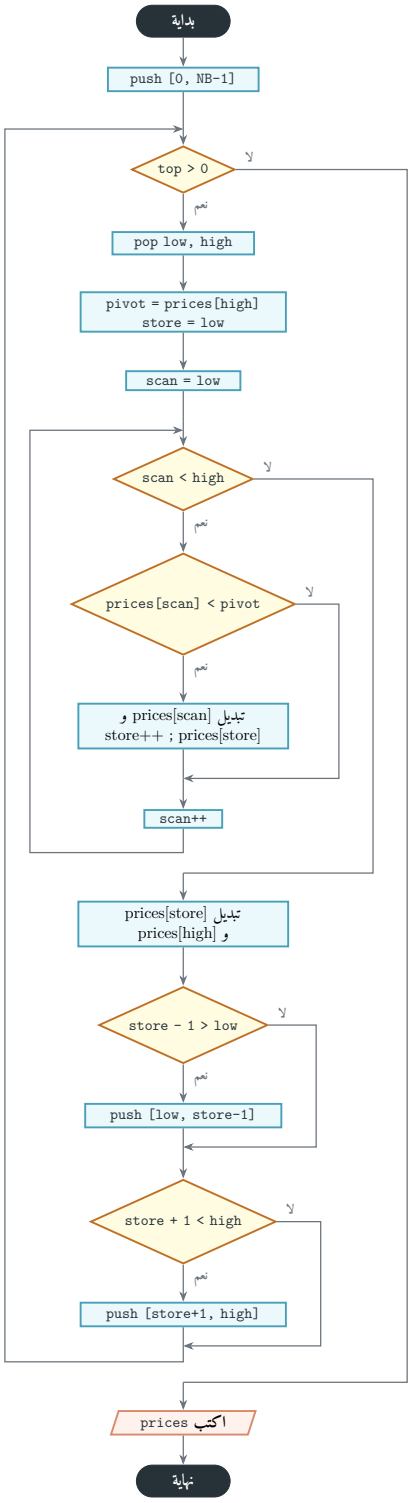
#define NB 12

int main(void) {
    int prices[NB] = {890, 120, 450, 3000, 75, 640, 1200, 300, 999, 15, 2200, 640};
    int stackLow[NB], stackHigh[NB], top = 0;
    int low, high, pivot, store, scan, tmp, index;
    stackLow[top] = 0; stackHigh[top] = NB - 1; top++;   // المجال الكامل
    while (top > 0) {
        top--;                                  // نسحب مجالًا
        low = stackLow[top];
        high = stackHigh[top];
        pivot = prices[high];                   // المحور: آخر عنصر
        store = low;                            // حد منطقة "الأصغر من المحور"
        for (scan = low; scan < high; scan++) {
            if (prices[scan] < pivot) {
                tmp = prices[scan]; prices[scan] = prices[store]; prices[store] = tmp;
                store++;
            }
        }
        tmp = prices[store]; prices[store] = prices[high]; prices[high] = tmp;
        // المحور الآن في مكانه النهائي store
        if (store - 1 > low)  { stackLow[top] = low;       stackHigh[top] = store - 1; top++; }
        if (store + 1 < high) { stackLow[top] = store + 1; stackHigh[top] = high;      top++; }
    }
    for (index = 0; index < NB; index++)
        printf("%d ", prices[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
15 75 120 300 450 640 640 890 999 1200 2200 3000
```

المخطط الإنسيابي (Organigramme)



تمرين 4.43 الترتيب بالدمج من الأسفل إلى الأعلى

علم البيانات ★★★★ صعب

اكتب برنامجًا يرتب أزمنة استجابة الخادم بخوارزمية **الترتيب بالدمج** في نسختها التكرارية (Bottom-Up): ندمج أجزاء بطول 1 ثم 2 ثم 4 ... مستعملين جدولًا مساعدًا، ونعرض الجدول بعد كل مرحلة.

خطوات الحل

1) `width` يبدأ من 1 ويتضاعف.
2) لكل زوج من الأجزاء المتجاورة `[left, mid)` و`[mid, right)`: ندمجهما في `buffer`.
3) ننسخ `buffer` إلى الجدول الأصلي بعد كل مرحلة.

البرنامج بلغة C

```c
#include <stdio.h>

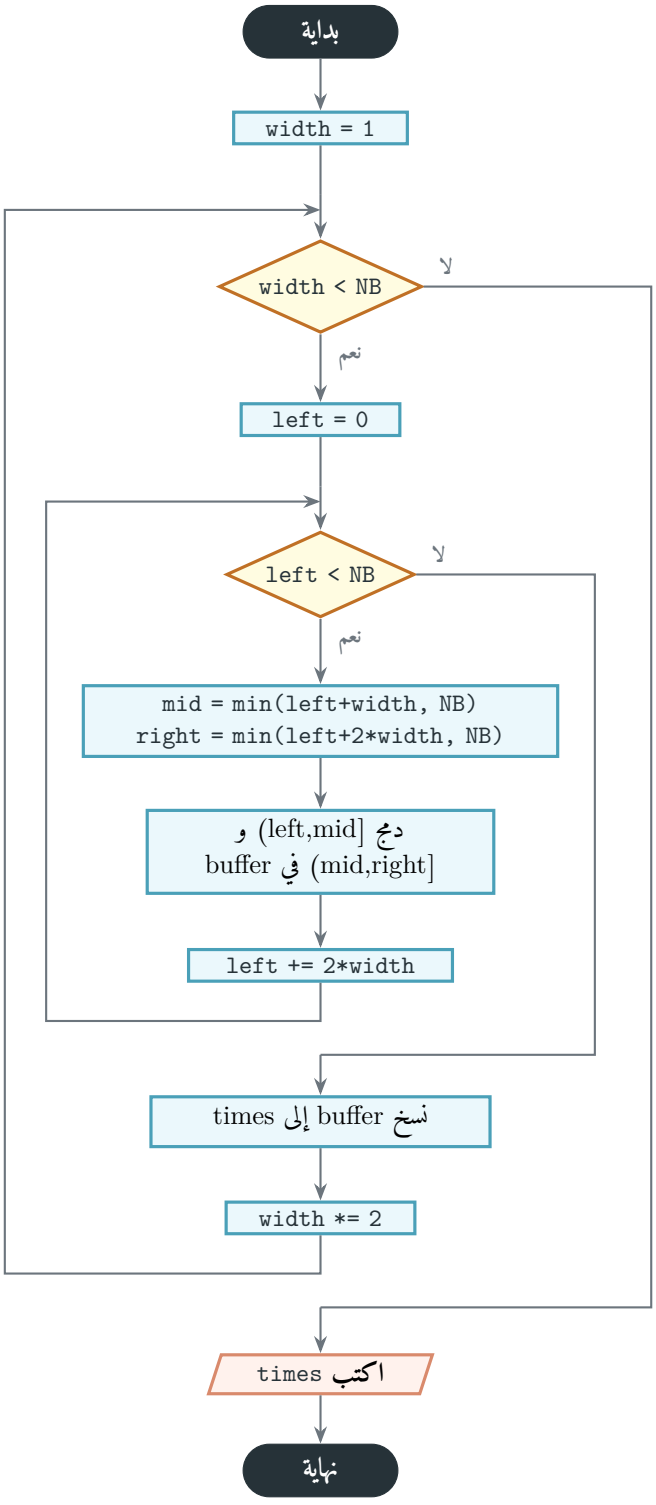
#define NB 10

int main(void) {
    int times[NB] = {120, 45, 300, 80, 95, 15, 210, 60, 33, 150};
    int buffer[NB];
    int width, left, mid, right, posA, posB, out, index;
    for (width = 1; width < NB; width *= 2) {          // طول الأجزاء المرتبة
        for (left = 0; left < NB; left += 2 * width) {
            mid = (left + width < NB) ? left + width : NB;
            right = (left + 2 * width < NB) ? left + 2 * width : NB;
            posA = left; posB = mid; out = left;
            while (posA < mid && posB < right)         // دمج جزأين مرتبين
                buffer[out++] = (times[posA] <= times[posB]) ? times[posA++] : times[posB++];
            while (posA < mid)   buffer[out++] = times[posA++];
            while (posB < right) buffer[out++] = times[posB++];
        }
        for (index = 0; index < NB; index++)
            times[index] = buffer[index];
        printf("width %d:", width);
        for (index = 0; index < NB; index++)
            printf(" %d", times[index]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
width 1: 45 120 80 300 15 95 60 210 33 150
width 2: 45 80 120 300 15 60 95 210 33 150
width 4: 15 45 60 80 95 120 210 300 33 150
width 8: 15 33 45 60 80 95 120 150 210 300
```

المخطط الإنسيابي (Organigramme)



تمرين 4.44 الترتيب بالعد لمليون تقييم

علم البيانات صعب ★★★★

لدينا عدد كبير من تقييمات المنتجات (0 إلى 10). الترتيب بالمقارنات بطيء، بينما **الترتيب بالعد** (Counting Sort) خطي. اكتب برنامجًا يولد 1000000 تقييم بمولد LCG يدوي، يعدها في جدول `count[11]`، ثم يحسب **الوسيط** و**المئين 90** مباشرة من العدادات دون تخزين الملايين.

خطوات الحل

1) مولد: `seed = (seed * 1103515245 + 12345) % 2^31`، والتقييم `(seed / 65536) % 11`.
2) `count[rating]++`.
3) نمر على العدادات تراكميًا: الوسيط هو أول قيمة يبلغ عندها التراكم `NB / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

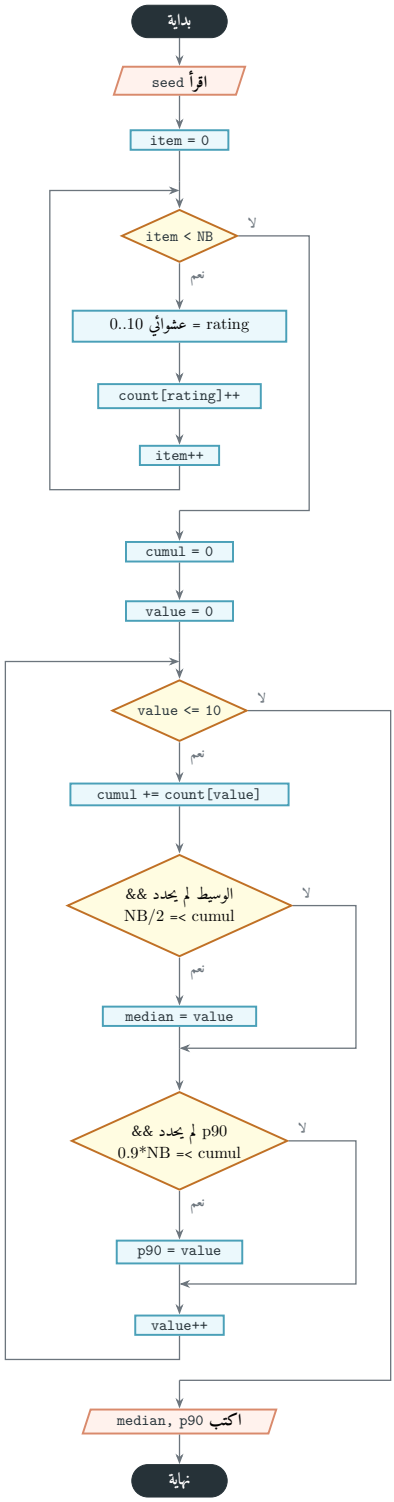
#define NB 1000000L

int main(void) {
    unsigned long long seed;
    long count[11] = {0}, cumul = 0, item;
    int value, rating, median = -1, p90 = -1;
    scanf("%llu", &seed);
    for (item = 0; item < NB; item++) {
        seed = (seed * 1103515245ULL + 12345ULL) % 2147483648ULL;
        rating = (int)((seed / 65536) % 11);  // تقييم من 0 إلى 10
        count[rating]++;                      // لا نخزن التقييم، فقط نعده
    }
    for (value = 0; value <= 10; value++) {
        cumul += count[value];                // كم تقييمًا <= value
        printf("%2d: %ld\n", value, count[value]);
        if (median < 0 && cumul >= NB / 2) median = value;
        if (p90 < 0 && cumul >= NB * 9 / 10) p90 = value;
    }
    printf("Median = %d, P90 = %d\n", median, p90);
    return 0;
}
```

نتيجة التنفيذ

```
 0: 90820
 1: 91304
 2: 91289
 3: 90572
 4: 91026
 5: 91112
 6: 90611
 7: 90940
 8: 90589
 9: 90691
10: 91046
Median = 5, P90 = 9
```

المخطط الإنسيابي (Organigramme)



تمرين 4.45 تصنيف زبون بـ k أقرب جيران — علم البيانات — صعب ★★★★

لدينا 10 زبائن معروفين (عدد الزيارات الشهرية، متوسط السلة بالآلاف) وصنف كل منهم: 1 = سيشتري الاشتراك المميز، 0 = لا. اكتب برنامجًا يصنف زبونًا جديدًا بخوارزمية **k-NN** مع `k = 3`: نحسب المسافات، نختار الثلاثة الأقرب (انتقاء جزئي)، ونصوت.

خطوات الحل

1) `dist[index]` = مربع المسافة إلى كل زبون.
2) لاختيار الأقرب 3 مرات: نبحث عن أصغر مسافة غير مستعملة ونعلّمها `used[index] = 1`.
3) نجمع أصوات الصنف 1، والقرار بالأغلبية.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 10
#define NB_NEIGHBOURS 3

int main(void) {
    double visitsData[NB] = {1, 2, 3, 8, 9, 7, 5, 10, 4, 6};
    double basketData[NB] = {1.0, 1.5, 2.0, 6.0, 5.5, 4.0, 3.5, 7.0, 2.5, 5.0};
    int label[NB]         = {0, 0, 0, 1, 1, 1, 0, 1, 0, 1};
    double dist[NB], visits, basket, dv, db;
    int used[NB] = {0}, index, round, nearest, votes = 0;
    scanf("%lf %lf", &visits, &basket);
    for (index = 0; index < NB; index++) {
        dv = visits - visitsData[index];
        db = basket - basketData[index];
        dist[index] = dv * dv + db * db;      // مربع المسافة يكفي للمقارنة
    }
    for (round = 0; round < NB_NEIGHBOURS; round++) {     // نختار الأقرب NB_NEIGHBOURS مرات
        nearest = -1;
        for (index = 0; index < NB; index++)
            if (!used[index] && (nearest < 0 || dist[index] < dist[nearest]))
                nearest = index;
        used[nearest] = 1;
        votes += label[nearest];
        printf("Neighbour %d: customer %d (d2 = %.2f, class %d)\n",
               round + 1, nearest, dist[nearest], label[nearest]);
    }
    if (votes * 2 > NB_NEIGHBOURS)                        // الأغلبية
        printf("Prediction: premium subscriber\n");
    else
        printf("Prediction: standard\n");
    return 0;
}
```

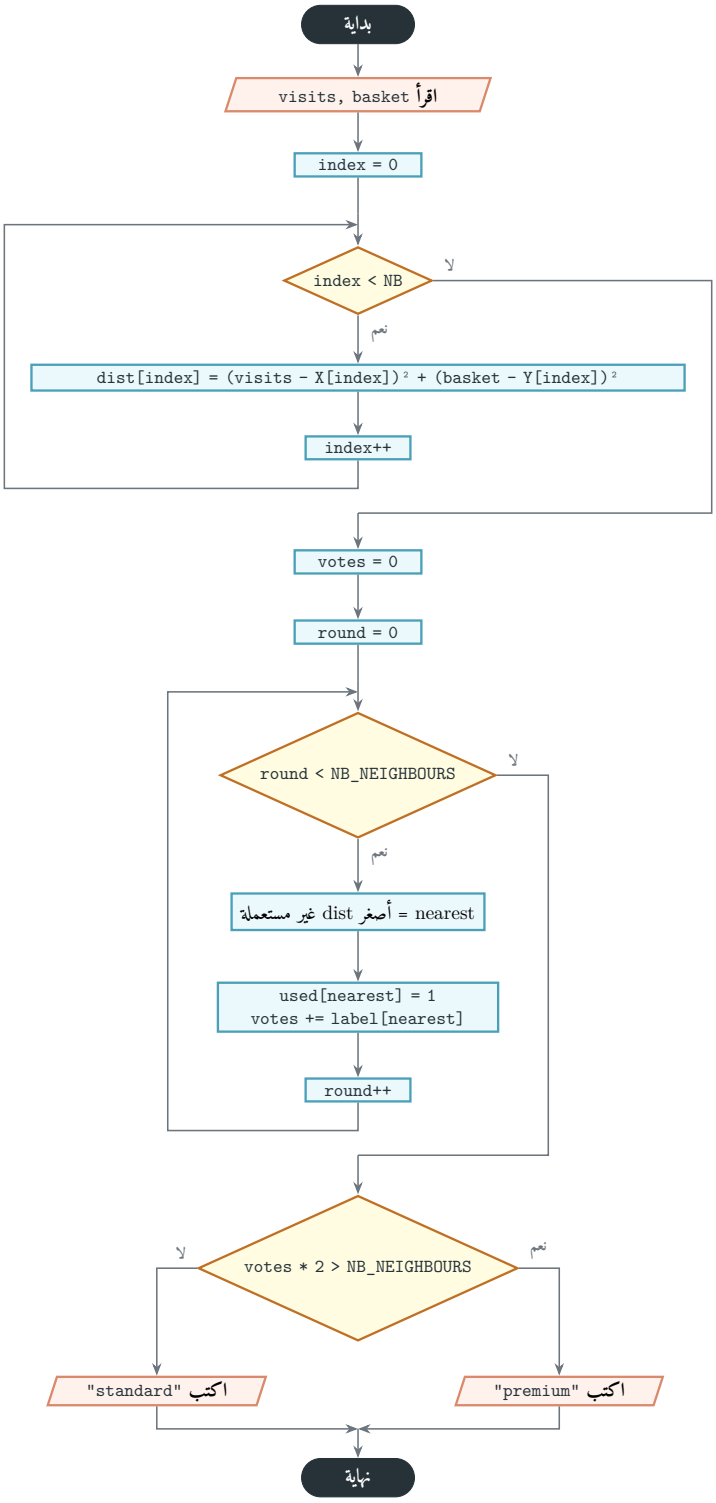
المدخلات

```
6 4.5
```

نتيجة التنفيذ

```
Neighbour 1: customer 9 (d2 = 0.25, class 1)
Neighbour 2: customer 5 (d2 = 1.25, class 1)
Neighbour 3: customer 6 (d2 = 2.00, class 0)
Prediction: premium subscriber
```

المخطط الإنسيابي (Organigramme)



تمرين 4.46 تجميع الزبائن إلى ثلاث فئات

علم البيانات ★★★ صعب

اكتب برنامجًا يطبق **K-Means** بثلاث مجموعات على إنفاق 15 زبونًا مخزن في جدول: المراكز الأولية هي أصغر قيمة، الوسط، وأكبر قيمة. نكرر الإسناد وإعادة الحساب حتى لا يتغير أي إسناد، ونعرض المراكز وأعضاء كل مجموعة.

خطوات الحل

1) `group[index]` رقم مجموعة كل زبون (يبدأ بـ -1).
2) الإسناد: أقرب مركز بالقيمة المطلقة للفرق.
3) `changed` يعد الزبائن الذين غيروا مجموعتهم؛ نتوقف عندما يساوي 0.
4) إعادة الحساب: كل مركز = متوسط أعضائه.

البرنامج بلغة C

```c
#include <stdio.h>

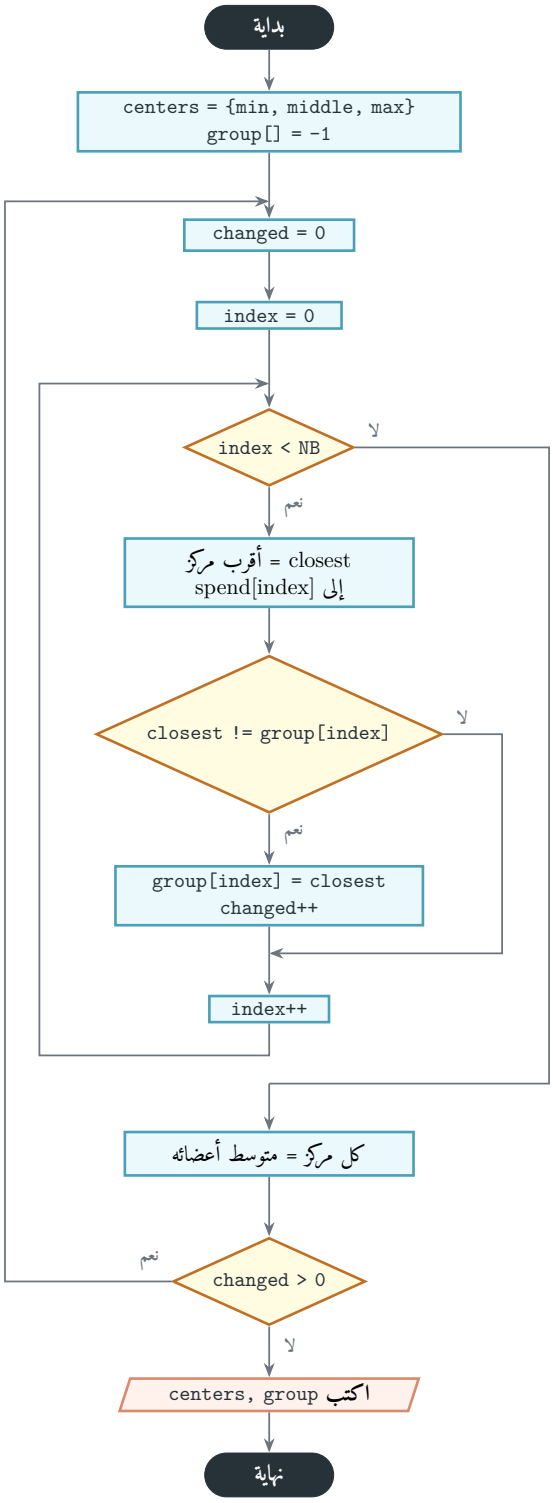
#define NB 15
#define NB_GROUPS 3

int main(void) {
    double spend[NB] = {12, 15, 90, 14, 52, 48, 95, 11, 55, 100, 47, 13, 88, 50, 92};
    double centers[NB_GROUPS] = {11, 50, 100};    // أصغر، وسط، أكبر
    double sum[NB_GROUPS], gap, bestGap;
    int group[NB], members[NB_GROUPS];
    int index, cluster, closest, changed, iteration = 0;
    for (index = 0; index < NB; index++)
        group[index] = -1;                // لا مجموعة بعد
    do {
        changed = 0;
        for (index = 0; index < NB; index++) {      // مرحلة الإسناد
            closest = 0;
            bestGap = -1;
            for (cluster = 0; cluster < NB_GROUPS; cluster++) {
                gap = spend[index] - centers[cluster];
                if (gap < 0) gap = -gap;
                if (bestGap < 0 || gap < bestGap) { bestGap = gap; closest = cluster; }
            }
            if (closest != group[index]) { group[index] = closest; changed++; }
        }
        for (cluster = 0; cluster < NB_GROUPS; cluster++) { sum[cluster] = 0; members[cluster] = 0; }
        for (index = 0; index < NB; index++) {      // مرحلة إعادة الحساب
            sum[group[index]] += spend[index];
            members[group[index]]++;
        }
        for (cluster = 0; cluster < NB_GROUPS; cluster++)
            if (members[cluster] > 0)
                centers[cluster] = sum[cluster] / members[cluster];
        iteration++;
    } while (changed > 0);
    printf("Converged in %d iterations\n", iteration);
    for (cluster = 0; cluster < NB_GROUPS; cluster++) {
        printf("Group %d (center %.2f):", cluster, centers[cluster]);
        for (index = 0; index < NB; index++)
            if (group[index] == cluster)
                printf(" %.0f", spend[index]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Converged in 2 iterations
Group 0 (center 13.00): 12 15 14 11 13
Group 1 (center 50.40): 52 48 55 47 50
Group 2 (center 93.00): 90 95 100 88 92
```

المخطط الإنسيابي (Organigramme)



تمرين 4.47 اختيار الحملات الإعلانية بميزانية محدودة — تجارة إلكترونية — صعب ★★★★

ميزانية التسويق `BUDGET` (بالآلاف). لكل حملة تكلفة وعائد متوقع، ولا يمكن تنفيذ الحملة إلا مرة واحدة. اكتب برنامجًا يجد أكبر عائد ممكن بالبرمجة الديناميكية (**مسألة حقيبة الظهر 0/1**) بجدول أحادي `best[budget]`، ثم يستنتج الحملات المختارة بجدول قرارات.

خطوات الحل

1) best[money] أكبر عائد بميزانية money باستعمال الحملات المعالجة حتى الآن.

2) لكل حملة نمر على الميزانيات تنازليًا لكي لا نستعمل الحملة مرتين:

```
best[money] = max(best[money], best[money - cost] + gain)
```

.

3) نسجل take[campaign][money] في جدول مسطح لإعادة بناء الاختيار.

البرنامج بلغة C

```c
#include <stdio.h>

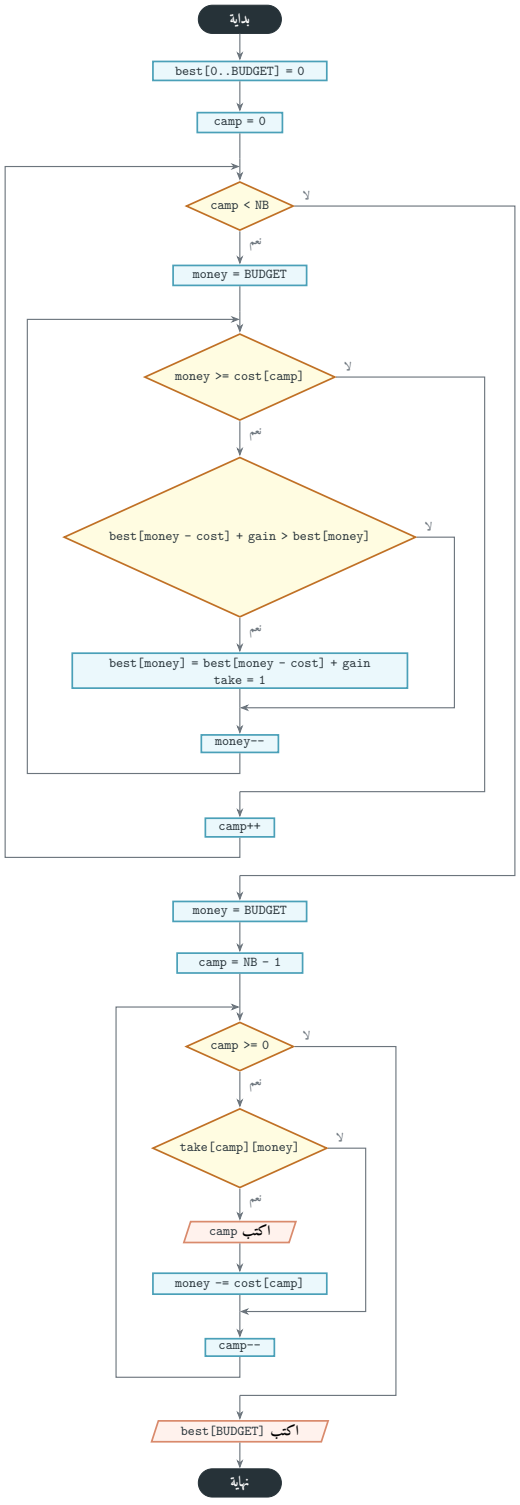
#define NB 6
#define BUDGET 20

int main(void) {
    int cost[NB] = {5, 8, 4, 10, 3, 7};        // بالآلاف
    int gain[NB] = {12, 20, 9, 25, 5, 14};     // العائد المتوقع
    int best[BUDGET + 1] = {0};
    int take[NB * (BUDGET + 1)] = {0};         // جدول قرارات مسطح
    int camp, money;
    for (camp = 0; camp < NB; camp++) {
        // تنازليًا: best[money - cost] ما زال يخص الحملات السابقة
        for (money = BUDGET; money >= cost[camp]; money--) {
            if (best[money - cost[camp]] + gain[camp] > best[money]) {
                best[money] = best[money - cost[camp]] + gain[camp];
                take[camp * (BUDGET + 1) + money] = 1;
            }
        }
    }
    printf("Max gain = %d with budget %d\n", best[BUDGET], BUDGET);
    money = BUDGET;
    printf("Campaigns:");
    for (camp = NB - 1; camp >= 0; camp--) {  // نعيد بناء الاختيار من الآخر
        if (take[camp * (BUDGET + 1) + money]) {
            printf(" #%d(cost %d)", camp, cost[camp]);
            money -= cost[camp];
        }
    }
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Max gain = 46 with budget 20
Campaigns: #3(cost 10) #2(cost 4) #0(cost 5)
```

المخطط الإنسيابي (Organigramme)



تمرين 4.48 المنتجات التي تُشترى معًا

علم البيانات ★★★★ صعب

نمثل كل سلة بقناع بتات: البت p يساوي 1 إذا احتوت السلة المنتج p (6 منتجات). اكتب برنامجًا يحسب لكل زوج من المنتجات **الدعم** (عدد السلات التي تحتوي الاثنين) ويعرض الأزواج التي يتجاوز دعمها 30%، مع **الثقة** support(A,B) / support(A) — أساس خوارزمية Apriori.

خطوات الحل

1) `single[p]` عدد السلات التي فيها المنتج p: نختبر `(basket >> p) & 1`.
2) لكل زوج `first < second`: نعد السلات التي فيها البتان.
3) نعرض الأزواج التي `pairCount * 100 / NB_BASKETS >= 30`.

البرنامج بلغة C

```c
#include <stdio.h>

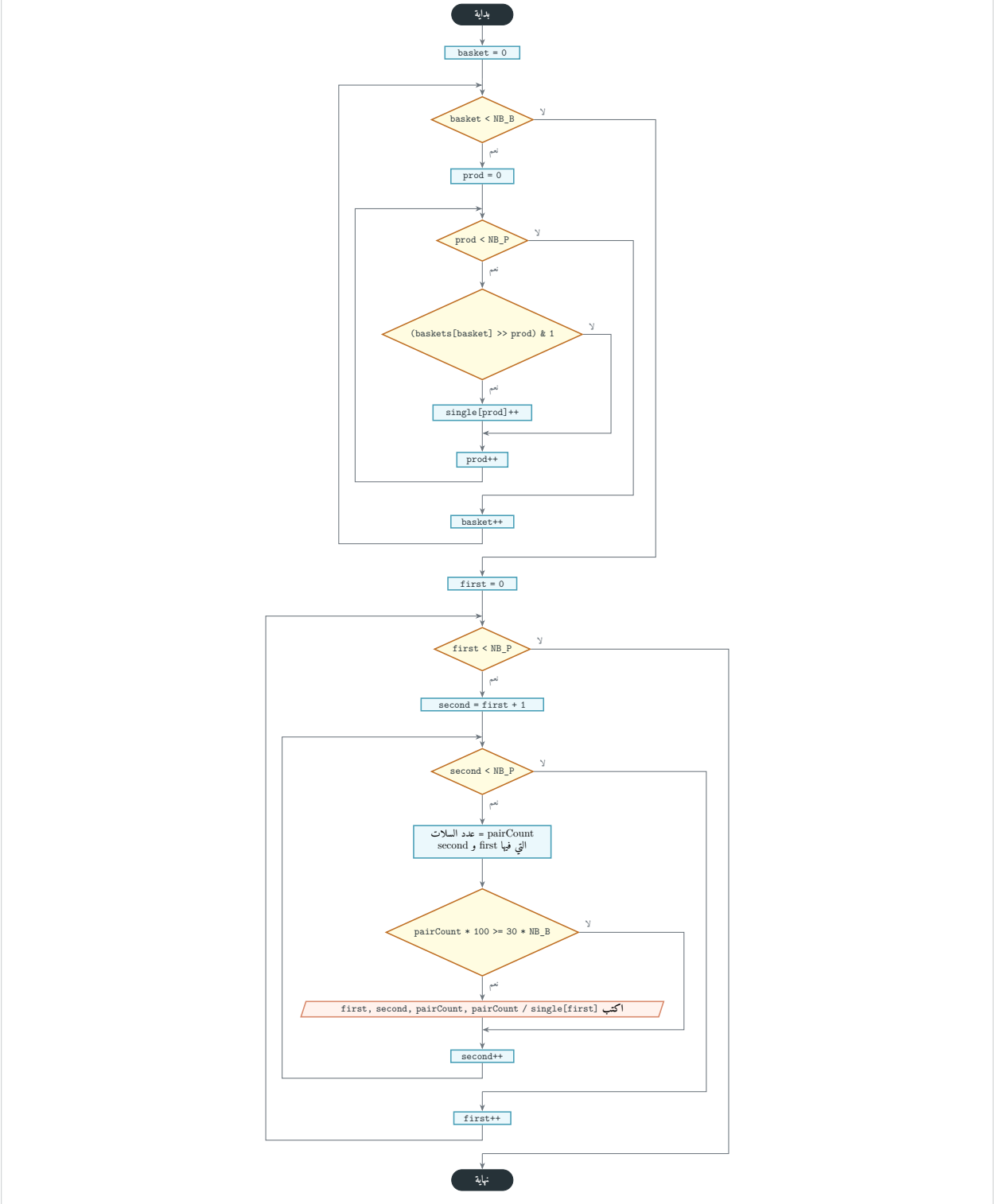
#define NB_B 10
#define NB_P 6

int main(void) {
    // 0 هاتف، 1 غلاف، 2 شاحن، 3 سماعات، 4 ساعة، 5 حاسوب
    int baskets[NB_B] = {0x07, 0x03, 0x0B, 0x01, 0x23, 0x0F, 0x30, 0x03, 0x13, 0x07};
    int single[NB_P] = {0};
    int basket, prod, first, second, pairCount, mask;
    for (basket = 0; basket < NB_B; basket++)
        for (prod = 0; prod < NB_P; prod++)
            if ((baskets[basket] >> prod) & 1)    // هل البت prod يساوي 1؟
                single[prod]++;
    for (first = 0; first < NB_P; first++) {
        for (second = first + 1; second < NB_P; second++) {
            mask = (1 << first) | (1 << second);  // البتان معًا
            pairCount = 0;
            for (basket = 0; basket < NB_B; basket++)
                if ((baskets[basket] & mask) == mask)
                    pairCount++;
            if (pairCount * 100 >= 30 * NB_B)
                printf("{%d,%d}: support %d%%, confidence %d->%d = %.2f\n",
                       first, second, pairCount * 100 / NB_B, first, second,
                       (double)pairCount / single[first]);
        }
    }
    return 0;
}
```

نتيجة التنفيذ

```
{0,1}: support 80%, confidence 0->1 = 0.89
{0,2}: support 30%, confidence 0->2 = 0.33
{1,2}: support 30%, confidence 1->2 = 0.38
```

المخطط الإنسيابي (Organigramme)



تمرين 4.49 معامل جيني لتركز الإنفاق

علم البيانات ★★★★ صعب

يقيس **معامل جيني** مدى تركز الإنفاق عند قلة من الزبائن (0 = توزيع متساوٍ، 1 = زبون واحد ينفق كل شيء). بعد ترتيب الإنفاق تصاعديًا: `G = (2 Σ i·x_i) / (n Σ x_i) - (n + 1) / n` حيث `i` من 1 إلى `n`. اكتب برنامجًا يحسبه، ويحسب أيضًا حصة أكبر 20% من الزبائن.

خطوات الحل

1) ترتيب تصاعدي.
2) `weighted = Σ (rank + 1) * spend[rank]` و`total = Σ spend`.
3) نطبق الصيغة.
4) أكبر 20% هم آخر `NB / 5` عناصر بعد الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>

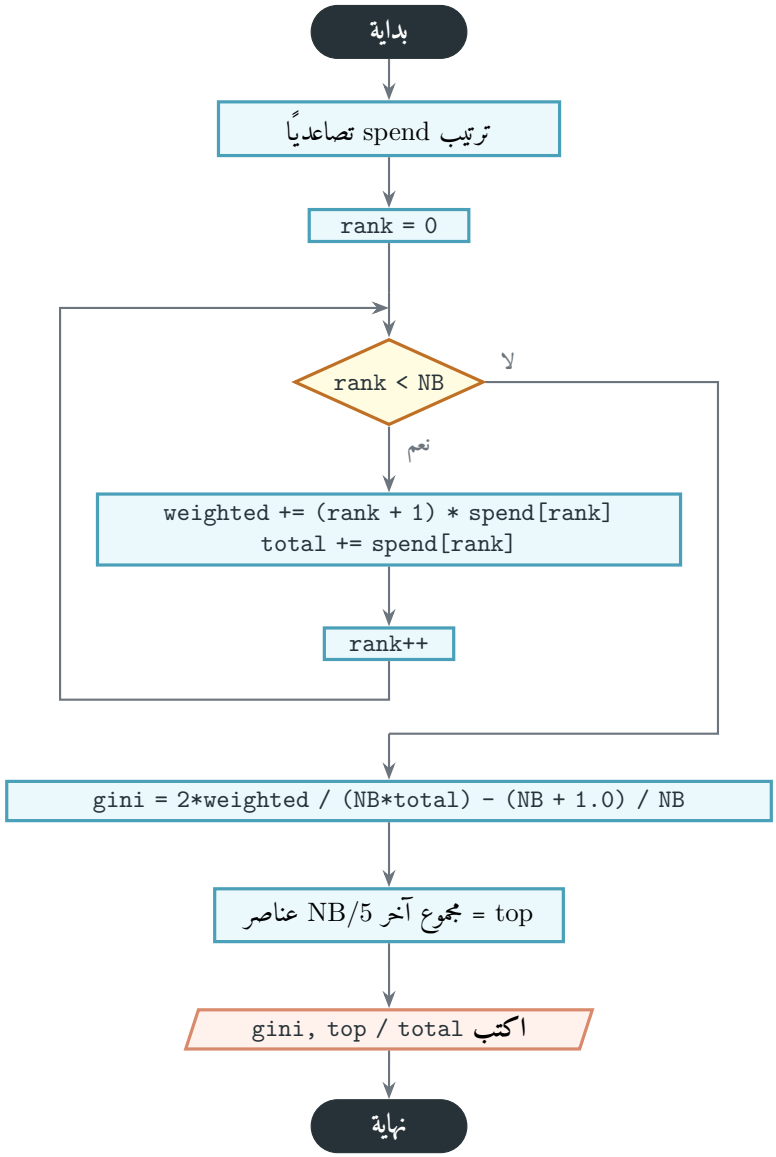
#define NB 10

int main(void) {
    double spend[NB] = {1200, 800, 45000, 3000, 950, 2200, 18000, 600, 1500, 7000};
    double key, weighted = 0, total = 0, gini, top = 0;
    int rank, scan;
    for (rank = 1; rank < NB; rank++) {          // ترتيب بالإدراج
        key = spend[rank];
        for (scan = rank - 1; scan >= 0 && spend[scan] > key; scan--)
            spend[scan + 1] = spend[scan];
        spend[scan + 1] = key;
    }
    for (rank = 0; rank < NB; rank++) {
        weighted += (rank + 1) * spend[rank];    // الرتبة تبدأ من 1
        total += spend[rank];
    }
    gini = 2 * weighted / (NB * total) - (NB + 1.0) / NB;
    for (rank = NB - NB / 5; rank < NB; rank++)  // أكبر 20%
        top += spend[rank];
    printf("Gini = %.3f\n", gini);
    printf("Top 20%% customers = %.1f%% of revenue\n", top * 100 / total);
    return 0;
}
```

نتيجة التنفيذ

```
Gini = 0.693
Top 20% customers = 78.5% of revenue
```

المخطط الإنسيابي (Organigramme)



تمرين 4.50 أطول فترة ضمن الميزانية

تجارة إلكترونية صعب ★★★

يريد مسؤول الإعلانات تشغيل حملة في **أيام متتالية**، وتكلفة كل يوم معروفة. اكتب برنامجًا يجد **أطول** سلسلة أيام متتالية مجموع تكاليفها لا يتجاوز الميزانية، بطريقة **النافذة المنزلقة** في `O(n)`.

خطوات الحل

1) مؤشرا النافذة `left` و`right`، ومجموعها `windowCost`.
2) نوسع النافذة يمينًا بإضافة `cost[right]`.
3) ما دام المجموع يتجاوز الميزانية نقلصها من اليسار.
4) نحفظ أطول نافذة صالحة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 14

int main(void) {
    int cost[NB] = {300, 150, 200, 500, 100, 120, 90, 400, 80, 60, 250, 700, 50, 100};
    int budget, left = 0, right, windowCost = 0, bestLen = 0, bestLeft = 0, day;
    scanf("%d", &budget);
    for (right = 0; right < NB; right++) {
        windowCost += cost[right];        // نضيف يومًا من اليمين
        while (windowCost > budget) {     // تجاوزنا: نحذف من اليسار
            windowCost -= cost[left];
            left++;
        }
        if (right - left + 1 > bestLen) {
            bestLen = right - left + 1;
            bestLeft = left;
        }
    }
    printf("Longest campaign: %d days (day %d to %d):",
           bestLen, bestLeft + 1, bestLeft + bestLen);
    windowCost = 0;
    for (day = bestLeft; day < bestLeft + bestLen; day++) {
        printf(" %d", cost[day]);
        windowCost += cost[day];
    }
    printf(" = %d DA\n", windowCost);
    return 0;
}
```

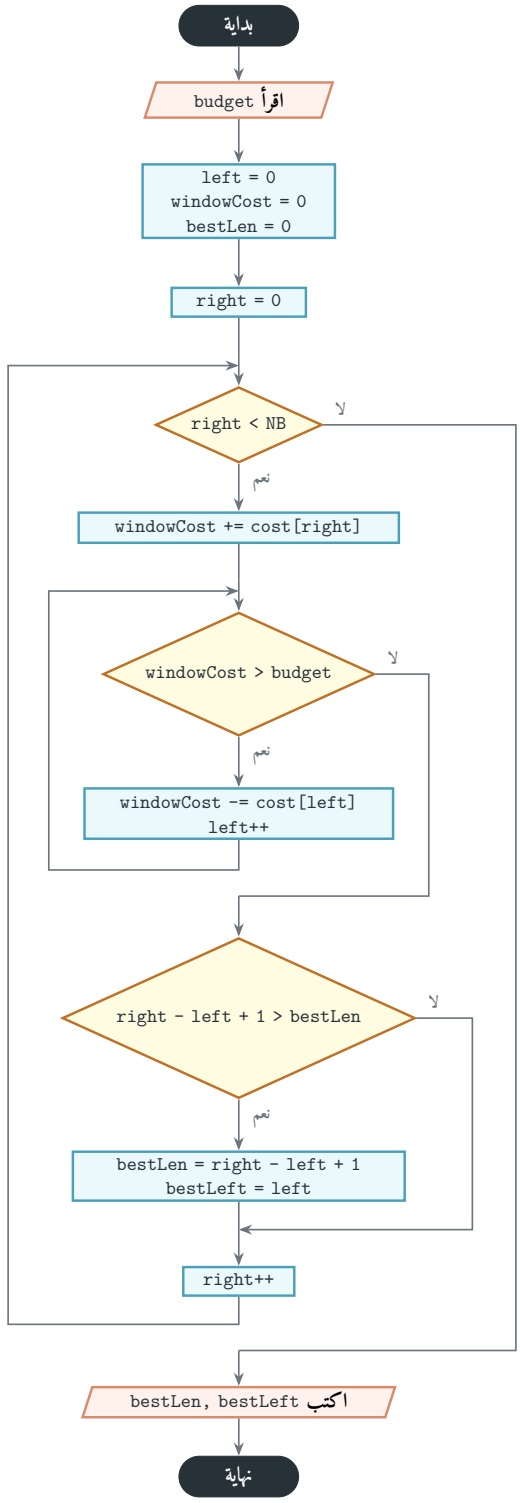
المدخلات

```
1000
```

نتيجة التنفيذ

```
Longest campaign: 6 days (day 5 to 10): 100 120 90 400 80 60 = 850 DA
```

المخطط الإنسيابي (Organigramme)



الفصل 5

الجداول ثنائية البعد

تعريفات

- **المصفوفة** (جدول ثنائي البعد): جدول من الأسطر والأعمدة، نصل إلى كل خانة بفهرسين `matrix[row][col]`.
- **التصريح**: `int sales[3][4];` ثلاثة أسطر وأربعة أعمدة (12 خانة).
- **التخزين**: الأسطر متتالية في الذاكرة (سطر بعد سطر).
- **المرور**: حلقتان متداخلتان، الخارجية على الأسطر والداخلية على الأعمدة (أو العكس لجميع الأعمدة).

	day 3	day 2	day 1	day 0	
	180	140	95	120	store 0
	90	75	60	80	store 1
sales[2][...] →	250	190	210	200	store 2

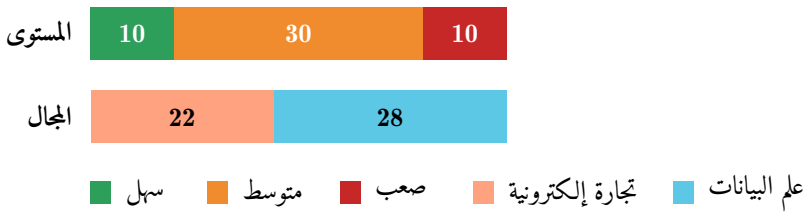
العملية	الفكرة
مجموع سطر	الحلقة الداخلية على الأعمدة، والمجمّع يُهيأ لكل سطر
مجموع عمود	نبدل الحلقتين: الخارجية على الأعمدة
المنقول	`transposed[col][row] = matrix[row][col]`
الجداء	`product[row][col] += left[row][mid] * right[mid][col]` (ثلاث حلقات)
البرمجة الديناميكية	كل خانة تُحسب من خانات سبق حسابها (فوق، يسار)

ترتيب الفهارس

`matrix[row][col]`: السطر أولًا ثم العمود. الخلط بينهما يقرأ خانات خاطئة أو يتجاوز حدود المصفوفة.

للاستزادة: [5, 12, 13]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 5.1 جدول مبيعات المتاجر

تجارة إلكترونية ⭐ سهل

مبيعات 3 متاجر خلال 4 أيام مخزنة في **مصفوفة** `sales[3][4]` (السطر = المتجر، العمود = اليوم). اكتب برنامجًا يعرضها على شكل جدول منسق.

خطوات الحل

1) تهيئ المصفوفة بأقواس متداخلة: سطر لكل متجر.
2) حلقة على الأسطر، وداخلها حلقة على الأعمدة.
3) نطبع سطرًا جديدًا بعد كل متجر.

البرنامج بلغة C

```c
#include <stdio.h>

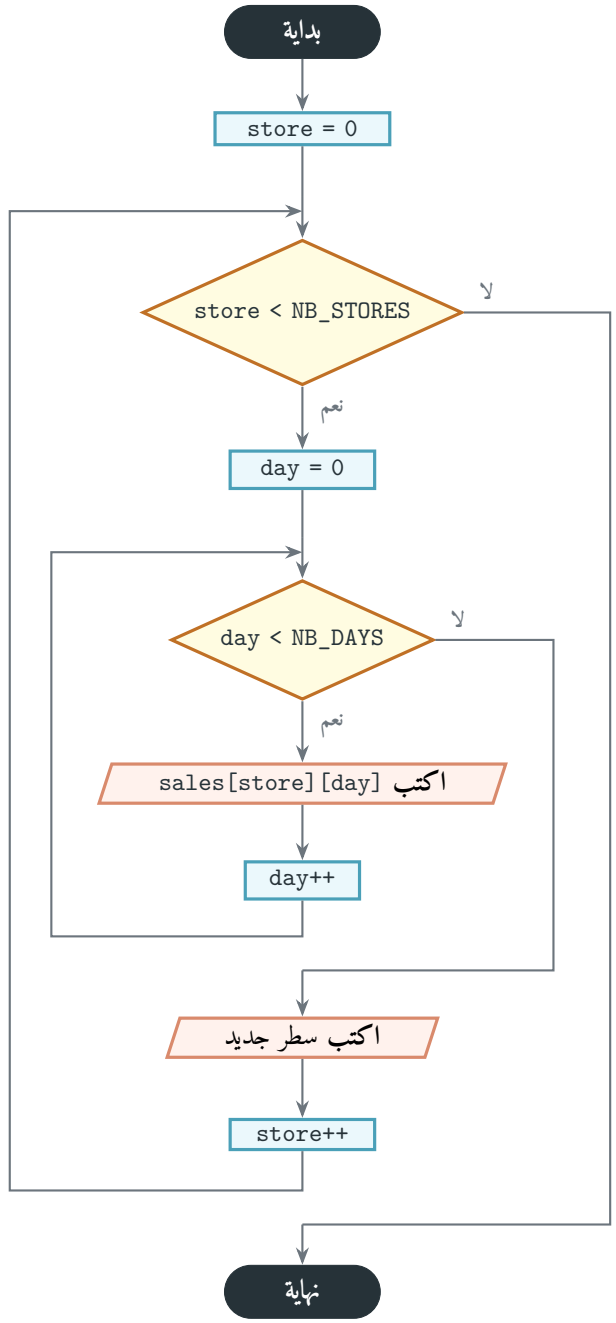
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {
        {120, 95, 140, 180},        // المتجر 0
        { 80, 60,  75,  90},        // المتجر 1
        {200, 210, 190, 250}        // المتجر 2
    };
    int store, day;
    printf("Store | D1  D2  D3  D4\n");
    for (store = 0; store < NB_STORES; store++) {      // الأسطر
        printf("%5d |", store);
        for (day = 0; day < NB_DAYS; day++)           // الأعمدة
            printf(" %3d", sales[store][day]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store | D1  D2  D3  D4
    0 | 120  95 140 180
    1 |  80  60  75  90
    2 | 200 210 190 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.2 المجموع الكلي للمبيعات

علم البيانات ★ سهل

اكتب برنامجًا يحسب مجموع كل خانات مصفوفة المبيعات (3 متاجر × 4 أيام) والمتوسط لكل خانة.

خطوات الحل

1) مجمّع واحد `total`.
2) حلقتان متداخلتان تضيفان كل خانة.
3) المتوسط: `total / (NB_STORES * NB_DAYS)`.

البرنامج بلغة C

```c
#include <stdio.h>

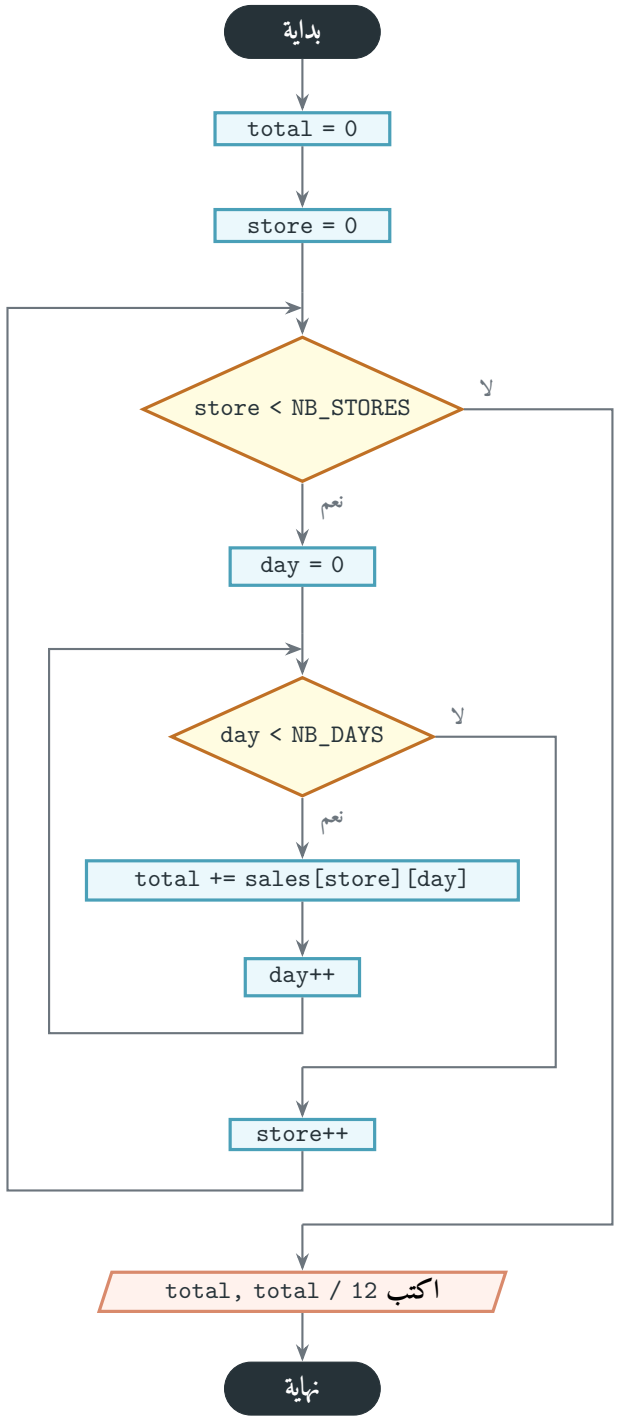
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, total = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            total += sales[store][day];   // كل خانة مرة واحدة
    printf("Total = %d\n", total);
    printf("Average per cell = %.2f\n", (double)total / (NB_STORES * NB_DAYS));
    return 0;
}
```

نتيجة التنفيذ

```
Total = 1690
Average per cell = 140.83
```

المخطط الإنسيابي (Organigramme)



تمرين 5.3 مجموع كل متجر

تجارة إلكترونية

سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل متجر** (مجموع كل سطر) ويخزنها في جدول `storeTotal`.

خطوات الحل

1) لكل سطر: نهيئ `storeTotal[store] = 0` ثم نجمع أعمدته.

البرنامج بلغة C

```c
#include <stdio.h>

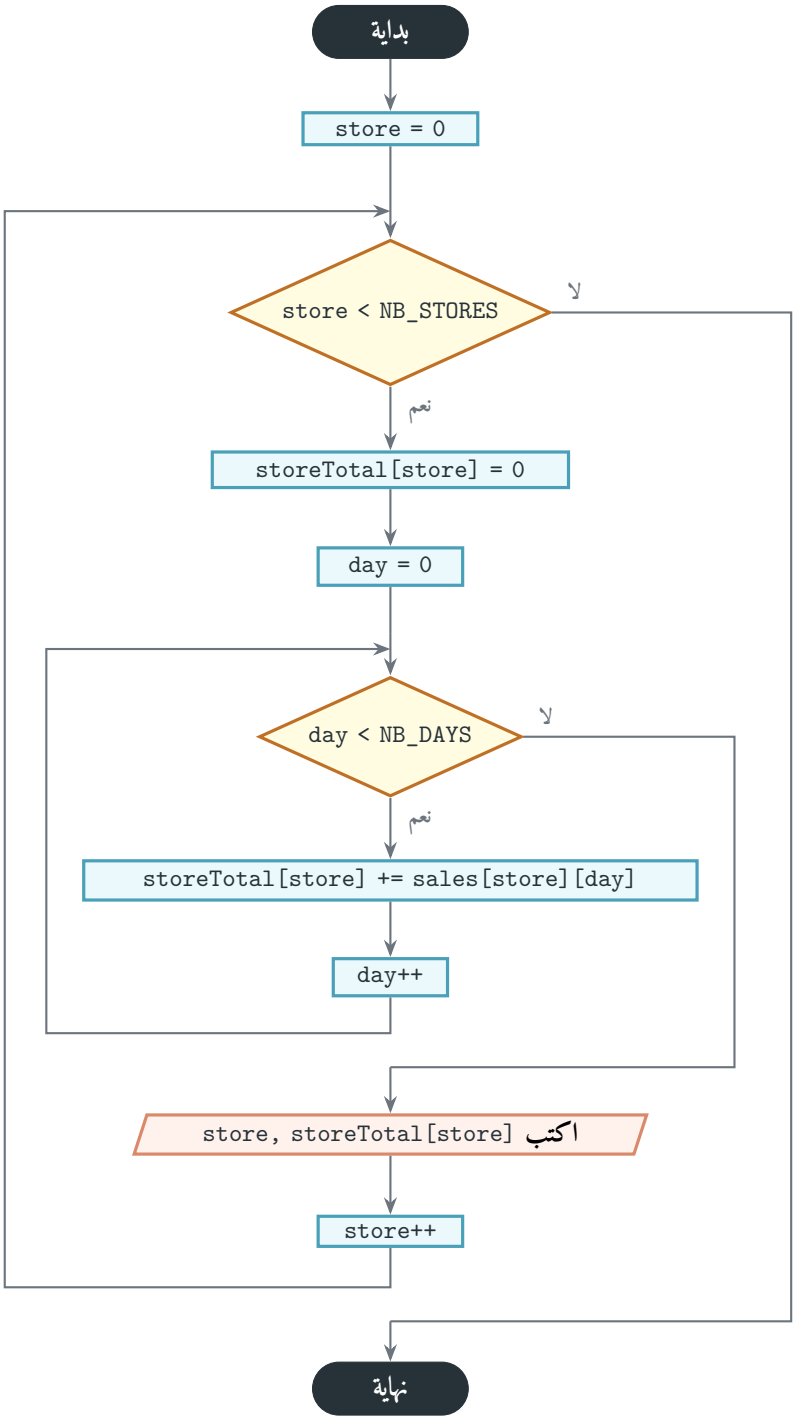
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int storeTotal[NB_STORES];
    int store, day;
    for (store = 0; store < NB_STORES; store++) {
        storeTotal[store] = 0;            // مجمّع خاص بكل سطر
        for (day = 0; day < NB_DAYS; day++)
            storeTotal[store] += sales[store][day];
        printf("Store %d: %d\n", store, storeTotal[store]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Store 0: 535
Store 1: 305
Store 2: 850
```

المخطط الإنسيابي (Organigramme)



تمرين 5.4 مجموع كل يوم

تجارة إلكترونية

سهل

اكتب برنامجًا يحسب مجموع مبيعات **كل يوم** في كل المتاجر (مجموع كل عمود). لاحظ أن الحلقة الخارجية تصبح على الأعمدة.

خطوات الحل

1) الحلقة الخارجية على `day`، والداخلية على `store`.
2) `dayTotal[day] += sales[store][day]`.

البرنامج بلغة C

```c
#include <stdio.h>

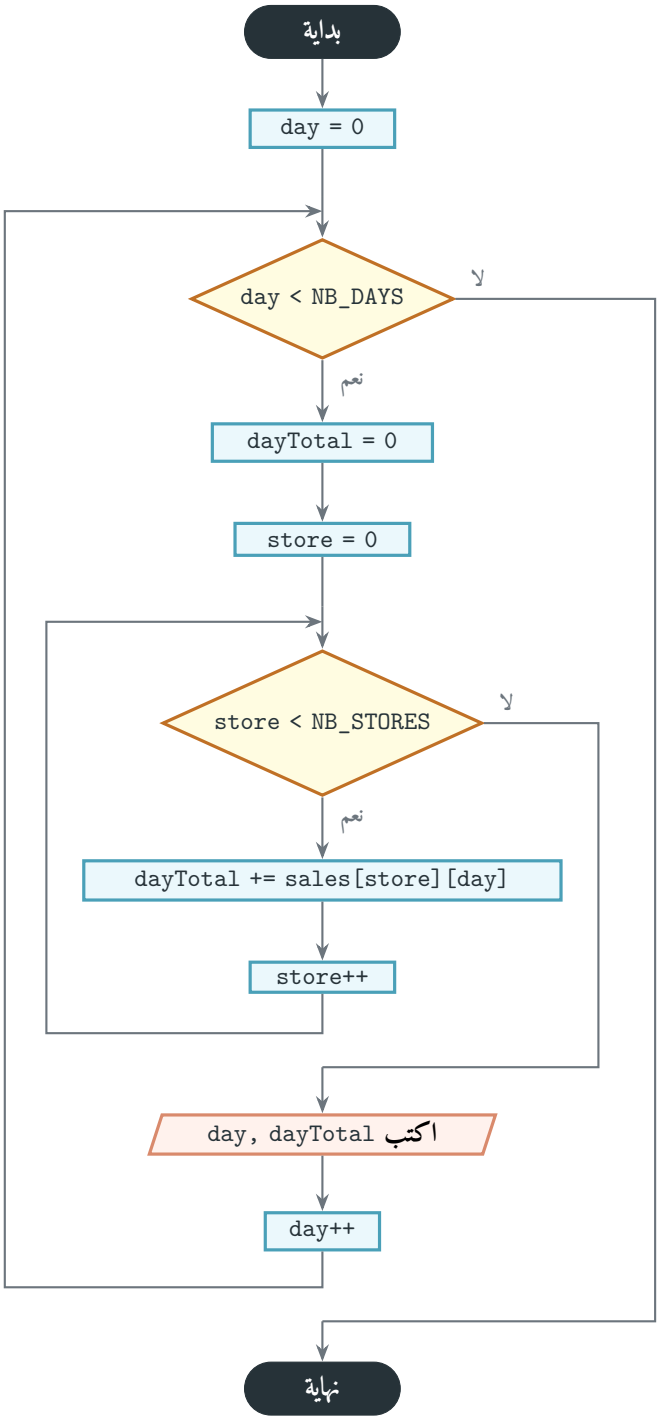
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, dayTotal;
    for (day = 0; day < NB_DAYS; day++) {          // الأعمدة في الخارج
        dayTotal = 0;
        for (store = 0; store < NB_STORES; store++)
            dayTotal += sales[store][day];
        printf("Day %d: %d\n", day + 1, dayTotal);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: 400
Day 2: 365
Day 3: 405
Day 4: 520
```

المخطط الإنسيابي (Organigramme)



تمرين 5.5 أفضل خانة في المصفوفة

علم البيانات | سهل

اكتب برنامجًا يجد أكبر قيمة مبيعات في المصفوفة مع المتجر واليوم الموافقين.

خطوات الحل

1) نحفظ موضع الأكبر `bestStore` و`bestDay` (نبدأ بالخانة `[0][0]`).
2) نقارن كل خانة بـ `sales[bestStore][bestDay]`.

البرنامج بلغة C

```c
#include <stdio.h>

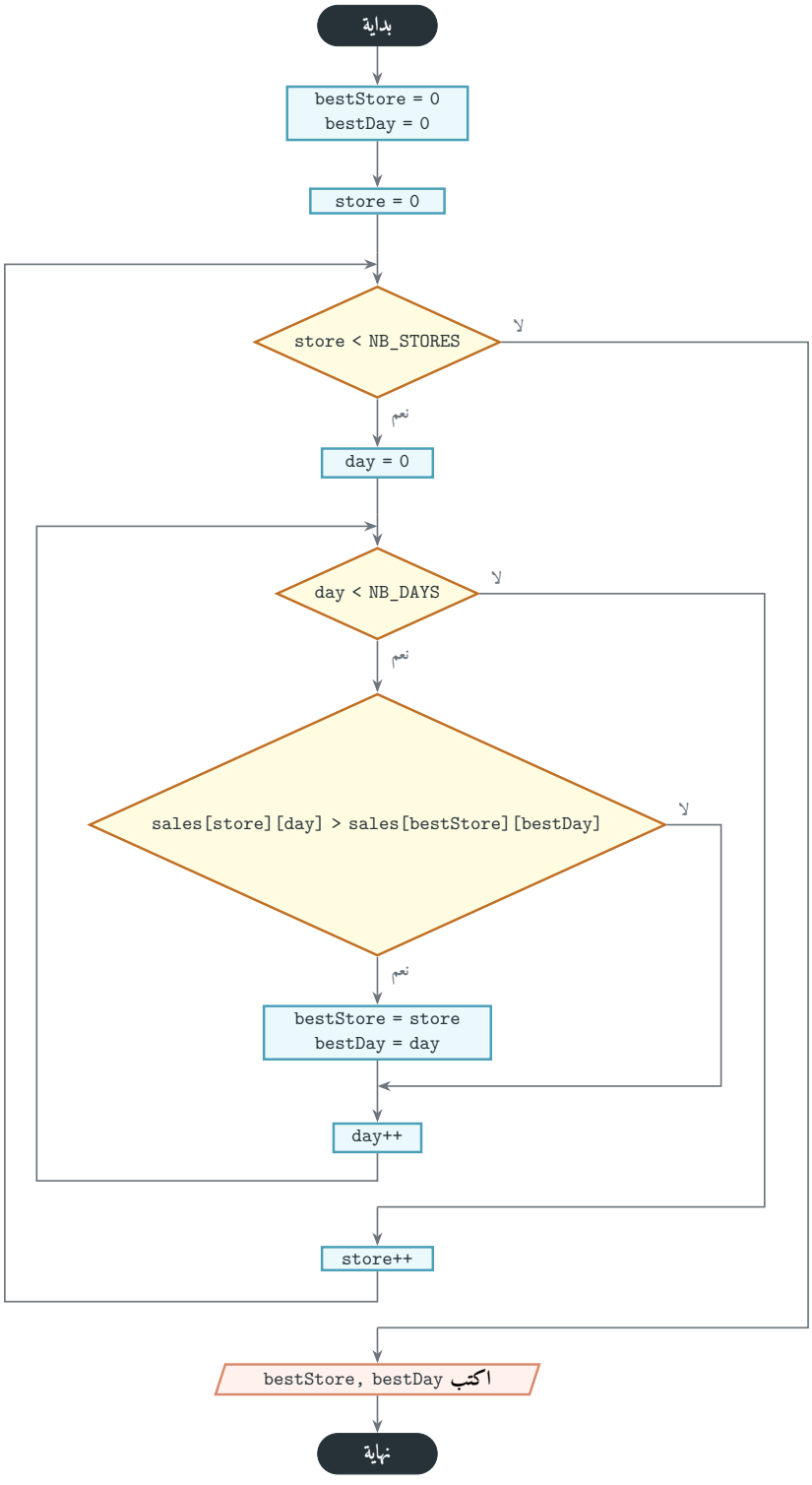
#define NB_STORES 3
#define NB_DAYS 4

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180},
                                     { 80, 60,  75,  90},
                                     {200, 210, 190, 250}};
    int store, day, bestStore = 0, bestDay = 0;
    for (store = 0; store < NB_STORES; store++)
        for (day = 0; day < NB_DAYS; day++)
            if (sales[store][day] > sales[bestStore][bestDay]) {
                bestStore = store;        // نحفظ الإحداثيتين
                bestDay = day;
            }
    printf("Best: store %d, day %d -> %d\n", bestStore, bestDay + 1,
           sales[bestStore][bestDay]);
    return 0;
}
```

نتيجة التنفيذ

```
Best: store 2, day 4 -> 250
```

المخطط الإنسيابي (Organigramme)



تمرين 5.6 الرفوف الفارغة في المستودع

تجارة إلكترونية | سهل

مصفوفة `stock[4][5]` تمثل مخزون المنتجات (الأعمدة) في 4 مستودعات (الأسطر). اكتب برنامجًا يعرض إحداثيات كل خانة مخزونها **0** ويعد عددها.

خطوات الحل

1) حلقتان متداخلتان.
2) إذا كان `stock[depot][product] == 0` نعرض الإحداثيتين ونزيد العداد.

البرنامج بلغة C

```c
#include <stdio.h>

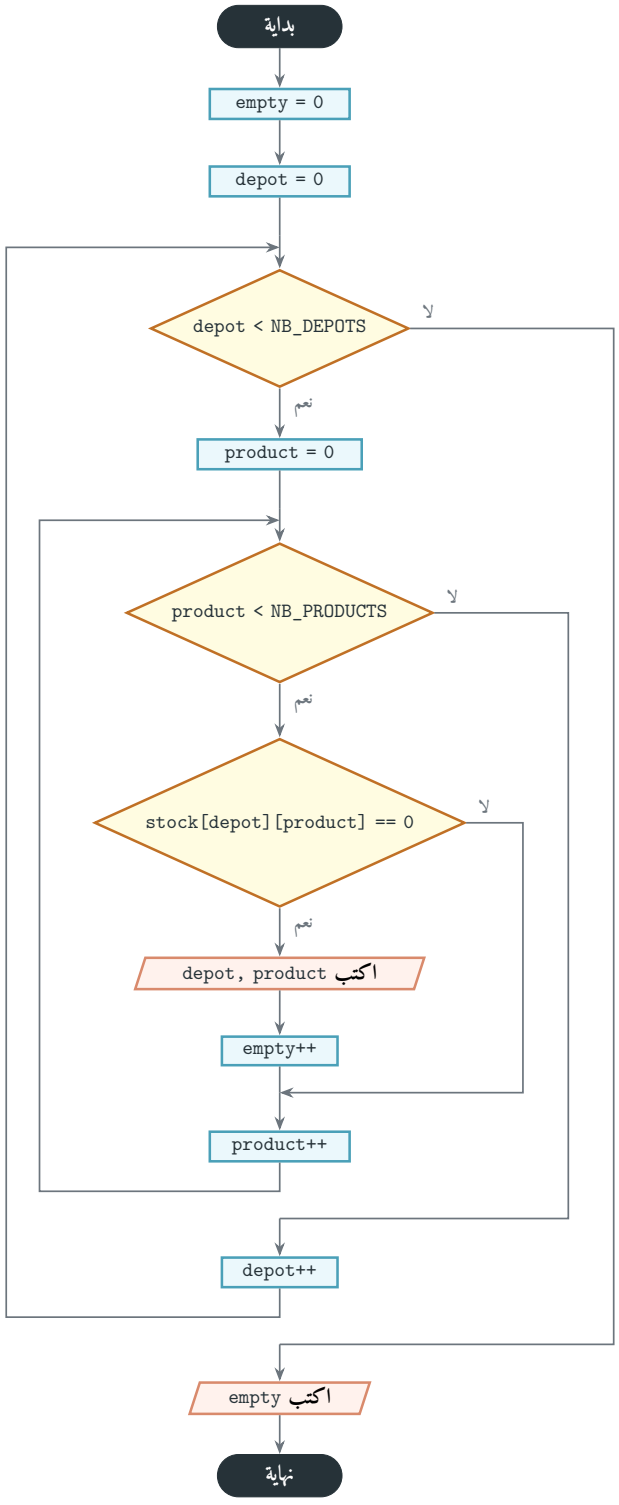
#define NB_DEPOTS 4
#define NB_PRODUCTS 5

int main(void) {
    int stock[NB_DEPOTS][NB_PRODUCTS] = {{5, 0, 12, 3, 8},
                                         {0, 4,  0, 9, 1},
                                         {7, 7,  2, 0, 6},
                                         {3, 0,  5, 5, 0}};
    int depot, product, empty = 0;
    for (depot = 0; depot < NB_DEPOTS; depot++)
        for (product = 0; product < NB_PRODUCTS; product++)
            if (stock[depot][product] == 0) {
                printf("Depot %d, product %d is empty\n", depot, product);
                empty++;
            }
    printf("%d empty slots\n", empty);
    return 0;
}
```

نتيجة التنفيذ

```
Depot 0, product 1 is empty
Depot 1, product 0 is empty
Depot 1, product 2 is empty
Depot 2, product 3 is empty
Depot 3, product 1 is empty
Depot 3, product 4 is empty
6 empty slots
```

المخطط الإنسيابي (Organigramme)



تمرين 5.7 تعديل كل الأسعار في المصفوفة

تجارة إلكترونية ★ سهل

مصفوفة `prices[3][3]` تحتوي أسعار 3 منتجات (أسطر) بثلاثة أحجام (أعمدة). اكتب برنامجًا يطبق تخفيضًا `percent`% على **كل** الأسعار في مكانها ثم يعرضها.

خطوات الحل

1) نقرأ النسبة.

2) `prices[product][size] *= (1 - percent / 100)`

لكل خانة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    double prices[3][3] = {{1200, 1500, 1800},     // صغير، متوسط، كبير
                           { 450,  600,  750},
                           {3000, 3600, 4200}};
    double percent;
    int product, size;
    scanf("%lf", &percent);
    for (product = 0; product < 3; product++) {
        for (size = 0; size < 3; size++) {
            prices[product][size] *= 1 - percent / 100;    // التعديل في المكان
            printf("%8.2f", prices[product][size]);
        }
        printf("\n");
    }
    return 0;
}
```

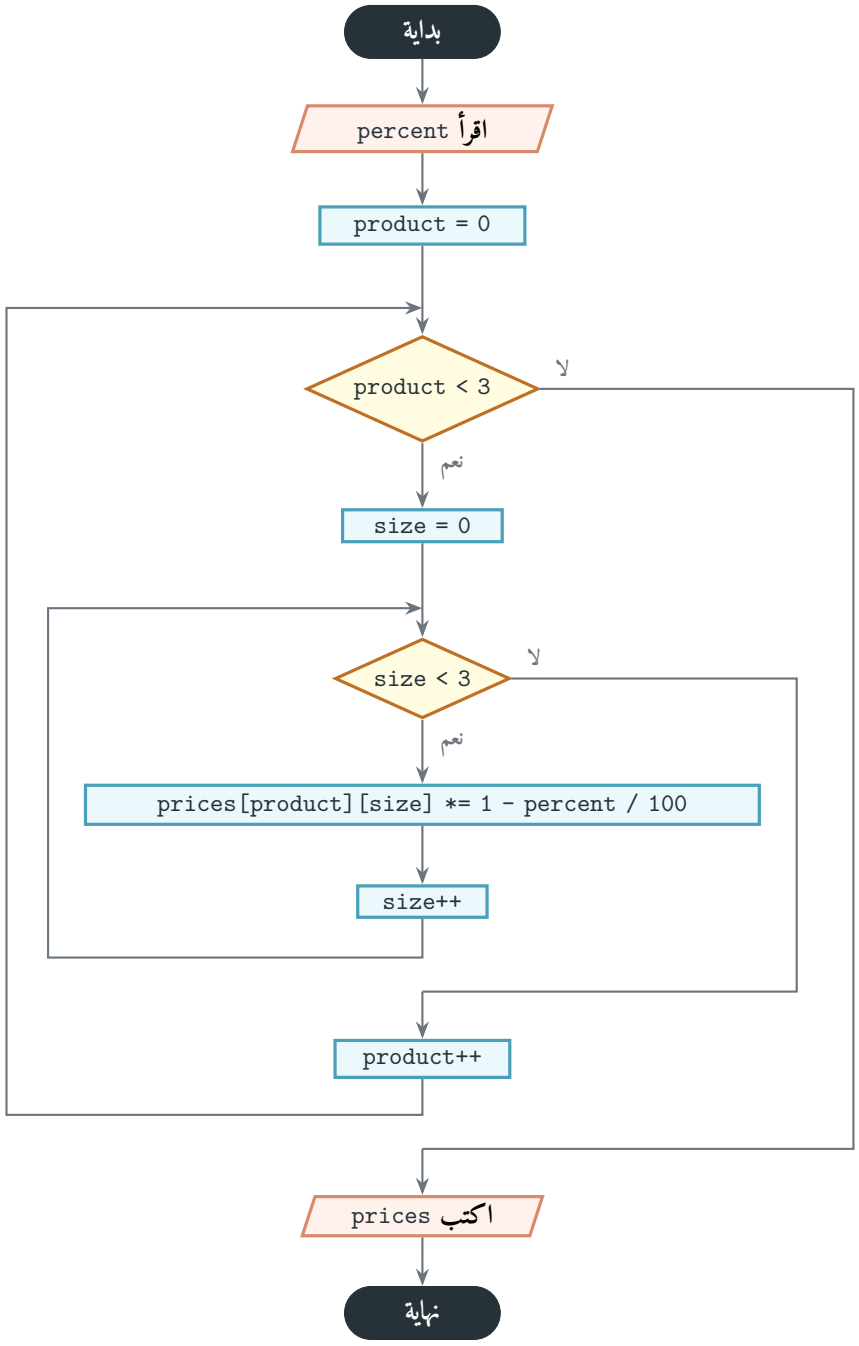
المدخلات

```
15
```

نتيجة التنفيذ

```
 1020.00 1275.00 1530.00
  382.50  510.00  637.50
2550.00 3060.00 3570.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.8 قراءة مصفوفة تقييمات

علم البيانات ⭐ سهل

اكتب برنامجًا يقرأ عدد الزبائن `nbUsers` وعدد المنتجات `nbItems` (لا يتجاوزان 10) ثم مصفوفة التقييمات، ويعرضها مع متوسط كل زبون.

خطوات الحل

1) نصرح بمصفوفة بالسعة القصوى `ratings[10][10]` ونستعمل جزءًا منها.
2) القراءة بحلقتين متداخلتين.
3) متوسط كل سطر عند العرض.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 10

int main(void) {
    int ratings[MAX][MAX];
    int nbUsers, nbItems, user, item, rowSum;
    scanf("%d %d", &nbUsers, &nbItems);
    for (user = 0; user < nbUsers; user++)
        for (item = 0; item < nbItems; item++)
            scanf("%d", &ratings[user][item]);     // عنوان الخانة
    for (user = 0; user < nbUsers; user++) {
        rowSum = 0;
        printf("User %d:", user);
        for (item = 0; item < nbItems; item++) {
            printf(" %d", ratings[user][item]);
            rowSum += ratings[user][item];
        }
        printf("  avg %.2f\n", (double)rowSum / nbItems);
    }
    return 0;
}
```

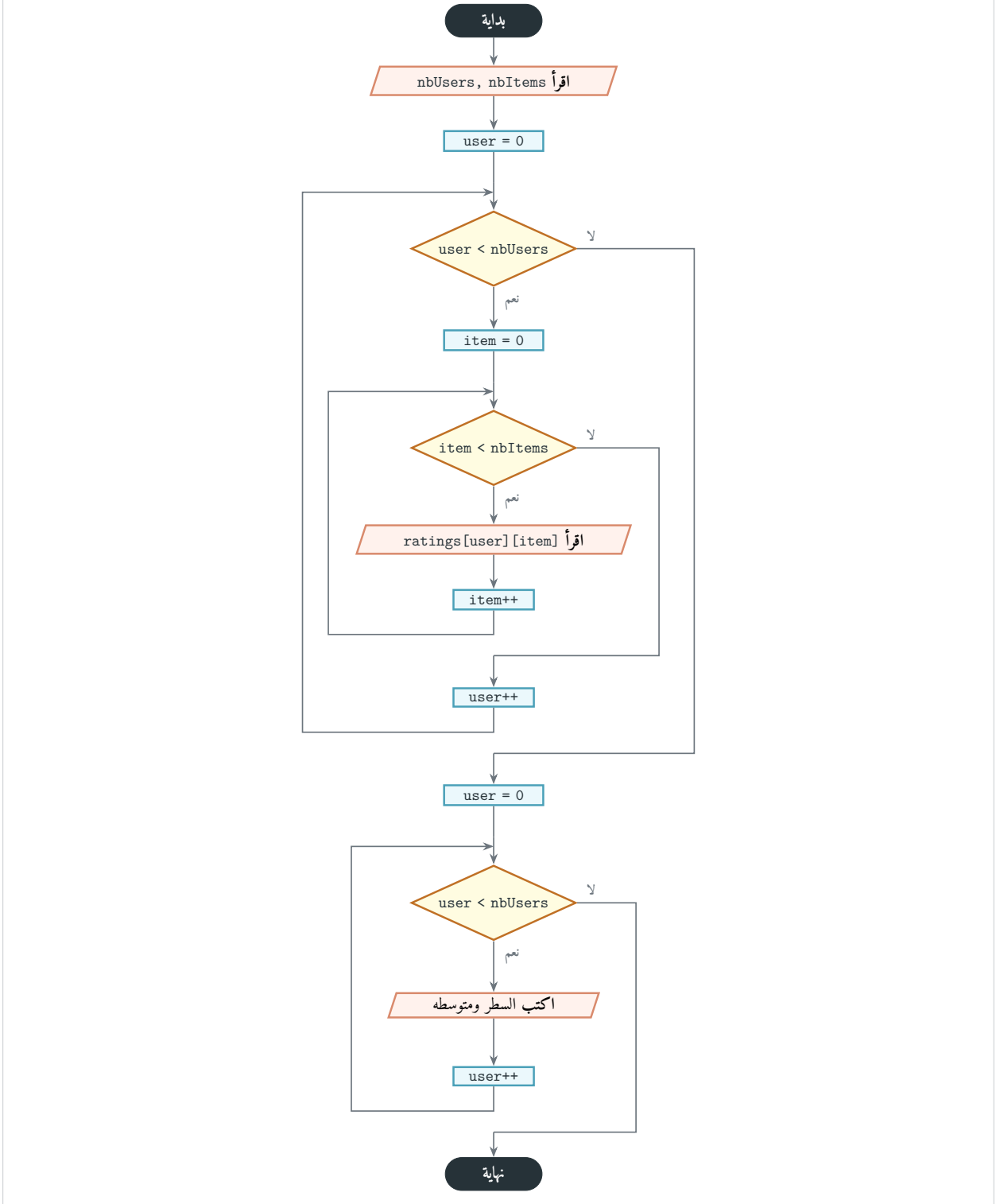
المدخلات

```
3 4
5 3 0 4
4 0 4 5
1 2 5 0
```

نتيجة التنفيذ

```
User 0: 5 3 0 4  avg 3.00
User 1: 4 0 4 5  avg 3.25
User 2: 1 2 5 0  avg 2.00
```

المخطط الإنسيابي (Organigramme)



تمرين 5.9 القطر الرئيسي لمصفوفة التحويلات

تجارة إلكترونية

سهل

مصفوفة مربعة `transfers[4][4]` تعطي عدد الطرود المرسلة من مستودع (سطر) إلى مستودع (عمود). القطر الرئيسي يمثل الطرود التي بقيت في نفس المستودع. اكتب برنامجًا يعرض القطر ومجموعه (الأثر) ومجموع الطرود المنقولة فعلًا.

خطوات الحل

1) عناصر القطر هي `transfers[depot][depot]`: حلقة واحدة تكفي.

2) المنقولة = المجموع الكلي − الأثر.

البرنامج بلغة C

```c
#include <stdio.h>

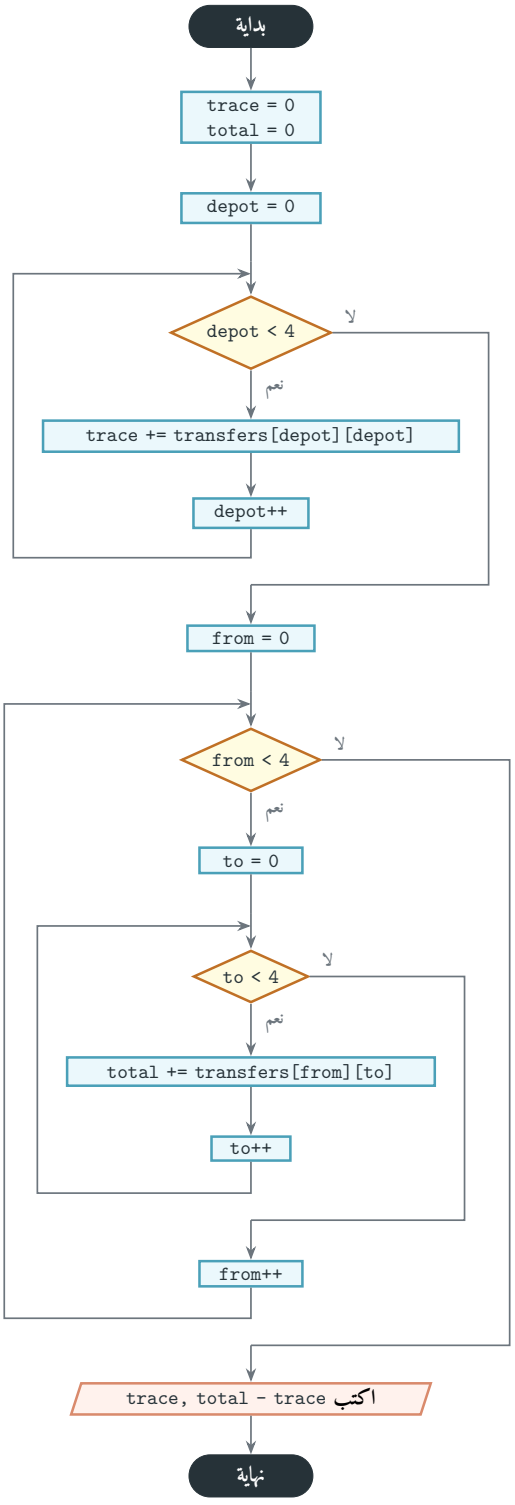
#define NB 4

int main(void) {
    int transfers[NB][NB] = {{30,  5,  2, 0},
                             { 4, 25,  6, 1},
                             { 0,  7, 40, 3},
                             { 2,  1,  5, 18}};
    int depot, from, to, trace = 0, total = 0;
    printf("Diagonal:");
    for (depot = 0; depot < NB; depot++) {
        printf(" %d", transfers[depot][depot]);    // السطر = العمود
        trace += transfers[depot][depot];
    }
    for (from = 0; from < NB; from++)
        for (to = 0; to < NB; to++)
            total += transfers[from][to];
    printf("\nKept = %d, moved = %d\n", trace, total - trace);
    return 0;
}
```

نتيجة التنفيذ

```
Diagonal: 30 25 40 18
Kept = 113, moved = 36
```

المخطط الإنسيابي (Organigramme)



تمرين 5.10 البحث عن منتج في الرفوف

تجارة إلكترونية

سهل

مصفوفة `shelves[3][4]` تحتوي رقم المنتج الموضوع في كل رف. اكتب برنامجًا يبحث عن منتج ويعرض رقم الممر والرف أو "غير موجود"، ويتوقف عند أول تطابق.

خطوات الحل

1) `found = 0`؛ الحلقتان تستمران ما دام `!found`.
2) عند التطابق نحفظ الموضع.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int shelves[3][4] = {{101, 204, 330, 415},
                         {512, 618, 777, 801},
                         {905, 999, 120, 343}};
    int target, aisle, slot, found = 0, foundAisle = 0, foundSlot = 0;
    scanf("%d", &target);
    for (aisle = 0; aisle < 3 && !found; aisle++)          // نخرج من الحلقتين
        for (slot = 0; slot < 4 && !found; slot++)
            if (shelves[aisle][slot] == target) {
                found = 1;
                foundAisle = aisle;
                foundSlot = slot;
            }
    if (found)
        printf("Product %d: aisle %d, slot %d\n", target, foundAisle, foundSlot);
    else
        printf("Product %d not found\n", target);
    return 0;
}
```

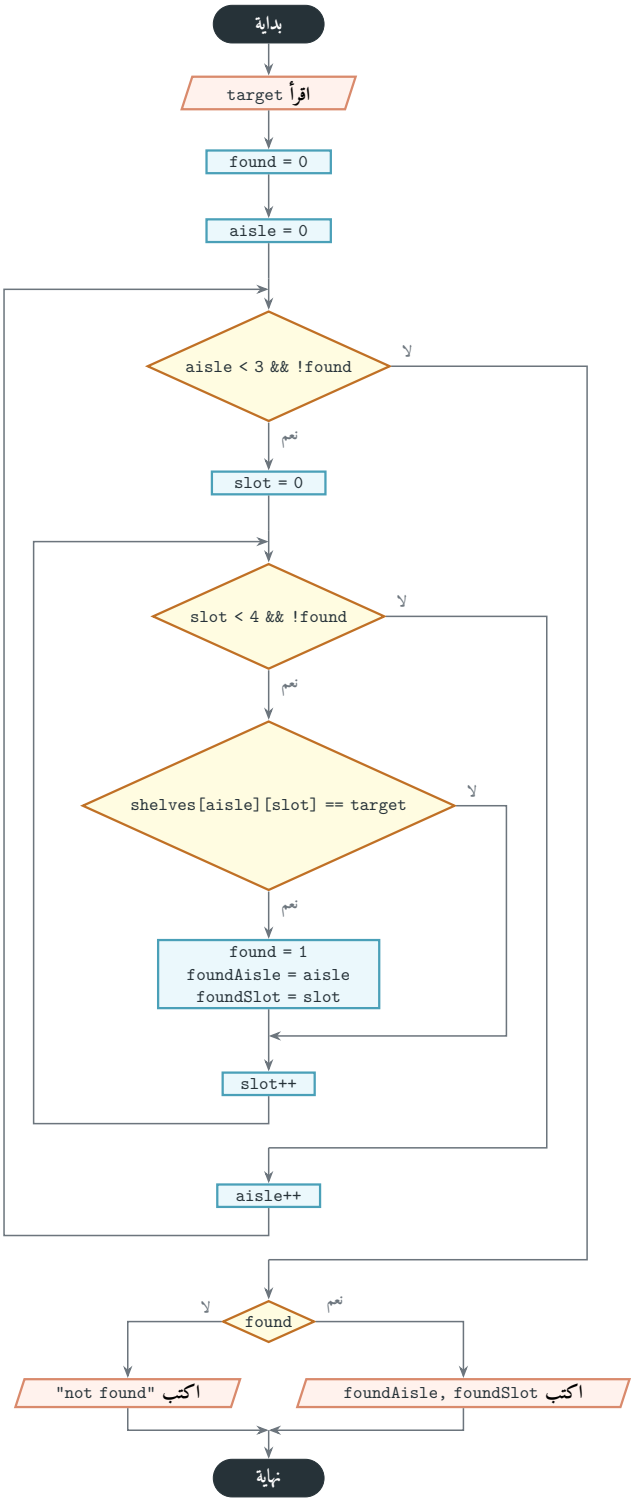
المدخلات

```
512
```

نتيجة التنفيذ

```
Product 512: aisle 1, slot 0
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 5.11 منقول المصفوفة

علم البيانات ★★ متوسط

مصفوفة `purchases[3][5]` (3 زبائن × 5 منتجات). يحتاج المحلل النظرة المعاكسة: منتجات × زبائن. اكتب برنامجًا يحسب المنقول `transposed[5][3]` ويعرضه.

خطوات الحل

1) `transposed[item][user] = purchases[user][item]`

.

2) أبعاد المنقول معكوسة.

البرنامج بلغة C

```c
#include <stdio.h>

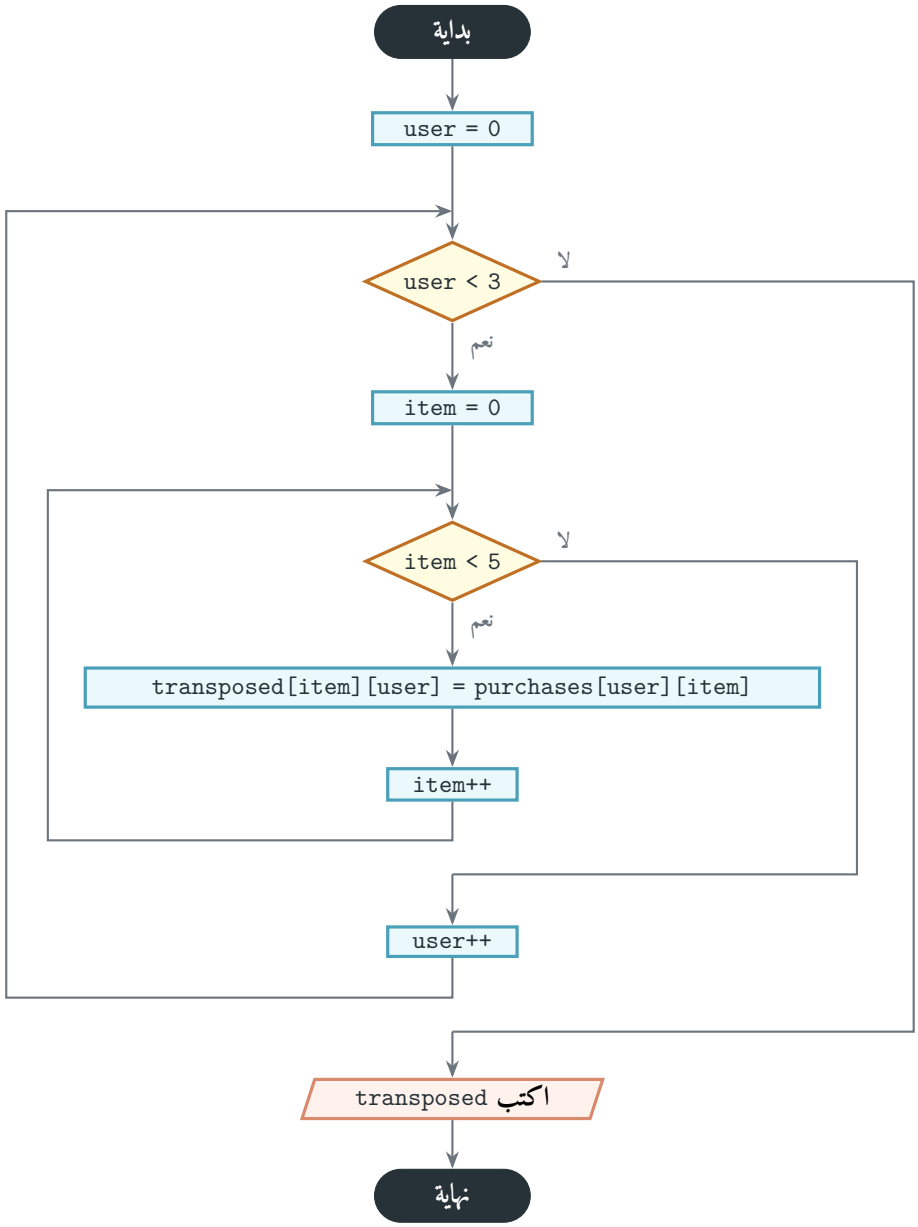
#define NB_USERS 3
#define NB_ITEMS 5

int main(void) {
    int purchases[NB_USERS][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                         {0, 3, 0, 1, 0},
                                         {2, 1, 0, 0, 4}};
    int transposed[NB_ITEMS][NB_USERS];
    int user, item;
    for (user = 0; user < NB_USERS; user++)
        for (item = 0; item < NB_ITEMS; item++)
            transposed[item][user] = purchases[user][item];   // تبديل الفهرسين
    for (item = 0; item < NB_ITEMS; item++) {
        printf("Item %d:", item);
        for (user = 0; user < NB_USERS; user++)
            printf(" %d", transposed[item][user]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 1 0 2
Item 1: 0 3 1
Item 2: 2 0 0
Item 3: 0 1 0
Item 4: 1 0 4
```

المخطط الإنسيابي (Organigramme)



تمرين 5.12 جمع مبيعات شهرين

تجارة إلكترونية ★★ متوسط

لدينا مصفوفتا مبيعات (منتجات × متاجر) لشهري جانفي وفيفري. اكتب برنامجًا يحسب **مجموعهما** خانة بخانة، ثم **نسبة النمو** لكل خانة من الشهر الأول إلى الثاني.

خطوات الحل

1) `total[row][col] = jan[row][col] + feb[row][col]`

.

2) `growth = (feb - jan) / jan * 100` إذا كان `jan > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

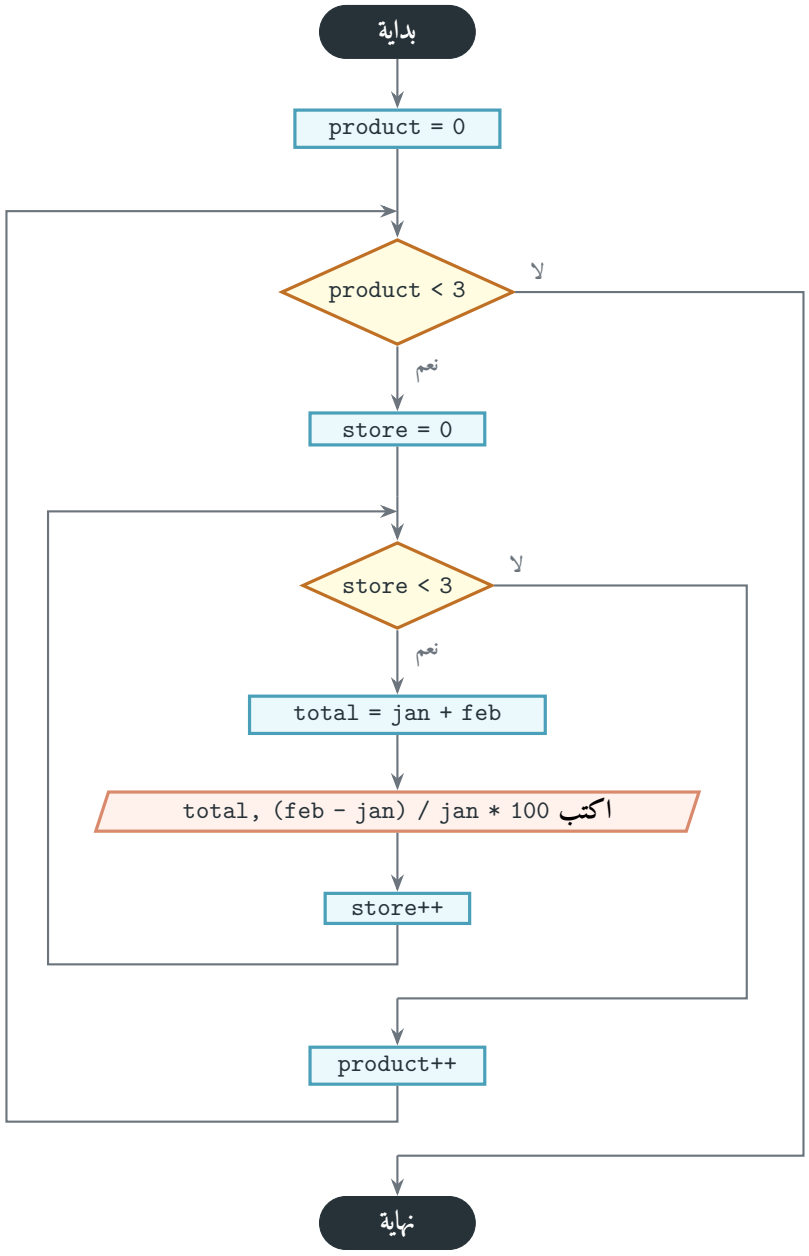
#define NB 3

int main(void) {
    int jan[NB][NB] = {{120, 80, 60}, {40, 55, 70}, {300, 210, 150}};
    int feb[NB][NB] = {{132, 70, 66}, {52, 55, 63}, {330, 240, 150}};
    int total[NB][NB], product, store;
    for (product = 0; product < NB; product++) {
        for (store = 0; store < NB; store++) {
            total[product][store] = jan[product][store] + feb[product][store];
            printf("%4d (%+5.1f%%)", total[product][store],
                   (feb[product][store] - jan[product][store]) * 100.0 / jan[product][store]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 252 (+10.0%) 150 (-12.5%) 126 (+10.0%)
  92 (+30.0%) 110 ( +0.0%) 133 (-10.0%)
 630 (+10.0%) 450 (+14.3%) 300 ( +0.0%)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.13 جداء المصفوفات: كلفة الطلبيات عند الموردين

متوسط ★★ تجارة إلكترونية

`quantities[2][3]`: كمية كل منتج (3 منتجات) في كل طلبية (2 طلبيات). `unitPrice[3][2]`: سعر كل منتج عند كل مورد (2 موردين). اكتب برنامجًا يحسب **جداء المصفوفتين** `cost[2][2]`: كلفة كل طلبية عند كل مورد، ويحدد المورد الأرخص لكل طلبية.

خطوات الحل

1) `cost[order][supplier] = Σ quantities[order][prod] * unitPrice[prod][supplier]`

.

2) ثلاث حلقات متداخلة: السطر، العمود، ثم المجموع.

3) الأرخص: مقارنة عمودي كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

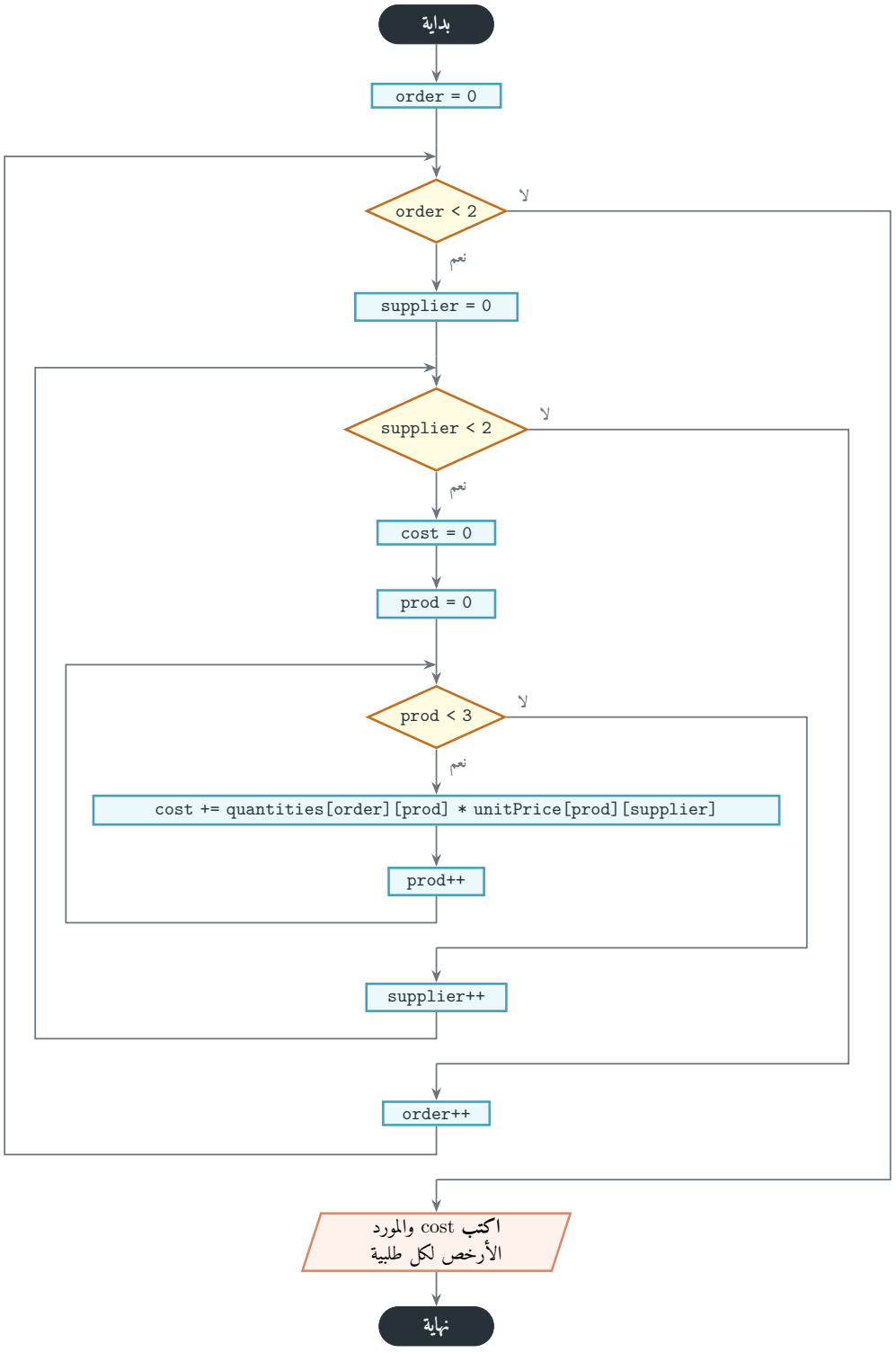
#define NB_ORDERS 2
#define NB_PRODUCTS 3
#define NB_SUPPLIERS 2

int main(void) {
    int quantities[NB_ORDERS][NB_PRODUCTS] = {{10, 5, 2}, {0, 20, 8}};
    int unitPrice[NB_PRODUCTS][NB_SUPPLIERS] = {{100, 95}, {40, 48}, {250, 230}};
    int cost[NB_ORDERS][NB_SUPPLIERS];
    int order, supplier, prod;
    for (order = 0; order < NB_ORDERS; order++) {
        for (supplier = 0; supplier < NB_SUPPLIERS; supplier++) {
            cost[order][supplier] = 0;
            for (prod = 0; prod < NB_PRODUCTS; prod++)   // سطر × عمود
                cost[order][supplier] += quantities[order][prod] * unitPrice[prod][supplier];
        }
        printf("Order %d: supplier A = %d, supplier B = %d -> choose %c\n", order,
               cost[order][0], cost[order][1], cost[order][0] <= cost[order][1] ? 'A' : 'B');
    }
    return 0;
}
```

نتيجة التنفيذ

```
Order 0: supplier A = 1700, supplier B = 1650 -> choose B
Order 1: supplier A = 2800, supplier B = 2800 -> choose A
```

المخطط الإنسيابي (Organigramme)



تمرين 5.14 فاتورة كل زبون بجداء مصفوفة في شعاع

تجارة إلكترونية ★★ متوسط

مصفوفة `cart[4][5]`: كمية كل منتج في سلة كل زبون. جدول `price[5]`: أسعار المنتجات. اكتب برنامجًا يحسب **جداء المصفوفة في الشعاع**: فاتورة كل زبون، والزبون الأكبر إنفاقًا.

خطوات الحل

1) `bill[customer] = Σ cart[customer][item] * price[item]`

.

2) نحتفظ بأكبر فاتورة.

البرنامج بلغة C

```c
#include <stdio.h>

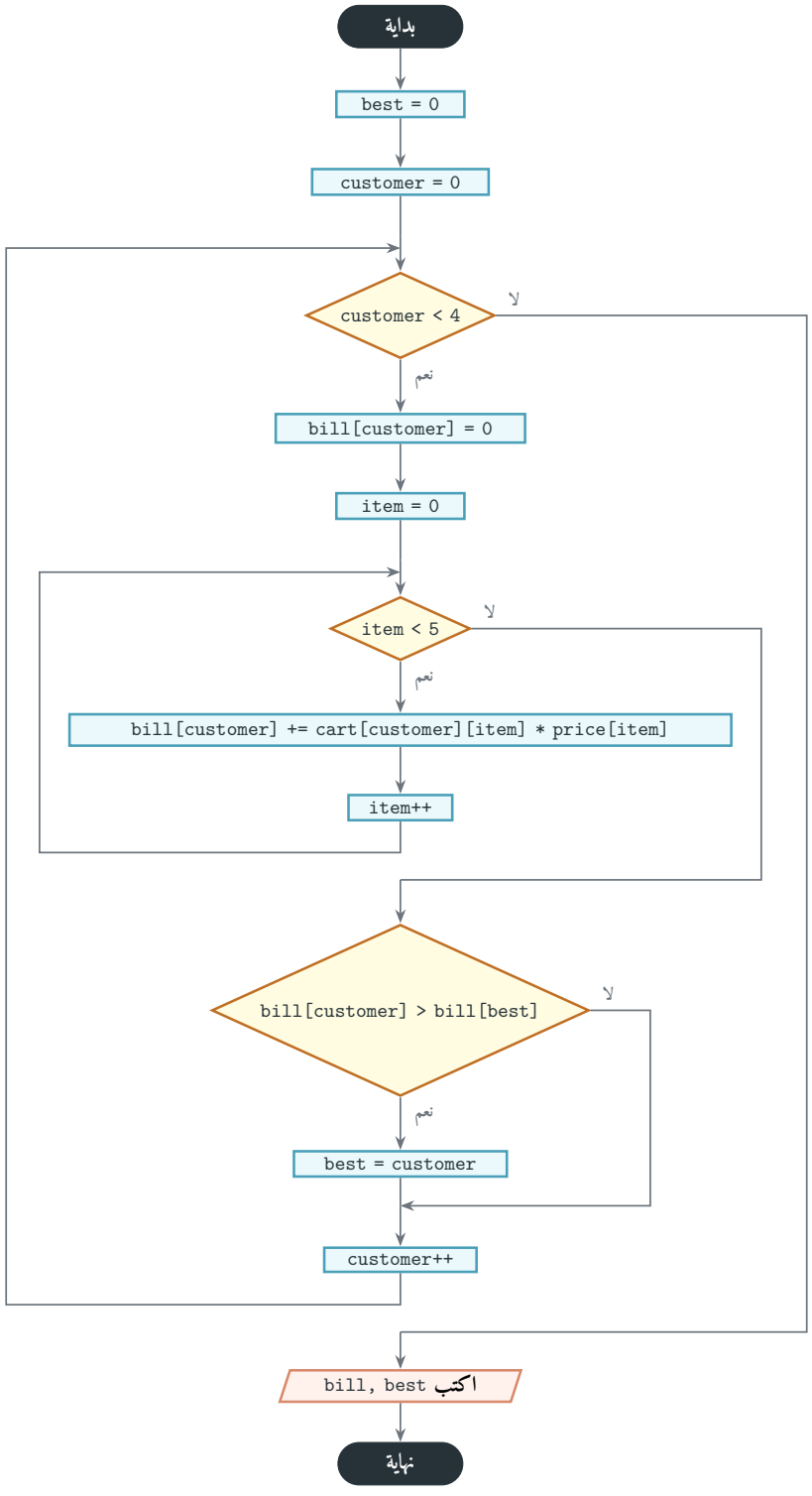
#define NB_CUST 4
#define NB_ITEMS 5

int main(void) {
    int cart[NB_CUST][NB_ITEMS] = {{1, 0, 2, 0, 1},
                                   {0, 3, 0, 1, 0},
                                   {2, 1, 0, 0, 4},
                                   {0, 0, 1, 2, 2}};
    double price[NB_ITEMS] = {1500, 350, 2200, 999, 120};
    double bill[NB_CUST];
    int customer, item, best = 0;
    for (customer = 0; customer < NB_CUST; customer++) {
        bill[customer] = 0;
        for (item = 0; item < NB_ITEMS; item++)
            bill[customer] += cart[customer][item] * price[item];  // كمية × سعر
        printf("Customer %d: %.2f DA\n", customer, bill[customer]);
        if (bill[customer] > bill[best])
            best = customer;
    }
    printf("Top spender: customer %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 0: 6020.00 DA
Customer 1: 2049.00 DA
Customer 2: 3830.00 DA
Customer 3: 4438.00 DA
Top spender: customer 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.15 أفضل متجر في كل يوم

علم البيانات ★★ متوسط

اكتب برنامجًا يحدد لكل يوم (عمود) **المتجر الأفضل** (السطر ذو الأكبر قيمة)، ثم يعد عدد الأيام التي فاز بها كل متجر.

خطوات الحل

1) لكل عمود: بحث عن فهرس السطر الأكبر `winner`.
2) `wins[winner]++`.

البرنامج بلغة C

```c
#include <stdio.h>

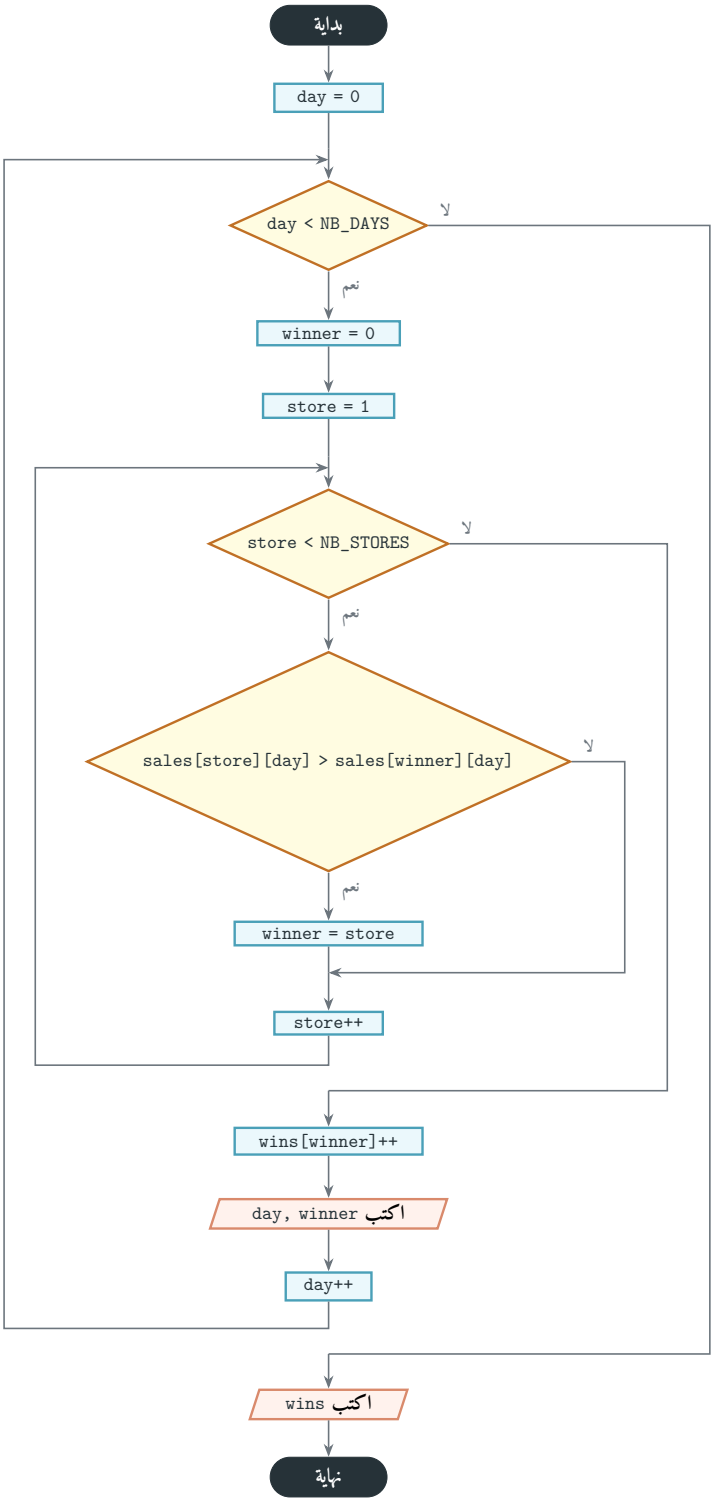
#define NB_STORES 3
#define NB_DAYS 6

int main(void) {
    int sales[NB_STORES][NB_DAYS] = {{120, 95, 140, 180, 90, 210},
                                     {130, 60, 175,  90, 95, 100},
                                     {100, 99, 150, 170, 80, 240}};
    int wins[NB_STORES] = {0};
    int day, store, winner;
    for (day = 0; day < NB_DAYS; day++) {
        winner = 0;                          // نبحث في العمود day
        for (store = 1; store < NB_STORES; store++)
            if (sales[store][day] > sales[winner][day])
                winner = store;
        wins[winner]++;
        printf("Day %d: store %d\n", day + 1, winner);
    }
    for (store = 0; store < NB_STORES; store++)
        printf("Store %d won %d day(s)\n", store, wins[store]);
    return 0;
}
```

نتيجة التنفيذ

```
Day 1: store 1
Day 2: store 2
Day 3: store 1
Day 4: store 0
Day 5: store 1
Day 6: store 2
Store 0 won 1 day(s)
Store 1 won 3 day(s)
Store 2 won 2 day(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.16 متوسط تقييم كل منتج دون الخانات الفارغة

علم البيانات ★★ متوسط

في مصفوفة التقييمات (زبائن × منتجات) القيمة **0** تعني "**لم يقيّم**". اكتب برنامجًا يحسب متوسط كل منتج على التقييمات الموجودة فقط، وعدد مقيّميه، ثم المنتج الأعلى تقييمًا.

خطوات الحل

1) لكل عمود: نجمع الخانات غير الصفرية ونعدها.
2) المتوسط `sum / count` إذا كان `count > 0`.

البرنامج بلغة C

```c
#include <stdio.h>

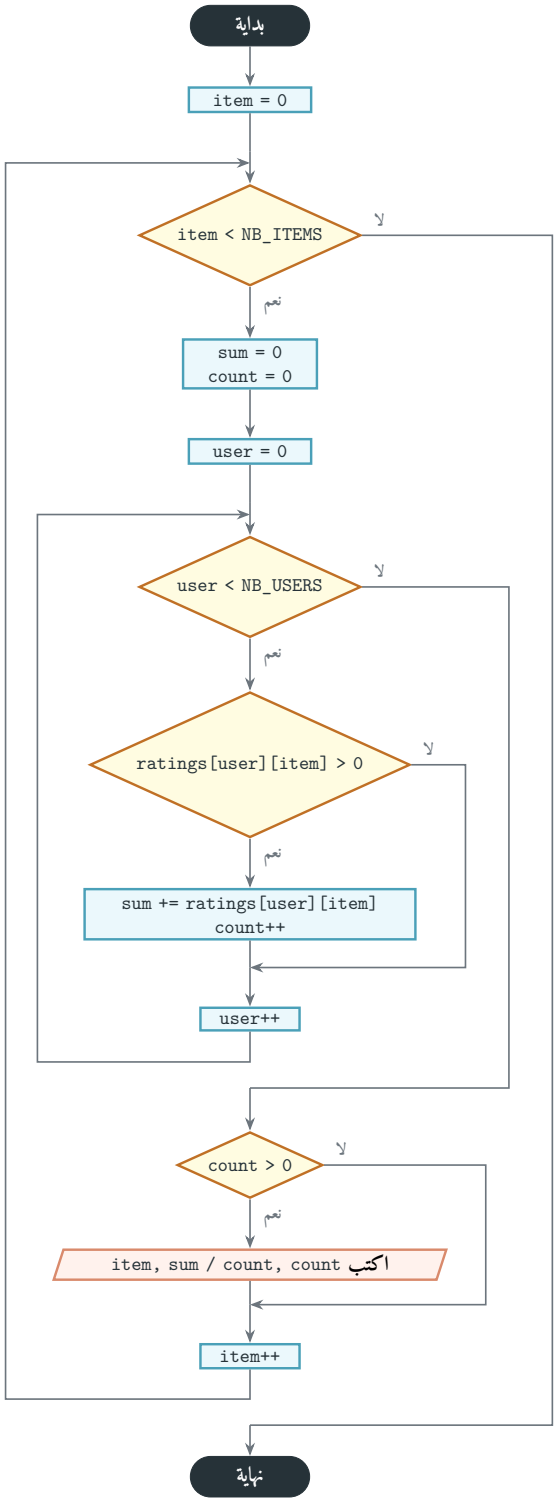
#define NB_USERS 5
#define NB_ITEMS 4

int main(void) {
    int ratings[NB_USERS][NB_ITEMS] = {{5, 3, 0, 4},
                                       {4, 0, 0, 5},
                                       {1, 2, 5, 0},
                                       {0, 3, 4, 4},
                                       {5, 0, 3, 0}};
    int user, item, sum, count, best = -1;
    double avg, bestAvg = 0;
    for (item = 0; item < NB_ITEMS; item++) {
        sum = 0;
        count = 0;
        for (user = 0; user < NB_USERS; user++)
            if (ratings[user][item] > 0) {    // نتجاهل الخانات الفارغة
                sum += ratings[user][item];
                count++;
            }
        if (count > 0) {
            avg = (double)sum / count;
            printf("Item %d: %.2f (%d ratings)\n", item, avg, count);
            if (avg > bestAvg) { bestAvg = avg; best = item; }
        }
    }
    printf("Best rated: item %d\n", best);
    return 0;
}
```

نتيجة التنفيذ

```
Item 0: 3.75 (4 ratings)
Item 1: 2.67 (3 ratings)
Item 2: 4.00 (3 ratings)
Item 3: 4.33 (3 ratings)
Best rated: item 3
```

المخطط الإنسيابي (Organigramme)



تمرين 5.17 تطبيع أعمدة جدول الخصائص

علم البيانات ★★ متوسط

مصفوفة `features[6][3]`: لكل منتج (سطر) ثلاث خصائص (السعر، الوزن، المبيعات) بوحدات مختلفة. اكتب برنامجًا يطبع **كل عمود** على حدة إلى `[0, 1]` (Min-Max)، لأن نماذج التعلم الآلي حساسة لاختلاف المقاييس.

خطوات الحل

1) لكل عمود: نجد `colMin` و`colMax`.
2) ثم نحول كل خانة في العمود.

البرنامج بلغة C

```c
#include <stdio.h>

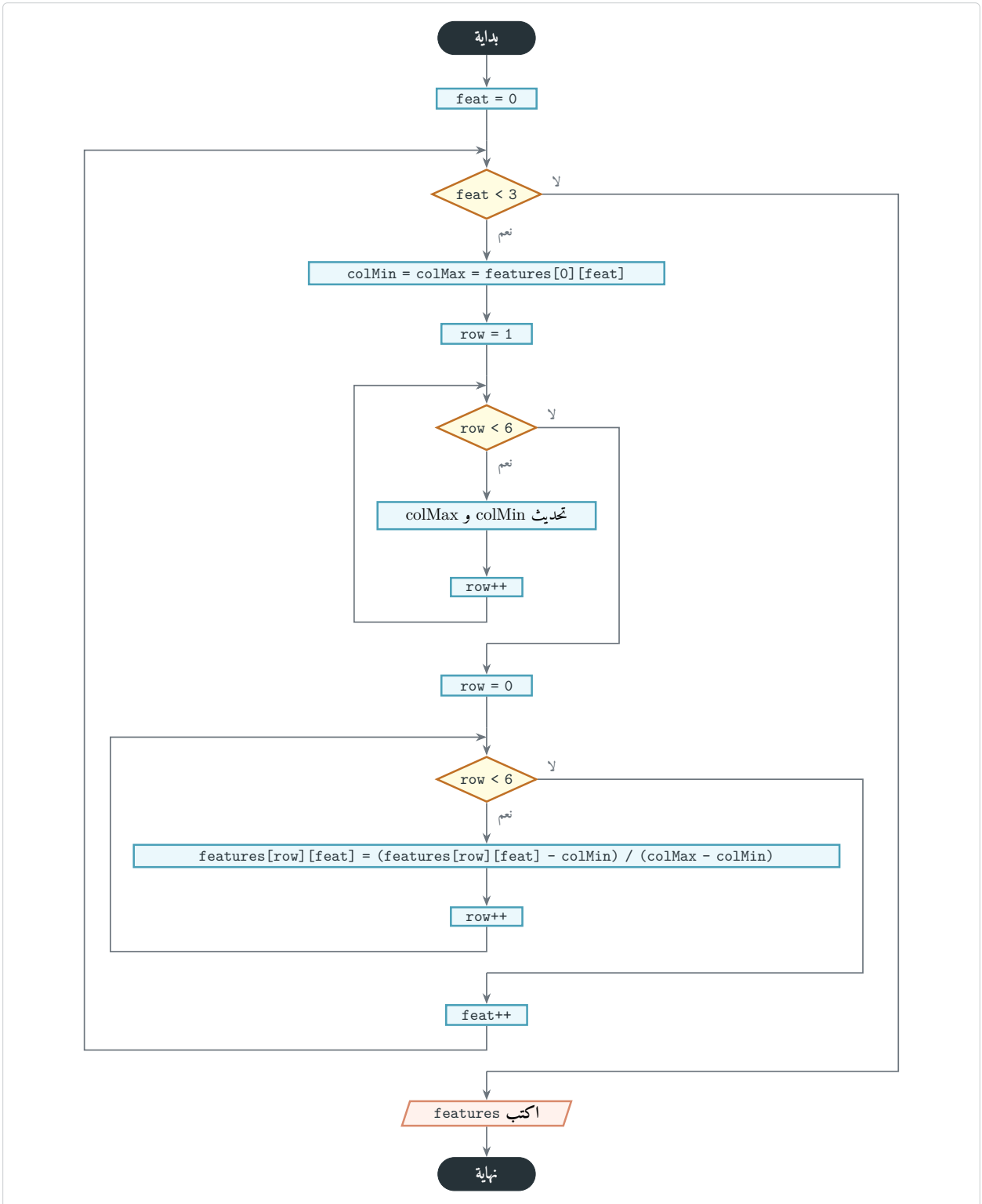
#define NB_ROWS 6
#define NB_FEAT 3

int main(void) {
    double features[NB_ROWS][NB_FEAT] = {{1200, 0.5, 340},
                                         {4500, 2.0,  80},
                                         { 300, 0.2, 900},
                                         {8000, 3.5,  45},
                                         {2500, 1.1, 210},
                                         { 950, 0.8, 500}};
    double colMin, colMax;
    int feat, row;
    for (feat = 0; feat < NB_FEAT; feat++) {          // عمود بعد عمود
        colMin = colMax = features[0][feat];
        for (row = 1; row < NB_ROWS; row++) {
            if (features[row][feat] < colMin) colMin = features[row][feat];
            if (features[row][feat] > colMax) colMax = features[row][feat];
        }
        for (row = 0; row < NB_ROWS; row++)
            features[row][feat] = (features[row][feat] - colMin) / (colMax - colMin);
    }
    printf(" price weight sales\n");
    for (row = 0; row < NB_ROWS; row++)
        printf(" %.2f  %.2f   %.2f\n", features[row][0], features[row][1], features[row][2]);
    return 0;
}
```

نتيجة التنفيذ

```
price weight sales
0.12  0.09   0.35
0.55  0.55   0.04
0.00  0.00   1.00
1.00  1.00   0.00
0.29  0.27   0.19
0.08  0.18   0.53
```

المخطط الإنسيابي (Organigramme)



تمرين 5.18 مصفوفة الإرتباط

علم البيانات ★★ متوسط

لثلاث خصائص لكل زبون (الزيارات، الدقائق، المبلغ) اكتب برنامجًا يحسب **مصفوفة الارتباط** `corr[3][3]` حيث `corr[a][b]` معامل بيرسون بين الخاصيتين، باستعمال الجذر بطريقة نيوتن.

خطوات الحل

1) نحسب متوسط كل عمود.

2) `cov[first][second] = Σ (x - mean)(y - mean)`

.

3) `corr = cov[first][second] / cov[first][first]) * cov[second][second])`

.

البرنامج بلغة C

```c
#include <stdio.h>

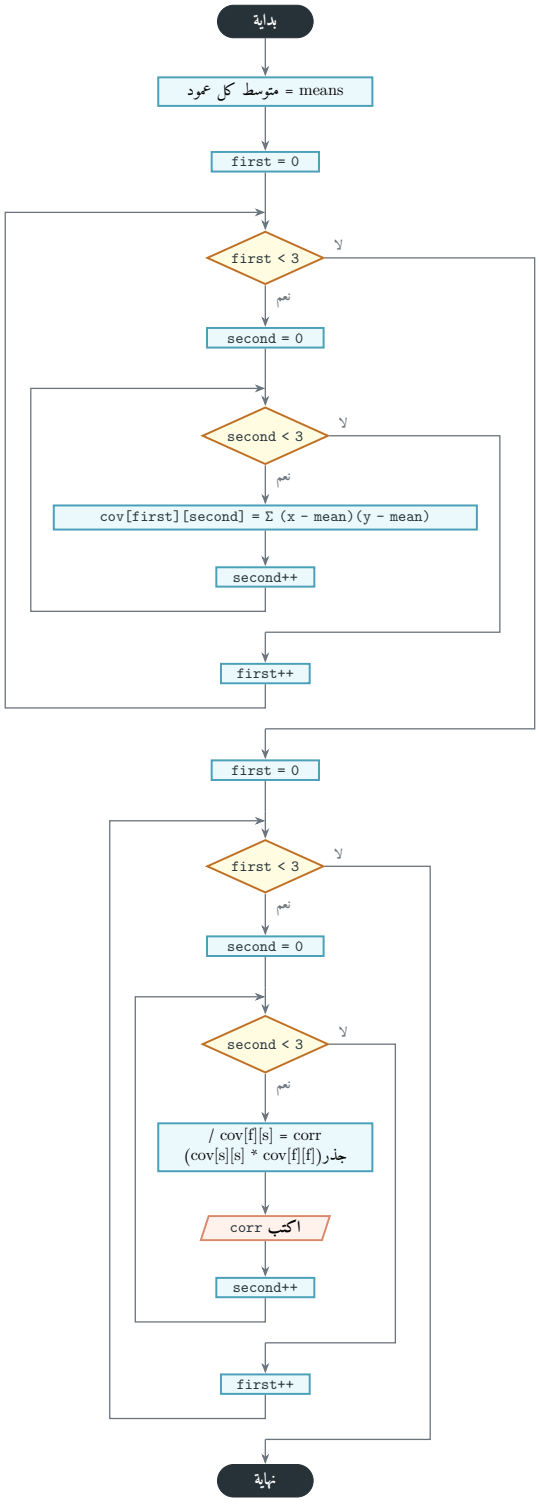
#define NB 6
#define NB_FEAT 3

int main(void) {
    double data[NB][NB_FEAT] = {{3, 12, 1500}, {8, 40, 5200}, {5, 22, 2400},
                                {1,  4,  300}, {9, 50, 6100}, {4, 25, 1900}};
    double mean[NB_FEAT] = {0}, cov[NB_FEAT][NB_FEAT], product, root;
    int row, first, second, step;
    for (row = 0; row < NB; row++)
        for (first = 0; first < NB_FEAT; first++)
            mean[first] += data[row][first] / NB;
    for (first = 0; first < NB_FEAT; first++)
        for (second = 0; second < NB_FEAT; second++) {
            cov[first][second] = 0;           // التغاير (دون القسمة على n)
            for (row = 0; row < NB; row++)
                cov[first][second] += (data[row][first] - mean[first])
                                    * (data[row][second] - mean[second]);
        }
    for (first = 0; first < NB_FEAT; first++) {
        for (second = 0; second < NB_FEAT; second++) {
            product = cov[first][first] * cov[second][second];
            root = product;                   // الجذر بطريقة نيوتن
            for (step = 0; step < 60; step++)
                root = (root + product / root) / 2;
            printf(" %6.3f", cov[first][second] / root);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 1.000  0.981  0.990
 0.981  1.000  0.976
 0.990  0.976  1.000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.19 مصفوفة المسافات بين الزبائن

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب **مصفوفة المسافات** `dist[5][5]` (مربع المسافة الإقليدية) بين 5 زبائن ممثلين بخاصيتين، مستغلًا **التناظر** `dist[a][b] = dist[b][a]` لحساب نصف الخانات فقط، ثم يجد أقرب زوج.

خطوات الحل

1) القطر أصفار.
2) لكل `first < second`: نحسب المسافة ونكتبها في الخانتين.
3) نحتفظ بأصغر مسافة.

البرنامج بلغة C

```c
#include <stdio.h>

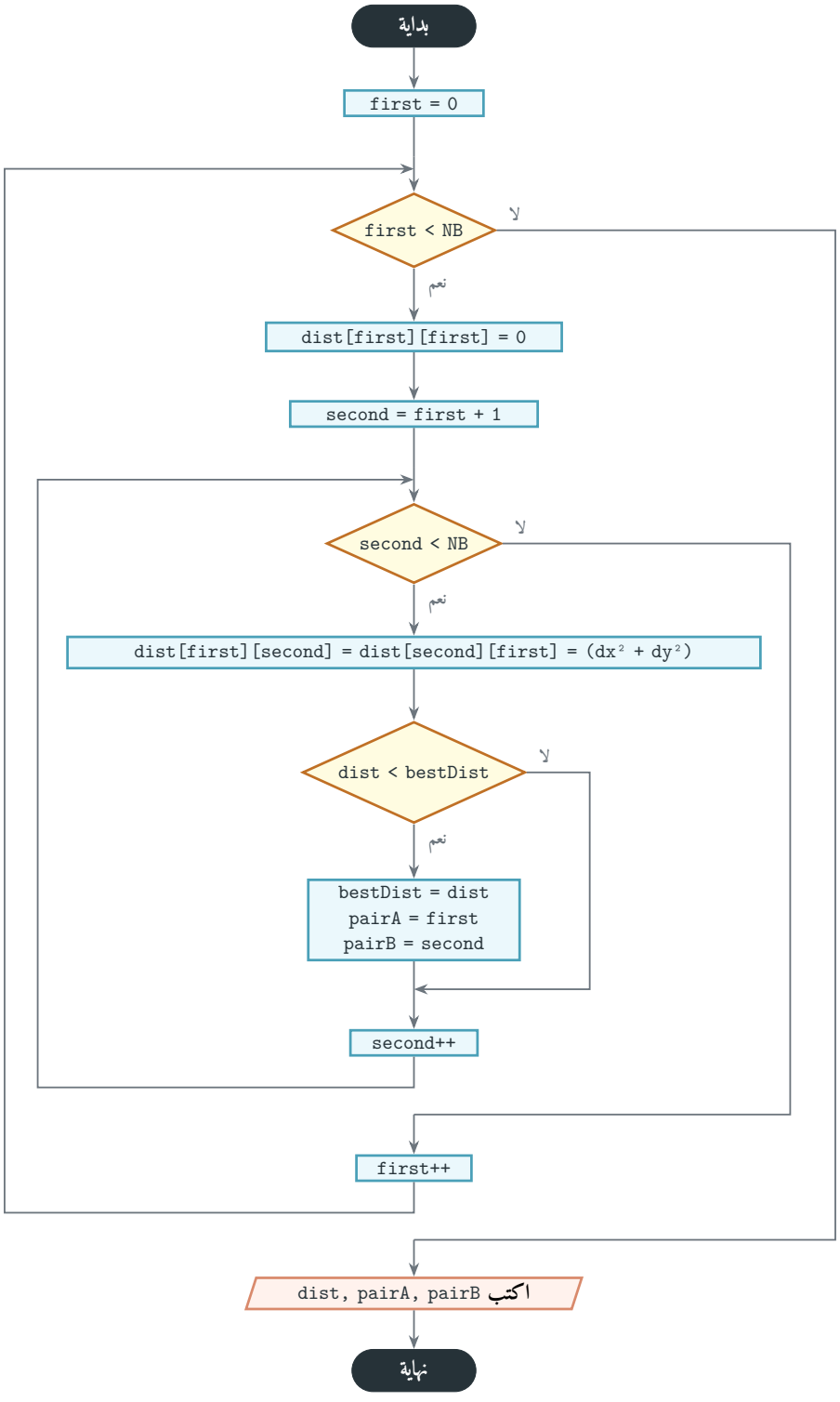
#define NB 5

int main(void) {
    double points[NB][2] = {{2, 1.5}, {8, 6}, {3, 2}, {9, 7.5}, {5, 3}};
    double dist[NB][NB], dx, dy, bestDist = -1;
    int first, second, pairA = 0, pairB = 0;
    for (first = 0; first < NB; first++) {
        dist[first][first] = 0;
        for (second = first + 1; second < NB; second++) {    // النصف العلوي فقط
            dx = points[first][0] - points[second][0];
            dy = points[first][1] - points[second][1];
            dist[first][second] = dist[second][first] = dx * dx + dy * dy;
            if (bestDist < 0 || dist[first][second] < bestDist) {
                bestDist = dist[first][second];
                pairA = first;
                pairB = second;
            }
        }
    }
    for (first = 0; first < NB; first++) {
        for (second = 0; second < NB; second++)
            printf("%7.2f", dist[first][second]);
        printf("\n");
    }
    printf("Closest customers: %d and %d\n", pairA, pairB);
    return 0;
}
```

نتيجة التنفيذ

```
   0.00  56.25   1.25  85.00  11.25
  56.25   0.00  41.00   3.25  18.00
   1.25  41.00   0.00  66.25   5.00
  85.00   3.25  66.25   0.00  36.25
  11.25  18.00   5.00  36.25   0.00
Closest customers: 0 and 2
```

المخطط الإنسيابي (Organigramme)



تمرين 5.20 مصفوفة الإلتباس والدقة

علم البيانات ★★ متوسط

نموذج يصنف التعليقات إلى 3 أصناف (0 سلبي، 1 محايد، 2 إيجابي). لدينا جدولا القيم الحقيقية والمتنبأ بها لـ 15 تعليقًا. اكتب برنامجًا يبني **مصفوفة الالتباس** `confusion[actual][predicted]` ويحسب **الدقة** (مجموع القطر / الكل).

خطوات الحل

1) نهيئ المصفوفة بأصفار.
2) لكل تعليق:

```
confusion[actual[index]][predicted[index]]++
```

.
3) الدقة = الأثر / عدد التعليقات.

البرنامج بلغة C

```c
#include <stdio.h>

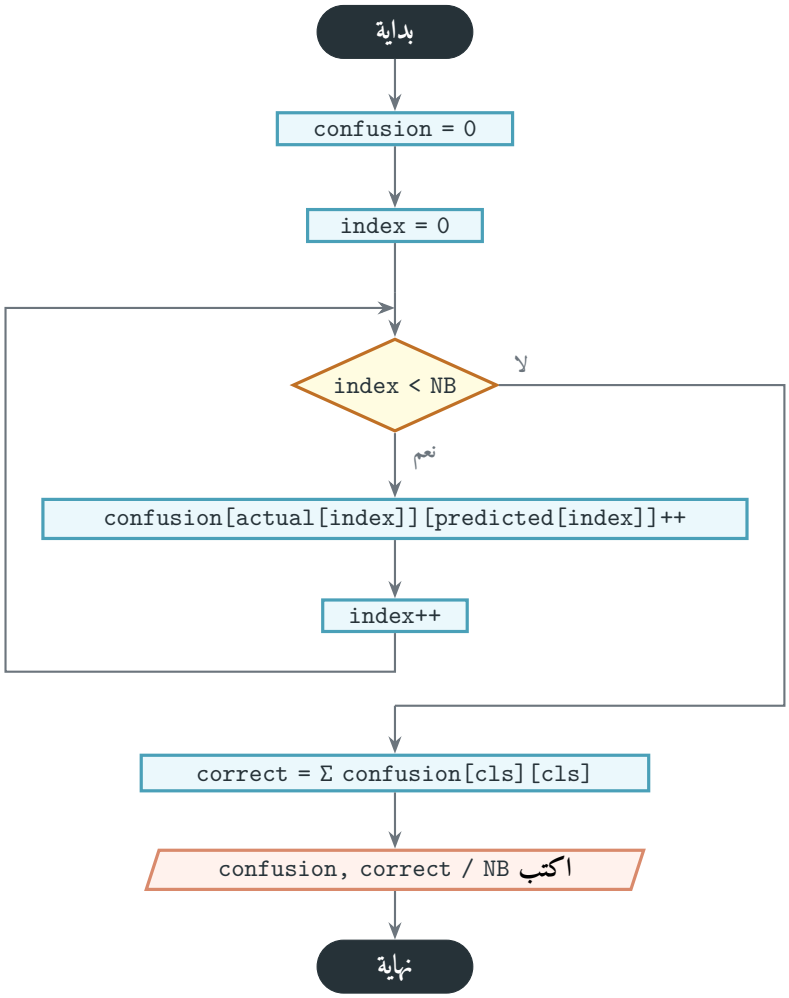
#define NB 15
#define NB_CLASSES 3

int main(void) {
    int actual[NB]    = {2, 0, 1, 2, 2, 0, 1, 1, 2, 0, 2, 1, 0, 2, 1};
    int predicted[NB] = {2, 0, 2, 2, 1, 0, 1, 0, 2, 0, 2, 1, 1, 2, 1};
    int confusion[NB_CLASSES][NB_CLASSES] = {{0}};
    int index, row, col, correct = 0;
    for (index = 0; index < NB; index++)
        confusion[actual[index]][predicted[index]]++;   // الفهرسان من الجدولين
    printf("actual\\pred  neg neu pos\n");
    for (row = 0; row < NB_CLASSES; row++) {
        printf("%11d", row);
        for (col = 0; col < NB_CLASSES; col++)
            printf(" %3d", confusion[row][col]);
        printf("\n");
        correct += confusion[row][row];   // القطر = التنبؤات الصحيحة
    }
    printf("Accuracy = %d/%d = %.2f\n", correct, NB, (double)correct / NB);
    return 0;
}
```

نتيجة التنفيذ

```
actual\pred  neg neu pos
          0   3   1   0
          1   1   3   1
          2   0   1   5
Accuracy = 11/15 = 0.73
```

المخطط الإنسيابي (Organigramme)



تمرين 5.21 الدقة والإستدعاء لكل صنف

علم البيانات ★★ متوسط

انطلاقًا من مصفوفة التباس جاهزة (3 أصناف)، اكتب برنامجًا يحسب لكل صنف **الدقة** `precision = TP /` و**الاستدعاء** `recall = TP /` و**F1** = `2PR / (P + R)`.

خطوات الحل

1) `TP = confusion[cls][cls]`.
2) مجموع العمود `cls`: كل التنبؤات بهذا الصنف.
3) مجموع السطر `cls`: كل العناصر التي صنفها الحقيقي `cls`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_CLASSES 3

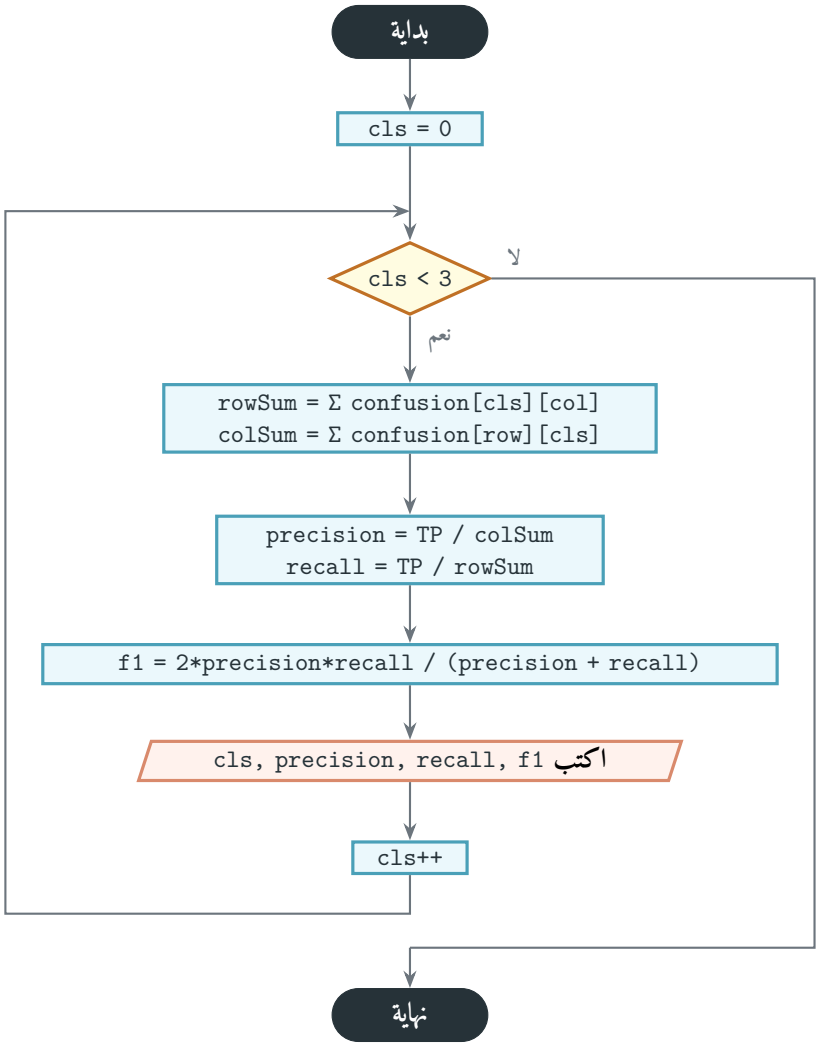
int main(void) {
    int confusion[NB_CLASSES][NB_CLASSES] = {{50,  8,  2},
                                             { 6, 30, 14},
```

```
                                            { 1,  9, 80}};
    int cls, other, rowSum, colSum, truePos;
    double precision, recall, f1;
    for (cls = 0; cls < NB_CLASSES; cls++) {
        rowSum = colSum = 0;
        for (other = 0; other < NB_CLASSES; other++) {
            rowSum += confusion[cls][other];  // cls = الحقيقي
            colSum += confusion[other][cls];  // cls = المتنبأ
        }
        truePos = confusion[cls][cls];
        precision = (double)truePos / colSum;
        recall = (double)truePos / rowSum;
        f1 = 2 * precision * recall / (precision + recall);
        printf("Class %d: precision %.3f, recall %.3f, F1 %.3f\n",
               cls, precision, recall, f1);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Class 0: precision 0.877, recall 0.833, F1 0.855
Class 1: precision 0.638, recall 0.600, F1 0.619
Class 2: precision 0.833, recall 0.889, F1 0.860
```

المخطط الإنسيابي (Organigramme)



تمرين 5.22 تناظر مصفوفة الشراء المشترك

علم البيانات ★★ متوسط

`together[a][b]` عدد المرات التي اشتُري فيها المنتجان `a` و`b` معًا؛ يجب أن تكون المصفوفة **متناظرة**. اكتب برنامجًا يتحقق من ذلك ويعرض الخانات غير المتطابقة (أخطاء تسجيل).

خطوات الحل

1) نقارن فقط النصف العلوي `row < col` بالنصف السفلي.
2) كل اختلاف خطأ نعرضه.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

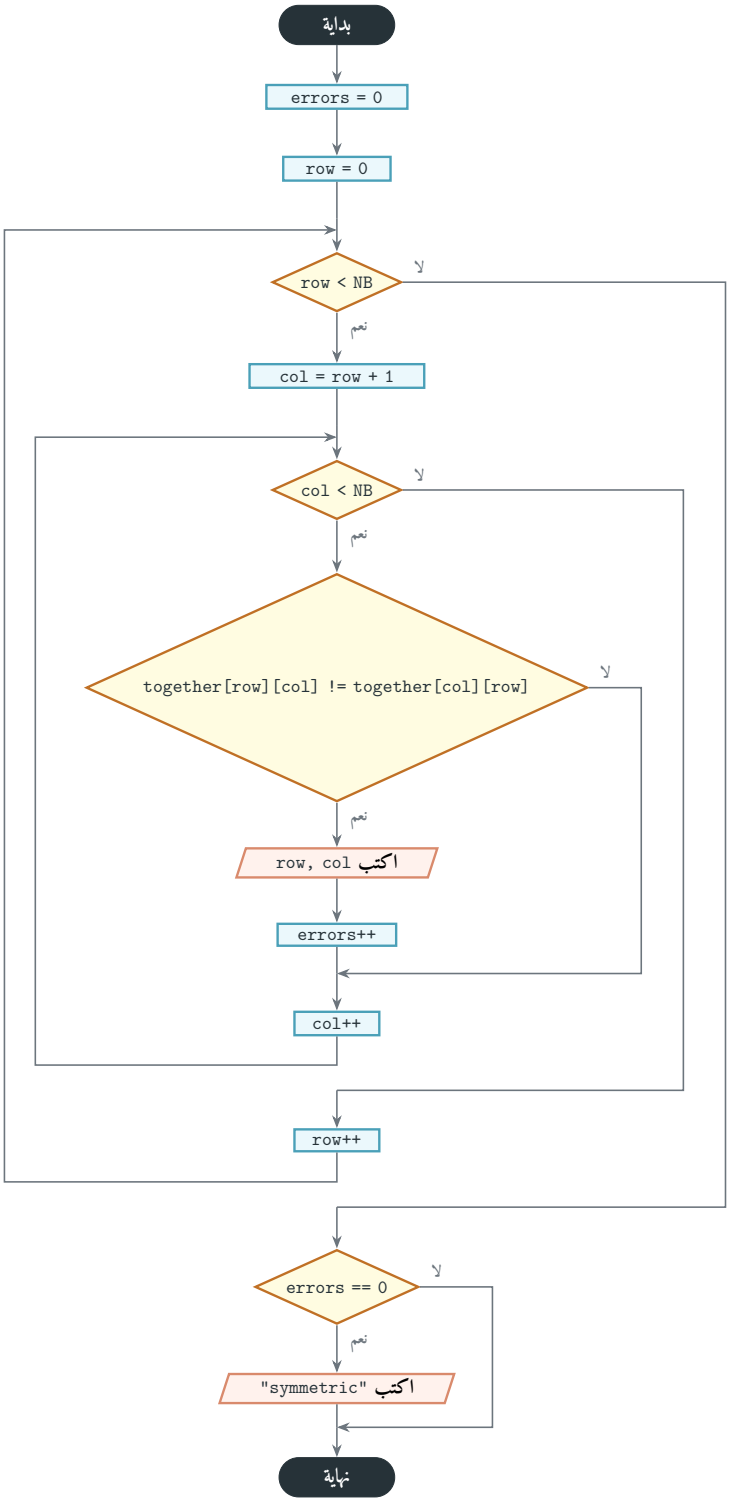
int main(void) {
    int together[NB][NB] = {{ 0, 12,  5,  3},
                            {12,  0,  7,  9},
```

```
                            { 5,  8,  0,  2},
                            { 3,  9,  2,  0}};
    int row, col, errors = 0;
    for (row = 0; row < NB; row++)
        for (col = row + 1; col < NB; col++)      // نصف المصفوفة يكفي
            if (together[row][col] != together[col][row]) {
                printf("Mismatch (%d,%d): %d vs %d\n", row, col,
                       together[row][col], together[col][row]);
                errors++;
            }
    if (errors == 0)
        printf("Matrix is symmetric\n");
    else
        printf("%d error(s) found\n", errors);
    return 0;
}
```

نتيجة التنفيذ

```
Mismatch (1,2): 7 vs 8
1 error(s) found
```

المخطط الإنسيابي (Organigramme)



تمرين 5.23 حساب الشراء المشترك من السلات

علم البيانات ★★ متوسط

مصفوفة `baskets[6][4]` تحتوي 1 إذا احتوت السلة (سطر) المنتج (عمود). اكتب برنامجًا يحسب مصفوفة **الشراء المشترك** `co = B × B`، حيث `co[a][b]` عدد السلات التي فيها `a` و`b` معًا، والقطر عدد السلات التي فيها كل منتج.

خطوات الحل

1) `co[first][second] = Σ baskets[basket][first] * baskets[basket][second]`

.

2) الجداء يساوي 1 فقط إذا كان المنتجان موجودين.

البرنامج بلغة C

```c
#include <stdio.h>

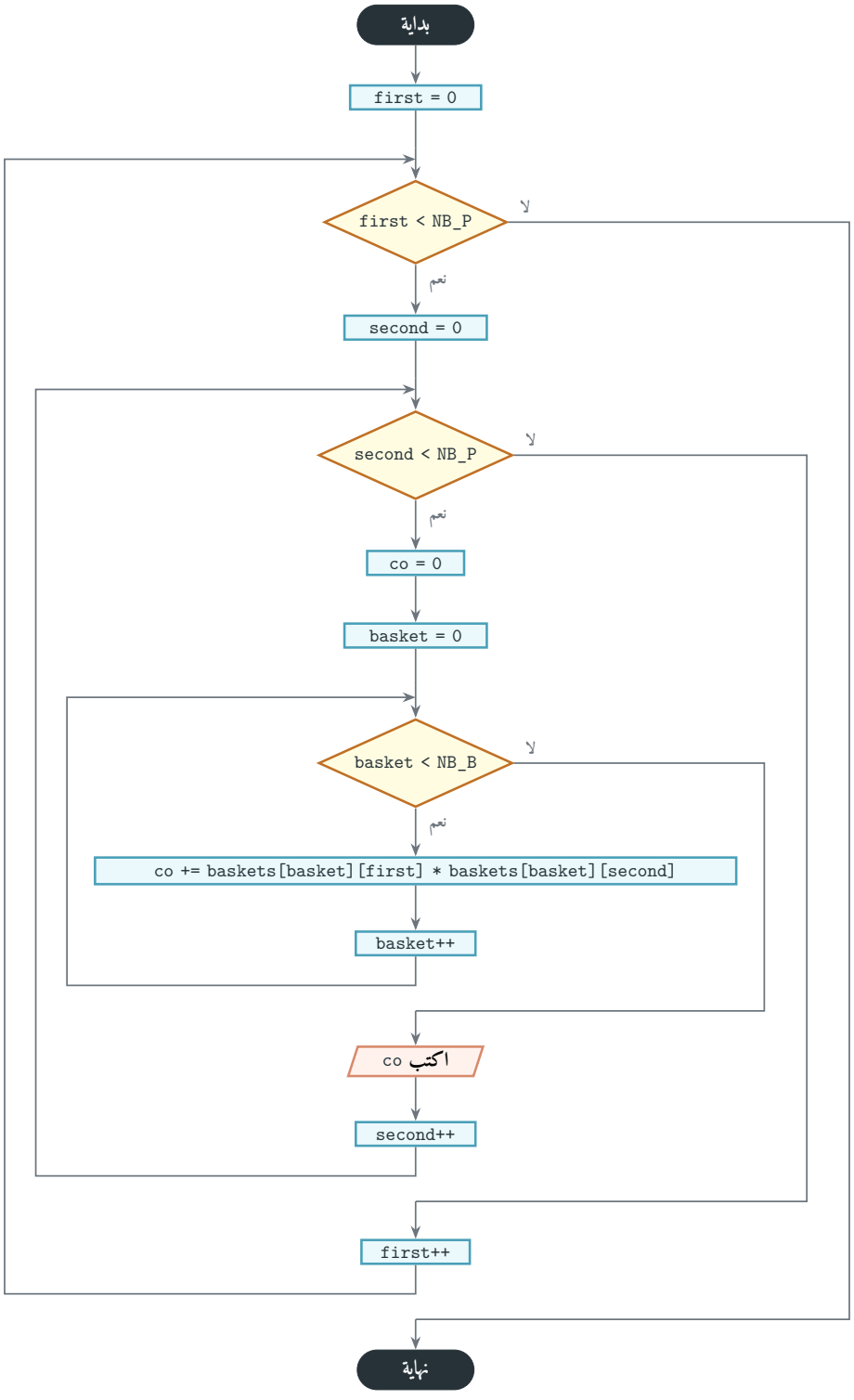
#define NB_B 6
#define NB_P 4

int main(void) {
    int baskets[NB_B][NB_P] = {{1, 1, 0, 0},
                               {1, 1, 1, 0},
                               {0, 1, 1, 0},
                               {1, 0, 0, 1},
                               {1, 1, 0, 1},
                               {0, 0, 1, 1}};
    int co[NB_P][NB_P], first, second, basket;
    for (first = 0; first < NB_P; first++) {
        for (second = 0; second < NB_P; second++) {
            co[first][second] = 0;
            for (basket = 0; basket < NB_B; basket++)   // عمود × عمود
                co[first][second] += baskets[basket][first] * baskets[basket][second];
            printf(" %d", co[first][second]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
4 3 1 2
3 4 2 1
1 2 3 1
2 1 1 3
```

المخطط الإنسيابي (Organigramme)



تمرين 5.24 أقرب رف فارغ

تجارة إلكترونية ★★ متوسط

المستودع شبكة رفوف `grid[5][6]` (1 مشغول، 0 فارغ). العامل في الموقع `(row, col)`. اكتب برنامجًا يجد **أقرب رف** فارغ بمسافة مانهاتن `|dr| + |dc|` (عدد الخطوات في الممرات).

خطوات الحل

1) نمر على كل الخانات الفارغة.
2) المسافة: القيمة المطلقة لفرق الأسطر + القيمة المطلقة لفرق الأعمدة.
3) نحتفظ بالأقرب.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int grid[5][6] = {{1, 1, 0, 1, 1, 1},
                      {1, 1, 1, 1, 1, 0},
                      {1, 1, 1, 1, 1, 1},
                      {0, 1, 1, 1, 1, 1},
                      {1, 1, 1, 1, 0, 1}};
    int myRow, myCol, row, col, dRow, dCol, dist;
    int best = -1, bestRow = -1, bestCol = -1;
    scanf("%d %d", &myRow, &myCol);
    for (row = 0; row < 5; row++)
        for (col = 0; col < 6; col++)
            if (grid[row][col] == 0) {
                dRow = row - myRow; if (dRow < 0) dRow = -dRow;
                dCol = col - myCol; if (dCol < 0) dCol = -dCol;
                dist = dRow + dCol;               // مسافة مانهاتن
                if (best < 0 || dist < best) {
                    best = dist; bestRow = row; bestCol = col;
                }
            }
    printf("Nearest empty shelf: (%d, %d), %d steps\n", bestRow, bestCol, best);
    return 0;
}
```

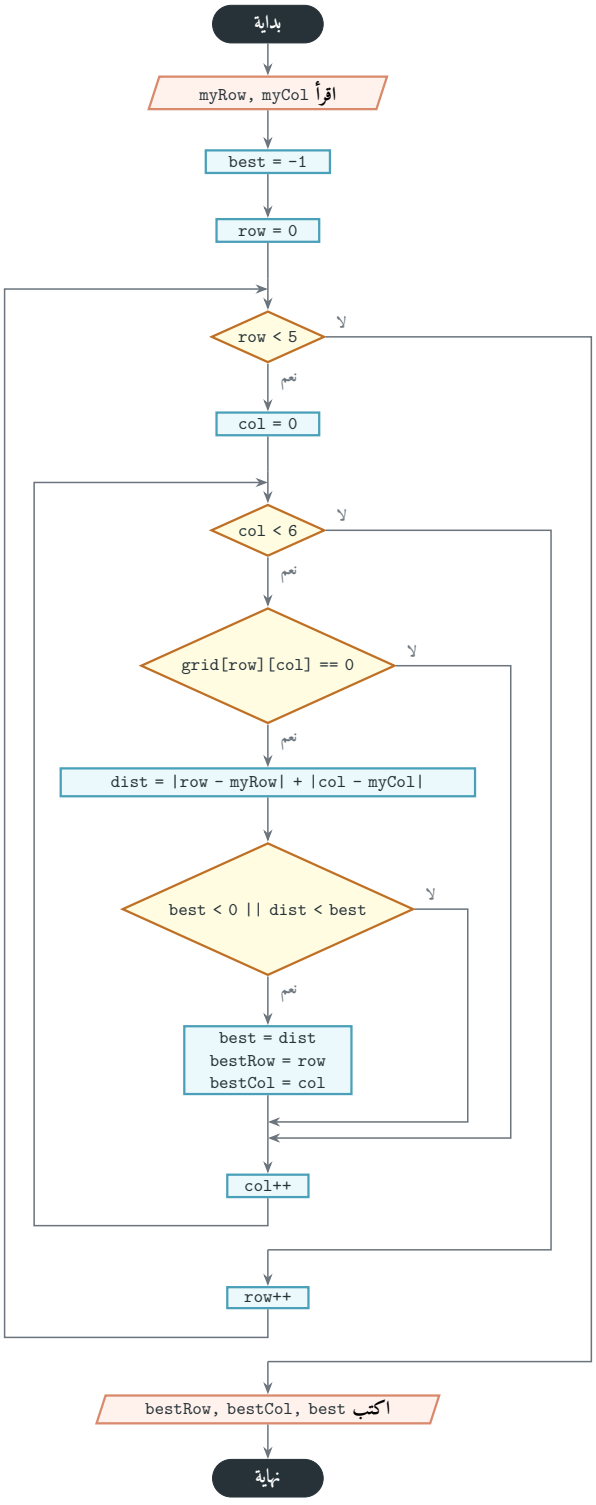
المدخلات

```
2 3
```

نتيجة التنفيذ

```
Nearest empty shelf: (0, 2), 3 steps
```

المخطط الإنسيابي (Organigramme)



تمرين 5.25 خريطة حرارية للطلبيات

علم البيانات ★★ متوسط

مصفوفة `orders[7][6]`: عدد الطلبيات لكل يوم من الأسبوع (أسطر) في 6 فترات من اليوم (أعمدة). اكتب برنامجًا يرسم خريطة حرارية نصية: كل خانة تُرسم برمز حسب شدتها . ثم - ثم + ثم #، بعد تحديد الأكبر.

خطوات الحل

1) نجد القيمة القصوى `peak`.
2) المستوى: `level = value * 4 / (peak + 1)` بين 0 و3.
3) `switch` على المستوى لاختيار الرمز.

البرنامج بلغة C

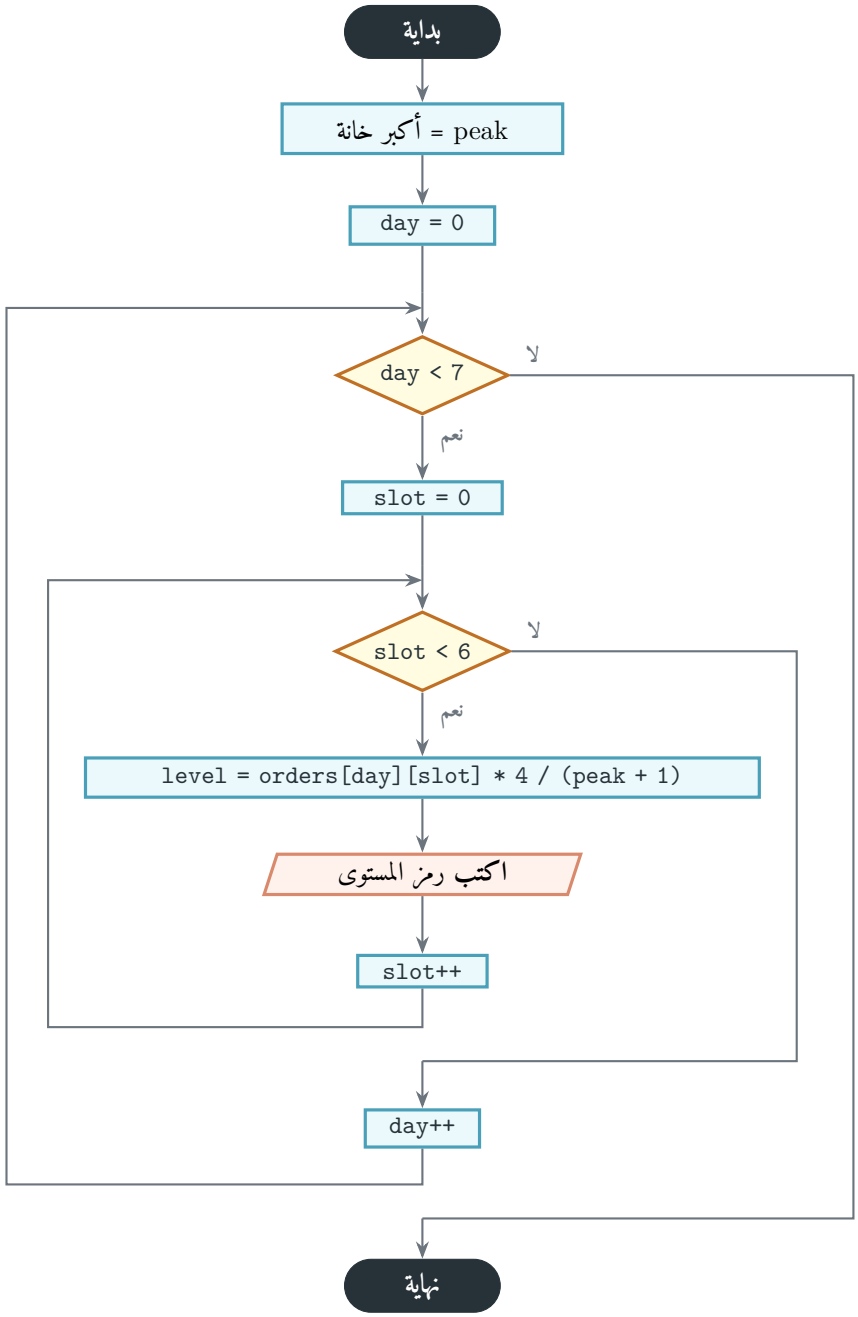
```c
#include <stdio.h>

int main(void) {
    int orders[7][6] = {{ 2,  5, 14, 10, 18, 25},
                        { 1,  6, 12,  9, 20, 22},
                        { 3,  4, 15, 11, 17, 28},
                        { 2,  7, 13, 12, 21, 30},
                        { 5,  9, 20, 16, 26, 38},
                        { 8, 15, 25, 22, 30, 41},
                        { 7, 12, 22, 18, 24, 29}};
    int day, slot, level, peak = 0;
    char symbol;
    for (day = 0; day < 7; day++)
        for (slot = 0; slot < 6; slot++)
            if (orders[day][slot] > peak)
                peak = orders[day][slot];
    printf("     0-4 4-8 8-12 12-16 16-20 20-24\n");
    for (day = 0; day < 7; day++) {
        printf("Day %d ", day + 1);
        for (slot = 0; slot < 6; slot++) {
            level = orders[day][slot] * 4 / (peak + 1);   // 0..3
            switch (level) {
                case 0:  symbol = '.'; break;
                case 1:  symbol = '-'; break;
                case 2:  symbol = '+'; break;
                default: symbol = '#';
            }
            printf("  %c  ", symbol);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
     0-4 4-8 8-12 12-16 16-20 20-24
Day 1   .    .    -    .    -    +
Day 2   .    .    -    .    -    +
Day 3   .    .    -    -    -    +
Day 4   .    .    -    -    +    +
Day 5   .    .    -    -    +    #
Day 6   .    -    +    +    +    #
Day 7   .    -    +    -    +    +
```

المخطط الإنسيابي (Organigramme)



تمرين 5.26 جدول محوري من السجلات

علم البيانات ★★ متوسط

لدينا سجلات بيع: لكل عملية (الصنف 0..2، الشهر 0..3، المبلغ). اكتب برنامجًا يبني **جدولًا محوريًا** `pivot[3][4]` يجمع المبالغ حسب الصنف والشهر، مع مجاميع الأسطر والأعمدة.

خطوات الحل

1) نهيئ المصفوفة بأصفار.
2) لكل سجل:

```
pivot[category[rec]][month[rec]] += amount[rec]
```

.

3) نعرض الجدول مع مجموع كل سطر، ثم سطر مجاميع الأعمدة.

البرنامج بلغة C

```c
#include <stdio.h>

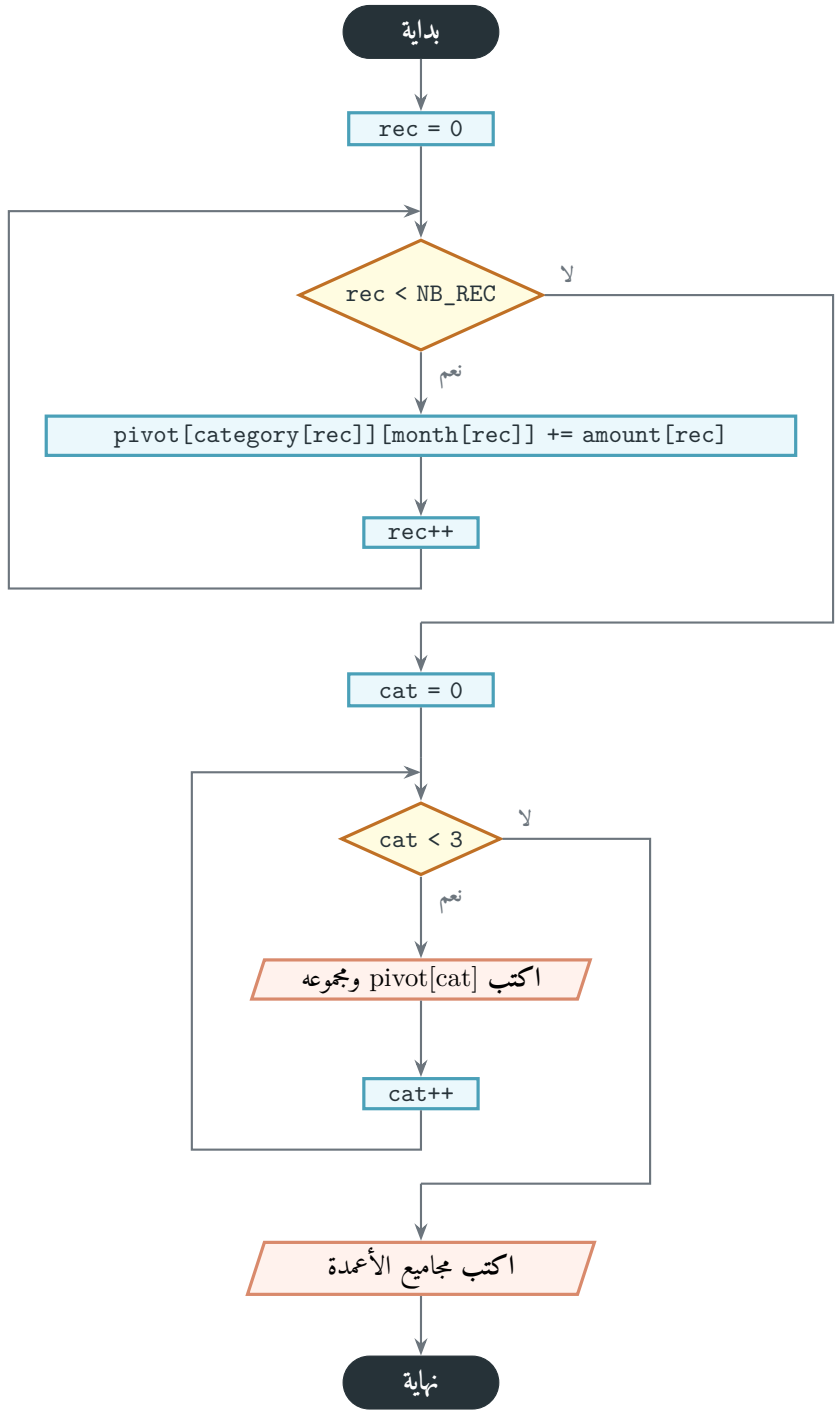
#define NB_REC 12

int main(void) {
    int category[NB_REC] = {0, 1, 0, 2, 1, 1, 0, 2, 2, 0, 1, 2};
    int month[NB_REC]    = {0, 0, 1, 1, 2, 0, 3, 3, 0, 2, 3, 2};
    int amount[NB_REC]   = {500, 300, 450, 900, 120, 220, 610, 400, 350, 200, 180, 700};
    int pivot[3][4] = {{0}}, colTotal[4] = {0};
    int rec, cat, mon, rowTotal;
    for (rec = 0; rec < NB_REC; rec++)
        pivot[category[rec]][month[rec]] += amount[rec];   // التجميع
    printf("cat |  M1   M2   M3   M4 | total\n");
    for (cat = 0; cat < 3; cat++) {
        rowTotal = 0;
        printf("%3d |", cat);
        for (mon = 0; mon < 4; mon++) {
            printf(" %4d", pivot[cat][mon]);
            rowTotal += pivot[cat][mon];
            colTotal[mon] += pivot[cat][mon];
        }
        printf(" | %5d\n", rowTotal);
    }
    printf("sum |");
    for (mon = 0; mon < 4; mon++)
        printf(" %4d", colTotal[mon]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
cat |  M1   M2   M3   M4 | total
  0 |  500  450  200  610 |  1760
  1 |  520    0  120  180 |   820
  2 |  350  900  700  400 |  2350
sum | 1370 1350 1020 1190
```

المخطط الإنسيابي (Organigramme)



تمرين 5.27 مجاميع المستطيلات بالمجاميع التراكمية الثنائية

علم البيانات ★★ متوسط

للإجابة بسرعة عن "مجموع المبيعات للمتاجر من `r1` إلى `r2` في الأيام من `c1` إلى `c2`"، نبني مصفوفة **مجاميع تراكمية ثنائية** `prefix[row+1][col+1]` = مجموع المستطيل من `(0,0)` إلى `(row,col)`. كل استعلام يصبح 4 عمليات.

خطوات الحل

1) `prefix[r+1][c+1] = sales[r][c] + prefix[r][c+1] + prefix[r+1][c] - prefix[r][c]`

.

2) المستطيل:

```
prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]
```

.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int sales[3][4] = {{120, 95, 140, 180},
                       { 80, 60,  75,  90},
                       {200, 210, 190, 250}};
    int prefix[4][5] = {{0}};              // سطر وعمود إضافيان من الأصفار
    int row, col, r1, r2, c1, c2;
    for (row = 0; row < 3; row++)
        for (col = 0; col < 4; col++)
            // نطرح التداخل الذي جُمع مرتين
            prefix[row + 1][col + 1] = sales[row][col] + prefix[row][col + 1]
                                     + prefix[row + 1][col] - prefix[row][col];
    scanf("%d %d %d %d", &r1, &r2, &c1, &c2);
    printf("Stores %d-%d, days %d-%d: %d\n", r1, r2, c1 + 1, c2 + 1,
           prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
           - prefix[r2 + 1][c1] + prefix[r1][c1]);
    return 0;
}
```

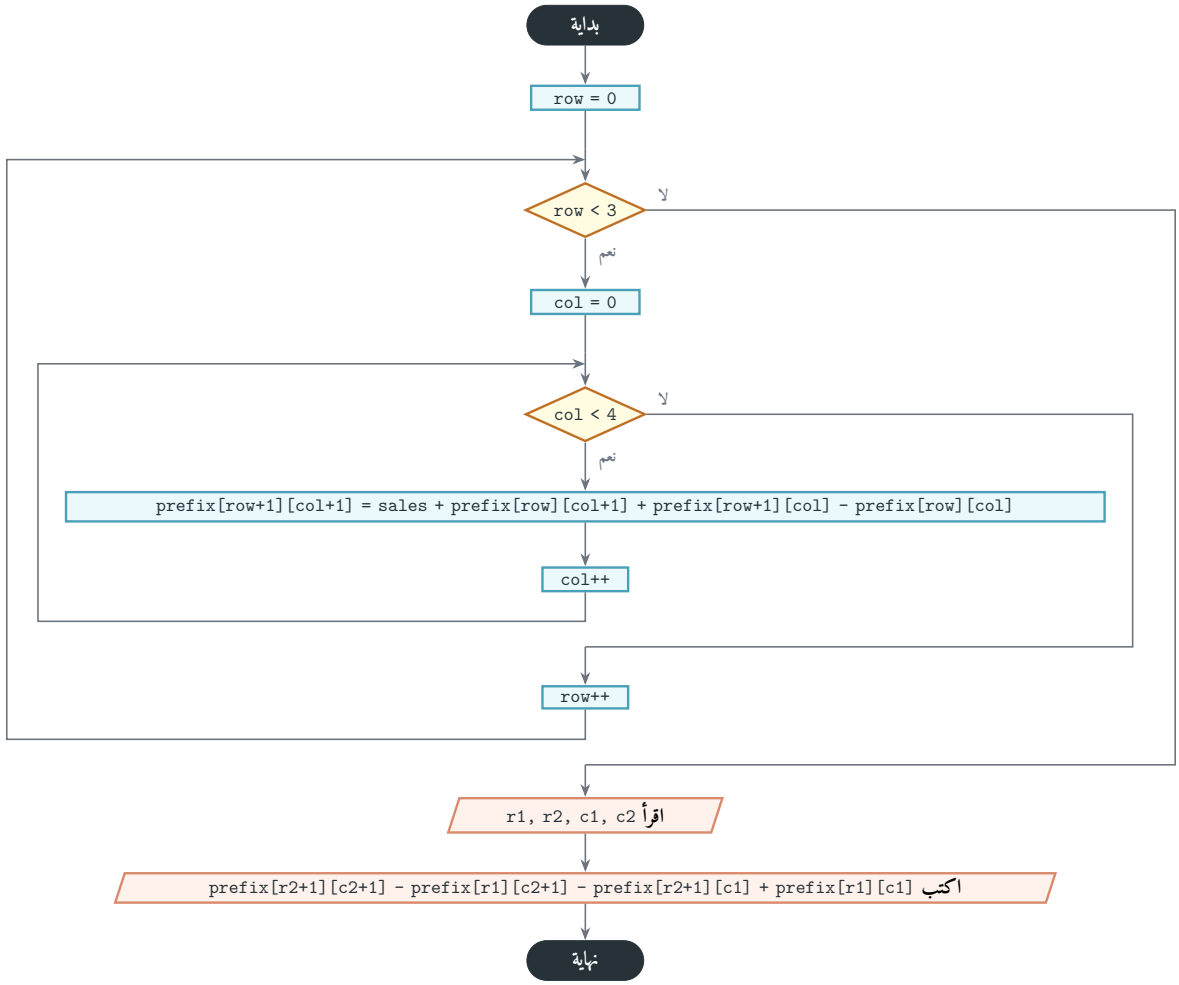
المدخلات

```
1 2 1 3
```

نتيجة التنفيذ

```
Stores 1-2, days 2-4: 875
```

المخطط الإنسيابي (Organigramme)



تمرين 5.28 تدوير صورة المنتج

متوسط ★★ تجارة إلكترونية

صورة منتج صغيرة مخزنة كمصفوفة بكسلات `image[4][6]` (درجات رمادي). اكتب برنامجًا يديرها **90° مع عقارب الساعة** في مصفوفة جديدة `rotated[6][4]`.

خطوات الحل

1) البكسل `(row, col)` ينتقل إلى `(col, HEIGHT - 1 - row)`.

2) أبعاد الصورة تتبادل.

البرنامج بلغة C

```c
#include <stdio.h>

#define HEIGHT 4
#define WIDTH 6

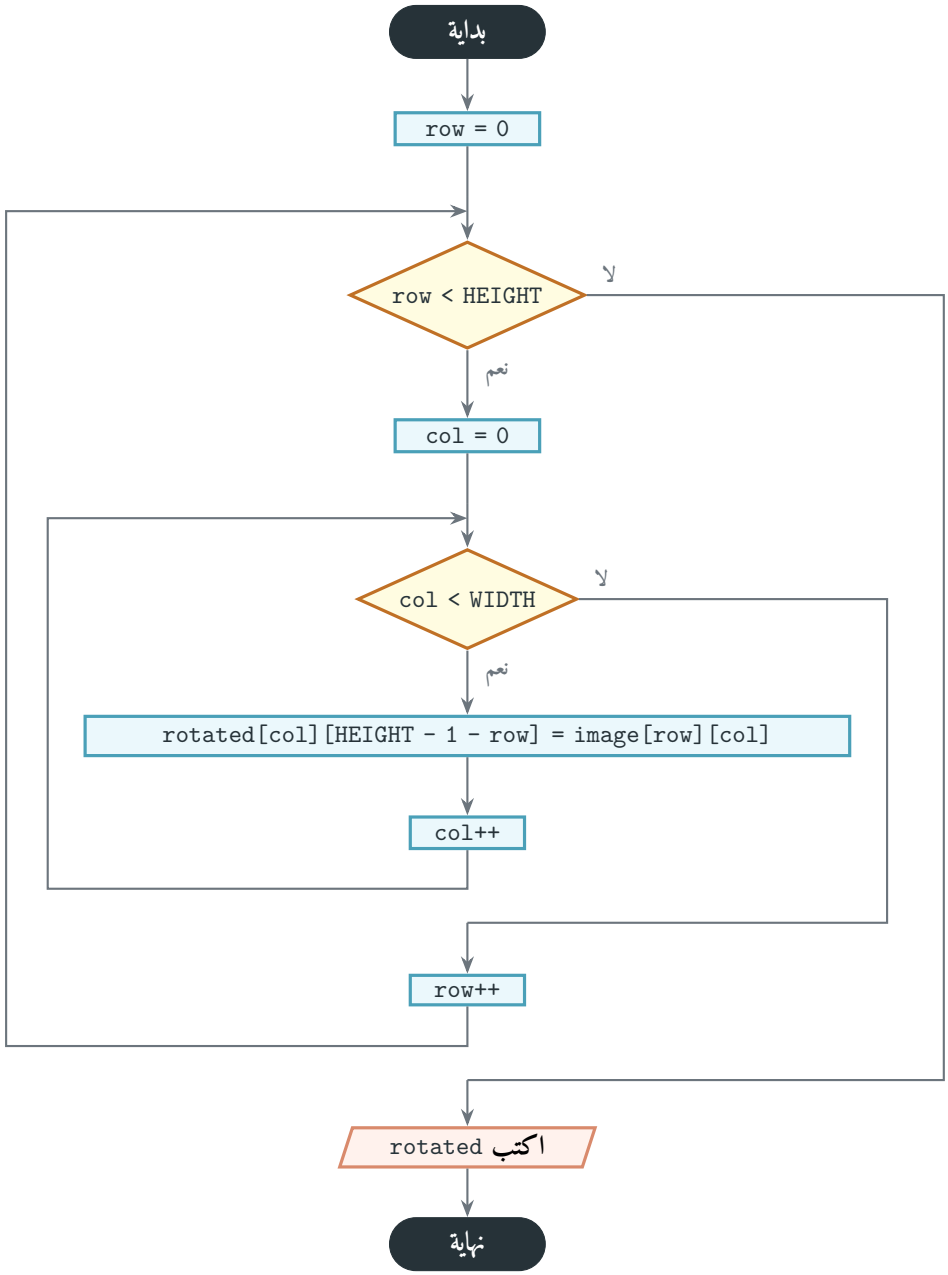
int main(void) {
    int image[HEIGHT][WIDTH] = {{ 1,  2,  3,  4,  5,  6},
                                { 7,  8,  9, 10, 11, 12},
                                {13, 14, 15, 16, 17, 18},
```

```
                                {19, 20, 21, 22, 23, 24}};
    int rotated[WIDTH][HEIGHT], row, col;
    for (row = 0; row < HEIGHT; row++)
        for (col = 0; col < WIDTH; col++)
            // السطر الأول يصبح العمود الأخير
            rotated[col][HEIGHT - 1 - row] = image[row][col];
    for (row = 0; row < WIDTH; row++) {
        for (col = 0; col < HEIGHT; col++)
            printf("%3d", rotated[row][col]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
 19 13  7  1
 20 14  8  2
 21 15  9  3
 22 16 10  4
 23 17 11  5
 24 18 12  6
```

المخطط الإنسيابي (Organigramme)



تمرين 5.29 تصغير صورة إلى مصغّرة

تجارة إلكترونية ★★ متوسط

لعرض مصغّرات المنتجات، اكتب برنامجًا يصغر صورة `image[6][8]` إلى النصف: كل بكسل في المصغرة متوسط مربع 2×2 من الصورة الأصلية.

خطوات الحل

1) المصغرة أبعادها 3×4.
2) `thumb[row][col]` = متوسط البكسلات `(2row, 2col)`، `(2row, 2col+1)`، `(2row+1, 2col)`، `(2row+1, 2col+1)`.

البرنامج بلغة C

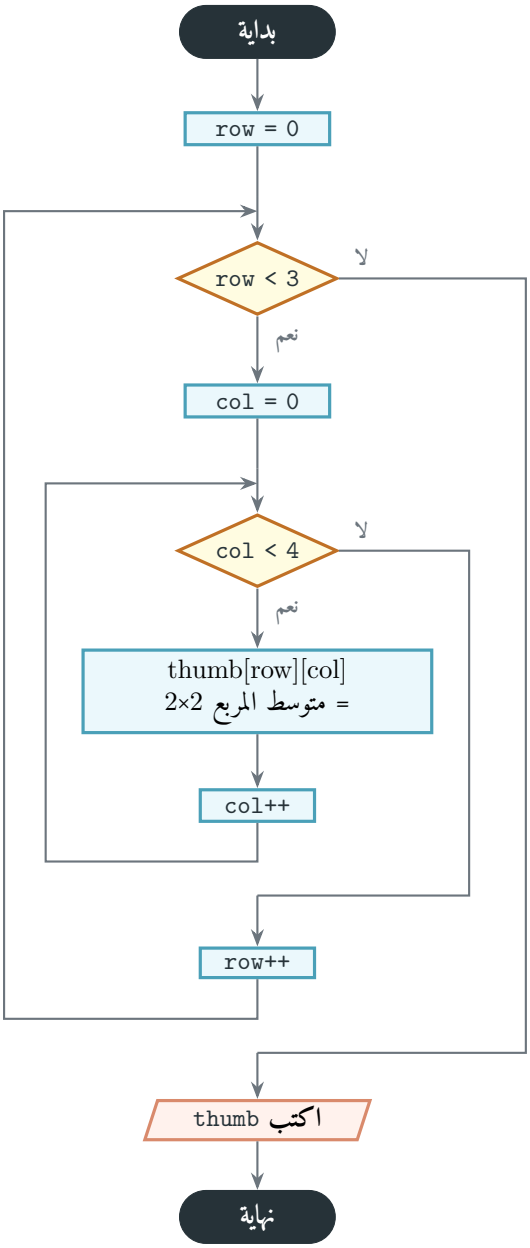
```c
#include <stdio.h>

int main(void) {
    int image[6][8] = {{ 10,  12, 200, 210,  50,  52, 90,  94},
                       { 14,  16, 205, 215,  54,  56, 92,  96},
                       {100, 104,  30,  34, 250, 246, 10,  12},
                       {102, 106,  32,  36, 248, 252, 14,  16},
                       { 60,  60,  60,  60, 120, 124, 180, 184},
                       { 60,  60,  60,  60, 128, 132, 188, 192}};
    int thumb[3][4], row, col;
    for (row = 0; row < 3; row++) {
        for (col = 0; col < 4; col++) {
            thumb[row][col] = (image[2 * row][2 * col] + image[2 * row][2 * col + 1]
                               + image[2 * row + 1][2 * col] + image[2 * row + 1][2 * col + 1]) / 4;
            printf("%4d", thumb[row][col]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
  13 207  53  93
 103  33 249  13
  60  60 126 186
```

المخطط الإنسيابي (Organigramme)



تمرين 5.30 تنعيم صورة بمرشح المتوسط

علم البيانات ★★ متوسط

اكتب برنامجًا يطبق **مرشح المتوسط** 3×3 على صورة `image[5][5]` لإزالة الضجيج: كل بكسل داخلي يصبح متوسط جيرانه التسعة، بكسلات الحافة تبقى كما هي.

خطوات الحل

1) نعمل على مصفوفة جديدة `blurred` لكي لا نستعمل قيمًا معدلة.
2) لكل بكسل داخلي: حلقتان صغيرتان على `dRow` و`dCol` من -1 إلى 1.

البرنامج بلغة C

```c
#include <stdio.h>

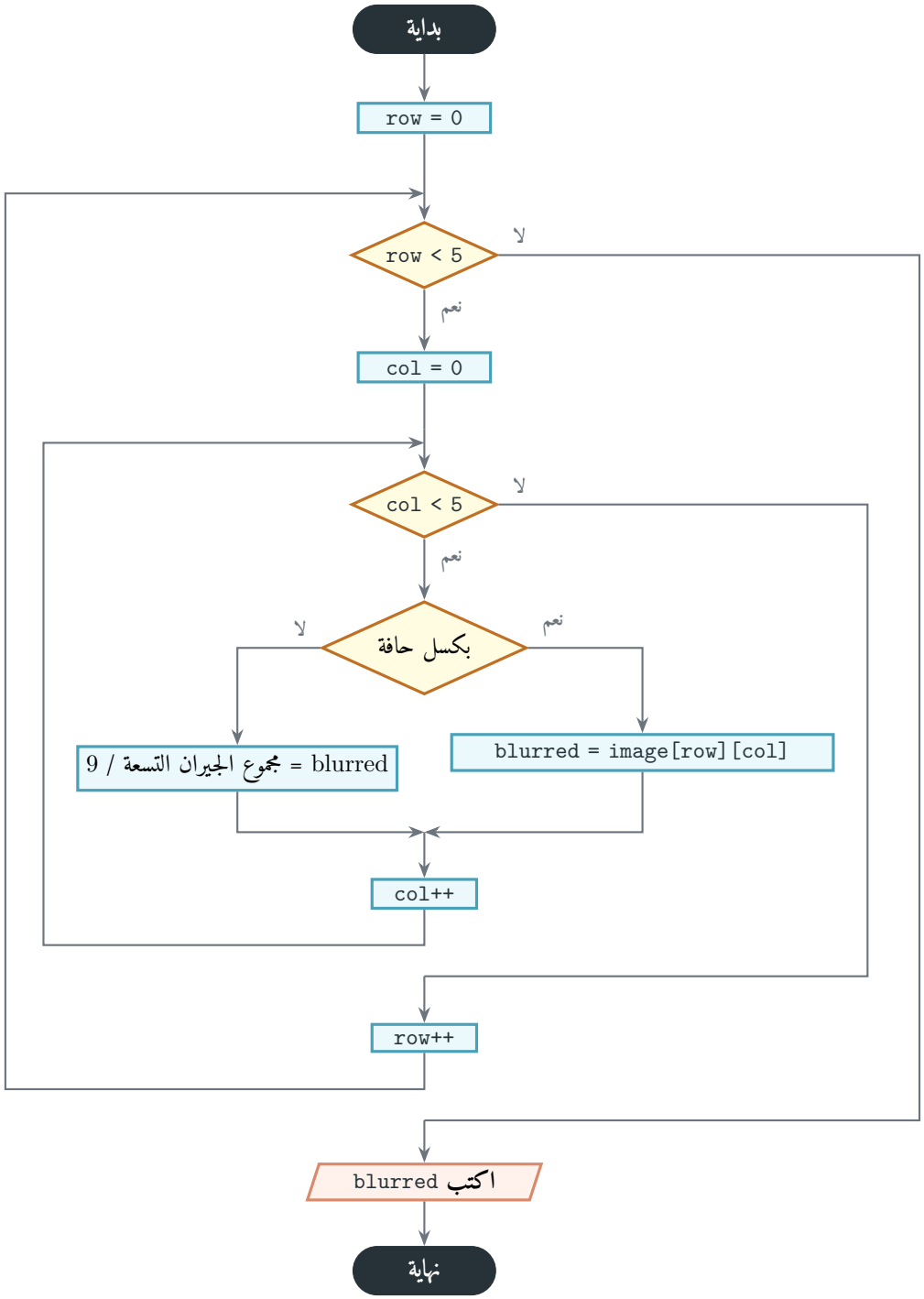
#define SIZE 5

int main(void) {
    int image[SIZE][SIZE] = {{10, 10,  10, 10, 10},
                             {10, 10, 255, 10, 10},
                             {10, 10,  10, 10, 10},
                             {10, 10,  10,  0, 10},
                             {10, 10,  10, 10, 10}};
    int blurred[SIZE][SIZE], row, col, dRow, dCol, sum;
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++) {
            if (row == 0 || col == 0 || row == SIZE - 1 || col == SIZE - 1) {
                blurred[row][col] = image[row][col];   // الحافة دون تغيير
            } else {
                sum = 0;
                for (dRow = -1; dRow <= 1; dRow++)     // الجوار 3×3
                    for (dCol = -1; dCol <= 1; dCol++)
                        sum += image[row + dRow][col + dCol];
                blurred[row][col] = sum / 9;
            }
            printf("%4d", blurred[row][col]);
        }
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
  10  10  10  10  10
  10  37  37  37  10
  10  37  36  36  10
  10  10   8   8  10
  10  10  10  10  10
```

المخطط الإنسيابي (Organigramme)



تمرين 5.31 سلسلة ماركوف لحالات الزبائن

علم البيانات ★★ متوسط

للزبون 3 حالات: 0 جديد، 1 نشط، 2 خامل. مصفوفة الانتقال `transition[from][to]` تعطي احتمال الانتقال شهريًا. انطلاقًا من توزيع ابتدائي (1000 جديد)، اكتب برنامجًا يحسب توزيع الزبائن بعد كل شهر لمدة `months` أشهر.

خطوات الحل

1) `next[to] = Σ current[from] * transition[from][to]` (جداء شعاع في مصفوفة).
2) ننسخ `next` إلى `current` بعد كل شهر.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_STATES 3

int main(void) {
    double transition[NB_STATES][NB_STATES] = {{0.0, 0.6, 0.4},   // جديد
                                               {0.0, 0.8, 0.2},   // نشط
                                               {0.0, 0.3, 0.7}};  // خامل
    double current[NB_STATES] = {1000, 0, 0}, next[NB_STATES];
    int months, month, from, to;
    scanf("%d", &months);
    for (month = 1; month <= months; month++) {
        for (to = 0; to < NB_STATES; to++) {
            next[to] = 0;
            for (from = 0; from < NB_STATES; from++)
                next[to] += current[from] * transition[from][to];
        }
        for (to = 0; to < NB_STATES; to++)
            current[to] = next[to];        // الشهر التالي ينطلق من هنا
        printf("Month %d: new %.0f, active %.1f, idle %.1f\n",
               month, current[0], current[1], current[2]);
    }
    return 0;
}
```

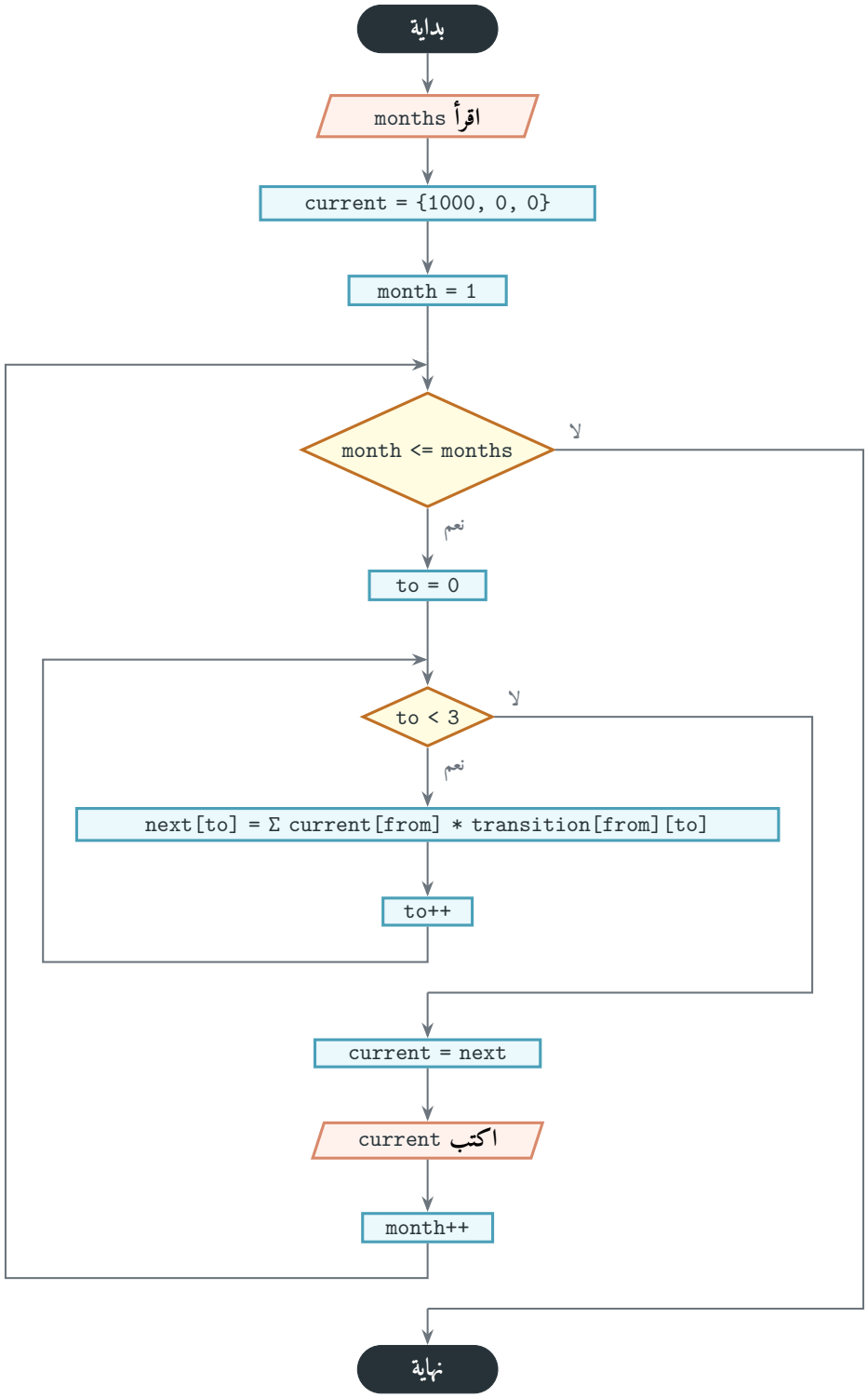
المدخلات

```
6
```

نتيجة التنفيذ

```
Month 1: new 0, active 600.0, idle 400.0
Month 2: new 0, active 600.0, idle 400.0
Month 3: new 0, active 600.0, idle 400.0
Month 4: new 0, active 600.0, idle 400.0
Month 5: new 0, active 600.0, idle 400.0
Month 6: new 0, active 600.0, idle 400.0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.32 جدول المسافات المثلثي

تجارة إلكترونية ★★ متوسط

جدول المسافات بين المدن متناظر، فتكفي مصفوفة مثلثية سفلية. اكتب برنامجًا يتحقق أن مصفوفة مقروءة مثلثية سفلية (كل الخانات فوق القطر أصفار)، ثم يستعملها لإيجاد المسافة بين مدينتين في أي ترتيب.

خطوات الحل

1) مثلثية سفلية: `matrix[row][col] == 0` لكل `col > row`.

2) المسافة بين a وb: نقرأ دائمًا `matrix[max][min]`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

int main(void) {
    int dist[NB][NB] = {{  0,   0,   0, 0},     // الجزائر
                        {430,   0,   0, 0},     // وهران
                        {390, 820,   0, 0},     // قسنطينة
                        {200, 600, 340, 0}};    // سطيف
    int row, col, lower = 1, cityA, cityB, tmp;
    for (row = 0; row < NB; row++)
        for (col = row + 1; col < NB; col++)    // فوق القطر
            if (dist[row][col] != 0)
                lower = 0;
    printf("Lower triangular: %s\n", lower ? "yes" : "no");
    scanf("%d %d", &cityA, &cityB);
    if (cityA < cityB) {                        // نقرأ دائمًا تحت القطر
        tmp = cityA; cityA = cityB; cityB = tmp;
    }
    printf("Distance = %d km\n", dist[cityA][cityB]);
    return 0;
}
```

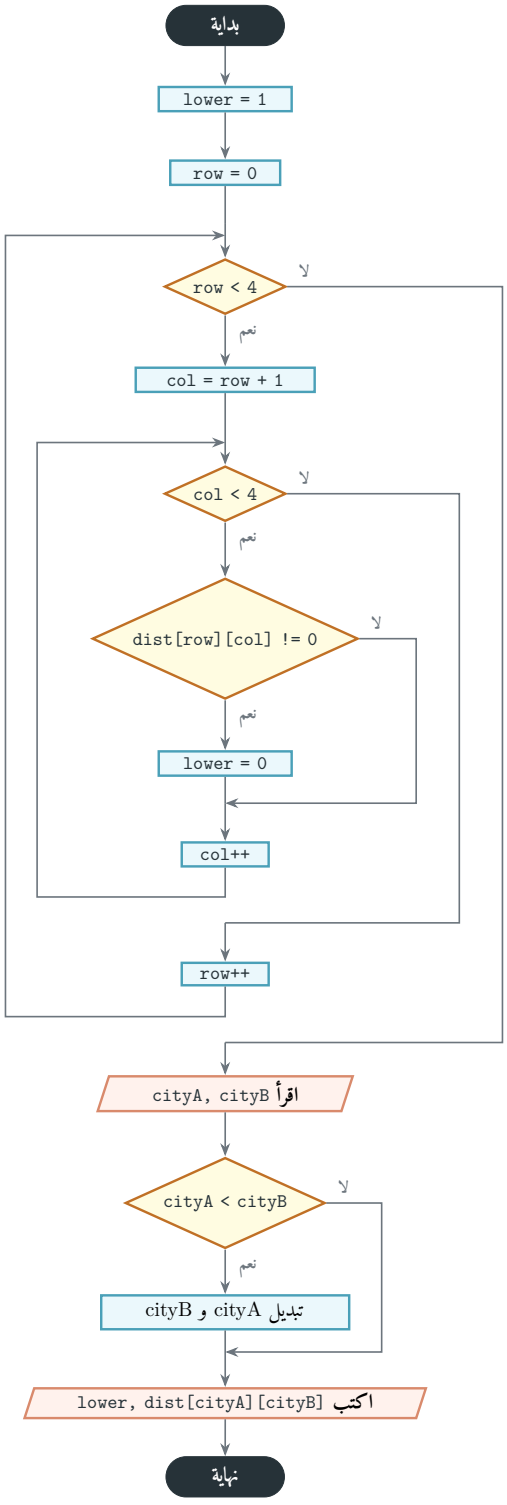
المدخلات

```
1 3
```

نتيجة التنفيذ

```
Lower triangular: yes
Distance = 600 km
```

المخطط الإنسيابي (Organigramme)



تمرين 5.33 المستودع الأرخص لكل زبون

تجارة إلكترونية

متوسط

مصفوفة `shipCost[3][5]`: تكلفة الشحن من كل مستودع (سطر) إلى كل زبون (عمود). اكتب برنامجًا يختار لكل زبون **المستودع الأرخص**، ويحسب التكلفة الكلية وعدد الزبائن المخدومين من كل مستودع.

خطوات الحل

1) لكل عمود: بحث عن أصغر قيمة.
2) `served[bestDepot]++` و`total += shipCost[bestDepot][customer]`.

البرنامج بلغة C

```c
#include <stdio.h>

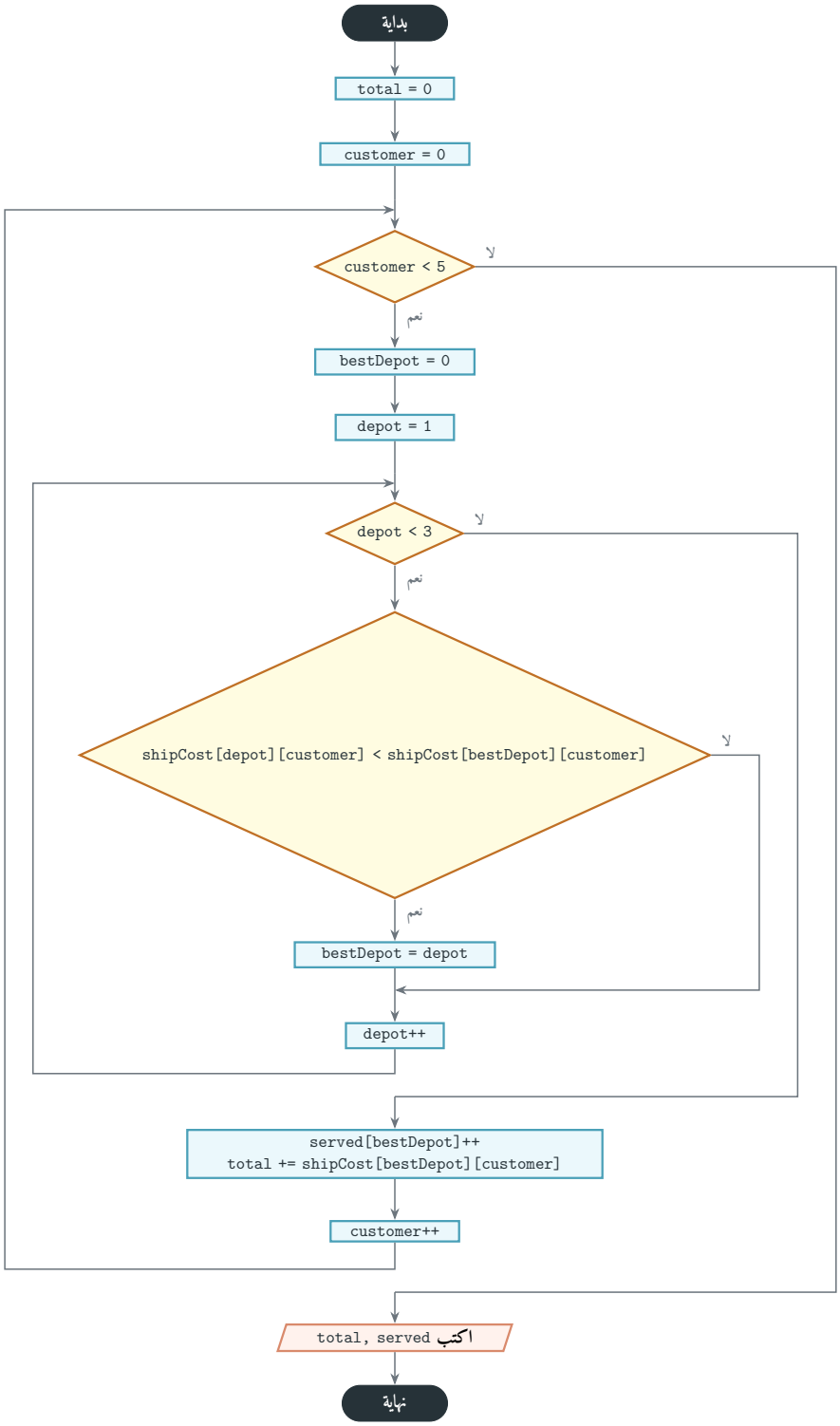
#define NB_DEPOTS 3
#define NB_CUST 5

int main(void) {
    int shipCost[NB_DEPOTS][NB_CUST] = {{400, 650, 300, 900, 500},
                                        {550, 300, 450, 600, 700},
                                        {700, 500, 350, 350, 450}};
    int served[NB_DEPOTS] = {0};
    int customer, depot, bestDepot, total = 0;
    for (customer = 0; customer < NB_CUST; customer++) {
        bestDepot = 0;
        for (depot = 1; depot < NB_DEPOTS; depot++)
            if (shipCost[depot][customer] < shipCost[bestDepot][customer])
                bestDepot = depot;
        served[bestDepot]++;
        total += shipCost[bestDepot][customer];
        printf("Customer %d <- depot %d (%d DA)\n", customer, bestDepot,
               shipCost[bestDepot][customer]);
    }
    printf("Total cost = %d DA\n", total);
    for (depot = 0; depot < NB_DEPOTS; depot++)
        printf("Depot %d serves %d customer(s)\n", depot, served[depot]);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 0 <- depot 0 (400 DA)
Customer 1 <- depot 1 (300 DA)
Customer 2 <- depot 0 (300 DA)
Customer 3 <- depot 2 (350 DA)
Customer 4 <- depot 2 (450 DA)
Total cost = 1800 DA
Depot 0 serves 2 customer(s)
Depot 1 serves 1 customer(s)
Depot 2 serves 2 customer(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.34 كثافة مصفوفة متفرقة

علم البيانات ★★ متوسط

مصفوفة زبائن × منتجات أغلب خاناتها أصفار (**متفرقة**). لتوفير الذاكرة نخزن فقط الخانات غير الصفرية بصيغة **الإحداثيات** `(row, col, value)` في ثلاثة جداول. اكتب برنامجًا يحسب الكثافة وينشئ هذا التمثيل.

خطوات الحل

1) نمر على المصفوفة، كل خانة غير صفرية تضاف إلى الجداول الثلاثة.
2) الكثافة = عدد الخانات غير الصفرية / عدد الخانات.

البرنامج بلغة C

```c
#include <stdio.h>

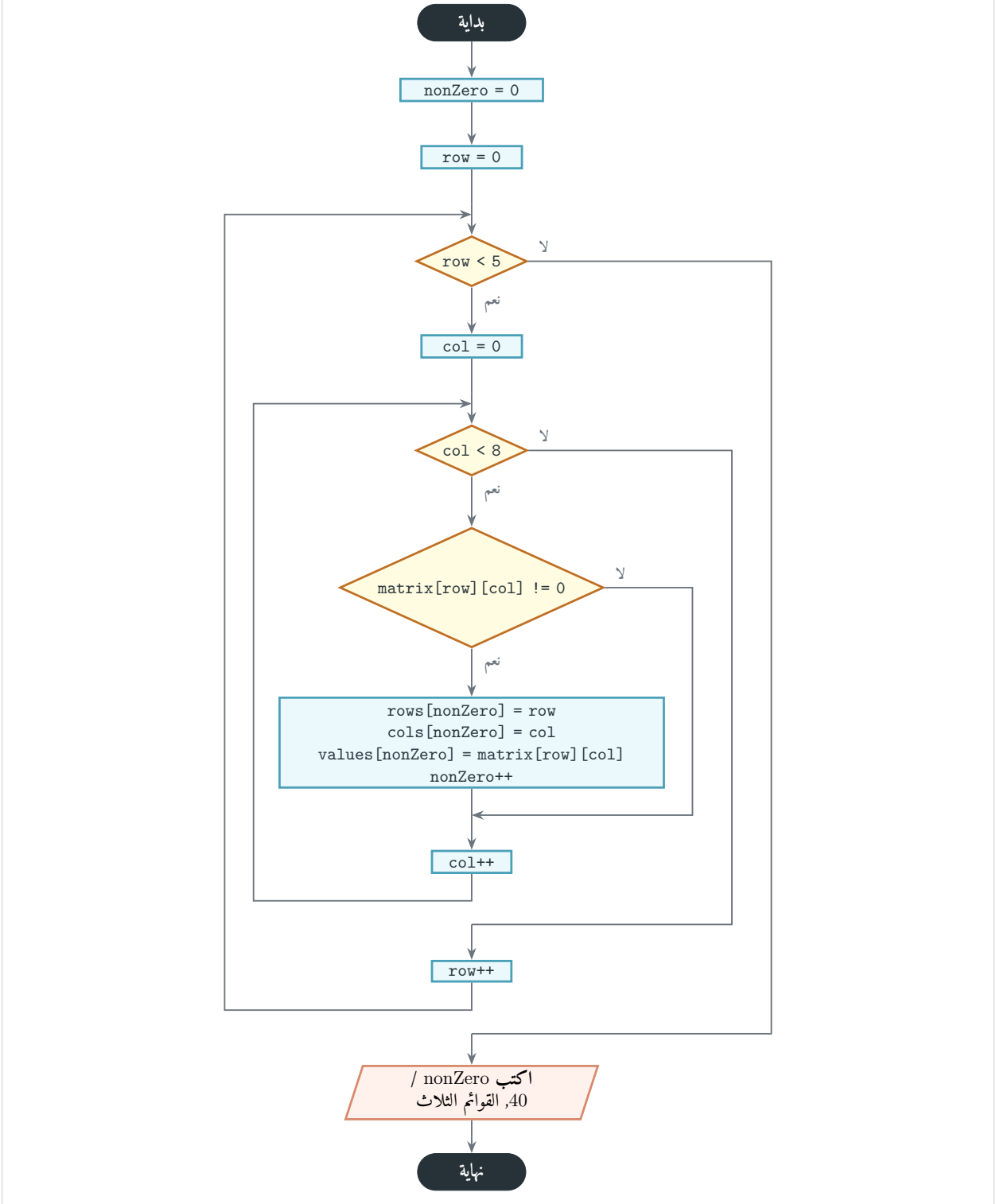
#define NB_ROWS 5
#define NB_COLS 8

int main(void) {
    int matrix[NB_ROWS][NB_COLS] = {{0, 0, 3, 0, 0, 0, 0, 1},
                                    {0, 0, 0, 0, 0, 0, 0, 0},
                                    {2, 0, 0, 0, 5, 0, 0, 0},
                                    {0, 0, 0, 0, 0, 0, 4, 0},
                                    {0, 1, 0, 0, 0, 0, 0, 0}};
    int rows[NB_ROWS * NB_COLS], cols[NB_ROWS * NB_COLS], values[NB_ROWS * NB_COLS];
    int row, col, nonZero = 0, index;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++)
            if (matrix[row][col] != 0) {      // نحفظ الإحداثيات والقيمة فقط
                rows[nonZero] = row;
                cols[nonZero] = col;
                values[nonZero] = matrix[row][col];
                nonZero++;
            }
    printf("Density = %d/%d = %.1f %%\n", nonZero, NB_ROWS * NB_COLS,
           nonZero * 100.0 / (NB_ROWS * NB_COLS));
    for (index = 0; index < nonZero; index++)
        printf("(%d,%d)=%d ", rows[index], cols[index], values[index]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Density = 6/40 = 15.0 %
(0,2)=3 (0,7)=1 (2,0)=2 (2,4)=5 (3,6)=4 (4,1)=1
```

المخطط الإنسيابي (Organigramme)



تمرين 5.35 عدد مسارات الروبوت في المستودع

تجارة إلكترونية ★★ متوسط

روبوت المستودع يتحرك من الزاوية العليا اليسرى إلى السفلى اليمنى، **يمينًا أو أسفل فقط**، وبعض الخانات محجوبة (1). اكتب برنامجًا يحسب **عدد المسارات** الممكنة بالبرمجة الديناميكية:

```
paths[r][c] = paths[r-1][c] + paths[r][c-1]
```

.

خطوات الحل

1) خانة محجوبة: 0 مسار.
2) خانة البداية: مسار واحد إن لم تكن محجوبة.
3) الخانات الأخرى: مجموع الخانة فوقها والخانة على يسارها (إن وجدتا).

البرنامج بلغة C

```c
#include <stdio.h>

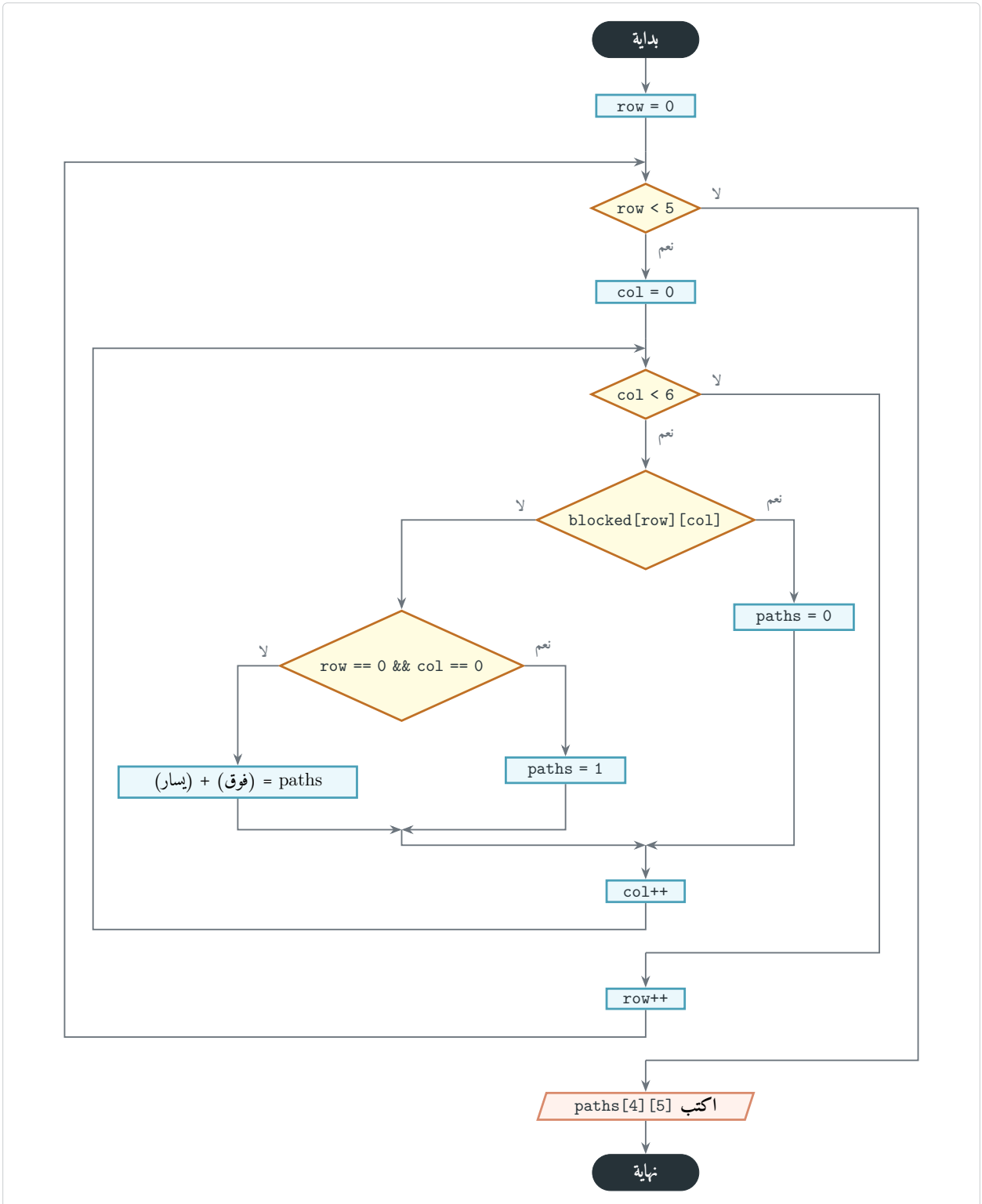
#define NB_ROWS 5
#define NB_COLS 6

int main(void) {
    int blocked[NB_ROWS][NB_COLS] = {{0, 0, 0, 0, 0, 0},
                                     {0, 1, 0, 0, 1, 0},
                                     {0, 0, 0, 1, 0, 0},
                                     {1, 0, 0, 0, 0, 0},
                                     {0, 0, 1, 0, 0, 0}};
    long paths[NB_ROWS][NB_COLS];
    int row, col;
    for (row = 0; row < NB_ROWS; row++) {
        for (col = 0; col < NB_COLS; col++) {
            if (blocked[row][col])
                paths[row][col] = 0;              // لا يمر منها أحد
            else if (row == 0 && col == 0)
                paths[row][col] = 1;              // نقطة الانطلاق
            else
                paths[row][col] = (row > 0 ? paths[row - 1][col] : 0)
                                + (col > 0 ? paths[row][col - 1] : 0);
            printf("%4ld", paths[row][col]);
        }
        printf("\n");
    }
    printf("Paths to the exit: %ld\n", paths[NB_ROWS - 1][NB_COLS - 1]);
    return 0;
}
```

نتيجة التنفيذ

```
   1   1   1   1   1   1
   1   0   1   2   0   1
   1   1   2   0   0   1
   0   1   3   3   3   4
   0   1   0   3   6  10
Paths to the exit: 10
```

المخطط الإنسيابي (Organigramme)



تمرين 5.36 أرخص مسار للتوصيل في المدينة

تجارة إلكترونية ★★ متوسط

المدينة شبكة أحياء `cost[4][5]` (زمن عبور كل حي بالدقائق). سائق التوصيل يتحرك **يمينًا أو أسفل**. اكتب برنامجًا يحسب **أقل زمن** من الزاوية العليا اليسرى إلى السفلى اليمنى، ويعيد بناء المسار.

خطوات الحل

1) `best[r][c] = cost[r][c] + min(best[r-1][c], best[r][c-1])`

.

2) إعادة البناء: من النهاية نعود إلى الخانة (فوق أو يسار) ذات القيمة الأصغر.

</> البرنامج بلغة C

```c
#include <stdio.h>

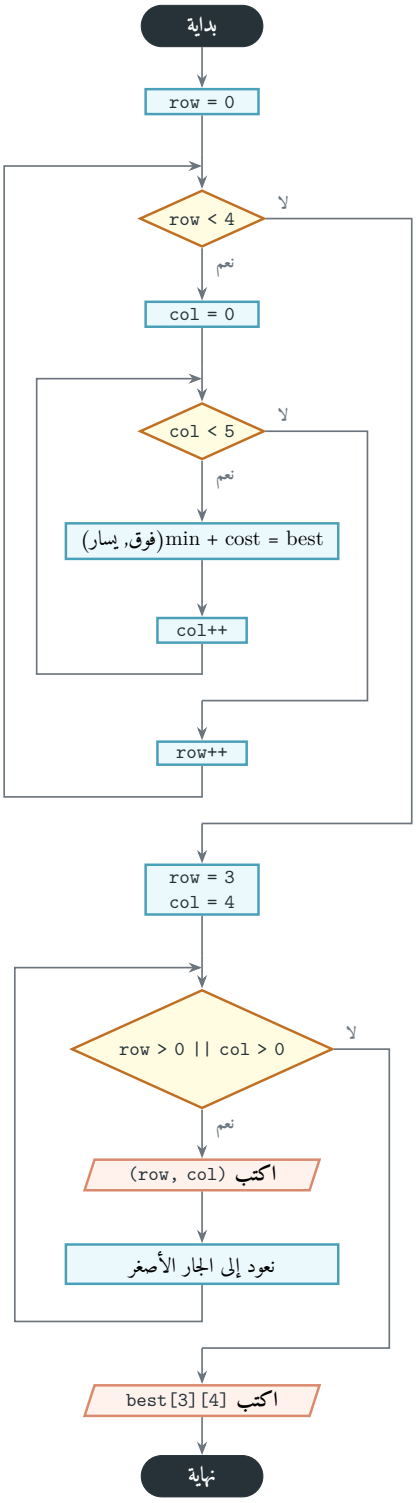
#define NB_ROWS 4
#define NB_COLS 5

int main(void) {
    int cost[NB_ROWS][NB_COLS] = {{3, 7, 2, 8, 1},
                                  {4, 1, 9, 2, 6},
                                  {8, 2, 1, 5, 3},
                                  {6, 9, 4, 1, 2}};
    int best[NB_ROWS][NB_COLS], row, col, up, left;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++) {
            if (row == 0 && col == 0) { best[row][col] = cost[row][col]; continue; }
            up = (row > 0) ? best[row - 1][col] : -1;     // غير موجود :1-
            left = (col > 0) ? best[row][col - 1] : -1;
            if (up < 0 || (left >= 0 && left < up))
                best[row][col] = cost[row][col] + left;
            else
                best[row][col] = cost[row][col] + up;
        }
    printf("Minimum time = %d min\nPath (reversed):", best[NB_ROWS - 1][NB_COLS - 1]);
    row = NB_ROWS - 1; col = NB_COLS - 1;
    while (row > 0 || col > 0) {          // نعود من النهاية إلى البداية
        printf(" (%d,%d)", row, col);
        if (row == 0) col--;
        else if (col == 0) row--;
        else if (best[row - 1][col] <= best[row][col - 1]) row--;
        else col--;
    }
    printf(" (0,0)\n");
    return 0;
}
```

>_ نتيجة التنفيذ

```
Minimum time = 18 min
Path (reversed): (3,4) (3,3) (3,2) (2,2) (2,1) (1,1) (1,0) (0,0)
```

المخطط الإنسيابي (Organigramme)



تمرين 5.37 أسعار الوحدة من ثلاث فواتير (قاعدة كرامر)

تجارة إلكترونية ★★ متوسط

لدينا ثلاث فواتير، كل واحدة تحتوي كميات من نفس ثلاثة منتجات ومبلغًا إجماليًا، لكن الأسعار مفقودة. اكتب برنامجًا يحل جملة المعادلات $A \times prices = totals$ بـ **قاعدة كرامر**: المحدد بطريقة ساروس.

خطوات الحل

1) `det(A)` بصيغة ساروس للمصفوفة 3×3.
2) لكل مجهول `unknown`: نعوض العمود `unknown` بشعاع المبالغ ونحسب المحدد.
3) `price[unknown] = detReplaced / det`.

البرنامج بلغة C

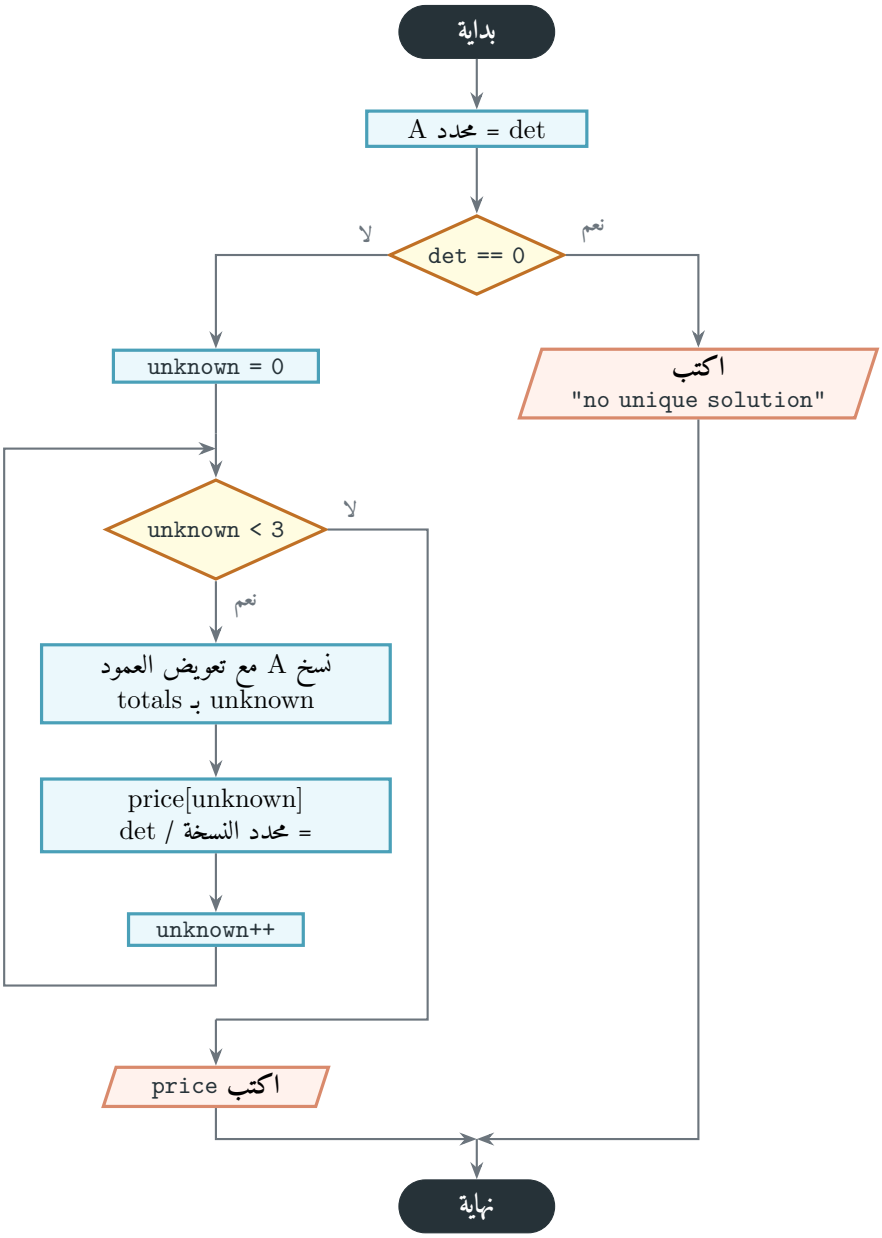
```c
#include <stdio.h>

int main(void) {
    double qty[3][3] = {{2, 1, 3},        // الفاتورة 1: الكميات
                        {1, 4, 1},
                        {3, 2, 2}};
    double totals[3] = {5100, 4300, 6200};
    double work[3][3], det, detReplaced, price[3];
    int unknown, row, col;
    // قاعدة ساروس: مجموع الأقطار ناقص مجموع الأقطار المعاكسة
    det = qty[0][0] * qty[1][1] * qty[2][2] + qty[0][1] * qty[1][2] * qty[2][0]
        + qty[0][2] * qty[1][0] * qty[2][1] - qty[0][2] * qty[1][1] * qty[2][0]
        - qty[0][0] * qty[1][2] * qty[2][1] - qty[0][1] * qty[1][0] * qty[2][2];
    if (det == 0) {
        printf("No unique solution\n");
        return 0;
    }
    for (unknown = 0; unknown < 3; unknown++) {
        for (row = 0; row < 3; row++)
            for (col = 0; col < 3; col++)
                work[row][col] = (col == unknown) ? totals[row] : qty[row][col];
        detReplaced = work[0][0] * work[1][1] * work[2][2] + work[0][1] * work[1][2] * work[2][0]
                    + work[0][2] * work[1][0] * work[2][1] - work[0][2] * work[1][1] * work[2][0]
                    - work[0][0] * work[1][2] * work[2][1] - work[0][1] * work[1][0] * work[2][2];
        price[unknown] = detReplaced / det;
        printf("Product %d: %.2f DA\n", unknown, price[unknown]);
    }
    printf("det = %.0f\n", det);
    return 0;
}
```

نتيجة التنفيذ

```
Product 0: 1200.00 DA
Product 1: 600.00 DA
Product 2: 700.00 DA
det = -17
```

المخطط الإنسيابي (Organigramme)



تمرين 5.38 ترتيب المتاجر حسب مجموع مبيعاتها

علم البيانات ★★ متوسط

اكتب برنامجًا يرتب **أسطر** مصفوفة المبيعات (متاجر × أيام) تنازليًا حسب مجموع كل سطر، بتبديل الأسطر كاملة، ثم يعرض الترتيب.

خطوات الحل

1) نحسب `total[store]` لكل سطر.

2) ترتيب بالانتقاء على `total`، عند التبديل نبدل أيضًا السطرين كاملين وأرقام المتاجر.

البرنامج بلغة C

```c
#include <stdio.h>

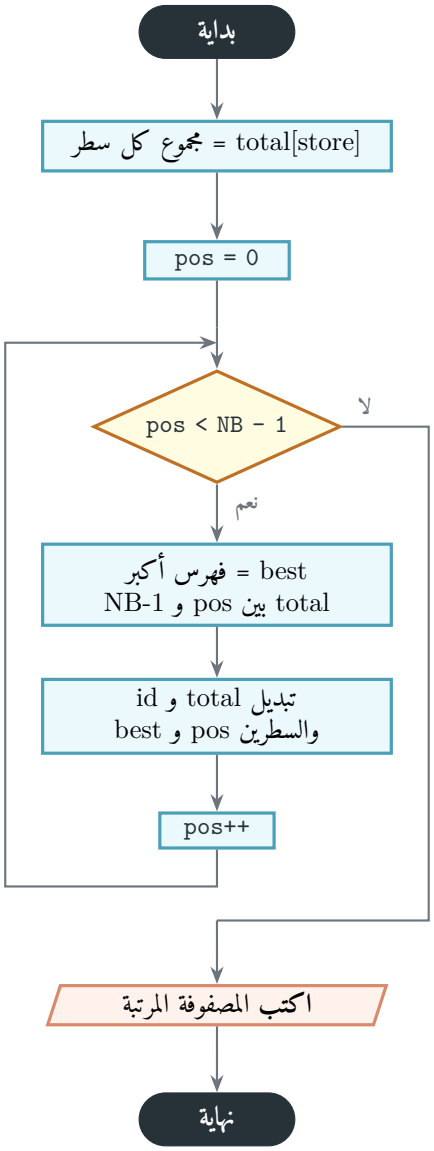
#define NB 4
#define NB_DAYS 3

int main(void) {
    int sales[NB][NB_DAYS] = {{80, 60, 75}, {200, 210, 190}, {120, 95, 140}, {10, 300, 20}};
    int id[NB] = {0, 1, 2, 3}, total[NB];
    int store, day, pos, scan, best, tmp;
    for (store = 0; store < NB; store++) {
        total[store] = 0;
        for (day = 0; day < NB_DAYS; day++)
            total[store] += sales[store][day];
    }
    for (pos = 0; pos < NB - 1; pos++) {          // ترتيب بالانتقاء
        best = pos;
        for (scan = pos + 1; scan < NB; scan++)
            if (total[scan] > total[best])
                best = scan;
        tmp = total[pos]; total[pos] = total[best]; total[best] = tmp;
        tmp = id[pos]; id[pos] = id[best]; id[best] = tmp;
        for (day = 0; day < NB_DAYS; day++) {     // تبديل السطرين كاملين
            tmp = sales[pos][day];
            sales[pos][day] = sales[best][day];
            sales[best][day] = tmp;
        }
    }
    for (store = 0; store < NB; store++) {
        printf("#%d store %d:", store + 1, id[store]);
        for (day = 0; day < NB_DAYS; day++)
            printf(" %3d", sales[store][day]);
        printf(" = %d\n", total[store]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
#1 store 1: 200 210 190 = 600
#2 store 2: 120  95 140 = 355
#3 store 3:  10 300  20 = 330
#4 store 0:  80  60  75 = 215
```

المخطط الإنسيابي (Organigramme)



تمرين 5.39 حجز فترات توصيل متتالية

تجارة إلكترونية ★★ متوسط

مصفوفة `slots[5][8]` تمثل فترات التوصيل لـ 5 أيام (8 فترات يومياً): 0 حرة و1 محجوزة. زبون يحتاج `need` فترات **متتالية** في نفس اليوم. اكتب برنامجًا يجد **أول** يوم وفترة بداية ممكنين ويحجزهما.

خطوات الحل

1) لكل يوم نعد الفترات الحرة المتتالية `run`، نعيده إلى 0 عند فترة محجوزة.
2) عندما يبلغ `run == need`: البداية `slot - need + 1`.
3) نحجز الفترات بوضع 1.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    int slots[5][8] = {{1, 1, 0, 0, 1, 0, 1, 1},
                       {0, 1, 1, 0, 0, 1, 0, 0},
                       {1, 0, 0, 0, 0, 1, 1, 0},
                       {0, 0, 0, 1, 1, 1, 1, 1},
                       {1, 1, 1, 1, 1, 1, 1, 1}};
    int need, day, slot, run, found = 0, start = 0, bookDay = 0, pos;
    scanf("%d", &need);
    for (day = 0; day < 5 && !found; day++) {
        run = 0;                            // طول السلسلة الحرة الحالية
        for (slot = 0; slot < 8 && !found; slot++) {
            if (slots[day][slot] == 0) run++;
            else run = 0;
            if (run == need) {
                found = 1;
                start = slot - need + 1;
                bookDay = day;
            }
        }
    }
    if (!found) {
        printf("No %d consecutive free slots\n", need);
    } else {
        for (pos = start; pos < start + need; pos++)
            slots[bookDay][pos] = 1;        // الحجز
        printf("Booked day %d, slots %d to %d:", bookDay + 1, start, start + need - 1);
        for (pos = 0; pos < 8; pos++)
            printf(" %d", slots[bookDay][pos]);
        printf("\n");
    }
    return 0;
}
```

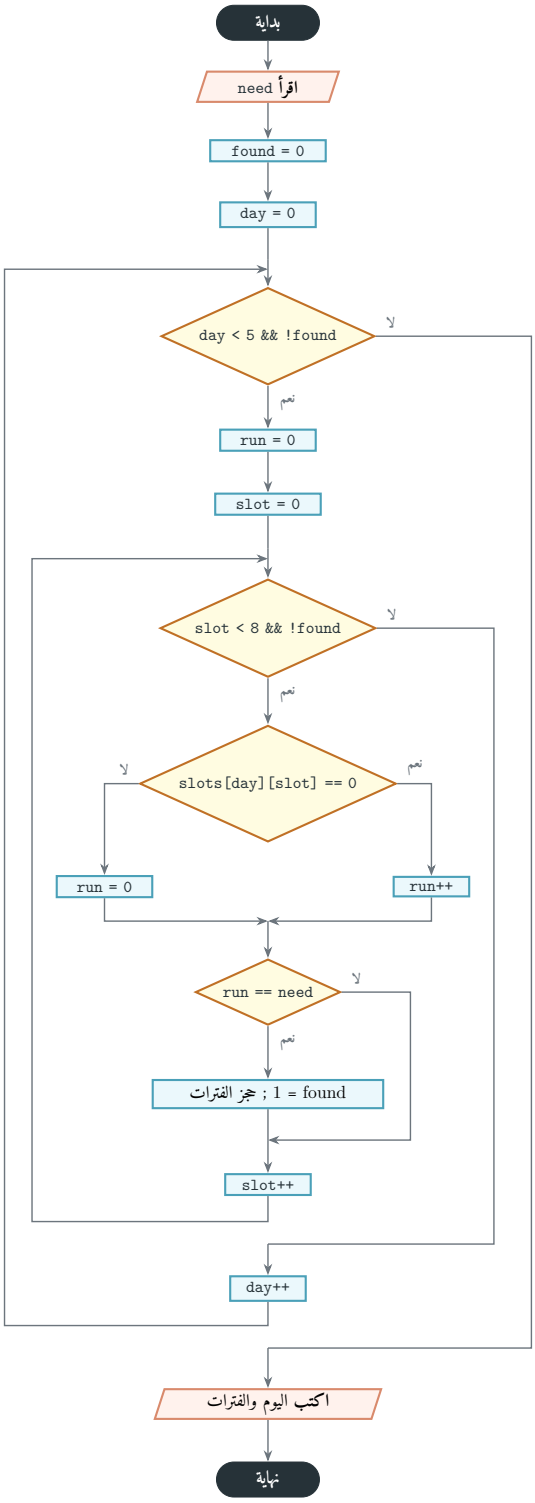
المدخلات

```
3
```

نتيجة التنفيذ

```
Booked day 3, slots 1 to 3: 1 1 1 1 0 1 1 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.40 قوة مصفوفة الإنتقال

علم البيانات

متوسط

احتمال انتقال زبون بين 3 صفحات من الموقع في خطوة واحدة معطى بالمصفوفة `step`. اكتب برنامجًا يحسب **مصفوفة الانتقال بعد خطوتين** `step × step`، ويتحقق أن مجموع كل سطر يبقى 1.

خطوات الحل

1) `twoSteps[from][to] = Σ step[from][mid] * step[mid][to]` : نمر بكل صفحة وسيطة.

2) مجموع كل سطر يجب أن يساوي 1 (توزيع احتمالي).

البرنامج بلغة C

```c
#include <stdio.h>

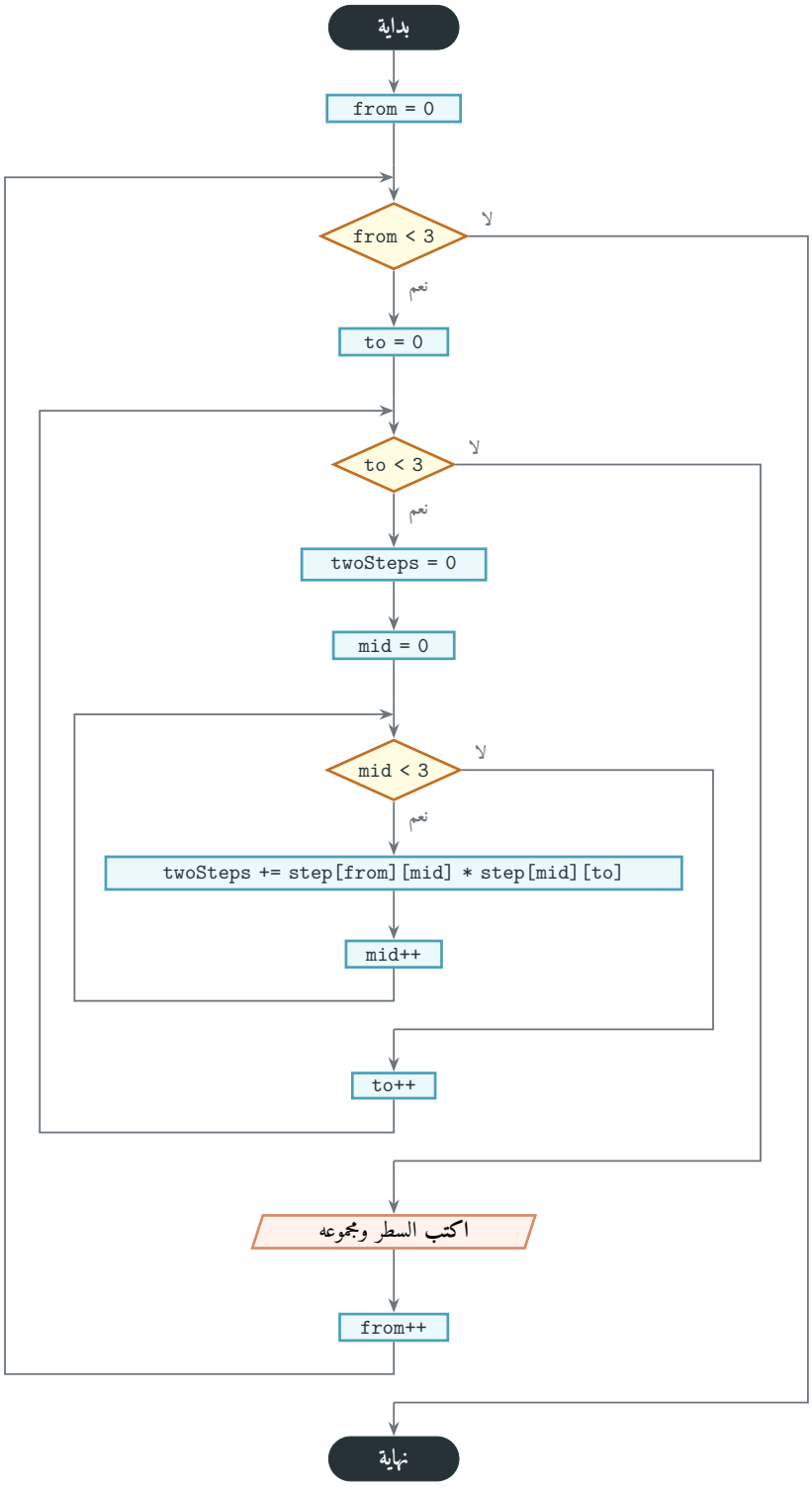
#define NB 3

int main(void) {
    double step[NB][NB] = {{0.2, 0.5, 0.3},        // الرئيسية
                           {0.1, 0.6, 0.3},        // المنتج
                           {0.4, 0.2, 0.4}};       // السلة
    double twoSteps[NB][NB], rowSum;
    int from, to, mid;
    for (from = 0; from < NB; from++) {
        rowSum = 0;
        for (to = 0; to < NB; to++) {
            twoSteps[from][to] = 0;
            for (mid = 0; mid < NB; mid++)         // كل الطرق عبر صفحة وسيطة
                twoSteps[from][to] += step[from][mid] * step[mid][to];
            rowSum += twoSteps[from][to];
            printf(" %.3f", twoSteps[from][to]);
        }
        printf("  (sum %.2f)\n", rowSum);
    }
    return 0;
}
```

نتيجة التنفيذ

```
0.210 0.460 0.330  (sum 1.00)
0.200 0.470 0.330  (sum 1.00)
0.260 0.400 0.340  (sum 1.00)
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 5.41 حل جملة معادلات بطريقة غاوس

علم البيانات ★★★ صعب

نموذج تسعير يقود إلى جملة من 4 معادلات خطية بـ 4 مجاهيل `A x = b`. اكتب برنامجًا يحلها بطريقة **الحذف لغاوس مع الارتكاز الجزئي** (اختيار أكبر محور بالقيمة المطلقة لتقليل أخطاء التقريب) ثم **التعويض التراجعي**.

خطوات الحل

1) نضع `b` كعمود إضافي في المصفوفة الموسعة `aug[4][5]`.
2) لكل عمود `pivotCol`: نختار السطر ذا أكبر `|aug[row][pivotCol]|` ونبدله مع سطر المحور.
3) نحذف المجهول من الأسطر السفلى: `aug[row] -= factor * aug[pivotCol]`.
4) التعويض التراجعي من آخر سطر إلى الأول.

البرنامج بلغة C

```c
#include <stdio.h>

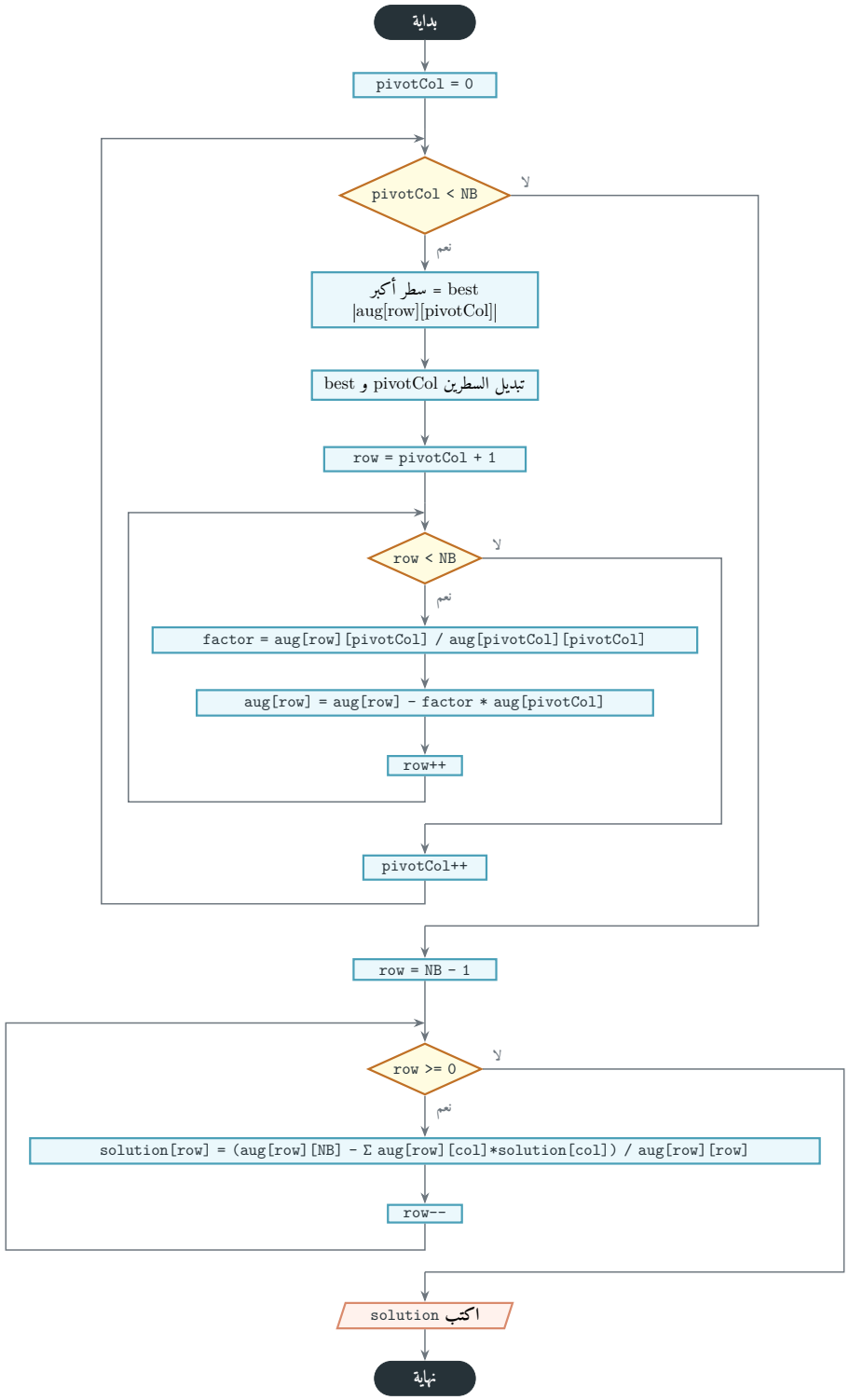
#define NB 4

int main(void) {
    double aug[NB][NB + 1] = {{ 2,  1, -1,  3,  18},     // المصفوفة الموسعة [A | b]
                              { 4,  1,  2, -1,  14},
                              {-2,  5,  1,  1,  20},
                              { 1, -1,  3,  2,  21}};
    double solution[NB], factor, tmp, bestVal, value, sum;
    int pivotCol, row, col, best;
    for (pivotCol = 0; pivotCol < NB; pivotCol++) {
        best = pivotCol;                          // الارتكاز الجزئي
        bestVal = aug[pivotCol][pivotCol] < 0 ? -aug[pivotCol][pivotCol] : aug[pivotCol][pivotCol];
        for (row = pivotCol + 1; row < NB; row++) {
            value = aug[row][pivotCol] < 0 ? -aug[row][pivotCol] : aug[row][pivotCol];
            if (value > bestVal) { bestVal = value; best = row; }
        }
        for (col = 0; col <= NB; col++) {         // تبديل السطرين
            tmp = aug[pivotCol][col]; aug[pivotCol][col] = aug[best][col]; aug[best][col] = tmp;
        }
        for (row = pivotCol + 1; row < NB; row++) {   // الحذف تحت المحور
            factor = aug[row][pivotCol] / aug[pivotCol][pivotCol];
            for (col = pivotCol; col <= NB; col++)
                aug[row][col] -= factor * aug[pivotCol][col];
        }
    }
    for (row = NB - 1; row >= 0; row--) {         // التعويض التراجعي
        sum = aug[row][NB];
        for (col = row + 1; col < NB; col++)
            sum -= aug[row][col] * solution[col];
        solution[row] = sum / aug[row][row];
    }
    for (row = 0; row < NB; row++)
        printf("x%d = %.4f\n", row + 1, solution[row]);
    return 0;
}
```

نتيجة التنفيذ

```
x1 = 2.0000
x2 = 3.0000
x3 = 4.0000
x4 = 5.0000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.42 مقلوب مصفوفة بطريقة غاوس-جوردان

علم البيانات

صعب ★★★

اكتب برنامجًا يحسب **مقلوب** مصفوفة 3×3 بطريقة **غاوس-جوردان**: نلصق مصفوفة الوحدة على يمين المصفوفة، ونحول الجزء الأيسر إلى الوحدة؛ الجزء الأيمن يصبح المقلوب. نتحقق بحساب $A \times A^{-1}$.

خطوات الحل

1) `work[3][6]`: الأعمدة 0..2 هي `A` والأعمدة 3..5 هي الوحدة.
2) لكل محور: نقسم سطره على المحور ليصبح 1، ثم نصفر العمود في **كل** الأسطر الأخرى (فوق وتحت).
3) إذا كان المحور 0 نبحث عن سطر نبدله معه؛ إذا لم نجد فالمصفوفة غير قابلة للقلب.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 3

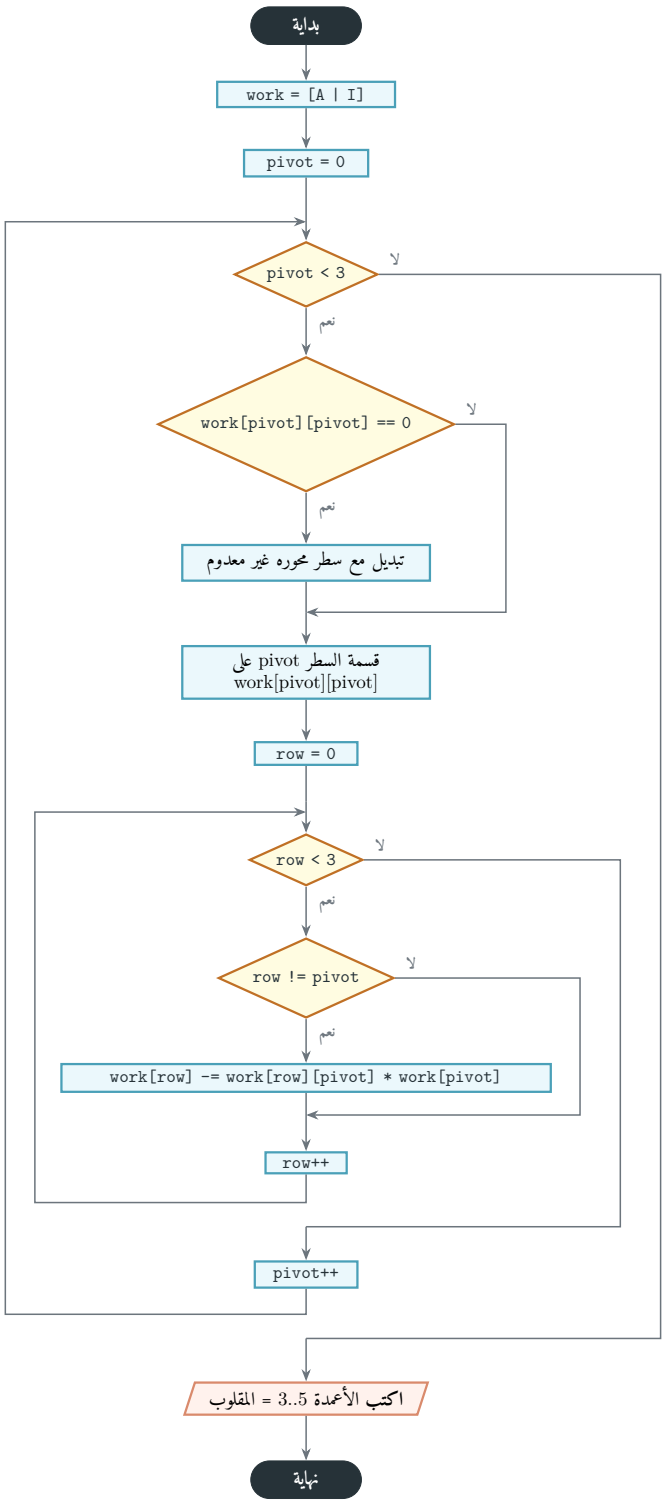
int main(void) {
    double matrix[NB][NB] = {{2, 1, 1}, {1, 3, 2}, {1, 0, 0}};
    double work[NB][2 * NB], factor, tmp, check;
    int pivot, row, col, swapRow, inner;
    for (row = 0; row < NB; row++)
        for (col = 0; col < 2 * NB; col++)
            work[row][col] = (col < NB) ? matrix[row][col] : (col - NB == row);  // [A | I]
    for (pivot = 0; pivot < NB; pivot++) {
        if (work[pivot][pivot] == 0) {            // محور معدوم: نبحث عن سطر آخر
            for (swapRow = pivot + 1; swapRow < NB && work[swapRow][pivot] == 0; swapRow++)
                ;
            if (swapRow == NB) { printf("Matrix is not invertible\n"); return 0; }
            for (col = 0; col < 2 * NB; col++) {
                tmp = work[pivot][col]; work[pivot][col] = work[swapRow][col]; work[swapRow][col] = tmp;
            }
        }
        factor = work[pivot][pivot];
        for (col = 0; col < 2 * NB; col++)
            work[pivot][col] /= factor;           // المحور يصبح 1
        for (row = 0; row < NB; row++)
            if (row != pivot) {                   // نصفر العمود في باقي الأسطر
                factor = work[row][pivot];
                for (col = 0; col < 2 * NB; col++)
                    work[row][col] -= factor * work[pivot][col];
            }
    }
    printf("Inverse:\n");
    for (row = 0; row < NB; row++) {
        for (col = NB; col < 2 * NB; col++)
            printf(" %7.3f", work[row][col]);
        printf("\n");
    }
    printf("Check A x inv (diagonal):");
    for (row = 0; row < NB; row++) {
        check = 0;
        for (inner = 0; inner < NB; inner++)
            check += matrix[row][inner] * work[inner][NB + row];
        printf(" %.3f", check);
    }
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Inverse:
   0.000   0.000   1.000
  -2.000   1.000   3.000
   3.000  -1.000  -5.000
```

```
Check A x inv (diagonal): 1.000 1.000 1.000
```

المخطط الإنسيابي (Organigramme)



تمرين 5.43 ترتيب صفحات المنتجات بخوارزمية PageRank

علم البيانات ★★★ صعب

صفحات 5 منتجات مرتبطة بروابط "منتجات مشابهة": `links[from][to] = 1`. اكتب برنامجًا يحسب **PageRank** لكل صفحة بطريقة **القوة المتكررة**:

$$rank_new[to] = (1 - d)/N + d \times \Sigma\ rank[from] / outDegree[from]$$

مع $d = 0.85$، حتى يصبح التغير الكلي أقل من $1e\text{-}6$.

خطوات الحل

1) $outDegree[from]$ عدد الروابط الخارجة من كل صفحة.
2) نبدأ بـ $rank = 1/N$ لكل صفحة.
3) في كل تكرار: كل صفحة توزع رتبتها بالتساوي على الصفحات التي تشير إليها.
4) تتوقف عندما يصبح $\Sigma\ |rank_new - rank|$ صغيرًا.

البرنامج بلغة C

```c
#include <stdio.h>

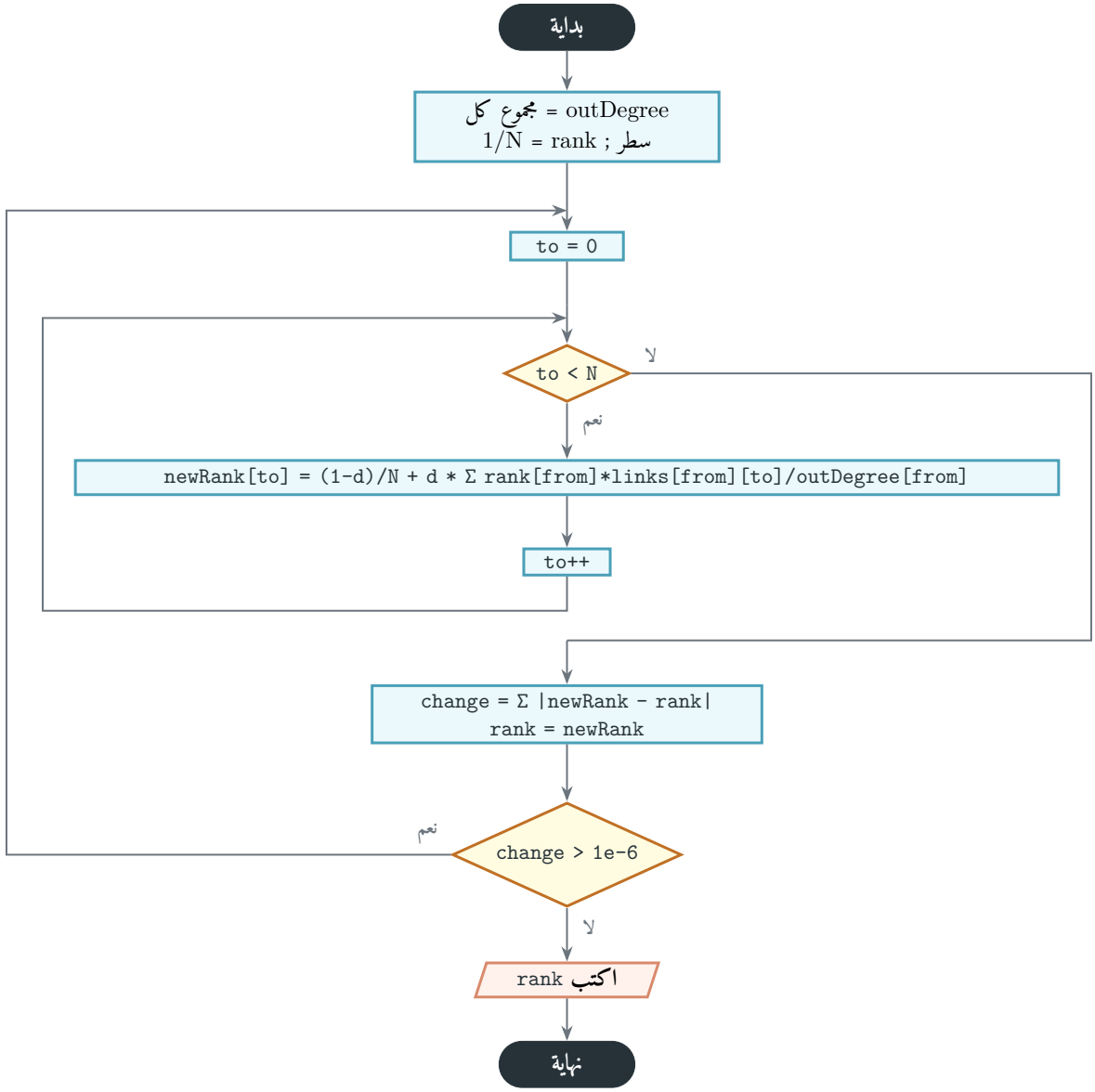
#define NB 5
#define DAMPING 0.85

int main(void) {
    int links[NB][NB] = {{0, 1, 1, 0, 0},
                         {0, 0, 1, 0, 0},
                         {1, 0, 0, 1, 0},
                         {0, 0, 1, 0, 1},
                         {1, 0, 1, 0, 0}};
    double rank[NB], newRank[NB], change, gap;
    int outDegree[NB], from, to, iteration = 0;
    for (from = 0; from < NB; from++) {
        outDegree[from] = 0;
        for (to = 0; to < NB; to++)
            outDegree[from] += links[from][to];
        rank[from] = 1.0 / NB;                    // توزيع ابتدائي متساوٍ
    }
    do {
        change = 0;
        for (to = 0; to < NB; to++) {
            newRank[to] = (1 - DAMPING) / NB;     // القفز العشوائي
            for (from = 0; from < NB; from++)
                if (links[from][to])
                    newRank[to] += DAMPING * rank[from] / outDegree[from];
        }
        for (to = 0; to < NB; to++) {
            gap = newRank[to] - rank[to];
            change += (gap < 0) ? -gap : gap;
            rank[to] = newRank[to];
        }
        iteration++;
    } while (change > 1e-6);
    printf("Converged after %d iterations\n", iteration);
    for (to = 0; to < NB; to++)
        printf("Page %d: %.4f\n", to, rank[to]);
    return 0;
}
```

نتيجة التنفيذ

```
Converged after 25 iterations
Page 0: 0.2273
Page 1: 0.1266
Page 2: 0.3571
Page 3: 0.1818
Page 4: 0.1072
```

المخطط الإنسيابي (Organigramme)



تمرين 5.44 أقصر المسافات بين كل المستودعات (فلويد-وارشال)

تجارة إلكترونية

صعب

مصفوفة `dist[5][5]` تعطي طول الطريق المباشر بين المستودعات (`INF` إذا لم يوجد طريق مباشر). اكتب برنامجًا يحسب **أقصر مسافة بين كل زوج** بخوارزمية **فلويد-وارشال**: لكل مستودع وسيط `via`، نتحقق هل المرور عبره أقصر.

خطوات الحل

1) ثلاث حلقات متداخلة: الوسيط `via` في الخارج.
2) `if (dist[from][via] + dist[via][to] < dist[from][to])` نحدّث.
3) نتجنب جمع `INF` مع قيمة.

البرنامج بلغة C

```c
#include <stdio.h>

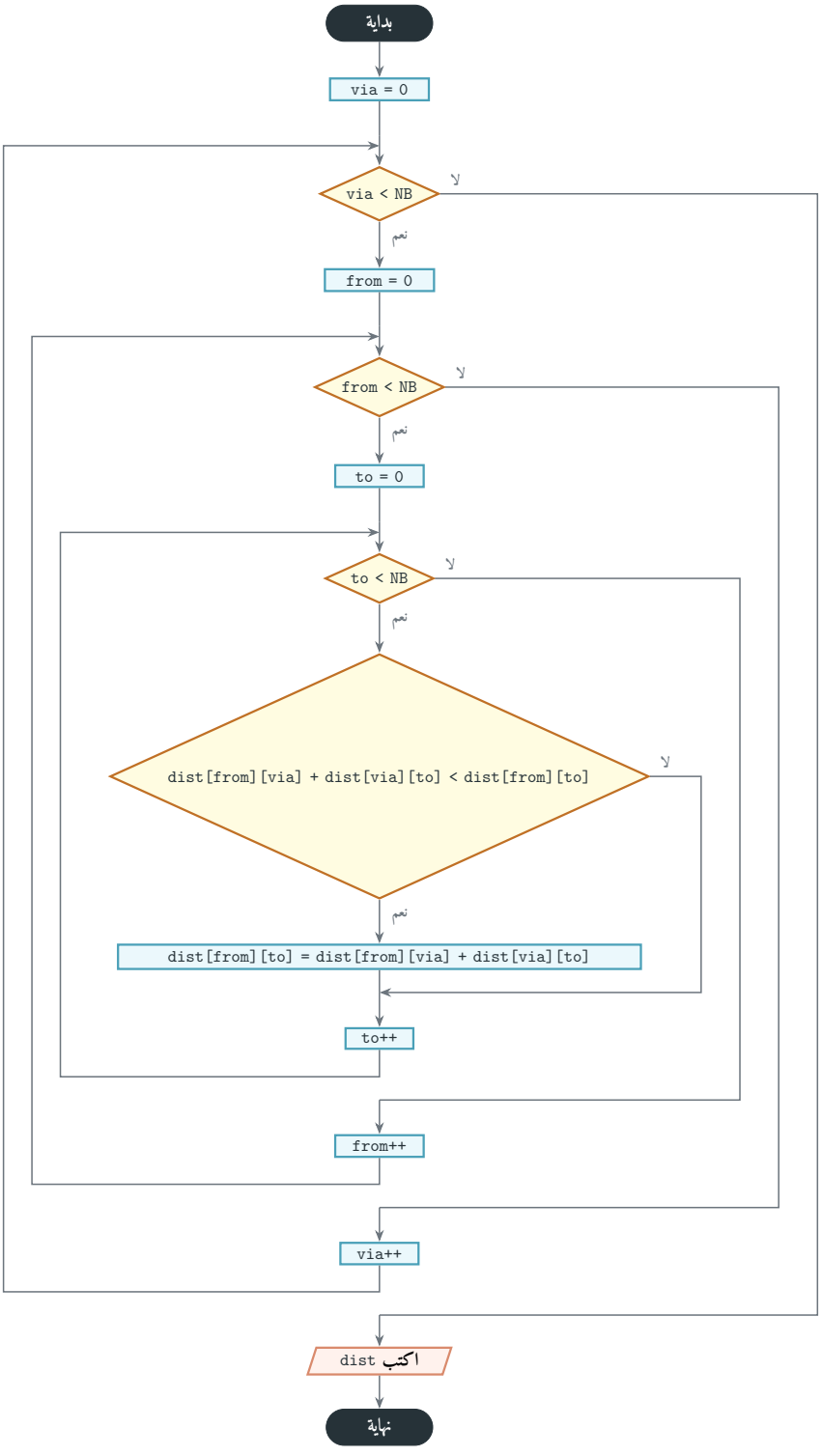
#define NB 5
#define INF 1000000

int main(void) {
    int dist[NB][NB] = {{  0, 120, INF, 300, INF},
                        {120,   0,  90, INF, 400},
                        {INF,  90,   0,  60, 150},
                        {300, INF,  60,   0, 110},
                        {INF, 400, 150, 110,   0}};
    int via, from, to;
    for (via = 0; via < NB; via++)                // المستودع الوسيط
        for (from = 0; from < NB; from++)
            for (to = 0; to < NB; to++)
                if (dist[from][via] < INF && dist[via][to] < INF
                    && dist[from][via] + dist[via][to] < dist[from][to])
                    dist[from][to] = dist[from][via] + dist[via][to];   // طريق أقصر
    for (from = 0; from < NB; from++) {
        for (to = 0; to < NB; to++)
            printf("%5d", dist[from][to]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
    0  120  210  270  360
  120    0   90  150  240
  210   90    0   60  150
  270  150   60    0  110
  360  240  150  110    0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.45 أقصر طريق للتوصيل (ديكسترا)

تجارة إلكترونية

صعب

اكتب برنامجًا يحسب أقصر طريق من المستودع (العقدة 0) إلى كل حي في المدينة بخوارزمية **ديكسترا** على مصفوفة المجاورة (0 = لا طريق)، ويعيد بناء المسار إلى حي مقروء باستعمال جدول `previous`.

خطوات الحل

1) `best[node] = INF` إلا المصدر 0، و`done[node] = 0`.
2) في كل دورة: نختار العقدة غير المعالجة ذات أصغر `best`، ونعلمها معالجة.
3) نحدّث جيرانها: إذا كان `best[node] + weight < best[next]`.
4) المسار: نتبع `previous` من الوجهة إلى المصدر.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6
#define INF 1000000

int main(void) {
    int road[NB][NB] = {{0, 7, 9, 0, 0, 14},
                        {7, 0, 10, 15, 0, 0},
                        {9, 10, 0, 11, 0, 2},
                        {0, 15, 11, 0, 6, 0},
                        {0, 0, 0, 6, 0, 9},
                        {14, 0, 2, 0, 9, 0}};
    int best[NB], done[NB], previous[NB], path[NB];
    int node, next, round, target, length = 0;
    for (node = 0; node < NB; node++) {
        best[node] = INF; done[node] = 0; previous[node] = -1;
    }
    best[0] = 0;                                  // المستودع
    for (round = 0; round < NB; round++) {
        node = -1;                                // أقرب عقدة غير معالجة
        for (next = 0; next < NB; next++)
            if (!done[next] && (node < 0 || best[next] < best[node]))
                node = next;
        done[node] = 1;                           // مسافتها نهائية الآن
        for (next = 0; next < NB; next++)
            if (road[node][next] && best[node] + road[node][next] < best[next]) {
                best[next] = best[node] + road[node][next];
                previous[next] = node;
            }
    }
    scanf("%d", &target);
    for (node = target; node != -1; node = previous[node])
        path[length++] = node;                    // المسار معكوسًا
    printf("Distance to %d = %d min\nPath:", target, best[target]);
    while (length > 0)
        printf(" %d", path[--length]);
    printf("\n");
    return 0;
}
```

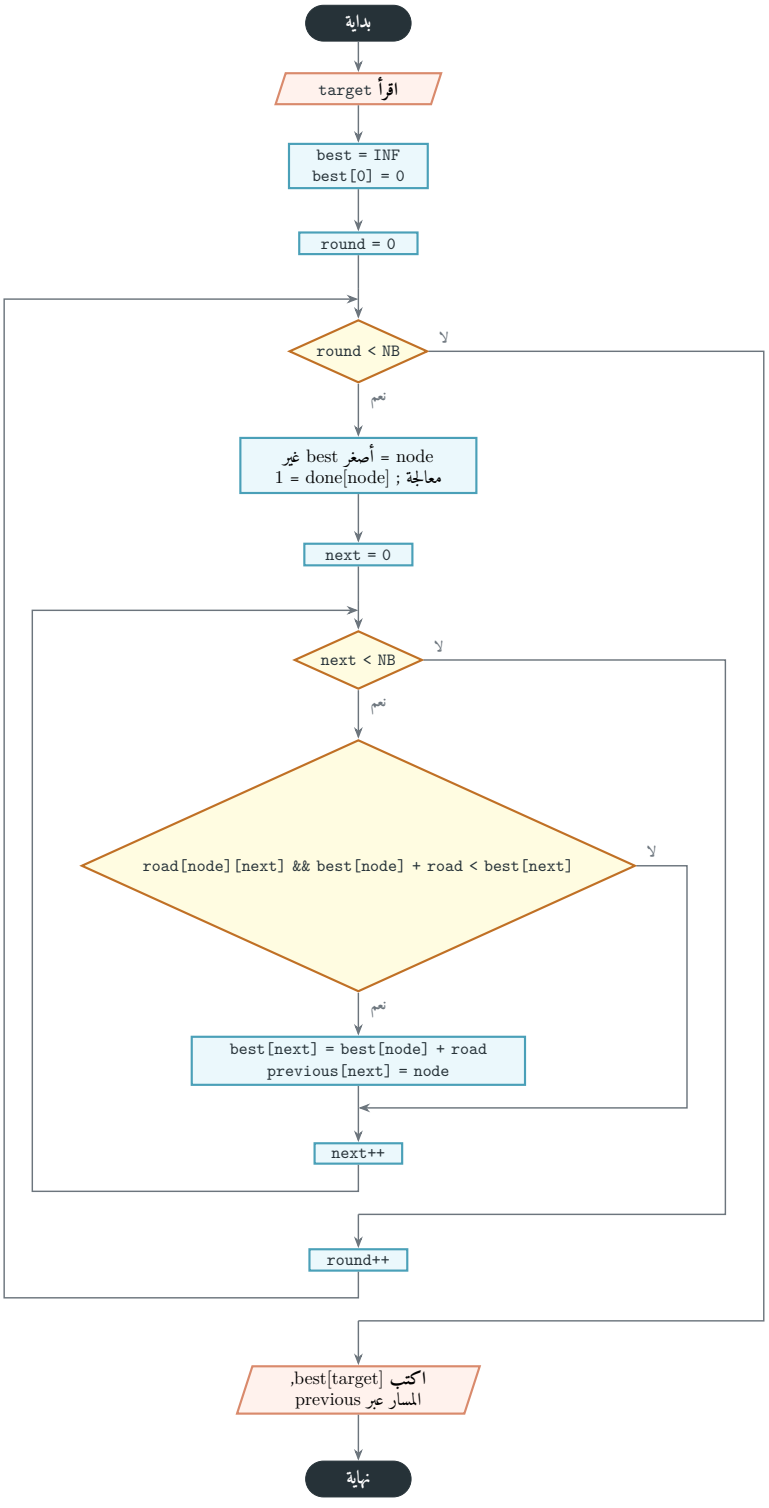
المدخلات

```
5
```

نتيجة التنفيذ

```
Distance to 5 = 11 min
Path: 0 2 5
```

المخطط الإنسيابي (Organigramme)



تمرين 5.46 الإنحدار الخطي المتعدد بالمعادلات الطبيعية

علم البيانات ★★★ صعب

نريد التنبؤ بالمبيعات `y` من ميزانية الإعلان `x1` وعدد التخفيضات `x2`: `y = b0 + b1 x1 + b2 x2`. اكتب برنامجًا يبني المعادلات الطبيعية `(X X) = X y` (مصفوفة 3×3) ثم يحلها بالحذف لغاوس، ويحسب معامل التحديد R^2.

خطوات الحل

1) مصفوفة التصميم X: عمود من 1 ثم x1 ثم x2.
2) xtx[a][b] = Σ X[row][a] * X[row][b] و xty[a] = Σ X[row][a] * sales[row].
3) حل الجملة 3×3 بالحذف ثم التعويض.
4) R^2 = 1 - SSres / SStot.

البرنامج بلغة C

```c
#include <stdio.h>

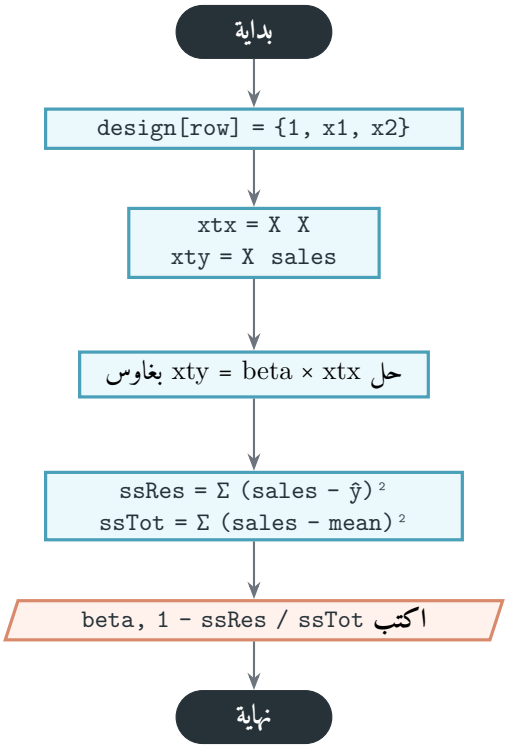
#define NB 8
#define NB_COEF 3

int main(void) {
    double x1[NB] = {1, 2, 3, 4, 5, 6, 7, 8};              // الإعلان
    double x2[NB] = {0, 1, 0, 2, 1, 3, 2, 3};              // التخفيضات
    double sales[NB]  = {14, 22, 21, 35, 33, 47, 44, 52};     // المبيعات
    double design[NB][NB_COEF], aug[NB_COEF][NB_COEF + 1], beta[NB_COEF];
    double factor, sum, mean = 0, ssRes = 0, ssTot = 0, predicted;
    int row, colA, colB, pivot;
    for (row = 0; row < NB; row++) {
        design[row][0] = 1; design[row][1] = x1[row]; design[row][2] = x2[row];
        mean += sales[row] / NB;
    }
    for (colA = 0; colA < NB_COEF; colA++) {      // المعادلات الطبيعية
        for (colB = 0; colB < NB_COEF; colB++) {
            aug[colA][colB] = 0;
            for (row = 0; row < NB; row++)
                aug[colA][colB] += design[row][colA] * design[row][colB];
        }
        aug[colA][NB_COEF] = 0;
        for (row = 0; row < NB; row++)
            aug[colA][NB_COEF] += design[row][colA] * sales[row];
    }
    for (pivot = 0; pivot < NB_COEF; pivot++)     // الحذف (XtX موجبة: لا ارتكاز)
        for (row = pivot + 1; row < NB_COEF; row++) {
            factor = aug[row][pivot] / aug[pivot][pivot];
            for (colB = pivot; colB <= NB_COEF; colB++)
                aug[row][colB] -= factor * aug[pivot][colB];
        }
    for (row = NB_COEF - 1; row >= 0; row--) {    // التعويض التراجعي
        sum = aug[row][NB_COEF];
        for (colB = row + 1; colB < NB_COEF; colB++)
            sum -= aug[row][colB] * beta[colB];
        beta[row] = sum / aug[row][row];
    }
    for (row = 0; row < NB; row++) {
        predicted = beta[0] + beta[1] * x1[row] + beta[2] * x2[row];
        ssRes += (sales[row] - predicted) * (sales[row] - predicted);
        ssTot += (sales[row] - mean) * (sales[row] - mean);
    }
    printf("sales = %.3f + %.3f*ads + %.3f*promos\n", beta[0], beta[1], beta[2]);
    printf("R^2 = %.4f\n", 1 - ssRes / ssTot);
    return 0;
}
```

نتيجة التنفيذ

```
sales = 10.989 + 3.237*ads + 5.298*promos
R^2 = 0.9982
```

المخطط الإنسيابي (Organigramme)



تمرين 5.47 تجميع الزبائن في بعدين

علم البيانات صعب

مصفوفة points[12][2]: لكل زبون (التكرار الشهري، متوسط السلة). اكتب برنامجًا يطبق **K-Means** بمجموعتين في بعدين: المراكز مصفوفة centers[2][2] تبدأ بأول زبونين، والمسافة مربع إقليدي. نعرض المراكز النهائية وحجم كل مجموعة ومجموع المربعات داخل المجموعات.

خطوات الحل

1) إسناد كل زبون لأقرب مركز في بعدين.
2) إعادة الحساب: متوسط كل إحداثية لأعضاء المجموعة.
3) التوقف عندما لا يتغير أي إسناد.
4) inertia = Σ مربع المسافة إلى المركز.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12
#define NB_K 2

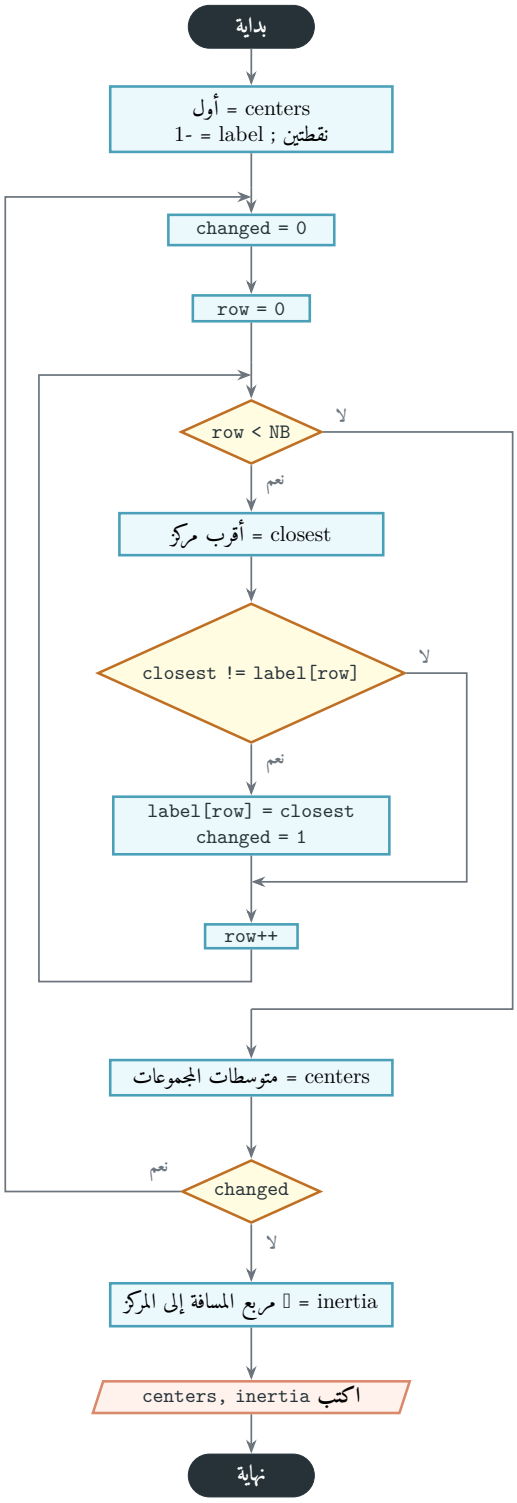
int main(void) {
    double points[NB][2] = {{1, 2.0}, {2, 1.5}, {1, 1.0}, {3, 2.5}, {2, 2.2}, {9, 8.0},
                            {8, 7.5}, {10, 9.0}, {9, 7.0}, {7, 8.5}, {3, 1.8}, {8, 9.2}};
    double centers[NB_K][2], sum[NB_K][2], dx, dy, dist, bestDist, inertia = 0;
    int label[NB], count[NB_K], row, cluster, closest, changed, axis;
    for (cluster = 0; cluster < NB_K; cluster++)
        for (axis = 0; axis < 2; axis++)
```

```
            centers[cluster][axis] = points[cluster][axis];    // أول نقطتين
    for (row = 0; row < NB; row++)
        label[row] = -1;
    do {
        changed = 0;
        for (row = 0; row < NB; row++) {
            closest = 0; bestDist = -1;
            for (cluster = 0; cluster < NB_K; cluster++) {
                dx = points[row][0] - centers[cluster][0];
                dy = points[row][1] - centers[cluster][1];
                dist = dx * dx + dy * dy;
                if (bestDist < 0 || dist < bestDist) { bestDist = dist; closest = cluster; }
            }
            if (closest != label[row]) { label[row] = closest; changed = 1; }
        }
        for (cluster = 0; cluster < NB_K; cluster++) {
            sum[cluster][0] = sum[cluster][1] = 0; count[cluster] = 0;
        }
        for (row = 0; row < NB; row++) {          // مجموع الإحداثيات لكل مجموعة
            sum[label[row]][0] += points[row][0];
            sum[label[row]][1] += points[row][1];
            count[label[row]]++;
        }
        for (cluster = 0; cluster < NB_K; cluster++)
            if (count[cluster] > 0)
                for (axis = 0; axis < 2; axis++)
                    centers[cluster][axis] = sum[cluster][axis] / count[cluster];
    } while (changed);
    for (row = 0; row < NB; row++) {
        dx = points[row][0] - centers[label[row]][0];
        dy = points[row][1] - centers[label[row]][1];
        inertia += dx * dx + dy * dy;
    }
    for (cluster = 0; cluster < NB_K; cluster++)
        printf("Cluster %d: center (%.2f, %.2f), %d customers\n", cluster,
               centers[cluster][0], centers[cluster][1], count[cluster]);
    printf("Inertia = %.3f\n", inertia);
    return 0;
}
```

نتيجة التنفيذ

```
Cluster 0: center (2.00, 1.83), 6 customers
Cluster 1: center (8.50, 8.20), 6 customers
Inertia = 14.613
```

المخطط الإنسيابي (Organigramme)



تمرين 5.48 التنبؤ بتقييم مفقود (الترشيح التعاوني)

علم البيانات ★★★ صعب

في مصفوفة التقييمات (زبائن × منتجات) القيمة 0 تعني "لم يقيّم". نريد التنبؤ بتقييم الزبون `target` للمنتج `item` بطريقة **الترشيح التعاوني**: نحسب **تشابه جيب التمام** بينه وبين كل زبون آخر قيّم المنتج (على المنتجات المشتركة فقط)، ثم المتوسط الموزون بالتشابه.

خطوات الحل

1) لكل زبون آخر other قيّم item: نحسب dot، وnormA، وnormB على المنتجات التي قيّمها الاثنان.
2) similarity = dot / normA) * normB) (الجذر بطريقة نيوتن).
3) prediction = Σ similarity × rating[other][item] / Σ similarity

.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_USERS 5
#define NB_ITEMS 5

int main(void) {
    int ratings[NB_USERS][NB_ITEMS] = {{5, 4, 0, 1, 0},
                                       {4, 5, 4, 1, 1},
                                       {1, 1, 2, 5, 4},
                                       {5, 5, 5, 0, 1},
                                       {2, 0, 1, 4, 5}};
    int target, item, other, col, step;
    double dot, normA, normB, product, root, similarity;
    double weightedSum = 0, simSum = 0;
    scanf("%d %d", &target, &item);
    for (other = 0; other < NB_USERS; other++) {
        if (other == target || ratings[other][item] == 0)
            continue;                               // لم يقيّم المنتج المطلوب
        dot = normA = normB = 0;
        for (col = 0; col < NB_ITEMS; col++)
            if (ratings[target][col] > 0 && ratings[other][col] > 0) {   // مشترك
                dot += ratings[target][col] * ratings[other][col];
                normA += ratings[target][col] * ratings[target][col];
                normB += ratings[other][col] * ratings[other][col];
            }
        if (dot == 0)
            continue;
        product = normA * normB;
        root = product;
        for (step = 0; step < 60; step++)           // الجذر بطريقة نيوتن
            root = (root + product / root) / 2;
        similarity = dot / root;
        printf("User %d: similarity %.3f, rating %d\n", other, similarity, ratings[other][item]);
        weightedSum += similarity * ratings[other][item];
        simSum += similarity;
    }
    if (simSum > 0)
        printf("Predicted rating of user %d for item %d = %.2f\n", target, item, weightedSum / simSum);
    else
        printf("Not enough data\n");
    return 0;
}
```

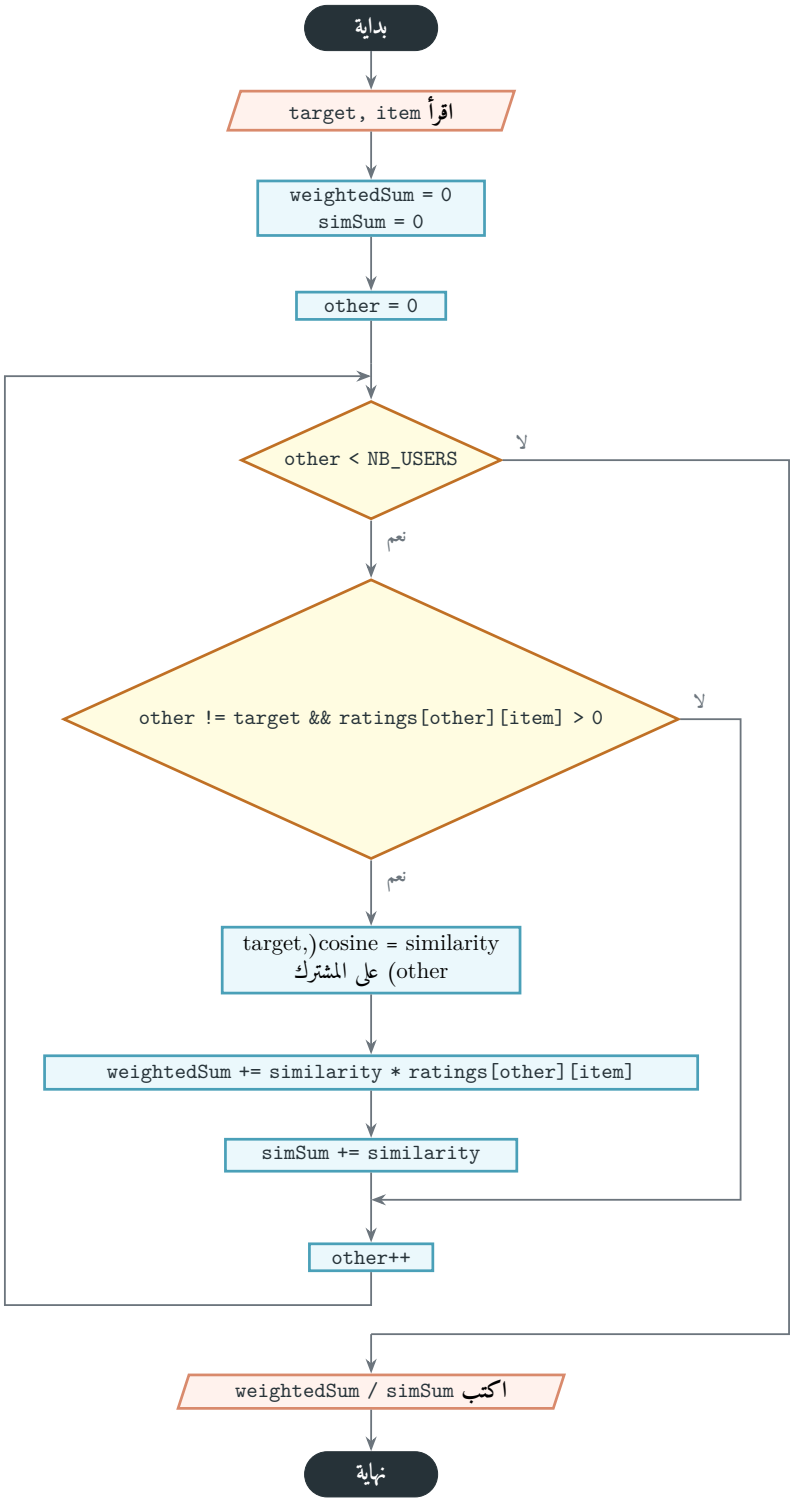
المدخلات

```
0 2
```

نتيجة التنفيذ

```
User 1: similarity 0.976, rating 4
User 2: similarity 0.416, rating 2
User 3: similarity 0.994, rating 5
User 4: similarity 0.614, rating 1
Predicted rating of user 0 for item 2 = 3.44
```

المخطط الإنسيابي (Organigramme)



تمرين 5.49 مناطق التوصيل المتصلة (ملء المناطق) تجارة إلكترونية صعب

خريطة المدينة شبكة zone[6][8]: 1 حي مخدوم و0 غير مخدوم. نريد عد **المناطق المتصلة** (أحياء مخدومة متجاورة أفقيًا أو عموديًا) وحجم كل منطقة، لتعيين سائق لكل منطقة. اكتب البرنامج بخوارزمية **ملء المنطقة** باستعمال **مكدس يدوي** (دون عودية).

خطوات الحل

1) `region[row][col]` رقم المنطقة لكل خانة (0 = لم تُزر).
2) لكل خانة مخدومة لم تُزر: نبدأ منطقة جديدة وندفعها في المكدس.
3) ما دام المكدس غير فارغ: نسحب خانة ونعلّم جيرانها الأربعة المخدومين غير المزارين وندفعهم.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB_ROWS 6
#define NB_COLS 8

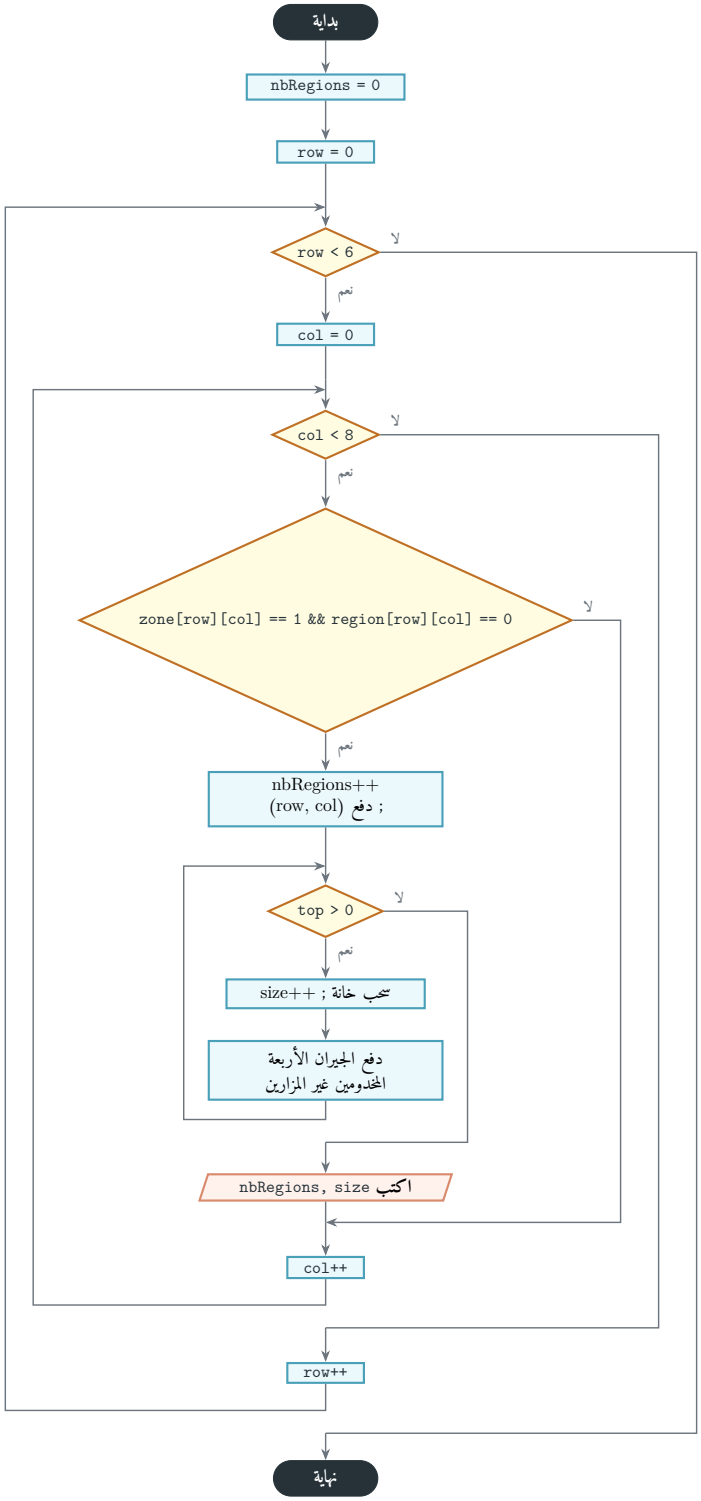
int main(void) {
    int zone[NB_ROWS][NB_COLS] = {{1, 1, 0, 0, 0, 1, 1, 0},
                                  {1, 0, 0, 1, 0, 0, 1, 0},
                                  {0, 0, 1, 1, 1, 0, 0, 0},
                                  {1, 0, 0, 1, 0, 0, 1, 1},
                                  {1, 1, 0, 0, 0, 0, 1, 0},
                                  {0, 1, 0, 1, 1, 0, 0, 0}};
    int region[NB_ROWS][NB_COLS] = {{0}};
    int stackRow[NB_ROWS * NB_COLS], stackCol[NB_ROWS * NB_COLS];
    int moveRow[4] = {-1, 1, 0, 0}, moveCol[4] = {0, 0, -1, 1};
    int row, col, top, curRow, curCol, nextRow, nextCol, dir, size, nbRegions = 0;
    for (row = 0; row < NB_ROWS; row++)
        for (col = 0; col < NB_COLS; col++) {
            if (zone[row][col] != 1 || region[row][col] != 0)
                continue;
            nbRegions++;                          // منطقة جديدة
            region[row][col] = nbRegions;
            top = 0; size = 0;
            stackRow[top] = row; stackCol[top] = col; top++;
            while (top > 0) {
                top--;
                curRow = stackRow[top]; curCol = stackCol[top];
                size++;
                for (dir = 0; dir < 4; dir++) {   // الجيران الأربعة
                    nextRow = curRow + moveRow[dir];
                    nextCol = curCol + moveCol[dir];
                    if (nextRow >= 0 && nextRow < NB_ROWS && nextCol >= 0 && nextCol < NB_COLS
                        && zone[nextRow][nextCol] == 1 && region[nextRow][nextCol] == 0) {
                        region[nextRow][nextCol] = nbRegions;   // نعلّم قبل الدفع
                        stackRow[top] = nextRow; stackCol[top] = nextCol; top++;
                    }
                }
            }
            printf("Region %d: %d block(s)\n", nbRegions, size);
        }
    for (row = 0; row < NB_ROWS; row++) {
        for (col = 0; col < NB_COLS; col++)
            printf("%2d", region[row][col]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Region 1: 3 block(s)
Region 2: 3 block(s)
Region 3: 5 block(s)
Region 4: 4 block(s)
Region 5: 3 block(s)
Region 6: 2 block(s)
 1 1 0 0 0 2 2 0
```

```
1 0 0 3 0 0 2 0
0 0 3 3 3 0 0 0
4 0 0 3 0 0 5 5
4 4 0 0 0 0 5 0
0 4 0 6 6 0 0 0
```

المخطط الإنسيابي (Organigramme)



تمرين 5.50 أطول مسار تصفح مشترك

علم البيانات صعب ★★★

مسارا تصفح لزبونين ممثلان بجدولي أرقام صفحات. اكتب برنامجًا يجد **أطول متتالية مشتركة** (LCS) بينهما بالبرمجة الديناميكية: `table[i][j]` طول LCS لأول `i` صفحة من الأول وأول `j` من الثاني، ثم يعيد بناء المتتالية.

خطوات الحل

1) إذا تساوت الصفحتان:

```
table[posA][posB] = table[posA-1][posB-1] + 1
```

.

2) وإلا: الأكبر بين `table[posA-1][posB]` و`table[posA][posB-1]`.

3) إعادة البناء من الخانة الأخيرة نحو `[0][0]`.

البرنامج بلغة C

```c
#include <stdio.h>

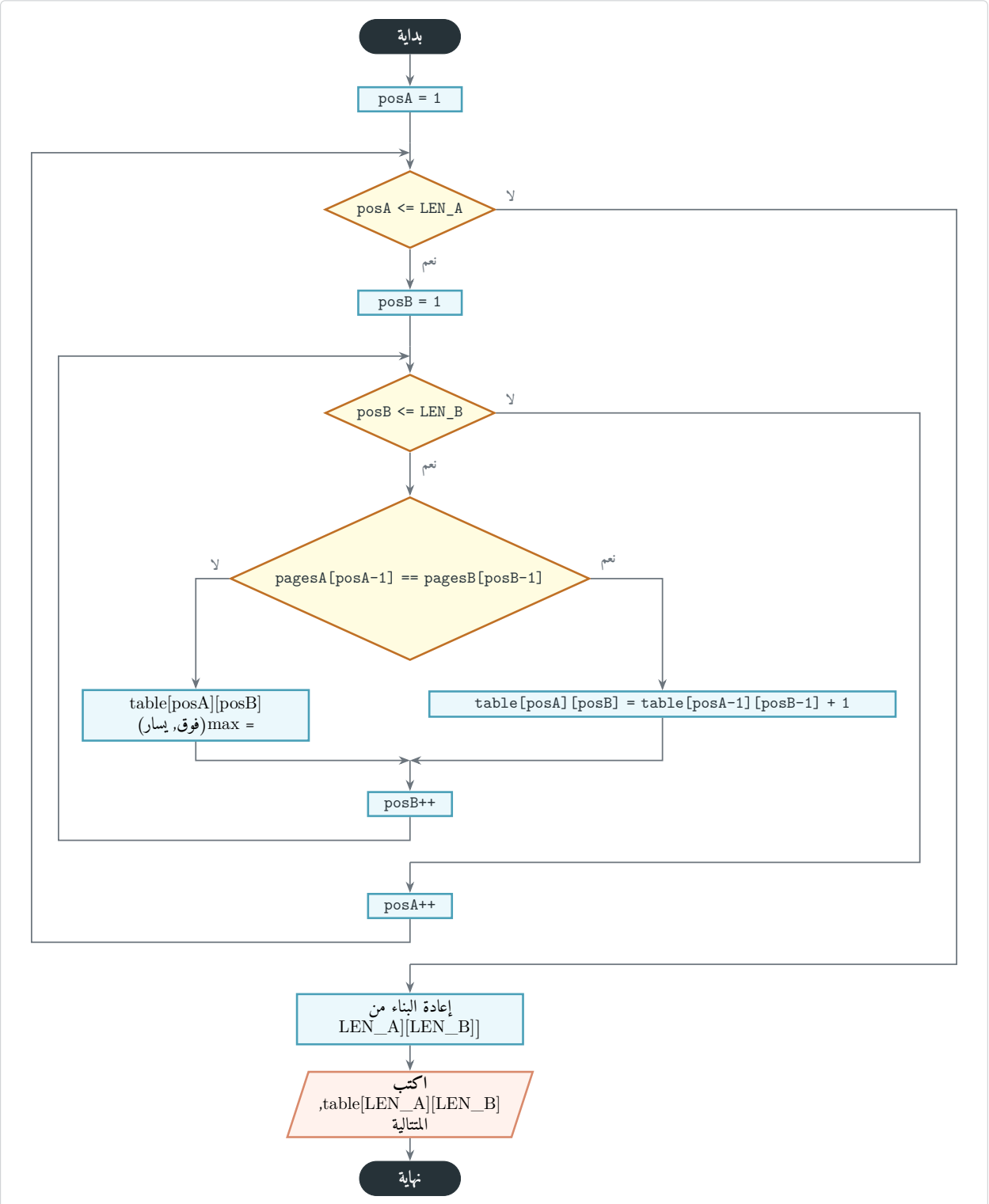
#define LEN_A 8
#define LEN_B 7

int main(void) {
    int pagesA[LEN_A] = {1, 4, 2, 7, 3, 9, 5, 6};      // الصفحة 1 = الرئيسية
    int pagesB[LEN_B] = {1, 2, 4, 7, 9, 6, 5};
    int table[LEN_A + 1][LEN_B + 1], common[LEN_B];
    int posA, posB, length;
    for (posA = 0; posA <= LEN_A; posA++)
        for (posB = 0; posB <= LEN_B; posB++) {
            if (posA == 0 || posB == 0)
                table[posA][posB] = 0;                 // متتالية فارغة
            else if (pagesA[posA - 1] == pagesB[posB - 1])
                table[posA][posB] = table[posA - 1][posB - 1] + 1;
            else if (table[posA - 1][posB] >= table[posA][posB - 1])
                table[posA][posB] = table[posA - 1][posB];
            else
                table[posA][posB] = table[posA][posB - 1];
        }
    length = table[LEN_A][LEN_B];
    posA = LEN_A; posB = LEN_B;
    while (posA > 0 && posB > 0) {                     // إعادة البناء
        if (pagesA[posA - 1] == pagesB[posB - 1]) {
            common[table[posA][posB] - 1] = pagesA[posA - 1];
            posA--; posB--;
        } else if (table[posA - 1][posB] >= table[posA][posB - 1]) {
            posA--;
        } else {
            posB--;
        }
    }
    printf("LCS length = %d\nCommon path:", length);
    for (posA = 0; posA < length; posA++)
        printf(" %d", common[posA]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
LCS length = 5
Common path: 1 4 7 9 5
```

المخطط الإنسيابي (Organigramme)

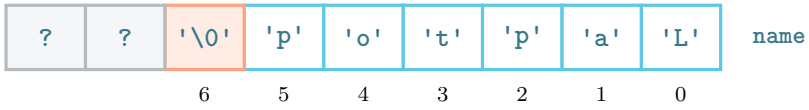


الفصل 6

السلاسل النصية

تعريفات

- **السلسلة النصية في C**: جدول من `char` ينتهي بالحرف الخاص `'\0'`.
- **التصريح**: `char name[20] = "Laptop";` يحجز 20 خانة، منها 7 مستعملة (6 حروف + `'\0'`).
- **الطول** هو عدد الحروف قبل `'\0'`، ونحسبه بحلقة.
- كل حرف عدد صحيح (رمز ASCII): `'A'` = 65، `'a'` = 97، `'0'` = 48.



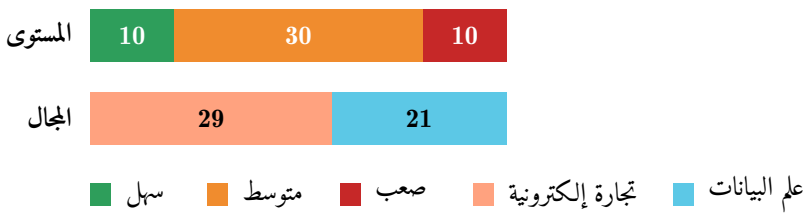
العملية	الفكرة (دون دوال جاهزة)
الطول	`while (text[len] != '\0') len++;`
النسخ	حرفًا بحرف ثم نضع `'\0'` في النهاية
المقارنة	نقارن الحروف حتى أول اختلاف أو نهاية إحدى السلسلتين
حرف كبير	`ch - 'a' + 'A'` إذا كان `ch` بين `'a'` و`'z'`
رقم إلى عدد	`value = value * 10 + (ch - '0')`

لا تنسَ `'\0'`

كل سلسلة نبنيها بأنفسنا يجب أن تنتهي بـ `'\0'`، وإلا طبع `printf` حروفًا عشوائية بعدها. والجدول يجب أن يتسع لعدد الحروف **زائد واحد**.

للاستزادة: [1, 2, 8]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 6.1 طول اسم المنتج

تجارة إلكترونية سهل

اكتب برنامجًا يحسب طول اسم منتج (عدد الحروف) دون الدالة `strlen`، مستغلًا أن كل سلسلة في C تنتهي بالحرف الخاص `'\0'`.

خطوات الحل

1) نبدأ بـ `length = 0`.
2) ما دام `name[length] != '\0'` نزيد `length`.
3) عند التوقف، `length` هو عدد الحروف.

البرنامج بلغة C

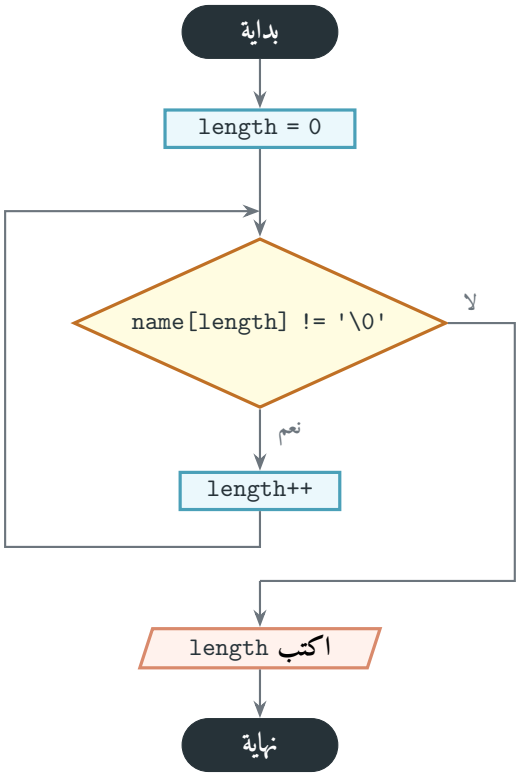
```c
#include <stdio.h>

int main(void) {
    char name[] = "Wireless Headphones";  // المترجم يضيف '\0' في النهاية
    int length = 0;
    while (name[length] != '\0')          // نتوقف عند علامة النهاية
        length++;
    printf("\"%s\" has %d characters\n", name, length);
    return 0;
}
```

نتيجة التنفيذ

```
"Wireless Headphones" has 19 characters
```

المخطط الإنسيابي (Organigramme)



تمرين 6.2 عدد كلمات التعليق

علم البيانات
سهل

اكتب برنامجًا يعد **الكلمات** في تعليق زبون. الكلمة سلسلة من الحروف غير الفراغية، قد توجد عدة فراغات متتالية.

خطوات الحل

1) `inWord = 0` يعني أننا خارج كلمة.
2) عند حرف غير فراغ ونحن خارج كلمة: كلمة جديدة `words++` و`inWord = 1`.
3) عند فراغ: `inWord = 0`.

البرنامج بلغة C

```c
#include <stdio.h>

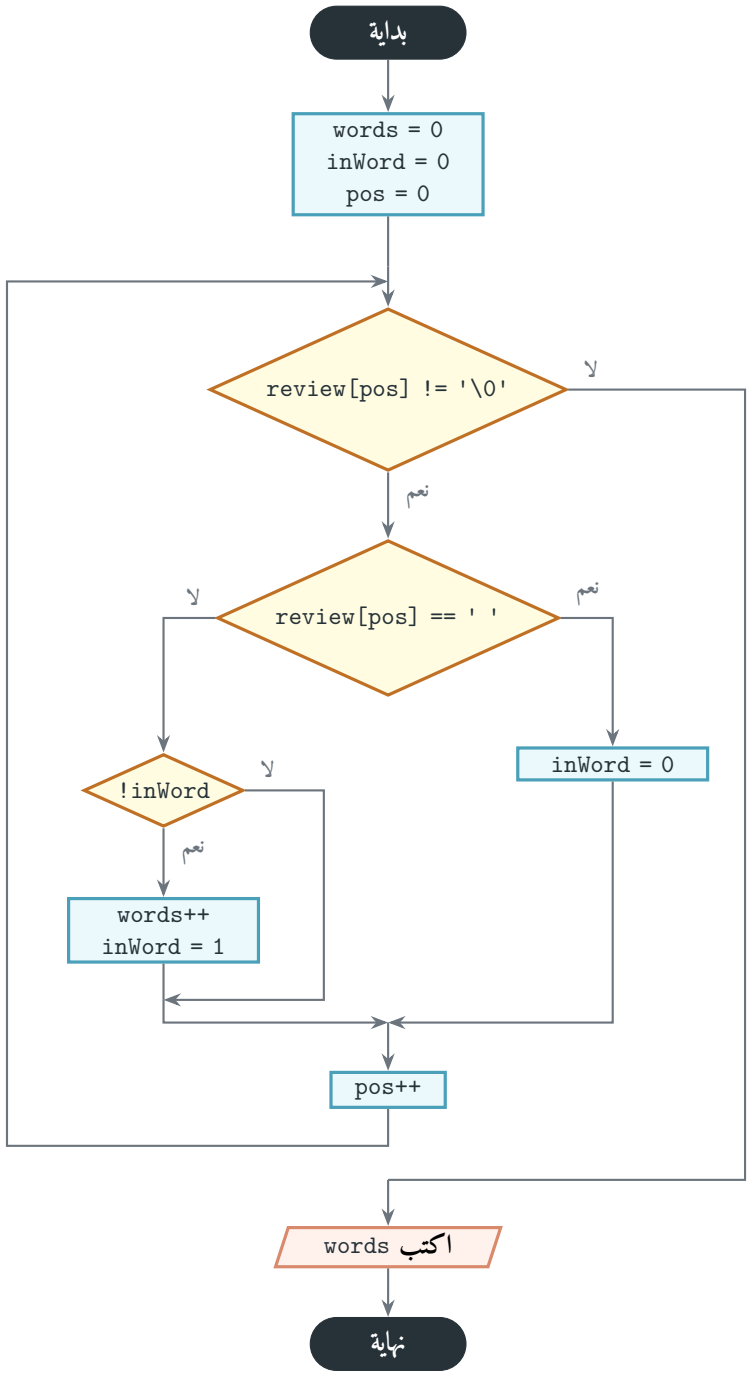
int main(void) {
    char review[] = "  Great   phone, fast delivery  and good price ";
    int words = 0, inWord = 0, pos;
    for (pos = 0; review[pos] != '\0'; pos++) {
        if (review[pos] == ' ') {
            inWord = 0;                 // الفراغ ينهي الكلمة
        } else if (!inWord) {
            words++;                    // أول حرف من كلمة جديدة
            inWord = 1;
        }
    }
    printf("Words: %d\n", words);
```

```
15      return 0;
16  }
```

نتيجة التنفيذ

```
Words: 7
```

المخطط الإنسيابي (Organigramme)



تمرين 6.3 رمز المنتج بالحروف الكبيرة

تجارة إلكترونية

سهل

رموز المنتجات (SKU) يجب أن تكون بالحروف **الكبيرة**. اكتب برنامجًا يحول رمزًا في مكانه دون `toupper`: كل حرف بين `'a'` و`'z'` نطرح منه `'a' - 'A'`.

خطوات الحل

1) نمر على السلسلة حتى `'\0'`.
2) إذا كان الحرف صغيرًا: `code[pos] = code[pos] - 'a' + 'A'`.
3) الأرقام والرموز تبقى كما هي.

البرنامج بلغة C

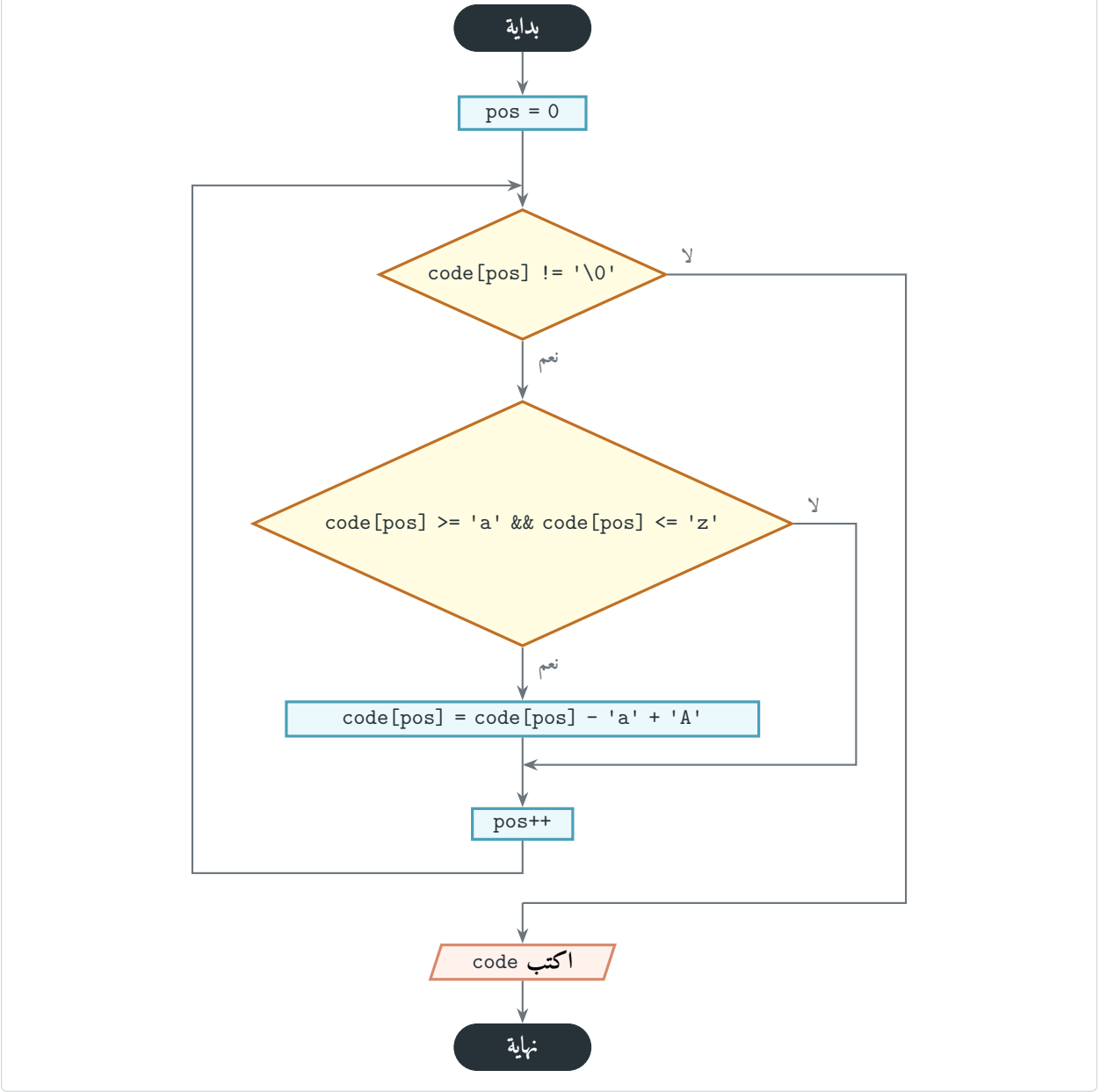
```c
#include <stdio.h>

int main(void) {
    char code[] = "tv-sam55-qled-2026";
    int pos;
    for (pos = 0; code[pos] != '\0'; pos++)
        if (code[pos] >= 'a' && code[pos] <= 'z')
            code[pos] = code[pos] - 'a' + 'A';    // الإزاحة بين الصغير والكبير
    printf("SKU: %s\n", code);
    return 0;
}
```

نتيجة التنفيذ

```
SKU: TV-SAM55-QLED-2026
```

المخطط الإنسيابي (Organigramme)



تمرين 6.4 عرض الإسم معكوسًا

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ اسم مستخدم (كلمة واحدة) ويعرضه **معكوسًا** حرفًا بحرف.

خطوات الحل

1) نقرأ بـ `scanf("%s", user)` (دون `&` لأن اسم الجدول عنوان).
2) نحسب الطول، ثم نطبع من `length - 1` نزولًا إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

```

```c
int main(void) {
    char user[50];
    int length = 0, pos;
    scanf("%49s", user);                // cen0" + حرفًا كحد أقصى 49
    while (user[length] != '\0')
        length++;
    for (pos = length - 1; pos >= 0; pos--)
        putchar(user[pos]);             // نطبع حرفًا واحدًا
    putchar('\n');
    return 0;
}
```

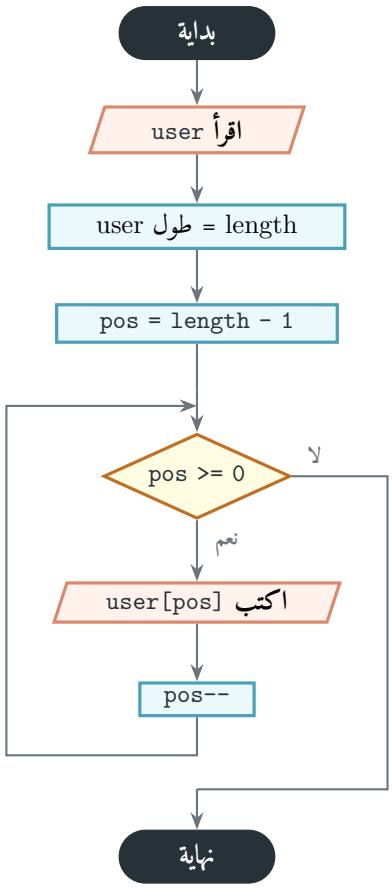
المدخلات

```
amina2026
```

نتيجة التنفيذ

```
6202anima
```

المخطط الإنسيابي (Organigramme)



تمرين 6.5 عدد الأرقام في رمز ترويجي

تجارة إلكترونية
سهل

اكتب برنامجًا يعد في رمز ترويجي عدد **الأرقام** وعدد **الحروف** وعدد الرموز الأخرى.

خطوات الحل

1) رقم: `'0' <= ch <= '9'`.
2) حرف: بين `'a'` و`'z'` أو بين `'A'` و`'Z'`.
3) غير ذلك: رمز.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char promo[40], ch;
    int pos, digits = 0, letters = 0, others = 0;
    scanf("%39s", promo);
    for (pos = 0; promo[pos] != '\0'; pos++) {
        ch = promo[pos];
        if (ch >= '0' && ch <= '9')
            digits++;
        else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z'))
            letters++;
        else
            others++;                 // أو _ أو غيرهما -
    }
    printf("Digits %d, letters %d, others %d\n", digits, letters, others);
    return 0;
}
```

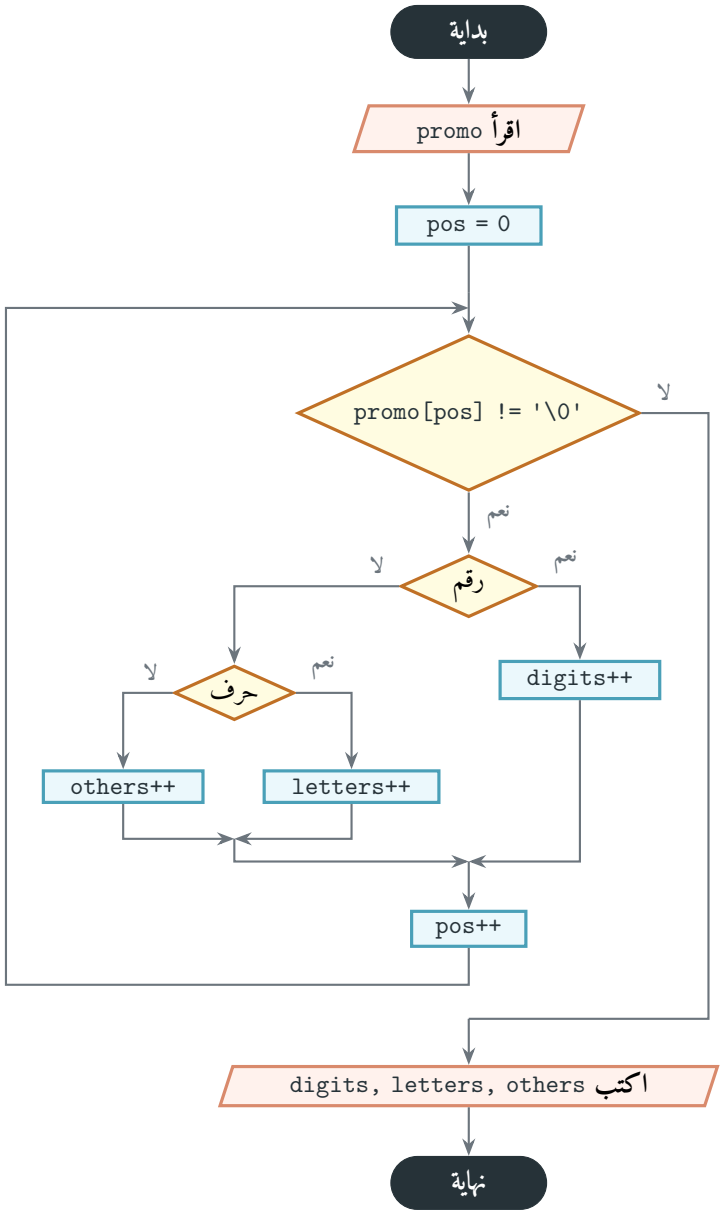
المدخلات

```
SUMMER-2026-X7
```

نتيجة التنفيذ

```
Digits 5, letters 7, others 2
```

المخطط الإنسيابي (Organigramme)



تمرين 6.6 نسخ سلسلة يدويًا

تجارة إلكترونية
سهل

اكتب برنامجًا ينسخ اسم منتج إلى جدول جديد حرفًا بحرف **دون** `strcpy`، دون أن ننسى `'\0'`، ثم يعدل النسخة ليتأكد أن الأصل لم يتغير.

خطوات الحل

1) ننسخ ما دام الحرف ليس `'\0'`.
2) بعد الحلقة نضع `'\0'` في النسخة.

البرنامج بلغة C

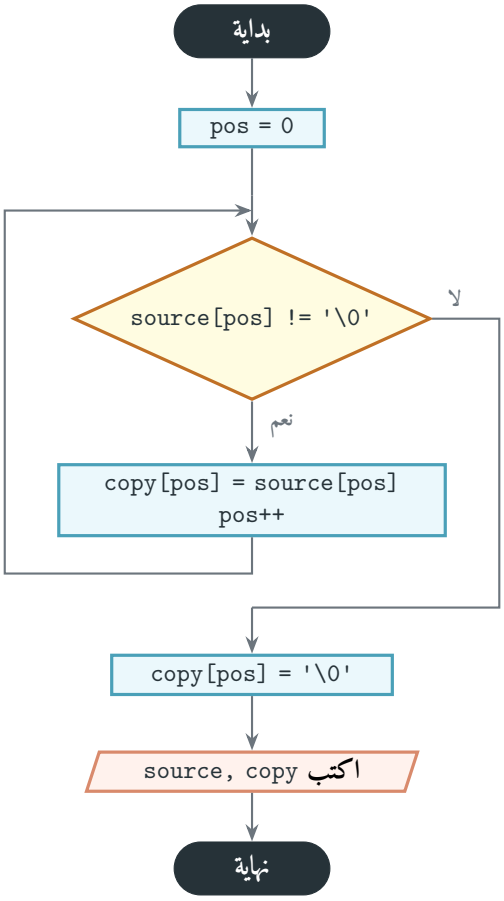
```c
#include <stdio.h>

int main(void) {
    char source[] = "Gaming Laptop";
    char copy[50];
    int pos = 0;
    while (source[pos] != '\0') {
        copy[pos] = source[pos];       // حرفًا بحرف
        pos++;
    }
    copy[pos] = '\0';                  // ضروري: علامة النهاية
    copy[0] = 'X';                     // نعدل النسخة فقط
    printf("Source: %s\nCopy:   %s\n", source, copy);
    return 0;
}
```

نتيجة التنفيذ

```
Source: Gaming Laptop
Copy:   Xaming Laptop
```

المخطط الإنسيابي (Organigramme)



تمرين 6.7 الإسم الكامل للزبون

تجارة إلكترونية سهل

اكتب برنامجًا يقرأ الاسم واللقب ويكوّن **الاسم الكامل** "اللقب الاسم" في جدول ثالث دون `strcat`.

خطوات الحل

1) ننسخ اللقب في `full`، ثم نضيف فراغًا.
2) نلصق الاسم بعده، ثم `'\0'`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char first[30], last[30], full[62];
    int pos = 0, src;
    scanf("%29s %29s", first, last);
    for (src = 0; last[src] != '\0'; src++)
        full[pos++] = last[src];      // اللقب أولًا
    full[pos++] = ' ';
    for (src = 0; first[src] != '\0'; src++)
        full[pos++] = first[src];     // ثم الاسم
    full[pos] = '\0';
    printf("Customer: %s (%d chars)\n", full, pos);
    return 0;
}
```

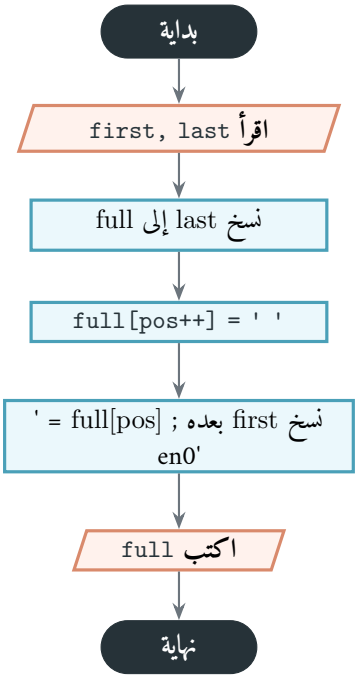
المدخلات

```
Yacine Belkacem
```

نتيجة التنفيذ

```
Customer: Belkacem Yacine (15 chars)
```

المخطط الانسيابي (Organigramme)



تمرين 6.8 مطابقة كلمة المرور

تجارة إلكترونية سهل

عند التسجيل يكتب المستخدم كلمة المرور مرتين. اكتب برنامجًا يتحقق من **تطابقهما** حرفًا بحرف دون `strcmp`.

خطوات الحل

1) نتقدم ما دام الحرفان متساويين وليسا نهاية السلسلة.
2) السلسلتان متطابقتان إذا كان الحرفان عند التوقف متساويين (أي كلاهما `'\0'`).

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char first[40], second[40];
    int pos = 0;
    scanf("%39s %39s", first, second);
    while (first[pos] == second[pos] && first[pos] != '\0')
        pos++;                          // نتقدم ما دام الحرفان متطابقين
    if (first[pos] == second[pos])      // الاثنان "cen0
        printf("Passwords match\n");
    else
        printf("Mismatch at position %d ('%c' vs '%c')\n", pos, first[pos], second[pos]);
    return 0;
}
```

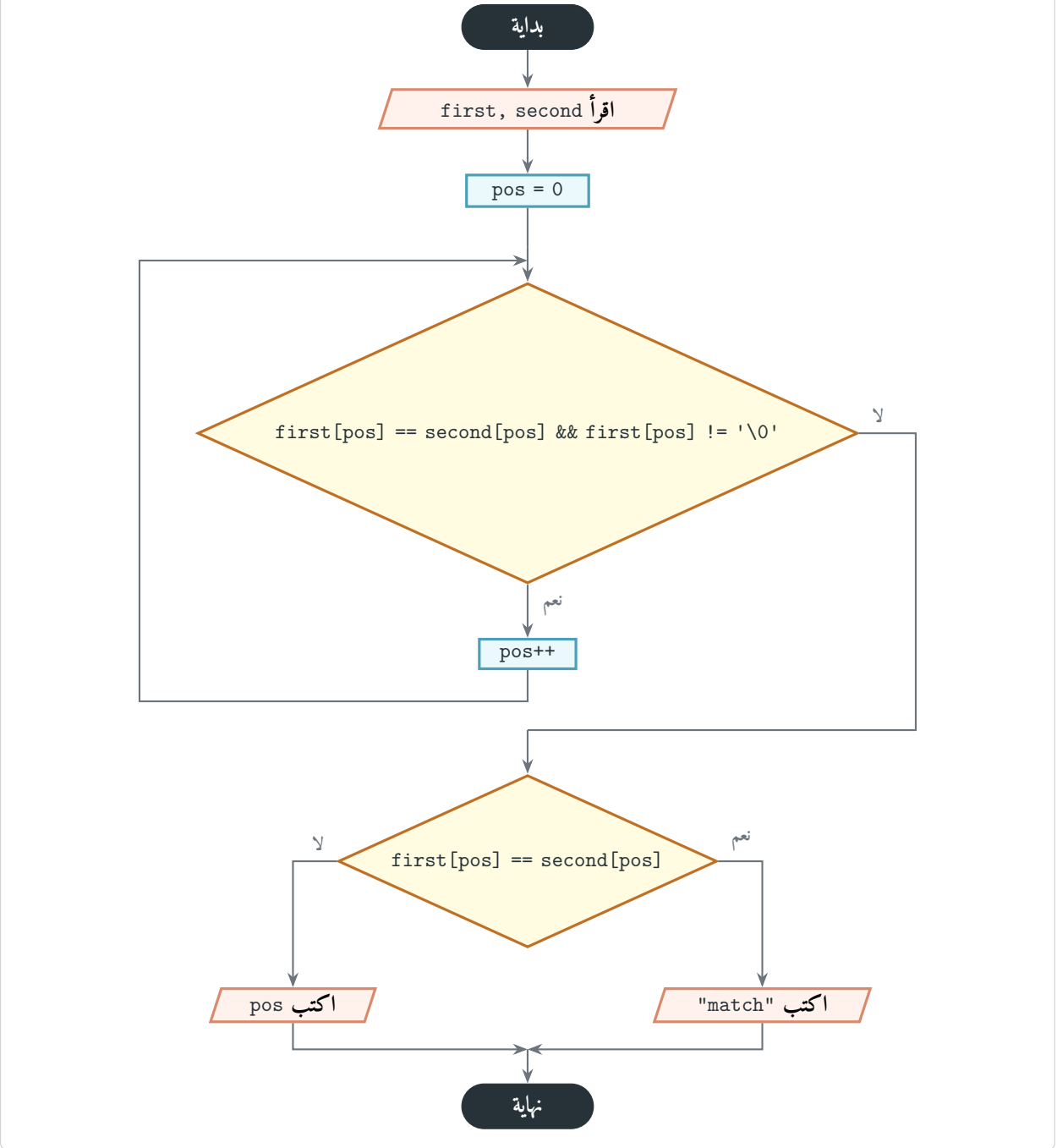
المدخلات

```
Dz2026!pass Dz2026!Pass
```

نتيجة التنفيذ

```
Mismatch at position 7 ('p' vs 'P')
```

المخطط الإنسيابي (Organigramme)



تمرين 6.9 علامات التعجب في التعليقات

علم البيانات ⭐ سهل

كثرة علامات التعجب قد تدل على تعليق انفعالي. اكتب برنامجًا يعد تكرار حرف مقروء (مثل !) في تعليق ويعرض نسبته من طول التعليق.

خطوات الحل

1) نقرأ الحرف المطلوب بـ `scanf(" %c")`.
2) نمر على التعليق ونعد التطابقات ونحسب الطول.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char review[] = "Worst delivery ever!!! Never again!!";
    char target;
    int count = 0, length = 0;
    scanf(" %c", &target);
    while (review[length] != '\0') {
        if (review[length] == target)
            count++;
        length++;                    // نحسب الطول في نفس المرور
    }
    printf("'%c' appears %d times (%.1f %% of text)\n", target, count, count * 100.0 / length);
    return 0;
}
```

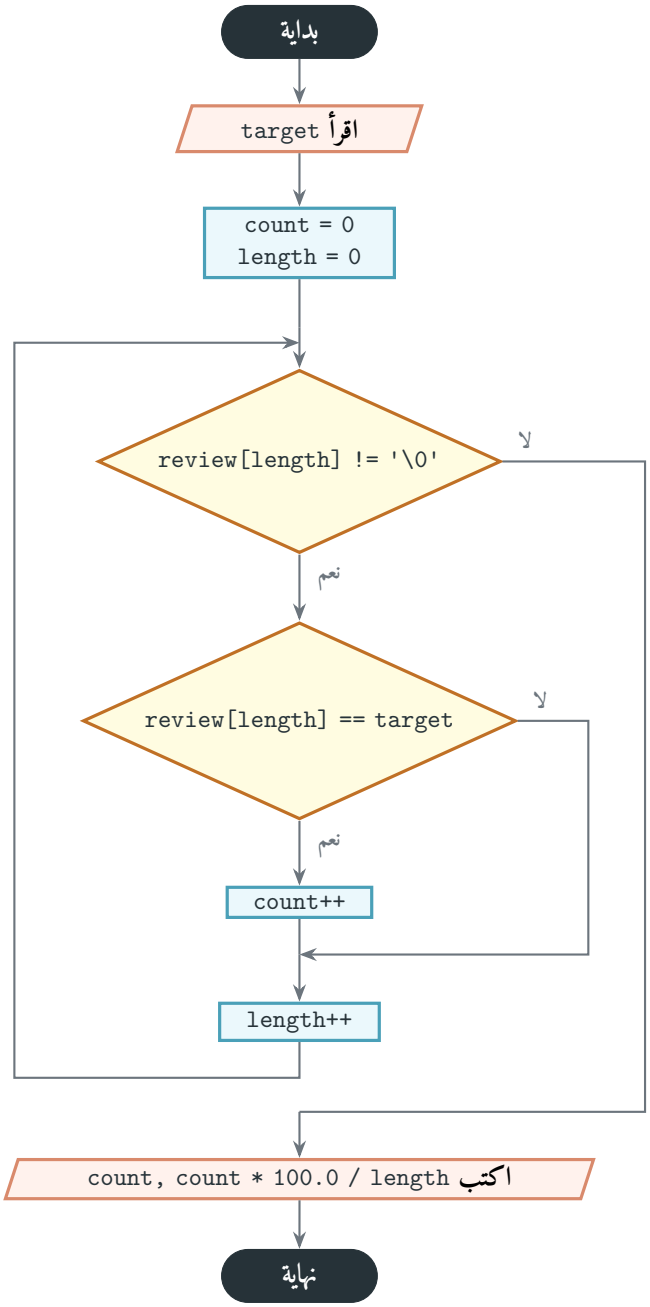
المدخلات

```
!
```

نتيجة التنفيذ

```
'!' appears 5 times (13.9 % of text)
```

المخطط الإنسيابي (Organigramme)



تمرين 6.10 تحقق بسيط من البريد الإلكتروني

تجارة إلكترونية | سهل

اكتب برنامجًا يقبل البريد الإلكتروني إذا احتوى @ **واحدة فقط** ليست في البداية، وبعدها **نقطة** واحدة على الأقل ليست مباشرة بعد @ ولا في النهاية.

خطوات الحل

1) نعد علامات @ ونحفظ موضعها `atPos`.
2) نبحث عن نقطة في موضع `atPos + 1` <.
3) نتحقق أن الحرف الأخير ليس نقطة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char email[80];
    int pos, atCount = 0, atPos = -1, dotAfter = 0, length = 0;
    scanf("%79s", email);
    for (pos = 0; email[pos] != '\0'; pos++) {
        if (email[pos] == '@') {
            atCount++;
            atPos = pos;
        } else if (email[pos] == '.' && atPos >= 0 && pos > atPos + 1) {
            dotAfter = 1;              // نقطة في اسم النطاق
        }
        length++;
    }
    if (atCount == 1 && atPos > 0 && dotAfter && email[length - 1] != '.')
        printf("%s: valid\n", email);
    else
        printf("%s: invalid\n", email);
    return 0;
}
```

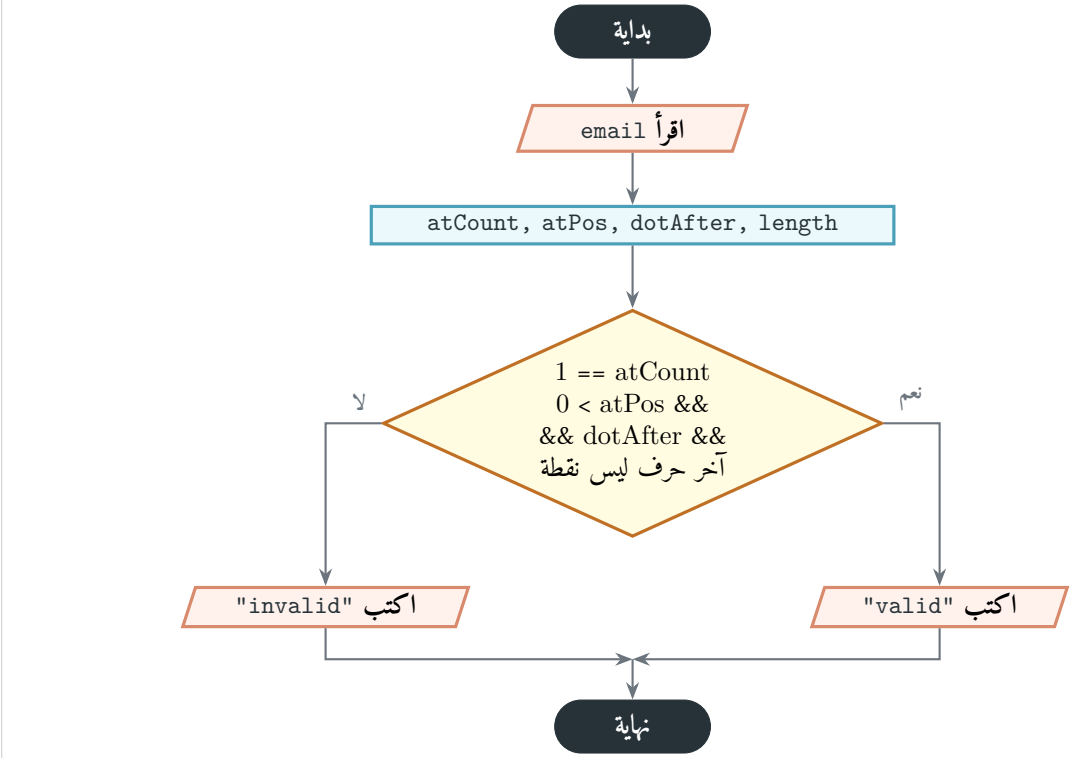
المدخلات

```
client.dz@shop.example.com
```

نتيجة التنفيذ

```
client.dz@shop.example.com: valid
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 6.11 اسم متناظر

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يتحقق إن كانت عبارة متناظرة مع تجاهل الفراغات وحالة الحروف (مثل "Step on no pets").

خطوات الحل

1) مؤشران: `left` من البداية و`right` من النهاية.
2) نتخطى الفراغات من الجهتين.
3) نقارن الحرفين بعد تحويلهما إلى صغيرين يدويًا.

البرنامج بلغة C

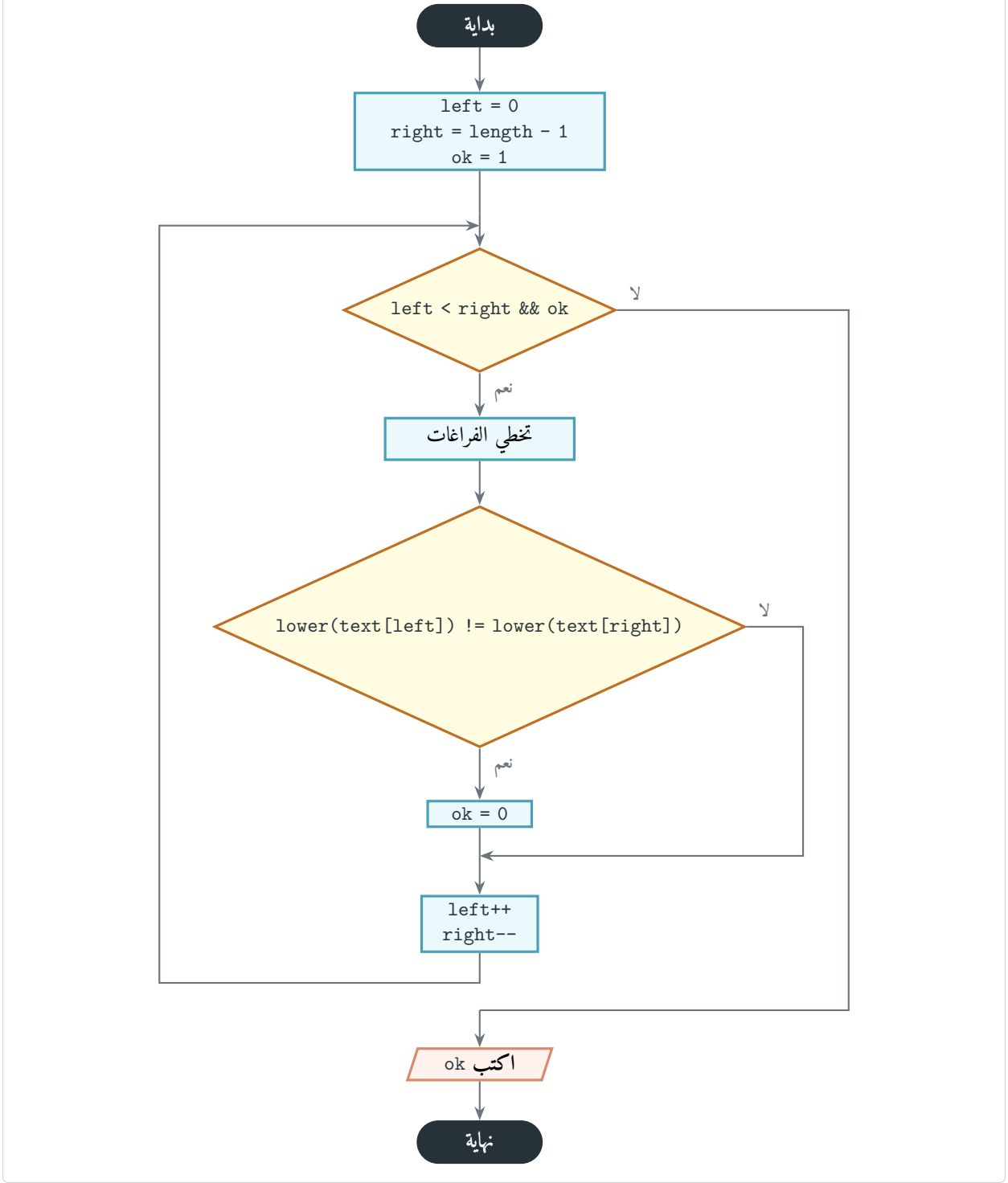
```c
#include <stdio.h>

int main(void) {
    char text[] = "Step on no pets";
    int left = 0, right = 0, ok = 1;
    char chLeft, chRight;
    while (text[right] != '\0')
        right++;
    right--;                                    // آخر حرف
    while (left < right && ok) {
        while (text[left] == ' ') left++;       // نتخطى الفراغات
        while (text[right] == ' ') right--;
        chLeft = text[left];
        chRight = text[right];
        if (chLeft >= 'A' && chLeft <= 'Z') chLeft += 'a' - 'A';
        if (chRight >= 'A' && chRight <= 'Z') chRight += 'a' - 'A';
        if (chLeft != chRight)
            ok = 0;
        left++;
        right--;
    }
    printf("\"%s\" is %sa palindrome\n", text, ok ? "" : "not ");
    return 0;
}
```

نتيجة التنفيذ

```
"Step on no pets" is a palindrome
```

المخطط الإنسيابي (Organigramme)



تمرين 6.12 أطول كلمة في الوصف

علم البيانات ★★ متوسط

اكتب برنامجًا يجد **أطول كلمة** في وصف منتج، ويعرضها مع طولها وموضع بدايتها، وكذلك متوسط طول الكلمات.

خطوات الحل

1) نمر على النص ونحسب طول الكلمة الحالية `current` وبدايتها.

2) عند نهاية كلمة (فراغ أو '\0') نقارنها بالأطول.

3) نجمع الأطوال ونعد الكلمات للمتوسط.

البرنامج بلغة C

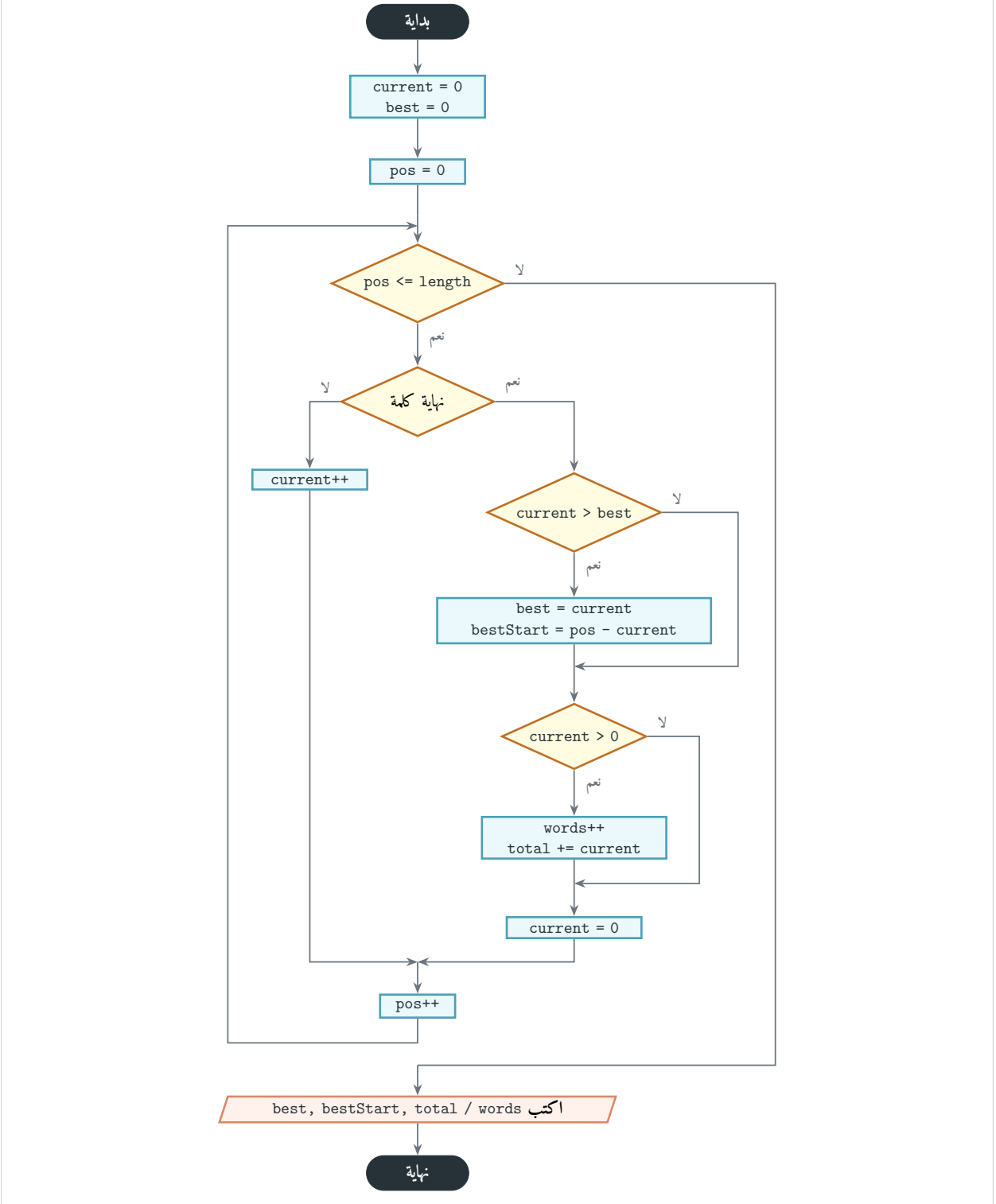
```c
#include <stdio.h>

int main(void) {
    char text[] = "Ultra thin waterproof smartwatch with heart monitor";
    int pos, current = 0, best = 0, bestStart = 0, words = 0, total = 0, done = 0;
    for (pos = 0; !done; pos++) {
        if (text[pos] == ' ' || text[pos] == '\0') {  // نهاية كلمة
            if (current > best) {
                best = current;
                bestStart = pos - current;
            }
            if (current > 0) { words++; total += current; }
            current = 0;
            if (text[pos] == '\0') done = 1;
        } else {
            current++;
        }
    }
    printf("Longest word: ");
    for (pos = bestStart; pos < bestStart + best; pos++)
        putchar(text[pos]);
    printf(" (%d letters, starts at %d)\n", best, bestStart);
    printf("Average word length: %.2f\n", (double)total / words);
    return 0;
}
```

نتيجة التنفيذ

```
Longest word: waterproof (10 letters, starts at 11)
Average word length: 6.43
```

المخطط الإنسيابي (Organigramme)



تمرين 6.13 تنسيق أسماء المنتجات

تجارة إلكترونية ★★ متوسط

تصل أسماء المنتجات من الموردين بحروف عشوائية مثل `"sAMSUNG gALAXY s24 ULTRA"`. اكتب برنامجًا يحولها إلى **صيغة العنوان**: الحرف الأول من كل كلمة كبير والباقي صغير.

خطوات الحل

1) `start = 1` عند بداية كل كلمة.
2) إذا كان الحرف أول كلمة: نحوله إلى كبير، وإلا إلى صغير.
3) الفراغ يعيد `start = 1`.

البرنامج بلغة C

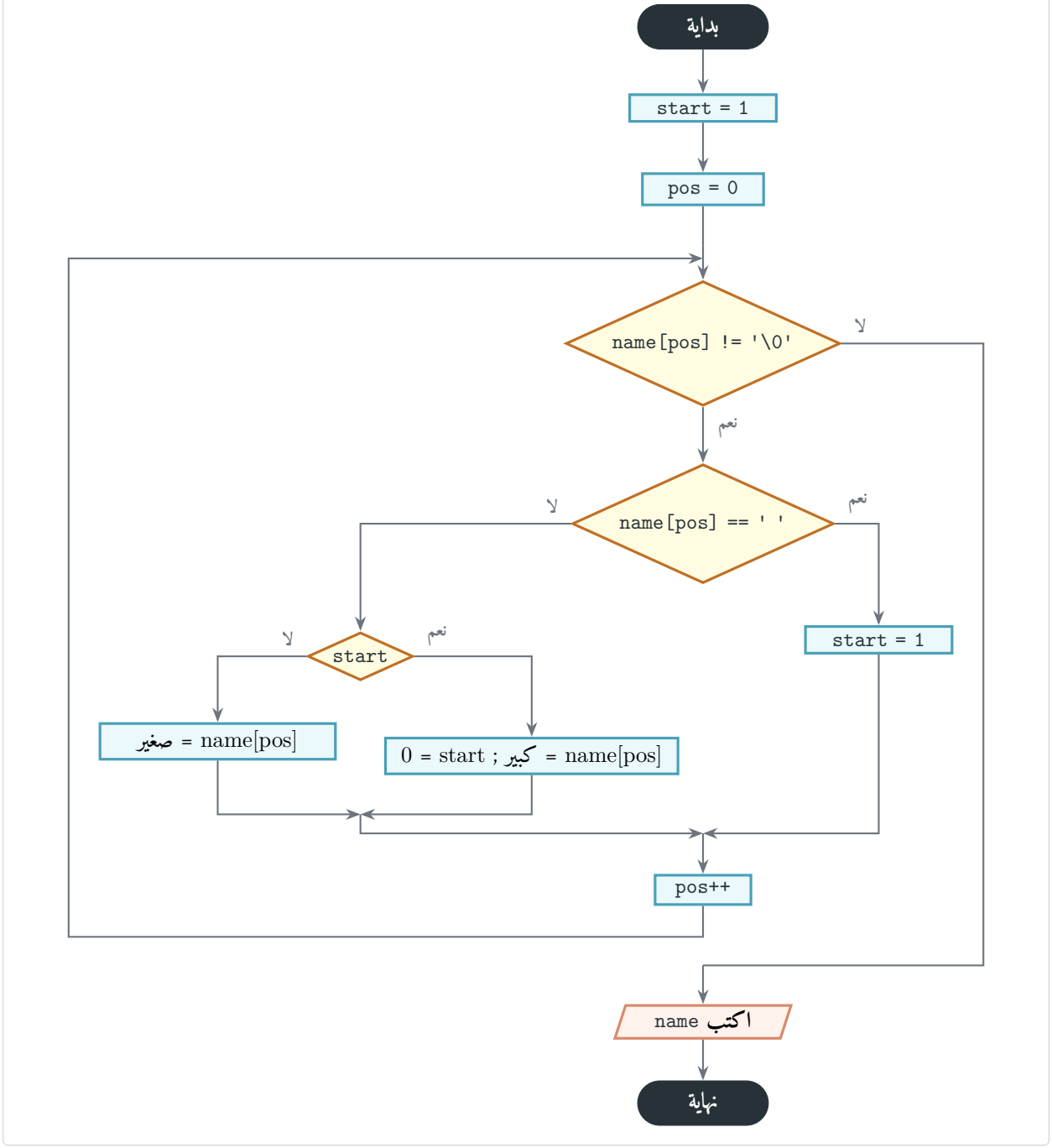
```c
#include <stdio.h>

int main(void) {
    char name[] = "sAMSUNG gALAXY s24 ULTRA";
    int pos, start = 1;
    for (pos = 0; name[pos] != '\0'; pos++) {
        if (name[pos] == ' ') {
            start = 1;                    // الكلمة التالية تبدأ بعد الفراغ
        } else if (start) {
            if (name[pos] >= 'a' && name[pos] <= 'z')
                name[pos] -= 'a' - 'A';   // أول حرف: كبير
            start = 0;
        } else if (name[pos] >= 'A' && name[pos] <= 'Z') {
            name[pos] += 'a' - 'A';       // بقية الحروف: صغيرة
        }
    }
    printf("%s\n", name);
    return 0;
}
```

نتيجة التنفيذ

```
Samsung Galaxy S24 Ultra
```

المخطط الإنسيابي (Organigramme)



تمرين 6.14 حذف الفراغات الزائدة

علم البيانات

متوسط

تنظيف البيانات: اكتب برنامجًا يحذف من نص الفراغات في **البداية** و**النهاية** ويختصر الفراغات المتتالية في فراغ واحد، **في مكانه**.

خطوات الحل

1) مؤشر قراءة `read` ومؤشر كتابة `write`.
2) ننسخ الحرف إلا إذا كان فراغًا وكان آخر حرف مكتوب فراغًا (أو لم نكتب شيئًا بعد).
3) في النهاية نحذف فراغًا أخيرًا إن وُجد.

البرنامج بلغة C

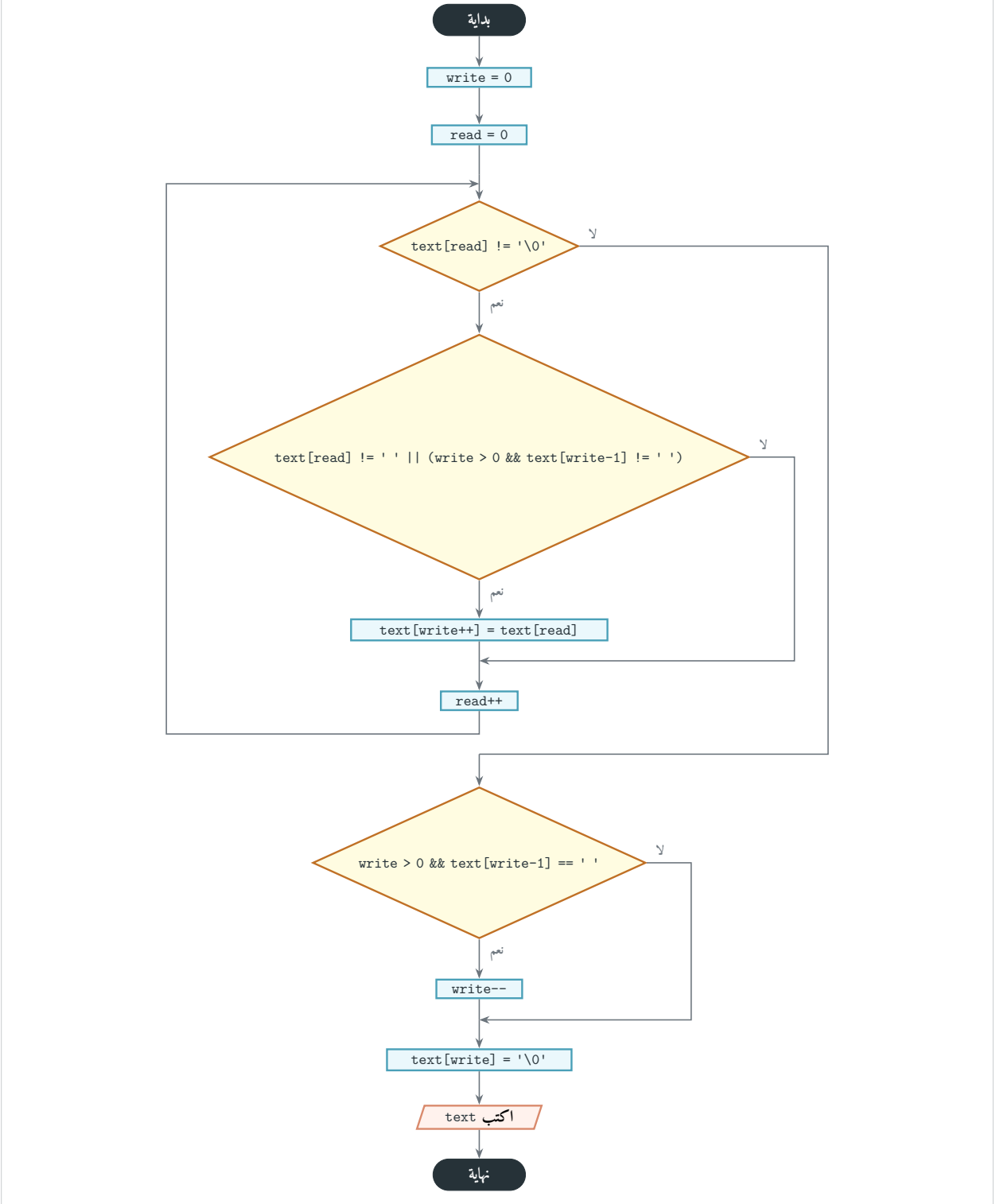
```c
#include <stdio.h>

int main(void) {
    char text[] = "   Free    shipping   on   all   orders   ";
    int read, write = 0;
    for (read = 0; text[read] != '\0'; read++)
        // نكتب الحرف إلا إذا كان فراغًا مكررًا أو فراغًا في البداية
        if (text[read] != ' ' || (write > 0 && text[write - 1] != ' '))
            text[write++] = text[read];
    if (write > 0 && text[write - 1] == ' ')
        write--;                            // فراغ النهاية
    text[write] = '\0';
    printf("[%s]\n", text);
    return 0;
}
```

نتيجة التنفيذ

```
[Free shipping on all orders]
```

المخطط الإنسيابي (Organigramme)



تمرين 6.15 تحويل نص إلى عدد صحيح — تجارة إلكترونية — متوسط

تصل الأسعار من نموذج الويب كنصوص مثل `"-12500"`. اكتب برنامجًا يحولها إلى `int` دون `atoi`، مع معالجة الإشارة ورفض النص إذا احتوى حرفًا غير رقمي.

خطوات الحل

1) إذا كان الحرف الأول - أو + نحفظ الإشارة ونتقدم.
2) لكل رقم: `value = value * 10 + (ch - '0')`.
3) حرف غير رقمي: النص غير صالح.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char text[30];
    int pos = 0, sign = 1, valid = 1;
    long value = 0;
    scanf("%29s", text);
    if (text[0] == '-' || text[0] == '+') {
        if (text[0] == '-') sign = -1;
        pos = 1;
    }
    if (text[pos] == '\0') valid = 0;      // إشارة بلا أرقام
    for (; text[pos] != '\0' && valid; pos++) {
        if (text[pos] >= '0' && text[pos] <= '9')
            value = value * 10 + (text[pos] - '0');    // نزيح ونضيف الرقم
        else
            valid = 0;
    }
    if (valid)
        printf("\"%s\" -> %ld (double: %ld)\n", text, sign * value, 2 * sign * value);
    else
        printf("\"%s\" is not a number\n", text);
    return 0;
}
```

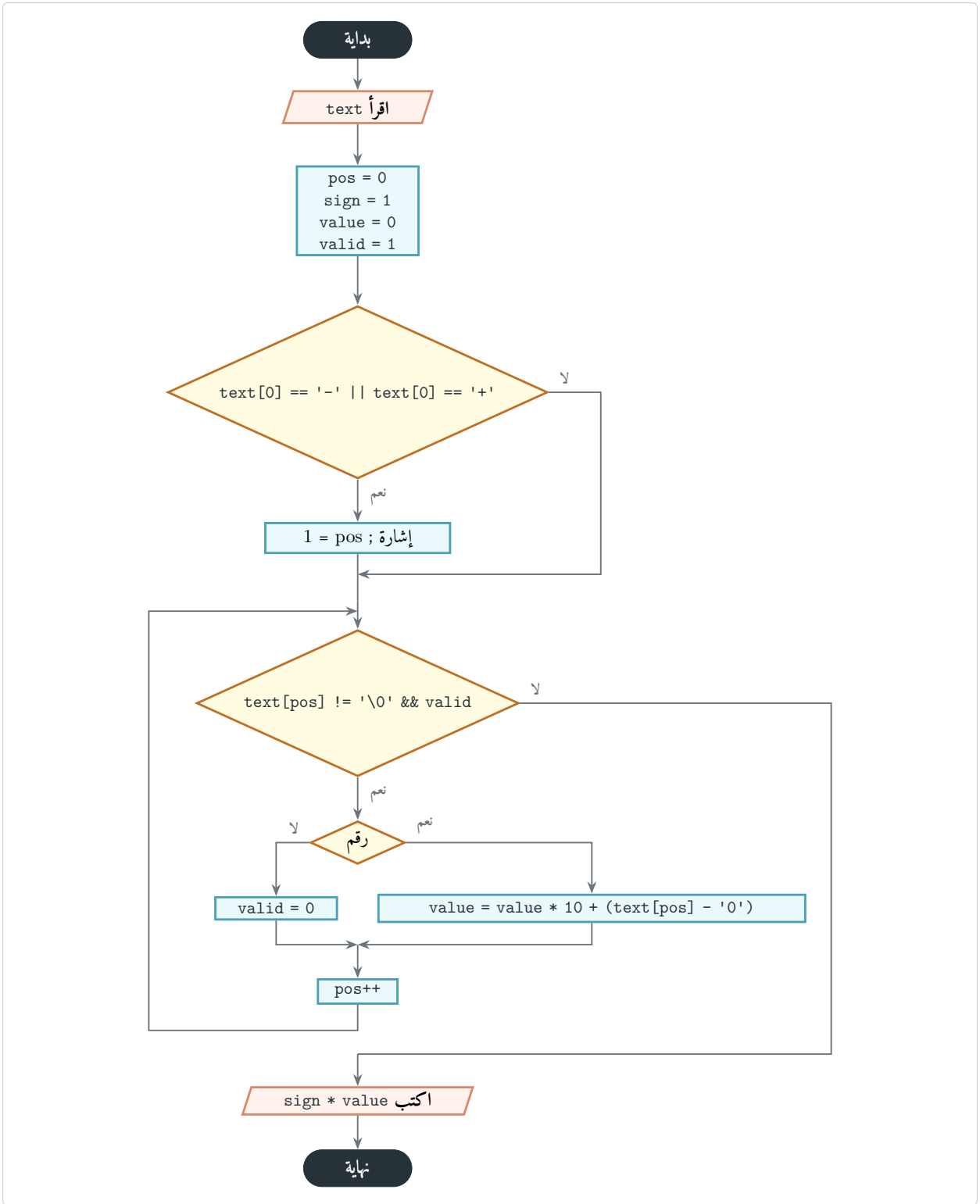
المدخلات

```
-12500
```

نتيجة التنفيذ

```
"-12500" -> -12500 (double: -25000)
```

المخطط الإنسيابي (Organigramme)



تمرين 6.16 تحويل عدد صحيح إلى نص

تجارة إلكترونية ★★ متوسط

لطباعة رقم الطلبية على الملصق كنص، اكتب برنامجًا يحول عددًا صحيحًا (قد يكون 0) إلى سلسلة حروف دون `sprintf`، ثم يضيف البادئة `ORD-`.

خطوات الحل

1) نستخرج الأرقام من اليمين `number % 10` ونخزنها في جدول مؤقت.
2) نكتب البادئة ثم الأرقام بالترتيب المعاكس، ونحول كل رقم بـ `'0' + digit`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char label[20] = "ORD-";
    int digits[12], count = 0, pos = 4;
    long number;
    scanf("%ld", &number);
    do {                                    // do-while: العدد 0 يعطي "0"
        digits[count++] = number % 10;
        number /= 10;
    } while (number > 0);
    while (count > 0)
        label[pos++] = '0' + digits[--count];  // من أكبر رقم إلى أصغرها
    label[pos] = '\0';
    printf("Label: %s\n", label);
    return 0;
}
```

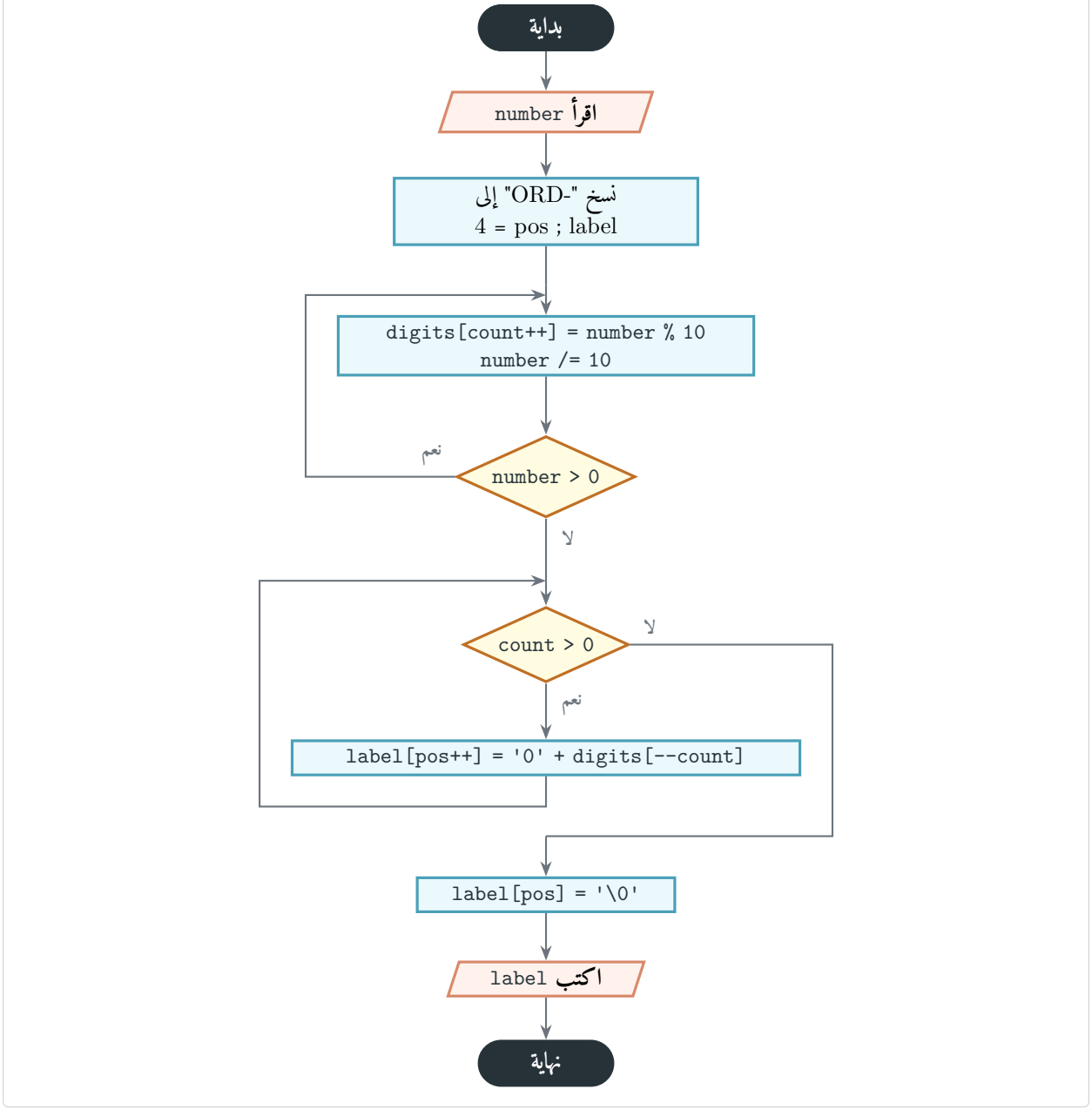
المدخلات

```
470213
```

نتيجة التنفيذ

```
Label: ORD-470213
```

المخطط الإنسيابي (Organigramme)



تمرين 6.17 قراءة سعر عشري من نص

تجارة إلكترونية

متوسط

اكتب برنامجًا يحول نص سعر مثل `"1299.95"` إلى `double` دون `atof`: الجزء الصحيح كالعادة، وبعد النقطة كل رقم يُضرب في 0.1 ثم 0.01 ...

خطوات الحل

1) قبل النقطة: `value = value * 10 + digit`.
2) بعد النقطة: `scale = scale / 10` و`value += digit * scale`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char text[30];
    double value = 0, scale = 1;
    int pos, afterDot = 0, digit;
    scanf("%29s", text);
    for (pos = 0; text[pos] != '\0'; pos++) {
        if (text[pos] == '.') {
            afterDot = 1;
            continue;
        }
        digit = text[pos] - '0';
        if (!afterDot) {
            value = value * 10 + digit;       // الجزء الصحيح
        } else {
            scale /= 10;                      // 0.1 ثم 0.01 ...
            value += digit * scale;
        }
    }
    printf("Price = %.2f, with tax = %.2f\n", value, value * 1.19);
    return 0;
}
```

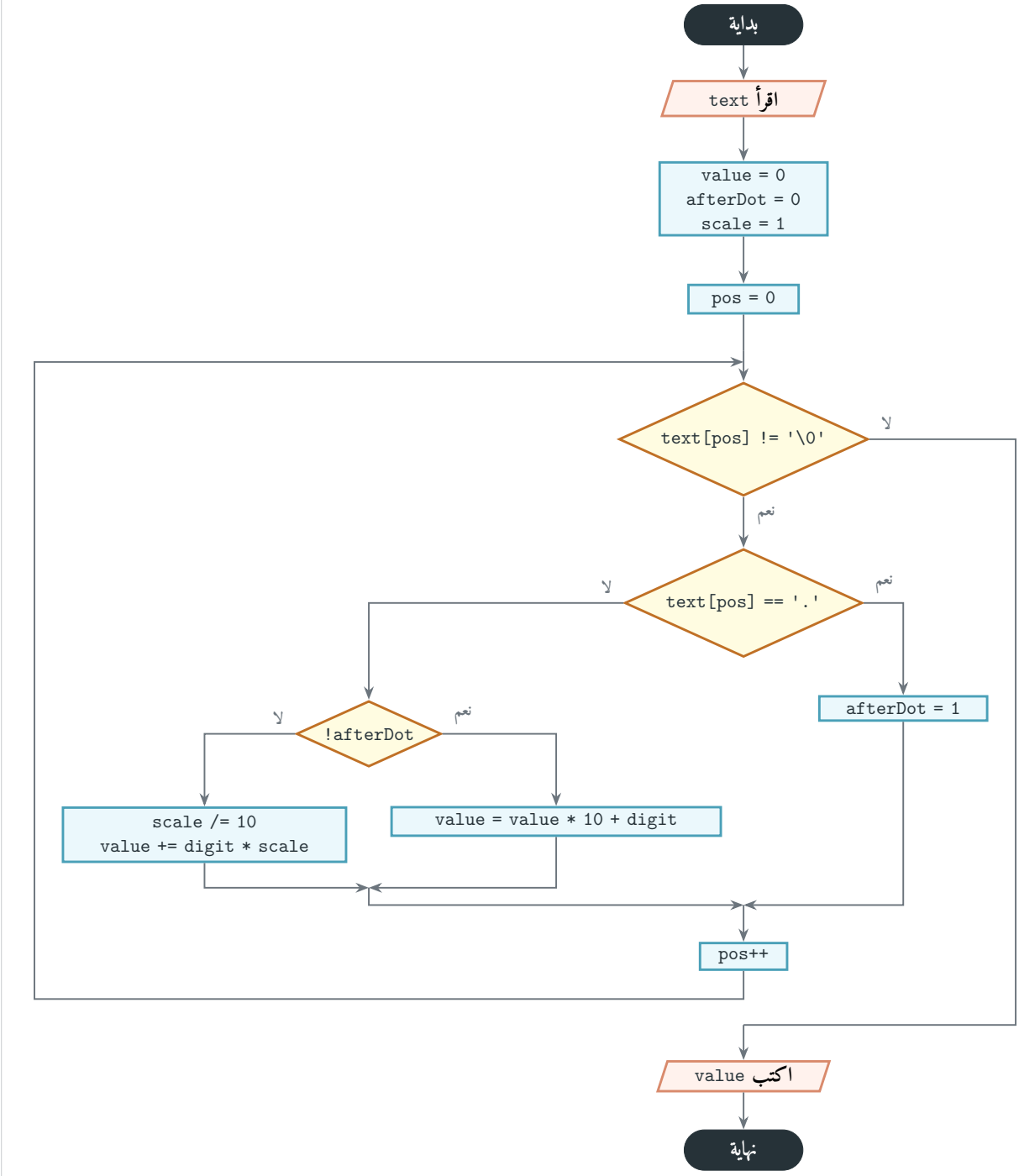
المدخلات

```
1299.95
```

نتيجة التنفيذ

```
Price = 1299.95, with tax = 1546.94
```

المخطط الإنسيابي (Organigramme)



تمرين 6.18 رابط صديق لمحركات البحث

تجارة إلكترونية متوسط

لإنشاء رابط صفحة المنتج (slug)، اكتب برنامجًا يحول الاسم إلى حروف صغيرة، ويعوض كل سلسلة من الرموز غير الحرفية/الرقمية بـ - واحدة، دون - في البداية أو النهاية. مثال: `"Smart TV 55\" (4K)!"` يصبح `smart-tv-55-4k`.

خطوات الحل

1) إذا كان الحرف حرفًا أو رقمًا: نكتبه صغيرًا.

2) وإلا: نكتب - فقط إذا كان آخر حرف مكتوب ليس - وكان `write > 0`.

3) نحذف - الأخيرة إن وجدت.

البرنامج بلغة C

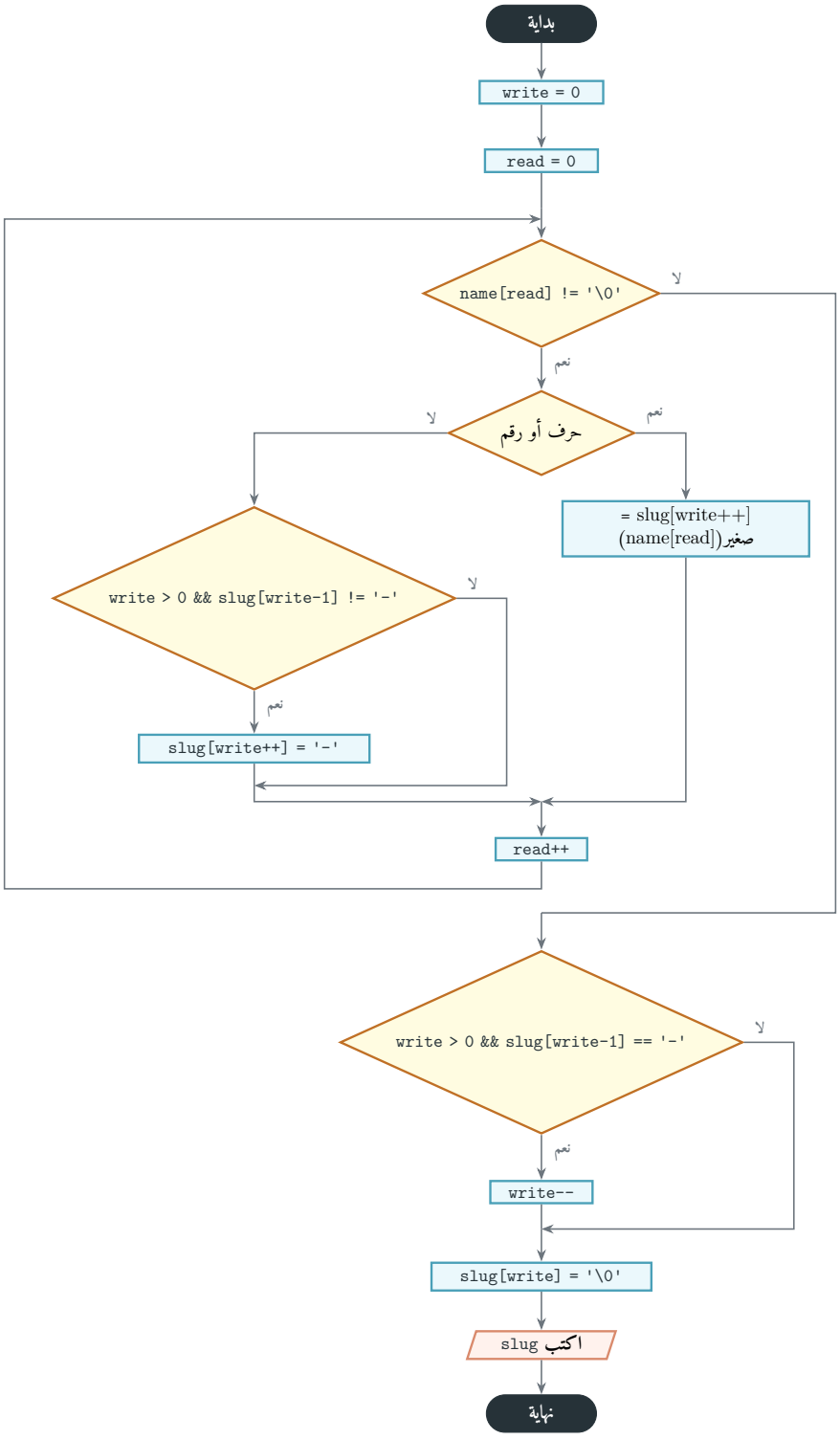
```c
#include <stdio.h>

int main(void) {
    char name[] = "Smart TV 55\" (4K) -- Ultra HD!";
    char slug[60], ch;
    int read, write = 0;
    for (read = 0; name[read] != '\0'; read++) {
        ch = name[read];
        if (ch >= 'A' && ch <= 'Z')
            ch += 'a' - 'A';                    // حروف صغيرة
        if ((ch >= 'a' && ch <= 'z') || (ch >= '0' && ch <= '9'))
            slug[write++] = ch;
        else if (write > 0 && slug[write - 1] != '-')
            slug[write++] = '-';                // فاصل واحد فقط
    }
    if (write > 0 && slug[write - 1] == '-')
        write--;
    slug[write] = '\0';
    printf("https://shop.dz/p/%s\n", slug);
    return 0;
}
```

نتيجة التنفيذ

```
https://shop.dz/p/smart-tv-55-4k-ultra-hd
```

المخطط الإنسيابي (Organigramme)



تمرين 6.19 البحث عن كلمة مفتاحية

علم البيانات

متوسط

اكتب برنامجًا يبحث عن كلمة مفتاحية داخل وصف منتج (دون `strstr`) ويعرض **كل** مواضع ظهورها وعددها.

خطوات الحل

1) لكل موضع بداية ممكن `start` في النص: نقارن الحروف واحدًا واحدًا.

2) إذا وصلنا إلى `'\0'` الكلمة: وجدنا تطابقًا.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char text[] = "Long battery life. The battery charges fast; spare battery included.";
    char word[30];
    int start, match, count = 0;
    scanf("%29s", word);
    for (start = 0; text[start] != '\0'; start++) {
        match = 0;
        // تتقدم ما دامت الحروف متطابقة
        while (word[match] != '\0' && text[start + match] == word[match])
            match++;
        if (word[match] == '\0') {            // كل حروف الكلمة تطابقت
            printf("Found at position %d\n", start);
            count++;
        }
    }
    printf("\"%s\" appears %d time(s)\n", word, count);
    return 0;
}
```

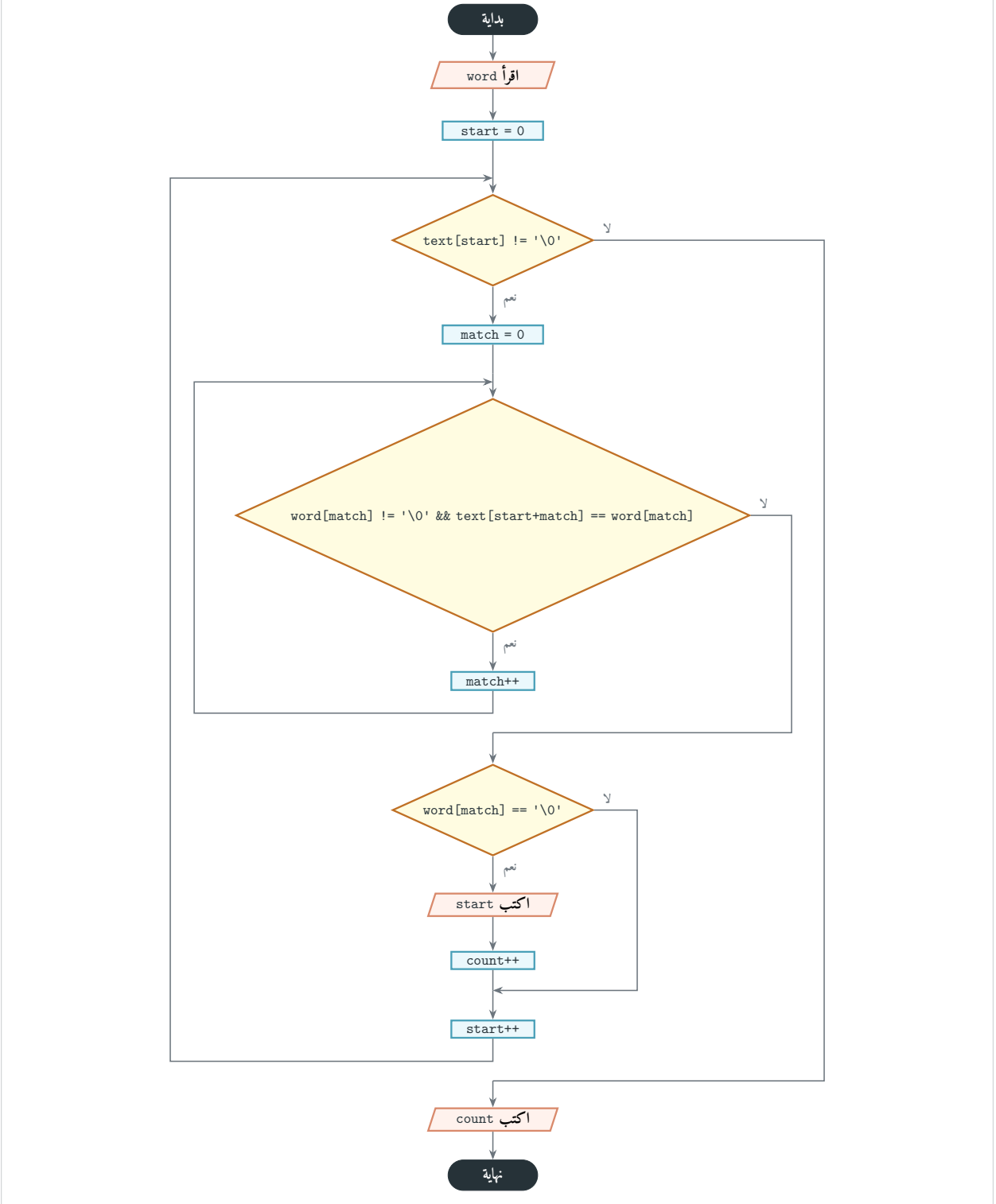
المدخلات

```
battery
```

نتيجة التنفيذ

```
Found at position 5
Found at position 23
Found at position 51
"battery" appears 3 time(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 6.20 حجب الكلمات المسيئة

علم البيانات

متوسط

قبل نشر التعليقات يحجب الموقع كلمة ممنوعة بتعويض كل حروفها بـ *. اكتب برنامجًا يقوم بذلك في مكانه، **فقط** عندما تكون الكلمة كاملة (ليست جزءًا من كلمة أطول).

خطوات الحل

1) نبحث عن الكلمة في كل موضع.
2) نتحقق أن الحرف قبلها وبعدها ليس حرفًا (حدود كلمة).
3) نعوض حروفها بـ *.

البرنامج بلغة C

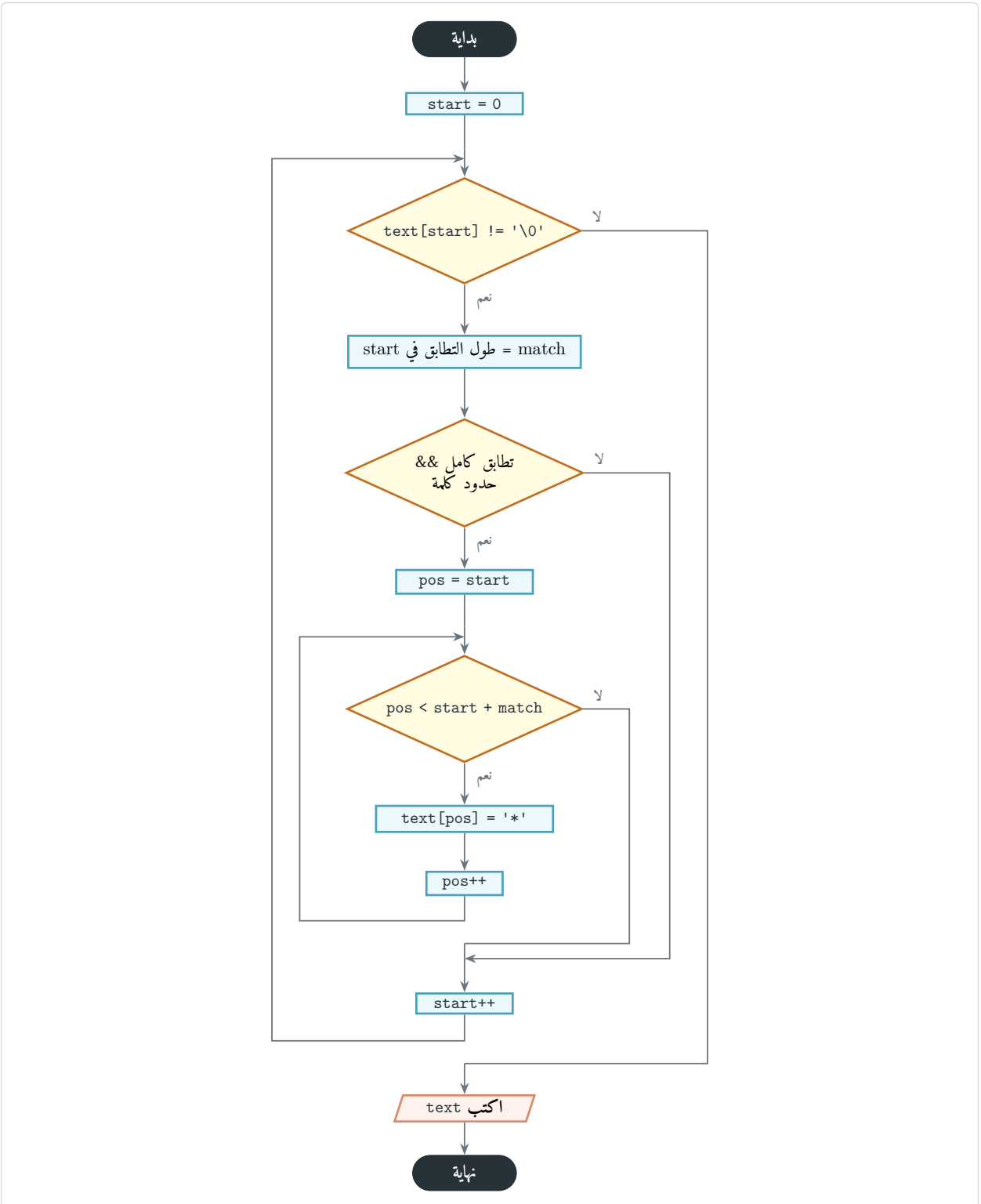
```c
#include <stdio.h>

int main(void) {
    char text[] = "This seller is a scam, total scam! scampi is fine.";
    char banned[] = "scam";
    int start, match, pos, before, after;
    for (start = 0; text[start] != '\0'; start++) {
        match = 0;
        while (banned[match] != '\0' && text[start + match] == banned[match])
            match++;
        if (banned[match] != '\0')
            continue;                          // لا تطابق
        before = (start > 0) ? text[start - 1] : ' ';
        after = text[start + match];
        // حدود كلمة: ما قبلها وما بعدها ليسا حرفين
        if (!((before >= 'a' && before <= 'z') || (before >= 'A' && before <= 'Z')) &&
            !((after >= 'a' && after <= 'z') || (after >= 'A' && after <= 'Z')))
            for (pos = start; pos < start + match; pos++)
                text[pos] = '*';
    }
    printf("%s\n", text);
    return 0;
}
```

نتيجة التنفيذ

```
This seller is a ****, total ****! scampi is fine.
```

المخطط الإنسيابي (Organigramme)



تمرين 6.21 تكرار الحروف في الأوصاف

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب **تكرار كل حرف** إنجليزي (دون تمييز بين الكبير والصغير) في نص، باستعمال جدول من 26 عدادًا، ثم يعرض الحروف الخمسة الأكثر تكرارًا.

خطوات الحل

1) `count[ch - 'a']++` بعد تحويل الحرف إلى صغير.
2) لاختيار الأكثر 5 مرات: نبحث عن الأكبر، نعرضه، ثم نجعله -1.

البرنامج بلغة C

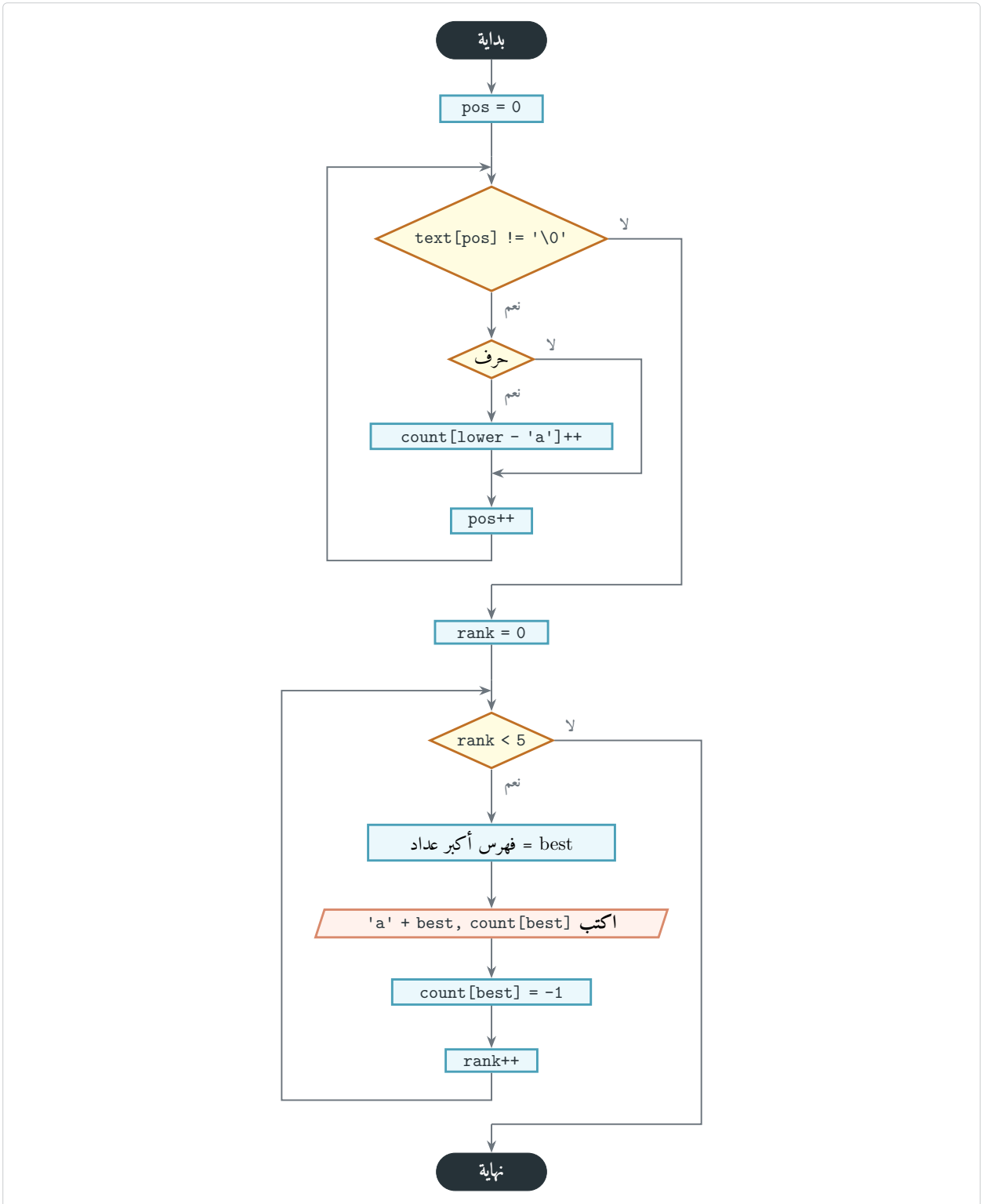
```c
#include <stdio.h>

int main(void) {
    char text[] = "Fast delivery and excellent customer service. Recommended seller!";
    int count[26] = {0}, pos, rank, letter, best;
    char ch;
    for (pos = 0; text[pos] != '\0'; pos++) {
        ch = text[pos];
        if (ch >= 'A' && ch <= 'Z') ch += 'a' - 'A';
        if (ch >= 'a' && ch <= 'z')
            count[ch - 'a']++;                // 'a' -> 0 ... 'z' -> 25
    }
    for (rank = 1; rank <= 5; rank++) {
        best = 0;
        for (letter = 1; letter < 26; letter++)
            if (count[letter] > count[best])
                best = letter;
        printf("%d. '%c': %d\n", rank, 'a' + best, count[best]);
        count[best] = -1;                     // لا نختاره مرة أخرى
    }
    return 0;
}
```

نتيجة التنفيذ

```
1. 'e': 13
2. 'l': 5
3. 'r': 5
4. 'c': 4
5. 'd': 4
```

المخطط الإنسيابي (Organigramme)



تمرين 6.22 أسماء متجانسة الحروف

تجارة إلكترونية

متوسط

لاكتشاف الحسابات المكررة بأسماء مقلوبة الحروف، اكتب برنامجًا يتحقق إن كانت كلمتان **متجانستين** (نفس الحروف بنفس التكرار، بترتيب مختلف) باستعمال جدول عدادات واحد.

خطوات الحل

1) الكلمة الأولى تزيد العدادات، والثانية تنقصها.
2) الكلمتان متجانستان إذا عادت كل العدادات إلى 0.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char first[30], second[30];
    int count[26] = {0}, pos, letter, ok = 1;
    scanf("%29s %29s", first, second);
    for (pos = 0; first[pos] != '\0'; pos++)
        count[first[pos] - 'a']++;        // +1 لكل حرف من الأولى
    for (pos = 0; second[pos] != '\0'; pos++)
        count[second[pos] - 'a']--;       // -1 لكل حرف من الثانية
    for (letter = 0; letter < 26; letter++)
        if (count[letter] != 0)
            ok = 0;
    printf("%s / %s: %s\n", first, second, ok ? "anagrams" : "different");
    return 0;
}
```

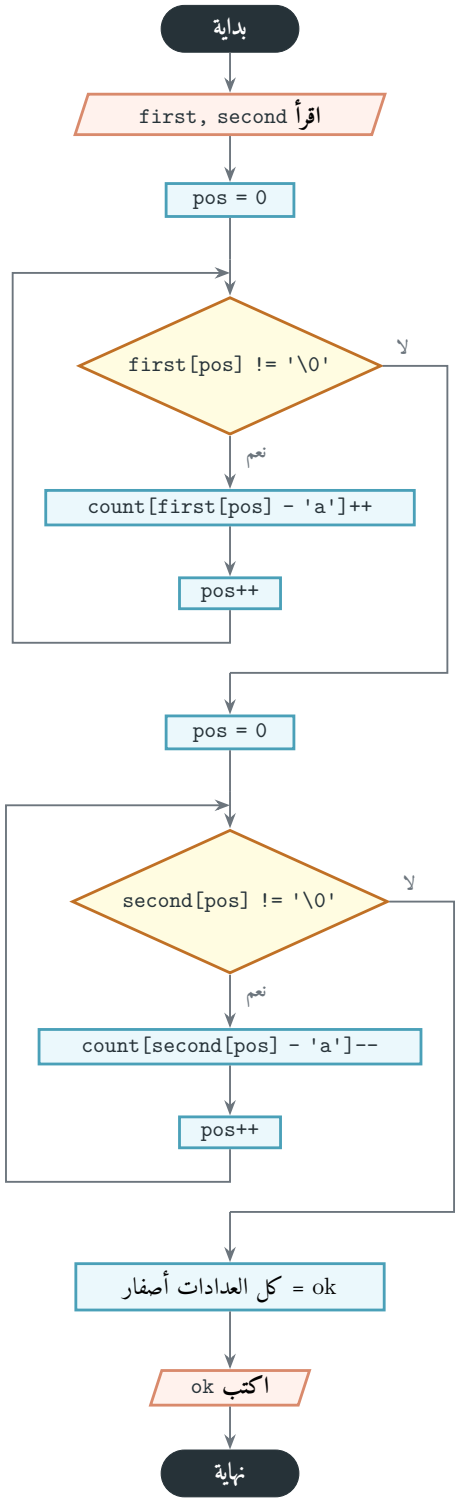
المدخلات

```
listen silent
```

نتيجة التنفيذ

```
listen / silent: anagrams
```

المخطط الإنسيابي (Organigramme)



تمرين 6.23 تشفير رمز القسيمة بإزاحة قيصر

تجارة إلكترونية

متوسط

لإخفاء رموز القسائم في الرسائل التسويقية نستعمل **تشفير قيصر**: كل حرف يُزاح بـ `shift` موضعًا في الأبجدية (مع الدوران بعد Z)، والأرقام تُزاح في 0..9. اكتب برنامجًا يشفر ثم يفك التشفير للتحقق.

خطوات الحل

1) حرف كبير: `'A' + (ch - 'A' + shift) % 26`.
2) رقم: `'0' + (ch - '0' + shift) % 10`.
3) فك التشفير: إزاحة بـ `26 - shift` و`10 - shift % 10`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char code[30];
    int shift, pos, round, step;
    scanf("%29s %d", code, &shift);
    for (round = 0; round < 2; round++) {
        // الجولة 0: تشفير، الجولة 1: فك التشفير
        step = (round == 0) ? shift : 26 - shift % 26;
        for (pos = 0; code[pos] != '\0'; pos++) {
            if (code[pos] >= 'A' && code[pos] <= 'Z')
                code[pos] = 'A' + (code[pos] - 'A' + step) % 26;   // دوران
            else if (code[pos] >= '0' && code[pos] <= '9')
                code[pos] = '0' + (code[pos] - '0' + (round == 0 ? shift : 10 - shift % 10)) % 10;
        }
        printf("%s: %s\n", round == 0 ? "Encrypted" : "Decrypted", code);
    }
    return 0;
}
```

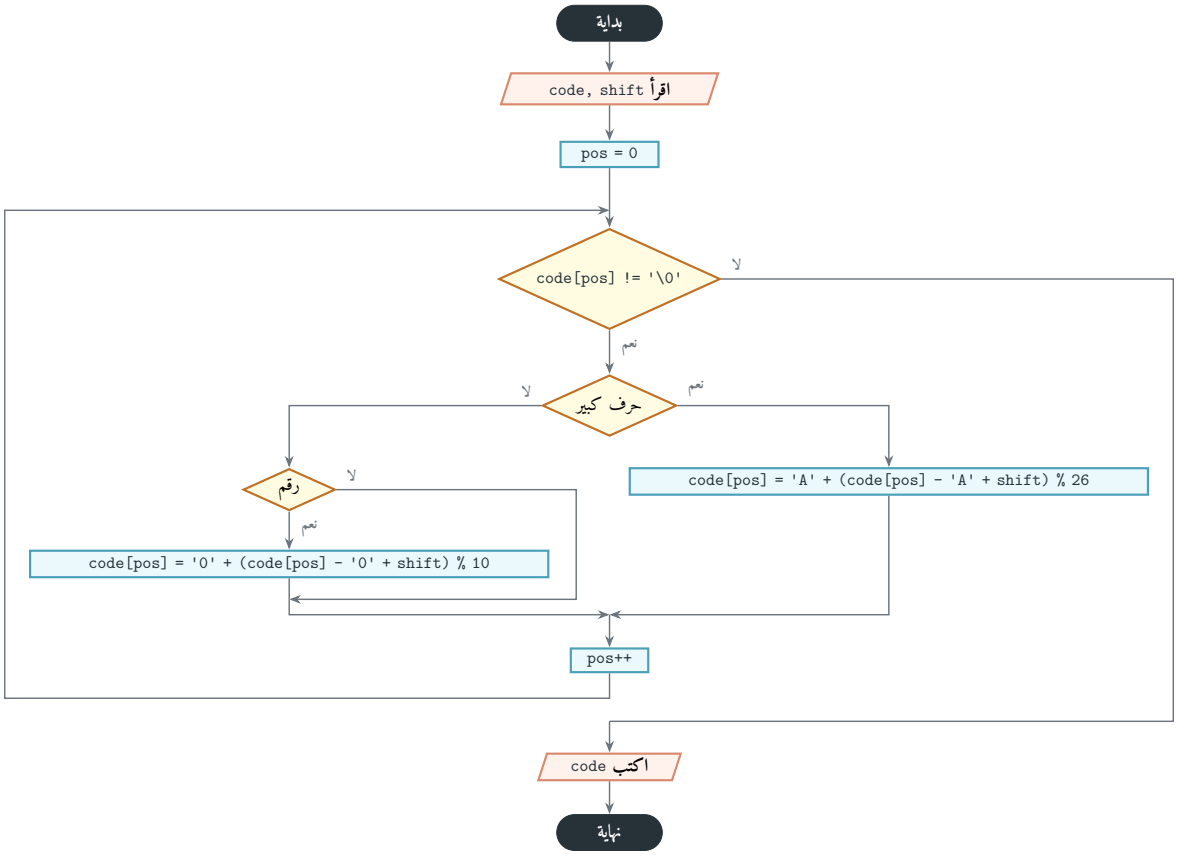
المدخلات

```
SALE2026 3
```

نتيجة التنفيذ

```
Encrypted: VDOH5359
Decrypted: SALE2026
```

المخطط الإنسيابي (Organigramme)



تمرين 6.24 قوة كلمة المرور

تجارة إلكترونية

متوسط

اكتب برنامجًا يقيّم **قوة كلمة المرور**: نقطة لكل شرط محقق (الطول ≥ 8، حرف كبير، حرف صغير، رقم، رمز خاص)، ثم يعرض "ضعيفة" (0-2)، "متوسطة" (3-4)، أو "قوية" (5) مع الشروط الناقصة.

خطوات الحل

1) مرور واحد يرفع الأعلام `hasUpper` و`hasLower` و`hasDigit` و`hasSymbol` ويحسب الطول.
2) `score` = عدد الشروط المحققة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char password[50], ch;
    int pos, length = 0, hasUpper = 0, hasLower = 0, hasDigit = 0, hasSymbol = 0, score;
    scanf("%49s", password);
    for (pos = 0; password[pos] != '\0'; pos++) {
        ch = password[pos];
        if (ch >= 'A' && ch <= 'Z')      hasUpper = 1;
        else if (ch >= 'a' && ch <= 'z') hasLower = 1;
        else if (ch >= '0' && ch <= '9') hasDigit = 1;
        else                             hasSymbol = 1;   // ! @ # ...
        length++;
    }
    score = (length >= 8) + hasUpper + hasLower + hasDigit + hasSymbol;  // كل شرط 0 أو 1
    printf("Score %d/5: %s\n", score, score == 5 ? "strong" : score >= 3 ? "medium" : "weak");
```

```c
    if (length < 8) printf("- use at least 8 characters\n");
    if (!hasUpper)  printf("- add an uppercase letter\n");
    if (!hasLower)  printf("- add a lowercase letter\n");
    if (!hasDigit)  printf("- add a digit\n");
    if (!hasSymbol) printf("- add a symbol\n");
    return 0;
}
```

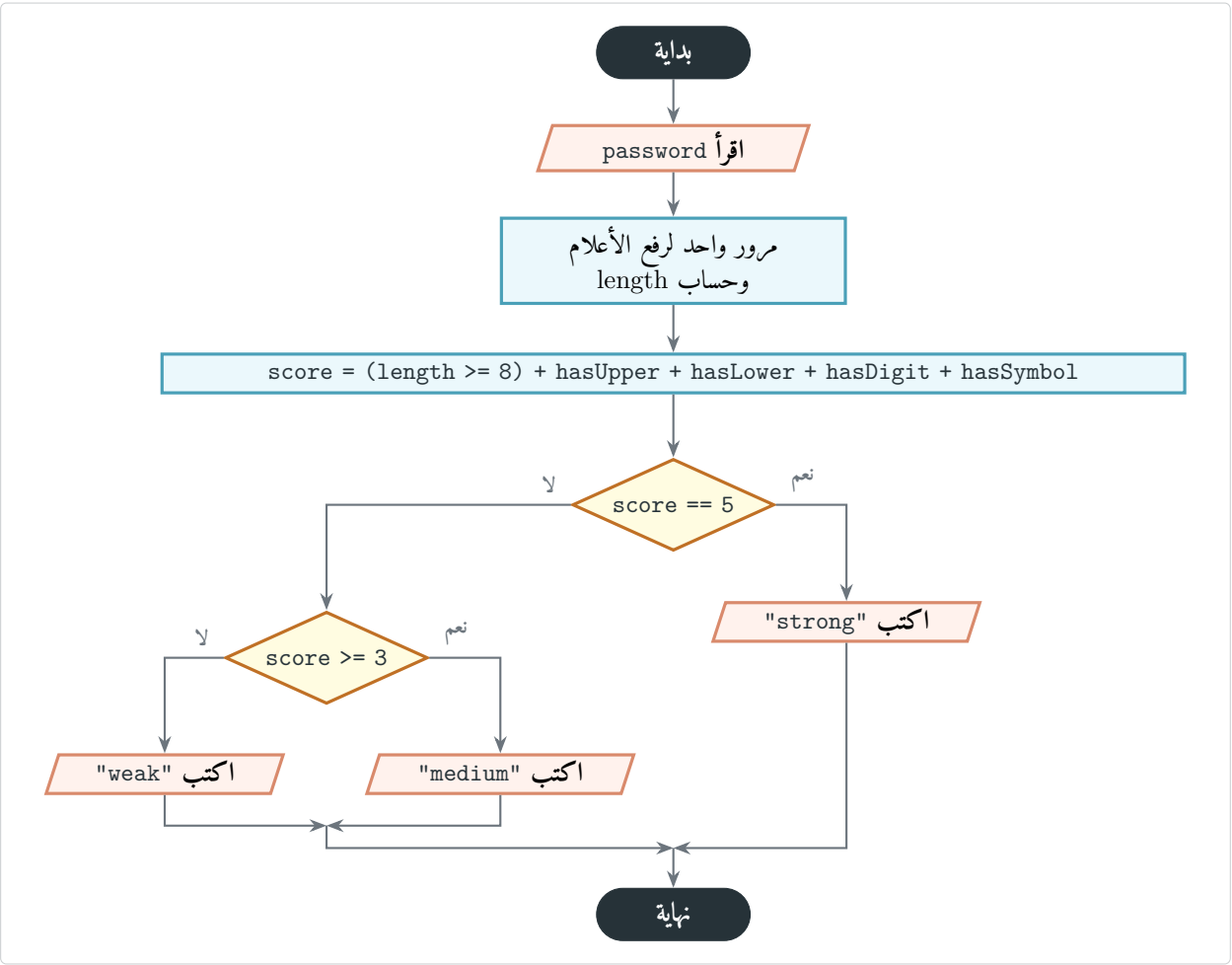
المدخلات

```
Shop2026
```

نتيجة التنفيذ

```
Score 4/5: medium
- add a symbol
```

المخطط الإنسيابي (Organigramme)



تمرين 6.25 التحقق من رقم الهاتف

تجارة إلكترونية ★★ متوسط

رقم الهاتف النقال الجزائري يتكون من **10 أرقام** ويبدأ بـ 05 أو 06 أو 07، وقد يُكتب بالصيغة الدولية +213 متبوعة بـ 9 أرقام (دون الصفر). اكتب برنامجًا يتحقق من الرقم ويعرضه بالصيغة الموحدة 0XXXXXXXXX.

خطوات الحل

1) إذا بدأ بـ +213: نتخطى 4 حروف ونضيف 0 في البداية.
2) نتحقق أن الباقي أرقام فقط وأن الطول الكلي 10.
3) الحرف الثاني يجب أن يكون 5 أو 6 أو 7.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char phone[20], normal[20];
    int read = 0, write = 0, valid = 1;
    scanf("%19s", phone);
    if (phone[0] == '+' && phone[1] == '2' && phone[2] == '1' && phone[3] == '3') {
        normal[write++] = '0';            // الصيغة الدولية: نضيف الصفر
        read = 4;
    }
    for (; phone[read] != '\0' && write < 19; read++) {
        if (phone[read] < '0' || phone[read] > '9')
            valid = 0;                    // حرف غير رقمي
        normal[write++] = phone[read];
    }
    normal[write] = '\0';
    if (write != 10 || normal[0] != '0' || normal[1] < '5' || normal[1] > '7')
        valid = 0;
    if (valid)
        printf("Valid mobile number: %s\n", normal);
    else
        printf("Invalid number: %s\n", phone);
    return 0;
}
```

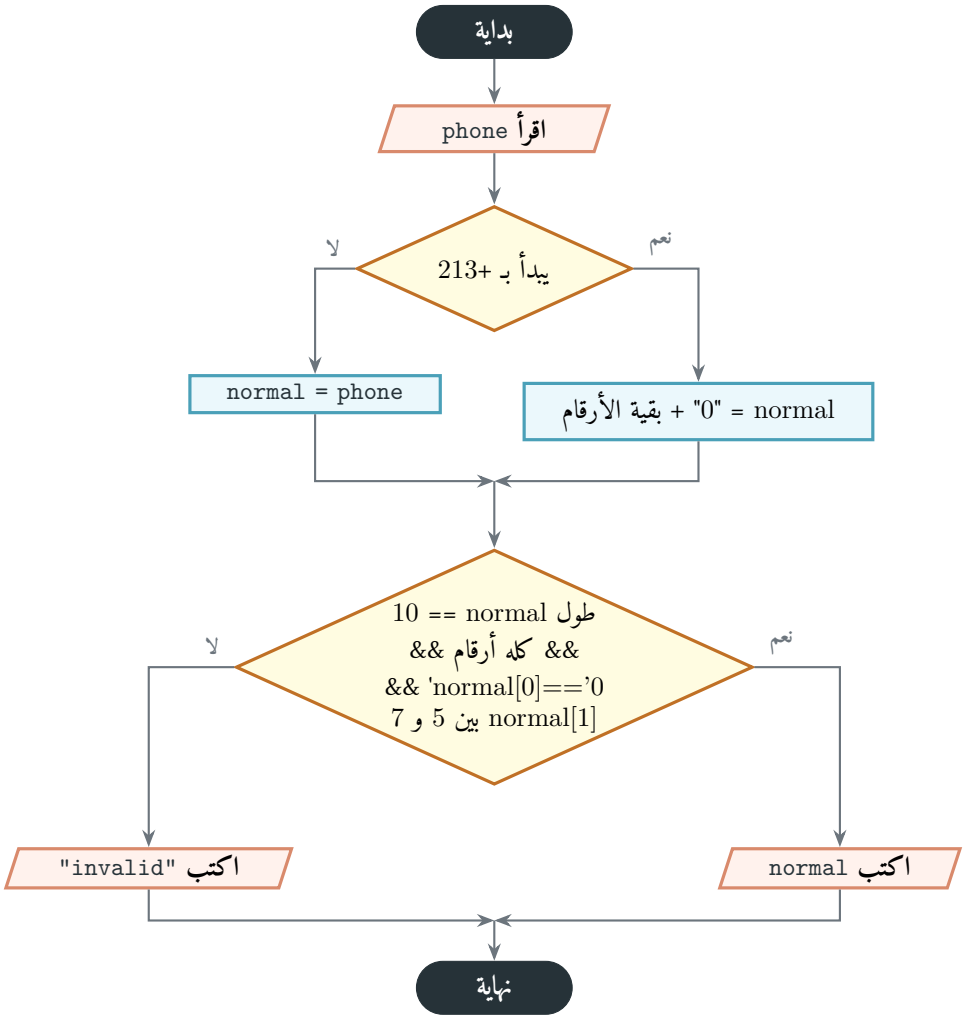
المدخلات

```
+213661234567
```

نتيجة التنفيذ

```
Valid mobile number: 0661234567
```

المخطط الانسيابي (Organigramme)



تمرين 6.26 تفكيك سطر CSV

علم البيانات ★★ متوسط

تصدير المنتجات يعطي أسطرًا بصيغة `id;name;price;qty`. اكتب برنامجًا يفكك سطرًا إلى حقوله (دون `strtok`)، ويحول السعر والكمية إلى أعداد، ويحسب قيمة المخزون.

خطوات الحل

1) تمر على السطر وتنسخ الحروف في الحقل الحالي `field[fieldNo]`.
2) عند `;` تنهي الحقل بـ `'\0'` وتنتقل إلى الحقل التالي.
3) تحول الحقل 0 و3 إلى `int` والحقل 2 إلى `double` يدويًا.

البرنامج بلغة C

```c
#include <stdio.h>

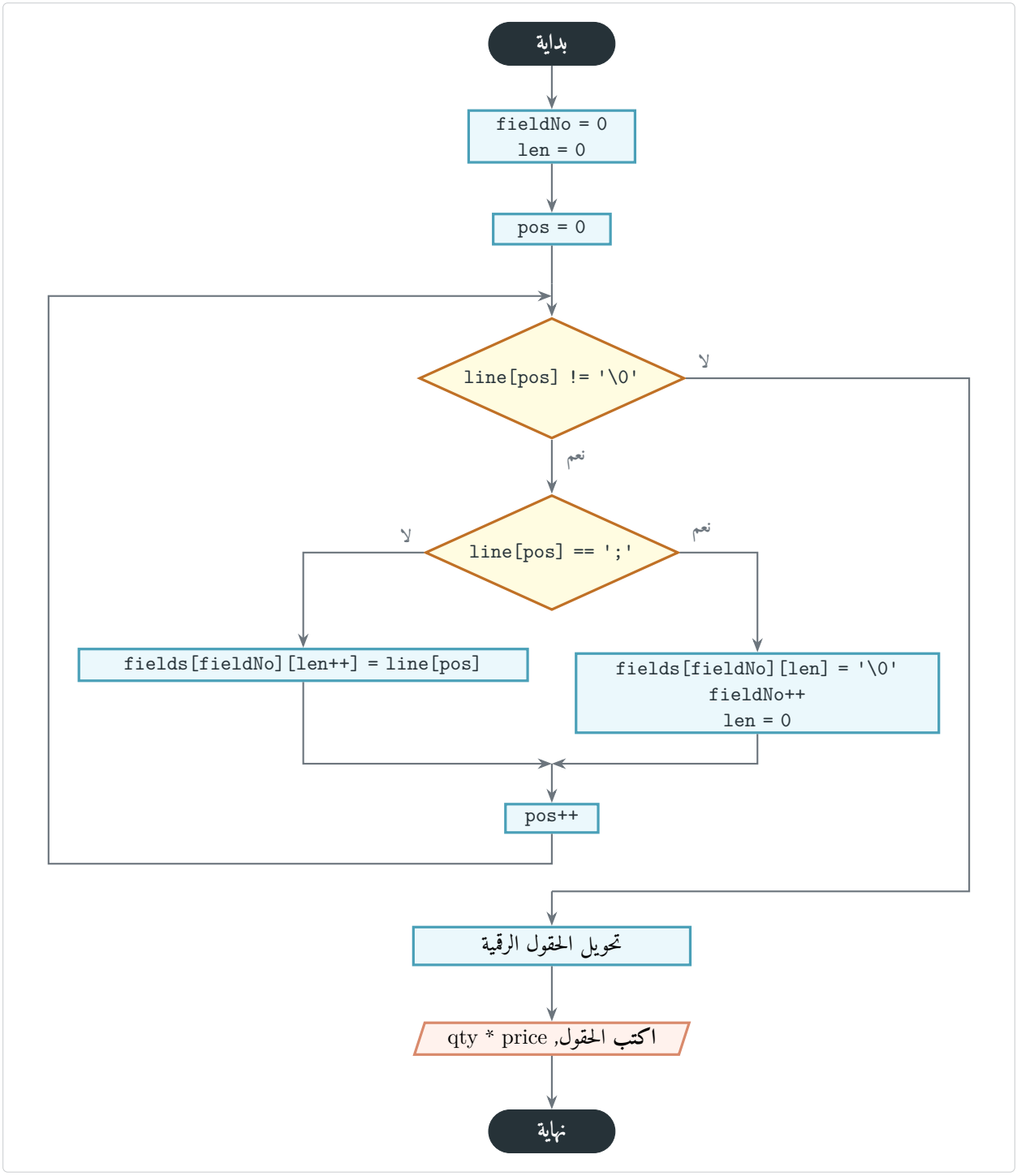
int main(void) {
    char line[] = "1042;USB-C Charger;2450.50;37";
    char fields[4][40];
    int pos, fieldNo = 0, length = 0, id = 0, qty = 0, afterDot = 0;
```

```c
    double price = 0, scale = 1;
    for (pos = 0; line[pos] != '\0'; pos++) {
        if (line[pos] == ';') {
            fields[fieldNo][length] = '\0';   // نهاية الحقل
            fieldNo++;
            length = 0;
        } else {
            fields[fieldNo][length++] = line[pos];
        }
    }
    fields[fieldNo][length] = '\0';           // الحقل الأخير
    for (pos = 0; fields[0][pos] != '\0'; pos++)
        id = id * 10 + (fields[0][pos] - '0');
    for (pos = 0; fields[3][pos] != '\0'; pos++)
        qty = qty * 10 + (fields[3][pos] - '0');
    for (pos = 0; fields[2][pos] != '\0'; pos++) {   // السعر العشري
        if (fields[2][pos] == '.') { afterDot = 1; continue; }
        if (!afterDot) price = price * 10 + (fields[2][pos] - '0');
        else { scale /= 10; price += (fields[2][pos] - '0') * scale; }
    }
    printf("id=%d name=\"%s\" price=%.2f qty=%d\n", id, fields[1], price, qty);
    printf("Stock value = %.2f DA\n", price * qty);
    return 0;
}
```

نتيجة التنفيذ

```
id=1042 name="USB-C Charger" price=2450.50 qty=37
Stock value = 90668.50 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 6.27 ترتيب أسماء المنتجات أبجديًا

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يرتب أسماء 6 منتجات **أبجديًا** (ترتيب القاموس) بالترتيب بالفقاعات، مع مقارنة يدوية بين سلسلتين بدل `strcmp`، وتبديل الأسماء حرفًا بحرف.

خطوات الحل

1) مقارنة سلسلتين: تتقدم ما دام الحرفان متساويين وغير نهائيين؛ الفرق `a[pos] - b[pos]` يحدد الترتيب.

2) إذا كان الفرق موجباً نبدل الاسمين (نسخ عبر جدول مؤقت).

البرنامج بلغة C

```c
#include <stdio.h>

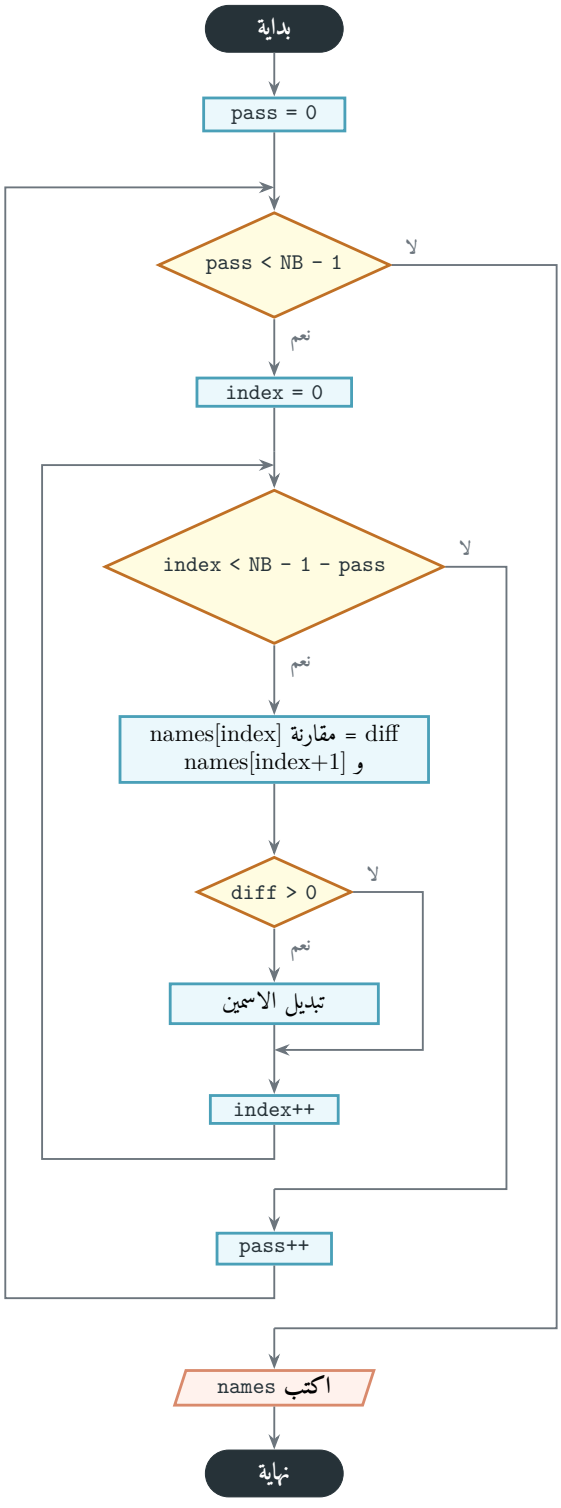
#define NB 6
#define LEN 30

int main(void) {
    char names[NB][LEN] = {"Mouse", "Keyboard", "Monitor", "Cable", "Microphone", "Camera"};
    char tmp[LEN];
    int pass, index, pos, diff;
    for (pass = 0; pass < NB - 1; pass++) {
        for (index = 0; index < NB - 1 - pass; index++) {
            pos = 0;                            // مقارنة يدوية
            while (names[index][pos] == names[index + 1][pos] && names[index][pos] != '\0')
                pos++;
            diff = names[index][pos] - names[index + 1][pos];
            if (diff > 0) {                     // الأول بعد الثاني في القاموس
                for (pos = 0; (tmp[pos] = names[index][pos]) != '\0'; pos++) ;
                for (pos = 0; (names[index][pos] = names[index + 1][pos]) != '\0'; pos++) ;
                for (pos = 0; (names[index + 1][pos] = tmp[pos]) != '\0'; pos++) ;
            }
        }
    }
    for (index = 0; index < NB; index++)
        printf("%d. %s\n", index + 1, names[index]);
    return 0;
}
```

نتيجة التنفيذ

```
1. Cable
2. Camera
3. Keyboard
4. Microphone
5. Monitor
6. Mouse
```

المخطط الإنسيابي (Organigramme)



تمرين 6.28 ضغط الرموز بالترميز الطولي

علم البيانات ★★ متوسط

سجل الأحداث يحتوي سلاسل طويلة من نفس الحرف مثل `"VVVVVCCCAAAAAAAB"` (V زيارة، C سلة، A إضافة، B شراء). اكتب برنامجًا يضغطها **بالترميز الطولي** (RLE): `5V3C7A1B`، ويعرض نسبة الضغط.

خطوات الحل

1) نعد طول السلسلة الحالية من نفس الحرف run.
2) عند تغير الحرف: نكتب العدد (قد يتكون من عدة أرقام) ثم الحرف.

البرنامج بلغة C

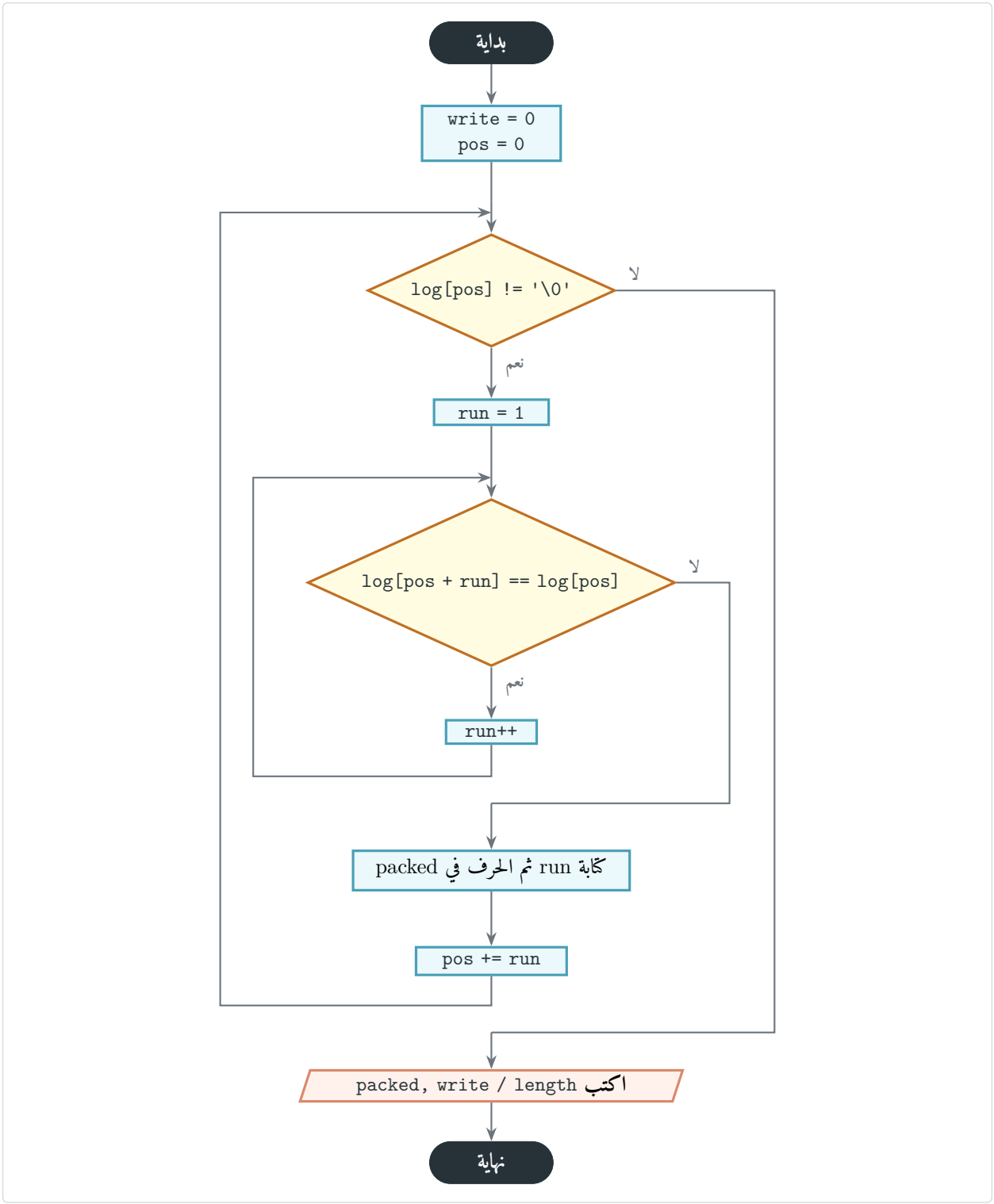
```c
#include <stdio.h>

int main(void) {
    char events[] = "VVVVVVVVVVVVCCCAAAAAAABVVVVC";
    char packed[80], digits[10];
    int pos = 0, run, write = 0, nbDigits;
    while (events[pos] != '\0') {
        run = 1;
        while (events[pos + run] == events[pos])  // طول السلسلة المتكررة
            run++;
        nbDigits = 0;
        do {                                      // العدد قد يكون > 9
            digits[nbDigits++] = '0' + run % 10;
            run /= 10;
        } while (run > 0);
        while (nbDigits > 0)
            packed[write++] = digits[--nbDigits];
        packed[write++] = events[pos];
        while (events[pos] == packed[write - 1])
            pos++;                                // نتجاوز السلسلة
    }
    packed[write] = '\0';
    printf("Original: %s (%d)\nPacked:   %s (%d)\n", events, pos, packed, write);
    printf("Ratio = %.1f %%\n", write * 100.0 / pos);
    return 0;
}
```

نتيجة التنفيذ

```
Original: VVVVVVVVVVVVCCCAAAAAAABVVVVC (28)
Packed:   12V3C7A1B4V1C (13)
Ratio = 46.4 %
```

المخطط الإنسيابي (Organigramme)



تمرين 6.29 إخفاء رقم البطاقة

تجارة إلكترونية ★★ متوسط

لا يجوز عرض رقم البطاقة كاملًا في صفحة التأكيد. اكتب برنامجًا يأخذ رقمًا من 16 رقمًا ويعرضه بالصيغة 1234 **** **** **** (آخر 4 أرقام فقط ظاهرة، وفراغ كل 4 خانات).

خطوات الحل

1) الطول يجب أن يكون 16.
2) لكل موضع pos: إذا كان pos < 12 نكتب * وإلا الرقم.
3) نضيف فراغًا بعد كل 4 خانات ما عدا الأخيرة.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char card[20], masked[25];
    int pos, write = 0, length = 0;
    scanf("%19s", card);
    while (card[length] != '\0') length++;
    if (length != 16) {
        printf("Invalid card length\n");
        return 0;
    }
    for (pos = 0; pos < 16; pos++) {
        masked[write++] = (pos < 12) ? '*' : card[pos];    // آخر 4 أرقام فقط
        if (pos % 4 == 3 && pos < 15)
            masked[write++] = ' ';                         // مجموعات من 4
    }
    masked[write] = '\0';
    printf("Card: %s\n", masked);
    return 0;
}
```

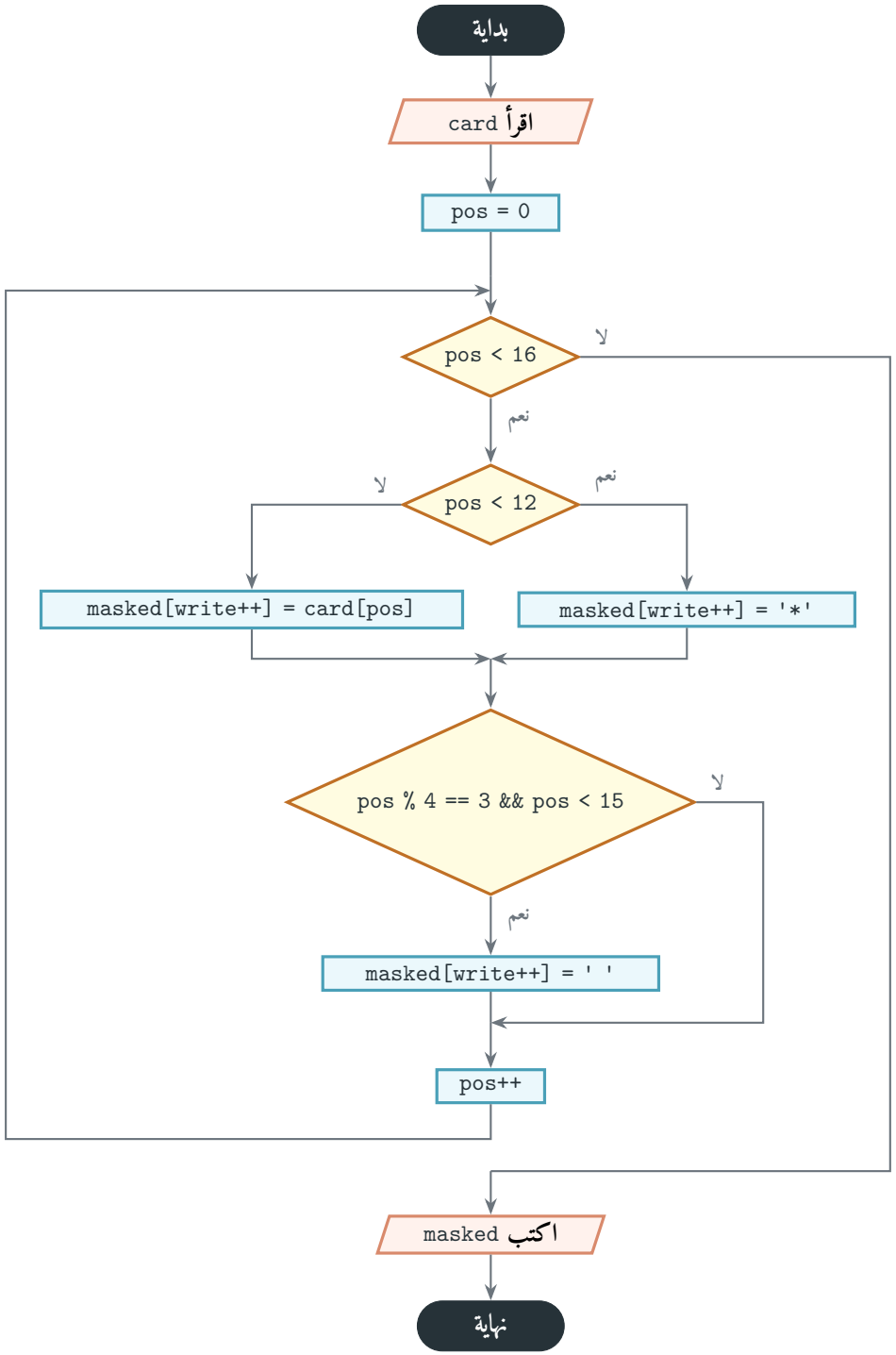
المدخلات

```
4539578763621486
```

نتيجة التنفيذ

```
Card: **** **** **** 1486
```

المخطط الإنسيابي (Organigramme)



تمرين 6.30 تنسيق المبالغ بفواصل الآلاف

تجارة إلكترونية ★★ متوسط

اكتب برنامجًا يحول مبلغًا صحيحًا مثل `12500750` إلى نص منسق `12 500 750 DA` (فراغ كل ثلاثة أرقام من اليمين) لعرضه في الفاتورة.

خطوات الحل

1) نحول العدد إلى أرقام في جدول (من اليمين).

2) عند الكتابة من اليسار: نضيف فراغًا قبل كل رقم يكون عدد الأرقام الباقية بعده مضاعفًا لـ 3.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    long amount;
    int digits[20], count = 0, pos, write = 0;
    char text[40];
    scanf("%ld", &amount);
    do {
        digits[count++] = amount % 10;    // الأرقام من الآحاد
        amount /= 10;
    } while (amount > 0);
    for (pos = count - 1; pos >= 0; pos--) {
        text[write++] = '0' + digits[pos];
        if (pos > 0 && pos % 3 == 0)      // بقي مضاعف لـ 3 أرقام
            text[write++] = ' ';
    }
    text[write] = '\0';
    printf("%s DA\n", text);
    return 0;
}
```

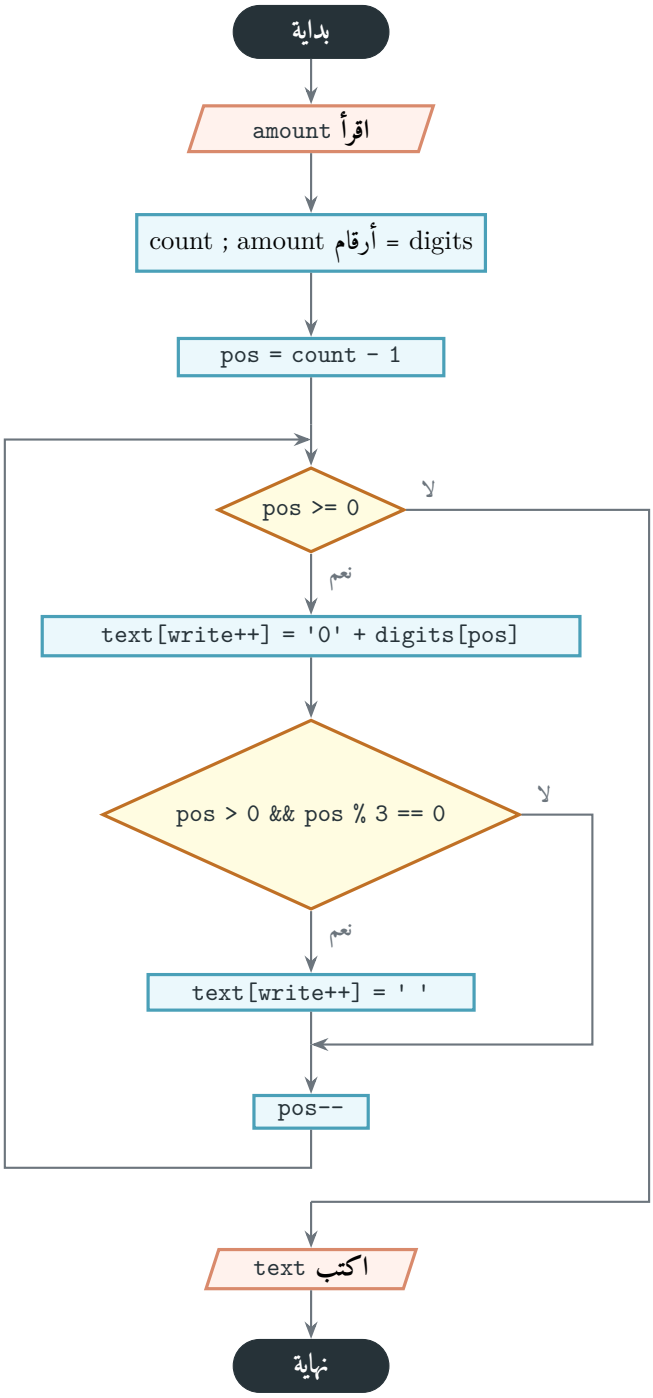
المدخلات

```
12500750
```

نتيجة التنفيذ

```
12 500 750 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 6.31 قراءة تاريخ بصيغة ISO

علم البيانات ★★ متوسط

تخزن قاعدة البيانات التواريخ بصيغة YYYY-MM-DD. اكتب برنامجًا يقرأ تاريخًا نصيًا، يتحقق من الصيغة (الطول 10، الشرطتان في الموضعين 4 و7، والباقي أرقام)، ويستخرج السنة والشهر واليوم كأعداد ويعرضه بصيغة DD/MM/YYYY.

خطوات الحل

1) تحقق من الطول والشرطتين والأرقام.

2) نحول كل جزء بحلقة صغيرة: `value = value * 10 + digit`.
3) نتحقق من مجال الشهر واليوم.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char text[20];
    int pos, length = 0, valid = 1, year = 0, month = 0, day = 0;
    scanf("%19s", text);
    while (text[length] != '\0') length++;
    if (length != 10 || text[4] != '-' || text[7] != '-')
        valid = 0;
    for (pos = 0; pos < length && valid; pos++)
        if (pos != 4 && pos != 7 && (text[pos] < '0' || text[pos] > '9'))
            valid = 0;
    if (!valid) {
        printf("Bad format\n");
        return 0;
    }
    for (pos = 0; pos < 4; pos++)  year = year * 10 + (text[pos] - '0');
    for (pos = 5; pos < 7; pos++)  month = month * 10 + (text[pos] - '0');
    for (pos = 8; pos < 10; pos++) day = day * 10 + (text[pos] - '0');
    if (month >= 1 && month <= 12 && day >= 1 && day <= 31)
        printf("%02d/%02d/%d\n", day, month, year);
    else
        printf("Bad values\n");
    return 0;
}
```

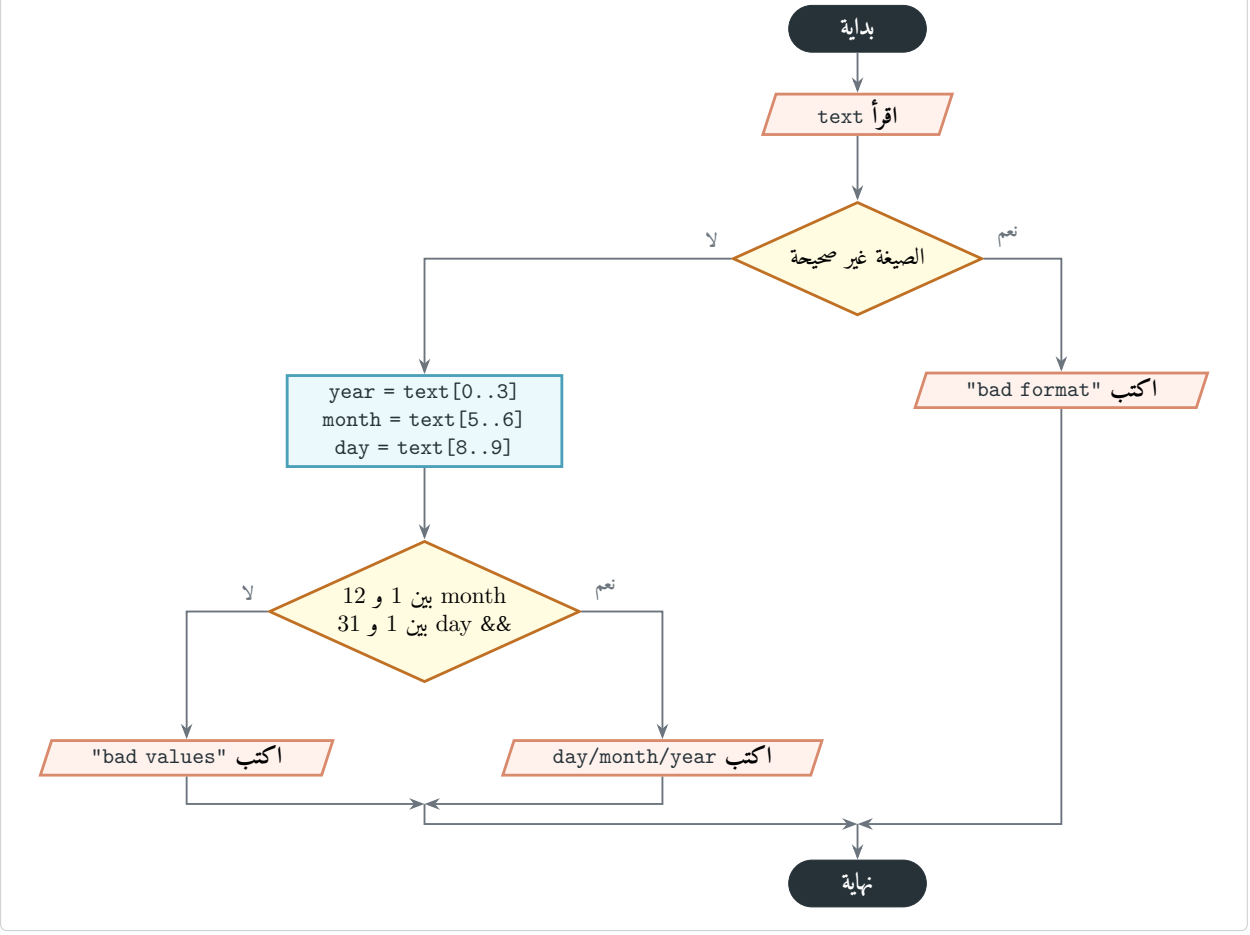
المدخلات

```
2026-11-02
```

نتيجة التنفيذ

```
02/11/2026
```

المخطط الإنسيابي (Organigramme)



تمرين 6.32 استخراج الوسوم من التعليقات

علم البيانات ★★ متوسط

اكتب برنامجًا يستخرج كل الوسوم (hashtags) من منشور تسويقي: كل كلمة تبدأ بـ # وتتكون من حروف وأرقام و_، ويعرضها واحدًا في كل سطر بحروف صغيرة.

خطوات الحل

1) عند #: نبدأ وسمًا جديدًا ونجمع الحروف المسموحة بعده.
2) نتوقف عند أول حرف غير مسموح.
3) نتجاهل # الوحيدة (وسم فارغ).

البرنامج بلغة C

```c
#include <stdio.h>

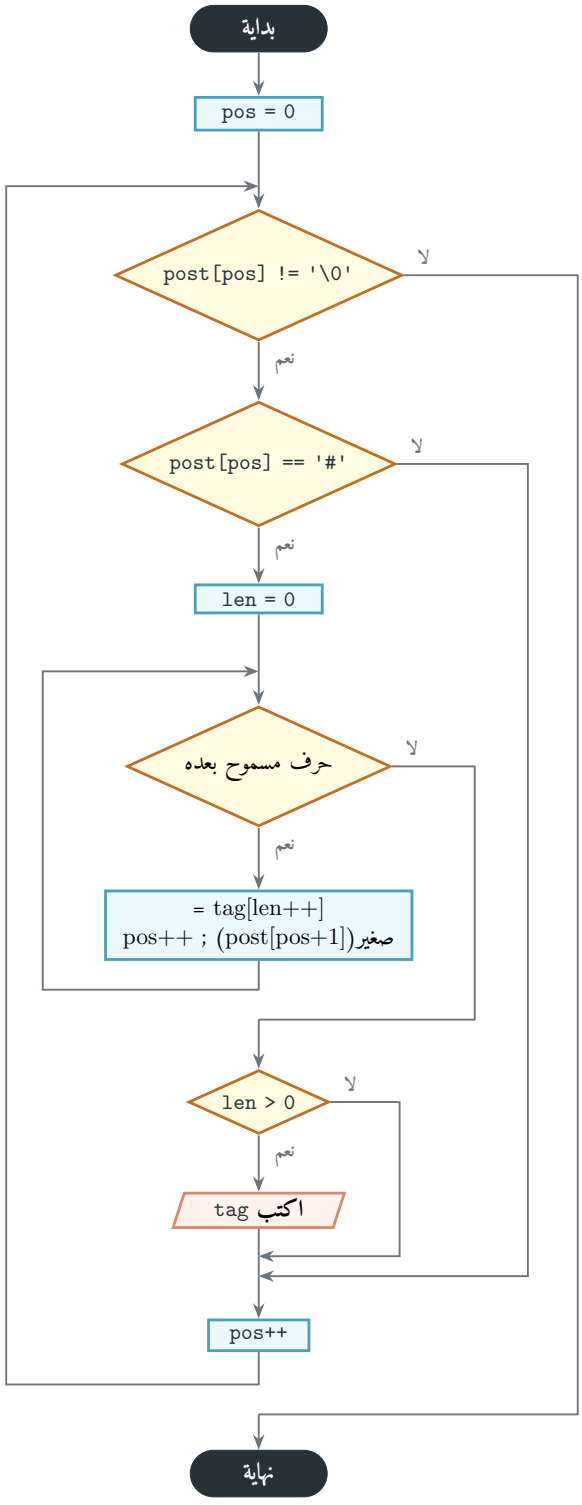
int main(void) {
    char post[] = "New drop! #BlackFriday deals on #Phones and #Smart_TV, # not a tag #2026Sale";
    char tag[40], ch;
    int pos, length, count = 0;
    for (pos = 0; post[pos] != '\0'; pos++) {
        if (post[pos] != '#')
            continue;
        length = 0;
        while (1) {                         // نجمع حروف الوسم
```

```
            ch = post[pos + 1];
            if (ch >= 'A' && ch <= 'Z') ch += 'a' - 'A';
            if (!((ch >= 'a' && ch <= 'z') || (ch >= '0' && ch <= '9') || ch == '_'))
                break;
            tag[length++] = ch;
            pos++;
        }
        tag[length] = '\0';
        if (length > 0) {
            printf("#%s\n", tag);
            count++;
        }
    }
    printf("%d hashtags\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
#blackfriday
#phones
#smart_tv
#2026sale
4 hashtags
```

المخطط الإنسيابي (Organigramme)



تمرين 6.33 تحليل المشاعر بالكلمات المفتاحية

علم البيانات ★★ متوسط

اكتب برنامجًا يعطي تعليقًا **درجة مشاعر**: +1 لكل كلمة من قائمة الكلمات الإيجابية و-1 لكل كلمة سلبية (مقارنة كلمات كاملة بحروف صغيرة)، ثم يصنف التعليق.

خطوات الحل

1) نقطع النص إلى كلمات (حروف فقط) ونحولها إلى حروف صغيرة.
2) نقارن كل كلمة بكل كلمة في القائمتين (مقارنة يدوية).
3) الدرجة > 0 إيجابي، < 0 سلبي، وإلا محايد.

البرنامج بلغة C

```c
#include <stdio.h>

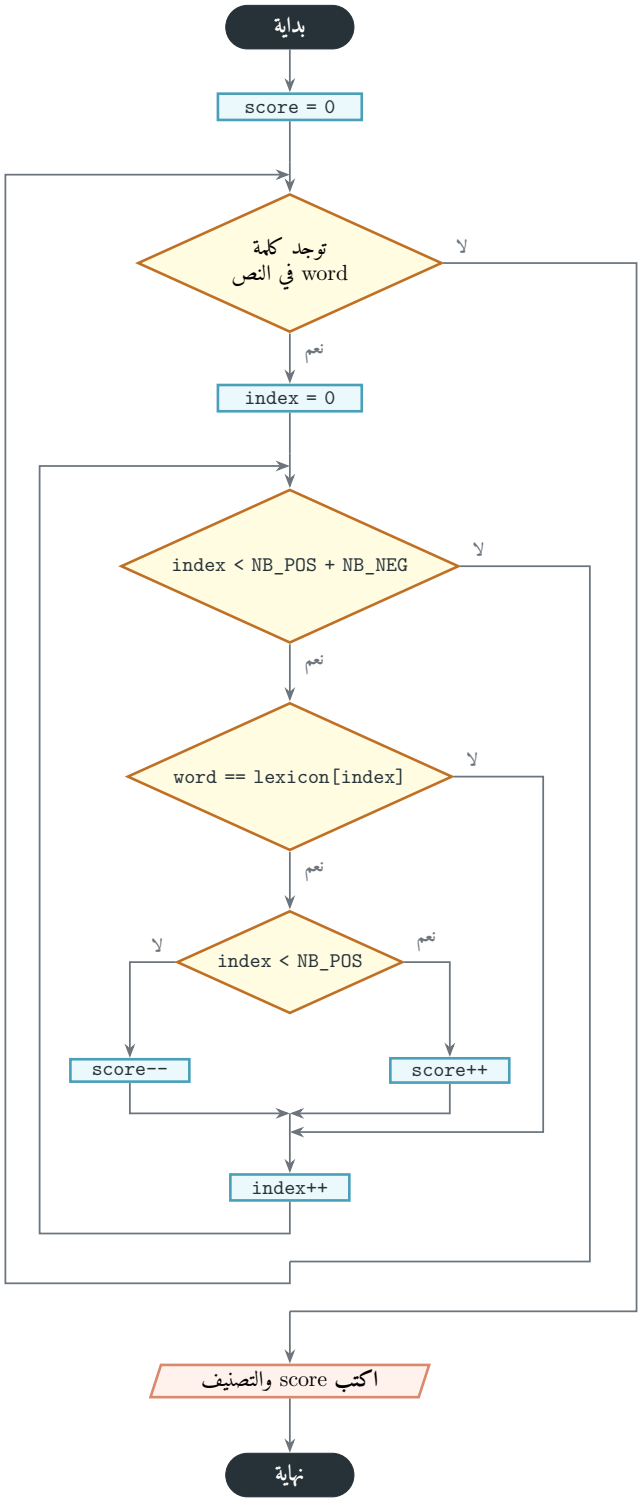
#define NB_POS 5
#define NB_NEG 5

int main(void) {
    char review[] = "Good product, fast delivery but the box was broken. Excellent price!";
    // الكلمات 0..4 إيجابية، و 5..9 سلبية
    char lexicon[NB_POS + NB_NEG][12] = {"good", "fast", "excellent", "great", "cheap",
                                         "bad", "slow", "broken", "late", "fake"};
    char word[30], ch;
    int pos = 0, length, index, cmp, score = 0;
    while (review[pos] != '\0') {
        length = 0;
        while (1) {                          // نقرأ الكلمة التالية
            ch = review[pos];
            if (ch >= 'A' && ch <= 'Z') ch += 'a' - 'A';
            if (ch < 'a' || ch > 'z') break;
            word[length++] = ch;
            pos++;
        }
        word[length] = '\0';
        if (review[pos] != '\0') pos++;      // نتجاوز الفاصل
        if (length == 0) continue;
        for (index = 0; index < NB_POS + NB_NEG; index++) {
            for (cmp = 0; word[cmp] == lexicon[index][cmp] && word[cmp] != '\0'; cmp++) ;
            if (word[cmp] == lexicon[index][cmp]) {   // تطابق كامل
                score += (index < NB_POS) ? 1 : -1;
                printf("%s: %+d\n", word, (index < NB_POS) ? 1 : -1);
            }
        }
    }
    printf("Score = %d -> %s\n", score, score > 0 ? "positive" : score < 0 ? "negative" : "neutral");
    return 0;
}
```

نتيجة التنفيذ

```
good: +1
fast: +1
broken: -1
excellent: +1
Score = 2 -> positive
```

المخطط الإنسيابي (Organigramme)



تمرين 6.34 الكلمات المختلفة في تعليق

علم البيانات ★★ متوسط

اكتب برنامجًا يقطع تعليقًا إلى كلمات (حروف صغيرة، دون علامات الترقيم) ويخزن الكلمات المختلفة في جدول من السلاسل مع عدد تكرار كل منها.

خطوات الحل

1) نستخرج كل كلمة في word.

2) نبحث عنها في جدول vocab؛ إذا وُجدت نزيد عدادها، وإلا نضيفها بعداد 1.

البرنامج بلغة C

```c
#include <stdio.h>

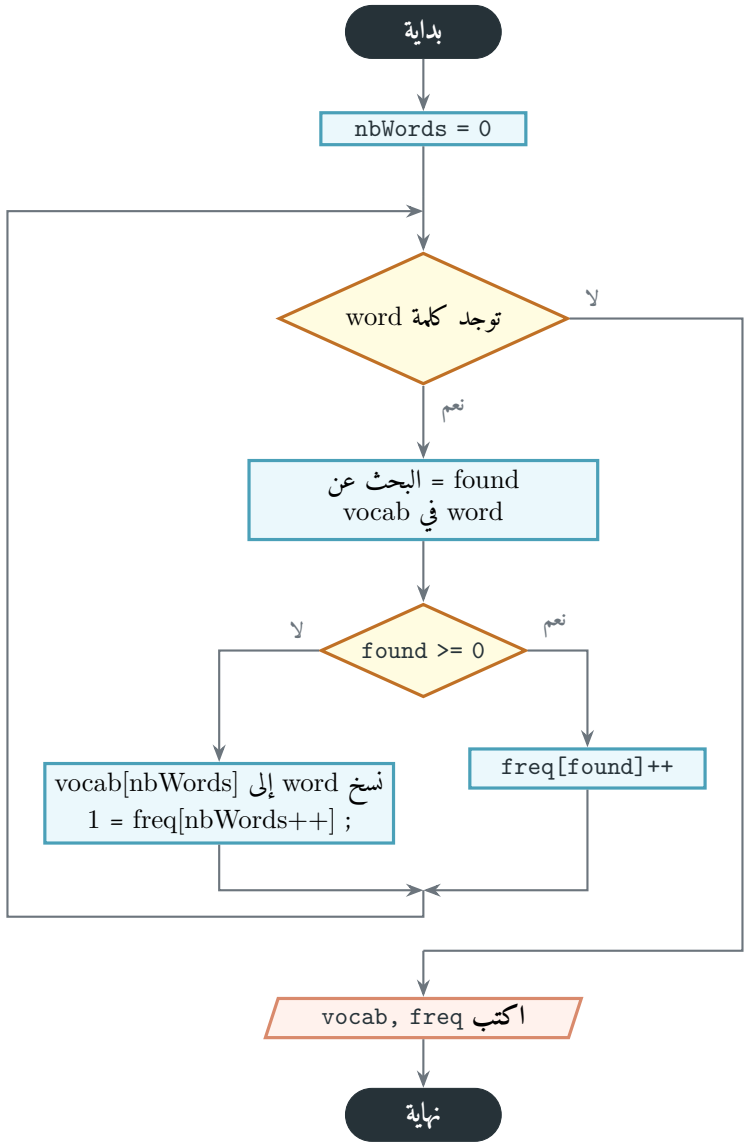
#define MAX_WORDS 30

int main(void) {
    char text[] = "The phone is great. The battery is great, the screen is OK.";
    char vocab[MAX_WORDS][20], word[20], ch;
    int freq[MAX_WORDS], nbWords = 0, pos = 0, length, index, cmp, found;
    while (text[pos] != '\0') {
        length = 0;
        while (1) {
            ch = text[pos];
            if (ch >= 'A' && ch <= 'Z') ch += 'a' - 'A';
            if (ch < 'a' || ch > 'z') break;
            word[length++] = ch;
            pos++;
        }
        word[length] = '\0';
        if (text[pos] != '\0') pos++;
        if (length == 0) continue;
        found = -1;
        for (index = 0; index < nbWords && found < 0; index++) {
            for (cmp = 0; word[cmp] == vocab[index][cmp] && word[cmp] != '\0'; cmp++) ;
            if (word[cmp] == vocab[index][cmp])
                found = index;
        }
        if (found >= 0) {
            freq[found]++;                    // كلمة معروفة
        } else {
            for (cmp = 0; (vocab[nbWords][cmp] = word[cmp]) != '\0'; cmp++) ;
            freq[nbWords++] = 1;              // كلمة جديدة
        }
    }
    for (index = 0; index < nbWords; index++)
        printf("%-8s %d\n", vocab[index], freq[index]);
    printf("%d distinct words\n", nbWords);
    return 0;
}
```

نتيجة التنفيذ

```
the      3
phone    1
is       3
great    2
battery  1
screen   1
ok       1
7 distinct words
```

المخطط الإنسيابي (Organigramme)



تمرين 6.35 أطول بادئة مشتركة لرموز المنتجات

تجارة إلكترونية ★★ متوسط

رموز منتجات نفس العائلة تشترك في **بادئة**. اكتب برنامجًا يجد **أطول بادئة مشتركة** بين كل الرموز في جدول.

خطوات الحل

1) نقارن الحرف `pos` في كل الرموز.
2) نتوقف عند أول موضع يختلف فيه رمز أو ينتهي.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

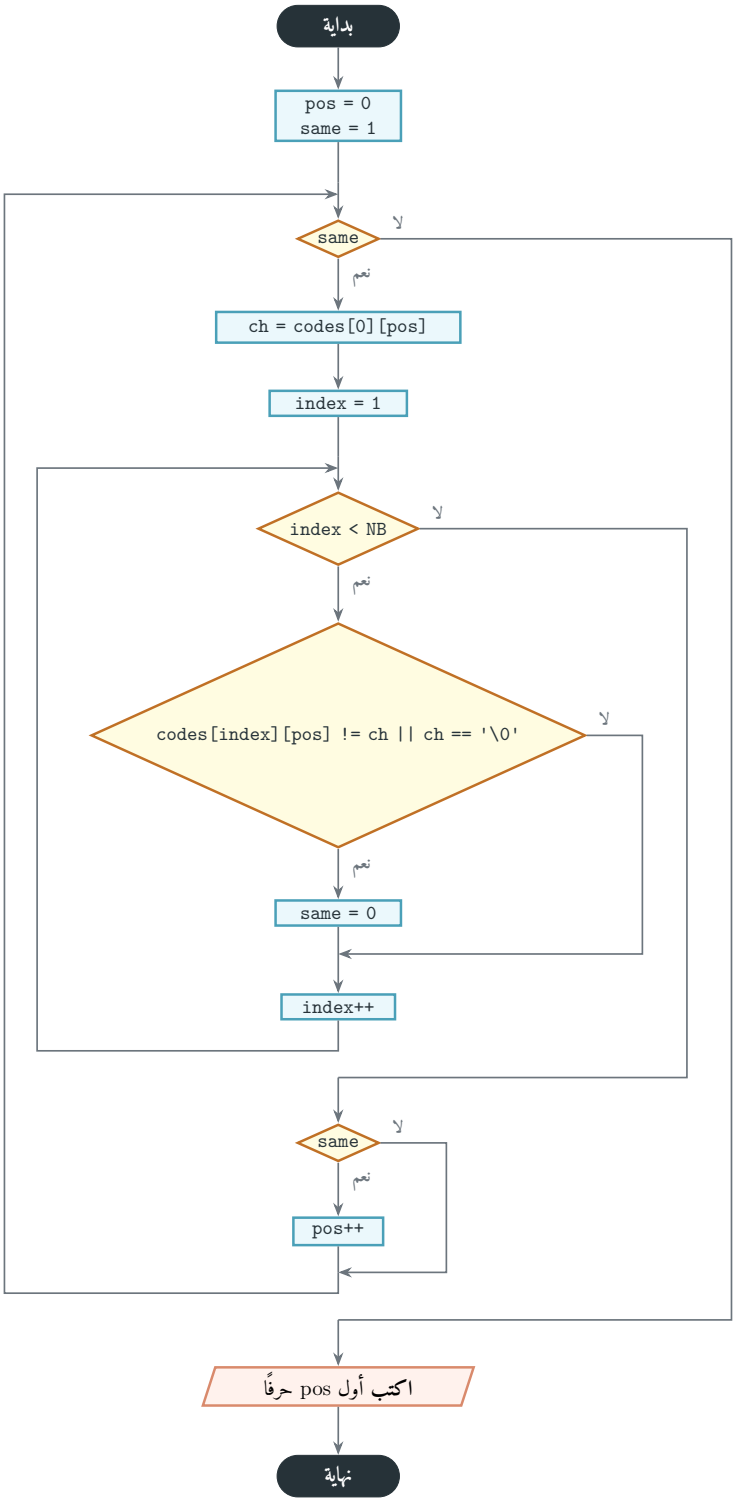
int main(void) {
    char codes[NB][20] = {"TV-SAM-55-Q80", "TV-SAM-65-Q90", "TV-SAM-55-S95", "TV-SAM-43-AU7"};
    int pos = 0, index, same = 1;
```

```c
    char ch;
    while (same) {
        ch = codes[0][pos];
        if (ch == '\0') same = 0;              // الرمز الأول انتهى
        for (index = 1; index < NB && same; index++)
            if (codes[index][pos] != ch)
                same = 0;                      // اختلاف في الموضع pos
        if (same) pos++;
    }
    printf("Common prefix: \"");
    for (index = 0; index < pos; index++)
        putchar(codes[0][index]);
    printf("\" (%d chars)\n", pos);
    return 0;
}
```

نتيجة التنفيذ

```
Common prefix: "TV-SAM-" (7 chars)
```

المخطط الإنسيابي (Organigramme)



تمرين 6.36 توازن الأقواس في ملف إعدادات

علم البيانات ★★ متوسط

ملف إعدادات المتجر بصيغة شبيهة بـ JSON. اكتب برنامجًا يتحقق من توازن الأقواس `()` و`[]` و`{}` باستعمال **مكدس** (جدول حروف)، ويتجاهل ما بين علامتي الاقتباس.

خطوات الحل

1) قوس فاتح: ندفعه في المكدس.
2) قوس مغلق: يجب أن يطابق قمة المكدس، ثم نسحبها.
3) في النهاية يجب أن يكون المكدس فارغًا.

البرنامج بلغة C

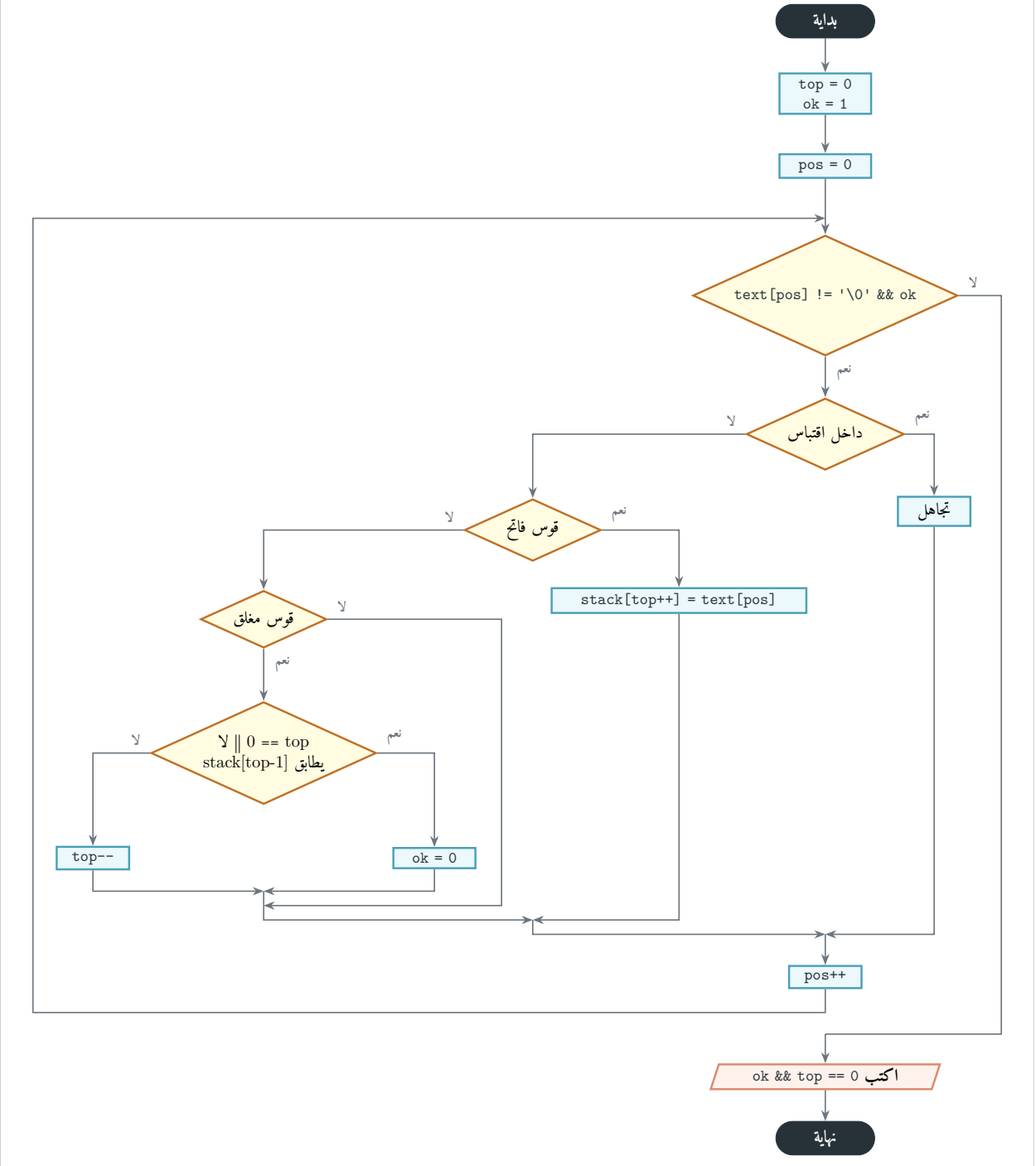
```c
#include <stdio.h>

int main(void) {
    char text[] = "{\"shop\": {\"name\": \"Dz(Store\", \"tags\": [\"tech\", \"home\"]}, \"open\": [8, 20]}";
    char stack[50], ch, expected;
    int pos, top = 0, ok = 1, inQuote = 0;
    for (pos = 0; text[pos] != '\0' && ok; pos++) {
        ch = text[pos];
        if (ch == '"') { inQuote = !inQuote; continue; }
        if (inQuote) continue;                // الأقواس داخل النص لا تُحسب
        if (ch == '(' || ch == '[' || ch == '{') {
            stack[top++] = ch;                // دفع
        } else if (ch == ')' || ch == ']' || ch == '}') {
            expected = (ch == ')') ? '(' : (ch == ']') ? '[' : '{';
            if (top == 0 || stack[top - 1] != expected)
                ok = 0;
            else
                top--;                        // سحب
        }
    }
    if (ok && top == 0)
        printf("Brackets are balanced\n");
    else
        printf("Unbalanced at position %d\n", pos - 1);
    return 0;
}
```

نتيجة التنفيذ

```
Brackets are balanced
```

المخطط الانسيابي (Organigramme)



تمرين 6.37 لون المنتج من الشفرة الست عشرية

تجارة إلكترونية

متوسط

ألوان المنتجات مخزنة بصيغة الويب `#RRGGBB`. اكتب برنامجًا يحول الشفرة إلى ثلاث قيم عشرية (أحمر، أخضر، أزرق) بين 0 و255، ويحسب سطوعها `0.299R + 0.587G + 0.114B` ليختار نصًا أبيض أو أسود فوقها.

خطوات الحل

1) قيمة رقم ست عشري: `0-9` أو `A-F` (أو `a-f`).

2) كل لون رقمان: `16 * high + low`.

3) السطوع > 128: نص أسود، وإلا أبيض.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char color[10], ch;
    int rgb[3], part, digit, half, value;
    double light;
    scanf("%9s", color);
    for (part = 0; part < 3; part++) {
        value = 0;
        for (half = 0; half < 2; half++) {    // رقمان لكل لون
            ch = color[1 + 2 * part + half];
            if (ch >= '0' && ch <= '9')      digit = ch - '0';
            else if (ch >= 'A' && ch <= 'F') digit = ch - 'A' + 10;
            else                             digit = ch - 'a' + 10;
            value = value * 16 + digit;
        }
        rgb[part] = value;
    }
    light = 0.299 * rgb[0] + 0.587 * rgb[1] + 0.114 * rgb[2];
    printf("%s -> R=%d G=%d B=%d\n", color, rgb[0], rgb[1], rgb[2]);
    printf("Brightness %.1f -> use %s text\n", light, light > 128 ? "black" : "white");
    return 0;
}
```

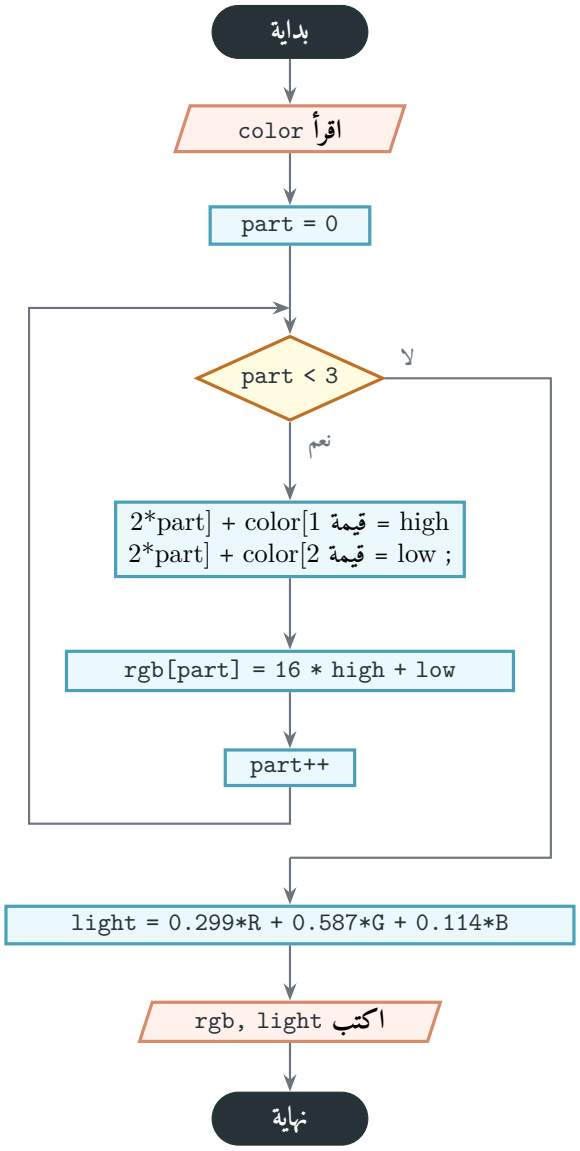
المدخلات

```
#FF8800
```

نتيجة التنفيذ

```
#FF8800 -> R=255 G=136 B=0
Brightness 156.1 -> use black text
```

المخطط الإنسيابي (Organigramme)



تمرين 6.38 سهولة قراءة وصف المنتج

علم البيانات ★★ متوسط

اكتب برنامجًا يحسب مؤشرات **سهولة القراءة** لوصف منتج: عدد الجمل (تنتهي بـ . أو ! أو ?)، عدد الكلمات، عدد الحروف، متوسط الكلمات في الجملة ومتوسط الحروف في الكلمة.

خطوات الحل

1) مرور واحد: عد الحروف، بدايات الكلمات، ونهايات الجمل.
2) المؤشران: `words / sentences` و`letters / words`.

البرنامج بلغة C

```c
#include <stdio.h>

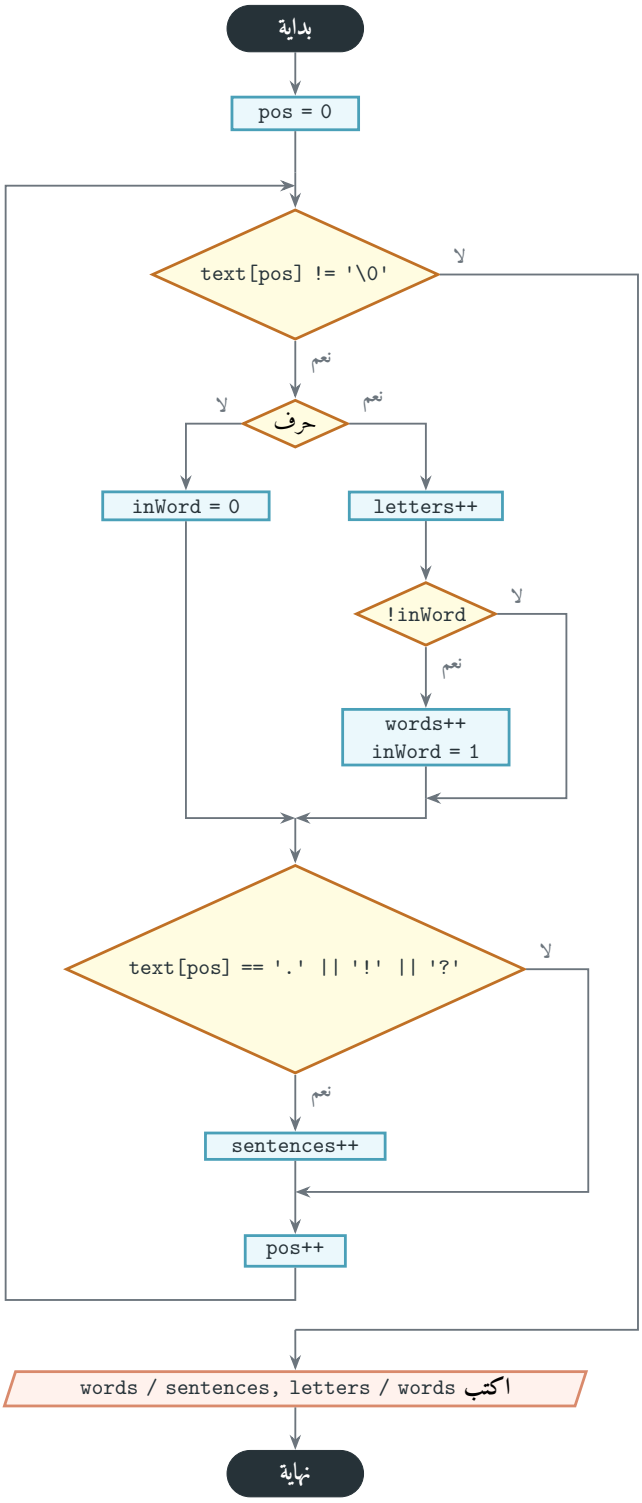
int main(void) {
    char text[] = "This blender is powerful. It crushes ice in seconds! Easy to clean? Yes, very easy.";
    int pos, letters = 0, words = 0, sentences = 0, inWord = 0;
```

```c
    char ch;
    for (pos = 0; text[pos] != '\0'; pos++) {
        ch = text[pos];
        if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
            letters++;
            if (!inWord) { words++; inWord = 1; }    // بداية كلمة
        } else {
            inWord = 0;
        }
        if (ch == '.' || ch == '!' || ch == '?')
            sentences++;                              // نهاية جملة
    }
    printf("Sentences %d, words %d, letters %d\n", sentences, words, letters);
    printf("Words per sentence = %.2f\n", (double)words / sentences);
    printf("Letters per word = %.2f\n", (double)letters / words);
    return 0;
}
```

نتيجة التنفيذ

```
Sentences 4, words 15, letters 64
Words per sentence = 3.75
Letters per word = 4.27
```

المخطط الإنسيابي (Organigramme)



تمرين 6.39 مسافة هامينغ بين رمزين

علم البيانات

متوسط

لاكتشاف أخطاء الإدخال في رموز المنتجات ذات الطول الثابت، نحسب **مسافة هامينغ**: عدد المواضع التي يختلف فيها الحرفان.
اكتب برنامجًا يحسبها ويعرض المواضع المختلفة، ويعتبر الرمز "خطأ إدخال محتمل" إذا كانت المسافة 1.

خطوات الحل

1) نتحقق أن للرمزين نفس الطول.
2) نقارن الحروف موضعًا بموضع ونعد الاختلافات.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char codeA[30], codeB[30];
    int pos, lengthA = 0, lengthB = 0, distance = 0;
    scanf("%29s %29s", codeA, codeB);
    while (codeA[lengthA] != '\0') lengthA++;
    while (codeB[lengthB] != '\0') lengthB++;
    if (lengthA != lengthB) {
        printf("Different lengths: not comparable\n");
        return 0;
    }
    for (pos = 0; pos < lengthA; pos++)
        if (codeA[pos] != codeB[pos]) {
            distance++;
            printf("Position %d: %c vs %c\n", pos, codeA[pos], codeB[pos]);
        }
    printf("Hamming distance = %d%s\n", distance,
           distance == 1 ? " -> probable typo" : "");
    return 0;
}
```

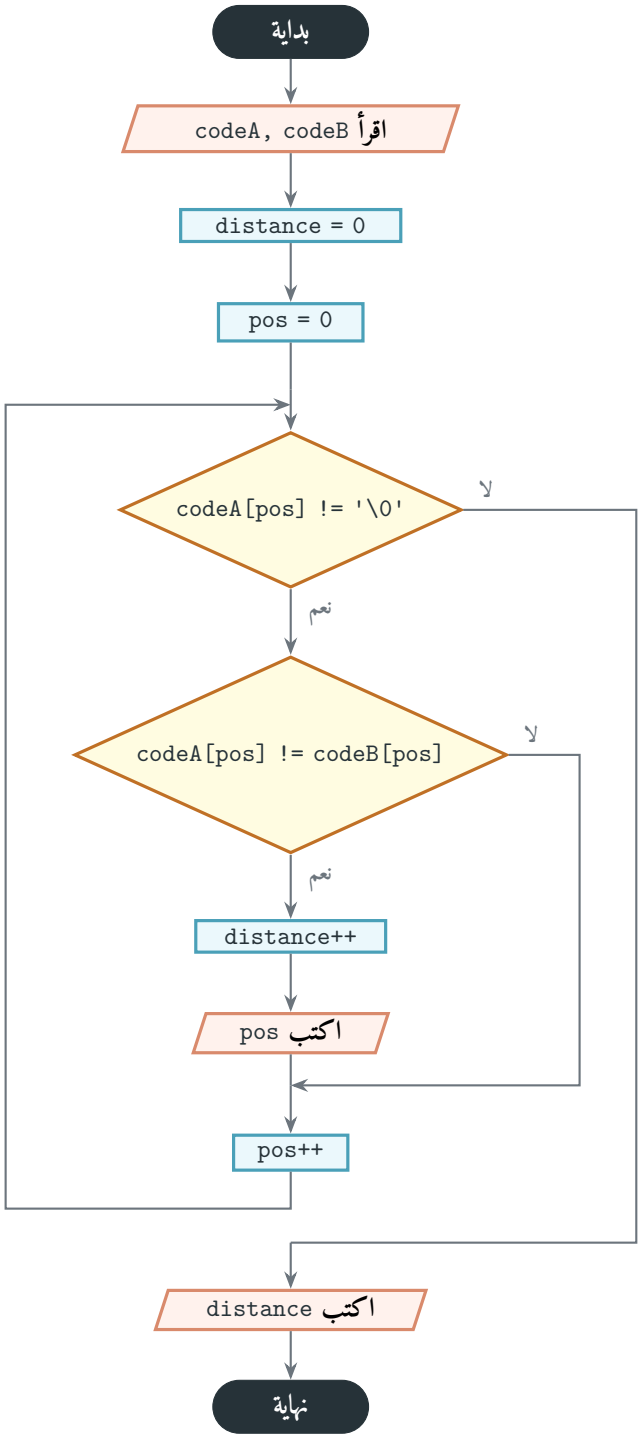
المدخلات

```
DZ48K7Q2 DZ48K1Q2
```

نتيجة التنفيذ

```
Position 5: 7 vs 1
Hamming distance = 1 -> probable typo
```

المخطط الإنسيابي (Organigramme)



تمرين 6.40 تحليل معاملات رابط البحث

تجارة إلكترونية ★★ متوسط

رابط صفحة البحث يحتوي معاملات بعد ? على شكل key=value مفصولة بـ &. اكتب برنامجًا يستخرج كل زوج ويعرضه، ويستخرج بالخصوص قيمة maxprice كعدد.

خطوات الحل

1) نتقدم إلى ما بعد ?.
2) نقرأ المفتاح حتى = ثم القيمة حتى & أو النهاية.
3) نقارن المفتاح بـ "maxprice" يدويًا، ونحول قيمته.

البرنامج بلغة C

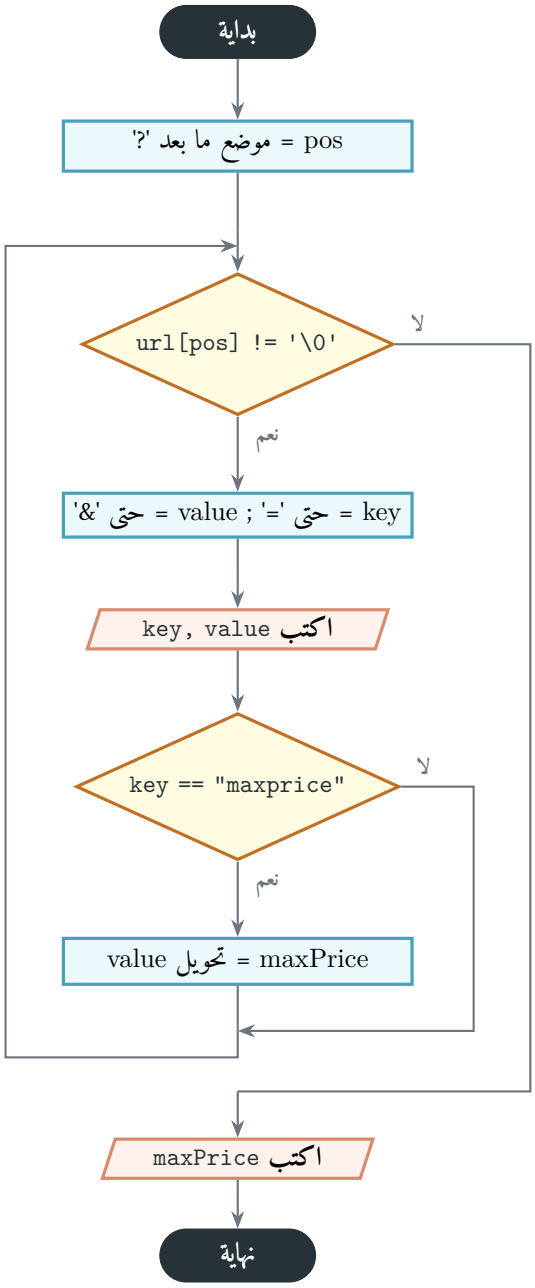
```c
#include <stdio.h>

int main(void) {
    char url[] = "https://shop.dz/search?q=laptop&brand=hp&maxprice=95000&sort=price";
    char key[20], value[30], target[] = "maxprice";
    int pos = 0, length, cmp;
    long maxPrice = -1;
    while (url[pos] != '?' && url[pos] != '\0') pos++;
    if (url[pos] == '?') pos++;
    while (url[pos] != '\0') {
        length = 0;
        while (url[pos] != '=' && url[pos] != '\0') key[length++] = url[pos++];
        key[length] = '\0';
        if (url[pos] == '=') pos++;
        length = 0;
        while (url[pos] != '&' && url[pos] != '\0') value[length++] = url[pos++];
        value[length] = '\0';
        if (url[pos] == '&') pos++;           // الزوج التالي
        printf("%-9s = %s\n", key, value);
        for (cmp = 0; key[cmp] == target[cmp] && key[cmp] != '\0'; cmp++) ;
        if (key[cmp] == target[cmp]) {        // المفتاح هو maxprice
            maxPrice = 0;
            for (cmp = 0; value[cmp] != '\0'; cmp++)
                maxPrice = maxPrice * 10 + (value[cmp] - '0');
        }
    }
    printf("Max price filter: %ld DA\n", maxPrice);
    return 0;
}
```

نتيجة التنفيذ

```
q         = laptop
brand     = hp
maxprice  = 95000
sort      = price
Max price filter: 95000 DA
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 6.41 مسافة التحرير بين كلمتي بحث

تجارة إلكترونية صعب

محرك البحث في المتجر يقترح التصحيح عندما يخطئ الزبون في الكتابة. اكتب برنامجًا يحسب **مسافة ليفنشتاين** بين كلمة البحث `"labtop"` وكل منتج من قائمة، أي أقل عدد من العمليات (إدراج، حذف، تبديل حرف) لتحويل الأولى إلى الثانية، ثم يقترح الأقرب.

خطوات الحل

1) `dist[row][col]` = مسافة التحرير بين أول `row` حرفًا من الكلمة وأول `col` حرفًا من المنتج.
2) الحالات الابتدائية: `dist[row][0] = row` و `dist[0][col] = col`.
3) إذا تساوى الحرفان: `dist[row][col] = dist[row-1][col-1]`، وإلا 1 + أصغر الخانات الثلاث المجاورة.
4) المنتج ذو أصغر مسافة هو الاقتراح.

البرنامج بلغة C

```c
#include <stdio.h>

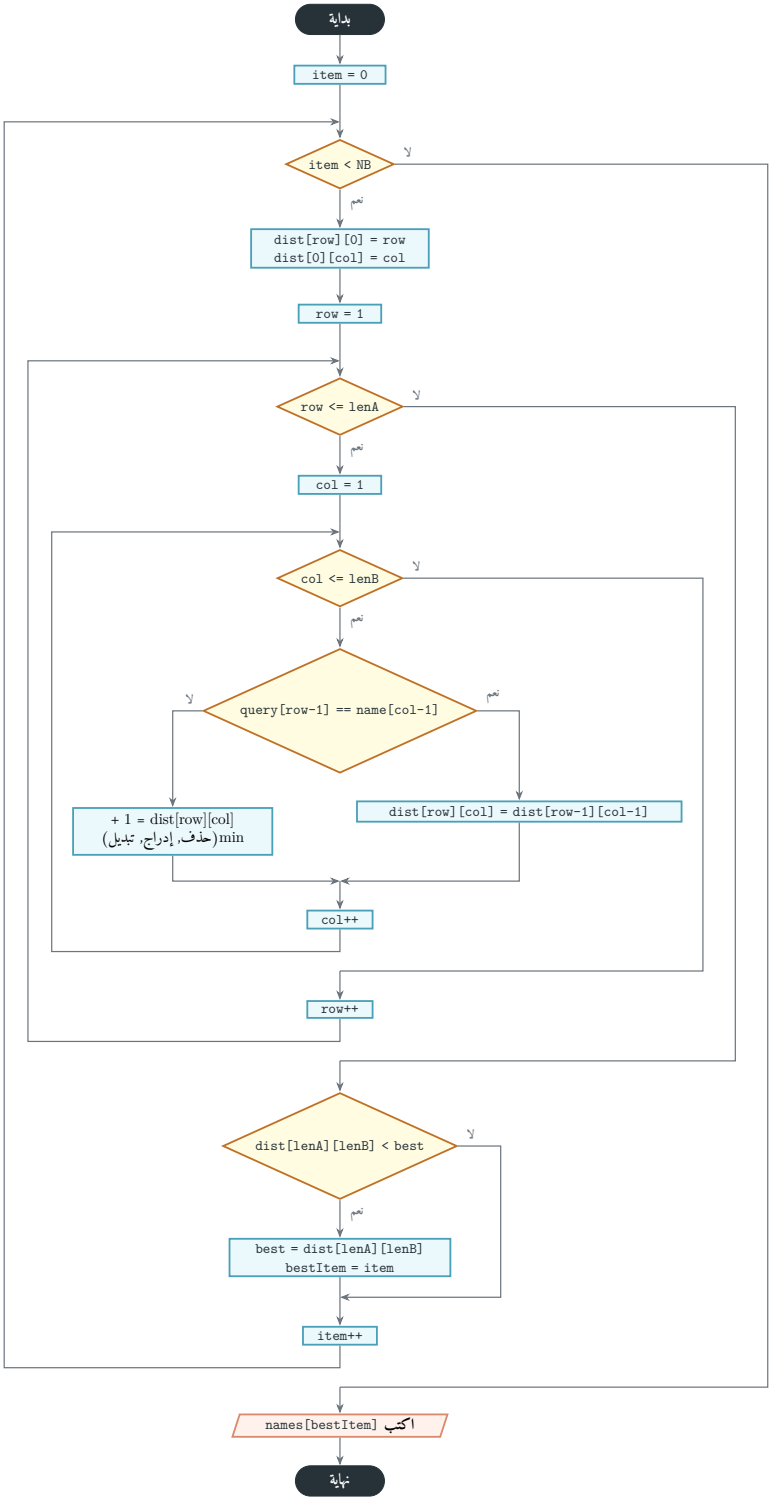
#define NB 5
#define MAXLEN 20

int main(void) {
    char query[] = "labtop";
    char names[NB][MAXLEN] = {"laptop", "tablet", "desktop", "lamp", "backup"};
    int dist[MAXLEN][MAXLEN];
    int item, row, col, lenA = 0, lenB, best = 1000, bestItem = 0, smallest;
    while (query[lenA] != '\0') lenA++;
    for (item = 0; item < NB; item++) {
        lenB = 0;
        while (names[item][lenB] != '\0') lenB++;
        for (row = 0; row <= lenA; row++) dist[row][0] = row;   // حذف كل الحروف
        for (col = 0; col <= lenB; col++) dist[0][col] = col;   // إدراج كل الحروف
        for (row = 1; row <= lenA; row++)
            for (col = 1; col <= lenB; col++) {
                if (query[row - 1] == names[item][col - 1]) {
                    dist[row][col] = dist[row - 1][col - 1];   // لا تكلفة
                } else {
                    smallest = dist[row - 1][col];              // حذف
                    if (dist[row][col - 1] < smallest) smallest = dist[row][col - 1];         // إدراج
                    if (dist[row - 1][col - 1] < smallest) smallest = dist[row - 1][col - 1]; // تبديل
                    dist[row][col] = 1 + smallest;
                }
            }
        printf("%-8s -> distance %d\n", names[item], dist[lenA][lenB]);
        if (dist[lenA][lenB] < best) {
            best = dist[lenA][lenB];
            bestItem = item;
        }
    }
    printf("Did you mean \"%s\"?\n", names[bestItem]);
    return 0;
}
```

نتيجة التنفيذ

```
laptop   -> distance 1
tablet   -> distance 4
desktop  -> distance 4
lamp     -> distance 3
backup   -> distance 4
Did you mean "laptop"?
```

المخطط الإنسيابي (Organigramme)



تمرين 6.42 البحث السريع عن نمط بخوارزمية KMP

علم البيانات صعب ★★★

نبحث في سجل نصي طويل عن كل مواضع ظهور نمط مثل `"abab"`. البحث الساذج يعيد المقارنة من الصفر عند كل فشل. اكتب برنامجًا يطبق خوارزمية **كنوث-موريس-برات (KMP)**: نحسب أولًا جدول **أطول بادئة هي لاحقة** `fail[]` للنمط، ثم نمر على النص مرة واحدة.

خطوات الحل

1) `fail[pos]` = طول أطول بادئة حقيقية للنمط `pattern[0..pos]` تكون لاحقة له أيضًا.
2) أثناء البحث: عند عدم التطابق نرجع في النمط إلى `fail[matched-1]` بدل الرجوع في النص.
3) عندما يبلغ `matched` طول النمط نسجل موضعًا ونكمل من `fail[matched-1]`.

البرنامج بلغة C

```c
#include <stdio.h>

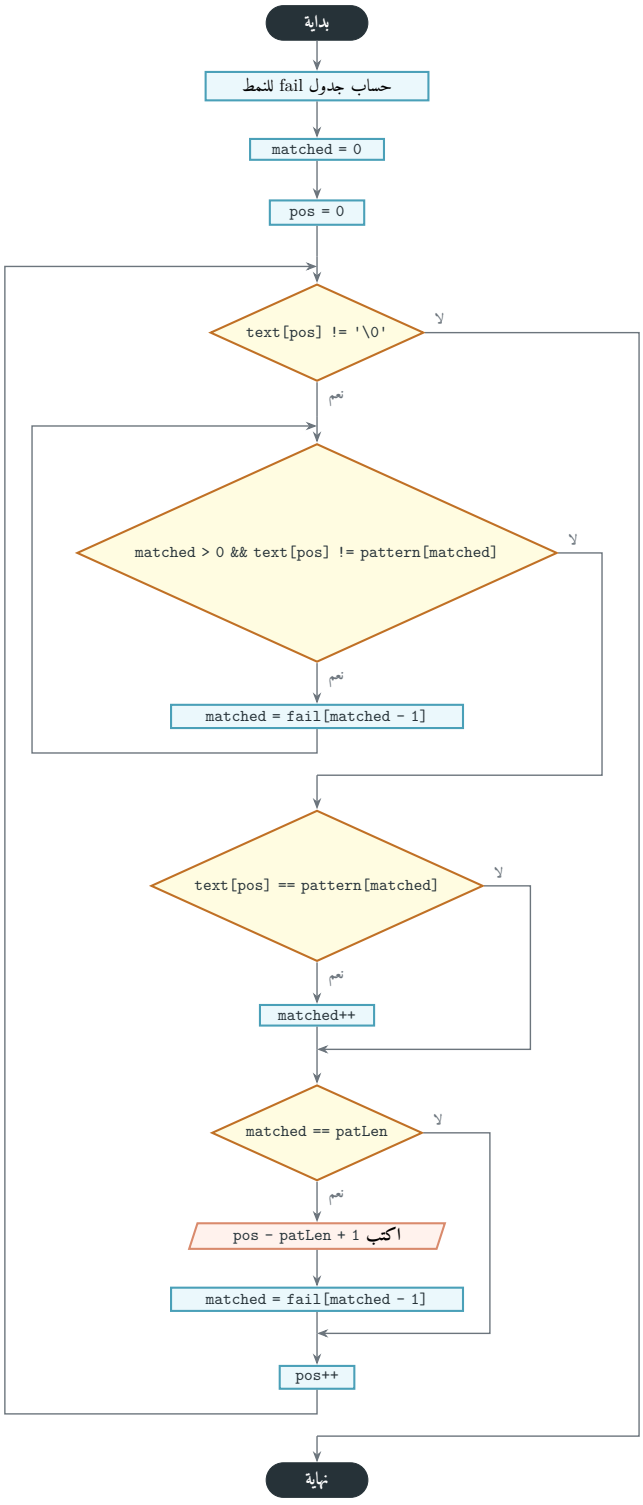
int main(void) {
    char text[] = "abababcababababab";
    char pattern[] = "abab";
    int fail[20], patLen = 0, pos, matched, count = 0;
    while (pattern[patLen] != '\0') patLen++;
    fail[0] = 0;                                    // جدول البادئات
    matched = 0;
    for (pos = 1; pos < patLen; pos++) {
        while (matched > 0 && pattern[pos] != pattern[matched])
            matched = fail[matched - 1];
        if (pattern[pos] == pattern[matched]) matched++;
        fail[pos] = matched;
    }
    printf("fail table:");
    for (pos = 0; pos < patLen; pos++) printf(" %d", fail[pos]);
    printf("\n");
    matched = 0;                                    // البحث في النص
    for (pos = 0; text[pos] != '\0'; pos++) {
        while (matched > 0 && text[pos] != pattern[matched])
            matched = fail[matched - 1];            // لا نرجع في النص أبدًا
        if (text[pos] == pattern[matched]) matched++;
        if (matched == patLen) {
            printf("Match at position %d\n", pos - patLen + 1);
            count++;
            matched = fail[matched - 1];            // نسمح بالتداخل
        }
    }
    printf("%d occurrence(s)\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
fail table: 0 0 1 2
Match at position 0
Match at position 2
Match at position 7
```

```
Match at position 9
Match at position 11
5 occurrence(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 6.43 أطول جزء مشترك بين وصفين

علم البيانات

صعب ★★★

لكشف الأوصاف المنسوخة بين منتجين، اكتب برنامجًا يحسب **أطول متتالية جزئية مشتركة (LCS)** بين نصين بالبرمجة الديناميكية، ثم يعيد بناءها ويحسب نسبة التشابه `2*LCS/(lenA+lenB)`.

خطوات الحل

1) `lcs[row][col]` = طول LCS بين أول `row` حرفًا من `first` وأول `col` حرفًا من `second`.
2) إذا تساوى الحرفان: `lcs[row-1][col-1] + 1`، وإلا أكبر `lcs[row-1][col]` و `lcs[row][col-1]`.
3) إعادة البناء: نبدأ من الخانة الأخيرة ونصعد قطريًا عند التطابق، وإلا نحو الخانة الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

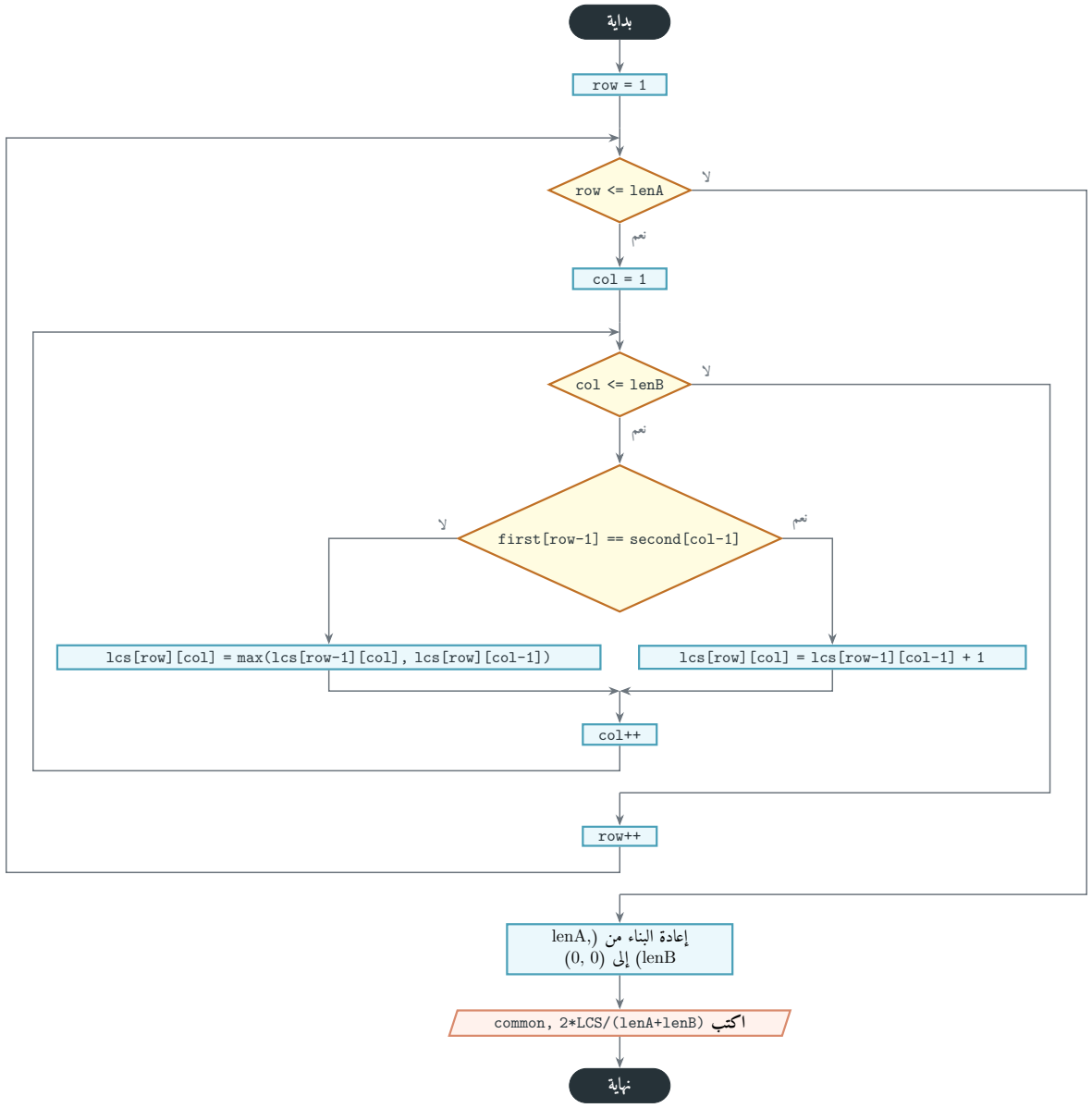
#define MAXLEN 40

int main(void) {
    char first[] = "fast usb charger 20w";
    char second[] = "usb-c fast charger";
    int lcs[MAXLEN][MAXLEN];
    char common[MAXLEN];
    int lenA = 0, lenB = 0, row, col, size;
    while (first[lenA] != '\0') lenA++;
    while (second[lenB] != '\0') lenB++;
    for (row = 0; row <= lenA; row++)
        for (col = 0; col <= lenB; col++) {
            if (row == 0 || col == 0)
                lcs[row][col] = 0;                          // نص فارغ
            else if (first[row - 1] == second[col - 1])
                lcs[row][col] = lcs[row - 1][col - 1] + 1;
            else
                lcs[row][col] = lcs[row - 1][col] > lcs[row][col - 1]
                                ? lcs[row - 1][col] : lcs[row][col - 1];
        }
    size = lcs[lenA][lenB];
    common[size] = '\0';
    row = lenA; col = lenB;
    while (row > 0 && col > 0) {                            // إعادة البناء من النهاية
        if (first[row - 1] == second[col - 1]) {
            common[--size] = first[row - 1];
            row--; col--;
        } else if (lcs[row - 1][col] >= lcs[row][col - 1]) {
            row--;
        } else {
            col--;
        }
    }
    printf("LCS length: %d\n", lcs[lenA][lenB]);
    printf("LCS: \"%s\"\n", common);
    printf("Similarity: %.2f\n", 2.0 * lcs[lenA][lenB] / (lenA + lenB));
    return 0;
}
```

نتيجة التنفيذ

```
LCS length: 12
LCS: "fast charger"
Similarity: 0.63
```

المخطط الإنسيابي (Organigramme)



تمرين 6.44 أكثر الكلمات تكرارًا في التعليقات

علم البيانات ★★★ صعب

اكتب برنامجًا يقرأ نص تعليقات الزبائن، ويقطعه إلى كلمات (بالحروف الصغيرة، ويتجاهل علامات الترقيم)، ويحسب تكرار كل كلمة دون دوال السلاسل، ثم يعرض **أكثر 5 كلمات** تكرارًا مرتبة تنازليًا.

خطوات الحل

1) نستخرج الكلمة الحالية حرفًا بحرف في `word` ونحولها إلى الحروف الصغيرة.
2) نبحث عنها في جدول الكلمات بمقارنة يدوية؛ إن وُجدت نزيد عدادها وإلا نضيفها.
3) نرتب الجدول تنازليًا حسب العداد بالترتيب بالإدراج (مع تبديل الكلمات).

البرنامج بلغة C

```c
#include <stdio.h>

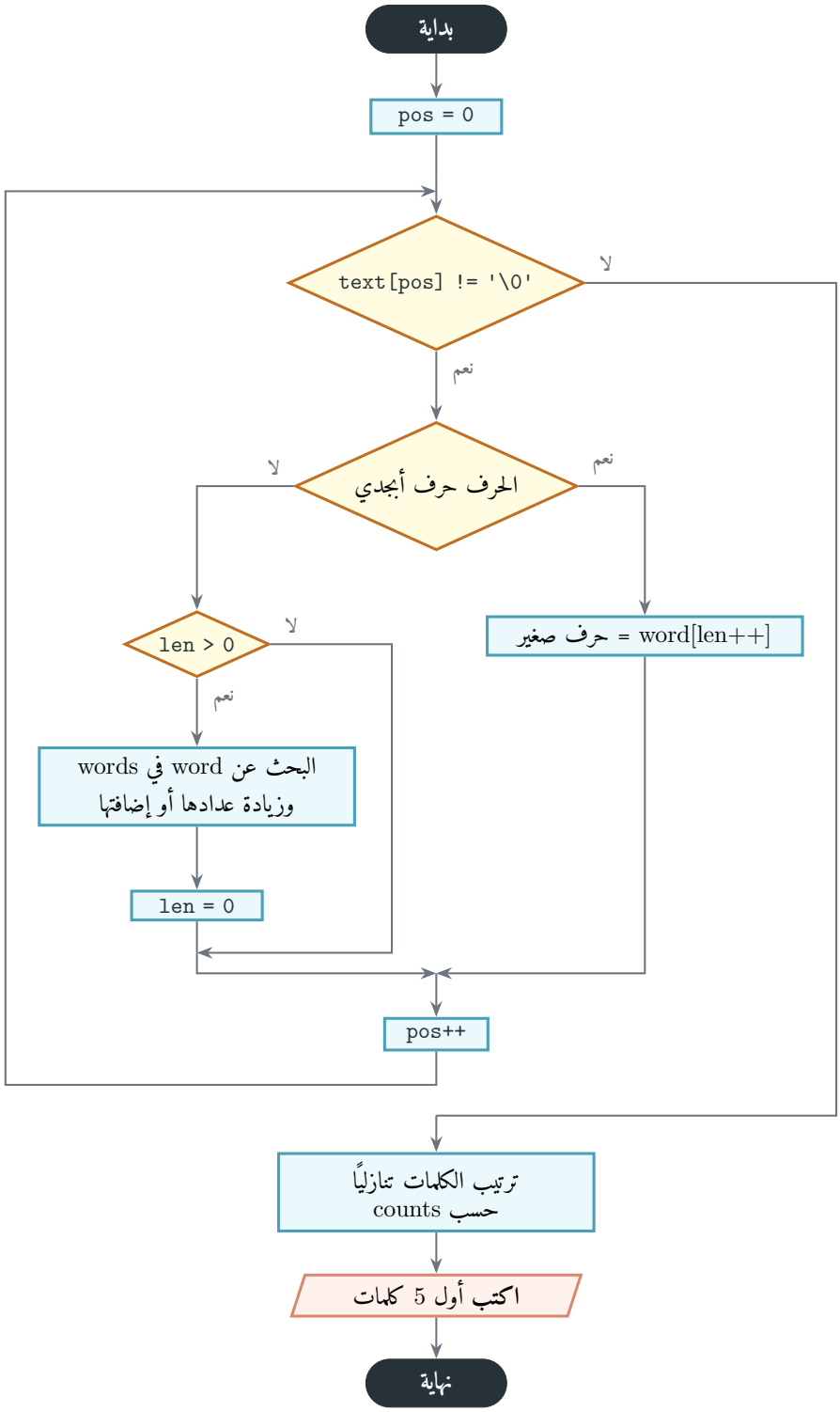
#define MAXW 60
#define WLEN 16

int main(void) {
    char text[] = "Great phone, great battery! Battery lasts long. "
                  "Screen is great, camera is good. Good price, great value.";
    char words[MAXW][WLEN], word[WLEN], tmpWord[WLEN];
    int counts[MAXW], nbWords = 0, len = 0, pos, idx, found, same, letter, tmpCount, back;
    char ch;
    for (pos = 0; ; pos++) {
        ch = text[pos];
        if (ch >= 'A' && ch <= 'Z') ch = ch - 'A' + 'a';        // حروف صغيرة
        if (ch >= 'a' && ch <= 'z') {
            if (len < WLEN - 1) word[len++] = ch;
        } else if (len > 0) {                                    // نهاية كلمة
            word[len] = '\0';
            found = -1;
            for (idx = 0; idx < nbWords && found < 0; idx++) {
                same = 1;                                        // مقارنة يدوية
                for (letter = 0; word[letter] != '\0' || words[idx][letter] != '\0'; letter++)
                    if (word[letter] != words[idx][letter]) { same = 0; break; }
                if (same) found = idx;
            }
            if (found >= 0) {
                counts[found]++;
            } else if (nbWords < MAXW) {
                for (letter = 0; letter <= len; letter++) words[nbWords][letter] = word[letter];
                counts[nbWords++] = 1;
            }
            len = 0;
        }
        if (text[pos] == '\0') break;
    }
    for (idx = 1; idx < nbWords; idx++) {                        // ترتيب بالإدراج تنازليًا
        tmpCount = counts[idx];
        for (letter = 0; letter < WLEN; letter++) tmpWord[letter] = words[idx][letter];
        back = idx - 1;
        while (back >= 0 && counts[back] < tmpCount) {
            counts[back + 1] = counts[back];
            for (letter = 0; letter < WLEN; letter++) words[back + 1][letter] = words[back][letter];
            back--;
        }
        counts[back + 1] = tmpCount;
        for (letter = 0; letter < WLEN; letter++) words[back + 1][letter] = tmpWord[letter];
    }
    printf("%d distinct words. Top 5:\n", nbWords);
    for (idx = 0; idx < 5 && idx < nbWords; idx++)
        printf("%d. %-8s %d\n", idx + 1, words[idx], counts[idx]);
    return 0;
}
```

نتيجة التنفيذ

```
11 distinct words. Top 5:
1. great    4
2. battery  2
3. is       2
4. good     2
5. phone    1
```

المخطط الإنسيابي (Organigramme)



تمرين 6.45 حساب عبارة سعر مكتوبة كنص

تجارة إلكترونية ★★★ صعب

تسمح لوحة الإدارة بكتابة السعر كعبارة مثل `"2*1500+3*250-10*20"`. اكتب برنامجًا يقيّم العبارة يدويًا مع احترام أولوية الضرب والقسمة على الجمع والطرح، دون أي دالة تحويل.

خطوات الحل

1) نقرأ العدد الحالي رقمًا برقم: `number = number*10 + (ch-'0')`.
2) نحتفظ بـ `term` (حاصل الضرب/القسمة الجاري) و `total` (مجموع الحدود) والعملية السابقة `op`.
3) عند `+` أو `-` أو نهاية النص: نطبق `op` على `term`، ثم نضيف `term` بإشارته إلى `total`.
4) عند `*` أو `/`: نطبق `op` على `term` فقط.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    char expr[100], ch, op = '+';
    long total = 0, term = 0, number = 0;
    int pos, sign = 1;
    scanf("%99s", expr);
    for (pos = 0; ; pos++) {
        ch = expr[pos];
        if (ch >= '0' && ch <= '9') {
            number = number * 10 + (ch - '0');        // تحويل يدوي
            continue;
        }
        if (op == '*')      term = term * number;     // أولوية الضرب
        else if (op == '/') term = term / number;
        else                term = number;            // بداية حد جديد
        if (ch == '+' || ch == '-' || ch == '\0') {
            total += sign * term;                     // الحد اكتمل
            sign = (ch == '-') ? -1 : 1;
            op = '+';
        } else {
            op = ch;                                  // أو / *
        }
        number = 0;
        if (ch == '\0') break;
    }
    printf("%s = %ld DA\n", expr, total);
    return 0;
}
```

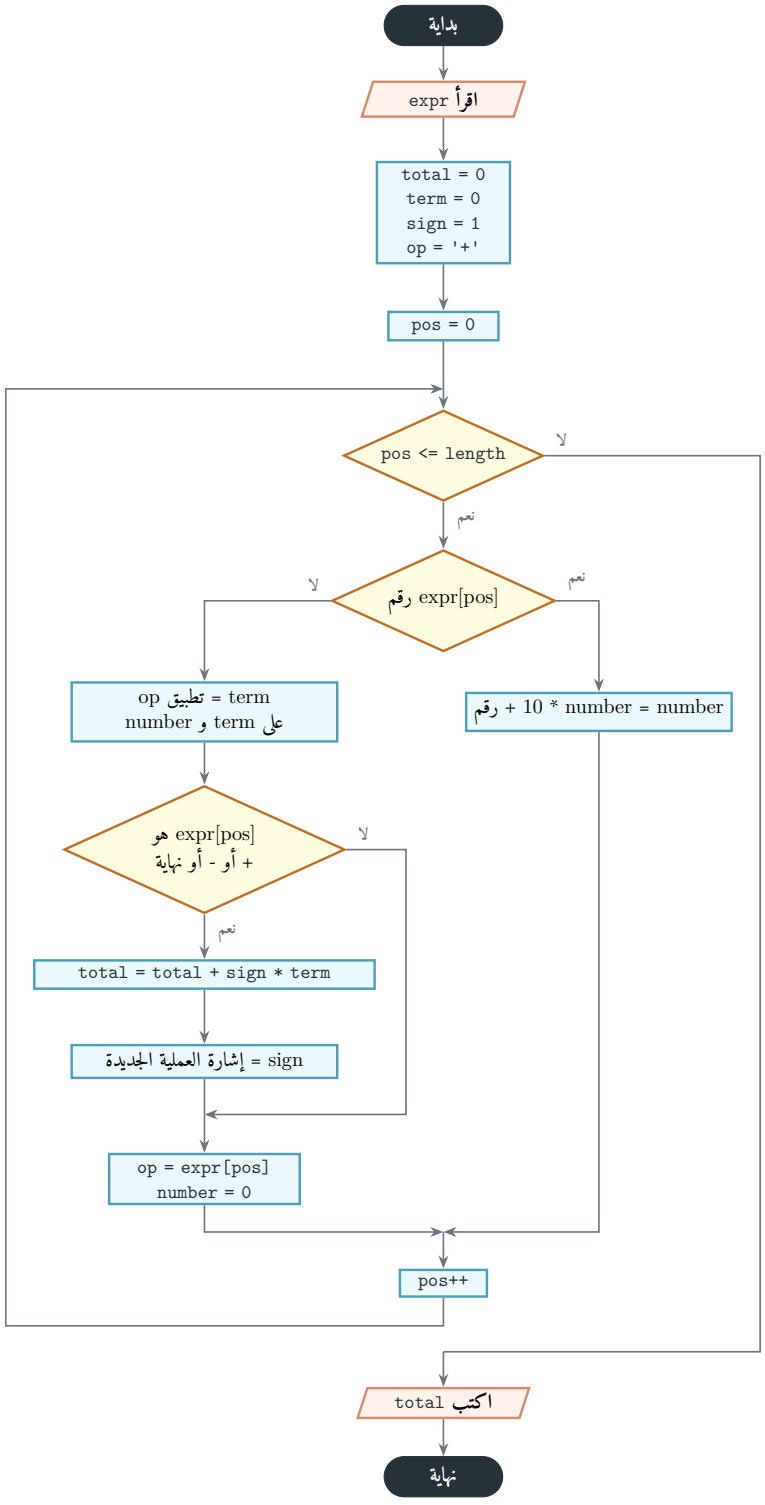
المدخلات

```
2*1500+3*250-10*20/4
```

نتيجة التنفيذ

```
2*1500+3*250-10*20/4 = 3700 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 6.46 مطابقة رموز المنتجات بالأحرف البديلة

تجارة إلكترونية

صعب ★★★

يكتب مسير المخزون أنماطًا مثل `"TV-*-202?"` حيث `?` يطابق **حرفًا واحدًا** و `*` يطابق **أي سلسلة** (حتى الفارغة). اكتب برنامجًا يحدد الرموز المطابقة للنمط بالبرمجة الديناميكية.

خطوات الحل

1) `match[row][col]` = هل يطابق أول `row` حرفًا من النمط أول `col` حرفًا من الرمز؟
2) `match[0][0] = 1`، والنمط المكوّن من `*` فقط يطابق الفراغ.
3) `*`: `match[row-1][col]` (لا شيء) أو `match[row][col-1]` (حرف إضافي).
4) `?` أو حرفان متساويان: `match[row-1][col-1]`.

البرنامج بلغة C

```c
#include <stdio.h>

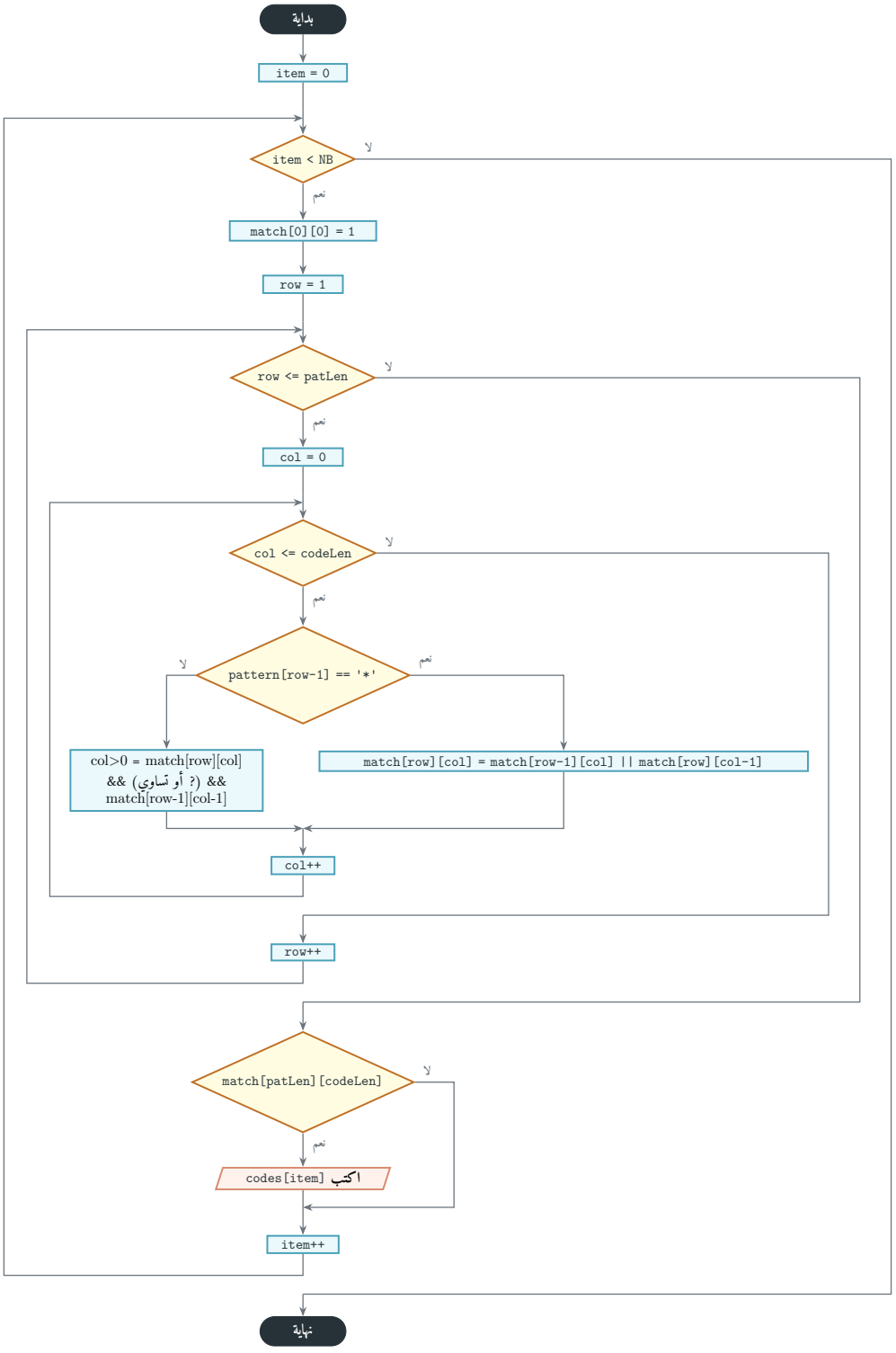
#define NB 6
#define MAXLEN 24

int main(void) {
    char pattern[] = "TV-*-202?";
    char codes[NB][MAXLEN] = {"TV-SAM-2024", "TV-LG-OLED-2025", "PC-HP-2024",
                              "TV--2023", "TV-SONY-2019", "TV-TCL-20245"};
    int match[MAXLEN][MAXLEN];
    int item, row, col, patLen = 0, codeLen;
    char pchar;
    while (pattern[patLen] != '\0') patLen++;
    printf("Pattern %s\n", pattern);
    for (item = 0; item < NB; item++) {
        codeLen = 0;
        while (codes[item][codeLen] != '\0') codeLen++;
        match[0][0] = 1;
        for (col = 1; col <= codeLen; col++) match[0][col] = 0;   // نمط فارغ
        for (row = 1; row <= patLen; row++) {
            pchar = pattern[row - 1];
            for (col = 0; col <= codeLen; col++) {
                if (pchar == '*')
                    match[row][col] = match[row - 1][col] || (col > 0 && match[row][col - 1]);
                else
                    match[row][col] = col > 0 && match[row - 1][col - 1] &&
                                      (pchar == '?' || pchar == codes[item][col - 1]);
            }
        }
        printf("  %-16s %s\n", codes[item], match[patLen][codeLen] ? "MATCH" : "-");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Pattern TV-*-202?
  TV-SAM-2024      MATCH
  TV-LG-OLED-2025  MATCH
  PC-HP-2024       -
  TV--2023         MATCH
  TV-SONY-2019     -
  TV-TCL-20245     -
```

المخطط الإنسيابي (Organigramme)



تمرين 6.47 أطول اسم متناظر داخل نص

علم البيانات ★★★ صعب

اكتب برنامجًا يجد **أطول سلسلة جزئية متناظرة** (تُقرأ من اليمين ومن اليسار) داخل سلسلة DNA أو رمز، بطريقة **التوسع حول المركز**: كل موضع (وكل فراغ بين حرفين) مركز محتمل.

خطوات الحل

1) عدد المراكز `2*length - 1`: حرف واحد (طول فردي) أو حرفان متجاوران (طول زوجي).
2) من كل مركز نوسع يسارًا ويمينًا ما دام الحرفان متساويين.
3) نحتفظ بأطول مجال `[bestStart, bestStart + bestLen)`.

البرنامج بلغة C

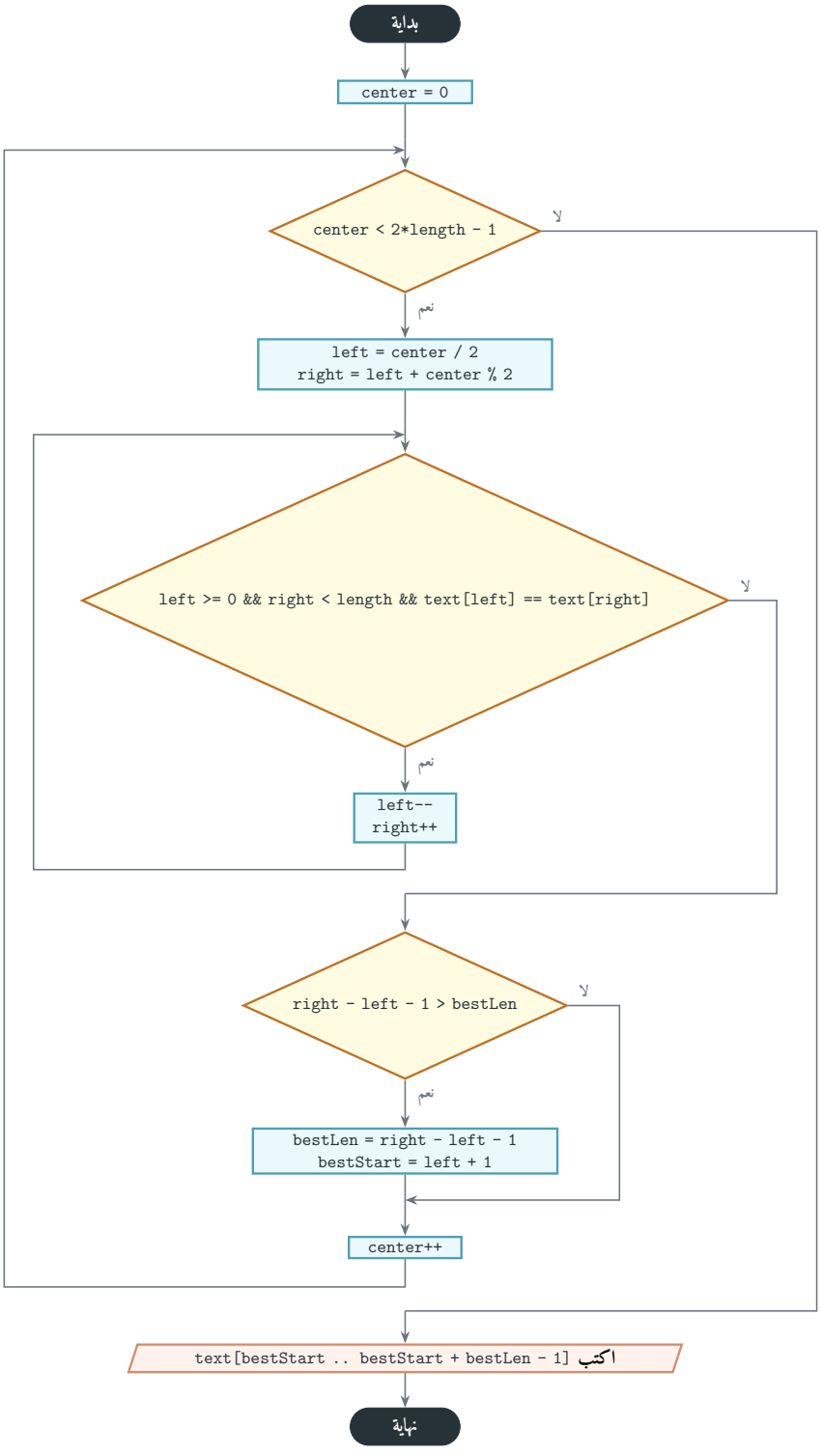
```c
#include <stdio.h>

int main(void) {
    char text[] = "GATTACATCGCTAGCGATCCA";
    int length = 0, center, left, right, bestLen = 0, bestStart = 0, pos;
    while (text[length] != '\0') length++;
    for (center = 0; center < 2 * length - 1; center++) {
        left = center / 2;                // مركز زوجي: حرف، فردي: بين حرفين
        right = left + center % 2;
        while (left >= 0 && right < length && text[left] == text[right]) {
            left--;                       // نتوسع ما دام التناظر قائمًا
            right++;
        }
        if (right - left - 1 > bestLen) {
            bestLen = right - left - 1;
            bestStart = left + 1;
        }
    }
    printf("Text: %s\n", text);
    printf("Longest palindrome (%d chars) at %d: ", bestLen, bestStart);
    for (pos = bestStart; pos < bestStart + bestLen; pos++)
        putchar(text[pos]);
    putchar('\n');
    return 0;
}
```

نتيجة التنفيذ

```
Text: GATTACATCGCTAGCGATCCA
Longest palindrome (9 chars) at 10: CTAGCGATC
```

المخطط الإنسيابي (Organigramme)



تمرين 6.48 ترميز Base64 لمفتاح الواجهة البرمجية

تجارة إلكترونية صعب ★★★

بوابة الدفع تطلب إرسال `user:key` مرمّزًا بـ **Base64**. اكتب برنامجًا يرمّز السلسلة يدويًا: كل 3 بايتات (24 بتًا) تتحول إلى 4 رموز من 6 بتات، مع الحشو بـ `=` في النهاية.

خطوات الحل

1) نجمع 3 بايتات في عدد واحد: `(b0 << 16) | (b1 << 8) | b2`.
2) نستخرج 4 مجموعات من 6 بتات بالإزاحة و `& 63` ونستعمل كل واحدة فهرسًا في جدول الرموز.
3) إذا بقي بايت أو بايتان: نكمل بأصفار ونستبدل الرموز الناقصة بـ `=`.

البرنامج بلغة C

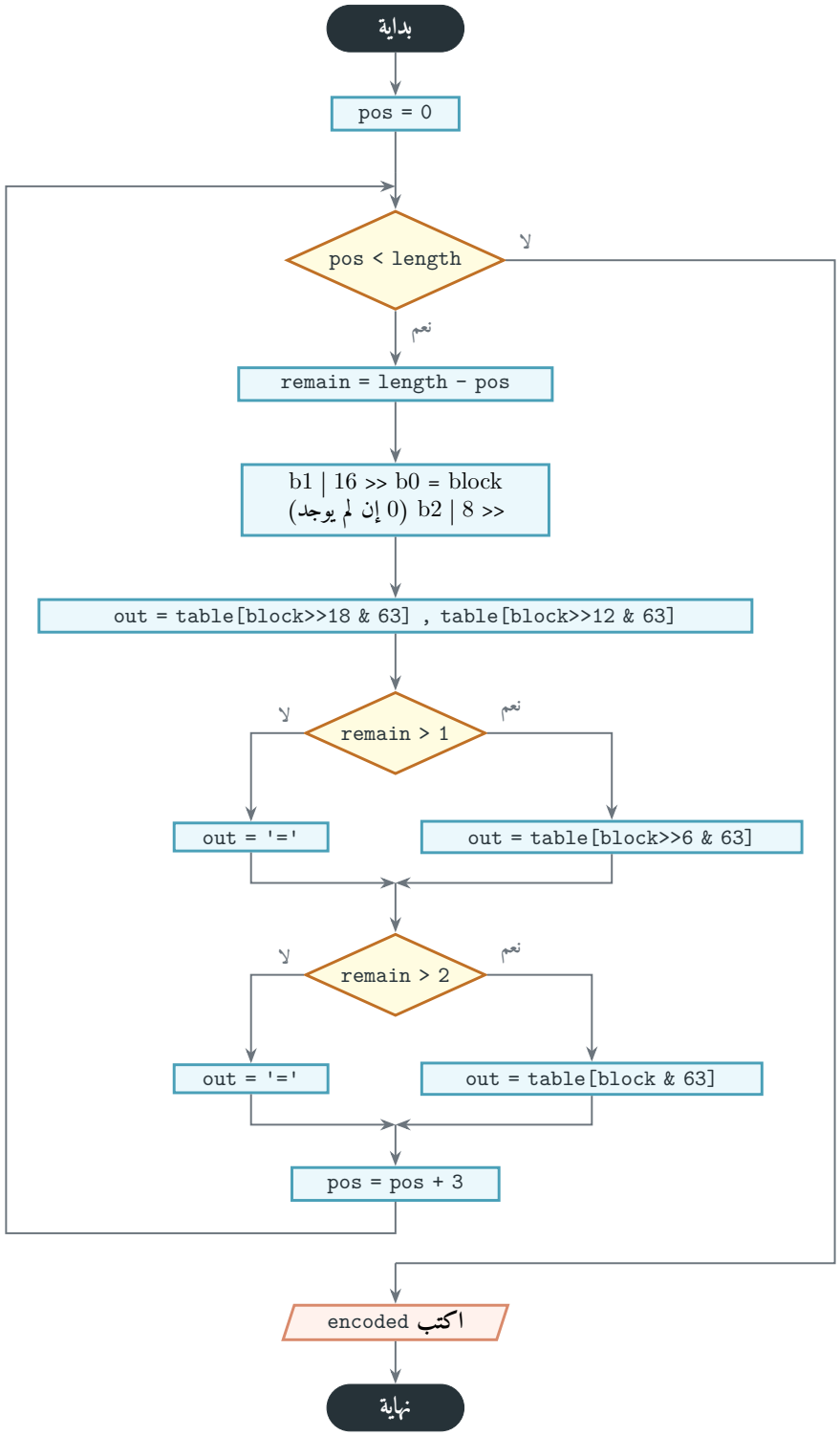
```c
#include <stdio.h>

int main(void) {
    char table[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    char input[] = "shop42:Key#2026";
    char encoded[64];
    unsigned long block;
    int length = 0, pos, outPos = 0, remain;
    while (input[length] != '\0') length++;
    for (pos = 0; pos < length; pos += 3) {
        remain = length - pos;
        block = (unsigned long)(unsigned char)input[pos] << 16;            // البايت الأول
        if (remain > 1) block |= (unsigned long)(unsigned char)input[pos + 1] << 8;
        if (remain > 2) block |= (unsigned long)(unsigned char)input[pos + 2];
        encoded[outPos++] = table[(block >> 18) & 63];                     // 6 بتات
        encoded[outPos++] = table[(block >> 12) & 63];
        encoded[outPos++] = remain > 1 ? table[(block >> 6) & 63] : '=';   // حشو
        encoded[outPos++] = remain > 2 ? table[block & 63] : '=';
    }
    encoded[outPos] = '\0';
    printf("Input : %s (%d bytes)\n", input, length);
    printf("Base64: %s (%d chars)\n", encoded, outPos);
    printf("Header: Authorization: Basic %s\n", encoded);
    return 0;
}
```

نتيجة التنفيذ

```
Input : shop42:Key#2026 (15 bytes)
Base64: c2hvcDQyOktleSMyMDI2 (20 chars)
Header: Authorization: Basic c2hvcDQyOktleSMyMDI2
```

المخطط الإنسيابي (Organigramme)



تمرين 6.49 تنسيق وصف المنتج بعرض ثابت

تجارة إلكترونية صعب ★★★

الملصق الحراري للطرد يطبع 24 حرفًا في السطر. اكتب برنامجًا يوزع كلمات الوصف على الأسطر دون قطع أي كلمة، ثم يضبط كل سطر (عدا الأخير) بتوزيع الفراغات الإضافية بين الكلمات بالتساوي (الفراغات الزائدة تذهب إلى اليسار أولًا).

خطوات الحل

1) نجمع مواضع بداية الكلمات وأطوالها.
2) نأخذ أكبر عدد من الكلمات يسعه السطر: مجموع الأطوال + عدد الفجوات 24.
3) الفراغات = WIDTH − ، تُقسم على عدد الفجوات، والباقي يوزع على الفجوات الأولى.
4) السطر الأخير أو سطر بكلمة واحدة: محاذاة عادية إلى اليسار.

البرنامج بلغة C

```c
#include <stdio.h>

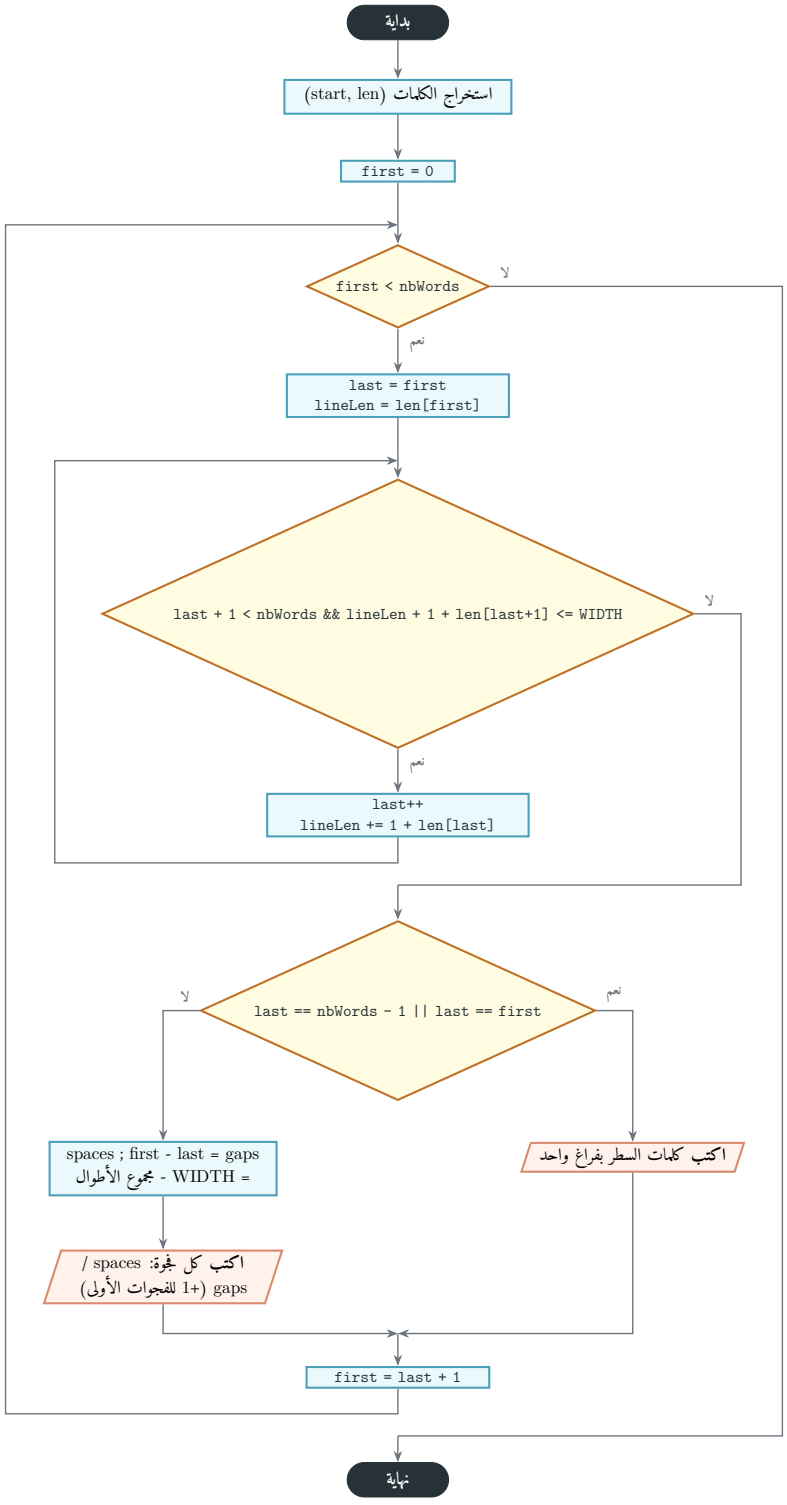
#define WIDTH 24

int main(void) {
    char text[] = "Wireless noise cancelling headphones with long battery life and fast charging case";
    int start[40], len[40], nbWords = 0, pos = 0;
    int first, last, lineLen, letters, gaps, spaces, word, gap, extra, col;
    while (text[pos] != '\0') {                          // استخراج الكلمات
        while (text[pos] == ' ') pos++;
        if (text[pos] == '\0') break;
        start[nbWords] = pos;
        while (text[pos] != ' ' && text[pos] != '\0') pos++;
        len[nbWords] = pos - start[nbWords];
        nbWords++;
    }
    printf("+------------------------+\n");
    for (first = 0; first < nbWords; first = last + 1) {
        last = first;
        lineLen = len[first];
        letters = len[first];
        while (last + 1 < nbWords && lineLen + 1 + len[last + 1] <= WIDTH) {
            last++;
            lineLen += 1 + len[last];
            letters += len[last];
        }
        gaps = last - first;
        spaces = WIDTH - letters;
        col = 0;
        putchar('|');
        for (word = first; word <= last; word++) {
            for (pos = 0; pos < len[word]; pos++) putchar(text[start[word] + pos]);
            col += len[word];
            if (word < last) {
                if (last == nbWords - 1) extra = 1;      // السطر الأخير: فراغ واحد
                else extra = spaces / gaps + (word - first < spaces % gaps ? 1 : 0);
                for (gap = 0; gap < extra; gap++) putchar(' ');
                col += extra;
            }
        }
        while (col++ < WIDTH) putchar(' ');              // إكمال السطر
        printf("|\n");
    }
    printf("+------------------------+\n");
    return 0;
}
```

نتيجة التنفيذ

```
+------------------------+
|Wireless           noise|
|cancelling    headphones|
|with  long  battery life|
|and fast charging case  |
+------------------------+
```

المخطط الإنسيابي (Organigramme)



تمرين 6.50 ضغط النص بالقاموس على طريقة LZW

علم البيانات صعب

لتقليل حجم سجلات النقرات، اكتب برنامجًا يضغط سلسلة من الحروف `A..Z` بخوارزمية **LZW**: القاموس يبدأ بالحروف المفردة (الرموز 0..25)، ثم يضيف كل تسلسل جديد يصادفه. اعرض الرموز الناتجة ونسبة الضغط.

خطوات الحل

1) القاموس: جدول سلاسل `dict[code]`، والرموز 0..25 للحروف.
2) `current` = أطول تسلسل موجود في القاموس؛ نجرب `current + next`.
3) إن وُجد في القاموس نمدد `current`، وإلا نُخرج رمز `current`، ونضيف `current + next`، ونبدأ من `next`.
4) نسبة الضغط: (عدد الرموز × 9 بتات) / (عدد الحروف × 8 بتات).

البرنامج بلغة C

```c
#include <stdio.h>

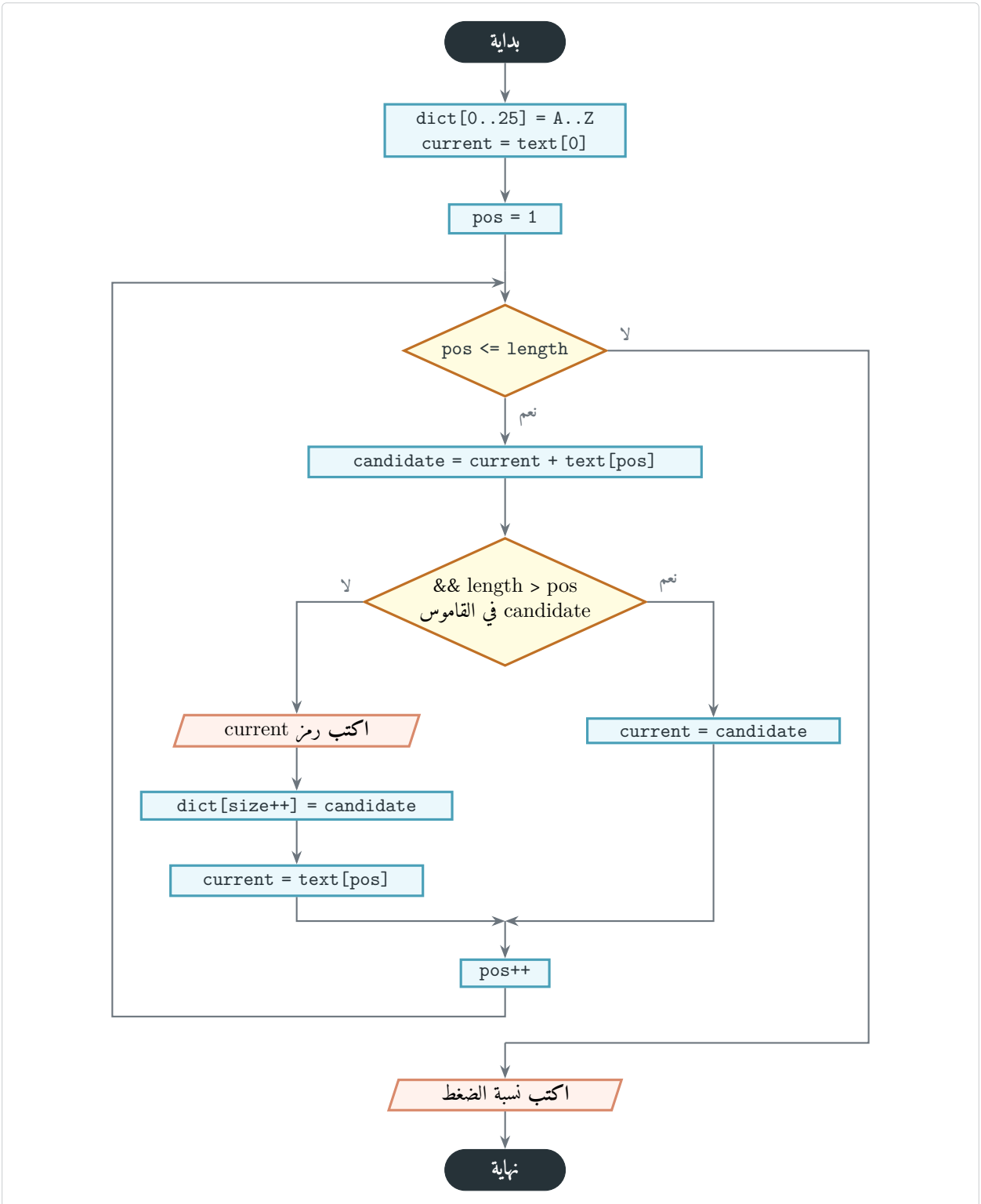
#define MAXD 128
#define MAXL 16

int main(void) {
    char text[] = "ABABABABCABCABCABCABAB";
    char dict[MAXD][MAXL], candidate[MAXL];
    int size = 26, length = 0, pos, code, letter, curLen, found, same, outCodes = 0, currentCode;
    for (code = 0; code < 26; code++) {              // القاموس الابتدائي
        dict[code][0] = (char)('A' + code);
        dict[code][1] = '\0';
    }
    while (text[length] != '\0') length++;
    candidate[0] = text[0];
    curLen = 1;
    currentCode = text[0] - 'A';
    printf("Codes:");
    for (pos = 1; pos <= length; pos++) {
        candidate[curLen] = text[pos];               // current + next
        candidate[curLen + 1] = '\0';
        found = -1;
        for (code = 0; pos < length && code < size && found < 0; code++) {
            same = 1;
            for (letter = 0; letter <= curLen + 1; letter++)
                if (dict[code][letter] != candidate[letter]) { same = 0; break; }
            if (same) found = code;
        }
        if (found >= 0) {
            currentCode = found;                     // نمدد التسلسل
            curLen++;
        } else {
            printf(" %d", currentCode);
            outCodes++;
            if (size < MAXD && curLen + 1 < MAXL) {  // تسلسل جديد في القاموس
                for (letter = 0; letter <= curLen + 1; letter++) dict[size][letter] = candidate[letter];
                size++;
            }
            candidate[0] = text[pos];
            curLen = 1;
            currentCode = text[pos] - 'A';
        }
    }
    printf("\n%d chars -> %d codes, dictionary size %d\n", length, outCodes, size);
    printf("Compression ratio: %.2f\n", (outCodes * 9.0) / (length * 8.0));
    return 0;
}
```

نتيجة التنفيذ

```
Codes: 0 1 26 28 1 2 26 31 30 32 29
22 chars -> 11 codes, dictionary size 37
Compression ratio: 0.56
```

المخطط الإنسيابي (Organigramme)



الفصل 7

الدوال

تعريفات

- **الدالة**: مجموعة تعليمات لها اسم، تستقبل **معاملات** وقد تعيد **قيمة** بـ `return`.
- **التعريف**: `double priceWithTax(double price) { return price * 1.19; }`
- **الاستدعاء**: `total = priceWithTax(4500);` تُنسخ القيمة 4500 في المعامل `price`.
- **الدالة** `void` لا تعيد شيئًا: تطبع أو تعدل جدولًا ممررًا.



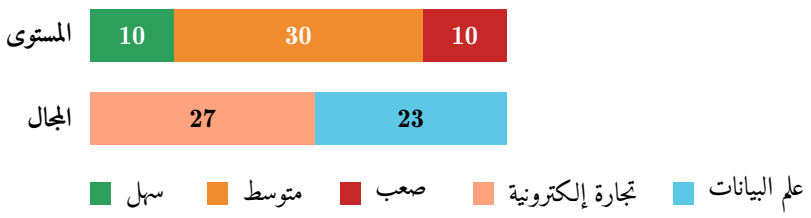
نوع المعامل	ما يحدث عند الاستدعاء
عدد (`double`، `int`)	**تمرير بالقيمة**: الدالة تعمل على نسخة، والأصل لا يتغير
جدول (`double values[]`)	يُمرر **عنوانه**: تعديلات الدالة تبقى بعد الخروج منها
حجم الجدول	يُمرر دائمًا في معامل منفصل `int size`

دالة واحدة لمهمة واحدة

الدالة القصيرة ذات الاسم الواضح (`subtotal`، `tax`، `shipping`) أسهل للفهم والاختبار وإعادة الاستعمال. نعرّف الدالة قبل `main` لكي يعرفها المترجم عند الاستدعاء.

للاستزادة: [1, 2, 3]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 7.1 دالة السعر مع الرسم على القيمة المضافة

تجارة إلكترونية سهل

اكتب دالة `priceWithTax(price)` تعيد السعر بعد إضافة الرسم على القيمة المضافة **19%**، ثم استعملها في `main` لعرض السعر النهائي لثلاثة منتجات.

خطوات الحل

1) الدالة تأخذ `double` وتعيد `double`: `return price * 1.19;`.
2) في `main` نستدعيها لكل منتج ونعرض النتيجة.

البرنامج بلغة C

```c
#include <stdio.h>

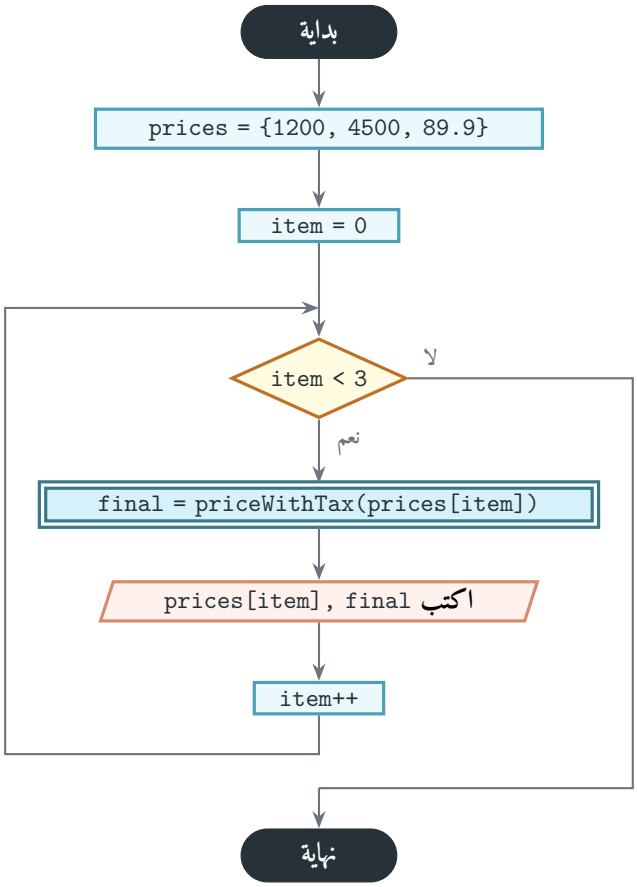
double priceWithTax(double price) {
    return price * 1.19;              // 19% TVA
}

int main(void) {
    double prices[3] = {1200, 4500, 89.9};
    int item;
    for (item = 0; item < 3; item++)
        printf("%8.2f DA -> %8.2f DA\n", prices[item], priceWithTax(prices[item]));
    return 0;
}
```

نتيجة التنفيذ

```
 1200.00 DA ->  1428.00 DA
 4500.00 DA ->  5355.00 DA
   89.90 DA ->   106.98 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.2 دالة الأكبر بين عرضين

تجارة إلكترونية سهل

يقارن الزبون بين عرضي سعر لنفس المنتج من بائعين. اكتب دالة `minPrice(first, second)` تعيد **الأصغر** بينهما، ودالة `maxPrice` تعيد الأكبر، ثم احسب الفرق (التوفير).

خطوات الحل

1) كل دالة تحتوي على `if` واحدة وتعيد إحدى القيمتين.
2) التوفير = `maxPrice - minPrice`.

البرنامج بلغة C

```c
#include <stdio.h>

double minPrice(double first, double second) {
    if (first < second)
        return first;
    return second;
}

double maxPrice(double first, double second) {
    return first > second ? first : second;      // نفس الفكرة بالعامل الشرطي
}

int main(void) {
```

```c
    double first, second;
    scanf("%lf %lf", &first, &second);
    printf("Best offer: %.2f DA\n", minPrice(first, second));
    printf("You save:   %.2f DA\n", maxPrice(first, second) - minPrice(first, second));
    return 0;
}
```

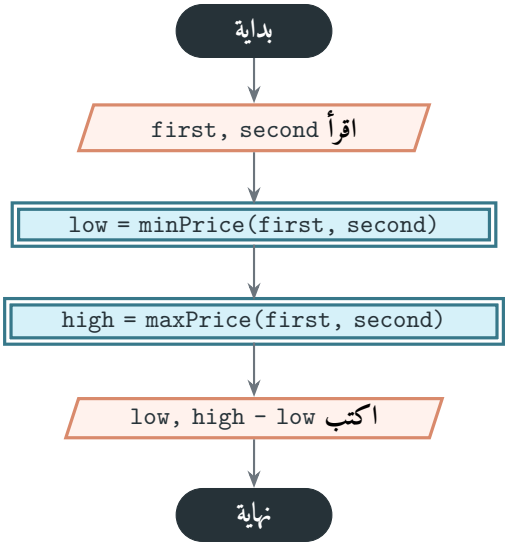
المدخلات

```
5400 4950
```

نتيجة التنفيذ

```
Best offer: 4950.00 DA
You save:   450.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.3 دالة تحويل العملة

تجارة إلكترونية
سهل

اكتب دالة `toEuro(dinars, rate)` تحول مبلغًا بالدينار إلى الأورو، ودالة `toDinar(euros, rate)` تقوم بالتحويل العكسي، ثم تحقق أن التحويل ذهابًا وإيابًا يعيد المبلغ الأصلي.

خطوات الحل

1) `toEuro` تقسم على سعر الصرف، و `toDinar` تضرب فيه.
2) نمرر سعر الصرف كمعامل بدل متغير عام.

البرنامج بلغة C

```c
#include <stdio.h>

double toEuro(double dinars, double rate) {
    return dinars / rate;
}

```

```c
double toDinar(double euros, double rate) {
    return euros * rate;
}

int main(void) {
    double dinars, rate, euros;
    scanf("%lf %lf", &dinars, &rate);
    euros = toEuro(dinars, rate);                  // استدعاء أول
    printf("%.2f DA = %.2f EUR\n", dinars, euros);
    printf("Back: %.2f DA\n", toDinar(euros, rate));  // التحويل العكسي
    return 0;
}
```

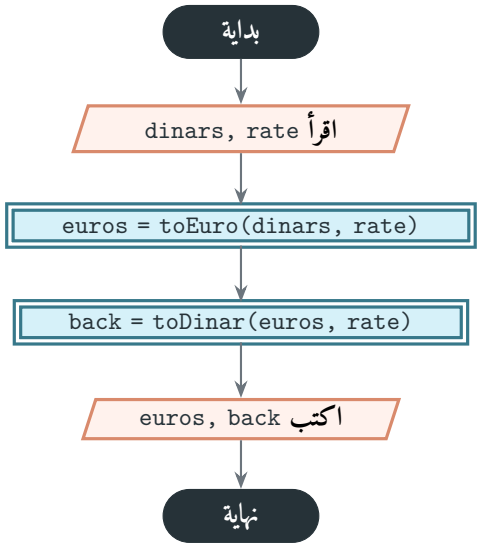
المدخلات

```
25000 148.5
```

نتيجة التنفيذ

```
25000.00 DA = 168.35 EUR
Back: 25000.00 DA
```

المخطط الانسيابي (Organigramme)



تمرين 7.4 دالة توفر المنتج في المخزون

تجارة إلكترونية سهل

اكتب دالة `isAvailable(stock, ordered)` تعيد 1 إذا كان المخزون يكفي للكمية المطلوبة و0 في الحالة المعاكسة، واستعملها لفحص أربع طلبيات.

خطوات الحل

1) الدالة تعيد `int` يُستعمل كقيمة منطقية.
2) في `main`: `if (isAvailable(...))`.

البرنامج بلغة C

```c
#include <stdio.h>

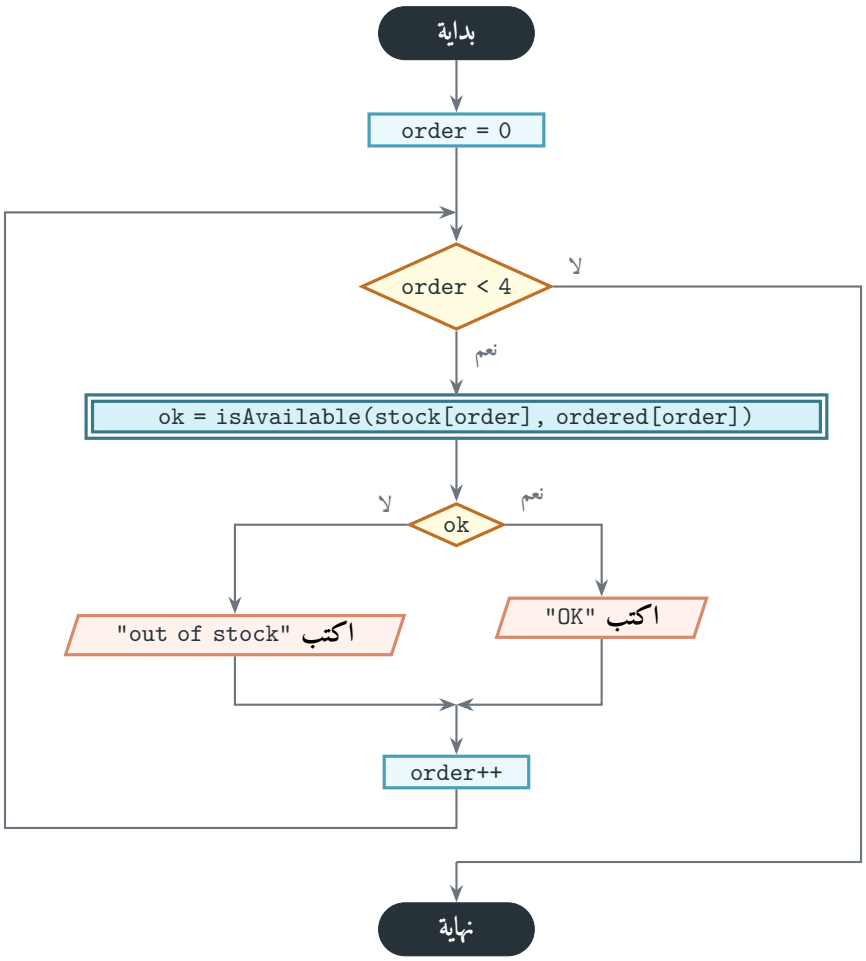
int isAvailable(int stock, int ordered) {
    return ordered <= stock;          // 0 أو 1
}

int main(void) {
    int stock[4] = {10, 0, 5, 7};
    int ordered[4] = {3, 1, 5, 9};
    int order;
    for (order = 0; order < 4; order++) {
        if (isAvailable(stock[order], ordered[order]))
            printf("Order %d: OK\n", order + 1);
        else
            printf("Order %d: out of stock (%d left)\n", order + 1, stock[order]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Order 1: OK
Order 2: out of stock (0 left)
Order 3: OK
Order 4: out of stock (7 left)
```

المخطط الإنسيابي (Organigramme)



تمرين 7.5 دالة رسم خط فاصل في الفاتورة

تجارة إلكترونية سهل

اكتب دالة من نوع `void` اسمها `printLine(symbol, width)` ترسم سطراً من الرمز المعطى بالطول المعطى، واستعملها لتأطير رأس فاتورة.

خطوات الحل

1) الدالة لا تعيد شيئًا (`void`) بل تطبع فقط.
2) نستدعيها قبل العنوان وبعده بعرضين ورمزين مختلفين.

البرنامج بلغة C

```c
#include <stdio.h>

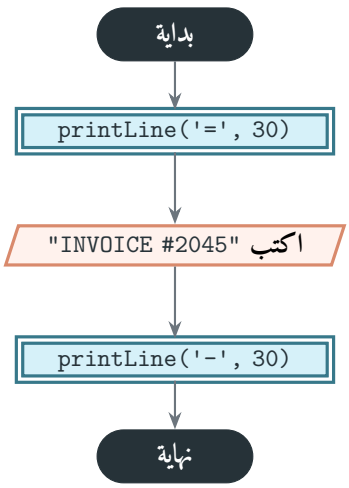
void printLine(char symbol, int width) {
    int col;
    for (col = 0; col < width; col++)
        putchar(symbol);
    putchar('\n');                    // لا قيمة معادة
}

int main(void) {
    printLine('=', 30);
    printf("  INVOICE #2045  - TechShop\n");
    printLine('-', 30);
    printf("  Total: 12500.00 DA\n");
    printLine('=', 30);
    return 0;
}
```

نتيجة التنفيذ

```
==============================
  INVOICE #2045  - TechShop
------------------------------
  Total: 12500.00 DA
==============================
```

المخطط الإنسيابي (Organigramme)



تمرين 7.6 دالة متوسط ثلاثة تقييمات

سهل ⭐ | علم البيانات

لكل منتج ثلاثة تقييمات من 5. اكتب دالة `average3(first, second, third)` تعيد المتوسط، ثم اعرض متوسط ثلاثة منتجات.

خطوات الحل

1) نقسم على `3.0` لنحصل على نتيجة عشرية.
2) نستدعي الدالة ثلاث مرات بقيم مختلفة.

البرنامج بلغة C

```c
#include <stdio.h>

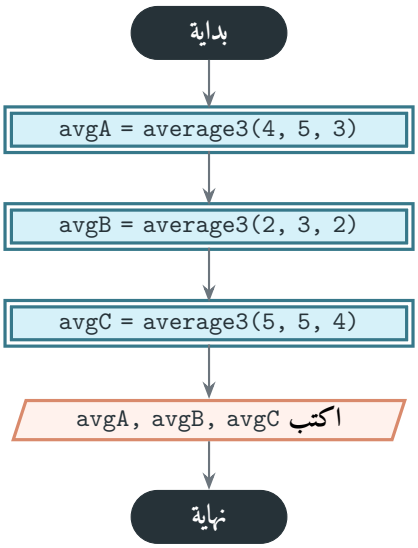
double average3(int first, int second, int third) {
    return (first + second + third) / 3.0;   // 3.0 لتفادي القسمة الصحيحة
}

int main(void) {
    printf("Headphones: %.2f\n", average3(4, 5, 3));
    printf("Charger:    %.2f\n", average3(2, 3, 2));
    printf("Keyboard:   %.2f\n", average3(5, 5, 4));
    return 0;
}
```

نتيجة التنفيذ

```
Headphones: 4.00
Charger:    2.33
Keyboard:   4.67
```

المخطط الإنسيابي (Organigramme)



تمرين 7.7 دالة تكلفة التوصيل حسب المسافة

سهل ⭐ | تجارة إلكترونية

تكلفة التوصيل: 300 دج حتى 10 كم، ثم 25 دج لكل كيلومتر إضافي، والتوصيل مجاني إذا تجاوز مبلغ الطلبية 10000 دج. اكتب دالة `deliveryFee(distance, amount)` تطبق هذه القواعد.

خطوات الحل

1) إذا `amount > 10000`: نعيد 0.
2) إذا `distance <= 10`: نعيد 300، وإلا `300 + (distance - 10) * 25`.

البرنامج بلغة C

```c
#include <stdio.h>

int deliveryFee(int distance, double amount) {
    if (amount > 10000)
        return 0;                          // توصيل مجاني
    if (distance <= 10)
        return 300;
    return 300 + (distance - 10) * 25;     // كيلومترات إضافية
}

int main(void) {
    int distance;
    double amount;
    scanf("%d %lf", &distance, &amount);
    printf("Delivery fee: %d DA\n", deliveryFee(distance, amount));
    printf("Same order above 10000 DA: %d DA\n", deliveryFee(distance, 12000));
    return 0;
}
```

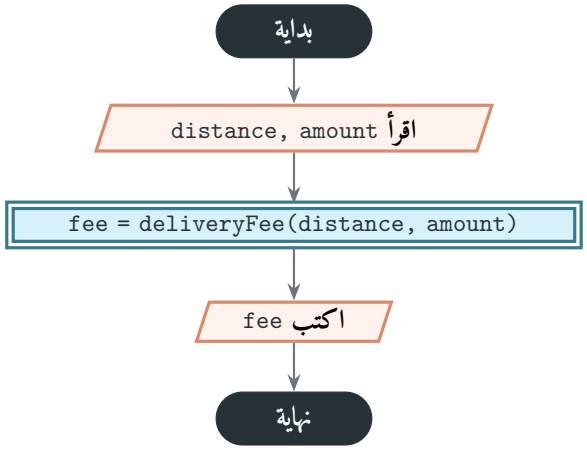
المدخلات

```
24 7800
```

نتيجة التنفيذ

```
Delivery fee: 650 DA
Same order above 10000 DA: 0 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.8 دالة المربع في حساب الخطأ — علم البيانات — سهل

اكتب دالة `square(value)` تعيد مربع عدد، واستعملها لحساب **مجموع مربعات الأخطاء** بين المبيعات المتوقعة والحقيقية لخمسة أيام.

خطوات الحل

1) `square` تعيد `value * value`.
2) الخطأ اليومي = الحقيقي - المتوقع، ونجمع مربعاته.

البرنامج بلغة C

```c
#include <stdio.h>

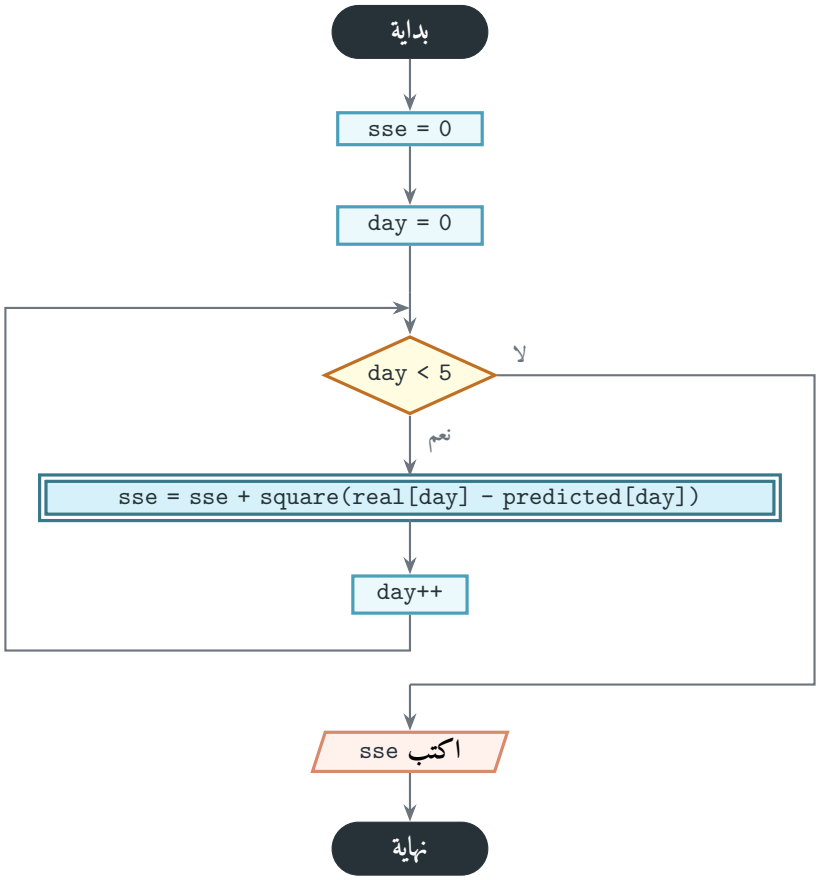
double square(double value) {
    return value * value;
}

int main(void) {
    double real[5] = {120, 135, 128, 150, 142};
    double predicted[5] = {118, 130, 133, 147, 145};
    double sse = 0;
    int day;
    for (day = 0; day < 5; day++)
        sse += square(real[day] - predicted[day]);   // الخطأ مرفوع للمربع
    printf("Sum of squared errors: %.1f\n", sse);
    printf("Mean squared error:    %.1f\n", sse / 5);
    return 0;
}
```

نتيجة التنفيذ

```
Sum of squared errors: 72.0
Mean squared error:    14.4
```

المخطط الإنسيابي (Organigramme)



تمرين 7.9 دالة السنة الكبيسة

علم البيانات سهل

لحساب عدد أيام شهر فيفري في تقارير المبيعات، اكتب دالة `isLeap(year)` تعيد 1 إذا كانت السنة كبيسة، ثم دالة `daysInFebruary(year)`.

خطوات الحل

1) كبيسة إذا قبلت القسمة على 4 دون 100، أو قبلت القسمة على 400.
2) `daysInFebruary` تستدعي `isLeap`: دالة تستعمل دالة أخرى.

البرنامج بلغة C

```c
#include <stdio.h>

int isLeap(int year) {
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}

int daysInFebruary(int year) {
    return isLeap(year) ? 29 : 28;      // دالة تستدعي دالة
}

int main(void) {
    int year, idx;
```

```c
    for (idx = 0; idx < 4; idx++) {
        scanf("%d", &year);
        printf("February %d: %d days\n", year, daysInFebruary(year));
    }
    return 0;
}
```

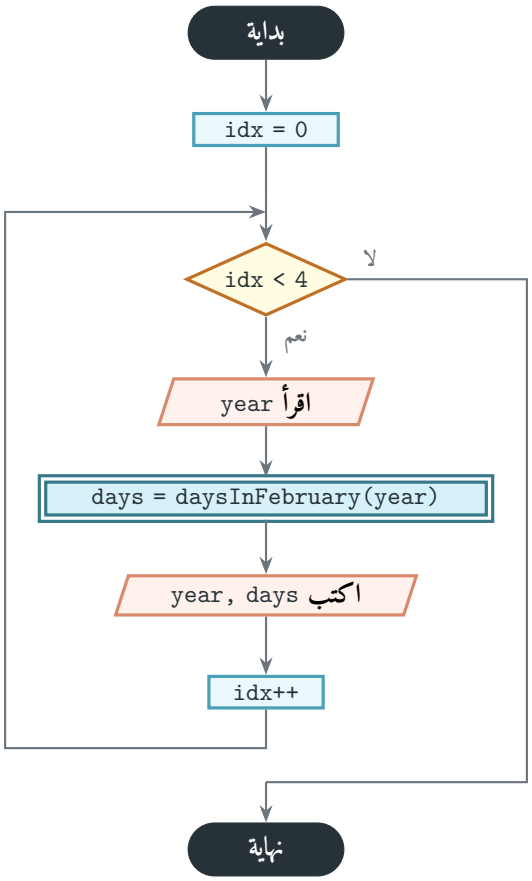
المدخلات

```
2024 2026 2100 2000
```

نتيجة التنفيذ

```
February 2024: 29 days
February 2026: 28 days
February 2100: 28 days
February 2000: 29 days
```

المخطط الإنسيابي (Organigramme)



تمرين 7.10 دالة النسبة المئوية للتغير

علم البيانات ★ سهل

اكتب دالة `percentChange(oldValue, newValue)` تعيد نسبة التغير بين مبيعات شهرين، واستعملها لعرض تطور المبيعات على أربعة أشهر.

خطوات الحل

1) النسبة = `(newValue - oldValue) / oldValue * 100`.
2) نستدعيها بين كل شهر والشهر الذي يليه.

البرنامج بلغة C

```c
#include <stdio.h>

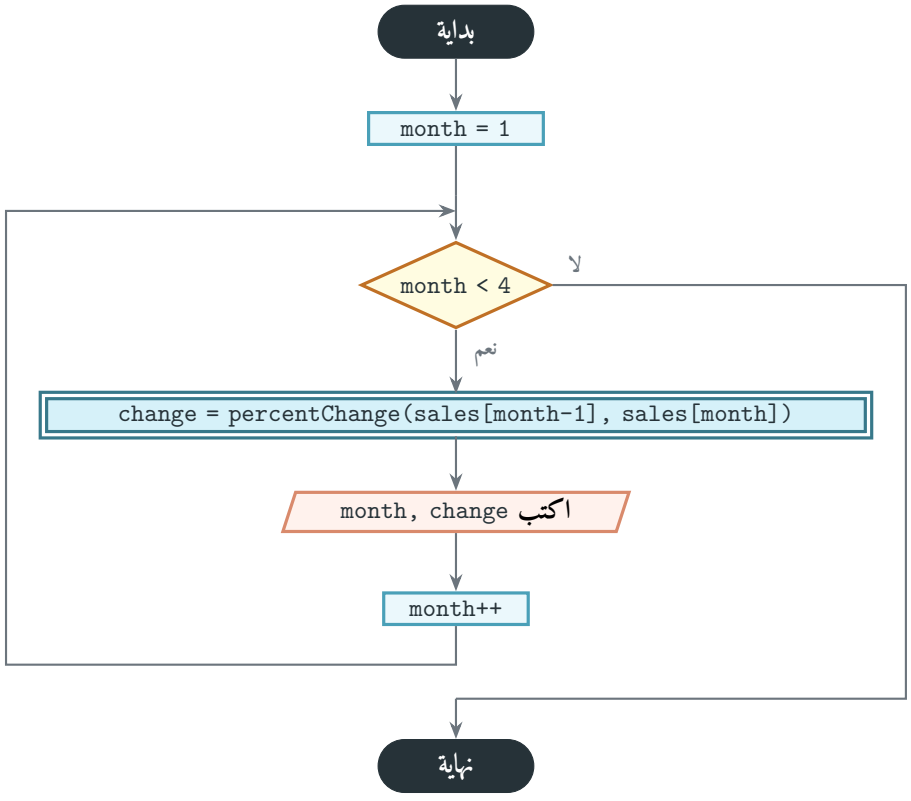
double percentChange(double oldValue, double newValue) {
    return (newValue - oldValue) / oldValue * 100;
}

int main(void) {
    double sales[4] = {80000, 92000, 87400, 104880};
    int month;
    for (month = 1; month < 4; month++)
        printf("Month %d -> %d: %+.1f%%\n", month, month + 1,
               percentChange(sales[month - 1], sales[month]));   // + لعرض الإشارة
    return 0;
}
```

نتيجة التنفيذ

```
Month 1 -> 2: +15.0%
Month 2 -> 3: -5.0%
Month 3 -> 4: +20.0%
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 7.11 دالة الجذر التربيعي بطريقة نيوتن

متوسط ★★ | علم البيانات

اكتب دالة `mySqrt(value)` تحسب الجذر التربيعي دون `sqrt` بطريقة نيوتن: `guess = (guess + value / guess) / 2` حتى يصبح الفرق بين تقريبين متتاليين أقل من `1e-9`. استعملها لحساب الانحراف المعياري لمبيعات أسبوع.

خطوات الحل

1) نبدأ بـ `guess = value` (أو 1 إذا كان `value < 1`)، ونعيد 0 إذا كان `value == 0`.
2) نكرر صيغة نيوتن حتى الاستقرار.
3) الانحراف المعياري = `mySqrt(  )`.

البرنامج بلغة C

```c
#include <stdio.h>

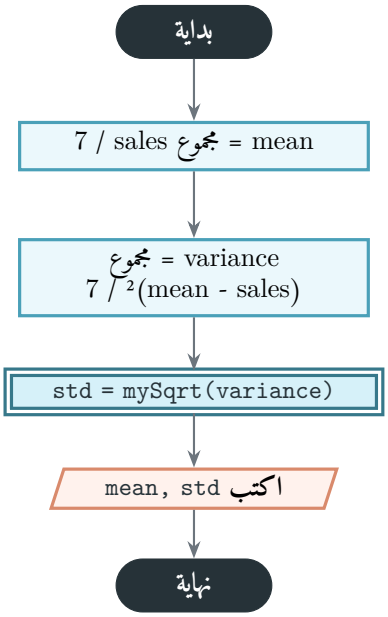
double mySqrt(double value) {
    double guess, previous;
    if (value <= 0) return 0;
    guess = value > 1 ? value : 1;
    do {
        previous = guess;
        guess = (guess + value / guess) / 2;      // صيغة نيوتن
    } while (previous - guess > 1e-9 || guess - previous > 1e-9);
    return guess;
}

int main(void) {
    double sales[7] = {52, 61, 48, 70, 66, 59, 64};
    double mean = 0, variance = 0;
    int day;
    for (day = 0; day < 7; day++) mean += sales[day];
    mean /= 7;
    for (day = 0; day < 7; day++)
        variance += (sales[day] - mean) * (sales[day] - mean);
    variance /= 7;
    printf("sqrt(2) = %.9f\n", mySqrt(2));
    printf("Mean = %.2f, std = %.3f\n", mean, mySqrt(variance));
    return 0;
}
```

نتيجة التنفيذ

```
sqrt(2) = 1.414213562
Mean = 60.00, std = 7.191
```

المخطط الإنسيابي (Organigramme)



تمرين 7.12 دالة القوة للفائدة المركبة

تجارة إلكترونية ★★ متوسط

اكتب دالة `power(base, exponent)` (أس صحيح موجب) دون `pow`، واستعملها لحساب قيمة رصيد محفظة الزبون بعد `years` سنة بفائدة سنوية `rate%`: `capital * (1 + rate/100)^years`.

خطوات الحل

1) `power` تضرب `base` في نفسها `exponent` مرة في حلقة.
2) نعرض الرصيد بعد كل سنة من 1 إلى `years`.

البرنامج بلغة C

```c
#include <stdio.h>

double power(double base, int exponent) {
    double result = 1;
    int step;
    for (step = 0; step < exponent; step++)
        result *= base;                         // ضرب متكرر
    return result;
}

int main(void) {
    double capital, rate;
    int years, year;
    scanf("%lf %lf %d", &capital, &rate, &years);
    for (year = 1; year <= years; year++)
        printf("Year %d: %.2f DA\n", year, capital * power(1 + rate / 100, year));
    return 0;
}
```

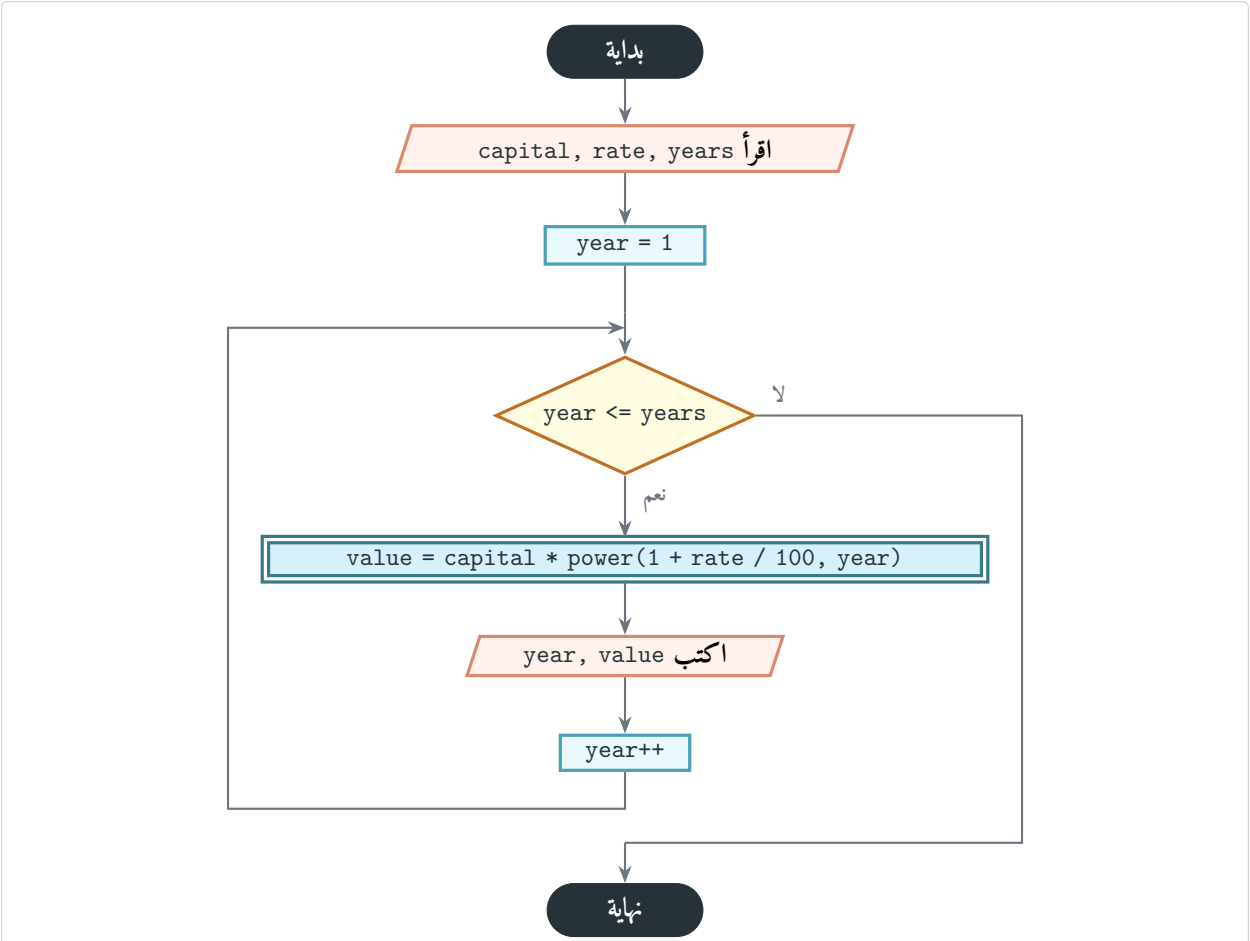
المدخلات

```
50000 6 5
```

نتيجة التنفيذ

```
Year 1: 53000.00 DA
Year 2: 56180.00 DA
Year 3: 59550.80 DA
Year 4: 63123.85 DA
Year 5: 66911.28 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.13 دالة مجموع جدول ومتوسطه

علم البيانات

متوسط

اكتب دالتين تستقبلان **جدولًا** وعدد عناصره: `sumArray(values, size)` و `meanArray(values, size)`، حيث تستدعي الثانية الأولى. استعملهما على مبيعات فرعين بأحجام مختلفة.

خطوات الحل

1) الجدول يُمرَّر بـ `double values[]` مع حجمه في معامل منفصل.
2) `meanArray` تعيد `sumArray(values, size) / size`.

البرنامج بلغة C

```c
#include <stdio.h>

double sumArray(double values[], int size) {
    double total = 0;
```

```c
    int idx;
    for (idx = 0; idx < size; idx++)
        total += values[idx];
    return total;
}

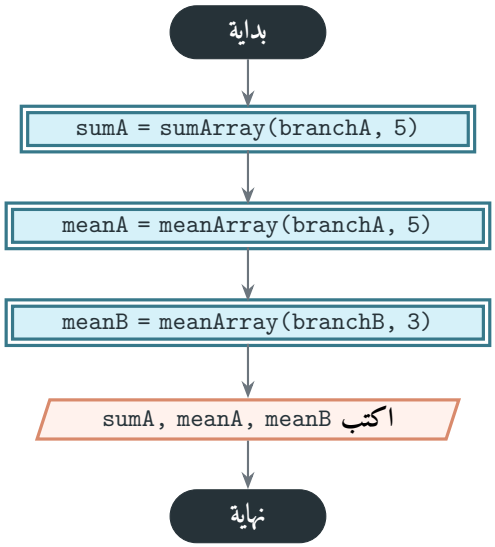
double meanArray(double values[], int size) {
    return sumArray(values, size) / size;    // إعادة استعمال الدالة الأولى
}

int main(void) {
    double branchA[5] = {1200, 980, 1430, 1100, 1290};
    double branchB[3] = {640, 710, 690};
    printf("Branch A: total %.0f, mean %.2f\n", sumArray(branchA, 5), meanArray(branchA, 5));
    printf("Branch B: total %.0f, mean %.2f\n", sumArray(branchB, 3), meanArray(branchB, 3));
    return 0;
}
```

نتيجة التنفيذ

```
Branch A: total 6000, mean 1200.00
Branch B: total 2040, mean 680.00
```

المخطط الإنسيابي (Organigramme)



تمرين 7.14 دالة البحث عن منتج بالرمز

تجارة إلكترونية ★★ متوسط

اكتب دالة `findProduct(codes, size, target)` تعيد **فهرس** المنتج ذي الرمز المطلوب أو `-1` إذا لم يوجد، ثم استعملها لعرض سعر المنتجات المطلوبة.

خطوات الحل

1) بحث تسلسلي: نعيد الفهرس فور العثور على الرمز (`return` داخل الحلقة).
2) بعد الحلقة نعيد `-1`.

البرنامج بلغة C

```c
#include <stdio.h>

int findProduct(int codes[], int size, int target) {
```

```
    int idx;
    for (idx = 0; idx < size; idx++)
        if (codes[idx] == target)
            return idx;                 // نخرج فور العثور
    return -1;                          // غير موجود
}

int main(void) {
    int codes[6] = {1001, 2045, 3310, 5120, 7781, 9002};
    double prices[6] = {450, 12500, 899, 3200, 27000, 650};
    int nbQueries, query, target, idx;
    scanf("%d", &nbQueries);
    for (query = 0; query < nbQueries; query++) {
        scanf("%d", &target);
        idx = findProduct(codes, 6, target);
        if (idx >= 0)
            printf("Product %d: %.2f DA\n", target, prices[idx]);
        else
            printf("Product %d: not found\n", target);
    }
    return 0;
}
```

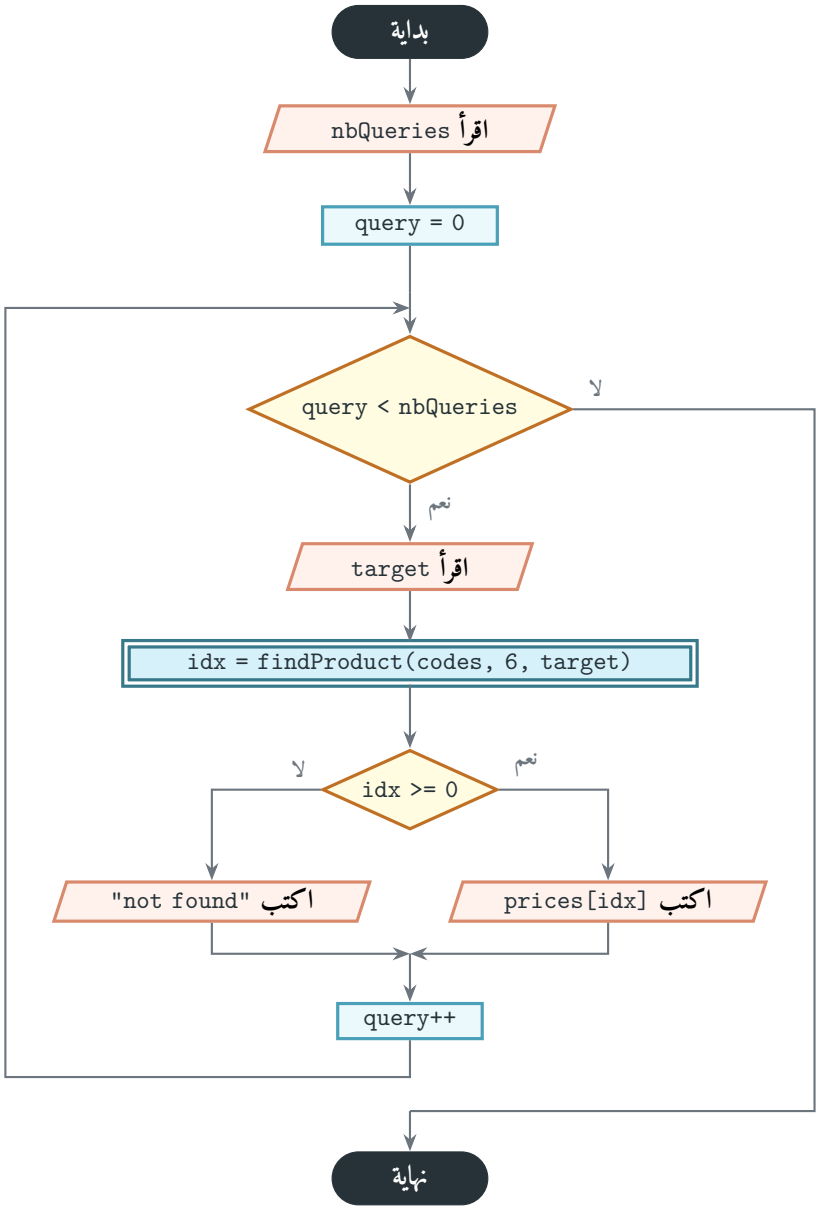
المدخلات

```
3 2045 1111 7781
```

نتيجة التنفيذ

```
Product 2045: 12500.00 DA
Product 1111: not found
Product 7781: 27000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.15 دالة ترتيب الأسعار

تجارة إلكترونية

متوسط

اكتب دالة `sortPrices(prices, size)` ترتب جدول الأسعار تصاعديًا بالترتيب بالفقاعات، ودالة `printPrices` تعرضه. لاحظ أن الجدول الممرر **يتغير فعلًا** بعد الاستدعاء.

خطوات الحل

1) الجدول يُمرَّر بعنوانه، لذلك التعديلات داخل الدالة تبقى بعد الخروج منها.
2) الترتيب بالفقاعات: نقارن كل عنصرين متجاورين ونبدلهما إذا كانا في غير الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>

void sortPrices(double prices[], int size) {
    int pass, idx;
    double tmp;
    for (pass = 0; pass < size - 1; pass++)
        for (idx = 0; idx < size - 1 - pass; idx++)
            if (prices[idx] > prices[idx + 1]) {
                tmp = prices[idx];                // تبديل
                prices[idx] = prices[idx + 1];
                prices[idx + 1] = tmp;
            }
}

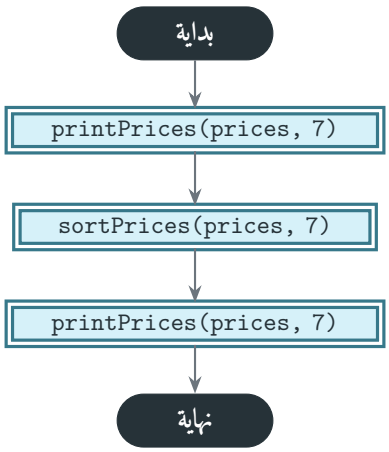
void printPrices(double prices[], int size) {
    int idx;
    for (idx = 0; idx < size; idx++)
        printf("%.0f ", prices[idx]);
    printf("\n");
}

int main(void) {
    double prices[7] = {4500, 1200, 899, 27000, 650, 3200, 12500};
    printf("Before: ");
    printPrices(prices, 7);
    sortPrices(prices, 7);                        // يعدل الجدول الأصلي
    printf("After:  ");
    printPrices(prices, 7);
    return 0;
}
```

نتيجة التنفيذ

```
Before: 4500 1200 899 27000 650 3200 12500
After:  650 899 1200 3200 4500 12500 27000
```

المخطط الإنسيابي (Organigramme)



تمرين 7.16 دالة طول السلسلة ودالة المقارنة

تجارة إلكترونية

متوسط

اكتب دالتين بديلتين لـ strlen و strcmp: textLength(text) و textCompare(first, second) (تعيد سالبًا أو 0 أو موجبًا)، واستعملهما للتحقق من اسم مستخدم وكلمة مروره.

خطوات الحل

1) textLength: تعد الحروف حتى '\0'.
2) textCompare: تتقدم ما دام الحرفان متساويين وغير منتهيين، ثم تعيد فرق الحرفين.

البرنامج بلغة C

```c
#include <stdio.h>

int textLength(char text[]) {
    int len = 0;
    while (text[len] != '\0')
        len++;
    return len;
}

int textCompare(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos])
        pos++;
    return first[pos] - second[pos];        // أول اختلاف يحدد النتيجة
}

int main(void) {
    char user[30], password[30];
    scanf("%29s %29s", user, password);
    printf("Password length: %d\n", textLength(password));
    printf("compare(\"apple\", \"apply\") = %d\n", textCompare("apple", "apply"));
    if (textCompare(user, "admin") == 0 && textLength(password) >= 8
        && textCompare(password, "shop2026") == 0)
        printf("Access granted\n");
    else
        printf("Access denied\n");
    return 0;
}
```

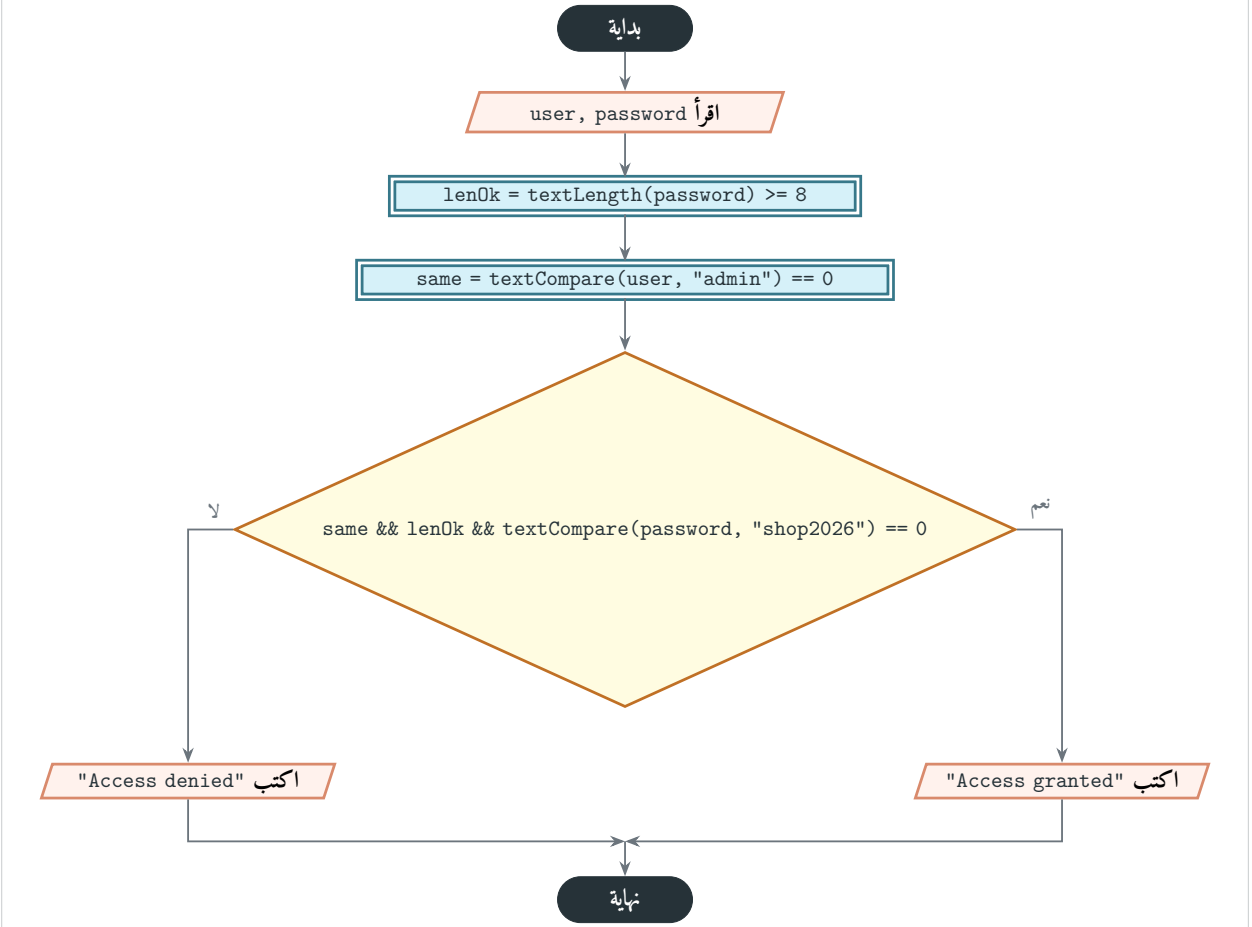
المدخلات

```
admin shop2026
```

نتيجة التنفيذ

```
Password length: 8
compare("apple", "apply") = -20
Access granted
```

المخطط الإنسيابي (Organigramme)



تمرين 7.17 دالة القاسم المشترك الأكبر لتوزيع العلب

تجارة إلكترونية ★★ متوسط

لدى المخزن 84 قلمًا و 120 دفترًا و 156 ممحاة. نريد تكوين أكبر عدد من **العلب المتطابقة** دون بقايا. اكتب دالة `gcd(first, second)` بخوارزمية **إقليدس** واستعملها لحساب القاسم المشترك الأكبر للثلاثة.

خطوات الحل

1) إقليدس: ما دام `second != 0` : `(first, second) = (second, first % second)`.
2) `gcd` لثلاثة أعداد = `gcd(gcd(first, second), third)`.
3) كل علبة تحتوي على `count / boxes` من كل صنف.

البرنامج بلغة C

```c
#include <stdio.h>

int gcd(int first, int second) {
    int rest;
    while (second != 0) {
        rest = first % second;          // إقليدس
        first = second;
        second = rest;
    }
```

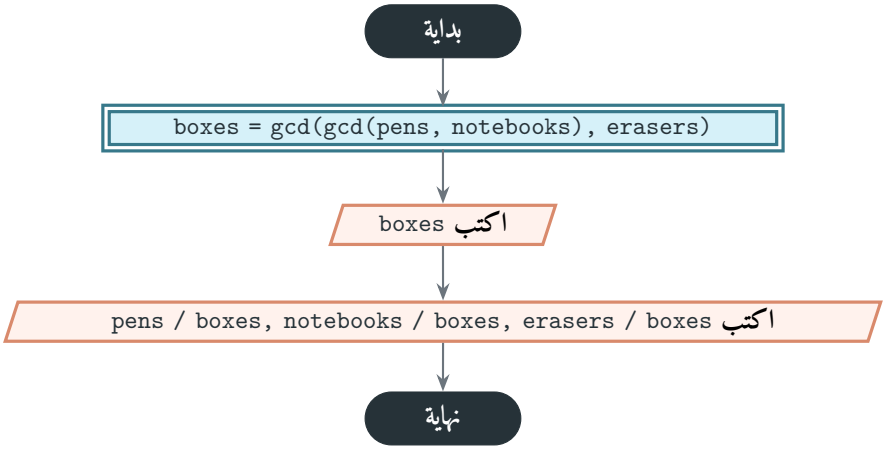
```c
    return first;
}

int main(void) {
    int pens = 84, notebooks = 120, erasers = 156;
    int boxes = gcd(gcd(pens, notebooks), erasers);   // استدعاء متداخل
    printf("Number of identical boxes: %d\n", boxes);
    printf("Each box: %d pens, %d notebooks, %d erasers\n",
           pens / boxes, notebooks / boxes, erasers / boxes);
    return 0;
}
```

نتيجة التنفيذ

```
Number of identical boxes: 12
Each box: 7 pens, 10 notebooks, 13 erasers
```

المخطط الإنسيابي (Organigramme)



تمرين 7.18 دالة العدد الأولي لأرقام القسائم

تجارة إلكترونية ★★ متوسط

يُعتبر رقم القسيمة صالحًا إذا كان عددًا أوليًا. اكتب دالة `isPrime(number)` تختبر القواسم حتى الجذر التربيعي فقط (`div * div <= number`)، ثم اعرض القسائم الصالحة بين 1000 و 1100.

خطوات الحل

1) الأعداد أقل من 2 ليست أولية؛ العدد 2 أولي؛ الأعداد الزوجية الأخرى ليست أولية.
2) نجرب القواسم الفردية `divisor` ما دام `divisor * divisor <= number`.

البرنامج بلغة C

```c
#include <stdio.h>

int isPrime(int number) {
    int divisor;
    if (number < 2) return 0;
    if (number % 2 == 0) return number == 2;
    for (divisor = 3; divisor * divisor <= number; divisor += 2)   // حتى الجذر فقط
        if (number % divisor == 0)
            return 0;
```

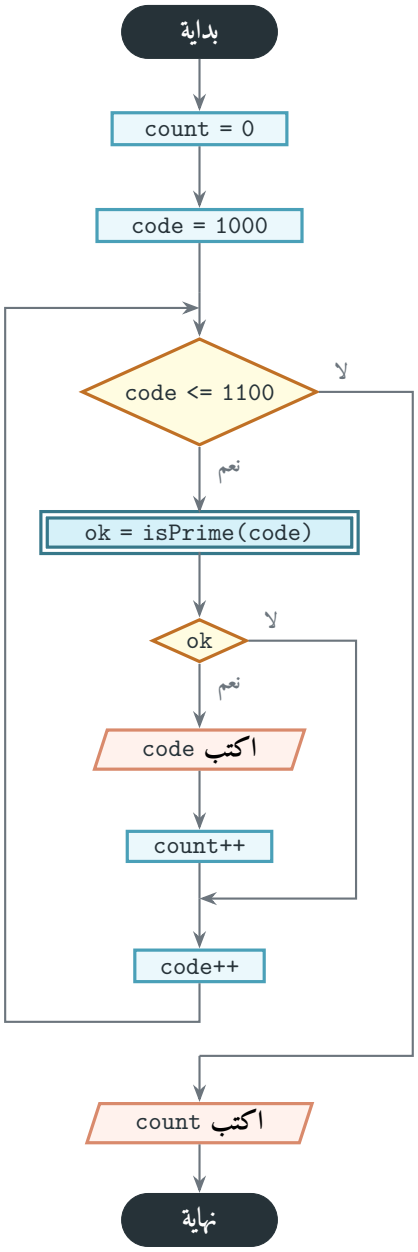
```c
    return 1;
}

int main(void) {
    int code, count = 0;
    for (code = 1000; code <= 1100; code++)
        if (isPrime(code)) {
            printf("%d ", code);
            count++;
        }
    printf("\n%d valid coupons\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
1009 1013 1019 1021 1031 1033 1039 1049 1051 1061 1063 1069 1087 1091 1093 1097
16 valid coupons
```

المخطط الإنسيابي (Organigramme)



تمرين 7.19 دالة تحويل المدة إلى ساعات ودقائق

علم البيانات ★★ متوسط

يسجل الموقع مدة كل جلسة بالثواني. اكتب دالة `printDuration(seconds)` تعرضها بصيغة `hh:mm:ss`، ودالة `toSeconds(hours, minutes, secs)` للتحويل العكسي، ثم احسب متوسط مدة الجلسات.

خطوات الحل

1) الساعات `seconds / 3600`، الدقائق `(seconds % 3600) / 60`، الثواني `seconds % 60`.

2) `%02d` يضيف صفراً على اليسار.

البرنامج بلغة C

```c
#include <stdio.h>

void printDuration(int seconds) {
    printf("%02d:%02d:%02d\n", seconds / 3600, (seconds % 3600) / 60, seconds % 60);
}

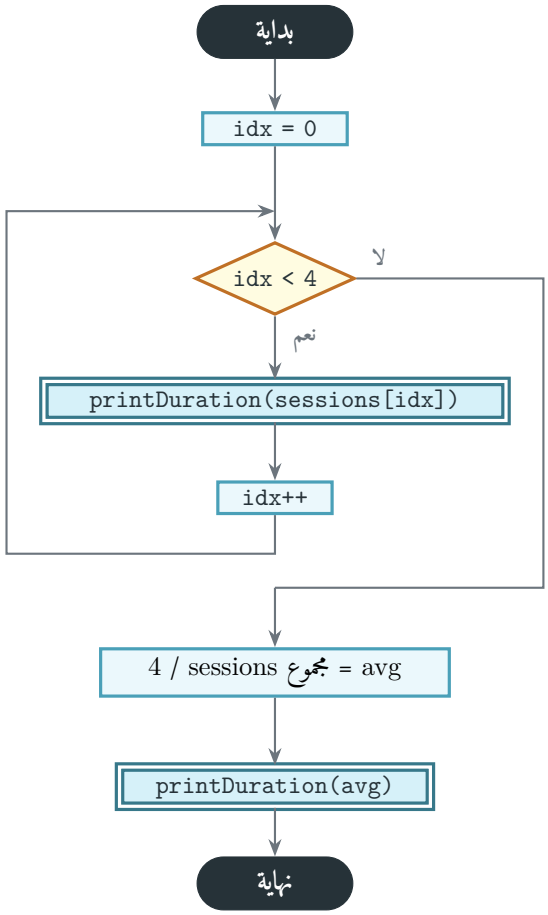
int toSeconds(int hours, int minutes, int secs) {
    return hours * 3600 + minutes * 60 + secs;
}

int main(void) {
    int sessions[4] = {754, 3725, 192, toSeconds(1, 5, 30)};
    int idx, total = 0;
    for (idx = 0; idx < 4; idx++) {
        printf("Session %d: ", idx + 1);
        printDuration(sessions[idx]);
        total += sessions[idx];
    }
    printf("Average:   ");
    printDuration(total / 4);
    return 0;
}
```

نتيجة التنفيذ

```
Session 1: 00:12:34
Session 2: 01:02:05
Session 3: 00:03:12
Session 4: 01:05:30
Average:   00:35:50
```

المخطط الإنسيابي (Organigramme)



تمرين 7.20 دالة الخصم حسب فئة الزبون

تجارة إلكترونية

متوسط

اكتب دالة `discountRate(category)` تعيد نسبة الخصم حسب فئة الزبون (`'B'` برونزي 5%، `'S'` فضي 10%، `'G'` ذهبي 15%، غير ذلك 0%) باستعمال `switch`، ودالة `finalAmount(amount, category)` تطبق الخصم ثم تضيف رسم التغليف 150 دج.

خطوات الحل

1) `switch (category)` مع `return` في كل حالة (لا حاجة إلى `break`).
2) `finalAmount = amount * (1 - rate) + 150`.

البرنامج بلغة C

```c
#include <stdio.h>

double discountRate(char category) {
    switch (category) {
        case 'B': return 0.05;
        case 'S': return 0.10;
        case 'G': return 0.15;
        default:  return 0.0;            // زبون عادي
    }
}
```

```c

double finalAmount(double amount, char category) {
    return amount * (1 - discountRate(category)) + 150;
}

int main(void) {
    int nbOrders, order;
    double amount;
    char category;
    scanf("%d", &nbOrders);
    for (order = 0; order < nbOrders; order++) {
        scanf("%lf %c", &amount, &category);
        printf("%8.2f (%c, -%2.0f%%) -> %8.2f DA\n", amount, category,
               discountRate(category) * 100, finalAmount(amount, category));
    }
    return 0;
}
```

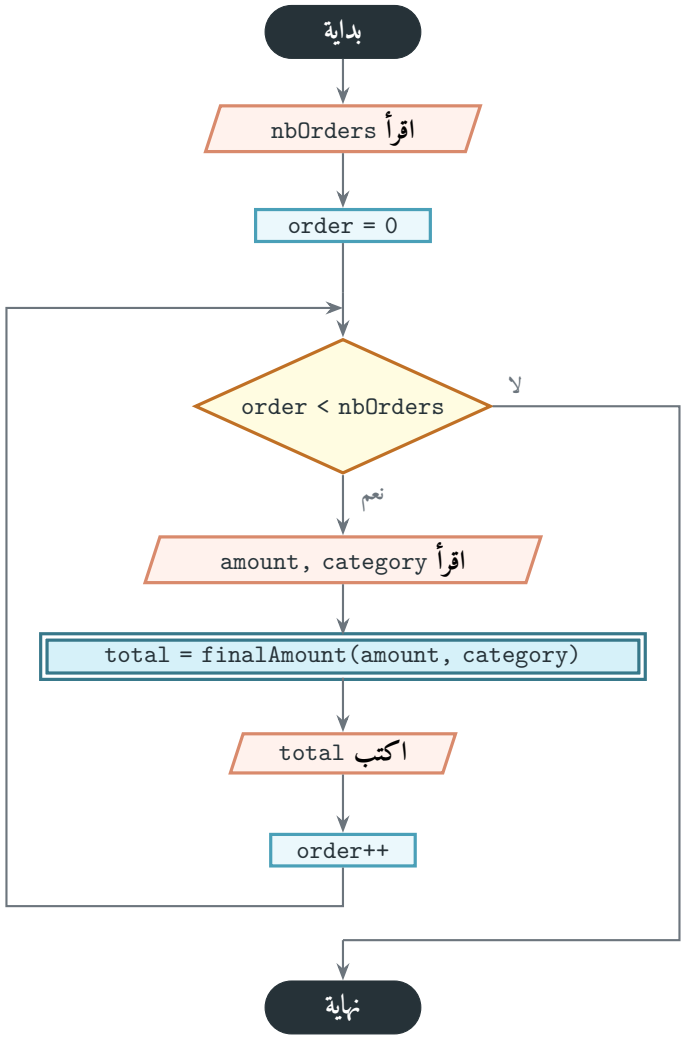
المدخلات

```
4
8000 G
2500 B
12000 S
900 X
```

نتيجة التنفيذ

```
 8000.00 (G, -15%) ->  6950.00 DA
 2500.00 (B, - 5%) ->  2525.00 DA
12000.00 (S, -10%) -> 10950.00 DA
  900.00 (X, - 0%) ->  1050.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.21 دالة التحقق من رقم البطاقة بخوارزمية لون

تجارة إلكترونية ★★ متوسط

قبل إرسال رقم البطاقة إلى البنك تحقق منه محليًا بخوارزمية لون **(Luhn)**. اكتب دالة `luhnValid(number)` تأخذ الرقم كسلسلة وتعيد 1 إذا كان صالحًا.

خطوات الحل

1) نمر على الأرقام من اليمين إلى اليسار، نضاعف رقمًا من كل اثنين (الثاني، الرابع، ...).
2) إذا تجاوز الرقم المضاعف 9 نطرح منه 9.
3) الرقم صالح إذا كان المجموع يقبل القسمة على 10.

البرنامج بلغة C

```c
#include <stdio.h>

int luhnValid(char number[]) {
    int length = 0, pos, digit, sum = 0, doubled = 0;
    while (number[length] != '\0') length++;
```

```c
    for (pos = length - 1; pos >= 0; pos--) {      // من اليمين
        digit = number[pos] - '0';
        if (doubled) {
            digit *= 2;
            if (digit > 9) digit -= 9;
        }
        sum += digit;
        doubled = !doubled;                        // رقم من كل اثنين
    }
    return sum % 10 == 0;
}

int main(void) {
    char card[25];
    int idx;
    for (idx = 0; idx < 2; idx++) {
        scanf("%24s", card);
        printf("%s: %s\n", card, luhnValid(card) ? "valid" : "invalid");
    }
    return 0;
}
```

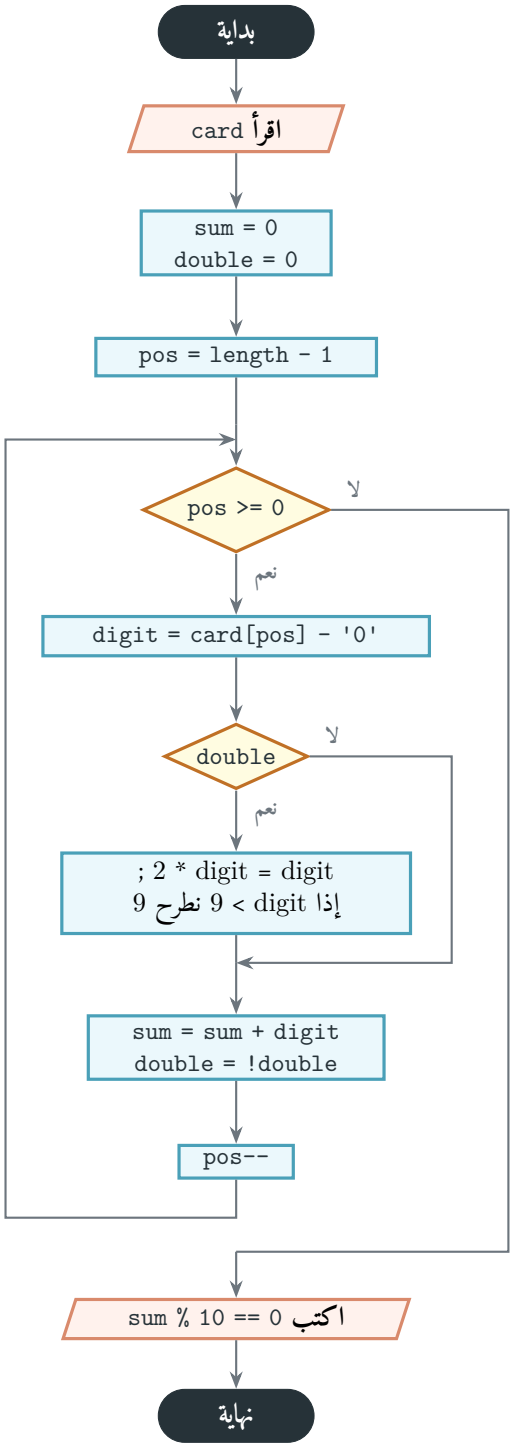
المدخلات

```
4539148803436467 4539148803436468
```

نتيجة التنفيذ

```
4539148803436467: valid
4539148803436468: invalid
```

المخطط الإنسيابي (Organigramme)



تمرين 7.22 دالة التطبيع بين الصفر والواحد

علم البيانات ★★ متوسط

قبل تدريب نموذج، نطبّع أعمدة البيانات إلى المجال `[0, 1]`. اكتب دالة `minMaxNormalize(values, normalized, size)` تملأ جدولًا ثانيًا بالقيم `(value - min) / (max - min)`، واستعملها على أسعار وعلى أعمار الزبائن.

خطوات الحل

1) مرور أول لإيجاد `min` و `max`.
2) إذا كان `max == min` نضع 0 لكل القيم لتفادي القسمة على 0.
3) النتيجة تُكتب في الجدول الثاني (الجدول الأصلي لا يتغير).

</> البرنامج بلغة C

```c
#include <stdio.h>

void minMaxNormalize(double values[], double normalized[], int size) {
    double low = values[0], high = values[0];
    int idx;
    for (idx = 1; idx < size; idx++) {
        if (values[idx] < low) low = values[idx];
        if (values[idx] > high) high = values[idx];
    }
    for (idx = 0; idx < size; idx++)
        normalized[idx] = high == low ? 0 : (values[idx] - low) / (high - low);
}

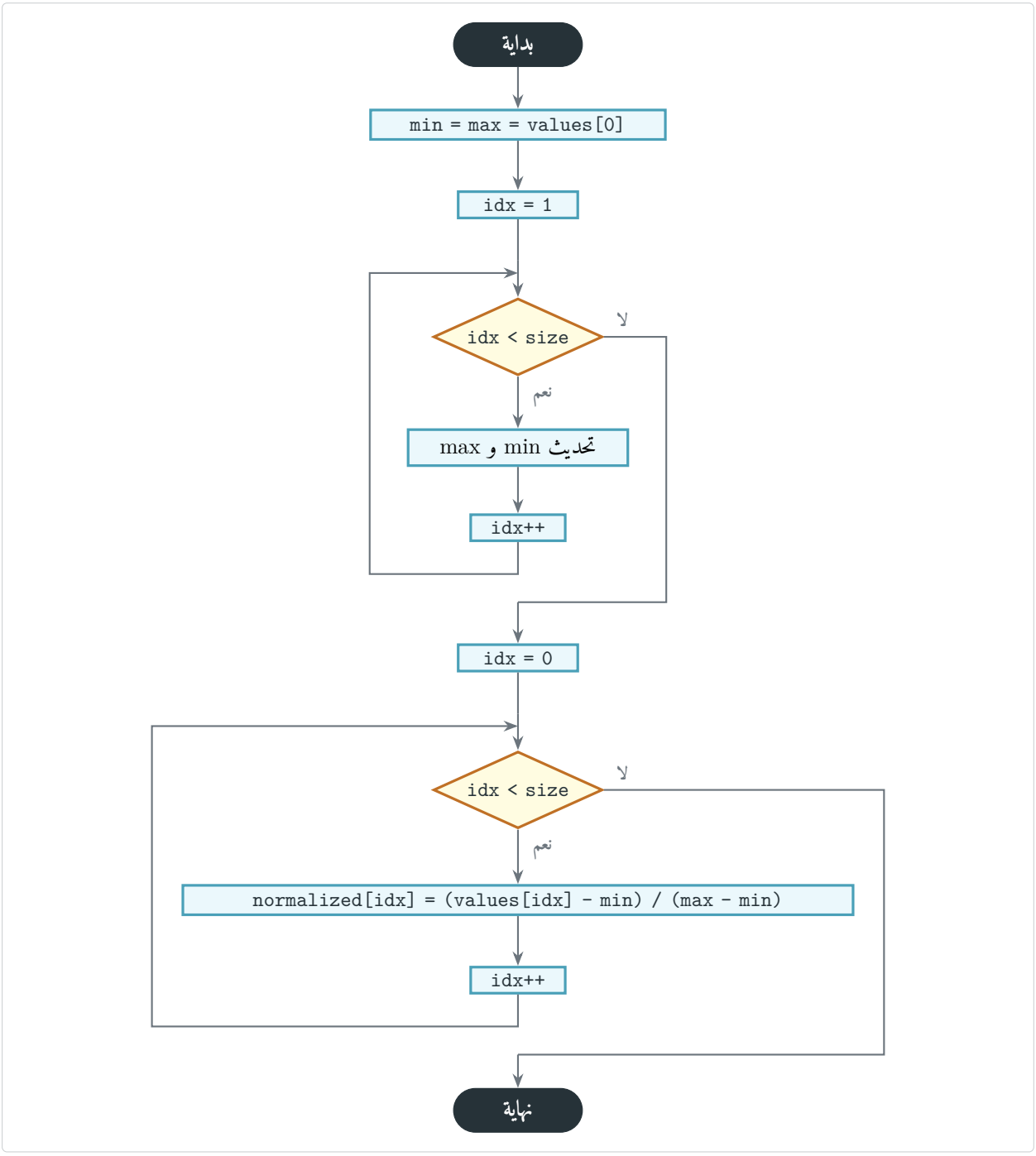
void printRow(char label[], double values[], int size) {
    int idx;
    printf("%s", label);
    for (idx = 0; idx < size; idx++) printf(" %.3f", values[idx]);
    printf("\n");
}

int main(void) {
    double prices[5] = {1200, 450, 3900, 2600, 900}, agesD[5] = {19, 34, 52, 27, 41};
    double result[5];
    minMaxNormalize(prices, result, 5);
    printRow("prices:", result, 5);
    minMaxNormalize(agesD, result, 5);          // نفس الدالة لعمود آخر
    printRow("ages:  ", result, 5);
    return 0;
}
```

>_ نتيجة التنفيذ

```
prices: 0.217 0.000 1.000 0.623 0.130
ages:   0.000 0.455 1.000 0.242 0.667
```

المخطط الإنسيابي (Organigramme)



تمرين 7.23 دالة عدّ المنتجات تحت عتبة

تجارة إلكترونية ★★ متوسط

اكتب دالة `countBelow(stock, size, threshold)` تعيد عدد المنتجات التي مخزونها أقل من العتبة، ودالة `printBelow` تعرض فهارسها. استعملهما لثلاث عتبات (5، 10، 20) لتقرير إعادة التموين.

خطوات الحل

1) دالة تعيد عددًا، وأخرى `void` تعرض.
2) نستدعيهما في حلقة على جدول العتبات.

البرنامج بلغة C

```c
#include <stdio.h>

int countBelow(int stock[], int size, int threshold) {
    int idx, count = 0;
    for (idx = 0; idx < size; idx++)
        if (stock[idx] < threshold) count++;
    return count;
}

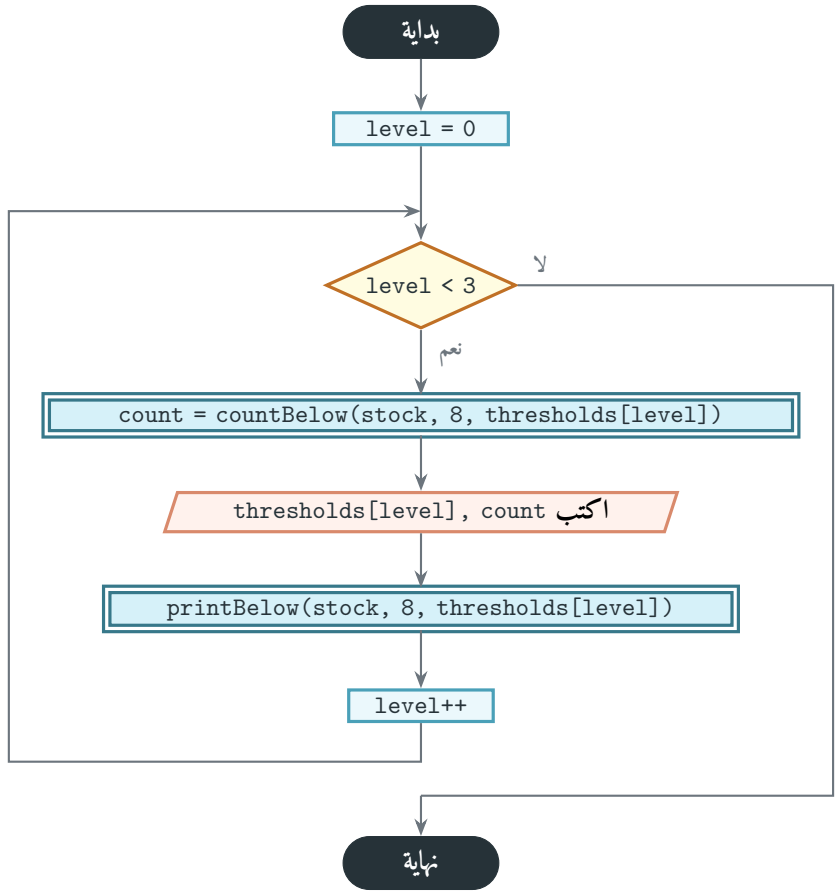
void printBelow(int stock[], int size, int threshold) {
    int idx;
    printf("   items:");
    for (idx = 0; idx < size; idx++)
        if (stock[idx] < threshold) printf(" #%d(%d)", idx, stock[idx]);
    printf("\n");
}

int main(void) {
    int stock[8] = {12, 3, 25, 8, 0, 17, 9, 30};
    int thresholds[3] = {5, 10, 20}, level;
    for (level = 0; level < 3; level++) {
        printf("Below %2d: %d products\n", thresholds[level],
               countBelow(stock, 8, thresholds[level]));
        printBelow(stock, 8, thresholds[level]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Below  5: 2 products
   items: #1(3) #4(0)
Below 10: 4 products
   items: #1(3) #3(8) #4(0) #6(9)
Below 20: 6 products
   items: #0(12) #1(3) #3(8) #4(0) #5(17) #6(9)
```

المخطط الإنسيابي (Organigramme)



تمرين 7.24 دالة الأس الأسي بسلسلة تايلور

علم البيانات ★★ متوسط

النمو المستمر للمستخدمين يتبع `users = start * e^(rate * t)`. اكتب دالة `myExp(value)` تحسب `e^value` دون exp بمجموع سلسلة تايلور `1 + value + value²/2! + value³/3! + ...` حتى يصبح الحد أقل من `1e-12`.

خطوات الحل

1) كل حد يُحسب من الذي قبله: `term = term * value / rank`.
2) نتوقف عندما يصبح الحد (بالقيمة المطلقة) صغيرًا جدًا.

البرنامج بلغة C

```c
#include <stdio.h>

double myExp(double value) {
    double sum = 1, term = 1;
    int rank = 1;
    while (term > 1e-12 || term < -1e-12) {
        term = term * value / rank;        // الحد التالي من السابق
        sum += term;
        rank++;
    }
    return sum;
```

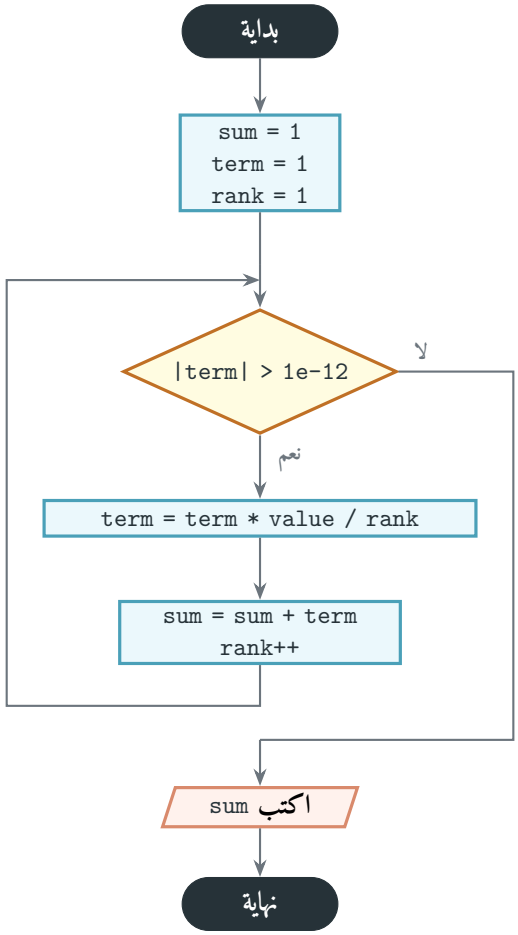
```c
}

int main(void) {
    double start = 1500, rate = 0.08;
    int month;
    printf("e = %.10f\n", myExp(1));
    for (month = 0; month <= 12; month += 3)
        printf("Month %2d: %.0f users\n", month, start * myExp(rate * month));
    return 0;
}
```

نتيجة التنفيذ

```
e = 2.7182818285
Month  0: 1500 users
Month  3: 1907 users
Month  6: 2424 users
Month  9: 3082 users
Month 12: 3918 users
```

المخطط الإنسيابي (Organigramme)



تمرين 7.25 دالة التقريب إلى أقرب مضاعف

تجارة إلكترونية

متوسط

تُعرض الأسعار التسويقية مقربة. اكتب دالة `roundTo(price, step)` تقرب السعر إلى أقرب مضاعف لـ `step`، ودالة

`charmPrice(price)` تعطي السعر "النفسي" (مثلًا 4990 بدل 5000).

خطوات الحل

1) `roundTo`: `(int)((price + step / 2.0) / step) * step`.

2) `charmPrice`: نقرب إلى أقرب 100 ثم نطرح 10.

البرنامج بلغة C

```c
#include <stdio.h>

int roundTo(double price, int step) {
    return (int)((price + step / 2.0) / step) * step;   // نصف الخطوة للتقريب
}

int charmPrice(double price) {
    return roundTo(price, 100) - 10;
}

int main(void) {
    double price;
    int step;
    scanf("%lf %d", &price, &step);
    printf("Rounded to %d: %d DA\n", step, roundTo(price, step));
    printf("Rounded to 1000: %d DA\n", roundTo(price, 1000));
    printf("Charm price: %d DA\n", charmPrice(price));
    return 0;
}
```

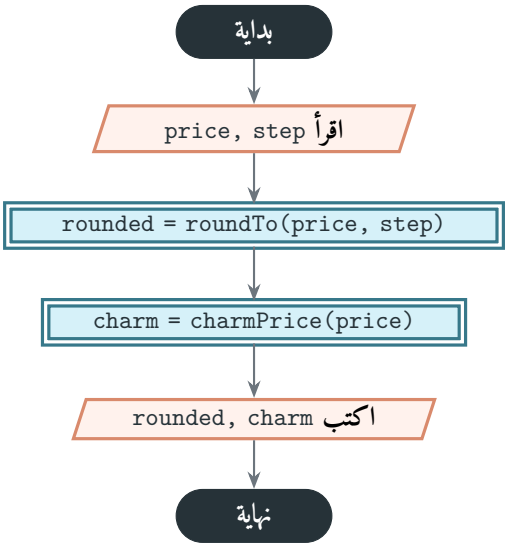
المدخلات

```
4873 50
```

نتيجة التنفيذ

```
Rounded to 50: 4850 DA
Rounded to 1000: 5000 DA
Charm price: 4890 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.26 دالة رقم الفاتورة المتناظر

تجارة إلكترونية
متوسط

اكتب دالة `reverseNumber(number)` تعيد العدد مقلوب الأرقام، ودالة `isPalindrome(number)` تستعملها لتحديد الفواتير ذات الرقم المتناظر (تحصل على هدية).

خطوات الحل

1) قلب الأرقام: `reversed = reversed * 10 + number % 10` ثم `number /= 10`.
2) العدد متناظر إذا ساوى مقلوبه.

البرنامج بلغة C

```c
#include <stdio.h>

long reverseNumber(long number) {
    long reversed = 0;
    while (number > 0) {
        reversed = reversed * 10 + number % 10;   // آخر رقم ينتقل إلى اليسار
        number /= 10;
    }
    return reversed;
}

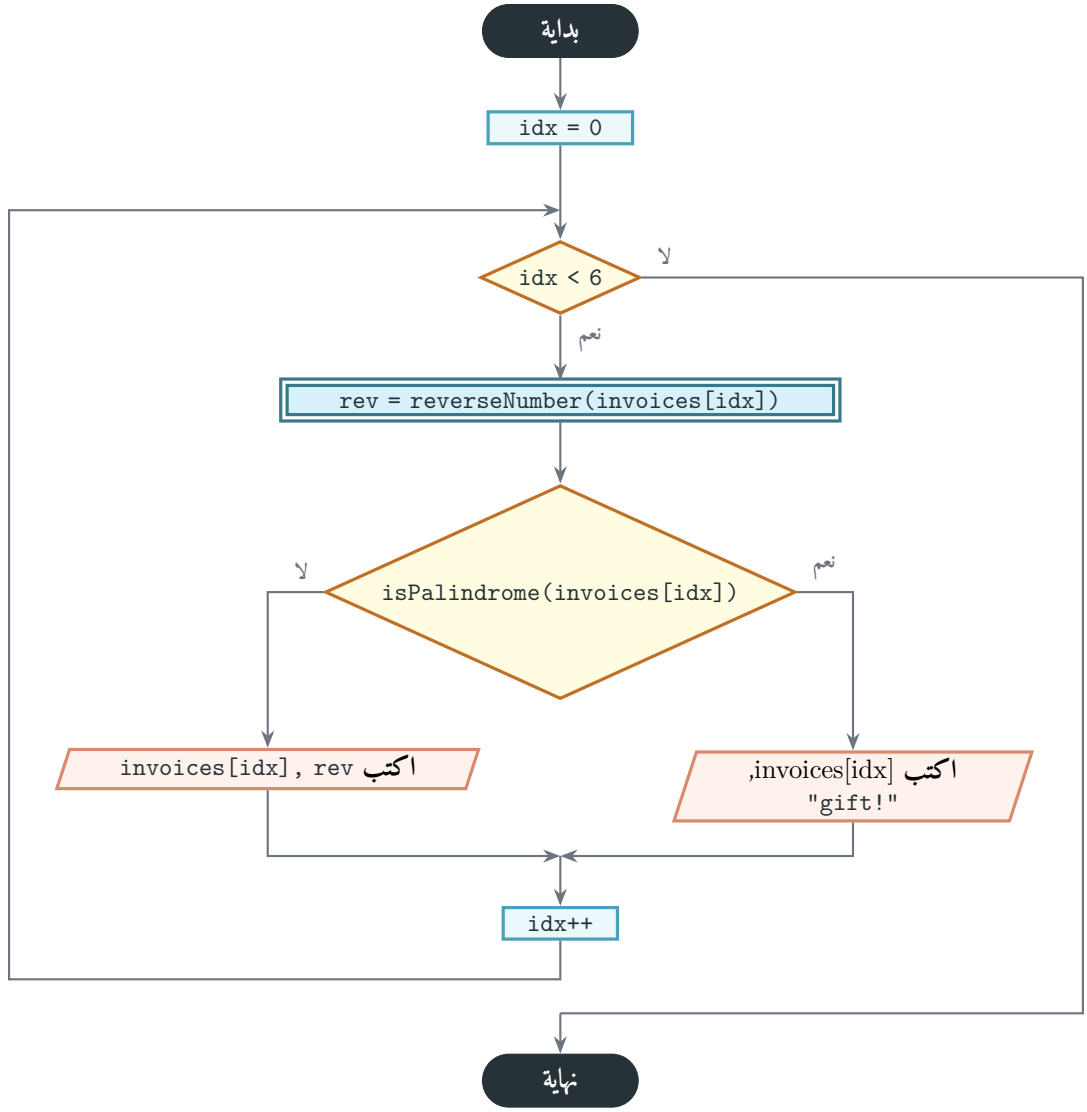
int isPalindrome(long number) {
    return reverseNumber(number) == number;
}

int main(void) {
    long invoices[6] = {12321, 20452, 45654, 100001, 98760, 7};
    int idx;
    for (idx = 0; idx < 6; idx++) {
        if (isPalindrome(invoices[idx]))
            printf("Invoice %6ld: palindrome -> gift!\n", invoices[idx]);
        else
            printf("Invoice %6ld: reversed %ld\n", invoices[idx], reverseNumber(invoices[idx]));
    }
    return 0;
}
```

نتيجة التنفيذ

```
Invoice  12321: palindrome -> gift!
Invoice  20452: reversed 25402
Invoice  45654: palindrome -> gift!
Invoice 100001: palindrome -> gift!
Invoice  98760: reversed 6789
Invoice      7: palindrome -> gift!
```

المخطط الإنسيابي (Organigramme)



تمرين 7.27 دالة رقم التحقق لرمز المنتج

تجارة إلكترونية ★★ متوسط

رموز المنتجات EAN-13 تنتهي برقم تحقق. اكتب دالة `eanCheckDigit(code)` تحسبه من الأرقام الـ 12 الأولى: مجموع أرقام المواضع الفردية + 3 × مجموع أرقام المواضع الزوجية، ثم `(10 - (... % 10) % 10)`. ودالة `eanValid` تتحقق من رمز كامل.

خطوات الحل

1) المواضع تُعد من 1 من اليسار: الموضع الفردي وزنه 1 والزوجي وزنه 3.

2) `eanValid` تقارن الرقم الثالث عشر بنتيجة `eanCheckDigit`.

البرنامج بلغة C

```c
#include <stdio.h>

int eanCheckDigit(char code[]) {
    int pos, sum = 0;
    for (pos = 0; pos < 12; pos++)
        sum += (code[pos] - '0') * (pos % 2 == 0 ? 1 : 3);   // الأوزان 1 و 3
    return (10 - sum % 10) % 10;
}

int eanValid(char code[]) {
    return code[12] - '0' == eanCheckDigit(code);
}

int main(void) {
    char code[20];
    int idx;
    for (idx = 0; idx < 2; idx++) {
        scanf("%19s", code);
        if (eanValid(code))
            printf("%s: valid\n", code);
        else
            printf("%s: invalid (check digit should be %d)\n", code, eanCheckDigit(code));
    }
    return 0;
}
```

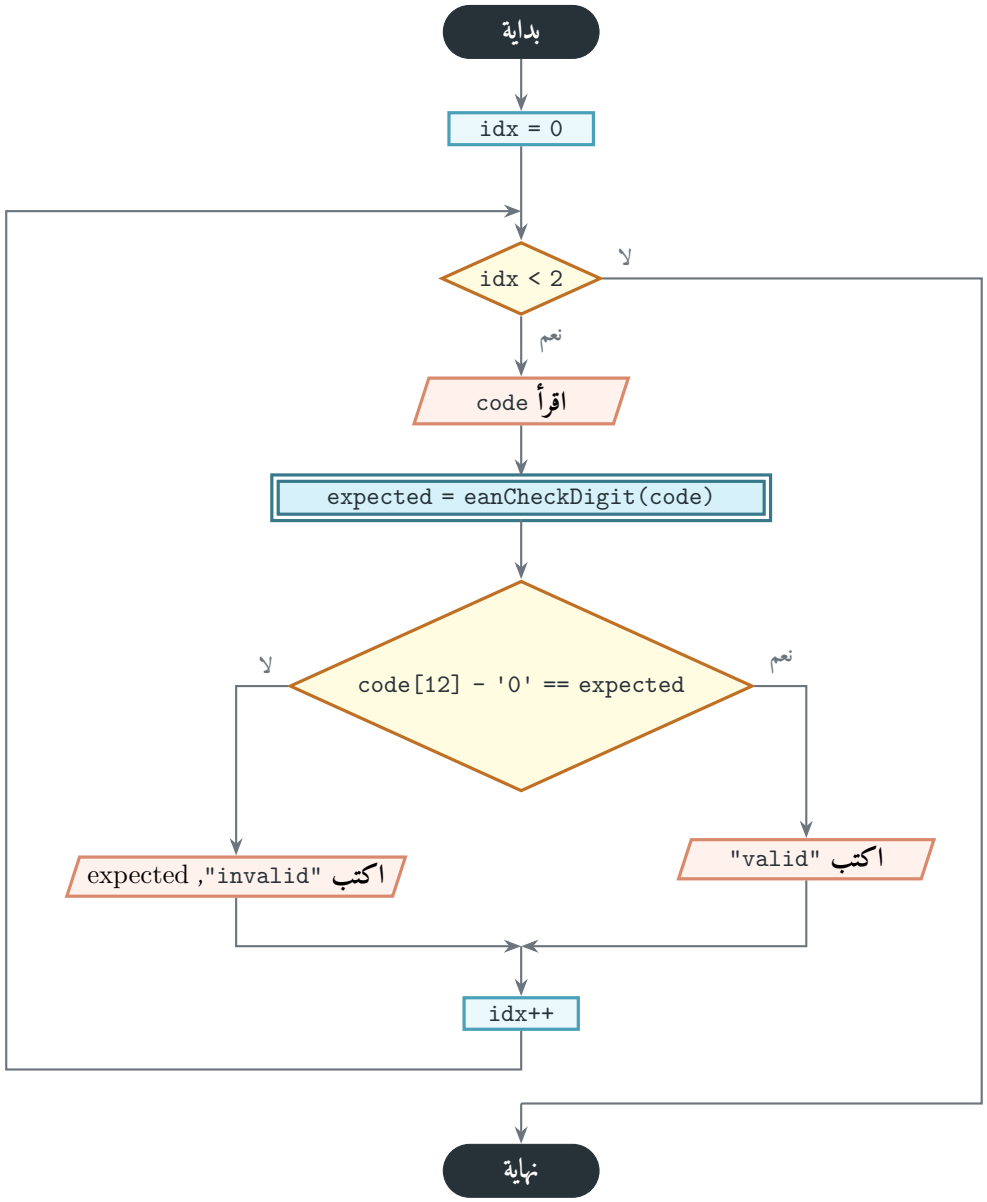
المدخلات

```
6131234567893 6131234567892
```

نتيجة التنفيذ

```
6131234567893: valid
6131234567892: invalid (check digit should be 3)
```

المخطط الإنسيابي (Organigramme)



تمرين 7.28 دالة عرض العدد بالنظام الثنائي

علم البيانات ★★ متوسط

خصائص كل منتج مخزنة كأعلام في عدد واحد (البت 0: متوفر، 1: جديد، 2: تخفيض، 3: توصيل مجاني). اكتب دالة `printBinary(value, bits)` تعرض العدد بالثنائي، ودالة `hasFlag(value, bit)` تختبر بتًا معينًا.

خطوات الحل

1) البت رقم `bit` يُستخرج بـ `(value >> bit) & 1`.
2) نعرض من البت الأعلى إلى الأدنى.

البرنامج بلغة C

```c
#include <stdio.h>

void printBinary(int value, int bits) {
    int bit;
    for (bit = bits - 1; bit >= 0; bit--)
        putchar(((value >> bit) & 1) ? '1' : '0');    // البت الأعلى أولًا
}

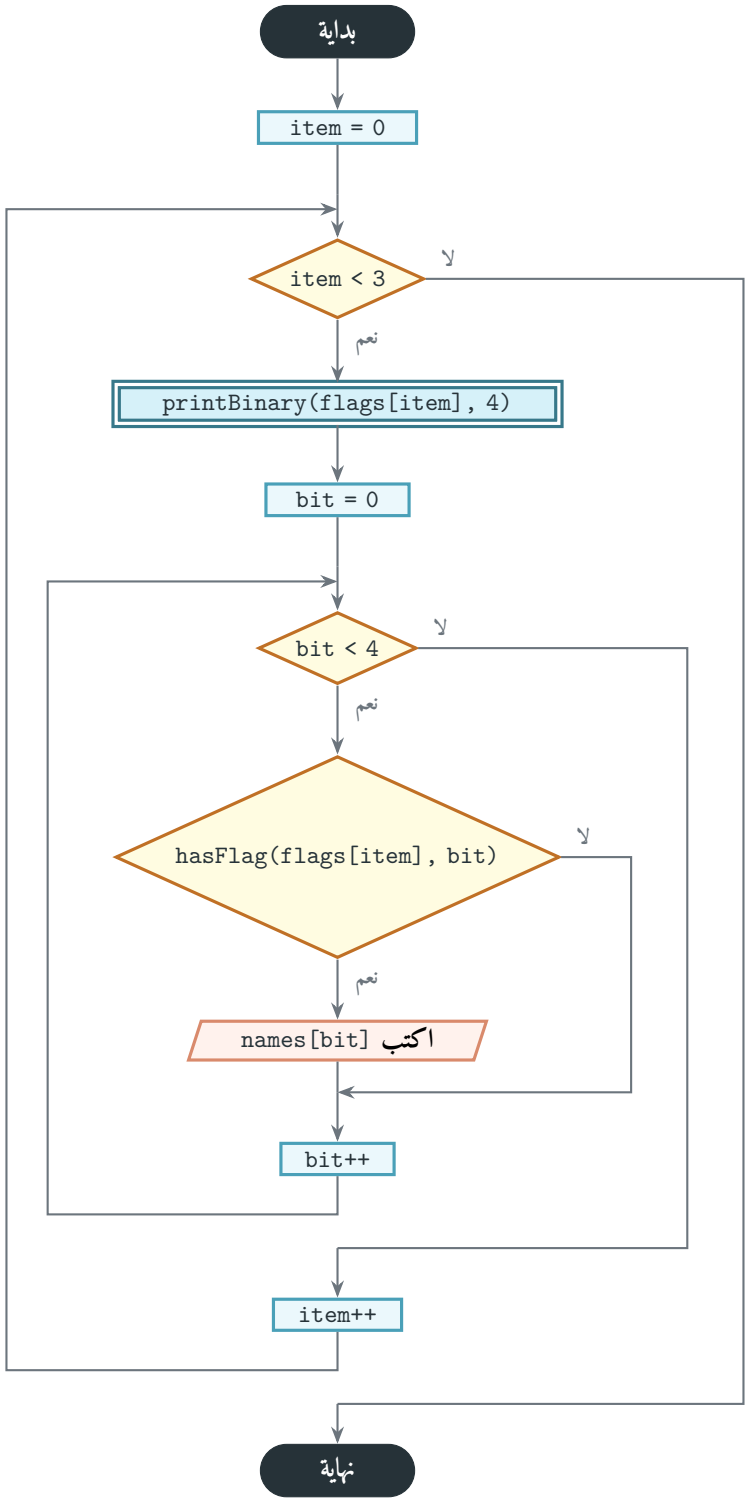
int hasFlag(int value, int bit) {
    return (value >> bit) & 1;
}

int main(void) {
    int flags[3] = {5, 11, 6};
    char names[4][16] = {"in-stock", "new", "on-sale", "free-ship"};
    int item, bit;
    for (item = 0; item < 3; item++) {
        printf("Product %d: %2d = ", item + 1, flags[item]);
        printBinary(flags[item], 4);
        printf(" ->");
        for (bit = 0; bit < 4; bit++)
            if (hasFlag(flags[item], bit))
                printf(" %s", names[bit]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Product 1:  5 = 0101 -> in-stock on-sale
Product 2: 11 = 1011 -> in-stock new free-ship
Product 3:  6 = 0110 -> new on-sale
```

المخطط الإنسيابي (Organigramme)



تمرين 7.29 دالة الوسيط لأسعار السوق

علم البيانات ★★ متوسط

المتوسط يتأثر بالقيم الشاذة، لذلك نستعمل **الوسيط**. اكتب دالة `median(values, size)` تنسخ الجدول إلى جدول محلي، ترتبه، ثم تعيد العنصر الأوسط (أو متوسط العنصرين الأوسطين إذا كان العدد زوجيًا)، دون تغيير الجدول الأصلي.

خطوات الحل

1) نسخ إلى `copy[]` المحلي لأن الترتيب يغير الجدول الممرر.
2) ترتيب بالإدراج، ثم: فردي `copy[size/2]`، زوجي `(copy[size/2-1] + copy[size/2]) / 2`.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 50

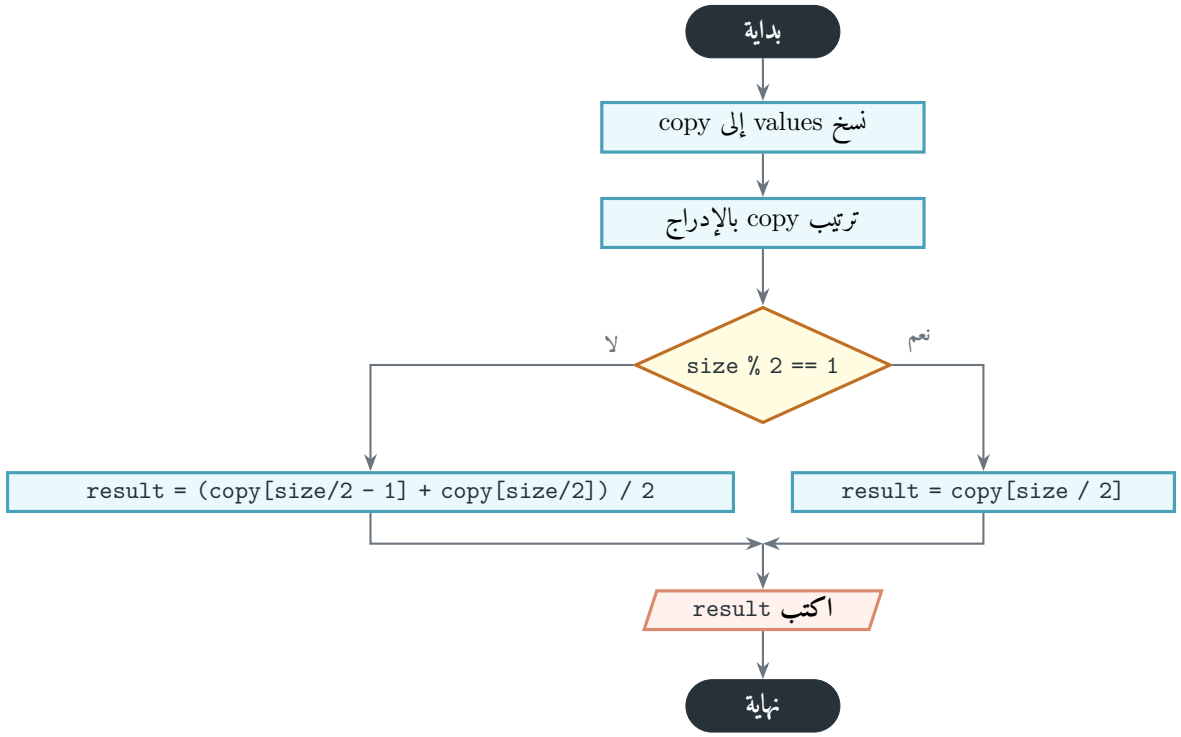
double median(double values[], int size) {
    double copy[MAX], key;
    int idx, back;
    for (idx = 0; idx < size; idx++) copy[idx] = values[idx];   // لا نغير الأصل
    for (idx = 1; idx < size; idx++) {                          // ترتيب بالإدراج
        key = copy[idx];
        back = idx - 1;
        while (back >= 0 && copy[back] > key) {
            copy[back + 1] = copy[back];
            back--;
        }
        copy[back + 1] = key;
    }
    if (size % 2 == 1) return copy[size / 2];
    return (copy[size / 2 - 1] + copy[size / 2]) / 2;
}

int main(void) {
    double phoneA[7] = {32000, 29500, 31000, 89000, 30500, 28900, 33000};
    double phoneB[6] = {15000, 14500, 16200, 15800, 14900, 15300};
    printf("Phone A median: %.0f DA (first value still %.0f)\n", median(phoneA, 7), phoneA[0]);
    printf("Phone B median: %.0f DA\n", median(phoneB, 6));
    return 0;
}
```

نتيجة التنفيذ

```
Phone A median: 31000 DA (first value still 32000)
Phone B median: 15150 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.30 دالة المنوال للمقاس الأكثر طلبًا

علم البيانات ★★ متوسط

اكتب دالة `mode(sizes, count)` تعيد **المقاس الأكثر تكرارًا** في طلبيات الأحذية (المقاسات بين 35 و 46)، ودالة `printHistogram` تعرض تكرار كل مقاس بالنجوم.

خطوات الحل

1) جدول عدادات `freq[12]` حيث فهرس المقاس `size - 35`.
2) المنوال = المقاس ذو أكبر عداد.

البرنامج بلغة C

```c
#include <stdio.h>

void countSizes(int sizes[], int count, int freq[]) {
    int idx;
    for (idx = 0; idx < 12; idx++) freq[idx] = 0;
    for (idx = 0; idx < count; idx++) freq[sizes[idx] - 35]++;   // مقاس -> فهرس
}

int mode(int sizes[], int count) {
    int freq[12], idx, best = 0;
    countSizes(sizes, count, freq);
    for (idx = 1; idx < 12; idx++)
        if (freq[idx] > freq[best]) best = idx;
    return best + 35;
}

void printHistogram(int sizes[], int count) {
    int freq[12], idx, star;
    countSizes(sizes, count, freq);
    for (idx = 0; idx < 12; idx++)
        if (freq[idx] > 0) {
```

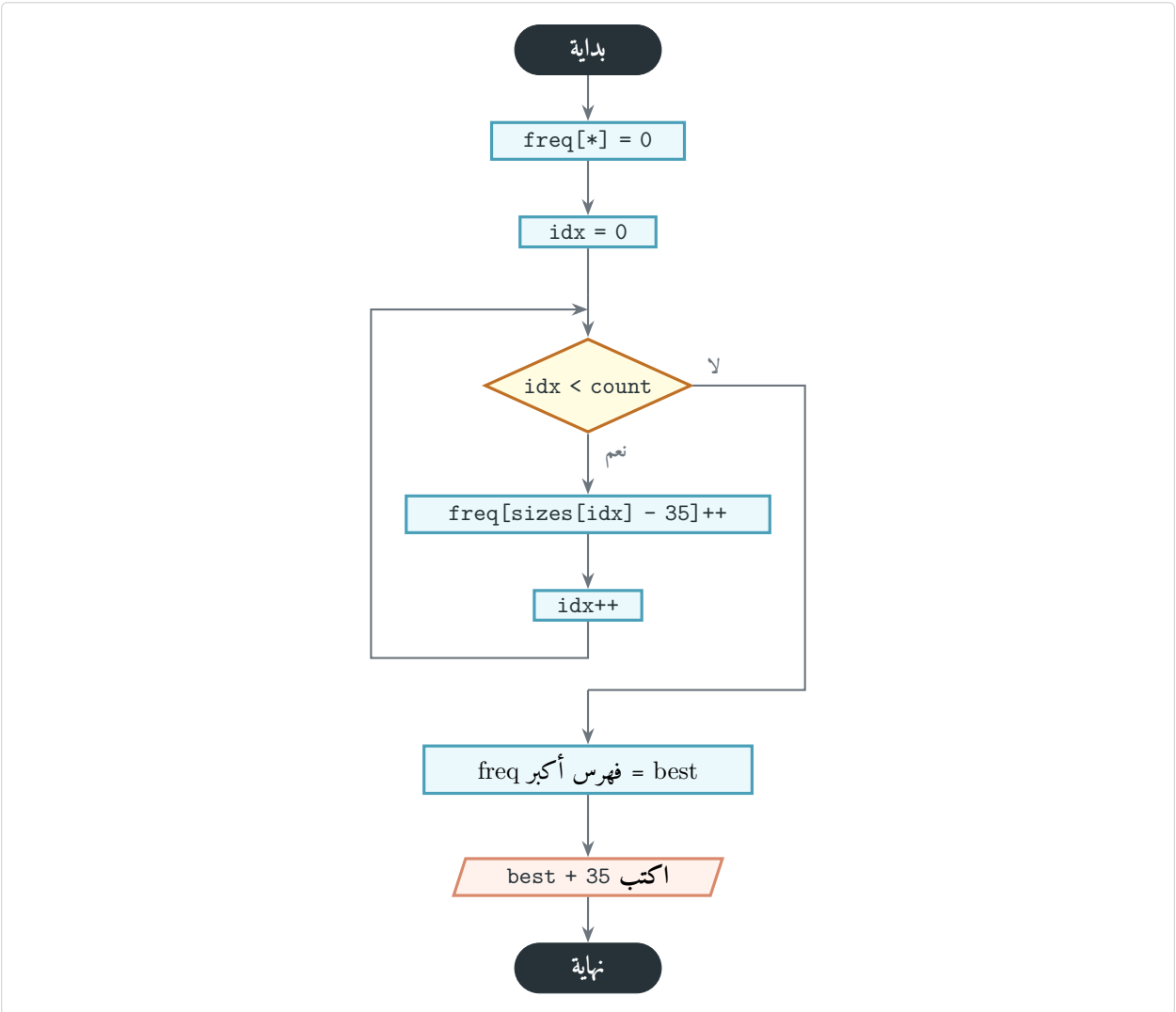
```c
            printf("%d | ", idx + 35);
            for (star = 0; star < freq[idx]; star++) putchar('*');
            printf(" %d\n", freq[idx]);
        }
}

int main(void) {
    int sizes[18] = {42, 41, 43, 42, 40, 44, 42, 39, 41, 42, 43, 45, 41, 42, 38, 43, 42, 41};
    printHistogram(sizes, 18);
    printf("Most ordered size: %d\n", mode(sizes, 18));
    return 0;
}
```

نتيجة التنفيذ

```
38 | * 1
39 | * 1
40 | * 1
41 | **** 4
42 | ****** 6
43 | *** 3
44 | * 1
45 | * 1
Most ordered size: 42
```

المخطط الإنسيابي (Organigramme)



تمرين 7.31 دالة معامل الارتباط بين الإعلان والمبيعات

علم البيانات ★★ متوسط

اكتب دالة `correlation(first, second, size)` تحسب **معامل ارتباط بيرسون** بين ميزانية الإعلان والمبيعات خلال 8 أسابيع، مستعملة دالتي `mean` و `mySqrt` (نيوتن).

خطوات الحل

1) `cov = Σ (first - meanF)(second - meanS)`، `varF = Σ (first - meanF)²`، `varS = Σ (second - meanS)²`.

2) `r = cov / sqrt(varF * varS)`؛ القيمة قرب 1 تعني ارتباطًا موجبًا قويًا.

البرنامج بلغة C

```c
#include <stdio.h>

double mean(double values[], int size) {
    double total = 0;
    int idx;
    for (idx = 0; idx < size; idx++) total += values[idx];
    return total / size;
}

double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    if (value <= 0) return 0;
    for (iter = 0; iter < 60; iter++)
        guess = (guess + value / guess) / 2;      // نيوتن
    return guess;
}

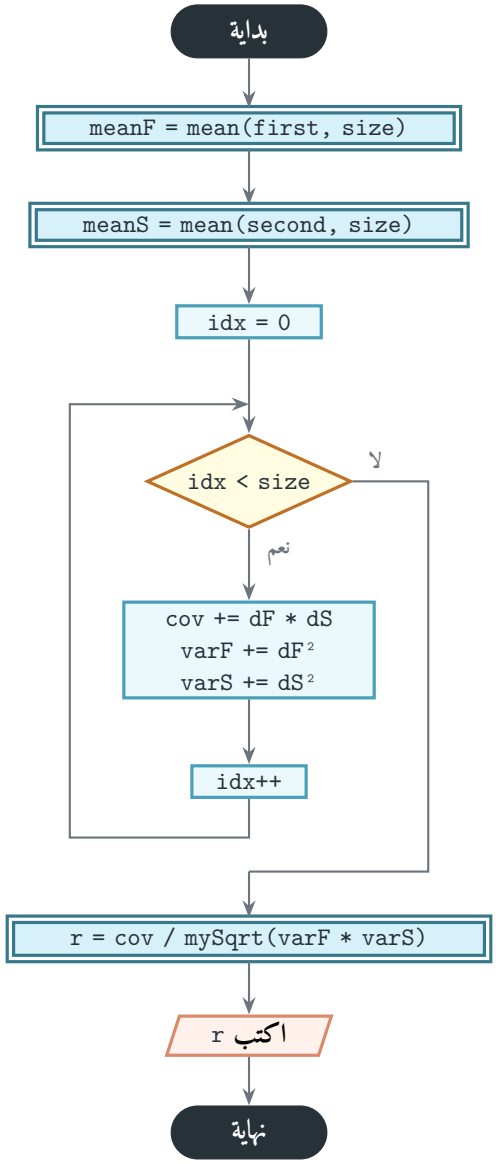
double correlation(double first[], double second[], int size) {
    double meanF = mean(first, size), meanS = mean(second, size);
    double cov = 0, varF = 0, varS = 0, devF, devS;
    int idx;
    for (idx = 0; idx < size; idx++) {
        devF = first[idx] - meanF;
        devS = second[idx] - meanS;
        cov += devF * devS;
        varF += devF * devF;
        varS += devS * devS;
    }
    return cov / mySqrt(varF * varS);
}

int main(void) {
    double ads[8] = {10, 15, 8, 20, 25, 12, 30, 18};
    double sales[8] = {210, 260, 190, 330, 370, 240, 420, 300};
    double returns[8] = {12, 9, 14, 10, 7, 15, 8, 11};
    printf("r(ads, sales)   = %.4f\n", correlation(ads, sales, 8));
    printf("r(ads, returns) = %.4f\n", correlation(ads, returns, 8));
    return 0;
}
```

نتيجة التنفيذ

```
r(ads, sales)   = 0.9974
r(ads, returns) = -0.8273
```

المخطط الإنسيابي (Organigramme)



تمرين 7.32 دوال الفاتورة المركبة

تجارة إلكترونية متوسط

قسّم حساب الفاتورة إلى دوال صغيرة: `subtotal` (مجموع الأسعار × الكميات)، `discount` (10% فوق 20000 دج)، `tax` (19% بعد الخصم)، `shipping` (500 دج أو مجاني فوق 30000 دج)، ثم `printInvoice` التي تستدعيها كلها.

خطوات الحل

1) كل دالة مسؤولة عن قاعدة واحدة: أسهل للفهم والتعديل.
2) `printInvoice` تستدعي الدوال بالترتيب وتعرض كل سطر.

البرنامج بلغة C

```c
#include <stdio.h>

double subtotal(double prices[], int qty[], int size) {
    double total = 0;
    int idx;
    for (idx = 0; idx < size; idx++) total += prices[idx] * qty[idx];
    return total;
}

double discount(double amount) { return amount > 20000 ? amount * 0.10 : 0; }
double tax(double amount)      { return amount * 0.19; }
double shipping(double amount) { return amount > 30000 ? 0 : 500; }

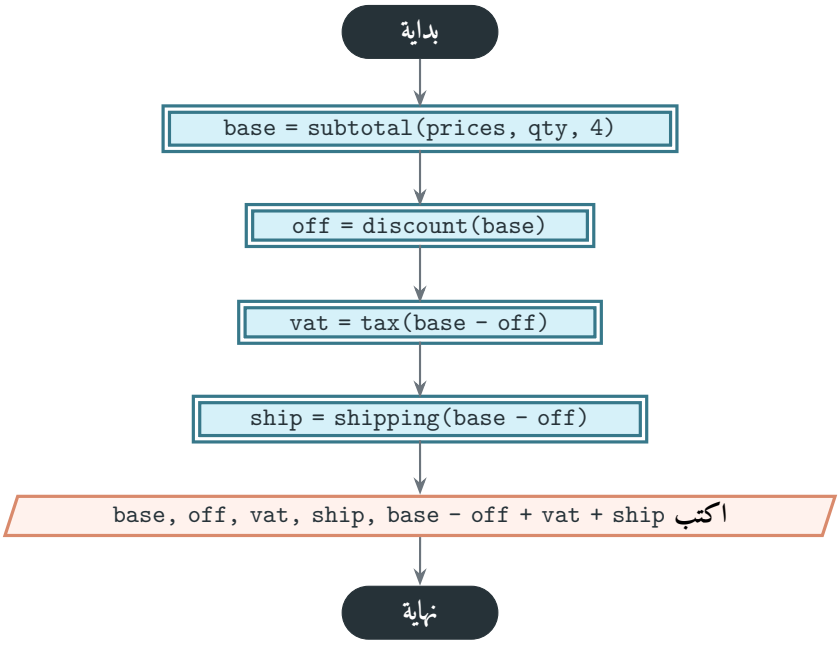
void printInvoice(double prices[], int qty[], int size) {
    double base = subtotal(prices, qty, size);
    double off = discount(base);
    double net = base - off;                        // المبلغ بعد الخصم
    printf("Subtotal : %10.2f\n", base);
    printf("Discount : %10.2f\n", -off);
    printf("VAT 19%%  : %10.2f\n", tax(net));
    printf("Shipping : %10.2f\n", shipping(net));
    printf("TOTAL    : %10.2f DA\n", net + tax(net) + shipping(net));
}

int main(void) {
    double prices[4] = {4500, 1200, 899, 6000};
    int qty[4] = {2, 3, 4, 1};
    printInvoice(prices, qty, 4);
    return 0;
}
```

نتيجة التنفيذ

```
Subtotal :   22196.00
Discount :   -2219.60
VAT 19%  :    3795.52
Shipping :     500.00
TOTAL    :   24271.92 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.33 دالة القسط الشهري للشراء بالتقسيط

تجارة إلكترونية ★★ متوسط

يتيح المتجر الشراء بالتقسيط. اكتب دالة `monthlyPayment(amount, annualRate, months)` بالصيغة `amount * r / (1 - (1 + r)^-months)` حيث `r = annualRate / 12 / 100`، مع دالة `power` مكتوبة يدويًا. اعرض القسط والتكلفة الكلية لثلاث مدد.

خطوات الحل

1) `(1 + r)^-months = 1 / power(1 + r, months)`.
2) إذا كانت النسبة 0: القسط = `amount / months`.
3) التكلفة الإضافية = `payment * months - amount`.

البرنامج بلغة C

```c
#include <stdio.h>

double power(double base, int exponent) {
    double result = 1;
    while (exponent-- > 0) result *= base;
    return result;
}

double monthlyPayment(double amount, double annualRate, int months) {
    double monthly = annualRate / 12 / 100;
    if (monthly == 0) return amount / months;
    return amount * monthly / (1 - 1 / power(1 + monthly, months));
}

int main(void) {
    double amount, rate, pay;
    int durations[3] = {6, 12, 24}, idx;
```

```c
    scanf("%lf %lf", &amount, &rate);
    for (idx = 0; idx < 3; idx++) {
        pay = monthlyPayment(amount, rate, durations[idx]);
        printf("%2d months: %9.2f DA/month, extra cost %8.2f DA\n",
               durations[idx], pay, pay * durations[idx] - amount);
    }
    return 0;
}
```

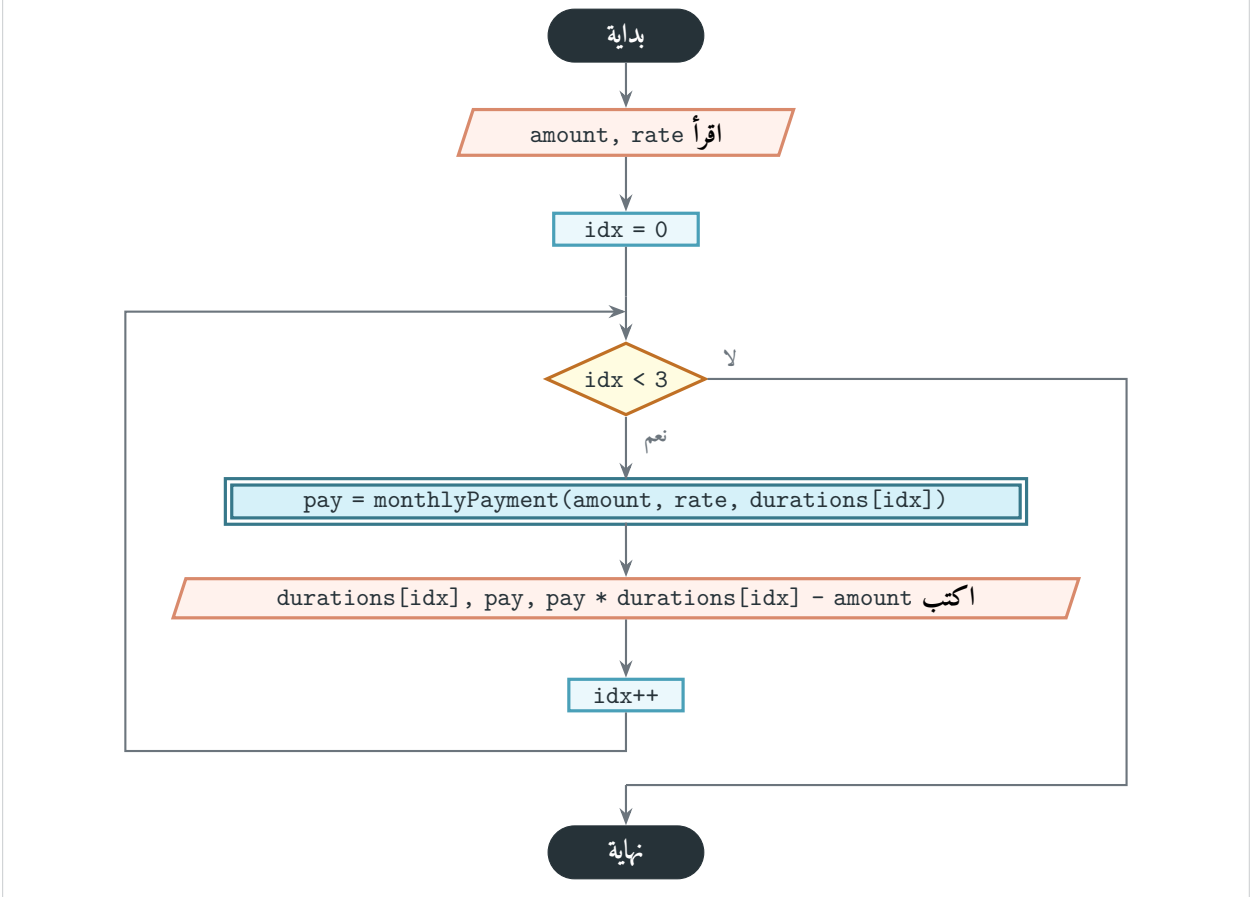
المدخلات

```
120000 9
```

نتيجة التنفيذ

```
 6 months:  20528.27 DA/month, extra cost  3169.61 DA
12 months:  10494.18 DA/month, extra cost  5930.13 DA
24 months:   5482.17 DA/month, extra cost 11572.06 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 7.34 دالة رسم مخطط الأعمدة النصي

علم البيانات ★★ متوسط

اكتب دالة `printBar(label, value, maxValue, width)` ترسم عمودًا أفقيًا متناسبًا مع القيمة، ودالة `maxOf(values, size)`، ثم ارسم مبيعات كل فئة منتجات بعرض أقصى 30 حرفًا.

خطوات الحل

1) طول العمود = `value * width / maxValue` (تناسب).
2) `maxOf` تعطي القيمة المرجعية للعمود الأطول.

البرنامج بلغة C

```c
#include <stdio.h>

int maxOf(int values[], int size) {
    int idx, best = values[0];
    for (idx = 1; idx < size; idx++)
        if (values[idx] > best) best = values[idx];
    return best;
}

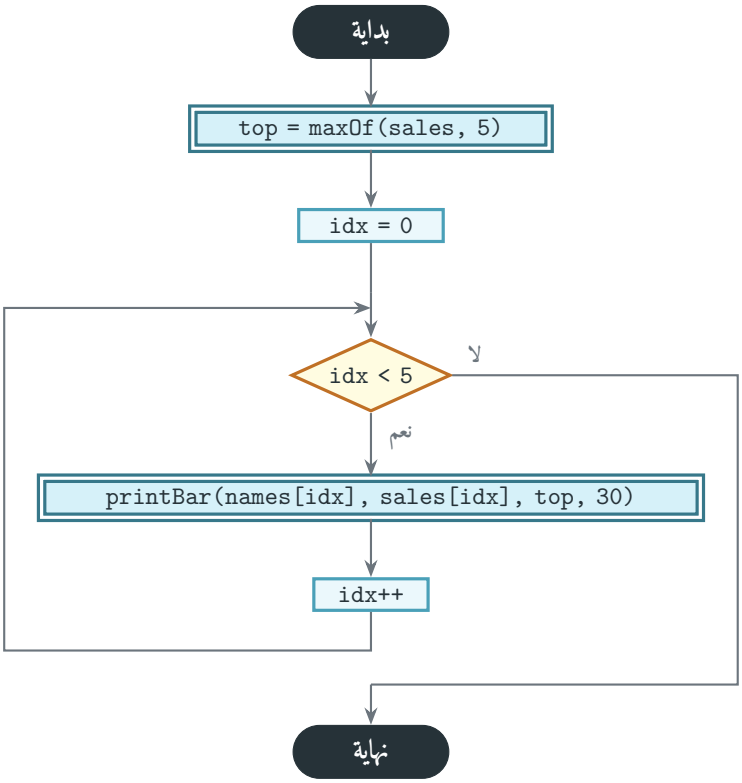
void printBar(char label[], int value, int maxValue, int width) {
    int len = value * width / maxValue, col;          // طول متناسب
    printf("%-11s|", label);
    for (col = 0; col < len; col++) putchar('#');
    printf(" %d\n", value);
}

int main(void) {
    char names[5][12] = {"Phones", "Laptops", "Audio", "Accessories", "Gaming"};
    int sales[5] = {420, 185, 260, 510, 140};
    int top = maxOf(sales, 5), idx;
    for (idx = 0; idx < 5; idx++)
        printBar(names[idx], sales[idx], top, 30);
    return 0;
}
```

نتيجة التنفيذ

```
Phones     |######################## 420
Laptops    |########## 185
Audio      |############### 260
Accessories|############################## 510
Gaming     |######## 140
```

المخطط الإنسيابي (Organigramme)



تمرين 7.35 دالة البحث الثنائي في الكتالوج

تجارة إلكترونية ★★ متوسط

الكتالوج مرتب حسب رمز المنتج. اكتب دالة `binarySearch(codes, size, target, steps)` تعيد فهرس الرمز أو `-1`، وتكتب في `steps[0]` عدد المقارنات المنجزة، ثم قارنها بالبحث التسلسلي.

خطوات الحل

1) `low = 0`، `high = size - 1`؛ `middle = (low + high) / 2`.
2) إذا كان الهدف أكبر من `codes[middle]`: `low = middle + 1`، وإذا كان أصغر: `high = middle - 1`.
3) نمرر جدولًا من خانة واحدة `steps[]` لنستطيع إعادة قيمة ثانية.

البرنامج بلغة C

```c
#include <stdio.h>

int binarySearch(int codes[], int size, int target, int steps[]) {
    int low = 0, high = size - 1, middle;
    steps[0] = 0;
    while (low <= high) {
        middle = (low + high) / 2;
        steps[0]++;                          // مقارنة إضافية
        if (codes[middle] == target) return middle;
        if (codes[middle] < target) low = middle + 1;   // النصف الأيمن
        else high = middle - 1;                         // النصف الأيسر
    }
    return -1;
}
```

```c

int main(void) {
    int codes[16], steps[1], idx, query, target, found;
    for (idx = 0; idx < 16; idx++) codes[idx] = 1015 + idx * 447;   // كتالوج مرتب
    for (query = 0; query < 3; query++) {
        scanf("%d", &target);
        found = binarySearch(codes, 16, target, steps);
        printf("Code %d: index %d after %d comparisons (linear: %d)\n",
               target, found, steps[0], found >= 0 ? found + 1 : 16);
    }
    return 0;
}
```

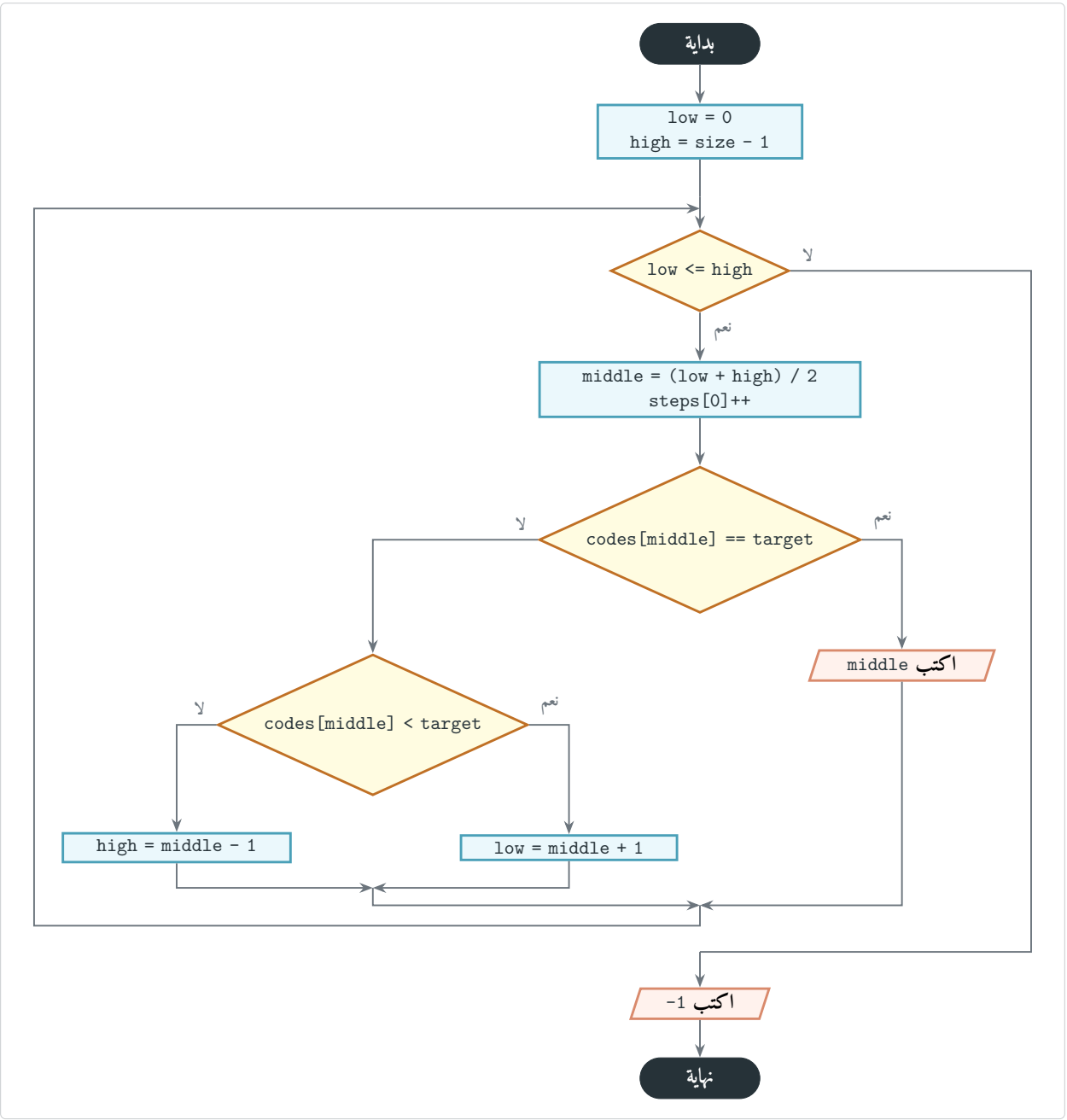
المدخلات

```
7720 1015 9999
```

نتيجة التنفيذ

```
Code 7720: index 15 after 5 comparisons (linear: 16)
Code 1015: index 0 after 4 comparisons (linear: 1)
Code 9999: index -1 after 5 comparisons (linear: 16)
```

المخطط الإنسيابي (Organigramme)



تمرين 7.36 دالة تحويل درجة الحرارة لمخزن التبريد

علم البيانات ★★ متوسط

حساسات المخزن المبرد ترسل القراءات بالفهرنهايت. اكتب دالة `toCelsius(fahrenheit)` ودالة `status(celsius)` تعيد رمزًا: `'L'` أقل من 2°، `'O'` بين 2° و 8°، `'H'` أكثر من 8°، ثم اعرض تقريرًا وعدد الإنذارات.

خطوات الحل

1) `celsius = (fahrenheit - 32) * 5 / 9`.

2) `status` تعيد `char`، و `main` تعد القراءات خارج المجال.

البرنامج بلغة C

```c
#include <stdio.h>

double toCelsius(double fahrenheit) {
    return (fahrenheit - 32) * 5 / 9;
}

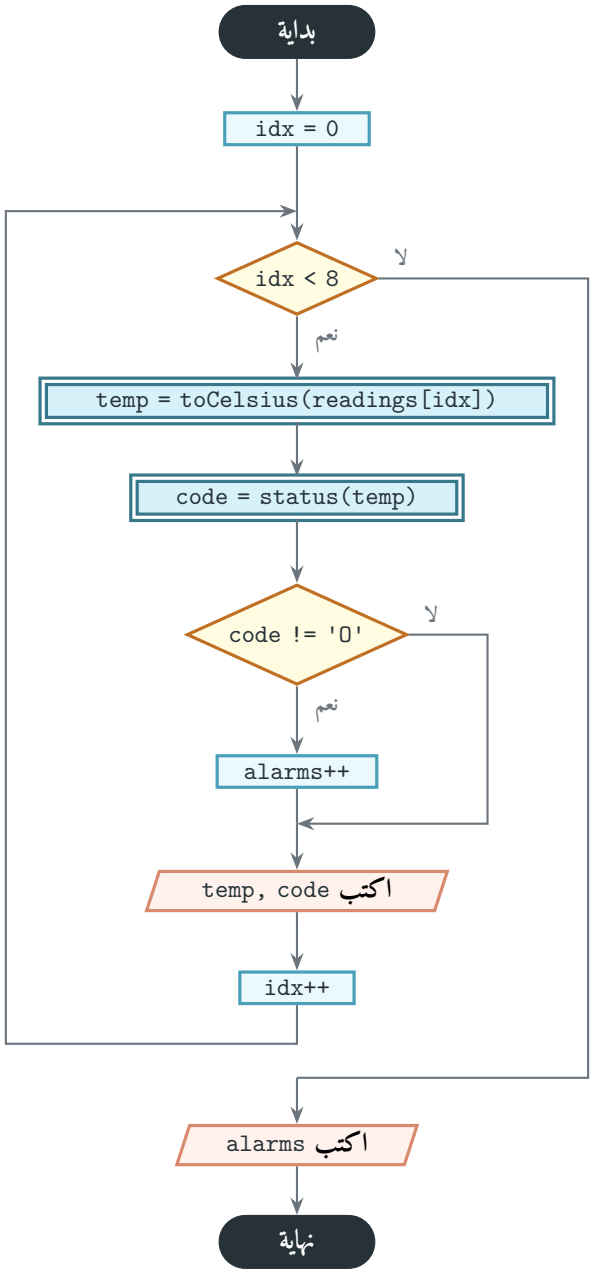
char status(double celsius) {
    if (celsius < 2) return 'L';          // بارد جدًا
    if (celsius <= 8) return 'O';         // مقبول
    return 'H';                           // ساخن
}

int main(void) {
    double readings[8] = {38.5, 41.0, 44.6, 47.3, 35.1, 39.2, 42.8, 50.0};
    double temp;
    int idx, alarms = 0;
    char code;
    for (idx = 0; idx < 8; idx++) {
        temp = toCelsius(readings[idx]);
        code = status(temp);
        if (code != 'O') alarms++;
        printf("%5.1f F = %5.2f C [%c]\n", readings[idx], temp, code);
    }
    printf("Alarms: %d\n", alarms);
    return 0;
}
```

نتيجة التنفيذ

```
 38.5 F =  3.61 C [O]
 41.0 F =  5.00 C [O]
 44.6 F =  7.00 C [O]
 47.3 F =  8.50 C [H]
 35.1 F =  1.72 C [L]
 39.2 F =  4.00 C [O]
 42.8 F =  6.00 C [O]
 50.0 F = 10.00 C [H]
Alarms: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 7.37 دالة تقييم المنتج بالنجوم

تجارة إلكترونية ★★ متوسط

اكتب دالة `printStars(rating)` تعرض تقييمًا عشريًا من 5 على شكل نجوم: `*` لكل نجمة كاملة، `+` لنصف نجمة (إذا كان الجزء العشري ≥ 0.5)، و `.` للباقي. ثم دالة `weightedRating` تحسب التقييم من عدد الأصوات لكل نجمة.

خطوات الحل

1) `weightedRating = (1*v1 + 2*v2 + ... + 5*v5) / (v1 + ... + v5)`

.

2) النجوم الكاملة `(int) rating`، ونصف نجمة إذا `rating - full >= 0.5`.

البرنامج بلغة C

```c
#include <stdio.h>

double weightedRating(int votes[]) {
    int star, total = 0, weighted = 0;
    for (star = 0; star < 5; star++) {
        weighted += (star + 1) * votes[star];   // وزن كل نجمة
        total += votes[star];
    }
    return total ? (double)weighted / total : 0;
}

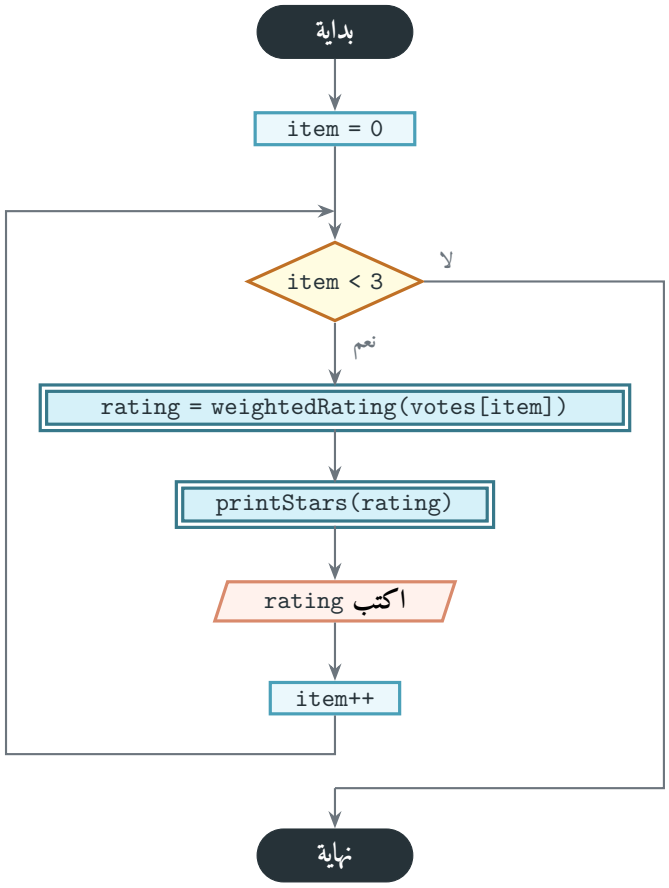
void printStars(double rating) {
    int full = (int)rating, pos;
    int half = rating - full >= 0.5;
    for (pos = 0; pos < 5; pos++) {
        if (pos < full) putchar('*');
        else if (pos == full && half) putchar('+');
        else putchar('.');
    }
}

int main(void) {
    int votes[3][5] = {{2, 3, 10, 40, 45}, {20, 15, 10, 5, 2}, {1, 4, 30, 25, 5}};
    char names[3][12] = {"Earbuds", "Cable", "Mouse"};
    double rating;
    int item;
    for (item = 0; item < 3; item++) {
        rating = weightedRating(votes[item]);
        printf("%-8s ", names[item]);
        printStars(rating);
        printf(" %.2f\n", rating);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Earbuds  ****. 4.23
Cable    **... 2.12
Mouse    ***.. 3.45
```

المخطط الإنسيابي (Organigramme)



تمرين 7.38 دالة الوقت المتوقع للتوصيل

تجارة إلكترونية ★★ متوسط

اكتب دالة `addHours(day, hour, extra)` تضيف عددًا من الساعات إلى لحظة (يوم من الأسبوع 0..6 وساعة 0..23) وتعرض اللحظة الناتجة، ودالة `dayName(day)` تعرض اسم اليوم. استعملهما لحساب موعد وصول ثلاث طلبيات.

خطوات الحل

1) نحول اللحظة إلى ساعات منذ بداية الأسبوع: `day * 24 + hour + extra`.
2) اليوم الجديد `(total / 24) % 7`، والساعة `total % 24`.
3) `dayName` تطبع من جدول أسماء.

البرنامج بلغة C

```c
#include <stdio.h>

void dayName(int day) {
    char names[7][4] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
    printf("%s", names[day]);
}

void addHours(int day, int hour, int extra) {
    int total = day * 24 + hour + extra;      // ساعات منذ بداية الأسبوع
    dayName(day);
```

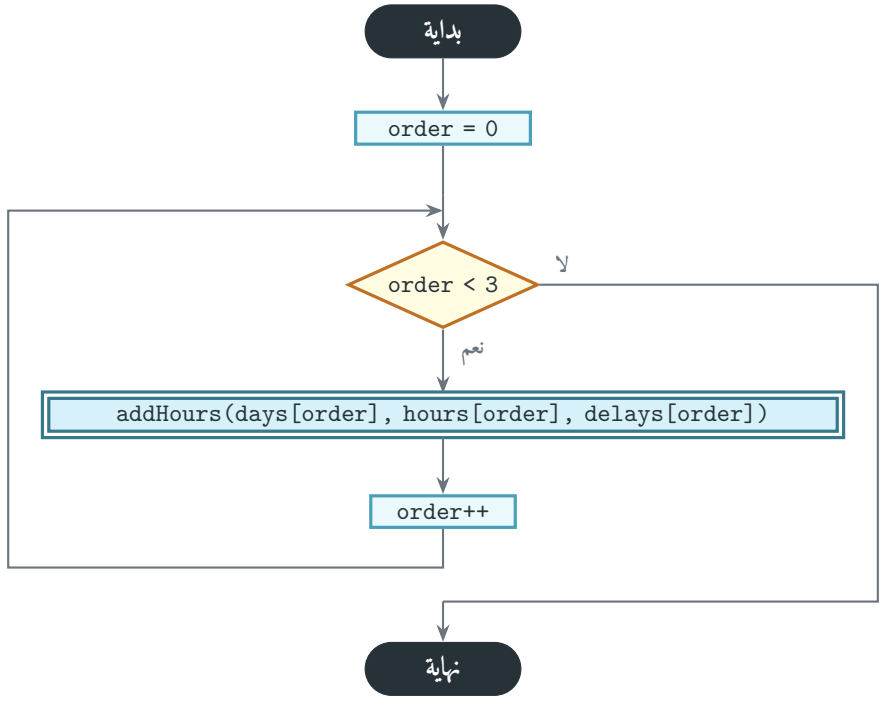
```c
    printf(" %02dh + %2dh -> ", hour, extra);
    dayName((total / 24) % 7);
    printf(" %02dh\n", total % 24);
}

int main(void) {
    int days[3] = {1, 4, 6}, hours[3] = {9, 16, 22}, delays[3] = {48, 30, 72};
    int order;
    for (order = 0; order < 3; order++)
        addHours(days[order], hours[order], delays[order]);
    return 0;
}
```

نتيجة التنفيذ

```
Mon 09h + 48h -> Wed 09h
Thu 16h + 30h -> Fri 22h
Sat 22h + 72h -> Tue 22h
```

المخطط الإنسيابي (Organigramme)



تمرين 7.39 دالة المتوسط المتحرك للمبيعات

علم البيانات

متوسط

اكتب دالة

```
movingAverage(values, size, window, result)
```

تحسب **المتوسط المتحرك** على نافذة من `window` يومًا لتنعيم المبيعات اليومية، وتعيد عدد القيم المحسوبة. قارن بين نافذة 3 ونافذة 5.

خطوات الحل

1) أول متوسط هو متوسط أول `window` قيمة.
2) كل متوسط تالٍ: نضيف القيمة الجديدة ونطرح القيمة الخارجة من النافذة (مجموع منزلق).

3) عدد النتائج `size - window + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

int movingAverage(double values[], int size, int window, double result[]) {
    double sum = 0;
    int idx;
    for (idx = 0; idx < window; idx++) sum += values[idx];
    result[0] = sum / window;
    for (idx = window; idx < size; idx++) {
        sum += values[idx] - values[idx - window];   // النافذة تنزلق بخانة
        result[idx - window + 1] = sum / window;
    }
    return size - window + 1;
}

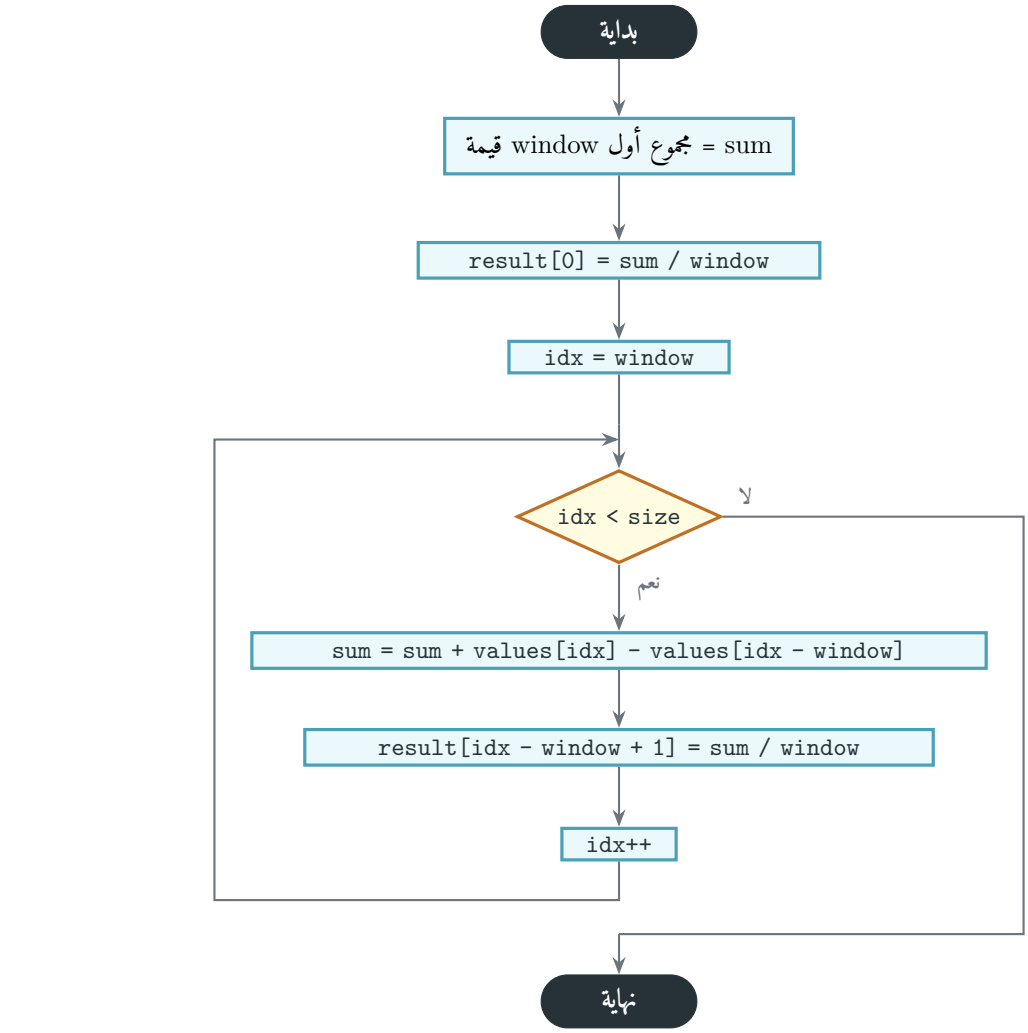
void printSeries(char label[], double values[], int size) {
    int idx;
    printf("%s", label);
    for (idx = 0; idx < size; idx++) printf(" %5.1f", values[idx]);
    printf("\n");
}

int main(void) {
    double sales[10] = {50, 72, 41, 66, 90, 58, 63, 95, 70, 81};
    double smooth[10];
    int count;
    printSeries("raw  :", sales, 10);
    count = movingAverage(sales, 10, 3, smooth);
    printSeries("MA(3):", smooth, count);
    count = movingAverage(sales, 10, 5, smooth);
    printSeries("MA(5):", smooth, count);
    return 0;
}
```

نتيجة التنفيذ

```
raw  :  50.0  72.0  41.0  66.0  90.0  58.0  63.0  95.0  70.0  81.0
MA(3):  54.3  59.7  65.7  71.3  70.3  72.0  76.0  82.0
MA(5):  63.8  65.4  63.6  74.4  75.2  73.4
```

المخطط الإنسيابي (Organigramme)



تمرين 7.40 دالة عمر الزبون وشريحته

علم البيانات ★★ متوسط

اكتب دالة

`ageAt(birthDay, birthMonth, birthYear, day, month, year)`

تحسب العمر بالسنوات الكاملة في تاريخ معين، ودالة `segment(age)` تعيد رقم الشريحة التسويقية (0: أقل من 18، 1: 18-24، 2: 25-39، 3: 40-59، 4: 60 فأكثر). احسب عدد الزبائن في كل شريحة.

خطوات الحل

1) العمر = فرق السنوات، ناقص 1 إذا لم يحل عيد الميلاد بعد في السنة الحالية.

2) `segment` سلسلة من `if` تعيد رقمًا، و `main` تزيد عداد الشريحة.

البرنامج بلغة C

```c
#include <stdio.h>

int ageAt(int birthDay, int birthMonth, int birthYear, int day, int month, int year) {
    int age = year - birthYear;
    if (month < birthMonth || (month == birthMonth && day < birthDay))
        age--;                                  // عيد الميلاد لم يحل بعد
```

```c
    return age;
}

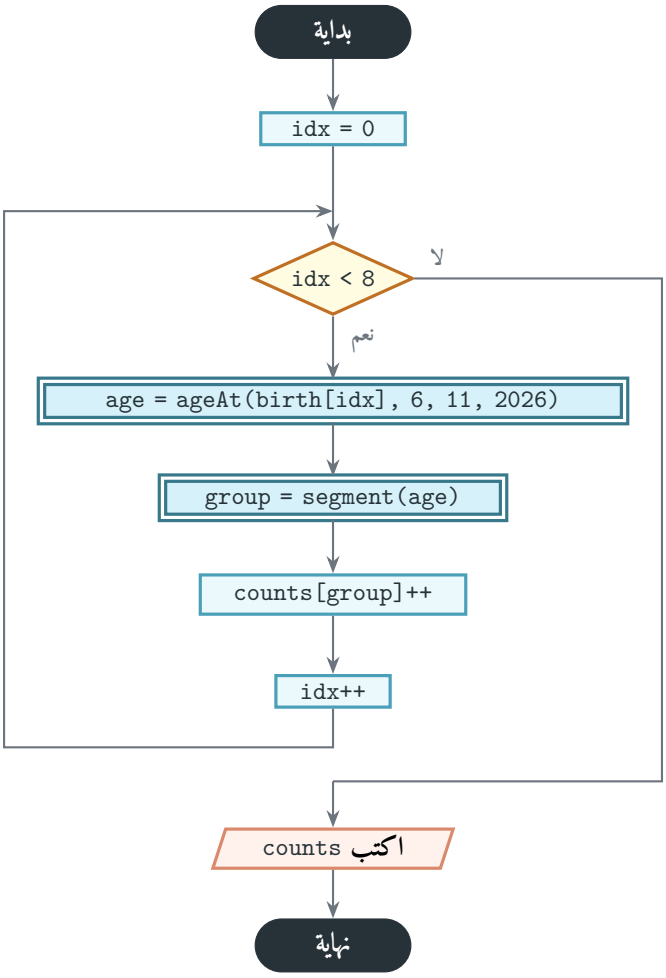
int segment(int age) {
    if (age < 18) return 0;
    if (age < 25) return 1;
    if (age < 40) return 2;
    if (age < 60) return 3;
    return 4;
}

int main(void) {
    int birth[8][3] = {{12, 3, 2001}, {30, 11, 2008}, {6, 11, 1990}, {7, 11, 1967},
                       {1, 1, 1958}, {15, 8, 2004}, {20, 12, 1986}, {9, 2, 2010}};
    char labels[5][8] = {"<18", "18-24", "25-39", "40-59", "60+"};
    int counts[5] = {0}, idx, age;
    for (idx = 0; idx < 8; idx++) {
        age = ageAt(birth[idx][0], birth[idx][1], birth[idx][2], 6, 11, 2026);
        counts[segment(age)]++;
        printf("Customer %d: %2d years\n", idx + 1, age);
    }
    for (idx = 0; idx < 5; idx++)
        printf("%-6s %d\n", labels[idx], counts[idx]);
    return 0;
}
```

نتيجة التنفيذ

```
Customer 1: 25 years
Customer 2: 17 years
Customer 3: 36 years
Customer 4: 58 years
Customer 5: 68 years
Customer 6: 22 years
Customer 7: 39 years
Customer 8: 16 years
<18    2
18-24  1
25-39  3
40-59  1
60+    1
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 7.41 الإنحدار الخطي بالدوال

علم البيانات صعب ★★★

اكتب مجموعة دوال لتدريب نموذج انحدار خطي sales = slope * ads + intercept بطريقة المربعات الصغرى: mean، fitSlope، fitIntercept، predict، و rSquared (معامل التحديد). درّب النموذج ثم تنبأ بالمبيعات لميزانيات جديدة.

خطوات الحل

1) slope = Σ(dx·dy) / Σ(dx²) حيث dx = x − meanX و dy = y − meanY.
2) intercept = meanY − slope * meanX.
3) R² = 1 − SSres / SStot، حيث SSres = Σ(y − predict(x))² و SStot = Σ(y − meanY)².

البرنامج بلغة C

```c
#include <stdio.h>

double mean(double values[], int size) {
    double total = 0;
    int idx;
    for (idx = 0; idx < size; idx++) total += values[idx];
    return total / size;
}

double fitSlope(double xs[], double ys[], int size) {
    double meanX = mean(xs, size), meanY = mean(ys, size), num = 0, den = 0;
    int idx;
    for (idx = 0; idx < size; idx++) {
        num += (xs[idx] - meanX) * (ys[idx] - meanY);
        den += (xs[idx] - meanX) * (xs[idx] - meanX);
    }
    return num / den;
}

double fitIntercept(double xs[], double ys[], int size, double slope) {
    return mean(ys, size) - slope * mean(xs, size);   // المستقيم يمر بالنقطة المتوسطة
}

double predict(double value, double slope, double intercept) {
    return slope * value + intercept;
}

double rSquared(double xs[], double ys[], int size, double slope, double intercept) {
    double meanY = mean(ys, size), ssRes = 0, ssTot = 0, err;
    int idx;
    for (idx = 0; idx < size; idx++) {
        err = ys[idx] - predict(xs[idx], slope, intercept);
        ssRes += err * err;                           // خطأ النموذج
        ssTot += (ys[idx] - meanY) * (ys[idx] - meanY);   // التشتت الكلي
    }
    return 1 - ssRes / ssTot;
}

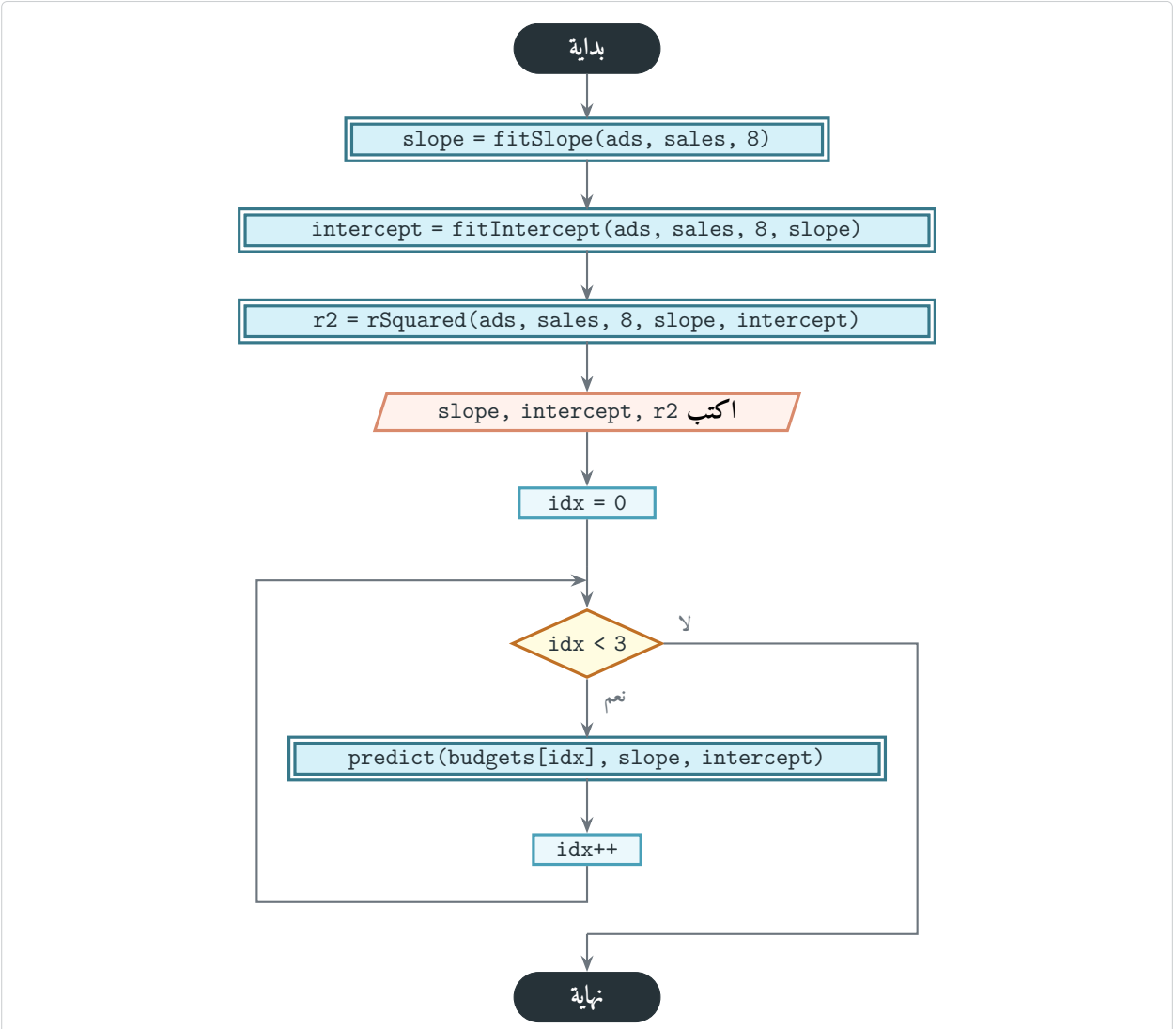
```

```c
int main(void) {
    double ads[8] = {10, 15, 8, 20, 25, 12, 30, 18};
    double sales[8] = {210, 260, 190, 330, 370, 240, 420, 300};
    double budgets[3] = {5, 22, 40};
    double slope = fitSlope(ads, sales, 8);
    double intercept = fitIntercept(ads, sales, 8, slope);
    int idx;
    printf("sales = %.3f * ads + %.3f\n", slope, intercept);
    printf("R^2 = %.4f\n", rSquared(ads, sales, 8, slope, intercept));
    for (idx = 0; idx < 3; idx++)
        printf("Budget %4.0f -> predicted sales %.1f\n", budgets[idx],
               predict(budgets[idx], slope, intercept));
    return 0;
}
```

نتيجة التنفيذ

```
sales = 10.535 * ads + 108.263
R^2 = 0.9948
Budget    5 -> predicted sales 160.9
Budget   22 -> predicted sales 340.0
Budget   40 -> predicted sales 529.7
```

المخطط الإنسيابي (Organigramme)



تمرين 7.42 الترتيب بالدمج التصاعدي بالدوال

تجارة إلكترونية صعب

اكتب دالة

`mergeRuns(source, target, left, middle, right)`

تدمج جزأين مرتبين متجاورين، ودالة `mergeSort(values, size)` تطبق **الترتيب بالدمج من الأسفل إلى الأعلى** (دون عودية): ندمج أجزاء طولها 1، ثم 2، ثم 4... حتى يرتب الجدول كله. رتب أسعار 12 منتجًا واعرض كل مرحلة.

خطوات الحل

1) في كل مرحلة `width`: ندمج `[left, left+width)` مع `[left+width, left+2*width)`.
2) الدمج يقرأ من `source` ويكتب في `target`، ثم ننسخ `target` إلى `source`.
3) عدد المراحل `log2(size)` تقريبًا، وكل مرحلة تمر على الجدول مرة واحدة.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAX 32

void mergeRuns(double source[], double target[], int left, int middle, int right) {
    int first = left, second = middle, out = left;
    while (first < middle && second < right)          // نأخذ الأصغر من الجزأين
        target[out++] = source[first] <= source[second] ? source[first++] : source[second++];
    while (first < middle) target[out++] = source[first++];
    while (second < right) target[out++] = source[second++];
}

void printArray(double values[], int size) {
    int idx;
    for (idx = 0; idx < size; idx++) printf("%.0f ", values[idx]);
    printf("\n");
}

void mergeSort(double values[], int size) {
    double buffer[MAX];
    int width, left, middle, right, idx;
    for (width = 1; width < size; width *= 2) {
        for (left = 0; left < size; left += 2 * width) {
            middle = left + width < size ? left + width : size;
            right = left + 2 * width < size ? left + 2 * width : size;
            mergeRuns(values, buffer, left, middle, right);
        }
        for (idx = 0; idx < size; idx++) values[idx] = buffer[idx];
        printf("width %2d: ", width);
        printArray(values, size);
    }
}

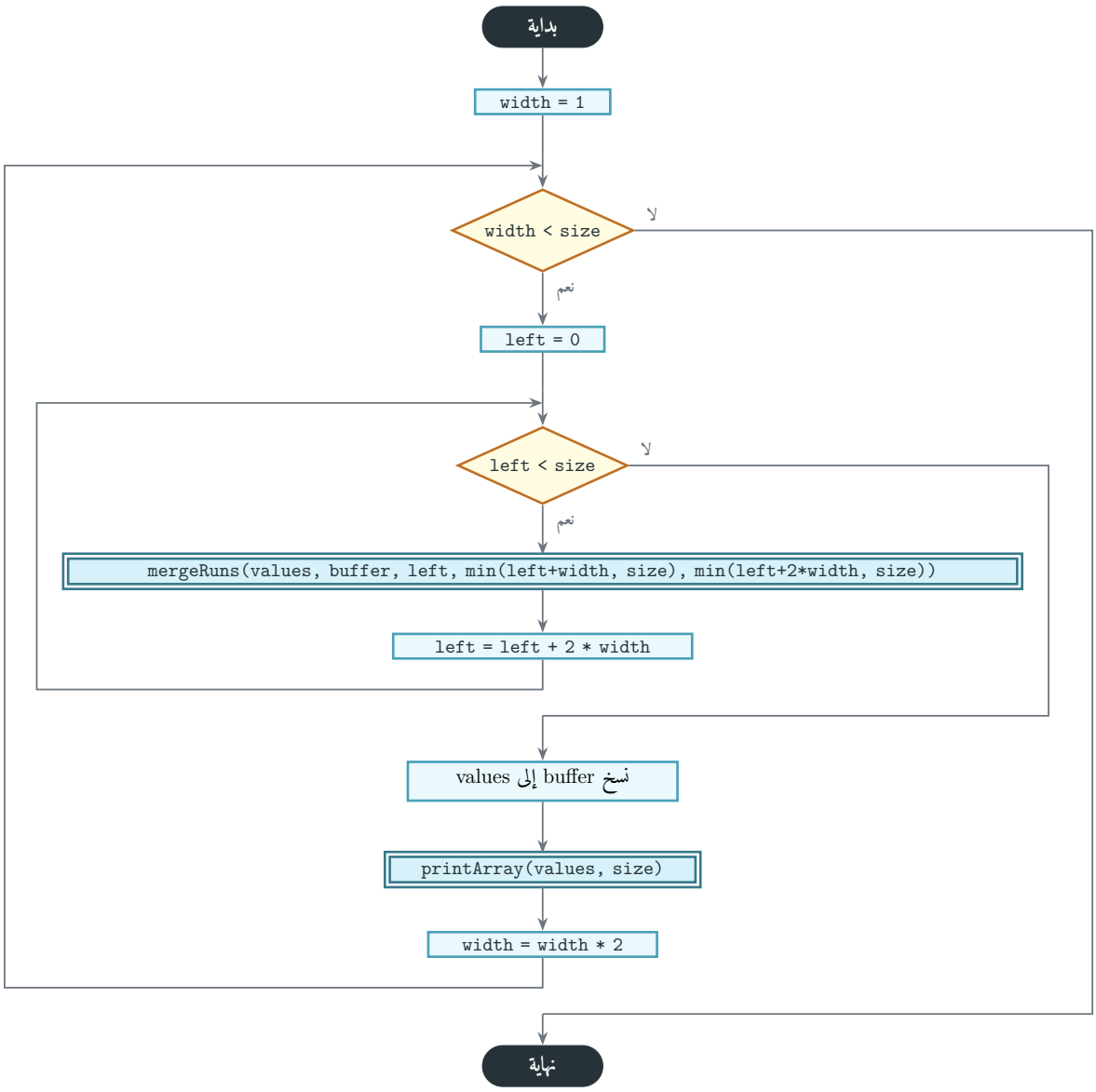
int main(void) {
    double prices[12] = {450, 12500, 899, 3200, 27000, 650, 1500, 99, 7800, 2300, 560, 4100};
    printf("start   : ");
    printArray(prices, 12);
    mergeSort(prices, 12);
    return 0;
}
```

نتيجة التنفيذ

```
start   : 450 12500 899 3200 27000 650 1500 99 7800 2300 560 4100
width  1: 450 12500 899 3200 650 27000 99 1500 2300 7800 560 4100
width  2: 450 899 3200 12500 99 650 1500 27000 560 2300 4100 7800
width  4: 99 450 650 899 1500 3200 12500 27000 560 2300 4100 7800
```

```
width  8: 99 450 560 650 899 1500 2300 3200 4100 7800 12500 27000
```

المخطط الإنسيابي (Organigramme)



تمرين 7.43 تجميع الزبائن بخوارزمية K-means أحادية البعد

علم البيانات صعب ★★★

نريد تقسيم الزبائن إلى **3 مجموعات** حسب مبلغ مشترياتهم السنوية. اكتب دوال `nearestCenter`، `assign`، و `updateCenters` لتطبيق **K-means**: نسند كل زبون إلى أقرب مركز، ثم نعيد حساب كل مركز كمتوسط مجموعته، ونكرر حتى لا يتغير أي إسناد.

خطوات الحل

1) المراكز الابتدائية: أصغر قيمة، قيمة وسطى، أكبر قيمة.
2) `assign` تعيد عدد الزبائن الذين تغيرت مجموعتهم؛ نتوقف عندما يصبح 0.
3) `updateCenters` تحسب متوسط كل مجموعة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12
#define NBK 3

int nearestCenter(double value, double centers[]) {
    int best = 0, idx;
    double dist, bestDist = -1;
    for (idx = 0; idx < NBK; idx++) {
        dist = value > centers[idx] ? value - centers[idx] : centers[idx] - value;
        if (bestDist < 0 || dist < bestDist) { bestDist = dist; best = idx; }
    }
    return best;
}

int assign(double spend[], int groups[], double centers[]) {
    int idx, group, changed = 0;
    for (idx = 0; idx < NB; idx++) {
        group = nearestCenter(spend[idx], centers);
        if (group != groups[idx]) { groups[idx] = group; changed++; }
    }
    return changed;
}

void updateCenters(double spend[], int groups[], double centers[]) {
    double sums[NBK] = {0};
    int counts[NBK] = {0}, idx;
    for (idx = 0; idx < NB; idx++) {
        sums[groups[idx]] += spend[idx];
        counts[groups[idx]]++;
    }
    for (idx = 0; idx < NBK; idx++)
        if (counts[idx] > 0) centers[idx] = sums[idx] / counts[idx];   // المركز = المتوسط
}

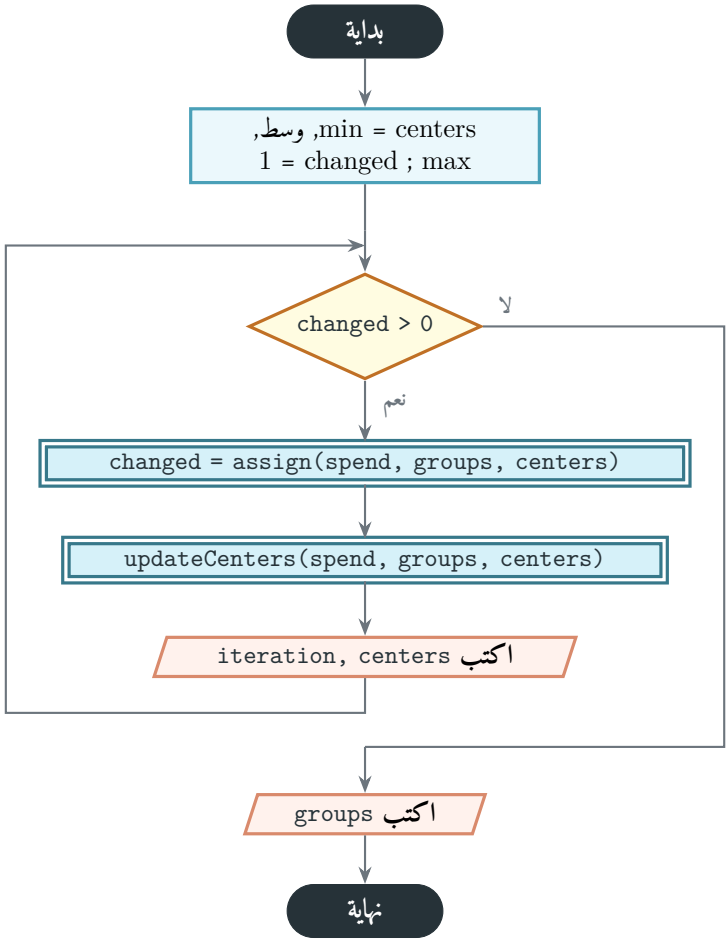
int main(void) {
    double spend[NB] = {12, 15, 9, 48, 52, 45, 110, 18, 55, 98, 120, 50};   // بالآلاف
    double centers[NBK] = {9, 50, 120};
    int groups[NB], idx, iteration = 0, changed = 1, group;
    for (idx = 0; idx < NB; idx++) groups[idx] = -1;
    while (changed > 0) {
        changed = assign(spend, groups, centers);
        updateCenters(spend, groups, centers);
        iteration++;
        printf("Iteration %d: %d changed, centers %.1f %.1f %.1f\n",
               iteration, changed, centers[0], centers[1], centers[2]);
    }
    for (group = 0; group < NBK; group++) {
        printf("Group %d:", group);
        for (idx = 0; idx < NB; idx++)
            if (groups[idx] == group) printf(" %.0f", spend[idx]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Iteration 1: 12 changed, centers 13.5 50.0 109.3
Iteration 2: 0 changed, centers 13.5 50.0 109.3
Group 0: 12 15 9 18
Group 1: 48 52 45 55 50
```

```
Group 2: 110 98 120
```

المخطط الإنسيابي (Organigramme)



تمرين 7.44 اختيار المنتجات الترويجية بمسألة حقيبة الظهر

تجارة إلكترونية صعب

ميزانية الحملة الإعلانية 50 ألف دج. لكل منتج **تكلفة** ترويج وربح متوقع. اكتب دالة `knapsack(costs, gains, size, budget, chosen)` بالبرمجة الديناميكية تعيد أكبر ربح ممكن وتحدد المنتجات المختارة (كل منتج مرة واحدة على الأكثر).

خطوات الحل

1) `best[item][money]` = أكبر ربح باستعمال أول `item` منتجًا وميزانية `money`.
2) إما لا نأخذ المنتج: `best[item-1][money]`، أو نأخذه: `gains + best[item-1][money - cost]`.
3) إعادة البناء: إذا اختلف `best[item][money]` عن `best[item-1][money]` فالمنتج مأخوذ.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6
#define BUDGET 50

```

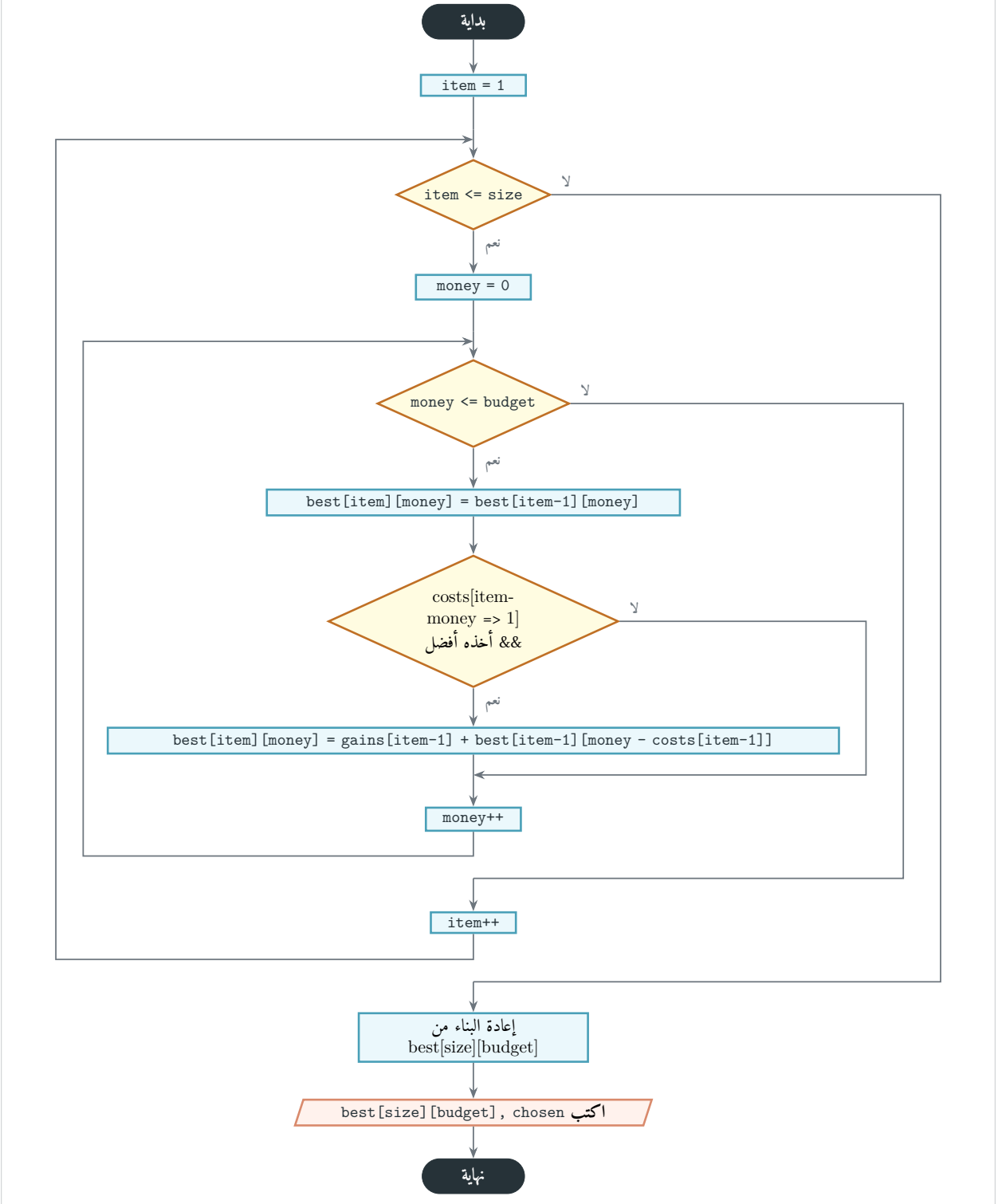
```c
int knapsack(int costs[], int gains[], int size, int budget, int chosen[]) {
    int best[NB + 1][BUDGET + 1], item, money, withItem;
    for (money = 0; money <= budget; money++) best[0][money] = 0;
    for (item = 1; item <= size; item++)
        for (money = 0; money <= budget; money++) {
            best[item][money] = best[item - 1][money];            // لا نأخذه
            if (costs[item - 1] <= money) {
                withItem = gains[item - 1] + best[item - 1][money - costs[item - 1]];
                if (withItem > best[item][money]) best[item][money] = withItem;
            }
        }
    money = budget;
    for (item = size; item >= 1; item--) {                        // إعادة البناء
        chosen[item - 1] = best[item][money] != best[item - 1][money];
        if (chosen[item - 1]) money -= costs[item - 1];
    }
    return best[size][budget];
}

int main(void) {
    char names[NB][12] = {"Phone", "Laptop", "Earbuds", "Watch", "Tablet", "Speaker"};
    int costs[NB] = {20, 30, 10, 15, 25, 5};       // بالآلاف
    int gains[NB] = {60, 90, 35, 40, 70, 18};
    int chosen[NB], idx, spent = 0;
    int total = knapsack(costs, gains, NB, BUDGET, chosen);
    for (idx = 0; idx < NB; idx++)
        if (chosen[idx]) {
            printf("Promote %-8s cost %2d gain %2d\n", names[idx], costs[idx], gains[idx]);
            spent += costs[idx];
        }
    printf("Budget used %d/%d, expected gain %d (thousand DA)\n", spent, BUDGET, total);
    return 0;
}
```

نتيجة التنفيذ

```
Promote Phone    cost 20 gain 60
Promote Earbuds  cost 10 gain 35
Promote Watch    cost 15 gain 40
Promote Speaker  cost  5 gain 18
Budget used 50/50, expected gain 153 (thousand DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 7.45 سلسلة ماركوف لحالات الزبائن — علم البيانات — صعب ★★★★

لكل زبون حالة شهرية: **نشط**، **خامل**، **مغادر**. مصفوفة الانتقال `P[from][to]` تعطي احتمال المرور من حالة إلى أخرى. اكتب دوال `multiplyVector` و `multiplyMatrix` و `printVector` لحساب توزيع الزبائن بعد 1، 3، 6، 12 شهراً انطلاقاً من 1000 زبون نشط.

خطوات الحل

1) توزيع الشهر التالي: `next[to] = Σ current[from] * P[from][to]`.

2) لـ 12 شهرًا يمكن رفع المصفوفة للأس بالضرب المتكرر `P^12`.

3) نتحقق أن مجموع كل توزيع يبقى 1000.

البرنامج بلغة C

```c
#include <stdio.h>

#define NS 3

void multiplyVector(double vec[], double matrix[NS][NS], double result[]) {
    int from, to;
    for (to = 0; to < NS; to++) {
        result[to] = 0;
        for (from = 0; from < NS; from++)
            result[to] += vec[from] * matrix[from][to];   // من كل حالة نحو to
    }
}

void multiplyMatrix(double left[NS][NS], double right[NS][NS], double result[NS][NS]) {
    int row, col, mid;
    for (row = 0; row < NS; row++)
        for (col = 0; col < NS; col++) {
            result[row][col] = 0;
            for (mid = 0; mid < NS; mid++)
                result[row][col] += left[row][mid] * right[mid][col];
        }
}

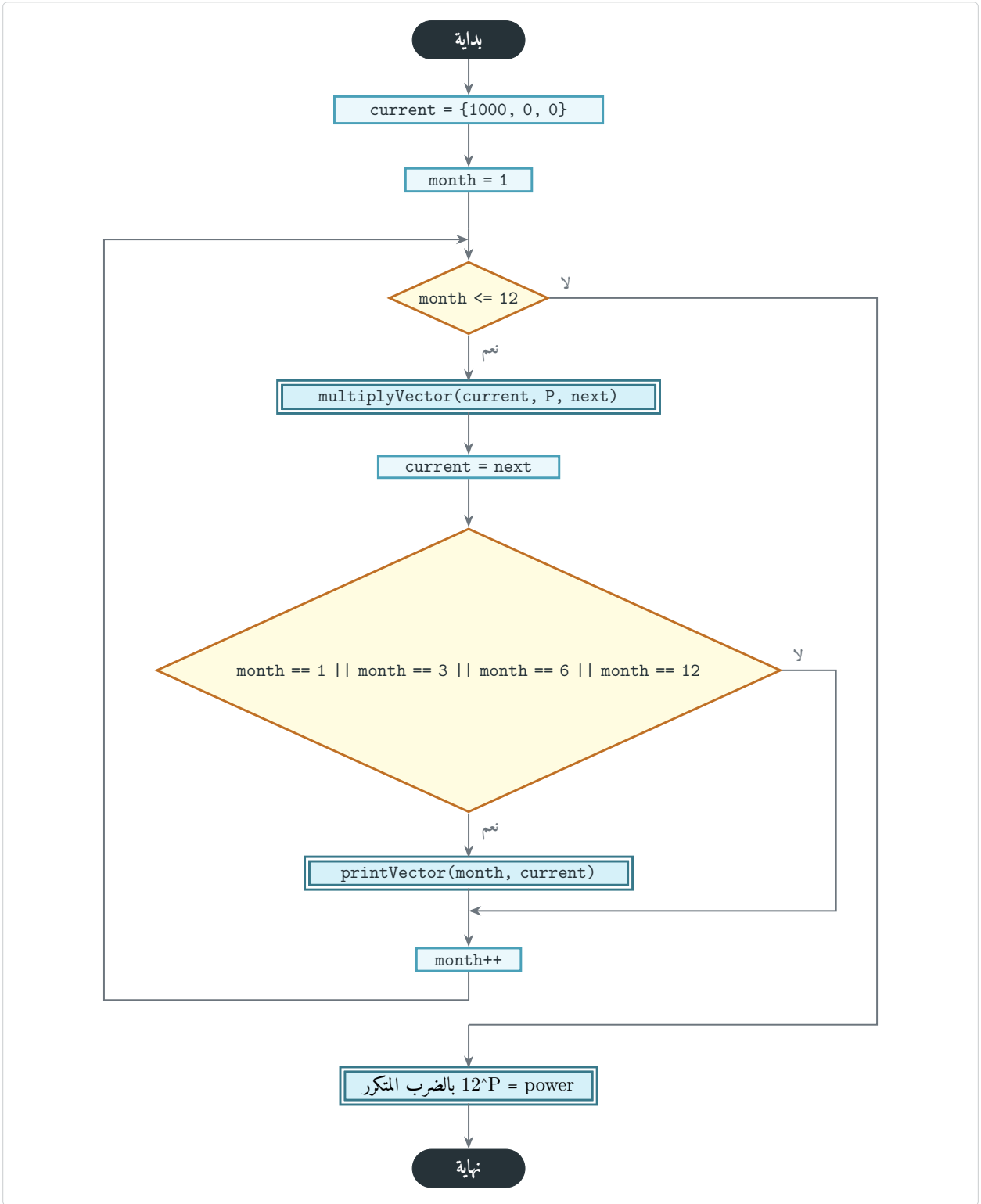
void printVector(int month, double vec[]) {
    printf("Month %2d: active %6.1f  idle %6.1f  churned %6.1f  (sum %.1f)\n",
           month, vec[0], vec[1], vec[2], vec[0] + vec[1] + vec[2]);
}

int main(void) {
    double trans[NS][NS] = {{0.80, 0.15, 0.05},
                            {0.30, 0.50, 0.20},
                            {0.00, 0.00, 1.00}};      // المغادر لا يعود
    double current[NS] = {1000, 0, 0}, next[NS];
    double power[NS][NS], tmp[NS][NS];
    int month, row, col;
    for (month = 1; month <= 12; month++) {
        multiplyVector(current, trans, next);
        for (row = 0; row < NS; row++) current[row] = next[row];
        if (month == 1 || month == 3 || month == 6 || month == 12)
            printVector(month, current);
    }
    for (row = 0; row < NS; row++)
        for (col = 0; col < NS; col++) power[row][col] = trans[row][col];
    for (month = 2; month <= 12; month++) {             // P^12
        multiplyMatrix(power, trans, tmp);
        for (row = 0; row < NS; row++)
            for (col = 0; col < NS; col++) power[row][col] = tmp[row][col];
    }
    printf("P^12 first row: %.4f %.4f %.4f\n", power[0][0], power[0][1], power[0][2]);
    return 0;
}
```

نتيجة التنفيذ

```
Month  1: active  800.0  idle  150.0  churned   50.0  (sum 1000.0)
Month  3: active  606.5  idle  200.2  churned  193.2  (sum 1000.0)
Month  6: active  448.0  idle  162.7  churned  389.3  (sum 1000.0)
Month 12: active  253.7  idle   92.9  churned  653.5  (sum 1000.0)
P^12 first row: 0.2537 0.0929 0.6535
```

المخطط الإنسيابي (Organigramme)



تمرين 7.46 سعر التعادل بطريقة التنصيف — تجارة إلكترونية — صعب ★★★

ربح المنتج بدلالة سعر البيع p (بالآلاف): profit(p) = (p - 2) * demand(p) - 40 حيث demand(p) = 120 / (1 + 0.04 p²). اكتب دالة bisection(low, high, tolerance) تجد سعر التعادل (حيث الربح = 0) بطريقة التنصيف، ودالة bestPrice تبحث عن السعر الذي يعظّم الربح بطريقة البحث الثلاثي.

خطوات الحل

1) التنصيف: إذا اختلفت إشارة profit عند طرفي المجال، فالجذر في المنتصف أو أحد النصفين؛ نحتفظ بالنصف الذي تتغير فيه الإشارة.
2) البحث الثلاثي: نقسم المجال بنقطتين m1 < m2، إذا profit(m1) < profit(m2) نحذف الثلث الأيسر، وإلا الأيمن.

البرنامج بلغة C

```c
#include <stdio.h>

double demand(double price) {
    return 120 / (1 + 0.04 * price * price);
}

double profit(double price) {
    return (price - 2) * demand(price) - 40;
}

double bisection(double low, double high, double tolerance) {
    double middle;
    int steps = 0;
    while (high - low > tolerance) {
        middle = (low + high) / 2;
        if (profit(low) * profit(middle) <= 0) high = middle;   // الجذر في النصف الأيسر
        else low = middle;
        steps++;
    }
    printf("Bisection: %d steps\n", steps);
    return (low + high) / 2;
}

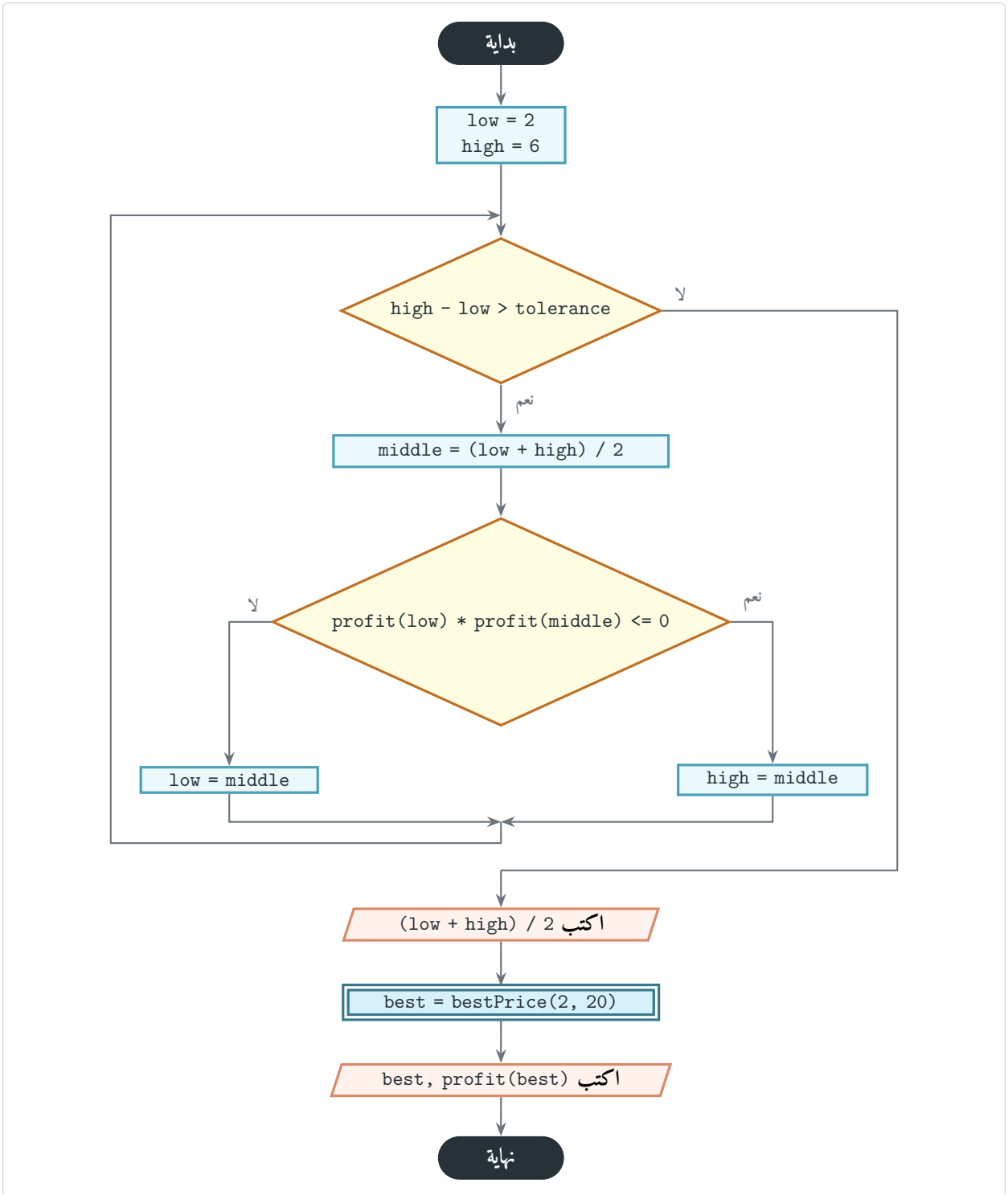
double bestPrice(double low, double high) {
    double third1, third2;
    while (high - low > 1e-6) {
        third1 = low + (high - low) / 3;
        third2 = high - (high - low) / 3;
        if (profit(third1) < profit(third2)) low = third1;    // القمة ليست في الثلث الأيسر
        else high = third2;
    }
    return (low + high) / 2;
}

int main(void) {
    double breakEven = bisection(2, 6, 1e-6);
    double best = bestPrice(2, 20);
    printf("Break-even price: %.4f thousand DA (profit %.6f)\n", breakEven, profit(breakEven));
    printf("Best price: %.4f thousand DA, max profit %.3f\n", best, profit(best));
    printf("Demand at best price: %.1f units\n", demand(best));
    return 0;
}
```

نتيجة التنفيذ

```
Bisection: 22 steps
Break-even price: 2.4108 thousand DA (profit 0.000021)
Best price: 7.3852 thousand DA, max profit 163.110
Demand at best price: 37.7 units
```

المخطط الإنسيابي (Organigramme)



تمرين 7.47 التكامل العددي لمنحنى الطلب

علم البيانات صعب

معدل الطلبات خلال اليوم (طلبية/ساعة) يتبع `rate(h) = 20 + 60 * h * (24 - h) / 144`. عدد الطلبات بين ساعتين هو **التكامل**. اكتب دالتي `trapezoid(start, end, steps)` و `simpson(start, end, steps)` وقارن دقتهما مع القيمة الدقيقة (1440) لعدد متزايد من الأقسام.

خطوات الحل

1) شبه المنحرف:

```
h * (f(a)/2 + f(a+h) + ... + f(b-h) + f(b)/2)
```

.

2) سمبسون (عدد أقسام زوجي):

```
h/3 * (f(a) + 4 f(  ) + 2 f(  ) + f(b))
```

.

3) الخطأ = |1440 - | .

البرنامج بلغة C

```c
#include <stdio.h>

double rate(double hour) {
    return 20 + 60 * hour * (24 - hour) / 144;
}

double absValue(double value) { return value < 0 ? -value : value; }

double trapezoid(double start, double end, int steps) {
    double width = (end - start) / steps, sum = (rate(start) + rate(end)) / 2;
    int idx;
    for (idx = 1; idx < steps; idx++) sum += rate(start + idx * width);
    return sum * width;
}

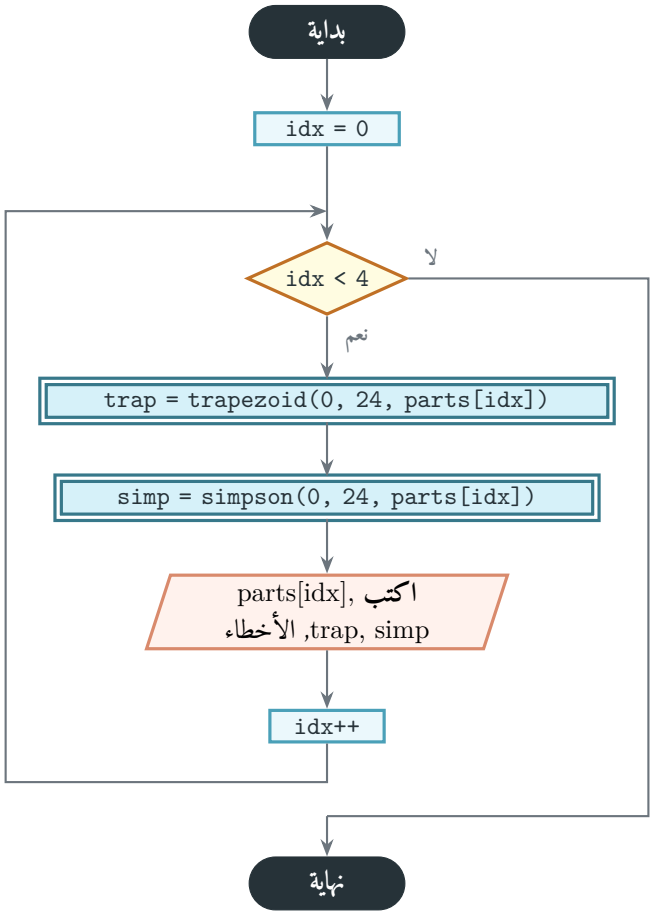
double simpson(double start, double end, int steps) {
    double width = (end - start) / steps, sum = rate(start) + rate(end);
    int idx;
    for (idx = 1; idx < steps; idx++)
        sum += (idx % 2 == 1 ? 4 : 2) * rate(start + idx * width);   // أوزان 4 و 2
    return sum * width / 3;
}

int main(void) {
    int parts[4] = {2, 4, 8, 16}, idx;
    double trap, simp, exact = 1440;
    printf("Exact daily orders: %.1f\n", exact);
    printf("steps  trapezoid   error   simpson   error\n");
    for (idx = 0; idx < 4; idx++) {
        trap = trapezoid(0, 24, parts[idx]);
        simp = simpson(0, 24, parts[idx]);
        printf("%5d %10.3f %7.3f %9.3f %7.3f\n", parts[idx], trap,
               absValue(trap - exact), simp, absValue(simp - exact));
    }
    printf("Orders 18h-22h: %.1f\n", simpson(18, 22, 8));
    return 0;
}
```

نتيجة التنفيذ

```
Exact daily orders: 1440.0
steps  trapezoid   error   simpson   error
    2   1200.000 240.000  1440.000   0.000
    4   1380.000  60.000  1440.000   0.000
    8   1425.000  15.000  1440.000   0.000
   16   1436.250   3.750  1440.000   0.000
Orders 18h-22h: 211.1
```

المخطط الإنسيابي (Organigramme)



تمرين 7.48 غربال إراتوستينس وتحليل الأعداد

علم البيانات صعب

اكتب دالة `sieve(limit, isPrime)` تعلّم الأعداد الأولية حتى `limit` بطريقة **غربال إراتوستينس**، ودالة `factorize(number, isPrime)` تحلل عددًا إلى جداء عوامل أولية باستعمال الجدول. حلل كميات الطلبيات لإيجاد أحجام العلب الممكنة.

خطوات الحل

1) نبدأ باعتبار كل الأعداد أولية، لكل أولي `prime` نشطب مضاعفاته بدءًا من `prime * prime`.
2) التحليل: نقسم العدد على كل أولي ما دام القسمة ممكنة ونعد الأس.

البرنامج بلغة C

```c
#include <stdio.h>

#define LIMIT 10000

void sieve(int limit, char isPrime[]) {
    int number, multiple;
    for (number = 0; number <= limit; number++) isPrime[number] = number >= 2;
    for (number = 2; number * number <= limit; number++)
        if (isPrime[number])
            for (multiple = number * number; multiple <= limit; multiple += number)
```

```c
                isPrime[multiple] = 0;                  // مضاعف = غير أولي
}

void factorize(int number, char isPrime[]) {
    int prime, exponent, first = 1;
    printf("%d =", number);
    for (prime = 2; prime <= number; prime++) {
        if (!isPrime[prime]) continue;
        exponent = 0;
        while (number % prime == 0) {
            number /= prime;
            exponent++;
        }
        if (exponent > 0) {
            printf("%s %d", first ? "" : " x", prime);
            if (exponent > 1) printf("^%d", exponent);
            first = 0;
        }
    }
    printf("\n");
}

int main(void) {
    char isPrime[LIMIT + 1];
    int number, query, count = 0;
    sieve(LIMIT, isPrime);
    for (number = 0; number <= LIMIT; number++) count += isPrime[number];
    printf("%d primes up to %d\n", count, LIMIT);
    for (query = 0; query < 4; query++) {
        scanf("%d", &number);
        factorize(number, isPrime);
    }
    return 0;
}
```

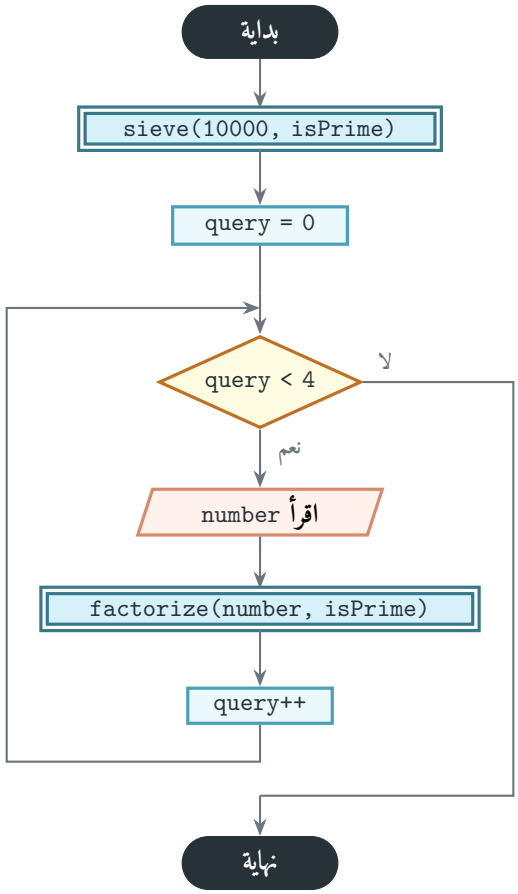
المدخلات

```
360 1001 9973 4096
```

نتيجة التنفيذ

```
1229 primes up to 10000
360 = 2^3 x 3^2 x 5
1001 = 7 x 11 x 13
9973 = 9973
4096 = 2^12
```

المخطط الإنسيابي (Organigramme)



تمرين 7.49 دوال التواريخ لحساب موعد التسليم

تجارة إلكترونية صعب ★★★

اكتب دوال التواريخ التالية: `daysInMonth(month, year)`، `dayOfWeek(day, month, year)` (صيغة زيلر)، `dayNumber` (عدد الأيام منذ 1/1/2000)، و `addBusinessDays` التي تضيف أيام عمل (تتخطى الجمعة والسبت). احسب موعد تسليم طلبيات ومدة التأخير.

خطوات الحل

1) زيلر: نعتبر جانفي وفيفري الشهرين 13 و 14 من السنة السابقة، ثم

```
(day + 13(month+1)/5 + yearOfCentury + yearOfCentury/4 + century/4 + 5*century) % 7
```

(0 = السبت).

2) `addBusinessDays`: تتقدم يومًا يومًا، ونعد اليوم فقط إذا لم يكن جمعة أو سبتًا.

3) الفرق بين تاريخين = فرق `dayNumber`.

البرنامج بلغة C

```c
#include <stdio.h>

int daysInMonth(int month, int year) {
    int days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (month == 2 && ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)) return 29;
    return days[month - 1];
}
```

```c

int dayOfWeek(int day, int month, int year) {    // السبت، 6 = الجمعة = 0
    int century, yearOfCentury;
    if (month < 3) { month += 12; year--; }
    century = year / 100;
    yearOfCentury = year % 100;
    return (day + 13 * (month + 1) / 5 + yearOfCentury + yearOfCentury / 4
            + century / 4 + 5 * century) % 7;
}

long dayNumber(int day, int month, int year) {
    long total = 0;
    int walk;
    for (walk = 2000; walk < year; walk++) total += 365 + (daysInMonth(2, walk) == 29);
    for (walk = 1; walk < month; walk++) total += daysInMonth(walk, year);
    return total + day - 1;
}

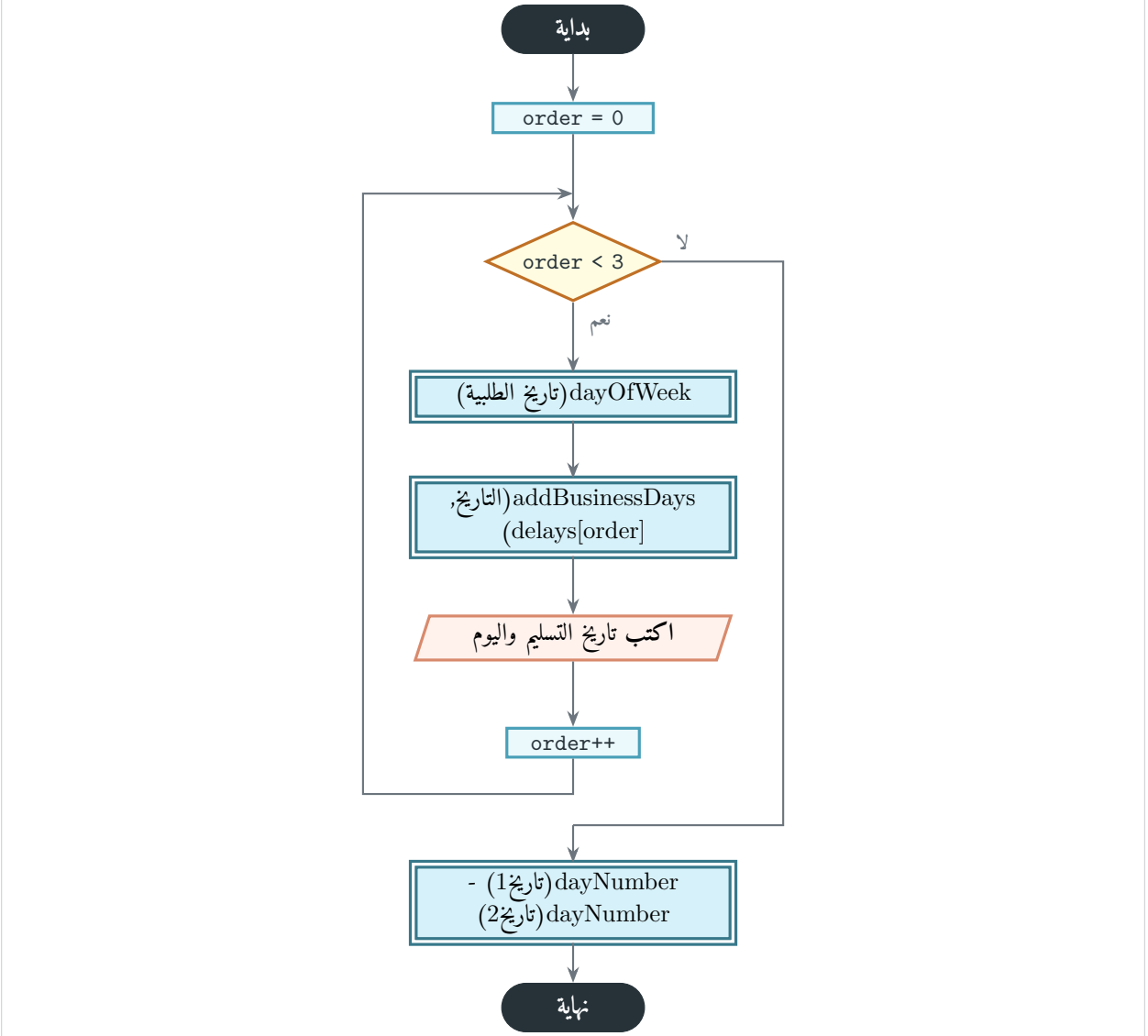
void addBusinessDays(int date[], int count) {
    int weekday;
    while (count > 0) {
        date[0]++;                                    // اليوم التالي
        if (date[0] > daysInMonth(date[1], date[2])) {
            date[0] = 1;
            if (++date[1] > 12) { date[1] = 1; date[2]++; }
        }
        weekday = dayOfWeek(date[0], date[1], date[2]);
        if (weekday != 6 && weekday != 0) count--;    // تخطى الجمعة والسبت
    }
}

int main(void) {
    char names[7][4] = {"Sat", "Sun", "Mon", "Tue", "Wed", "Thu", "Fri"};
    int orders[3][3] = {{27, 11, 2026}, {29, 12, 2026}, {25, 2, 2028}};
    int delays[3] = {3, 5, 4}, date[3], order;
    for (order = 0; order < 3; order++) {
        date[0] = orders[order][0]; date[1] = orders[order][1]; date[2] = orders[order][2];
        printf("%s %02d/%02d/%d + %d business days -> ", names[dayOfWeek(date[0], date[1], date[2])],
               date[0], date[1], date[2], delays[order]);
        addBusinessDays(date, delays[order]);
        printf("%s %02d/%02d/%d\n", names[dayOfWeek(date[0], date[1], date[2])],
               date[0], date[1], date[2]);
    }
    printf("Days between 01/03/2026 and 06/11/2026: %ld\n",
           dayNumber(6, 11, 2026) - dayNumber(1, 3, 2026));
    return 0;
}
```

نتيجة التنفيذ

```
Fri 27/11/2026 + 3 business days -> Tue 01/12/2026
Tue 29/12/2026 + 5 business days -> Tue 05/01/2027
Fri 25/02/2028 + 4 business days -> Wed 01/03/2028
Days between 01/03/2026 and 06/11/2026: 250
```

المخطط الإنسيابي (Organigramme)



تمرين 7.50 تصنيف الزبائن بأقرب الجيران

علم البيانات ★★★ صعب

لدينا 10 زبائن معروفين بخاصيتين (عدد الزيارات الشهرية، متوسط السلة بالآلاف) وتصنيفهم (يشتري 1 / لا يشتري 0). اكتب دوال `distance2` و `findNearest` و `classify` لتطبيق خوارزمية **k أقرب جار (k = 3)** على زبائن جدد بالتصويت.

خطوات الحل

1) المسافة الإقليدية المربعة كافية للمقارنة (لا حاجة للجذر).
2) `findNearest` تختار `k` مرات أقرب نقطة لم تُختر بعد.
3) التصنيف = الفئة الأكثر بين الجيران `k`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 10
```

```c
#define NBK 3

double distance2(double visitsA, double basketA, double visitsB, double basketB) {
    return (visitsA - visitsB) * (visitsA - visitsB) + (basketA - basketB) * (basketA - basketB);
}

void findNearest(double visits[], double basket[], double qVisits, double qBasket, int nearest[]) {
    int used[NB] = {0}, rank, idx, best;
    double dist, bestDist;
    for (rank = 0; rank < NBK; rank++) {
        best = -1;
        bestDist = 0;
        for (idx = 0; idx < NB; idx++) {
            if (used[idx]) continue;
            dist = distance2(visits[idx], basket[idx], qVisits, qBasket);
            if (best < 0 || dist < bestDist) { best = idx; bestDist = dist; }
        }
        used[best] = 1;                        // لا نختار نفس الجار مرتين
        nearest[rank] = best;
    }
}

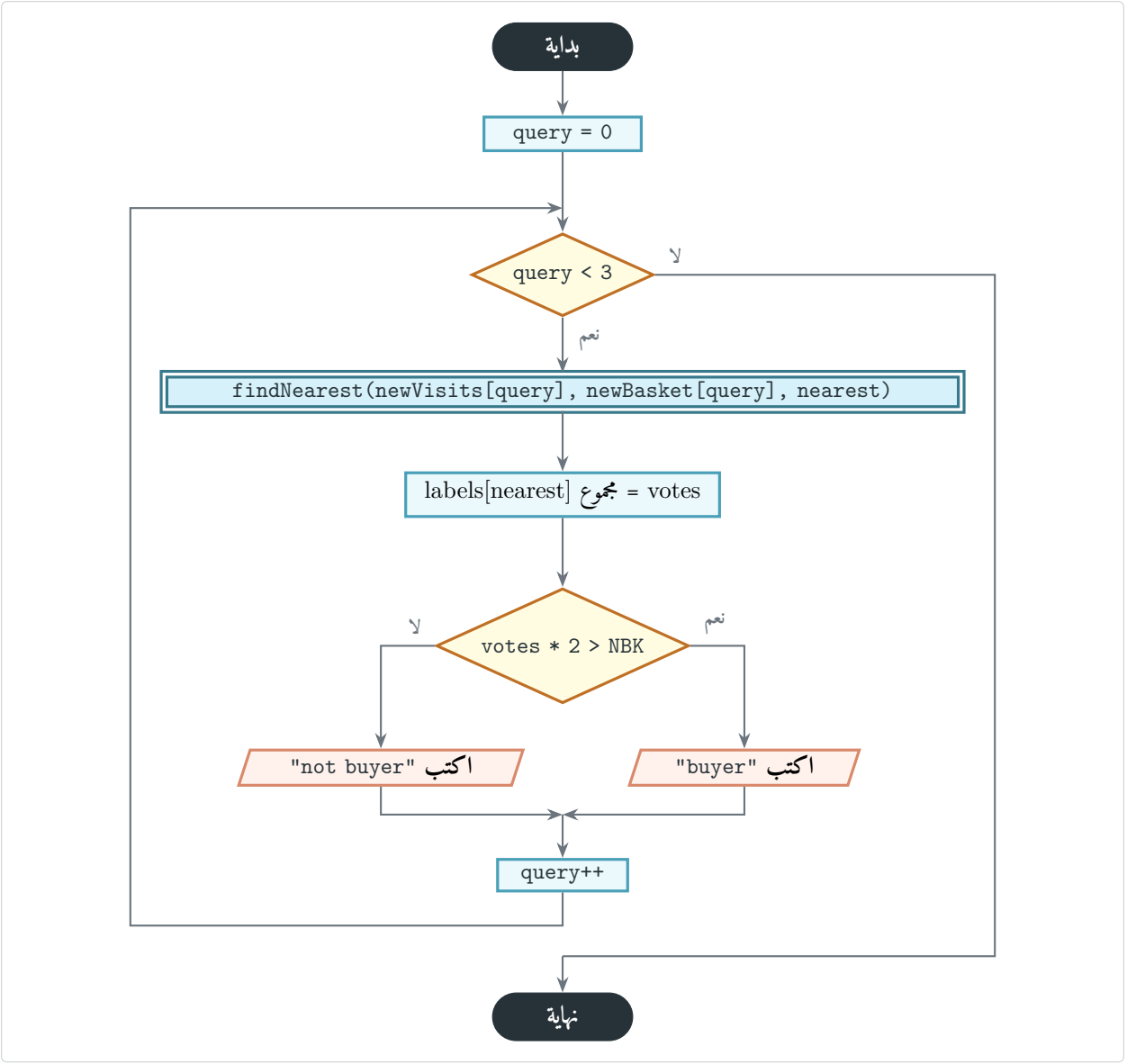
int classify(double visits[], double basket[], int labels[], double qVisits, double qBasket) {
    int nearest[NBK], rank, votes = 0;
    findNearest(visits, basket, qVisits, qBasket, nearest);
    printf("  neighbours:");
    for (rank = 0; rank < NBK; rank++) {
        printf(" #%d", nearest[rank]);
        votes += labels[nearest[rank]];
    }
    return votes * 2 > NBK;                    // أغلبية الأصوات
}

int main(void) {
    double visits[NB] = {1, 2, 8, 9, 3, 10, 2, 7, 6, 1};
    double basket[NB] = {2.0, 1.5, 6.0, 7.5, 2.5, 5.0, 3.0, 8.0, 4.5, 1.0};
    int labels[NB] = {0, 0, 1, 1, 0, 1, 0, 1, 1, 0};
    double newVisits[3] = {7, 2, 5}, newBasket[3] = {5.5, 2.0, 3.5};
    int query, result;
    for (query = 0; query < 3; query++) {
        printf("Customer (%.0f visits, %.1fk):", newVisits[query], newBasket[query]);
        result = classify(visits, basket, labels, newVisits[query], newBasket[query]);
        printf(" -> %s\n", result ? "buyer" : "not buyer");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Customer (7 visits, 5.5k):  neighbours: #2 #8 #7 -> buyer
Customer (2 visits, 2.0k):  neighbours: #1 #0 #6 -> not buyer
Customer (5 visits, 3.5k):  neighbours: #8 #4 #6 -> not buyer
```

المخطط الإنسيابي (Organigramme)

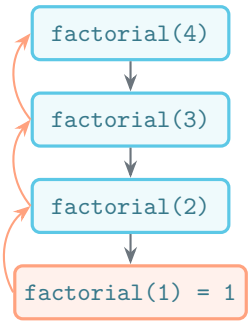


الفصل 8

العودية

تعريفات

- **الدالة العودية**: دالة تستدعي نفسها على مسألة **أصغر** من نفس النوع.
- **الحالة الأساسية**: حالة صغيرة نعرف جوابها مباشرة وتوقف الاستدعاءات (مثلًا `0! = 1`).
- **الحالة العودية**: تقرّب المسألة من الحالة الأساسية (`count * factorial(count - 1)`).
- **التذكر** (memoization): حفظ النتائج المحسوبة في جدول لتفادي إعادة حسابها.
- **التراجع** (backtracking): نجرب اختيارًا، ونستدعي عوديًا، ثم **نلغي** الاختيار إذا فشل.



الاستدعاءات تنزل حتى الحالة الأساسية، ثم تصعد النتائج (بالبرتقالي) وكل مستوى يكمل حسابه.

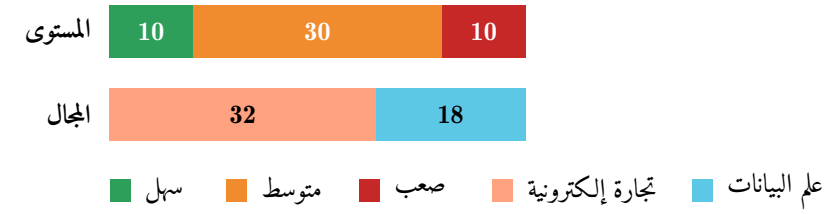
النمط	أمثلة في هذا الفصل
عودية بسيطة	المضروب، مجموع الأرقام، القوة، البحث الثنائي
فرّق تسد	الترتيب بالدمج، الترتيب السريع، أقرب زوج من النقاط
التذكر	فيبوناتشي، عدد طرق الدفع، مسارات الشبكة
التراجع	التباديل، الملكات، السودوكو، المتاهة

لا تنسَ الحالة الأساسية

دالة عودية دون حالة أساسية (أو لا تقترب منها) تستدعي نفسها إلى ما لا نهاية حتى تمتلئ **مكدس الاستدعاءات** ويتوقف البرنامج (stack overflow).

للاستزادة: [5, 10, 9]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 8.1 عدد طرق ترتيب المنتجات في الواجهة

تجارة إلكترونية ⭐ سهل

بكم طريقة يمكن ترتيب `count` منتجات في واجهة المتجر؟ الجواب هو `count!`. اكتب دالة **عودية** `factorial(count)` تعتمد على `count! = count × (count-1)!` مع الحالة الأساسية `0! = 1`.

خطوات الحل

1) الحالة الأساسية: إذا `count <= 1` نعيد 1.
2) الحالة العودية: `count * factorial(count - 1)`.

البرنامج بلغة C

```c
#include <stdio.h>

long factorial(int count) {
    if (count <= 1)
        return 1;                          // الحالة الأساسية
    return count * factorial(count - 1);   // الاستدعاء العودي
}

int main(void) {
    int count, idx;
    scanf("%d", &count);
    for (idx = 1; idx <= count; idx++)
        printf("%d products: %ld arrangements\n", idx, factorial(idx));
    return 0;
}
```

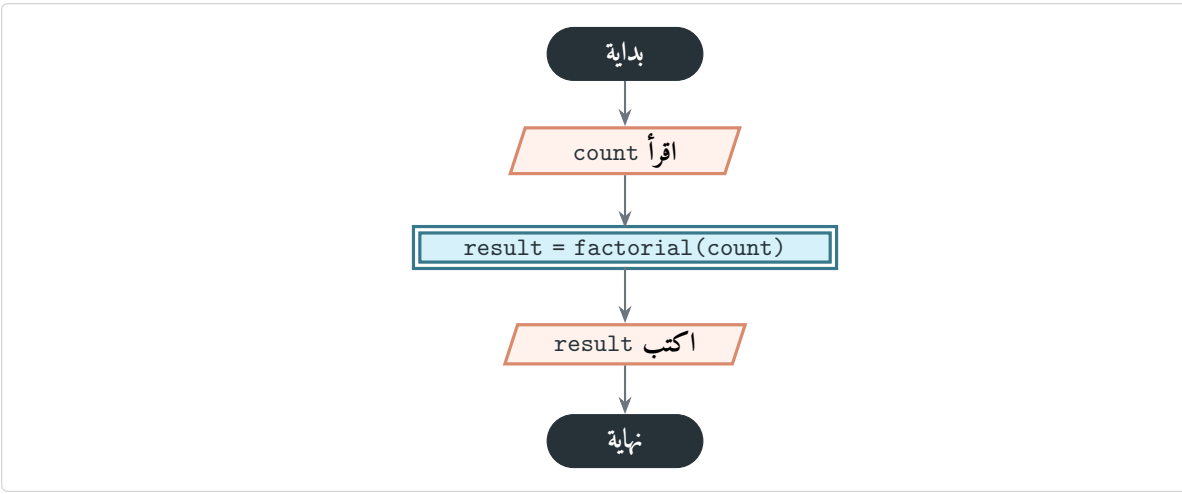
المدخلات

```
6
```

نتيجة التنفيذ

```
1 products: 1 arrangements
2 products: 2 arrangements
3 products: 6 arrangements
4 products: 24 arrangements
5 products: 120 arrangements
6 products: 720 arrangements
```

المخطط الإنسيابي (Organigramme)



تمرين 8.2 مجموع أرقام رقم الطلبية عوديًا

تجارة إلكترونية سهل

اكتب دالة عودية `digitSum(number)` تحسب مجموع أرقام رقم الطلبية: آخر رقم + مجموع أرقام الباقي.

خطوات الحل

1) الحالة الأساسية: `number < 10` نعيد `number`.
2) العودية: `number % 10 + digitSum(number / 10)`.

البرنامج بلغة C

```c
#include <stdio.h>

int digitSum(long number) {
    if (number < 10)
        return (int)number;                    // رقم واحد
    return number % 10 + digitSum(number / 10);  // آخر رقم + الباقي
}

int main(void) {
    long number;
    scanf("%ld", &number);
    printf("Digit sum of %ld = %d\n", number, digitSum(number));
    return 0;
}
```

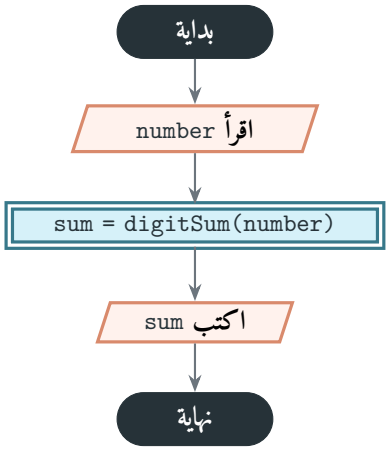
المدخلات

```
2026045
```

نتيجة التنفيذ

```
Digit sum of 2026045 = 19
```

المخطط الإنسيابي (Organigramme)



تمرين 8.3 القوة عوديًا لنمو المتابعين

علم البيانات سهل

عدد متابعي صفحة المتجر يتضاعف بنسبة 1.1 كل أسبوع. اكتب دالة عودية power(base, exponent) حيث base^0 = 1 و base^n = base × base^(n-1)، واحسب عدد المتابعين بعد 4 و 8 و 12 أسبوعًا انطلاقًا من 2000.

خطوات الحل

1) الحالة الأساسية exponent == 0.
2) كل استدعاء ينقص الأس بواحد.

البرنامج بلغة C

```c
#include <stdio.h>

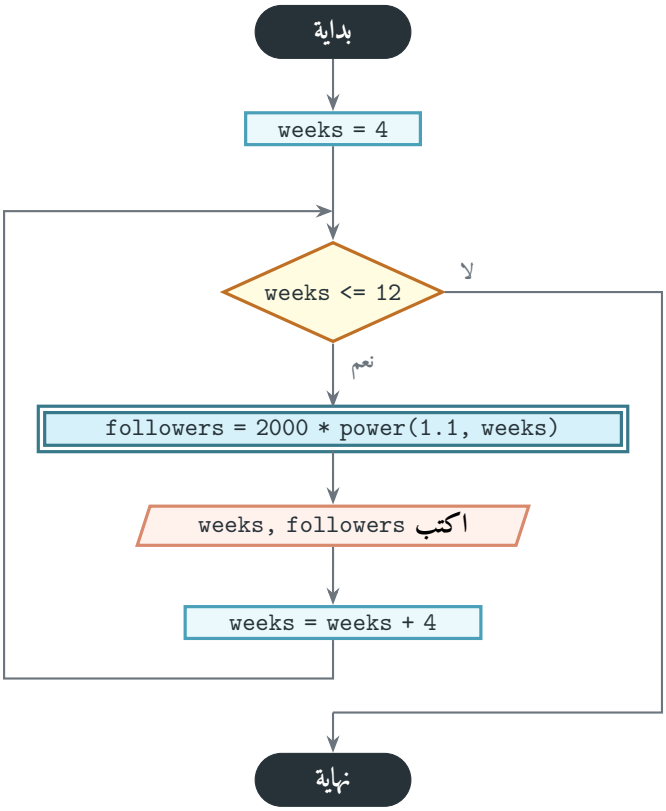
double power(double base, int exponent) {
    if (exponent == 0)
        return 1;
    return base * power(base, exponent - 1);
}

int main(void) {
    int weeks;
    for (weeks = 4; weeks <= 12; weeks += 4)
        printf("After %2d weeks: %.0f followers\n", weeks, 2000 * power(1.1, weeks));
    return 0;
}
```

نتيجة التنفيذ

```
After  4 weeks: 2928 followers
After  8 weeks: 4287 followers
After 12 weeks: 6277 followers
```

المخطط الإنسيابي (Organigramme)



تمرين 8.4 العد التنازلي لنهاية العرض

تجارة إلكترونية
سهل

اكتب دالة عودية `countdown(hours)` تعرض الساعات المتبقية لنهاية التخفيضات من `hours` إلى 1 ثم تعرض "Sale over!"، ودالة `countUp` تعرضها تصاعديًا (الطباعة بعد الاستدعاء العودي).

خطوات الحل

1) `countdown`: نطبع ثم نستدعي بـ `hours - 1`.
2) `countUp`: نستدعي أولًا ثم نطبع: الترتيب ينعكس.

البرنامج بلغة C

```c
#include <stdio.h>

void countdown(int hours) {
    if (hours == 0) {
        printf("Sale over!\n");
        return;
    }
    printf("%d ", hours);              // نطبع قبل الاستدعاء
    countdown(hours - 1);
}

void countUp(int hours) {
    if (hours == 0) return;
    countUp(hours - 1);                // نستدعي قبل الطباعة
```

```c
    printf("%d ", hours);
}

int main(void) {
    int hours;
    scanf("%d", &hours);
    countdown(hours);
    countUp(hours);
    printf("\n");
    return 0;
}
```

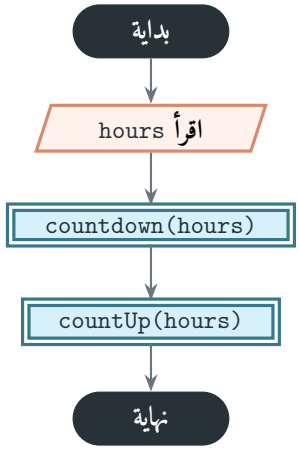
المدخلات

```
5
```

نتيجة التنفيذ

```
5 4 3 2 1 Sale over!
1 2 3 4 5
```

المخطط الإنسيابي (Organigramme)



تمرين 8.5 مجموع سلة المشتريات عوديًا

تجارة إلكترونية سهل

اكتب دالة عودية `basketTotal(prices, size)` تحسب مجموع أسعار السلة: مجموع الجدول = آخر عنصر + مجموع العناصر التي قبله.

خطوات الحل

1) الحالة الأساسية: جدول فارغ (`size == 0`) مجموعه 0.
2) العودية:

```
prices[size - 1] + basketTotal(prices, size - 1)
```

.

البرنامج بلغة C

```c
#include <stdio.h>

```

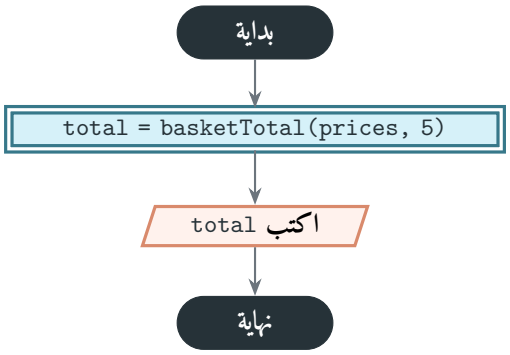
```c
double basketTotal(double prices[], int size) {
    if (size == 0)
        return 0;                                          // سلة فارغة
    return prices[size - 1] + basketTotal(prices, size - 1);
}

int main(void) {
    double prices[5] = {1200, 450, 3999, 89.5, 700};
    printf("Basket total: %.2f DA\n", basketTotal(prices, 5));
    printf("First 3 items: %.2f DA\n", basketTotal(prices, 3));
    return 0;
}
```

نتيجة التنفيذ

```
Basket total: 6438.50 DA
First 3 items: 5649.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 8.6 عرض رمز المنتج معكوسًا عوديًا

تجارة إلكترونية سهل

اكتب دالة عودية `printReversed(text, pos)` تعرض السلسلة معكوسة: نستدعي الدالة على الحرف التالي أولًا، ثم نطبع الحرف الحالي.

خطوات الحل

1) الحالة الأساسية: `text[pos] == '\0'` لا نطبع شيئًا.
2) الطباعة بعد الاستدعاء تعكس الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>

void printReversed(char text[], int pos) {
    if (text[pos] == '\0')
        return;                          // نهاية السلسلة
    printReversed(text, pos + 1);        // باقي السلسلة أولًا
    putchar(text[pos]);
}

int main(void) {
```

```
    char code[30];
    scanf("%29s", code);
    printReversed(code, 0);
    putchar('\n');
    return 0;
}
```

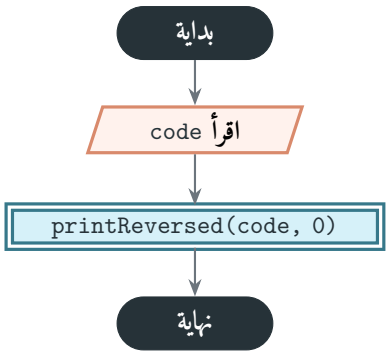
المدخلات

```
SKU-7781
```

نتيجة التنفيذ

```
1877-UKS
```

المخطط الانسيابي (Organigramme)



تمرين 8.7 أغلى منتج عوديًا

تجارة إلكترونية سهل

اكتب دالة عودية `maxPrice(prices, size)` تعيد أكبر سعر: الأكبر بين آخر عنصر وأكبر عناصر الجزء المتبقي.

خطوات الحل

1) الحالة الأساسية: `size == 1` نعيد `prices[0]`.
2) `rest = maxPrice(prices, size - 1)` ثم نقارنه مع `prices[size - 1]`.

البرنامج بلغة C

```c
#include <stdio.h>

double maxPrice(double prices[], int size) {
    double rest;
    if (size == 1)
        return prices[0];
    rest = maxPrice(prices, size - 1);                // أكبر الباقي
    return prices[size - 1] > rest ? prices[size - 1] : rest;
}

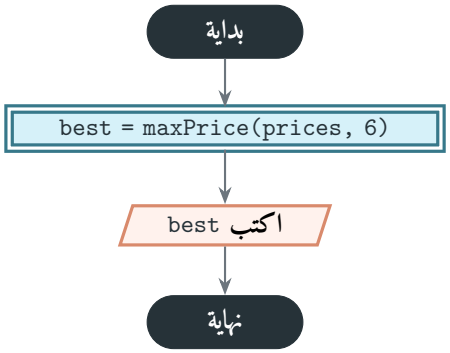
int main(void) {
    double prices[6] = {4500, 1200, 27000, 899, 12500, 650};
    printf("Most expensive: %.0f DA\n", maxPrice(prices, 6));
    return 0;
```

```
15 }
```

نتيجة التنفيذ

```
Most expensive: 27000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 8.8 متتالية فيبوناتشي لنمو المشتركين

علم البيانات سهل

نموذج مبسط لنمو المشتركين: عدد الشهر `month` يساوي مجموع عددي الشهرين السابقين، مع `fib(1) = fib(2) = 1`. اكتب الدالة العودية `fib(month)` واعرض أول 12 حدًا.

خطوات الحل

1) حالتان أساسيتان: الشهر 1 والشهر 2.
2) كل استدعاء يولد استدعاءين: الطريقة بطيئة للقيم الكبيرة (سنحسنها لاحقًا).

البرنامج بلغة C

```c
#include <stdio.h>

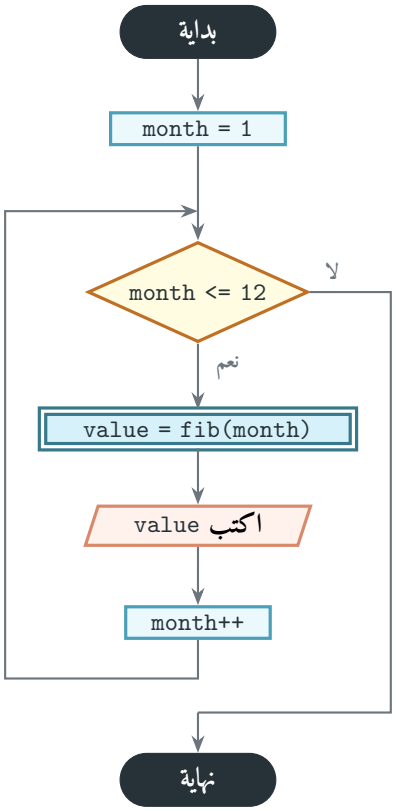
long fib(int month) {
    if (month <= 2)
        return 1;                          // الشهران الأولان
    return fib(month - 1) + fib(month - 2);  // استدعاءان عوديان
}

int main(void) {
    int month;
    for (month = 1; month <= 12; month++)
        printf("%ld ", fib(month));
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
1 1 2 3 5 8 13 21 34 55 89 144
```

المخطط الإنسيابي (Organigramme)



تمرين 8.9 القاسم المشترك الأكبر عوديًا

تجارة إلكترونية
سهل

اكتب صيغة عودية لخوارزمية إقليدس: `gcd(first, 0) = first` و
`gcd(first, second) = gcd(second, first % second)`
، واستعملها لتبسيط نسبة الطلبيات المرتجعة إلى المُسلّمة.

خطوات الحل

1) الحالة الأساسية: `second == 0`.
2) النسبة المبسطة: `returned/g` إلى `delivered/g`.

البرنامج بلغة C

```c
#include <stdio.h>

int gcd(int first, int second) {
    if (second == 0)
        return first;
    return gcd(second, first % second);
}

int main(void) {
    int returned, delivered, divisor;
    scanf("%d %d", &returned, &delivered);
    divisor = gcd(returned, delivered);
    printf("gcd = %d\n", divisor);
    printf("Return ratio: %d / %d\n", returned / divisor, delivered / divisor);
    return 0;
```

```
16 }
```

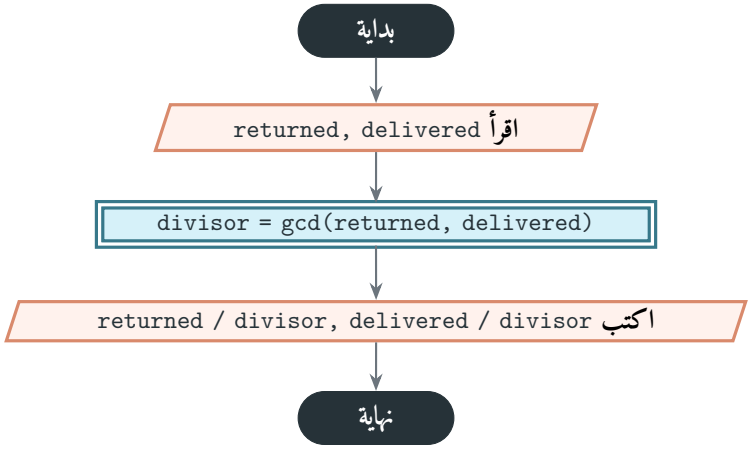
المدخلات

```
84 1260
```

نتيجة التنفيذ

```
gcd = 84
Return ratio: 1 / 15
```

المخطط الإنسيابي (Organigramme)



تمرين 8.10 عدّ تكرار تقييم معين عوديًا

علم البيانات سهل

اكتب دالة عودية `countValue(ratings, size, target)` تعد عدد المرات التي ظهر فيها تقييم معين (من 1 إلى 5) في جدول التقييمات.

خطوات الحل

1) جدول فارغ: 0.

2) `(ratings[size-1] == target) + countValue(ratings, size - 1, target)`

.

البرنامج بلغة C

```c
#include <stdio.h>

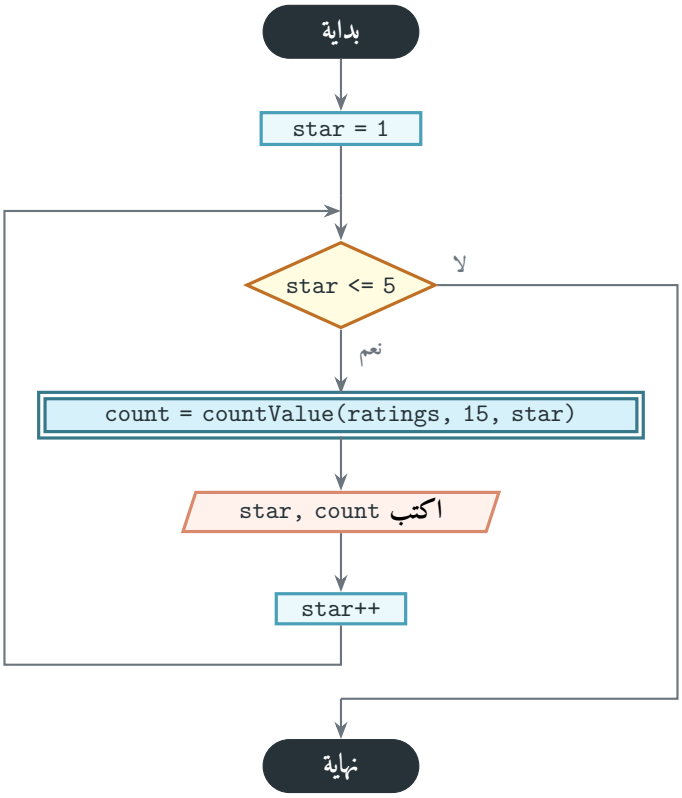
int countValue(int ratings[], int size, int target) {
    if (size == 0)
        return 0;
    return (ratings[size - 1] == target) + countValue(ratings, size - 1, target);
}

int main(void) {
    int ratings[15] = {5, 4, 5, 3, 5, 2, 4, 5, 1, 4, 5, 3, 4, 5, 4};
    int star;
    for (star = 1; star <= 5; star++)
        printf("%d stars: %d\n", star, countValue(ratings, 15, star));
    return 0;
}
```

نتيجة التنفيذ

```
1 stars: 1
2 stars: 1
3 stars: 2
4 stars: 5
5 stars: 6
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 8.11 القوة السريعة بالتربيع

علم البيانات ★★ متوسط

حساب `base^exponent` بالضرب المتكرر يتطلب `exponent` عملية. اكتب دالة عودية `fastPower` تستعمل: `base^n = (base^(n/2))²` إذا كان `n` زوجيًا، و `base × base^(n-1)` إذا كان فرديًا، وعدّ عدد عمليات الضرب.

خطوات الحل

1) نحسب `half = fastPower(base, n / 2)` **مرة واحدة ثم نعيد** `half * half`.
2) عدد الضربات من رتبة `log2(n)`.
3) نستعمل جدولًا من خانة واحدة لعد الضربات.

البرنامج بلغة C

```c
#include <stdio.h>

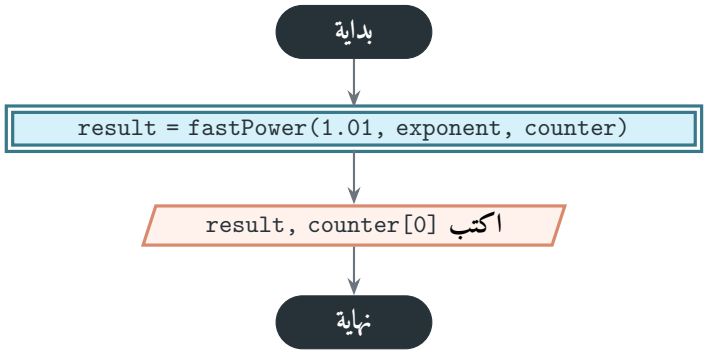
double fastPower(double base, int exponent, int counter[]) {
    double half;
    if (exponent == 0) return 1;
    if (exponent % 2 == 1) {
        counter[0]++;
        return base * fastPower(base, exponent - 1, counter);   // أس فردي
    }
    half = fastPower(base, exponent / 2, counter);              // نحسب النصف مرة واحدة
    counter[0]++;
    return half * half;
}

int main(void) {
    int exponents[4] = {10, 100, 365, 1000}, counter[1], idx;
    double result;
    for (idx = 0; idx < 4; idx++) {
        counter[0] = 0;
        result = fastPower(1.01, exponents[idx], counter);
        printf("1.01^%-4d = %14.4f with %2d multiplications (naive: %d)\n",
               exponents[idx], result, counter[0], exponents[idx]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
1.01^10   =         1.1046 with  5 multiplications (naive: 10)
1.01^100  =         2.7048 with  9 multiplications (naive: 100)
1.01^365  =        37.7834 with 14 multiplications (naive: 365)
1.01^1000 =     20959.1556 with 15 multiplications (naive: 1000)
```

المخطط الإنسيابي (Organigramme)



تمرين 8.12 البحث الثنائي العودي في الأسعار

تجارة إلكترونية ★★ متوسط

اكتب دالة عودية search(prices, low, high, target) تبحث عن سعر في جدول مرتب: نقارن بالعنصر الأوسط ثم نبحث عودياً في النصف المناسب فقط. اعرض المجالات التي فُحصت في كل مستوى.

خطوات الحل

1) الحالة الأساسية 1: low > high غير موجود (-1).
2) الحالة الأساسية 2: prices[middle] == target.
3) وإلا نستدعي على [low, middle-1] أو [middle+1, high].

البرنامج بلغة C

```c
#include <stdio.h>

int search(double prices[], int low, int high, double target) {
    int middle;
    if (low > high) return -1;                  // مجال فارغ
    middle = (low + high) / 2;
    printf("  [%d..%d] middle %d\n", low, high, middle);
    if (prices[middle] == target) return middle;
    if (prices[middle] < target)
        return search(prices, middle + 1, high, target);
    return search(prices, low, middle - 1, target);
}

int main(void) {
    double prices[12] = {99, 450, 560, 650, 899, 1200, 1500, 2300, 3200, 4100, 7800, 12500};
    double target;
    int query, idx;
    for (query = 0; query < 2; query++) {
        scanf("%lf", &target);
        printf("Search %.0f:\n", target);
        idx = search(prices, 0, 11, target);
        if (idx >= 0) printf("  found at index %d\n", idx);
        else printf("  not found\n");
    }
    return 0;
}
```

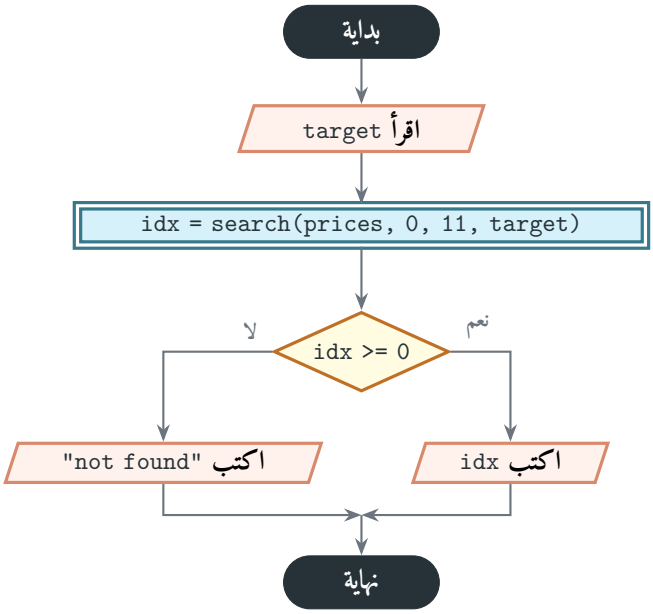
المدخلات

```
3200 700
```

نتيجة التنفيذ

```
Search 3200:
  [0..11] middle 5
  [6..11] middle 8
  found at index 8
Search 700:
  [0..11] middle 5
  [0..4] middle 2
  [3..4] middle 3
  [4..4] middle 4
  not found
```

المخطط الإنسيابي (Organigramme)



تمرين 8.13 الرمز المتناظر عوديًا

تجارة إلكترونية ★★ متوسط

اكتب دالة عودية `isPalindrome(text, left, right)` تتحقق من أن رمزًا متناظر: الحرفان الطرفيان متساويان و الجزء الداخلي متناظر. اختبرها على عدة رموز.

خطوات الحل

1) الحالة الأساسية: `left >= right` (حرف واحد أو لا شيء) متناظر.
2) إذا `text[left] != text[right]` غير متناظر.
3) وإلا نستدعي على `left + 1` و `right - 1`.

البرنامج بلغة C

```c
#include <stdio.h>

int isPalindrome(char text[], int left, int right) {
```

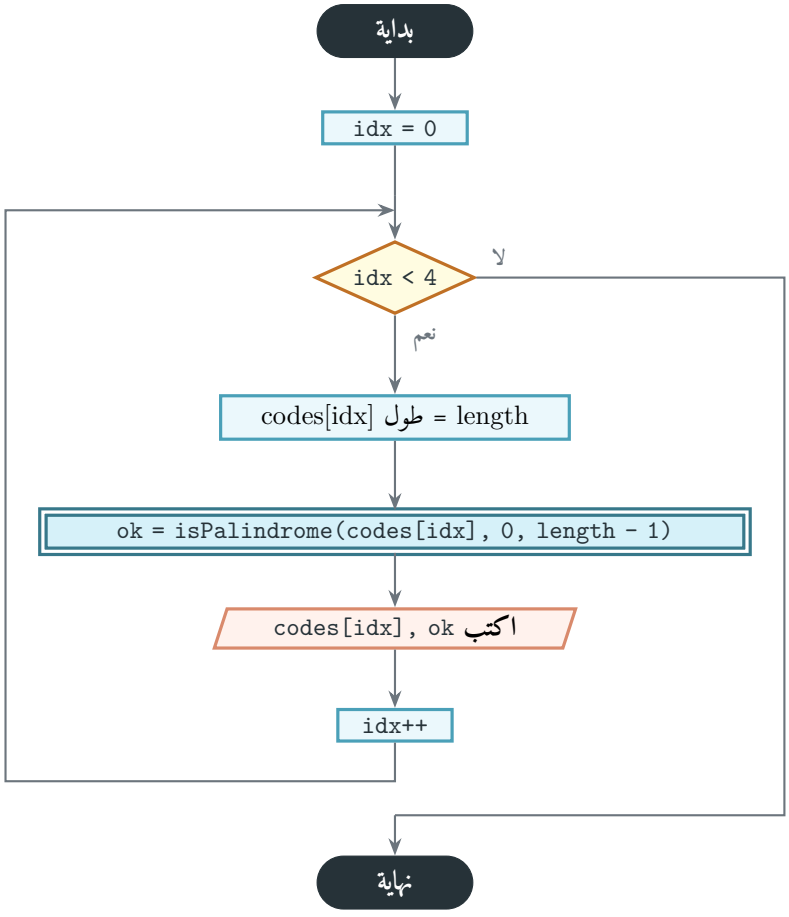
```c
    if (left >= right) return 1;                    // انتهت المقارنات
    if (text[left] != text[right]) return 0;
    return isPalindrome(text, left + 1, right - 1);  // الجزء الداخلي
}

int main(void) {
    char codes[4][20] = {"AB12321BA", "XY-77-YX", "RADAR", "SKU2026"};
    int idx, length;
    for (idx = 0; idx < 4; idx++) {
        length = 0;
        while (codes[idx][length] != '\0') length++;
        printf("%-10s %s\n", codes[idx],
               isPalindrome(codes[idx], 0, length - 1) ? "palindrome" : "no");
    }
    return 0;
}
```

نتيجة التنفيذ

```
AB12321BA  palindrome
XY-77-YX   palindrome
RADAR      palindrome
SKU2026    no
```

المخطط الإنسيابي (Organigramme)



تمرين 8.14 التحويل إلى الثنائي عوديًا

علم البيانات ★★ متوسط

اكتب دالة عودية `printBinary(number)` تعرض عددًا بالنظام الثنائي: نعرض أولًا التمثيل الثنائي لـ `number / 2`، ثم الرقم `number % 2`. اعرض معرّفات بعض الحساسات بالثنائي.

خطوات الحل

1) الحالة الأساسية: `number < 2` نطبع الرقم مباشرة.
2) الاستدعاء قبل الطباعة يجعل الأرقام تظهر من الأعلى إلى الأدنى.

البرنامج بلغة C

```c
#include <stdio.h>

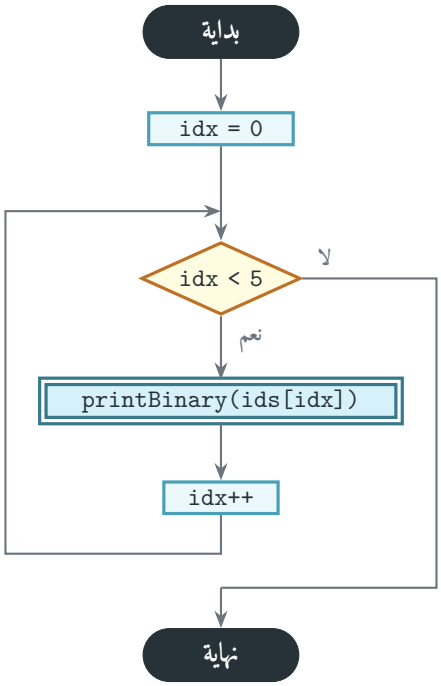
void printBinary(int number) {
    if (number < 2) {
        printf("%d", number);
        return;
    }
    printBinary(number / 2);          // الأرقام الأعلى أولًا
    printf("%d", number % 2);
}

int main(void) {
    int ids[5] = {5, 13, 64, 255, 2026}, idx;
    for (idx = 0; idx < 5; idx++) {
        printf("%4d = ", ids[idx]);
        printBinary(ids[idx]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
   5 = 101
  13 = 1101
  64 = 1000000
 255 = 11111111
2026 = 11111101010
```

المخطط الإنسيابي (Organigramme)



تمرين 8.15 لون المنتج بالست عشري عوديًا

تجارة إلكترونية ★★ متوسط

يُخزَّن لون المنتج كعدد صحيح ويُعرض في الموقع بصيغة #RRGGBB. اكتب دالة عودية `printHex(number, digits)` تعرض العدد بالأساس 16 على `digits` خانة بالضبط (مع أصفار على اليسار).

خطوات الحل

1) نطبع أولًا `printHex(number / 16, digits - 1)` ثم رقم `number % 16`.
2) الرقم الست عشري: `"0123456789ABCDEF"[value]`.
3) الحالة الأساسية `digits == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

void printHex(long number, int digits) {
    char symbols[] = "0123456789ABCDEF";
    if (digits == 0) return;
    printHex(number / 16, digits - 1);        // الخانات الأعلى
    putchar(symbols[number % 16]);
}

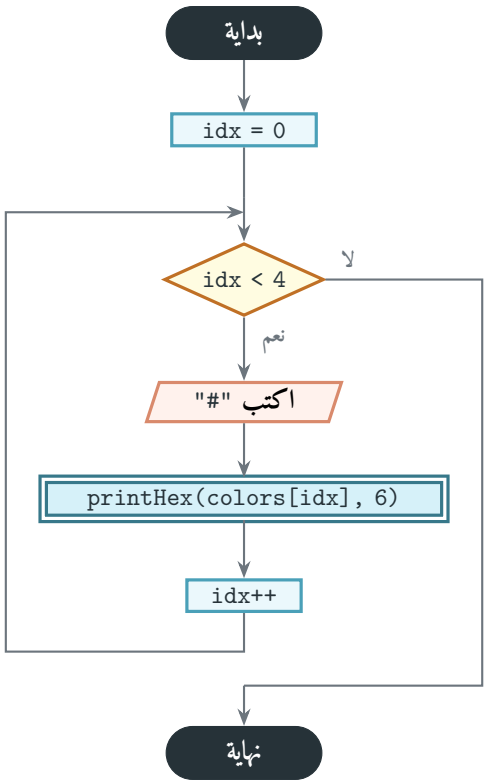
int main(void) {
    long colors[4] = {16711680, 65280, 3368601, 16753920};
    char names[4][8] = {"red", "green", "teal", "orange"};
    int idx;
    for (idx = 0; idx < 4; idx++) {
        printf("%-7s %8ld -> #", names[idx], colors[idx]);
```

```
        printHex(colors[idx], 6);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
red      16711680 -> #FF0000
green       65280 -> #00FF00
teal      3368601 -> #336699
orange   16753920 -> #FFA500
```

المخطط الإنسيابي (Organigramme)



تمرين 8.16 فواصل الآلاف في المبالغ عوديًا

تجارة إلكترونية ★★ متوسط

اكتب دالة عودية `printAmount(amount)` تعرض مبلغًا بفواصل الآلاف (1,250,000): نعرض عوديًا `amount / 1000` ثم فاصلة ثم آخر ثلاثة أرقام بالصيغة `%03ld`.

خطوات الحل

1) الحالة الأساسية: `amount < 1000` نطبعه دون أصفار.
2) الحالة العودية: `printAmount(amount / 1000)` ثم `%03ld,`.

البرنامج بلغة C

```c
#include <stdio.h>

void printAmount(long amount) {
    if (amount < 1000) {
        printf("%ld", amount);            // الجزء الأول دون أصفار
        return;
    }
    printAmount(amount / 1000);
    printf(",%03ld", amount % 1000);      // مجموعات من ثلاثة أرقام
}

int main(void) {
    long amount;
    int idx;
    for (idx = 0; idx < 4; idx++) {
        scanf("%ld", &amount);
        printAmount(amount);
        printf(" DA\n");
    }
    return 0;
}
```

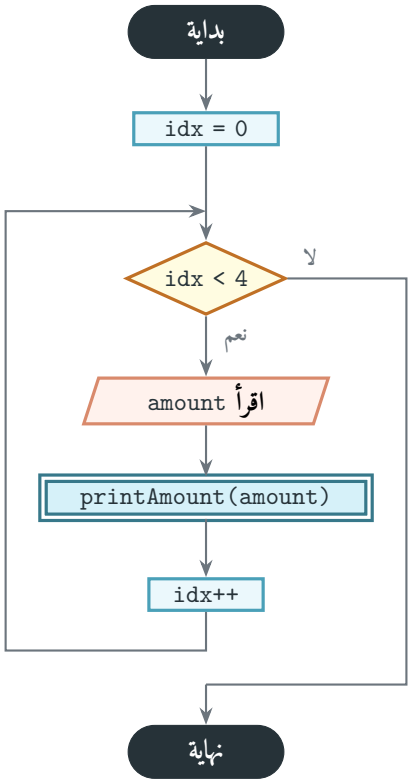
المدخلات

```
1250000 987 45000 2026000500
```

نتيجة التنفيذ

```
1,250,000 DA
987 DA
45,000 DA
2,026,000,500 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 8.17 الجذر الرقمي لرمز التحقق

تجارة إلكترونية متوسط

الجذر الرقمي لعدد هو نتيجة جمع أرقامه بشكل متكرر حتى نحصل على رقم واحد. اكتب دالتين عوديتين: `digitSum` و `digitalRoot` (تستدعي نفسها على `digitSum` ما دامت النتيجة ⩾ 10)، واعرض المراحل.

خطوات الحل

1) `digitalRoot(number)`: إذا `number < 10` نعيده، وإلا `digitalRoot(digitSum(number))`.

2) عودية متداخلة: دالة عودية تستدعي دالة عودية أخرى.

البرنامج بلغة C

```c
#include <stdio.h>

long digitSum(long number) {
    if (number < 10) return number;
    return number % 10 + digitSum(number / 10);
}

long digitalRoot(long number) {
    if (number < 10) return number;
    printf("%ld -> ", number);
    return digitalRoot(digitSum(number));      // نكرر على مجموع الأرقام
}

int main(void) {
    long number, root;
```

```c
    scanf("%ld", &number);
    root = digitalRoot(number);
    printf("%ld\n", root);
    printf("Check: 1 + (%ld - 1) %% 9 = %ld\n", number, 1 + (number - 1) % 9);
    return 0;
}
```

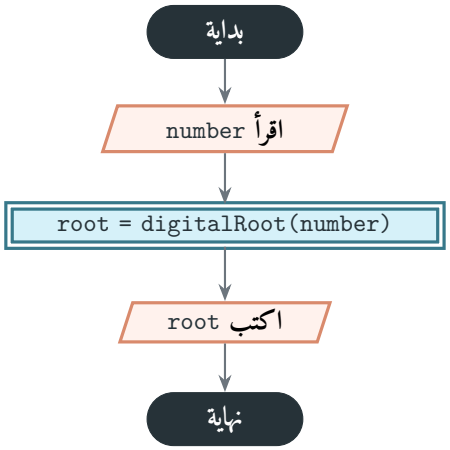
المدخلات

```
987654321
```

نتيجة التنفيذ

```
987654321 -> 45 -> 9
Check: 1 + (987654321 - 1) % 9 = 9
```

المخطط الإنسيابي (Organigramme)



تمرين 8.18 طول متتالية كولاتز

علم البيانات ★★ متوسط

متتالية كولاتز: إذا كان العدد زوجيًا نقسمه على 2، وإلا نضربه في 3 ونضيف 1، حتى نصل إلى 1. اكتب دالة عودية `collatzLength(number)` تعيد عدد الخطوات، وجد العدد بين 1 و 1000 ذا أطول متتالية.

خطوات الحل

1) الحالة الأساسية: `number == 1` ← 0 خطوة.

2) `1 + collatzLength(number / 2)` أو `1 + collatzLength(3 * number + 1)`.

البرنامج بلغة C

```c
#include <stdio.h>

int collatzLength(long number) {
    if (number == 1) return 0;
    if (number % 2 == 0) return 1 + collatzLength(number / 2);
    return 1 + collatzLength(3 * number + 1);
}

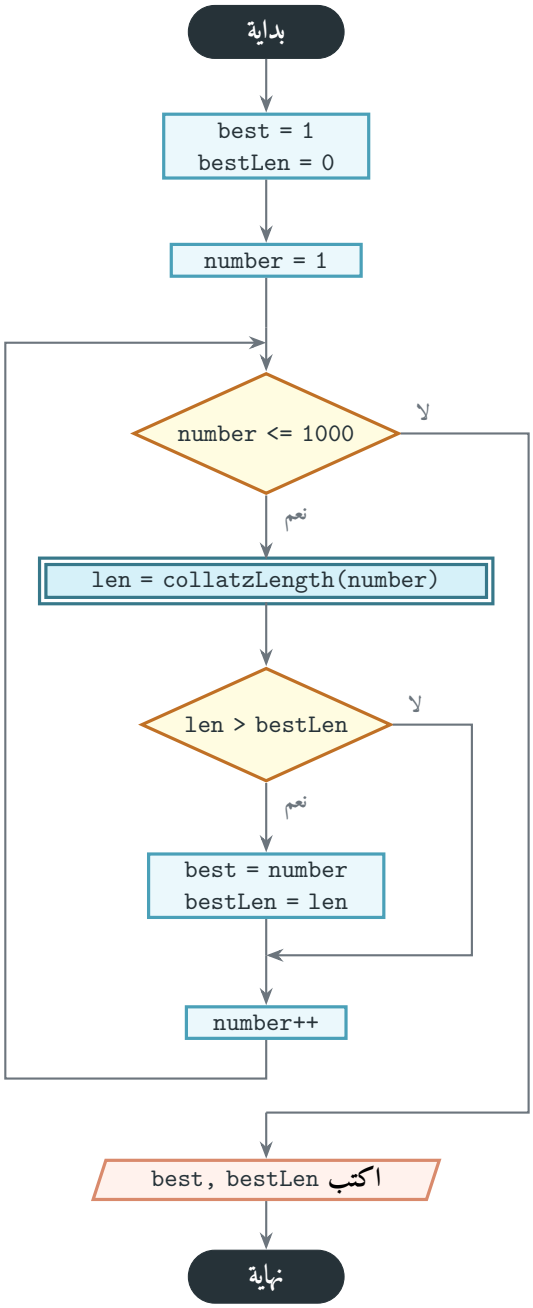
int main(void) {
```

```c
    int number, len, best = 1, bestLen = 0;
    printf("27 needs %d steps\n", collatzLength(27));
    for (number = 1; number <= 1000; number++) {
        len = collatzLength(number);
        if (len > bestLen) {
            bestLen = len;
            best = number;
        }
    }
    printf("Longest under 1000: %d with %d steps\n", best, bestLen);
    return 0;
}
```

نتيجة التنفيذ

```
27 needs 111 steps
Longest under 1000: 871 with 178 steps
```

المخطط الإنسيابي (Organigramme)



تمرين 8.19 أبراج هانوي لنقل الصناديق

تجارة إلكترونية ★★ متوسط

في المخزن ثلاثة أماكن A و B و C، وكومة من count صناديق مختلفة الحجم في A. ننقل صندوقًا واحدًا في كل مرة، ولا يوضع صندوق كبير فوق صغير. اكتب الدالة العودية hanoi(count, from, to, via) لنقل الكومة إلى C.

خطوات الحل

1) ننقل count - 1 صندوقًا من from إلى via.
2) ننقل الصندوق الأكبر من from إلى to.
3) ننقل count - 1 صندوقًا من via إلى to.

4) العدد الكلي للنقلات `2^count - 1`.

البرنامج بلغة C

```c
#include <stdio.h>

int moves = 0;

void hanoi(int count, char from, char to, char via) {
    if (count == 0) return;
    hanoi(count - 1, from, via, to);           // نفرغ الطريق
    moves++;
    printf("Move %d: box %d from %c to %c\n", moves, count, from, to);
    hanoi(count - 1, via, to, from);           // نعيد الكومة فوقه
}

int main(void) {
    int count;
    scanf("%d", &count);
    hanoi(count, 'A', 'C', 'B');
    printf("Total moves: %d\n", moves);
    return 0;
}
```

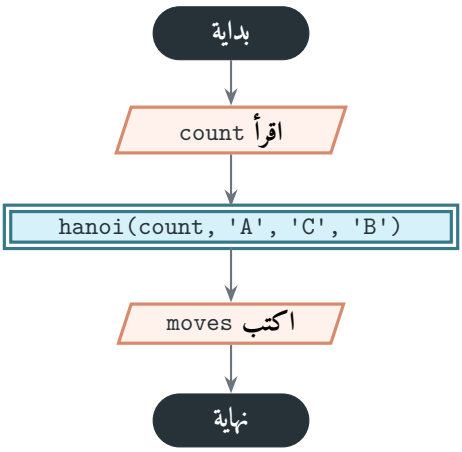
المدخلات

```
3
```

نتيجة التنفيذ

```
Move 1: box 1 from A to C
Move 2: box 2 from A to B
Move 3: box 1 from C to B
Move 4: box 3 from A to C
Move 5: box 1 from B to A
Move 6: box 2 from B to C
Move 7: box 1 from A to C
Total moves: 7
```

المخطط الإنسيابي (Organigramme)



تمرين 8.20 فيبوناتشي بالتذكر وعدد الإستدعاءات

علم البيانات ★★ متوسط

الدالة العودية الساذجة `fib` تعيد حساب نفس القيم آلاف المرات. اكتب نسخة بالتذكر (memoization) تحفظ كل نتيجة في جدول، وقارن عدد الاستدعاءات بين النسختين لـ `fib(30)`.

خطوات الحل

1) جدول `memo[]` مهيأ بـ 0 (= لم يُحسب بعد).
2) قبل الحساب: إذا `memo[month] != 0` نعيدها مباشرة.
3) عداد عام لكل نسخة.

البرنامج بلغة C

```c
#include <stdio.h>

long slowCalls = 0, fastCalls = 0;
long memo[100];

long fibSlow(int month) {
    slowCalls++;
    if (month <= 2) return 1;
    return fibSlow(month - 1) + fibSlow(month - 2);
}

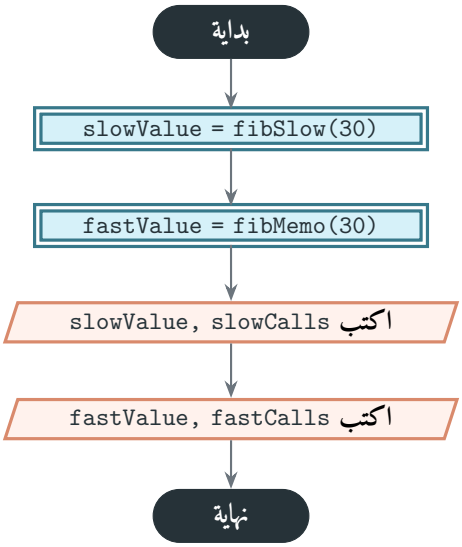
long fibMemo(int month) {
    fastCalls++;
    if (month <= 2) return 1;
    if (memo[month] != 0) return memo[month];          // محسوبة سابقًا
    memo[month] = fibMemo(month - 1) + fibMemo(month - 2);
    return memo[month];
}

int main(void) {
    long slowValue = fibSlow(30), fastValue = fibMemo(30);
    printf("Naive:    fib(30) = %ld with %ld calls\n", slowValue, slowCalls);
    printf("Memoized: fib(30) = %ld with %ld calls\n", fastValue, fastCalls);
    fastCalls = 0;
    printf("fib(90) = %ld\n", fibMemo(90));
    printf("calls for fib(90) after cache: %ld\n", fastCalls);
    return 0;
}
```

نتيجة التنفيذ

```
Naive:    fib(30) = 832040 with 1664079 calls
Memoized: fib(30) = 832040 with 57 calls
fib(90) = 2880067194370816120
calls for fib(90) after cache: 121
```

المخطط الإنسيابي (Organigramme)



تمرين 8.21 عدد الحزم الممكنة بمثلث باسكال

تجارة إلكترونية ★★ متوسط

بكم طريقة نختار `pick` منتجات من `total` منتج لتكوين حزمة؟ اكتب الدالة العودية

```
choose(total, pick) = choose(total-1, pick-1) + choose(total-1, pick)
```

مع التذكر في جدول ثنائي البعد، واعرض الأسطر الأولى من **مثلث باسكال**.

خطوات الحل

1) الحالتان الأساسيتان: `pick == 0` أو `pick == total` ⟹ 1.
2) إما أن المنتج الأخير في الحزمة (`pick - 1` من الباقي)، أو لا (`pick` من الباقي).
3) `memo[total][pick]` يمنع تكرار الحساب.

البرنامج بلغة C

```c
#include <stdio.h>

long memo[60][60];

long choose(int total, int pick) {
    if (pick == 0 || pick == total) return 1;
    if (memo[total][pick] != 0) return memo[total][pick];
    memo[total][pick] = choose(total - 1, pick - 1) + choose(total - 1, pick);
    return memo[total][pick];
}

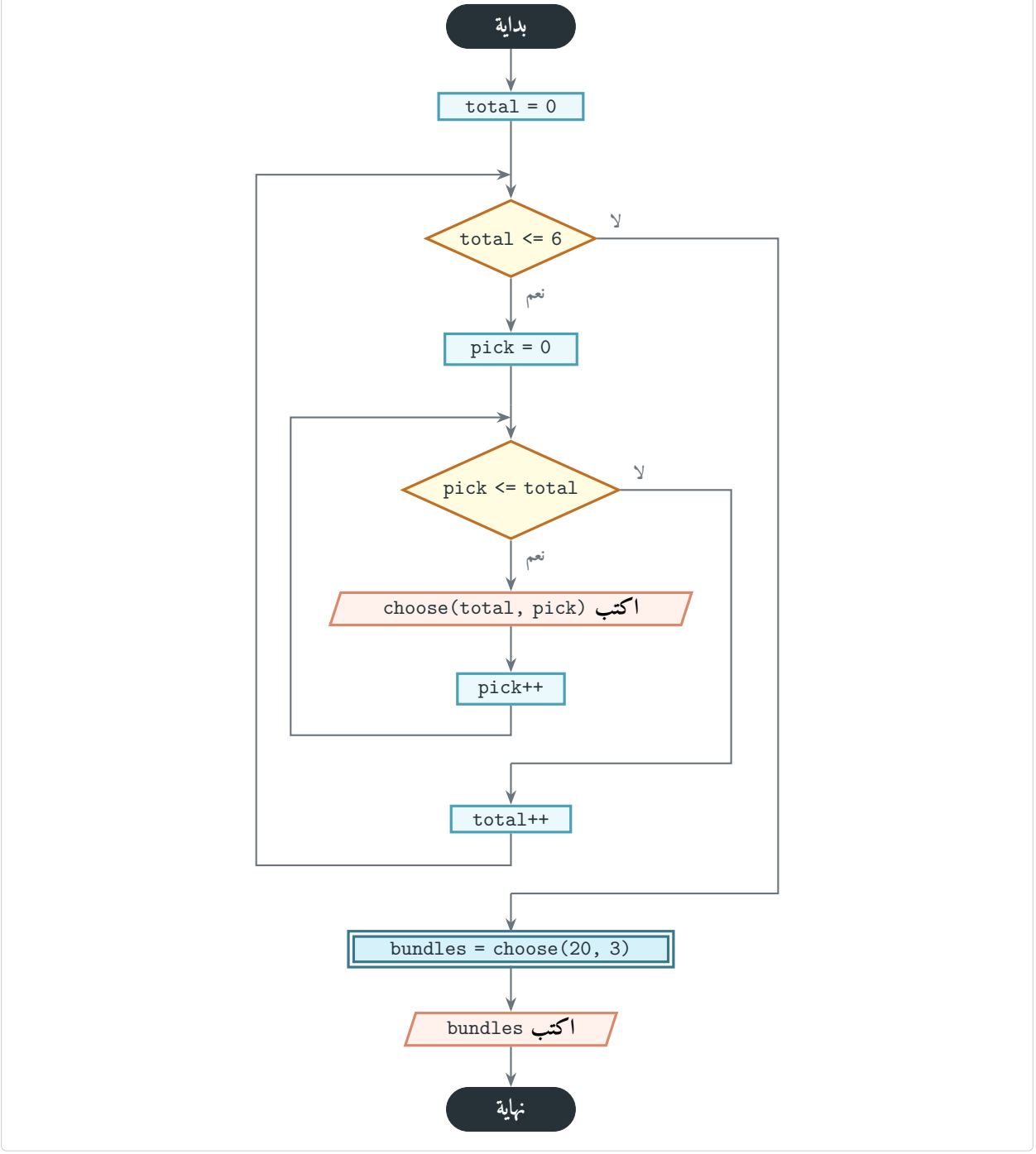
int main(void) {
    int total, pick;
    for (total = 0; total <= 6; total++) {
        printf("%*s", 2 * (6 - total), "");          // إزاحة لشكل المثلث
        for (pick = 0; pick <= total; pick++)
            printf("%4ld", choose(total, pick));
        printf("\n");
    }
    printf("Bundles of 3 among 20 products: %ld\n", choose(20, 3));
```

```
    printf("Bundles of 10 among 50 products: %ld\n", choose(50, 10));
    return 0;
}
```

نتيجة التنفيذ

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5  10  10   5   1
    1   6  15  20  15   6   1
Bundles of 3 among 20 products: 1140
Bundles of 10 among 50 products: 10272278170
```

المخطط الإنسيابي (Organigramme)



تمرين 8.22 كل الحزم الممكنة من أربعة منتجات

تجارة إلكترونية ★★ متوسط

اكتب دالة عودية `subsets(index, chosen)` تولّد **كل المجموعات الجزئية** لأربعة منتجات (16 حزمة)، وتعرض لكل حزمة محتواها وسعرها الإجمالي، وعدد الحزم التي لا يتجاوز سعرها 6000 دج.

خطوات الحل

1) لكل منتج قراران: نأخذه (`chosen[index] = 1`) أو لا نأخذه، ثم ننتقل إلى المنتج التالي.
2) الحالة الأساسية: `index == NB` ⟹ حزمة كاملة نعرضها.

3) عدد الحزم 2^NB.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

char names[NB][10] = {"Mouse", "Keyboard", "Headset", "Webcam"};
double prices[NB] = {1500, 3200, 2800, 4100};
int chosen[NB], affordable = 0;

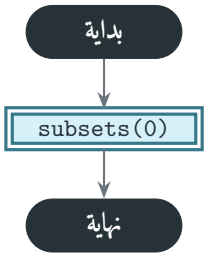
void subsets(int index) {
    int idx;
    double total = 0;
    if (index == NB) {                          // حزمة كاملة
        printf("{");
        for (idx = 0; idx < NB; idx++)
            if (chosen[idx]) { printf(" %s", names[idx]); total += prices[idx]; }
        printf(" } = %.0f\n", total);
        if (total <= 6000) affordable++;
        return;
    }
    chosen[index] = 0;                          // دون المنتج
    subsets(index + 1);
    chosen[index] = 1;                          // مع المنتج
    subsets(index + 1);
}

int main(void) {
    subsets(0);
    printf("Bundles up to 6000 DA: %d\n", affordable);
    return 0;
}
```

نتيجة التنفيذ

```
{ } = 0
{ Webcam } = 4100
{ Headset } = 2800
{ Headset Webcam } = 6900
{ Keyboard } = 3200
{ Keyboard Webcam } = 7300
{ Keyboard Headset } = 6000
{ Keyboard Headset Webcam } = 10100
{ Mouse } = 1500
{ Mouse Webcam } = 5600
{ Mouse Headset } = 4300
{ Mouse Headset Webcam } = 8400
{ Mouse Keyboard } = 4700
{ Mouse Keyboard Webcam } = 8800
{ Mouse Keyboard Headset } = 7500
{ Mouse Keyboard Headset Webcam } = 11600
Bundles up to 6000 DA: 9
```

المخطط الإنسيابي (Organigramme)



تمرين 8.23 أقصر جولة توصيل بالتباديل

تجارة إلكترونية ★★ متوسط

يبدأ السائق من المستودع (0) ويزور 4 زبائن ثم يعود. اكتب دالة عودية `permute(depth)` تولد **كل الترتيبات** الممكنة للزبائن (24 ترتيباً)، وتحسب طول كل جولة من مصفوفة المسافات، وتحتفظ بأقصرها.

خطوات الحل

1) `order[]` يحتوي الزبائن 1..4؛ عند المستوى `depth` نبدل `order[depth]` مع كل عنصر بعده، نستدعي `permute(depth + 1)`، ثم نعيد التبديل (**التراجع**).

2) الحالة الأساسية `depth == NB`: نحسب الطول `0 → order[0] → ... → order[3] → 0`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

int dist[NB + 1][NB + 1] = {{0, 12, 18, 9, 22},
                            {12, 0, 7, 14, 11},
                            {18, 7, 0, 10, 6},
                            {9, 14, 10, 0, 15},
                            {22, 11, 6, 15, 0}};
int order[NB] = {1, 2, 3, 4}, bestOrder[NB], bestLength = 100000, tours = 0;

void permute(int depth) {
    int idx, tmp, length;
    if (depth == NB) {                                  // ترتيب كامل
        length = dist[0][order[0]] + dist[order[NB - 1]][0];
        for (idx = 0; idx < NB - 1; idx++) length += dist[order[idx]][order[idx + 1]];
        tours++;
        if (length < bestLength) {
            bestLength = length;
            for (idx = 0; idx < NB; idx++) bestOrder[idx] = order[idx];
        }
        return;
    }
    for (idx = depth; idx < NB; idx++) {
        tmp = order[depth]; order[depth] = order[idx]; order[idx] = tmp;
        permute(depth + 1);
        tmp = order[depth]; order[depth] = order[idx]; order[idx] = tmp;   // تراجع
    }
}

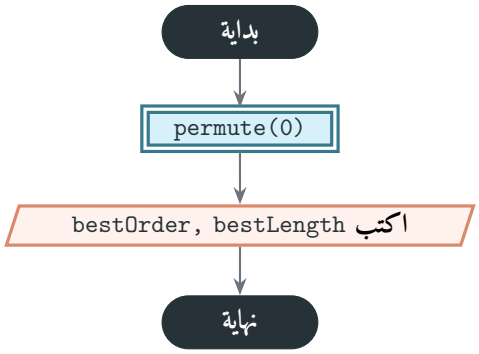
int main(void) {
    int idx;
    permute(0);
```

```
    printf("%d tours tested\nBest: 0", tours);
    for (idx = 0; idx < NB; idx++) printf(" -> %d", bestOrder[idx]);
    printf(" -> 0 = %d km\n", bestLength);
    return 0;
}
```

نتيجة التنفيذ

```
24 tours tested
Best: 0 -> 1 -> 4 -> 2 -> 3 -> 0 = 48 km
```

المخطط الإنسيابي (Organigramme)



تمرين 8.24 عدد طرق دفع مبلغ بالقطع النقدية

تجارة إلكترونية ★★ متوسط

بكم طريقة يمكن دفع مبلغ باستعمال قطع 5 و 10 و 20 و 50 و 100 دج (الترتيب لا يهم)؟ اكتب دالة عودية `ways(amount, coinIndex)`: إما نستعمل القطعة الحالية مرة أخرى، أو ننتقل إلى القطعة التالية. أضف التذكر لتسريعها.

خطوات الحل

1) `ways(0, ...) = 1` (دفعنا المبلغ بالضبط)، و `ways( , ...) = 0`، و `ways(..., NB) = 0`.

2) `ways(amount, idx) = ways(amount - coins[idx], idx) + ways(amount, idx + 1)`

.

3) `memo[amount][idx]` يحفظ النتائج (-1 = غير محسوبة).

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 5

int coins[NB] = {5, 10, 20, 50, 100};
long memo[1001][NB];

long ways(int amount, int coinIndex) {
    if (amount == 0) return 1;                          // مبلغ مدفوع بالضبط
    if (amount < 0 || coinIndex == NB) return 0;
    if (memo[amount][coinIndex] >= 0) return memo[amount][coinIndex];
    memo[amount][coinIndex] = ways(amount - coins[coinIndex], coinIndex)   // نستعمل القطعة
                            + ways(amount, coinIndex + 1);                 // ننتقل إلى التالية
    return memo[amount][coinIndex];
}

int main(void) {
    int amount, query, row, col;
```

```
    for (row = 0; row <= 1000; row++)
        for (col = 0; col < NB; col++) memo[row][col] = -1;
    for (query = 0; query < 2; query++) {
        scanf("%d", &amount);
        printf("%d DA can be paid in %ld ways\n", amount, ways(amount, 0));
    }
    return 0;
}
```

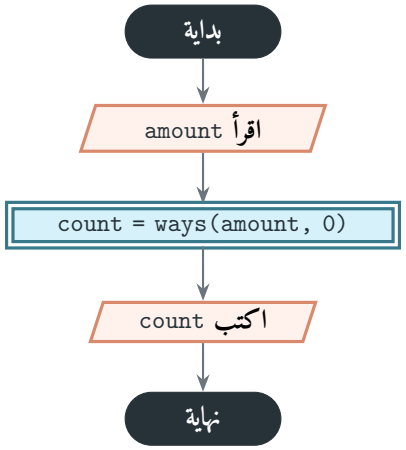
المدخلات

```
100 500
```

نتيجة التنفيذ

```
100 DA can be paid in 50 ways
500 DA can be paid in 5126 ways
```

المخطط الإنسيابي (Organigramme)



تمرين 8.25 أقل عدد من القطع لإرجاع الباقي

تجارة إلكترونية
متوسط

آلة الدفع الذاتي تملك قطع 1 و 7 و 10 و 25 دج. الطريقة الجشعة (أكبر قطعة أولًا) ليست دائمًا الأفضل. اكتب دالة عودية بالتذكر `minCoins(amount)` تعيد أقل عدد من القطع، وقارنها بالطريقة الجشعة.

خطوات الحل

1) `minCoins(0) = 0`؛ وإلا `1 + min(minCoins(amount - coin))` على كل قطعة لا تتجاوز المبلغ.
2) `memo[amount]` يحفظ النتيجة.
3) الجشعة: نأخذ أكبر قطعة ممكنة في كل مرة.

البرنامج بلغة C

```
#include <stdio.h>

#define NB 4

int coins[NB] = {25, 10, 7, 1};
int memo[200];

int minCoins(int amount) {
    int idx, candidate, best = 100000;
```

```c
    if (amount == 0) return 0;
    if (memo[amount] > 0) return memo[amount];
    for (idx = 0; idx < NB; idx++)
        if (coins[idx] <= amount) {
            candidate = 1 + minCoins(amount - coins[idx]);   // نجرب كل قطعة
            if (candidate < best) best = candidate;
        }
    memo[amount] = best;
    return best;
}

int greedy(int amount) {
    int idx, count = 0;
    for (idx = 0; idx < NB; idx++)
        while (amount >= coins[idx]) { amount -= coins[idx]; count++; }
    return count;
}

int main(void) {
    int amount, query;
    for (query = 0; query < 3; query++) {
        scanf("%d", &amount);
        printf("%3d DA: optimal %d coins, greedy %d coins\n", amount, minCoins(amount), greedy(amount));
    }
    return 0;
}
```

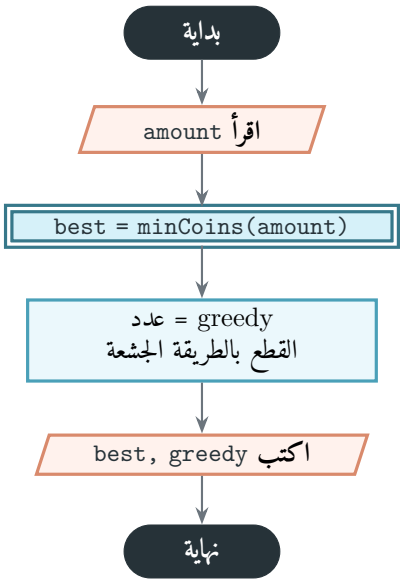
المدخلات

```
14 31 63
```

نتيجة التنفيذ

```
 14 DA: optimal 2 coins, greedy 5 coins
 31 DA: optimal 4 coins, greedy 7 coins
 63 DA: optimal 6 coins, greedy 6 coins
```

المخطط الإنسيابي (Organigramme)



تمرين 8.26 عدد مسارات التوصيل في شبكة الشوارع

تجارة إلكترونية ★★ متوسط

المدينة شبكة 6×6 من التقاطعات. ينطلق الدراج من الزاوية العليا اليسرى نحو السفلى اليمنى متحركًا **يمينًا أو نزولًا** فقط، وبعض التقاطعات مغلقة بالأشغال. اكتب دالة عودية بالتذكر `paths(row, col)` تعد المسارات الممكنة.

خطوات الحل

1) `paths(row, col) = paths(row + 1, col) + paths(row, col + 1)`

.

2) خارج الشبكة أو تقاطع مغلق: 0؛ نقطة الوصول: 1.

3) `memo[row][col]` (-1 = غير محسوب).

البرنامج بلغة C

```c
#include <stdio.h>

#define SIZE 6

int closed[SIZE][SIZE] = {{0, 0, 0, 0, 0, 0},
                          {0, 1, 0, 0, 1, 0},
                          {0, 0, 0, 0, 0, 0},
                          {1, 0, 0, 1, 0, 0},
                          {0, 0, 0, 0, 0, 1},
                          {0, 0, 1, 0, 0, 0}};
long memo[SIZE][SIZE];

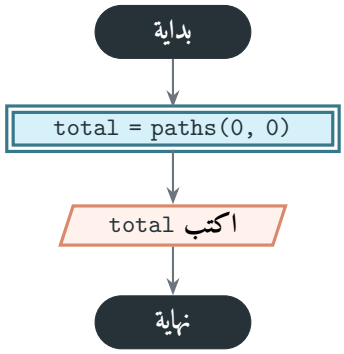
long paths(int row, int col) {
    if (row >= SIZE || col >= SIZE || closed[row][col]) return 0;
    if (row == SIZE - 1 && col == SIZE - 1) return 1;     // الوصول
    if (memo[row][col] >= 0) return memo[row][col];
    memo[row][col] = paths(row + 1, col) + paths(row, col + 1);
    return memo[row][col];
}

int main(void) {
    int row, col;
    for (row = 0; row < SIZE; row++)
        for (col = 0; col < SIZE; col++) memo[row][col] = -1;
    printf("Paths with roadworks: %ld\n", paths(0, 0));
    for (row = 0; row < SIZE; row++) {
        for (col = 0; col < SIZE; col++)
            printf("%4ld", closed[row][col] ? 0 : paths(row, col));
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Paths with roadworks: 12
  12   5   5   1   0   0
   7   0   4   1   0   0
   7   7   3   1   1   0
   0   4   2   0   1   0
   2   2   2   2   1   0
   0   0   0   1   1   1
```

المخطط الإنسيابي (Organigramme)



تمرين 8.27 أصغر وأكبر سعر بالتقسيم

علم البيانات ★★ متوسط

اكتب دالة عودية بطريقة **فرّق تسد** `minMax(prices, low, high, result)` تقسم الجدول إلى نصفين، تجد الأصغر والأكبر في كل نصف، ثم تجمع النتيجتين. عدّ المقارنات وقارنها بـ `2(n-1)` للطريقة العادية.

خطوات الحل

1) عنصر واحد: هو الأصغر والأكبر (0 مقارنة)، عنصران: مقارنة واحدة.
2) وإلا: `minMax` للنصف الأيسر والأيمن، ثم مقارنتان للدمج.
3) `result[0]` الأصغر و `result[1]` الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>

int comparisons = 0;

void minMax(double prices[], int low, int high, double result[]) {
    double left[2], right[2];
    int middle;
    if (low == high) {
        result[0] = result[1] = prices[low];
        return;
    }
    if (high == low + 1) {
        comparisons++;
        if (prices[low] < prices[high]) { result[0] = prices[low]; result[1] = prices[high]; }
        else { result[0] = prices[high]; result[1] = prices[low]; }
        return;
    }
    middle = (low + high) / 2;
    minMax(prices, low, middle, left);          // النصف الأيسر
    minMax(prices, middle + 1, high, right);    // النصف الأيمن
    comparisons += 2;
    result[0] = left[0] < right[0] ? left[0] : right[0];
    result[1] = left[1] > right[1] ? left[1] : right[1];
}

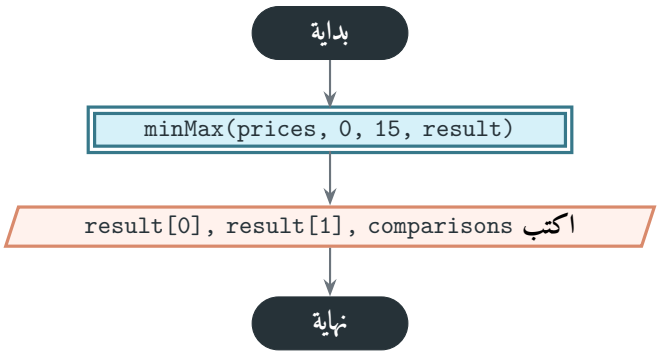
int main(void) {
    double prices[16] = {4500, 1200, 899, 27000, 650, 3200, 12500, 99,
                         7800, 2300, 560, 4100, 1500, 18900, 740, 3100};
    double result[2];
    minMax(prices, 0, 15, result);
    printf("Min %.0f, max %.0f\n", result[0], result[1]);
    printf("Comparisons: %d (simple method: %d)\n", comparisons, 2 * (16 - 1));
    return 0;
}
```

```
34 }
```

نتيجة التنفيذ

```
Min 99, max 27000
Comparisons: 22 (simple method: 30)
```

المخطط الإنسيابي (Organigramme)



تمرين 8.28 صرف بطاقة الهدية بالضبط

تجارة إلكترونية ★★ متوسط

لدى الزبون بطاقة هدية بقيمة محددة ويريد صرفها **بالضبط** دون باقٍ. اكتب دالة عودية `canSpend(index, remaining)` تحدد إن كانت توجد مجموعة من المنتجات (كل منتج مرة واحدة) مجموع أسعارها يساوي القيمة، وتعرض أحد الحلول.

خطوات الحل

1) `remaining == 0`: وجدنا حلًا. `index == NB` أو `remaining < 0`: لا حل في هذا الفرع.
2) نجرب أخذ المنتج `index` (نعلمه في `taken`)، وإن فشلنا نجرب عدم أخذه.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 7

int prices[NB] = {1200, 3500, 899, 2100, 650, 1750, 4100};
int taken[NB];

int canSpend(int index, int remaining) {
    if (remaining == 0) return 1;                       // صرفنا البطاقة بالضبط
    if (index == NB || remaining < 0) return 0;
    taken[index] = 1;
    if (canSpend(index + 1, remaining - prices[index])) return 1;   // نأخذ المنتج
    taken[index] = 0;
    return canSpend(index + 1, remaining);                          // لا نأخذه
}

int main(void) {
    int card, query, idx;
    for (query = 0; query < 2; query++) {
        scanf("%d", &card);
        for (idx = 0; idx < NB; idx++) taken[idx] = 0;
        printf("Card %d DA: ", card);
```

```
        if (canSpend(0, card)) {
            for (idx = 0; idx < NB; idx++)
                if (taken[idx]) printf("%d ", prices[idx]);
            printf("\n");
        } else {
            printf("impossible\n");
        }
    }
    return 0;
}
```

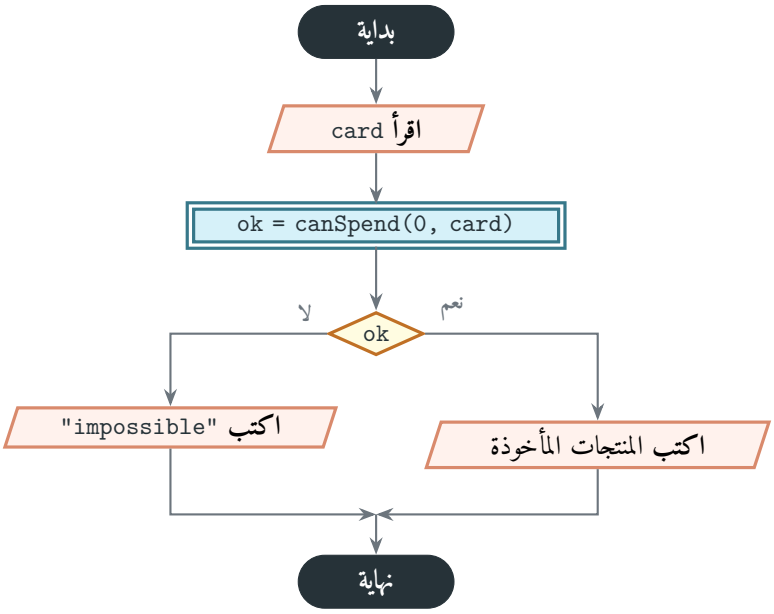
المدخلات

```
4700 5000
```

نتيجة التنفيذ

```
Card 4700 DA: 1200 3500
Card 5000 DA: impossible
```

المخطط الإنسيابي (Organigramme)



تمرين 8.29 تجزئة الكمية إلى دفعات

تجارة إلكترونية ★★ متوسط

بكم طريقة يمكن تقسيم شحنة من `total` وحدة إلى دفعات (الترتيب لا يهم)؟ مثلًا 4 = 3+1 = 2+2 = 2+1+1 = 1+1+1+1 (5 طرق). اكتب الدالة العودية `partitions(total, largest)` حيث `largest` أكبر حجم دفعة مسموح.

خطوات الحل

1) `partitions(0, ...) = 1`؛ `largest == 0` أو `total < 0`: 0.

2) إما نستعمل دفعة بحجم `largest`: `partitions(total - largest, largest)`، أو لا نستعمل هذا الحجم: `partitions(total, largest - 1)`.

البرنامج بلغة C

```c
#include <stdio.h>

long memo[101][101];

long partitions(int total, int largest) {
    if (total == 0) return 1;
    if (total < 0 || largest == 0) return 0;
    if (memo[total][largest] > 0) return memo[total][largest];
    memo[total][largest] = partitions(total - largest, largest)   // دفعة بحجم largest
                         + partitions(total, largest - 1);        // دفعات أصغر فقط
    return memo[total][largest];
}

int main(void) {
    int total, query;
    for (query = 0; query < 3; query++) {
        scanf("%d", &total);
        printf("%d units: %ld ways\n", total, partitions(total, total));
    }
    printf("30 units with batches of at most 5: %ld ways\n", partitions(30, 5));
    return 0;
}
```

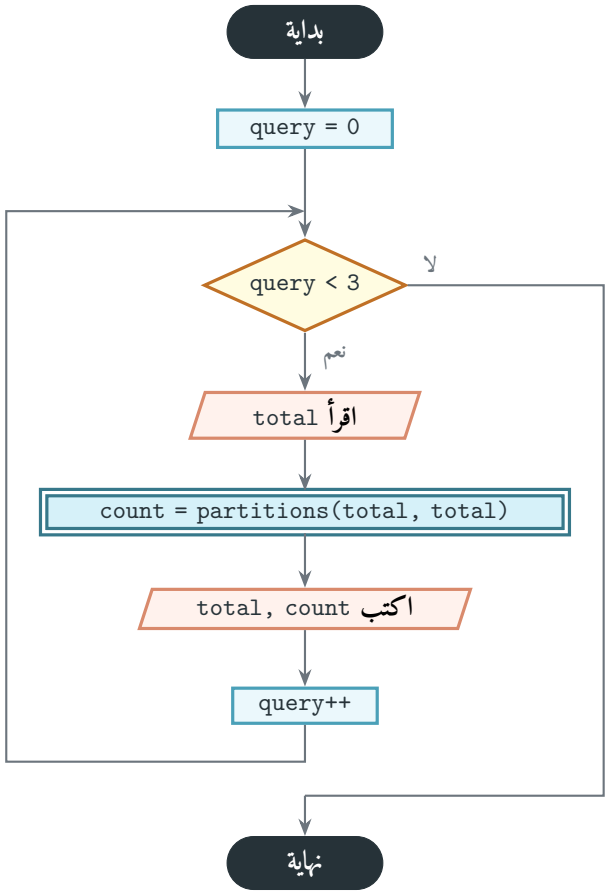
المدخلات

```
4 10 30
```

نتيجة التنفيذ

```
4 units: 5 ways
10 units: 42 ways
30 units: 5604 ways
30 units with batches of at most 5: 674 ways
```

المخطط الإنسيابي (Organigramme)



تمرين 8.30 أعداد كاتالان وأشجار القرار

علم البيانات ★★ متوسط

عدد أشجار القرار الثنائية المختلفة التي يمكن بناؤها بـ `nodes` عقدة هو **عدد كاتالان** `C(n) = Σ C(left) × C(n-1-left)` لـ `left` من 0 إلى `n-1`. اكتب الدالة العودية بالتذكر واعرض الأعداد من 0 إلى 15.

خطوات الحل

1) `C(0) = 1` (شجرة فارغة).

2) نختار الجذر، ثم `left` عقدة يسارًا و `n - 1 - left` يمينًا: نضرب عدد الإمكانيات.

البرنامج بلغة C

```c
#include <stdio.h>

long memo[40];

long catalan(int nodes) {
    long total = 0;
    int left;
    if (nodes <= 1) return 1;
    if (memo[nodes] != 0) return memo[nodes];
    for (left = 0; left < nodes; left++)
        total += catalan(left) * catalan(nodes - 1 - left);   // يسار × يمين
    memo[nodes] = total;
```

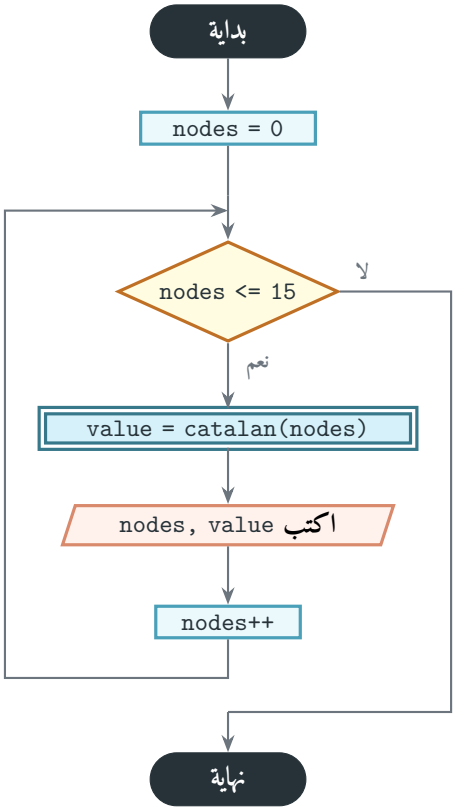
```c
    return total;
}

int main(void) {
    int nodes;
    for (nodes = 0; nodes <= 15; nodes++) {
        printf("C(%2d) = %-9ld", nodes, catalan(nodes));
        if (nodes % 4 == 3) printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
C( 0) = 1         C( 1) = 1         C( 2) = 2         C( 3) = 5
C( 4) = 14        C( 5) = 42        C( 6) = 132       C( 7) = 429
C( 8) = 1430      C( 9) = 4862      C(10) = 16796     C(11) = 58786
C(12) = 208012    C(13) = 742900    C(14) = 2674440   C(15) = 9694845
```

المخطط الإنسيابي (Organigramme)



تمرين 8.31 مساحة المنطقة الحرة في المخزن

تجارة إلكترونية ★★ متوسط

مخطط المخزن شبكة: # رف، . ممر حر. اكتب دالة عودية `fill(row, col)` (**ملء الفيضان**) تعد خانات المنطقة الحرة المتصلة انطلاقًا من خانة معينة (الحركة أفقية وعمودية)، وتعلمها بـ o، ثم عدّ عدد المناطق المنفصلة.

خطوات الحل

1) إذا كانت الخانة خارج الشبكة أو ليست `.`: نعيد 0.
2) نعلمها بـ `o` (لكي لا نعدها مرتين) ثم `1 + fill` للجيران الأربعة.
3) كل خانة `.` باقية بعد ملء منطقة هي بداية منطقة جديدة.

البرنامج بلغة C

```c
#include <stdio.h>

#define ROWS 6
#define COLS 10

char plan[ROWS][COLS + 1] = {"..##......",
                             "..##.####.",
                             "####.#..#.",
                             "....##..#.",
                             ".##.######",
                             ".#........"};

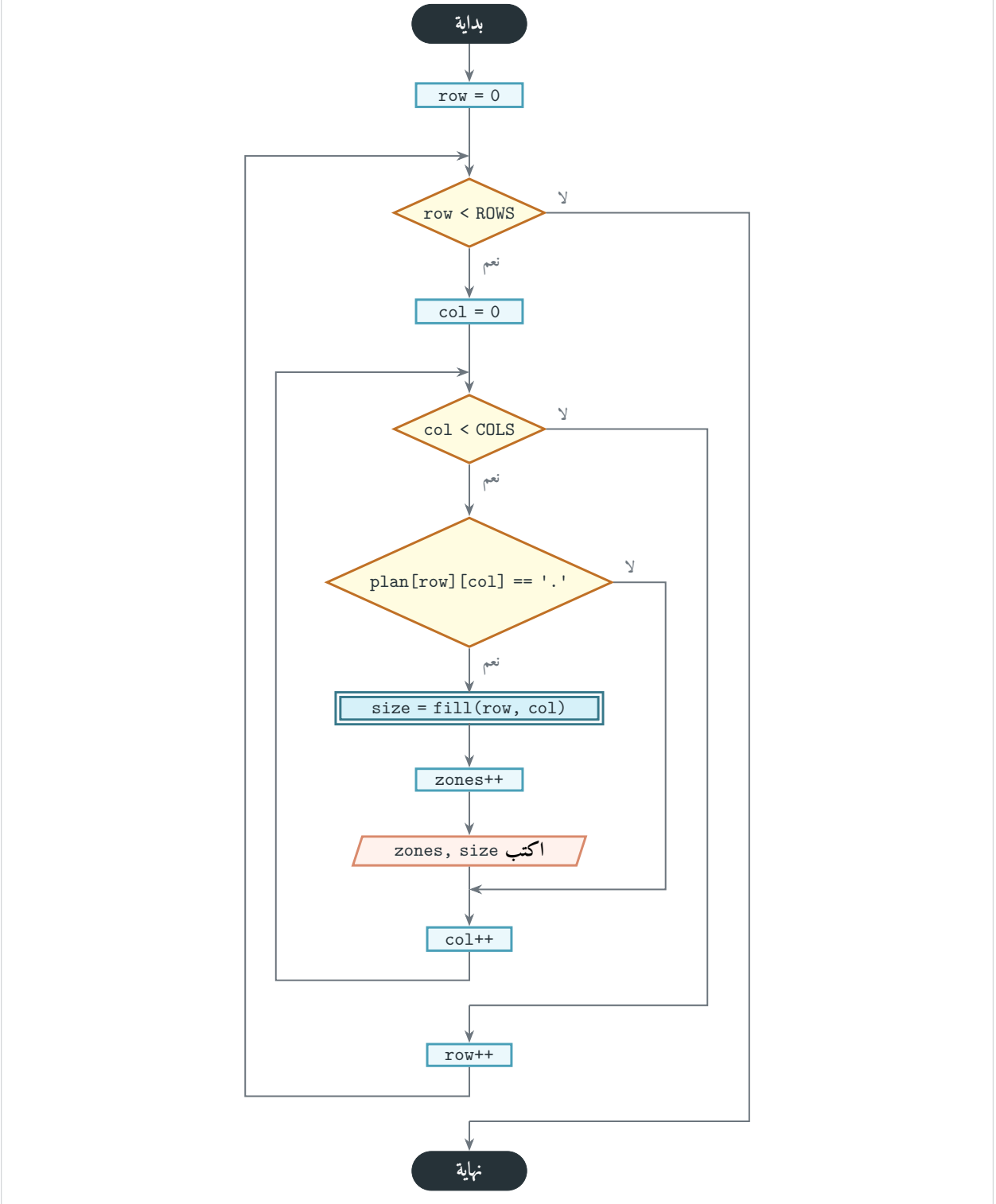
int fill(int row, int col) {
    if (row < 0 || row >= ROWS || col < 0 || col >= COLS || plan[row][col] != '.')
        return 0;
    plan[row][col] = 'o';                          // تمت زيارتها
    return 1 + fill(row + 1, col) + fill(row - 1, col)
             + fill(row, col + 1) + fill(row, col - 1);
}

int main(void) {
    int row, col, zones = 0;
    for (row = 0; row < ROWS; row++)
        for (col = 0; col < COLS; col++)
            if (plan[row][col] == '.') {
                zones++;
                printf("Zone %d from (%d,%d): %d cells\n", zones, row, col, fill(row, col));
            }
    for (row = 0; row < ROWS; row++) printf("%s\n", plan[row]);
    return 0;
}
```

نتيجة التنفيذ

```
Zone 1 from (0,0): 4 cells
Zone 2 from (0,4): 11 cells
Zone 3 from (2,6): 4 cells
Zone 4 from (3,0): 15 cells
oo##oooooo
oo##o####o
####o#oo#o
oooo##oo#o
o##o######
o#oooooooo
```

المخطط الإنسيابي (Organigramme)



تمرين 8.32 مسار الروبوت في متاهة المخزن

تجارة إلكترونية

متوسط

روبوت جمع الطلبيات يبحث عن طريق من S إلى E في المخزن (# جدار). اكتب دالة عودية `solve(row, col)` بطريقة التراجع: نعلم الخانة كجزء من المسار *، نجرب الاتجاهات الأربعة، وإذا فشلت كلها نزيل العلامة.

خطوات الحل

1) الحالة الأساسية: الوصول إلى E نجاح.

2) خانة غير صالحة (جدار، خارج، أو زرناها): فشل.

3) عند الفشل في كل الاتجاهات نتراجع: نعلم الخانة بـ x (طريق مسدود) لكي لا نزورها مجددًا.

البرنامج بلغة C

```c
#include <stdio.h>

#define ROWS 7
#define COLS 11

char maze[ROWS][COLS + 1] = {"###########",
                             "#S..#.....#",
                             "#.#.#.###.#",
                             "#.#...#...#",
                             "#.#####.#.#",
                             "#.......#E#",
                             "###########"};

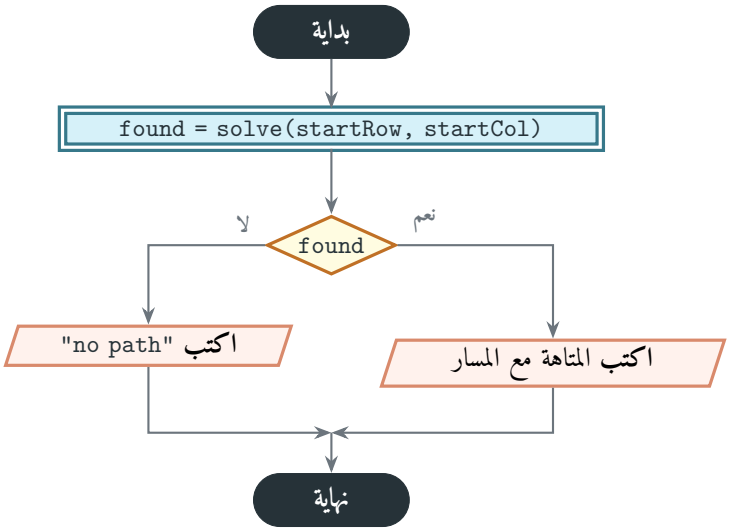
int solve(int row, int col) {
    if (maze[row][col] == 'E') return 1;                    // وصلنا
    if (maze[row][col] != '.' && maze[row][col] != 'S') return 0;
    if (maze[row][col] == '.') maze[row][col] = '*';        // جزء من المسار
    if (solve(row, col + 1) || solve(row + 1, col) || solve(row, col - 1) || solve(row - 1, col))
        return 1;
    if (maze[row][col] == '*') maze[row][col] = 'x';        // طريق مسدود
    return 0;
}

int main(void) {
    int row, steps = 0, col;
    maze[1][1] = 'S';
    if (solve(1, 1)) {
        for (row = 0; row < ROWS; row++) {
            for (col = 0; col < COLS; col++) {
                if (maze[row][col] == 'x') maze[row][col] = '.';
                if (maze[row][col] == '*') steps++;
            }
            printf("%s\n", maze[row]);
        }
        printf("Path found: %d cells\n", steps);
    } else {
        printf("no path\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
###########
#S**#######
#.#**####*#
#.#***#..*#
#.#####.#*#
#.......#E#
###########
Path found: 15 cells
```

المخطط الإنسيابي (Organigramme)



تمرين 8.33 مبيعات شجرة الأقسام

علم البيانات ★★ متوسط

أقسام المتجر شجرة: كل قسم له أب (`parent[]`، والجذر أبوه -1) ومبيعات خاصة به. اكتب دالة عودية `totalSales(node)` تعيد مبيعات القسم مع **كل الأقسام التابعة له**، ودالة `printTree(node, depth)` تعرض الشجرة مع الإزاحة.

خطوات الحل

1) مبيعات قسم = مبيعاته الخاصة + `totalSales` لكل ابن (الأبناء: العقد التي `parent == node`).

2) `printTree` تطبع القسم بإزاحة `2 * depth` ثم تستدعي نفسها على الأبناء بـ `depth + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 9

char names[NB][14] = {"Store", "Electronics", "Phones", "Laptops", "Home",
                      "Kitchen", "Garden", "Accessories", "Chargers"};
int parent[NB] = {-1, 0, 1, 1, 0, 4, 4, 1, 7};
double own[NB] = {0, 1200, 8400, 6100, 300, 2500, 1800, 900, 1350};

double totalSales(int node) {
    double total = own[node];
    int child;
    for (child = 0; child < NB; child++)
        if (parent[child] == node)
            total += totalSales(child);          // مبيعات الفرع
    return total;
}

void printTree(int node, int depth) {
    int child;
    printf("%*s%-*s %8.0f\n", 2 * depth, "", 16 - 2 * depth, names[node], totalSales(node));
    for (child = 0; child < NB; child++)
        if (parent[child] == node)
```

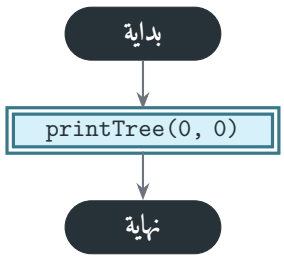
```
            printTree(child, depth + 1);
}

int main(void) {
    printTree(0, 0);
    return 0;
}
```

نتيجة التنفيذ

```
Store               22550
  Electronics       17950
    Phones           8400
    Laptops          6100
    Accessories      2250
      Chargers       1350
  Home              4600
    Kitchen          2500
    Garden           1800
```

المخطط الإنسيابي (Organigramme)



تمرين 8.34 الجذر التربيعي بالتنصيف العودي

علم البيانات متوسط

اكتب دالة عودية `rootBisect(value, low, high)` تجد الجذر التربيعي: نحسب المنتصف `middle`، إذا كان $\texttt{middle}^2$ أكبر من القيمة نبحث في `[low, middle]`، وإلا في `[middle, high]`، حتى يصبح طول المجال أقل من `1e-9`.

خطوات الحل

1) الحالة الأساسية: `high - low < 1e-9`.
2) المجال الابتدائي `[0, max(1, value)]`.
3) كل استدعاء يقسم المجال على 2.

البرنامج بلغة C

```c
#include <stdio.h>

int depthReached = 0;

double rootBisect(double value, double low, double high) {
    double middle = (low + high) / 2;
    depthReached++;
    if (high - low < 1e-9) return middle;            // دقة كافية
    if (middle * middle > value)
        return rootBisect(value, low, middle);
```

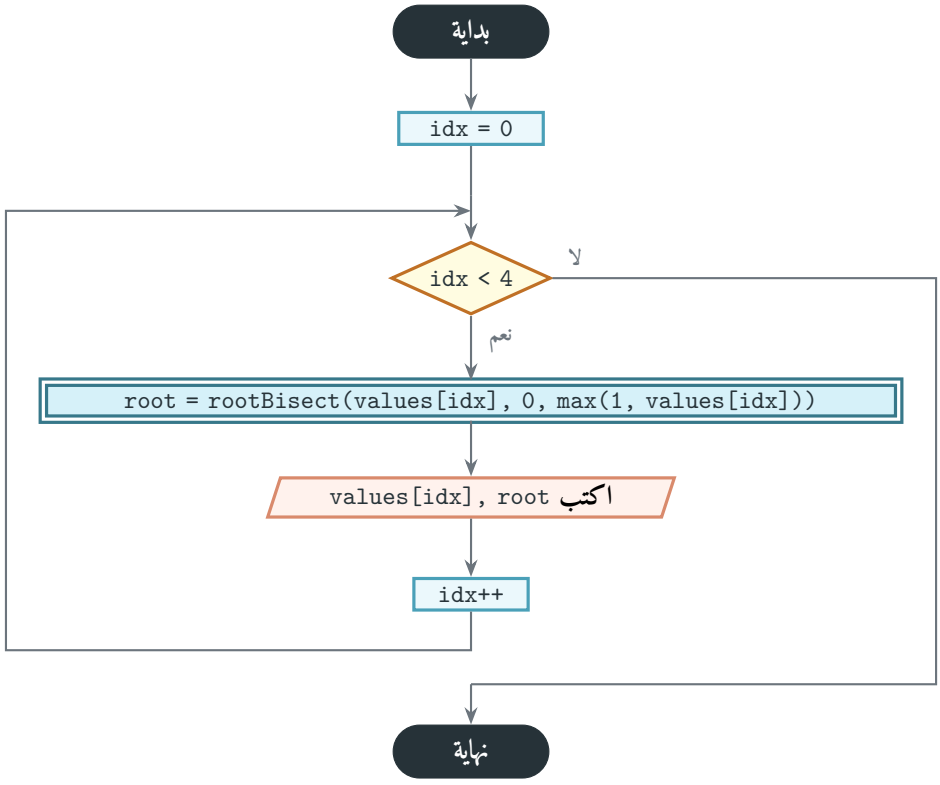
```
    return rootBisect(value, middle, high);
}

int main(void) {
    double values[4] = {2, 0.25, 144, 1000000}, root;
    int idx;
    for (idx = 0; idx < 4; idx++) {
        depthReached = 0;
        root = rootBisect(values[idx], 0, values[idx] > 1 ? values[idx] : 1);
        printf("sqrt(%.2f) = %.8f (%d calls)\n", values[idx], root, depthReached);
    }
    return 0;
}
```

نتيجة التنفيذ

```
sqrt(2.00) = 1.41421356 (32 calls)
sqrt(0.25) = 0.50000000 (31 calls)
sqrt(144.00) = 12.00000000 (39 calls)
sqrt(1000000.00) = 1000.00000000 (51 calls)
```

المخطط الإنسيابي (Organigramme)



تمرين 8.35 قلب السلسلة في مكانها عوديًا

تجارة إلكترونية متوسط

اكتب دالة عودية `reverseInPlace(text, left, right)` تقلب السلسلة **داخل نفس الجدول**: تبدل الحرفين الطرفيين ثم تستدعي نفسها على الجزء الداخلي. استعملها لقلب كل كلمة من جملة على حدة.

خطوات الحل

1) الحالة الأساسية `left >= right`.
2) لقلب كل كلمة: نجد بداية الكلمة ونهايتها ثم نستدعي الدالة على هذا المجال.

البرنامج بلغة C

```c
#include <stdio.h>

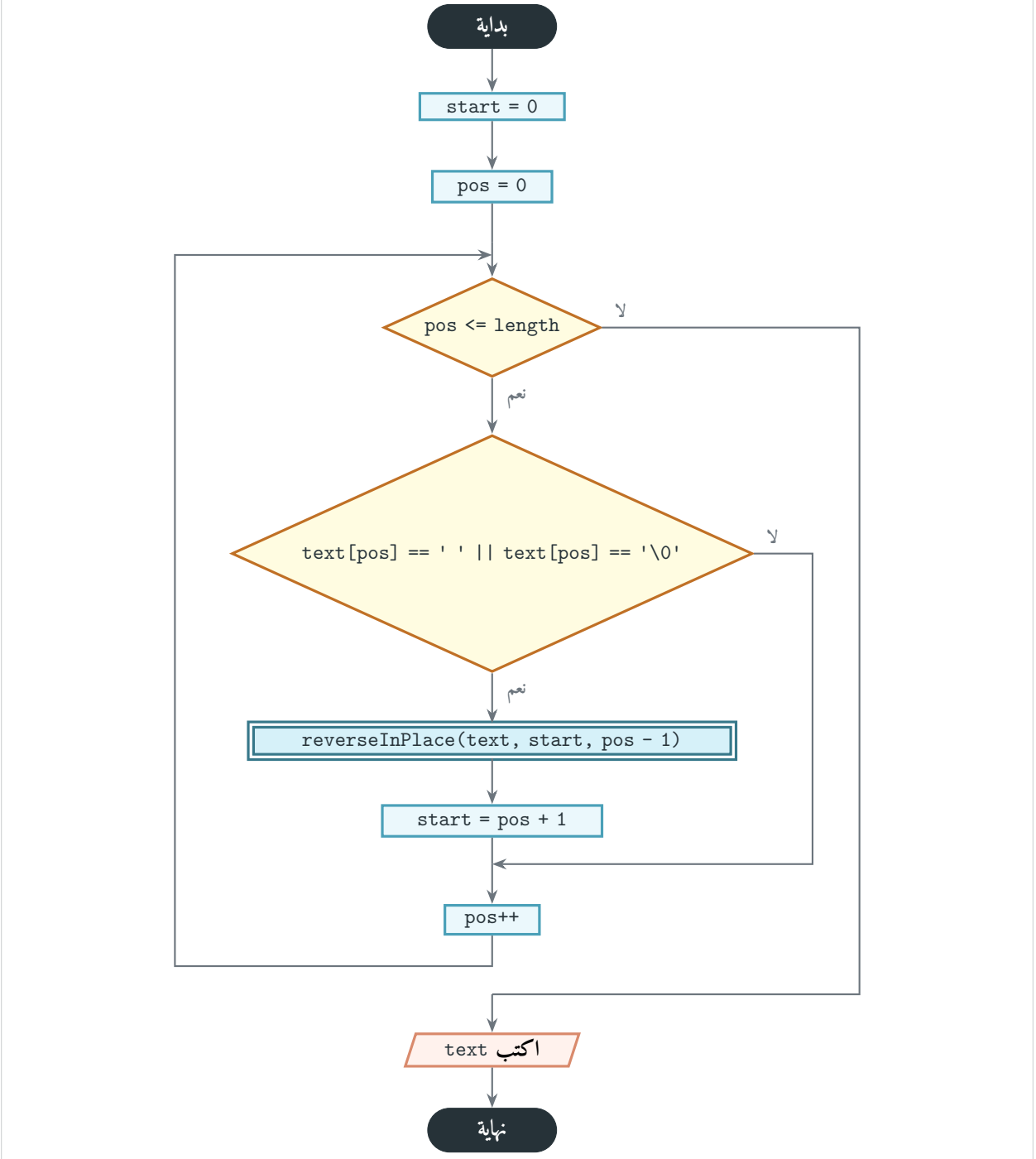
void reverseInPlace(char text[], int left, int right) {
    char tmp;
    if (left >= right) return;
    tmp = text[left];                  // تبديل الطرفين
    text[left] = text[right];
    text[right] = tmp;
    reverseInPlace(text, left + 1, right - 1);
}

int main(void) {
    char text[] = "fast delivery for every order";
    int pos, start = 0, length = 0;
    while (text[length] != '\0') length++;
    reverseInPlace(text, 0, length - 1);
    printf("Whole: %s\n", text);
    reverseInPlace(text, 0, length - 1);          // نعيدها كما كانت
    for (pos = 0; pos <= length; pos++)
        if (text[pos] == ' ' || text[pos] == '\0') {
            reverseInPlace(text, start, pos - 1);  // كل كلمة على حدة
            start = pos + 1;
        }
    printf("Words: %s\n", text);
    return 0;
}
```

نتيجة التنفيذ

```
Whole: redro yreve rof yreviled tsaf
Words: tsaf yreviled rof yreve redro
```

المخطط الإنسيابي (Organigramme)



تمرين 8.36 حساب كثير الحدود بطريقة هورنر العودية

علم البيانات ★★ متوسط

نموذج التكلفة كثير حدود `cost(q) = 0.002q³ - 0.5q² + 60q + 1500`. اكتب دالة عودية `horner(coeffs, degree, value)` تحسب `p(q) = a0 + q·(a1 + q·(a2 + ...))` بعدد ضربات يساوي الدرجة فقط، واحسب التكلفة المتوسطة لعدة كميات.

خطوات الحل

1) `coeffs[0]` الحد الثابت، `coeffs[degree]` معامل أعلى درجة.

2) `horner(coeffs, idx) = coeffs[idx] + value * horner(coeffs, idx + 1)`

، والحالة الأساسية `idx == degree`.

البرنامج بلغة C

```c
#include <stdio.h>

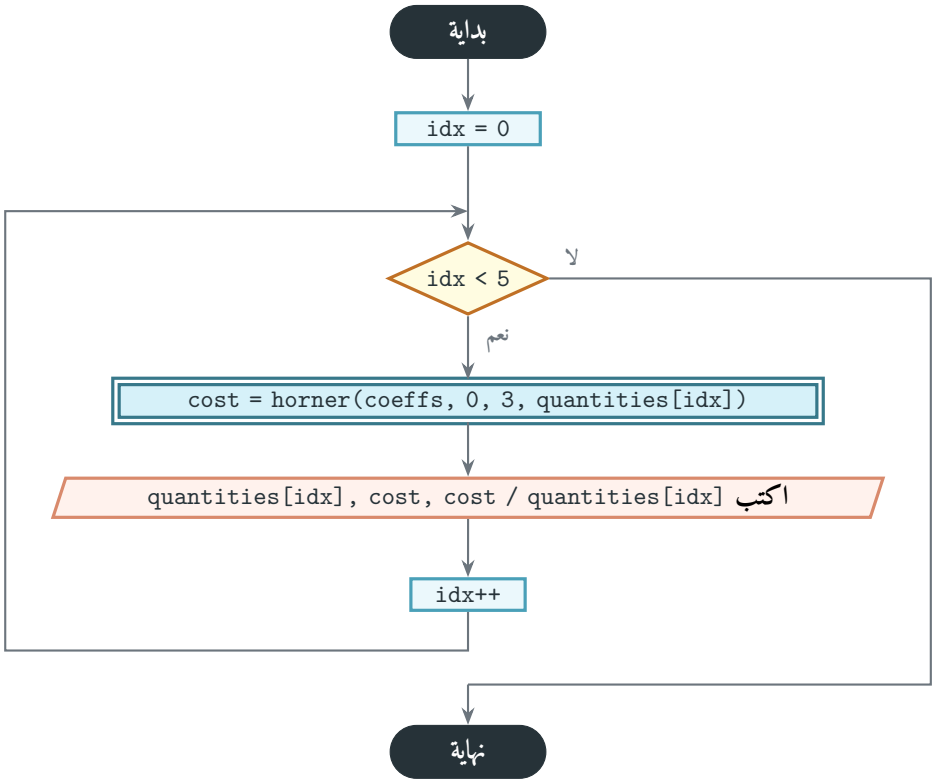
double horner(double coeffs[], int index, int degree, double value) {
    if (index == degree) return coeffs[degree];
    return coeffs[index] + value * horner(coeffs, index + 1, degree, value);
}

int main(void) {
    double coeffs[4] = {1500, 60, -0.5, 0.002};   // a0 + a1 q + a2 q^2 + a3 q^3
    double quantities[5] = {10, 50, 100, 150, 200}, cost;
    int idx;
    for (idx = 0; idx < 5; idx++) {
        cost = horner(coeffs, 0, 3, quantities[idx]);
        printf("q = %3.0f: cost %9.2f, per unit %7.2f\n",
               quantities[idx], cost, cost / quantities[idx]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
q =  10: cost   2052.00, per unit  205.20
q =  50: cost   3500.00, per unit   70.00
q = 100: cost   4500.00, per unit   45.00
q = 150: cost   6000.00, per unit   40.00
q = 200: cost   9500.00, per unit   47.50
```

المخطط الإنسيابي (Organigramme)



تمرين 8.37 دمج قائمتين مرتبتين عوديًا

تجارة إلكترونية ★★ متوسط

لدينا قائمتا أسعار مرتبتان من مورّدين. اكتب دالة عودية `mergeRec(first, sizeA, second, sizeB, out)` تدمجهما في قائمة مرتبة واحدة: نأخذ الأصغر من رأسي القائمتين ثم ندمج الباقي عوديًا.

خطوات الحل

1) إذا انتهت إحدى القائمتين ننسخ الأخرى.

2) وإلا نضع الأصغر في `out[0]` ونستدعي الدالة على القائمتين مع تقدم القائمة التي أُخذ منها العنصر، و `out + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

void mergeRec(int first[], int sizeA, int second[], int sizeB, int out[]) {
    int idx;
    if (sizeA == 0) { for (idx = 0; idx < sizeB; idx++) out[idx] = second[idx]; return; }
    if (sizeB == 0) { for (idx = 0; idx < sizeA; idx++) out[idx] = first[idx]; return; }
    if (first[0] <= second[0]) {
        out[0] = first[0];
        mergeRec(first + 1, sizeA - 1, second, sizeB, out + 1);     // باقي الأولى
    } else {
        out[0] = second[0];
        mergeRec(first, sizeA, second + 1, sizeB - 1, out + 1);     // باقي الثانية
    }
}

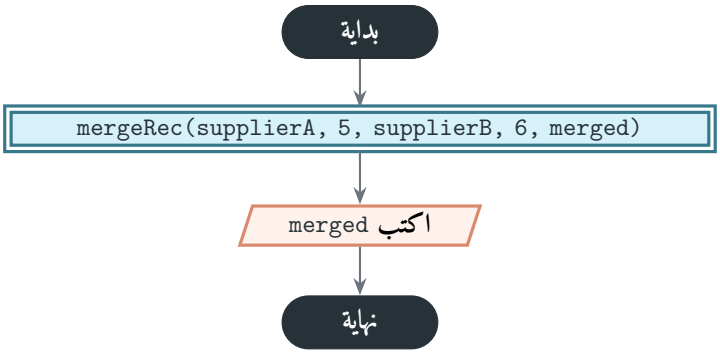
```

```c
int main(void) {
    int supplierA[5] = {450, 900, 1500, 3200, 7800};
    int supplierB[6] = {300, 899, 1200, 2500, 4100, 9900};
    int merged[11], idx;
    mergeRec(supplierA, 5, supplierB, 6, merged);
    for (idx = 0; idx < 11; idx++) printf("%d ", merged[idx]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
300 450 899 900 1200 1500 2500 3200 4100 7800 9900
```

المخطط الانسيابي (Organigramme)



تمرين 8.38 رموز ثنائية دون واحدين متتاليين

علم البيانات متوسط

نظام قراءة البطاقات يرفض الرموز الثنائية التي تحتوي على 11. اكتب دالة عودية `generate(pos, previous)` تعرض كل الرموز الصالحة بطول 5، وعدّها (يجب أن يكون العدد من متتالية فيبوناتشي).

خطوات الحل

1) في كل موضع نضع 0 دائمًا، ونضع 1 فقط إذا كان الرقم السابق 0.
2) الحالة الأساسية `pos == LEN`: نطبع الرمز.

البرنامج بلغة C

```c
#include <stdio.h>

#define LEN 5

char code[LEN + 1];
int count = 0;

void generate(int pos, int previous) {
    if (pos == LEN) {
        code[LEN] = '\0';
        printf("%s ", code);
        if (++count % 7 == 0) printf("\n");
        return;
    }
```

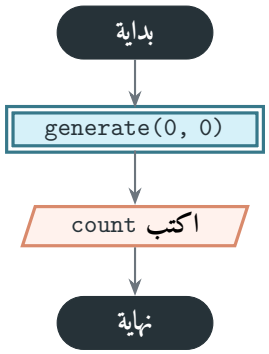
```
    code[pos] = '0';
    generate(pos + 1, 0);
    if (previous == 0) {                    // 0 بعد 1 مسموح فقط
        code[pos] = '1';
        generate(pos + 1, 1);
    }
}

int main(void) {
    generate(0, 0);
    printf("\n%d valid codes of length %d\n", count, LEN);
    return 0;
}
```

نتيجة التنفيذ

```
00000 00001 00010 00100 00101 01000 01001
01010 10000 10001 10010 10100 10101
13 valid codes of length 5
```

المخطط الإنسيابي (Organigramme)



تمرين 8.39 اختيار ثلاثة منتجات ضمن الميزانية

تجارة إلكترونية ★★ متوسط

اكتب دالة عودية `combine(start, picked, spent)` تولد كل **التوافيق** لثلاثة منتجات من ستة (20 توفيقة) دون تكرار ودون اعتبار الترتيب، وتعرض فقط التي لا يتجاوز مجموعها ميزانية 9000 دج.

خطوات الحل

1) نختار المنتج التالي من `start` فما بعد فقط (لتفادي التكرار).
2) الحالة الأساسية `picked == 3`.
3) **التقليم**: إذا تجاوز `spent` الميزانية نتوقف عن التوسيع.

البرنامج بلغة C

```
#include <stdio.h>

#define NB 6
#define PICK 3
#define BUDGET 9000

char names[NB][10] = {"Mouse", "Keyboard", "Headset", "Webcam", "Hub", "Stand"};
```

```c
int prices[NB] = {1500, 3200, 2800, 4100, 2200, 1900};
int chosen[PICK], valid = 0;

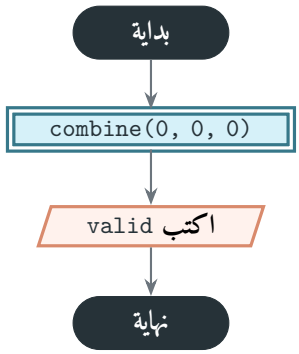
void combine(int start, int picked, int spent) {
    int idx;
    if (spent > BUDGET) return;                    // تقليم
    if (picked == PICK) {
        valid++;
        for (idx = 0; idx < PICK; idx++) printf("%-9s", names[chosen[idx]]);
        printf("= %d\n", spent);
        return;
    }
    for (idx = start; idx < NB; idx++) {
        chosen[picked] = idx;
        combine(idx + 1, picked + 1, spent + prices[idx]);
    }
}

int main(void) {
    combine(0, 0, 0);
    printf("%d bundles within %d DA\n", valid, BUDGET);
    return 0;
}
```

نتيجة التنفيذ

```
Mouse    Keyboard Headset  = 7500
Mouse    Keyboard Webcam   = 8800
Mouse    Keyboard Hub      = 6900
Mouse    Keyboard Stand    = 6600
Mouse    Headset  Webcam   = 8400
Mouse    Headset  Hub      = 6500
Mouse    Headset  Stand    = 6200
Mouse    Webcam   Hub      = 7800
Mouse    Webcam   Stand    = 7500
Mouse    Hub      Stand    = 5600
Keyboard Headset  Hub      = 8200
Keyboard Headset  Stand    = 7900
Keyboard Hub      Stand    = 7300
Headset  Webcam   Stand    = 8800
Headset  Hub      Stand    = 6900
Webcam   Hub      Stand    = 8200
16 bundles within 9000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 8.40 عمق التداخل في ملف إعدادات

علم البيانات متوسط

ملف إعدادات المتجر يستعمل الأقواس {} و [] بشكل متداخل. اكتب دالة عودية `parse(text, pos, depth, closing)` تمر على النص، وعند كل قوس فتح تستدعي نفسها بعمق أكبر، وتعيد الموضع بعد قوس الغلق المطابق. احسب **أقصى عمق** وتحقق من صحة التوازن.

خطوات الحل

1) الدالة تعالج كتلة كاملة: تتقدم حتى تجد قوس الغلق المطابق لقوسها.
2) عند قوس فتح:

```
pos = parse(text, pos + 1, depth + 1, closing)
```

حيث `closing` قوس الغلق المنتظر.
3) قوس غلق غير مطابق أو نهاية مبكرة: خطأ `(-1)`.

البرنامج بلغة C

```c
#include <stdio.h>

int maxDepth = 0;

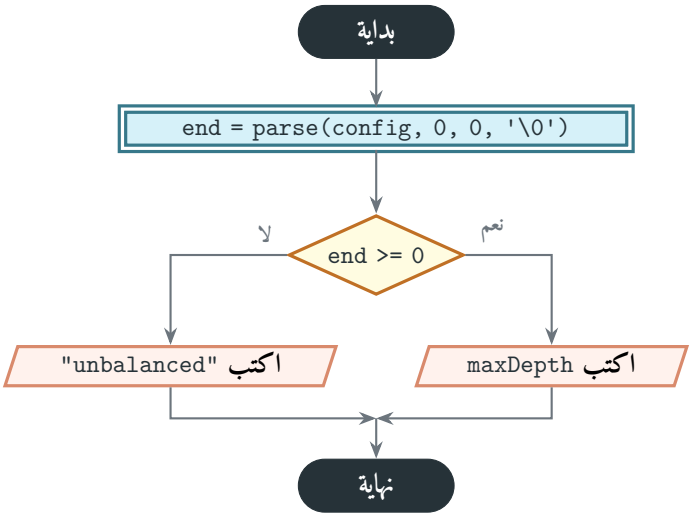
int parse(char text[], int pos, int depth, char closing) {
    char ch;
    if (depth > maxDepth) maxDepth = depth;
    while (text[pos] != '\0') {
        ch = text[pos];
        if (ch == '{' || ch == '[') {
            pos = parse(text, pos + 1, depth + 1, ch == '{' ? '}' : ']');   // كتلة داخلية
            if (pos < 0) return -1;
        } else if (ch == '}' || ch == ']') {
            return ch == closing ? pos + 1 : -1;      // يجب أن يطابق قوس الفتح
        } else {
            pos++;
        }
    }
    return closing == '\0' ? pos : -1;                // نهاية النص
}

int main(void) {
    char good[] = "{shop:{name:x,cats:[{id:1,tags:[a,b]},{id:2}]},pay:[cash]}";
    char bad[] = "{shop:{cats:[{id:1]}}";
    int end;
    end = parse(good, 0, 0, '\0');
    printf("Config 1: %s, max depth %d\n", end >= 0 ? "balanced" : "unbalanced", maxDepth);
    maxDepth = 0;
    end = parse(bad, 0, 0, '\0');
    printf("Config 2: %s\n", end >= 0 ? "balanced" : "unbalanced");
    return 0;
}
```

نتيجة التنفيذ

```
Config 1: balanced, max depth 5
Config 2: unbalanced
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 8.41 الترتيب بالدمج وعدد الانقلابات

علم البيانات صعب

رتّب محرك التوصيات 10 منتجات، ورتّبها الزبون بطريقته. عدد **الانقلابات** (أزواج المنتجات المرتبة في اتجاهين مختلفين) يقيس الاختلاف بين الترتيبين. اكتب دالة `sortCount(values, low, high)` **عودية** ترتب بالدمج وتعد الانقلابات في نفس الوقت بتعقيد `n log n`.

خطوات الحل

1) نقسم إلى نصفين ونرتب كل نصف عودياً (ونعد انقلاباته).
2) أثناء الدمج: إذا أخذنا عنصرًا من النصف الأيمن قبل عناصر باقية في الأيسر، فكل تلك العناصر الباقية (`middle - first + 1`) تشكل انقلابات معه.
3) المجموع = انقلابات اليسار + اليمين + الدمج.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 10

int buffer[NB];

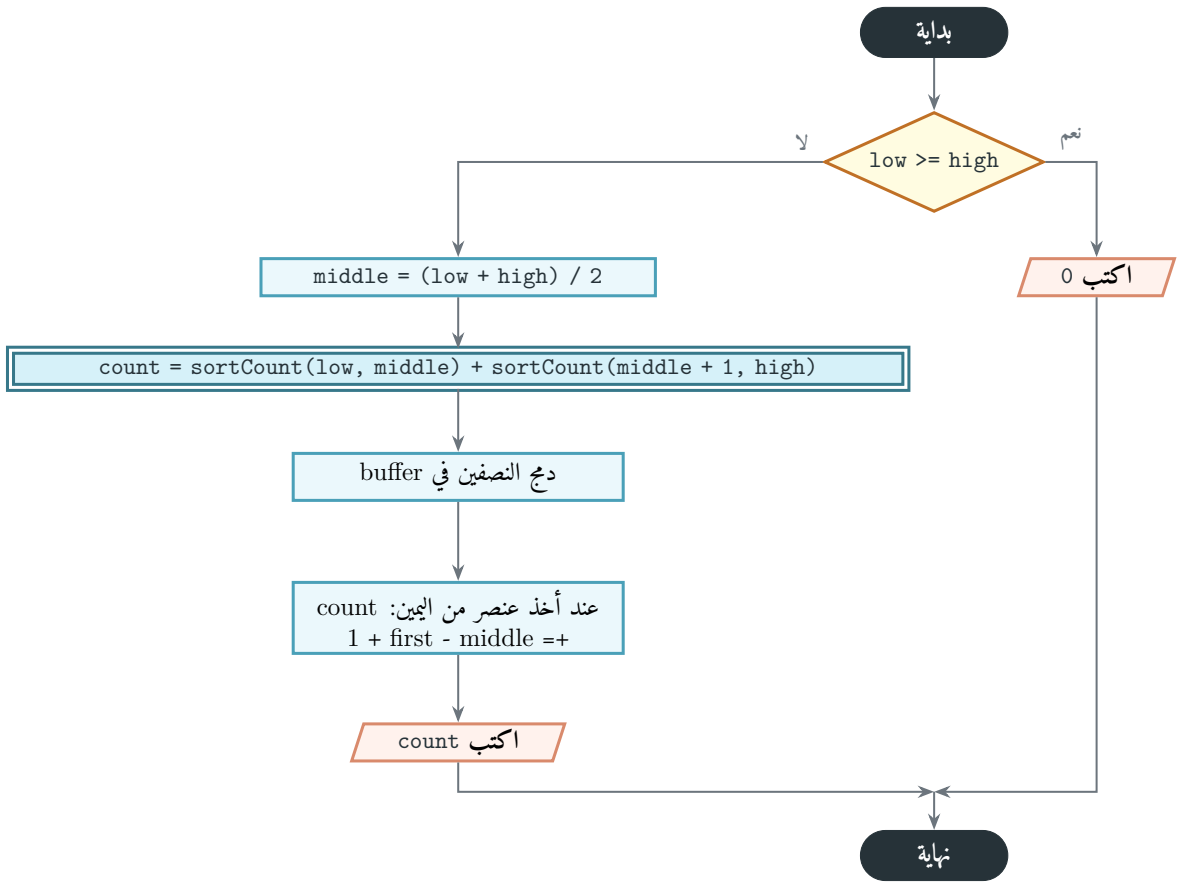
long sortCount(int values[], int low, int high) {
    int middle, first, second, out, idx;
    long count;
    if (low >= high) return 0;
    middle = (low + high) / 2;
    count = sortCount(values, low, middle) + sortCount(values, middle + 1, high);
    first = low; second = middle + 1; out = low;
    while (first <= middle && second <= high) {
        if (values[first] <= values[second]) {
            buffer[out++] = values[first++];
        } else {
            buffer[out++] = values[second++];
            count += middle - first + 1;          // كل الباقي في اليسار أكبر منه
        }
    }
    while (first <= middle) buffer[out++] = values[first++];
    while (second <= high) buffer[out++] = values[second++];
    for (idx = low; idx <= high; idx++) values[idx] = buffer[idx];
    return count;
}

int main(void) {
    int customer[NB] = {3, 1, 2, 5, 4, 7, 10, 6, 9, 8};   // رتبة المحرك لكل منتج حسب ترتيب الزبون
    int reversed[NB] = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1};
    long inv;
    int idx;
    inv = sortCount(customer, 0, NB - 1);
    printf("Inversions: %ld of %d pairs\n", inv, NB * (NB - 1) / 2);
    printf("Kendall tau = %.3f\n", 1 - 4.0 * inv / (NB * (NB - 1)));
    printf("Sorted: ");
    for (idx = 0; idx < NB; idx++) printf("%d ", customer[idx]);
    printf("\nFully reversed list: %ld inversions\n", sortCount(reversed, 0, NB - 1));
    return 0;
}
```

نتيجة التنفيذ

```
Inversions: 8 of 45 pairs
Kendall tau = 0.644
Sorted: 1 2 3 4 5 6 7 8 9 10
Fully reversed list: 45 inversions
```

المخطط الإنسيابي (Organigramme)



تمرين 8.42 الترتيب السريع للأسعار

تجارة إلكترونية صعب

اكتب الترتيب السريع **العودي** `quickSort(prices, low, high)`: نختار محورًا (العنصر الأوسط)، ونقسم الجدول بحيث يكون كل ما قبل المحور أصغر منه وكل ما بعده أكبر، ثم نرتب الجزأين عوديًا. اعرض كل تقسيم وعدد المقارنات.

خطوات الحل

1) التقسيم (طريقة هور): مؤشران `left` و `right` يتقاربان، نتوقف عند عنصر في غير مكانه من كل جهة ونبدلهما.
2) بعد التقسيم: `quickSort(low, right)` و `quickSort(left, high)`.
3) الحالة الأساسية `low >= high`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12
```

```c

int comparisons = 0;

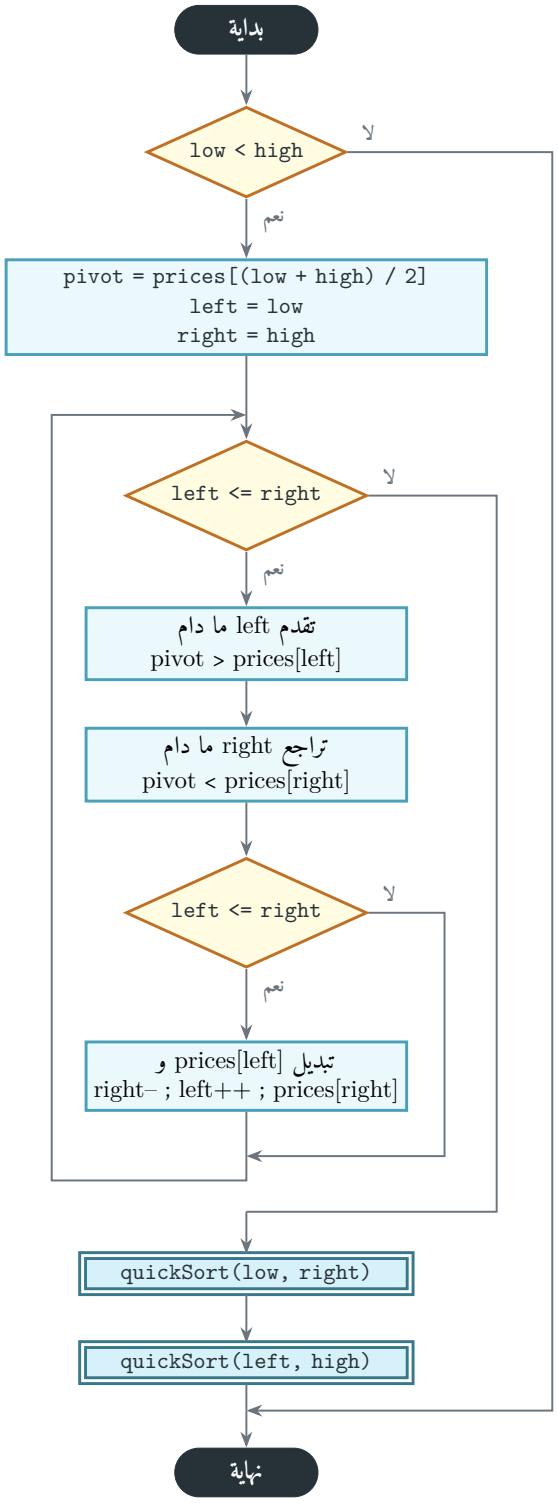
void quickSort(int prices[], int low, int high) {
    int pivot, left = low, right = high, tmp;
    if (low >= high) return;
    pivot = prices[(low + high) / 2];
    while (left <= right) {
        while (++comparisons && prices[left] < pivot) left++;
        while (++comparisons && prices[right] > pivot) right--;
        if (left <= right) {
            tmp = prices[left]; prices[left] = prices[right]; prices[right] = tmp;
            left++;
            right--;
        }
    }
    printf("pivot %5d on [%2d..%2d] -> [%2d..%2d] | [%2d..%2d]\n",
           pivot, low, high, low, right, left, high);
    quickSort(prices, low, right);              // الجزء الأصغر من المحور
    quickSort(prices, left, high);              // الجزء الأكبر
}

int main(void) {
    int prices[NB] = {4500, 1200, 899, 27000, 650, 3200, 12500, 99, 7800, 2300, 560, 4100};
    int idx;
    quickSort(prices, 0, NB - 1);
    for (idx = 0; idx < NB; idx++) printf("%d ", prices[idx]);
    printf("\n%d comparisons\n", comparisons);
    return 0;
}
```

نتيجة التنفيذ

```
pivot  3200 on [ 0..11] -> [ 0.. 5] | [ 6..11]
pivot   899 on [ 0.. 5] -> [ 0.. 2] | [ 3.. 5]
pivot    99 on [ 0.. 2] -> [ 0.. 0] | [ 1.. 2]
pivot   560 on [ 1.. 2] -> [ 1.. 0] | [ 2.. 2]
pivot   899 on [ 3.. 5] -> [ 3.. 3] | [ 4.. 5]
pivot  2300 on [ 4.. 5] -> [ 4.. 4] | [ 5.. 5]
pivot  7800 on [ 6..11] -> [ 6.. 8] | [ 9..11]
pivot  3200 on [ 6.. 8] -> [ 6.. 6] | [ 7.. 8]
pivot  4100 on [ 7.. 8] -> [ 7.. 6] | [ 8.. 8]
pivot  7800 on [ 9..11] -> [ 9.. 9] | [10..11]
pivot 27000 on [10..11] -> [10..10] | [11..11]
99 560 650 899 1200 2300 3200 4100 4500 7800 12500 27000
52 comparisons
```

المخطط الإنسيابي (Organigramme)



تمرين 8.43 توزيع الأكشاك دون تهديد متبادل

تجارة إلكترونية

صعب

في قاعة معرض على شكل شبكة 6×6 نريد وضع 6 أكشاك لعلامات متنافسة بحيث لا يتشارك كشكان نفس السطر أو العمود أو القطر (مسألة الملكات). اكتب دالة عودية بالتراجع `place(row)` تعد كل الحلول وتعرض الأول.

خطوات الحل

1) في كل سطر نضع كشكًا واحدًا، نجرب كل عمود.
2) الموضع آمن إذا لم يكن في سطر سابق كشك في نفس العمود أو على نفس القطر (`|col1 - col2| == |row1 - row2|`).
3) إذا وصلنا إلى `row == NB` فقد وجدنا حلًا.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6

int cols[NB], first[NB], solutions = 0;

int safe(int row, int col) {
    int prev, diff;
    for (prev = 0; prev < row; prev++) {
        diff = cols[prev] > col ? cols[prev] - col : col - cols[prev];
        if (cols[prev] == col || diff == row - prev)    // نفس العمود أو القطر
            return 0;
    }
    return 1;
}

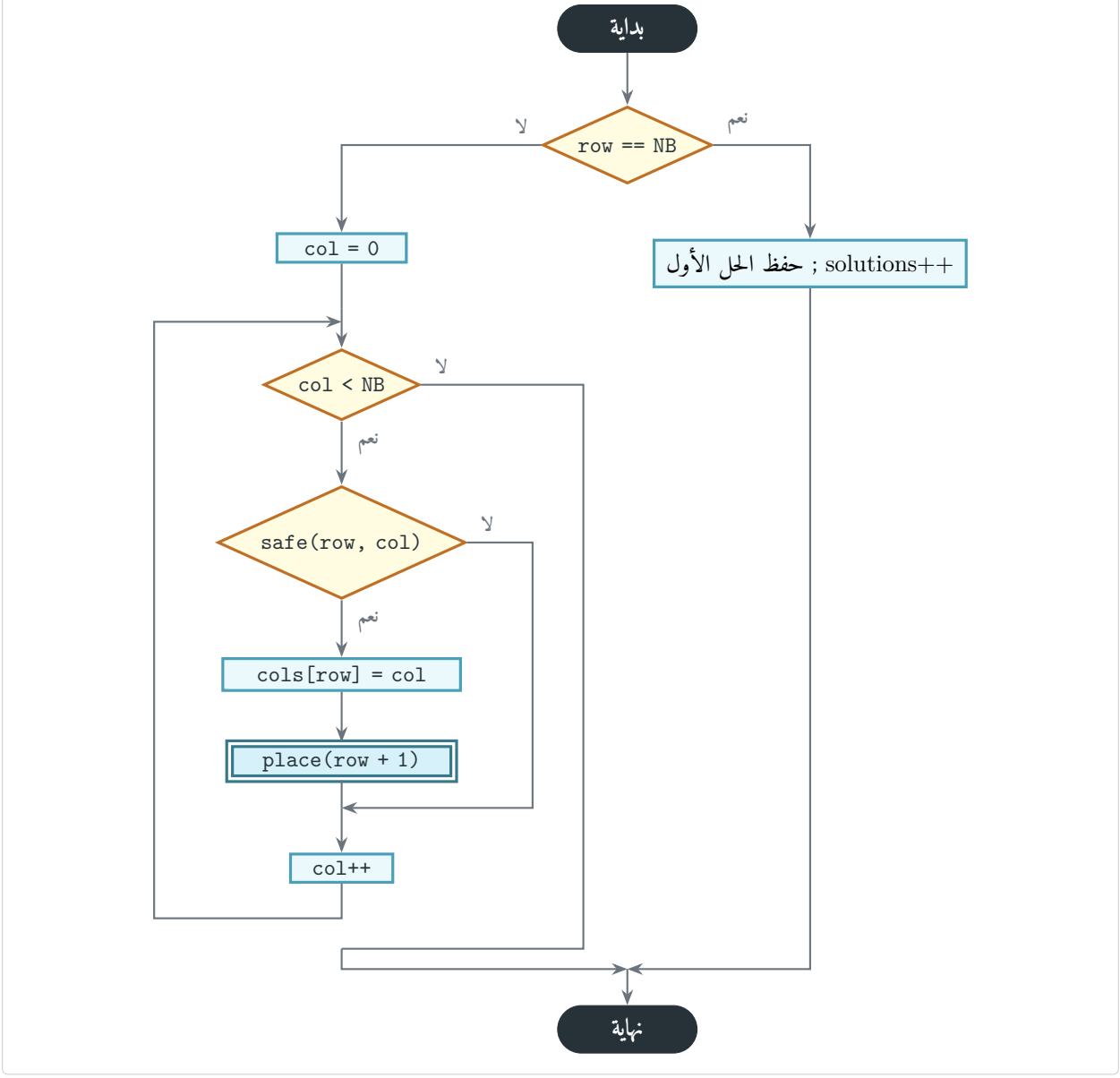
void place(int row) {
    int col, idx;
    if (row == NB) {
        if (solutions == 0)
            for (idx = 0; idx < NB; idx++) first[idx] = cols[idx];
        solutions++;
        return;
    }
    for (col = 0; col < NB; col++)
        if (safe(row, col)) {
            cols[row] = col;
            place(row + 1);                             // التراجع تلقائي عند العودة
        }
}

int main(void) {
    int row, col;
    place(0);
    printf("%d solutions. First one:\n", solutions);
    for (row = 0; row < NB; row++) {
        for (col = 0; col < NB; col++) printf(" %c", first[row] == col ? 'K' : '.');
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
4 solutions. First one:
 . K . . . .
 . . . K . .
 . . . . . K
 K . . . . .
 . . K . . .
 . . . . K .
```

المخطط الإنسيابي (Organigramme)



تمرين 8.44 حل لغز السودوكو الترويجي

تجارة إلكترونية ★★★ صعب

ينشر المتجر لغز سودوكو أسبوعيًا ومن يحله يربح قسيمة. اكتب دالة عودية بالتراجع `solve(cell)` تملأ الخانات الفارغة (0) بحيث يحتوي كل سطر وكل عمود وكل مربع 3×3 على الأرقام 1 إلى 9 مرة واحدة.

خطوات الحل

1) نمر على الخانات من 0 إلى 80؛ الخانة المملوءة نتخطاها.
2) نجرب الأرقام 1..9؛ الرقم مقبول إذا لم يظهر في السطر ولا العمود ولا المربع.
3) إذا فشلت كل الأرقام نعيد الخانة إلى 0 ونرجع 0 (تراجع).

البرنامج بلغة C

```c
#include <stdio.h>

int grid[9][9] = {{5, 3, 0, 0, 7, 0, 0, 0, 0}, {6, 0, 0, 1, 9, 5, 0, 0, 0},
                  {0, 9, 8, 0, 0, 0, 0, 6, 0}, {8, 0, 0, 0, 6, 0, 0, 0, 3},
                  {4, 0, 0, 8, 0, 3, 0, 0, 1}, {7, 0, 0, 0, 2, 0, 0, 0, 6},
                  {0, 6, 0, 0, 0, 0, 2, 8, 0}, {0, 0, 0, 4, 1, 9, 0, 0, 5},
                  {0, 0, 0, 0, 8, 0, 0, 7, 9}};
long tries = 0;

int allowed(int row, int col, int digit) {
    int idx, boxRow = row / 3 * 3, boxCol = col / 3 * 3;
    for (idx = 0; idx < 9; idx++) {
        if (grid[row][idx] == digit || grid[idx][col] == digit) return 0;
        if (grid[boxRow + idx / 3][boxCol + idx % 3] == digit) return 0;   // 3x3 المربع
    }
    return 1;
}

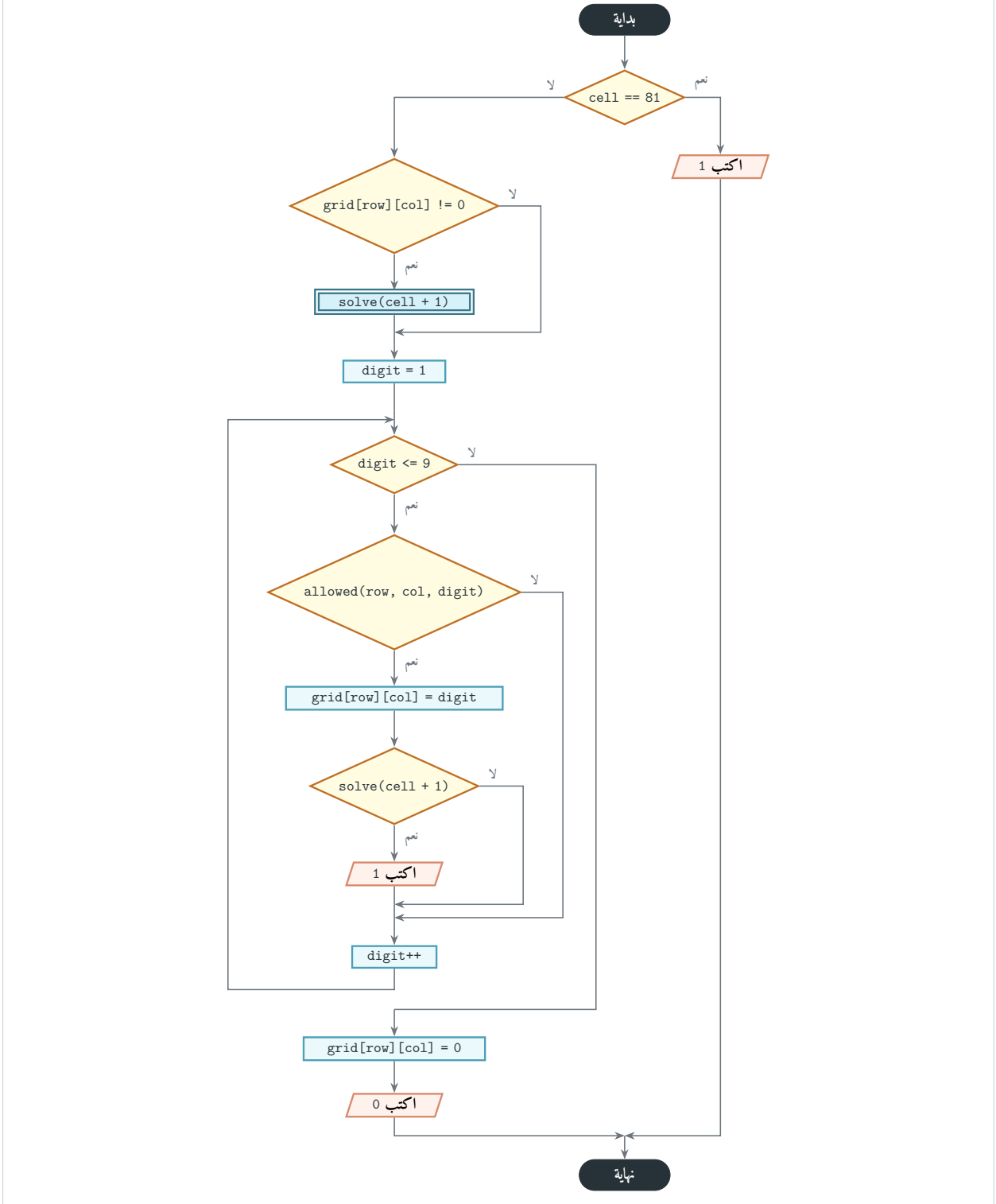
int solve(int cell) {
    int row = cell / 9, col = cell % 9, digit;
    if (cell == 81) return 1;                          // كل الخانات مملوءة
    if (grid[row][col] != 0) return solve(cell + 1);
    for (digit = 1; digit <= 9; digit++)
        if (allowed(row, col, digit)) {
            grid[row][col] = digit;
            tries++;
            if (solve(cell + 1)) return 1;
        }
    grid[row][col] = 0;                                // تراجع
    return 0;
}

int main(void) {
    int row, col;
    if (solve(0)) {
        for (row = 0; row < 9; row++) {
            for (col = 0; col < 9; col++)
                printf("%d%s", grid[row][col], col % 3 == 2 && col < 8 ? " | " : " ");
            printf("\n");
            if (row % 3 == 2 && row < 8) printf("------+-------+------\n");
        }
        printf("Solved with %ld placements\n", tries);
    }
    return 0;
}
```

نتيجة التنفيذ

```
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
------+-------+------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
------+-------+------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
Solved with 4208 placements
```

المخطط الإنسيابي (Organigramme)



تمرين 8.45 محلل صيغ الأسعار بالأقواس

تجارة إلكترونية

صعب ★★★

وسّع حاسبة صيغ الأسعار لتقبل **الأقواس** والأعداد العشرية: `(1200+800)*1.19-(2*150)/3`. اكتب **محللًا تنازليًا عوديًا** من ثلاث دوال متبادلة: `expression` (جمع وطرح)، `term` (ضرب وقسمة)، `factor` (عدد أو عبارة بين قوسين).

خطوات الحل

1) `expression := term (('+'|'-') term)*`.
2) `term := factor (('*'|'/') factor)*`.
3) `factor := number | '(' expression ')'`: هنا تحدث العودية غير المباشرة.
4) موضع القراءة متغير عام `pos` تتقدم فيه كل الدوال.

</> البرنامج بلغة C

```c
#include <stdio.h>

char formula[100];
int pos = 0;

double expression(void);

double factor(void) {
    double value = 0, scale = 1;
    if (formula[pos] == '(') {
        pos++;                                  // (
        value = expression();                   // عودية غير مباشرة
        pos++;                                  // )
        return value;
    }
    while (formula[pos] >= '0' && formula[pos] <= '9')
        value = value * 10 + (formula[pos++] - '0');
    if (formula[pos] == '.') {
        pos++;
        while (formula[pos] >= '0' && formula[pos] <= '9') {
            scale /= 10;
            value += (formula[pos++] - '0') * scale;
        }
    }
    return value;
}

double term(void) {
    double value = factor();
    while (formula[pos] == '*' || formula[pos] == '/') {
        if (formula[pos++] == '*') value *= factor();
        else value /= factor();
    }
    return value;
}

double expression(void) {
    double value = term();
    while (formula[pos] == '+' || formula[pos] == '-') {
        if (formula[pos++] == '+') value += term();
        else value -= term();
    }
    return value;
}

int main(void) {
    scanf("%99s", formula);
    printf("%s = %.2f DA\n", formula, expression());
    return 0;
```

```
50 }
```

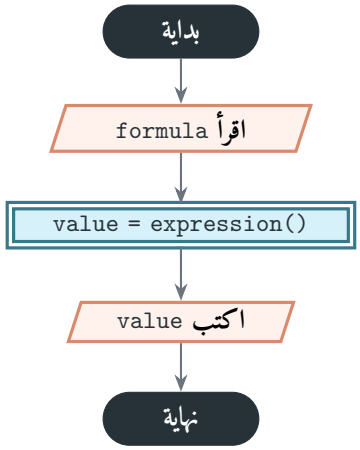
المدخلات

```
(1200+800)*1.19-(2*150)/3
```

نتيجة التنفيذ

```
(1200+800)*1.19-(2*150)/3 = 2280.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 8.46 أطول اتجاه صاعد في الأسعار

علم البيانات صعب ★★★

لتحليل اتجاه سعر منتج، نبحث عن **أطول متتالية جزئية متزايدة تمامًا** (ليست بالضرورة متجاورة) في أسعاره الأسبوعية. اكتب دالة عودية بالتذكر `lisFrom(idx)` = طول أطول متتالية متزايدة تبدأ من `idx`، ثم أعد بناء المتتالية.

خطوات الحل

1) `lisFrom(idx) = 1 + max(lisFrom(next))` على كل `next > idx` حيث `prices[next] > prices[idx]`.
2) `memo[idx]` و `nextOf[idx]` (العنصر التالي في أفضل متتالية) لإعادة البناء.
3) الجواب = أكبر `lisFrom(idx)` على كل نقاط البداية.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 14

int prices[NB] = {520, 480, 510, 530, 470, 545, 560, 500, 575, 590, 540, 600, 580, 620};
int memo[NB], nextOf[NB];

int lisFrom(int idx) {
    int next, len, best = 1;
    if (memo[idx] > 0) return memo[idx];
    nextOf[idx] = -1;
    for (next = idx + 1; next < NB; next++)
        if (prices[next] > prices[idx]) {
```

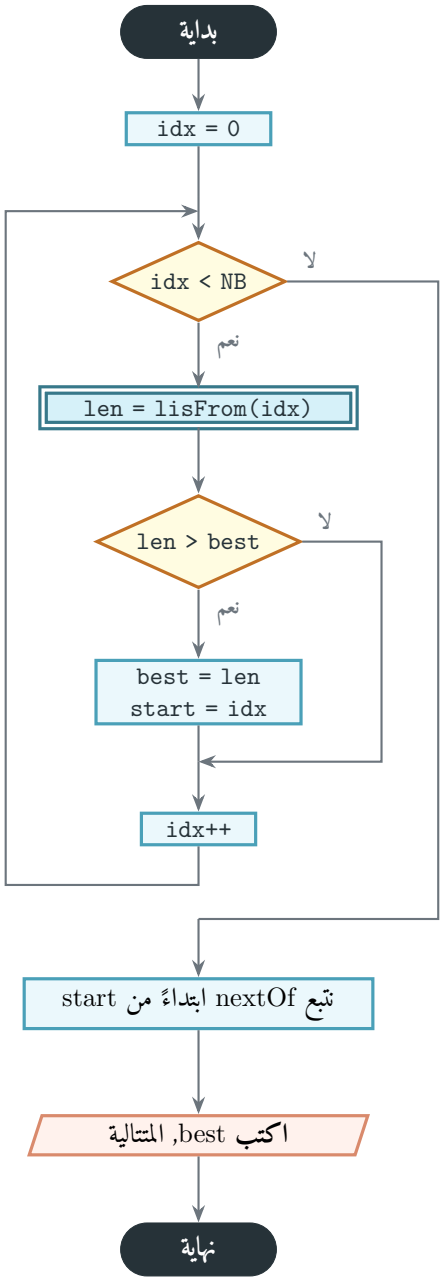
```
            len = 1 + lisFrom(next);
            if (len > best) { best = len; nextOf[idx] = next; }
        }
    memo[idx] = best;
    return best;
}

int main(void) {
    int idx, best = 0, start = 0, len;
    for (idx = 0; idx < NB; idx++) {
        len = lisFrom(idx);
        if (len > best) { best = len; start = idx; }
    }
    printf("Longest rising trend: %d weeks\n", best);
    for (idx = start; idx != -1; idx = nextOf[idx])     // إعادة البناء
        printf("w%d:%d ", idx + 1, prices[idx]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
Longest rising trend: 9 weeks
w2:480 w3:510 w4:530 w6:545 w7:560 w9:575 w10:590 w12:600 w14:620
```

المخطط الإنسيابي (Organigramme)



تمرين 8.47 الإختيار السريع لوسيط الأسعار

علم البيانات

صعب

لإيجاد العنصر رقم **k** (مثل الوسيط) لا نحتاج إلى ترتيب الجدول كله. اكتب دالة عودية `quickSelect(values, low, high, rank)`: نقسم حول محور كما في الترتيب السريع، ثم نستدعي الدالة على **جهة واحدة فقط** حسب موضع المحور.

خطوات الحل

1) التقسيم (لوموتو): المحور آخر عنصر؛ نضع كل الأصغر منه على اليسار، ثم نضع المحور في موضعه النهائي `pivotPos`.
2) إذا `rank == pivotPos` وجدناه؛ إذا كان أصغر نبحث يسارًا، وإلا يمينًا.

3) التعقيد المتوسط خطي.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 15

int calls = 0;

int partition(double values[], int low, int high) {
    double pivot = values[high], tmp;
    int store = low, idx;
    for (idx = low; idx < high; idx++)
        if (values[idx] < pivot) {
            tmp = values[idx]; values[idx] = values[store]; values[store] = tmp;
            store++;
        }
    tmp = values[store]; values[store] = values[high]; values[high] = tmp;
    return store;                                   // الموضع النهائي للمحور
}

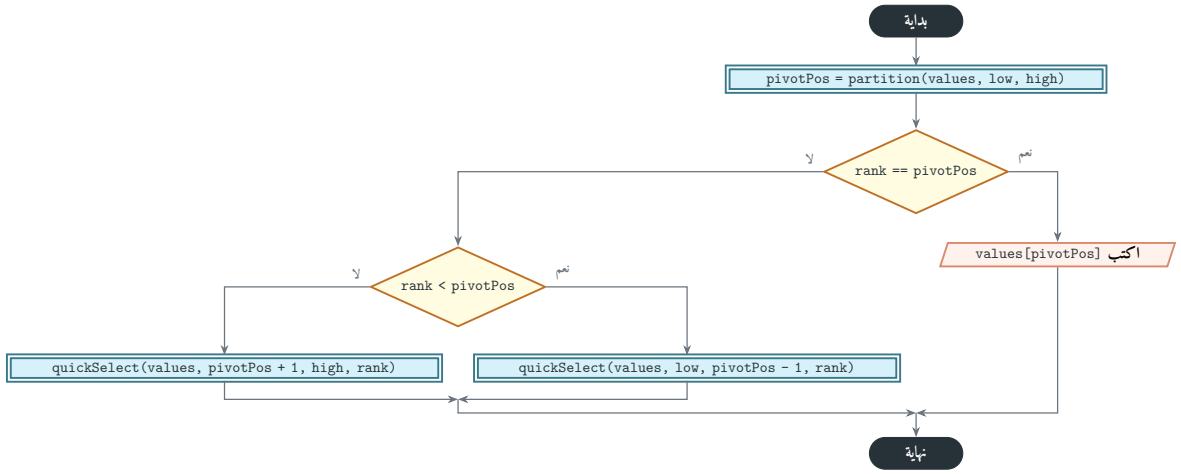
double quickSelect(double values[], int low, int high, int rank) {
    int pivotPos;
    calls++;
    if (low == high) return values[low];
    pivotPos = partition(values, low, high);
    if (rank == pivotPos) return values[pivotPos];
    if (rank < pivotPos) return quickSelect(values, low, pivotPos - 1, rank);
    return quickSelect(values, pivotPos + 1, high, rank);   // جهة واحدة فقط
}

int main(void) {
    double prices[NB] = {32000, 29500, 31000, 89000, 30500, 28900, 33000, 27500,
                         35000, 30000, 29900, 31500, 120000, 30800, 28000};
    double median, low, high;
    median = quickSelect(prices, 0, NB - 1, NB / 2);
    printf("Median price: %.0f DA (%d calls)\n", median, calls);
    low = quickSelect(prices, 0, NB - 1, 3);
    high = quickSelect(prices, 0, NB - 1, NB - 4);
    printf("4th lowest: %.0f, 4th highest: %.0f\n", low, high);
    return 0;
}
```

نتيجة التنفيذ

```
Median price: 30800 DA (4 calls)
4th lowest: 29500, 4th highest: 33000
```

المخطط الإنسيابي (Organigramme)



تمرين 8.48 أقرب مستودعين بطريقة فرّق تسد

تجارة إلكترونية ★★★ صعب

لشركة التوصيل 12 مستودعًا بإحداثيات (كم). اكتب دالة عودية `closest(low, high)` تجد **أقرب زوج من المستودعين** بطريقة فرّق تسد: نرتب حسب x، نقسم إلى نصفين، نحل كل نصف، ثم نفحص الشريط الضيق حول خط القسمة.

خطوات الحل

1) الحالة الأساسية: 3 نقاط أو أقل ← نقارن كل الأزواج.
2) `best = min(closest( ), closest( ))`.
3) في الشريط `|x - xMiddle| < best` نقارن كل نقطة بالنقاط القريبة منها في y.
4) نقارن مربعات المسافات، ونأخذ الجذر في النهاية بطريقة نيوتن.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12

double xs[NB] = {2, 15, 7, 31, 22, 9, 40, 18, 27, 4, 35, 12};
double ys[NB] = {3, 8, 14, 5, 19, 2, 11, 25, 9, 20, 16, 13};
int pairA, pairB;

double dist2(int first, int second) {
    return (xs[first] - xs[second]) * (xs[first] - xs[second])
         + (ys[first] - ys[second]) * (ys[first] - ys[second]);
}

double check(int first, int second, double best) {
    double value = dist2(first, second);
    if (value < best) { pairA = first; pairB = second; return value; }
    return best;
}

double closest(int low, int high) {
    double best = 1e18, right, gap;
    int first, second, middle;
    if (high - low < 3) {                                  // حالة أساسية صغيرة
```

```
        for (first = low; first <= high; first++)
            for (second = first + 1; second <= high; second++)
                best = check(first, second, best);
        return best;
    }
    middle = (low + high) / 2;
    best = closest(low, middle);
    right = closest(middle + 1, high);
    if (right < best) best = right;
    for (first = low; first <= high; first++) {              // الشريط حول خط القسمة
        gap = xs[first] - xs[middle];
        if (gap * gap >= best) continue;
        for (second = first + 1; second <= high; second++) {
            gap = xs[second] - xs[middle];
            if (gap * gap < best) best = check(first, second, best);
        }
    }
    return best;
}

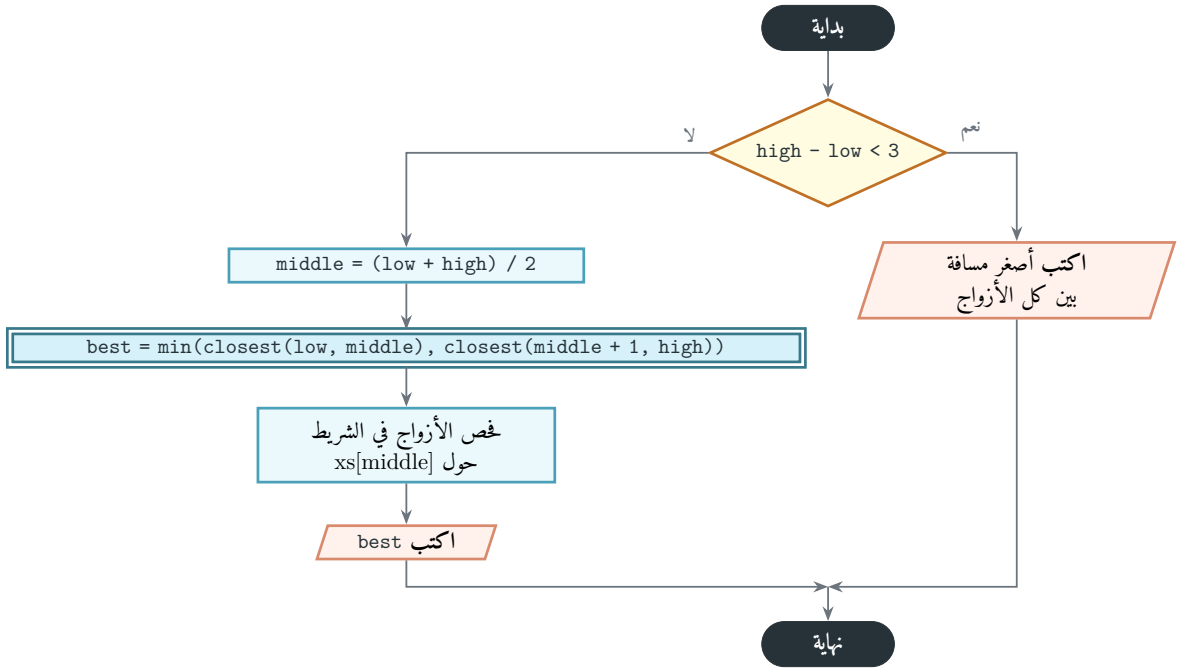
double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    for (iter = 0; iter < 50; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

int main(void) {
    int idx, back;
    double keyX, keyY, best;
    for (idx = 1; idx < NB; idx++) {                        // ترتيب حسب x
        keyX = xs[idx]; keyY = ys[idx]; back = idx - 1;
        while (back >= 0 && xs[back] > keyX) {
            xs[back + 1] = xs[back]; ys[back + 1] = ys[back]; back--;
        }
        xs[back + 1] = keyX; ys[back + 1] = keyY;
    }
    best = closest(0, NB - 1);
    printf("Closest warehouses: (%.0f,%.0f) and (%.0f,%.0f)\n",
           xs[pairA], ys[pairA], xs[pairB], ys[pairB]);
    printf("Distance: %.3f km\n", mySqrt(best));
    return 0;
}
```

نتيجة التنفيذ

```
Closest warehouses: (27,9) and (31,5)
Distance: 5.099 km
```

المخطط الإنسيابي (Organigramme)



تمرين 8.49 جولة التوصيل المثلى بالتفرع والتحديد

تجارة إلكترونية صعب ★★★

يزور السائق 6 مدن انطلاقًا من المستودع (المدينة 0) ثم يعود. تجربة كل الترتيبات (720) مكلفة. اكتب دالة عودية `explore(city, visitedCount, length)` بطريقة **التفرع والتحديد**: نتوقف عن توسيع أي جولة جزئية طولها الحالي أفضل جولة وجدناها.

خطوات الحل

1) `visited[]` يعلّم المدن المزارة، و `path[]` يحفظ الترتيب الحالي.
2) عند زيارة كل المدن: نضيف طريق العودة ونقارن بالأفضل.
3) **التحديد**: `if (length >= best) return;` يقطع فروعًا كاملة.
4) نعد العقد المزارة لقياس فائدة التقليم.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 7

int dist[NB][NB] = {{0, 29, 20, 21, 16, 31, 100},
                    {29, 0, 15, 29, 28, 40, 72},
                    {20, 15, 0, 15, 14, 25, 81},
                    {21, 29, 15, 0, 4, 12, 92},
                    {16, 28, 14, 4, 0, 16, 94},
                    {31, 40, 25, 12, 16, 0, 95},
                    {100, 72, 81, 92, 94, 95, 0}};
int visited[NB], path[NB], bestPath[NB], best = 1000000;
long nodes = 0;

```

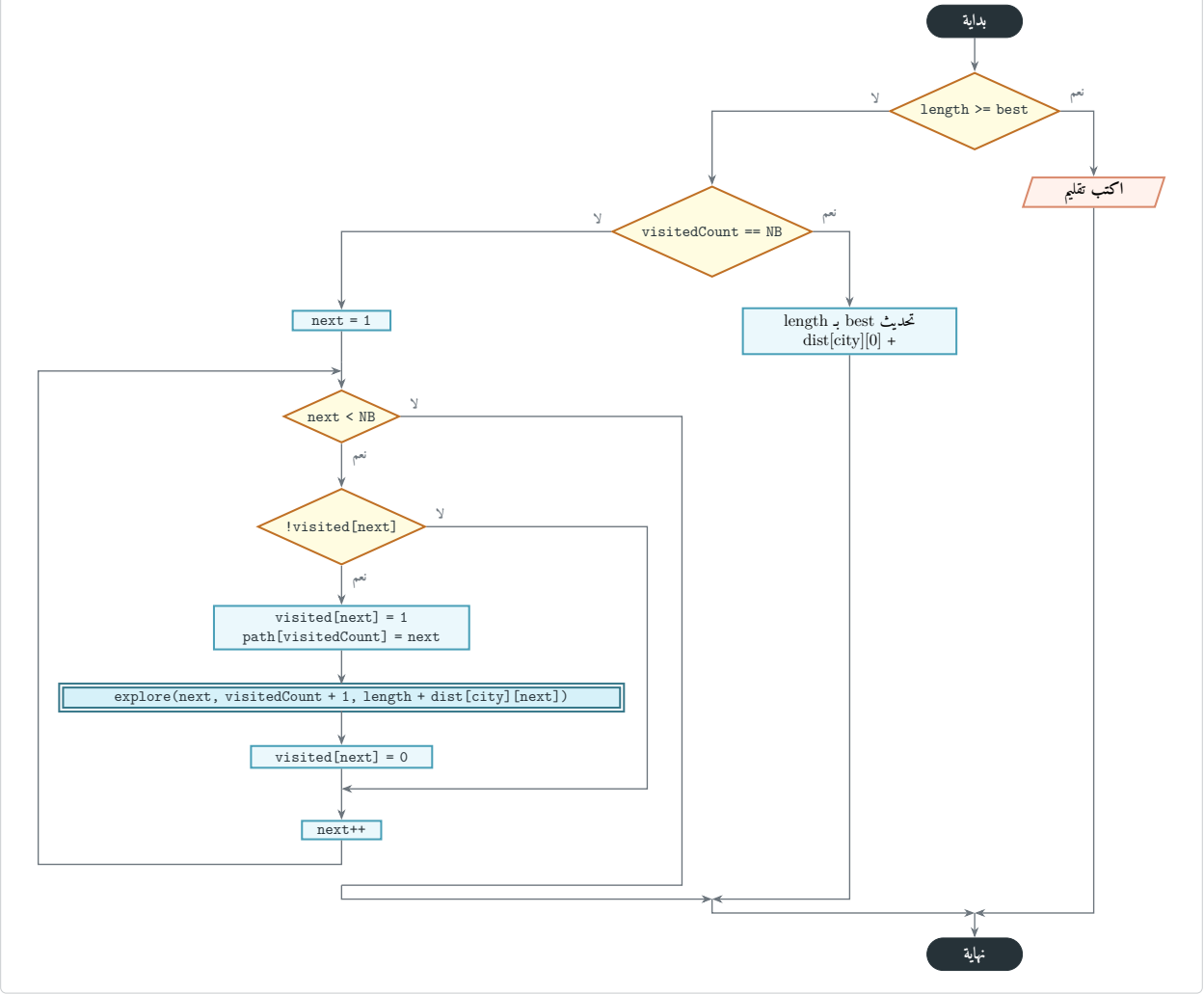
```c
void explore(int city, int visitedCount, int length) {
    int next, idx;
    nodes++;
    if (length >= best) return;                          // تحديد: لا فائدة من المواصلة
    if (visitedCount == NB) {
        if (length + dist[city][0] < best) {
            best = length + dist[city][0];
            for (idx = 0; idx < NB; idx++) bestPath[idx] = path[idx];
        }
        return;
    }
    for (next = 1; next < NB; next++)
        if (!visited[next]) {
            visited[next] = 1;
            path[visitedCount] = next;
            explore(next, visitedCount + 1, length + dist[city][next]);
            visited[next] = 0;                           // تراجع
        }
}

int main(void) {
    int idx;
    path[0] = 0;
    visited[0] = 1;
    explore(0, 1, 0);
    printf("Best tour:");
    for (idx = 0; idx < NB; idx++) printf(" %d ->", bestPath[idx]);
    printf(" 0\nLength: %d km\n", best);
    printf("Nodes explored: %ld (full tree: 1957)\n", nodes);
    return 0;
}
```

نتيجة التنفيذ

```
Best tour: 0 -> 2 -> 1 -> 6 -> 5 -> 3 -> 4 -> 0
Length: 234 km
Nodes explored: 1624 (full tree: 1957)
```

المخطط الإنسيابي (Organigramme)



تمرين 8.50 تقطيع الوسم إلى كلمات من القاموس

علم البيانات صعب ★★★

وسوم الحملات تُكتب دون فراغات مثل `#freeshippingforsummersale`. اكتب دالة عودية بالتذكر `canSplit(start)` تحدد إن كان الوسم يمكن تقطيعه إلى كلمات من قاموس، وتعيد بناء التقطيع، وتعد عدد التقطيعات الممكنة.

خطوات الحل

1) `canSplit(start)`: نجرب كل كلمة من القاموس تطابق النص ابتداءً من `start`، ثم نستدعي `canSplit(start + )`.
2) الحالة الأساسية: `start == length` ⟹ نجاح.
3) `ways[start]` يحفظ عدد التقطيعات من `start` (-1 = غير محسوب)، و `choice[start]` أول كلمة صالحة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBW 12

```

```c
char dict[NBW][10] = {"free", "shipping", "ship", "ping", "for", "summer", "sale",
                      "sum", "mer", "fors", "um", "s"};
char tag[60];
int length, choice[60];
long ways[60];

int matchAt(int start, int word) {
    int pos = 0;
    while (dict[word][pos] != '\0') {
        if (tag[start + pos] != dict[word][pos]) return 0;
        pos++;
    }
    return pos;                                          // طول الكلمة المطابقة
}

long countWays(int start) {
    int word, len;
    long total = 0, sub;
    if (start == length) return 1;
    if (ways[start] >= 0) return ways[start];
    choice[start] = -1;
    for (word = 0; word < NBW; word++) {
        len = matchAt(start, word);
        if (len > 0) {
            sub = countWays(start + len);
            if (sub > 0 && choice[start] < 0) choice[start] = word;
            total += sub;
        }
    }
    ways[start] = total;
    return total;
}

int main(void) {
    int start;
    long total;
    scanf("%59s", tag);
    for (length = 0; tag[length] != '\0'; length++) ways[length] = -1;
    total = countWays(0);
    printf("#%s: %ld possible splits\n", tag, total);
    if (total > 0) {
        printf("First split:");
        for (start = 0; start < length; start += matchAt(start, choice[start]))
            printf(" %s", dict[choice[start]]);
        printf("\n");
    }
    return 0;
}
```

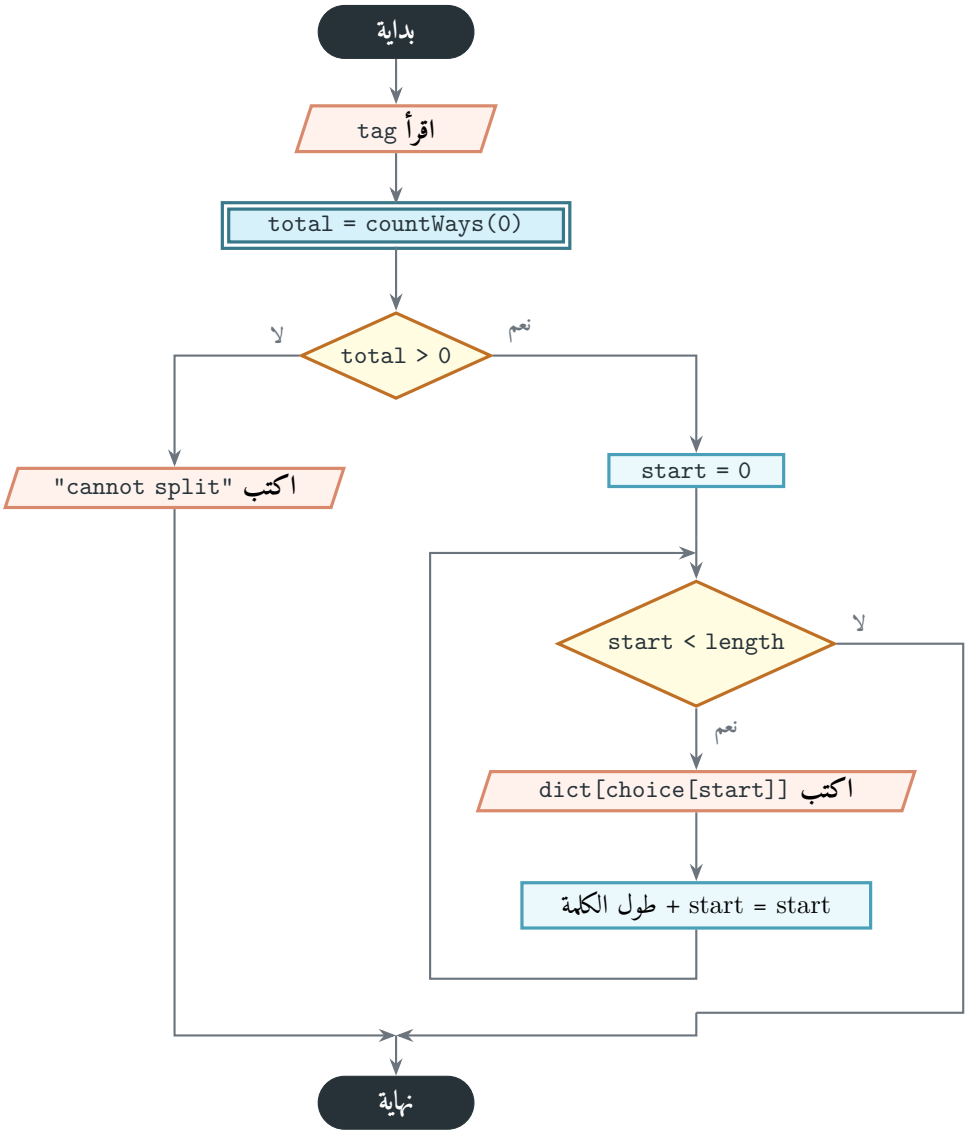
المدخلات

```
freeshippingforsummersale
```

نتيجة التنفيذ

```
#freeshippingforsummersale: 8 possible splits
First split: free shipping for summer sale
```

المخطط الإنسيابي (Organigramme)

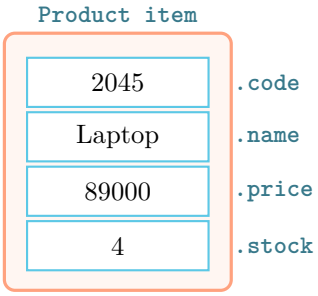


الفصل 9

الهياكل

تعريفات

- **الهيكل** `struct`: نوع جديد يجمع **حقولًا** من أنواع مختلفة تصف كيانًا واحدًا (منتج، زبون، طلبية).
- **التعريف**: `typedef struct { int code; double price; } Product;`
- **الوصول إلى الحقل** بالنقطة: `item.price`، وفي هيكل متداخل: `order.client.name`.
- **الإسناد** `copy = item;` ينسخ كل الحقول، بما فيها الجداول الداخلية.
- **التعداد** `enum`: أسماء لقيم صحيحة ثابتة (`PENDING`، `PAID`، ...).



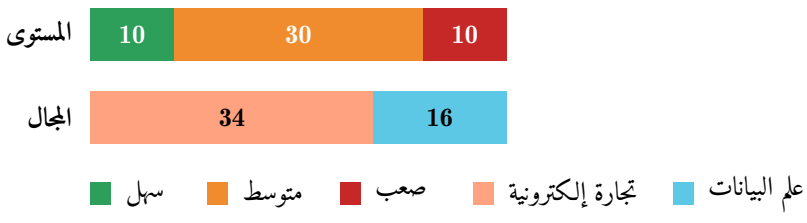
الاستعمال	مثال
جدول من الهياكل	`Product items[50];` ثم `items[idx].price`
تمرير إلى دالة	بالقيمة (نسخة)، لتعديله نمرر جدولًا من خانة واحدة `Cart cart[]`
إعادة عدة نتائج	الدالة تعيد هيكلًا: `Summary describe(...)`
بنى المعطيات	مكدس، طابور، كومة، جدول تجزئة: هيكل = جدول + عدادات

المقارنة غير مباشرة

لا يمكن مقارنة هيكلين بـ `==`: نقارن الحقول واحدًا واحدًا، أو نكتب دالة مقارنة مثل `compareDates(first, second)`.

للاستزادة: [2, 3, 11, 13]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 9.1 هيكل المنتج

تجارة إلكترونية سهل

عرّف هيكلًا `struct Product` يجمع رمز المنتج واسمه وسعره وكميته في المخزون، ثم أنشئ منتجًا مهيأً واعرض بطاقته وقيمة مخزونه.

خطوات الحل

1) الهيكل يجمع حقولًا من أنواع مختلفة تحت اسم واحد.
2) نصل إلى الحقل بالنقطة: `item.price`.

البرنامج بلغة C

```c
#include <stdio.h>

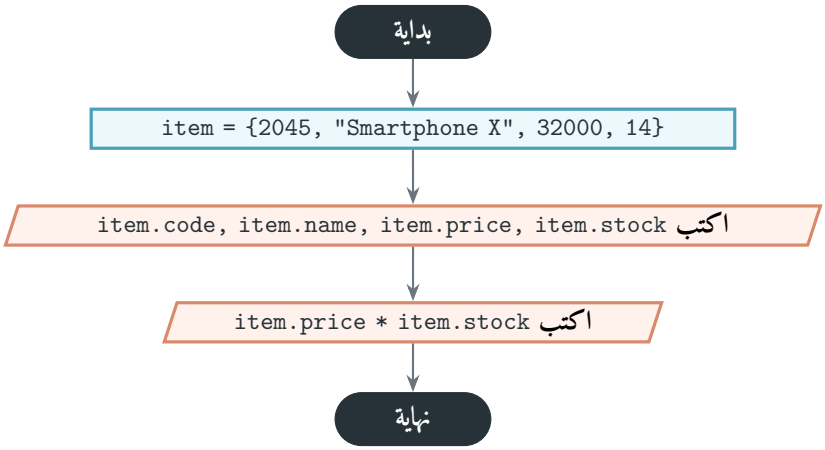
struct Product {
    int code;
    char name[30];
    double price;
    int stock;
};

int main(void) {
    struct Product item = {2045, "Smartphone X", 32000, 14};   // تهيئة بالترتيب
    printf("Code : %d\n", item.code);
    printf("Name : %s\n", item.name);
    printf("Price: %.2f DA\n", item.price);
    printf("Stock: %d units\n", item.stock);
    printf("Stock value: %.2f DA\n", item.price * item.stock);
    return 0;
}
```

نتيجة التنفيذ

```
Code : 2045
Name : Smartphone X
Price: 32000.00 DA
Stock: 14 units
Stock value: 448000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.2 قراءة بيانات زبون

تجارة إلكترونية
سهل

عرّف بـ typedef نوعًا Customer (المعرف، الاسم، المدينة، عدد الطلبيات)، ثم اقرأ زبونًا من لوحة المفاتيح واعرض بطاقته مع فئته: "وفي" إذا تجاوز 10 طلبيات.

خطوات الحل

1) typedef struct {...} Customer; يسمح بكتابة Customer بدل struct ....

2) scanf يحتاج إلى عنوان الحقل: &client.id، أما الجدول client.name فهو عنوان.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    int id;
    char name[30];
    char city[20];
    int orders;
} Customer;

int main(void) {
    Customer client;
    scanf("%d %29s %19s %d", &client.id, client.name, client.city, &client.orders);
    printf("#%d %s (%s): %d orders -> %s\n", client.id, client.name, client.city,
           client.orders, client.orders > 10 ? "loyal" : "regular");
    return 0;
}
```

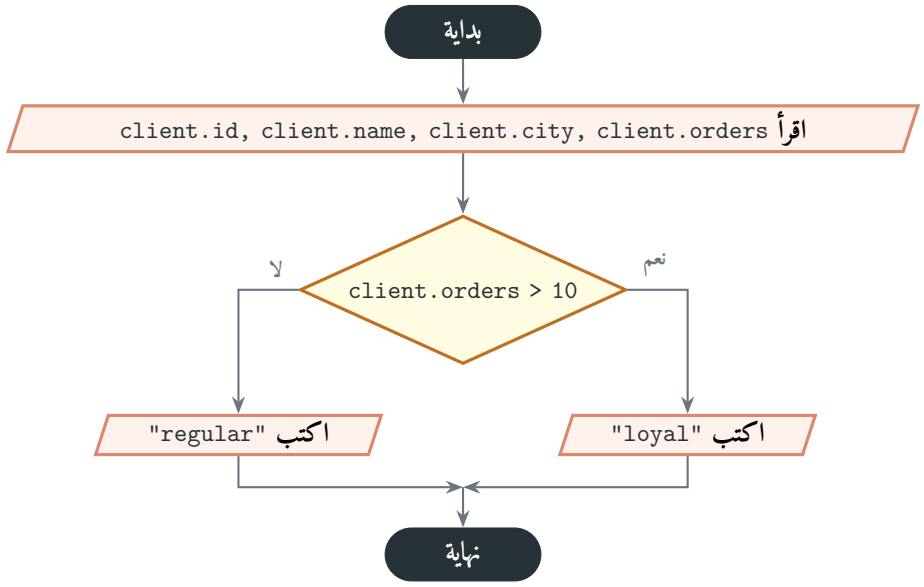
المدخلات

```
507 Amina Oran 12
```

نتيجة التنفيذ

```
#507 Amina (Oran): 12 orders -> loyal
```

المخطط الإنسيابي (Organigramme)



تمرين 9.3 مقارنة تاريخي طلبيتين

تجارة إلكترونية سهل

عرّف هيكلًا Date (اليوم، الشهر، السنة) واكتب دالة compareDates(first, second) تعيد -1 أو 0 أو 1، ثم حدد أي الطلبيتين أقدم.

خطوات الحل

1) نقارن السنوات أولًا، ثم الأشهر، ثم الأيام.
2) الهيكل يُمرر إلى الدالة **بالقيمة** (نسخة).

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    int day, month, year;
} Date;

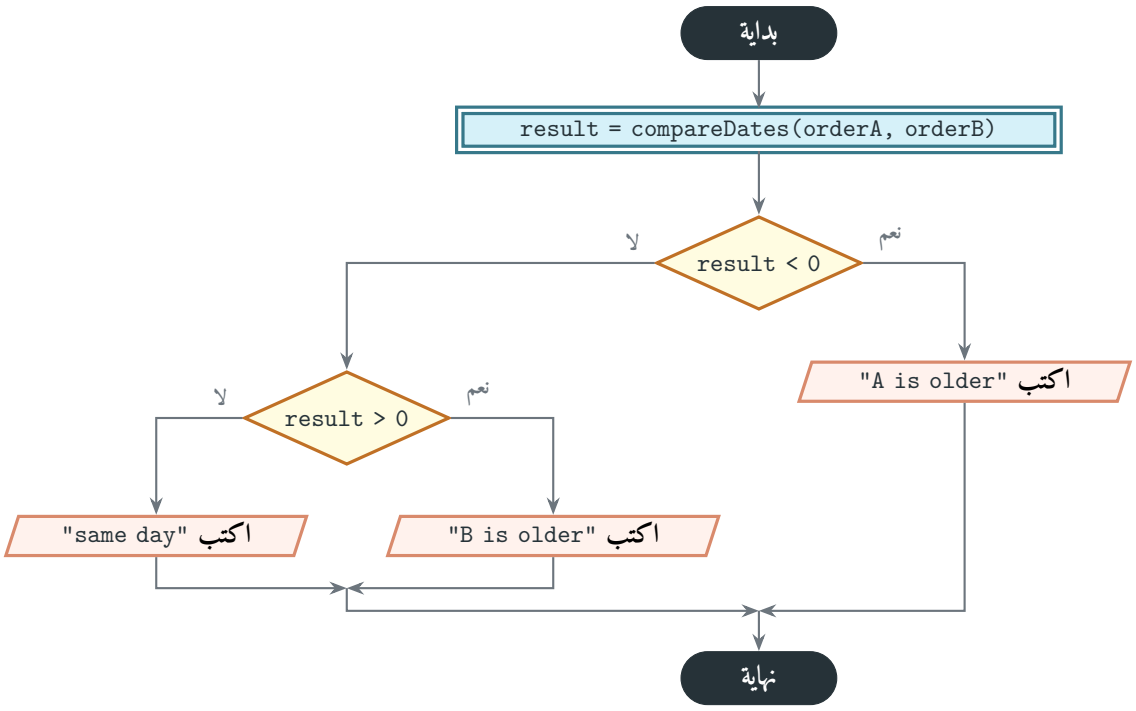
int compareDates(Date first, Date second) {
    if (first.year != second.year) return first.year < second.year ? -1 : 1;
    if (first.month != second.month) return first.month < second.month ? -1 : 1;
    if (first.day != second.day) return first.day < second.day ? -1 : 1;
    return 0;
}

int main(void) {
    Date orderA = {28, 10, 2026}, orderB = {3, 11, 2026};
    int result = compareDates(orderA, orderB);
    printf("A: %02d/%02d/%d  B: %02d/%02d/%d\n", orderA.day, orderA.month, orderA.year,
           orderB.day, orderB.month, orderB.year);
    if (result < 0) printf("Order A is older\n");
    else if (result > 0) printf("Order B is older\n");
    else printf("Same day\n");
    return 0;
}
```

نتيجة التنفيذ

```
A: 28/10/2026  B: 03/11/2026
Order A is older
```

المخطط الانسيابي (Organigramme)



تمرين 9.4 المسافة بين المتجر والزبون

تجارة إلكترونية

سهل

عرّف هيكلًا Point بإحداثيتين posX و posY (بالكم)، واكتب دالة manhattan(first, second) تحسب مسافة التنقل في شوارع متعامدة |dx| + |dy|، ثم احسب مسافة كل زبون عن المتجر.

خطوات الحل

1) القيمة المطلقة نكتبها بأنفسنا بالعامل الشرطي.
2) جدول من الهياكل: Point clients[3].

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    double posX, posY;
} Point;

double absValue(double value) { return value < 0 ? -value : value; }

double manhattan(Point first, Point second) {
    return absValue(first.posX - second.posX) + absValue(first.posY - second.posY);
}

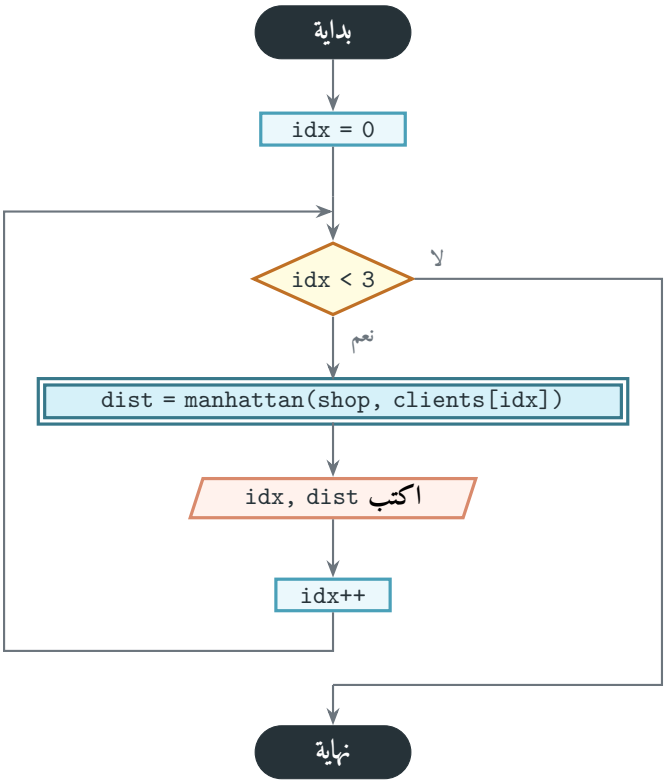
int main(void) {
    Point shop = {4, 3};
```

```c
    Point clients[3] = {{1, 7}, {9, 2}, {4, 10}};
    int idx;
    for (idx = 0; idx < 3; idx++)
        printf("Client %d at (%.0f,%.0f): %.0f km\n", idx + 1,
               clients[idx].posX, clients[idx].posY, manhattan(shop, clients[idx]));
    return 0;
}
```

نتيجة التنفيذ

```
Client 1 at (1,7): 7 km
Client 2 at (9,2): 6 km
Client 3 at (4,10): 7 km
```

المخطط الإنسيابي (Organigramme)



تمرين 9.5 مبلغ الطلبية من سطورها

تجارة إلكترونية

سهل

عرّف هيكلًا `OrderLine` (اسم المنتج، سعر الوحدة، الكمية) وجدولًا من 4 سطور، ثم احسب مبلغ كل سطر ومبلغ الطلبية الكلي.

خطوات الحل

1) `lines[idx].price * lines[idx].qty` لكل سطر.
2) نجمع في `total`.

البرنامج بلغة C

```c
#include <stdio.h>

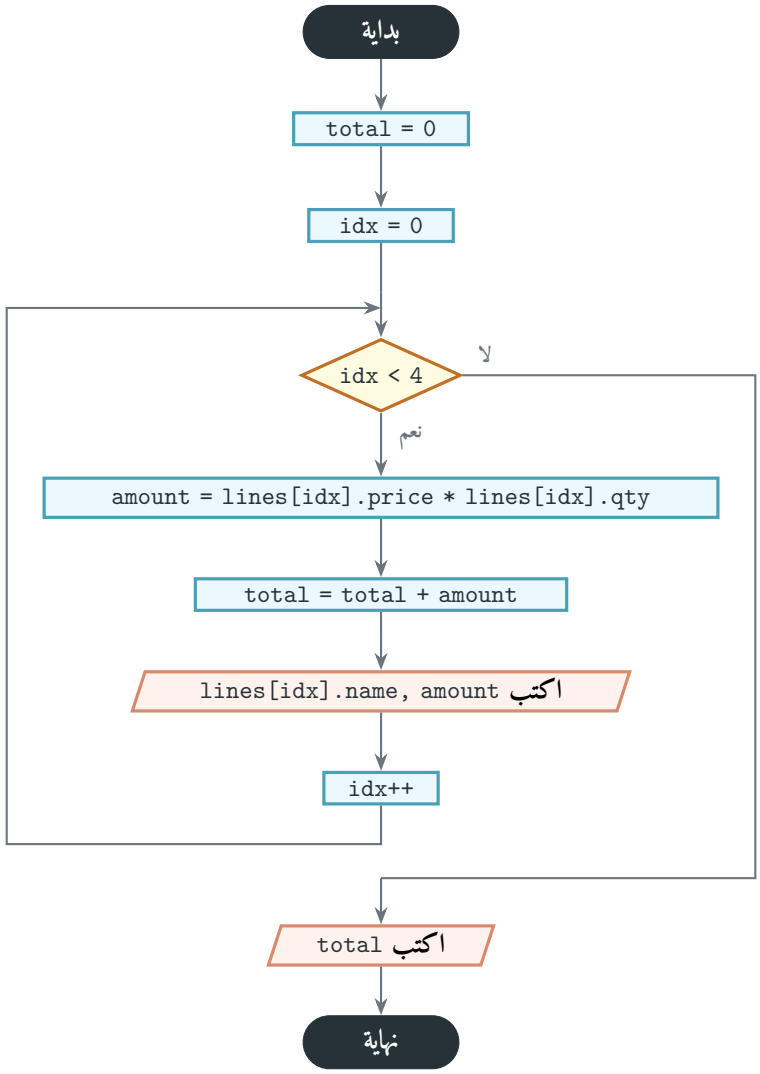
typedef struct {
    char name[20];
    double price;
    int qty;
} OrderLine;

int main(void) {
    OrderLine lines[4] = {{"USB cable", 450, 3}, {"Charger", 2200, 1},
                          {"Case", 900, 2}, {"Screen film", 350, 4}};
    double total = 0, amount;
    int idx;
    for (idx = 0; idx < 4; idx++) {
        amount = lines[idx].price * lines[idx].qty;
        total += amount;
        printf("%-12s %2d x %7.2f = %8.2f\n", lines[idx].name, lines[idx].qty,
               lines[idx].price, amount);
    }
    printf("TOTAL %30.2f DA\n", total);
    return 0;
}
```

نتيجة التنفيذ

```
USB cable     3 x  450.00 =  1350.00
Charger       1 x 2200.00 =  2200.00
Case          2 x  900.00 =  1800.00
Screen film   4 x  350.00 =  1400.00
TOTAL                        6750.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.6 قيمة المخزون الإجمالية

تجارة إلكترونية

سهل

لدينا جدول من 5 منتجات (هيكل بالاسم والسعر والمخزون). احسب قيمة المخزون الإجمالية، وعدد المنتجات النافدة (مخزون 0).

خطوات الحل

1) حلقة واحدة على جدول الهياكل.
2) نجمع `price * stock` ونعد الحالات `stock == 0`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char name[20];
    double price;
    int stock;
} Product;

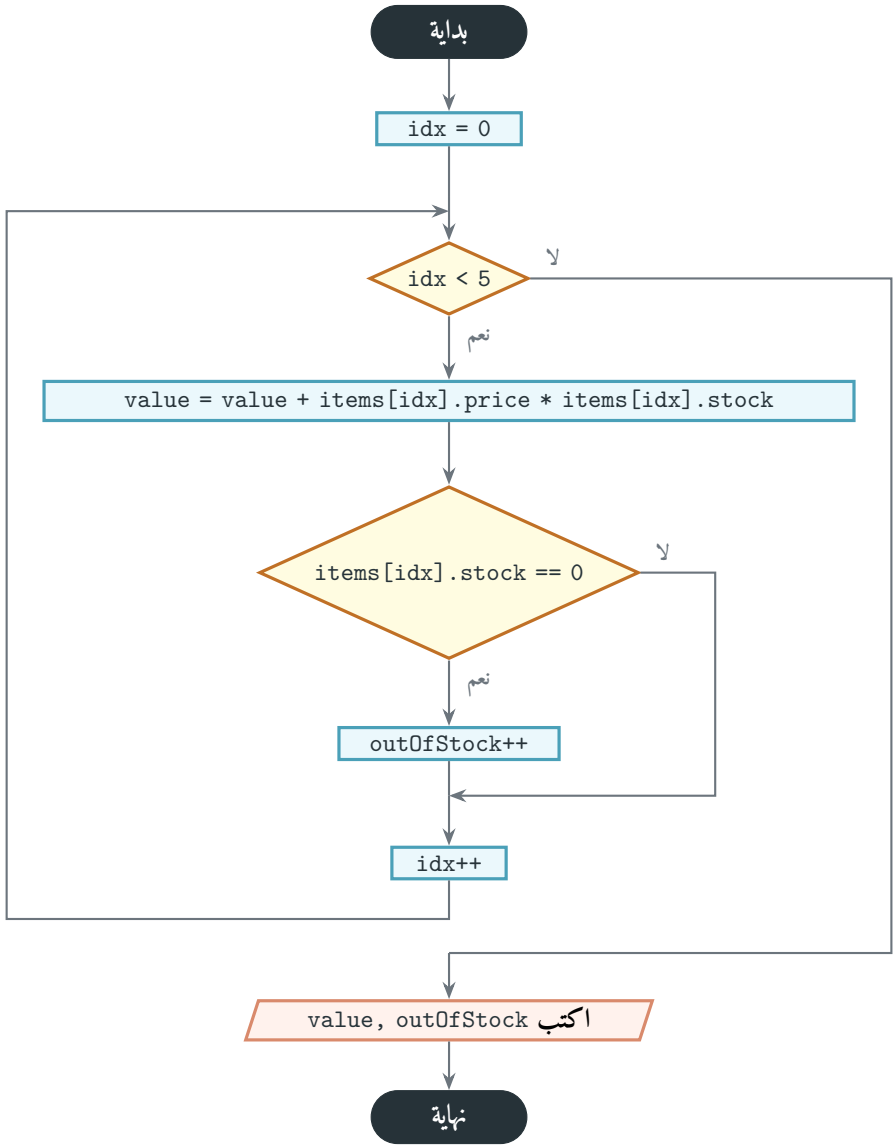
```

```c
int main(void) {
    Product items[5] = {{"Laptop", 89000, 4}, {"Mouse", 1500, 0}, {"Monitor", 32000, 6},
                        {"Keyboard", 3200, 15}, {"Webcam", 4100, 0}};
    double value = 0;
    int idx, outOfStock = 0;
    for (idx = 0; idx < 5; idx++) {
        value += items[idx].price * items[idx].stock;
        if (items[idx].stock == 0) {
            outOfStock++;
            printf("Out of stock: %s\n", items[idx].name);
        }
    }
    printf("Inventory value: %.2f DA\n", value);
    printf("%d product(s) to reorder\n", outOfStock);
    return 0;
}
```

نتيجة التنفيذ

```
Out of stock: Mouse
Out of stock: Webcam
Inventory value: 596000.00 DA
2 product(s) to reorder
```

المخطط الإنسيابي (Organigramme)



تمرين 9.7 دالة تعيد هيكلًا معدلًا

تجارة إلكترونية سهل

اكتب دالة `applyDiscount(item, percent)` تستقبل منتجًا (بالقيمة) وتعيد **نسخة** منه بسعر مخفض. اعرض المنتج الأصلي والنسخة للتأكد أن الأصل لم يتغير.

خطوات الحل

1) الهيكل يُمرر بالقيمة: التعديل داخل الدالة على نسخة.
2) الدالة تعيد هيكلًا كاملًا بـ `return`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
```

```c
    char name[20];
    double price;
} Product;

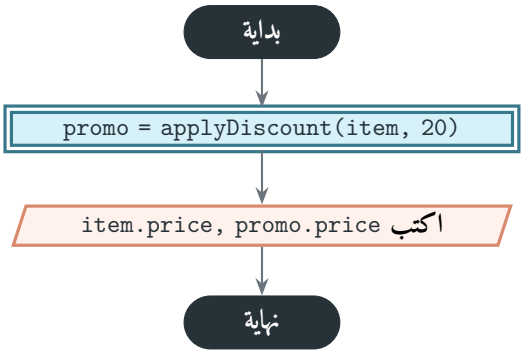
Product applyDiscount(Product item, double percent) {
    item.price = item.price * (100 - percent) / 100;   // نعدل النسخة فقط
    return item;
}

int main(void) {
    Product item = {"Headphones", 12500};
    Product promo = applyDiscount(item, 20);
    printf("Original: %s %.2f DA\n", item.name, item.price);
    printf("Promo:    %s %.2f DA\n", promo.name, promo.price);
    return 0;
}
```

نتيجة التنفيذ

```
Original: Headphones 12500.00 DA
Promo:    Headphones 10000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.8 متوسط تقييمات منتج

علم البيانات سهل

عرّف هيكلًا Review (اسم الزبون، النجوم من 1 إلى 5، هل الشراء موثق). احسب متوسط النجوم لكل التقييمات ومتوسط التقييمات الموثقة فقط.

خطوات الحل

1) حقل verified قيمته 0 أو 1.
2) مجموعان وعدادان.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char author[16];
    int stars;
    int verified;
} Review;

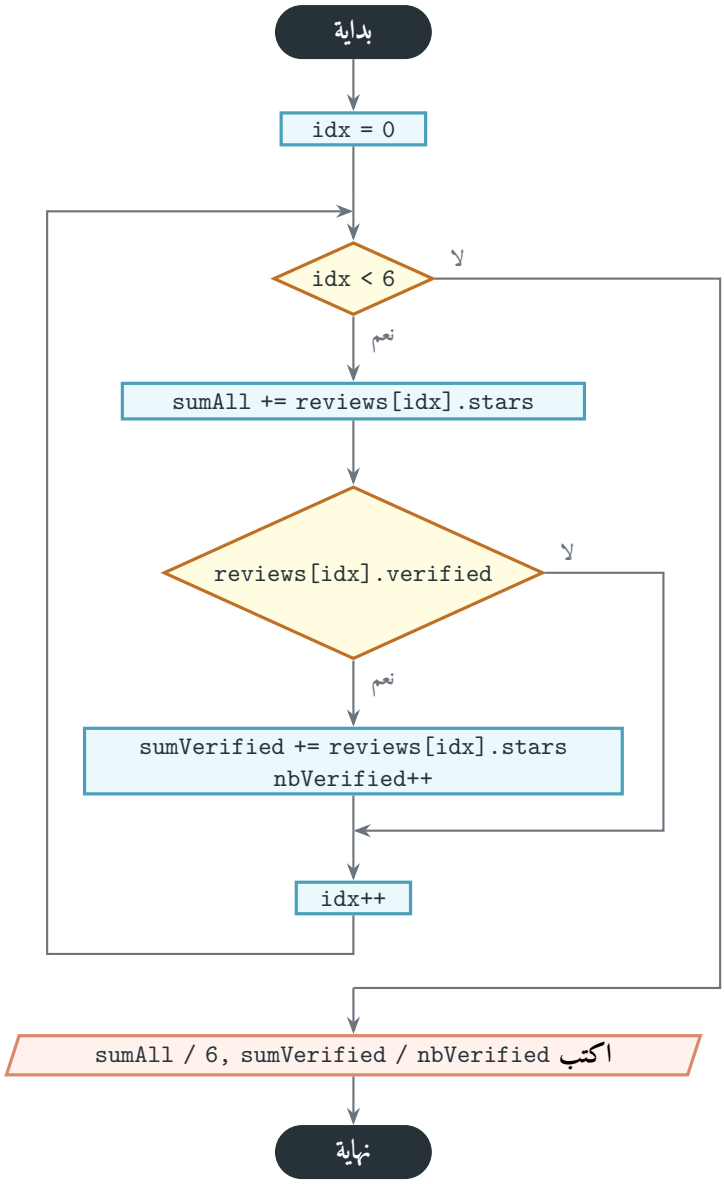
```

```c
int main(void) {
    Review reviews[6] = {{"Karim", 5, 1}, {"Lina", 1, 0}, {"Yacine", 4, 1},
                         {"Sara", 1, 0}, {"Nadir", 4, 1}, {"Meriem", 5, 1}};
    int idx, sumAll = 0, sumVerified = 0, nbVerified = 0;
    for (idx = 0; idx < 6; idx++) {
        sumAll += reviews[idx].stars;
        if (reviews[idx].verified) {            // شراء موثق فقط
            sumVerified += reviews[idx].stars;
            nbVerified++;
        }
    }
    printf("All reviews:      %.2f\n", sumAll / 6.0);
    printf("Verified reviews: %.2f (%d reviews)\n", (double)sumVerified / nbVerified, nbVerified);
    return 0;
}
```

نتيجة التنفيذ

```
All reviews:      3.33
Verified reviews: 4.50 (4 reviews)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.9 جمع مدتين بهيكل الوقت

علم البيانات ⭐ سهل

عرّف هيكلًا `Duration` (ساعات، دقائق، ثوان) ودالة `addDurations(first, second)` تعيد مجموعهما مع ترحيل الثواني والدقائق الزائدة، ثم احسب مدة تصفح زبون خلال ثلاث جلسات.

خطوات الحل

1) نجمع كل حقل، ثم: إذا تجاوزت الثواني 59 نرحّل `seconds / 60` إلى الدقائق، ونفس الشيء للدقائق.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    int hours, minutes, seconds;
} Duration;

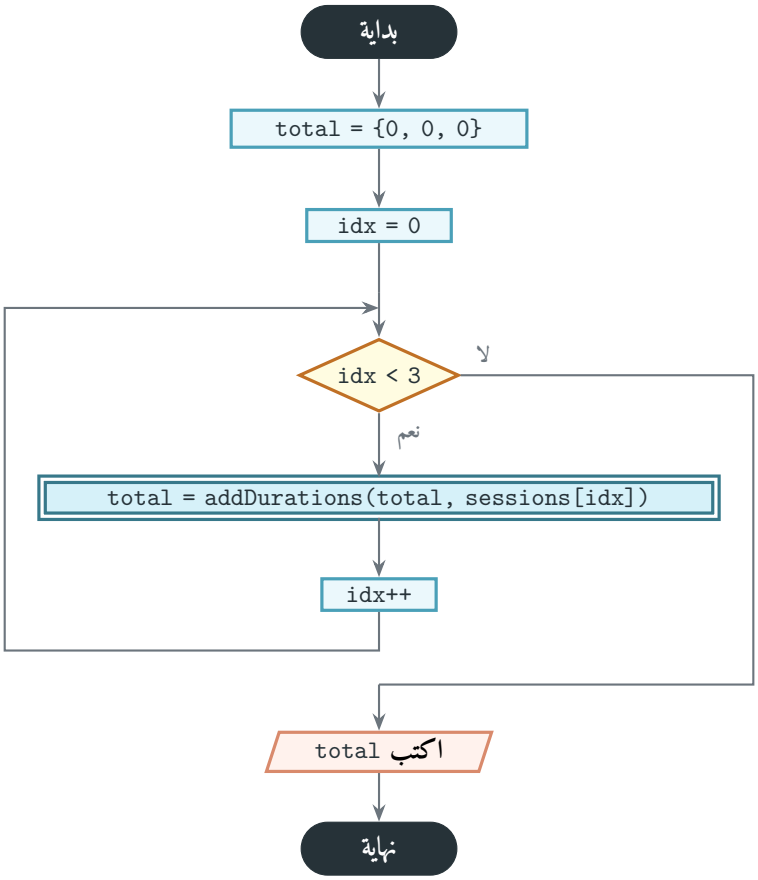
Duration addDurations(Duration first, Duration second) {
    Duration sum;
    sum.seconds = first.seconds + second.seconds;
    sum.minutes = first.minutes + second.minutes + sum.seconds / 60;   // ترحيل
    sum.seconds %= 60;
    sum.hours = first.hours + second.hours + sum.minutes / 60;
    sum.minutes %= 60;
    return sum;
}

int main(void) {
    Duration sessions[3] = {{0, 42, 35}, {1, 18, 50}, {0, 55, 47}};
    Duration total = {0, 0, 0};
    int idx;
    for (idx = 0; idx < 3; idx++)
        total = addDurations(total, sessions[idx]);
    printf("Total browsing time: %dh %02dm %02ds\n", total.hours, total.minutes, total.seconds);
    return 0;
}
```

نتيجة التنفيذ

```
Total browsing time: 2h 57m 12s
```

المخطط الإنسيابي (Organigramme)



تمرين 9.10 نسخ الهياكل بالإسناد

تجارة إلكترونية

سهل

وضّح أن الإسناد `backup = cart;` ينسخ **كل** حقول الهيكل، بما فيها الجداول الداخلية، على عكس الجداول العادية. عدّل السلة بعد النسخ وقارن.

خطوات الحل

1) الإسناد بين هيكلين من نفس النوع ينسخ الذاكرة كلها.
2) تعديل `cart` لا يؤثر على `backup`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    int items;
    int qty[5];
} Cart;

void show(char label[], Cart basket) {
    int idx;
    printf("%s %d items:", label, basket.items);
    for (idx = 0; idx < basket.items; idx++) printf(" %d", basket.qty[idx]);
    printf("\n");
```

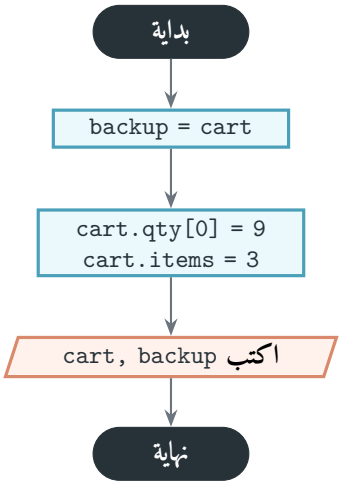
```
}

int main(void) {
    Cart cart = {2, {1, 4}};
    Cart backup = cart;                    // نسخة كاملة (بما فيها الجدول)
    cart.qty[0] = 9;
    cart.qty[2] = 2;
    cart.items = 3;
    show("cart:  ", cart);
    show("backup:", backup);
    return 0;
}
```

نتيجة التنفيذ

```
cart:   3 items: 9 4 2
backup: 2 items: 1 4
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 9.11 أرخص منتج في كل فئة

تجارة إلكترونية ★★ متوسط

لدينا 8 منتجات، لكل منتج اسم وفئة (`'P'` هواتف، `'L'` حواسيب، `'A'` ملحقات) وسعر. اكتب دالة `cheapestOf(items, size, category)` تعيد **فهرس** أرخص منتج في الفئة المعطاة، واعرض النتيجة لكل فئة.

خطوات الحل

1) `best = -1` في البداية؛ أول منتج من الفئة يصبح المرشح.
2) نقارن السعر فقط مع منتجات نفس الفئة.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char name[20];
    char category;
    double price;
} Product;

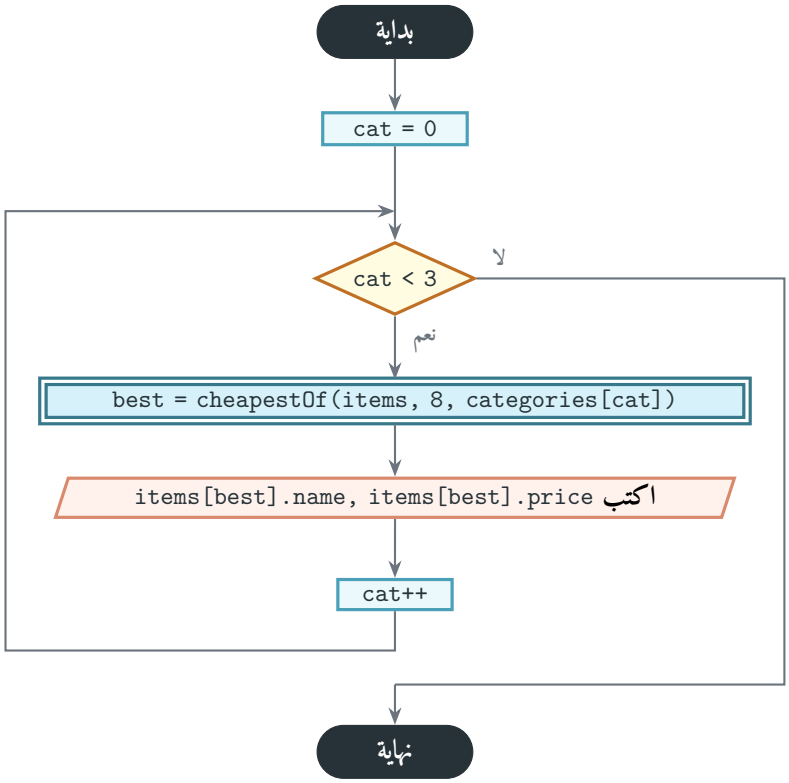
int cheapestOf(Product items[], int size, char category) {
    int idx, best = -1;
    for (idx = 0; idx < size; idx++)
        if (items[idx].category == category &&
            (best < 0 || items[idx].price < items[best].price))
            best = idx;
    return best;
}

int main(void) {
    Product items[8] = {{"Galaxy A", 'P', 38000}, {"ThinkPad", 'L', 95000},
                        {"Charger", 'A', 2200}, {"Redmi 13", 'P', 27500},
                        {"IdeaPad", 'L', 72000}, {"Cable", 'A', 450},
                        {"iPhone", 'P', 165000}, {"Hub", 'A', 3100}};
    char categories[3] = {'P', 'L', 'A'};
    int cat, best;
    for (cat = 0; cat < 3; cat++) {
        best = cheapestOf(items, 8, categories[cat]);
        printf("Category %c: %-9s %9.2f DA\n", categories[cat], items[best].name, items[best].price);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Category P: Redmi 13   27500.00 DA
Category L: IdeaPad    72000.00 DA
Category A: Cable        450.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.12 ترتيب المنتجات حسب السعر

متوسط ★★ تجارة إلكترونية

رتّب جدولًا من الهياكل `Product` تصاعديًا حسب السعر بالترتيب بالاختيار، مع تبديل **الهياكل كاملة** (لا الأسعار فقط)، ثم اعرض الكتالوج.

خطوات الحل

1) نبحث عن فهرس أصغر سعر في الجزء غير المرتب.
2) نبدل الهيكلين بمتغير مؤقت `Product tmp` (إسناد هياكل).

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6

typedef struct {
    int code;
    char name[20];
    double price;
} Product;

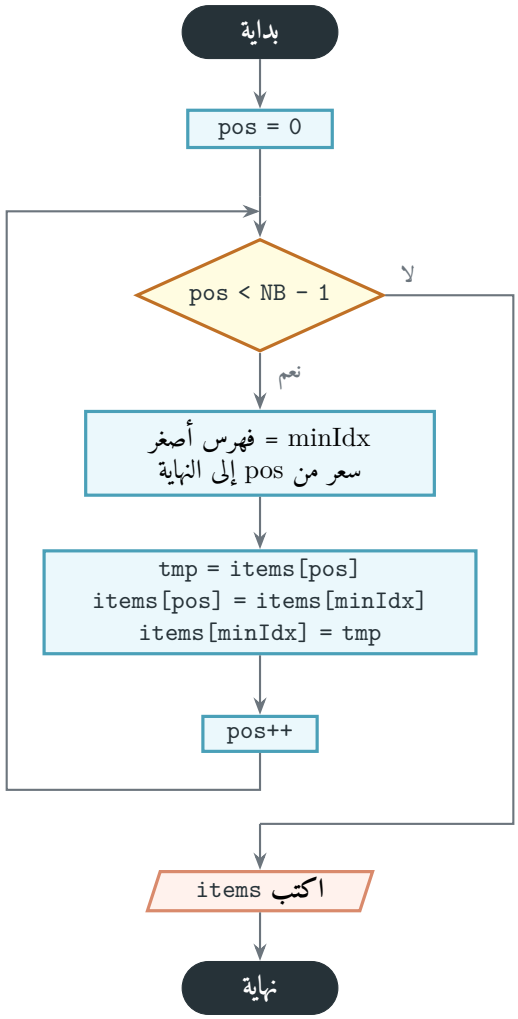
int main(void) {
    Product items[NB] = {{101, "Monitor", 32000}, {102, "Mouse", 1500}, {103, "Laptop", 89000},
                         {104, "Keyboard", 3200}, {105, "Webcam", 4100}, {106, "Cable", 450}};
    Product tmp;
    int pos, idx, minIdx;
    for (pos = 0; pos < NB - 1; pos++) {
        minIdx = pos;
        for (idx = pos + 1; idx < NB; idx++)
            if (items[idx].price < items[minIdx].price) minIdx = idx;
```

```
20          tmp = items[pos];                // تبديل الهيكلين كاملين
21          items[pos] = items[minIdx];
22          items[minIdx] = tmp;
23      }
24      for (idx = 0; idx < NB; idx++)
25          printf("%d %-9s %9.2f\n", items[idx].code, items[idx].name, items[idx].price);
26      return 0;
27  }
```

نتيجة التنفيذ

```
106 Cable        450.00
102 Mouse       1500.00
104 Keyboard    3200.00
105 Webcam      4100.00
101 Monitor    32000.00
103 Laptop     89000.00
```

المخطط الإنسيابي (Organigramme)



تمرين 9.13 ترتيب الزبائن أبجديًا

تجارة إلكترونية

متوسط

رتّب قائمة زبائن (الاسم، المدينة، المشتريات) **أبجديًا حسب الاسم** بالترتيب بالإدراج، مع دالة مقارنة يدوية `compareNames`، وفي حالة تساوي الاسمين حسب المدينة.

خطوات الحل

1) `compareNames` تقارن الحروف حتى أول اختلاف.
2) مفتاح المقارنة المركب: الاسم ثم المدينة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 6

typedef struct {
    char name[16];
    char city[16];
    double spent;
} Customer;

int compareNames(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos]) pos++;
    return first[pos] - second[pos];
}

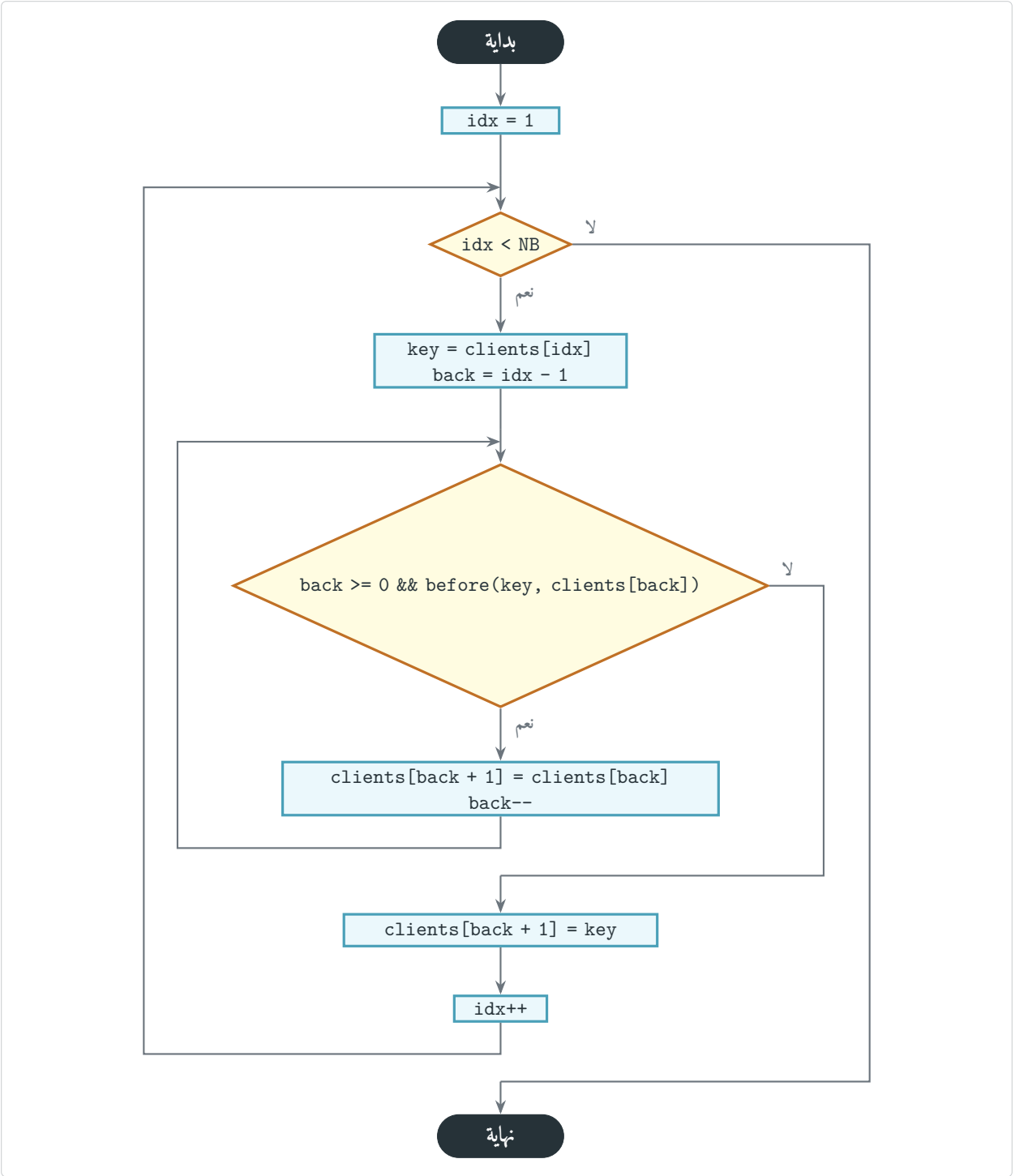
int before(Customer first, Customer second) {
    int byName = compareNames(first.name, second.name);
    if (byName != 0) return byName < 0;
    return compareNames(first.city, second.city) < 0;    // نفس الاسم: المدينة
}

int main(void) {
    Customer clients[NB] = {{"Yacine", "Oran", 15000}, {"Amina", "Setif", 8200},
                            {"Karim", "Annaba", 23000}, {"Amina", "Blida", 4100},
                            {"Lina", "Algiers", 9900}, {"Bilal", "Oran", 12000}};
    Customer key;
    int idx, back;
    for (idx = 1; idx < NB; idx++) {
        key = clients[idx];
        back = idx - 1;
        while (back >= 0 && before(key, clients[back])) {
            clients[back + 1] = clients[back];
            back--;
        }
        clients[back + 1] = key;
    }
    for (idx = 0; idx < NB; idx++)
        printf("%-7s %-8s %8.0f\n", clients[idx].name, clients[idx].city, clients[idx].spent);
    return 0;
}
```

نتيجة التنفيذ

```
Amina   Blida        4100
Amina   Setif        8200
Bilal   Oran        12000
Karim   Annaba      23000
Lina    Algiers      9900
Yacine  Oran        15000
```

المخطط الإنسيابي (Organigramme)



تمرين 9.14 البحث عن منتج برمزه النصي

تجارة إلكترونية ★★ متوسط

لكل منتج رمز نصي SKU مثل `"KB-204"`. اكتب دالة `findBySku(items, size, sku)` تعيد فهرس المنتج أو -1، واستعملها لمعالجة طلبات أسعار تُقرأ من لوحة المفاتيح.

خطوات الحل

1) دالة `sameText` تقارن سلسلتين حرفًا بحرف.
2) بحث تسلسلي على حقل `sku` في جدول الهياكل.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char sku[10];
    char name[20];
    double price;
} Product;

int sameText(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos]) pos++;
    return first[pos] == second[pos];          // انتهتا معًا
}

int findBySku(Product items[], int size, char sku[]) {
    int idx;
    for (idx = 0; idx < size; idx++)
        if (sameText(items[idx].sku, sku)) return idx;
    return -1;
}

int main(void) {
    Product items[5] = {{"KB-204", "Keyboard", 3200}, {"MS-110", "Mouse", 1500},
                        {"MN-270", "Monitor", 32000}, {"HD-500", "Headset", 5400},
                        {"CB-001", "Cable", 450}};
    char sku[10];
    int nbQueries, query, idx;
    scanf("%d", &nbQueries);
    for (query = 0; query < nbQueries; query++) {
        scanf("%9s", sku);
        idx = findBySku(items, 5, sku);
        if (idx >= 0) printf("%s: %s, %.2f DA\n", sku, items[idx].name, items[idx].price);
        else printf("%s: unknown SKU\n", sku);
    }
    return 0;
}
```

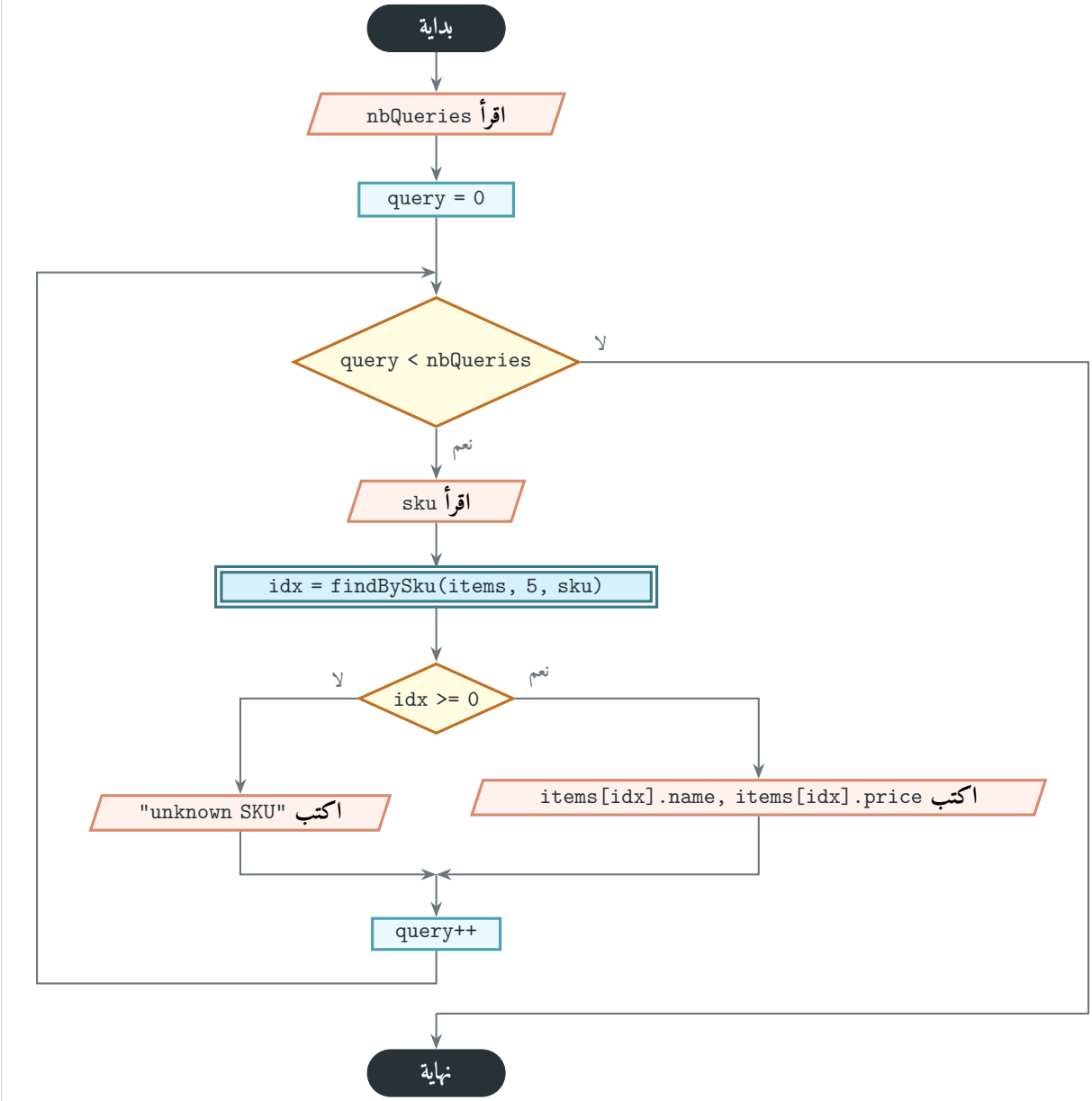
المدخلات

```
3
MS-110
KB-204
TV-999
```

نتيجة التنفيذ

```
MS-110: Mouse, 1500.00 DA
KB-204: Keyboard, 3200.00 DA
TV-999: unknown SKU
```

المخطط الإنسيابي (Organigramme)



تمرين 9.15 الطلبية بهياكل متداخلة

تجارة إلكترونية

متوسط

عرّف هيكل `Order` يحتوي على هيكل `Customer` وهيكل `Date` وجدول من المنتجات المطلوبة. اكتب دالة `printOrder(order)` تعرض ملخص الطلبية: الزبون، التاريخ، عدد القطع، والمبلغ.

خطوات الحل

1) الوصول إلى حقل متداخل: `order.client.name` و `order.date.day`.
2) جدول داخل هيكل: `order.prices[idx]`.

البرنامج بلغة C

```c
#include <stdio.h>

```

```c
typedef struct { int day, month, year; } Date;
typedef struct { char name[20]; char city[16]; } Customer;
typedef struct {
    int number;
    Customer client;              // هيكل داخل هيكل
    Date date;
    int nbLines;
    double prices[5];
    int qty[5];
} Order;

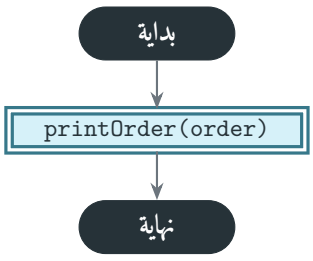
void printOrder(Order order) {
    double total = 0;
    int idx, pieces = 0;
    for (idx = 0; idx < order.nbLines; idx++) {
        total += order.prices[idx] * order.qty[idx];
        pieces += order.qty[idx];
    }
    printf("Order #%d for %s (%s)\n", order.number, order.client.name, order.client.city);
    printf("Date: %02d/%02d/%d\n", order.date.day, order.date.month, order.date.year);
    printf("%d lines, %d pieces, total %.2f DA\n", order.nbLines, pieces, total);
}

int main(void) {
    Order order = {7781, {"Nadir", "Tlemcen"}, {5, 11, 2026}, 3,
                   {1500, 3200, 450}, {2, 1, 4}};
    printOrder(order);
    return 0;
}
```

نتيجة التنفيذ

```
Order #7781 for Nadir (Tlemcen)
Date: 05/11/2026
3 lines, 7 pieces, total 8000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.16 تقرير إعادة التموين

تجارة إلكترونية ★★ متوسط

لكل منتج: الاسم، المخزون الحالي، متوسط المبيعات اليومية، ومدة التوريد بالأيام. نعيد الطلب إذا كان المخزون لا يكفي لمدة التوريد + 3 أيام أمان. اكتب دالة `needsRestock(item)` و `orderQuantity(item)` (كمية تكفي 30 يومًا) واعرض التقرير.

خطوات الحل

1) نقطة إعادة الطلب = `dailySales * (leadDays + 3)`.
2) الكمية المطلوبة = `dailySales * 30 - stock` (تقريب إلى عدد صحيح).

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char name[16];
    int stock;
    double dailySales;
    int leadDays;
} Item;

int needsRestock(Item item) {
    return item.stock < item.dailySales * (item.leadDays + 3);   // نقطة إعادة الطلب
}

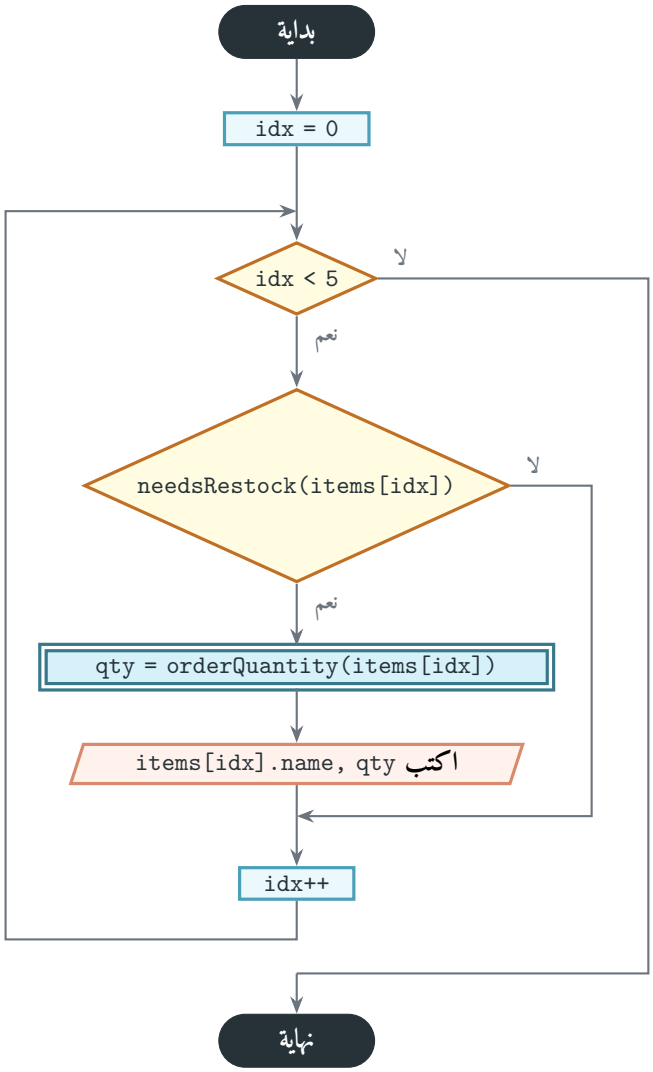
int orderQuantity(Item item) {
    return (int)(item.dailySales * 30 - item.stock + 0.5);
}

int main(void) {
    Item items[5] = {{"Phone case", 40, 6.5, 4}, {"Charger", 120, 5.0, 7},
                     {"Earbuds", 15, 2.2, 10}, {"Cable", 300, 12.0, 3},
                     {"Power bank", 9, 1.5, 5}};
    int idx, count = 0;
    for (idx = 0; idx < 5; idx++)
        if (needsRestock(items[idx])) {
            printf("Reorder %-11s stock %3d -> order %3d units\n", items[idx].name,
                   items[idx].stock, orderQuantity(items[idx]));
            count++;
        }
    printf("%d product(s) to reorder\n", count);
    return 0;
}
```

نتيجة التنفيذ

```
Reorder Phone case  stock  40 -> order 155 units
Reorder Earbuds     stock  15 -> order  51 units
Reorder Power bank  stock   9 -> order  36 units
3 product(s) to reorder
```

المخطط الإنسيابي (Organigramme)



تمرين 9.17 حساب الكسور لتقسيم الحصص

علم البيانات ★★ متوسط

عرّف هيكل `Fraction` (بسط ومقام) ودوال `simplify` و `add` و `multiply`، واستعملها لحساب حصة كل شريك من أرباح متجر: الأول يملك 1/3، والثاني 1/4، والثالث ثلث الباقي.

خطوات الحل

1) `add`: `a/b + c/d = (ad + cb) / bd` ثم التبسيط بالقاسم المشترك الأكبر.
2) `multiply`: `(a·c) / (b·d)`.
3) الباقي = `1 - (1/3 + 1/4)`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    long num, den;
} Fraction;

```

```c
long gcd(long first, long second) {
    while (second != 0) { long rest = first % second; first = second; second = rest; }
    return first < 0 ? -first : first;
}

Fraction simplify(Fraction value) {
    long divisor = gcd(value.num, value.den);
    value.num /= divisor;
    value.den /= divisor;
    return value;
}

Fraction add(Fraction first, Fraction second) {
    Fraction sum = {first.num * second.den + second.num * first.den, first.den * second.den};
    return simplify(sum);
}

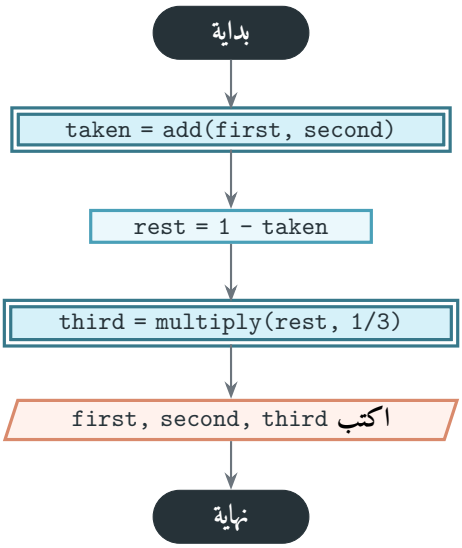
Fraction multiply(Fraction first, Fraction second) {
    Fraction product = {first.num * second.num, first.den * second.den};
    return simplify(product);
}

int main(void) {
    Fraction first = {1, 3}, second = {1, 4}, oneThird = {1, 3};
    Fraction taken = add(first, second);
    Fraction rest = {taken.den - taken.num, taken.den};     // 1 - taken
    Fraction third = multiply(rest, oneThird);
    Fraction left = add(taken, third);
    printf("Partner 1: %ld/%ld\nPartner 2: %ld/%ld\nPartner 3: %ld/%ld\n",
           first.num, first.den, second.num, second.den, third.num, third.den);
    printf("Reinvested: %ld/%ld of profit\n", left.den - left.num, left.den);
    return 0;
}
```

نتيجة التنفيذ

```
Partner 1: 1/3
Partner 2: 1/4
Partner 3: 5/36
Reinvested: 5/18 of profit
```

المخطط الإنسيابي (Organigramme)



تمرين 9.18 المبالغ بالدينار والسنتيم دون أخطاء تقريب

تجارة إلكترونية

متوسط

الأعداد العشرية `double` قد تعطي أخطاء تقريب في المحاسبة. عرّف هيكل `Money` بحقلين صحيحين `dinars` و `centimes` (0..99) ودوال `addMoney` و `multiplyMoney(amount, qty)` و `printMoney`، ثم احسب فاتورة.

خطوات الحل

1) نحول إلى سنتيمات (`long`) للحساب، ثم نعيد التقسيم: `total / 100` و `total % 100`.
2) الضرب في الكمية يتم على السنتيمات.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    long dinars;
    int centimes;
} Money;

long toCentimes(Money amount) { return amount.dinars * 100 + amount.centimes; }

Money fromCentimes(long value) {
    Money amount;
    amount.dinars = value / 100;
    amount.centimes = (int)(value % 100);
    return amount;
}

Money addMoney(Money first, Money second) {
    return fromCentimes(toCentimes(first) + toCentimes(second));
}

Money multiplyMoney(Money amount, int qty) {
    return fromCentimes(toCentimes(amount) * qty);      // حساب بالأعداد الصحيحة
}

void printMoney(Money amount) {
    printf("%ld.%02d DA", amount.dinars, amount.centimes);
}

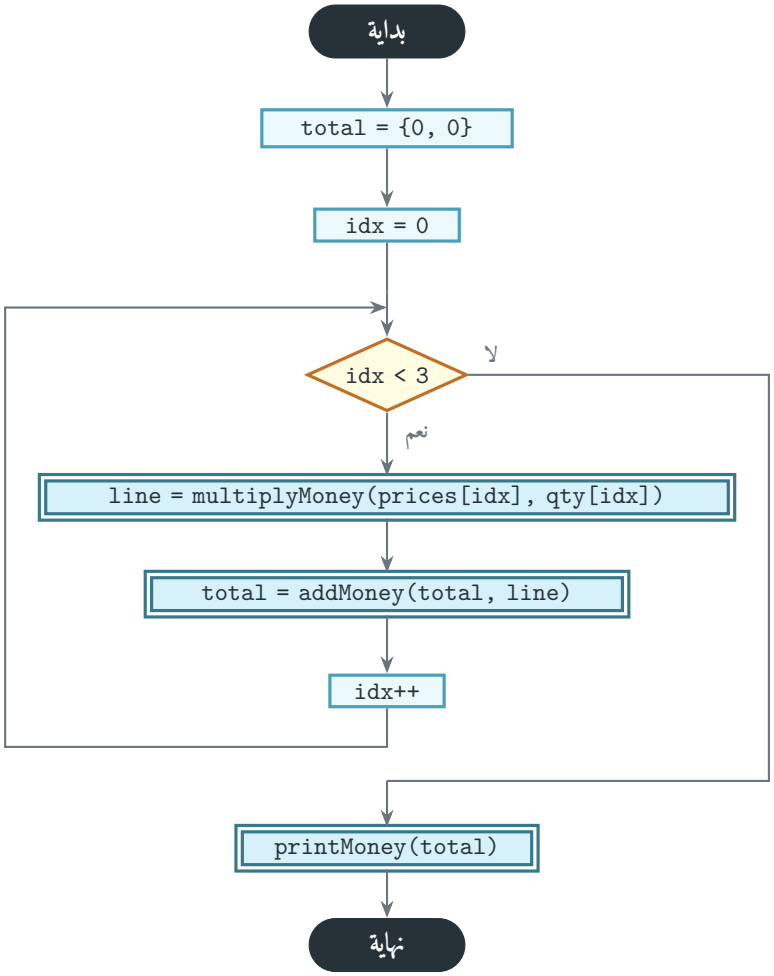
int main(void) {
    Money prices[3] = {{199, 99}, {1450, 50}, {89, 95}}, total = {0, 0}, line;
    int qty[3] = {7, 3, 11}, idx;
    double check = 0;
    for (idx = 0; idx < 3; idx++) {
        line = multiplyMoney(prices[idx], qty[idx]);
        total = addMoney(total, line);
        printf("%2d x ", qty[idx]); printMoney(prices[idx]);
        printf(" = "); printMoney(line); printf("\n");
        check += (prices[idx].dinars + prices[idx].centimes / 100.0) * qty[idx];
    }
    printf("Total: "); printMoney(total); printf("\n");
    printf("Same total with double: %.17f\n", check);
    return 0;
}
```

نتيجة التنفيذ

```
 7 x 199.99 DA = 1399.93 DA
 3 x 1450.50 DA = 4351.50 DA
11 x 89.95 DA = 989.45 DA
Total: 6740.88 DA
```

```
Same total with double: 6740.88000000000010914
```

المخطط الإنسيابي (Organigramme)



تمرين 9.19 هل يدخل الصندوق في صندوق الشحن

تجارة إلكترونية

متوسط

عرّف هيكل Box (طول، عرض، ارتفاع بالسم). اكتب دالة fitsIn(item, carton) تتحقق إن كان المنتج يدخل في صندوق الشحن مع **إمكانية تدويره** (رتب الأبعاد الثلاثة لكل صندوق ثم نقارنها)، ودالة volume، ثم اختر أصغر صندوق مناسب لكل منتج.

خطوات الحل

1) دالة sortDims ترتب أبعاد الصندوق تصاعديًا (3 مقارنات وتبديلات).
2) يدخل إذا كان كل بعد مرتب للمنتج ≤ البعد المرتب الموافق للصندوق.
3) أصغر صندوق مناسب = أصغر حجم بين الصناديق المناسبة.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    double length, width, height;
```

```c
} Box;

Box sortDims(Box box) {
    double tmp;
    if (box.length > box.width)  { tmp = box.length; box.length = box.width; box.width = tmp; }
    if (box.width > box.height)  { tmp = box.width; box.width = box.height; box.height = tmp; }
    if (box.length > box.width)  { tmp = box.length; box.length = box.width; box.width = tmp; }
    return box;                    // length <= width <= height
}

double volume(Box box) { return box.length * box.width * box.height; }

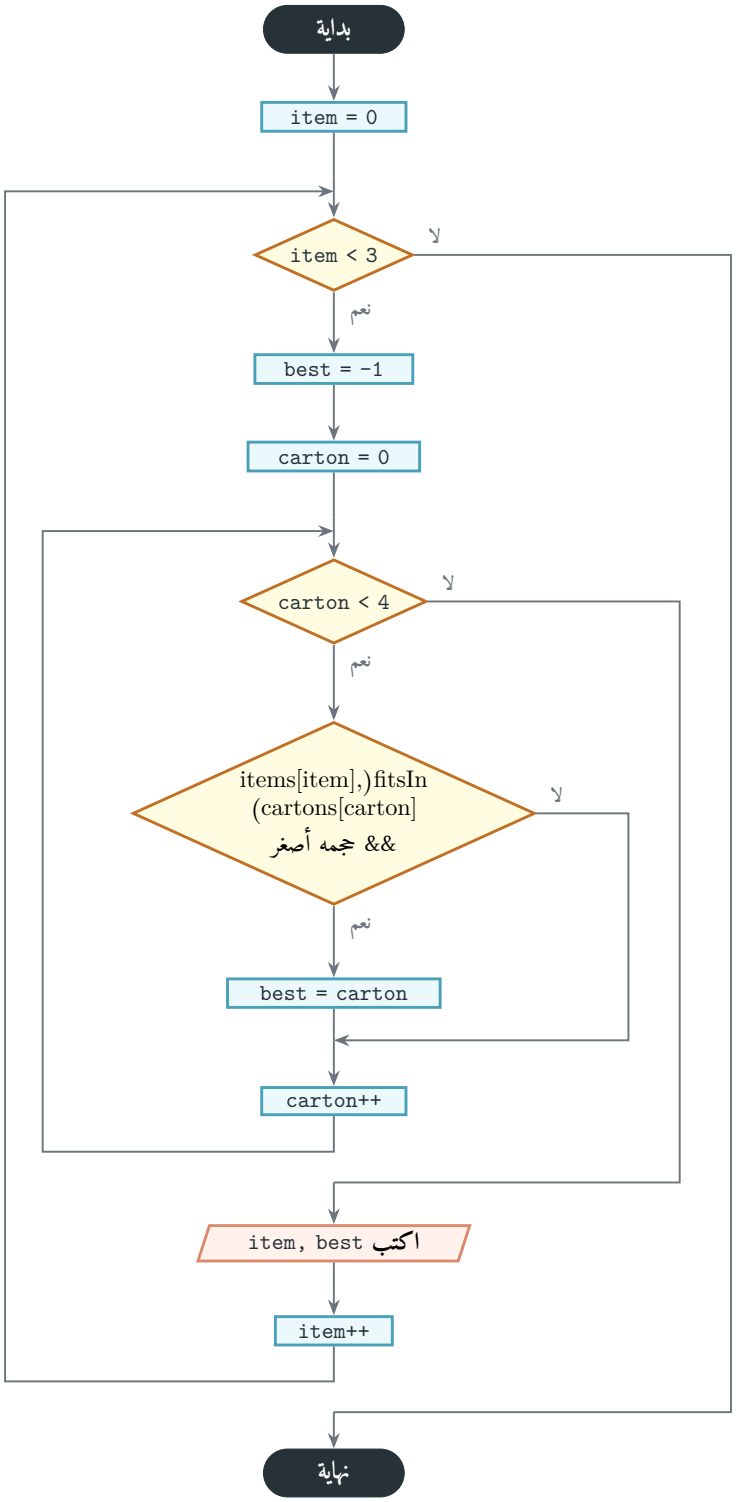
int fitsIn(Box item, Box carton) {
    item = sortDims(item);
    carton = sortDims(carton);
    return item.length <= carton.length && item.width <= carton.width && item.height <= carton.height;
}

int main(void) {
    Box cartons[4] = {{30, 20, 10}, {40, 30, 20}, {15, 60, 15}, {50, 40, 40}};
    Box items[3] = {{12, 55, 9}, {35, 18, 25}, {28, 8, 19}};
    int item, carton, best;
    for (item = 0; item < 3; item++) {
        best = -1;
        for (carton = 0; carton < 4; carton++)
            if (fitsIn(items[item], cartons[carton]) &&
                (best < 0 || volume(cartons[carton]) < volume(cartons[best])))
                best = carton;
        printf("Item %d (%.0fx%.0fx%.0f): carton %d (%.0f cm3)\n", item + 1, items[item].length,
               items[item].width, items[item].height, best + 1, volume(cartons[best]));
    }
    return 0;
}
```

نتيجة التنفيذ

```
Item 1 (12x55x9): carton 3 (13500 cm3)
Item 2 (35x18x25): carton 2 (24000 cm3)
Item 3 (28x8x19): carton 1 (6000 cm3)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.20 دالة تعيد ملخصًا إحصائيًا

علم البيانات ★★ متوسط

الدالة في C تعيد قيمة واحدة، لكن يمكنها أن تعيد **هيكلًا** يحتوي على عدة نتائج. اكتب دالة `describe(values, size)` تعيد هيكل `Summary` (العدد، الأصغر، الأكبر، المتوسط، الانحراف المعياري) لمبيعات فرعين.

خطوات الحل

1) مرور أول: الأصغر والأكبر والمجموع؛ مرور ثانٍ: مجموع مربعات الانحرافات.
2) الجذر بطريقة نيوتن.
3) `main` تعرض الهيكلين بدالة `printSummary`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    int count;
    double low, high, mean, std;
} Summary;

double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    for (iter = 0; iter < 50; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

Summary describe(double values[], int size) {
    Summary result;
    double sum = 0, squares = 0;
    int idx;
    result.count = size;
    result.low = result.high = values[0];
    for (idx = 0; idx < size; idx++) {
        sum += values[idx];
        if (values[idx] < result.low) result.low = values[idx];
        if (values[idx] > result.high) result.high = values[idx];
    }
    result.mean = sum / size;
    for (idx = 0; idx < size; idx++)
        squares += (values[idx] - result.mean) * (values[idx] - result.mean);
    result.std = mySqrt(squares / size);
    return result;                          // عدة نتائج في هيكل واحد
}

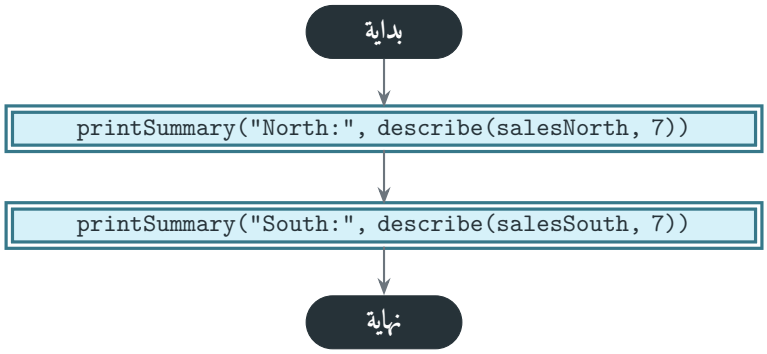
void printSummary(char label[], Summary info) {
    printf("%s n=%d min=%.0f max=%.0f mean=%.2f std=%.2f\n", label, info.count,
           info.low, info.high, info.mean, info.std);
}

int main(void) {
    double salesNorth[7] = {120, 135, 128, 150, 142, 160, 131};
    double salesSouth[7] = {90, 210, 60, 180, 75, 200, 95};
    printSummary("North:", describe(salesNorth, 7));
    printSummary("South:", describe(salesSouth, 7));
    return 0;
}
```

نتيجة التنفيذ

```
North: n=7 min=120 max=160 mean=138.00 std=12.68
South: n=7 min=60 max=210 mean=130.00 std=59.22
```

المخطط الإنسيابي (Organigramme)



تمرين 9.21 المبيعات حسب الفئة

علم البيانات متوسط

لدينا 10 عمليات بيع (اسم الفئة، المبلغ). اكتب برنامجًا يجمعها حسب الفئة في جدول من الهياكل `CategoryTotal` (الاسم، المجموع، العدد) يُبنى تدريجيًا، ثم يعرض لكل فئة مجموعها ومتوسطها وحصتها من رقم الأعمال.

خطوات الحل

1) لكل عملية: نبحث عن فئتها في جدول الإجماليات، إن لم توجد نضيف خانة جديدة.
2) الحصة = `100 *` / .

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char category[12]; double amount; } Sale;
typedef struct { char category[12]; double sum; int count; } CategoryTotal;

int sameText(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos]) pos++;
    return first[pos] == second[pos];
}

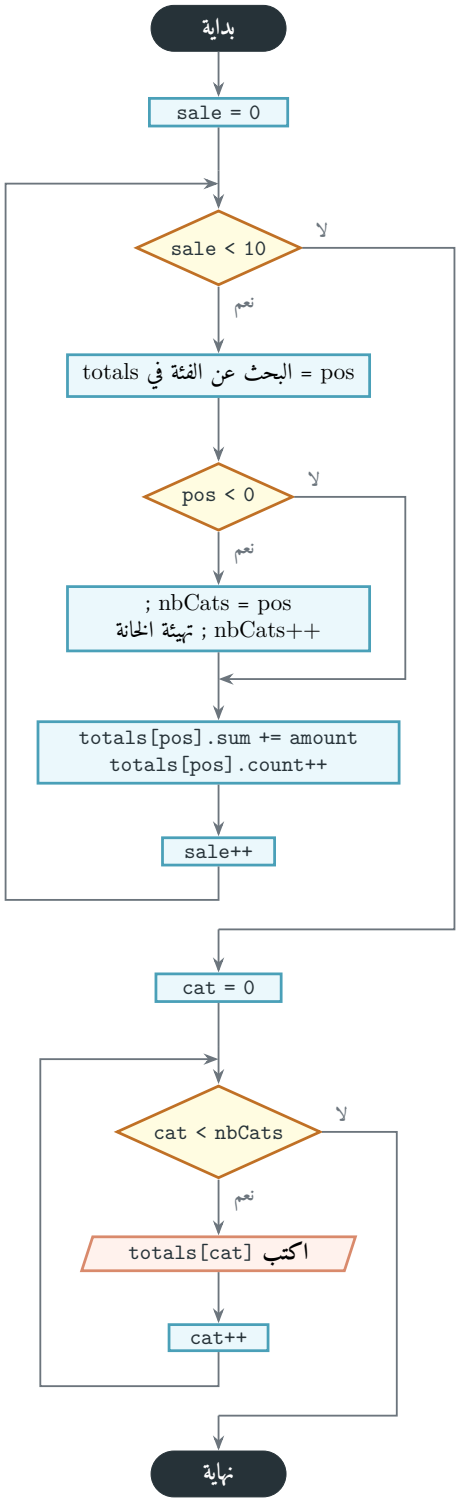
int main(void) {
    Sale sales[10] = {{"Phones", 38000}, {"Audio", 5400}, {"Phones", 27500}, {"Laptops", 95000},
                      {"Audio", 12500}, {"Cables", 450}, {"Laptops", 72000}, {"Phones", 41000},
                      {"Cables", 900}, {"Audio", 3100}};
    CategoryTotal totals[10];
    int sale, cat, pos, nbCats = 0, letter;
    double grand = 0;
    for (sale = 0; sale < 10; sale++) {
        pos = -1;
        for (cat = 0; cat < nbCats; cat++)
            if (sameText(totals[cat].category, sales[sale].category)) pos = cat;
        if (pos < 0) {                                  // فئة جديدة
            pos = nbCats++;
            for (letter = 0; letter < 12; letter++) totals[pos].category[letter] =
  ↪ sales[sale].category[letter];
            totals[pos].sum = 0;
            totals[pos].count = 0;
        }
        totals[pos].sum += sales[sale].amount;
        totals[pos].count++;
        grand += sales[sale].amount;
    }
    for (cat = 0; cat < nbCats; cat++)
        printf("%-8s %d sales, total %9.0f, mean %8.0f, share %5.1f%%\n", totals[cat].category,
               totals[cat].count, totals[cat].sum, totals[cat].sum / totals[cat].count,
```

```
36                 totals[cat].sum / grand * 100);
37     return 0;
38 }
```

نتيجة التنفيذ

```
Phones   3 sales, total    106500, mean    35500, share  36.0%
Audio    3 sales, total     21000, mean     7000, share   7.1%
Laptops  2 sales, total    167000, mean    83500, share  56.4%
Cables   2 sales, total      1350, mean      675, share   0.5%
```

المخطط الإنسيابي (Organigramme)



تمرين 9.22 صلاحية قسيمة التخفيض

تجارة إلكترونية

متوسط

عرّف هيكل `Coupon` (الرمز، النسبة، تاريخ البداية، تاريخ النهاية، الحد الأدنى للسلة). اكتب دالة `isValid(coupon, today, basket)` تتحقق من أن التاريخ داخل فترة الصلاحية وأن السلة تتجاوز الحد الأدنى، ثم طبّق أفضل قسيمة صالحة.

خطوات الحل

1) نحول التاريخ إلى عدد `year*10000 + month*100 + day` لتسهيل المقارنة.

2) من بين القسائم الصالحة نختار ذات أكبر نسبة.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int day, month, year; } Date;
typedef struct {
    char code[12];
    int percent;
    Date start, end;
    double minBasket;
} Coupon;

long dateKey(Date date) { return date.year * 10000L + date.month * 100 + date.day; }

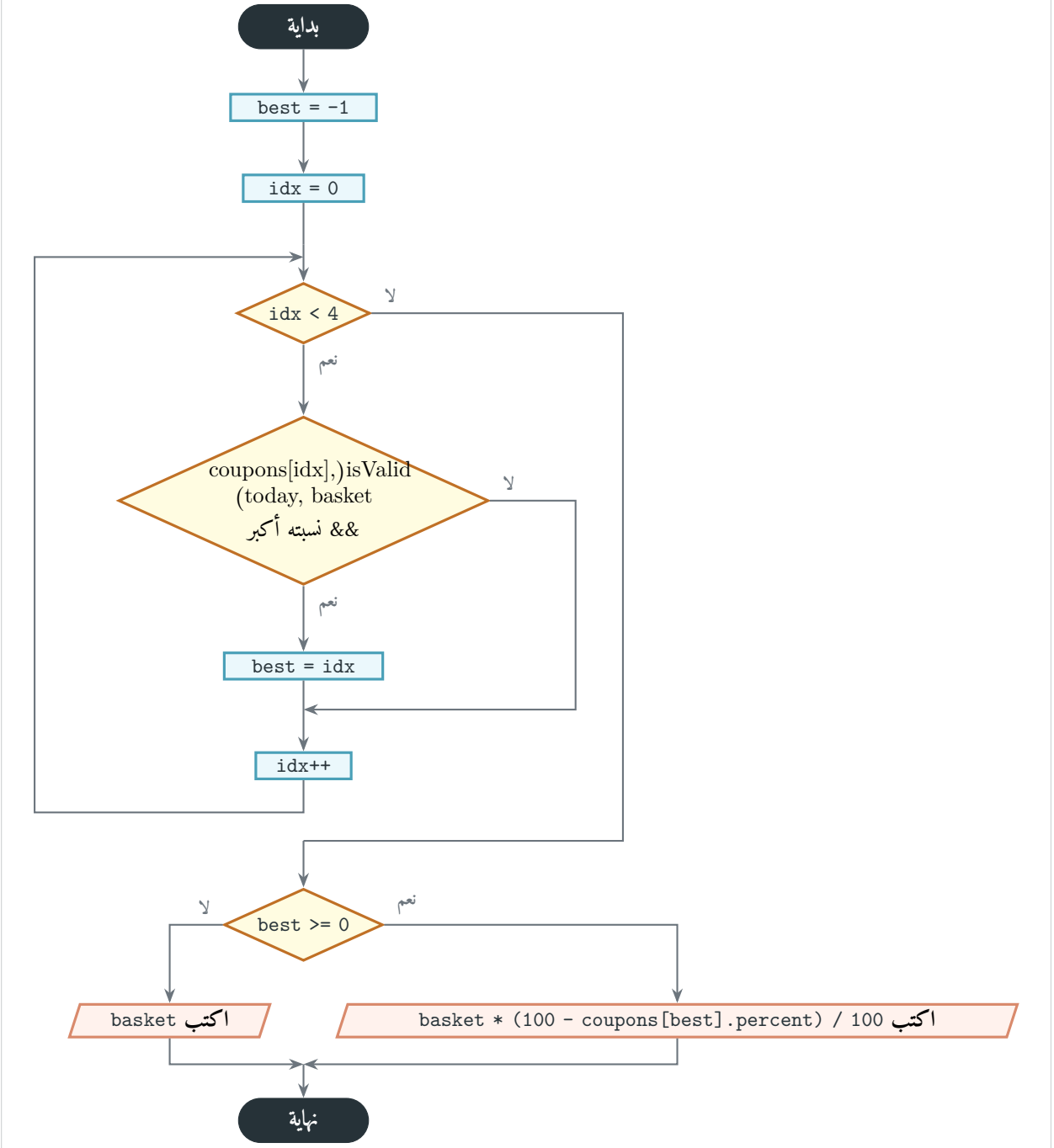
int isValid(Coupon coupon, Date today, double basket) {
    long key = dateKey(today);
    return key >= dateKey(coupon.start) && key <= dateKey(coupon.end) && basket >= coupon.minBasket;
}

int main(void) {
    Coupon coupons[4] = {{"WELCOME10", 10, {1, 1, 2026}, {31, 12, 2026}, 0},
                         {"AUTUMN20", 20, {1, 9, 2026}, {31, 10, 2026}, 5000},
                         {"BIG15", 15, {1, 11, 2026}, {30, 11, 2026}, 20000},
                         {"BLACK30", 30, {27, 11, 2026}, {30, 11, 2026}, 10000}};
    Date today = {6, 11, 2026};
    double basket = 24500;
    int idx, best = -1;
    for (idx = 0; idx < 4; idx++) {
        printf("%-10s %s\n", coupons[idx].code, isValid(coupons[idx], today, basket) ? "valid" : "-");
        if (isValid(coupons[idx], today, basket) &&
            (best < 0 || coupons[idx].percent > coupons[best].percent))
            best = idx;
    }
    if (best >= 0)
        printf("Apply %s: %.2f -> %.2f DA\n", coupons[best].code, basket,
               basket * (100 - coupons[best].percent) / 100);
    return 0;
}
```

نتيجة التنفيذ

```
WELCOME10  valid
AUTUMN20   -
BIG15      valid
BLACK30    -
Apply BIG15: 24500.00 -> 20825.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.23 عمولات فريق المبيعات — تجارة إلكترونية — متوسط

لكل بائع: الاسم، المبيعات الشهرية، والهدف. العمولة 3% من المبيعات، وتصبح 5% على الجزء الذي يتجاوز الهدف، مع مكافأة 10000 دج لمن تجاوز الهدف بأكثر من 20%. اكتب دالة `commission(seller)` واعرض الترتيب حسب العمولة.

خطوات الحل

1) الجزء العادي `min(sales, target) * 0.03`، والجزء الإضافي `(sales - target) * 0.05`.
2) نخزن العمولة في حقل من الهيكل ثم نرتب.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char name[12];
    double sales, target, bonus;
} Seller;

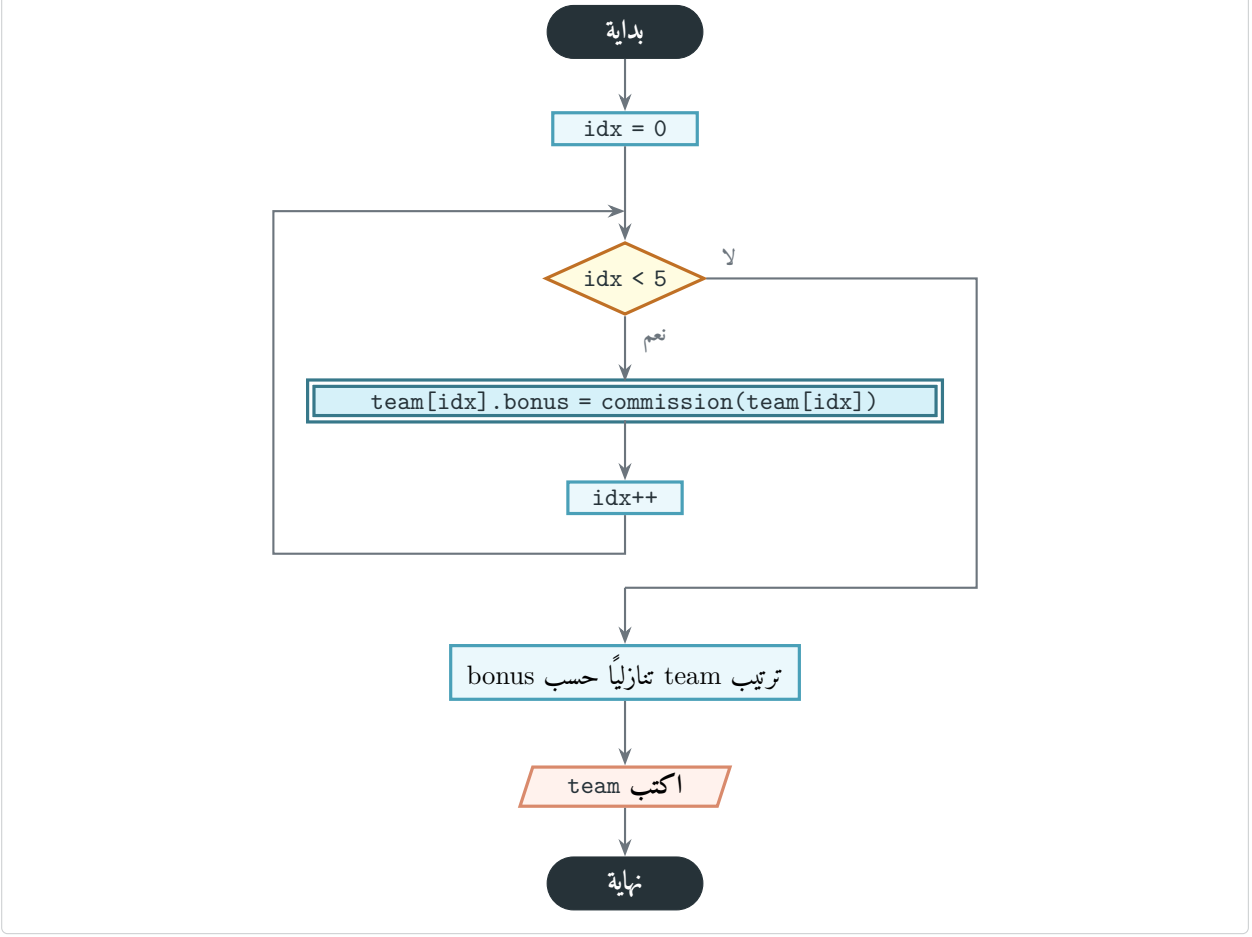
double commission(Seller seller) {
    double amount;
    if (seller.sales <= seller.target) return seller.sales * 0.03;
    amount = seller.target * 0.03 + (seller.sales - seller.target) * 0.05;
    if (seller.sales > seller.target * 1.2) amount += 10000;    // مكافأة
    return amount;
}

int main(void) {
    Seller team[5] = {{"Rania", 820000, 700000, 0}, {"Hakim", 640000, 700000, 0},
                      {"Selma", 910000, 750000, 0}, {"Omar", 505000, 500000, 0},
                      {"Imane", 300000, 450000, 0}};
    Seller tmp;
    int idx, pass;
    for (idx = 0; idx < 5; idx++) team[idx].bonus = commission(team[idx]);
    for (pass = 0; pass < 4; pass++)
        for (idx = 0; idx < 4 - pass; idx++)
            if (team[idx].bonus < team[idx + 1].bonus) {
                tmp = team[idx]; team[idx] = team[idx + 1]; team[idx + 1] = tmp;
            }
    for (idx = 0; idx < 5; idx++)
        printf("%d. %-6s sales %7.0f / %7.0f -> %8.2f DA\n", idx + 1, team[idx].name,
               team[idx].sales, team[idx].target, team[idx].bonus);
    return 0;
}
```

نتيجة التنفيذ

```
1. Selma  sales  910000 /  750000 -> 40500.00 DA
2. Rania  sales  820000 /  700000 -> 27000.00 DA
3. Hakim  sales  640000 /  700000 -> 19200.00 DA
4. Omar   sales  505000 /  500000 -> 15250.00 DA
5. Imane  sales  300000 /  450000 ->  9000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.24 أفضل ثلاثة زبائن

علم البيانات ★★ متوسط

من سجل 12 طلبية (رقم الزبون، المبلغ)، اجمع إنفاق كل زبون في جدول هياكل Spending، ثم اعرض **أفضل ثلاثة زبائن** دون ترتيب الجدول كله (ثلاث عمليات اختيار للأكبر).

خطوات الحل

1) تجميع: بحث عن الزبون في الجدول، أو إضافته.
2) لكل رتبة من 1 إلى 3: نبحث عن الأكبر بين غير المختارين، ونعلّمه مختارًا.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int client; double amount; } Order;
typedef struct { int client; double total; int orders; int picked; } Spending;

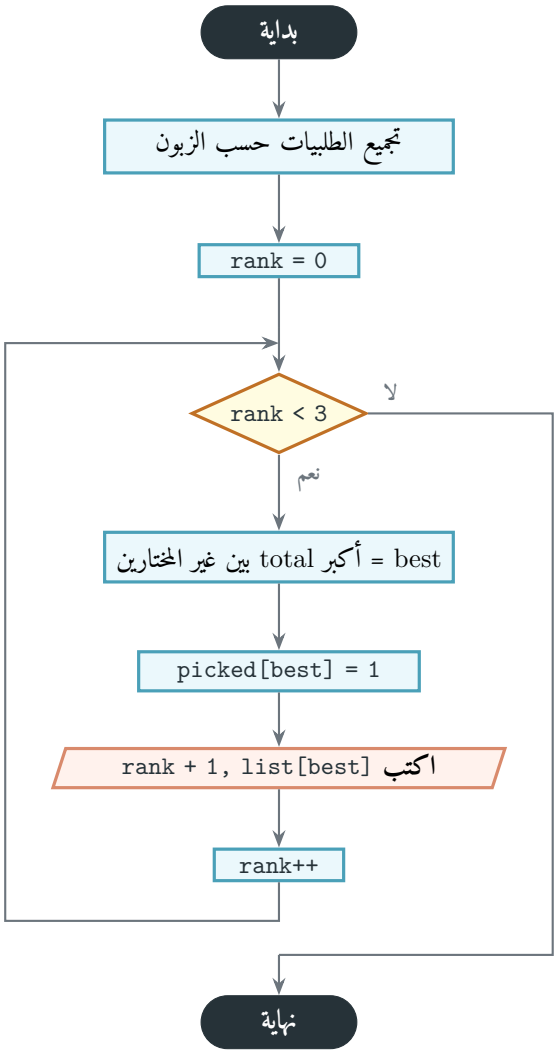
int main(void) {
    Order orders[12] = {{7, 4500}, {3, 12000}, {7, 8200}, {9, 1500}, {3, 3300}, {5, 22000},
                        {9, 6400}, {7, 2100}, {1, 9800}, {5, 1200}, {3, 7600}, {1, 400}};
    Spending list[12];
    int nb = 0, idx, pos, rank, best;
    for (idx = 0; idx < 12; idx++) {
        for (pos = 0; pos < nb && list[pos].client != orders[idx].client; pos++) {}
        if (pos == nb) {                          // زبون جديد
            list[nb].client = orders[idx].client;
```

```
            list[nb].total = 0;
            list[nb].orders = 0;
            list[nb].picked = 0;
            nb++;
        }
        list[pos].total += orders[idx].amount;
        list[pos].orders++;
    }
    printf("%d distinct customers\n", nb);
    for (rank = 0; rank < 3; rank++) {
        best = -1;
        for (pos = 0; pos < nb; pos++)
            if (!list[pos].picked && (best < 0 || list[pos].total > list[best].total)) best = pos;
        list[best].picked = 1;
        printf("#%d customer %d: %.0f DA in %d orders\n", rank + 1, list[best].client,
               list[best].total, list[best].orders);
    }
    return 0;
}
```

نتيجة التنفيذ

```
5 distinct customers
#1 customer 5: 23200 DA in 2 orders
#2 customer 3: 22900 DA in 3 orders
#3 customer 7: 14800 DA in 3 orders
```

المخطط الإنسيابي (Organigramme)



تمرين 9.25 دمج نوافذ التوصيل المتداخلة

تجارة إلكترونية ★★ متوسط

اختار الزبائن نوافذ توصيل (ساعة البداية، ساعة النهاية) قد تتداخل. عرّف هيكل `Window` ورتب النوافذ حسب البداية، ثم **ادمج** المتداخلة منها لتحديد الفترات التي يجب أن يعمل فيها السائق.

خطوات الحل

1) ترتيب بالإدراج حسب `start`.

2) تمر على النوافذ: إذا بدأت النافذة قبل نهاية الفترة الحالية (أو عندها) نمدد النهاية، وإلا نبدأ فترة جديدة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 8

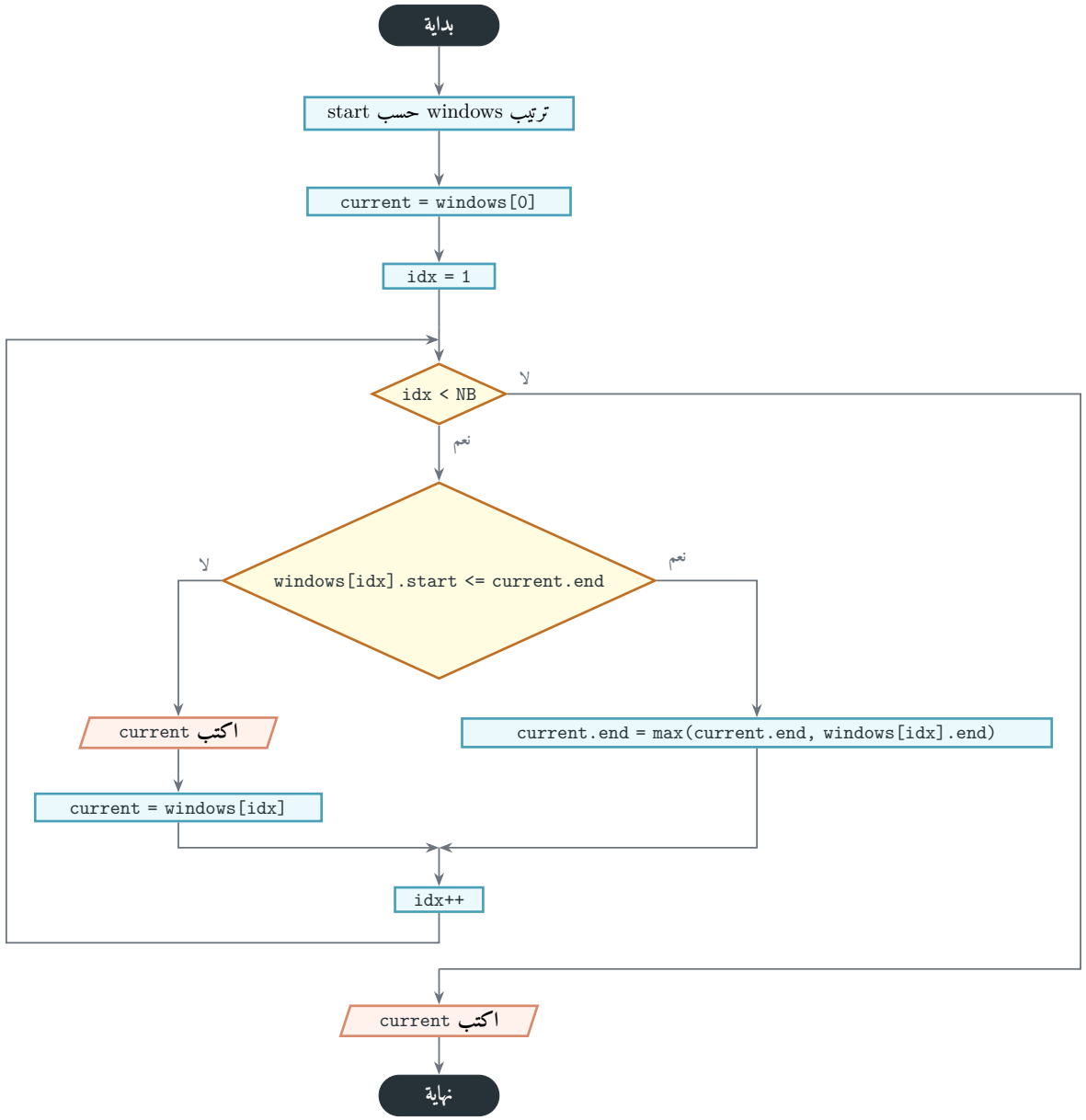
typedef struct { int start, end; } Window;

int main(void) {
    Window windows[NB] = {{14, 16}, {9, 11}, {10, 12}, {18, 20}, {15, 17}, {8, 9}, {19, 21}, {13, 14}};
    Window key, current;
    int idx, back, busy = 0;
    for (idx = 1; idx < NB; idx++) {                // ترتيب حسب البداية
        key = windows[idx];
        for (back = idx - 1; back >= 0 && windows[back].start > key.start; back--)
            windows[back + 1] = windows[back];
        windows[back + 1] = key;
    }
    current = windows[0];
    for (idx = 1; idx < NB; idx++) {
        if (windows[idx].start <= current.end) {     // تداخل أو تلامس
            if (windows[idx].end > current.end) current.end = windows[idx].end;
        } else {
            printf("Shift %02dh-%02dh\n", current.start, current.end);
            busy += current.end - current.start;
            current = windows[idx];
        }
    }
    printf("Shift %02dh-%02dh\n", current.start, current.end);
    busy += current.end - current.start;
    printf("Driver works %d hours\n", busy);
    return 0;
}
```

نتيجة التنفيذ

```
Shift 08h-12h
Shift 13h-17h
Shift 18h-21h
Driver works 11 hours
```

المخطط الإنسيابي (Organigramme)



تمرين 9.26 موقع المستودع المركزي

علم البيانات ★★ متوسط

لكل مدينة إحداثيات (هيكل `City`) وعدد طلبيات شهرية. أفضل موقع تقريبي لمستودع مركزي هو **المركز المرجح** $\Sigma(w \cdot x)/\Sigma w$ و $\Sigma(w \cdot y)/\Sigma w$. احسبه، ثم احسب متوسط المسافة المرجحة عنه.

خطوات الحل

1) الوزن `w` هو عدد الطلبيات.
2) المسافة الإقليدية بالجذر (نيوتن).

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
```

```c
    char name[12];
    double posX, posY;
    int orders;
} City;

typedef struct { double posX, posY; } Point;

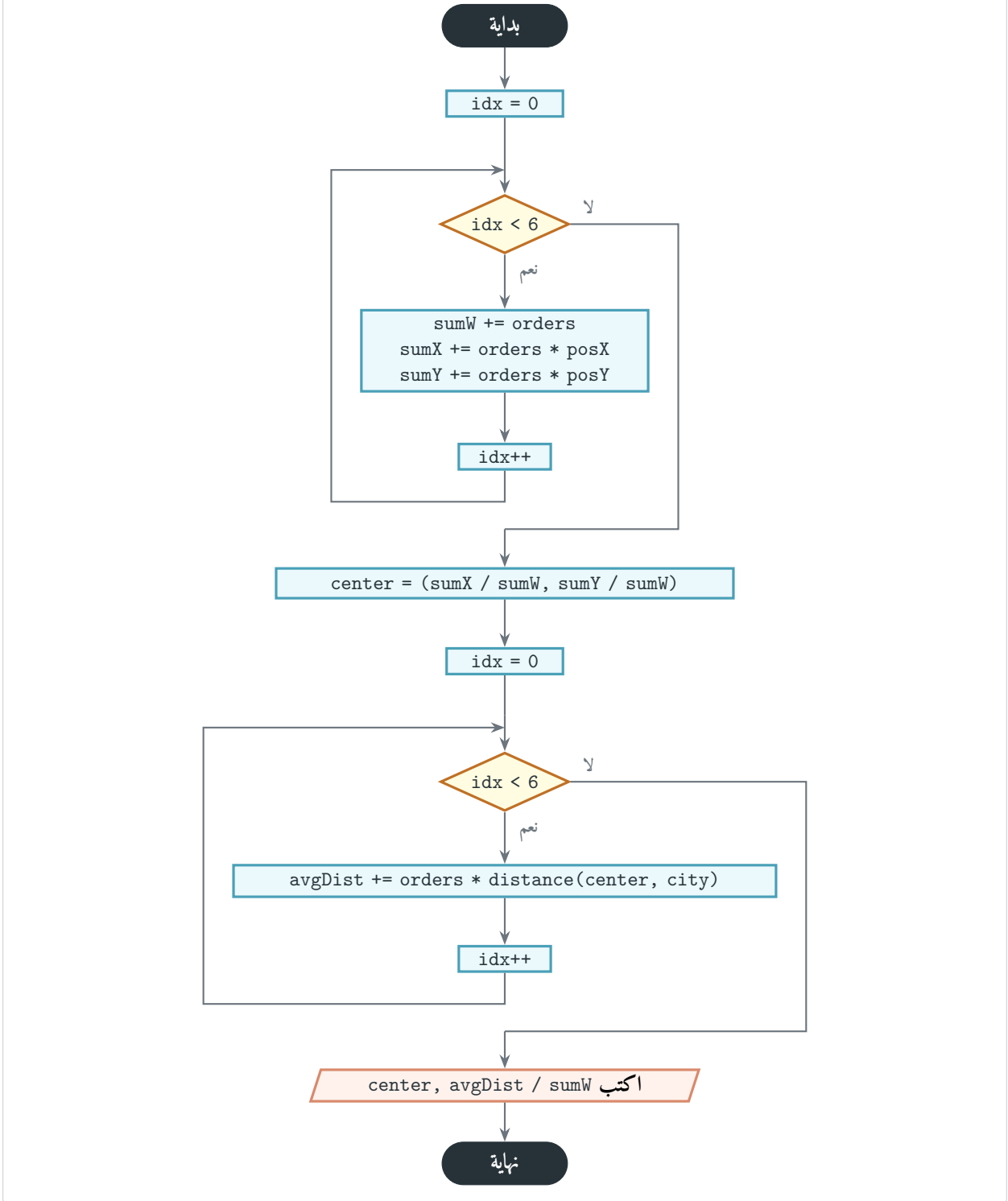
double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    for (iter = 0; iter < 50; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

int main(void) {
    City cities[6] = {{"Algiers", 0, 0, 520}, {"Blida", -10, -45, 180}, {"Tizi", 95, -20, 140},
                      {"Bouira", 110, -90, 60}, {"Medea", -20, -80, 90}, {"Boumerdes", 45, -5, 110}};
    Point center;
    double sumW = 0, sumX = 0, sumY = 0, avgDist = 0, dx, dy;
    int idx;
    for (idx = 0; idx < 6; idx++) {
        sumW += cities[idx].orders;
        sumX += cities[idx].orders * cities[idx].posX;
        sumY += cities[idx].orders * cities[idx].posY;
    }
    center.posX = sumX / sumW;
    center.posY = sumY / sumW;
    for (idx = 0; idx < 6; idx++) {
        dx = cities[idx].posX - center.posX;
        dy = cities[idx].posY - center.posY;
        avgDist += cities[idx].orders * mySqrt(dx * dx + dy * dy);   // مسافة مرجحة
    }
    printf("Warehouse at (%.1f, %.1f)\n", center.posX, center.posY);
    printf("Average distance per order: %.1f km\n", avgDist / sumW);
    return 0;
}
```

نتيجة التنفيذ

```
Warehouse at (19.3, -21.9)
Average distance per order: 44.5 km
```

المخطط الإنسيابي (Organigramme)



تمرين 9.27 حالات الطلبية بالتعداد

تجارة إلكترونية ★★ متوسط

عرّف تعدادًا

```
enum Status { PENDING, PAID, SHIPPED, DELIVERED, CANCELLED }
```

وهيكل `Order`. اكتب دالة `advance(order, event)` تطبق انتقالًا بين الحالات حسب الحدث (`'p'` دفع، `'s'` شحن، `'d'` تسليم، `'c'` إلغاء) وترفض الانتقالات غير المسموحة.

خطوات الحل

1) الإلغاء مسموح فقط قبل الشحن.
2) كل حدث مسموح فقط من حالة واحدة (الدفع من PENDING، الشحن من PAID، ...).
3) جدول أسماء الحالات لعرضها نصيًا.

البرنامج بلغة C

```c
#include <stdio.h>

enum Status { PENDING, PAID, SHIPPED, DELIVERED, CANCELLED };

typedef struct {
    int number;
    enum Status status;
} Order;

char names[5][10] = {"PENDING", "PAID", "SHIPPED", "DELIVERED", "CANCELLED"};

int advance(Order order[], char event) {
    enum Status now = order[0].status;
    if (event == 'p' && now == PENDING) order[0].status = PAID;
    else if (event == 's' && now == PAID) order[0].status = SHIPPED;
    else if (event == 'd' && now == SHIPPED) order[0].status = DELIVERED;
    else if (event == 'c' && (now == PENDING || now == PAID)) order[0].status = CANCELLED;
    else return 0;                                      // انتقال مرفوض
    return 1;
}

int main(void) {
    Order order[1] = {{7781, PENDING}};
    char events[20];
    int idx, ok;
    scanf("%19s", events);
    for (idx = 0; events[idx] != '\0'; idx++) {
        ok = advance(order, events[idx]);
        printf("event '%c': %-9s %s\n", events[idx], names[order[0].status], ok ? "" : "(rejected)");
    }
    return 0;
}
```

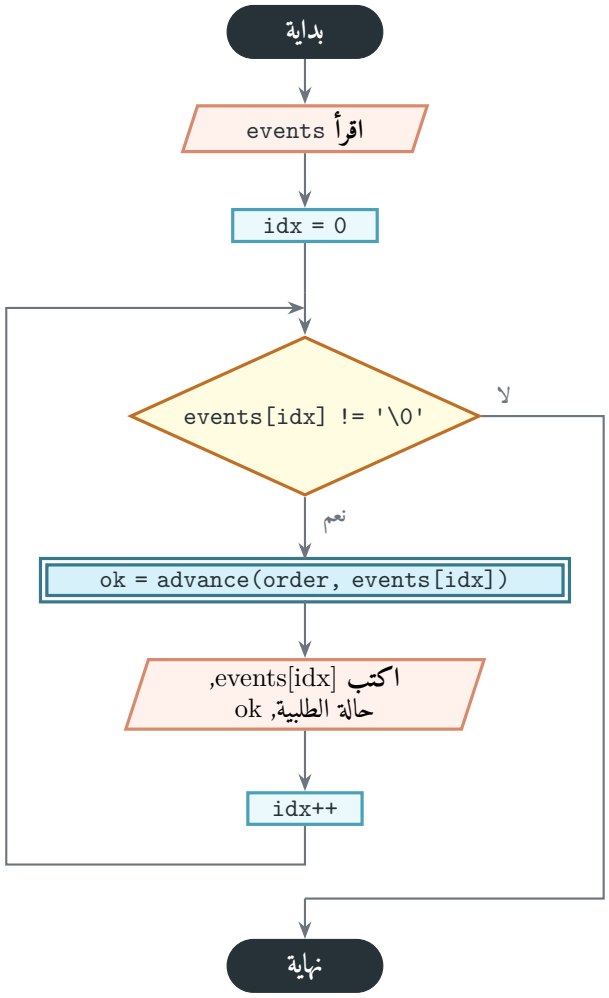
المدخلات

```
pscdd
```

نتيجة التنفيذ

```
event 'p': PAID
event 's': SHIPPED
event 'c': SHIPPED   (rejected)
event 'd': DELIVERED
event 'd': DELIVERED (rejected)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.28 تاريخ الإستحقاق بإضافة أيام

تجارة إلكترونية ★★ متوسط

اكتب دالة `addDays(date, count)` تعيد هيكل `Date` جديدًا بعد إضافة عدد من الأيام (مع تغيير الشهر والسنة والسنوات الكبيسة)، واستعملها لحساب تواريخ استحقاق أقساط شهرية كل 30 يومًا.

خطوات الحل

1) نضيف الأيام واحدًا واحدًا: إذا تجاوز اليوم عدد أيام الشهر ننتقل إلى الشهر التالي، وإذا تجاوز الشهر 12 ننتقل إلى السنة التالية.
2) دالة `daysIn(month, year)` مع الكبيسة.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int day, month, year; } Date;

int daysIn(int month, int year) {
    int days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (month == 2 && ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)) return 29;
```

```c
    return days[month - 1];
}

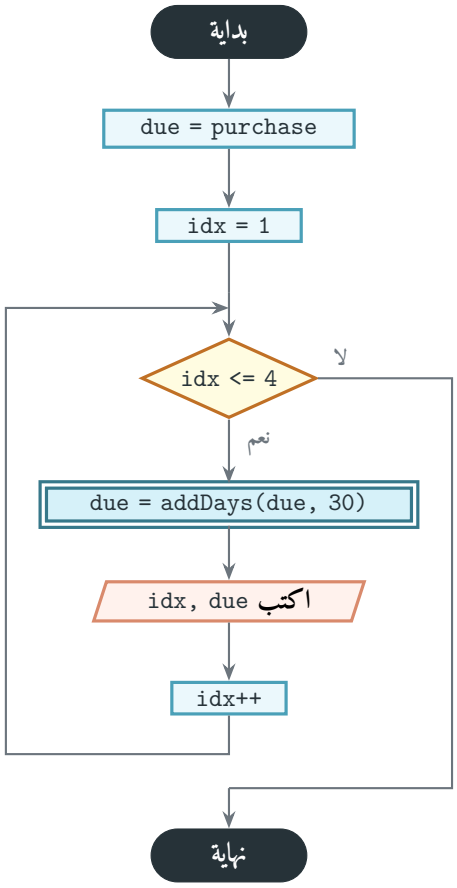
Date addDays(Date date, int count) {
    while (count-- > 0) {
        date.day++;
        if (date.day > daysIn(date.month, date.year)) {   // شهر جديد
            date.day = 1;
            date.month++;
            if (date.month > 12) { date.month = 1; date.year++; }
        }
    }
    return date;
}

int main(void) {
    Date purchase = {15, 11, 2027}, due = purchase;
    int idx;
    printf("Purchase: %02d/%02d/%d\n", purchase.day, purchase.month, purchase.year);
    for (idx = 1; idx <= 4; idx++) {
        due = addDays(due, 30);
        printf("Installment %d due %02d/%02d/%d\n", idx, due.day, due.month, due.year);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Purchase: 15/11/2027
Installment 1 due 15/12/2027
Installment 2 due 14/01/2028
Installment 3 due 13/02/2028
Installment 4 due 14/03/2028
```

المخطط الإنسيابي (Organigramme)



تمرين 9.29 إدارة سلة المشتريات

تجارة إلكترونية ★★ متوسط

عرّف هيكل Cart يحتوي على جدول من CartItem (الرمز، السعر، الكمية) وعدد العناصر. اكتب دوال addItem (تزيد الكمية إذا كان المنتج موجودًا) و removeItem (تحذف السطر وتزيح الباقي) و cartTotal، ثم طبّق سلسلة عمليات.

خطوات الحل

1) نمرر السلة كجدول من خانة واحدة Cart cart[] لكي تعدلها الدوال.
2) الحذف: نزيح العناصر التي بعده بخانة إلى اليسار وننقص العدد.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; double price; int qty; } CartItem;
typedef struct { CartItem items[10]; int count; } Cart;

void addItem(Cart cart[], int code, double price, int qty) {
    int idx;
    for (idx = 0; idx < cart[0].count; idx++)
        if (cart[0].items[idx].code == code) { cart[0].items[idx].qty += qty; return; }
    cart[0].items[cart[0].count].code = code;          // سطر جديد
    cart[0].items[cart[0].count].price = price;
    cart[0].items[cart[0].count].qty = qty;
    cart[0].count++;
```

```c
}

void removeItem(Cart cart[], int code) {
    int idx, found = -1;
    for (idx = 0; idx < cart[0].count; idx++)
        if (cart[0].items[idx].code == code) found = idx;
    if (found < 0) return;
    for (idx = found; idx < cart[0].count - 1; idx++)
        cart[0].items[idx] = cart[0].items[idx + 1];      // إزاحة
    cart[0].count--;
}

double cartTotal(Cart cart[]) {
    double total = 0;
    int idx;
    for (idx = 0; idx < cart[0].count; idx++)
        total += cart[0].items[idx].price * cart[0].items[idx].qty;
    return total;
}

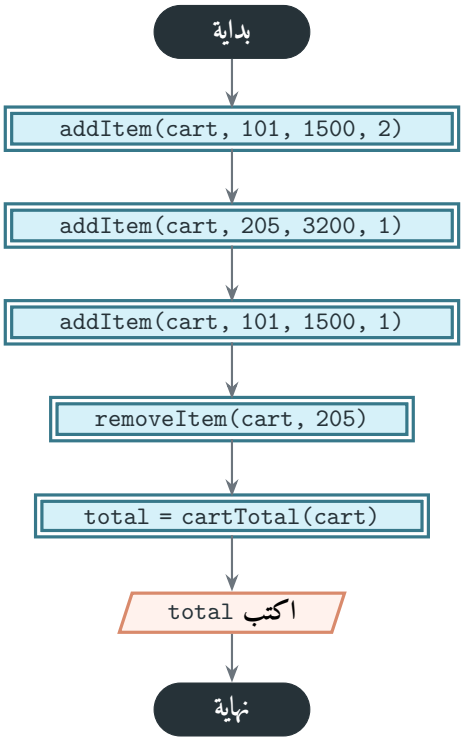
void show(Cart cart[]) {
    int idx;
    for (idx = 0; idx < cart[0].count; idx++)
        printf("[%d x%d] ", cart[0].items[idx].code, cart[0].items[idx].qty);
    printf("= %.2f DA\n", cartTotal(cart));
}

int main(void) {
    Cart cart[1];
    cart[0].count = 0;
    addItem(cart, 101, 1500, 2); show(cart);
    addItem(cart, 205, 3200, 1); show(cart);
    addItem(cart, 330, 899, 3);  show(cart);
    addItem(cart, 101, 1500, 1); show(cart);
    removeItem(cart, 205);       show(cart);
    return 0;
}
```

نتيجة التنفيذ

```
[101 x2] = 3000.00 DA
[101 x2] [205 x1] = 6200.00 DA
[101 x2] [205 x1] [330 x3] = 8897.00 DA
[101 x3] [205 x1] [330 x3] = 10397.00 DA
[101 x3] [330 x3] = 7197.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.30 توزيع قيم السلات على فئات

علم البيانات متوسط

عرّف هيكل Bucket (الحد الأدنى، الحد الأعلى، العدد). وزّع 20 قيمة سلة على 5 فئات متساوية العرض بين أصغر وأكبر قيمة، ثم اعرض المدرج التكراري بالنجوم.

خطوات الحل

1) العرض `width = (max - min) / 5`.

2) فهرس الفئة `(value - min) / width`، والقيمة العظمى تُوضع في الفئة الأخيرة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 20
#define NBB 5

typedef struct { double low, high; int count; } Bucket;

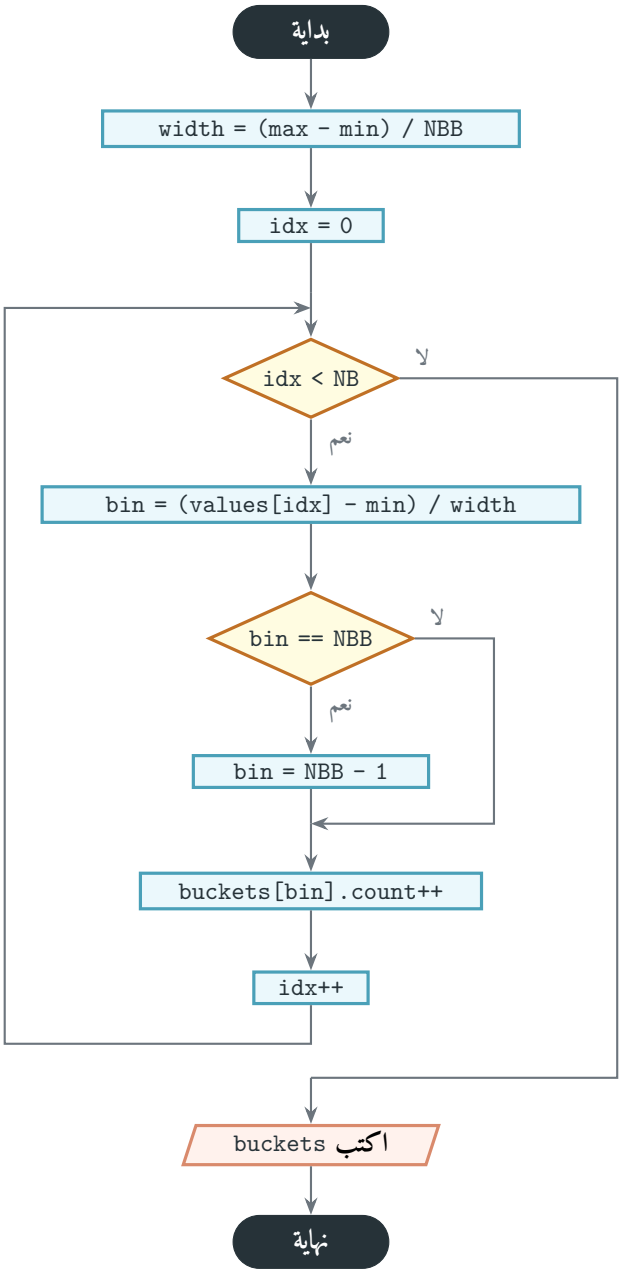
int main(void) {
    double values[NB] = {1200, 4500, 3100, 890, 7600, 2300, 5400, 1500, 9800, 3300,
                         2700, 450, 6100, 3900, 2100, 1800, 4200, 3600, 2900, 5100};
    Bucket buckets[NBB];
    double low = values[0], high = values[0], width;
    int idx, bin, star;
    for (idx = 1; idx < NB; idx++) {
        if (values[idx] < low) low = values[idx];
        if (values[idx] > high) high = values[idx];
    }
    width = (high - low) / NBB;
```

```c
    for (bin = 0; bin < NBB; bin++) {
        buckets[bin].low = low + bin * width;
        buckets[bin].high = low + (bin + 1) * width;
        buckets[bin].count = 0;
    }
    for (idx = 0; idx < NB; idx++) {
        bin = (int)((values[idx] - low) / width);
        if (bin == NBB) bin = NBB - 1;               // القيمة العظمى
        buckets[bin].count++;
    }
    for (bin = 0; bin < NBB; bin++) {
        printf("%6.0f-%6.0f | ", buckets[bin].low, buckets[bin].high);
        for (star = 0; star < buckets[bin].count; star++) putchar('*');
        printf(" %d\n", buckets[bin].count);
    }
    return 0;
}
```

نتيجة التنفيذ

```
 450-  2320 | ******* 7
2320-  4190 | ****** 6
4190-  6060 | **** 4
6060-  7930 | ** 2
7930-  9800 | * 1
```

المخطط الإنسيابي (Organigramme)



تمرين 9.31 معدل الإرتداد ومدة الجلسات

علم البيانات

متوسط

كل جلسة تصفح هيكل `Session` (المصدر: `'S'` بحث، `'A'` إعلان، `'D'` مباشر؛ عدد الصفحات؛ المدة بالثواني؛ هل اشترى). الجلسة **مرتدة** إذا زار صفحة واحدة. احسب لكل مصدر: عدد الجلسات، معدل الارتداد، متوسط المدة، ومعدل التحويل.

خطوات الحل

1) هيكل `SourceStats` لكل مصدر يجمع العدادات.
2) فهرس المصدر من جدول الرموز `"SAD"`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12

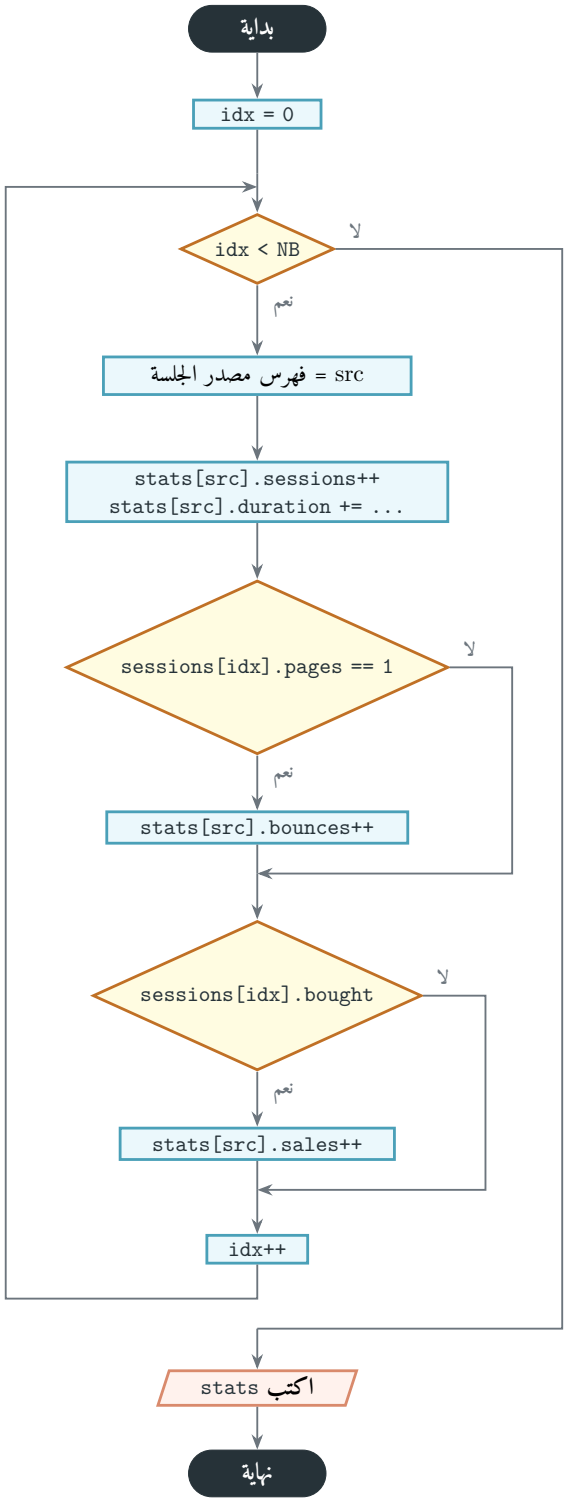
typedef struct { char source; int pages, seconds, bought; } Session;
typedef struct { int sessions, bounces, seconds, sales; } SourceStats;

int main(void) {
    Session log[NB] = {{'S', 5, 340, 1}, {'A', 1, 12, 0}, {'D', 8, 610, 1}, {'A', 1, 9, 0},
                       {'S', 3, 150, 0}, {'A', 2, 45, 0}, {'D', 1, 20, 0}, {'S', 7, 420, 1},
                       {'A', 1, 15, 0}, {'S', 1, 30, 0}, {'D', 4, 260, 1}, {'A', 6, 380, 1}};
    char codes[4] = "SAD";
    char labels[3][8] = {"Search", "Ads", "Direct"};
    SourceStats stats[3] = {{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}};
    int idx, src;
    for (idx = 0; idx < NB; idx++) {
        for (src = 0; codes[src] != log[idx].source; src++) {}   // فهرس المصدر
        stats[src].sessions++;
        stats[src].seconds += log[idx].seconds;
        if (log[idx].pages == 1) stats[src].bounces++;
        if (log[idx].bought) stats[src].sales++;
    }
    for (src = 0; src < 3; src++)
        printf("%-6s %d sessions, bounce %5.1f%%, avg %5.1fs, conversion %5.1f%%\n", labels[src],
               stats[src].sessions, 100.0 * stats[src].bounces / stats[src].sessions,
               (double)stats[src].seconds / stats[src].sessions,
               100.0 * stats[src].sales / stats[src].sessions);
    return 0;
}
```

نتيجة التنفيذ

```
Search 4 sessions, bounce  25.0%, avg 235.0s, conversion  50.0%
Ads    5 sessions, bounce  60.0%, avg  92.2s, conversion  20.0%
Direct 3 sessions, bounce  33.3%, avg 296.7s, conversion  66.7%
```

المخطط الإنسيابي (Organigramme)



تمرين 9.32 تكلفة الشحن حسب المنطقة والوزن

تجارة إلكترونية ★★ متوسط

عرّف هيكل `Zone` (الاسم، السعر الأساسي حتى 1 كغ، سعر كل كغ إضافي) وهيكل `Parcel` (رقم الطرد، المنطقة، الوزن الحقيقي، الأبعاد). الوزن المعتمد هو الأكبر بين الوزن الحقيقي والوزن **الحجمي** `/ 5000`. احسب تكلفة كل طرد.

خطوات الحل

1) الوزن المعتمد يُقرب إلى أعلى كيلوغرام (دون `ceil`).

2) التكلفة = `base + (kg - 1) * perKg`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[10]; double base, perKg; } Zone;
typedef struct { int id, zone; double weight, length, width, height; } Parcel;

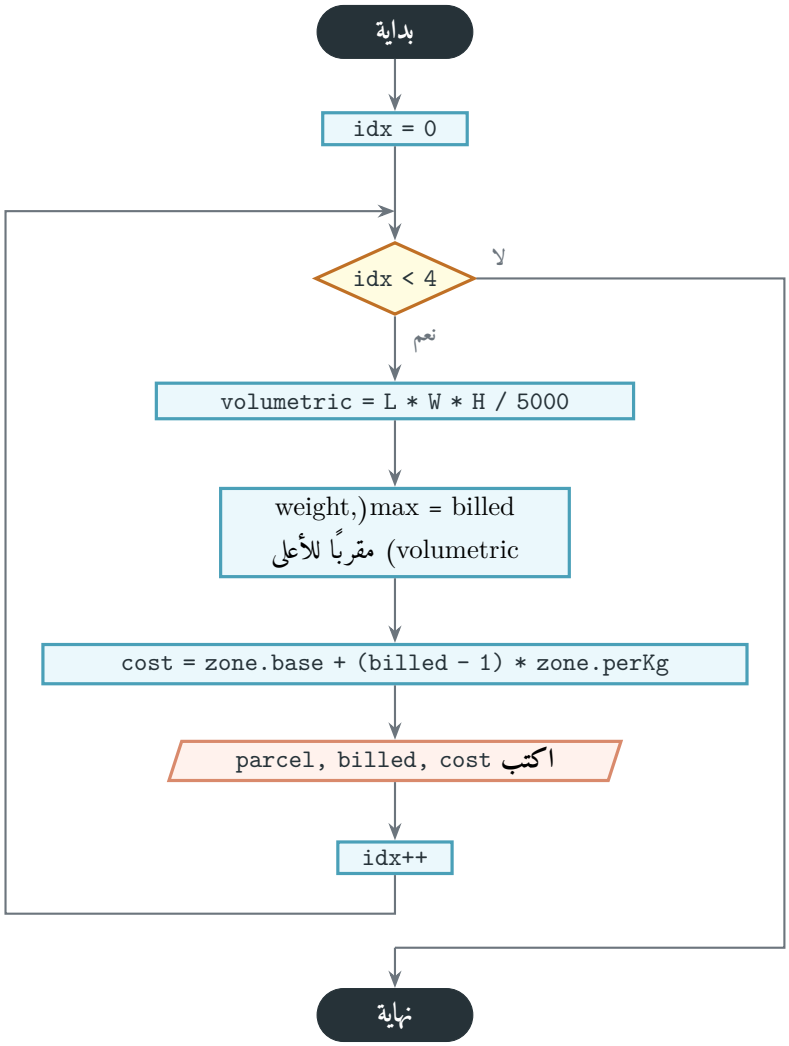
int roundUp(double value) {
    int whole = (int)value;
    return value > whole ? whole + 1 : whole;      // تقريب إلى الأعلى
}

int main(void) {
    Zone zones[3] = {{"Local", 400, 80}, {"North", 650, 120}, {"South", 900, 180}};
    Parcel parcels[4] = {{1, 0, 2.3, 30, 20, 15}, {2, 2, 0.8, 60, 40, 40},
                         {3, 1, 7.6, 40, 30, 20}, {4, 2, 12.0, 50, 40, 30}};
    double volumetric, weight;
    int idx, billed;
    for (idx = 0; idx < 4; idx++) {
        volumetric = parcels[idx].length * parcels[idx].width * parcels[idx].height / 5000;
        weight = parcels[idx].weight > volumetric ? parcels[idx].weight : volumetric;
        billed = roundUp(weight);
        printf("Parcel %d (%s): real %.1f kg, volumetric %.1f kg -> %d kg, %.0f DA\n",
               parcels[idx].id, zones[parcels[idx].zone].name, parcels[idx].weight, volumetric,
               billed, zones[parcels[idx].zone].base + (billed - 1) * zones[parcels[idx].zone].perKg);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Parcel 1 (Local): real 2.3 kg, volumetric 1.8 kg -> 3 kg, 560 DA
Parcel 2 (South): real 0.8 kg, volumetric 19.2 kg -> 20 kg, 4320 DA
Parcel 3 (North): real 7.6 kg, volumetric 4.8 kg -> 8 kg, 1490 DA
Parcel 4 (South): real 12.0 kg, volumetric 12.0 kg -> 12 kg, 2880 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.33 تقييم الزبائن بطريقة RFM

علم البيانات ★★ متوسط

تقسيم **RFM** يمنح كل زبون ثلاث علامات من 1 إلى 3: **الحداثة** (أيام منذ آخر شراء: أقل = أفضل)، **التكرار** (عدد الطلبيات)، **القيمة** (المبلغ الكلي). عرّف هيكل `Client` بهذه الحقول وبعلاماتها، واحسب العلامات بعتبات ثابتة ثم صنّف الزبائن.

خطوات الحل

1) `R`: ≤ 30 يومًا ⇒ 3، ≤ 90 ⇒ 2، وإلا 1. `F`: ≥ 10 ⇒ 3، ≥ 4 ⇒ 2، وإلا 1. `M`: ≥ 50000 ⇒ 3، ≥ 15000 ⇒ 2.
2) المجموع 8-9: "Champion"، 6-7: "Loyal"، 4-5: "At risk"، وإلا "Lost".

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct {
    char name[10];
    int recency, frequency;
    double monetary;
    int scoreR, scoreF, scoreM;
} Client;
```

```c

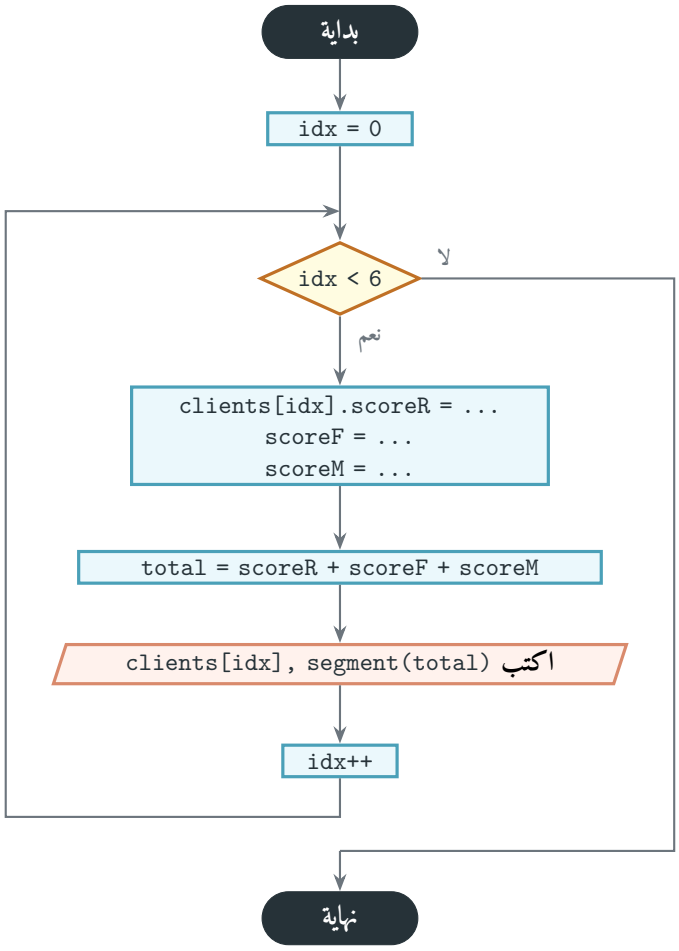
int grade(double value, double good, double medium, int higherIsBetter) {
    if (higherIsBetter) return value >= good ? 3 : value >= medium ? 2 : 1;
    return value <= good ? 3 : value <= medium ? 2 : 1;
}

int main(void) {
    Client clients[6] = {{"Amina", 12, 15, 82000, 0, 0, 0}, {"Bilal", 200, 2, 6000, 0, 0, 0},
                         {"Karim", 45, 6, 21000, 0, 0, 0}, {"Lina", 20, 3, 9000, 0, 0, 0},
                         {"Omar", 120, 11, 64000, 0, 0, 0}, {"Sara", 75, 4, 16000, 0, 0, 0}};
    char segments[4][10] = {"Lost", "At risk", "Loyal", "Champion"};
    int idx, total;
    for (idx = 0; idx < 6; idx++) {
        clients[idx].scoreR = grade(clients[idx].recency, 30, 90, 0);    // الأقل أفضل
        clients[idx].scoreF = grade(clients[idx].frequency, 10, 4, 1);
        clients[idx].scoreM = grade(clients[idx].monetary, 50000, 15000, 1);
        total = clients[idx].scoreR + clients[idx].scoreF + clients[idx].scoreM;
        printf("%-6s R%d F%d M%d = %d -> %s\n", clients[idx].name, clients[idx].scoreR,
               clients[idx].scoreF, clients[idx].scoreM, total,
               segments[total >= 8 ? 3 : total >= 6 ? 2 : total >= 4 ? 1 : 0]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Amina  R3 F3 M3 = 9 -> Champion
Bilal  R1 F1 M1 = 3 -> Lost
Karim  R2 F2 M2 = 6 -> Loyal
Lina   R3 F1 M1 = 5 -> At risk
Omar   R1 F3 M3 = 7 -> Loyal
Sara   R2 F2 M2 = 6 -> Loyal
```

المخطط الإنسيابي (Organigramme)



تمرين 9.34 منطقة تغطية التوصيلات

تجارة إلكترونية ★★ متوسط

عرّف هيكل Rect (الزاوية الدنيا والعليا) ودالة boundingBox(points, size) تعيد أصغر مستطيل يحتوي كل نقاط التسليم لسائق، ودالة overlap(first, second) تحسب مساحة تقاطع منطقتي سائقين (لتفادي التداخل).

خطوات الحل

1) المستطيل المحيط: أصغر وأكبر posX و posY.

2) التقاطع: width = min(highs) - max(lows) في كل محور، إذا كان سالبًا فلا تقاطع.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { double posX, posY; } Point;
typedef struct { Point low, high; } Rect;

Rect boundingBox(Point points[], int size) {
    Rect box;
    int idx;
    box.low = box.high = points[0];
    for (idx = 1; idx < size; idx++) {
        if (points[idx].posX < box.low.posX) box.low.posX = points[idx].posX;
```

```
        if (points[idx].posY < box.low.posY) box.low.posY = points[idx].posY;
        if (points[idx].posX > box.high.posX) box.high.posX = points[idx].posX;
        if (points[idx].posY > box.high.posY) box.high.posY = points[idx].posY;
    }
    return box;
}

double overlap(Rect first, Rect second) {
    double left = first.low.posX > second.low.posX ? first.low.posX : second.low.posX;
    double right = first.high.posX < second.high.posX ? first.high.posX : second.high.posX;
    double bottom = first.low.posY > second.low.posY ? first.low.posY : second.low.posY;
    double top = first.high.posY < second.high.posY ? first.high.posY : second.high.posY;
    if (right <= left || top <= bottom) return 0;          // لا تقاطع
    return (right - left) * (top - bottom);
}

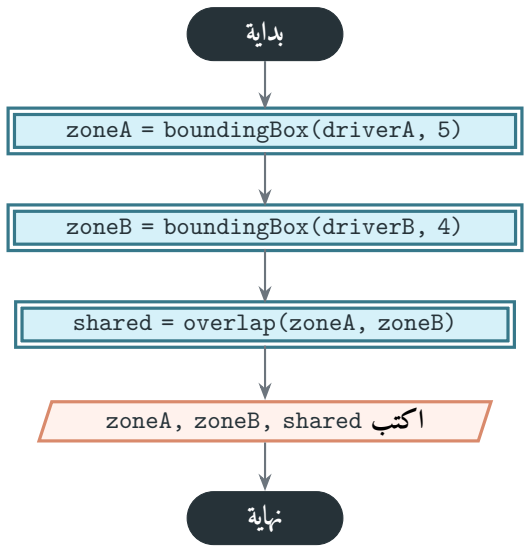
void show(char label[], Rect box) {
    printf("%s (%.0f,%.0f)-(%.0f,%.0f), area %.0f km2\n", label, box.low.posX, box.low.posY,
           box.high.posX, box.high.posY,
           (box.high.posX - box.low.posX) * (box.high.posY - box.low.posY));
}

int main(void) {
    Point driverA[5] = {{2, 3}, {8, 1}, {5, 9}, {3, 6}, {7, 7}};
    Point driverB[4] = {{6, 5}, {12, 4}, {10, 11}, {7, 8}};
    Rect zoneA = boundingBox(driverA, 5), zoneB = boundingBox(driverB, 4);
    show("Driver A:", zoneA);
    show("Driver B:", zoneB);
    printf("Shared area: %.0f km2\n", overlap(zoneA, zoneB));
    return 0;
}
```

نتيجة التنفيذ

```
Driver A: (2,1)-(8,9), area 48 km2
Driver B: (6,4)-(12,11), area 42 km2
Shared area: 10 km2
```

المخطط الإنسيابي (Organigramme)



تمرين 9.35 تصفية الكتالوج بعدة معايير — تجارة إلكترونية — متوسط

عرّف هيكل `Filter` (الفئة أو `'*'` للكل، السعر الأدنى والأعلى، التقييم الأدنى، المتوفر فقط) ودالة `matches(item, filter)`، ثم طبّق ثلاثة مرشحات مختلفة على كتالوج من 8 منتجات.

خطوات الحل

1) المنتج يُقبل إذا حقق **كل** شروط المرشح.

2) الشرط `inStockOnly` يُطبق فقط إذا كانت قيمته 1.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[14]; char category; double price, rating; int stock; } Product;
typedef struct { char category; double minPrice, maxPrice, minRating; int inStockOnly; } Filter;

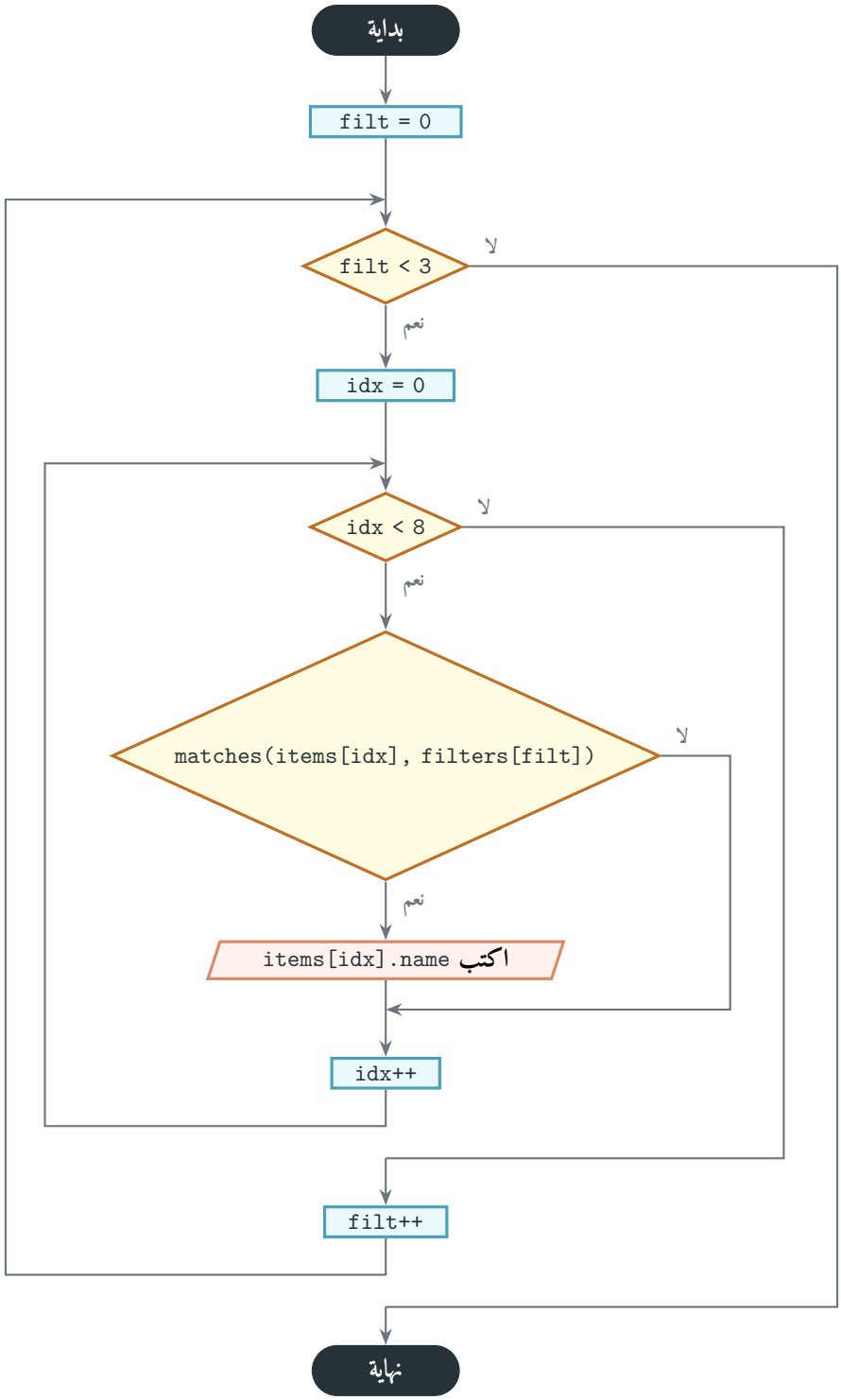
int matches(Product item, Filter filter) {
    if (filter.category != '*' && item.category != filter.category) return 0;
    if (item.price < filter.minPrice || item.price > filter.maxPrice) return 0;
    if (item.rating < filter.minRating) return 0;
    if (filter.inStockOnly && item.stock == 0) return 0;
    return 1;                                          // كل الشروط محققة
}

int main(void) {
    Product items[8] = {{"Galaxy A", 'P', 38000, 4.3, 5}, {"Redmi 13", 'P', 27500, 4.1, 0},
                        {"iPhone", 'P', 165000, 4.7, 2}, {"ThinkPad", 'L', 95000, 4.5, 3},
                        {"IdeaPad", 'L', 72000, 3.9, 6}, {"Earbuds", 'A', 5400, 4.6, 20},
                        {"Charger", 'A', 2200, 3.5, 0}, {"Hub", 'A', 3100, 4.2, 8}};
    Filter filters[3] = {{'P', 0, 50000, 4.0, 0}, {'*', 3000, 100000, 4.2, 1}, {'A', 0, 10000, 0, 1}};
    int filt, idx;
    for (filt = 0; filt < 3; filt++) {
        printf("Filter %d:", filt + 1);
        for (idx = 0; idx < 8; idx++)
            if (matches(items[idx], filters[filt])) printf(" %s;", items[idx].name);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
Filter 1: Galaxy A; Redmi 13;
Filter 2: Galaxy A; ThinkPad; Earbuds; Hub;
Filter 3: Earbuds; Hub;
```

المخطط الإنسيابي (Organigramme)



تمرين 9.36 سجل معاملات المحفظة الإلكترونية

تجارة إلكترونية

متوسط

كل معاملة في المحفظة هيكل `Transaction` (اليوم، النوع `'D'` إيداع أو `'P'` دفع أو `'R'` استرجاع، المبلغ). عالج السجل بالترتيب: ارفض أي دفع يتجاوز الرصيد، واعرض الرصيد بعد كل معاملة، وأدنى رصيد بلغته المحفظة.

خطوات الحل

1) الإيداع والاسترجاع يزيدان الرصيد؛ الدفع ينقصه إذا كان الرصيد كافيًا.
2) نحتفظ بحقل `accepted` في كل معاملة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 8

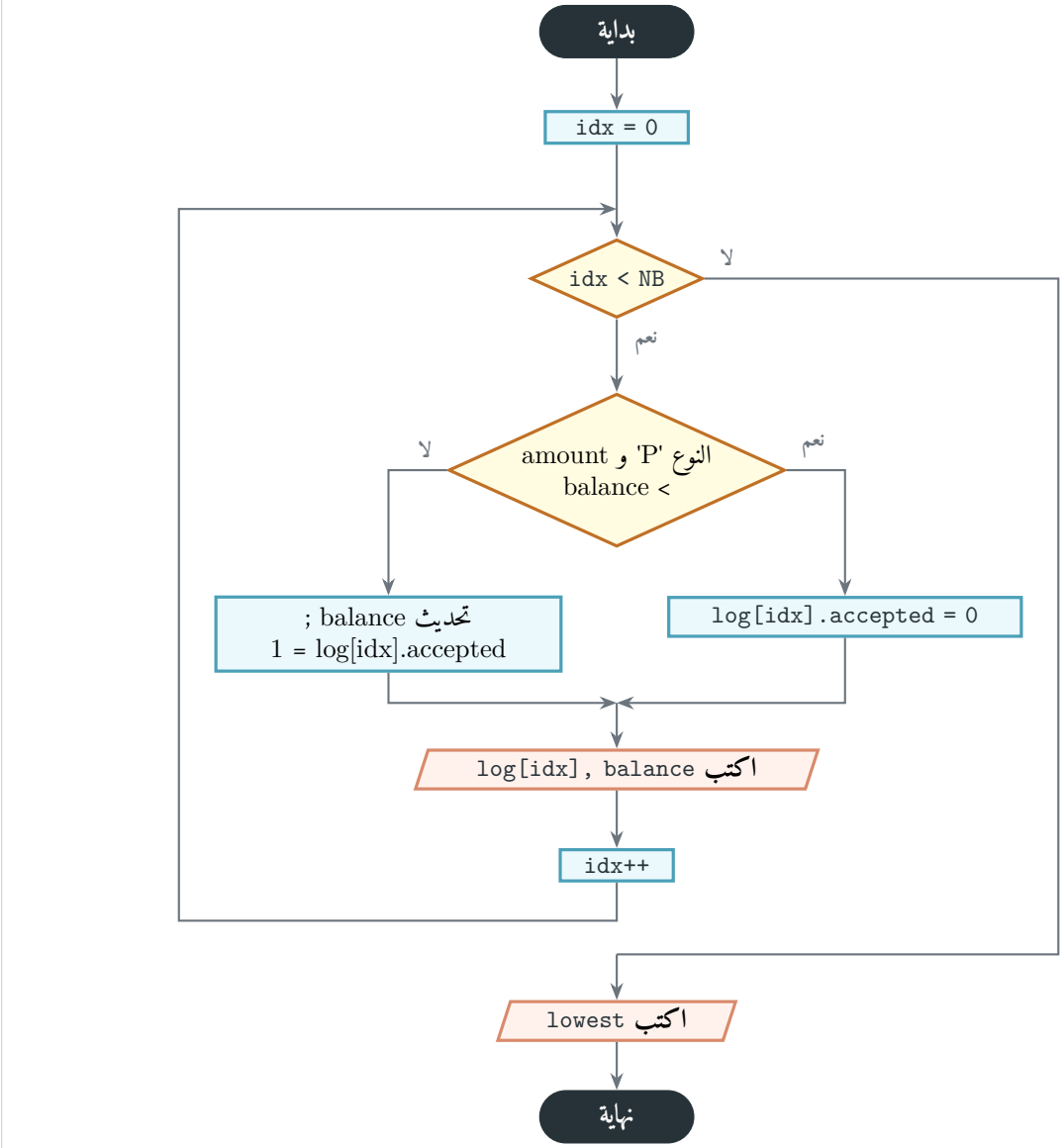
typedef struct { int day; char kind; double amount; int accepted; } Transaction;

int main(void) {
    Transaction log[NB] = {{1, 'D', 10000, 0}, {3, 'P', 4500, 0}, {4, 'P', 7000, 0},
                           {6, 'R', 1200, 0}, {9, 'P', 6200, 0}, {12, 'D', 5000, 0},
                           {15, 'P', 3300, 0}, {20, 'P', 1800, 0}};
    double balance = 0, lowest = 0;
    int idx;
    for (idx = 0; idx < NB; idx++) {
        if (log[idx].kind == 'P' && log[idx].amount > balance) {
            log[idx].accepted = 0;                      // رصيد غير كافٍ
        } else {
            balance += log[idx].kind == 'P' ? -log[idx].amount : log[idx].amount;
            log[idx].accepted = 1;
            if (balance < lowest || idx == 0) lowest = balance;
        }
        printf("Day %2d %c %7.2f %-8s balance %8.2f\n", log[idx].day, log[idx].kind,
               log[idx].amount, log[idx].accepted ? "ok" : "REFUSED", balance);
    }
    printf("Lowest balance: %.2f DA\n", lowest);
    return 0;
}
```

نتيجة التنفيذ

```
Day  1 D 10000.00 ok       balance 10000.00
Day  3 P 4500.00 ok        balance  5500.00
Day  4 P 7000.00 REFUSED   balance  5500.00
Day  6 R 1200.00 ok        balance  6700.00
Day  9 P 6200.00 ok        balance   500.00
Day 12 D 5000.00 ok        balance  5500.00
Day 15 P 3300.00 ok        balance  2200.00
Day 20 P 1800.00 ok        balance   400.00
Lowest balance: 400.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 9.37 اختيار المورد بالعلامة المرجحة

علم البيانات متوسط

لكل مورد هيكل يحتوي على: السعر، مدة التسليم بالأيام، نسبة العيوب، والتقييم. نطبّع كل معيار إلى `[0, 1]` (الأفضل = 1)، ثم نحسب علامة مرجحة: السعر 40%، التسليم 25%، العيوب 20%، التقييم 15%. اعرض الترتيب.

خطوات الحل

1) للمعايير التي "الأقل أفضل" (السعر، التسليم، العيوب): `(max - value) / (max - min)`.
2) للتقييم: `(value - min) / (max - min)`.
3) العلامة = مجموع الأوزان × العلامات المطبّعة.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 4

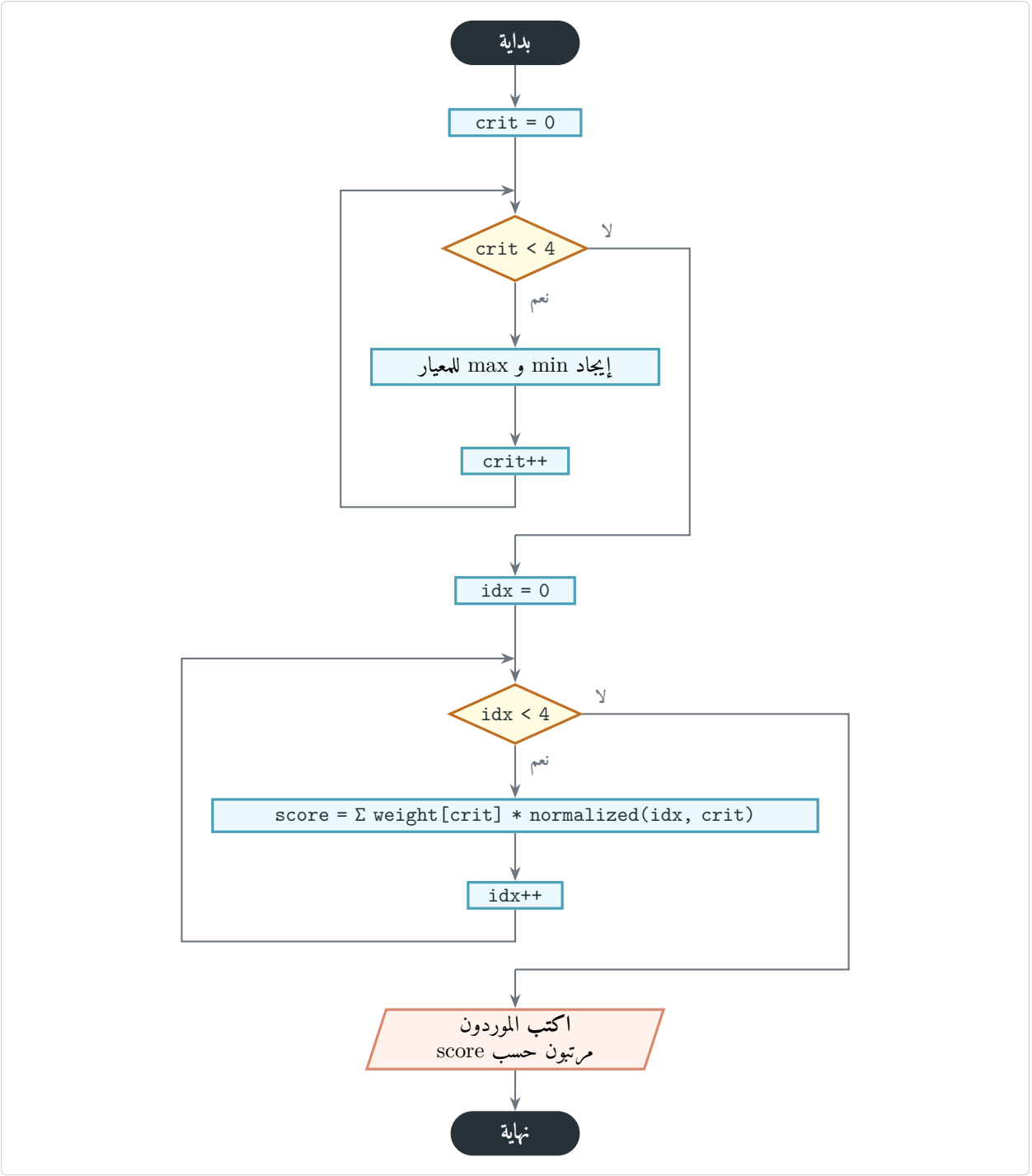
typedef struct { char name[10]; double values[4]; double score; } Supplier;

int main(void) {
    Supplier list[NB] = {{"AlphaTec", {1200, 5, 2.0, 4.1}, 0}, {"BetaCom", {1050, 9, 3.5, 3.8}, 0},
                         {"Gamma", {1350, 3, 0.8, 4.6}, 0}, {"Delta", {1100, 6, 1.5, 4.0}, 0}};
    double weights[4] = {0.40, 0.25, 0.20, 0.15};
    int lowerBetter[4] = {1, 1, 1, 0};
    double low, high, norm;
    Supplier tmp;
    int crit, idx, pass;
    for (crit = 0; crit < 4; crit++) {
        low = high = list[0].values[crit];
        for (idx = 1; idx < NB; idx++) {
            if (list[idx].values[crit] < low) low = list[idx].values[crit];
            if (list[idx].values[crit] > high) high = list[idx].values[crit];
        }
        for (idx = 0; idx < NB; idx++) {
            norm = lowerBetter[crit] ? (high - list[idx].values[crit]) / (high - low)
                                     : (list[idx].values[crit] - low) / (high - low);
            list[idx].score += weights[crit] * norm;          // مساهمة المعيار
        }
    }
    for (pass = 0; pass < NB - 1; pass++)
        for (idx = 0; idx < NB - 1 - pass; idx++)
            if (list[idx].score < list[idx + 1].score) {
                tmp = list[idx]; list[idx] = list[idx + 1]; list[idx + 1] = tmp;
            }
    for (idx = 0; idx < NB; idx++)
        printf("%d. %-9s score %.3f\n", idx + 1, list[idx].name, list[idx].score);
    return 0;
}
```

نتيجة التنفيذ

```
1. Delta     score 0.644
2. Gamma     score 0.600
3. AlphaTec  score 0.534
4. BetaCom   score 0.400
```

المخطط الإنسيابي (Organigramme)



تمرين 9.38 أكبر انخفاض في تاريخ السعر

علم البيانات ★★ متوسط

لكل منتج هيكل يحتوي على اسمه وجدول أسعاره خلال 10 أسابيع. اكتب دالة `largestDrop(item, from, to)` تجد **أكبر انخفاض** بين أسبوع وأسبوع لاحق (ليس بالضرورة متتاليين) في مرور واحد، وتكتب الأسبوعين في جدول.

خطوات الحل

1) نحتفظ بأعلى سعر سابق `peak` وأسبوعه.

2) لكل أسبوع: الانخفاض = `peak - price`؛ إذا كان أكبر من الأفضل نحفظه.

3) مرور واحد: تعقيد خطي بدل مقارنة كل الأزواج.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[12]; double prices[10]; } History;

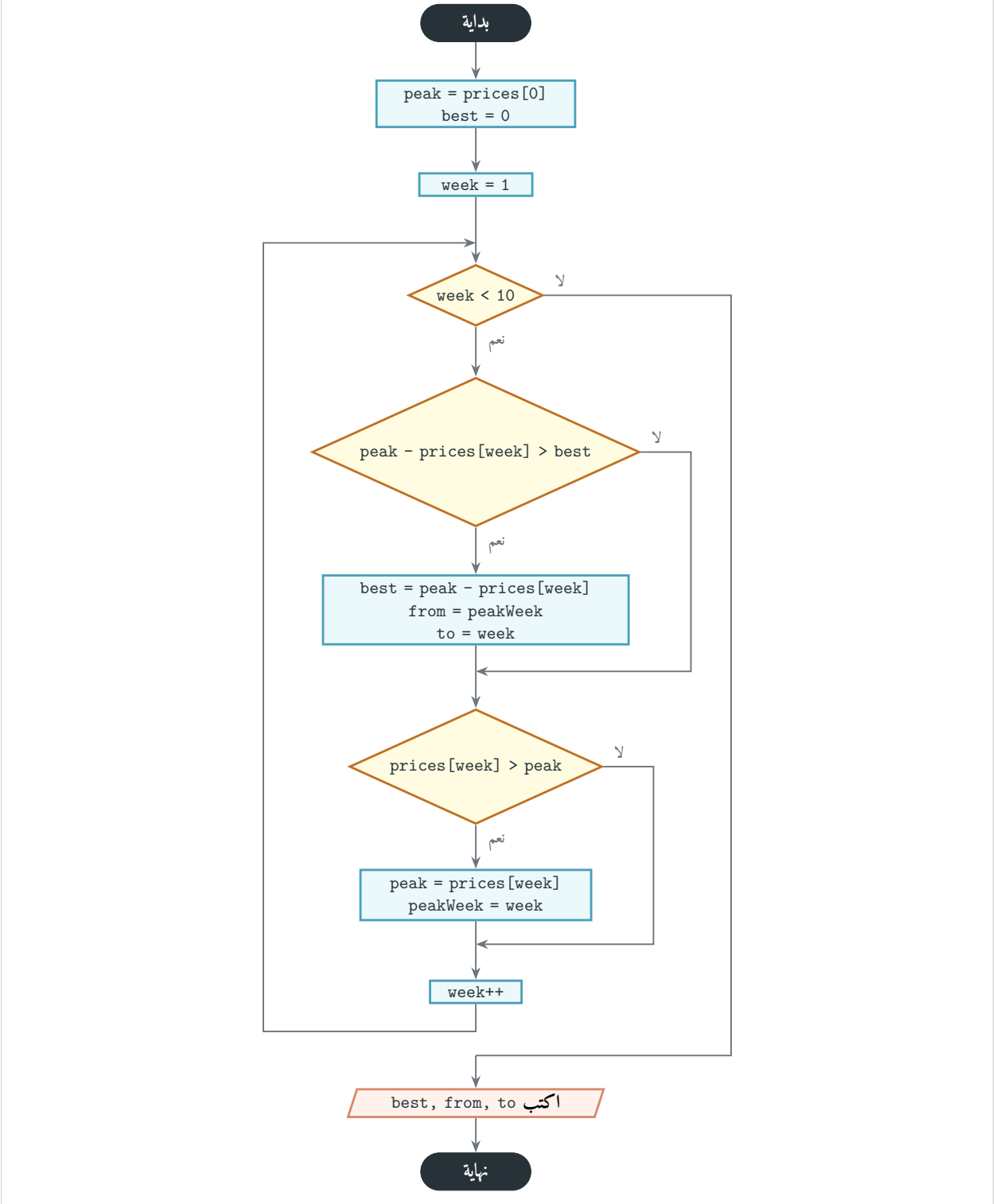
double largestDrop(History item, int weeks[]) {
    double peak = item.prices[0], best = 0;
    int week, peakWeek = 0;
    weeks[0] = weeks[1] = 0;
    for (week = 1; week < 10; week++) {
        if (peak - item.prices[week] > best) {
            best = peak - item.prices[week];
            weeks[0] = peakWeek;
            weeks[1] = week;
        }
        if (item.prices[week] > peak) { peak = item.prices[week]; peakWeek = week; }   // قمة جديدة
    }
    return best;
}

int main(void) {
    History items[3] = {{"Phone", {32000, 33500, 31000, 34500, 30000, 29000, 31500, 28000, 29500, 30500}},
                        {"Laptop", {89000, 87000, 91000, 90500, 88000, 92000, 95000, 94000, 93000, 96000}},
                        {"Earbuds", {5400, 5200, 5000, 4800, 4600, 4500, 4400, 4300, 4200, 4100}}};
    int idx, weeks[2];
    double drop;
    for (idx = 0; idx < 3; idx++) {
        drop = largestDrop(items[idx], weeks);
        printf("%-8s largest drop %6.0f DA (week %d -> week %d, -%.1f%%)\n", items[idx].name,
               drop, weeks[0] + 1, weeks[1] + 1, 100 * drop / items[idx].prices[weeks[0]]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Phone    largest drop   6500 DA (week 4 -> week 8, -18.8%)
Laptop   largest drop   3000 DA (week 3 -> week 5, -3.3%)
Earbuds  largest drop   1300 DA (week 1 -> week 10, -24.1%)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.39 مقارنة إصدارات تطبيق المتجر

تجارة إلكترونية

متوسط

إصدار التطبيق يُكتب `major.minor.patch` مثل `2.10.3`. اقرأ الإصدارات كنصوص، وحوّلها يدويًا إلى هيكل `Version`، واكتب دالة `compareVersions`، ثم حدد لكل جهاز هل يحتاج إلى تحديث نحو الإصدار الأخير `2.10.3`، مع التنبيه إذا كان التحديث رئيسيًا.

خطوات الحل

1) التحويل: نقرأ الأرقام حرفًا بحرف ونمر إلى الحقل التالي عند كل نقطة.

2) المقارنة: `major` ثم `minor` ثم `patch` (المقارنة النصية خاطئة: `"2.10" > "2.9"` نصيًا).

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int major, minor, patch; } Version;

Version parseVersion(char text[]) {
    int parts[3] = {0, 0, 0}, field = 0, pos;
    Version result;
    for (pos = 0; text[pos] != '\0' && field < 3; pos++) {
        if (text[pos] == '.') field++;                      // الحقل التالي
        else parts[field] = parts[field] * 10 + (text[pos] - '0');
    }
    result.major = parts[0];
    result.minor = parts[1];
    result.patch = parts[2];
    return result;
}

int compareVersions(Version first, Version second) {
    if (first.major != second.major) return first.major - second.major;
    if (first.minor != second.minor) return first.minor - second.minor;
    return first.patch - second.patch;
}

int main(void) {
    Version latest = {2, 10, 3}, device;
    char text[20];
    int nb, idx;
    scanf("%d", &nb);
    for (idx = 0; idx < nb; idx++) {
        scanf("%19s", text);
        device = parseVersion(text);
        if (compareVersions(device, latest) < 0)
            printf("%-8s -> update to 2.10.3%s\n", text, device.major < latest.major ? " (MAJOR)" : "");
        else
            printf("%-8s -> up to date\n", text);
    }
    return 0;
}
```

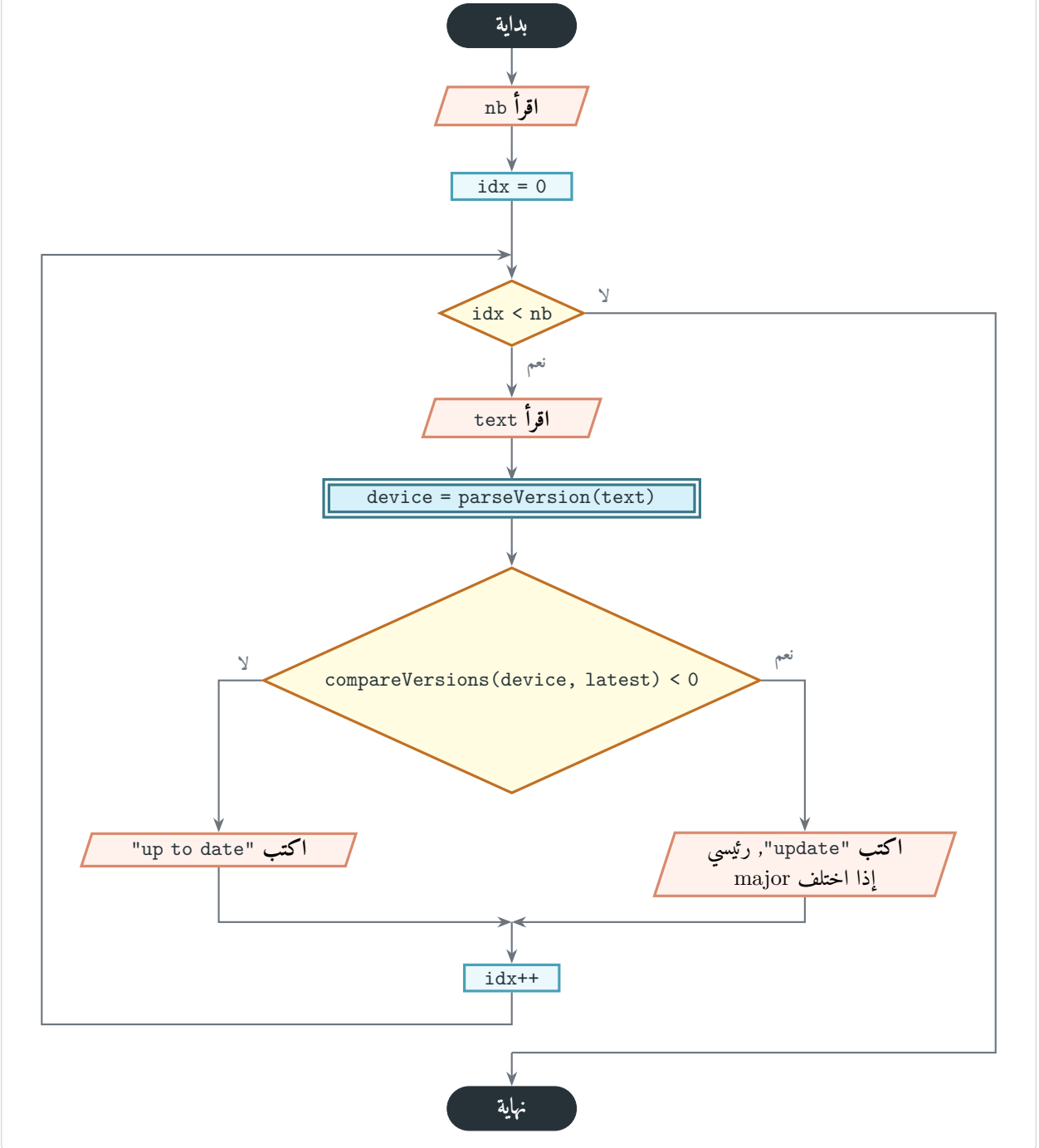
المدخلات

```
4
2.9.12
2.10.3
1.15.0
2.10.1
```

نتيجة التنفيذ

```
2.9.12   -> update to 2.10.3
2.10.3   -> up to date
1.15.0   -> update to 2.10.3 (MAJOR)
2.10.1   -> update to 2.10.3
```

المخطط الإنسيابي (Organigramme)



تمرين 9.40 نتائج اختبار A/B لصفحة المنتج

علم البيانات ★★ متوسط

اختبر المتجر نسختين من صفحة المنتج. لكل نسخة هيكل `Variant` (الاسم، عدد الزوار، عدد المشترين). احسب معدل التحويل لكل نسخة، والارتفاع النسبي (lift)، وإحصائية z لاختبار الفرق بين نسبتين، وقرر إن كان الفرق معنوياً (`|z| > 1.96`).

خطوات الحل

1) `rate = buyers / visitors`؛ `lift = (rateB - rateA) / rateA`.
2) النسبة المجمعة

$$\text{pooled} = (\text{buyersA} + \text{buyersB}) / (\text{visitorsA} + \text{visitorsB})$$

$$z = (\text{rateB} - \text{rateA}) / \text{sqrt}(\text{pooled} \cdot (1 - \text{pooled}) \cdot (1/nA + 1/nB)) \quad (3$$

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[16]; int visitors, buyers; } Variant;

double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    for (iter = 0; iter < 60; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

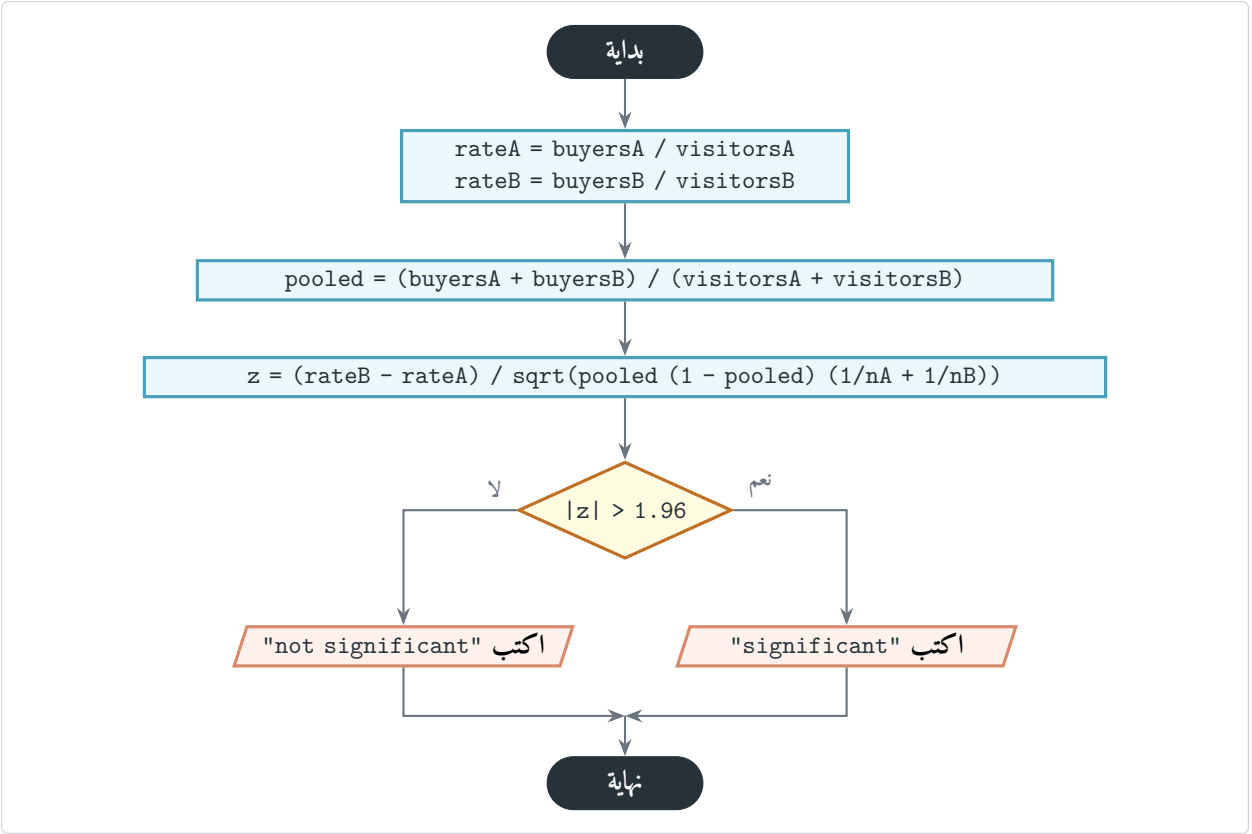
double rate(Variant variant) { return (double)variant.buyers / variant.visitors; }

int main(void) {
    Variant tests[2][2] = {{{"A: old page", 4800, 192}, {"B: new photos", 4750, 238}},
                           {{"A: green btn", 1200, 54}, {"B: red btn", 1180, 59}}};
    double rateA, rateB, pooled, zValue;
    int test;
    for (test = 0; test < 2; test++) {
        rateA = rate(tests[test][0]);
        rateB = rate(tests[test][1]);
        pooled = (double)(tests[test][0].buyers + tests[test][1].buyers)
               / (tests[test][0].visitors + tests[test][1].visitors);
        zValue = (rateB - rateA) / mySqrt(pooled * (1 - pooled) *
                 (1.0 / tests[test][0].visitors + 1.0 / tests[test][1].visitors));
        printf("%s %.2f%% vs %s %.2f%%\n", tests[test][0].name, 100 * rateA,
               tests[test][1].name, 100 * rateB);
        printf("  lift %+.1f%%, z = %.2f -> %s\n", 100 * (rateB - rateA) / rateA, zValue,
               (zValue > 1.96 || zValue < -1.96) ? "significant" : "not significant");
    }
    return 0;
}
```

نتيجة التنفيذ

```
A: old page 4.00% vs B: new photos 5.01%
  lift +25.3%, z = 2.38 -> significant
A: green btn 4.50% vs B: red btn 5.00%
  lift +11.1%, z = 0.57 -> not significant
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 9.41 قاعدة بيانات مصغرة بالربط بين الجداول

تجارة إلكترونية ★★★ صعب

لدينا ثلاثة جداول من الهياكل كما في قاعدة بيانات: `customers` و `products` و `orders` (كل طلبية تشير إلى زبون ومنتج بمعرّفيهما). اكتب دوال البحث بالمعرّف، ثم أنتج تقريرًا يشبه الاستعلام `SELECT city, SUM(qty*price) ... GROUP BY city ORDER BY total DESC`.

خطوات الحل

1) `findCustomer(id)` و `findProduct(id)` تعيدان الفهرس (الربط **join**).
2) لكل طلبية: مدينة الزبون + مبلغ السطر، ثم التجميع حسب المدينة في جدول هياكل `CityTotal`.
3) ترتيب النتيجة تنازليًا.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBC 5
#define NBP 4
#define NBO 10

typedef struct { int id; char name[10]; char city[10]; } Customer;
typedef struct { int id; char name[10]; double price; } Product;
typedef struct { int customerId, productId, qty; } Order;
typedef struct { char city[10]; double total; int orders; } CityTotal;

Customer customers[NBC] = {{1, "Amina", "Oran"}, {2, "Karim", "Algiers"}, {3, "Lina", "Oran"},
                           {4, "Omar", "Setif"}, {5, "Sara", "Algiers"}};
Product products[NBP] = {{10, "Phone", 32000}, {11, "Case", 900}, {12, "Earbuds", 5400}, {13, "Laptop",
    ↪ 89000}};
Order orders[NBO] = {{1, 10, 1}, {2, 11, 3}, {3, 12, 2}, {4, 13, 1}, {5, 10, 2},
                     {1, 12, 1}, {2, 13, 1}, {3, 11, 5}, {4, 11, 2}, {5, 12, 1}};

int findCustomer(int id) {
    int idx;
    for (idx = 0; idx < NBC; idx++) if (customers[idx].id == id) return idx;
    return -1;
}

int findProduct(int id) {
    int idx;
    for (idx = 0; idx < NBP; idx++) if (products[idx].id == id) return idx;
    return -1;
}

int sameText(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos]) pos++;
    return first[pos] == second[pos];
}

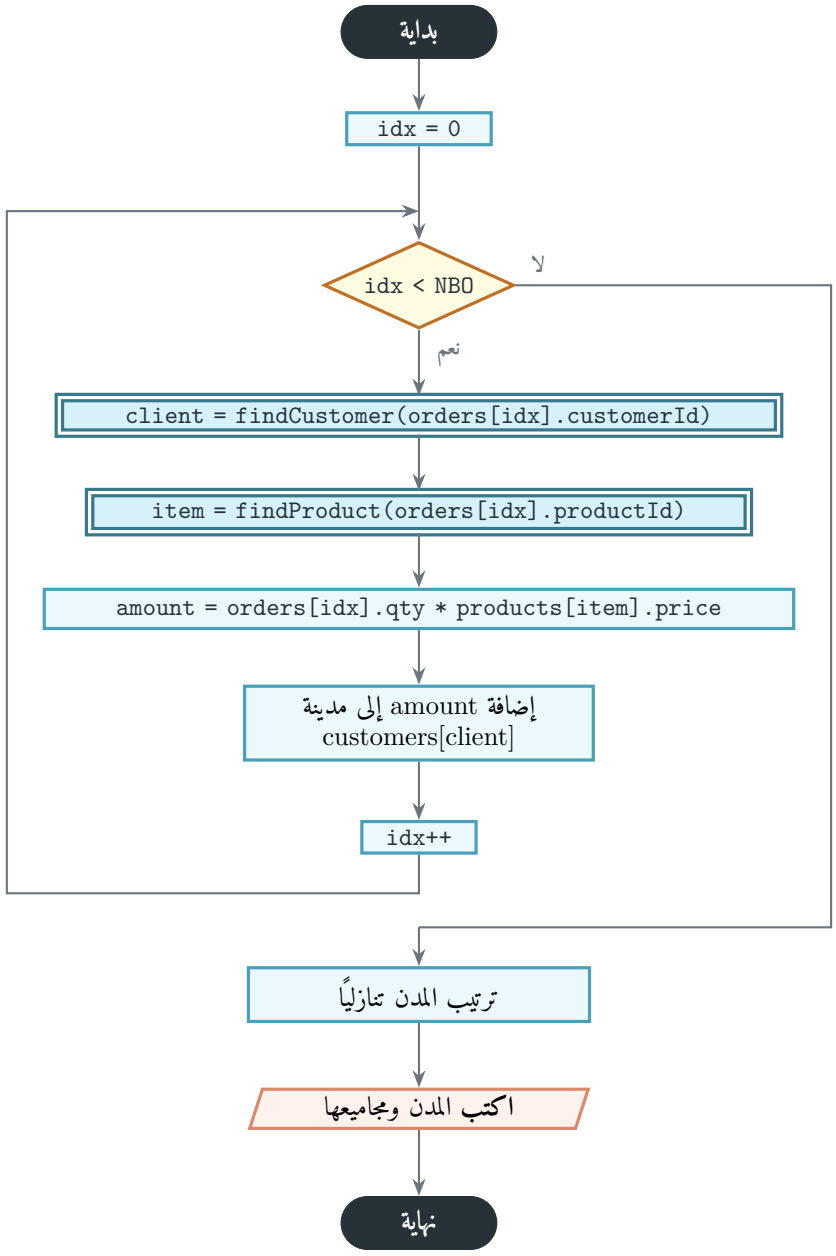
int main(void) {
    CityTotal report[NBC], tmp;
    int nbCities = 0, idx, client, item, pos, letter, pass;
    for (idx = 0; idx < NBO; idx++) {
        client = findCustomer(orders[idx].customerId);       // ربط مع الزبائن
        item = findProduct(orders[idx].productId);           // ربط مع المنتجات
        for (pos = 0; pos < nbCities && !sameText(report[pos].city, customers[client].city); pos++) {}
```

```
43          if (pos == nbCities) {                              // GROUP BY city
44              for (letter = 0; letter < 10; letter++) report[pos].city[letter] =
        ↪ customers[client].city[letter];
45              report[pos].total = 0;
46              report[pos].orders = 0;
47              nbCities++;
48          }
49          report[pos].total += orders[idx].qty * products[item].price;
50          report[pos].orders++;
51      }
52      for (pass = 0; pass < nbCities - 1; pass++)             // ORDER BY total DESC
53          for (idx = 0; idx < nbCities - 1 - pass; idx++)
54              if (report[idx].total < report[idx + 1].total) {
55                  tmp = report[idx]; report[idx] = report[idx + 1]; report[idx + 1] = tmp;
56              }
57      printf("city      orders      total\n");
58      for (idx = 0; idx < nbCities; idx++)
59          printf("%-9s %6d %10.0f\n", report[idx].city, report[idx].orders, report[idx].total);
60      return 0;
61  }
```

نتيجة التنفيذ

```
city      orders      total
Algiers        4     161100
Setif          2      90800
Oran           4      52700
```

المخطط الإنسيابي (Organigramme)



تمرين 9.42 طابور الأولوية للطلبيات بكومة ثنائية

تجارة إلكترونية صعب

تُعالج الطلبيات حسب الأولوية (التوصيل السريع أولًا، ثم الأقدم). اكتب هيكل `Heap` (جدول من `Order` + الحجم) ودوال `push` (إدراج مع الصعود) و `pop` (سحب الجذر مع النزول) لكومة عظمى، ثم عالج الطلبيات بالترتيب الصحيح.

خطوات الحل

1) الأب في الكومة `(idx - 1) / 2` والأبناء `2*idx + 1` و `2*idx + 2`.
2) `higher(first, second)`: أولوية أكبر، وعند التساوي رقم أصغر (أقدم).
3) `push`: نضع العنصر في الأخير ثم نبدله مع أبيه ما دام أعلى منه.

4) pop: نأخذ الجذر، نضع الأخير مكانه، ثم ننزله نحو الابن الأعلى.

البرنامج بلغة C

```c
#include <stdio.h>

#define CAP 20

typedef struct { int number; int priority; char client[10]; } Order;
typedef struct { Order items[CAP]; int size; } Heap;

int higher(Order first, Order second) {
    if (first.priority != second.priority) return first.priority > second.priority;
    return first.number < second.number;            // الأقدم أولًا
}

void swapOrders(Heap heap[], int first, int second) {
    Order tmp = heap[0].items[first];
    heap[0].items[first] = heap[0].items[second];
    heap[0].items[second] = tmp;
}

void push(Heap heap[], Order order) {
    int idx = heap[0].size++;
    heap[0].items[idx] = order;
    while (idx > 0 && higher(heap[0].items[idx], heap[0].items[(idx - 1) / 2])) {
        swapOrders(heap, idx, (idx - 1) / 2);       // صعود
        idx = (idx - 1) / 2;
    }
}

Order pop(Heap heap[]) {
    Order top = heap[0].items[0];
    int idx = 0, child;
    heap[0].items[0] = heap[0].items[--heap[0].size];
    while (2 * idx + 1 < heap[0].size) {            // نزول
        child = 2 * idx + 1;
        if (child + 1 < heap[0].size && higher(heap[0].items[child + 1], heap[0].items[child]))
            child++;
        if (!higher(heap[0].items[child], heap[0].items[idx])) break;
        swapOrders(heap, idx, child);
        idx = child;
    }
    return top;
}

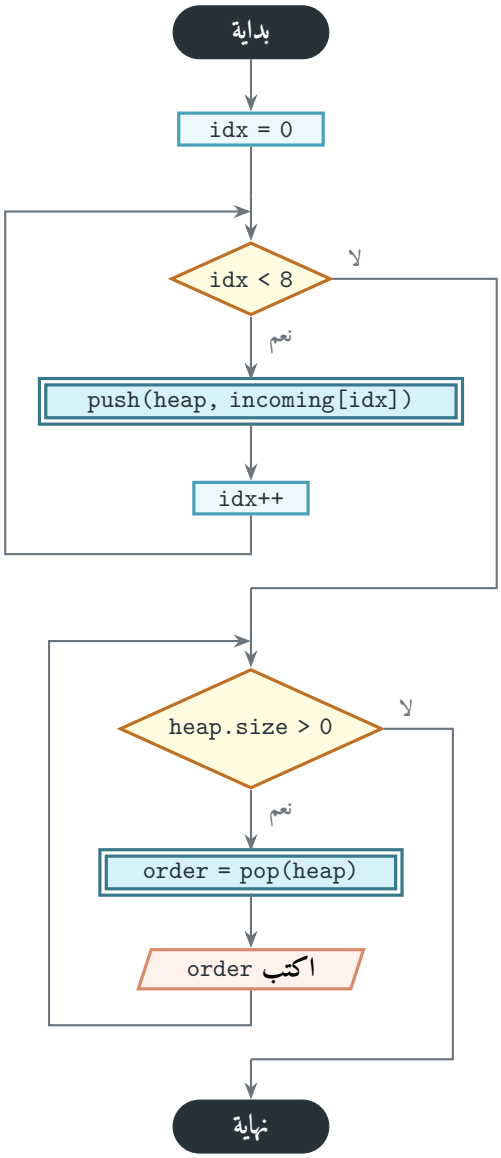
int main(void) {
    Order incoming[8] = {{101, 1, "Amina"}, {102, 3, "Karim"}, {103, 2, "Lina"}, {104, 3, "Omar"},
                         {105, 1, "Sara"}, {106, 2, "Nadir"}, {107, 3, "Rania"}, {108, 1, "Yacine"}};
    Heap heap[1];
    Order order;
    int idx;
    heap[0].size = 0;
    for (idx = 0; idx < 8; idx++) push(heap, incoming[idx]);
    printf("Root after inserts: #%d\n", heap[0].items[0].number);
    while (heap[0].size > 0) {
        order = pop(heap);
        printf("Process #%d (priority %d, %s)\n", order.number, order.priority, order.client);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Root after inserts: #102
Process #102 (priority 3, Karim)
Process #104 (priority 3, Omar)
Process #107 (priority 3, Rania)
Process #103 (priority 2, Lina)
Process #106 (priority 2, Nadir)
```

```
Process #101 (priority 1, Amina)
Process #105 (priority 1, Sara)
Process #108 (priority 1, Yacine)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.43 التراجع في السلة بمكدس

تجارة إلكترونية

صعب

يتيح الموقع زر **تراجع** (Undo) و**إعادة** (Redo) لعمليات السلة. اكتب هيكل `Stack` لعمليات `Action` (نوع الإضافة/الحذف، المنتج، الكمية)، ودوال `pushAction` و `popAction`، ثم نفّذ سلسلة أوامر: كل عملية جديدة تُكدس في مكدس التراجع وتفرغ مكدس الإعادة.

خطوات الحل

1) `aN`: إضافة المنتج N، `rN`: حذفه، `u`: تراجع، `y`: إعادة.

2) التراجع: نسحب من `undo`، نطبّق **العملية المعاكسة**، ونكدسها في `redo`.

3) الإعادة: نسحب من redo، نطبّقها، ونكدسها في undo.

البرنامج بلغة C

```c
#include <stdio.h>

#define CAP 20

typedef struct { char kind; int product; } Action;
typedef struct { Action items[CAP]; int top; } Stack;

int cart[5];

void pushAction(Stack stack[], Action action) { stack[0].items[stack[0].top++] = action; }
Action popAction(Stack stack[]) { return stack[0].items[--stack[0].top]; }

void apply(Action action, int reverse) {
    int add = (action.kind == 'a') != reverse;        // المعاكسة تقلب العملية
    cart[action.product] += add ? 1 : -1;
}

void show(char command[]) {
    int idx;
    printf("%-3s cart:", command);
    for (idx = 1; idx < 5; idx++) if (cart[idx] > 0) printf(" P%d x%d", idx, cart[idx]);
    printf("\n");
}

int main(void) {
    Stack undo[1], redo[1];
    Action action;
    char command[4];
    undo[0].top = redo[0].top = 0;
    while (scanf("%3s", command) == 1) {
        if (command[0] == 'a' || command[0] == 'r') {
            action.kind = command[0];
            action.product = command[1] - '0';
            apply(action, 0);
            pushAction(undo, action);
            redo[0].top = 0;                          // عملية جديدة تلغي الإعادة
        } else if (command[0] == 'u' && undo[0].top > 0) {
            action = popAction(undo);
            apply(action, 1);
            pushAction(redo, action);
        } else if (command[0] == 'y' && redo[0].top > 0) {
            action = popAction(redo);
            apply(action, 0);
            pushAction(undo, action);
        }
        show(command);
    }
    return 0;
}
```

المدخلات

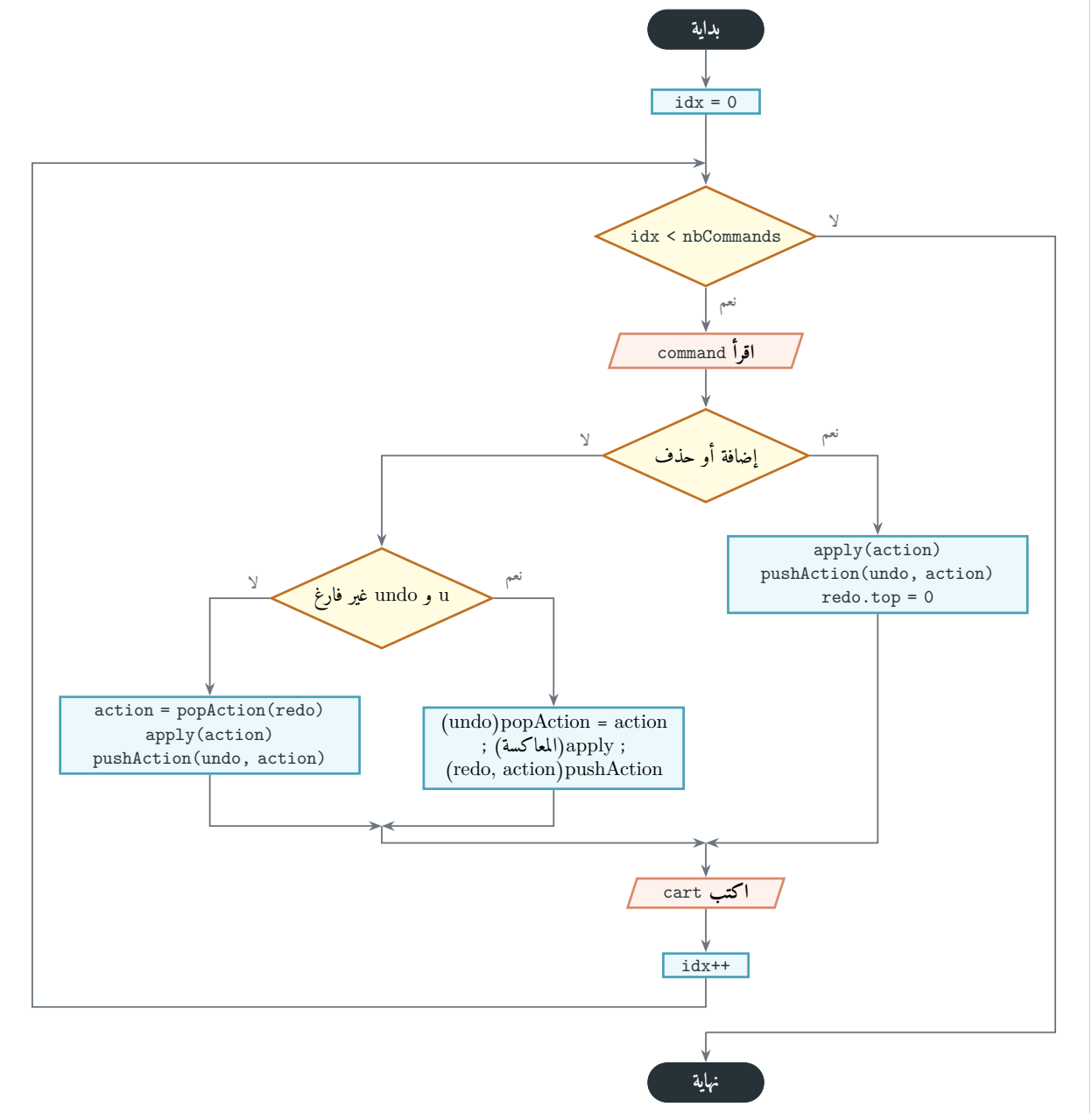
```
a1 a2 a1 r2 u u u y y a3 u
```

نتيجة التنفيذ

```
a1  cart: P1 x1
a2  cart: P1 x1 P2 x1
```

```
a1  cart: P1 x2 P2 x1
r2  cart: P1 x2
u   cart: P1 x2 P2 x1
u   cart: P1 x1 P2 x1
u   cart: P1 x1
y   cart: P1 x1 P2 x1
y   cart: P1 x2 P2 x1
a3  cart: P1 x2 P2 x1 P3 x1
u   cart: P1 x2 P2 x1
```

المخطط الإنسيابي (Organigramme)



تمرين 9.44 طابور تذاكر الدعم الدائري

تجارة إلكترونية ★★★ صعب

يستقبل فريق الدعم التذاكر في **طابور دائري** بسعة 5. اكتب هيكل `Queue` (جدول، رأس، ذيل، عدد) ودوال `enqueue`

(ترفض إذا امتلأ الطابور) و dequeue. حاكِ يومًا من العمل: في كل دقيقة تصل تذاكر و/أو يعالج الموظف تذكرة، واحسب متوسط وقت الانتظار.

خطوات الحل

1) الذيل يدور: tail = (tail + 1) % CAP، وكذلك الرأس.
2) كل تذكرة تحفظ دقيقة وصولها؛ الانتظار = دقيقة المعالجة - دقيقة الوصول.
3) الحدث + وصول تذكرة، - معالجة تذكرة.

البرنامج بلغة C

```c
#include <stdio.h>

#define CAP 5

typedef struct { int id, arrival; } Ticket;
typedef struct { Ticket items[CAP]; int head, tail, count; } Queue;

int enqueue(Queue queue[], Ticket ticket) {
    if (queue[0].count == CAP) return 0;                // الطابور ممتلئ
    queue[0].items[queue[0].tail] = ticket;
    queue[0].tail = (queue[0].tail + 1) % CAP;          // دوران
    queue[0].count++;
    return 1;
}

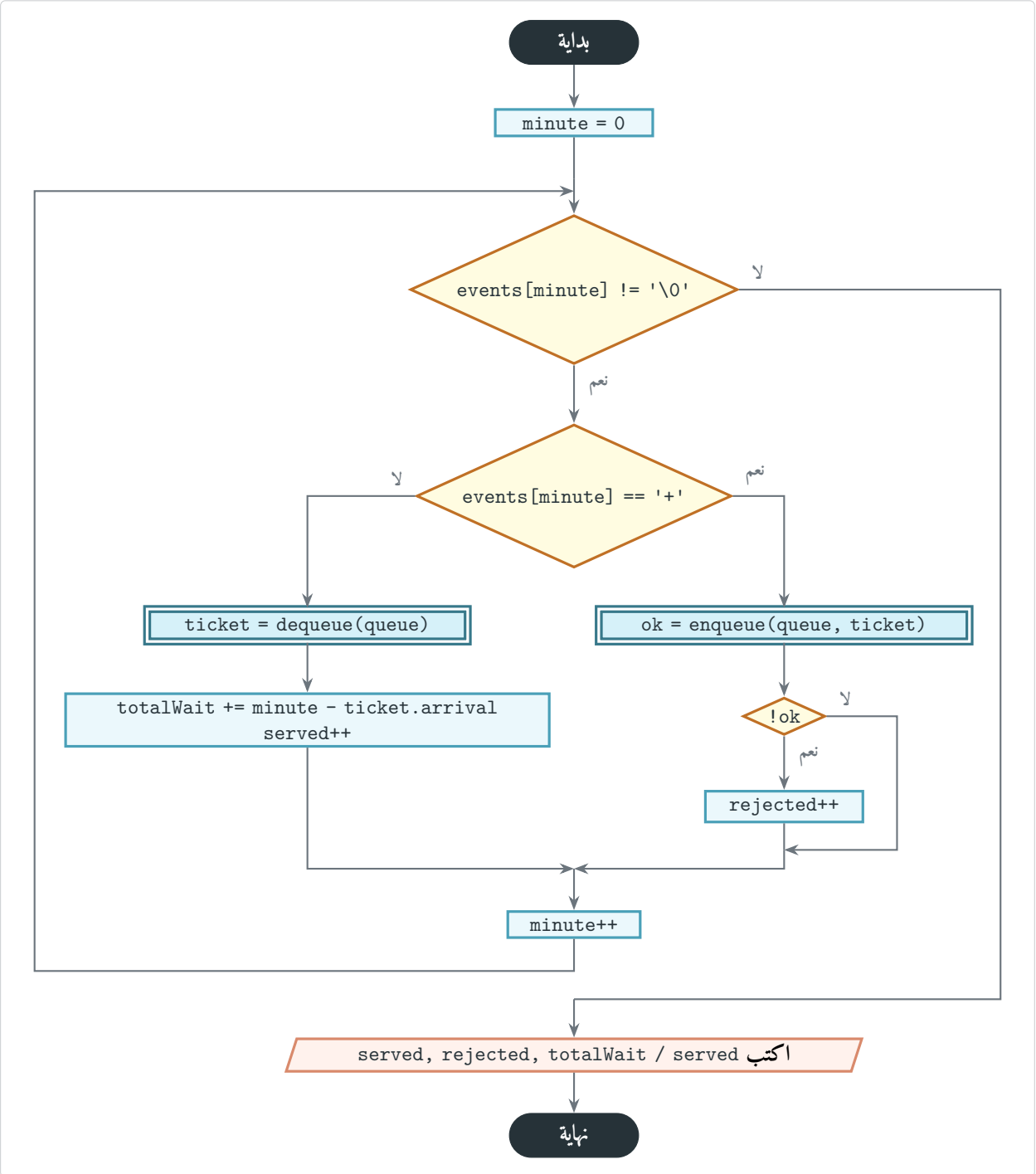
Ticket dequeue(Queue queue[]) {
    Ticket ticket = queue[0].items[queue[0].head];
    queue[0].head = (queue[0].head + 1) % CAP;
    queue[0].count--;
    return ticket;
}

int main(void) {
    char events[] = "+++-++-+++++--+-----+-";
    Queue queue[1] = {{{{0, 0}}, 0, 0, 0}};
    Ticket ticket;
    int minute, nextId = 1, served = 0, rejected = 0, totalWait = 0;
    for (minute = 0; events[minute] != '\0'; minute++) {
        if (events[minute] == '+') {
            ticket.id = nextId++;
            ticket.arrival = minute;
            if (!enqueue(queue, ticket)) {
                rejected++;
                printf("t=%2d ticket %d rejected (queue full)\n", minute, ticket.id);
            }
        } else if (queue[0].count > 0) {
            ticket = dequeue(queue);
            totalWait += minute - ticket.arrival;
            served++;
        }
    }
    printf("Served %d, rejected %d, still waiting %d\n", served, rejected, queue[0].count);
    printf("Average wait: %.2f minutes\n", (double)totalWait / served);
    return 0;
}
```

نتيجة التنفيذ

```
t= 9 ticket 8 rejected (queue full)
t=10 ticket 9 rejected (queue full)
t=11 ticket 10 rejected (queue full)
Served 9, rejected 3, still waiting 0
Average wait: 6.67 minutes
```

المخطط الإنسيابي (Organigramme)



تمرين 9.45 أقصر طريق: توصيل بخوارزمية ديكسترا — تجارة إلكترونية — صعب ★★★

شبكة الطرق بين 7 نقاط (المستودع 0) ممثلة بجدول هياكل Road (من، إلى، المسافة) في الاتجاهين. اكتب خوارزمية **ديكسترا**

لحساب أقصر مسافة من المستودع إلى كل نقطة، مع إعادة بناء المسار عبر جدول `previous`.

خطوات الحل

1) `dist[node] = ∞` ما عدا المستودع 0، `done[node] = 0`.
2) نكرر: نختار أقرب عقدة غير منتهية `current`، ونعلمها منتهية، ثم نرخي كل طريق يخرج منها: إذا `dist[current] + length < dist[other]` نحدث المسافة و `previous`.
3) المسار يُبنى بالرجوع من الهدف عبر `previous` حتى 0.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBN 7
#define NBR 11
#define INF 1000000

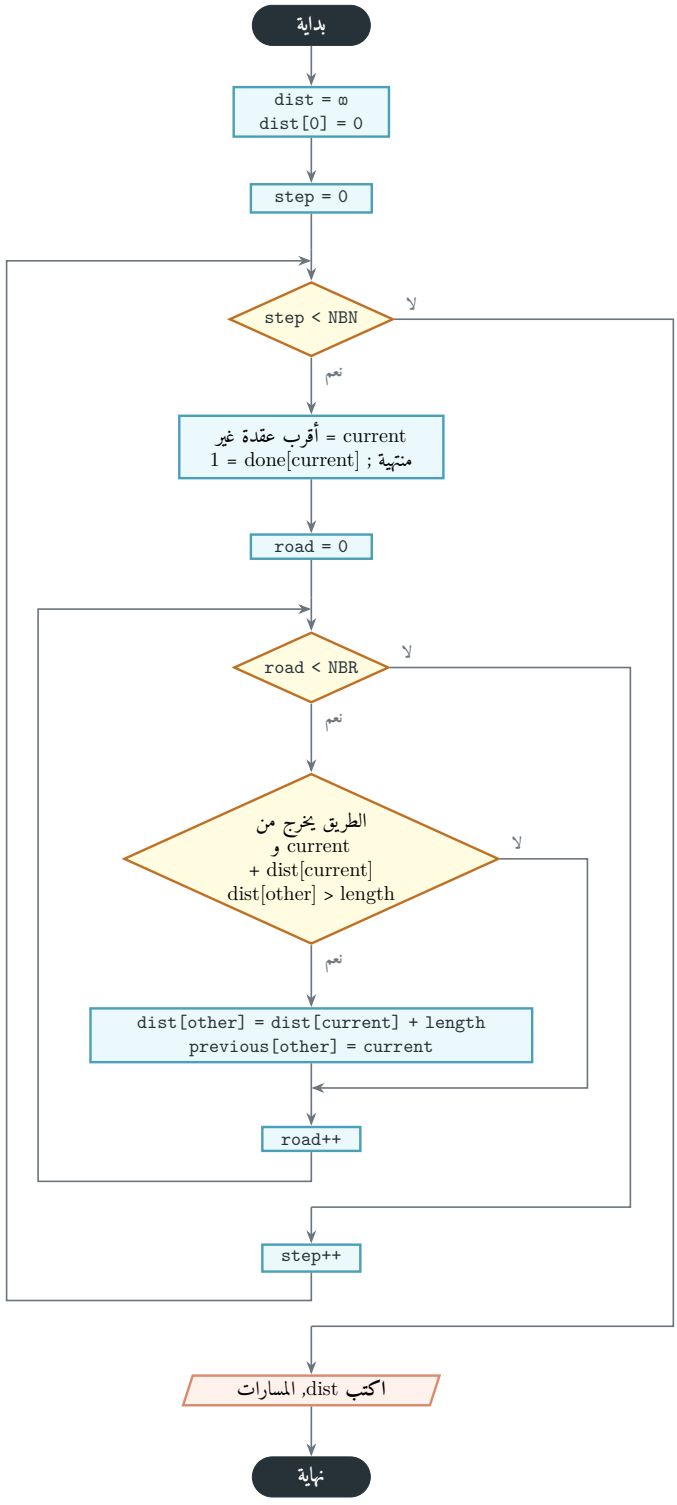
typedef struct { int from, to, length; } Road;

int main(void) {
    Road roads[NBR] = {{0, 1, 7}, {0, 2, 9}, {0, 5, 14}, {1, 2, 10}, {1, 3, 15}, {2, 3, 11},
                       {2, 5, 2}, {3, 4, 6}, {4, 5, 9}, {4, 6, 5}, {3, 6, 12}};
    int dist[NBN], previous[NBN], done[NBN], path[NBN];
    int node, step, current, road, other, len, idx;
    for (node = 0; node < NBN; node++) { dist[node] = INF; done[node] = 0; previous[node] = -1; }
    dist[0] = 0;
    for (step = 0; step < NBN; step++) {
        current = -1;
        for (node = 0; node < NBN; node++)                // أقرب عقدة غير منتهية
            if (!done[node] && (current < 0 || dist[node] < dist[current])) current = node;
        done[current] = 1;
        for (road = 0; road < NBR; road++) {
            if (roads[road].from == current) other = roads[road].to;
            else if (roads[road].to == current) other = roads[road].from;
            else continue;
            if (dist[current] + roads[road].length < dist[other]) {   // إرخاء
                dist[other] = dist[current] + roads[road].length;
                previous[other] = current;
            }
        }
    }
    for (node = 1; node < NBN; node++) {
        len = 0;
        for (other = node; other != -1; other = previous[other]) path[len++] = other;
        printf("to %d: %2d km via", node, dist[node]);
        for (idx = len - 1; idx >= 0; idx--) printf(" %d", path[idx]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
to 1:  7 km via 0 1
to 2:  9 km via 0 2
to 3: 20 km via 0 2 3
to 4: 20 km via 0 2 5 4
to 5: 11 km via 0 2 5
to 6: 25 km via 0 2 5 4 6
```

المخطط الإنسيابي (Organigramme)



تمرين 9.46 جذع شجرة قرار بمؤشر جيني

علم البيانات

صعب ★★★

نريد قاعدة بسيطة تتنبأ إن كان الزائر سيشتري: "إذا كانت الخاصية `feature` ≥ عتبة فهو مشترٍ". لكل زائر هيكل `Visitor` (مدة الزيارة، عدد الصفحات، عدد النقرات على الصور، اشترى أم لا). جرّب كل خاصية وكل عتبة ممكنة، واختر الأفضل بأقل **مؤشر جيني** مرجّح، ثم احسب دقة القاعدة.

خطوات الحل

1) جيني لمجموعة: $1 - p^2 - (1-p)^2$ حيث p نسبة المشترين فيها.
2) لكل (خاصية، عتبة): نقسم الزوار إلى يسار (< عتبة) ويمين (≥ عتبة)، والجيني المرجح $(nL \cdot gL + nR \cdot gR) / n$.
3) العتبات المرشحة: قيم الخاصية نفسها.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 12

typedef struct { double values[3]; int bought; } Visitor;

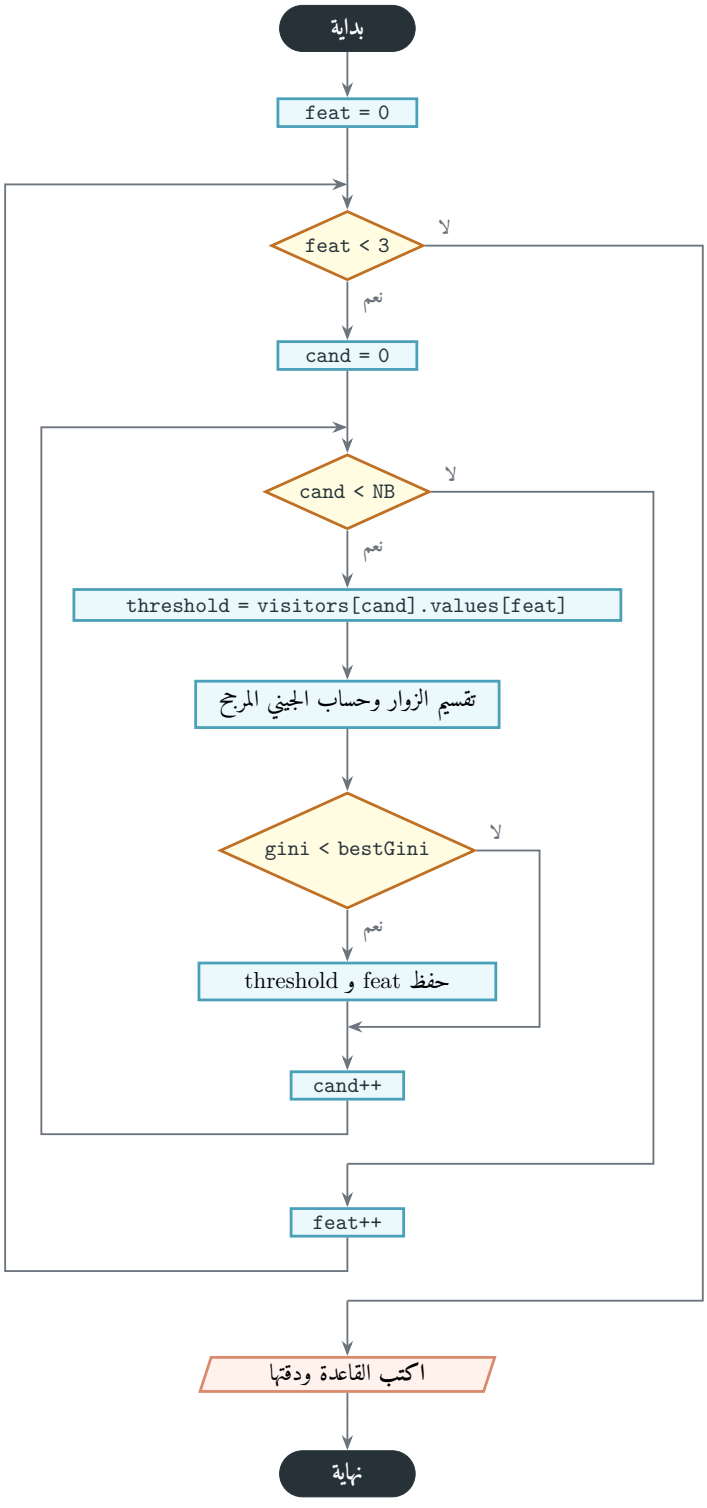
double gini(int buyers, int total) {
    double ratio;
    if (total == 0) return 0;
    ratio = (double)buyers / total;
    return 1 - ratio * ratio - (1 - ratio) * (1 - ratio);
}

int main(void) {
    Visitor visitors[NB] = {{{30, 2, 0}, 0}, {{240, 8, 5}, 1}, {{95, 4, 1}, 0}, {{310, 6, 7}, 1},
                            {{60, 3, 2}, 0}, {{180, 9, 4}, 1}, {{150, 5, 0}, 0}, {{20, 1, 0}, 0},
                            {{200, 4, 6}, 1}, {{130, 7, 3}, 1}, {{110, 6, 4}, 0}, {{270, 3, 4}, 1}};
    char names[3][10] = {"duration", "pages", "photos"};
    int feat, cand, idx, nLeft, nRight, buyLeft, buyRight, bestFeat = 0, correct = 0;
    double threshold, score, bestGini = 2, bestThreshold = 0;
    for (feat = 0; feat < 3; feat++)
        for (cand = 0; cand < NB; cand++) {
            threshold = visitors[cand].values[feat];
            nLeft = nRight = buyLeft = buyRight = 0;
            for (idx = 0; idx < NB; idx++) {
                if (visitors[idx].values[feat] < threshold) { nLeft++; buyLeft += visitors[idx].bought; }
                else { nRight++; buyRight += visitors[idx].bought; }
            }
            score = (nLeft * gini(buyLeft, nLeft) + nRight * gini(buyRight, nRight)) / NB;
            if (score < bestGini) {                    // أنقى تقسيم
                bestGini = score;
                bestFeat = feat;
                bestThreshold = threshold;
            }
        }
    for (idx = 0; idx < NB; idx++)
        correct += (visitors[idx].values[bestFeat] >= bestThreshold) == visitors[idx].bought;
    printf("Rule: buyer if %s >= %.0f\n", names[bestFeat], bestThreshold);
    printf("Weighted Gini: %.4f\n", bestGini);
    printf("Training accuracy: %d/%d\n", correct, NB);
    return 0;
}
```

نتيجة التنفيذ

```
Rule: buyer if duration >= 180
Weighted Gini: 0.1429
Training accuracy: 11/12
```

المخطط الإنسيابي (Organigramme)



تمرين 9.47 جدول تجزئة لرموز المنتجات

تجارة إلكترونية ★★★ صعب

البحث التسلسلي في كتالوج كبير بطيء. اكتب **جدول تجزئة** بالعنونة المفتوحة: هيكل `Slot` (مستعمل أم لا، الرمز، السعر)، ودالة تجزئة `hashCode(sku)` (طريقة djb2 : `hash = hash * 33 + ch`)، ودالتي `insert` و `lookup` مع **السبر الخطي** عند التصادم. اعرض عدد التصادمات وعدد عمليات السبر لكل بحث.

خطوات الحل

1) الخانة الأولية hashCode(sku) % SIZE؛ إذا كانت مشغولة نجرب التالية (slot + 1) % SIZE.
2) lookup يتوقف عند العثور على الرمز أو عند خانة فارغة (غير موجود).
3) معامل الامتلاء count / SIZE يجب أن يبقى صغيرًا.

البرنامج بلغة C

```c
#include <stdio.h>

#define SIZE 17

typedef struct { int used; char sku[10]; double price; } Slot;

Slot table[SIZE];
int collisions = 0;

unsigned long hashCode(char sku[]) {
    unsigned long hash = 5381;
    int pos;
    for (pos = 0; sku[pos] != '\0'; pos++) hash = hash * 33 + (unsigned char)sku[pos];
    return hash;
}

int sameText(char first[], char second[]) {
    int pos = 0;
    while (first[pos] != '\0' && first[pos] == second[pos]) pos++;
    return first[pos] == second[pos];
}

void insert(char sku[], double price) {
    int slot = (int)(hashCode(sku) % SIZE), pos;
    while (table[slot].used) {                          // تصادم: سبر خطي
        collisions++;
        slot = (slot + 1) % SIZE;
    }
    table[slot].used = 1;
    for (pos = 0; pos < 10; pos++) table[slot].sku[pos] = sku[pos];
    table[slot].price = price;
}

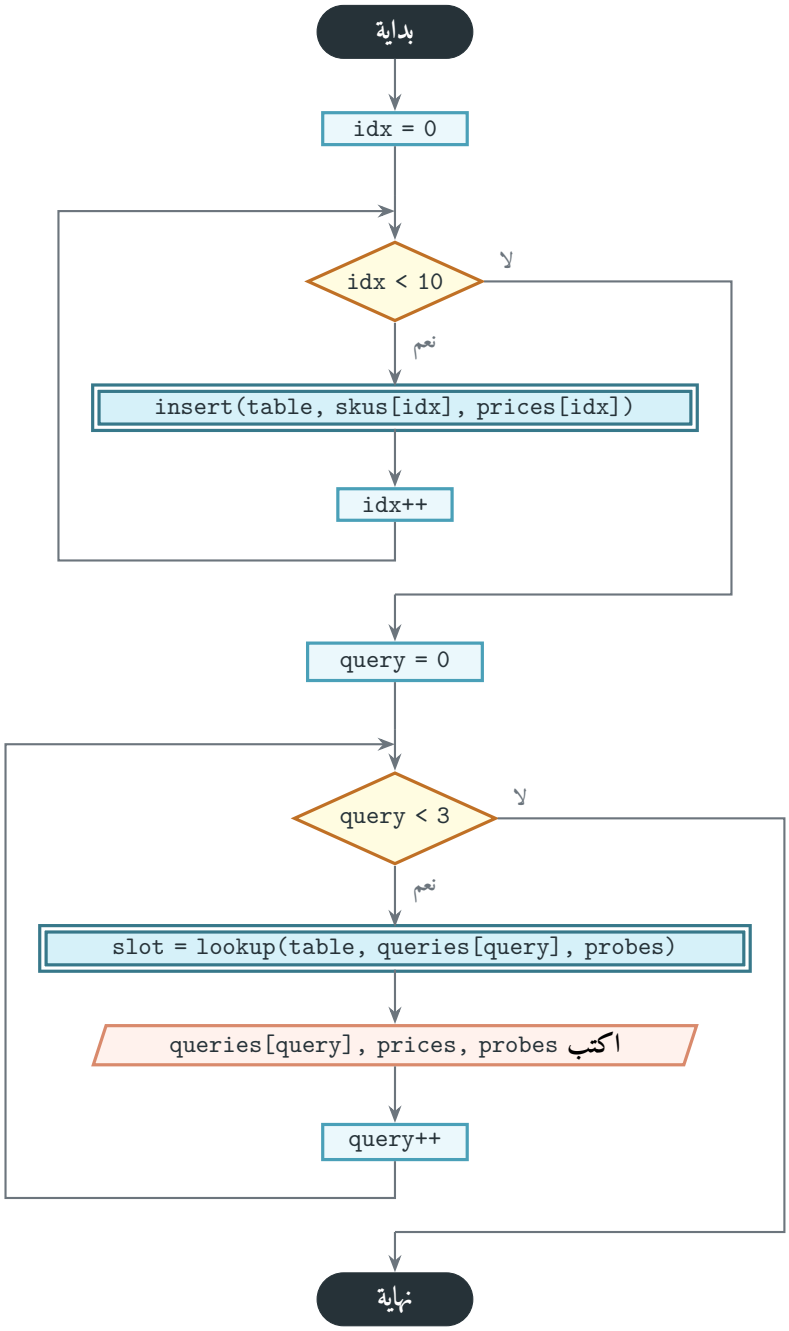
int lookup(char sku[], int probes[]) {
    int slot = (int)(hashCode(sku) % SIZE);
    probes[0] = 1;
    while (table[slot].used) {
        if (sameText(table[slot].sku, sku)) return slot;
        slot = (slot + 1) % SIZE;
        probes[0]++;
    }
    return -1;                                          // خانة فارغة: غير موجود
}

int main(void) {
    char skus[10][10] = {"KB-204", "MS-110", "MN-270", "HD-500", "CB-001",
                         "PH-330", "LP-900", "WC-410", "HB-220", "SP-150"};
    double prices[10] = {3200, 1500, 32000, 5400, 450, 38000, 89000, 4100, 3100, 6800};
    char queries[3][10] = {"LP-900", "CB-001", "TV-999"};
    int idx, slot, probes[1];
    for (idx = 0; idx < 10; idx++) insert(skus[idx], prices[idx]);
    printf("10 items in %d slots, %d collisions\n", SIZE, collisions);
    for (idx = 0; idx < 3; idx++) {
        slot = lookup(queries[idx], probes);
        if (slot >= 0) printf("%s -> slot %2d, %.0f DA (%d probes)\n", queries[idx], slot,
          ↪ table[slot].price, probes[0]);
        else printf("%s -> not found (%d probes)\n", queries[idx], probes[0]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
10 items in 17 slots, 4 collisions
LP-900 -> slot 10, 89000 DA (3 probes)
CB-001 -> slot 12, 450 DA (1 probes)
TV-999 -> not found (2 probes)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.48 جدولة أكبر عدد من العروض الترويجية

تجارة إلكترونية ★★★ صعب

تريد الصفحة الرئيسية عرض لافتات ترويجية، ولا تُعرض لافتتان في نفس الوقت. لكل عرض هيكل `Promo` (الاسم، ساعة البداية، ساعة النهاية، الربح المتوقع). (1) اختر **أكبر عدد** من العروض بالطريقة الجشعة (الأبكر انتهاءً أولًا). (2) اختر مجموعة

ذات أكبر ربح بالبرمجة الديناميكية وقارن.

خطوات الحل

1) نرتب العروض حسب ساعة النهاية.
2) الجشعة: نأخذ العرض إذا بدأ بعد نهاية آخر عرض مأخوذ.
3) الديناميكية:

```
best[idx] = max(best[idx-1], gain[idx] + best[last])
```

حيث `last` آخر عرض متوافق ينتهي قبل بداية `idx`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 8

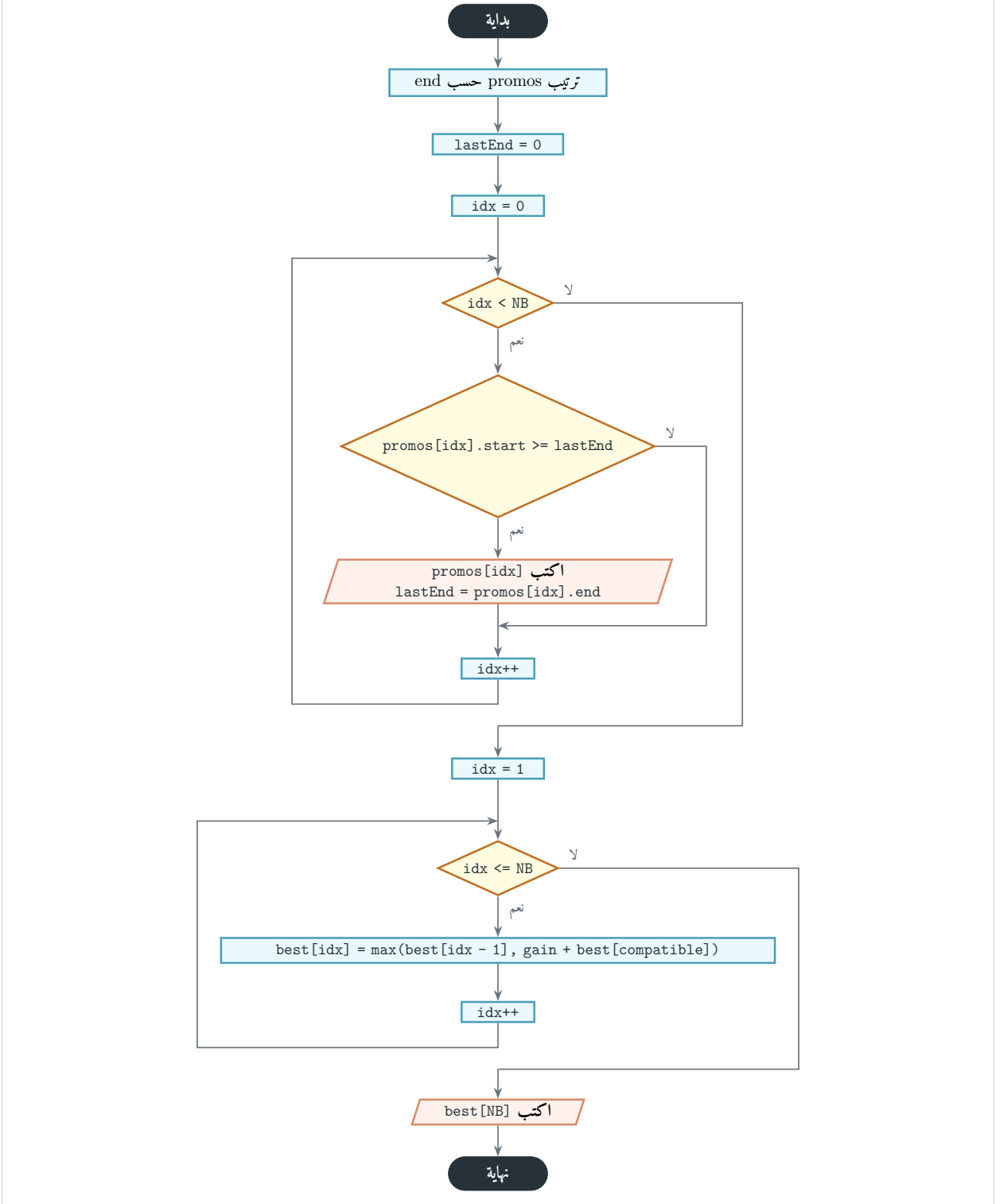
typedef struct { char name[10]; int start, end, gain; } Promo;

int main(void) {
    Promo promos[NB] = {{"Flash", 8, 11, 50}, {"Phones", 9, 12, 70}, {"Audio", 11, 14, 40},
                        {"Laptops", 10, 16, 120}, {"Cables", 13, 15, 20}, {"Gaming", 14, 18, 65},
                        {"Night", 18, 22, 55}, {"Bundle", 16, 20, 110}};
    Promo key;
    int best[NB + 1], take[NB + 1], idx, back, lastEnd = 0, count = 0, compat, withIt;
    for (idx = 1; idx < NB; idx++) {                    // ترتيب حسب النهاية
        key = promos[idx];
        for (back = idx - 1; back >= 0 && promos[back].end > key.end; back--) promos[back + 1] =
      ↪ promos[back];
        promos[back + 1] = key;
    }
    printf("Greedy (max count):");
    for (idx = 0; idx < NB; idx++)
        if (promos[idx].start >= lastEnd) {
            printf(" %s", promos[idx].name);
            lastEnd = promos[idx].end;
            count++;
        }
    printf(" -> %d promos\n", count);
    best[0] = 0;
    for (idx = 1; idx <= NB; idx++) {
        for (compat = idx - 1; compat >= 1 && promos[compat - 1].end > promos[idx - 1].start; compat--) {}
        withIt = promos[idx - 1].gain + best[compat];   // عدد العروض المتوافقة قبله = compat
        take[idx] = withIt > best[idx - 1];
        best[idx] = take[idx] ? withIt : best[idx - 1];
    }
    printf("DP (max gain):");
    for (idx = NB; idx >= 1; ) {                        // إعادة البناء
        if (take[idx]) {
            printf(" %s", promos[idx - 1].name);
            for (compat = idx - 1; compat >= 1 && promos[compat - 1].end > promos[idx - 1].start; compat--)
      ↪ {}
            idx = compat;
        } else {
            idx--;
        }
    }
    printf(" -> gain %d\n", best[NB]);
    return 0;
}
```

نتيجة التنفيذ

```
Greedy (max count): Flash Audio Gaming Night -> 4 promos
DP (max gain): Bundle Laptops -> gain 230
```

المخطط الإنسيابي (Organigramme)



تمرين 9.49 الأعداد الكبيرة لرقم الأعمال التراكمي

علم البيانات ★★★ صعب

النوع long محدود بحوالي 9.2 × 10^18. لحساب أعداد ضخمة (مثل 2^100 أو 30!) لعدد ترتيبات المنتجات)، عرّف هيكل BigNum يخزن الأرقام في جدول (الرقم الأقل وزنًا أولًا) ودوال addBig و multiplySmall و printBig.

خطوات الحل

1) الجمع: رقمًا برقم مع حمل carry.
2) الضرب في عدد صغير: كل رقم × العدد + الحمل، نحتفظ بـ 10 % ونحمل 10 /.
3) 30 × ... × 2 × 1 = 30! و 2^100 بضرب متكرر في 2.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAXD 80

typedef struct { int digits[MAXD]; int length; } BigNum;

BigNum fromInt(long value) {
    BigNum number;
    number.length = 0;
    do {
        number.digits[number.length++] = (int)(value % 10);
        value /= 10;
    } while (value > 0);
    return number;
}

BigNum multiplySmall(BigNum number, int factor) {
    int pos, carry = 0, product;
    for (pos = 0; pos < number.length; pos++) {
        product = number.digits[pos] * factor + carry;
        number.digits[pos] = product % 10;
        carry = product / 10;                          // الحمل
    }
    while (carry > 0) {
        number.digits[number.length++] = carry % 10;
        carry /= 10;
    }
    return number;
}

BigNum addBig(BigNum first, BigNum second) {
    BigNum sum;
    int pos, carry = 0, digit;
    sum.length = first.length > second.length ? first.length : second.length;
    for (pos = 0; pos < sum.length; pos++) {
        digit = carry + (pos < first.length ? first.digits[pos] : 0)
                      + (pos < second.length ? second.digits[pos] : 0);
        sum.digits[pos] = digit % 10;
        carry = digit / 10;
    }
    if (carry) sum.digits[sum.length++] = carry;
    return sum;
}

void printBig(char label[], BigNum number) {
    int pos;
    printf("%s", label);
    for (pos = number.length - 1; pos >= 0; pos--) putchar('0' + number.digits[pos]);
    printf(" (%d digits)\n", number.length);
}

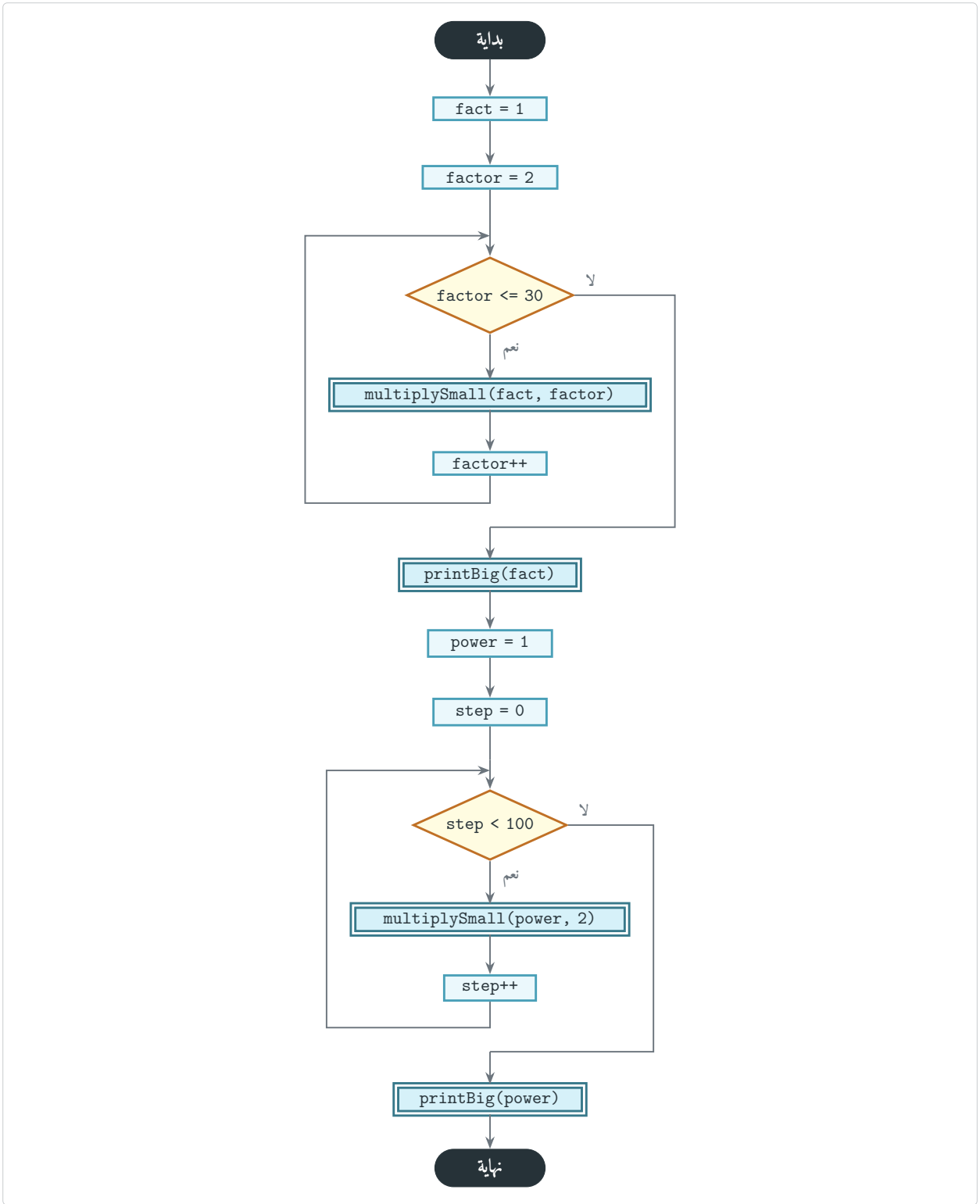
int main(void) {
    BigNum fact = fromInt(1), power = fromInt(1);
    int factor, step;
    for (factor = 2; factor <= 30; factor++) fact = multiplySmall(fact, factor);
```

```
56      for (step = 0; step < 100; step++) power = multiplySmall(power, 2);
57      printBig("30!   = ", fact);
58      printBig("2^100 = ", power);
59      printBig("sum   = ", addBig(fact, power));
60      printBig("long max = ", fromInt(9223372036854775807L));
61      return 0;
62  }
```

نتيجة التنفيذ

```
30!   = 265252859812191058636308480000000 (33 digits)
2^100 = 1267650600228229401496703205376 (31 digits)
sum   = 266520510412419288037805183205376 (33 digits)
long max = 9223372036854775807 (19 digits)
```

المخطط الإنسيابي (Organigramme)



تمرين 9.50 انحدار متعدد بحل المعادلات الطبيعية

علم البيانات ★★★★ صعب

نتنبأ بالمبيعات الأسبوعية بدلالة ميزانية الإعلان والتخفيض: `sales = b0 + b1·ads + b2·discount`. عرّف هيكل `Matrix3` (3×4 موسعة) واكتب دوال `buildNormal` (تبني المعادلات الطبيعية `X X b = X y`) و `solve` (غاوس) و `predict`، ثم احسب المعاملات والتنبؤ.

خطوات الحل

1) $X\,X[row][col] = \Sigma\ feature[row] \cdot feature[col]$ حيث $feature = (1, ads, discount)$.
2) $X\,y[row] = \Sigma\ feature[row] \cdot sales$.
3) حل جملة 3×3 بالحذف ثم التعويض التراجعي.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { double ads, discount, sales; } Week;
typedef struct { double cell[3][4]; } Matrix3;
typedef struct { double coef[3]; } Model;

Matrix3 buildNormal(Week weeks[], int size) {
    Matrix3 system;
    double feature[3];
    int idx, row, col;
    for (row = 0; row < 3; row++) for (col = 0; col < 4; col++) system.cell[row][col] = 0;
    for (idx = 0; idx < size; idx++) {
        feature[0] = 1; feature[1] = weeks[idx].ads; feature[2] = weeks[idx].discount;
        for (row = 0; row < 3; row++) {
            for (col = 0; col < 3; col++) system.cell[row][col] += feature[row] * feature[col];
            system.cell[row][3] += feature[row] * weeks[idx].sales;   // X^T y
        }
    }
    return system;
}

Model solve(Matrix3 system) {
    Model model;
    double factor, sum;
    int pivot, row, col;
    for (pivot = 0; pivot < 3; pivot++)                 // حذف غاوس
        for (row = pivot + 1; row < 3; row++) {
            factor = system.cell[row][pivot] / system.cell[pivot][pivot];
            for (col = pivot; col < 4; col++) system.cell[row][col] -= factor * system.cell[pivot][col];
        }
    for (row = 2; row >= 0; row--) {                    // تعويض تراجعي
        sum = system.cell[row][3];
        for (col = row + 1; col < 3; col++) sum -= system.cell[row][col] * model.coef[col];
        model.coef[row] = sum / system.cell[row][row];
    }
    return model;
}

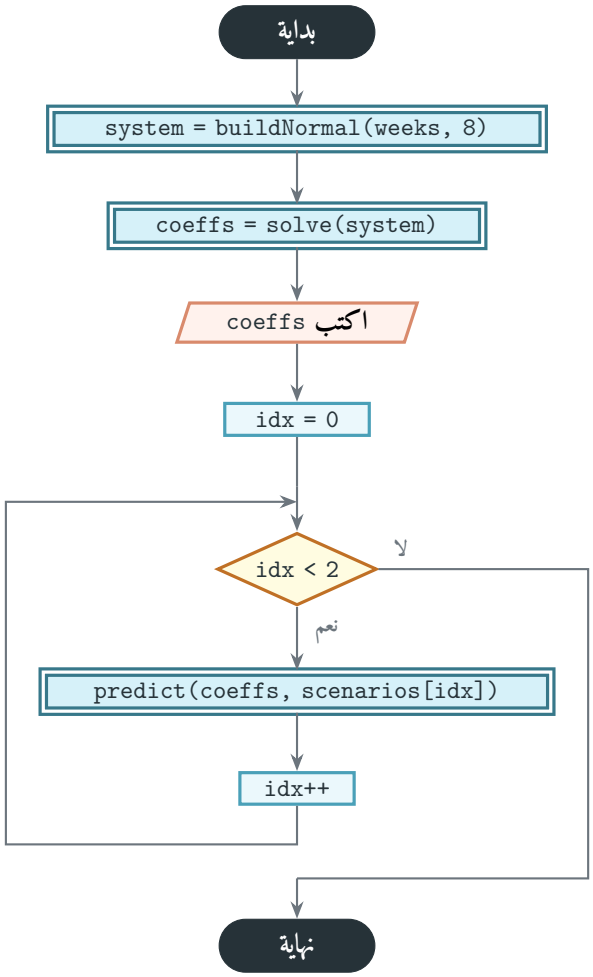
double predict(Model model, double ads, double discount) {
    return model.coef[0] + model.coef[1] * ads + model.coef[2] * discount;
}

int main(void) {
    Week weeks[8] = {{10, 0, 205}, {15, 5, 290}, {8, 10, 230}, {20, 0, 320},
                     {25, 10, 430}, {12, 15, 300}, {30, 5, 455}, {18, 20, 400}};
    Model model = solve(buildNormal(weeks, 8));
    double error = 0, diff;
    int idx;
    printf("sales = %.2f + %.2f * ads + %.2f * discount\n", model.coef[0], model.coef[1], model.coef[2]);
    for (idx = 0; idx < 8; idx++) {
        diff = weeks[idx].sales - predict(model, weeks[idx].ads, weeks[idx].discount);
        error += diff * diff;
    }
    printf("Mean squared error: %.2f\n", error / 8);
    printf("Forecast (ads 22, discount 10%%): %.1f\n", predict(model, 22, 10));
    printf("Forecast (ads 35, discount 0%%):  %.1f\n", predict(model, 35, 0));
    return 0;
}
```

نتيجة التنفيذ

```
sales = 89.72 + 11.47 * ads + 5.07 * discount
Mean squared error: 7.13
Forecast (ads 22, discount 10%): 392.7
Forecast (ads 35, discount 0%):  491.2
```

المخطط الإنسيابي (Organigramme)

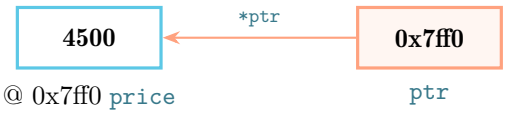


الفصل 10

المؤشرات والحجز الديناميكي

تعريفات

- **المؤشر**: متغير يحمل عنوان متغير آخر: `double *ptr = &price;`
- `*ptr` القيمة الموجودة في العنوان (قراءة وكتابة)، و`&price` عنوان المتغير.
- **حساب المؤشرات**: `ptr + 1` يشير إلى العنصر التالي في الجدول، و`end - start` عدد العناصر بينهما.
- **الحجز الديناميكي**: `malloc(count * sizeof(double))` يحجز كتلة أثناء التنفيذ، و`free` يعيدها.
- **مؤشر الدالة**: `double (*policy)(double)` يسمح باختيار الدالة المستدعاة أثناء التنفيذ.



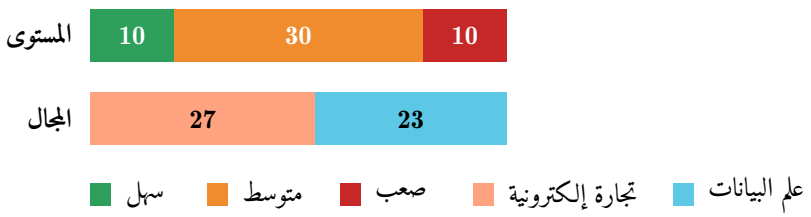
الحاجة	الحل بالمؤشرات
دالة تعدل متغيرًا	نمرر عنوانه: `sell(&stock, 3)`
عدة نتائج من دالة	نكتبها عبر مؤشرات: `range(values, size, &low, &high)`
حجم غير معروف مسبقًا	`malloc` ثم مضاعفة السعة عند الامتلاء (دون `realloc`)
دالة تحجز للمستدعي	مؤشر على مؤشر: `double **result`

قواعد الذاكرة

تحقق دائمًا من أن `malloc` لم يعد `NULL`. لكل `malloc` يوجد `free` واحد بالضبط، ولا نستعمل الكتلة بعد تحريرها، ولا نحررها مرتين.

للاستزادة: [1, 2, 11]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 10.1 عنوان السعر وقيمته

تجارة إلكترونية سهل

صرّح بمتغير price ومؤشر ptr يشير إليه. اعرض القيمة عبر المتغير وعبر المؤشر (*ptr)، ثم عدّل السعر عبر المؤشر وتحقق أن المتغير تغيّر.

خطوات الحل

1) ptr = &price; يخزن عنوان المتغير.
2) *ptr يعني "القيمة الموجودة في العنوان": القراءة والكتابة.

البرنامج بلغة C

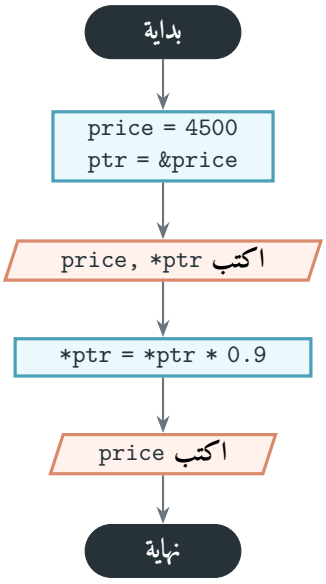
```c
#include <stdio.h>

int main(void) {
    double price = 4500;
    double *ptr = &price;                    // ptr يحمل عنوان price
    printf("price = %.2f, *ptr = %.2f\n", price, *ptr);
    printf("same address: %s\n", ptr == &price ? "yes" : "no");
    *ptr = *ptr * 0.9;                       // نعدل price عبر المؤشر
    printf("after -10%% via pointer: price = %.2f\n", price);
    return 0;
}
```

نتيجة التنفيذ

```
price = 4500.00, *ptr = 4500.00
same address: yes
after -10% via pointer: price = 4050.00
```

المخطط الإنسيابي (Organigramme)



تمرين 10.2 تبديل سعرين بالمؤشرات

تجارة إلكترونية ⭐ سهل

اكتب دالة `swap(first, second)` تستقبل **مؤشرين** وتبدل القيمتين اللتين يشيران إليهما، واستعملها لترتيب سعرين تصاعديًا. لماذا لا تنجح الدالة إذا استقبلت القيم مباشرة؟

خطوات الحل

1) التمرير بالقيمة ينسخ المتغيرات: التبديل يحدث على النسخ فقط.
2) بالمؤشرات: الدالة تصل إلى المتغيرات الأصلية `*first` و `*second`.

البرنامج بلغة C

```c
#include <stdio.h>

void swap(double *first, double *second) {
    double tmp = *first;
    *first = *second;                 // نعدل المتغيرات الأصلية
    *second = tmp;
}

int main(void) {
    double low, high;
    scanf("%lf %lf", &low, &high);
    if (low > high)
        swap(&low, &high);            // نمرر العناوين
    printf("Price range: %.2f - %.2f DA\n", low, high);
    return 0;
}
```

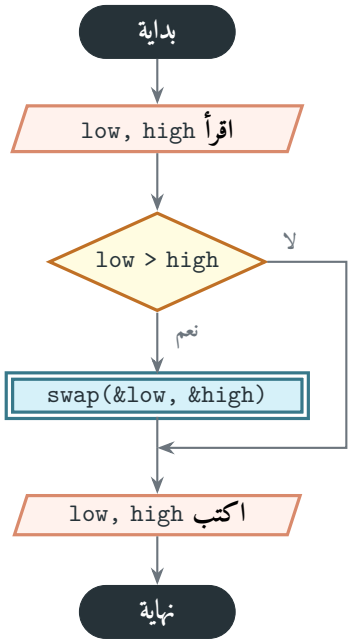
المدخلات

```
5400 3200
```

نتيجة التنفيذ

```
Price range: 3200.00 - 5400.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.3 إنقاص المخزون عبر مؤشر

تجارة إلكترونية سهل

اكتب دالة `sell(stock, qty)` تستقبل **مؤشرًا** على المخزون وتنقصه بالكمية المباعة إذا كان كافيًا، وتعيد 1 أو 0. طبّقها على سلسلة عمليات بيع.

خطوات الحل

1) `if (*stock < qty) return 0;` ثم `*stock -= qty;`.
2) الاستدعاء بـ `sell(&stock, 3)`.

البرنامج بلغة C

```c
#include <stdio.h>

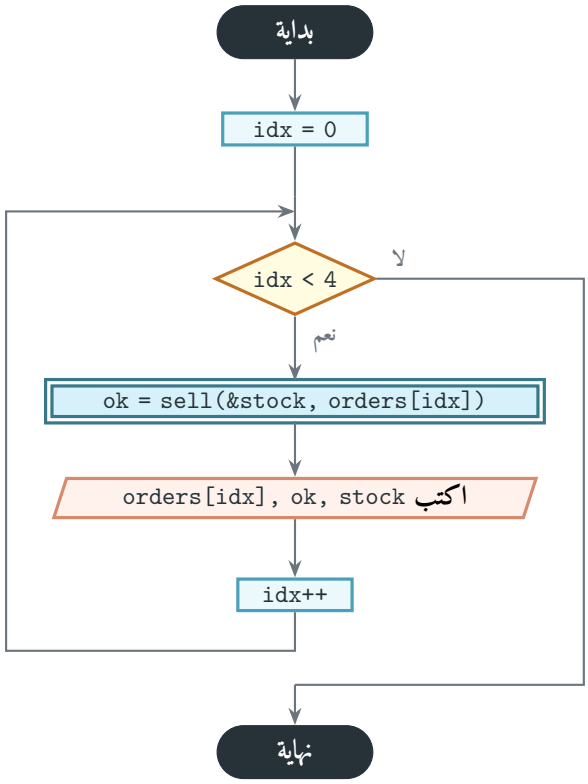
int sell(int *stock, int qty) {
    if (*stock < qty)
        return 0;                    // مخزون غير كافٍ
    *stock -= qty;
    return 1;
}

int main(void) {
    int stock = 10, orders[4] = {3, 5, 4, 2}, idx, ok;
    for (idx = 0; idx < 4; idx++) {
        ok = sell(&stock, orders[idx]);
        printf("Sell %d: %s, stock now %d\n", orders[idx], ok ? "OK" : "refused", stock);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Sell 3: OK, stock now 7
Sell 5: OK, stock now 2
Sell 4: refused, stock now 2
Sell 2: OK, stock now 0
```

المخطط الإنسيابي (Organigramme)



تمرين 10.4 المرور على جدول بحساب المؤشرات

علم البيانات سهل

احسب مجموع المبيعات اليومية بالمرور على الجدول بمؤشر يتقدم خانة بخانة (`ptr++`) بدل الفهرس، وتوقف عندما يصل المؤشر إلى نهاية الجدول `sales + 7`.

خطوات الحل

1) اسم الجدول `sales` هو عنوان أول عنصر.
2) `ptr + 1` يشير إلى العنصر التالي (يتقدم بحجم `double`).

البرنامج بلغة C

```c
#include <stdio.h>

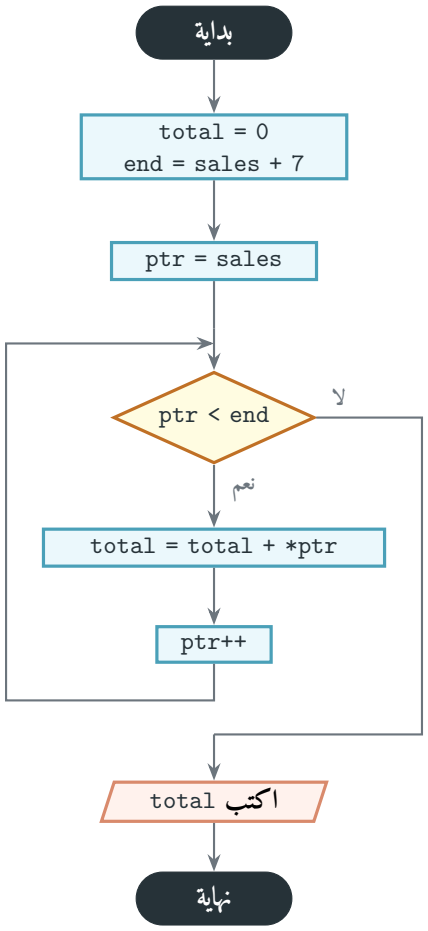
int main(void) {
    double sales[7] = {1200, 980, 1430, 1100, 1290, 1650, 1520};
    double *ptr, *end = sales + 7, total = 0;
    for (ptr = sales; ptr < end; ptr++)        // المؤشر يتقدم بخانة
        total += *ptr;
    printf("Week total: %.2f DA\n", total);
    printf("Third day via pointer: %.2f (= sales[2])\n", *(sales + 2));
    return 0;
```

```
}
```

نتيجة التنفيذ

```
Week total: 9170.00 DA
Third day via pointer: 1430.00 (= sales[2])
```

المخطط الإنسيابي (Organigramme)



تمرين 10.5 دالة تعيد الأصغر والأكبر

علم البيانات
سهل

الدالة تعيد قيمة واحدة فقط، لكن يمكنها كتابة نتائج إضافية عبر مؤشرات. اكتب دالة `range(values, size, low, high)` تكتب أصغر وأكبر قيمة في `*low` و `*high`.

خطوات الحل

1) المعاملان `double *low, double *high` عنوانا متغيرين في `main`.
2) `*low = values[0]` ثم نحدّث أثناء المرور.

البرنامج بلغة C

```c
#include <stdio.h>

void range(double values[], int size, double *low, double *high) {
```

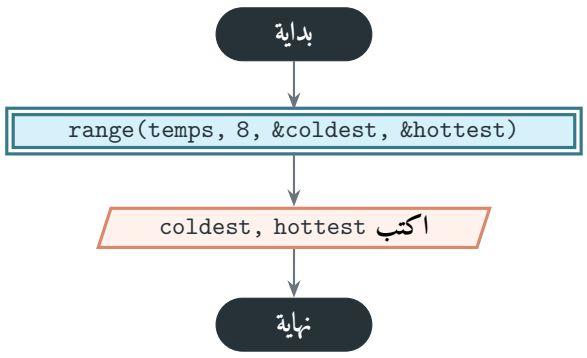
```c
    int idx;
    *low = *high = values[0];
    for (idx = 1; idx < size; idx++) {
        if (values[idx] < *low) *low = values[idx];
        if (values[idx] > *high) *high = values[idx];
    }
}

int main(void) {
    double temps[8] = {4.2, 3.8, 5.1, 6.9, 4.4, 2.7, 3.3, 5.6};
    double coldest, hottest;
    range(temps, 8, &coldest, &hottest);      // نتيجتان عبر العناوين
    printf("Cold room range: %.1f C to %.1f C\n", coldest, hottest);
    return 0;
}
```

نتيجة التنفيذ

```
Cold room range: 2.7 C to 6.9 C
```

المخطط الإنسيابي (Organigramme)



تمرين 10.6 جدول ديناميكي بحجم يحدده المستخدم

تجارة إلكترونية سهل

اقرأ عدد الطلبيات count، ثم احجز جدولًا بالحجم المطلوب بـ malloc، واقرأ المبالغ، واحسب المتوسط، ثم حرر الذاكرة بـ free.

خطوات الحل

1) malloc(count * sizeof(double)) يعيد عنوان كتلة جديدة، أو NULL إذا فشل.
2) نستعمل الكتلة كجدول عادي amounts[idx].
3) كل malloc يقابله free.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int count, idx;
    double *amounts, total = 0;
    scanf("%d", &count);
```

```c
    amounts = malloc(count * sizeof(double));      // حجم يُعرف أثناء التنفيذ
    if (amounts == NULL) {
        printf("Not enough memory\n");
        return 1;
    }
    for (idx = 0; idx < count; idx++) {
        scanf("%lf", &amounts[idx]);
        total += amounts[idx];
    }
    printf("%d orders, average %.2f DA\n", count, total / count);
    free(amounts);                                 // نعيد الذاكرة
    return 0;
}
```

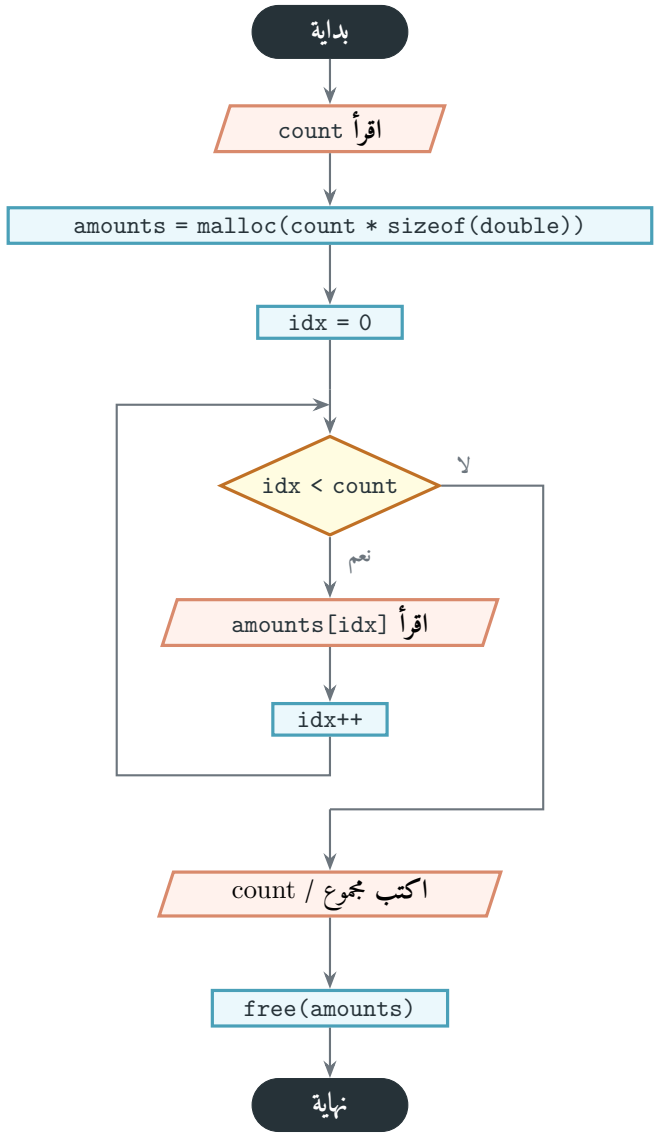
المدخلات

```
5
1200 4500 890 3100 2600
```

نتيجة التنفيذ

```
5 orders, average 2458.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.7 مؤشر على هيكل والسهم

تجارة إلكترونية

سهل

اكتب دالة `applyDiscount(item, percent)` تستقبل **مؤشرًا على هيكل** منتج وتعدل سعره مباشرة باستعمال العامل `->`، ثم طبّقها على عناصر جدول.

خطوات الحل

1) `item->price` يكافئ `(*item).price`.
2) نمرر `&items[idx]` لكي تعدل الدالة الأصل.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[16]; double price; } Product;

```

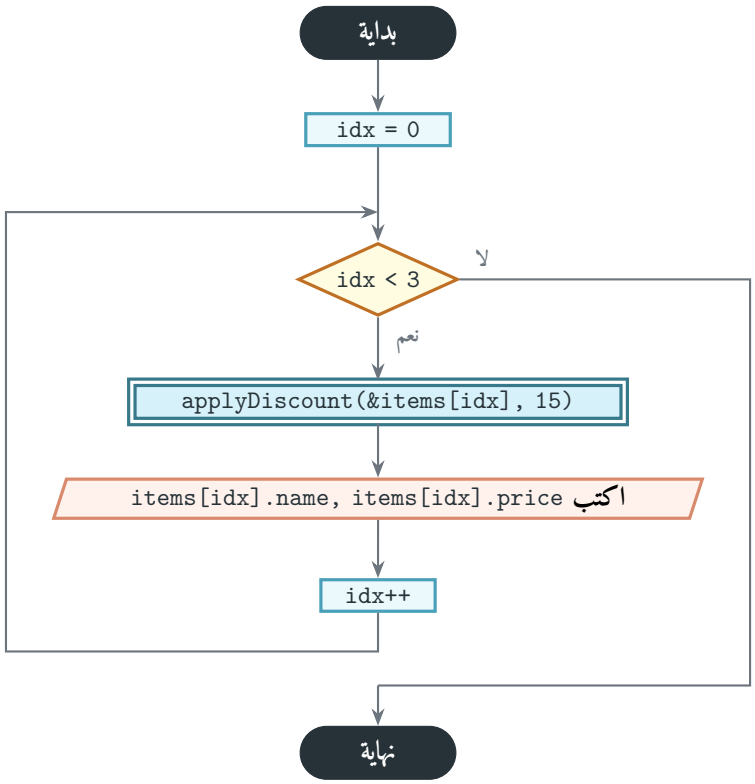
```c
void applyDiscount(Product *item, double percent) {
    item->price = item->price * (100 - percent) / 100;    // يعدل الأصل
}

int main(void) {
    Product items[3] = {{"Monitor", 32000}, {"Keyboard", 3200}, {"Webcam", 4100}};
    int idx;
    for (idx = 0; idx < 3; idx++) {
        applyDiscount(&items[idx], 15);
        printf("%-9s %9.2f DA\n", items[idx].name, items[idx].price);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Monitor    27200.00 DA
Keyboard    2720.00 DA
Webcam      3485.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.8 طول السلسلة بالمؤشرات

تجارة إلكترونية سهل

اكتب دالة `textLength(text)` تحسب طول سلسلة بمؤشر يتقدم حتى `'\0'`، ثم تعيد الفرق بين المؤشر النهائي ومؤشر البداية.

خطوات الحل

1) `const char *walk = text;` ثم `while (*walk) walk++;`.
2) طرح مؤشرين على نفس الجدول يعطي عدد الخانات بينهما.

البرنامج بلغة C

```c
#include <stdio.h>

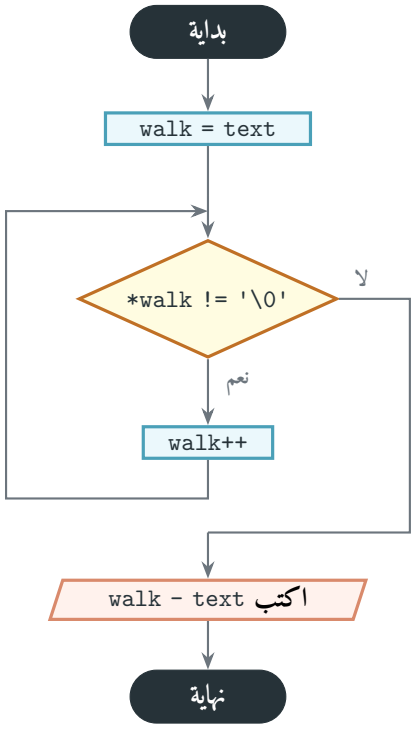
int textLength(const char *text) {
    const char *walk = text;
    while (*walk != '\0')
        walk++;                         // نتقدم حتى النهاية
    return (int)(walk - text);          // الفرق = عدد الحروف
}

int main(void) {
    const char *names[3] = {"Wireless Mouse", "USB-C Hub", "4K Monitor 27 inch"};
    int idx;
    for (idx = 0; idx < 3; idx++)
        printf("%-20s %d chars\n", names[idx], textLength(names[idx]));
    return 0;
}
```

نتيجة التنفيذ

```
Wireless Mouse       14 chars
USB-C Hub            9 chars
4K Monitor 27 inch   18 chars
```

المخطط الإنسيابي (Organigramme)



تمرين 10.9 حجز هيكل منتج ديناميكيًا

تجارة إلكترونية سهل

اجز هيكل منتج بـ `malloc(sizeof(Product))`، املأ حقوله عبر المؤشر، اعرضه، ثم حرره. وضّح أن المؤشر بعد `free` يجب ألا يُستعمل (نضعه `NULL`).

خطوات الحل

1) `Product *item = malloc(sizeof(Product));`
2) الحقول عبر `item->...`.
3) بعد `free(item)`: `item = NULL;`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

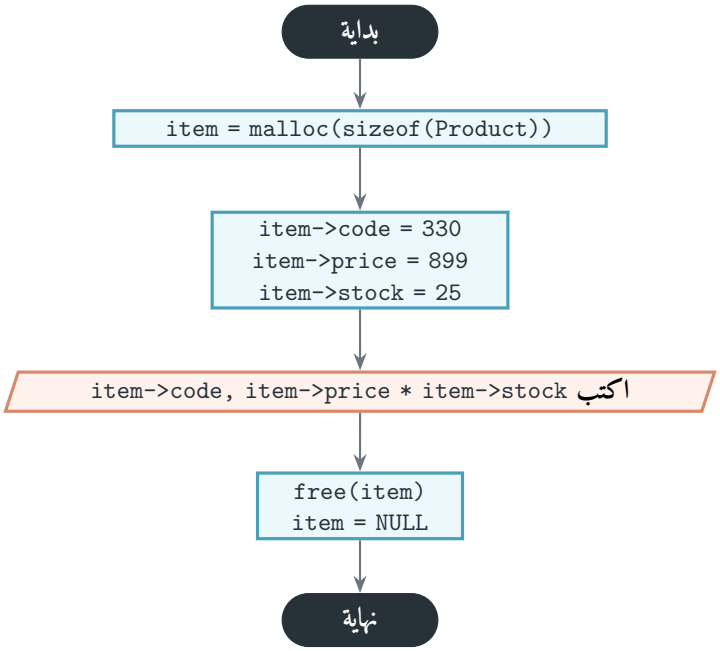
typedef struct { int code; double price; int stock; } Product;

int main(void) {
    Product *item = malloc(sizeof(Product));
    if (item == NULL) return 1;
    item->code = 330;                  // كتابة الحقول عبر المؤشر
    item->price = 899;
    item->stock = 25;
    printf("Product %d: %.2f DA x %d = %.2f DA\n", item->code, item->price,
           item->stock, item->price * item->stock);
    printf("Block size: %d bytes\n", (int)sizeof(Product));
    free(item);
    item = NULL;                       // لا نستعمل ذاكرة محررة
    printf("Freed: %s\n", item == NULL ? "pointer reset" : "?");
    return 0;
}
```

نتيجة التنفيذ

```
Product 330: 899.00 DA x 25 = 22475.00 DA
Block size: 24 bytes
Freed: pointer reset
```

المخطط الإنسيابي (Organigramme)



تمرين 10.10 عدد العناصر بين مؤشرين

علم البيانات سهل

في جدول قراءات حرارة مرتب، ابحث بمؤشرين عن أول قراءة ≥ 4°C وأول قراءة > 6°C، ثم استنتج **عدد القراءات** في المجال `[4, 6]` بطرح المؤشرين.

خطوات الحل

1) `first` يتقدم حتى `*first >= 4`، و `last` يتقدم حتى `*last > 6`.
2) العدد = `last - first`.

البرنامج بلغة C

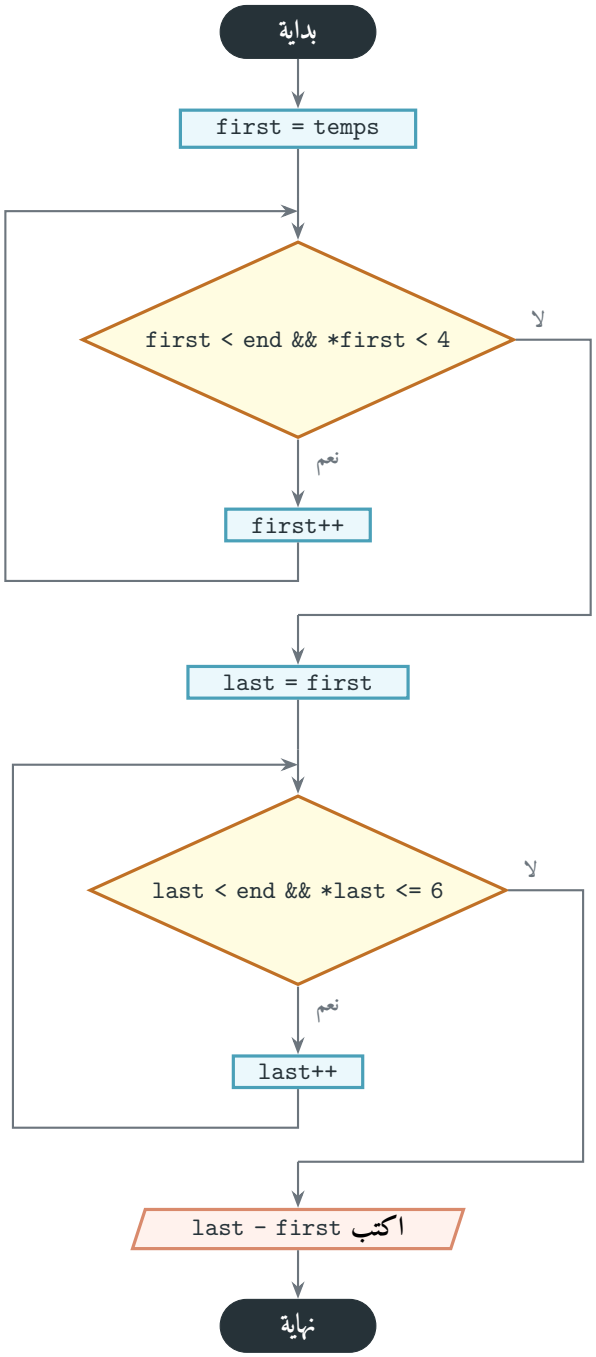
```c
#include <stdio.h>

int main(void) {
    double temps[10] = {2.1, 3.0, 3.8, 4.0, 4.6, 5.2, 5.9, 6.0, 6.8, 7.5};
    double *end = temps + 10, *first = temps, *last;
    while (first < end && *first < 4) first++;
    last = first;
    while (last < end && *last <= 6) last++;
    printf("First in range: %.1f (index %d)\n", *first, (int)(first - temps));
    printf("Readings in [4, 6]: %d\n", (int)(last - first));   // طرح مؤشرين
    return 0;
}
```

نتيجة التنفيذ

```
First in range: 4.0 (index 3)
Readings in [4, 6]: 5
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 10.11 جدول يكبر تلقائيًا

علم البيانات ★★ متوسط

لا نعرف مسبقًا عدد المبيعات التي ستُقرأ (تنتهي القراءة بـ `-1`). اكتب برنامجًا يبدأ بجدول ديناميكي سعته 2، وكلما امتلأ **يضاعف** سعته: يحجز كتلة جديدة، ينسخ العناصر، ويحرر القديمة (دون `realloc`).

خطوات الحل

1) `capacity` السعة، و `count` العدد المستعمل.
2) عند `count == capacity`: `bigger = malloc(2 * capacity ...)`، نسخ، `free(old)`.
3) المضاعفة تجعل التكلفة المتوسطة لكل إضافة ثابتة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int capacity = 2, count = 0, idx, copies = 0;
    double *data = malloc(capacity * sizeof(double)), *bigger, value, total = 0;
    if (data == NULL) return 1;
    scanf("%lf", &value);
    while (value != -1) {
        if (count == capacity) {                    // الجدول ممتلئ
            bigger = malloc(2 * capacity * sizeof(double));
            if (bigger == NULL) { free(data); return 1; }
            for (idx = 0; idx < count; idx++) bigger[idx] = data[idx];
            copies += count;
            free(data);                             // نحرر القديم
            data = bigger;
            capacity *= 2;
            printf("grow -> capacity %d\n", capacity);
        }
        data[count++] = value;
        scanf("%lf", &value);
    }
    for (idx = 0; idx < count; idx++) total += data[idx];
    printf("%d sales, capacity %d, %d copies, total %.0f\n", count, capacity, copies, total);
    free(data);
    return 0;
}
```

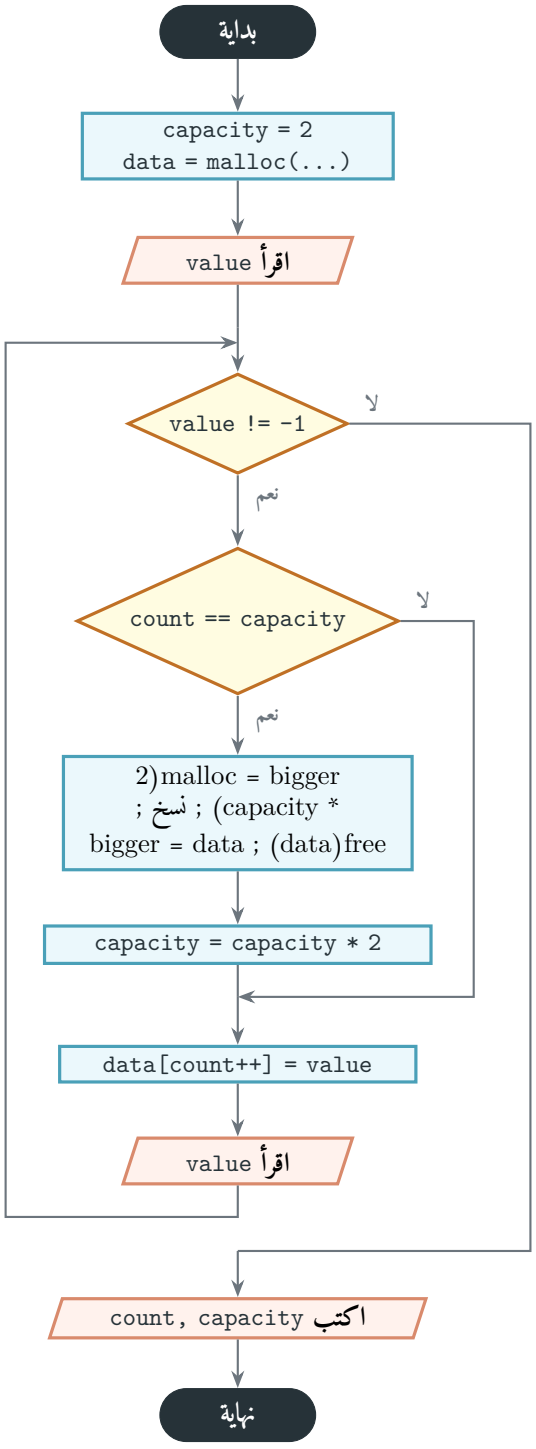
المدخلات

```
120 340 90 560 210 480 75 330 -1
```

نتيجة التنفيذ

```
grow -> capacity 4
grow -> capacity 8
8 sales, capacity 8, 6 copies, total 2205
```

المخطط الإنسيابي (Organigramme)



تمرين 10.12 نسخة ديناميكية من اسم المنتج

تجارة إلكترونية ★★ متوسط

اكتب دالة `duplicate(text)` تحجز بالضبط الذاكرة اللازمة لنسخة من السلسلة (الطول + 1)، وتنسخها، وتعيد المؤشر (بديل `strdup`). استعملها لتخزين أسماء منتجات بأطوال مختلفة دون إهدار ذاكرة.

خطوات الحل

1) نحسب الطول، ثم `malloc(length + 1)`.
2) ننسخ الحروف ثم `'\0'`.
3) المستدعي مسؤول عن `free`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

char *duplicate(const char *text) {
    int length = 0, pos;
    char *copy;
    while (text[length] != '\0') length++;
    copy = malloc(length + 1);                  // 1+ لعلامة النهاية
    if (copy == NULL) return NULL;
    for (pos = 0; pos <= length; pos++) copy[pos] = text[pos];
    return copy;
}

int main(void) {
    char buffer[100];
    char *names[3];
    int idx, bytes = 0, len;
    for (idx = 0; idx < 3; idx++) {
        scanf("%99s", buffer);
        names[idx] = duplicate(buffer);         // نسخة بحجمها الحقيقي
        for (len = 0; names[idx][len] != '\0'; len++) {}
        bytes += len + 1;
    }
    for (idx = 0; idx < 3; idx++) printf("%s\n", names[idx]);
    printf("Used %d bytes instead of %d\n", bytes, 3 * 100);
    for (idx = 0; idx < 3; idx++) free(names[idx]);
    return 0;
}
```

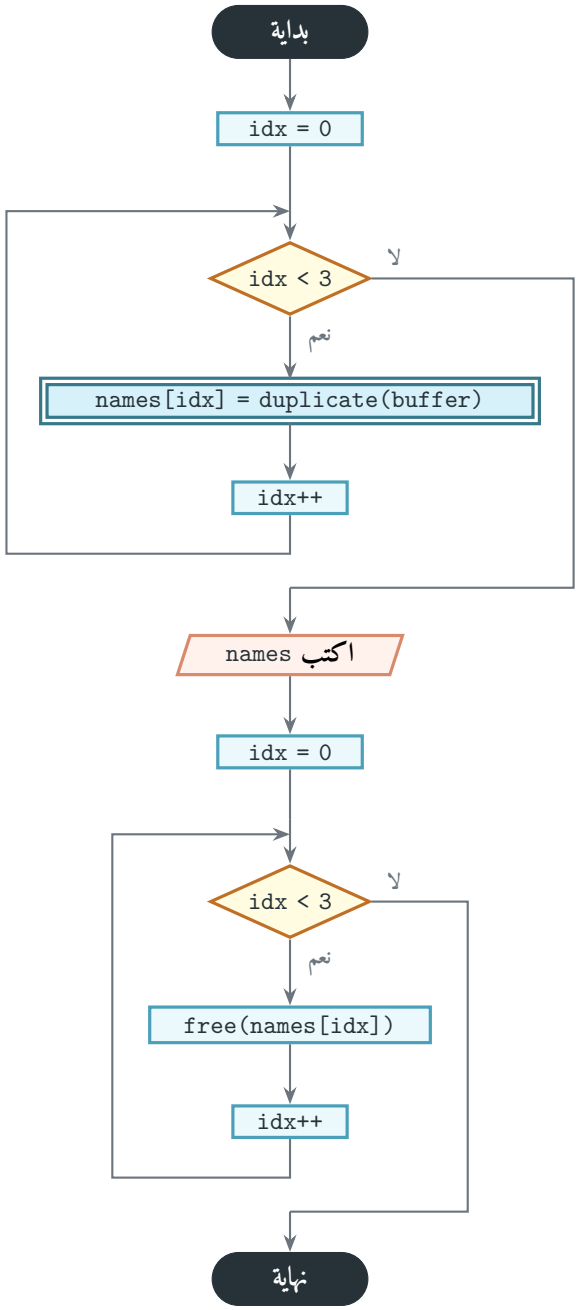
المدخلات

```
Mouse
Ultra-Wide-Curved-Monitor-34
Hub
```

نتيجة التنفيذ

```
Mouse
Ultra-Wide-Curved-Monitor-34
Hub
Used 39 bytes instead of 300
```

المخطط الإنسيابي (Organigramme)



تمرين 10.13 الإسم الكامل في ذاكرة ديناميكية

تجارة إلكترونية

متوسط

اكتب دالة `fullName(first, last)` تحجز سلسلة جديدة تحتوي على `"LAST, First"` (اللقب بحروف كبيرة)، وتعيد مؤشرها. استعملها لطباعة ملصقات الطرود.

خطوات الحل

1) الحجم = طول اللقب + 2 (فاصلة وفراغ) + طول الاسم + 1.
2) ننسخ اللقب محولًا إلى حروف كبيرة، ثم `", "`، ثم الاسم مع أول حرف كبير.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int length(const char *text) {
    int len = 0;
    while (text[len] != '\0') len++;
    return len;
}

char upper(char ch) { return (ch >= 'a' && ch <= 'z') ? ch - 'a' + 'A' : ch; }

char *fullName(const char *first, const char *last) {
    int lenF = length(first), lenL = length(last), pos, out = 0;
    char *label = malloc(lenL + 2 + lenF + 1);
    if (label == NULL) return NULL;
    for (pos = 0; pos < lenL; pos++) label[out++] = upper(last[pos]);   // اللقب كبير
    label[out++] = ',';
    label[out++] = ' ';
    for (pos = 0; pos < lenF; pos++) label[out++] = pos == 0 ? upper(first[pos]) : first[pos];
    label[out] = '\0';
    return label;
}

int main(void) {
    char first[30], last[30], *label;
    int idx;
    for (idx = 0; idx < 2; idx++) {
        scanf("%29s %29s", first, last);
        label = fullName(first, last);
        printf("Ship to: %s\n", label);
        free(label);
    }
    return 0;
}
```

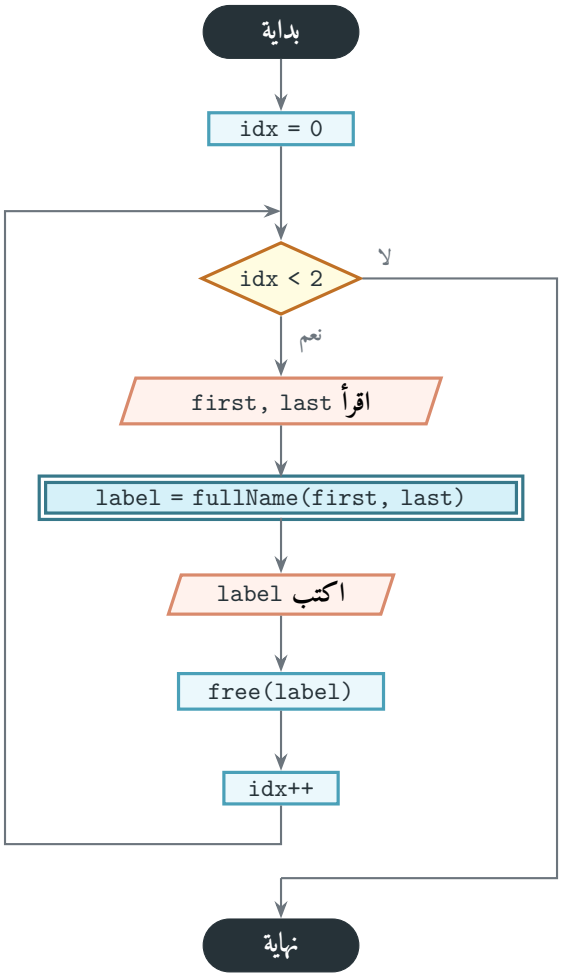
المدخلات

```
amina benali
karim zerrouki
```

نتيجة التنفيذ

```
Ship to: BENALI, Amina
Ship to: ZERROUKI, Karim
```

المخطط الإنسيابي (Organigramme)



تمرين 10.14 قلب الجدول بمؤشرين متقابلين

علم البيانات ★★ متوسط

اكتب دالة `reverse(start, end)` تقلب جزءًا من جدول في مكانه بمؤشرين: الأول من البداية والثاني من النهاية، يتبادلان ثم يتقاربان. استعملها لقلب الجدول كله، ثم لـ تدوير سجل المبيعات بـ 3 أيام (قلب ثلاث مرات).

خطوات الحل

1) ما دام `start < end`: تبديل `*start` و `*end`، ثم `start++` و `end--`.

2) التدوير يسارًا بـ `shift`: قلب `[0, shift)`، قلب `[shift, n)`، ثم قلب الكل.

البرنامج بلغة C

```c
#include <stdio.h>

void reverse(int *start, int *end) {
    int tmp;
    while (start < end) {
        tmp = *start;
        *start++ = *end;            // نكتب ثم نتقدم
        *end-- = tmp;
    }
```

```c
}

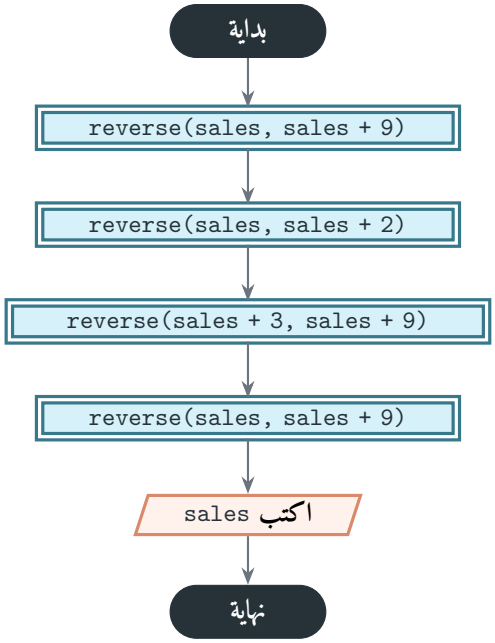
void show(char label[], int *values, int size) {
    int *ptr;
    printf("%s", label);
    for (ptr = values; ptr < values + size; ptr++) printf(" %d", *ptr);
    printf("\n");
}

int main(void) {
    int sales[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    reverse(sales, sales + 9);
    show("reversed:", sales, 10);
    reverse(sales, sales + 9);                  // نعيده كما كان
    reverse(sales, sales + 2);                  // تدوير يسارًا بـ 3
    reverse(sales + 3, sales + 9);
    reverse(sales, sales + 9);
    show("rotated: ", sales, 10);
    return 0;
}
```

نتيجة التنفيذ

```
reversed: 10 9 8 7 6 5 4 3 2 1
rotated:  4 5 6 7 8 9 10 1 2 3
```

المخطط الانسيابي (Organigramme)



تمرين 10.15 حذف المكررات في مكانها

تجارة إلكترونية ★★ متوسط

قائمة رموز المنتجات المطلوبة مرتبة وتحتوي على مكررات. اكتب دالة `unique(codes, size)` تحذف المكررات **في نفس الجدول** بمؤشري قراءة وكتابة، وتعيد الطول الجديد.

خطوات الحل

1) `write` يشير إلى آخر عنصر فريد، و `read` يمر على كل العناصر.
2) إذا اختلف `*read` عن `*write`: نتقدم بـ `write` وننسخ.
3) الطول الجديد `write - codes + 1`.

البرنامج بلغة C

```c
#include <stdio.h>

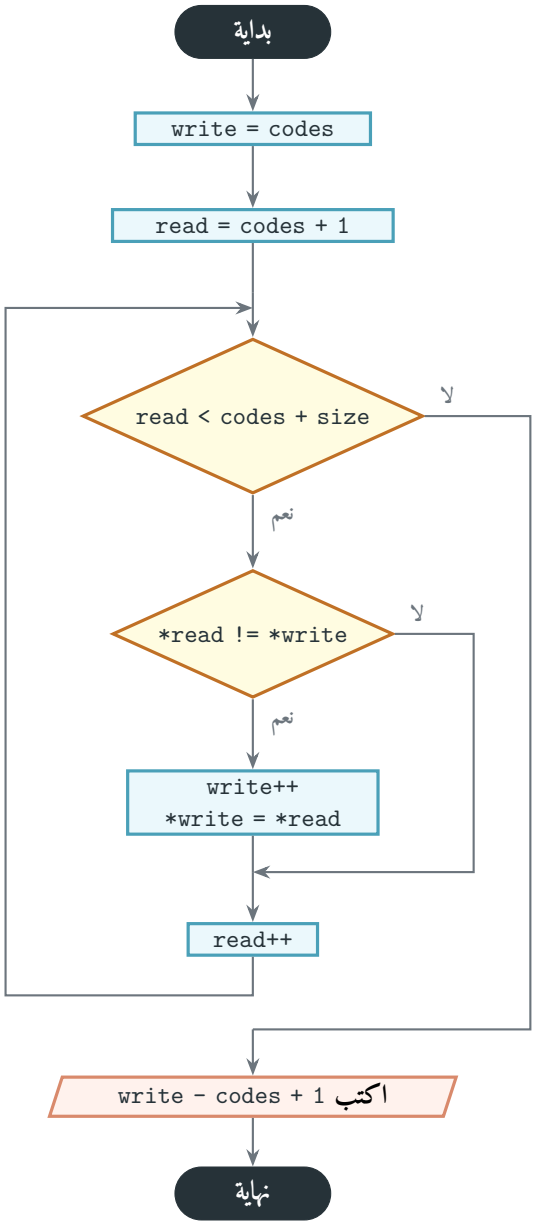
int unique(int *codes, int size) {
    int *write = codes, *read;
    if (size == 0) return 0;
    for (read = codes + 1; read < codes + size; read++)
        if (*read != *write)
            *++write = *read;            // عنصر جديد: نتقدم ثم ننسخ
    return (int)(write - codes) + 1;
}

int main(void) {
    int codes[14] = {101, 101, 105, 110, 110, 110, 204, 204, 330, 412, 412, 412, 412, 500};
    int size = unique(codes, 14), idx;
    printf("%d distinct codes:", size);
    for (idx = 0; idx < size; idx++) printf(" %d", codes[idx]);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
7 distinct codes: 101 105 110 204 330 412 500
```

المخطط الإنسيابي (Organigramme)



تمرين 10.16 تقطيع الوصف إلى كلمات بالمؤشرات

علم البيانات ★★ متوسط

اكتب دالة `splitWords(text, words, maxWords)` تقسم الجملة في مكانها: تستبدل الفراغات بـ `'\0'` وتخزن في جدول مؤشرات `char *words[]` عنوان بداية كل كلمة، دون نسخ أي حرف. ثم اعرض الكلمات وأطولها.

خطوات الحل

1) نمر بمؤشر على النص؛ عند بداية كلمة نخزن العنوان.
2) عند الفراغ بعد كلمة نكتب `'\0'` لإنهائها.
3) `words[idx]` سلسلة صالحة لأنها تنتهي بـ `'\0'` داخل النص الأصلي.

البرنامج بلغة C

```c
#include <stdio.h>

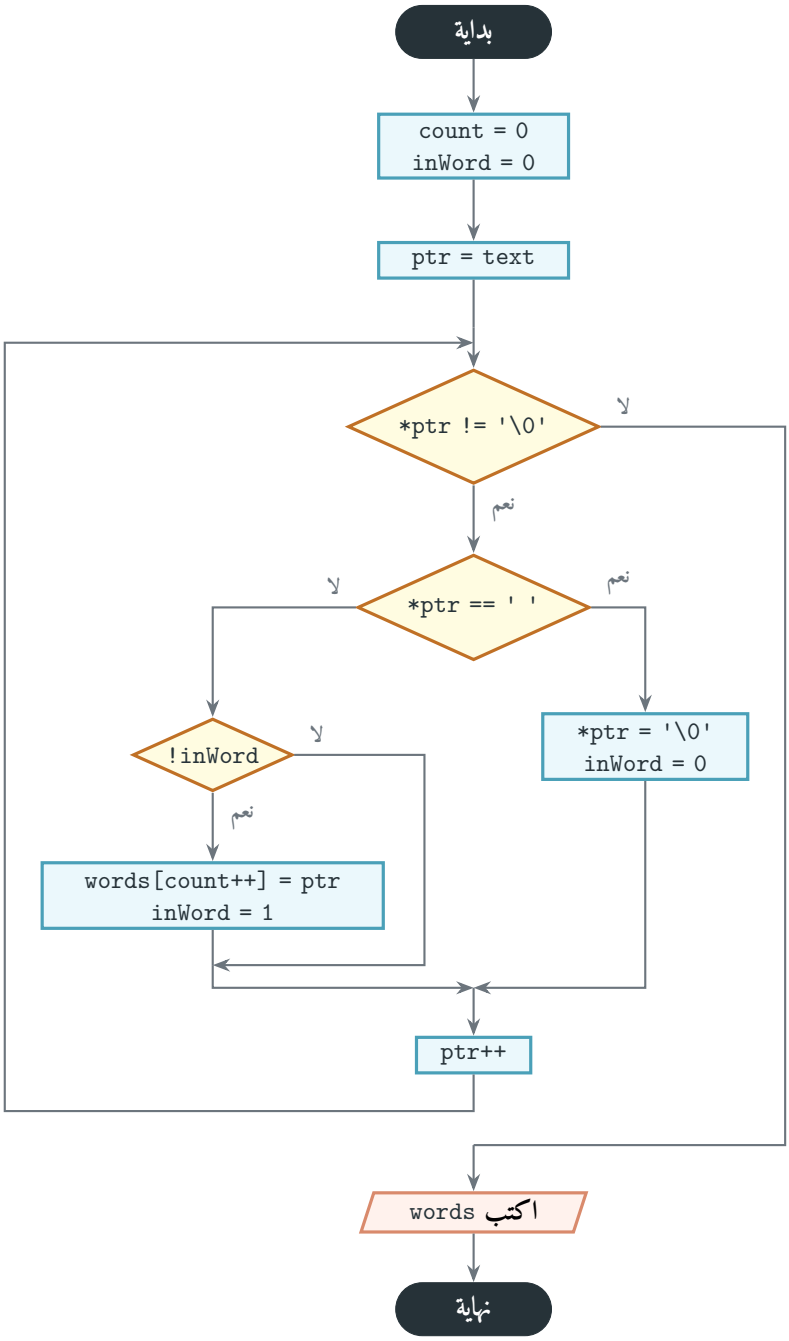
int splitWords(char *text, char *words[], int maxWords) {
    int count = 0, inWord = 0;
    char *ptr;
    for (ptr = text; *ptr != '\0'; ptr++) {
        if (*ptr == ' ') {
            *ptr = '\0';                        // نهاية الكلمة السابقة
            inWord = 0;
        } else if (!inWord && count < maxWords) {
            words[count++] = ptr;               // عنوان بداية الكلمة
            inWord = 1;
        }
    }
    return count;
}

int main(void) {
    char text[] = "  ergonomic  wireless keyboard with silent keys ";
    char *words[20], *longest;
    int count = splitWords(text, words, 20), idx, len, best = 0;
    longest = words[0];
    for (idx = 0; idx < count; idx++) {
        for (len = 0; words[idx][len] != '\0'; len++) {}
        printf("[%s] ", words[idx]);
        if (len > best) { best = len; longest = words[idx]; }
    }
    printf("\n%d words, longest: %s (%d)\n", count, longest, best);
    return 0;
}
```

نتيجة التنفيذ

```
[ergonomic] [wireless] [keyboard] [with] [silent] [keys]
6 words, longest: ergonomic (9)
```

المخطط الإنسيابي (Organigramme)



تمرين 10.17 ترتيب مؤشرات المنتجات لا المنتجات

متوسط ★★ تجارة إلكترونية

تبديل هياكل كبيرة مكلف. رتّب جدول مؤشرات `Product *view[]` يشير إلى المنتجات، حسب السعر مرة وحسب التقييم مرة أخرى، دون تحريك الهياكل الأصلية في الذاكرة.

خطوات الحل

1) `view[idx] = &items[idx]` في البداية.
2) الترتيب يبدل المؤشرات فقط (8 بايتات بدل الهيكل كله).

3) الجدول الأصلي يبقى بترتيبه.

البرنامج بلغة C

```c
#include <stdio.h>

#define NB 5

typedef struct { char name[16]; double price, rating; char details[200]; } Product;

void sortView(Product *view[], int byPrice) {
    Product *tmp;
    int pass, idx, swapIt;
    for (pass = 0; pass < NB - 1; pass++)
        for (idx = 0; idx < NB - 1 - pass; idx++) {
            swapIt = byPrice ? view[idx]->price > view[idx + 1]->price
                             : view[idx]->rating < view[idx + 1]->rating;
            if (swapIt) { tmp = view[idx]; view[idx] = view[idx + 1]; view[idx + 1] = tmp; }
        }
}

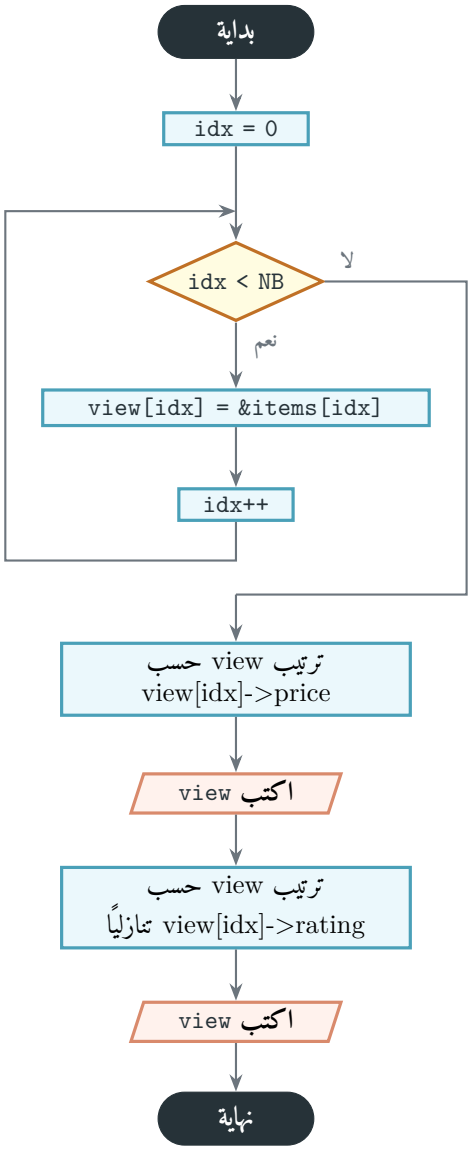
void show(char label[], Product *view[]) {
    int idx;
    printf("%s", label);
    for (idx = 0; idx < NB; idx++) printf(" %s;", view[idx]->name);
    printf("\n");
}

int main(void) {
    Product items[NB] = {{"Monitor", 32000, 4.4, ""}, {"Mouse", 1500, 4.7, ""},
                         {"Laptop", 89000, 4.2, ""}, {"Keyboard", 3200, 4.5, ""},
                         {"Webcam", 4100, 3.9, ""}};
    Product *view[NB];
    int idx;
    for (idx = 0; idx < NB; idx++) view[idx] = &items[idx];
    sortView(view, 1);
    show("By price: ", view);
    sortView(view, 0);
    show("By rating:", view);
    printf("Original first item still: %s\n", items[0].name);
    printf("Swapped %d bytes per move instead of %d\n", (int)sizeof(Product *), (int)sizeof(Product));
    return 0;
}
```

نتيجة التنفيذ

```
By price:  Mouse; Keyboard; Webcam; Monitor; Laptop;
By rating: Mouse; Keyboard; Monitor; Laptop; Webcam;
Original first item still: Monitor
Swapped 8 bytes per move instead of 232
```

المخطط الإنسيابي (Organigramme)



تمرين 10.18 مصفوفة مبيعات ديناميكية

علم البيانات ★★ متوسط

اقرأ عدد الفروع وعدد الأيام، ثم احجز مصفوفة ديناميكية `double **sales`: جدول مؤشرات على الأسطر، وكل سطر جدول محجوز على حدة. املأها، احسب مجموع كل فرع ومجموع كل يوم، ثم حرر كل الأسطر ثم جدول المؤشرات.

خطوات الحل

1) `sales = malloc(rows * sizeof(double *))` ثم لكل سطر `malloc(cols * sizeof(double))`.
2) `sales[row][col]` يعمل كمصفوفة عادية.
3) التحرير بالترتيب المعاكس: الأسطر أولًا.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int rows, cols, row, col;
    double **sales, sum;
    scanf("%d %d", &rows, &cols);
    sales = malloc(rows * sizeof(double *));          // جدول مؤشرات الأسطر
    for (row = 0; row < rows; row++)
        sales[row] = malloc(cols * sizeof(double));   // كل سطر على حدة
    for (row = 0; row < rows; row++)
        for (col = 0; col < cols; col++) scanf("%lf", &sales[row][col]);
    for (row = 0; row < rows; row++) {
        for (sum = 0, col = 0; col < cols; col++) sum += sales[row][col];
        printf("Branch %d: %.0f\n", row + 1, sum);
    }
    printf("Per day:");
    for (col = 0; col < cols; col++) {
        for (sum = 0, row = 0; row < rows; row++) sum += sales[row][col];
        printf(" %.0f", sum);
    }
    printf("\n");
    for (row = 0; row < rows; row++) free(sales[row]);  // الأسطر أولًا
    free(sales);
    return 0;
}
```

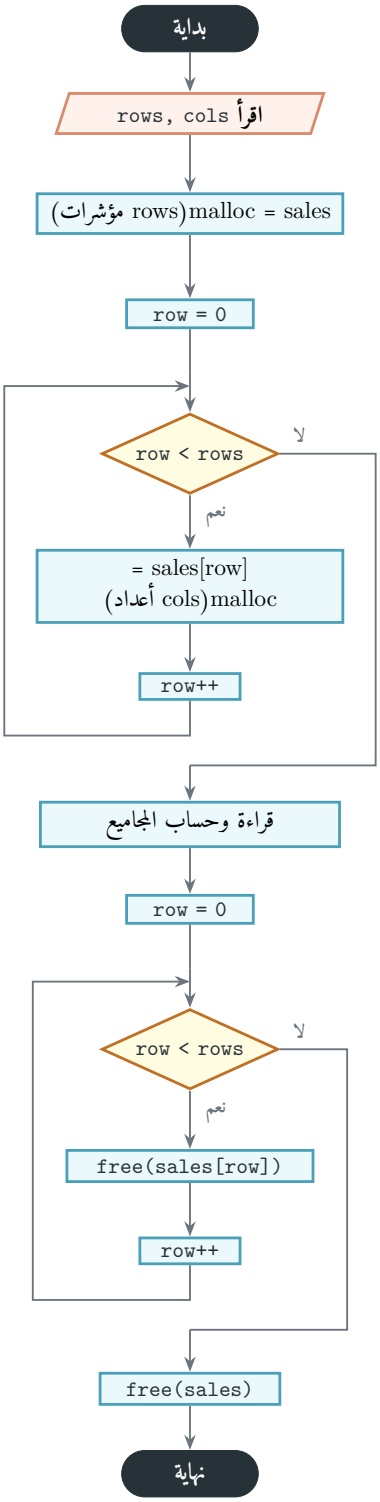
المدخلات

```
3 4
120 95 140 180
80 60 75 90
200 210 190 250
```

نتيجة التنفيذ

```
Branch 1: 535
Branch 2: 305
Branch 3: 850
Per day: 400 365 405 520
```

المخطط الإنسيابي (Organigramme)



تمرين 10.19 استراتيجية الخصم بمؤشر دالة

تجارة إلكترونية ★★ متوسط

لدى المتجر ثلاث سياسات خصم: بدون خصم، خصم ثابت 500 دج فوق 5000، و 10% فوق 10000. عرّف **مؤشر دالة** `double (*policy)(double)`، واختر السياسة حسب رمز الحملة، ثم طبّقها على عدة سلات.

خطوات الحل

1) كل سياسة دالة بنفس التوقيع `double name(double amount)`.
2) جدول مؤشرات دوال `policies[3]`، ونختار منه بالفهرس.
3) الاستدعاء: `policy(amount)` كأي دالة.

البرنامج بلغة C

```c
#include <stdio.h>

double noDiscount(double amount) { return amount; }
double fixedOff(double amount)   { return amount > 5000 ? amount - 500 : amount; }
double tenPercent(double amount) { return amount > 10000 ? amount * 0.9 : amount; }

int main(void) {
    double (*policies[3])(double) = {noDiscount, fixedOff, tenPercent};   // جدول دوال
    char codes[3] = {'N', 'F', 'P'}, campaign;
    double baskets[4] = {3000, 7500, 12000, 25000};
    double (*policy)(double) = noDiscount;
    int idx;
    scanf(" %c", &campaign);
    for (idx = 0; idx < 3; idx++)
        if (codes[idx] == campaign) policy = policies[idx];
    for (idx = 0; idx < 4; idx++)
        printf("%8.2f -> %8.2f DA\n", baskets[idx], policy(baskets[idx]));   // استدعاء عبر المؤشر
    printf("Best policy for 12000: %.2f\n",
           tenPercent(12000) < fixedOff(12000) ? tenPercent(12000) : fixedOff(12000));
    return 0;
}
```

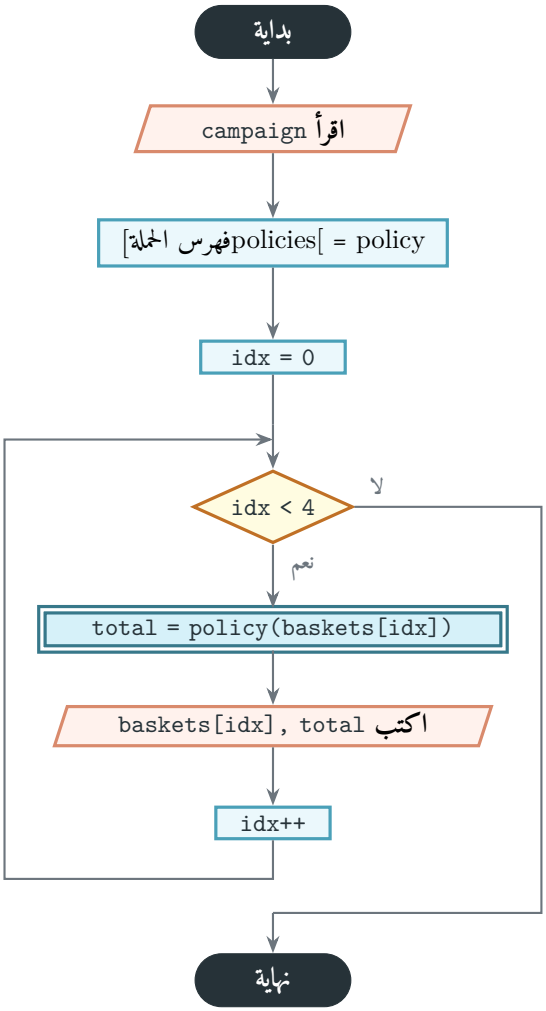
المدخلات

```
F
```

نتيجة التنفيذ

```
 3000.00 ->  3000.00 DA
 7500.00 ->  7000.00 DA
12000.00 -> 11500.00 DA
25000.00 -> 24500.00 DA
Best policy for 12000: 10800.00
```

المخطط الإنسيابي (Organigramme)



تمرين 10.20 ترتيب عام بدالة مقارنة

تجارة إلكترونية ★★ متوسط

اكتب دالة ترتيب عامة `sortProducts(items, size, compare)` تستقبل **مؤشر دالة مقارنة**، واكتب ثلاث دوال مقارنة (السعر تصاعديًا، التقييم تنازليًا، الاسم أبجديًا). هكذا نكتب الترتيب مرة واحدة ونغير المعيار فقط.

خطوات الحل

1) توقيع المقارنة:

```
int compare(const Product *first, const Product *second)
```

يعيد سالبًا إذا كان الأول قبل الثاني.

2) الترتيب بالإدراج يستدعي `compare(&key, &items[back])`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[12]; double price, rating; } Product;

int byPrice(const Product *first, const Product *second) {
    return first->price < second->price ? -1 : first->price > second->price;
}
```

```c
int byRating(const Product *first, const Product *second) {
    return first->rating > second->rating ? -1 : first->rating < second->rating;
}
int byName(const Product *first, const Product *second) {
    const char *left = first->name, *right = second->name;
    while (*left != '\0' && *left == *right) { left++; right++; }
    return *left - *right;
}

void sortProducts(Product items[], int size, int (*compare)(const Product *, const Product *)) {
    Product key;
    int idx, back;
    for (idx = 1; idx < size; idx++) {
        key = items[idx];
        for (back = idx - 1; back >= 0 && compare(&key, &items[back]) < 0; back--)
            items[back + 1] = items[back];
        items[back + 1] = key;
    }
}

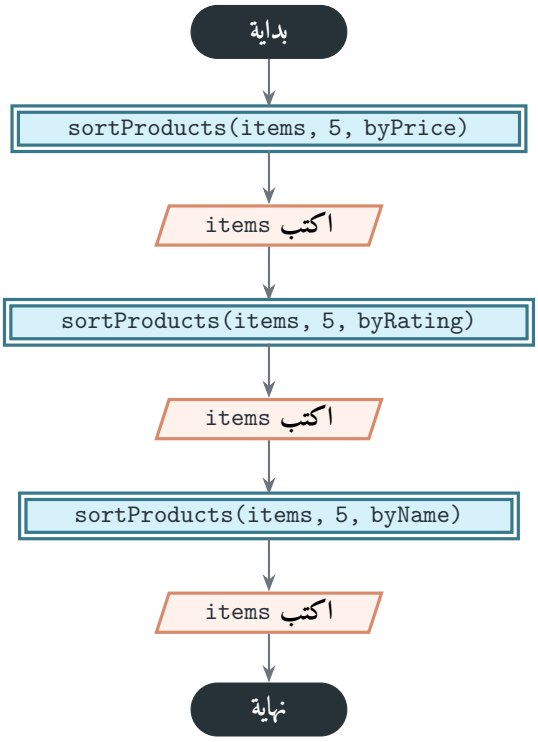
void show(char label[], Product items[], int size) {
    int idx;
    printf("%-7s", label);
    for (idx = 0; idx < size; idx++) printf(" %s", items[idx].name);
    printf("\n");
}

int main(void) {
    Product items[5] = {{"Monitor", 32000, 4.4}, {"Mouse", 1500, 4.7}, {"Laptop", 89000, 4.2},
                        {"Keyboard", 3200, 4.5}, {"Webcam", 4100, 3.9}};
    sortProducts(items, 5, byPrice);  show("price:", items, 5);
    sortProducts(items, 5, byRating); show("rating:", items, 5);
    sortProducts(items, 5, byName);   show("name:", items, 5);
    return 0;
}
```

نتيجة التنفيذ

```
price:  Mouse Keyboard Webcam Monitor Laptop
rating: Mouse Keyboard Monitor Laptop Webcam
name:   Keyboard Laptop Monitor Mouse Webcam
```

المخطط الإنسيابي (Organigramme)



تمرين 10.21 دالة تحجز جدولًا للمستدعي

علم البيانات ★★ متوسط

اكتب دالة `loadScores(result, count)` تقرأ عدد التقييمات ثم القيم، **تحجز** الجدول بنفسها، وتعيد عنوانه للمستدعي عبر مؤشر على مؤشر `double **result`، والعدد عبر `int *count`. احسب ثم المتوسط والوسيط.

خطوات الحل

1) لكي تغير الدالة مؤشرًا في `main` يجب أن تستقبل **عنوانه**: `double **result`.
2) `*result = malloc(...)` تكتب العنوان الجديد في متغير المستدعي.
3) الوسيط بعد ترتيب الجدول.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int loadScores(double **result, int *count) {
    int idx;
    scanf("%d", count);
    *result = malloc(*count * sizeof(double));   // نكتب العنوان في متغير المستدعي
    if (*result == NULL) return 0;
    for (idx = 0; idx < *count; idx++) scanf("%lf", &(*result)[idx]);
    return 1;
}

int main(void) {
    double *scores = NULL, total = 0, key;
    int count = 0, idx, back;
    if (!loadScores(&scores, &count)) return 1;
    for (idx = 0; idx < count; idx++) total += scores[idx];
    for (idx = 1; idx < count; idx++) {          // ترتيب بالإدراج
```

```
        key = scores[idx];
        for (back = idx - 1; back >= 0 && scores[back] > key; back--) scores[back + 1] = scores[back];
        scores[back + 1] = key;
    }
    printf("%d scores, mean %.2f, median %.2f\n", count, total / count, scores[count / 2]);
    free(scores);
    return 0;
}
```

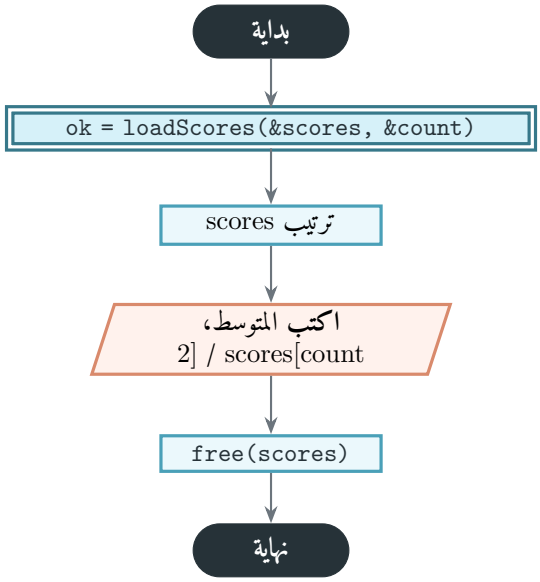
المدخلات

```
7
4.5 3.0 5.0 4.0 2.5 4.5 3.5
```

نتيجة التنفيذ

```
7 scores, mean 3.86, median 4.00
```

المخطط الإنسيابي (Organigramme)



تمرين 10.22 تصفية المنتجات في جدول جديد

تجارة إلكترونية ★★ متوسط

اكتب دالة

```
filterAbove(items, size, minRating, outCount)
```

تعيد جدولًا جديدًا محجوزًا يحتوي فقط على المنتجات التي تقييمها ≥ الحد، وتكتب عددها في *outCount. الجدول الأصلي لا يتغير.

خطوات الحل

1) مرور أول لعد العناصر المطابقة، ثم حجز بالحجم المضبوط.
2) مرور ثانٍ للنسخ.
3) إذا لم يوجد أي عنصر نعيد NULL.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { char name[12]; double price, rating; } Product;

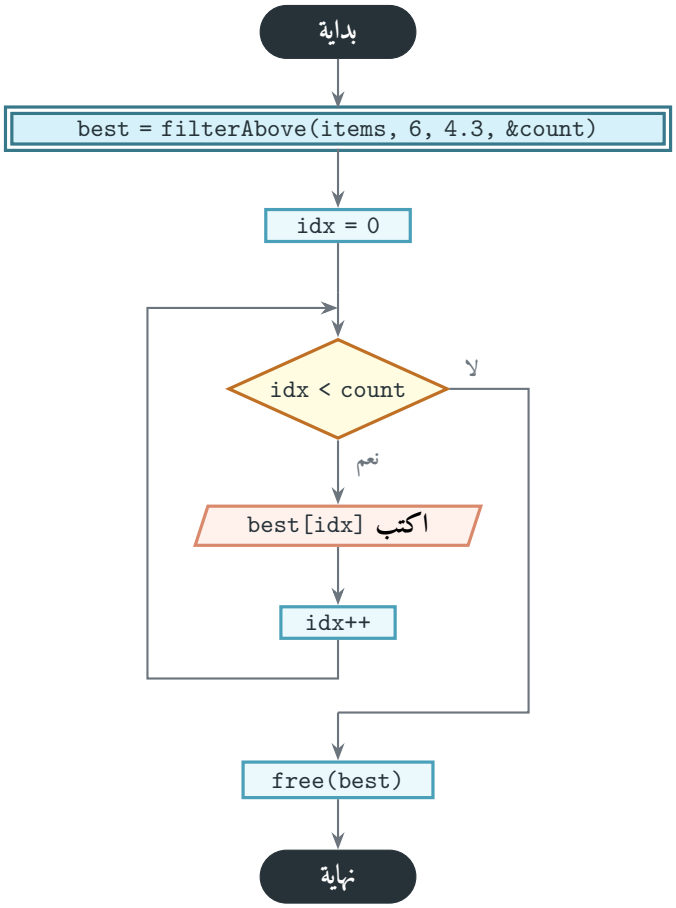
Product *filterAbove(const Product *items, int size, double minRating, int *outCount) {
    Product *result;
    int idx, count = 0;
    for (idx = 0; idx < size; idx++) if (items[idx].rating >= minRating) count++;
    *outCount = count;
    if (count == 0) return NULL;
    result = malloc(count * sizeof(Product));      // الحجم المضبوط
    if (result == NULL) { *outCount = 0; return NULL; }
    count = 0;
    for (idx = 0; idx < size; idx++)
        if (items[idx].rating >= minRating) result[count++] = items[idx];
    return result;
}

int main(void) {
    Product items[6] = {{"Monitor", 32000, 4.4}, {"Mouse", 1500, 4.7}, {"Laptop", 89000, 4.2},
                        {"Keyboard", 3200, 4.5}, {"Webcam", 4100, 3.9}, {"Hub", 3100, 4.3}};
    int count, idx;
    Product *best = filterAbove(items, 6, 4.3, &count);
    printf("%d products rated >= 4.3:\n", count);
    for (idx = 0; idx < count; idx++)
        printf("  %-9s %.1f  %8.2f DA\n", best[idx].name, best[idx].rating, best[idx].price);
    free(best);
    return 0;
}
```

نتيجة التنفيذ

```
4 products rated >= 4.3:
  Monitor   4.4  32000.00 DA
  Mouse     4.7   1500.00 DA
  Keyboard  4.5   3200.00 DA
  Hub       4.3   3100.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.23 دمج قائمتين في ذاكرة جديدة

تجارة إلكترونية ★★ متوسط

اكتب دالة `mergeNew(first, sizeA, second, sizeB)` تحجز جدولًا جديدًا بحجم `sizeA + sizeB` وتدمج فيه قائمتي أسعار مرتبتين بالمؤشرات (لا فهارس)، ثم تعيد عنوانه.

خطوات الحل

1) ثلاثة مؤشرات: `left` و `right` للقراءة و `out` للكتابة.
2) نسخ الأصغر بـ `*out++ = *left++`.
3) بعد الحلقة نسخ ما تبقى.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int *mergeNew(const int *first, int sizeA, const int *second, int sizeB) {
    int *merged = malloc((sizeA + sizeB) * sizeof(int)), *out = merged;
    const int *left = first, *right = second, *endA = first + sizeA, *endB = second + sizeB;
    if (merged == NULL) return NULL;
    while (left < endA && right < endB)
        *out++ = (*left <= *right) ? *left++ : *right++;   // الأصغر أولًا
    while (left < endA) *out++ = *left++;
```

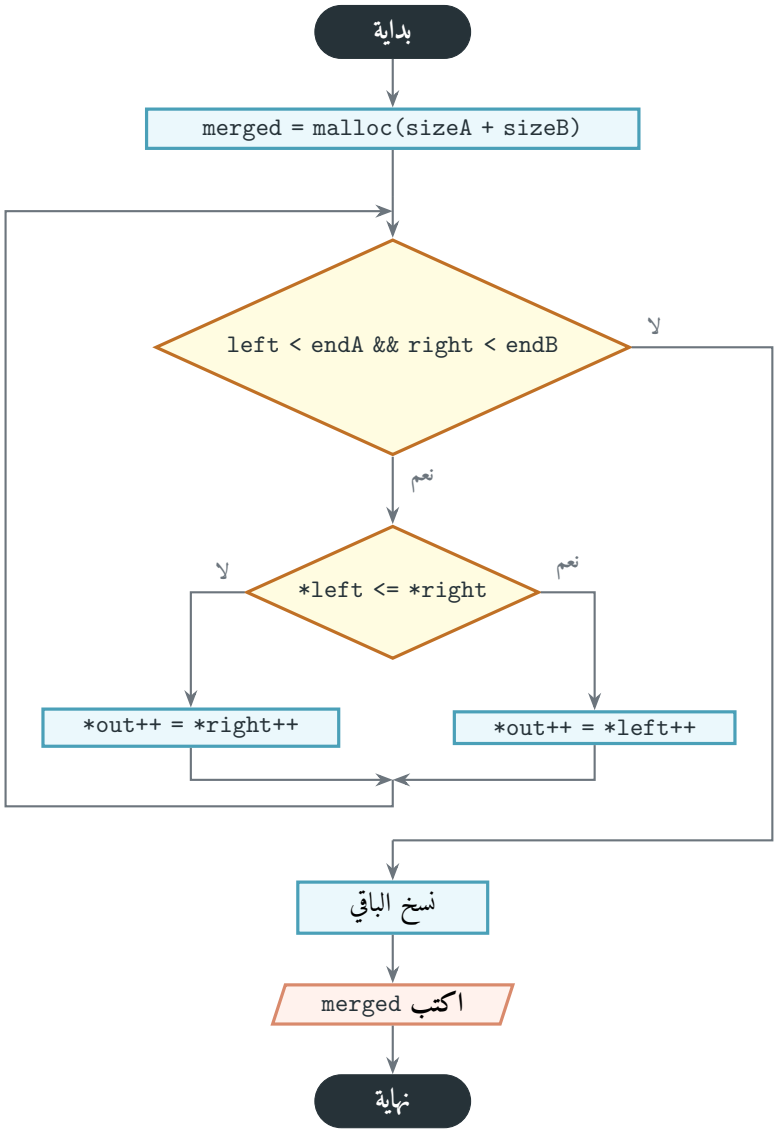
```c
    while (right < endB) *out++ = *right++;
    return merged;
}

int main(void) {
    int supplierA[5] = {450, 900, 1500, 3200, 7800};
    int supplierB[4] = {300, 1200, 2500, 9900};
    int *merged = mergeNew(supplierA, 5, supplierB, 4), *ptr;
    for (ptr = merged; ptr < merged + 9; ptr++) printf("%d ", *ptr);
    printf("\n");
    free(merged);
    return 0;
}
```

نتيجة التنفيذ

```
300 450 900 1200 1500 2500 3200 7800 9900
```

المخطط الإنسيابي (Organigramme)



تمرين 10.24 منقول مصفوفة التقييمات ديناميكيًا — علم البيانات — متوسط

مصفوفة التقييمات `ratings[ ][ ]` (3×4). احجز **ديناميكيًا** مصفوفتها المنقولة `[ ][ ]` في كتلة **واحدة** متصلة (`malloc(rows * cols)`) مع حساب الموضع `row * cols + col`، ثم احسب متوسط كل منتج من المنقولة.

خطوات الحل

1) كتلة واحدة أسرع ويحررها `free` واحد.

2) `transposed[col * nbClients + row] = ratings[row][col]`

.

3) سطر المنتج في المنقولة متصل في الذاكرة: حساب متوسطه مرور بسيط.

البرنامج بلغة C

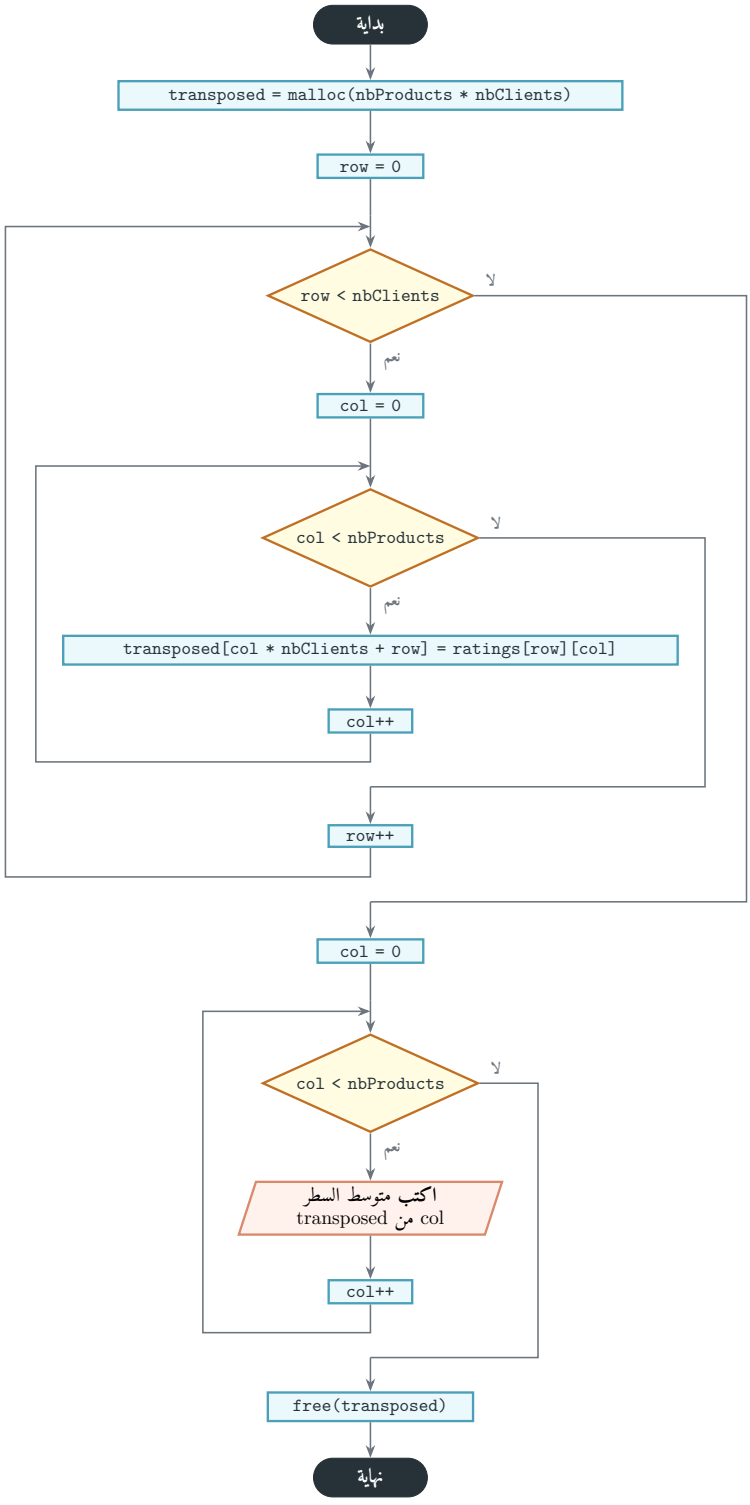
```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int ratings[4][3] = {{5, 3, 4}, {4, 2, 5}, {5, 4, 4}, {3, 1, 5}};
    int nbClients = 4, nbProducts = 3, row, col;
    int *transposed = malloc(nbProducts * nbClients * sizeof(int));   // كتلة واحدة
    double sum;
    if (transposed == NULL) return 1;
    for (row = 0; row < nbClients; row++)
        for (col = 0; col < nbProducts; col++)
            transposed[col * nbClients + row] = ratings[row][col];    // فهرس مسطح
    for (col = 0; col < nbProducts; col++) {
        printf("Product %d:", col + 1);
        for (sum = 0, row = 0; row < nbClients; row++) {
            printf(" %d", transposed[col * nbClients + row]);
            sum += transposed[col * nbClients + row];
        }
        printf("  mean %.2f\n", sum / nbClients);
    }
    free(transposed);
    return 0;
}
```

نتيجة التنفيذ

```
Product 1: 5 4 5 3  mean 4.25
Product 2: 3 2 4 1  mean 2.50
Product 3: 4 5 4 5  mean 4.50
```

المخطط الإنسيابي (Organigramme)



تمرين 10.25 تفكيك سطر CSV في مكانه

علم البيانات ★★ متوسط

اكتب دالة `splitCsv(line, fields, maxFields)` تقسم سطر CSV مفصولًا بـ `;` **في مكانه** (تستبدل `;` بـ `'\0'`) وتخزن مؤشرات الحقول، مع قبول **الحقول الفارغة** (`;;`). عالج عدة أسطر من تصدير الطلبيات.

خطوات الحل

1) الحقل الأول يبدأ عند `line`.
2) عند كل `;`: نكتبه `'\0'` ونبدأ حقلًا جديدًا عند الحرف التالي.
3) الحقل الفارغ مؤشر على `'\0'` مباشرة.

البرنامج بلغة C

```c
#include <stdio.h>

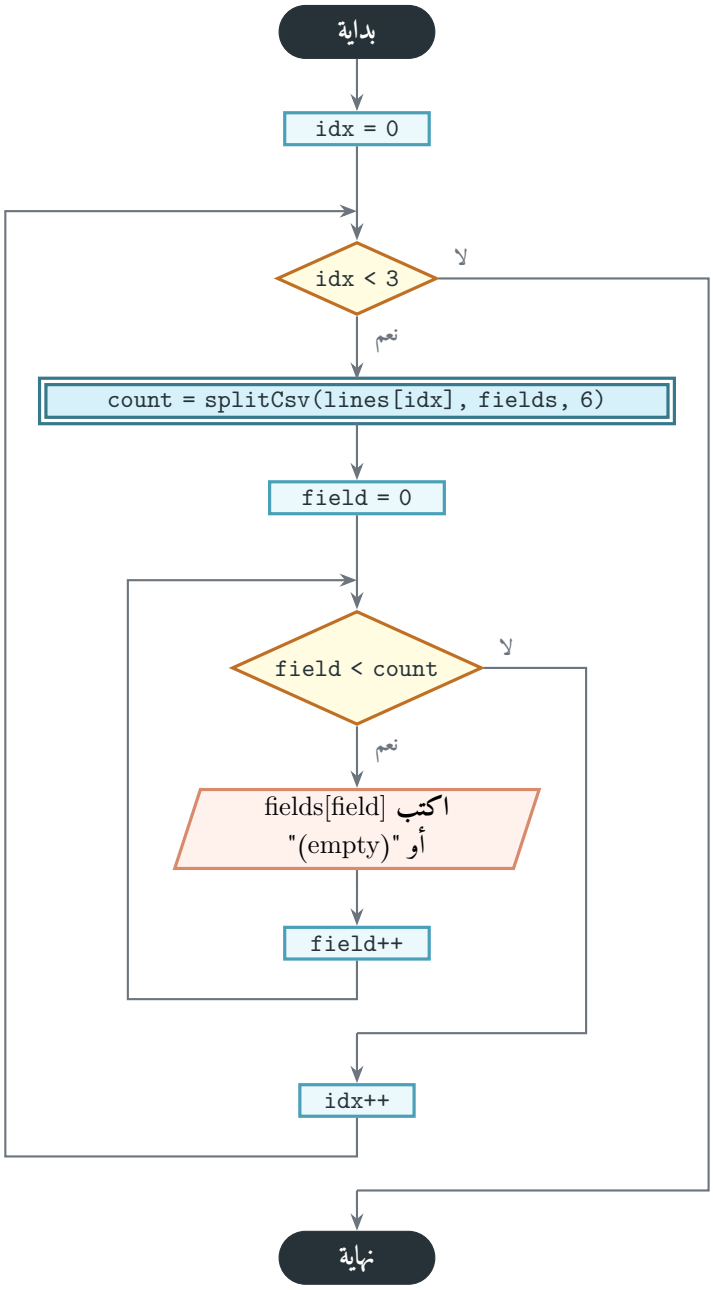
int splitCsv(char *line, char *fields[], int maxFields) {
    int count = 0;
    char *ptr = line;
    fields[count++] = ptr;                      // الحقل الأول
    for (; *ptr != '\0'; ptr++)
        if (*ptr == ';' && count < maxFields) {
            *ptr = '\0';                        // نهاية الحقل الحالي
            fields[count++] = ptr + 1;          // بداية التالي
        }
    return count;
}

int main(void) {
    char lines[3][60] = {"7781;Amina;Oran;12500;paid", "7782;Karim;;3200;", "7783;Lina;Setif;;pending"};
    char *fields[6];
    int idx, count, field;
    for (idx = 0; idx < 3; idx++) {
        count = splitCsv(lines[idx], fields, 6);
        printf("%d fields:", count);
        for (field = 0; field < count; field++)
            printf(" [%s]", *fields[field] != '\0' ? fields[field] : "-");
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
5 fields: [7781] [Amina] [Oran] [12500] [paid]
5 fields: [7782] [Karim] [-] [3200] [-]
5 fields: [7783] [Lina] [Setif] [-] [pending]
```

المخطط الإنسيابي (Organigramme)



تمرين 10.26 إحصاءات متجددة عبر مؤشر على هيكل

علم البيانات ★★ متوسط

تصل أزمنة استجابة الخادم واحدًا واحدًا ولا نريد تخزينها كلها. اكتب هيكل `Running` (العدد، المتوسط، `m2`، الأصغر، الأكبر) ودالة `update(stats, value)` تحدّثه **عبر مؤشر** بخوارزمية **ويلفورد**، ثم اعرض المتوسط والتباين في كل لحظة.

خطوات الحل

1) `count++`، `delta = value - mean`، `mean += delta / count`، `m2 += delta * (value - mean)`.
2) التباين = `m2 / count`.
3) خوارزمية بمرور واحد وذاكرة ثابتة.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int count; double mean, m2, low, high; } Running;

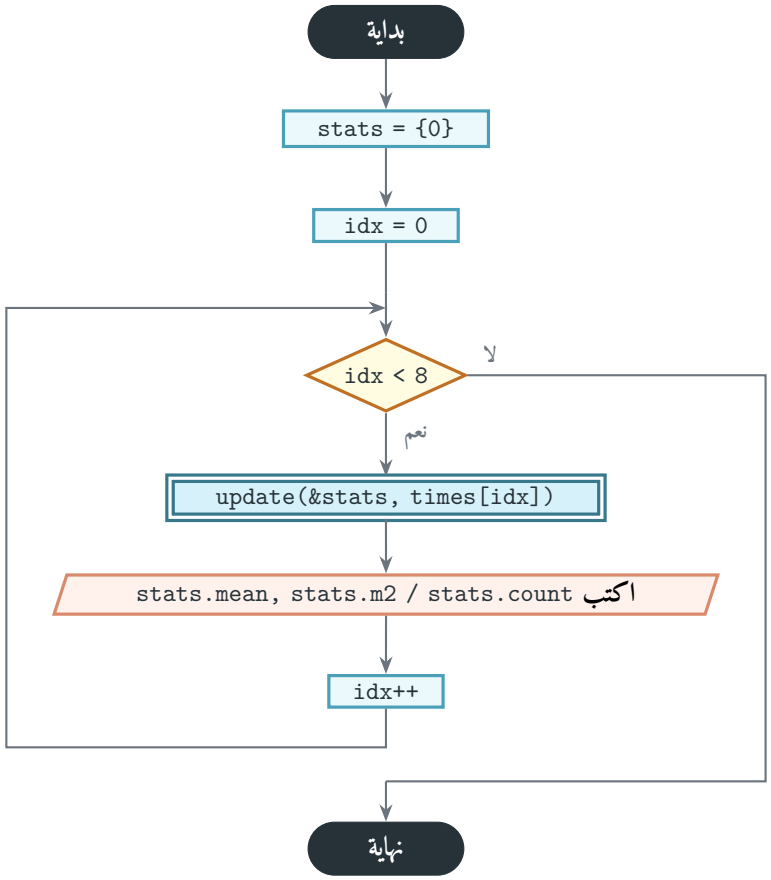
void update(Running *stats, double value) {
    double delta = value - stats->mean;
    stats->count++;
    stats->mean += delta / stats->count;
    stats->m2 += delta * (value - stats->mean);       // ويلفورد
    if (stats->count == 1 || value < stats->low) stats->low = value;
    if (stats->count == 1 || value > stats->high) stats->high = value;
}

int main(void) {
    double times[8] = {120, 135, 98, 410, 125, 140, 101, 132};
    Running stats = {0, 0, 0, 0, 0};
    int idx;
    for (idx = 0; idx < 8; idx++) {
        update(&stats, times[idx]);
        printf("n=%d value %3.0f ms -> mean %6.2f var %8.2f\n", stats.count, times[idx],
               stats.mean, stats.m2 / stats.count);
    }
    printf("min %.0f, max %.0f\n", stats.low, stats.high);
    return 0;
}
```

نتيجة التنفيذ

```
n=1 value 120 ms -> mean 120.00 var     0.00
n=2 value 135 ms -> mean 127.50 var    56.25
n=3 value  98 ms -> mean 117.67 var   230.89
n=4 value 410 ms -> mean 190.75 var 16196.69
n=5 value 125 ms -> mean 177.60 var 13649.04
n=6 value 140 ms -> mean 171.33 var 11570.56
n=7 value 101 ms -> mean 161.29 var 10523.35
n=8 value 132 ms -> mean 157.62 var  9301.73
min 98, max 410
```

المخطط الإنسيابي (Organigramme)



تمرين 10.27 المخزن الدائري لآخر الأسعار

علم البيانات ★★ متوسط

نعرض على صفحة المنتج متوسط آخر 4 أسعار. اكتب هيكل `Ring` يحتوي على جدول بسعة 4 ومؤشرًا `next` على الخانة التالية للكتابة، ودالة `addPrice` تكتب ثم تُرجع المؤشر إلى البداية عند الوصول إلى النهاية.

خطوات الحل

1) `*ring->next = price;` ثم `ring->next++`؛ إذا وصل إلى `ring->data + CAP` نعيده إلى `ring->data`.
2) `count` لا يتجاوز `CAP`.
3) المتوسط على الخانات المملوءة فقط.

البرنامج بلغة C

```c
#include <stdio.h>

#define CAP 4

typedef struct { double data[CAP]; double *next; int count; } Ring;

void addPrice(Ring *ring, double price) {
    *ring->next = price;
    ring->next++;
    if (ring->next == ring->data + CAP) ring->next = ring->data;   // دوران
    if (ring->count < CAP) ring->count++;
}
```

```c

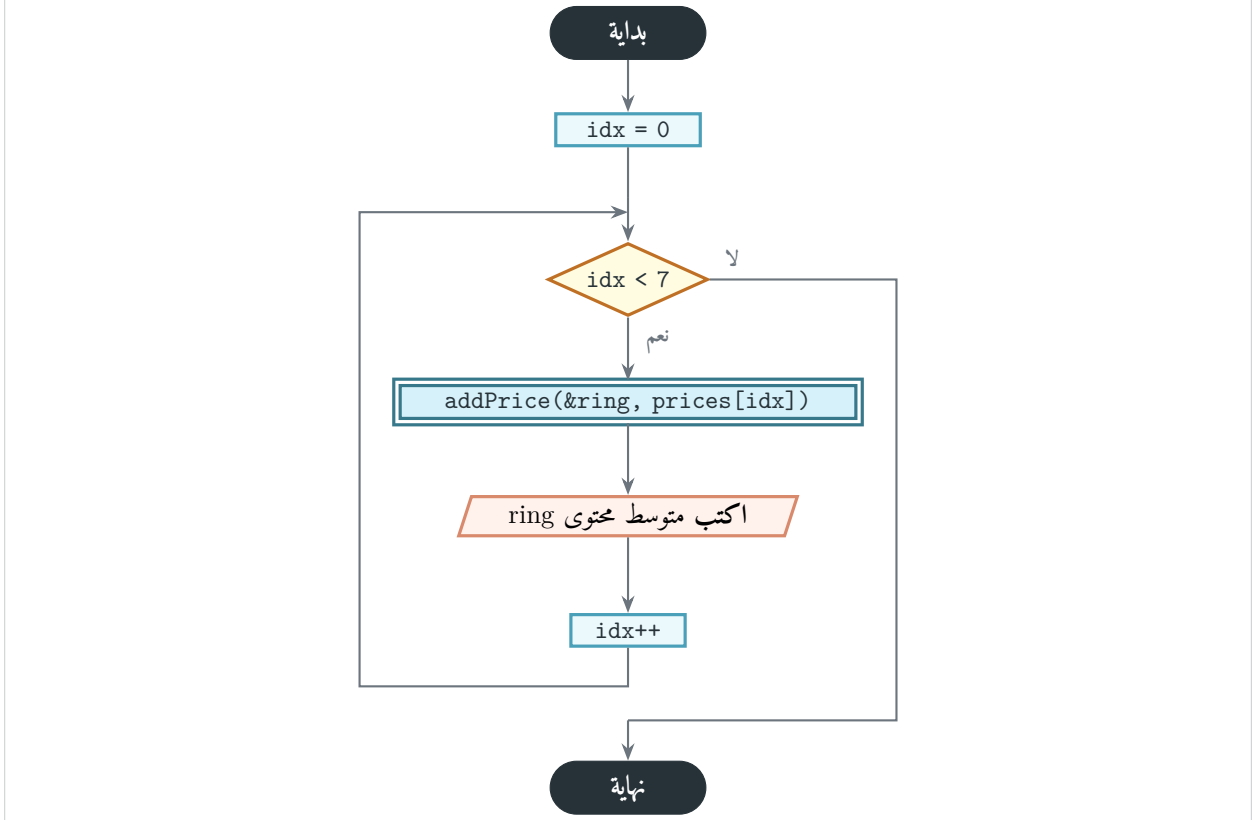
double average(const Ring *ring) {
    double sum = 0;
    int idx;
    for (idx = 0; idx < ring->count; idx++) sum += ring->data[idx];
    return sum / ring->count;
}

int main(void) {
    double prices[7] = {32000, 33500, 31000, 34500, 30000, 29000, 31500};
    Ring ring;
    int idx;
    ring.next = ring.data;
    ring.count = 0;
    for (idx = 0; idx < 7; idx++) {
        addPrice(&ring, prices[idx]);
        printf("add %.0f -> last %d avg %.2f\n", prices[idx], ring.count, average(&ring));
    }
    return 0;
}
```

نتيجة التنفيذ

```
add 32000 -> last 1 avg 32000.00
add 33500 -> last 2 avg 32750.00
add 31000 -> last 3 avg 32166.67
add 34500 -> last 4 avg 32750.00
add 30000 -> last 4 avg 32250.00
add 29000 -> last 4 avg 31125.00
add 31500 -> last 4 avg 31250.00
```

المخطط الإنسيابي (Organigramme)



تمرين 10.28 طلبيات الزبائن بأسطر متفاوتة الطول

تجارة إلكترونية ★★ متوسط

لكل زبون عدد مختلف من الطلبيات. بدل مصفوفة مستطيلة تهدر الذاكرة، احجز **جدولًا مسننًا**: جدول مؤشرات، وكل سطر محجوز بحجم طلبيات صاحبه بالضبط، مع جدول للأطوال.

خطوات الحل

1) `orders = malloc(nbClients * sizeof(double *))` و `lengths = malloc(nbClients * sizeof(int))`.

2) لكل زبون: نقرأ عدد طلبياته ثم `orders[client] = malloc(lengths[client] * sizeof(double))`.

3) نحسب المجموع لكل زبون، والذاكرة المستعملة مقارنة بمصفوفة مستطيلة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int nbClients, client, idx, cells = 0, longest = 0;
    double **orders, sum;
    int *lengths;
    scanf("%d", &nbClients);
    orders = malloc(nbClients * sizeof(double *));
    lengths = malloc(nbClients * sizeof(int));
    for (client = 0; client < nbClients; client++) {
        scanf("%d", &lengths[client]);
        orders[client] = malloc(lengths[client] * sizeof(double));   // سطر بطوله الحقيقي
        for (sum = 0, idx = 0; idx < lengths[client]; idx++) {
            scanf("%lf", &orders[client][idx]);
            sum += orders[client][idx];
        }
        cells += lengths[client];
        if (lengths[client] > longest) longest = lengths[client];
        printf("Client %d: %d orders, total %.0f\n", client + 1, lengths[client], sum);
    }
    printf("Cells used %d vs rectangular %d\n", cells, nbClients * longest);
    for (client = 0; client < nbClients; client++) free(orders[client]);
    free(orders);
    free(lengths);
    return 0;
}
```

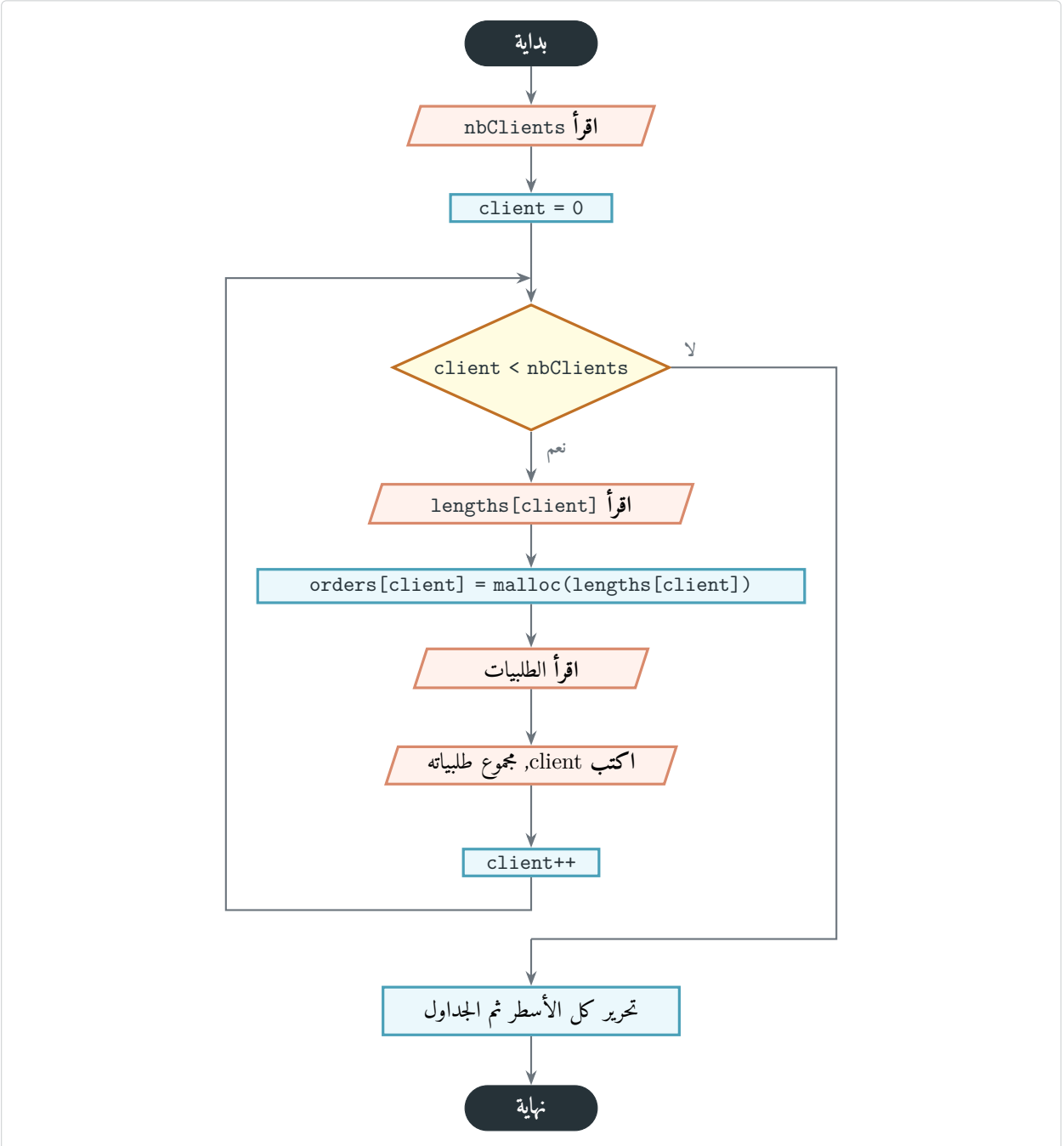
المدخلات

```
4
3 1200 4500 890
1 32000
5 450 900 1500 300 2200
2 7800 6400
```

نتيجة التنفيذ

```
Client 1: 3 orders, total 6590
Client 2: 1 orders, total 32000
Client 3: 5 orders, total 5350
Client 4: 2 orders, total 14200
Cells used 11 vs rectangular 20
```

المخطط الإنسيابي (Organigramme)



تمرين 10.29 البحث عن كلمة يعيد مؤشرًا — تجارة إلكترونية — متوسط

اكتب دالة `findText(haystack, needle)` (بديل `strstr`) تعيد **مؤشرًا** على أول ظهور للكلمة داخل الوصف أو `NULL`. استعملها لعدّ كل الظهورات بمواصلة البحث بعد المؤشر المعاد.

خطوات الحل

1) لكل موضع ممكن: نقارن الحروف بمؤشرين مؤقتين.
2) إذا انتهت `needle` فالمطابقة تامة: نعيد الموضع.
3) للعد: `pos = findText(pos + 1, needle)` حتى `NULL`.

البرنامج بلغة C

```c
#include <stdio.h>

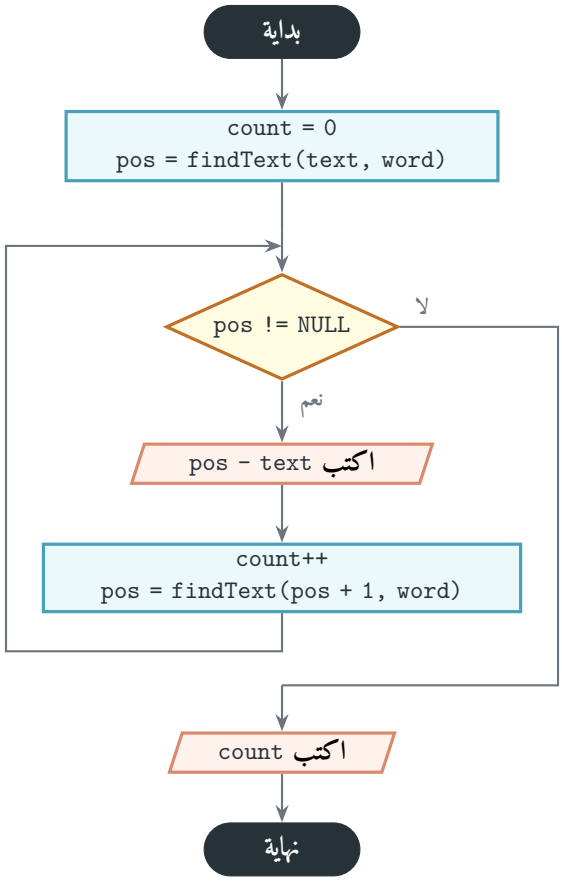
const char *findText(const char *haystack, const char *needle) {
    const char *start, *left, *right;
    for (start = haystack; *start != '\0'; start++) {
        left = start;
        right = needle;
        while (*right != '\0' && *left == *right) { left++; right++; }
        if (*right == '\0') return start;          // كل needle طابقت
    }
    return NULL;
}

int main(void) {
    const char *text = "usb cable, usb charger and usb-c hub with usb 3.0 support";
    const char *word = "usb";
    const char *pos = findText(text, word);
    int count = 0;
    printf("Positions of \"%s\":", word);
    while (pos != NULL) {
        printf(" %d", (int)(pos - text));
        count++;
        pos = findText(pos + 1, word);             // تواصل بعد آخر ظهور
    }
    printf("\n%d occurrences\n", count);
    printf("\"hdmi\" found: %s\n", findText(text, "hdmi") ? "yes" : "no");
    return 0;
}
```

نتيجة التنفيذ

```
Positions of "usb": 0 11 27 42
4 occurrences
"hdmi" found: no
```

المخطط الإنسيابي (Organigramme)



تمرين 10.30 حذف الفراغات الطرفية بالمؤشرات

تجارة إلكترونية ★★ متوسط

البيانات المدخلة في نماذج الموقع تحتوي على فراغات زائدة في البداية والنهاية. اكتب دالة `trim(text)` تعيد **مؤشرًا** على أول حرف غير فراغ، وتكتب `'\0'` بعد آخر حرف غير فراغ، دون نسخ.

خطوات الحل

1) مؤشر `start` يتقدم ما دام `*start == ' '`.
2) مؤشر `end` على آخر حرف، يتراجع ما دام فراغًا وما دام `end >= start`.
3) `*(end + 1) = '\0'`.

البرنامج بلغة C

```c
#include <stdio.h>

char *trim(char *text) {
    char *start = text, *end;
    while (*start == ' ') start++;              // فراغات البداية
    if (*start == '\0') return start;           // نص فارغ
    end = start;
    while (*(end + 1) != '\0') end++;           // آخر حرف
    while (end > start && *end == ' ') end--;   // فراغات النهاية
    *(end + 1) = '\0';
```

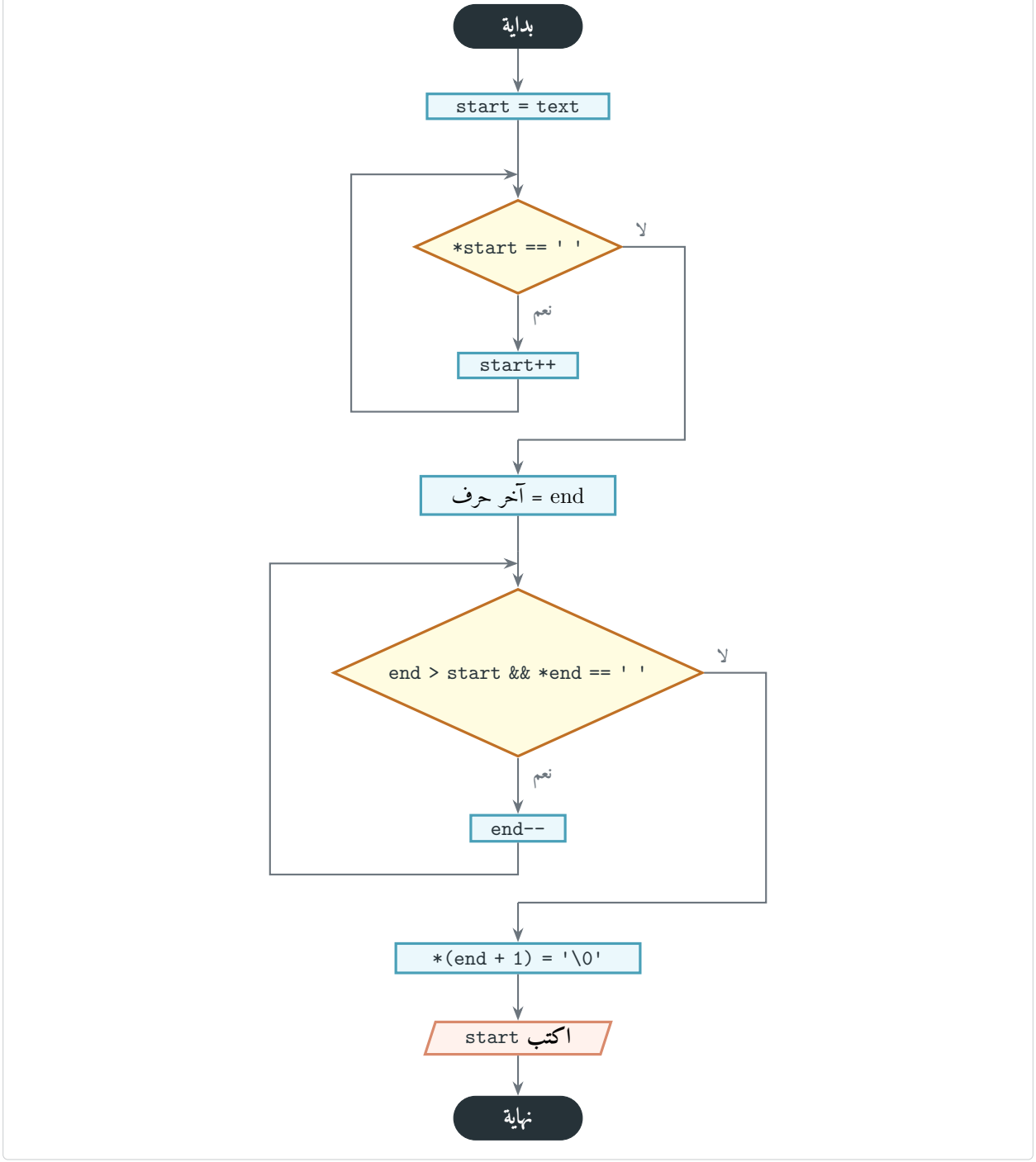
```c
    return start;
}

int main(void) {
    char inputs[3][40] = {"   amina@mail.dz  ", "Oran   ", "      "};
    int idx;
    for (idx = 0; idx < 3; idx++)
        printf("[%s]\n", trim(inputs[idx]));
    return 0;
}
```

نتيجة التنفيذ

```
[amina@mail.dz]
[Oran]
[]
```

المخطط الإنسيابي (Organigramme)



تمرين 10.31 البحث الثنائي يعيد مؤشرًا

تجارة إلكترونية متوسط

اكتب بحثًا ثنائيًا `findPrice(low, high, target)` يعمل بمؤشرين على بداية ونهاية جدول مرتب (لا فهارس)، ويعيد مؤشرًا على العنصر أو `NULL`. استعمل حساب `middle = low + (high - low) / 2`.

خطوات الحل

1) طرح مؤشرين يعطي عددًا، وإضافته لمؤشر تعطي مؤشرًا.

2) لا يمكن جمع مؤشرين (`(low + high) / 2`) خطأ.

البرنامج بلغة C

```c
#include <stdio.h>

const double *findPrice(const double *low, const double *high, double target) {
    const double *middle;
    while (low <= high) {
        middle = low + (high - low) / 2;          // لا نجمع مؤشرين
        if (*middle == target) return middle;
        if (*middle < target) low = middle + 1;
        else high = middle - 1;
    }
    return NULL;
}

int main(void) {
    double prices[10] = {99, 450, 650, 899, 1200, 1500, 2300, 3200, 4100, 7800};
    const double *found;
    double target;
    int query;
    for (query = 0; query < 2; query++) {
        scanf("%lf", &target);
        found = findPrice(prices, prices + 9, target);
        if (found != NULL) printf("%.0f found at index %d\n", target, (int)(found - prices));
        else printf("%.0f not found\n", target);
    }
    return 0;
}
```

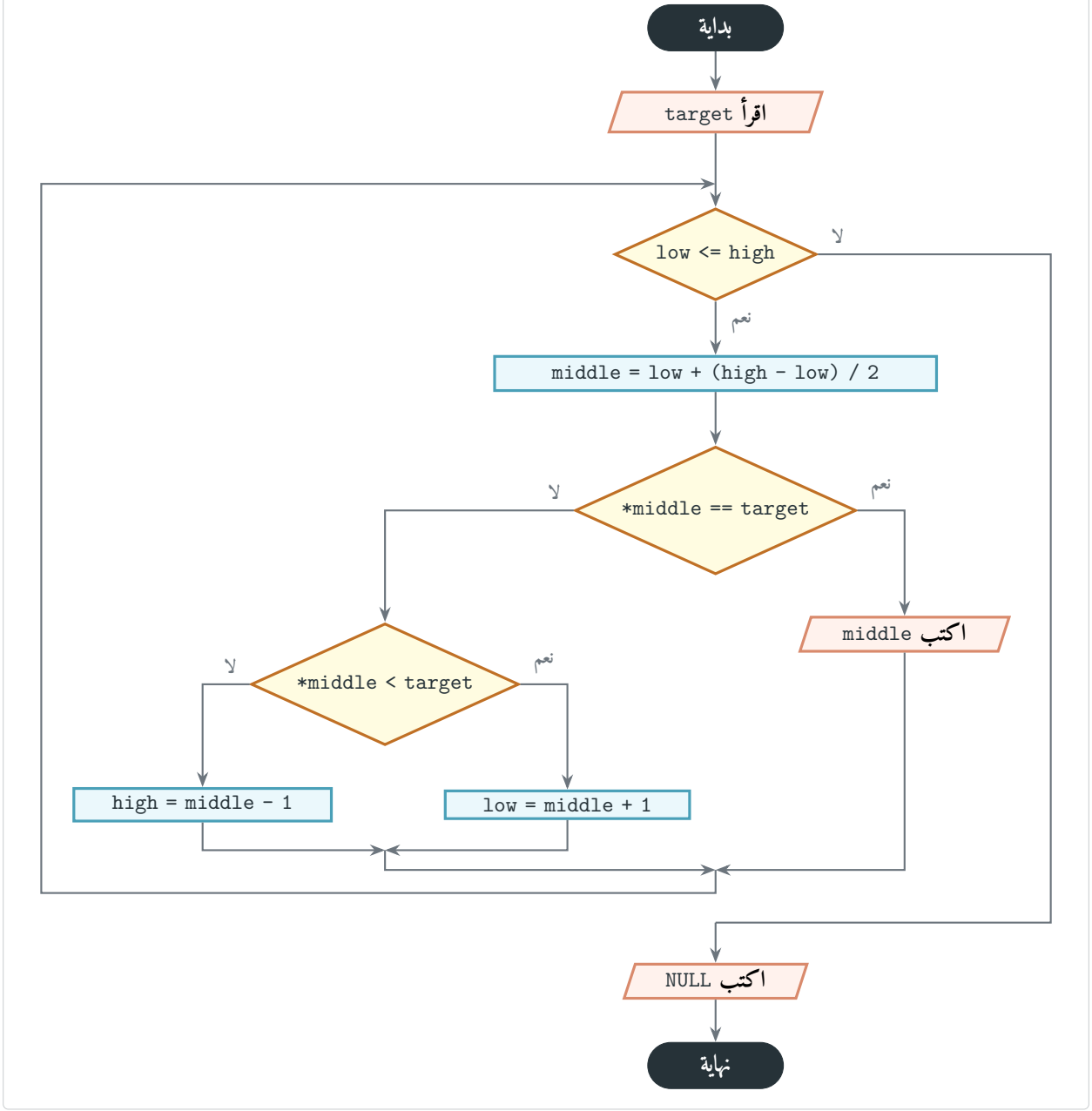
المدخلات

```
3200 4000
```

نتيجة التنفيذ

```
3200 found at index 7
4000 not found
```

المخطط الإنسيابي (Organigramme)



تمرين 10.32 نسخة عميقة من الكتالوج

تجارة إلكترونية ★★ متوسط

كل منتج هيكل يحتوي على مؤشر `char *name` على اسم محجوز ديناميكيًا. النسخ البسيط `copy = item` ينسخ المؤشر فقط (نسخة سطحية): الاسمان يتشاركان نفس الذاكرة. اكتب `deepCopy` تحجز اسمًا جديدًا، ووضّح الفرق بتعديل النسخة.

خطوات الحل

1) نسخة سطحية: تغيير `shallow.name[0]` يغير الأصل أيضًا.
2) نسخة عميقة: `malloc` جديد ونسخ الحروف.
3) كل اسم محجوز يُحرر مرة واحدة فقط.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { char *name; double price; } Product;

char *makeName(const char *text) {
    int len = 0, pos;
    char *copy;
    while (text[len] != '\0') len++;
    copy = malloc(len + 1);
    for (pos = 0; pos <= len; pos++) copy[pos] = text[pos];
    return copy;
}

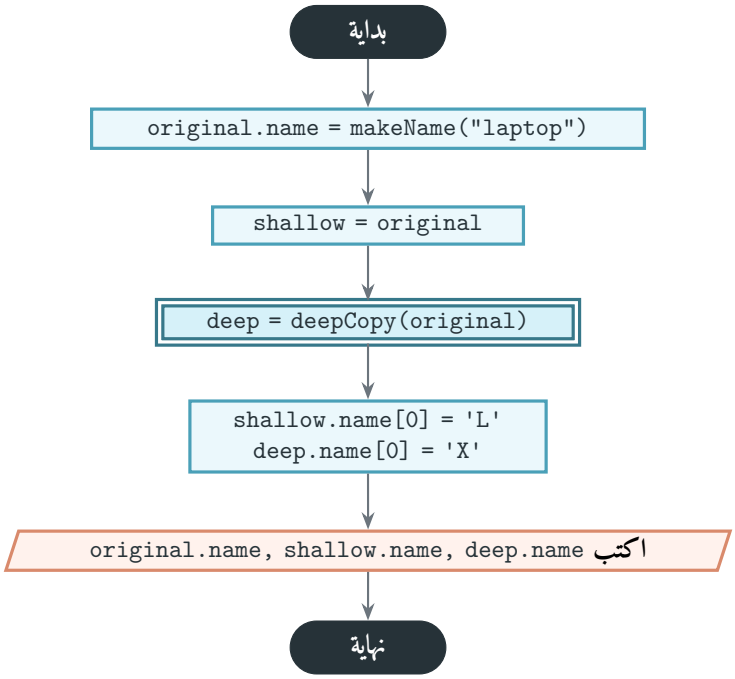
Product deepCopy(Product item) {
    Product copy = item;
    copy.name = makeName(item.name);          // ذاكرة مستقلة للاسم
    return copy;
}

int main(void) {
    Product original, shallow, deep;
    original.name = makeName("laptop");
    original.price = 89000;
    shallow = original;                       // نسخ المؤشر فقط
    deep = deepCopy(original);
    shallow.name[0] = 'L';
    deep.name[0] = 'X';
    printf("original: %s\nshallow:  %s\ndeep:     %s\n", original.name, shallow.name, deep.name);
    printf("shared memory: %s\n", original.name == shallow.name ? "yes" : "no");
    free(original.name);                      // يشير لنفس الكتلة: لا نحررها مرتين shallow
    free(deep.name);
    return 0;
}
```

نتيجة التنفيذ

```
original: Laptop
shallow:  Laptop
deep:     Xaptop
shared memory: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 10.33 تطبيق دالة على كل عناصر الجدول

علم البيانات ★★ متوسط

اكتب دالة عامة `mapArray(values, size, transform)` تطبق دالة (عبر مؤشر دالة) على كل عنصر في مكانه. استعملها لسلسلة معالجات على أسعار: إضافة الرسم، ثم التقريب، ثم التحويل إلى الأورو.

خطوات الحل

1) `void mapArray(double *values, int size, double (*transform)(double))`

.

2) كل معالجة دالة صغيرة `double f(double)`.

3) جدول من مؤشرات الدوال يمثل **خط معالجة**.

البرنامج بلغة C

```c
#include <stdio.h>

double addTax(double value)   { return value * 1.19; }
double roundTen(double value) { return (int)(value / 10 + 0.5) * 10.0; }
double toEuro(double value)   { return value / 148.5; }

void mapArray(double *values, int size, double (*transform)(double)) {
    double *ptr;
    for (ptr = values; ptr < values + size; ptr++)
        *ptr = transform(*ptr);                 // نفس الدالة لكل عنصر
}

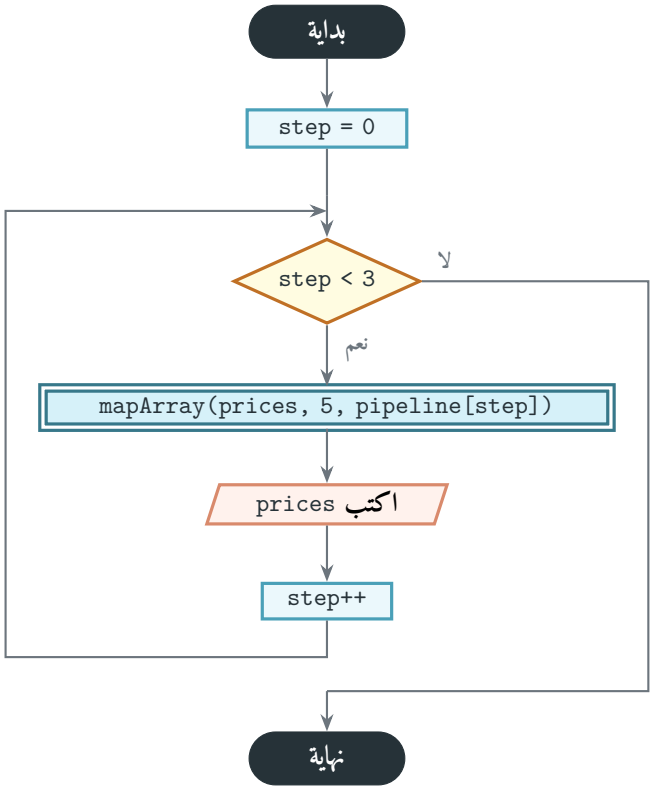
int main(void) {
    double prices[5] = {1500, 3200, 899, 4100, 32000};
    double (*pipeline[3])(double) = {addTax, roundTen, toEuro};
    char names[3][10] = {"tax", "round", "euro"};
```

```
    int step, idx;
    for (step = 0; step < 3; step++) {
        mapArray(prices, 5, pipeline[step]);
        printf("after %-5s:", names[step]);
        for (idx = 0; idx < 5; idx++) printf(" %9.2f", prices[idx]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
after tax  :   1785.00   3808.00   1069.81   4879.00  38080.00
after round:   1790.00   3810.00   1070.00   4880.00  38080.00
after euro :     12.05     25.66      7.21     32.86    256.43
```

المخطط الإنسيابي (Organigramme)



تمرين 10.34 تبديل عام بمؤشر void

علم البيانات ★★ متوسط

اكتب دالة `swapBytes(first, second, size)` تبدل أي نوع من البيانات بايتًا ببايت عبر مؤشرات `void *` (محولة إلى `unsigned char *`)، ثم دالة عامة `reverseAny(base, count, size)` تقلب جدولًا من أي نوع. اختبرها على `int` و `double` وجدول هياكل.

خطوات الحل

1) `void *` مؤشر عام لا يُفك مباشرة؛ نحوله إلى `unsigned char *` لنمر على البايتات.

2) العنصر رقم `idx` يبدأ عند `(unsigned char *)base + idx * size`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[8]; int stock; } Product;

void swapBytes(void *first, void *second, int size) {
    unsigned char *left = first, *right = second, tmp;
    int pos;
    for (pos = 0; pos < size; pos++) {              // بايتًا بايت
        tmp = left[pos];
        left[pos] = right[pos];
        right[pos] = tmp;
    }
}

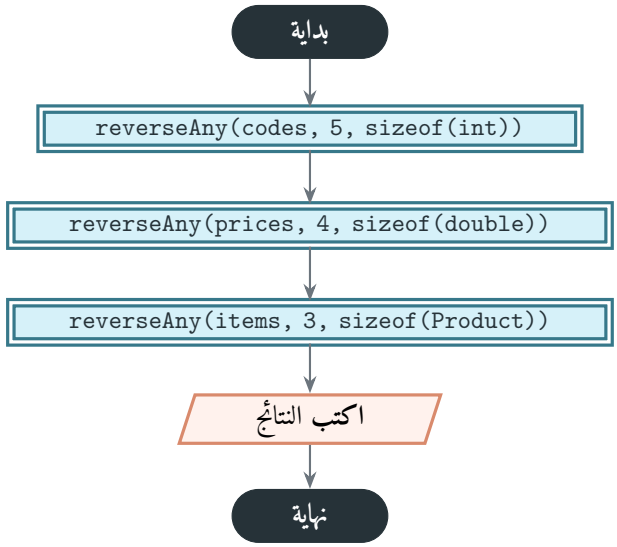
void reverseAny(void *base, int count, int size) {
    unsigned char *bytes = base;
    int low = 0, high = count - 1;
    while (low < high) {
        swapBytes(bytes + low * size, bytes + high * size, size);
        low++;
        high--;
    }
}

int main(void) {
    int codes[5] = {101, 205, 330, 412, 500}, idx;
    double prices[4] = {1.5, 32.0, 4.1, 89.9};
    Product items[3] = {{"Mouse", 12}, {"Hub", 4}, {"Cable", 40}};
    reverseAny(codes, 5, sizeof(int));
    reverseAny(prices, 4, sizeof(double));
    reverseAny(items, 3, sizeof(Product));
    for (idx = 0; idx < 5; idx++) printf("%d ", codes[idx]);
    printf("\n");
    for (idx = 0; idx < 4; idx++) printf("%.1f ", prices[idx]);
    printf("\n");
    for (idx = 0; idx < 3; idx++) printf("%s(%d) ", items[idx].name, items[idx].stock);
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
500 412 330 205 101
89.9 4.1 32.0 1.5
Cable(40) Hub(4) Mouse(12)
```

المخطط الإنسيابي (Organigramme)



تمرين 10.35 مكدس ديناميكي يكبر عند الحاجة

تجارة إلكترونية ★★ متوسط

اكتب هيكل `Stack` (مؤشر على البيانات، السعة، القمة) ودوال `createStack` و `push` (تضاعف السعة عند الامتلاء) و `pop` و `destroyStack`. استعمله للتحقق من توازن الأقواس في قالب صفحة HTML مبسط.

خطوات الحل

1) `push`: إذا `top == capacity` نحجز ضعف الحجم، ننسخ، ونحرر القديم.
2) لكل قوس فتح `{ [ (` نكدسه؛ لكل غلق نسحب ونتحقق من التطابق.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { char *data; int capacity, top; } Stack;

Stack createStack(int capacity) {
    Stack stack;
    stack.data = malloc(capacity);
    stack.capacity = capacity;
    stack.top = 0;
    return stack;
}

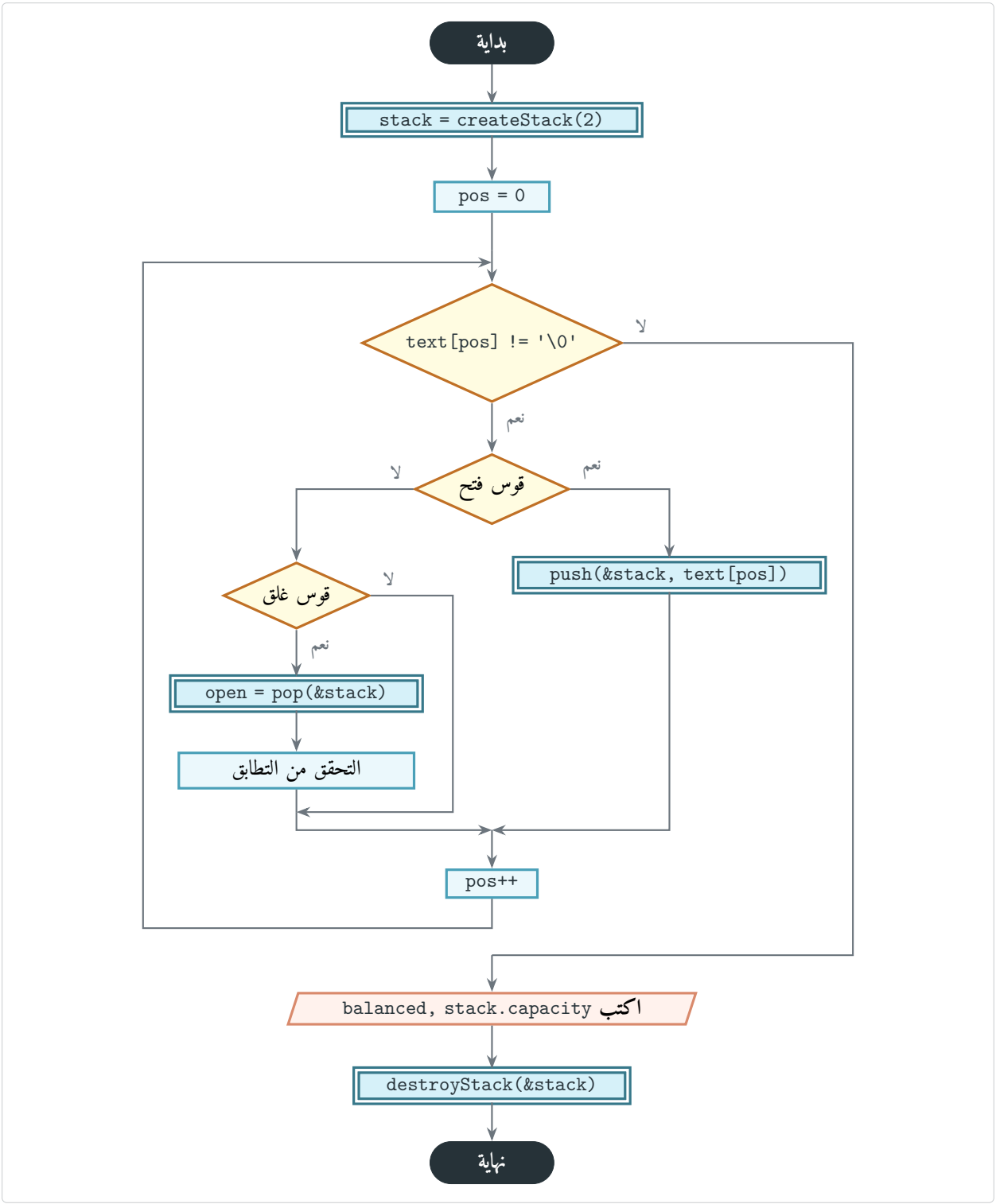
void push(Stack *stack, char value) {
    char *bigger;
    int idx;
    if (stack->top == stack->capacity) {            // تضاعف السعة
        bigger = malloc(2 * stack->capacity);
        for (idx = 0; idx < stack->top; idx++) bigger[idx] = stack->data[idx];
        free(stack->data);
        stack->data = bigger;
        stack->capacity *= 2;
    }
    stack->data[stack->top++] = value;
}
```

```
26
27 char pop(Stack *stack) { return stack->top > 0 ? stack->data[--stack->top] : '\0'; }
28 void destroyStack(Stack *stack) { free(stack->data); stack->data = NULL; }
29
30 int main(void) {
31     char text[] = "{div:[{p:(price*(1+tax))},{ul:[(a),(b),{c:[(d)]}]}]}";
32     Stack stack = createStack(2);
33     int pos, balanced = 1, deepest = 0;
34     char open, close;
35     for (pos = 0; text[pos] != '\0' && balanced; pos++) {
36         close = text[pos];
37         if (close == '(' || close == '[' || close == '{') {
38             push(&stack, close);
39             if (stack.top > deepest) deepest = stack.top;
40         } else if (close == ')' || close == ']' || close == '}') {
41             open = pop(&stack);
42             balanced = (open == '(' && close == ')') || (open == '[' && close == ']') ||
43                        (open == '{' && close == '}');
44         }
45     }
46     if (stack.top != 0) balanced = 0;
47     printf("%s, max depth %d, final capacity %d\n", balanced ? "balanced" : "unbalanced",
48            deepest, stack.capacity);
49     destroyStack(&stack);
50     return 0;
51 }
```

نتيجة التنفيذ

```
balanced, max depth 7, final capacity 8
```

المخطط الإنسيابي (Organigramme)



تمرين 10.36 جداء مصفوفتين ديناميكيتين

علم البيانات ★★ متوسط

مصفوفة الكميات `qty[ ][ ]` (3×4) ومصفوفة التكاليف `cost[ ][ ]` (4×2): سعر الشراء وتكلفة النقل). احجز المصفوفات ديناميكيًا بدالة `newMatrix(rows, cols)` واحسب الجداء `qty × cost` لإيجاد التكاليف لكل فرع.

خطوات الحل

1) `newMatrix` تحجز جدول مؤشرات وأسطرًا وتعيد `double **`.

2) `result[row][col] = Σ qty[row][mid] * cost[mid][col]`

.

3) `freeMatrix` تحرر الأسطر ثم الجدول.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

double **newMatrix(int rows, int cols) {
    double **matrix = malloc(rows * sizeof(double *));
    int row, col;
    for (row = 0; row < rows; row++) {
        matrix[row] = malloc(cols * sizeof(double));
        for (col = 0; col < cols; col++) matrix[row][col] = 0;
    }
    return matrix;
}

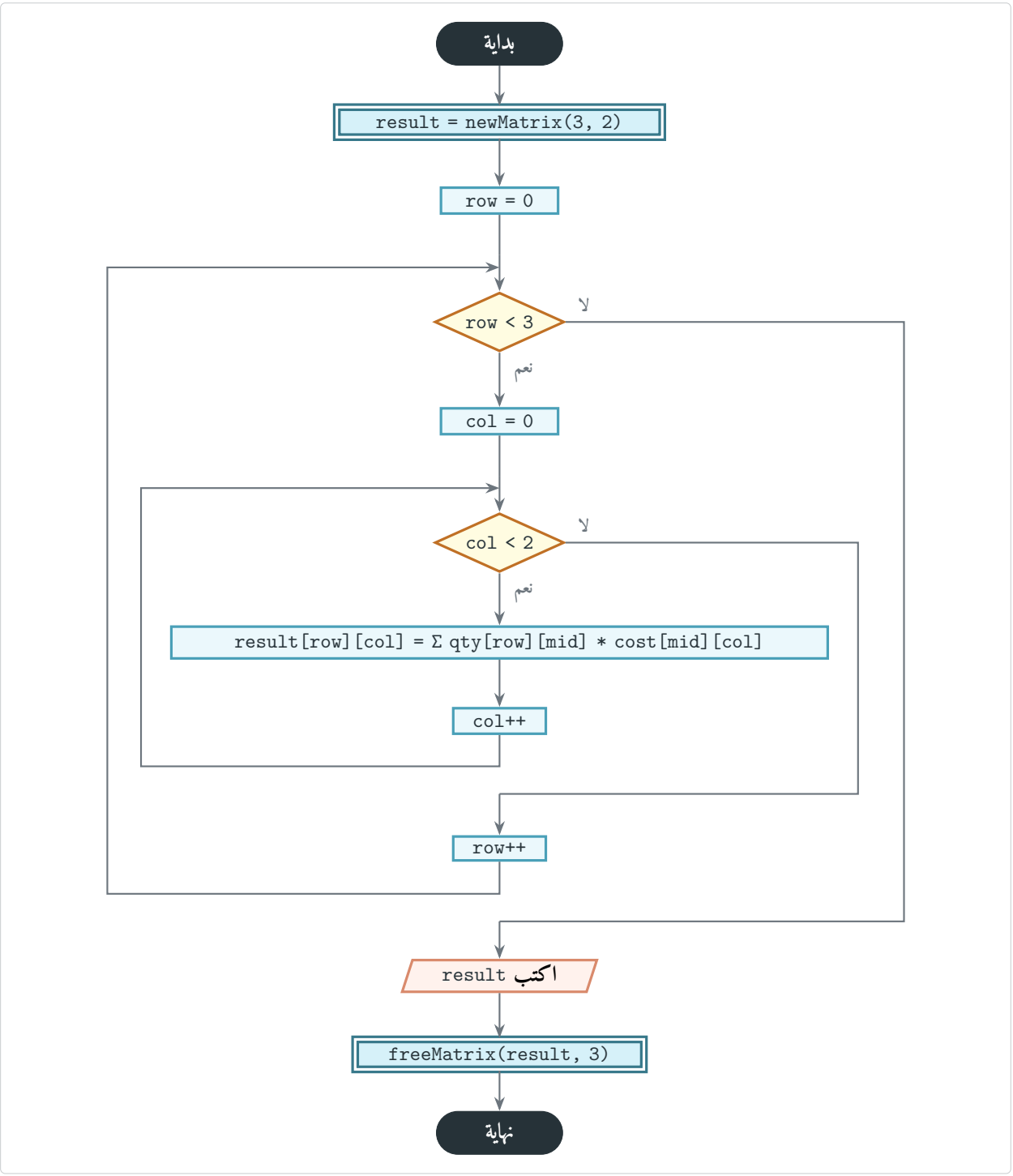
void freeMatrix(double **matrix, int rows) {
    int row;
    for (row = 0; row < rows; row++) free(matrix[row]);
    free(matrix);
}

int main(void) {
    double qtyData[3][4] = {{10, 5, 0, 8}, {4, 12, 6, 2}, {7, 0, 9, 5}};
    double costData[4][2] = {{1200, 50}, {3200, 120}, {899, 30}, {4100, 150}};
    double **qty = newMatrix(3, 4), **cost = newMatrix(4, 2), **result = newMatrix(3, 2);
    int row, col, mid;
    for (row = 0; row < 3; row++) for (col = 0; col < 4; col++) qty[row][col] = qtyData[row][col];
    for (row = 0; row < 4; row++) for (col = 0; col < 2; col++) cost[row][col] = costData[row][col];
    for (row = 0; row < 3; row++)
        for (col = 0; col < 2; col++)
            for (mid = 0; mid < 4; mid++)
                result[row][col] += qty[row][mid] * cost[mid][col];   // سطر × عمود
    for (row = 0; row < 3; row++)
        printf("Branch %d: purchase %8.0f DA, transport %6.0f DA\n", row + 1, result[row][0],
          ↪ result[row][1]);
    freeMatrix(qty, 3);
    freeMatrix(cost, 4);
    freeMatrix(result, 3);
    return 0;
}
```

نتيجة التنفيذ

```
Branch 1: purchase    60800 DA, transport   2300 DA
Branch 2: purchase    56794 DA, transport   2120 DA
Branch 3: purchase    36991 DA, transport   1370 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.37 المجاميع التراكمية للإستعلامات السريعة

علم البيانات ★★ متوسط

نريد الإجابة بسرعة عن أسئلة مثل "كم بعنا بين اليوم 5 واليوم 20؟". احجز جدول **مجاميع تراكمية** `prefix[day] = 0..day-1` بحجم `n + 1`، واملأه بمؤشرين (قراءة وكتابة)، ثم أجب عن كل استعلام بطرح واحد.

خطوات الحل

1) `prefix[0] = 0`، و `*(out + 1) = *out + *in`.
2) مبيعات المجال `[from, to]` (من 1) = `prefix[to] - prefix[from - 1]`.
3) البناء خطي، وكل استعلام بعملية واحدة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int days = 30, day, nbQueries, query, from, to;
    long *sales = malloc(days * sizeof(long)), *prefix = malloc((days + 1) * sizeof(long));
    long *in, *out;
    for (day = 0; day < days; day++) sales[day] = 1000 + (day * 37) % 500;   // بيانات تجريبية
    prefix[0] = 0;
    for (in = sales, out = prefix; in < sales + days; in++, out++)
        *(out + 1) = *out + *in;                    // المجموع التراكمي
    scanf("%d", &nbQueries);
    for (query = 0; query < nbQueries; query++) {
        scanf("%d %d", &from, &to);
        printf("Days %2d..%2d: %ld DA\n", from, to, prefix[to] - prefix[from - 1]);
    }
    free(sales);
    free(prefix);
    return 0;
}
```

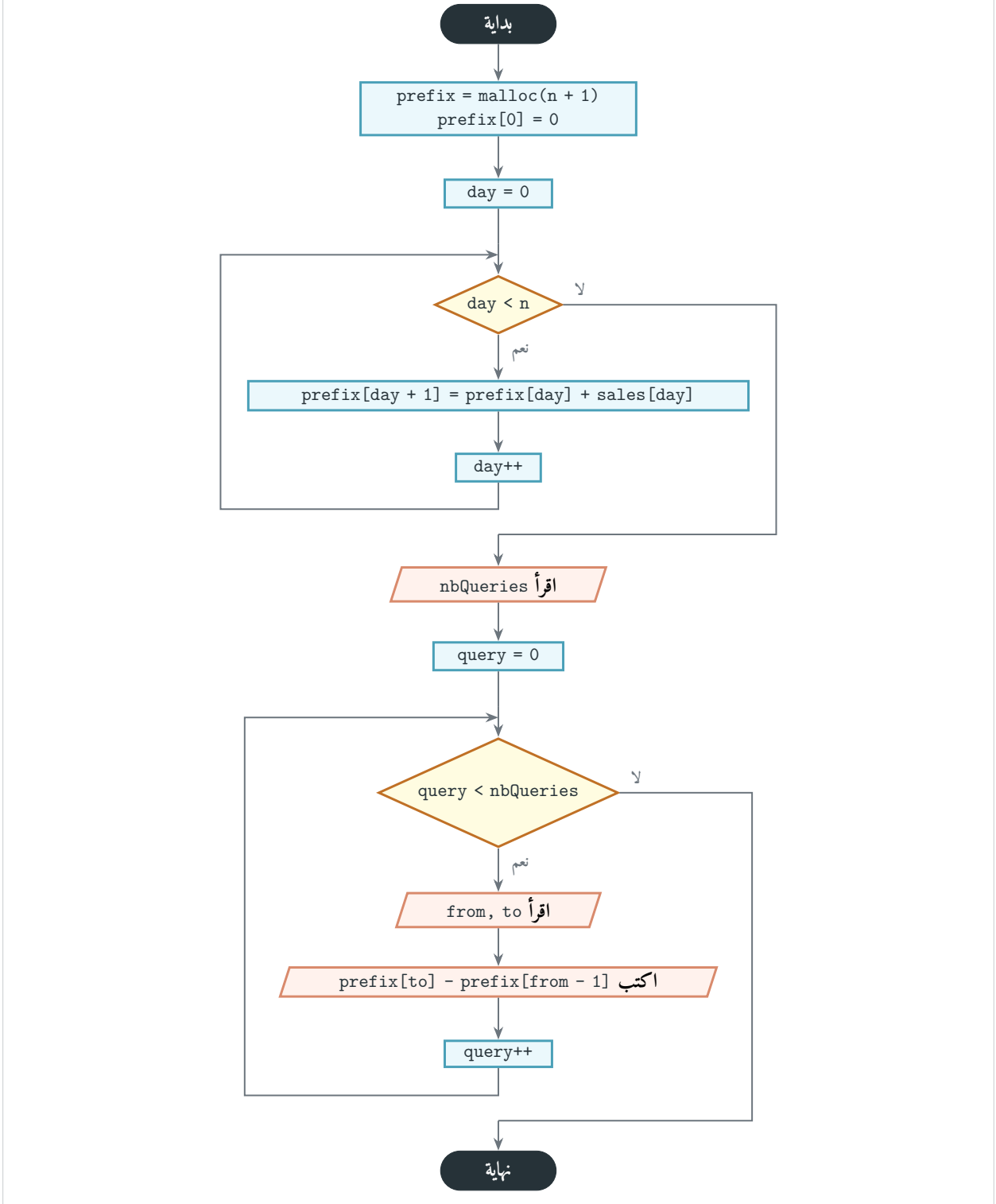
المدخلات

```
3
5 20
1 30
14 14
```

نتيجة التنفيذ

```
Days  5..20: 19808 DA
Days  1..30: 37095 DA
Days 14..14: 1481 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.38 كشف تسرب الذاكرة بعدّاد

تجارة إلكترونية ★★ متوسط

اكتب دالتين مغلفتين `trackedMalloc(size)` و `trackedFree(ptr)` تعدّان عدد الكتل المحجوزة والمحررة وعدد البايتات الحية. شغّل محاكاة سلة يُنسى فيها تحرير بعض الكتل، ثم اعرض تقرير **التسرب** في النهاية.

خطوات الحل

1) عدادات عامة: `allocations`، `releases`، `liveBytes`.
2) لمعرفة حجم الكتلة عند التحرير نحجز `size + sizeof(long)` ونخزن الحجم في بدايتها، ونعيد العنوان بعد الحجم.
3) التسرب = `allocations - releases`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int allocations = 0, releases = 0;
long liveBytes = 0;

void *trackedMalloc(long size) {
    long *block = malloc(size + sizeof(long));
    if (block == NULL) return NULL;
    *block = size;                          // نخزن الحجم قبل البيانات
    allocations++;
    liveBytes += size;
    return block + 1;                       // المستخدم يرى ما بعد الحجم
}

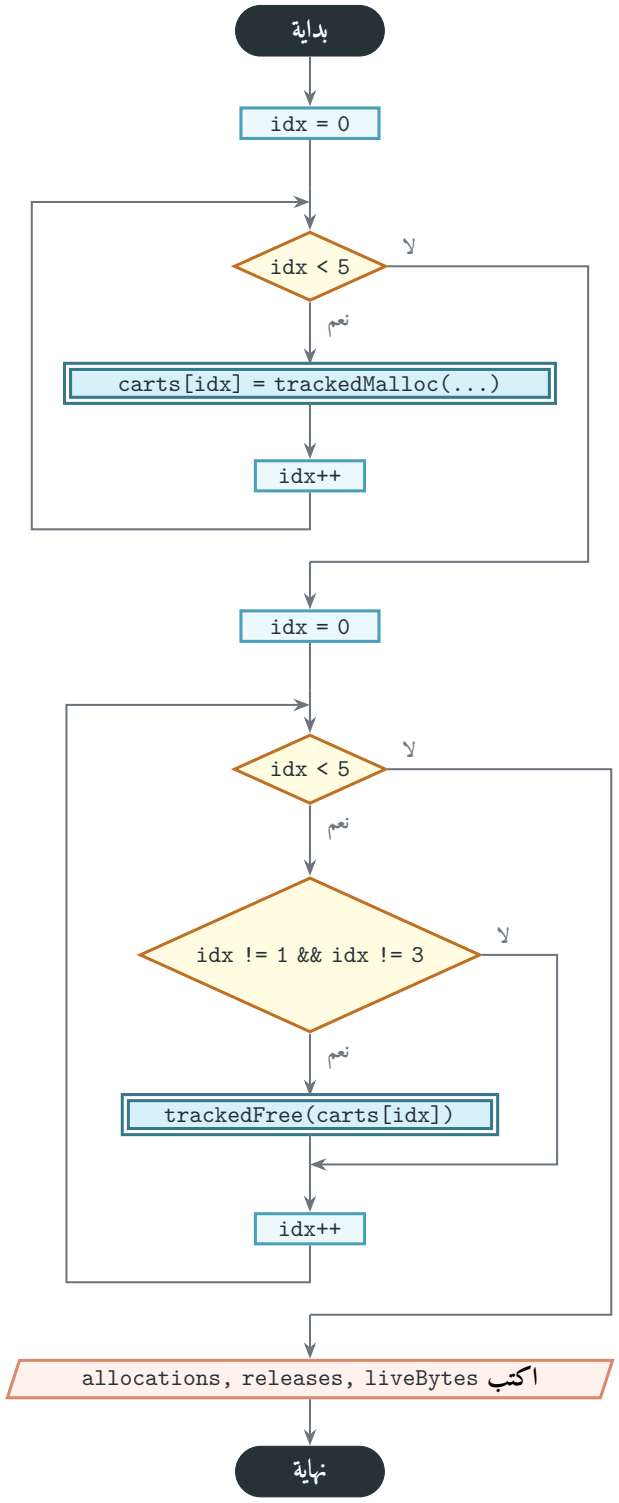
void trackedFree(void *ptr) {
    long *block = (long *)ptr - 1;          // نرجع إلى بداية الكتلة
    liveBytes -= *block;
    releases++;
    free(block);
}

int main(void) {
    double *carts[5];
    int idx, items;
    for (idx = 0; idx < 5; idx++) {
        items = 3 + idx * 2;
        carts[idx] = trackedMalloc(items * sizeof(double));
        carts[idx][0] = 100.0 * items;
    }
    for (idx = 0; idx < 5; idx++)
        if (idx != 1 && idx != 3)           // نسيان تحرير السلتين 1 و 3
            trackedFree(carts[idx]);
    printf("allocations %d, releases %d\n", allocations, releases);
    printf("LEAK: %d blocks, %ld bytes still allocated\n", allocations - releases, liveBytes);
    trackedFree(carts[1]);
    trackedFree(carts[3]);
    printf("after fix: %ld bytes\n", liveBytes);
    return 0;
}
```

نتيجة التنفيذ

```
allocations 5, releases 3
LEAK: 2 blocks, 112 bytes still allocated
after fix: 0 bytes
```

المخطط الإنسيابي (Organigramme)



تمرين 10.39 مدرج تكراري بعدد فئات متغير

علم البيانات

متوسط

اقرأ عدد الفئات المطلوب، ثم احجز ديناميكيًا جدول العدادات وجدول الحدود، ووزّع أعمار الزبائن على الفئات بعرض متساوٍ، واعرض المدرج. جرّب 4 فئات ثم 7.

خطوات الحل

1) الحجز يعتمد على قيمة تُقرأ أثناء التنفيذ.

2) `bin = (age - low) * bins / (high - low + 1)`

.

3) نحرر الجدولين بعد كل مدرج.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NB 20

int main(void) {
    int ages[NB] = {19, 23, 31, 45, 52, 27, 34, 22, 61, 38, 29, 47, 25, 33, 56, 41, 21, 36, 30, 66};
    int low = 18, high = 67, bins, query, idx, bin, star, *counts;
    for (query = 0; query < 2; query++) {
        scanf("%d", &bins);
        counts = malloc(bins * sizeof(int));              // حجم يحدده المستخدم
        for (bin = 0; bin < bins; bin++) counts[bin] = 0;
        for (idx = 0; idx < NB; idx++) counts[(ages[idx] - low) * bins / (high - low + 1)]++;
        printf("%d bins:\n", bins);
        for (bin = 0; bin < bins; bin++) {
            printf("  %2d-%2d |", low + bin * (high - low + 1) / bins,
                   low + (bin + 1) * (high - low + 1) / bins - 1);
            for (star = 0; star < counts[bin]; star++) putchar('#');
            printf(" %d\n", counts[bin]);
        }
        free(counts);
    }
    return 0;
}
```

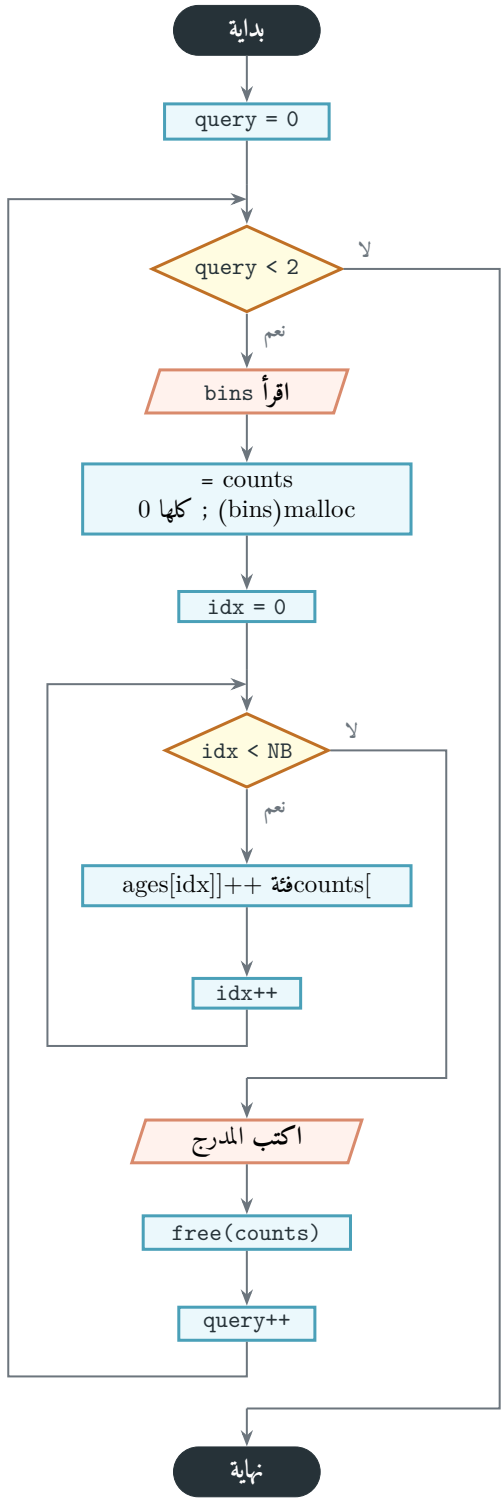
المدخلات

```
4 7
```

نتيجة التنفيذ

```
4 bins:
  18-29 |######## 8
  30-42 |###### 6
  43-54 |### 3
  55-67 |### 3
7 bins:
  18-24 |##### 5
  25-31 |#### 4
  32-38 |#### 4
  39-45 |## 2
  46-52 |## 2
  53-59 |# 1
  60-67 |## 2
```

المخطط الإنسيابي (Organigramme)



تمرين 10.40 إدراج اسم في قائمة مرتبة ديناميكية

تجارة إلكترونية

متوسط ★★

قائمة أسماء العلامات التجارية جدول ديناميكي من المؤشرات `char **brands` مرتب أبجديًا. اكتب دالة `insertSorted(&brands, &count, name)` تحجز جدولًا أكبر بخانة، وتنسخ المؤشرات مع إدراج الاسم الجديد (نسخة محجوزة) في مكانه الصحيح.

خطوات الحل

1) الدالة تغير المؤشر brands في main، لذلك تستقبل char ***list.
2) نجد موضع الإدراج بالمقارنة الأبجدية، ثم ننسخ المؤشرات قبله وبعده.
3) نحرر الجدول القديم (المؤشرات فقط، لا الأسماء).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int compareText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first - *second;
}

char *copyText(const char *text) {
    int len = 0, pos;
    char *copy;
    while (text[len] != '\0') len++;
    copy = malloc(len + 1);
    for (pos = 0; pos <= len; pos++) copy[pos] = text[pos];
    return copy;
}

void insertSorted(char ***list, int *count, const char *name) {
    char **bigger = malloc((*count + 1) * sizeof(char *));
    int pos = 0, idx;
    while (pos < *count && compareText((*list)[pos], name) < 0) pos++;   // موضع الإدراج
    for (idx = 0; idx < pos; idx++) bigger[idx] = (*list)[idx];
    bigger[pos] = copyText(name);
    for (idx = pos; idx < *count; idx++) bigger[idx + 1] = (*list)[idx];
    free(*list);                                  // الجدول القديم (المؤشرات فقط)
    *list = bigger;
    (*count)++;
}

int main(void) {
    char **brands = NULL, name[30];
    int count = 0, nb, idx;
    scanf("%d", &nb);
    for (idx = 0; idx < nb; idx++) {
        scanf("%29s", name);
        insertSorted(&brands, &count, name);
        printf("after %-8s:", name);
        for (int show = 0; show < count; show++) printf(" %s", brands[show]);
        printf("\n");
    }
    for (idx = 0; idx < count; idx++) free(brands[idx]);
    free(brands);
    return 0;
}
```

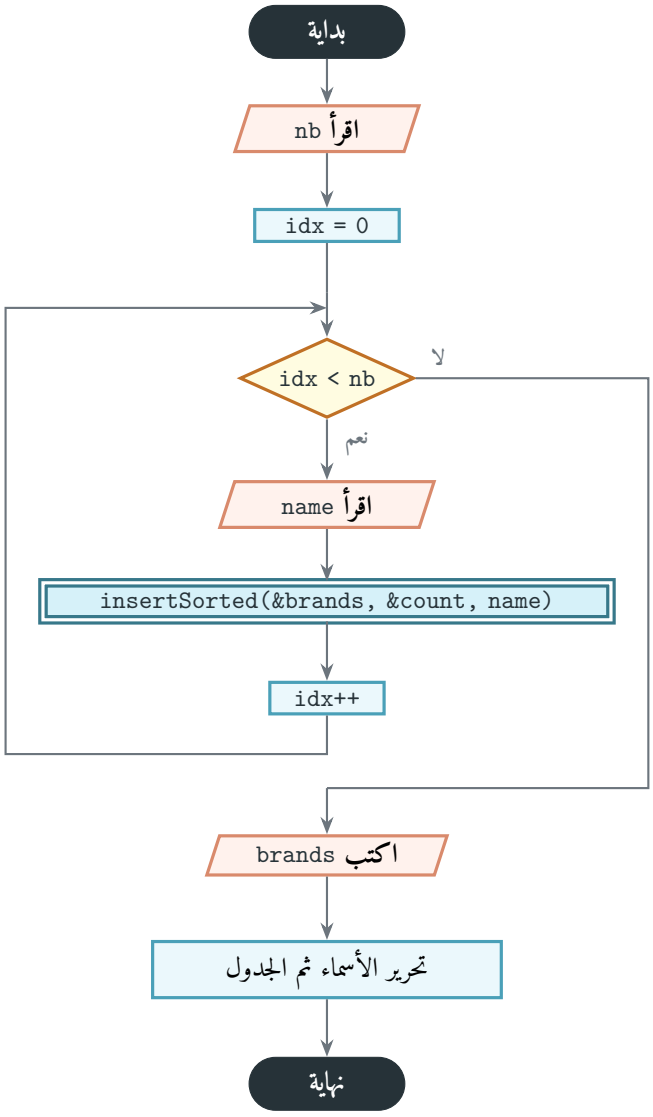
المدخلات

```
4
Samsung
Apple
Xiaomi
Lenovo
```

نتيجة التنفيذ

```
after Samsung : Samsung
after Apple   : Apple Samsung
after Xiaomi  : Apple Samsung Xiaomi
after Lenovo  : Apple Lenovo Samsung Xiaomi
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 10.41 الجدول الديناميكي الكامل

تجارة إلكترونية ★★★ صعب

اكتب نوعًا `Vector` كامل الوظائف لسجل الطلبيات: `vecInit`، `vecPush` (إضافة في النهاية مع مضاعفة السعة)، `vecInsert(vec, pos, value)` (إدراج مع إزاحة)، `vecRemove(vec, pos)`، و `vecFree`. عدّ عدد العناصر المنسوخة الكلي لإظهار أن تكلفة الإضافة **مستهلكة ثابتة**.

خطوات الحل

1) `grow` تحجز ضعف السعة، تنسخ، وتحرر القديم.
2) الإدراج: تأكد من السعة، نزيح `[pos, size)` بخانة يمينًا من النهاية، ثم نكتب.
3) الحذف: نزيح يسارًا وننقص الحجم.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { int *data; int size, capacity; long copies; } Vector;

void vecInit(Vector *vec, int capacity) {
    vec->data = malloc(capacity * sizeof(int));
    vec->size = 0;
    vec->capacity = capacity;
    vec->copies = 0;
}

void grow(Vector *vec) {
    int *bigger = malloc(2 * vec->capacity * sizeof(int)), idx;
    for (idx = 0; idx < vec->size; idx++) bigger[idx] = vec->data[idx];
    vec->copies += vec->size;
    free(vec->data);
    vec->data = bigger;
    vec->capacity *= 2;                                  // مضاعفة
}

void vecPush(Vector *vec, int value) {
    if (vec->size == vec->capacity) grow(vec);
    vec->data[vec->size++] = value;
}

void vecInsert(Vector *vec, int pos, int value) {
    int idx;
    if (vec->size == vec->capacity) grow(vec);
    for (idx = vec->size; idx > pos; idx--) vec->data[idx] = vec->data[idx - 1];   // إزاحة يمينًا
    vec->data[pos] = value;
    vec->size++;
}

void vecRemove(Vector *vec, int pos) {
    int idx;
    for (idx = pos; idx < vec->size - 1; idx++) vec->data[idx] = vec->data[idx + 1];
    vec->size--;
}

void vecFree(Vector *vec) { free(vec->data); vec->data = NULL; vec->size = vec->capacity = 0; }

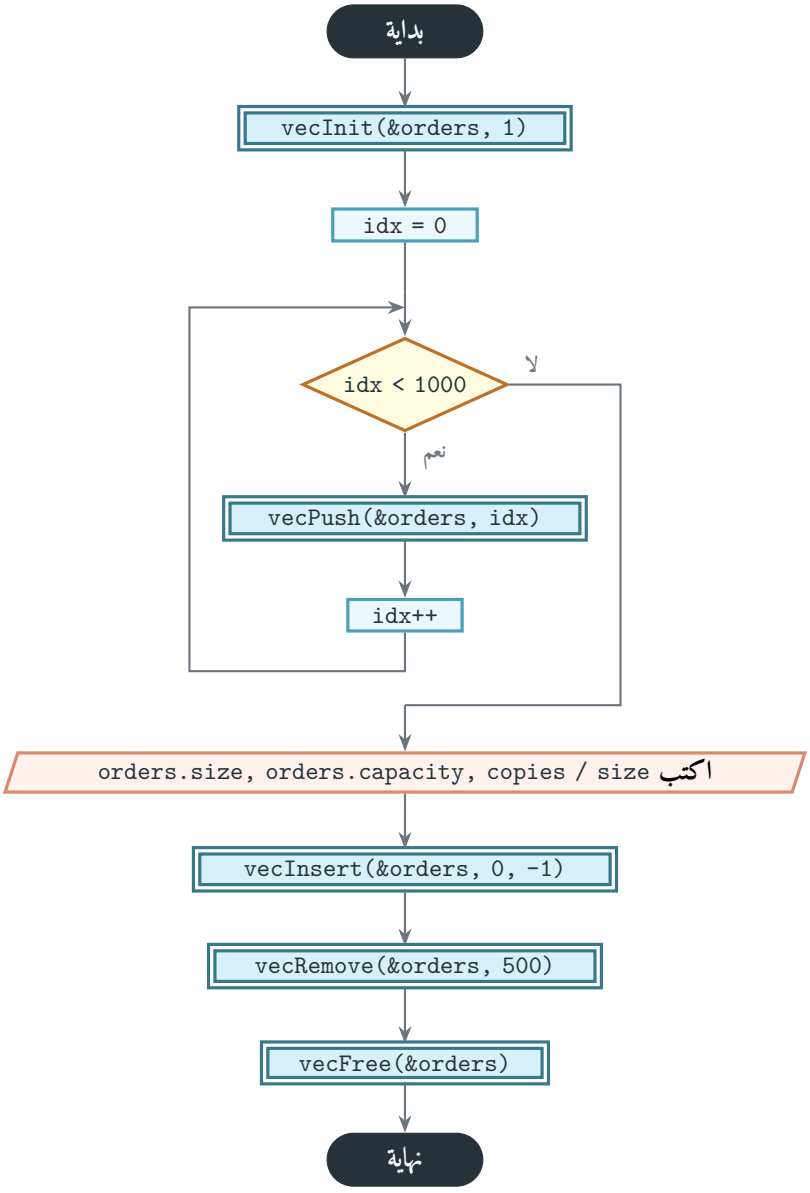
```

```c
int main(void) {
    Vector orders;
    int idx;
    vecInit(&orders, 1);
    for (idx = 0; idx < 1000; idx++) vecPush(&orders, 7000 + idx);
    printf("size %d, capacity %d\n", orders.size, orders.capacity);
    printf("copies %ld -> %.2f per push (amortized constant)\n", orders.copies,
           (double)orders.copies / orders.size);
    vecInsert(&orders, 0, 6999);
    vecRemove(&orders, 500);
    printf("first %d, at 500: %d, last %d, size %d\n", orders.data[0], orders.data[500],
           orders.data[orders.size - 1], orders.size);
    vecFree(&orders);
    return 0;
}
```

نتيجة التنفيذ

```
size 1000, capacity 1024
copies 1023 -> 1.02 per push (amortized constant)
first 6999, at 500: 7500, last 7999, size 1000
```

المخطط الإنسيابي (Organigramme)



تمرين 10.42 مصفوفة التقييمات المتناثرة CSR

علم البيانات صعب ★★★

أغلب خانات مصفوفة (زبون × منتج) فارغة. صيغة **CSR** تخزن فقط القيم غير المعدومة في ثلاثة جداول ديناميكية: `values`، `columns`، و `rowStart` (بداية كل سطر). ابنِ CSR من قائمة تقييمات، ثم احسب لزبون جديد **التشابه** (جداء سلمي) مع كل زبون، واقترح منتجًا من أقربهم.

خطوات الحل

1) نعد التقييمات لكل زبون، ثم

```
rowStart[row+1] = rowStart[row] + count[row]
```

.

2) نملأ `values` و `columns` في مواضعها.

3) الجداء السلمي لسطر `row`: Σ `values[idx] * query[columns[idx]]` لـ `idx` من `rowStart[row]` إلى `rowStart[row+1]`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBC 5
#define NBP 8
#define NBR 14

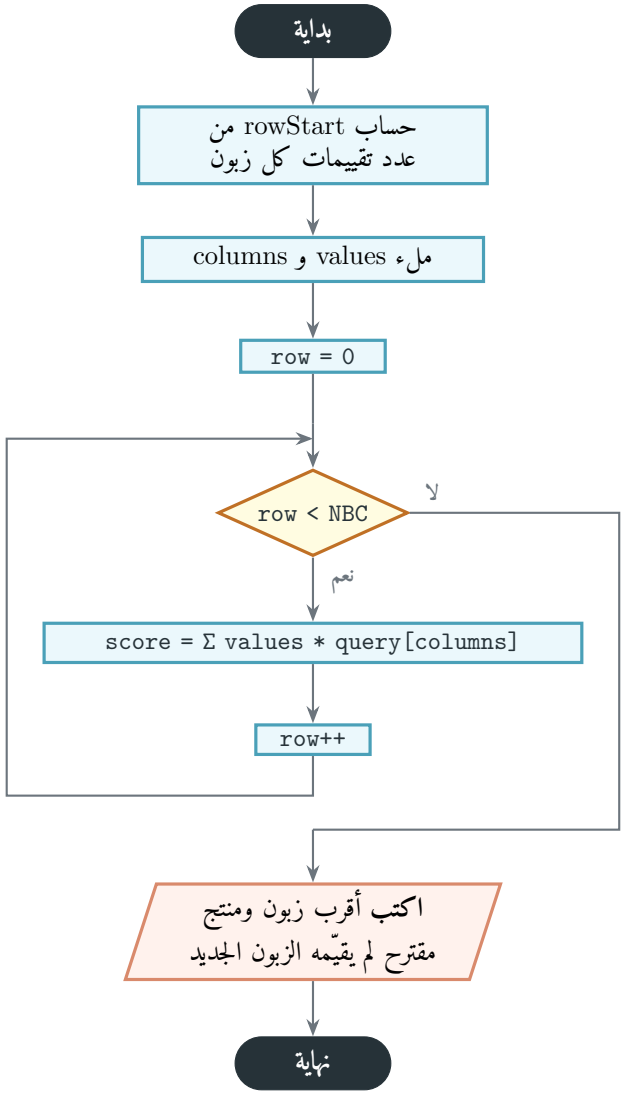
int main(void) {
    int ratings[NBR][3] = {{0, 1, 5}, {0, 4, 3}, {1, 0, 4}, {1, 2, 5}, {1, 7, 2}, {2, 1, 4},
                           {2, 4, 5}, {2, 6, 4}, {3, 3, 5}, {3, 5, 4}, {4, 0, 3}, {4, 2, 4},
                           {4, 7, 5}, {0, 6, 2}};          // (زبون، منتج، نجوم)
    int query[NBP] = {0, 5, 0, 0, 4, 0, 0, 0};             // الزبون الجديد
    int *rowStart = malloc((NBC + 1) * sizeof(int)), *fill = malloc(NBC * sizeof(int));
    int *columns = malloc(NBR * sizeof(int)), *values = malloc(NBR * sizeof(int));
    int row, idx, pos, best = 0, score, bestScore = -1, suggestion = -1, top = 0;
    for (row = 0; row <= NBC; row++) rowStart[row] = 0;
    for (idx = 0; idx < NBR; idx++) rowStart[ratings[idx][0] + 1]++;
    for (row = 0; row < NBC; row++) rowStart[row + 1] += rowStart[row];   // مجاميع تراكمية
    for (row = 0; row < NBC; row++) fill[row] = rowStart[row];
    for (idx = 0; idx < NBR; idx++) {
        pos = fill[ratings[idx][0]]++;
        columns[pos] = ratings[idx][1];
        values[pos] = ratings[idx][2];
    }
    printf("Stored %d of %d cells (%.0f%%)\n", NBR, NBC * NBP, 100.0 * NBR / (NBC * NBP));
    for (row = 0; row < NBC; row++) {
        score = 0;
        for (idx = rowStart[row]; idx < rowStart[row + 1]; idx++)
            score += values[idx] * query[columns[idx]];     // جداء سلمي متناثر
        printf("customer %d: similarity %d\n", row, score);
        if (score > bestScore) { bestScore = score; best = row; }
    }
    for (idx = rowStart[best]; idx < rowStart[best + 1]; idx++)
        if (query[columns[idx]] == 0 && values[idx] > top) { top = values[idx]; suggestion = columns[idx]; }
    printf("Nearest customer %d -> recommend product %d (%d stars)\n", best, suggestion, top);
    free(rowStart); free(fill); free(columns); free(values);
    return 0;
}
```

نتيجة التنفيذ

```
Stored 14 of 40 cells (35%)
customer 0: similarity 37
```

```
customer 1: similarity 0
customer 2: similarity 40
customer 3: similarity 0
customer 4: similarity 0
Nearest customer 2 -> recommend product 6 (4 stars)
```

المخطط الإنسيابي (Organigramme)



تمرين 10.43 منشئ النصوص لتوليد الفاتورة

تجارة إلكترونية صعب

لتوليد نص الفاتورة سطراً بسطر دون معرفة طوله مسبقًا، اكتب نوع Builder (مؤشر، طول، سعة) ودوال appendText و appendNumber (تحويل عدد صحيح إلى نص يدويًا) و appendChar، مع مضاعفة السعة عند الحاجة. ابنِ الفاتورة ثم اطبعها مرة واحدة.

خطوات الحل

1) كل إضافة تتأكد أولًا من السعة: ensure(builder, extra).
2) appendNumber: نستخرج الأرقام في جدول مؤقت بالترتيب المعكوس ثم نضيفها.

3) النص يبقى منتهيًا بـ `'\0'` بعد كل إضافة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { char *text; int length, capacity, grows; } Builder;

void init(Builder *builder, int capacity) {
    builder->text = malloc(capacity);
    builder->text[0] = '\0';
    builder->length = 0;
    builder->capacity = capacity;
    builder->grows = 0;
}

void ensure(Builder *builder, int extra) {
    char *bigger;
    int idx;
    while (builder->length + extra + 1 > builder->capacity) {
        bigger = malloc(2 * builder->capacity);
        for (idx = 0; idx <= builder->length; idx++) bigger[idx] = builder->text[idx];
        free(builder->text);
        builder->text = bigger;
        builder->capacity *= 2;
        builder->grows++;
    }
}

void appendChar(Builder *builder, char ch) {
    ensure(builder, 1);
    builder->text[builder->length++] = ch;
    builder->text[builder->length] = '\0';
}

void appendText(Builder *builder, const char *text) {
    while (*text != '\0') appendChar(builder, *text++);
}

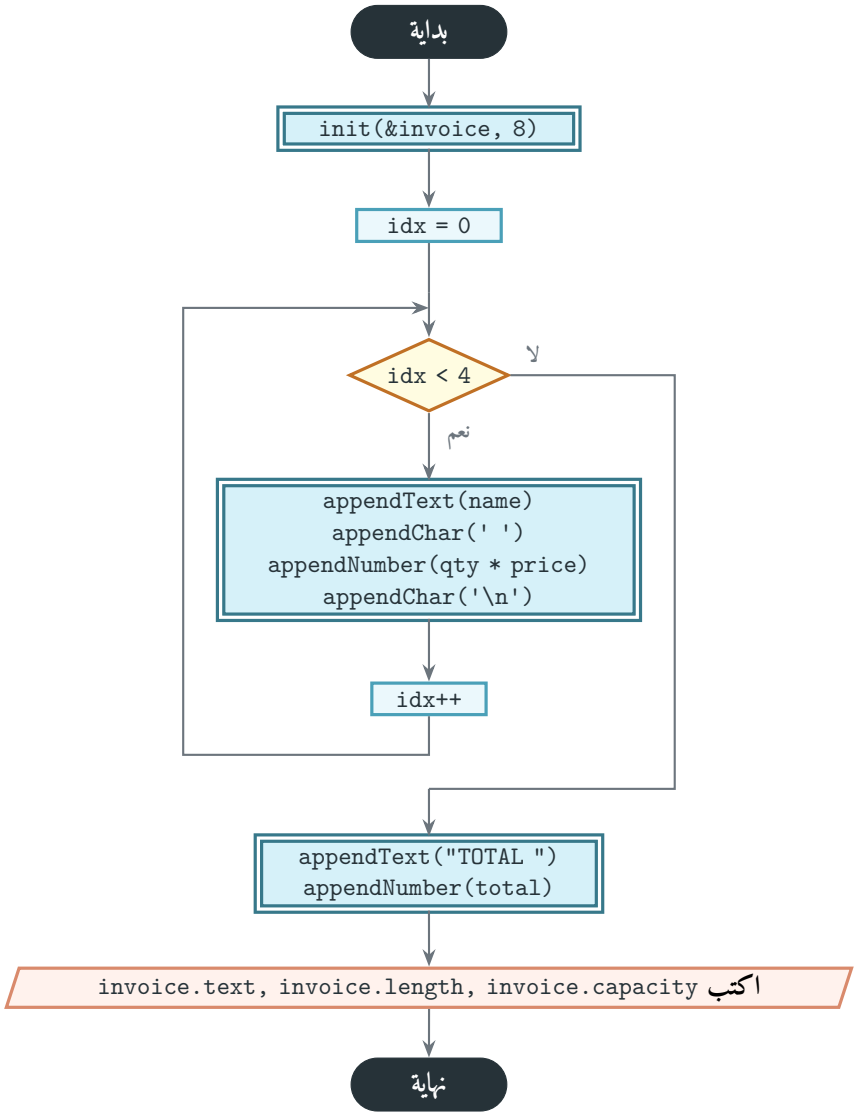
void appendNumber(Builder *builder, long value) {
    char digits[20];
    int count = 0;
    if (value == 0) digits[count++] = '0';
    while (value > 0) { digits[count++] = (char)('0' + value % 10); value /= 10; }   // معكوس
    while (count > 0) appendChar(builder, digits[--count]);
}

int main(void) {
    const char *names[4] = {"Mouse      x2", "Keyboard   x1", "Monitor    x1", "USB cable  x5"};
    long amounts[4] = {3000, 3200, 32000, 2250}, total = 0;
    Builder invoice;
    int idx;
    init(&invoice, 8);
    appendText(&invoice, "INVOICE #7781\n");
    for (idx = 0; idx < 4; idx++) {
        appendText(&invoice, names[idx]);
        appendText(&invoice, "  ");
        appendNumber(&invoice, amounts[idx]);
        appendText(&invoice, " DA\n");
        total += amounts[idx];
    }
    appendText(&invoice, "TOTAL ");
    appendNumber(&invoice, total);
    appendText(&invoice, " DA\n");
    printf("%s", invoice.text);
    printf("[%d chars, capacity %d, %d grows]\n", invoice.length, invoice.capacity, invoice.grows);
    free(invoice.text);
    return 0;
}
```

نتيجة التنفيذ

```
INVOICE #7781
Mouse       x2  3000 DA
Keyboard    x1  3200 DA
Monitor     x1  32000 DA
USB cable   x5  2250 DA
TOTAL 40450 DA
[122 chars, capacity 128, 4 grows]
```

المخطط الإنسيابي (Organigramme)



تمرين 10.44 جدول تجزئة بالسلاسل لعد الكلمات

علم البيانات ★★★ صعب

لعدّ تكرار الكلمات في التعليقات بسرعة، اكتب **جدول تجزئة بالسلاسل**: كل خانة رأس لقائمة عقد محجوزة بـ `malloc` (الكلمة، العداد، المؤشر على التالية). أدرج كل كلمة (أو زد عدادها)، ثم اعرض الكلمات التي ظهرت أكثر من مرة وطول أطول سلسلة، وحرر كل العقد.

خطوات الحل

1) `hashWord` (djb2) ثم `% SIZE` لتحديد الخانة.
2) البحث في القائمة؛ إذا لم توجد الكلمة نحجز عقدة جديدة ونضعها في رأس القائمة.
3) التحرير: لكل خانة نمر على القائمة ونحرر العقدة بعد حفظ التالية.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define SIZE 13

typedef struct Node { char word[16]; int count; struct Node *next; } Node;

unsigned long hashWord(const char *word) {
    unsigned long hash = 5381;
    while (*word != '\0') hash = hash * 33 + (unsigned char)*word++;
    return hash;
}

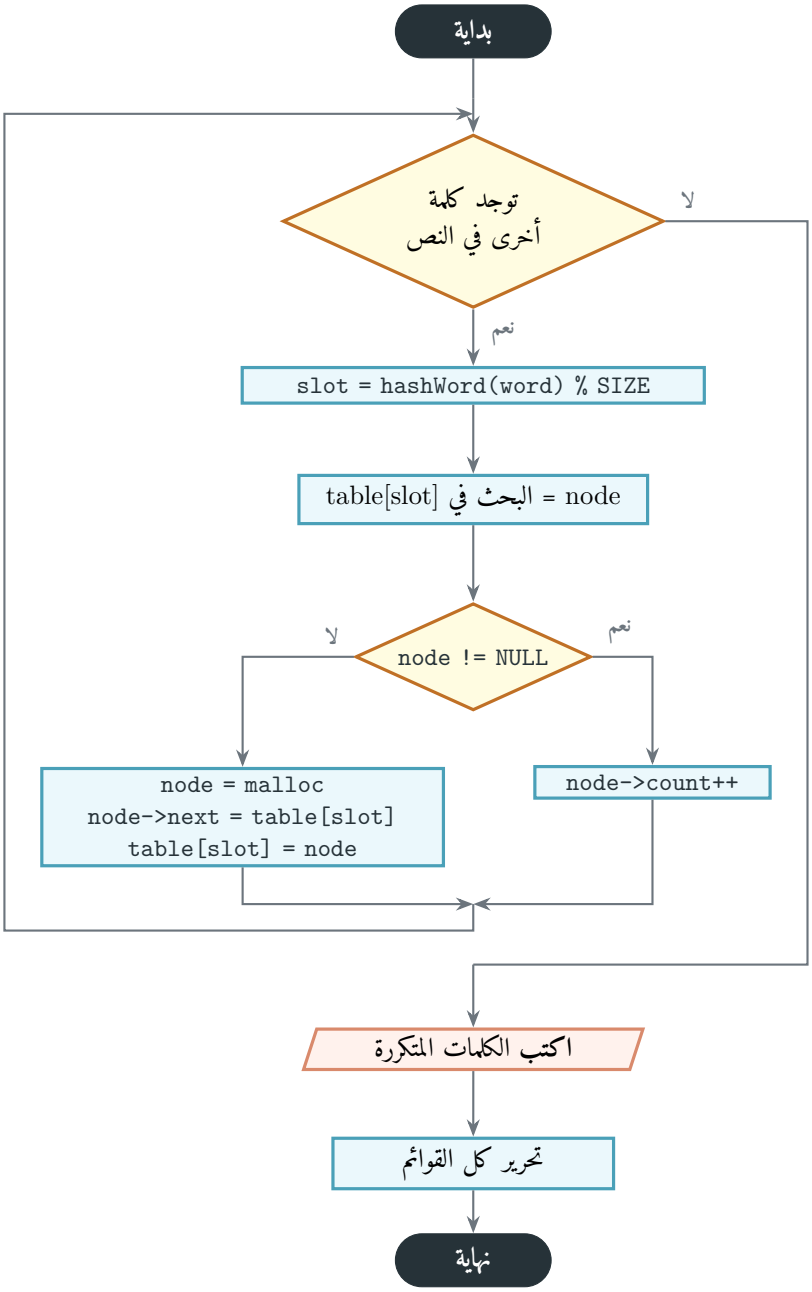
int sameWord(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

int main(void) {
    char text[] = "fast delivery good price fast service good quality fast refund "
                  "bad packaging good support price ok";
    Node *table[SIZE] = {NULL}, *node, *next;
    char word[16];
    int pos = 0, len, slot, chain, longest = 0, distinct = 0;
    while (text[pos] != '\0') {
        while (text[pos] == ' ') pos++;
        for (len = 0; text[pos] != ' ' && text[pos] != '\0' && len < 15; pos++) word[len++] = text[pos];
        word[len] = '\0';
        if (len == 0) continue;
        slot = (int)(hashWord(word) % SIZE);
        for (node = table[slot]; node != NULL && !sameWord(node->word, word); node = node->next) {}
        if (node != NULL) {
            node->count++;
        } else {
            node = malloc(sizeof(Node));                    // عقدة جديدة في رأس السلسلة
            for (len = 0; (node->word[len] = word[len]) != '\0'; len++) {}
            node->count = 1;
            node->next = table[slot];
            table[slot] = node;
            distinct++;
        }
    }
    printf("%d distinct words. Repeated:", distinct);
    for (slot = 0; slot < SIZE; slot++) {
        chain = 0;
        for (node = table[slot]; node != NULL; node = node->next) {
            chain++;
            if (node->count > 1) printf(" %s(%d)", node->word, node->count);
        }
        if (chain > longest) longest = chain;
    }
    printf("\nLongest chain: %d\n", longest);
    for (slot = 0; slot < SIZE; slot++)
        for (node = table[slot]; node != NULL; node = next) {
            next = node->next;                              // نحفظ التالي قبل التحرير
            free(node);
        }
    return 0;
}
```

نتيجة التنفيذ

```
11 distinct words. Repeated: fast(3) price(2) good(3)
Longest chain: 3
```

المخطط الانسيابي (Organigramme)



تمرين 10.45 الترتيب بالكومة لمؤشرات المنتجات

تجارة إلكترونية صعب ★★★

رتّب جدولًا **ديناميكيًا من المؤشرات** على هياكل منتجات حسب الإيراد (السعر × المبيعات) بخوارزمية **الترتيب بالكومة** (heap sort): نبني كومة عظمى في المكان، ثم ننقل الجذر إلى النهاية ونعيد ترميم الكومة. التعقيد `n log n` دون ذاكرة إضافية.

خطوات الحل

1) `siftDown(view, root, size)`: ننزل العنصر ما دام أحد أبنائه أكبر منه.

2) البناء: `siftDown` لكل عقدة داخلية من `size/2 - 1` إلى 0.

3) الاستخراج: نبدل `view[0]` مع `view[end]`، ثم `siftDown(view, 0, end)`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NB 8

typedef struct { char name[12]; double price; int sold; } Product;

double revenue(const Product *item) { return item->price * item->sold; }

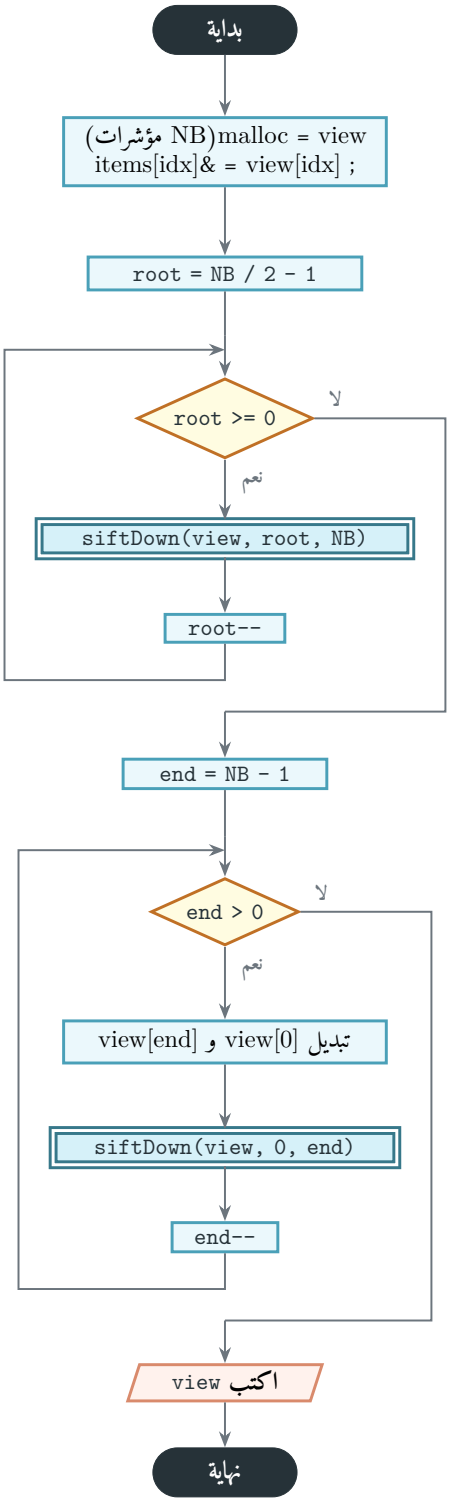
void siftDown(Product **view, int root, int size) {
    int child;
    Product *tmp;
    while (2 * root + 1 < size) {
        child = 2 * root + 1;
        if (child + 1 < size && revenue(view[child + 1]) > revenue(view[child])) child++;
        if (revenue(view[root]) >= revenue(view[child])) return;
        tmp = view[root]; view[root] = view[child]; view[child] = tmp;   // ننزل
        root = child;
    }
}

int main(void) {
    Product items[NB] = {{"Phone", 32000, 40}, {"Case", 900, 310}, {"Laptop", 89000, 12},
                         {"Earbuds", 5400, 95}, {"Charger", 2200, 150}, {"Watch", 15000, 28},
                         {"Cable", 450, 600}, {"Tablet", 41000, 18}};
    Product **view = malloc(NB * sizeof(Product *)), *tmp;
    int idx, root, end;
    for (idx = 0; idx < NB; idx++) view[idx] = &items[idx];
    for (root = NB / 2 - 1; root >= 0; root--) siftDown(view, root, NB);   // بناء الكومة
    printf("Top revenue: %s\n", view[0]->name);
    for (end = NB - 1; end > 0; end--) {
        tmp = view[0]; view[0] = view[end]; view[end] = tmp;            // الأكبر إلى النهاية
        siftDown(view, 0, end);
    }
    for (idx = NB - 1; idx >= 0; idx--)
        printf("%-8s %10.0f DA\n", view[idx]->name, revenue(view[idx]));
    free(view);
    return 0;
}
```

نتيجة التنفيذ

```
Top revenue: Phone
Phone       1280000 DA
Laptop      1068000 DA
Tablet       738000 DA
Earbuds      513000 DA
Watch        420000 DA
Charger      330000 DA
Case         279000 DA
Cable        270000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 10.46 مصفوفة التشابه بين المنتجات

علم البيانات ★★★ صعب

لاقتراح "منتجات مشابهة"، احسب **تشابه جيب التمام** بين كل زوج من المنتجات من مصفوفة المشتريات (زبون × منتج، 0 أو عدد المرات). احجز مصفوفة التشابه `NBP × NBP` ديناميكيًا في كتلة واحدة، واستغل **التناظر** لحساب النصف فقط، ثم اعرض لكل منتج أقرب منتج إليه.

خطوات الحل

1) `cos(a, b) = Σ buy[c][a]·buy[c][b] / (‖a‖·‖b‖)` .
2) نحسب الخانة `[a][b]` لـ `b > a` فقط ثم ننسخها في `[b][a]`.
3) الجذر بطريقة نيوتن؛ القطر = 1.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBC 6
#define NBP 5

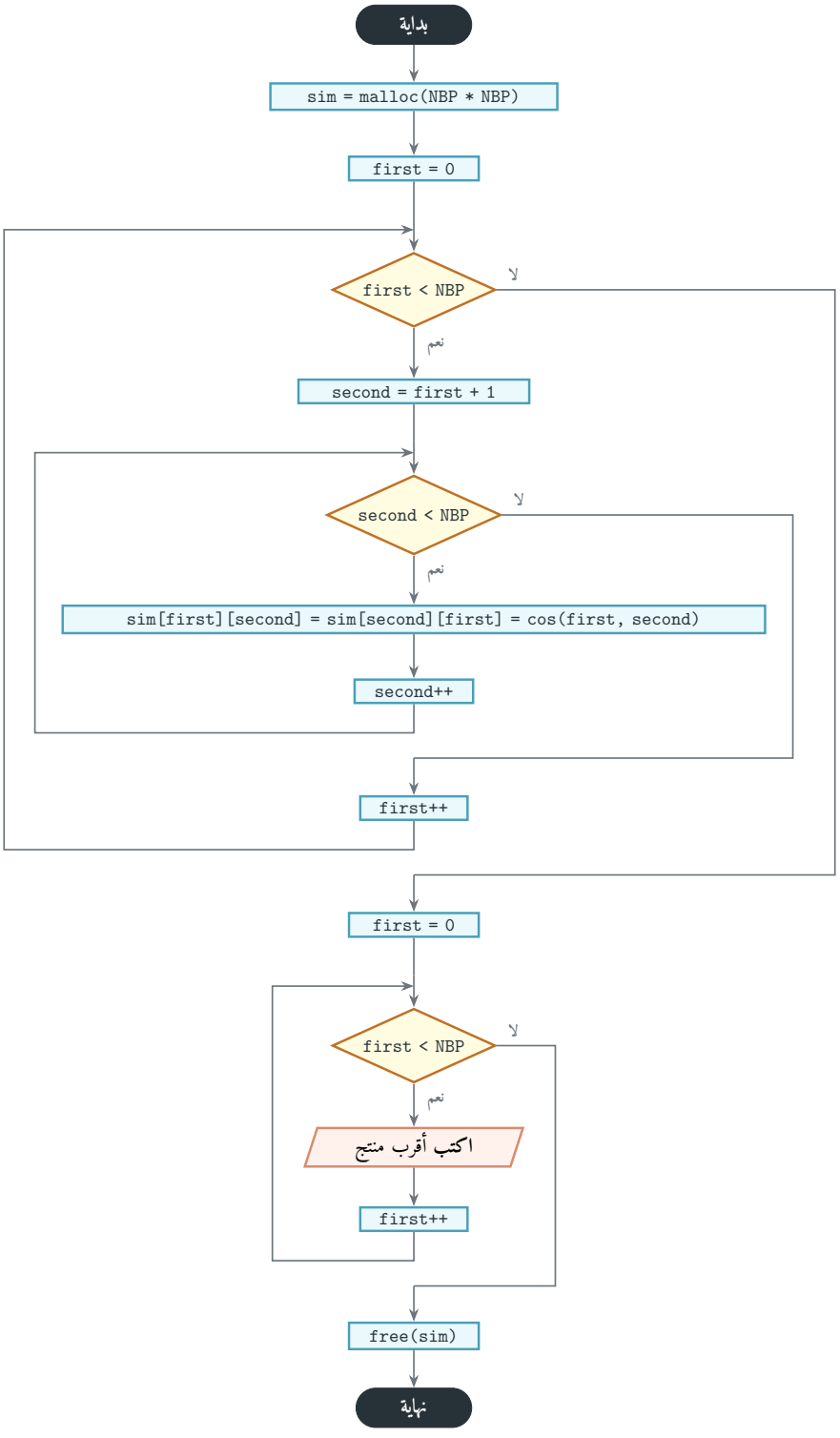
double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    if (value <= 0) return 0;
    for (iter = 0; iter < 50; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

int main(void) {
    int buy[NBC][NBP] = {{2, 1, 0, 0, 1}, {1, 1, 0, 1, 0}, {0, 0, 3, 1, 0},
                         {0, 1, 2, 2, 0}, {3, 2, 0, 0, 1}, {0, 0, 1, 2, 1}};
    char names[NBP][10] = {"Phone", "Case", "Laptop", "Mouse", "Charger"};
    double *sim = malloc(NBP * NBP * sizeof(double)), dot, normA, normB;
    int first, second, client, best;
    for (first = 0; first < NBP; first++) {
        sim[first * NBP + first] = 1;
        for (second = first + 1; second < NBP; second++) {
            dot = normA = normB = 0;
            for (client = 0; client < NBC; client++) {
                dot += buy[client][first] * buy[client][second];
                normA += buy[client][first] * buy[client][first];
                normB += buy[client][second] * buy[client][second];
            }
            sim[first * NBP + second] = sim[second * NBP + first] = dot / (mySqrt(normA) * mySqrt(normB));
        }
    }
    for (first = 0; first < NBP; first++) {
        best = first == 0 ? 1 : 0;
        for (second = 0; second < NBP; second++)
            if (second != first && sim[first * NBP + second] > sim[first * NBP + best]) best = second;
        printf("%-8s ->", names[first]);
        for (second = 0; second < NBP; second++) printf(" %.2f", sim[first * NBP + second]);
        printf("  similar: %s\n", names[best]);
    }
    free(sim);
    return 0;
}
```

نتيجة التنفيذ

```
Phone    -> 1.00 0.91 0.00 0.08 0.77  similar: Case
Case     -> 0.91 1.00 0.20 0.36 0.65  similar: Phone
Laptop   -> 0.00 0.20 1.00 0.76 0.15  similar: Mouse
Mouse    -> 0.08 0.36 0.76 1.00 0.37  similar: Laptop
Charger  -> 0.77 0.65 0.15 0.37 1.00  similar: Phone
```

المخطط الإنسيابي (Organigramme)



تمرين 10.47 أطول تسلسل مشترك بين سجلي تصفح

علم البيانات

صعب

سجل تصفح زبون هو تسلسل من رموز الصفحات. لقياس تشابه سلوك زبونين، اقرأ التسلسلين بطولين يحددهما المدخل، واحجز جدول البرمجة الديناميكية **بالحجم المضبوط** `(lenA+1) × (lenB+1)` في كتلة واحدة، واحسب **LCS** ثم أعد بناء التسلسل المشترك.

خطوات الحل

1) الخانة `(row, col)` في الموضع `row * (lenB + 1) + col`.
2) نفس معادلة LCS: تطابق ← قطري + 1، وإلا الأكبر من الأعلى واليسار.
3) نحرر الجداول الثلاثة في النهاية.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int lenA, lenB, row, col, width, size;
    char *first, *second, *common;
    int *table;
    scanf("%d", &lenA);
    first = malloc(lenA + 1);
    scanf("%s", first);
    scanf("%d", &lenB);
    second = malloc(lenB + 1);
    scanf("%s", second);
    width = lenB + 1;
    table = malloc((lenA + 1) * width * sizeof(int));        // جدول بالحجم المضبوط
    for (row = 0; row <= lenA; row++)
        for (col = 0; col <= lenB; col++) {
            if (row == 0 || col == 0) table[row * width + col] = 0;
            else if (first[row - 1] == second[col - 1])
                table[row * width + col] = table[(row - 1) * width + col - 1] + 1;
            else
                table[row * width + col] = table[(row - 1) * width + col] > table[row * width + col - 1]
                                         ? table[(row - 1) * width + col] : table[row * width + col - 1];
        }
    size = table[lenA * width + lenB];
    common = malloc(size + 1);
    common[size] = '\0';
    row = lenA; col = lenB;
    while (row > 0 && col > 0) {
        if (first[row - 1] == second[col - 1]) { common[--size] = first[row - 1]; row--; col--; }
        else if (table[(row - 1) * width + col] >= table[row * width + col - 1]) row--;
        else col--;
    }
    printf("A: %s\nB: %s\nCommon path: %s (%d pages)\n", first, second, common,
           table[lenA * width + lenB]);
    printf("Similarity: %.2f\n", 2.0 * table[lenA * width + lenB] / (lenA + lenB));
    free(first); free(second); free(common); free(table);
    return 0;
}
```

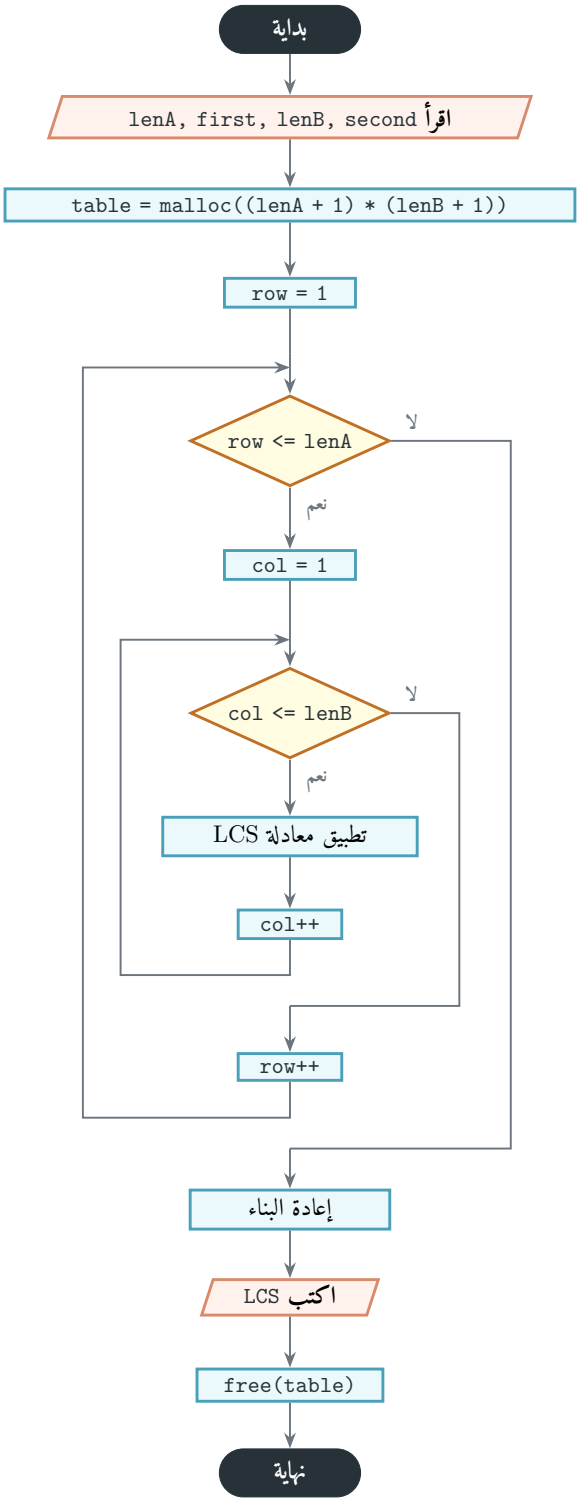
المدخلات

```
9 HPCPKCOPT
8 HSPCKPOT
```

نتيجة التنفيذ

```
A: HPCPKCOPT
B: HSPCKPOT
Common path: HPCPOT (6 pages)
Similarity: 0.71
```

المخطط الإنسيابي (Organigramme)



تمرين 10.48 الترتيب بالأساس لأرقام الطلبيات

تجارة إلكترونية

صعب ★★★

أرقام الطلبيات أعداد صحيحة من 6 أرقام. **الترتيب بالأساس** (radix sort) يرتبها دون مقارنات: نرتب حسب رقم الآحاد، ثم العشرات، ثم المئات... بترتيب **مستقر** بالعد في كل مرحلة. احجز جدول الإخراج ديناميكيًا وبدّل المؤشرين بين المراحل.

خطوات الحل

1) لكل رقم `digit = (value / place) % 10`: نعد التكرارات `count[10]`.
2) المواضع التراكمية تحدد مكان كل مجموعة؛ نكتب من النهاية للحفاظ على الاستقرار.
3) نبدل `source` و `target` (مؤشران) بدل النسخ.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

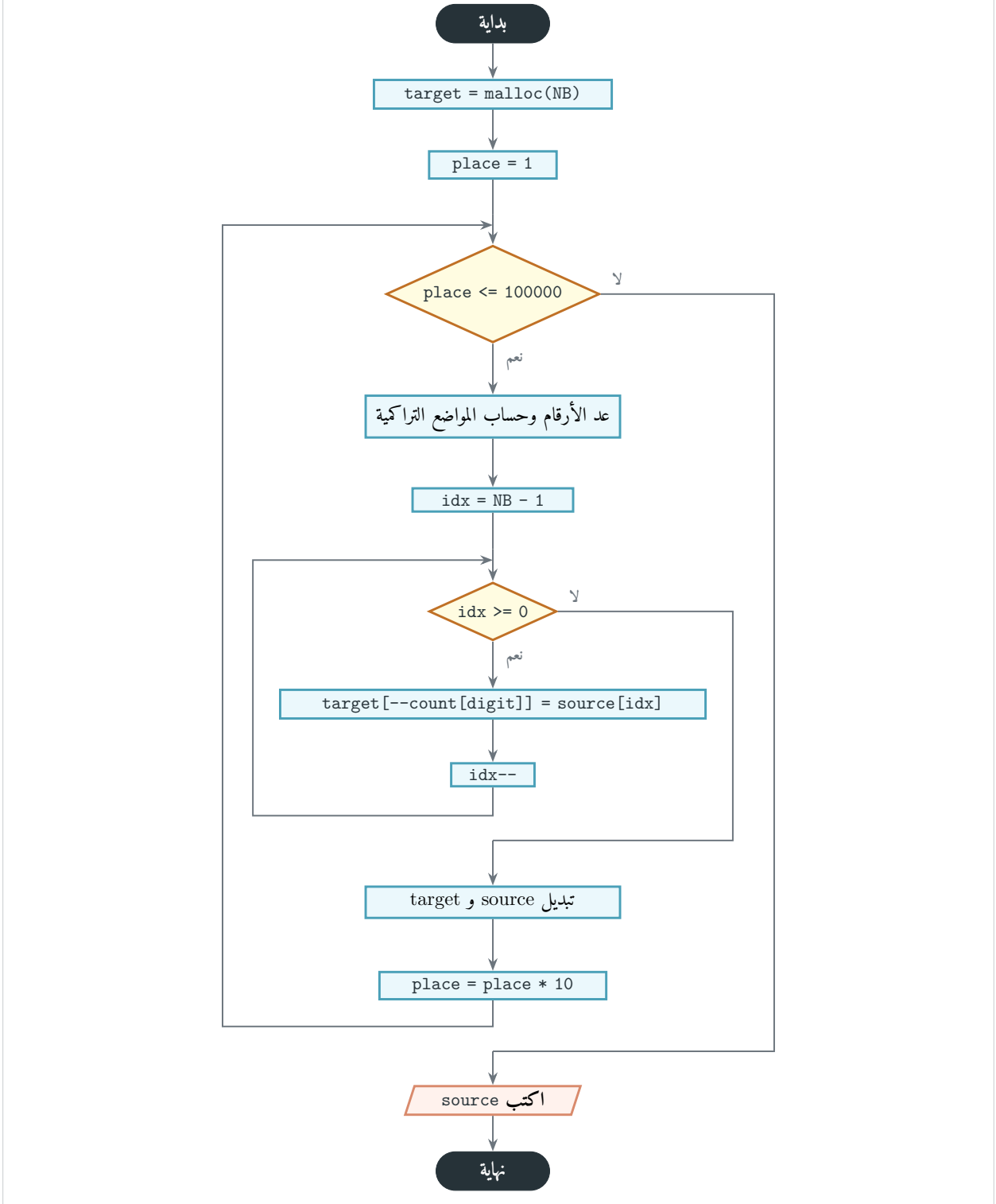
#define NB 10

int main(void) {
    long orders[NB] = {482019, 102345, 993201, 482001, 100045, 777310, 102344, 560120, 999999, 100000};
    long *source = orders, *target = malloc(NB * sizeof(long)), *tmp;
    long place;
    int count[10], idx, digit, pass = 0;
    for (place = 1; place <= 100000; place *= 10) {
        for (digit = 0; digit < 10; digit++) count[digit] = 0;
        for (idx = 0; idx < NB; idx++) count[(source[idx] / place) % 10]++;
        for (digit = 1; digit < 10; digit++) count[digit] += count[digit - 1];   // مواضع تراكمية
        for (idx = NB - 1; idx >= 0; idx--)                                      // من النهاية: مستقر
            target[--count[(source[idx] / place) % 10]] = source[idx];
        tmp = source; source = target; target = tmp;                             // تبديل المؤشرين
        pass++;
    }
    printf("%d passes, 0 comparisons\n", pass);
    for (idx = 0; idx < NB; idx++) printf("%ld ", source[idx]);
    printf("\n");
    free(source == orders ? target : source);       // نحرر الكتلة المحجوزة فقط
    return 0;
}
```

نتيجة التنفيذ

```
6 passes, 0 comparisons
100000 100045 102344 102345 482001 482019 560120 777310 993201 999999
```

المخطط الإنسيابي (Organigramme)



تمرين 10.49 محاكاة مدير الذاكرة بالملاءمة الأولى

علم البيانات صعب ★★★

كيف يعمل `malloc` من الداخل؟ حاكِ مديرًا بسيطًا على "ساحة" من 64 خانة: جدول كتل (البداية، الحجم، حرة أم لا). `allocate(size)` تختار **أول كتلة حرة كافية** وتقسمها، و `release(start)` تحرر الكتلة **وتدمجها** مع الكتل الحرة المجاورة. اعرض خريطة الذاكرة بعد كل عملية.

خطوات الحل

1) في البداية كتلة حرة واحدة `[0, 64)`.
2) التقسيم: الكتلة الحرة `size` تصبح مستعملة، والباقي كتلة حرة جديدة (إزاحة جدول الكتل).
3) الدمج: بعد التحرير نوحد كل كتلتين حرتين متتاليتين.
4) التجزئة: قد يوجد مجموع حر كافٍ دون كتلة واحدة كافية.

البرنامج بلغة C

```c
#include <stdio.h>

#define ARENA 64
#define MAXB 20

typedef struct { int start, size, isFree; } Block;

Block blocks[MAXB];
int nbBlocks = 1;

int allocate(int size) {
    int idx, move;
    for (idx = 0; idx < nbBlocks; idx++)
        if (blocks[idx].isFree && blocks[idx].size >= size) {          // الملاءمة الأولى
            if (blocks[idx].size > size) {                             // تقسيم
                for (move = nbBlocks; move > idx + 1; move--) blocks[move] = blocks[move - 1];
                blocks[idx + 1].start = blocks[idx].start + size;
                blocks[idx + 1].size = blocks[idx].size - size;
                blocks[idx + 1].isFree = 1;
                nbBlocks++;
            }
            blocks[idx].size = size;
            blocks[idx].isFree = 0;
            return blocks[idx].start;
        }
    return -1;                                                         // لا توجد كتلة كافية
}

void release(int start) {
    int idx, move;
    for (idx = 0; idx < nbBlocks; idx++) if (blocks[idx].start == start) blocks[idx].isFree = 1;
    for (idx = 0; idx < nbBlocks - 1; ) {
        if (blocks[idx].isFree && blocks[idx + 1].isFree) {            // دمج المتجاورتين
            blocks[idx].size += blocks[idx + 1].size;
            for (move = idx + 1; move < nbBlocks - 1; move++) blocks[move] = blocks[move + 1];
            nbBlocks--;
        } else {
            idx++;
        }
    }
}

void printMap(const char *label) {
    int idx, cell, freeTotal = 0;
    printf("%-12s|", label);
    for (idx = 0; idx < nbBlocks; idx++) {
        for (cell = 0; cell < blocks[idx].size; cell++) putchar(blocks[idx].isFree ? '.' : (idx % 2 ? '=' : '#'));   // رمزان بالتناوب لتمييز الكتل
        if (blocks[idx].isFree) freeTotal += blocks[idx].size;
    }
    printf("| free %d\n", freeTotal);
}

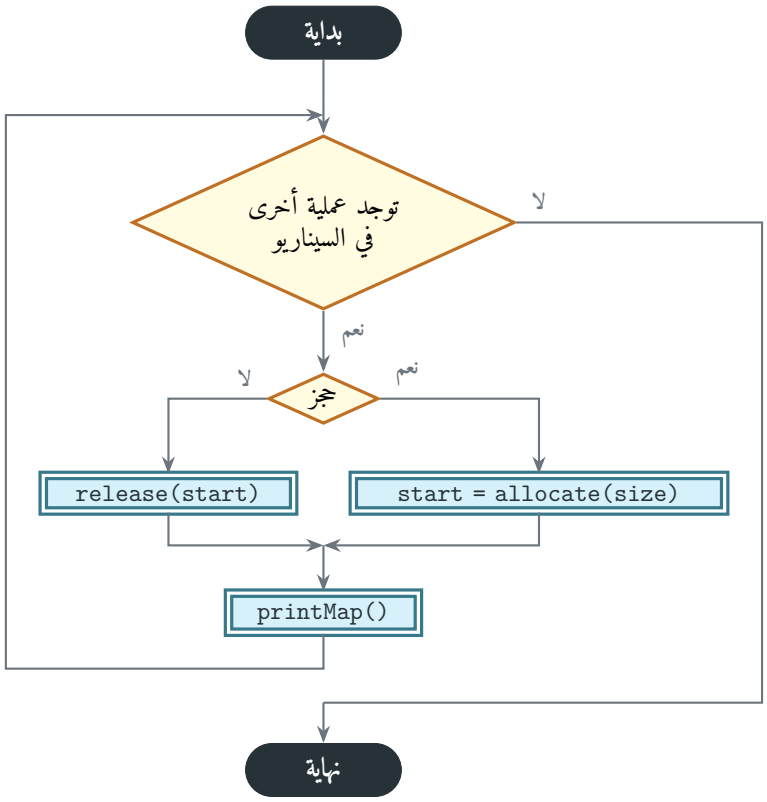
int main(void) {
    int cart, wish, logs, report;
    blocks[0].start = 0; blocks[0].size = ARENA; blocks[0].isFree = 1;
    cart = allocate(16);   printMap("alloc 16");
    wish = allocate(8);    printMap("alloc 8");
```

```
58      logs = allocate(20);   printMap("alloc 20");
59      release(wish);         printMap("free 8");
60      report = allocate(4);  printMap("alloc 4");
61      release(cart);         printMap("free 16");
62      printf("alloc 24 -> %d (40 cells free, but largest hole is 20)\n", allocate(24));
63      release(logs);         printMap("free 20");
64      printf("alloc 24 -> start %d\n", allocate(24));
65      printMap("final");
66      (void)report;
67      return 0;
68  }
```

نتيجة التنفيذ

```
alloc 16    |################..........................................| free 48
alloc 8     |################========..................................| free 40
alloc 20    |################========####################..............| free 20
free 8      |################........####################..............| free 28
alloc 4     |################====....====================..............| free 24
free 16     |................====....====================..............| free 40
alloc 24 -> -1 (40 cells free, but largest hole is 20)
free 20     |................====....................................| free 60
alloc 24 -> start 20
final       |................====########################..............| free 36
```

المخطط الإنسيابي (Organigramme)



تمرين 10.50 تجميع نقاط التوصيل بخوارزمية K-means

علم البيانات

صعب ★★★

نريد تقسيم نقاط التوصيل (إحداثيات ثنائية البعد، عددها يُقرأ) إلى groups منطقة لسائقين. احجز ديناميكيًا جداول النقاط والمراكز والإسنادات، وطبّق **K-means**: إسناد كل نقطة إلى أقرب مركز ثم تحديث المراكز، حتى الاستقرار. اعرض المراكز

وعدد النقاط ومجموع مربعات المسافات (SSE).

خطوات الحل

1) المراكز الابتدائية: أول groups نقاط مختلفة (للتبسيط).
2) مسافة مربعة لتفادي الجذر.
3) نتوقف عندما لا يتغير أي إسناد.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { double posX, posY; } Point;

double dist2(Point first, Point second) {
    return (first.posX - second.posX) * (first.posX - second.posX)
         + (first.posY - second.posY) * (first.posY - second.posY);
}

int main(void) {
    int nbPoints, groups, idx, group, best, changed, iterations = 0, *assign, *counts;
    Point *points, *centers, *sums;
    double sse = 0;
    scanf("%d %d", &nbPoints, &groups);
    points = malloc(nbPoints * sizeof(Point));
    assign = malloc(nbPoints * sizeof(int));
    centers = malloc(groups * sizeof(Point));
    sums = malloc(groups * sizeof(Point));
    counts = malloc(groups * sizeof(int));
    for (idx = 0; idx < nbPoints; idx++) { scanf("%lf %lf", &points[idx].posX, &points[idx].posY);
      ↪ assign[idx] = -1; }
    centers[0] = points[0]; centers[1] = points[3]; centers[2] = points[6];   // بداية متباعدة
    do {
        changed = 0;
        for (idx = 0; idx < nbPoints; idx++) {
            best = 0;
            for (group = 1; group < groups; group++)
                if (dist2(points[idx], centers[group]) < dist2(points[idx], centers[best])) best = group;
            if (best != assign[idx]) { assign[idx] = best; changed++; }
        }
        for (group = 0; group < groups; group++) { sums[group].posX = sums[group].posY = 0; counts[group] =
      ↪ 0; }
        for (idx = 0; idx < nbPoints; idx++) {
            sums[assign[idx]].posX += points[idx].posX;
            sums[assign[idx]].posY += points[idx].posY;
            counts[assign[idx]]++;
        }
        for (group = 0; group < groups; group++)
            if (counts[group] > 0) {
                centers[group].posX = sums[group].posX / counts[group];
                centers[group].posY = sums[group].posY / counts[group];
            }
        iterations++;
    } while (changed > 0);
    for (idx = 0; idx < nbPoints; idx++) sse += dist2(points[idx], centers[assign[idx]]);
    printf("Converged in %d iterations\n", iterations);
    for (group = 0; group < groups; group++)
        printf("Zone %d: center (%.2f, %.2f), %d stops\n", group + 1, centers[group].posX,
               centers[group].posY, counts[group]);
    printf("SSE = %.3f\n", sse);
    free(points); free(assign); free(centers); free(sums); free(counts);
    return 0;
}
```

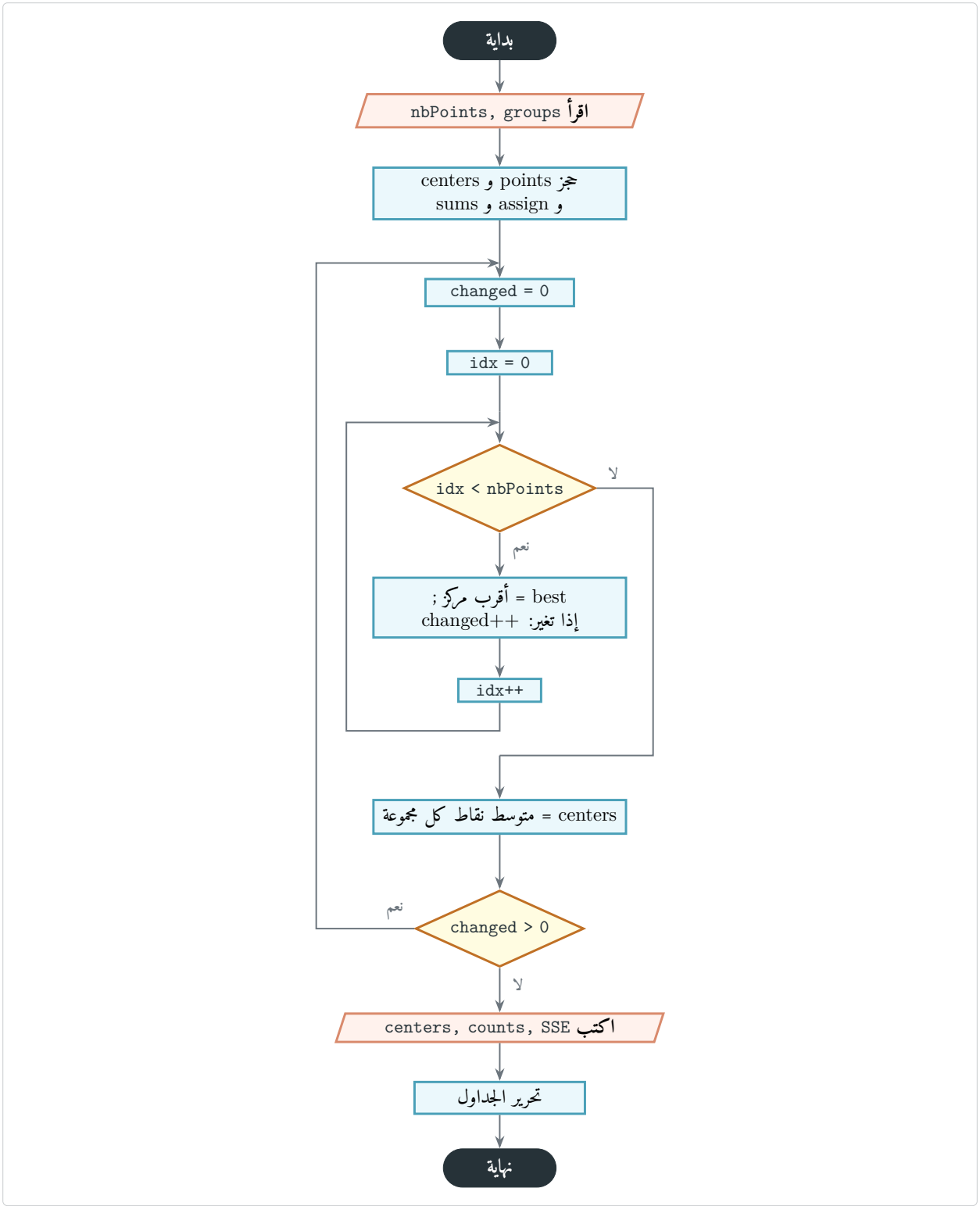
المدخلات

```
12 3
2 3 3 4 2 5 9 8 10 9 8 9 15 2 16 3 14 1 3 3 9 10 15 3
```

نتيجة التنفيذ

```
Converged in 2 iterations
Zone 1: center (2.50, 3.75), 4 stops
Zone 2: center (9.00, 9.00), 4 stops
Zone 3: center (15.00, 2.25), 4 stops
SSE = 12.500
```

المخطط الإنسيابي (Organigramme)

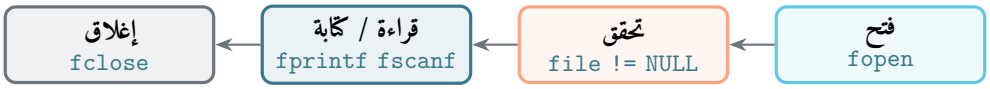


الفصل 11

الملفات

تعريفات

- **الملف**: بيانات تبقى بعد انتهاء البرنامج. نصل إليه عبر مؤشر `FILE *file = fopen(path, mode);`
- **الأوضاع**: `"r"` قراءة، `"w"` كتابة (يمسح المحتوى)، `"a"` إضافة في النهاية، `"r+"` قراءة وكتابة، ونضيف `b` للملفات الثنائية.
- **ملف نصي**: أسطر مقروءة (`fprintf`، `fscanf`، `fgets`). **ملف ثنائي**: نسخة من الذاكرة (`fwrite`، `fread`).
- **الموضع**: `fseek` للقفز، `ftell` لمعرفة الموضع، `rewind` للعودة إلى البداية.



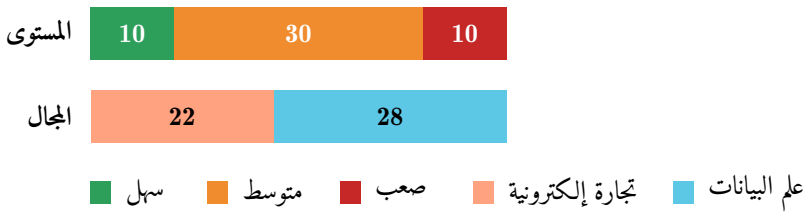
الدالة	القيمة المعادة عند النهاية أو الفشل
`fgetc(file)`	`EOF` (لذلك نخزن النتيجة في `int` لا في `char`)
`fscanf(file, ...)`	عدد القيم المقروءة فعلًا، أو `EOF`
`fgets(line, size, file)`	`NULL` (السطر المقروء يحتفظ بـ `'\n'`)
`fread(buf, size, count, file)`	عدد العناصر المقروءة (أقل من `count` في النهاية)

برامج هذا الفصل مستقلة

كل برنامج ينشئ ملفاته بنفسه ثم يقرؤها، لذلك يمكن تنفيذه مباشرة وإعادة إنتاج نتائجه. لا تنسَ `fclose`: البيانات المكتوبة قد تبقى في الذاكرة المؤقتة ولا تصل إلى القرص قبل الإغلاق.

للاستزادة: [1, 3, 4]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 11.1 كتابة وصل الشراء في ملف

تجارة إلكترونية | سهل

اكتب وصل شراء في ملف نصي `receipt.txt` بـ `fprintf`، ثم أغلقه، ثم أعد فتحه للقراءة واعرض محتواه حرفًا بحرف بـ `fgetc` حتى `EOF`.

خطوات الحل

1) `fopen("receipt.txt", "w")` للكتابة (ينشئ الملف أو يفرغه).
2) `fopen(..., "r")` للقراءة؛ `fgetc` يعيد `EOF` في النهاية.
3) كل `fopen` يقابله `fclose`.

البرنامج بلغة C

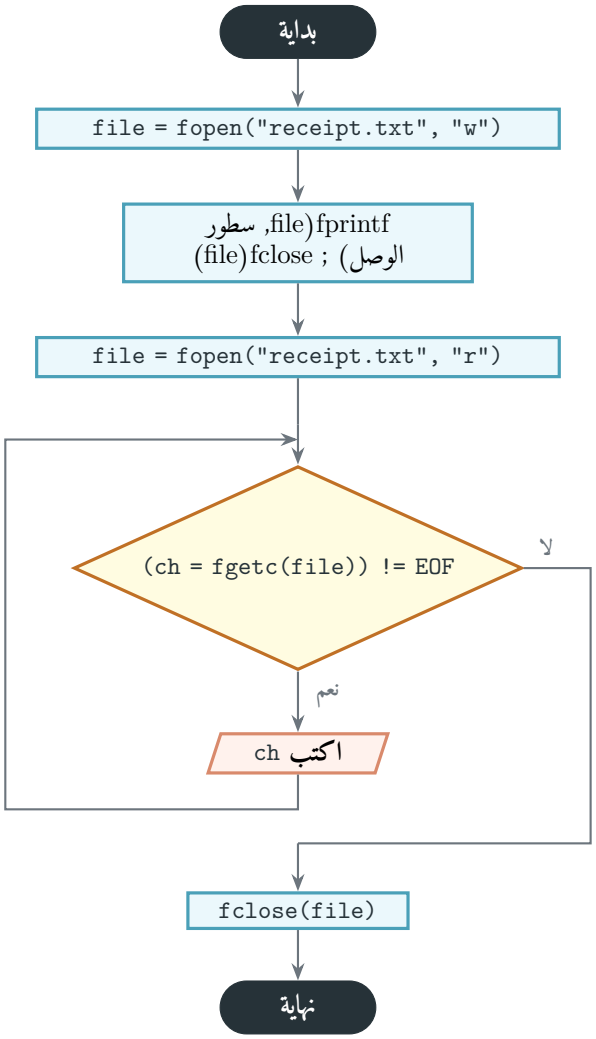
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("receipt.txt", "w");
    int ch;                                  // int لكي يسع EOF
    if (file == NULL) return 1;
    fprintf(file, "TechShop - receipt #2045\n");
    fprintf(file, "Mouse     1500.00\n");
    fprintf(file, "Cable      450.00\n");
    fprintf(file, "TOTAL     1950.00 DA\n");
    fclose(file);
    file = fopen("receipt.txt", "r");        // نعيد الفتح للقراءة
    if (file == NULL) return 1;
    while ((ch = fgetc(file)) != EOF)
        putchar(ch);
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
TechShop - receipt #2045
Mouse     1500.00
Cable      450.00
TOTAL     1950.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.2 قراءة قائمة المنتجات بـ fscanf

تجارة إلكترونية سهل

اكتب ملف `products.txt` يحتوي على سطر لكل منتج: . ثم اقرأه بـ `fscanf` (الذي يعيد عدد القيم المقروءة) واحسب قيمة المخزون.

خطوات الحل

1) الحلقة

```
while (fscanf(file, "%s %lf %d", ...) == 3)
```

تتوقف عند النهاية أو عند سطر غير صالح.

2) القيمة `price * qty`.

البرنامج بلغة C

```c
#include <stdio.h>

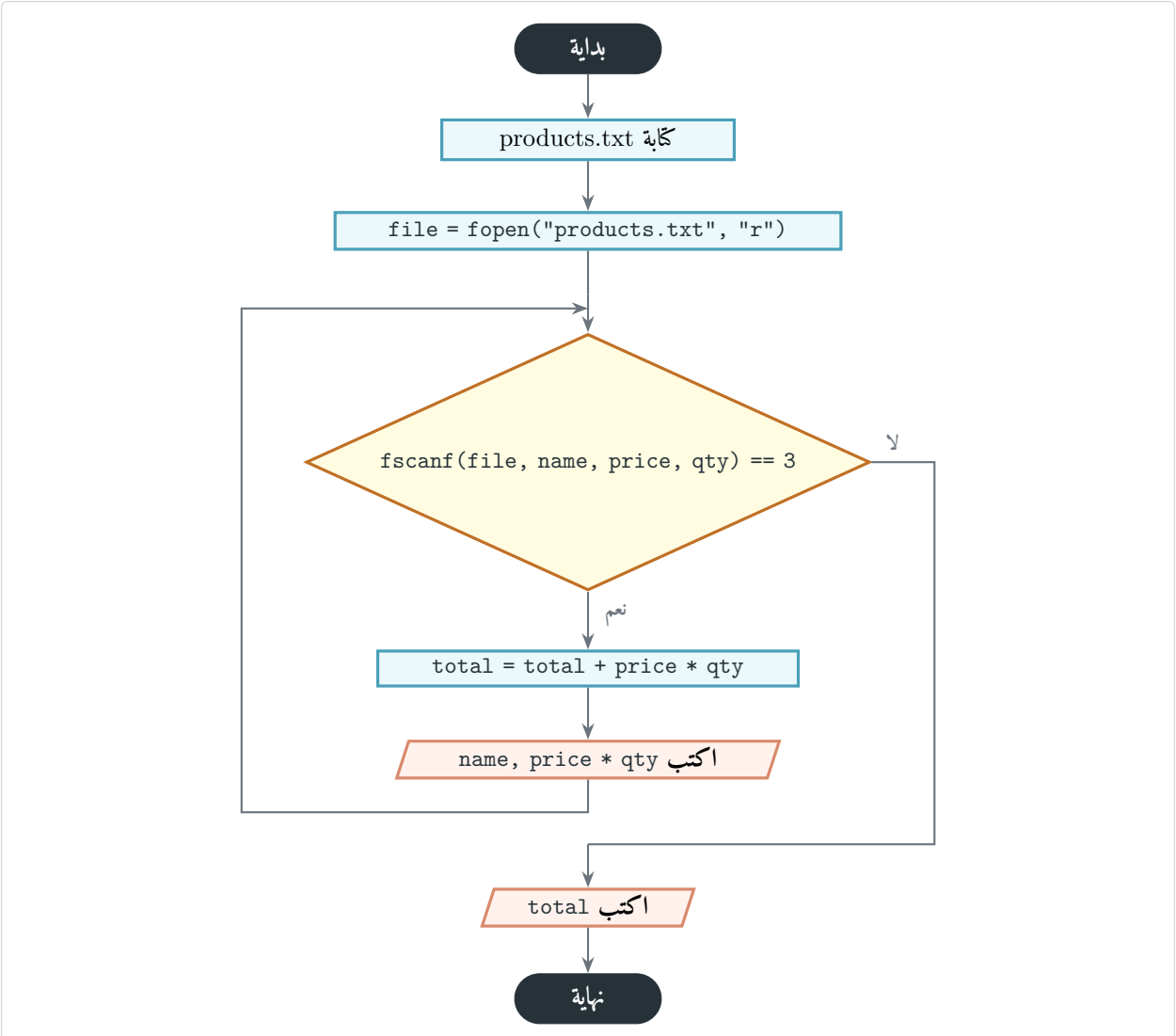
int main(void) {
    FILE *file = fopen("products.txt", "w");
    char name[30];
```

```c
    double price, total = 0;
    int qty;
    fprintf(file, "Mouse 1500 12\nKeyboard 3200 5\nMonitor 32000 3\nCable 450 40\n");
    fclose(file);
    file = fopen("products.txt", "r");
    if (file == NULL) return 1;
    while (fscanf(file, "%29s %lf %d", name, &price, &qty) == 3) {   // 3 قيم مقروءة
        printf("%-9s %9.2f\n", name, price * qty);
        total += price * qty;
    }
    fclose(file);
    printf("Stock value: %.2f DA\n", total);
    return 0;
}
```

نتيجة التنفيذ

```
Mouse      18000.00
Keyboard   16000.00
Monitor    96000.00
Cable      18000.00
Stock value: 148000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.3 عدّ أسطر سجل الطلبيات

علم البيانات سهل

أنشئ ملف سجل `orders.log` بعدة أسطر، ثم عدّ عدد الأسطر (عدد حروف `'\n'`) وعدد الحروف الكلي بقراءة الملف حرفًا بحرف.

خطوات الحل

1) كل `'\n'` يعني نهاية سطر.
2) إذا لم ينته الملف بـ `'\n'` نضيف سطرًا أخيرًا (نتذكر آخر حرف).

البرنامج بلغة C

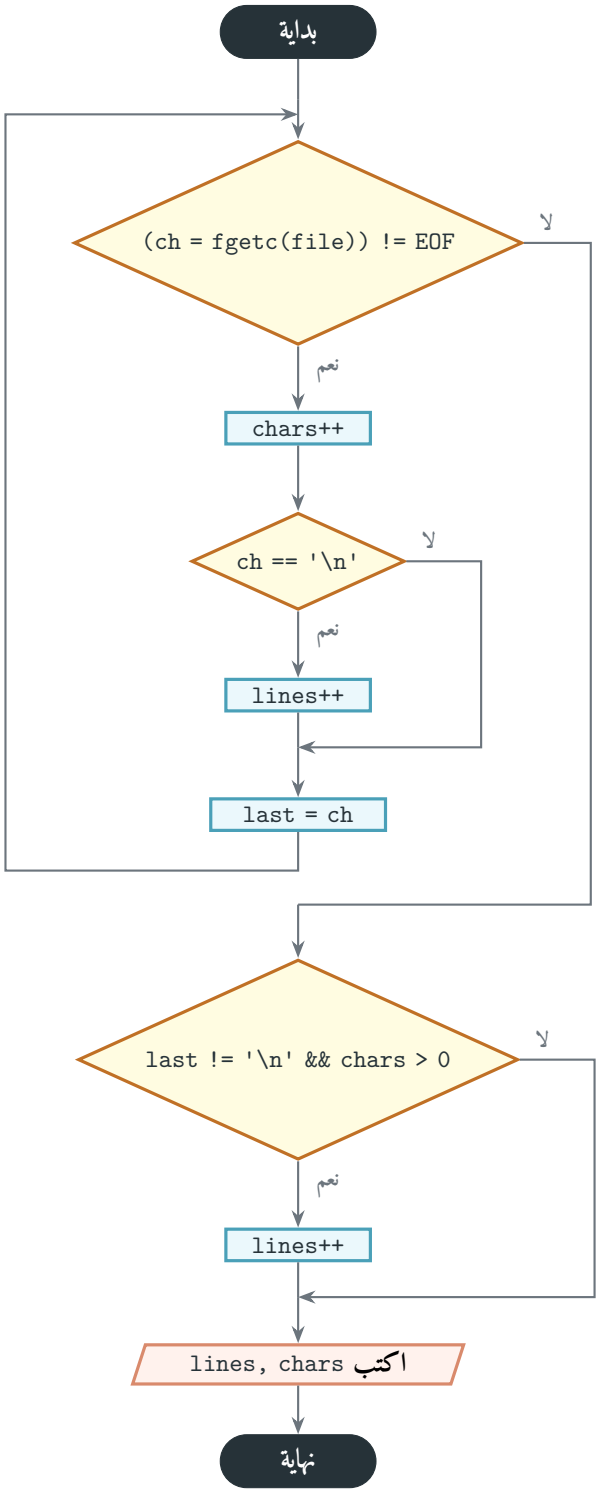
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("orders.log", "w");
    int ch, lines = 0, chars = 0, last = '\n';
    fprintf(file, "2026-11-01 #7781 paid\n2026-11-01 #7782 shipped\n");
    fprintf(file, "2026-11-02 #7783 paid\n2026-11-03 #7784 cancelled");   // دون سطر جديد في النهاية
    fclose(file);
    file = fopen("orders.log", "r");
    if (file == NULL) return 1;
    while ((ch = fgetc(file)) != EOF) {
        chars++;
        if (ch == '\n') lines++;
        last = ch;
    }
    if (last != '\n' && chars > 0) lines++;   // السطر الأخير دون "cenn"
    fclose(file);
    printf("%d lines, %d characters\n", lines, chars);
    return 0;
}
```

نتيجة التنفيذ

```
4 lines, 95 characters
```

المخطط الإنسيابي (Organigramme)



تمرين 11.4 إضافة طلبيات إلى نهاية الملف

تجارة إلكترونية

سهل

وضح الفرق بين الوضع `"w"` (يمسح المحتوى) والوضع `"a"` (يضيف في النهاية): اكتب طلبية بـ `"w"`، ثم أضف طلبيتين بـ `"a"` في فتحين منفصلين، ثم اعرض الملف.

خطوات الحل

1) `"a"` لا يمسح الملف، والكتابة تتم دائمًا في نهايته.
2) مناسب للسجلات التي تكبر مع الوقت.

البرنامج بلغة C

```c
#include <stdio.h>

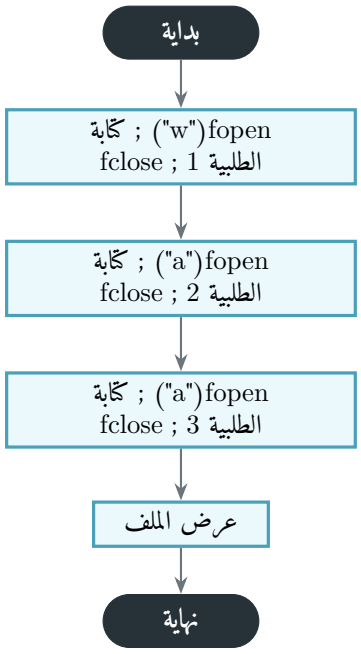
void addOrder(const char *mode, int number, double amount) {
    FILE *file = fopen("journal.txt", mode);
    if (file == NULL) return;
    fprintf(file, "#%d %.2f\n", number, amount);
    fclose(file);
}

int main(void) {
    FILE *file;
    int ch;
    addOrder("w", 7781, 12500);        // ينشئ الملف من جديد
    addOrder("a", 7782, 3200);         // يضيف في النهاية
    addOrder("a", 7783, 899.5);
    file = fopen("journal.txt", "r");
    if (file == NULL) return 1;
    while ((ch = fgetc(file)) != EOF) putchar(ch);
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
#7781 12500.00
#7782 3200.00
#7783 899.50
```

المخطط الإنسيابي (Organigramme)



تمرين 11.5 نسخة احتياطية من ملف الزبائن

تجارة إلكترونية سهل

اكتب برنامجًا ينسخ الملف customers.txt إلى customers.bak حرفًا بحرف بـ fgetc و fputc، ثم يتحقق أن الملفين بنفس الحجم.

خطوات الحل

1) نفتح المصدر بـ "r" والوجهة بـ "w".
2) كل حرف مقروء يُكتب ونعده.
3) نتحقق بإعادة قراءة النسخة وعدّ حروفها.

البرنامج بلغة C

```c
#include <stdio.h>

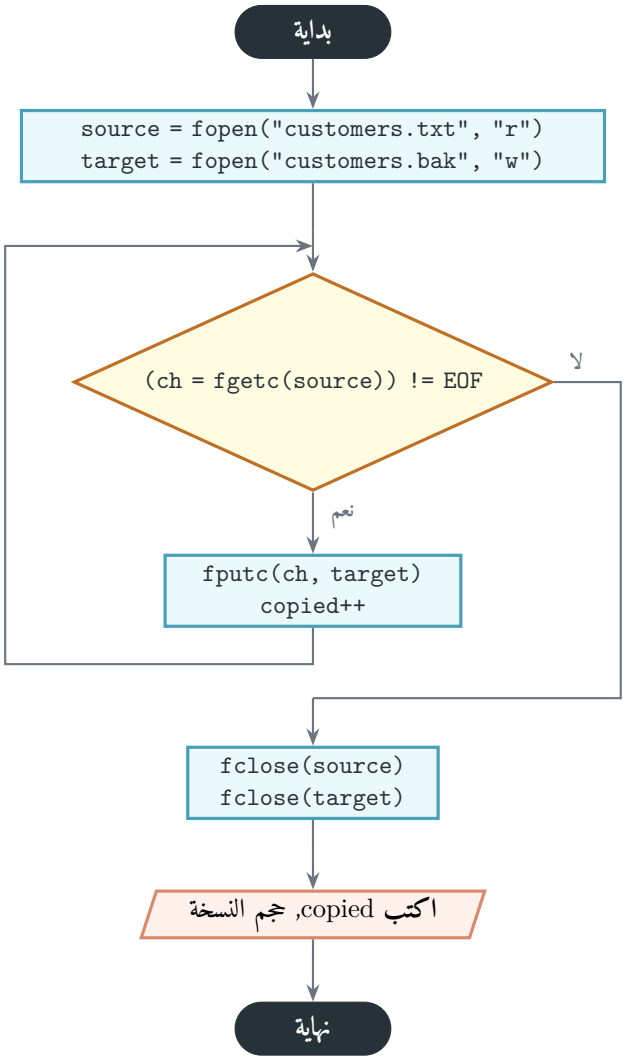
int countChars(const char *path) {
    FILE *file = fopen(path, "r");
    int count = 0;
    if (file == NULL) return -1;
    while (fgetc(file) != EOF) count++;
    fclose(file);
    return count;
}

int main(void) {
    FILE *source, *target;
    int ch, copied = 0;
    source = fopen("customers.txt", "w");
    fprintf(source, "507;Amina;Oran\n508;Karim;Algiers\n509;Lina;Setif\n");
    fclose(source);
    source = fopen("customers.txt", "r");
    target = fopen("customers.bak", "w");
    if (source == NULL || target == NULL) return 1;
    while ((ch = fgetc(source)) != EOF) {
        fputc(ch, target);                  // حرف بحرف
        copied++;
    }
    fclose(source);
    fclose(target);
    printf("Copied %d bytes\n", copied);
    printf("Original %d, backup %d -> %s\n", countChars("customers.txt"), countChars("customers.bak"),
           countChars("customers.txt") == countChars("customers.bak") ? "OK" : "MISMATCH");
    return 0;
}
```

نتيجة التنفيذ

```
Copied 48 bytes
Original 48, backup 48 -> OK
```

المخطط الإنسيابي (Organigramme)



تمرين 11.6 عدد كلمات تعليقات الزبائن

علم البيانات سهل

احفظ تعليقات الزبائن في ملف `reviews.txt`، ثم اقرأه حرفًا بحرف واحسب عدد الحروف والكلمات والأسطر (كما يفعل الأمر `wc`).

خطوات الحل

1) الكلمة تبدأ عند حرف غير فاصل بعد فاصل (فراغ أو سطر جديد).
2) متغير `inWord` يتذكر هل نحن داخل كلمة.

البرنامج بلغة C

```c
#include <stdio.h>

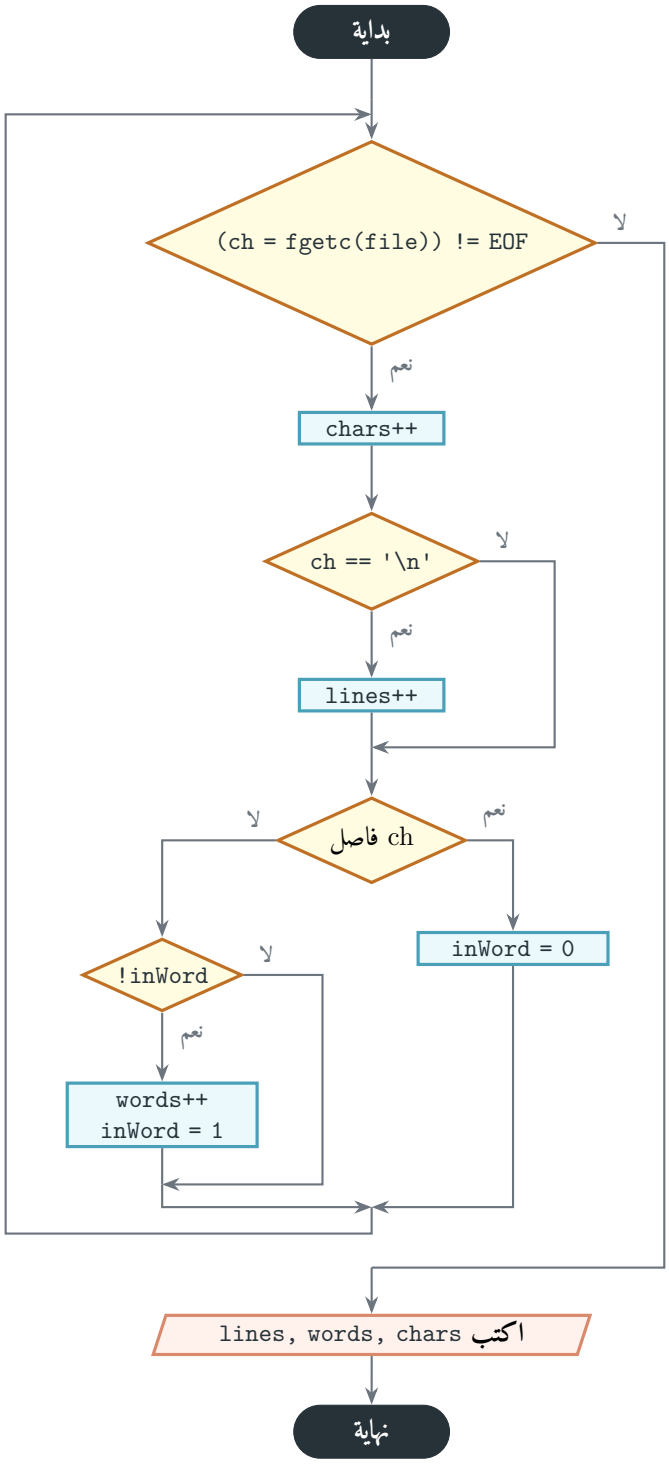
int main(void) {
    FILE *file = fopen("reviews.txt", "w");
    int ch, chars = 0, words = 0, lines = 0, inWord = 0;
    fprintf(file, "Great phone, fast delivery.\nBattery could be better\n");
```

```
    fprintf(file, "  Excellent   value for money!\n");
    fclose(file);
    file = fopen("reviews.txt", "r");
    if (file == NULL) return 1;
    while ((ch = fgetc(file)) != EOF) {
        chars++;
        if (ch == '\n') lines++;
        if (ch == ' ' || ch == '\n' || ch == '\t') {
            inWord = 0;
        } else if (!inWord) {
            words++;                            // بداية كلمة جديدة
            inWord = 1;
        }
    }
    fclose(file);
    printf("%d lines, %d words, %d chars\n", lines, words, chars);
    return 0;
}
```

نتيجة التنفيذ

```
3 lines, 12 words, 83 chars
```

المخطط الإنسيابي (Organigramme)



تمرين 11.7 التعامل مع ملف غير موجود

تجارة إلكترونية سهل

`fopen` يعيد `NULL` إذا تعذر فتح الملف (غير موجود، أو دون صلاحيات). اكتب دالة `safeTotal(path)` تحاول قراءة ملف أسعار وتعيد -1 مع رسالة واضحة إذا فشل الفتح، واختبرها على ملف موجود وآخر غير موجود.

خطوات الحل

1) نتحقق من `file == NULL` قبل أي قراءة.
2) نعيد قيمة خاصة يفهمها المستدعي.

البرنامج بلغة C

```c
#include <stdio.h>

double safeTotal(const char *path) {
    FILE *file = fopen(path, "r");
    double price, total = 0;
    if (file == NULL) {                          // فشل الفتح
        printf("Cannot open '%s'\n", path);
        return -1;
    }
    while (fscanf(file, "%lf", &price) == 1) total += price;
    fclose(file);
    return total;
}

int main(void) {
    FILE *file = fopen("prices.txt", "w");
    fprintf(file, "1500 3200 450 899\n");
    fclose(file);
    printf("prices.txt total: %.2f\n", safeTotal("prices.txt"));
    printf("old_prices.txt total: %.2f\n", safeTotal("old_prices.txt"));
    return 0;
}
```

نتيجة التنفيذ

```
prices.txt total: 6049.00
Cannot open 'old_prices.txt'
old_prices.txt total: -1.00
```

المخطط الإنسيابي (Organigramme)



تمرين 11.8 متوسط قراءات الحرارة من ملف — علم البيانات — سهل

حساس المخزن المبرد يكتب قراءة في كل سطر. احفظ القراءات في ملف ثم اقرأها حتى نهاية الملف لحساب العدد والمتوسط وأصغر وأكبر قيمة.

خطوات الحل

1) `fscanf(file, "%lf", &value) == 1` شرط الاستمرار.

2) لا نعرف العدد مسبقًا: نعد أثناء القراءة.

البرنامج بلغة C

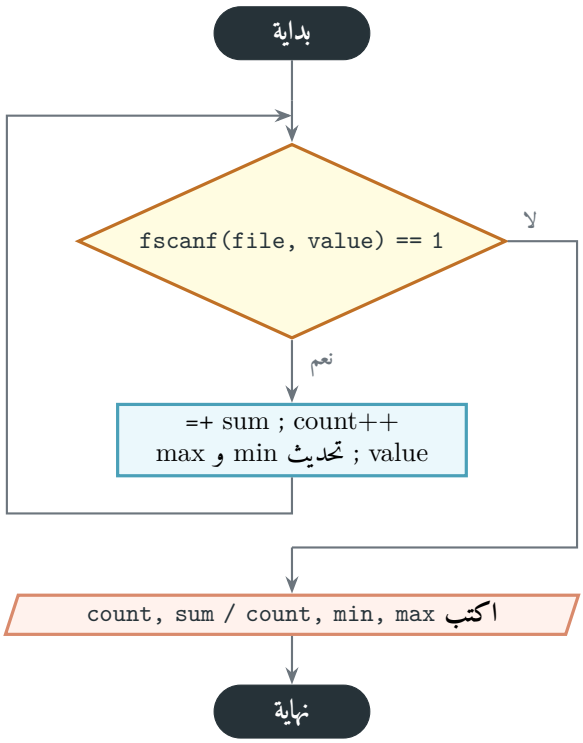
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("temps.txt", "w");
    double readings[9] = {4.2, 3.8, 5.1, 6.9, 4.4, 2.7, 3.3, 5.6, 4.0}, value, sum = 0, low = 0, high = 0;
    int idx, count = 0;
    for (idx = 0; idx < 9; idx++) fprintf(file, "%.1f\n", readings[idx]);
    fclose(file);
    file = fopen("temps.txt", "r");
    if (file == NULL) return 1;
    while (fscanf(file, "%lf", &value) == 1) {
        if (count == 0 || value < low) low = value;
        if (count == 0 || value > high) high = value;
        sum += value;
        count++;
    }
    fclose(file);
    printf("%d readings, mean %.2f C, min %.1f, max %.1f\n", count, sum / count, low, high);
    return 0;
}
```

نتيجة التنفيذ

```
9 readings, mean 4.44 C, min 2.7, max 6.9
```

المخطط الإنسيابي (Organigramme)



تمرين 11.9 قراءة الأسطر بـ fgets وترقيمها

تجارة إلكترونية ⭐ سهل

اقرأ ملف شروط البيع سطرًا سطرًا بـ `fgets` (الذي يحتفظ بـ `'\n'` في نهاية السطر)، واعرض كل سطر مسبوقًا برقمه وطوله (دون `'\n'`).

خطوات الحل

1) `fgets(line, size, file)` يعيد `NULL` في النهاية.
2) الطول: نعد الحروف حتى `'\0'` أو `'\n'`.

البرنامج بلغة C

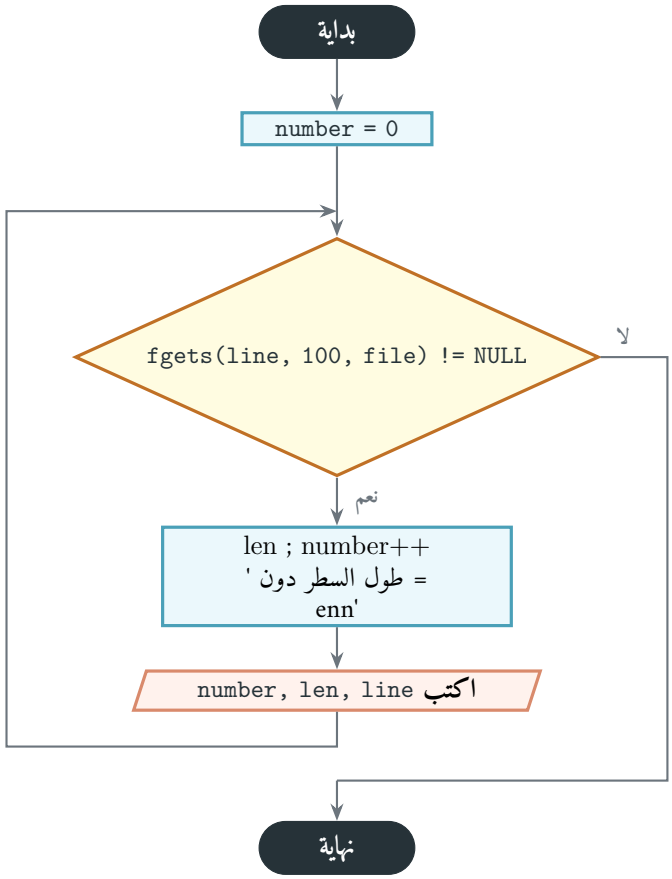
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("terms.txt", "w");
    char line[100];
    int number = 0, len;
    fprintf(file, "Returns accepted within 14 days.\nWarranty: 12 months.\n");
    fprintf(file, "Free delivery above 10000 DA.\n");
    fclose(file);
    file = fopen("terms.txt", "r");
    if (file == NULL) return 1;
    while (fgets(line, 100, file) != NULL) {
        number++;
        for (len = 0; line[len] != '\0' && line[len] != '\n'; len++) {}
        line[len] = '\0';                       // نحذف "cenn"
        printf("%2d (%2d chars) %s\n", number, len, line);
    }
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
 1 (32 chars) Returns accepted within 14 days.
 2 (20 chars) Warranty: 12 months.
 3 (29 chars) Free delivery above 10000 DA.
```

المخطط الإنسيابي (Organigramme)



تمرين 11.10 تصدير الطلبيات بصيغة CSV

علم البيانات سهل

صدّر جدول طلبيات إلى ملف `orders.csv` مع سطر عناوين، ثم أعد قراءة الملف بـ `fgets` وعدّ عدد أسطر البيانات (دون سطر العناوين) واعرض الملف.

خطوات الحل

1) سطر العناوين `id,client,amount` ثم سطر لكل طلبية.
2) عند القراءة: أول سطر هو العناوين فلا نعده.

البرنامج بلغة C

```c
#include <stdio.h>

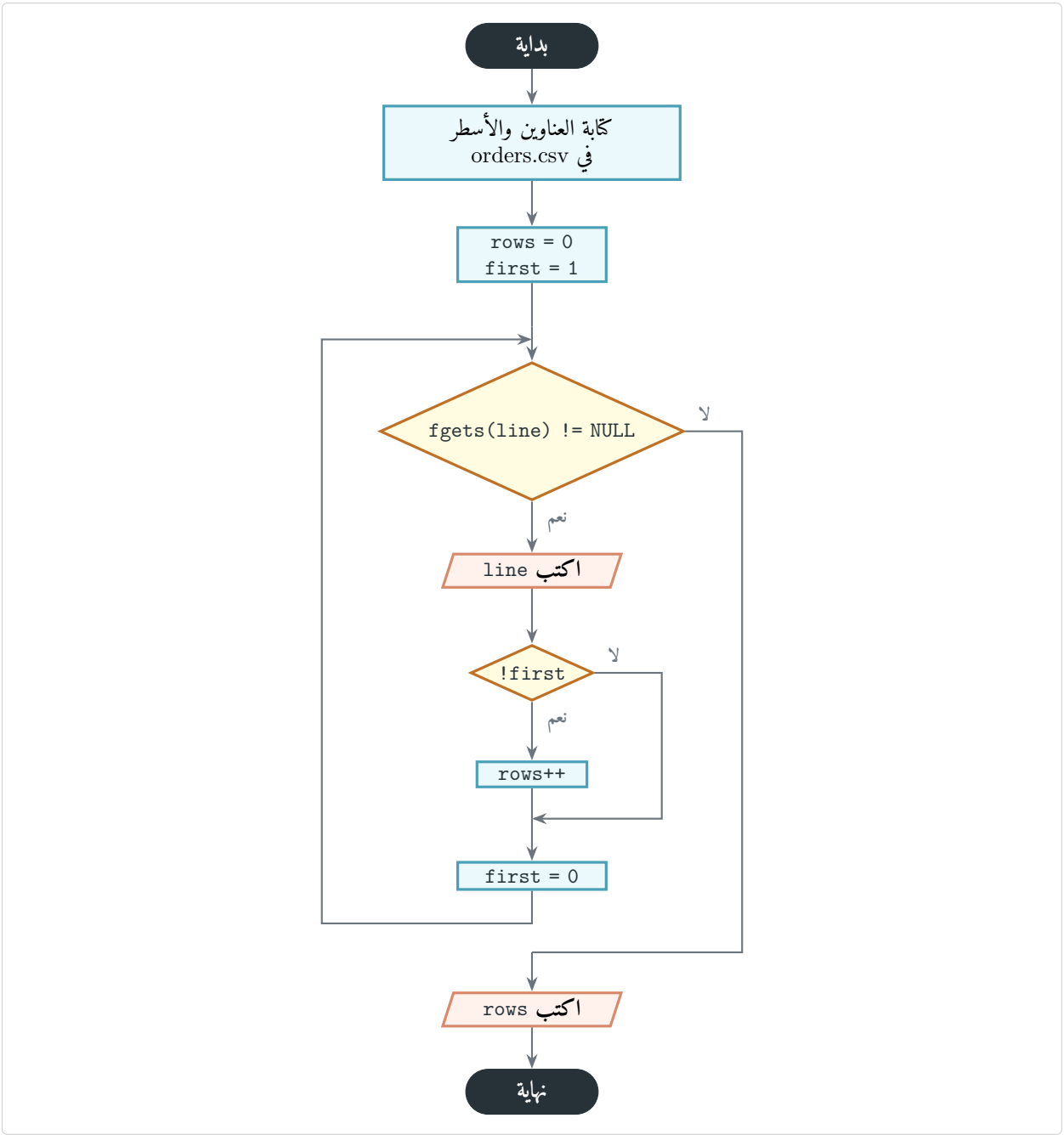
int main(void) {
    int ids[4] = {7781, 7782, 7783, 7784};
    char clients[4][10] = {"Amina", "Karim", "Lina", "Omar"};
    double amounts[4] = {12500, 3200, 899.5, 45000};
    FILE *file = fopen("orders.csv", "w");
    char line[100];
    int idx, rows = 0, first = 1;
    fprintf(file, "id,client,amount\n");                // سطر العناوين
    for (idx = 0; idx < 4; idx++)
        fprintf(file, "%d,%s,%.2f\n", ids[idx], clients[idx], amounts[idx]);
```

```
    fclose(file);
    file = fopen("orders.csv", "r");
    if (file == NULL) return 1;
    while (fgets(line, 100, file) != NULL) {
        printf("%s", line);
        if (!first) rows++;
        first = 0;
    }
    fclose(file);
    printf("%d data rows\n", rows);
    return 0;
}
```

نتيجة التنفيذ

```
id,client,amount
7781,Amina,12500.00
7782,Karim,3200.00
7783,Lina,899.50
7784,Omar,45000.00
4 data rows
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 11.11 الإيراد لكل منتج من ملف CSV

علم البيانات ★★ متوسط

ملف المبيعات sales.csv أسطره بالصيغة date;product;qty;price. اقرأ كل سطر بـ fgets، وفككه يدويًا (دون strtok ولا sscanf)، وحوّل الكمية والسعر إلى أعداد، ثم اجمع الإيراد لكل منتج.

خطوات الحل

1) نقسم السطر عند ; إلى 4 حقول في مكانه.
2) تحويل يدوي: toInt و toDouble.
3) جدول هياكل (اسم المنتج، الإيراد) يُبنى تدريجيًا.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { char name[16]; double revenue; } Total;

int toInt(const char *text) {
    int value = 0;
    while (*text >= '0' && *text <= '9') value = value * 10 + (*text++ - '0');
    return value;
}

double toDouble(const char *text) {
    double value = 0, scale = 1;
    while (*text >= '0' && *text <= '9') value = value * 10 + (*text++ - '0');
    if (*text == '.') for (text++; *text >= '0' && *text <= '9'; text++) value += (*text - '0') * (scale /= 10);
    return value;
}

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

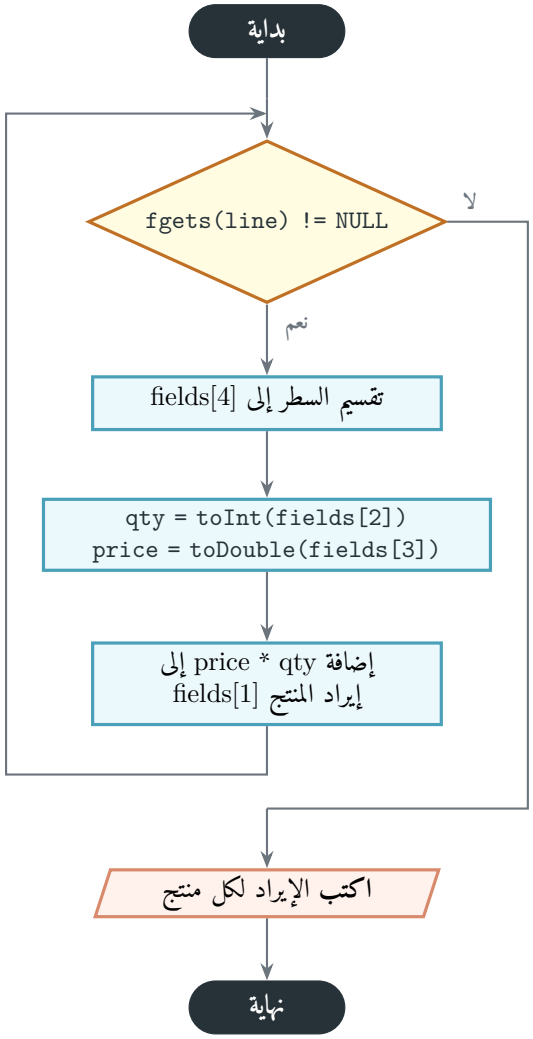
int main(void) {
    FILE *file = fopen("sales.csv", "w");
    char line[100], *fields[4];
    Total totals[10];
    int nbTotals = 0, count, idx, pos, letter;
    fprintf(file, "2026-11-01;Mouse;3;1500\n2026-11-01;Cable;10;450.5\n2026-11-02;Mouse;2;1450\n");
    fprintf(file, "2026-11-02;Monitor;1;32000\n2026-11-03;Cable;4;450.5\n2026-11-03;Mouse;1;1500\n");
    fclose(file);
    file = fopen("sales.csv", "r");
    if (file == NULL) return 1;
    while (fgets(line, 100, file) != NULL) {
        count = 1;
        fields[0] = line;
        for (pos = 0; line[pos] != '\0' && line[pos] != '\n'; pos++)
            if (line[pos] == ';' && count < 4) { line[pos] = '\0'; fields[count++] = line + pos + 1; }
        line[pos] = '\0';
        for (idx = 0; idx < nbTotals && !sameText(totals[idx].name, fields[1]); idx++) {}
        if (idx == nbTotals) {                                  // منتج جديد
            for (letter = 0; (totals[idx].name[letter] = fields[1][letter]) != '\0'; letter++) {}
            totals[idx].revenue = 0;
            nbTotals++;
        }
```

```
45          totals[idx].revenue += toInt(fields[2]) * toDouble(fields[3]);
46      }
47      fclose(file);
48      for (idx = 0; idx < nbTotals; idx++) printf("%-8s %10.2f DA\n", totals[idx].name, totals[idx].revenue);
49      return 0;
50  }
```

نتيجة التنفيذ

```
Mouse       8900.00 DA
Cable       6307.00 DA
Monitor    32000.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.12 استخراج الأخطاء من سجل الخادم

علم البيانات متوسط

سجل الخادم `server.log` يحتوي على أسطر بمستويات `INFO` و `WARN` و `ERROR`. اكتب برنامجًا ينسخ أسطر `ERROR` فقط إلى ملف `errors.log`، ويعد الأسطر حسب المستوى، ثم يعرض ملف الأخطاء.

خطوات الحل

1) دالة `startsWith(line, prefix)` بعد تخطي الطابع الزمني (أول 9 حروف).
2) نكتب السطر كما هو بـ `fputs` في الملف الثاني.

البرنامج بلغة C

```c
#include <stdio.h>

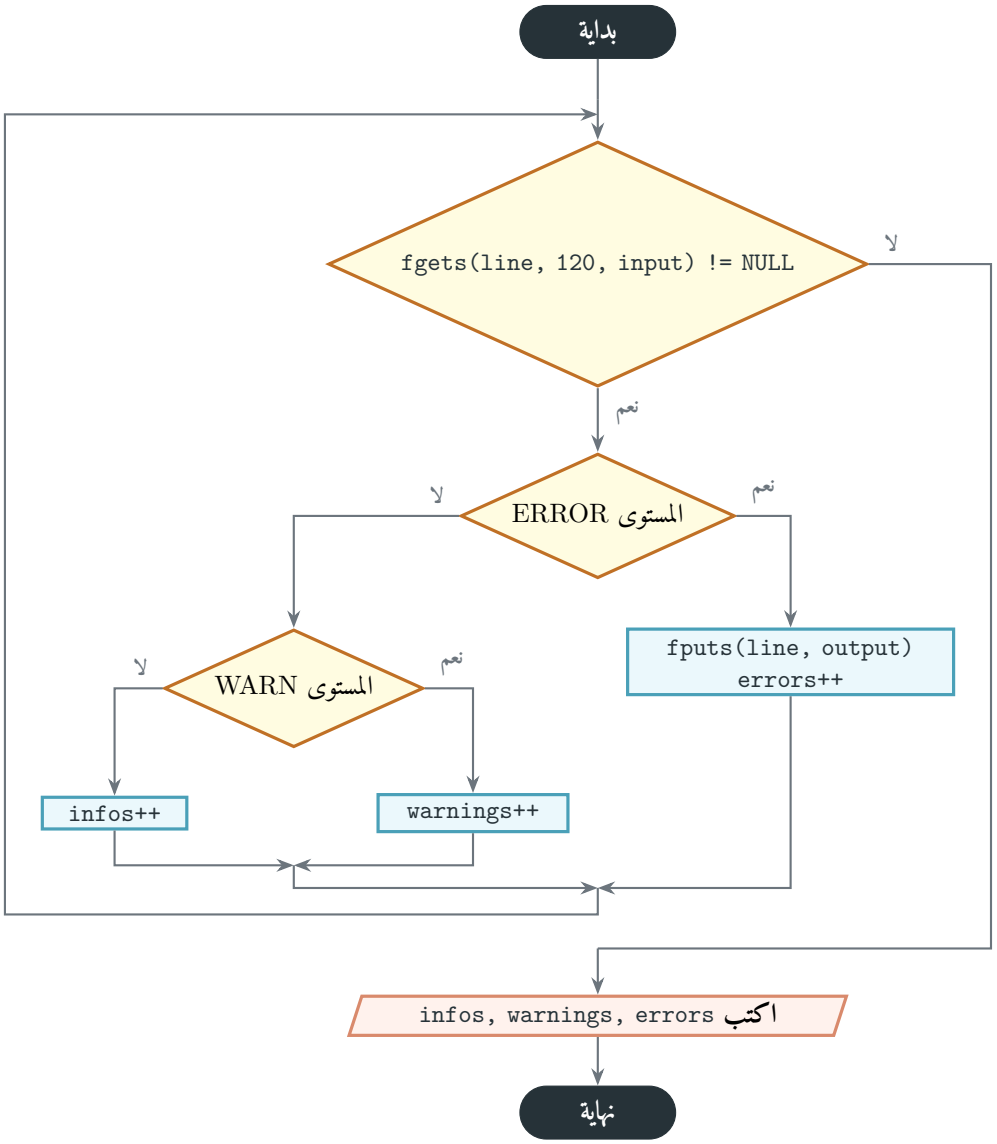
int startsWith(const char *text, const char *prefix) {
    while (*prefix != '\0') if (*text++ != *prefix++) return 0;
    return 1;
}

int main(void) {
    FILE *input = fopen("server.log", "w"), *output;
    char line[120];
    int infos = 0, warnings = 0, errors = 0;
    fprintf(input, "10:00:01 INFO  checkout started #7781\n10:00:02 ERROR payment timeout #7781\n");
    fprintf(input, "10:00:05 WARN  slow query 2300ms\n10:01:10 INFO  order paid #7782\n");
    fprintf(input, "10:02:33 ERROR stock negative sku=MS-110\n10:03:00 INFO  login user 507\n");
    fclose(input);
    input = fopen("server.log", "r");
    output = fopen("errors.log", "w");
    if (input == NULL || output == NULL) return 1;
    while (fgets(line, 120, input) != NULL) {
        if (startsWith(line + 9, "ERROR")) { fputs(line, output); errors++; }   // بعد الوقت
        else if (startsWith(line + 9, "WARN")) warnings++;
        else infos++;
    }
    fclose(input);
    fclose(output);
    printf("INFO %d, WARN %d, ERROR %d\n--- errors.log ---\n", infos, warnings, errors);
    output = fopen("errors.log", "r");
    while (fgets(line, 120, output) != NULL) printf("%s", line);
    fclose(output);
    return 0;
}
```

نتيجة التنفيذ

```
INFO 3, WARN 1, ERROR 2
--- errors.log ---
10:00:02 ERROR payment timeout #7781
10:02:33 ERROR stock negative sku=MS-110
```

المخطط الإنسيابي (Organigramme)



تمرين 11.13 دمج ملفين مرتبين للطلبيات

تجارة إلكترونية ★★ متوسط

لكل فرع ملف بأرقام طلبياته مرتبة. ادمج الملفين في ملف ثالث مرتب `all.txt` بقراءة رقم واحد من كل ملف في كل مرة (دون تحميل الملفين في الذاكرة)، ثم اعرض النتيجة.

خطوات الحل

1) نقرأ الرقم الأول من كل ملف؛ `hasA` و `hasB` تشيران إلى وجود قيمة.
2) نكتب الأصغر ثم نقرأ التالي من نفس الملف.
3) عند انتهاء ملف ننسخ باقي الآخر.

البرنامج بلغة C

```c
#include <stdio.h>

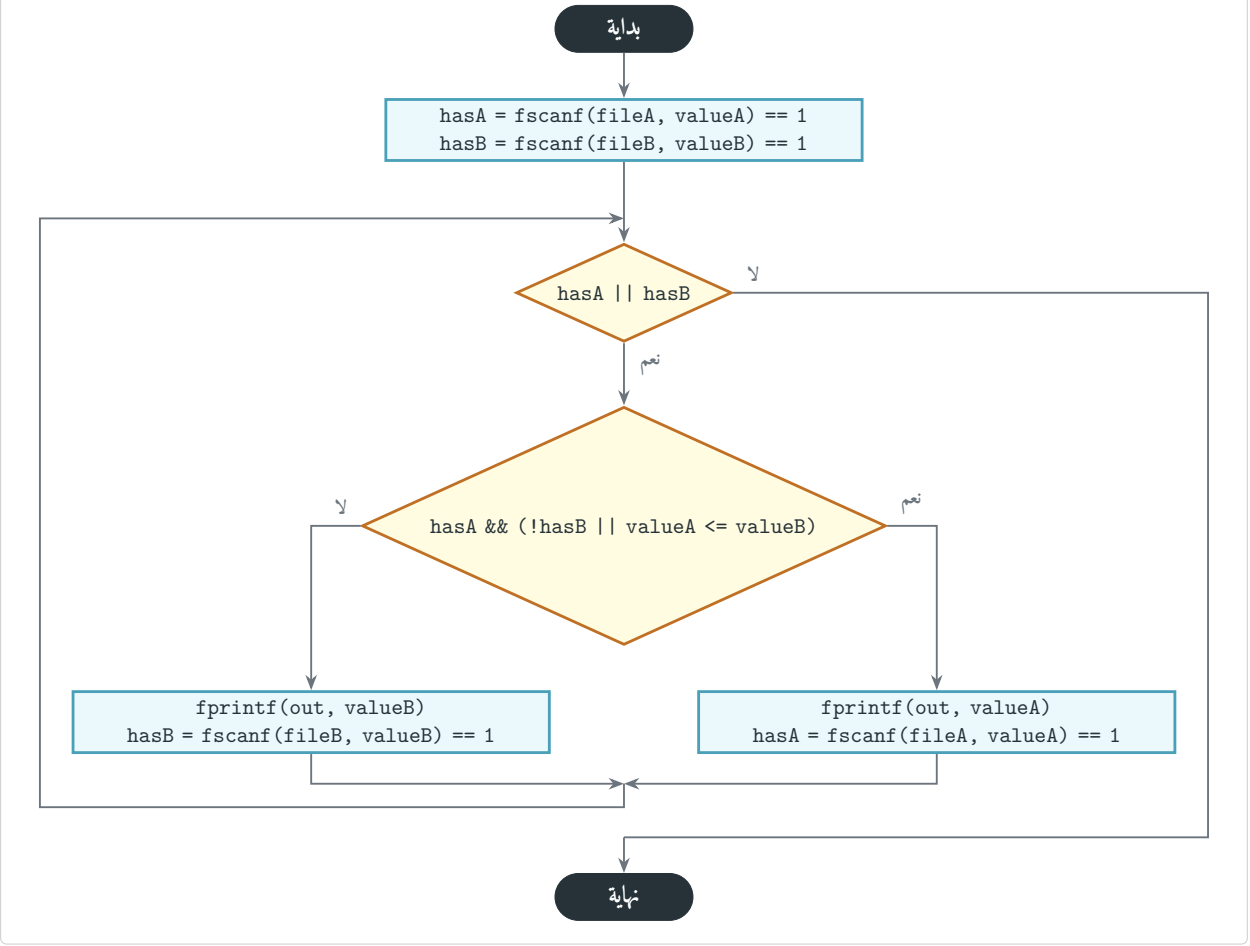
void writeList(const char *path, const int *values, int size) {
    FILE *file = fopen(path, "w");
    int idx;
    for (idx = 0; idx < size; idx++) fprintf(file, "%d\n", values[idx]);
    fclose(file);
}

int main(void) {
    int north[5] = {1003, 1010, 1011, 1025, 1040}, south[4] = {1001, 1012, 1030, 1045};
    FILE *fileA, *fileB, *out;
    int valueA, valueB, hasA, hasB, written = 0;
    writeList("north.txt", north, 5);
    writeList("south.txt", south, 4);
    fileA = fopen("north.txt", "r");
    fileB = fopen("south.txt", "r");
    out = fopen("all.txt", "w");
    hasA = fscanf(fileA, "%d", &valueA) == 1;
    hasB = fscanf(fileB, "%d", &valueB) == 1;
    while (hasA || hasB) {
        if (hasA && (!hasB || valueA <= valueB)) {
            fprintf(out, "%d\n", valueA);
            hasA = fscanf(fileA, "%d", &valueA) == 1;      // التالي من نفس الملف
        } else {
            fprintf(out, "%d\n", valueB);
            hasB = fscanf(fileB, "%d", &valueB) == 1;
        }
        written++;
    }
    fclose(fileA); fclose(fileB); fclose(out);
    out = fopen("all.txt", "r");
    printf("%d orders:", written);
    while (fscanf(out, "%d", &valueA) == 1) printf(" %d", valueA);
    printf("\n");
    fclose(out);
    return 0;
}
```

نتيجة التنفيذ

```
9 orders: 1001 1003 1010 1011 1012 1025 1030 1040 1045
```

المخطط الإنسيابي (Organigramme)



تمرين 11.14 حفظ الكتالوج في ملف ثنائي

تجارة إلكترونية ★★ متوسط

احفظ جدول هياكل `Product` في ملف ثنائي `catalog.bin` بـ `fwrite` دفعة واحدة، ثم اقرأه بـ `fread` في جدول جديد واعرضه. قارن حجم الملف الثنائي بحجم النسخة النصية.

خطوات الحل

1) `fwrite(items, sizeof(Product), count, file)` يكتب الذاكرة كما هي.
2) `fread` يعيد عدد العناصر المقروءة فعلًا.
3) الوضع `"wb"` و `"rb"` للملفات الثنائية.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; char name[16]; double price; int stock; } Product;

long fileSize(const char *path) {
    FILE *file = fopen(path, "rb");
    long size = 0;
    if (file == NULL) return -1;
    while (fgetc(file) != EOF) size++;
    fclose(file);
```

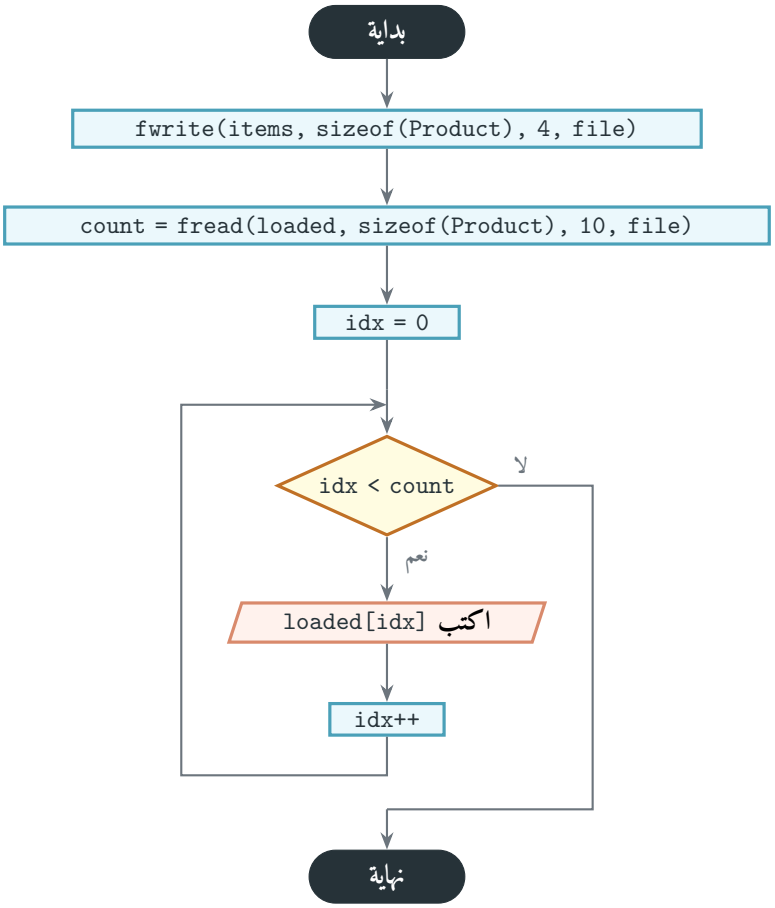
```c
    return size;
}

int main(void) {
    Product items[4] = {{101, "Mouse", 1500, 12}, {102, "Keyboard", 3200, 5},
                        {103, "Monitor", 32000, 3}, {104, "Cable", 450, 40}};
    Product loaded[10];
    FILE *file = fopen("catalog.bin", "wb");
    int idx, count;
    fwrite(items, sizeof(Product), 4, file);            // نسخة مباشرة للذاكرة
    fclose(file);
    file = fopen("catalog.txt", "w");
    for (idx = 0; idx < 4; idx++)
        fprintf(file, "%d %s %.2f %d\n", items[idx].code, items[idx].name, items[idx].price,
      ↪ items[idx].stock);
    fclose(file);
    file = fopen("catalog.bin", "rb");
    count = (int)fread(loaded, sizeof(Product), 10, file);
    fclose(file);
    for (idx = 0; idx < count; idx++)
        printf("%d %-9s %8.2f %3d\n", loaded[idx].code, loaded[idx].name, loaded[idx].price,
      ↪ loaded[idx].stock);
    printf("%d records read; binary %ld bytes, text %ld bytes\n", count,
           fileSize("catalog.bin"), fileSize("catalog.txt"));
    return 0;
}
```

نتيجة التنفيذ

```
101 Mouse      1500.00  12
102 Keyboard   3200.00   5
103 Monitor   32000.00   3
104 Cable       450.00  40
4 records read; binary 160 bytes, text 87 bytes
```

المخطط الانسيابي (Organigramme)



تمرين 11.15 تعديل سجل مباشرة بـ fseek

تجارة إلكترونية متوسط

في ملف ثنائي من سجلات ثابتة الحجم، يمكن الوصول إلى السجل رقم `idx` مباشرة عند الموضع `idx * sizeof(Product)`. اكتب دالة `updateStock(path, idx, delta)` تفتح الملف بـ `"r+b"`، تنتقل بـ `fseek`، تقرأ السجل، تعدل المخزون، ثم تعيد كتابته في نفس المكان.

خطوات الحل

1) `fseek(file, idx * sizeof(Product), SEEK_SET)` ثم `fread`.
2) بعد القراءة يتقدم موضع الملف: نعود بـ `fseek` قبل `fwrite`.
3) لا حاجة لإعادة كتابة الملف كله.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; char name[12]; int stock; } Product;

void updateStock(const char *path, int idx, int delta) {
    FILE *file = fopen(path, "r+b");                // قراءة وكتابة
    Product item;
    if (file == NULL) return;
    fseek(file, idx * (long)sizeof(Product), SEEK_SET);
```

```c
    fread(&item, sizeof(Product), 1, file);
    item.stock += delta;
    fseek(file, idx * (long)sizeof(Product), SEEK_SET);   // نعود إلى بداية السجل
    fwrite(&item, sizeof(Product), 1, file);
    fclose(file);
    printf("record %d (%s): stock %+d -> %d\n", idx, item.name, delta, item.stock);
}

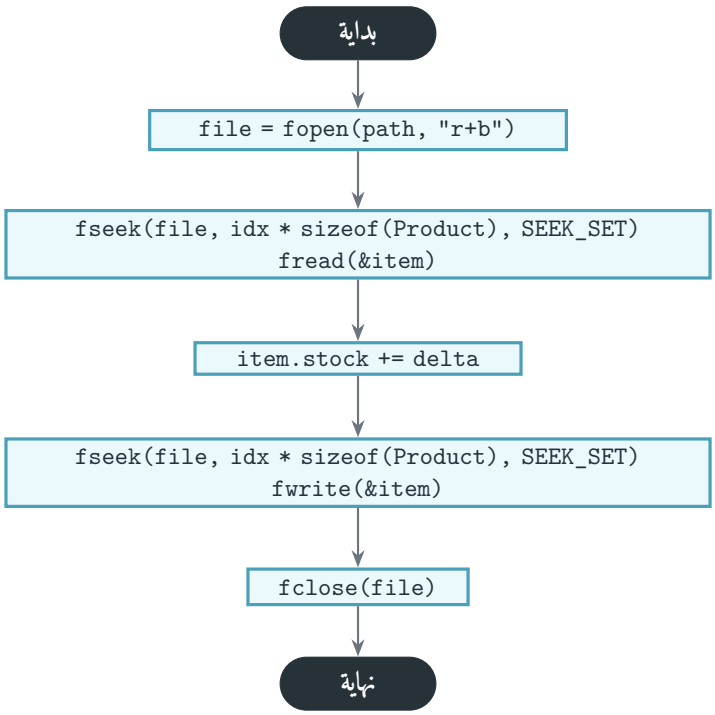
void show(const char *path) {
    FILE *file = fopen(path, "rb");
    Product item;
    while (fread(&item, sizeof(Product), 1, file) == 1) printf("[%d %s %d] ", item.code, item.name,
      ↪ item.stock);
    printf("\n");
    fclose(file);
}

int main(void) {
    Product items[4] = {{101, "Mouse", 12}, {102, "Keyboard", 5}, {103, "Monitor", 3}, {104, "Cable", 40}};
    FILE *file = fopen("stock.bin", "wb");
    fwrite(items, sizeof(Product), 4, file);
    fclose(file);
    show("stock.bin");
    updateStock("stock.bin", 2, -1);
    updateStock("stock.bin", 0, 20);
    show("stock.bin");
    return 0;
}
```

نتيجة التنفيذ

```
[101 Mouse 12] [102 Keyboard 5] [103 Monitor 3] [104 Cable 40]
record 2 (Monitor): stock -1 -> 2
record 0 (Mouse): stock +20 -> 32
[101 Mouse 32] [102 Keyboard 5] [103 Monitor 2] [104 Cable 40]
```

المخطط الإنسيابي (Organigramme)



تمرين 11.16 حجم الملف بـ fseek و ftell

علم البيانات متوسط

اكتب دالة `sizeOf(path)` تعطي حجم ملف بالبايتات **دون قراءته كله**: ننتقل إلى نهايته بـ `fseek(file, 0, SEEK_END)` ثم نسأل عن الموضع بـ `ftell`. استعملها لحساب عدد السجلات في ملف ثنائي وموضع آخر سجل.

خطوات الحل

1) `ftell` يعيد الموضع الحالي بالبايتات من البداية.
2) عدد السجلات = الحجم / حجم السجل.
3) `fseek(file, -sizeof(Record), SEEK_END)` يقفز إلى آخر سجل.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int day; double sales; } Record;

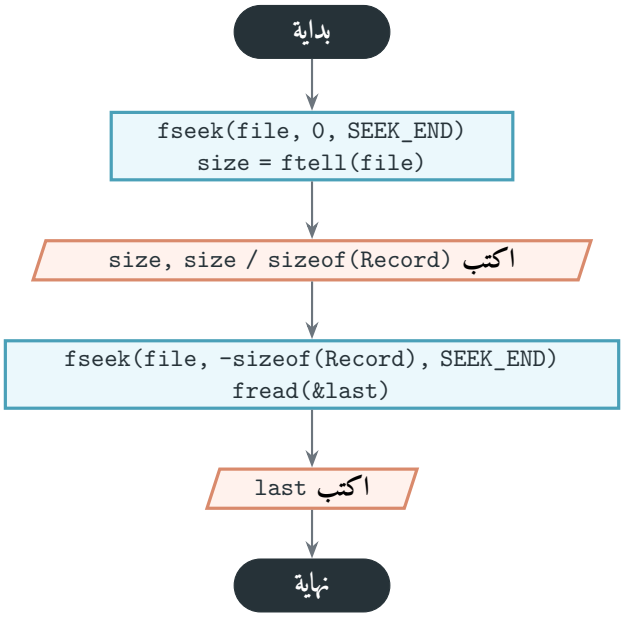
long sizeOf(const char *path) {
    FILE *file = fopen(path, "rb");
    long size;
    if (file == NULL) return -1;
    fseek(file, 0, SEEK_END);                  // القفز إلى النهاية
    size = ftell(file);
    fclose(file);
    return size;
}

int main(void) {
    FILE *file = fopen("daily.bin", "wb");
    Record record;
    long size;
    int day;
    for (day = 1; day <= 30; day++) {
        record.day = day;
        record.sales = 1000 + day * 35.5;
        fwrite(&record, sizeof(Record), 1, file);
    }
    fclose(file);
    size = sizeOf("daily.bin");
    printf("File size %ld bytes, record %d bytes -> %ld records\n", size,
           (int)sizeof(Record), size / (long)sizeof(Record));
    file = fopen("daily.bin", "rb");
    fseek(file, -(long)sizeof(Record), SEEK_END);  // آخر سجل مباشرة
    fread(&record, sizeof(Record), 1, file);
    printf("Last record: day %d, %.2f DA\n", record.day, record.sales);
    fseek(file, 14 * (long)sizeof(Record), SEEK_SET);
    fread(&record, sizeof(Record), 1, file);
    printf("Record 15: day %d, %.2f DA\n", record.day, record.sales);
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
File size 480 bytes, record 16 bytes -> 30 records
Last record: day 30, 2065.00 DA
Record 15: day 15, 1532.50 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.17 الكلمات الأكثر تكرارًا في ملف التعليقات

علم البيانات ★★ متوسط

اقرأ ملف التعليقات كلمة بكلمة بـ `fscanf(file, "%s", word)`، نظّف كل كلمة (حروف صغيرة، دون علامات ترقيم)، وتجاهل الكلمات القصيرة (أقل من 4 حروف)، ثم اعرض أكثر 5 كلمات تكرارًا.

خطوات الحل

1) `clean(word)`: نبقي الحروف فقط ونحولها إلى صغيرة.
2) جدول (كلمة، عداد) مع بحث يدوي.
3) اختيار الأكبر 5 مرات.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAXW 80

int clean(char *word) {
    int read, write = 0;
    for (read = 0; word[read] != '\0'; read++) {
        char ch = word[read];
        if (ch >= 'A' && ch <= 'Z') ch = ch - 'A' + 'a';
        if (ch >= 'a' && ch <= 'z') word[write++] = ch;    // نبقي الحروف فقط
    }
    word[write] = '\0';
    return write;
}

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

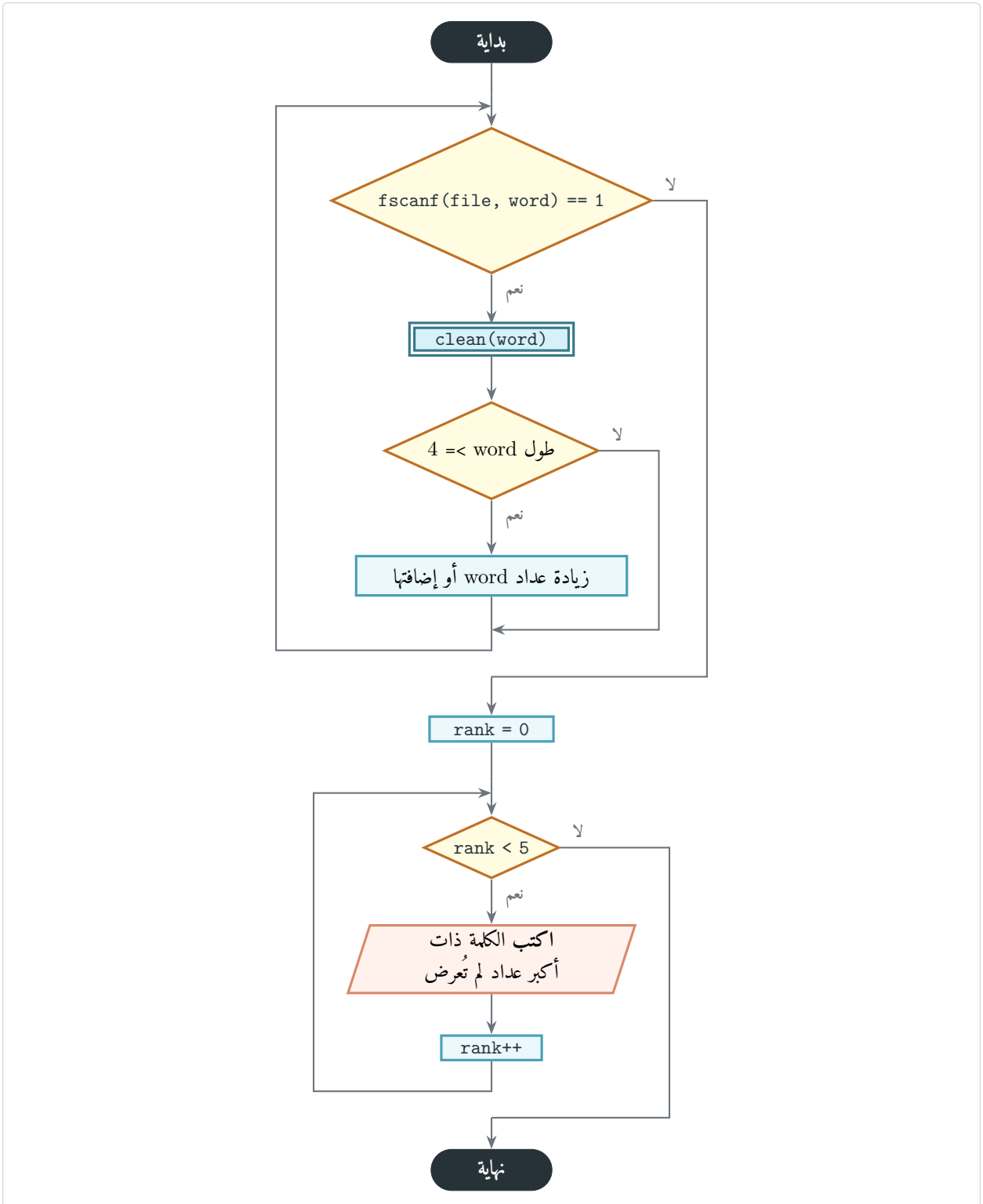
int main(void) {
    FILE *file = fopen("comments.txt", "w");
    char words[MAXW][20], word[40];
```

```c
    int counts[MAXW], used[MAXW], nb = 0, idx, rank, best, letter;
    fprintf(file, "Fast delivery! Great quality, great price.\nDelivery was fast and the quality is
      ↪ great.\n");
    fprintf(file, "Price is fair; delivery slow this time. Quality GREAT.\n");
    fclose(file);
    file = fopen("comments.txt", "r");
    if (file == NULL) return 1;
    while (fscanf(file, "%39s", word) == 1) {
        if (clean(word) < 4) continue;                    // كلمات قصيرة
        for (idx = 0; idx < nb && !sameText(words[idx], word); idx++) {}
        if (idx == nb && nb < MAXW) {
            for (letter = 0; (words[nb][letter] = word[letter]) != '\0'; letter++) {}
            counts[nb] = 0;
            used[nb] = 0;
            nb++;
        }
        counts[idx]++;
    }
    fclose(file);
    for (rank = 0; rank < 5 && rank < nb; rank++) {
        best = -1;
        for (idx = 0; idx < nb; idx++)
            if (!used[idx] && (best < 0 || counts[idx] > counts[best])) best = idx;
        used[best] = 1;
        printf("%d. %-9s %d\n", rank + 1, words[best], counts[best]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
1. great     4
2. delivery  3
3. quality   3
4. fast      2
5. price     2
```

المخطط الإنسيابي (Organigramme)



تمرين 11.18 تحديث المخزون من ملف الحركات

تجارة إلكترونية ★★ متوسط

ملف `stock.txt` يحتوي على . ، وملف `moves.txt` يحتوي على حركات `+/-` . حمّل المخزون، طبّق الحركات (مع رفض أي حركة تجعل المخزون سالبًا)، ثم اكتب المخزون الجديد في `stock_new.txt` واعرضه.

خطوات الحل

1) تحميل الملف الأول في جدولين `codes[]` و `qty[]`.
2) لكل حركة: بحث عن الرمز ثم التحقق والتطبيق.
3) الكتابة في ملف جديد (لا نفقد الأصل إذا حدث خطأ).

</> البرنامج بلغة C

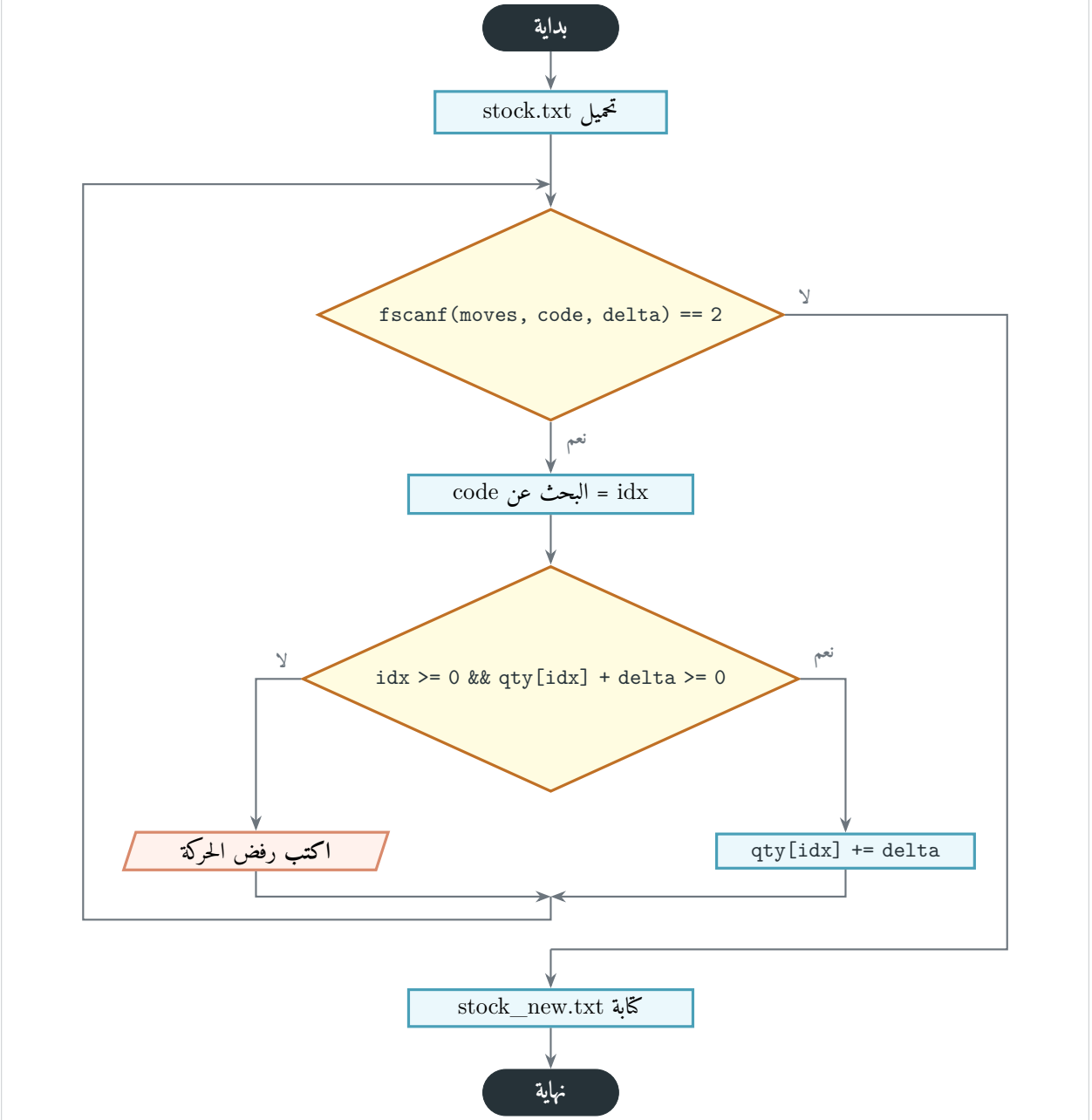
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("stock.txt", "w");
    int codes[20], qty[20], nb = 0, code, delta, idx, applied = 0;
    fprintf(file, "101 12\n102 5\n103 3\n104 40\n");
    fclose(file);
    file = fopen("moves.txt", "w");
    fprintf(file, "101 -4\n103 -5\n104 +10\n102 -5\n105 +3\n101 +20\n");
    fclose(file);
    file = fopen("stock.txt", "r");
    while (nb < 20 && fscanf(file, "%d %d", &codes[nb], &qty[nb]) == 2) nb++;
    fclose(file);
    file = fopen("moves.txt", "r");
    while (fscanf(file, "%d %d", &code, &delta) == 2) {    // + يقبل الإشارة d%
        for (idx = 0; idx < nb && codes[idx] != code; idx++) {}
        if (idx < nb && qty[idx] + delta >= 0) {
            qty[idx] += delta;
            applied++;
        } else {
            printf("rejected: %d %+d\n", code, delta);
        }
    }
    fclose(file);
    file = fopen("stock_new.txt", "w");
    for (idx = 0; idx < nb; idx++) fprintf(file, "%d %d\n", codes[idx], qty[idx]);
    fclose(file);
    printf("%d moves applied. New stock:\n", applied);
    file = fopen("stock_new.txt", "r");
    while (fscanf(file, "%d %d", &code, &delta) == 2) printf("  %d: %d\n", code, delta);
    fclose(file);
    return 0;
}
```

>_ نتيجة التنفيذ

```
rejected: 103 -5
rejected: 105 +3
4 moves applied. New stock:
  101: 28
  102: 0
  103: 3
  104: 50
```

المخطط الإنسيابي (Organigramme)



تمرين 11.19 قراءة ملف الإعدادات

تجارة إلكترونية متوسط

ملف إعدادات المتجر `shop.conf` يحتوي على أسطر `key=value`، وأسطر تعليق تبدأ بـ `#`، وأسطر فارغة. اكتب دالة `getSetting(path, key, value)` تبحث عن مفتاح وتنسخ قيمته، واقرأ بها عدة إعدادات مع قيمة افتراضية عند الغياب.

خطوات الحل

1) نتجاهل الأسطر التي تبدأ بـ `#` أو `'\n'`.
2) نقارن الجزء قبل `=` بالمفتاح، وننسخ ما بعده حتى نهاية السطر.
3) نعيد 1 إذا وُجد المفتاح، 0 وإلا.

البرنامج بلغة C

```c
#include <stdio.h>

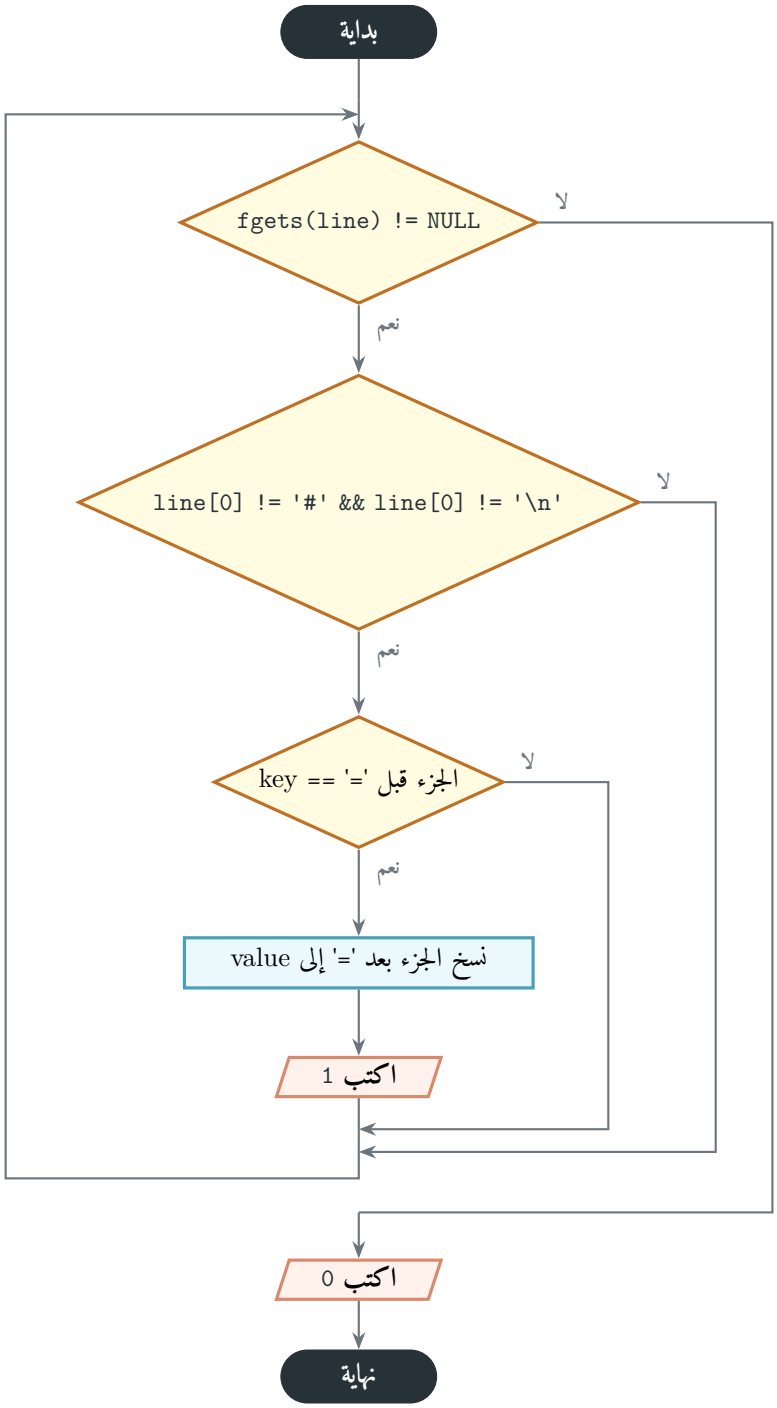
int getSetting(const char *path, const char *key, char *value) {
    FILE *file = fopen(path, "r");
    char line[100];
    int pos, out;
    if (file == NULL) return 0;
    while (fgets(line, 100, file) != NULL) {
        if (line[0] == '#' || line[0] == '\n') continue;      // تعليق أو فراغ
        for (pos = 0; key[pos] != '\0' && line[pos] == key[pos]; pos++) {}
        if (key[pos] == '\0' && line[pos] == '=') {
            for (pos++, out = 0; line[pos] != '\n' && line[pos] != '\0'; pos++) value[out++] = line[pos];
            value[out] = '\0';
            fclose(file);
            return 1;
        }
    }
    fclose(file);
    return 0;
}

int main(void) {
    FILE *file = fopen("shop.conf", "w");
    const char *keys[4] = {"currency", "vat", "free_shipping", "theme"};
    char value[50];
    int idx;
    fprintf(file, "# TechShop settings\nname=TechShop DZ\ncurrency=DZD\n\nvat=19\n");
    fprintf(file, "# shipping\nfree_shipping=10000\n");
    fclose(file);
    for (idx = 0; idx < 4; idx++) {
        if (getSetting("shop.conf", keys[idx], value))
            printf("%-14s = %s\n", keys[idx], value);
        else
            printf("%-14s = (default) light\n", keys[idx]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
currency       = DZD
vat            = 19
free_shipping  = 10000
theme          = (default) light
```

المخطط الإنسيابي (Organigramme)



تمرين 11.20 تقسيم السجل إلى ملفات شهرية

علم البيانات ★★ متوسط

ملف `history.txt` يحتوي على طلبيات السنة بالصيغة `month amount`. وزّعها على ملفات شهرية `m01.txt` ... `m12.txt` (نبني اسم الملف يدويًا دون `sprintf`)، ثم أعد فتح كل ملف شهري موجود واحسب مجموعه وعدد طلبياته.

خطوات الحل

1) اسم الملف: `name[1] = '0' + month / 10` و `name[2] = '0' + month % 10`.

2) نفتح الملف الشهري بـ `"a"` لإضافة كل طلبية.

3) في البداية نفرغ الملفات الـ 12 بفتحها بـ `"w"`.

البرنامج بلغة C

```c
#include <stdio.h>

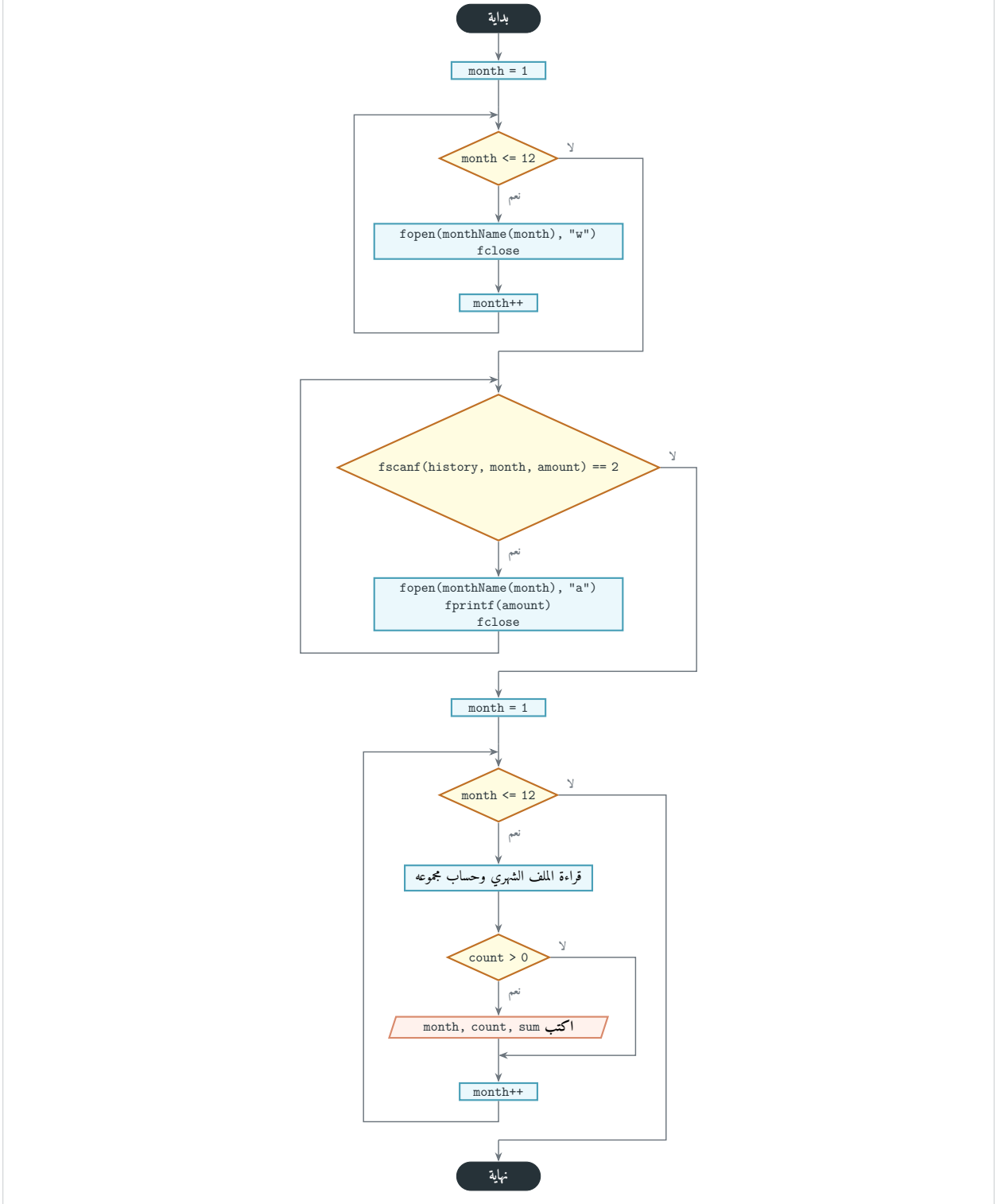
void monthName(int month, char *name) {
    name[0] = 'm';
    name[1] = (char)('0' + month / 10);          // بناء الاسم يدويًا
    name[2] = (char)('0' + month % 10);
    name[3] = '.'; name[4] = 't'; name[5] = 'x'; name[6] = 't'; name[7] = '\0';
}

int main(void) {
    FILE *file = fopen("history.txt", "w"), *part;
    char name[8];
    int month, count, data[10][2] = {{1, 1200}, {3, 4500}, {1, 800}, {11, 12500}, {3, 2300},
                                     {12, 9900}, {11, 3100}, {11, 450}, {7, 6000}, {12, 1500}};
    double amount, sum;
    for (count = 0; count < 10; count++) fprintf(file, "%d %d\n", data[count][0], data[count][1]);
    fclose(file);
    for (month = 1; month <= 12; month++) { monthName(month, name); part = fopen(name, "w"); fclose(part); }
    file = fopen("history.txt", "r");
    while (fscanf(file, "%d %lf", &month, &amount) == 2) {
        monthName(month, name);
        part = fopen(name, "a");
        fprintf(part, "%.2f\n", amount);
        fclose(part);
    }
    fclose(file);
    for (month = 1; month <= 12; month++) {
        monthName(month, name);
        part = fopen(name, "r");
        for (count = 0, sum = 0; fscanf(part, "%lf", &amount) == 1; count++) sum += amount;
        fclose(part);
        if (count > 0) printf("%s: %d orders, %.2f DA\n", name, count, sum);
    }
    return 0;
}
```

نتيجة التنفيذ

```
m01.txt: 2 orders, 2000.00 DA
m03.txt: 2 orders, 6800.00 DA
m07.txt: 1 orders, 6000.00 DA
m11.txt: 3 orders, 16050.00 DA
m12.txt: 2 orders, 11400.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.21 استبدال اسم منتج في ملف الوصف

تجارة إلكترونية

متوسط

غيّر المتجر اسم علامة تجارية. اكتب برنامجًا يقرأ ملف الأوصاف حرفًا بحرف ويكتب نسخة جديدة يُستبدل فيها كل ظهور لـ "SmartX" بـ "NovaTech"، ويعد عدد الاستبدالات.

خطوات الحل

1) نحتفظ بعدد الحروف المطابقة حتى الآن `matched`.
2) عند اكتمال التطابق نكتب الكلمة الجديدة.
3) عند انقطاع التطابق نكتب الحروف المحجوزة كما هي ثم نعيد فحص الحرف الحالي.

البرنامج بلغة C

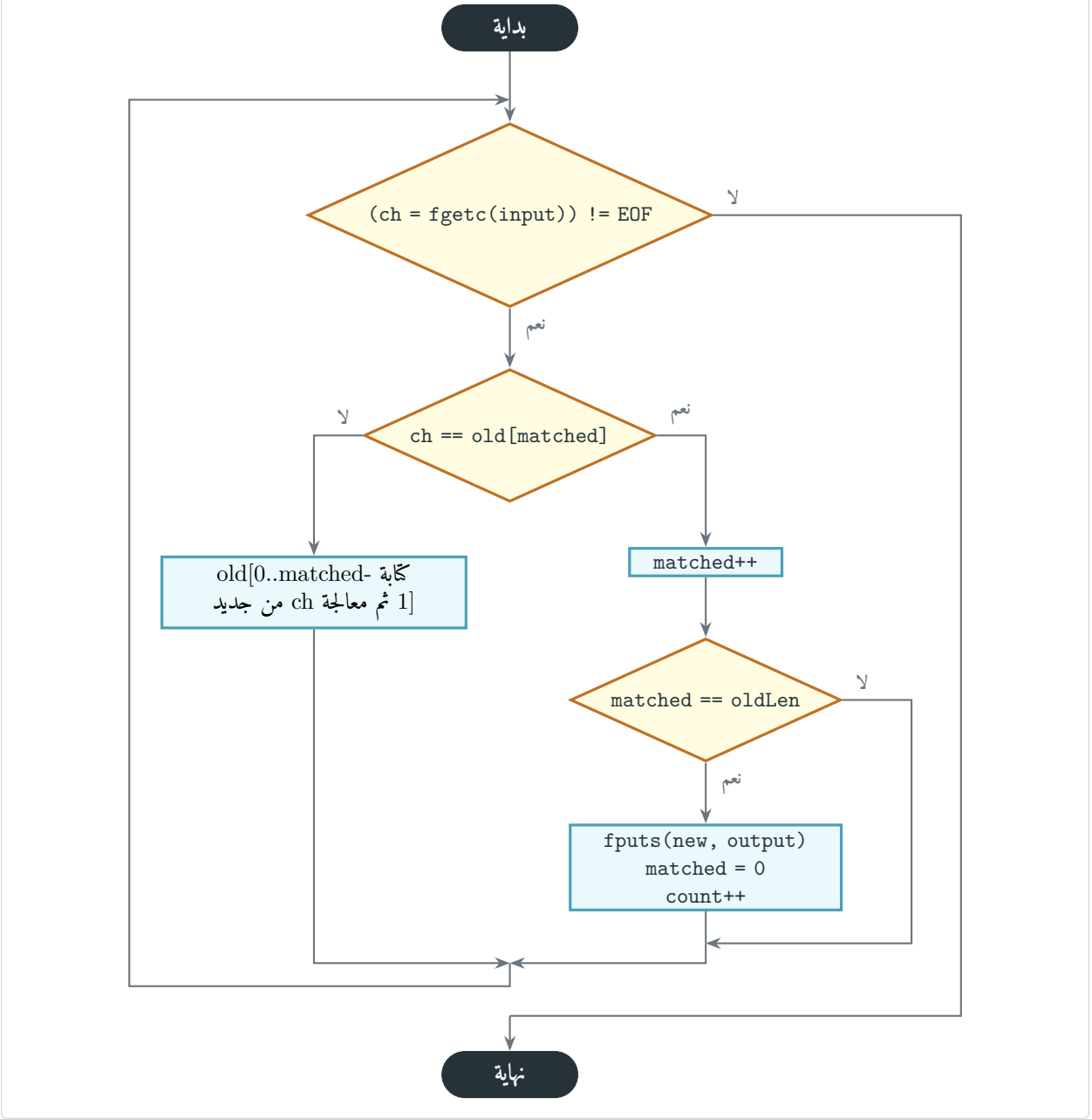
```c
#include <stdio.h>

int main(void) {
    const char *oldWord = "SmartX", *newWord = "NovaTech";
    FILE *input = fopen("desc.txt", "w"), *output;
    int ch, matched = 0, oldLen = 6, count = 0, idx;
    fprintf(input, "SmartX Phone 12: the best SmartX ever.\nCase for Smart phones and SmartX tablets.\n");
    fclose(input);
    input = fopen("desc.txt", "r");
    output = fopen("desc_new.txt", "w");
    while ((ch = fgetc(input)) != EOF) {
        while (1) {
            if (ch == oldWord[matched]) {
                matched++;
                if (matched == oldLen) { fputs(newWord, output); matched = 0; count++; }
                break;
            }
            if (matched == 0) { fputc(ch, output); break; }
            for (idx = 0; idx < matched; idx++) fputc(oldWord[idx], output);   // تطابق جزئي فاشل
            matched = 0;                                                       // نعيد فحص ch
        }
    }
    for (idx = 0; idx < matched; idx++) fputc(oldWord[idx], output);
    fclose(input);
    fclose(output);
    printf("%d replacements\n", count);
    output = fopen("desc_new.txt", "r");
    while ((ch = fgetc(output)) != EOF) putchar(ch);
    fclose(output);
    return 0;
}
```

نتيجة التنفيذ

```
3 replacements
NovaTech Phone 12: the best NovaTech ever.
Case for Smart phones and NovaTech tablets.
```

المخطط الإنسيابي (Organigramme)



تمرين 11.22 إحصاءات أعمدة ملف بيانات

علم البيانات ★★ متوسط

ملف `metrics.txt` يحتوي على سطر عناوين ثم أسطر بثلاثة أعمدة رقمية (الزيارات، الطلبيات، الإيراد). احسب لكل عمود: المتوسط والانحراف المعياري والأصغر والأكبر **بمرور واحد** على الملف (مجموع ومجموع مربعات).

خطوات الحل

1) تخطي سطر العناوين بـ `fgets`.
2) `variance = sumSq / count - mean²`.
3) الجذر بطريقة نيوتن.

البرنامج بلغة C

```c
#include <stdio.h>

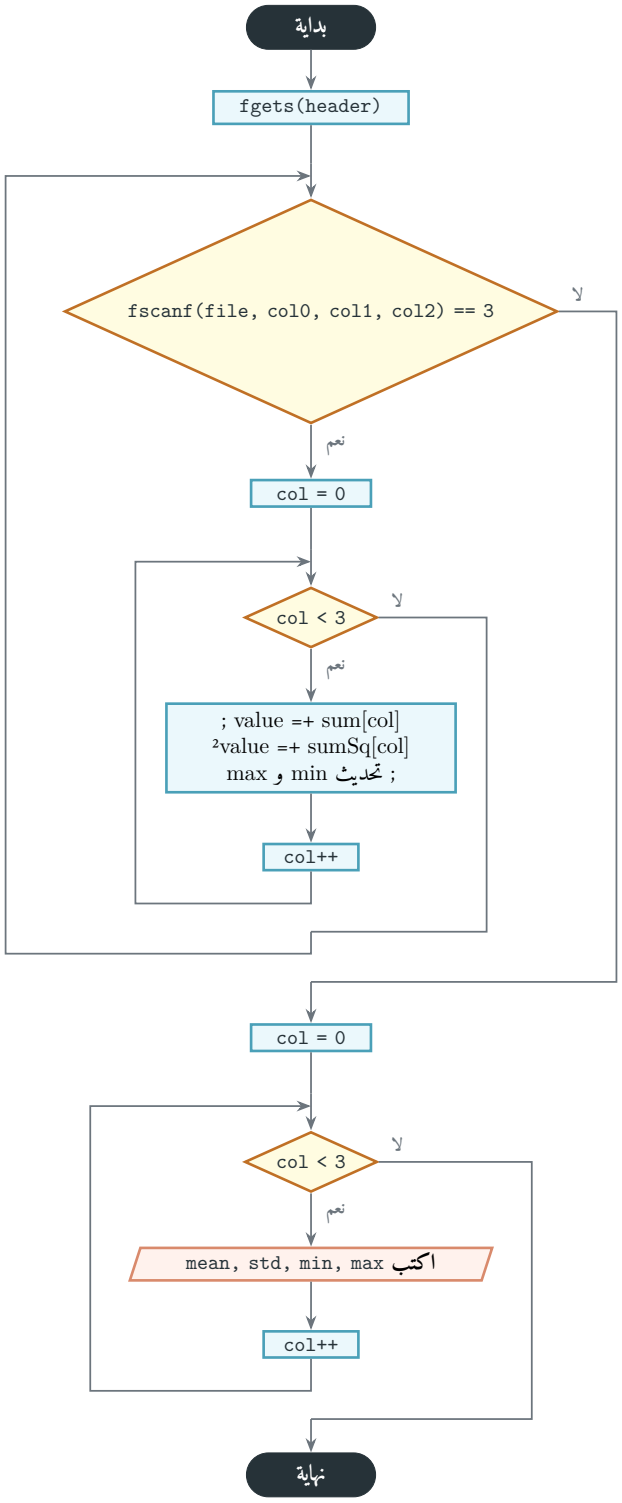
double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    if (value <= 0) return 0;
    for (iter = 0; iter < 60; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

int main(void) {
    FILE *file = fopen("metrics.txt", "w");
    char header[100], names[3][10] = {"visits", "orders", "revenue"};
    double row[3], sum[3] = {0}, sumSq[3] = {0}, low[3], high[3], mean;
    int col, count = 0;
    fprintf(file, "visits orders revenue\n1200 36 182000\n1350 41 205000\n980 25 131000\n");
    fprintf(file, "1610 52 268000\n1420 39 199500\n1105 30 150500\n");
    fclose(file);
    file = fopen("metrics.txt", "r");
    fgets(header, 100, file);                          // سطر العناوين
    while (fscanf(file, "%lf %lf %lf", &row[0], &row[1], &row[2]) == 3) {
        for (col = 0; col < 3; col++) {
            sum[col] += row[col];
            sumSq[col] += row[col] * row[col];
            if (count == 0 || row[col] < low[col]) low[col] = row[col];
            if (count == 0 || row[col] > high[col]) high[col] = row[col];
        }
        count++;
    }
    fclose(file);
    for (col = 0; col < 3; col++) {
        mean = sum[col] / count;
        printf("%-8s mean %10.2f  std %9.2f  min %8.0f  max %8.0f\n", names[col], mean,
               mySqrt(sumSq[col] / count - mean * mean), low[col], high[col]);
    }
    return 0;
}
```

نتيجة التنفيذ

```
visits   mean    1277.50  std    208.32  min      980  max     1610
orders   mean      37.17  std      8.55  min       25  max       52
revenue  mean  189333.33  std  43783.24  min   131000  max   268000
```

المخطط الإنسيابي (Organigramme)



تمرين 11.23 تصدير الكتالوج بصيغة JSON

تجارة إلكترونية متوسط

تطلب واجهة الهاتف الكتالوج بصيغة **JSON**. اكتب دالة `exportJson(path, items, size)` تكتب مصفوفة كائنات مع الفواصل والمسافات الصحيحة (لا فاصلة بعد آخر عنصر)، وتهرّب علامات الاقتباس داخل الأسماء، ثم اعرض الملف.

خطوات الحل

1) [ثم لكل منتج {"code": ..., "name": "...", "price": ...} ثم].
2) الفاصلة بين العناصر فقط: if (idx < size - 1).
3) الحرف " داخل الاسم يُكتب \".

</> البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; char name[30]; double price; int inStock; } Product;

void writeEscaped(FILE *file, const char *text) {
    for (; *text != '\0'; text++) {
        if (*text == '"' || *text == '\\') fputc('\\', file);   // تهريب
        fputc(*text, file);
    }
}

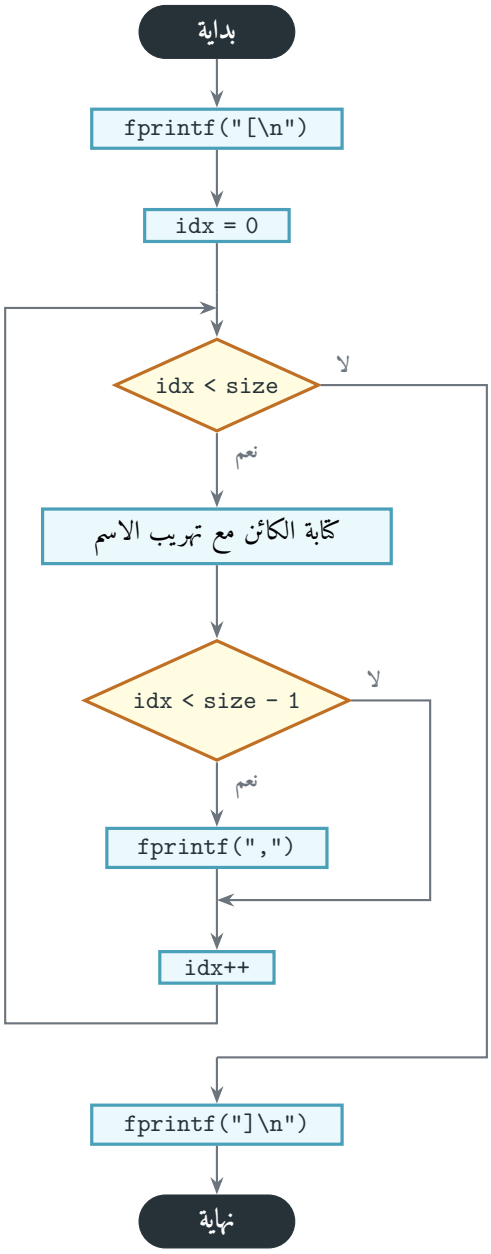
void exportJson(const char *path, const Product *items, int size) {
    FILE *file = fopen(path, "w");
    int idx;
    fprintf(file, "[\n");
    for (idx = 0; idx < size; idx++) {
        fprintf(file, "  {\"code\": %d, \"name\": \"", items[idx].code);
        writeEscaped(file, items[idx].name);
        fprintf(file, "\", \"price\": %.2f, \"inStock\": %s}", items[idx].price,
                items[idx].inStock ? "true" : "false");
        if (idx < size - 1) fprintf(file, ",");                    // لا فاصلة بعد الأخير
        fprintf(file, "\n");
    }
    fprintf(file, "]\n");
    fclose(file);
}

int main(void) {
    Product items[3] = {{101, "Mouse", 1500, 1}, {205, "Monitor 27\" 4K", 54000, 1}, {330, "Hub", 3100, 0}};
    FILE *file;
    int ch;
    exportJson("catalog.json", items, 3);
    file = fopen("catalog.json", "r");
    while ((ch = fgetc(file)) != EOF) putchar(ch);
    fclose(file);
    return 0;
}
```

>_ نتيجة التنفيذ

```
[
  {"code": 101, "name": "Mouse", "price": 1500.00, "inStock": true},
  {"code": 205, "name": "Monitor 27\" 4K", "price": 54000.00, "inStock": true},
  {"code": 330, "name": "Hub", "price": 3100.00, "inStock": false}
]
```

المخطط الإنسيابي (Organigramme)



تمرين 11.24 كشف البريد المكرر في ملف الزبائن

علم البيانات ★★ متوسط

ملف التسجيلات `signups.txt` يحتوي على بريد إلكتروني في كل سطر، بعضها مكرر بحروف مختلفة الحجم (`Amina@Mail.dz` و `amina@mail.dz`). اقرأ الملف، وحّد كل بريد إلى الحروف الصغيرة، واكتب البريد الفريد فقط في `unique.txt` مع عدّ المكررات.

خطوات الحل

1) `normalize`: تحويل إلى حروف صغيرة في مكانها.
2) نحتفظ بالبريد المرئي سابقًا في جدول ونبحث فيه.
3) نكتب في الملف الجديد عند أول ظهور فقط.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAXE 50

void normalize(char *text) {
    for (; *text != '\0'; text++)
        if (*text >= 'A' && *text <= 'Z') *text = *text - 'A' + 'a';
}

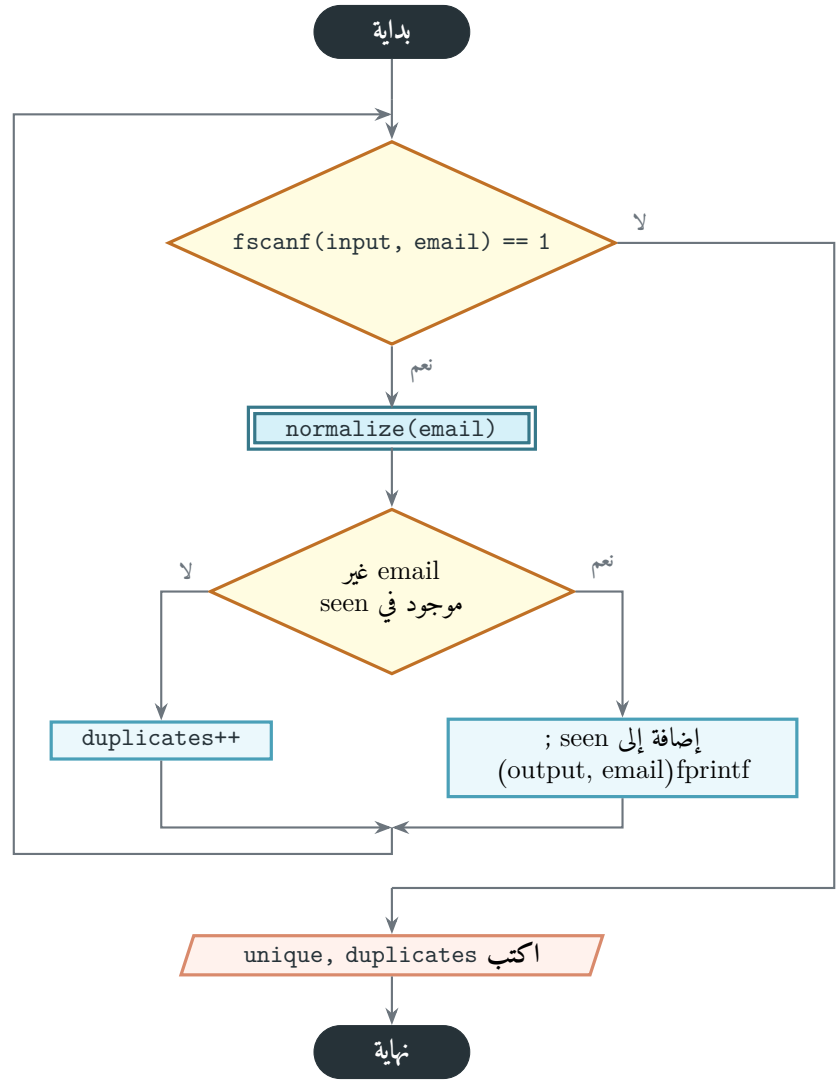
int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

int main(void) {
    FILE *input = fopen("signups.txt", "w"), *output;
    char seen[MAXE][40], email[40];
    int nb = 0, duplicates = 0, idx, letter;
    fprintf(input, "amina@mail.dz\nkarim@shop.dz\nAmina@Mail.dz\nlina@web.dz\nKARIM@shop.dz\n");
    fprintf(input, "omar@mail.dz\namina@mail.dz\n");
    fclose(input);
    input = fopen("signups.txt", "r");
    output = fopen("unique.txt", "w");
    while (fscanf(input, "%39s", email) == 1) {
        normalize(email);
        for (idx = 0; idx < nb && !sameText(seen[idx], email); idx++) {}
        if (idx == nb && nb < MAXE) {                    // أول ظهور
            for (letter = 0; (seen[nb][letter] = email[letter]) != '\0'; letter++) {}
            nb++;
            fprintf(output, "%s\n", email);
        } else {
            duplicates++;
        }
    }
    fclose(input);
    fclose(output);
    printf("%d unique, %d duplicates removed\n", nb, duplicates);
    output = fopen("unique.txt", "r");
    while (fscanf(output, "%39s", email) == 1) printf("  %s\n", email);
    fclose(output);
    return 0;
}
```

نتيجة التنفيذ

```
4 unique, 3 duplicates removed
  amina@mail.dz
  karim@shop.dz
  lina@web.dz
  omar@mail.dz
```

المخطط الإنسيابي (Organigramme)



تمرين 11.25 مدرج التقييمات من ملف

علم البيانات متوسط

ملف `ratings.txt` يحتوي على تقييمات من 1 إلى 5 (قيمة في كل سطر)، وقد يحتوي على قيم غير صالحة (0، 7، أو نص). اقرأ الملف، وتجاهل القيم غير الصالحة مع عدّها، ثم اكتب تقريرًا في `report.txt` (المدرج بالنجوم والمتوسط) واعرضه.

خطوات الحل

1) إذا فشل `fscanf` في قراءة عدد (نص)، تخطى الكلمة بـ `fscanf(file, "%*s")`.
2) `counts[value]` لـ `value` من 1 إلى 5.

البرنامج بلغة C

```c
#include <stdio.h>

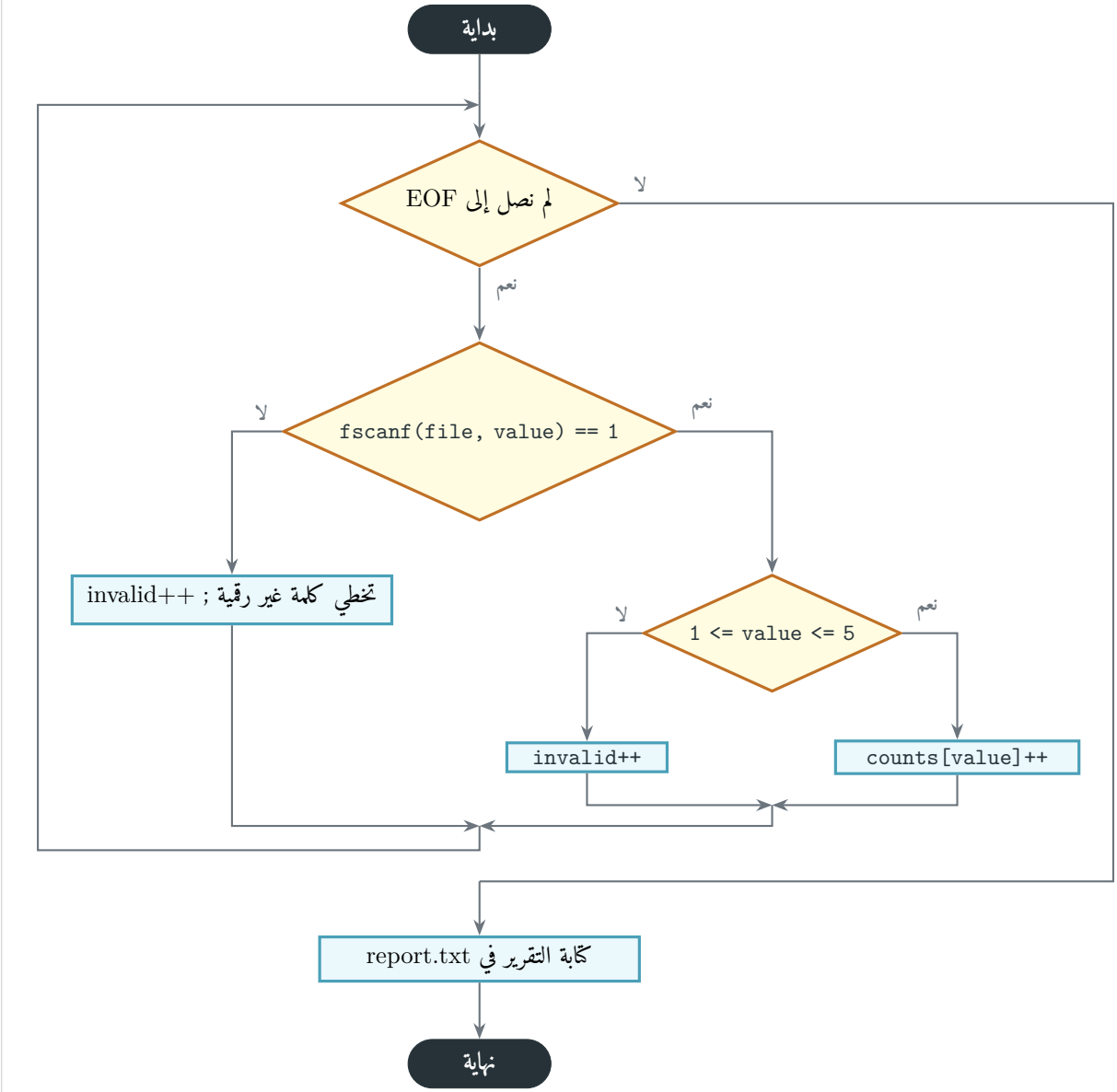
int main(void) {
    FILE *file = fopen("ratings.txt", "w");
    int counts[6] = {0}, value, invalid = 0, total = 0, sum = 0, star, mark, status;
    int ch;
```

```c
    fprintf(file, "5\n4\n5\n3\nabc\n5\n2\n4\n7\n5\n1\n4\n0\n5\n4\n");
    fclose(file);
    file = fopen("ratings.txt", "r");
    while ((status = fscanf(file, "%d", &value)) != EOF) {
        if (status == 1) {
            if (value >= 1 && value <= 5) { counts[value]++; total++; sum += value; }
            else invalid++;
        } else {
            fscanf(file, "%*s");                    // تخطى كلمة غير رقمية
            invalid++;
        }
    }
    fclose(file);
    file = fopen("report.txt", "w");
    for (star = 5; star >= 1; star--) {
        fprintf(file, "%d | ", star);
        for (mark = 0; mark < counts[star]; mark++) fputc('*', file);
        fprintf(file, " %d\n", counts[star]);
    }
    fprintf(file, "valid %d, invalid %d, mean %.2f\n", total, invalid, (double)sum / total);
    fclose(file);
    file = fopen("report.txt", "r");
    while ((ch = fgetc(file)) != EOF) putchar(ch);
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
5 | ***** 5
4 | **** 4
3 | * 1
2 | * 1
1 | * 1
valid 12, invalid 3, mean 3.92
```

المخطط الإنسيابي (Organigramme)



تمرين 11.26 إعادة بناء الرصيد من دفتر العمليات

تجارة إلكترونية ★★ متوسط

تسجل المحفظة كل عملية في ملف يُضاف إليه فقط (`ledger.txt`) بالصيغة `type amount` (`D` إيداع، `W` سحب، `F` رسوم). لا نخزن الرصيد أبدًا: نعيد حسابه **بإعادة تشغيل** الدفتر من البداية. أضف عمليات على دفعتين ثم أعد بناء الرصيد بعد كل دفعة.

خطوات الحل

1) `appendOp` تفتح بـ `"a"` وتكتب سطرًا.
2) `replay` تقرأ كل الأسطر وتطبقها بالترتيب.
3) الرصيد مشتق دائمًا من الدفتر: لا تناقض ممكن.

البرنامج بلغة C

```c
#include <stdio.h>

void appendOp(char type, double amount) {
    FILE *file = fopen("ledger.txt", "a");
    fprintf(file, "%c %.2f\n", type, amount);
    fclose(file);
}

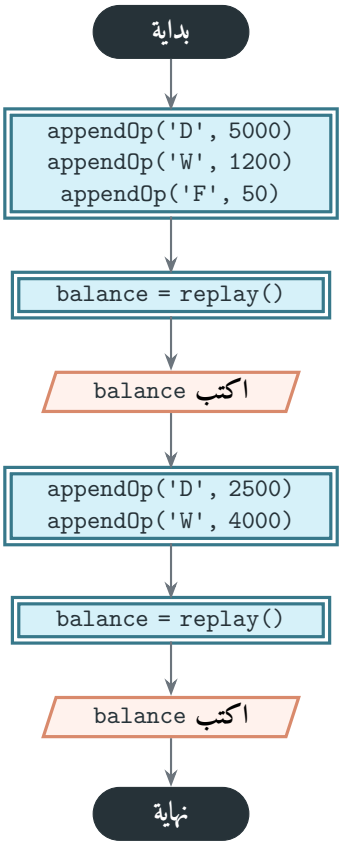
double replay(int *count) {
    FILE *file = fopen("ledger.txt", "r");
    char type;
    double amount, balance = 0;
    *count = 0;
    if (file == NULL) return 0;
    while (fscanf(file, " %c %lf", &type, &amount) == 2) {
        if (type == 'D') balance += amount;
        else balance -= amount;                        // W أو F
        (*count)++;
    }
    fclose(file);
    return balance;
}

int main(void) {
    FILE *file = fopen("ledger.txt", "w");          // دفتر جديد
    int count;
    double balance;
    fclose(file);
    appendOp('D', 5000); appendOp('W', 1200); appendOp('F', 50);
    balance = replay(&count);
    printf("After %d ops: %.2f DA\n", count, balance);
    appendOp('D', 2500); appendOp('W', 4000);
    balance = replay(&count);
    printf("After %d ops: %.2f DA\n", count, balance);
    return 0;
}
```

نتيجة التنفيذ

```
After 3 ops: 3750.00 DA
After 5 ops: 2250.00 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.27 تشفير ملف الزبائن بمفتاح XOR

تجارة إلكترونية ★★ متوسط

قبل إرسال ملف الزبائن نشفّره بعملية **XOR** مع مفتاح يتكرر: `out = in ^ key[pos % keyLen]`. اكتب دالة `xorFile(source, target, key)` تعمل بالملفات الثنائية، شفّر الملف ثم فك تشفيره بنفس الدالة وتحقق من التطابق.

خطوات الحل

1) XOR مرتين بنفس المفتاح يعيد القيمة الأصلية.
2) نقرأ ونكتب بايتًا ببايت بالوضعين `"rb"` و `"wb"`.
3) التحقق: مقارنة الملفين بايتًا ببايت.

البرنامج بلغة C

```c
#include <stdio.h>

void xorFile(const char *source, const char *target, const char *key) {
    FILE *input = fopen(source, "rb"), *output = fopen(target, "wb");
    int ch, keyLen = 0;
    long pos = 0;
    while (key[keyLen] != '\0') keyLen++;
    while ((ch = fgetc(input)) != EOF)
        fputc(ch ^ key[pos++ % keyLen], output);      // المفتاح يتكرر
    fclose(input);
    fclose(output);
}

int compareFiles(const char *first, const char *second) {
```

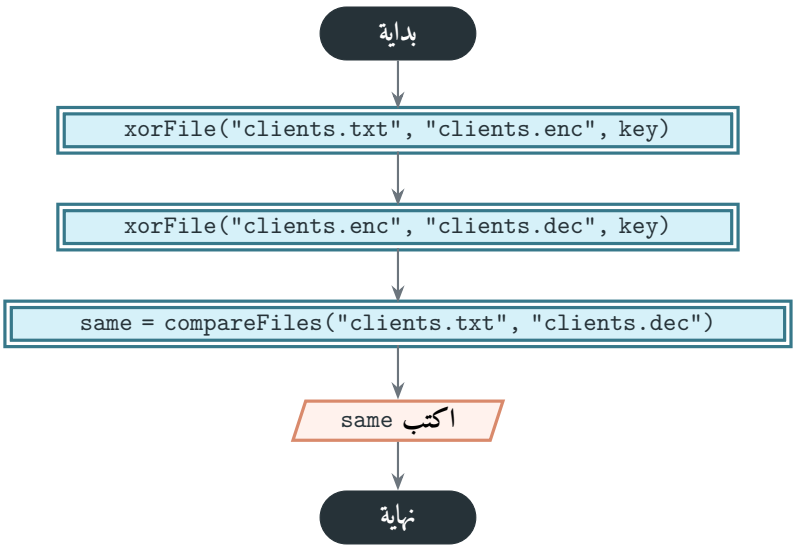
```c
    FILE *fileA = fopen(first, "rb"), *fileB = fopen(second, "rb");
    int chA, chB, same = 1;
    do {
        chA = fgetc(fileA);
        chB = fgetc(fileB);
        if (chA != chB) same = 0;
    } while (chA != EOF && chB != EOF && same);
    fclose(fileA);
    fclose(fileB);
    return same;
}

int main(void) {
    FILE *file = fopen("clients.txt", "w");
    int ch, shown = 0;
    fprintf(file, "507;Amina;0550123456\n508;Karim;0661987654\n");
    fclose(file);
    xorFile("clients.txt", "clients.enc", "K3y!");
    printf("Encrypted bytes (hex): ");
    file = fopen("clients.enc", "rb");
    while ((ch = fgetc(file)) != EOF && shown++ < 12) printf("%02X ", ch);
    fclose(file);
    xorFile("clients.enc", "clients.dec", "K3y!");    // نفس العملية تفك التشفير
    printf("...\nDecrypted identical to original: %s\n", compareFiles("clients.txt", "clients.dec") ? "yes"
      ↪ : "no");
    return 0;
}
```

نتيجة التنفيذ

```
Encrypted bytes (hex): 7E 03 4E 1A 0A 5E 10 4F 2A 08 49 14 ...
Decrypted identical to original: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 11.28 أطول سطر في ملف الأوصاف

علم البيانات ★★ متوسط

لضبط عرض عمود الأوصاف في واجهة الإدارة، ابحث عن **أطول سطر** في الملف، ورقمه، وطوله، ومتوسط أطوال الأسطر. اقرأ حرفًا بحرف لكي لا يُقيدك حجم مخزن `fgets`.

خطوات الحل

1) عداد طول السطر الحالي يرجع إلى 0 عند كل `'\n'`.
2) عند نهاية السطر: نقارن بالأطول ونجمع للمتوسط.
3) نعالج السطر الأخير إذا لم ينته بـ `'\n'`.

البرنامج بلغة C

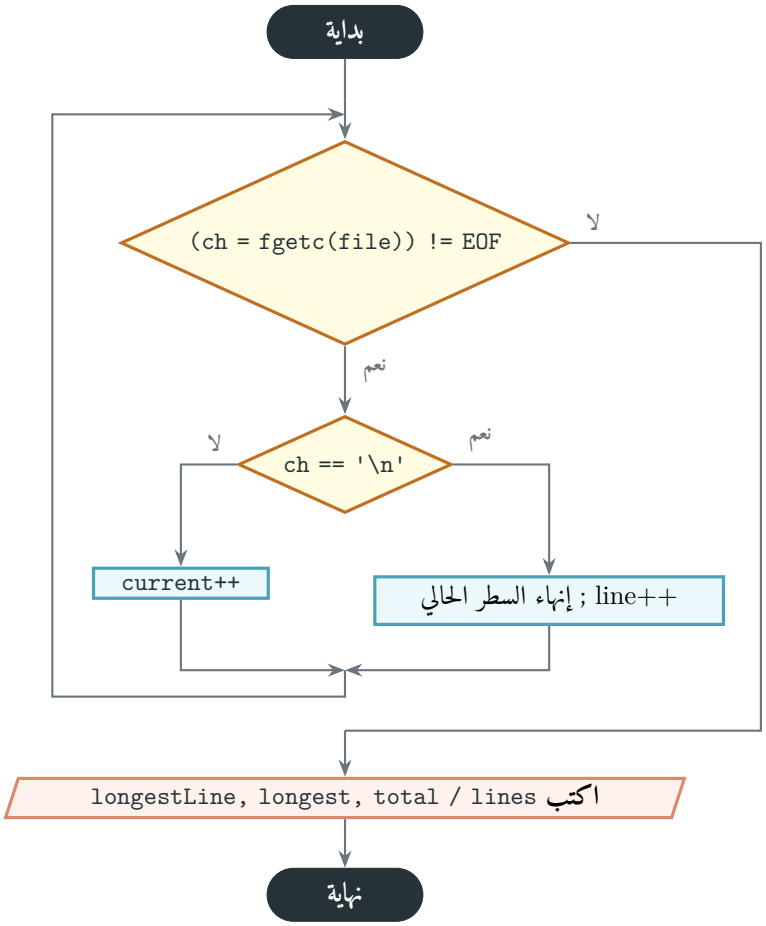
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("descriptions.txt", "w");
    int ch, current = 0, line = 0, longest = -1, longestLine = 0, total = 0;
    fprintf(file, "Wireless mouse 2.4 GHz\nMechanical keyboard with blue switches and RGB lighting\n");
    fprintf(file, "USB-C cable 1m\n27 inch 4K IPS monitor, HDR10, 60 Hz, adjustable stand\nHub");
    fclose(file);
    file = fopen("descriptions.txt", "r");
    while (1) {
        ch = fgetc(file);
        if (ch == '\n' || (ch == EOF && current > 0)) {   // نهاية سطر
            line++;
            total += current;
            if (current > longest) { longest = current; longestLine = line; }
            current = 0;
        } else if (ch != EOF) {
            current++;
        }
        if (ch == EOF) break;
    }
    fclose(file);
    printf("%d lines, longest is line %d (%d chars), average %.1f\n",
           line, longestLine, longest, (double)total / line);
    return 0;
}
```

نتيجة التنفيذ

```
5 lines, longest is line 2 (55 chars), average 29.6
```

المخطط الإنسيابي (Organigramme)



تمرين 11.29 آخر أسطر السجل بمخزن دائري

علم البيانات ★★ متوسط

اكتب أمرًا يشبه `tail -n 4`: يعرض آخر 4 أسطر من سجل طويل دون تحميله كله، بالاحتفاظ بآخر 4 أسطر فقط في مخزن دائري من الأسطر أثناء القراءة.

خطوات الحل

1) `ring[NBL][LEN]` و `next` فهرس الكتابة الدائري.
2) كل سطر جديد يُكتب في `ring[next]` ثم `next = (next + 1) % NBL`.
3) في النهاية نطبع ابتداءً من `next` (الأقدم) إذا امتلأ المخزن.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBL 4
#define LEN 60

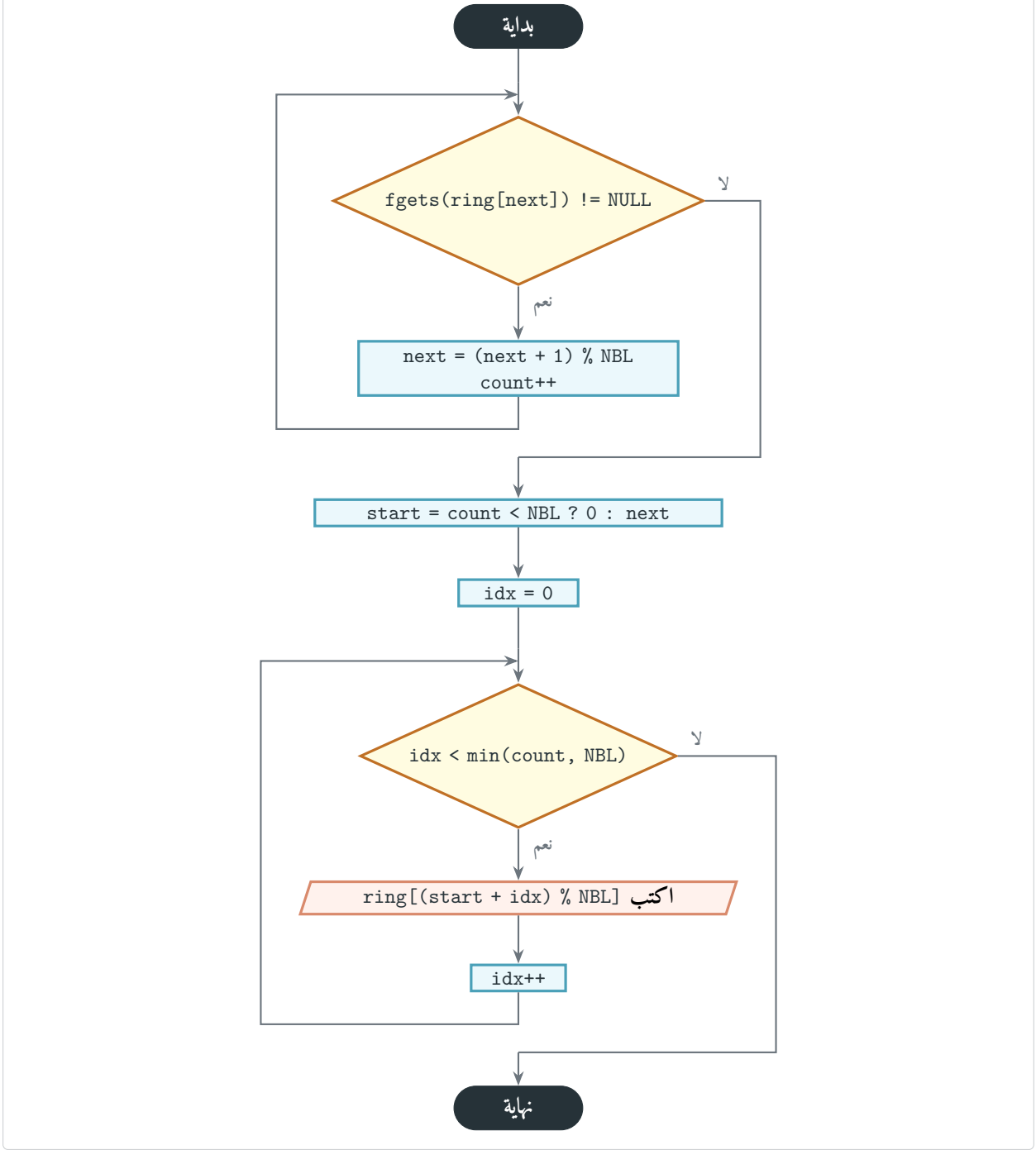
int main(void) {
    FILE *file = fopen("access.log", "w");
    char ring[NBL][LEN];
    int hour, next = 0, count = 0, idx, start, shown;
    for (hour = 0; hour < 24; hour++)
        fprintf(file, "%02d:00 visits=%d orders=%d\n", hour, 100 + hour * 13 % 70, hour * 7 % 11);
```

```
    fclose(file);
    file = fopen("access.log", "r");
    while (fgets(ring[next], LEN, file) != NULL) {
        next = (next + 1) % NBL;                  // نكتب فوق الأقدم
        count++;
    }
    fclose(file);
    start = count < NBL ? 0 : next;
    shown = count < NBL ? count : NBL;
    printf("Last %d of %d lines:\n", shown, count);
    for (idx = 0; idx < shown; idx++) printf("%s", ring[(start + idx) % NBL]);
    return 0;
}
```

نتيجة التنفيذ

```
Last 4 of 24 lines:
20:00 visits=150 orders=8
21:00 visits=163 orders=4
22:00 visits=106 orders=0
23:00 visits=119 orders=7
```

المخطط الإنسيابي (Organigramme)



تمرين 11.30 مقارنة نسختين من قائمة الأسعار

تجارة إلكترونية ★★ متوسط

قارن ملفي الأسعار `prices_v1.txt` و `prices_v2.txt` **سطراً بسطر** (مثل `diff` مبسط)، واعرض رقم كل سطر مختلف مع النسختين، والأسطر الزائدة في أحد الملفين.

خطوات الحل

1) نقرأ سطراً من كل ملف في كل دورة.
2) `sameLine` تقارن السلسلتين.

3) إذا انتهى أحد الملفين قبل الآخر: الأسطر الباقية "مضافة" أو "محذوفة".

البرنامج بلغة C

```c
#include <stdio.h>

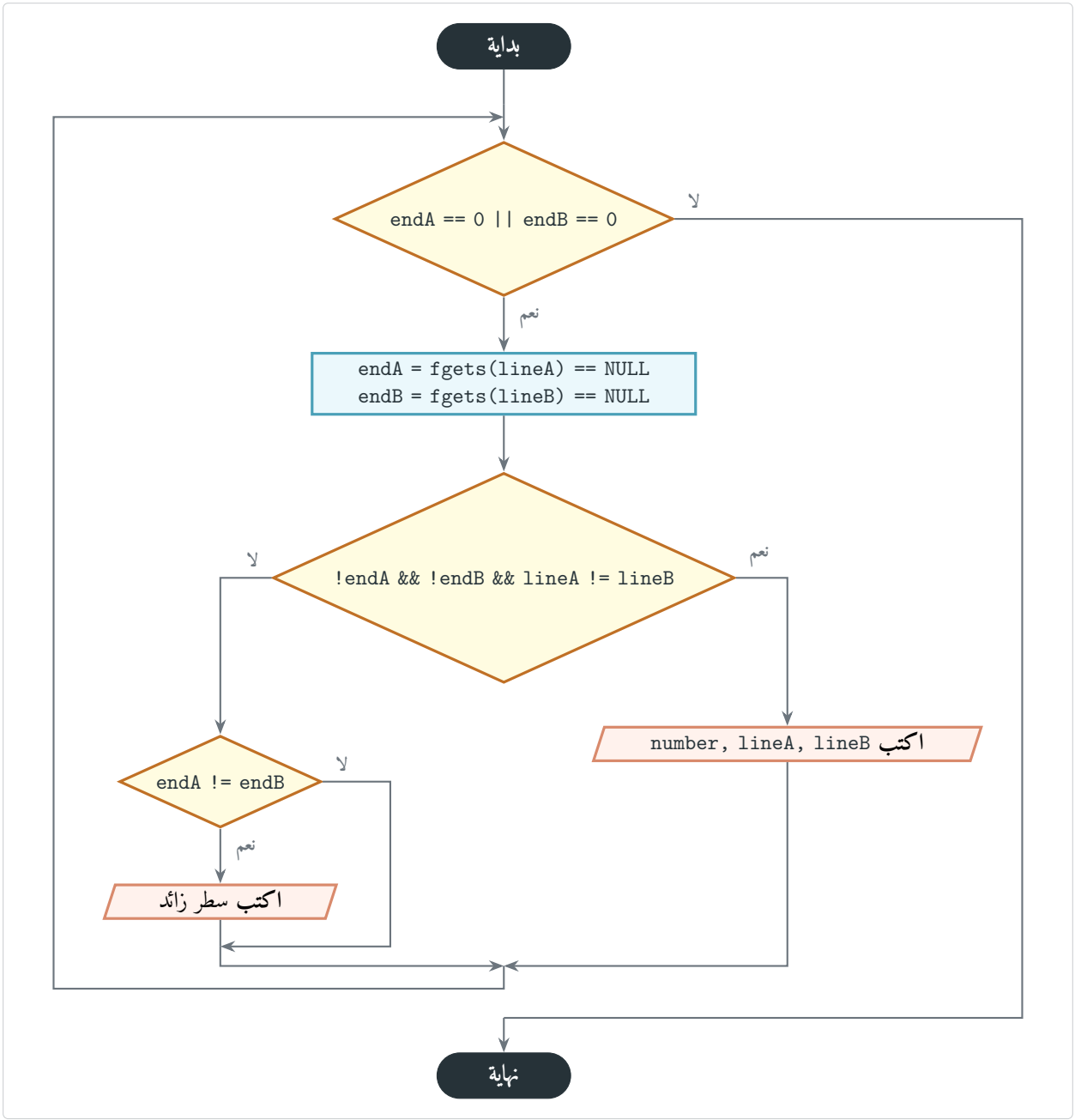
int sameLine(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

int main(void) {
    FILE *fileA = fopen("prices_v1.txt", "w"), *fileB = fopen("prices_v2.txt", "w");
    char lineA[80], lineB[80];
    int endA = 0, endB = 0, number = 0, changes = 0;
    fprintf(fileA, "Mouse 1500\nKeyboard 3200\nMonitor 32000\nCable 450\n");
    fprintf(fileB, "Mouse 1500\nKeyboard 2990\nMonitor 32000\nCable 500\nHub 3100\nWebcam 4100\n");
    fclose(fileA); fclose(fileB);
    fileA = fopen("prices_v1.txt", "r");
    fileB = fopen("prices_v2.txt", "r");
    while (1) {
        endA = fgets(lineA, 80, fileA) == NULL;
        endB = fgets(lineB, 80, fileB) == NULL;
        if (endA && endB) break;
        number++;
        if (!endA && !endB && !sameLine(lineA, lineB)) {
            printf("line %d changed:\n  - %s  + %s", number, lineA, lineB);
            changes++;
        } else if (endA && !endB) {
            printf("line %d added:   + %s", number, lineB);
            changes++;
        } else if (!endA && endB) {
            printf("line %d removed: - %s", number, lineA);
            changes++;
        }
    }
    fclose(fileA); fclose(fileB);
    printf("%d difference(s)\n", changes);
    return 0;
}
```

نتيجة التنفيذ

```
line 2 changed:
  - Keyboard 3200
  + Keyboard 2990
line 4 changed:
  - Cable 450
  + Cable 500
line 5 added:   + Hub 3100
line 6 added:   + Webcam 4100
4 difference(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 11.31 تقرير بأعمدة ثابتة العرض من CSV

علم البيانات ★★ متوسط

حوّل ملف `team.csv` (`name,city,sales`) إلى تقرير نصي بأعمدة **ثابتة العرض** في `team_report.txt` مع إطار وسطر مجموع، حيث يُقص الاسم الطويل إلى 10 حروف مع `.` في النهاية.

خطوات الحل

1) `%-10s` محاذاة يسارًا و `%10.2f` محاذاة يمينًا.
2) القص: نسخ 9 حروف ونضع `.` إذا كان الاسم أطول من 10.
3) نفكك كل سطر يدويًا عند الفواصل.

البرنامج بلغة C

```c
#include <stdio.h>

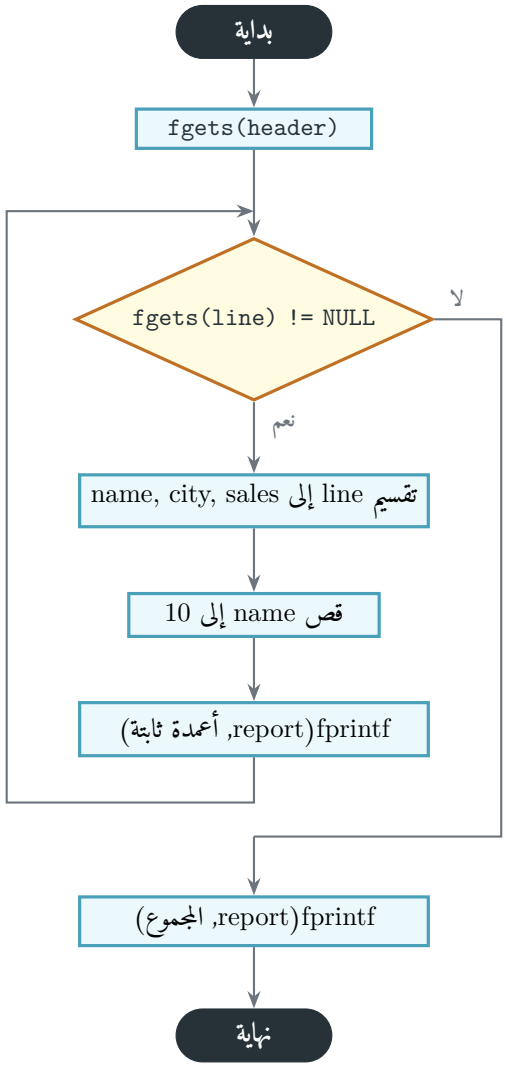
double toDouble(const char *text) {
    double value = 0, scale = 1;
    while (*text >= '0' && *text <= '9') value = value * 10 + (*text++ - '0');
    if (*text == '.') for (text++; *text >= '0' && *text <= '9'; text++) value += (*text - '0') * (scale /= 10);
    return value;
}

int main(void) {
    FILE *input = fopen("team.csv", "w"), *report;
    char line[100], *fields[3], shortName[12];
    double total = 0;
    int pos, count, len, ch;
    fprintf(input, "name,city,sales\nRania Belkacem,Oran,820000\nHakim,Algiers,640000.5\n");
    fprintf(input, "Selma Ait-Mohand,Tizi Ouzou,910000\nOmar,Setif,505000\n");
    fclose(input);
    input = fopen("team.csv", "r");
    report = fopen("team_report.txt", "w");
    fgets(line, 100, input);                                    // العناوين
    fprintf(report, "+------------+------------+--------------+\n");
    fprintf(report, "| %-10s | %-10s | %12s |\n", "Name", "City", "Sales");
    fprintf(report, "+------------+------------+--------------+\n");
    while (fgets(line, 100, input) != NULL) {
        fields[0] = line;
        for (pos = 0, count = 1; line[pos] != '\n' && line[pos] != '\0'; pos++)
            if (line[pos] == ',') { line[pos] = '\0'; fields[count++] = line + pos + 1; }
        line[pos] = '\0';
        for (len = 0; fields[0][len] != '\0'; len++) {}
        for (pos = 0; pos < 10 && pos < len; pos++) shortName[pos] = fields[0][pos];
        if (len > 10) shortName[9] = '.';                       // قص الاسم الطويل
        shortName[pos] = '\0';
        fprintf(report, "| %-10s | %-10.10s | %12.2f |\n", shortName, fields[1], toDouble(fields[2]));
        total += toDouble(fields[2]);
    }
    fprintf(report, "+------------+------------+--------------+\n");
    fprintf(report, "| %-23s | %12.2f |\n", "TOTAL", total);
    fclose(input);
    fclose(report);
    report = fopen("team_report.txt", "r");
    while ((ch = fgetc(report)) != EOF) putchar(ch);
    fclose(report);
    return 0;
}
```

نتيجة التنفيذ

```
+------------+------------+--------------+
| Name       | City       |        Sales |
+------------+------------+--------------+
| Rania Bel. | Oran       |    820000.00 |
| Hakim      | Algiers    |    640000.50 |
| Selma Ait. | Tizi Ouzou |    910000.00 |
| Omar       | Setif      |    505000.00 |
+------------+------------+--------------+
| TOTAL                   |   2875000.50 |
```

المخطط الإنسيابي (Organigramme)



تمرين 11.32 البحث في السجل دون اعتبار حجم الحروف

علم البيانات ★★ متوسط

اكتب أمرًا يشبه `grep -in`: يبحث عن كلمة في ملف سجل **دون اعتبار حجم الحروف**، ويعرض كل سطر مطابق مسبوقًا برقمه، وعدد الأسطر المطابقة في النهاية.

خطوات الحل

1) `lower(ch)` يحول الحرف إلى صغير للمقارنة فقط (السطر يُعرض كما هو).
2) `containsIgnoreCase(line, word)`: لكل موضع نقارن الحروف بعد التحويل.

البرنامج بلغة C

```c
#include <stdio.h>

char lower(char ch) { return (ch >= 'A' && ch <= 'Z') ? ch - 'A' + 'a' : ch; }

int containsIgnoreCase(const char *line, const char *word) {
    const char *start, *left, *right;
    for (start = line; *start != '\0'; start++) {
        for (left = start, right = word; *right != '\0' && lower(*left) == lower(*right); left++, right++) {}
```

```
        if (*right == '\0') return 1;
    }
    return 0;
}

int main(void) {
    FILE *file = fopen("support.log", "w");
    char word[30], line[120];
    int number = 0, matches = 0;
    fprintf(file, "#101 customer asks about delivery time\n#102 REFUND requested: item broken\n");
    fprintf(file, "#103 password reset\n#104 partial Refund approved for #102\n#105 invoice copy\n");
    fprintf(file, "#106 refunds policy question\n");
    fclose(file);
    scanf("%29s", word);
    file = fopen("support.log", "r");
    while (fgets(line, 120, file) != NULL) {
        number++;
        if (containsIgnoreCase(line, word)) {
            printf("%d: %s", number, line);
            matches++;
        }
    }
    fclose(file);
    printf("%d matching line(s) for \"%s\"\n", matches, word);
    return 0;
}
```

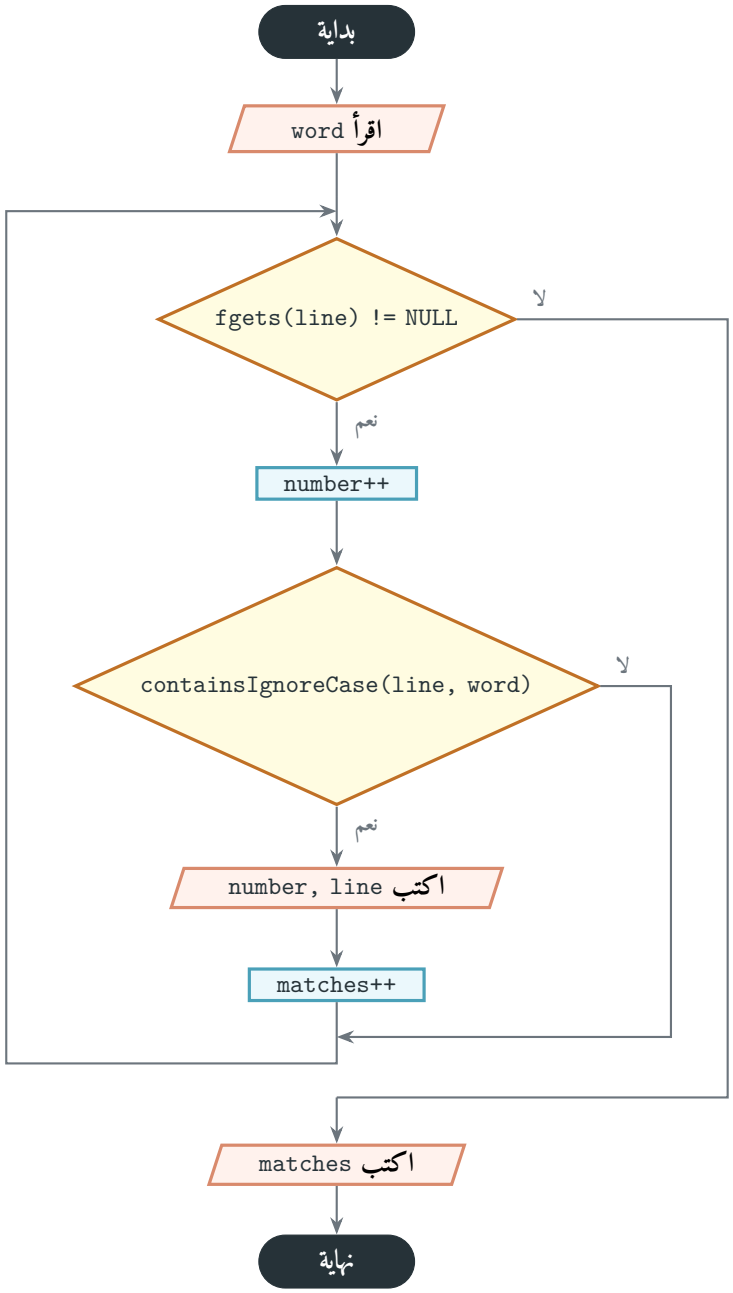
المدخلات

```
refund
```

نتيجة التنفيذ

```
2: #102 REFUND requested: item broken
4: #104 partial Refund approved for #102
6: #106 refunds policy question
3 matching line(s) for "refund"
```

المخطط الإنسيابي (Organigramme)



تمرين 11.33 قراءة مصفوفة من ملف وكتابة منقولتها

علم البيانات ★★ متوسط

ملف `matrix.txt` يبدأ بعدد الأسطر والأعمدة ثم القيم (مبيعات منتج × شهر). اقرأ المصفوفة، اكتب **منقولتها** في `transposed.txt` بنفس الصيغة، ثم أعد قراءة الملف الجديد للتحقق وعرضه.

خطوات الحل

1) الصيغة: `rows cols` ثم `rows` سطرًا من `cols` قيمة.
2) المنقولة: `cols` سطرًا من `rows` قيمة: `matrix[col][row]`.
3) دالة `printFile` تعرض أي ملف نصي.

البرنامج بلغة C

```c
#include <stdio.h>

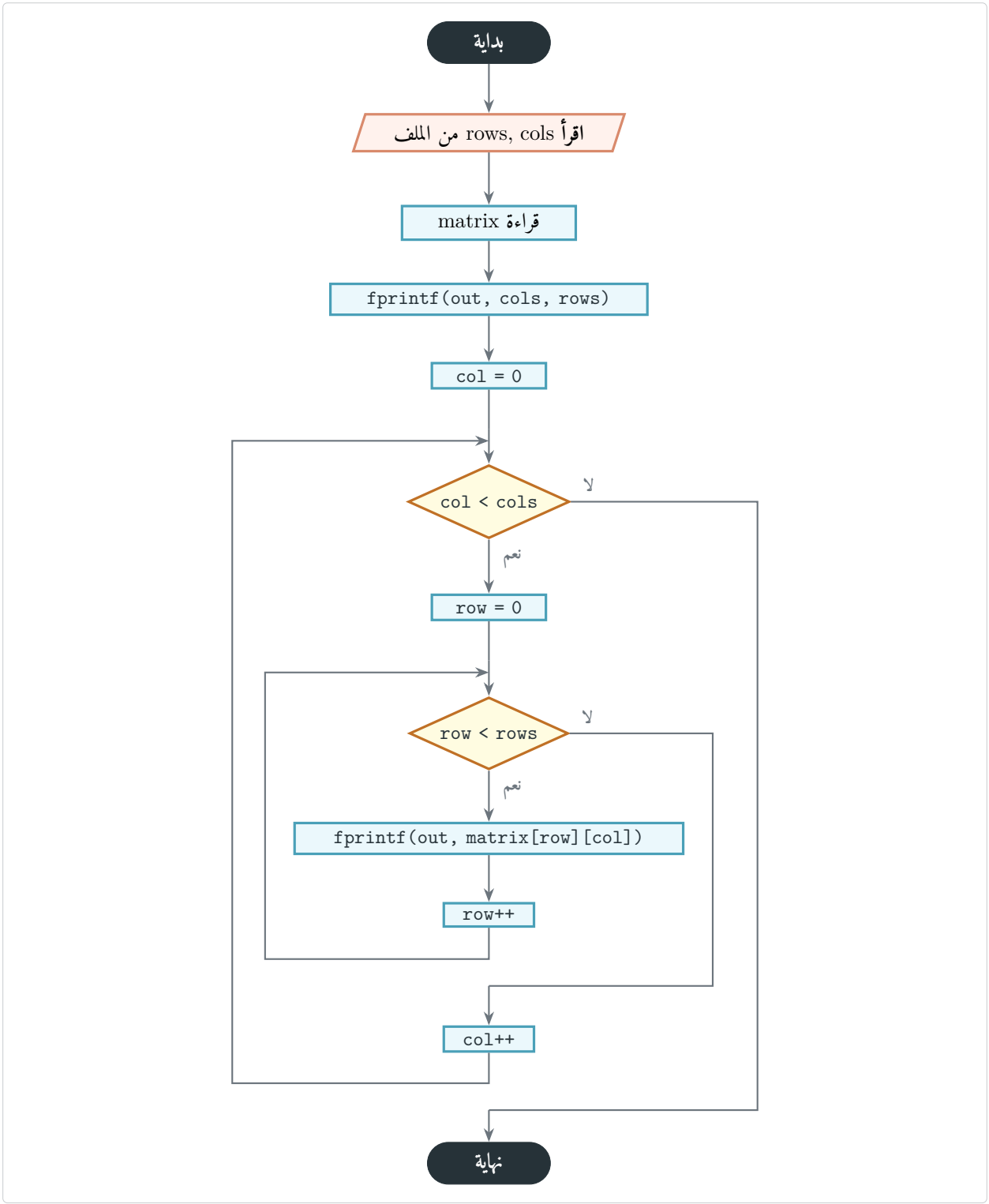
void printFile(const char *path) {
    FILE *file = fopen(path, "r");
    int ch;
    while ((ch = fgetc(file)) != EOF) putchar(ch);
    fclose(file);
}

int main(void) {
    FILE *file = fopen("matrix.txt", "w");
    int matrix[10][10], rows, cols, row, col;
    fprintf(file, "3 4\n120 95 140 180\n80 60 75 90\n200 210 190 250\n");
    fclose(file);
    file = fopen("matrix.txt", "r");
    fscanf(file, "%d %d", &rows, &cols);
    for (row = 0; row < rows; row++)
        for (col = 0; col < cols; col++) fscanf(file, "%d", &matrix[row][col]);
    fclose(file);
    file = fopen("transposed.txt", "w");
    fprintf(file, "%d %d\n", cols, rows);              // الأبعاد تتبادل
    for (col = 0; col < cols; col++) {
        for (row = 0; row < rows; row++) fprintf(file, "%d%c", matrix[row][col], row < rows - 1 ? ' ' :
        ↪ '\n');
    }
    fclose(file);
    printf("--- transposed.txt ---\n");
    printFile("transposed.txt");
    return 0;
}
```

نتيجة التنفيذ

```
--- transposed.txt ---
4 3
120 80 200
95 60 210
140 75 190
180 90 250
```

المخطط الإنسيابي (Organigramme)



تمرين 11.34 عدد الزوار المختلفين في سجل الدخول

علم البيانات

متوسط

كل سطر في سجل الدخول `hits.log` يحتوي على عنوان IP وصفحة. احسب عدد **الزوار المختلفين** (عناوين IP فريدة)، وعدد الزيارات لكل زائر، والزائر الأكثر نشاطًا.

خطوات الحل

1) `fscanf(file, "%s %s", ip, page)`.
2) جدول عناوين فريدة مع عدادات (بحث يدوي).

البرنامج بلغة C

```c
#include <stdio.h>

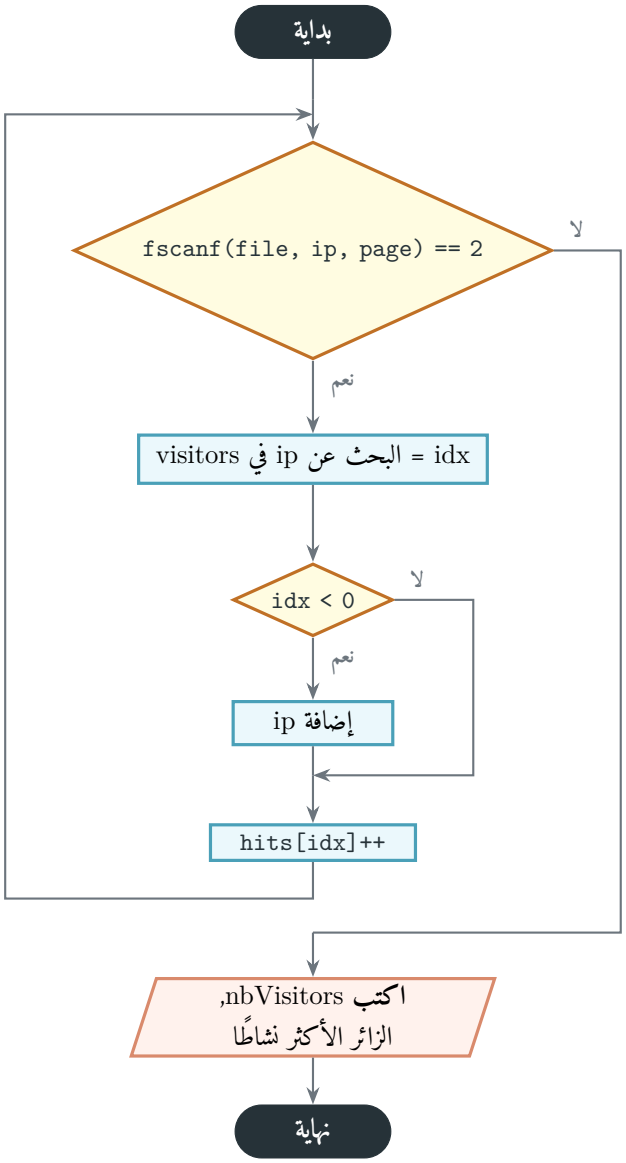
int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

int main(void) {
    FILE *file = fopen("hits.log", "w");
    char visitors[30][20], ip[20], page[40];
    int hits[30], nb = 0, idx, top = 0, total = 0, letter;
    fprintf(file, "41.200.3.7 /home\n105.98.1.20 /phones\n41.200.3.7 /phones\n197.112.5.9 /home\n");
    fprintf(file, "41.200.3.7 /cart\n105.98.1.20 /cart\n41.200.3.7 /checkout\n154.121.0.4 /home\n");
    fclose(file);
    file = fopen("hits.log", "r");
    while (fscanf(file, "%19s %39s", ip, page) == 2) {
        for (idx = 0; idx < nb && !sameText(visitors[idx], ip); idx++) {}
        if (idx == nb) {                                  // زائر جديد
            for (letter = 0; (visitors[nb][letter] = ip[letter]) != '\0'; letter++) {}
            hits[nb++] = 0;
        }
        hits[idx]++;
        total++;
    }
    fclose(file);
    for (idx = 0; idx < nb; idx++) {
        printf("%-12s %d hits\n", visitors[idx], hits[idx]);
        if (hits[idx] > hits[top]) top = idx;
    }
    printf("%d hits from %d unique visitors; most active %s\n", total, nb, visitors[top]);
    return 0;
}
```

نتيجة التنفيذ

```
41.200.3.7   4 hits
105.98.1.20  2 hits
197.112.5.9  1 hits
154.121.0.4  1 hits
8 hits from 4 unique visitors; most active 41.200.3.7
```

المخطط الإنسيابي (Organigramme)



تمرين 11.35 المجموع اليومي من سجل مؤرخ

علم البيانات ★★ متوسط

سجل المبيعات مرتب زمنيًا بأسطر `YYYY-MM-DD HH:MM amount`. احسب **مجموع كل يوم** بمرور واحد: نجمع ما دام التاريخ نفسه، وعند تغيره نكتب سطر اليوم السابق في `daily.txt`. اعرض النتيجة واليوم الأفضل.

خطوات الحل

1) نقارن تاريخ السطر الحالي بالتاريخ المحفوظ `currentDay`.
2) عند التغير: نكتب `currentDay total` ونعيد الضبط.
3) لا ننسى كتابة اليوم الأخير بعد الحلقة.

البرنامج بلغة C

```c
#include <stdio.h>

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

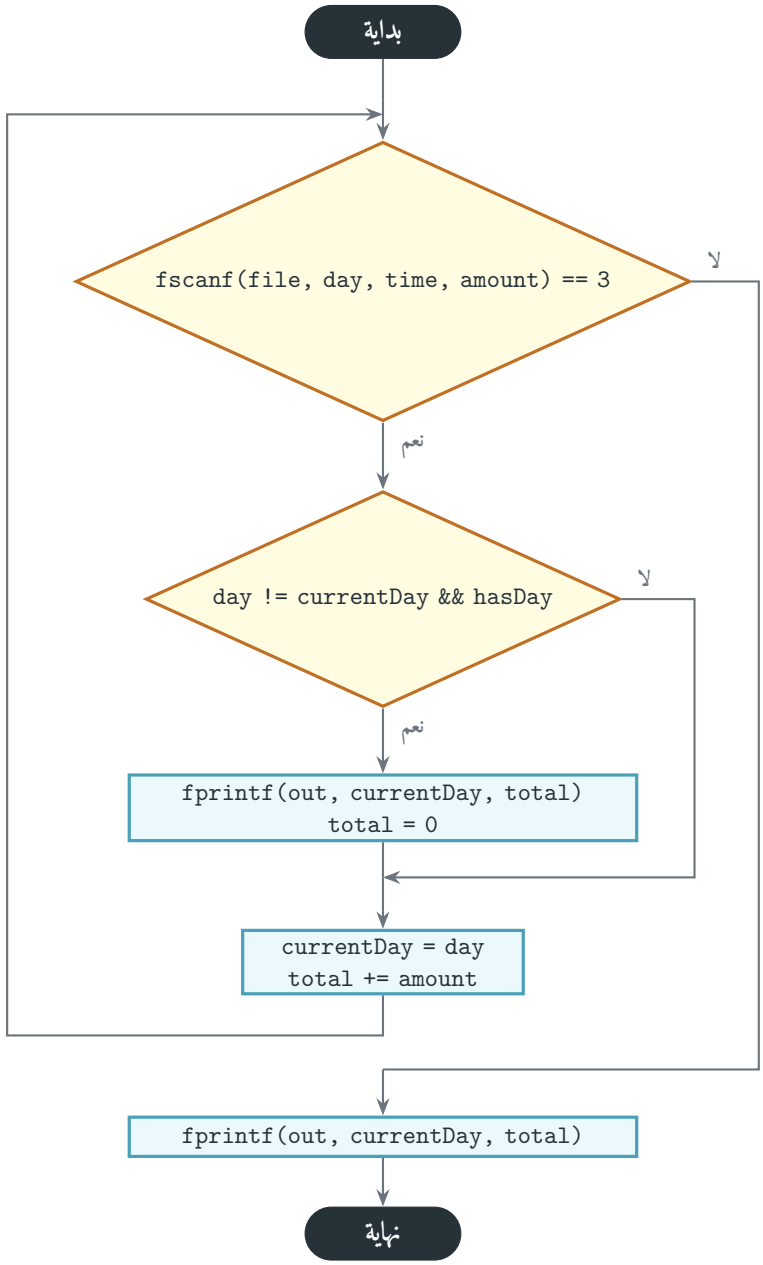
void copyText(char *target, const char *source) { while ((*target++ = *source++) != '\0') {} }

int main(void) {
    FILE *file = fopen("timeline.log", "w"), *out;
    char day[12], time[8], currentDay[12] = "", bestDay[12] = "";
    double amount, total = 0, best = -1;
    int hasDay = 0;
    fprintf(file, "2026-11-01 09:14 1200\n2026-11-01 13:40 4500\n2026-11-01 20:02 890\n");
    fprintf(file, "2026-11-02 10:30 32000\n2026-11-03 08:05 450\n2026-11-03 18:45 3100\n2026-11-03 22:10
      ↪ 2600\n");
    fclose(file);
    file = fopen("timeline.log", "r");
    out = fopen("daily.txt", "w");
    while (fscanf(file, "%11s %7s %lf", day, time, &amount) == 3) {
        if (hasDay && !sameText(day, currentDay)) {          // يوم جديد
            fprintf(out, "%s %.2f\n", currentDay, total);
            if (total > best) { best = total; copyText(bestDay, currentDay); }
            total = 0;
        }
        copyText(currentDay, day);
        hasDay = 1;
        total += amount;
    }
    fprintf(out, "%s %.2f\n", currentDay, total);             // اليوم الأخير
    if (total > best) { best = total; copyText(bestDay, currentDay); }
    fclose(file);
    fclose(out);
    out = fopen("daily.txt", "r");
    while (fscanf(out, "%11s %lf", day, &amount) == 2) printf("%s: %9.2f DA\n", day, amount);
    fclose(out);
    printf("Best day: %s\n", bestDay);
    return 0;
}
```

نتيجة التنفيذ

```
2026-11-01:   6590.00 DA
2026-11-02:  32000.00 DA
2026-11-03:   6150.00 DA
Best day: 2026-11-02
```

المخطط الإنسيابي (Organigramme)



تمرين 11.36 حفظ جلسة السلة واستعادتها

تجارة إلكترونية ★★ متوسط

عند إغلاق التطبيق نحفظ السلة في ملف ثنائي: رأس (رقم إصدار الصيغة وعدد العناصر) ثم العناصر. عند الفتح نستعيدها بعد التحقق من الإصدار. اكتب `saveCart` و `loadCart` واختبر الحفظ والاستعادة، ثم محاولة تحميل ملف بإصدار قديم.

خطوات الحل

1) الرأس هيكل `Header { int version; int count; }`.
2) `loadCart` ترفض الملف إذا اختلف الإصدار أو إذا كان العدد غير منطقي.
3) نكتب الرأس ثم العناصر بـ `fwrite`.

البرنامج بلغة C

```c
#include <stdio.h>

#define VERSION 2

typedef struct { int version; int count; } Header;
typedef struct { int code; int qty; double price; } Item;

void saveCart(const char *path, const Item *items, int count) {
    FILE *file = fopen(path, "wb");
    Header header = {VERSION, count};
    fwrite(&header, sizeof(Header), 1, file);          // الرأس أولًا
    fwrite(items, sizeof(Item), count, file);
    fclose(file);
}

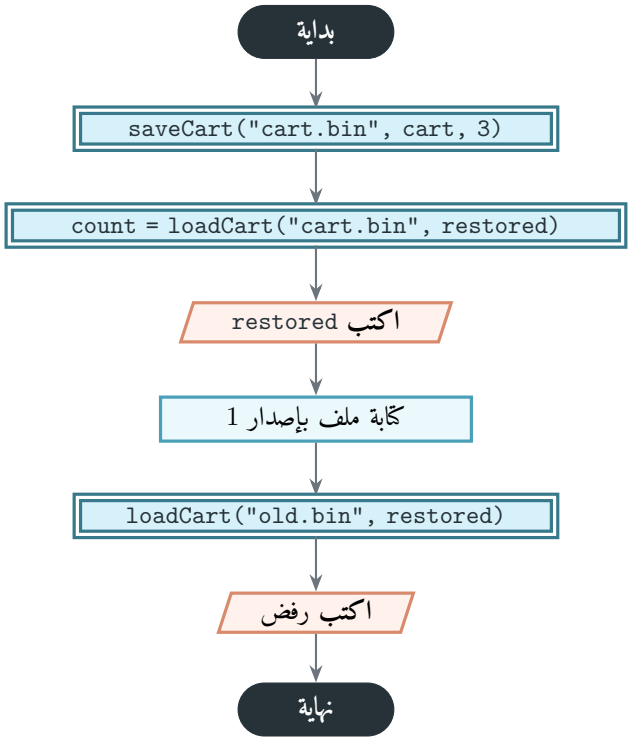
int loadCart(const char *path, Item *items, int capacity) {
    FILE *file = fopen(path, "rb");
    Header header;
    int count;
    if (file == NULL) return -1;
    if (fread(&header, sizeof(Header), 1, file) != 1 || header.version != VERSION
        || header.count < 0 || header.count > capacity) {
        fclose(file);
        return -1;                                     // ملف غير متوافق
    }
    count = (int)fread(items, sizeof(Item), header.count, file);
    fclose(file);
    return count;
}

int main(void) {
    Item cart[3] = {{101, 2, 1500}, {205, 1, 32000}, {330, 4, 450}}, restored[10];
    Header oldHeader = {1, 3};
    FILE *file;
    int count, idx;
    saveCart("cart.bin", cart, 3);
    count = loadCart("cart.bin", restored, 10);
    printf("Restored %d items:", count);
    for (idx = 0; idx < count; idx++) printf(" %d x%d", restored[idx].code, restored[idx].qty);
    printf("\n");
    file = fopen("old.bin", "wb");
    fwrite(&oldHeader, sizeof(Header), 1, file);
    fwrite(cart, sizeof(Item), 3, file);
    fclose(file);
    printf("Old format file: %s\n", loadCart("old.bin", restored, 10) < 0 ? "rejected" : "loaded");
    return 0;
}
```

نتيجة التنفيذ

```
Restored 3 items: 101 x2 205 x1 330 x4
Old format file: rejected
```

المخطط الإنسيابي (Organigramme)



تمرين 11.37 ضغط ملف بالترميز الطولي وفك ضغطه

علم البيانات متوسط

ملفات حالة المخزون اليومية تحتوي على تكرارات طويلة (AAAAAABBBCC...). اكتب encodeFile التي تكتب كل سلسلة متكررة كزوج (العدد بايت، الحرف) في ملف ثنائي (بحد أقصى 255 للعدد)، و decodeFile التي تعيد الأصل، ثم قارن الأحجام وتحقق من التطابق.

خطوات الحل

1) نقرأ الحرف الأول ثم نعد التكرارات حتى يتغير الحرف أو يبلغ العداد 255.
2) فك الضغط: لكل زوج نكتب الحرف count مرة.

البرنامج بلغة C

```c
#include <stdio.h>

long encodeFile(const char *source, const char *target) {
    FILE *input = fopen(source, "rb"), *output = fopen(target, "wb");
    int ch, prev = fgetc(input), run = 1;
    long written = 0;
    while (prev != EOF) {
        ch = fgetc(input);
        if (ch == prev && run < 255) {
            run++;
        } else {
            fputc(run, output);                    // (العدد، الحرف)
            fputc(prev, output);
            written += 2;
            prev = ch;
            run = 1;
        }
    }
    fclose(input);
```

```c
    fclose(output);
    return written;
}

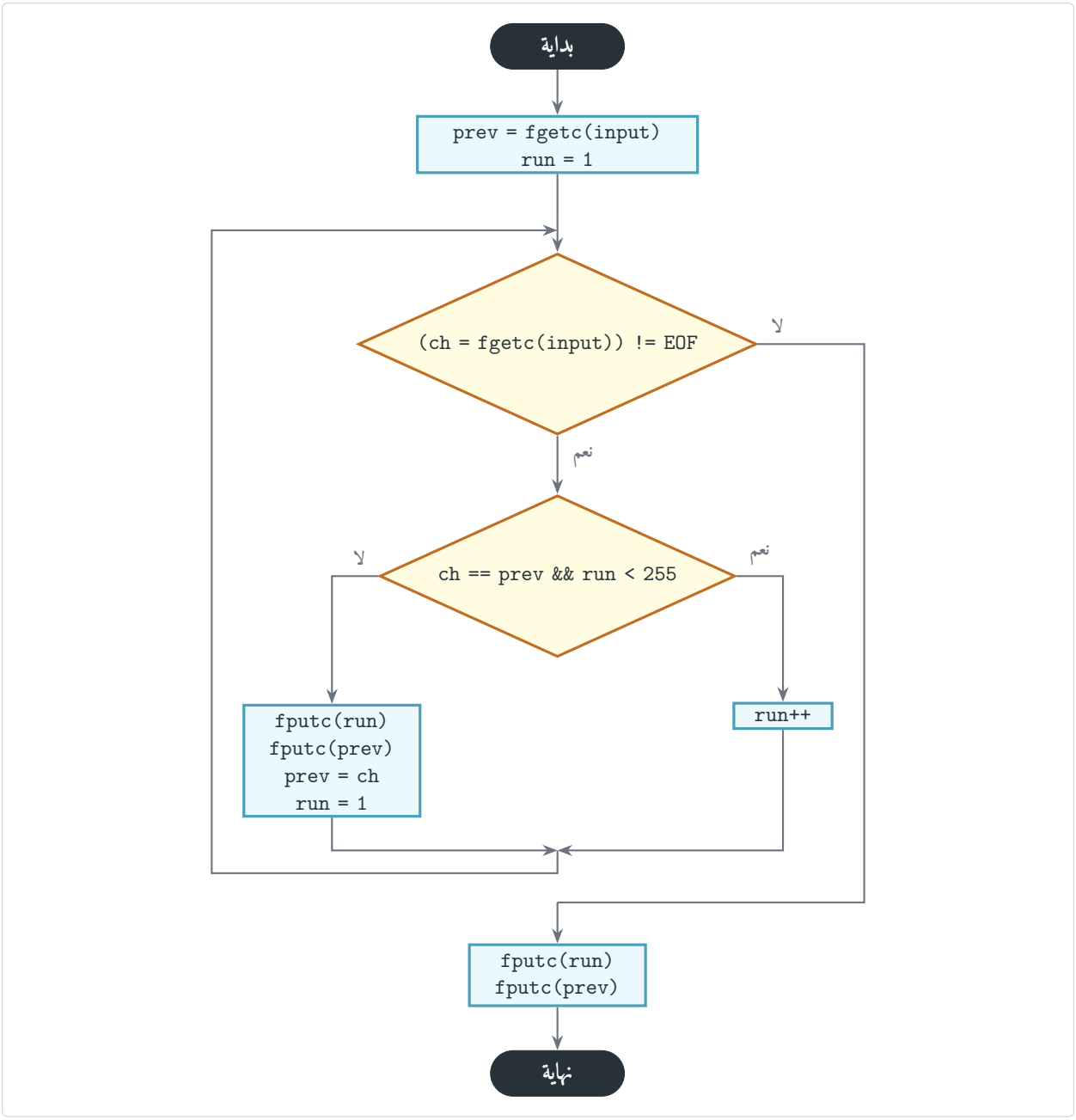
long decodeFile(const char *source, const char *target) {
    FILE *input = fopen(source, "rb"), *output = fopen(target, "wb");
    int run, ch, rep;
    long written = 0;
    while ((run = fgetc(input)) != EOF && (ch = fgetc(input)) != EOF)
        for (rep = 0; rep < run; rep++) { fputc(ch, output); written++; }
    fclose(input);
    fclose(output);
    return written;
}

int main(void) {
    FILE *file = fopen("status.txt", "wb");
    int idx;
    for (idx = 0; idx < 300; idx++) fputc(idx < 120 ? 'A' : idx < 125 ? 'B' : idx < 290 ? 'A' : 'C', file);
    fclose(file);
    printf("encoded size: %ld bytes (original 300)\n", encodeFile("status.txt", "status.rle"));
    printf("decoded size: %ld bytes\n", decodeFile("status.rle", "status.out"));
    file = fopen("status.rle", "rb");
    printf("pairs:");
    while ((idx = fgetc(file)) != EOF) printf(" %d%c", idx, fgetc(file));
    printf("\n");
    fclose(file);
    return 0;
}
```

نتيجة التنفيذ

```
encoded size: 8 bytes (original 300)
decoded size: 300 bytes
pairs: 120A 5B 165A 10C
```

المخطط الإنسيابي (Organigramme)



تمرين 11.38 عدّ الرموز المتتالية في ملف مرتب

تجارة إلكترونية ★★ متوسط

ملف `sold_codes.txt` يحتوي على رموز المنتجات المباعة مرتبة (رمز في كل سطر). اكتب أمرًا يشبه `uniq -c`: يكتب في `summary.txt` كل رمز مع عدد مرات ظهوره المتتالية، ثم اعرض أكثر 3 منتجات مبيعًا.

خطوات الحل

1) بما أن الملف مرتب: كل الظهورات لرمز واحد متتالية، لا نحتاج إلى جدول بحث.
2) عند تغير الرمز نكتب `previous count`.

البرنامج بلغة C

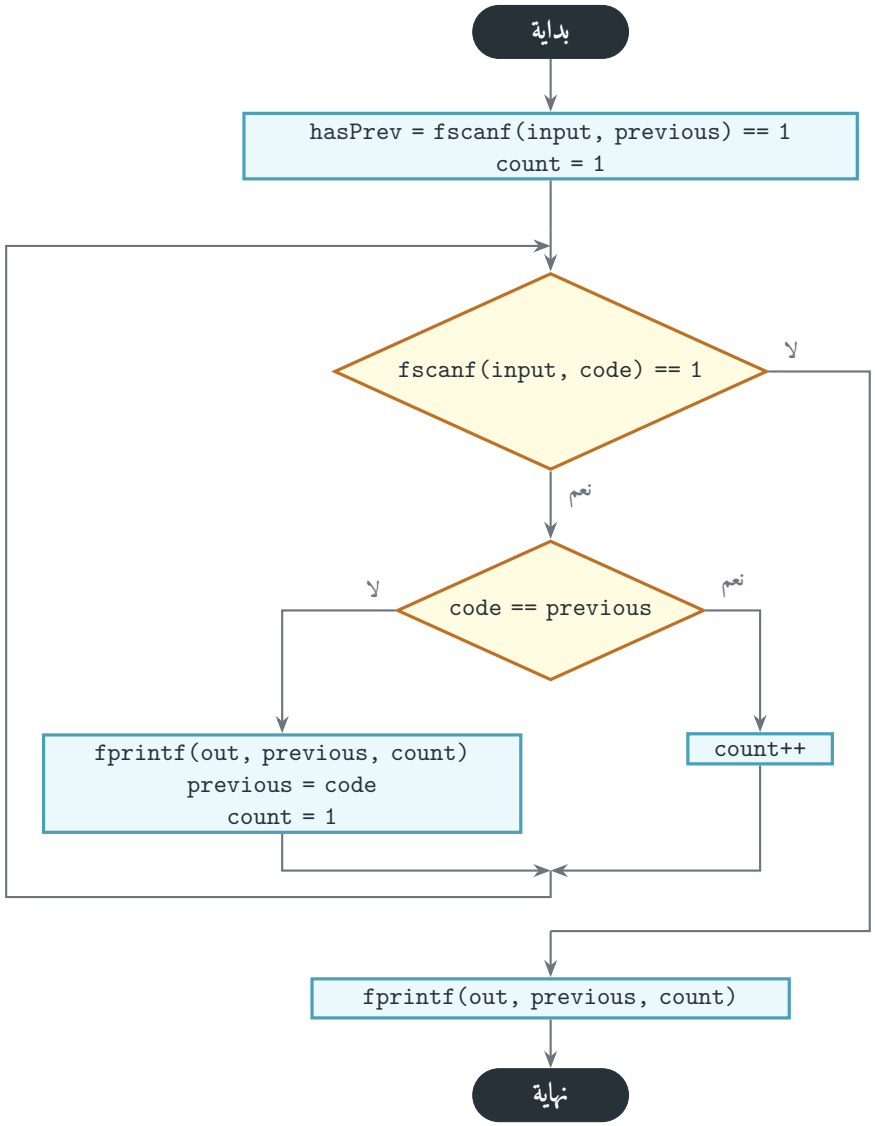
```c
#include <stdio.h>

int main(void) {
    FILE *input = fopen("sold_codes.txt", "w"), *out;
    int sold[16] = {101, 101, 101, 102, 205, 205, 205, 205, 205, 330, 330, 412, 412, 412, 500, 500};
    int previous, code, count, idx, codes[16], counts[16], nb = 0, rank, best, used[16] = {0};
    for (idx = 0; idx < 16; idx++) fprintf(input, "%d\n", sold[idx]);
    fclose(input);
    input = fopen("sold_codes.txt", "r");
    out = fopen("summary.txt", "w");
    if (fscanf(input, "%d", &previous) == 1) {
        count = 1;
        while (fscanf(input, "%d", &code) == 1) {
            if (code == previous) count++;
            else { fprintf(out, "%d %d\n", previous, count); previous = code; count = 1; }
        }
        fprintf(out, "%d %d\n", previous, count);        // المجموعة الأخيرة
    }
    fclose(input);
    fclose(out);
    out = fopen("summary.txt", "r");
    while (fscanf(out, "%d %d", &codes[nb], &counts[nb]) == 2) { printf("%7d %d\n", counts[nb], codes[nb]);
      ↪ nb++; }
    fclose(out);
    printf("Top 3:");
    for (rank = 0; rank < 3; rank++) {
        for (best = -1, idx = 0; idx < nb; idx++)
            if (!used[idx] && (best < 0 || counts[idx] > counts[best])) best = idx;
        used[best] = 1;
        printf(" %d(%d)", codes[best], counts[best]);
    }
    printf("\n");
    return 0;
}
```

نتيجة التنفيذ

```
      3 101
      1 102
      5 205
      2 330
      3 412
      2 500
Top 3: 205(5) 101(3) 412(3)
```

المخطط الإنسيابي (Organigramme)



تمرين 11.39 تنظيف ملف بيانات ناقص

علم البيانات ★★ متوسط

ملف التصدير `raw.csv` فيه أسطر سيئة: حقول فارغة، عدد حقول خاطئ، أو مبلغ غير رقمي. اكتب برنامج **تنظيف** ينقل الأسطر الصالحة فقط إلى `clean.csv`، ويكتب الأسطر المرفوضة مع سبب الرفض في `rejected.txt`، ثم يعرض ملخصًا.

خطوات الحل

1) السطر الصالح: 4 حقول بالضبط، لا حقل فارغ، والحقل الأخير رقمي (أرقام ونقطة واحدة على الأكثر).
2) نحتفظ بنسخة من السطر قبل تقسيمه في مكانه.

البرنامج بلغة C

```c
#include <stdio.h>

int isNumber(const char *text) {
    int dots = 0, digits = 0;
    for (; *text != '\0'; text++) {
```

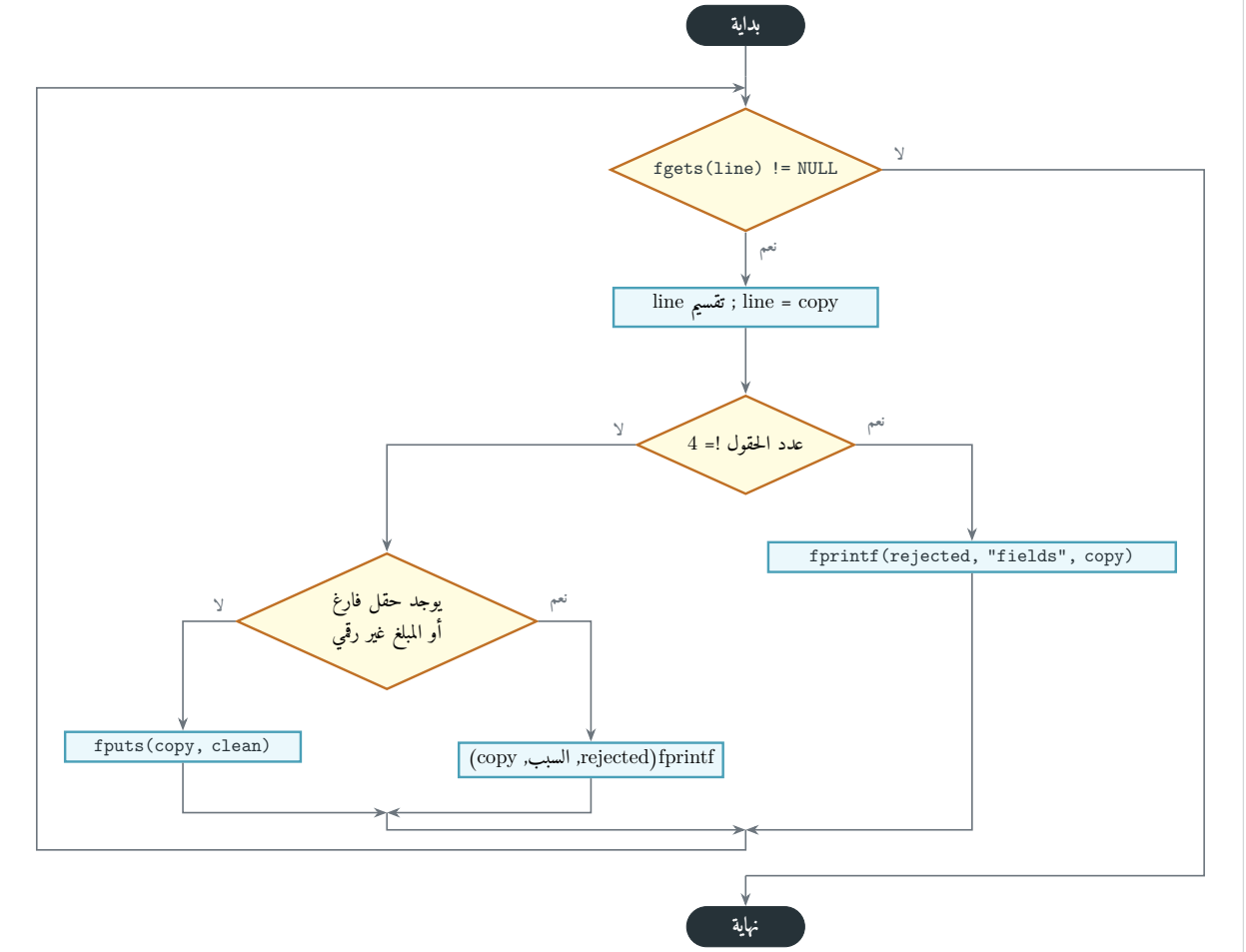
```c
        if (*text == '.') dots++;
        else if (*text >= '0' && *text <= '9') digits++;
        else return 0;
    }
    return digits > 0 && dots <= 1;
}

int main(void) {
    FILE *input = fopen("raw.csv", "w"), *clean, *rejected;
    char line[100], copy[100], *fields[8], *reason;
    int pos, count, idx, kept = 0, dropped = 0, ch;
    fprintf(input, "7781,Amina,Oran,12500\n7782,Karim,,3200\n7783,Lina,Setif,89x0\n");
    fprintf(input,
      ↪ "7784,Omar,Annaba\n7785,Sara,Blida,4500.50\n7786,Nadir,Oran,1.2.3\n7787,Rania,Bejaia,780\n");
    fclose(input);
    input = fopen("raw.csv", "r");
    clean = fopen("clean.csv", "w");
    rejected = fopen("rejected.txt", "w");
    while (fgets(line, 100, input) != NULL) {
        for (pos = 0; (copy[pos] = line[pos]) != '\0'; pos++) {}   // نسخة قبل التقسيم
        fields[0] = line;
        for (pos = 0, count = 1; line[pos] != '\n' && line[pos] != '\0'; pos++)
            if (line[pos] == ',' && count < 8) { line[pos] = '\0'; fields[count++] = line + pos + 1; }
        line[pos] = '\0';
        reason = NULL;
        if (count != 4) reason = "wrong field count";
        for (idx = 0; idx < count && reason == NULL; idx++) if (fields[idx][0] == '\0') reason = "empty
      ↪ field";
        if (reason == NULL && !isNumber(fields[3])) reason = "bad amount";
        if (reason != NULL) { fprintf(rejected, "%-18s | %s", reason, copy); dropped++; }
        else { fputs(copy, clean); kept++; }
    }
    fclose(input); fclose(clean); fclose(rejected);
    printf("kept %d, rejected %d\n--- rejected.txt ---\n", kept, dropped);
    rejected = fopen("rejected.txt", "r");
    while ((ch = fgetc(rejected)) != EOF) putchar(ch);
    fclose(rejected);
    return 0;
}
```

نتيجة التنفيذ

```
kept 3, rejected 4
--- rejected.txt ---
empty field        | 7782,Karim,,3200
bad amount         | 7783,Lina,Setif,89x0
wrong field count  | 7784,Omar,Annaba
bad amount         | 7786,Nadir,Oran,1.2.3
```

المخطط الإنسيابي (Organigramme)



تمرين 11.40 فهرس الأسطر للوصول المباشر

علم البيانات ★★ متوسط

أسطر الملف النصي مختلفة الطول، فلا يمكن حساب موضع السطر رقم `k`. ابنِ **فهرسًا**: مرور أول يحفظ موضع بداية كل سطر بـ `ftell`، ثم نصل إلى أي سطر مباشرة بـ `fseek` دون قراءة ما قبله. أجب عن طلبات أسطر بترتيب عشوائي.

خطوات الحل

1) قبل قراءة كل سطر: `offsets[count++] = ftell(file)`.
2) للوصول إلى السطر `k`: `fseek(file, offsets[k - 1], SEEK_SET)` ثم `fgets`.

البرنامج بلغة C

```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("products.txt", "w");
    const char *names[10] = {"Mouse", "Mechanical keyboard RGB", "USB cable", "Monitor 27 inch 4K",
                             "Hub", "Webcam full HD", "Laptop stand aluminium", "SSD 1TB",
                             "Wireless charger 15W", "Earbuds"};
    long offsets[20], offset;
    char line[80];
    int idx, count = 0, query, lineNo;
    for (idx = 0; idx < 10; idx++) fprintf(file, "%d %s\n", 101 + idx, names[idx]);
    fclose(file);
    file = fopen("products.txt", "r");
```

```c
    while (1) {                                     // بناء الفهرس
        offset = ftell(file);
        if (fgets(line, 80, file) == NULL) break;
        offsets[count++] = offset;
    }
    printf("Indexed %d lines\n", count);
    for (query = 0; query < 5; query++) {
        scanf("%d", &lineNo);
        fseek(file, offsets[lineNo - 1], SEEK_SET);   // قفز مباشر
        fgets(line, 80, file);
        printf("line %d @%3ld: %s", lineNo, offsets[lineNo - 1], line);
    }
    fclose(file);
    return 0;
}
```

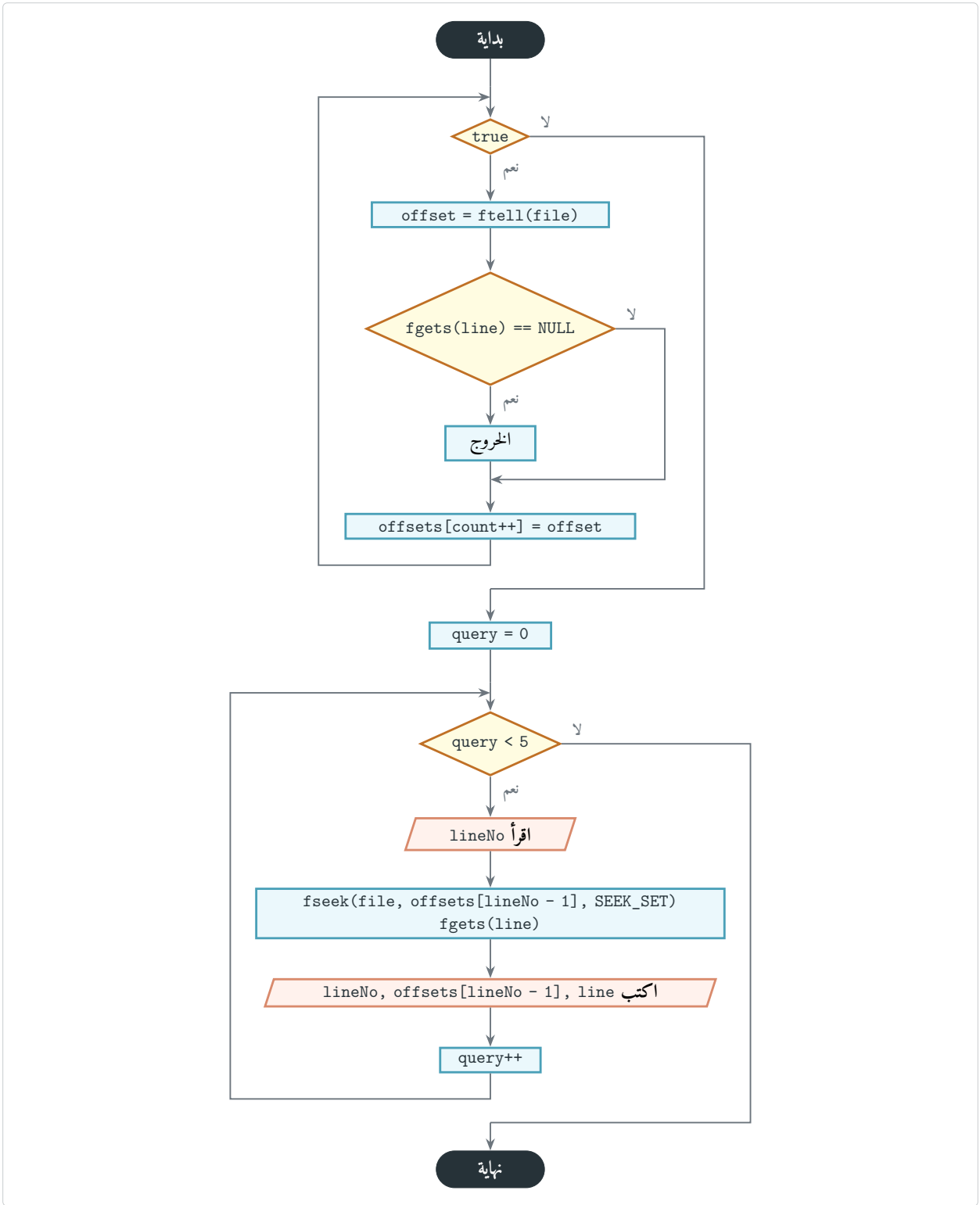
المدخلات

```
4 7 2 9 1
```

نتيجة التنفيذ

```
Indexed 10 lines
line 4 @ 52: 104 Monitor 27 inch 4K
line 7 @102: 107 Laptop stand aluminium
line 2 @ 10: 102 Mechanical keyboard RGB
line 9 @141: 109 Wireless charger 15W
line 1 @  0: 101 Mouse
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 11.41 الترتيب الخارجي لملف كبير

علم البيانات ★★★ صعب

عندما يكون ملف المبالغ أكبر من الذاكرة، نستعمل **الترتيب الخارجي**: (1) نقرأ الملف على دفعات صغيرة (4 قيم هنا)، نرتب كل دفعة في الذاكرة ونكتبها في ملف مؤقت `runK.txt`؛ (2) **ندمج** كل الملفات المؤقتة معًا باختيار الأصغر من رؤوسها في كل خطوة.

خطوات الحل

1) المرحلة 1: كل دفعة تُرتب بالإدراج وتُكتب في ملف منفصل.
2) المرحلة 2: نفتح كل الملفات المؤقتة، لكل ملف قيمة "رأس" حالية.
3) في كل خطوة نكتب أصغر رأس ونقرأ القيمة التالية من نفس الملف.

البرنامج بلغة C

```c
#include <stdio.h>

#define CHUNK 4
#define MAXR 10

void runName(int idx, char *name) {
    name[0] = 'r'; name[1] = 'u'; name[2] = 'n'; name[3] = (char)('0' + idx);
    name[4] = '.'; name[5] = 't'; name[6] = 'x'; name[7] = 't'; name[8] = '\0';
}

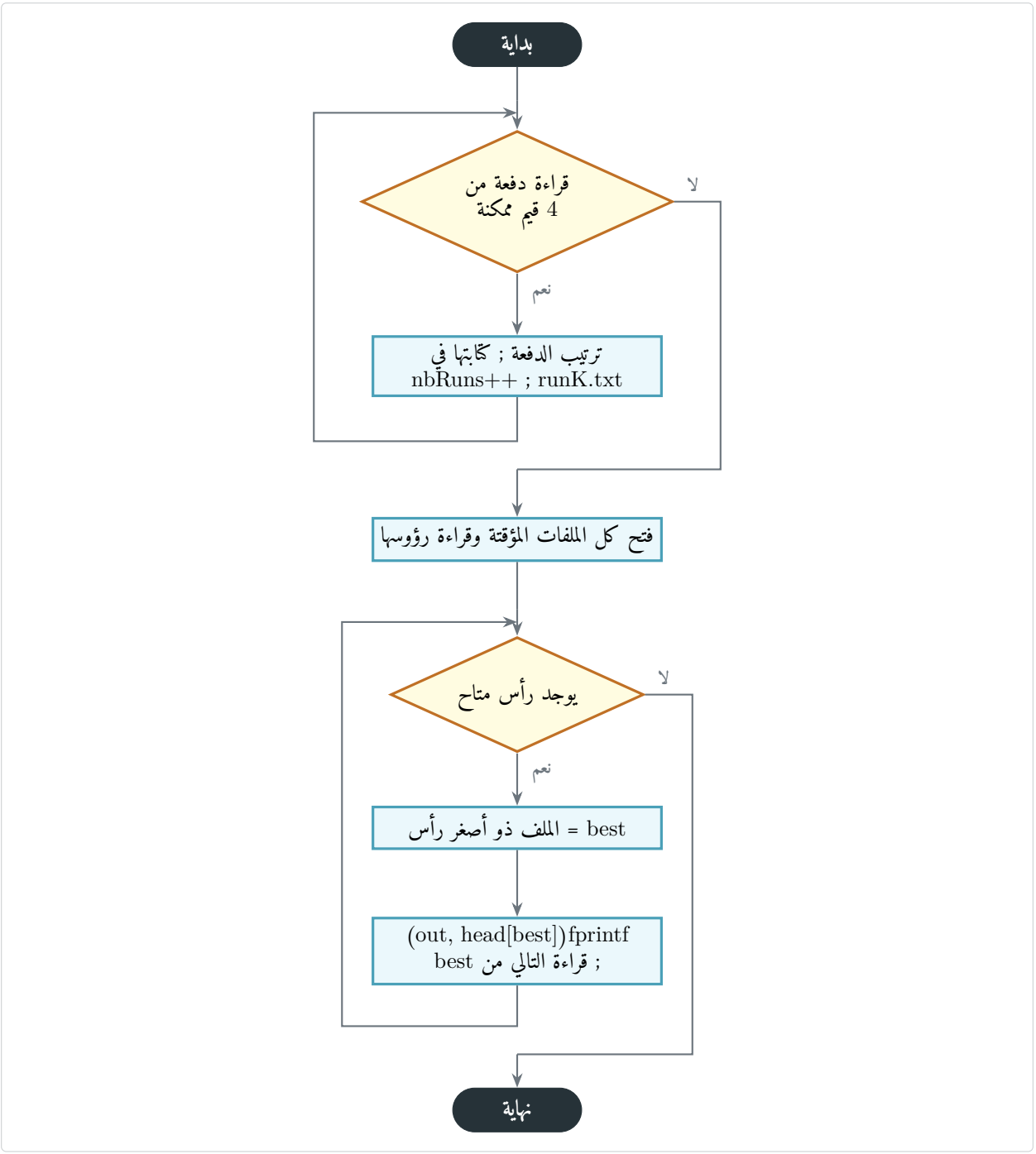
int main(void) {
    FILE *file = fopen("amounts.txt", "w"), *runs[MAXR], *out;
    int data[14] = {4500, 120, 9800, 3300, 760, 15000, 230, 8900, 4100, 50, 6200, 1100, 7300, 990};
    int buffer[CHUNK], count, idx, back, key, nbRuns = 0, heads[MAXR], alive[MAXR], best, value;
    char name[9];
    for (idx = 0; idx < 14; idx++) fprintf(file, "%d\n", data[idx]);
    fclose(file);
    file = fopen("amounts.txt", "r");
    do {                                                  // المرحلة 1: دفعات مرتبة
        for (count = 0; count < CHUNK && fscanf(file, "%d", &buffer[count]) == 1; count++) {}
        if (count == 0) break;
        for (idx = 1; idx < count; idx++) {
            key = buffer[idx];
            for (back = idx - 1; back >= 0 && buffer[back] > key; back--) buffer[back + 1] = buffer[back];
            buffer[back + 1] = key;
        }
        runName(nbRuns, name);
        out = fopen(name, "w");
        printf("%s:", name);
        for (idx = 0; idx < count; idx++) { fprintf(out, "%d\n", buffer[idx]); printf(" %d", buffer[idx]); }
        printf("\n");
        fclose(out);
        nbRuns++;
    } while (count == CHUNK);
    fclose(file);
    for (idx = 0; idx < nbRuns; idx++) {                  // المرحلة 2: دمج متعدد
        runName(idx, name);
        runs[idx] = fopen(name, "r");
        alive[idx] = fscanf(runs[idx], "%d", &heads[idx]) == 1;
    }
    out = fopen("sorted.txt", "w");
    while (1) {
        best = -1;
```

```
        for (idx = 0; idx < nbRuns; idx++)
            if (alive[idx] && (best < 0 || heads[idx] < heads[best])) best = idx;
        if (best < 0) break;
        fprintf(out, "%d\n", heads[best]);
        alive[best] = fscanf(runs[best], "%d", &heads[best]) == 1;
    }
    for (idx = 0; idx < nbRuns; idx++) fclose(runs[idx]);
    fclose(out);
    out = fopen("sorted.txt", "r");
    printf("merged:");
    while (fscanf(out, "%d", &value) == 1) printf(" %d", value);
    printf("\n");
    fclose(out);
    return 0;
}
```

نتيجة التنفيذ

```
run0.txt: 120 3300 4500 9800
run1.txt: 230 760 8900 15000
run2.txt: 50 1100 4100 6200
run3.txt: 990 7300
merged: 50 120 230 760 990 1100 3300 4100 4500 6200 7300 8900 9800 15000
```

المخطط الإنسيابي (Organigramme)



تمرين 11.42 ملف ثنائي مفهرس والبحث بالقفز

تجارة إلكترونية صعب

ملف المنتجات `data.bin` مكتوب بترتيب الإضافة. نبني ملف فهرس `index.bin` يحتوي على أزواج (الرمز، رقم السجل) **مرتبة حسب الرمز**. للبحث عن منتج: **بحث ثنائي داخل ملف الفهرس** بـ `fseek` (دون تحميله)، ثم قفزة واحدة إلى السجل في ملف البيانات. عدّ عمليات القراءة.

خطوات الحل

1) الفهرس: نقرأ الرموز، نرتب الأزواج في الذاكرة مرة واحدة، ونكتبها.

2) البحث الثنائي: `middle` رقم مدخل الفهرس `fseek(index, middle * sizeof(Entry))`.

3) بعد العثور:

```
fseek(data, entry.record * sizeof(Product))
```

.

</> البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; char name[16]; double price; } Product;
typedef struct { int code; int record; } Entry;

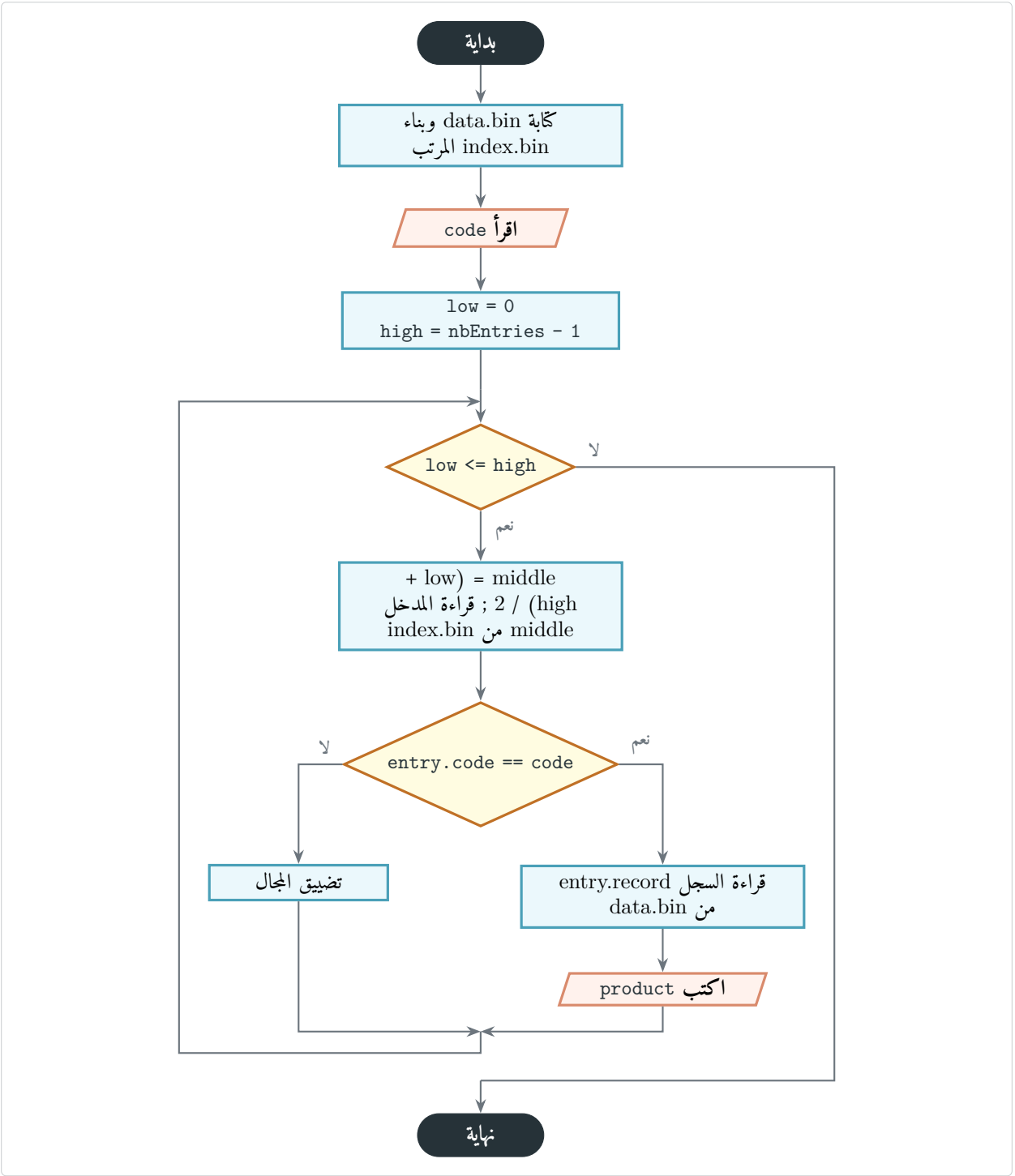
int findProduct(int code, Product *found, int *reads) {
    FILE *index = fopen("index.bin", "rb"), *data;
    Entry entry;
    long size;
    int low = 0, high, middle;
    fseek(index, 0, SEEK_END);
    size = ftell(index);
    high = (int)(size / (long)sizeof(Entry)) - 1;
    *reads = 0;
    while (low <= high) {
        middle = (low + high) / 2;
        fseek(index, middle * (long)sizeof(Entry), SEEK_SET);    // قفز داخل الفهرس
        fread(&entry, sizeof(Entry), 1, index);
        (*reads)++;
        if (entry.code == code) {
            fclose(index);
            data = fopen("data.bin", "rb");
            fseek(data, entry.record * (long)sizeof(Product), SEEK_SET);   // قفزة واحدة
            fread(found, sizeof(Product), 1, data);
            (*reads)++;
            fclose(data);
            return 1;
        }
        if (entry.code < code) low = middle + 1; else high = middle - 1;
    }
    fclose(index);
    return 0;
}

int main(void) {
    Product items[8] = {{5120, "Speaker", 6800}, {1001, "Cable", 450}, {7781, "Laptop", 89000},
                        {2045, "Phone", 32000}, {3310, "Mouse", 1500}, {9002, "Watch", 15000},
                        {1500, "Hub", 3100}, {4400, "Webcam", 4100}};
    Entry entries[8], key;
    Product found;
    FILE *file = fopen("data.bin", "wb");
    int idx, back, queries[3] = {4400, 9002, 6000}, reads;
    fwrite(items, sizeof(Product), 8, file);
    fclose(file);
    for (idx = 0; idx < 8; idx++) { entries[idx].code = items[idx].code; entries[idx].record = idx; }
    for (idx = 1; idx < 8; idx++) {                                  // ترتيب الفهرس
        key = entries[idx];
        for (back = idx - 1; back >= 0 && entries[back].code > key.code; back--) entries[back + 1] =
      ↪ entries[back];
        entries[back + 1] = key;
    }
    file = fopen("index.bin", "wb");
    fwrite(entries, sizeof(Entry), 8, file);
    fclose(file);
    for (idx = 0; idx < 3; idx++) {
        if (findProduct(queries[idx], &found, &reads))
            printf("%d -> %s %.0f DA (%d reads)\n", queries[idx], found.name, found.price, reads);
        else
            printf("%d -> not found (%d reads)\n", queries[idx], reads);
    }
    return 0;
}
```

نتيجة التنفيذ

```
4400 -> Webcam 4100 DA (4 reads)
9002 -> Watch 15000 DA (5 reads)
6000 -> not found (3 reads)
```

المخطط الإنسيابي (Organigramme)



تمرين 11.43 كشف تلف الملفات بمجموع CRC-32

علم البيانات ★★★ صعب

قبل استيراد ملف أسعار من مورد، تحقق من سلامته بمقارنة **CRC-32** المحسوب مع القيمة المرسلة. اكتب دالة `crcFile(path)` تحسب CRC-32 (كثير الحدود المعكوس `0xEDB88320`) بايتًا ببايت، ثم أفسد بايتًا واحدًا في الملف وبيّن أن المجموع يتغير.

خطوات الحل

1) `crc = 0xFFFFFFFF`؛ لكل بايت: `crc ^= byte` ثم 8 مرات: إذا كان البت الأدنى 1: `crc = (crc >> 1) ^ 0xEDB88320` وإلا `crc >>= 1`.
2) النتيجة النهائية `crc ^ 0xFFFFFFFF`.
3) القيمة المعروفة لـ `"123456789"` هي `CBF43926` (للتحقق من التطبيق).

البرنامج بلغة C

```c
#include <stdio.h>

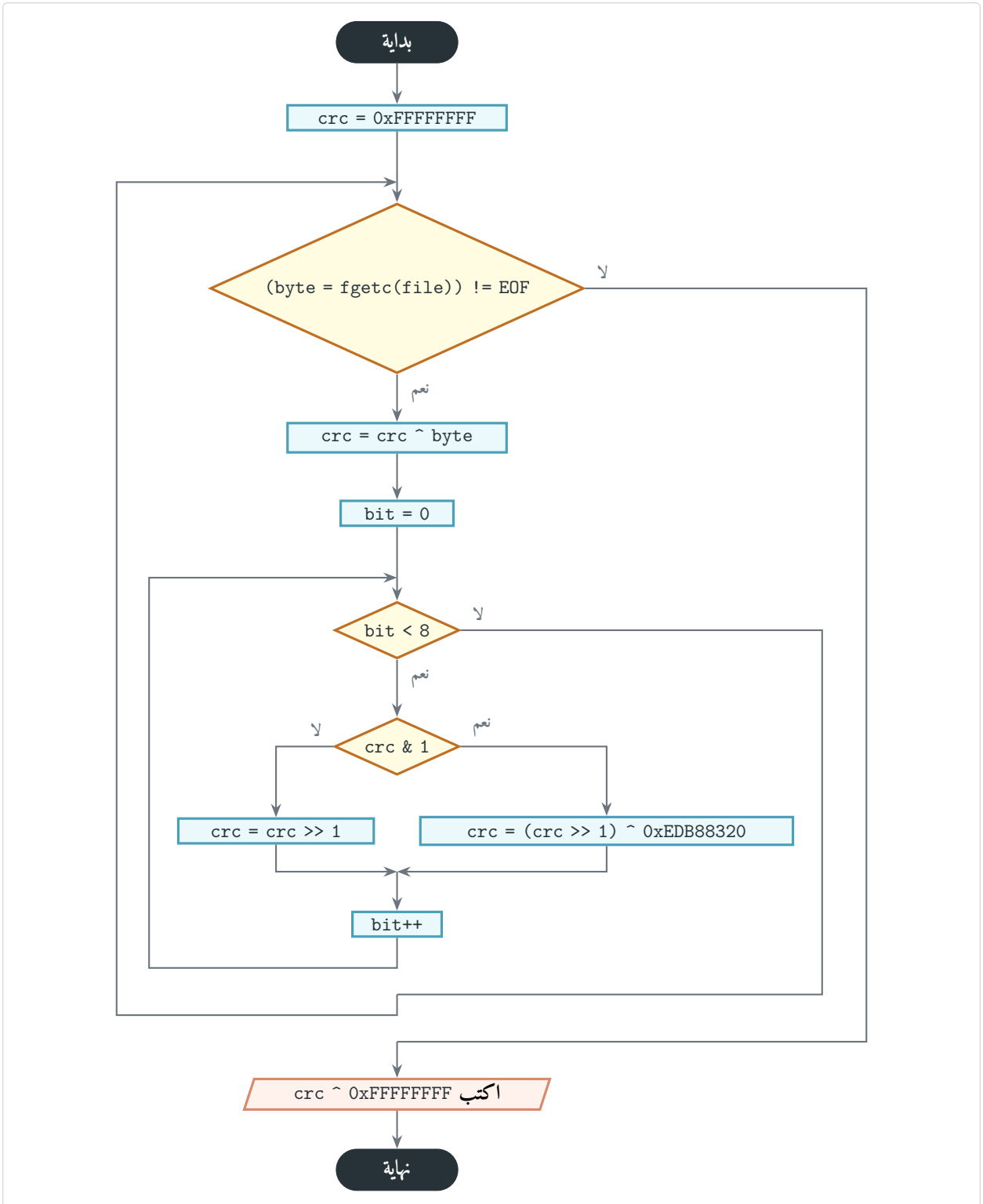
unsigned long crcFile(const char *path) {
    FILE *file = fopen(path, "rb");
    unsigned long crc = 0xFFFFFFFFUL;
    int byte, bit;
    if (file == NULL) return 0;
    while ((byte = fgetc(file)) != EOF) {
        crc ^= (unsigned long)byte;
        for (bit = 0; bit < 8; bit++)
            crc = (crc & 1) ? (crc >> 1) ^ 0xEDB88320UL : crc >> 1;   // قسمة متعددة الحدود
    }
    fclose(file);
    return (crc ^ 0xFFFFFFFFUL) & 0xFFFFFFFFUL;
}

int main(void) {
    FILE *file = fopen("check.txt", "w");
    unsigned long sent;
    fprintf(file, "123456789");
    fclose(file);
    printf("CRC32(\"123456789\") = %08lX (expected CBF43926)\n", crcFile("check.txt"));
    file = fopen("supplier.csv", "w");
    fprintf(file, "MS-110;1500\nKB-204;3200\nMN-270;32000\n");
    fclose(file);
    sent = crcFile("supplier.csv");                                  // القيمة المرسلة مع الملف
    printf("supplier.csv CRC = %08lX\n", sent);
    file = fopen("supplier.csv", "r+b");
    fseek(file, 20, SEEK_SET);
    fputc('9', file);                                                // تلف بايت واحد
    fclose(file);
    printf("after corruption = %08lX -> %s\n", crcFile("supplier.csv"),
           crcFile("supplier.csv") == sent ? "OK" : "FILE CORRUPTED");
    return 0;
}
```

نتيجة التنفيذ

```
CRC32("123456789") = CBF43926 (expected CBF43926)
supplier.csv CRC = B274C0DE
after corruption = 07FED4C4 -> FILE CORRUPTED
```

المخطط الإنسيابي (Organigramme)



تمرين 11.44 مصنف بايز للتعليقات المزيفة

علم البيانات صعب ★★★★

ملف تدريب `train.txt` يحتوي على تعليقات موسومة (`F` مزيف، `R` حقيقي) متبوعة بكلماتها. درّب **مصنف بايز الساذج**: احسب لكل كلمة عدد ظهورها في كل فئة، ثم صنّف تعليقات جديدة بمقارنة **مجموع لوغاريتمات** الاحتمالات (مع تنعيم لابلاس). اللوغاريتم مكتوب يدوياً.

خطوات الحل

(1) `P(word | class) = (count + 1) / (wordsInClass + vocabulary)` (لابلاس).

(2) `score(class) = log P(class) + Σ log P(word | class)`.

(3) `myLog(value)`: نخفض `value` بالقسمة على `e` حتى يقترب من 1، ثم سلسلة `log(1+t)`.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAXW 60

char vocab[MAXW][16];
int countF[MAXW], countR[MAXW], nbWords = 0;

double myLog(double value) {
    double result = 0, term, power, sum = 0;
    int rank;
    while (value > 1.5) { value /= 2.718281828459045; result += 1; }    // log(x) = 1 + log(x/e)
    while (value < 0.5) { value *= 2.718281828459045; result -= 1; }
    term = value - 1;
    power = term;
    for (rank = 1; rank < 40; rank++) { sum += (rank % 2 ? power : -power) / rank; power *= term; }
    return result + sum;
}

int wordIndex(const char *word, int add) {
    int idx, pos;
    for (idx = 0; idx < nbWords; idx++) {
        for (pos = 0; word[pos] != '\0' && word[pos] == vocab[idx][pos]; pos++) {}
        if (word[pos] == vocab[idx][pos]) return idx;
    }
    if (!add || nbWords == MAXW) return -1;
    for (pos = 0; (vocab[nbWords][pos] = word[pos]) != '\0'; pos++) {}
    countF[nbWords] = countR[nbWords] = 0;
    return nbWords++;
}

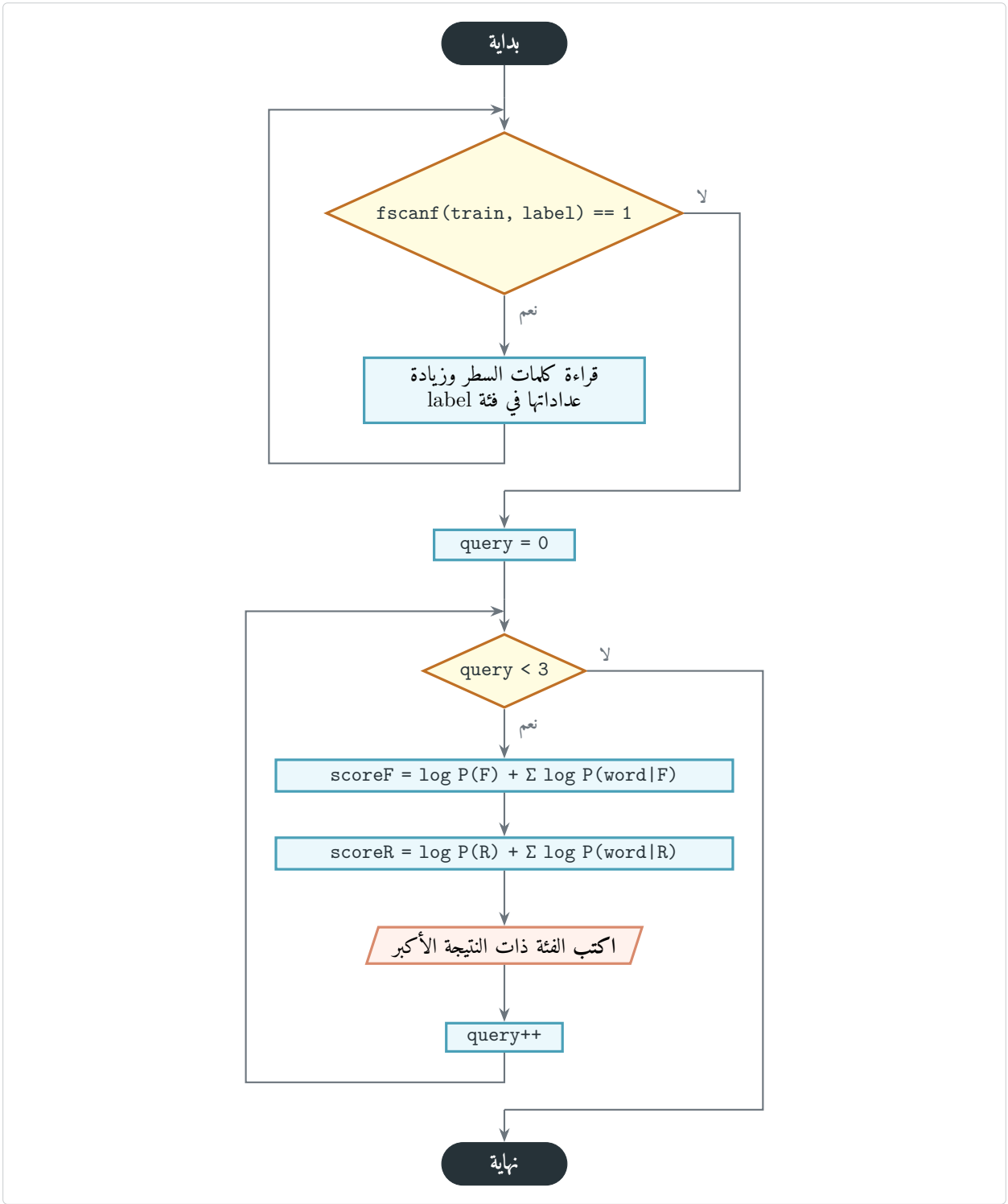
int main(void) {
    FILE *file = fopen("train.txt", "w");
    char label, word[16];
    const char *queries[3][5] = {{"best", "product", "ever", "buy", "now"},
                                 {"battery", "lasts", "two", "days", "ok"},
                                 {"amazing", "deal", "buy", "best", "fast"}};
    int docsF = 0, docsR = 0, wordsF = 0, wordsR = 0, idx, query, ch;
    double scoreF, scoreR;
    fprintf(file, "F best product ever buy now amazing\nF amazing deal best price buy\nF buy now best
      ↪ ever\n");
    fprintf(file, "R battery lasts one day\nR screen ok but slow delivery\nR good price fast delivery\n");
    fprintf(file, "R product works battery ok\n");
    fclose(file);
    file = fopen("train.txt", "r");
    while (fscanf(file, " %c", &label) == 1) {
        if (label == 'F') docsF++; else docsR++;
        while ((ch = fgetc(file)) != '\n' && ch != EOF) {
            if (ch == ' ') continue;
            ungetc(ch, file);
            fscanf(file, "%15s", word);
            idx = wordIndex(word, 1);
            if (label == 'F') { countF[idx]++; wordsF++; } else { countR[idx]++; wordsR++; }
        }
    }
    fclose(file);
    printf("Trained on %d fake / %d real reviews, vocabulary %d\n", docsF, docsR, nbWords);
    for (query = 0; query < 3; query++) {
        scoreF = myLog((double)docsF / (docsF + docsR));
        scoreR = myLog((double)docsR / (docsF + docsR));
```

```
59          for (idx = 0; idx < 5; idx++) {
60              int pos = wordIndex(queries[query][idx], 0);
61              int inF = pos >= 0 ? countF[pos] : 0, inR = pos >= 0 ? countR[pos] : 0;
62              scoreF += myLog((inF + 1.0) / (wordsF + nbWords));      // تنعيم لابلاس
63              scoreR += myLog((inR + 1.0) / (wordsR + nbWords));
64          }
65          printf("\"%s %s %s %s %s\": F %.2f R %.2f -> %s\n", queries[query][0], queries[query][1],
66                 queries[query][2], queries[query][3], queries[query][4], scoreF, scoreR,
67                 scoreF > scoreR ? "FAKE" : "real");
68      }
69      return 0;
70  }
```

نتيجة التنفيذ

```
Trained on 3 fake / 4 real reviews, vocabulary 20
"best product ever buy now": F -12.96 R -17.92 -> FAKE
"battery lasts two days ok": F -18.62 R -15.72 -> real
"amazing deal buy best fast": F -14.06 R -17.92 -> FAKE
```

المخطط الإنسيابي (Organigramme)



تمرين 11.45 تقسيم سجل النقرات إلى جلسات

علم البيانات ★★★ صعب

سجل النقرات `clicks.log` مرتب زمنيًا بأسطر `seconds user page`. **الجلسة** هي سلسلة نقرات لنفس المستخدم لا يفصل بين نقرتين متتاليتين فيها أكثر من 30 دقيقة (1800 ثانية). احسب لكل مستخدم عدد جلساته، ومتوسط مدة الجلسة، ومتوسط الصفحات لكل جلسة، واكتب الجلسات في `sessions.txt`.

خطوات الحل

1) لكل مستخدم نحفظ: وقت آخر نقرة، بداية الجلسة الحالية، عدد صفحاتها.
2) نقرة جديدة: إذا تجاوز الفاصل 1800 ثانية نغلق الجلسة السابقة (نكتبها) ونبدأ جديدة.
3) في النهاية نغلق كل الجلسات المفتوحة.

البرنامج بلغة C

```c
#include <stdio.h>

#define TIMEOUT 1800
#define MAXU 10

typedef struct { int id; long start, last; int pages, sessions, totalPages; long totalTime; } UserState;

UserState users[MAXU];
int nbUsers = 0;
FILE *out;

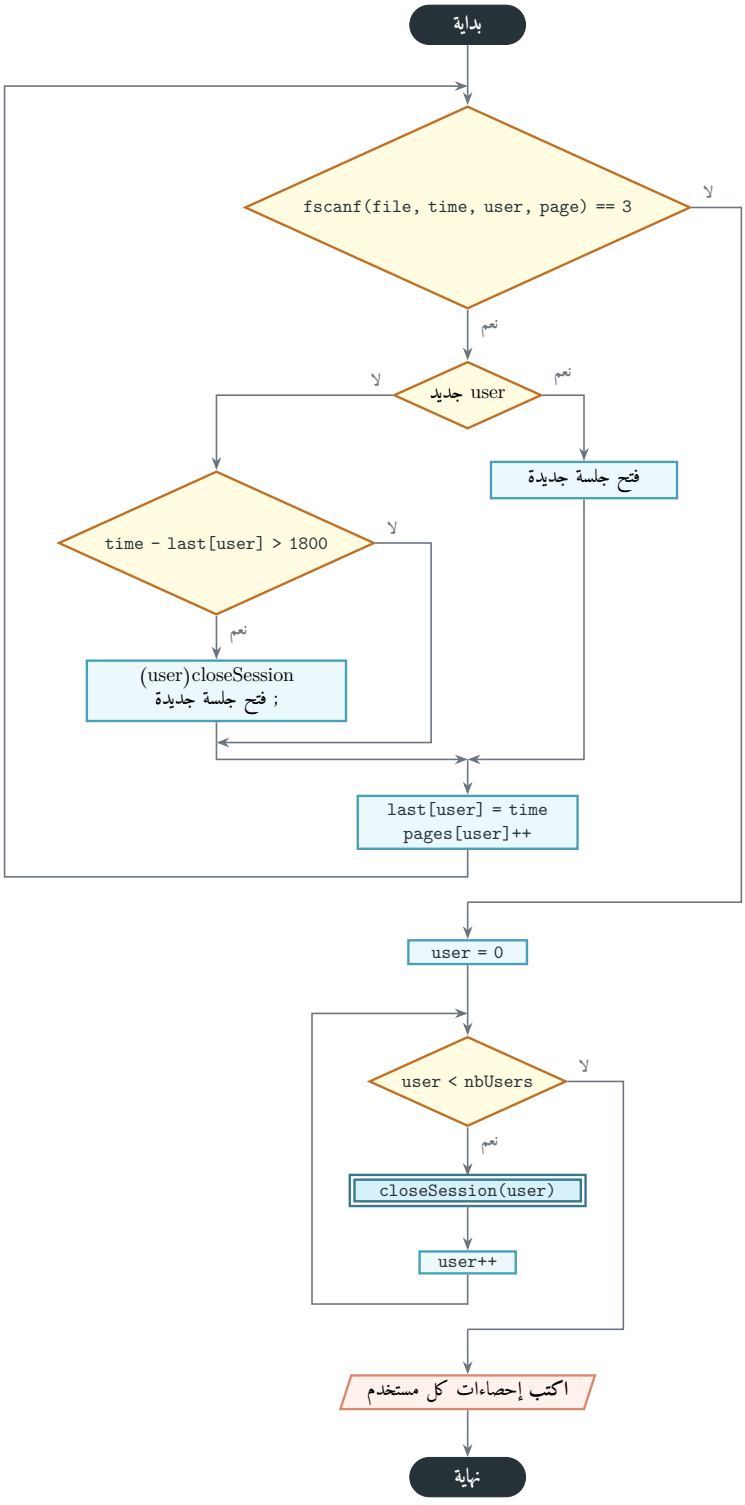
void closeSession(UserState *user) {
    fprintf(out, "user %d: %ld -> %ld (%ld s, %d pages)\n", user->id, user->start, user->last,
            user->last - user->start, user->pages);
    user->sessions++;
    user->totalPages += user->pages;
    user->totalTime += user->last - user->start;
}

int main(void) {
    FILE *file = fopen("clicks.log", "w");
    long data[14][2] = {{0, 7}, {60, 7}, {200, 9}, {400, 7}, {500, 9}, {2100, 9}, {2600, 7},
                        {2700, 7}, {2900, 9}, {5000, 7}, {5100, 9}, {5200, 7}, {7300, 9}, {7400, 9}};
    char page[20];
    long time;
    int idx, userId, ch;
    for (idx = 0; idx < 14; idx++) fprintf(file, "%ld %ld /p%d\n", data[idx][0], data[idx][1], idx);
    fclose(file);
    file = fopen("clicks.log", "r");
    out = fopen("sessions.txt", "w");
    while (fscanf(file, "%ld %d %19s", &time, &userId, page) == 3) {
        for (idx = 0; idx < nbUsers && users[idx].id != userId; idx++) {}
        if (idx == nbUsers) {                                   // مستخدم جديد
            users[idx].id = userId;
            users[idx].sessions = users[idx].totalPages = 0;
            users[idx].totalTime = 0;
            users[idx].start = time;
            users[idx].pages = 0;
            nbUsers++;
        } else if (time - users[idx].last > TIMEOUT) {          // انقطاع طويل: جلسة جديدة
            closeSession(&users[idx]);
            users[idx].start = time;
            users[idx].pages = 0;
        }
        users[idx].last = time;
        users[idx].pages++;
    }
    for (idx = 0; idx < nbUsers; idx++) closeSession(&users[idx]);
    fclose(file);
    fclose(out);
    out = fopen("sessions.txt", "r");
    while ((ch = fgetc(out)) != EOF) putchar(ch);
    fclose(out);
    for (idx = 0; idx < nbUsers; idx++)
        printf("user %d: %d sessions, avg %.1f s, avg %.1f pages\n", users[idx].id, users[idx].sessions,
               (double)users[idx].totalTime / users[idx].sessions,
               (double)users[idx].totalPages / users[idx].sessions);
    return 0;
}
```

نتيجة التنفيذ

```
user 7: 0 -> 400 (400 s, 3 pages)
user 7: 2600 -> 2700 (100 s, 2 pages)
user 9: 200 -> 2900 (2700 s, 4 pages)
user 9: 5100 -> 5100 (0 s, 1 pages)
user 7: 5000 -> 5200 (200 s, 2 pages)
user 9: 7300 -> 7400 (100 s, 2 pages)
user 7: 3 sessions, avg 233.3 s, avg 2.3 pages
user 9: 3 sessions, avg 933.3 s, avg 2.3 pages
```

المخطط الإنسيابي (Organigramme)



تمرين 11.46 مطابقة كشف البنك مع الطلبيات

تجارة إلكترونية صعب ★★★

المطابقة المحاسبية: ملف الطلبيات `orders.txt` (المرجع، المبلغ) وكشف البنك `bank.txt` (المرجع، المبلغ المستلم). الملفان **مرتبان حسب المرجع**. بمرور واحد متزامن على الملفين (مثل الدمج)، صنّف كل مرجع: مطابق، فرق في المبلغ، طلبية دون دفع، أو دفع دون طلبية، واكتب التقرير في `reconciliation.txt`.

خطوات الحل

1) نقرأ سطرًا من كل ملف، نقارن المرجعين.
2) متساويان: نقارن المبلغين ثم نتقدم في الملفين.
3) مرجع الطلبية أصغر: طلبية غير مدفوعة، نتقدم في الطلبيات فقط؛ والعكس للبنك.

البرنامج بلغة C

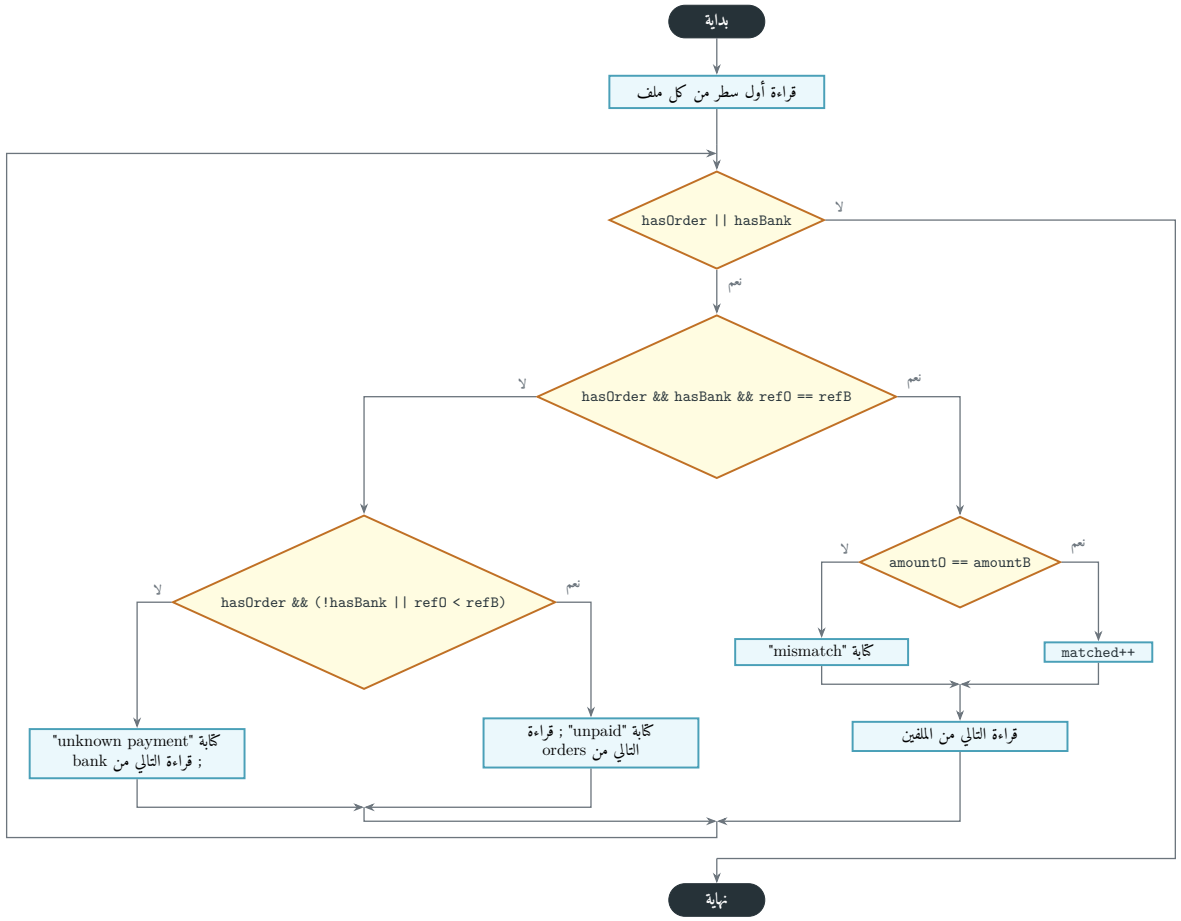
```c
#include <stdio.h>

int main(void) {
    FILE *orders = fopen("orders.txt", "w"), *bank = fopen("bank.txt", "w"), *report;
    int refO, refB, hasOrder, hasBank, matched = 0, issues = 0, ch;
    double amountO, amountB, missing = 0;
    fprintf(orders, "7781 12500\n7782 3200\n7783 899.5\n7785 45000\n7786 1500\n7788 2200\n");
    fprintf(bank, "7781 12500\n7783 899.5\n7784 650\n7785 44000\n7786 1500\n7788 2200\n7790 300\n");
    fclose(orders); fclose(bank);
    orders = fopen("orders.txt", "r");
    bank = fopen("bank.txt", "r");
    report = fopen("reconciliation.txt", "w");
    hasOrder = fscanf(orders, "%d %lf", &refO, &amountO) == 2;
    hasBank = fscanf(bank, "%d %lf", &refB, &amountB) == 2;
    while (hasOrder || hasBank) {
        if (hasOrder && hasBank && refO == refB) {
            if (amountO == amountB) matched++;
            else { fprintf(report, "%d MISMATCH  expected %.2f got %.2f\n", refO, amountO, amountB);
      ↪ issues++; missing += amountO - amountB; }
            hasOrder = fscanf(orders, "%d %lf", &refO, &amountO) == 2;
            hasBank = fscanf(bank, "%d %lf", &refB, &amountB) == 2;
        } else if (hasOrder && (!hasBank || refO < refB)) {            // لا دفع لهذه الطلبية
            fprintf(report, "%d UNPAID    %.2f\n", refO, amountO);
            issues++;
            missing += amountO;
            hasOrder = fscanf(orders, "%d %lf", &refO, &amountO) == 2;
        } else {                                                       // دفع دون طلبية
            fprintf(report, "%d UNKNOWN   payment %.2f\n", refB, amountB);
            issues++;
            hasBank = fscanf(bank, "%d %lf", &refB, &amountB) == 2;
        }
    }
    fclose(orders); fclose(bank); fclose(report);
    report = fopen("reconciliation.txt", "r");
    while ((ch = fgetc(report)) != EOF) putchar(ch);
    fclose(report);
    printf("%d matched, %d issues, %.2f DA not received\n", matched, issues, missing);
    return 0;
}
```

نتيجة التنفيذ

```
7782 UNPAID    3200.00
7784 UNKNOWN   payment 650.00
7785 MISMATCH  expected 45000.00 got 44000.00
7790 UNKNOWN   payment 300.00
4 matched, 4 issues, 4200.00 DA not received
```

المخطط الانسيابي (Organigramme)



تمرين 11.47 ترميز ملف بـ Base64 وفك ترميزه

صعب ★★★ تجارة إلكترونية

لإرفاق ملف ثنائي (صورة منتج مصغرة) في رسالة JSON، نرمّزه بـ **Base64**. اكتب `encodeFile(source, target)` التي تقرأ 3 بايتات في كل مرة وتكتب 4 رموز (مع الحشو `=`)، و `decodeFile` التي تعكس العملية بجدول بحث معكوس، ثم تحقق أن الملف المسترجع مطابق للأصل.

خطوات الحل

1) الترميز: كتلة 24 بت ← 4 مجموعات من 6 بتات ← جدول الرموز.
2) فك الترميز: نبني جدولًا معكوسًا `reverse[symbol] = value`، نقرأ 4 رموز، نعيد 3 بايتات (أو أقل إذا وُجد `=`).
3) التحقق بمقارنة الملفين بايتًا ببايت.

البرنامج بلغة C

```c
#include <stdio.h>

const char table[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

long encodeFile(const char *source, const char *target) {
    FILE *input = fopen(source, "rb"), *output = fopen(target, "w");
    int bytes[3], count, ch;
    long written = 0, block;
    while (1) {
```

```c
        for (count = 0; count < 3 && (ch = fgetc(input)) != EOF; count++) bytes[count] = ch;
        if (count == 0) break;
        block = (long)bytes[0] << 16 | (count > 1 ? bytes[1] : 0) << 8 | (count > 2 ? bytes[2] : 0);
        fputc(table[(block >> 18) & 63], output);
        fputc(table[(block >> 12) & 63], output);
        fputc(count > 1 ? table[(block >> 6) & 63] : '=', output);     // حشو
        fputc(count > 2 ? table[block & 63] : '=', output);
        written += 4;
        if (count < 3) break;
    }
    fclose(input); fclose(output);
    return written;
}

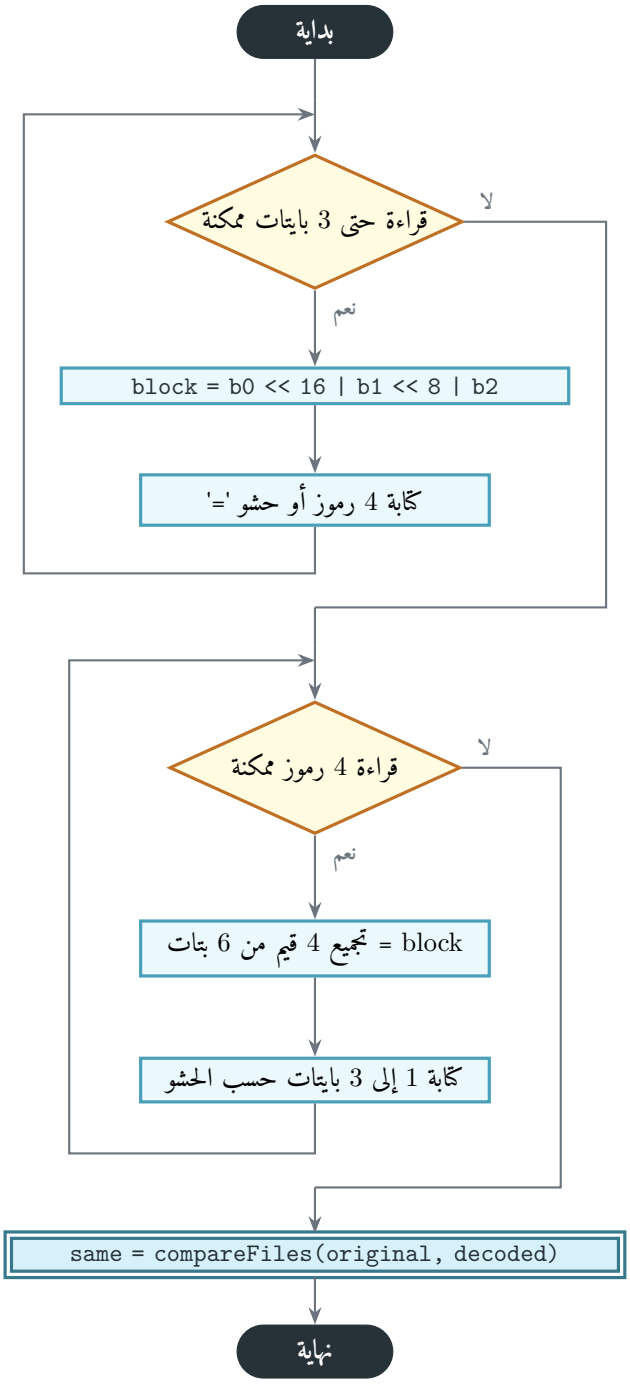
long decodeFile(const char *source, const char *target) {
    FILE *input = fopen(source, "r"), *output = fopen(target, "wb");
    int reverse[128], idx, symbols[4], count, pads;
    long block, written = 0;
    for (idx = 0; idx < 128; idx++) reverse[idx] = 0;
    for (idx = 0; idx < 64; idx++) reverse[(int)table[idx]] = idx;     // جدول معكوس
    while (1) {
        for (count = 0, pads = 0; count < 4 && (symbols[count] = fgetc(input)) != EOF; count++)
            if (symbols[count] == '=') pads++;
        if (count < 4) break;
        for (block = 0, idx = 0; idx < 4; idx++)
            block = block << 6 | (symbols[idx] == '=' ? 0 : reverse[symbols[idx]]);
        fputc((int)(block >> 16) & 255, output); written++;
        if (pads < 2) { fputc((int)(block >> 8) & 255, output); written++; }
        if (pads < 1) { fputc((int)block & 255, output); written++; }
    }
    fclose(input); fclose(output);
    return written;
}

int main(void) {
    FILE *file = fopen("thumb.bin", "wb"), *other;
    int idx, chA, chB, same = 1;
    for (idx = 0; idx < 50; idx++) fputc((idx * 37 + 11) % 256, file);    // بيانات ثنائية
    fclose(file);
    printf("encoded: %ld chars\n", encodeFile("thumb.bin", "thumb.b64"));
    file = fopen("thumb.b64", "r");
    printf("text: ");
    for (idx = 0; idx < 32 && (chA = fgetc(file)) != EOF; idx++) putchar(chA);
    printf("...\n");
    fclose(file);
    printf("decoded: %ld bytes\n", decodeFile("thumb.b64", "thumb.out"));
    file = fopen("thumb.bin", "rb");
    other = fopen("thumb.out", "rb");
    do { chA = fgetc(file); chB = fgetc(other); if (chA != chB) same = 0; } while (chA != EOF && chB != EOF);
    fclose(file); fclose(other);
    printf("round trip identical: %s\n", same ? "yes" : "no");
    return 0;
}
```

نتيجة التنفيذ

```
encoded: 68 chars
text: CzBVep/E6Q4zWH2ix+wRNluApcrvFDle...
decoded: 50 bytes
round trip identical: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 11.48 نموذج الإنحدار من ملف وكتابة التنبؤات

علم البيانات ★★★ صعب

ملف `history.csv` يحتوي على `week,price,units`. درّب نموذج انحدار خطي `units = a + b·price` بمرور أول على الملف (مجاميع)، ثم مرور ثانٍ بعد `rewind` لكتابة ملف `predictions.csv` يضيف لكل سطر التنبؤ والخطأ، واحسب مرونة الطلب عند متوسط السعر والسعر الذي يعظّم الإيراد.

خطوات الحل

1) المرور 1: $\Sigma x,\ \Sigma y,\ \Sigma xy,\ \Sigma x^2,\ n$ ثم $b = (n\Sigma xy - \Sigma x\Sigma y) / (n\Sigma x^2 - (\Sigma x)^2)$ و $a = (\Sigma y - b\Sigma x)/n$.

2) `rewind(file)` يعيد القراءة من البداية دون إغلاق.

3) المرونة = $b \times \text{meanPrice} / \text{meanUnits}$، الإيراد $p(a + bp)$ أعظمي عند $p = -a / (2b)$.

البرنامج بلغة C

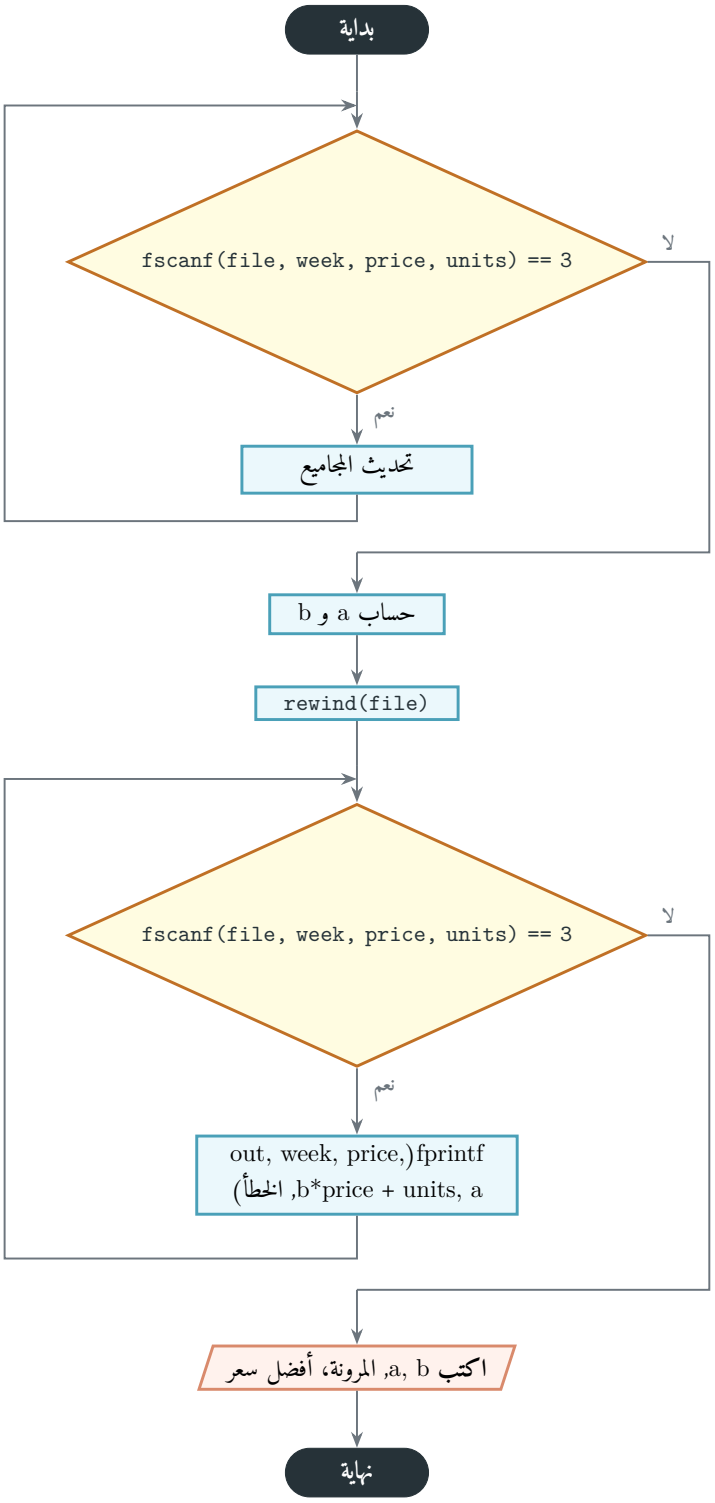
```c
#include <stdio.h>

int main(void) {
    FILE *file = fopen("history.csv", "w"), *out;
    double prices[8] = {3200, 3000, 3500, 2800, 3300, 3600, 2900, 3100};
    double units[8] = {410, 455, 360, 500, 385, 345, 480, 430};
    double sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0, slope, intercept, price, qty, sse = 0;
    char header[40];
    int week, count = 0, ch;
    fprintf(file, "week,price,units\n");
    for (week = 0; week < 8; week++) fprintf(file, "%d,%.0f,%.0f\n", week + 1, prices[week], units[week]);
    fclose(file);
    file = fopen("history.csv", "r");
    fgets(header, 40, file);
    while (fscanf(file, "%d,%lf,%lf", &week, &price, &qty) == 3) {        // المرور الأول
        sumX += price; sumY += qty; sumXY += price * qty; sumX2 += price * price;
        count++;
    }
    slope = (count * sumXY - sumX * sumY) / (count * sumX2 - sumX * sumX);
    intercept = (sumY - slope * sumX) / count;
    rewind(file);                                                          // من البداية
    fgets(header, 40, file);
    out = fopen("predictions.csv", "w");
    fprintf(out, "week,price,units,predicted,error\n");
    while (fscanf(file, "%d,%lf,%lf", &week, &price, &qty) == 3) {        // المرور الثاني
        fprintf(out, "%d,%.0f,%.0f,%.1f,%+.1f\n", week, price, qty, intercept + slope * price,
                qty - (intercept + slope * price));
        sse += (qty - intercept - slope * price) * (qty - intercept - slope * price);
    }
    fclose(file); fclose(out);
    out = fopen("predictions.csv", "r");
    while ((ch = fgetc(out)) != EOF) putchar(ch);
    fclose(out);
    printf("units = %.2f %+.4f * price, MSE = %.1f\n", intercept, slope, sse / count);
    printf("Elasticity at mean: %.2f\n", slope * (sumX / count) / (sumY / count));
    printf("Revenue-maximizing price: %.0f DA\n", -intercept / (2 * slope));
    return 0;
}
```

نتيجة التنفيذ

```
week,price,units,predicted,error
1,3200,410,415.7,-5.7
2,3000,455,455.1,-0.1
3,3500,360,356.6,+3.4
4,2800,500,494.5,+5.5
5,3300,385,396.0,-11.0
6,3600,345,336.9,+8.1
7,2900,480,474.8,+5.2
8,3100,430,435.4,-5.4
units = 1046.33 -0.1971 * price, MSE = 39.6
Elasticity at mean: -1.49
Revenue-maximizing price: 2655 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 11.49 قاعدة بيانات المنتجات بالحذف المنطقي والضغط تجارة إلكترونية صعب

ملف ثنائي `products.db` من سجلات ثابتة الحجم. **الحذف** لا يزيح الملف: نكتب علامة `deleted = 1` في السجل (حذف منطقي). اكتب دوال `insertProduct` (في النهاية)، `deleteProduct` (بحث ثم تعليم)، `updatePrice` (في المكان)، `listProducts`، و `compact` التي تنشئ ملفًا جديدًا دون السجلات المحذوفة وتستبدل به القديم.

خطوات الحل

1) البحث: قراءة تسلسلية حتى الرمز المطلوب مع `ftell` لتذكر موضع السجل.
2) التعديل في المكان: `fseek(file, position, SEEK_SET)` ثم `fwrite`.
3) الضغط: نسخ السجلات الحية إلى `products.tmp` ثم `remove` و `rename`.

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct { int code; char name[12]; double price; int deleted; } Record;

const char *DB = "products.db";

void insertProduct(int code, const char *name, double price) {
    FILE *file = fopen(DB, "ab");
    Record record = {0, "", 0, 0};
    int pos;
    record.code = code;
    for (pos = 0; pos < 11 && name[pos] != '\0'; pos++) record.name[pos] = name[pos];
    record.price = price;
    fwrite(&record, sizeof(Record), 1, file);
    fclose(file);
}

long findRecord(FILE *file, int code, Record *record) {
    long position;
    rewind(file);
    while (1) {
        position = ftell(file);
        if (fread(record, sizeof(Record), 1, file) != 1) return -1;
        if (record->code == code && !record->deleted) return position;
    }
}

void deleteProduct(int code) {
    FILE *file = fopen(DB, "r+b");
    Record record;
    long position = findRecord(file, code, &record);
    if (position >= 0) {
        record.deleted = 1;                                  // حذف منطقي
        fseek(file, position, SEEK_SET);
        fwrite(&record, sizeof(Record), 1, file);
    }
    fclose(file);
}

void updatePrice(int code, double price) {
    FILE *file = fopen(DB, "r+b");
    Record record;
    long position = findRecord(file, code, &record);
    if (position >= 0) {
        record.price = price;
        fseek(file, position, SEEK_SET);                     // تعديل في المكان
        fwrite(&record, sizeof(Record), 1, file);
    }
    fclose(file);
}

void listProducts(const char *label) {
    FILE *file = fopen(DB, "rb");
    Record record;
    int total = 0, alive = 0;
    printf("%s:", label);
    while (fread(&record, sizeof(Record), 1, file) == 1) {
        total++;
        if (!record.deleted) { alive++; printf(" %d %s %.0f;", record.code, record.name, record.price); }
    }
    fseek(file, 0, SEEK_END);
```

```c
    printf("\n  %d live / %d records, file %ld bytes\n", alive, total, ftell(file));
    fclose(file);
}

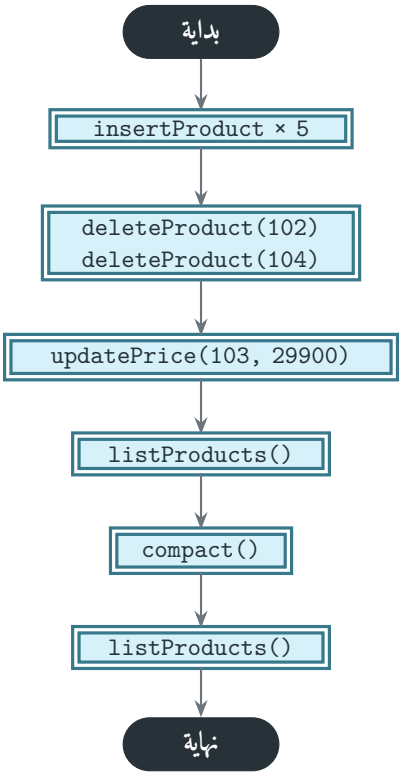
void compact(void) {
    FILE *input = fopen(DB, "rb"), *output = fopen("products.tmp", "wb");
    Record record;
    while (fread(&record, sizeof(Record), 1, input) == 1)
        if (!record.deleted) fwrite(&record, sizeof(Record), 1, output);
    fclose(input);
    fclose(output);
    remove(DB);                                          // نستبدل القديم بالجديد
    rename("products.tmp", DB);
}

int main(void) {
    FILE *file = fopen(DB, "wb");
    fclose(file);
    insertProduct(101, "Mouse", 1500);
    insertProduct(102, "Keyboard", 3200);
    insertProduct(103, "Monitor", 32000);
    insertProduct(104, "Cable", 450);
    insertProduct(105, "Webcam", 4100);
    deleteProduct(102);
    deleteProduct(104);
    updatePrice(103, 29900);
    listProducts("before compact");
    compact();
    listProducts("after compact");
    return 0;
}
```

نتيجة التنفيذ

```
before compact: 101 Mouse 1500; 103 Monitor 29900; 105 Webcam 4100;
  3 live / 5 records, file 160 bytes
after compact: 101 Mouse 1500; 103 Monitor 29900; 105 Webcam 4100;
  3 live / 3 records, file 96 bytes
```

المخطط الإنسيابي (Organigramme)



تمرين 11.50 فهرس معكوس للبحث في الكتالوج

علم البيانات صعب

محرك بحث المتجر: اقرأ ملف الأوصاف `catalog.txt` (`id:`) وابنِ **فهرسًا معكوسًا**: لكل كلمة قائمة مرتبة بمعرفات المنتجات التي تحتويها. احفظ الفهرس في `index.txt`، ثم أجب عن استعلامات **AND** متعددة الكلمات بتقاطع القوائم المرتبة (كما في الدمج).

خطوات الحل

1) لكل كلمة (بحروف صغيرة): نضيف المعرف إلى قائمتها إن لم يكن آخر عنصر فيها (الأسطر مرتبة حسب المعرف).
2) كتابة الفهرس: `word: id1 id2 ...`.
3) التقاطع: مؤشران على قائمتين مرتبتين، نتقدم بالأصغر، ونحتفظ بالمتساوي.

البرنامج بلغة C

```c
#include <stdio.h>

#define MAXW 60
#define MAXP 12

char words[MAXW][16];
int lists[MAXW][MAXP], sizes[MAXW], nbWords = 0;

int lookup(const char *word, int add) {
    int idx, pos;
    for (idx = 0; idx < nbWords; idx++) {
        for (pos = 0; word[pos] != '\0' && word[pos] == words[idx][pos]; pos++) {}
        if (word[pos] == words[idx][pos]) return idx;
    }
    if (!add) return -1;
```

```
    for (pos = 0; (words[nbWords][pos] = word[pos]) != '\0'; pos++) {}
    sizes[nbWords] = 0;
    return nbWords++;
}

int intersect(const int *first, int sizeA, const int *second, int sizeB, int *out) {
    int left = 0, right = 0, count = 0;
    while (left < sizeA && right < sizeB) {
        if (first[left] == second[right]) { out[count++] = first[left]; left++; right++; }
        else if (first[left] < second[right]) left++;          // تتقدم بالأصغر
        else right++;
    }
    return count;
}

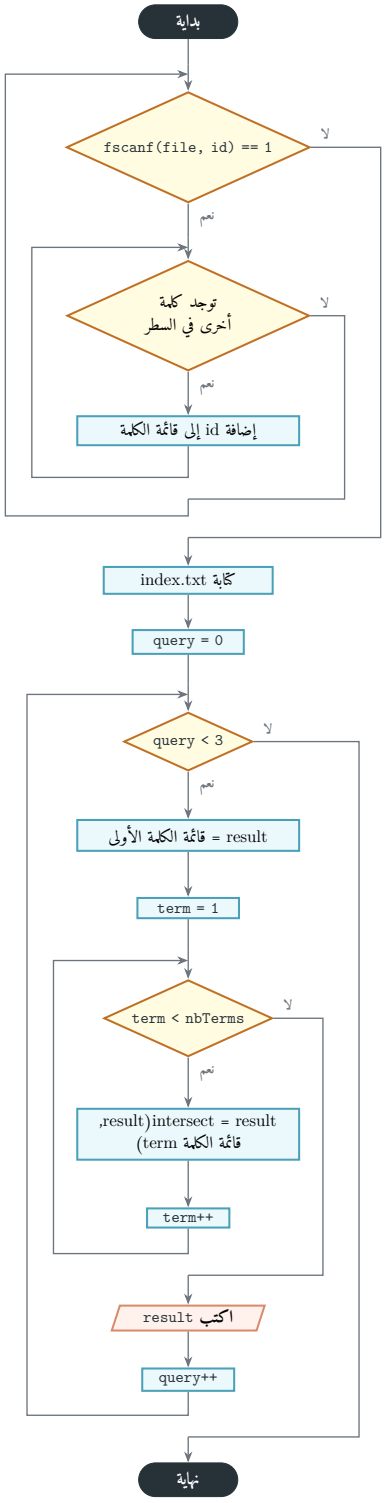
int main(void) {
    FILE *file = fopen("catalog.txt", "w");
    const char *queries[3][3] = {{"wireless", "mouse", ""}, {"usb", "c", "fast"}, {"gaming", "", ""}};
    char word[16];
    int id, idx, ch, pos, query, term, result[MAXP], tmp[MAXP], count;
    fprintf(file, "1: Wireless optical mouse\n2: Gaming mouse RGB wired\n3: USB C fast charger\n");
    fprintf(file, "4: Wireless keyboard and mouse combo\n5: USB C hub fast ethernet\n6: Gaming keyboard
      ↪ wireless\n");
    fclose(file);
    file = fopen("catalog.txt", "r");
    while (fscanf(file, "%d:", &id) == 1) {
        while ((ch = fgetc(file)) != '\n' && ch != EOF) {
            if (ch == ' ') continue;
            pos = 0;
            while (ch != ' ' && ch != '\n' && ch != EOF) {
                if (pos < 15) word[pos++] = (char)(ch >= 'A' && ch <= 'Z' ? ch - 'A' + 'a' : ch);
                ch = fgetc(file);
            }
            word[pos] = '\0';
            idx = lookup(word, 1);
            if (sizes[idx] == 0 || lists[idx][sizes[idx] - 1] != id) lists[idx][sizes[idx]++] = id;
            if (ch == '\n' || ch == EOF) break;
        }
    }
    fclose(file);
    file = fopen("index.txt", "w");
    for (idx = 0; idx < nbWords; idx++) {
        fprintf(file, "%s:", words[idx]);
        for (pos = 0; pos < sizes[idx]; pos++) fprintf(file, " %d", lists[idx][pos]);
        fprintf(file, "\n");
    }
    fclose(file);
    printf("index.txt: %d words\n", nbWords);
    for (query = 0; query < 3; query++) {
        idx = lookup(queries[query][0], 0);
        count = idx >= 0 ? sizes[idx] : 0;
        for (pos = 0; pos < count; pos++) result[pos] = lists[idx][pos];
        for (term = 1; term < 3 && queries[query][term][0] != '\0'; term++) {
            idx = lookup(queries[query][term], 0);
            count = idx >= 0 ? intersect(result, count, lists[idx], sizes[idx], tmp) : 0;
            for (pos = 0; pos < count; pos++) result[pos] = tmp[pos];
        }
        printf("\"%s", queries[query][0]);
        for (term = 1; term < 3 && queries[query][term][0] != '\0'; term++) printf(" %s",
      ↪ queries[query][term]);
        printf("\" ->");
        for (pos = 0; pos < count; pos++) printf(" #%d", result[pos]);
        printf("\n");
    }
    return 0;
}
```

نتيجة التنفيذ

```
index.txt: 15 words
"wireless mouse" -> #1 #4
"usb c fast" -> #3 #5
```

```
"gaming" -> #2 #6
```

المخطط الإنسيابي (Organigramme)



الفصل 12

القوائم المتصلة

تعريفات

- **القائمة المتصلة:** سلسلة من **العقد** المحجوزة ديناميكيًا، كل عقدة تحمل بياناتها ومؤشرًا `next` على العقدة التالية.
- **الرأس** `head` يشير إلى أول عقدة، وآخر عقدة `next = NULL`.
- **التعريف:** `typedef struct Node { int code; struct Node *next; } Node;`
- **القائمة المزدوجة:** مؤشران `prev` و`next`. **القائمة الدائرية:** آخر عقدة تشير إلى الأولى.



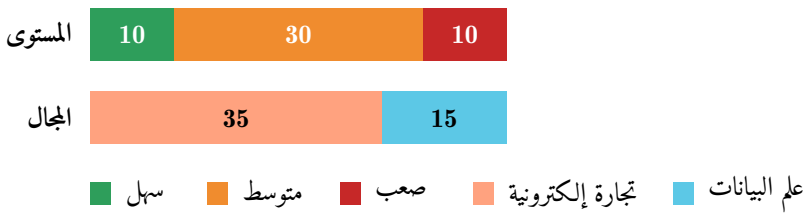
العملية	التكلفة والفكرة
إدراج / حذف في الرأس	وقت ثابت: نغير مؤشرين فقط
إدراج في النهاية	ثابت إذا احتفظنا بمؤشر `tail`
بحث، وصول إلى العنصر رقم `k`	خطي: نمر من الرأس عقدة بعقدة
حذف عقدة معلومة	ثابت في القائمة المزدوجة، يحتاج إلى السابقة في الأحادية

حيل مفيدة

العقدة الحارسة `dummy` تلغي حالة الرأس الخاصة، **والمؤشر على مؤشر** `Node **link` يوحد حذف الرأس وحذف عقدة في الوسط، **والمؤشران السريع والبطيء** يجدان المنتصف أو الحلقة في مرور واحد. احفظ دائمًا `next` قبل `free`.

للاستزادة: [11, 8, 6]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 12.1 أول قائمة متصلة للمنتجات

تجارة إلكترونية سهل

عرّف عقدة Node (رمز المنتج، السعر، مؤشر على العقدة التالية). أنشئ ثلاث عقد بـ malloc، اربطها يدويًا، ثم امرّ على القائمة من الرأس حتى NULL لعرضها، وحرر العقد.

خطوات الحل

1) كل عقدة تحتوي على بياناتها و next؛ العقدة الأخيرة next = NULL.
2) المرور:

```
for (walk = head; walk != NULL; walk = walk->next)
```

.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; double price; struct Node *next; } Node;

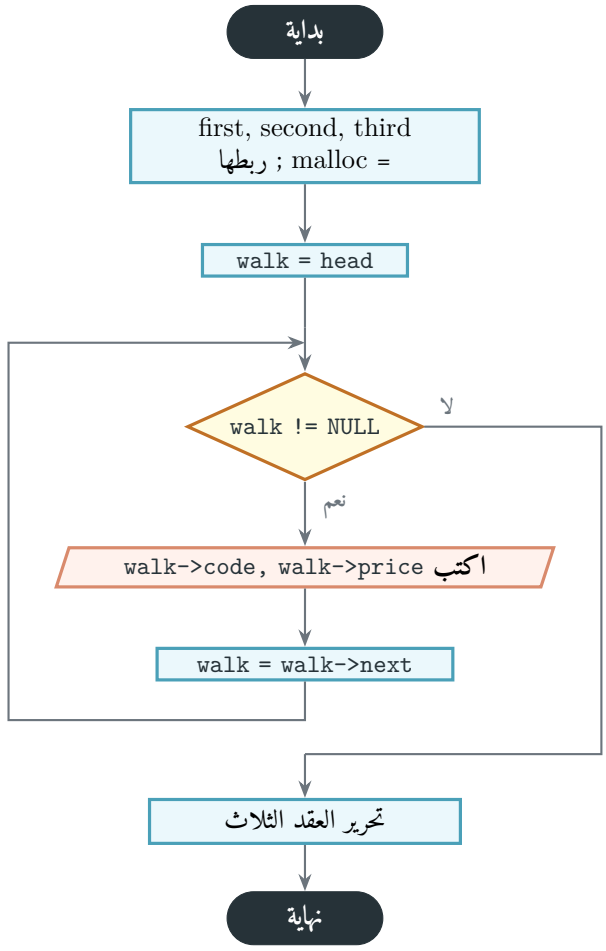
Node *newNode(int code, double price) {
    Node *node = malloc(sizeof(Node));
    node->code = code;
    node->price = price;
    node->next = NULL;
    return node;
}

int main(void) {
    Node *head = newNode(101, 1500);
    Node *walk;
    head->next = newNode(205, 32000);          // الربط يدويًا
    head->next->next = newNode(330, 450);
    for (walk = head; walk != NULL; walk = walk->next)
        printf("[%d %.0f] -> ", walk->code, walk->price);
    printf("NULL\n");
    free(head->next->next);
    free(head->next);
    free(head);
    return 0;
}
```

نتيجة التنفيذ

```
[101 1500] -> [205 32000] -> [330 450] -> NULL
```

المخطط الإنسيابي (Organigramme)



تمرين 12.2 الإدراج في رأس القائمة

تجارة إلكترونية سهل

اكتب دالة `pushFront(head, number)` تضيف طلبية جديدة في رأس القائمة وتعيد الرأس الجديد. أضف 5 طلبيات ولاحظ أن ترتيب العرض معكوس (آخر طلبية أولًا).

خطوات الحل

1) العقدة الجديدة تشير إلى الرأس القديم: `node->next = head`.
2) الرأس الجديد هو العقدة الجديدة.
3) الإدراج في الرأس يتم في وقت ثابت.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; struct Node *next; } Node;

Node *pushFront(Node *head, int number) {
    Node *node = malloc(sizeof(Node));
    node->number = number;
```

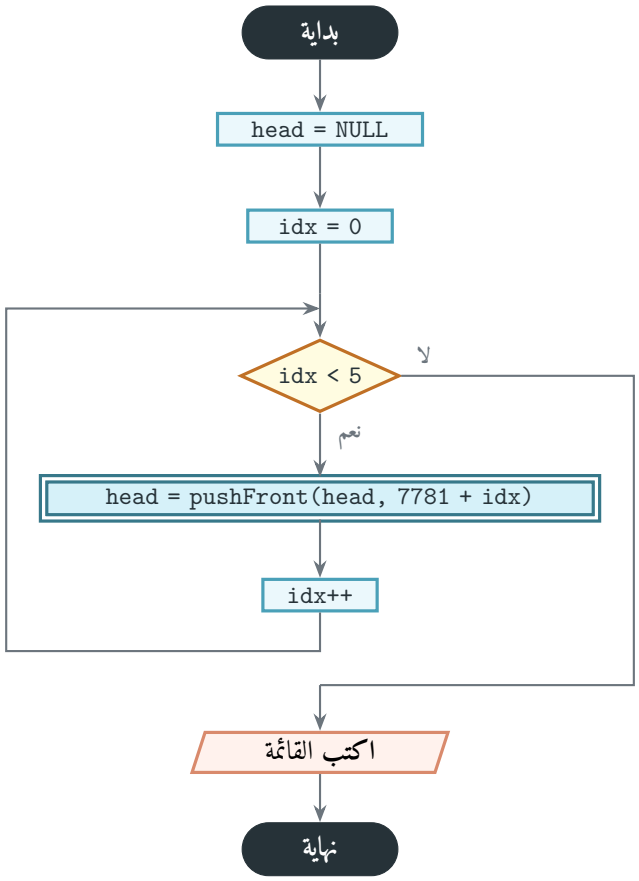
```c
    node->next = head;                  // تشير إلى الرأس القديم
    return node;                        // الرأس الجديد
}

int main(void) {
    Node *head = NULL, *walk, *next;
    int idx;
    for (idx = 0; idx < 5; idx++) head = pushFront(head, 7781 + idx);
    for (walk = head; walk != NULL; walk = walk->next) printf("#%d -> ", walk->number);
    printf("NULL\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
#7785 -> #7784 -> #7783 -> #7782 -> #7781 -> NULL
```

المخطط الإنسيابي (Organigramme)



تمرين 12.3 عدد العقد ومجموع الأسعار

علم البيانات سهل

اكتب دالتين: `length(head)` تعد عقد القائمة و `sumPrices(head)` تجمع أسعارها، ثم احسب متوسط سعر المنتجات في سلة مبنية كقائمة متصلة.

خطوات الحل

1) مرور واحد لكل دالة حتى NULL.
2) المتوسط = المجموع / العدد (مع التحقق من أن القائمة غير فارغة).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { double price; struct Node *next; } Node;

Node *pushFront(Node *head, double price) {
    Node *node = malloc(sizeof(Node));
    node->price = price;
    node->next = head;
    return node;
}

int length(const Node *head) {
    int count = 0;
    for (; head != NULL; head = head->next) count++;
    return count;
}

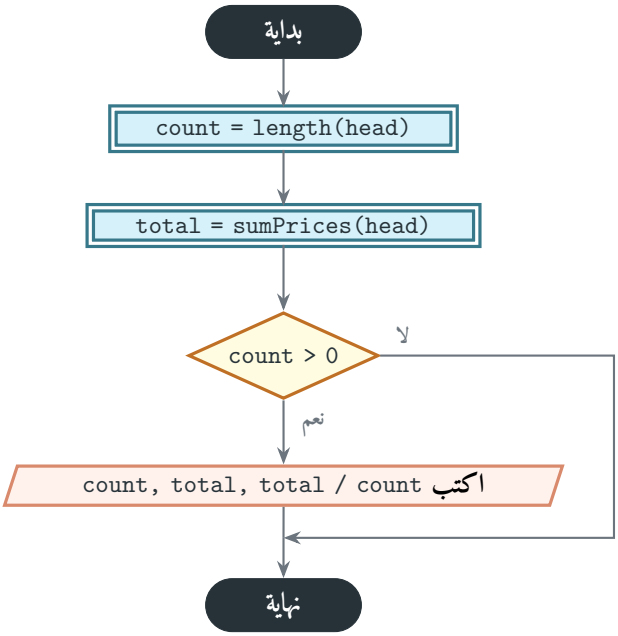
double sumPrices(const Node *head) {
    double total = 0;
    for (; head != NULL; head = head->next) total += head->price;
    return total;
}

int main(void) {
    double prices[6] = {1500, 3200, 899, 450, 12500, 2200};
    Node *head = NULL, *next;
    int idx, count;
    for (idx = 0; idx < 6; idx++) head = pushFront(head, prices[idx]);
    count = length(head);
    if (count > 0)
        printf("%d items, total %.2f, average %.2f DA\n", count, sumPrices(head), sumPrices(head) / count);
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

نتيجة التنفيذ

```
6 items, total 20749.00, average 3458.17 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 12.4 البحث عن منتج في القائمة

تجارة إلكترونية سهل

اكتب دالة `find(head, code)` تعيد **مؤشرًا على العقدة** التي تحمل الرمز المطلوب أو `NULL`. استعمل المؤشر المعاد لتعديل مخزون المنتج مباشرة داخل القائمة.

خطوات الحل

1) نمر حتى نجد الرمز أو نصل إلى `NULL`.
2) العقدة المعادة جزء من القائمة: تعديلها يعدل القائمة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code, stock; struct Node *next; } Node;

Node *pushFront(Node *head, int code, int stock) {
    Node *node = malloc(sizeof(Node));
    node->code = code; node->stock = stock; node->next = head;
    return node;
}

Node *find(Node *head, int code) {
    for (; head != NULL; head = head->next)
        if (head->code == code) return head;      // العقدة نفسها
    return NULL;
}

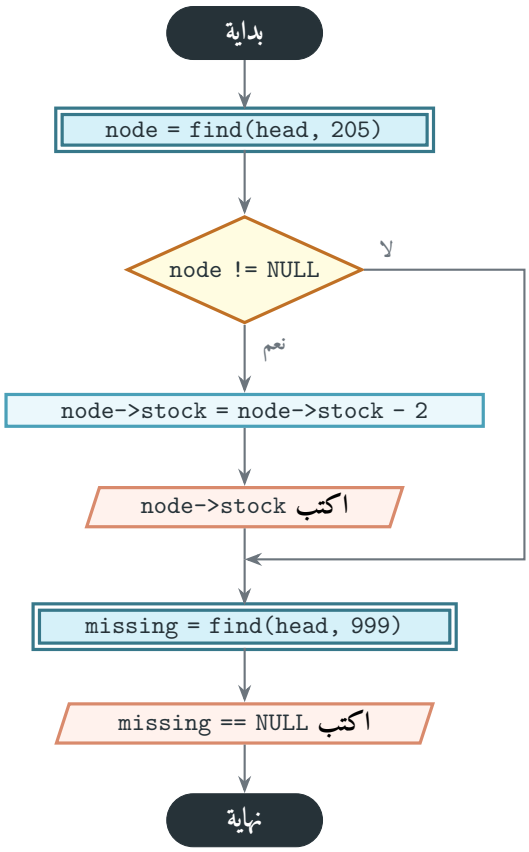
int main(void) {
    Node *head = NULL, *node, *next;
    head = pushFront(head, 101, 12);
    head = pushFront(head, 205, 5);
    head = pushFront(head, 330, 40);
```

```c
    node = find(head, 205);
    if (node != NULL) {
        node->stock -= 2;                          // تعديل داخل القائمة
        printf("Product 205: stock now %d\n", node->stock);
    }
    printf("Product 999: %s\n", find(head, 999) == NULL ? "not found" : "found");
    printf("Check via list: 205 -> %d\n", find(head, 205)->stock);
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

نتيجة التنفيذ

```
Product 205: stock now 3
Product 999: not found
Check via list: 205 -> 3
```

المخطط الإنسيابي (Organigramme)



تمرين 12.5 طابور الطلبيات بمؤشر الذيل

تجارة إلكترونية ⭐ سهل

للحفاظ على ترتيب الوصول، أضف الطلبيات في نهاية القائمة. لتفادي المرور على كل القائمة في كل إضافة، احتفظ بمؤشر `tail` على العقدة الأخيرة. اكتب `pushBack(&head, &tail, number)`.

خطوات الحل

1) إذا كانت القائمة فارغة: `head = tail = node`.
2) وإلا: `tail->next = node` ثم `tail = node`.

3) الدالة تعدل `head` و `tail`، لذلك نمرر عنوانيهما.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; struct Node *next; } Node;

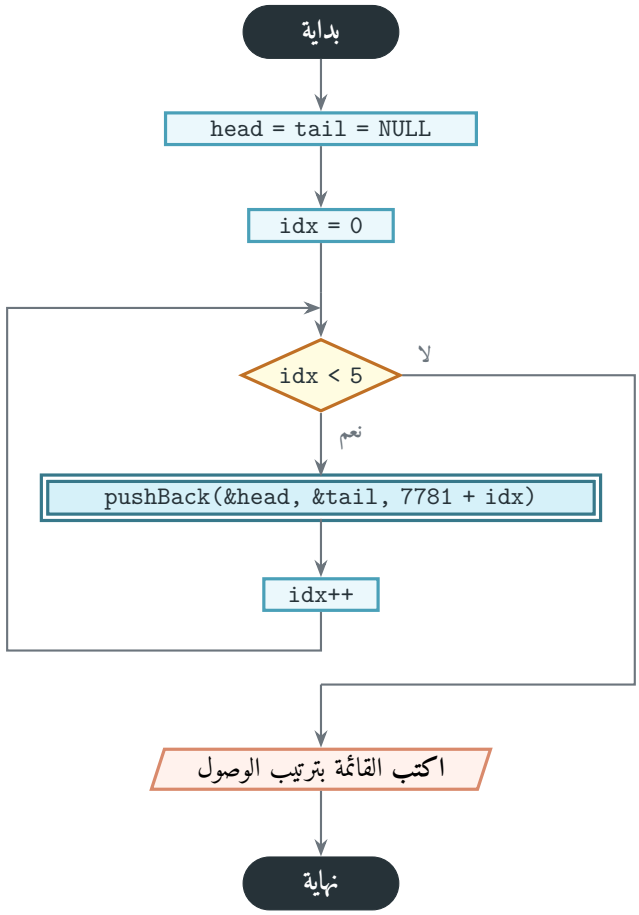
void pushBack(Node **head, Node **tail, int number) {
    Node *node = malloc(sizeof(Node));
    node->number = number;
    node->next = NULL;
    if (*head == NULL) *head = *tail = node;      // أول عقدة
    else { (*tail)->next = node; *tail = node; }  // بعد الأخيرة مباشرة
}

int main(void) {
    Node *head = NULL, *tail = NULL, *walk, *next;
    int idx;
    for (idx = 0; idx < 5; idx++) pushBack(&head, &tail, 7781 + idx);
    for (walk = head; walk != NULL; walk = walk->next) printf("#%d -> ", walk->number);
    printf("NULL\nhead #%d, tail #%d\n", head->number, tail->number);
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
#7781 -> #7782 -> #7783 -> #7784 -> #7785 -> NULL
head #7781, tail #7785
```

المخطط الإنسيابي (Organigramme)



تمرين 12.6 تحرير القائمة كاملة

تجارة إلكترونية سهل

اكتب دالة `freeList(head)` تحرر كل عقد القائمة **بالترتيب الصحيح** وتعيد عدد العقد المحررة. لماذا لا يمكن كتابة `free(walk); walk = walk->next;`؟

خطوات الحل

1) بعد `free(walk)` لا يجوز قراءة `walk->next`: نحفظه قبل التحرير.

2) `next = walk->next; free(walk); walk = next;`

.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *next; } Node;

int freeList(Node *head) {
    Node *walk = head, *next;
    int count = 0;
    while (walk != NULL) {
        next = walk->next;          // نحفظ التالي قبل التحرير
        free(walk);
```

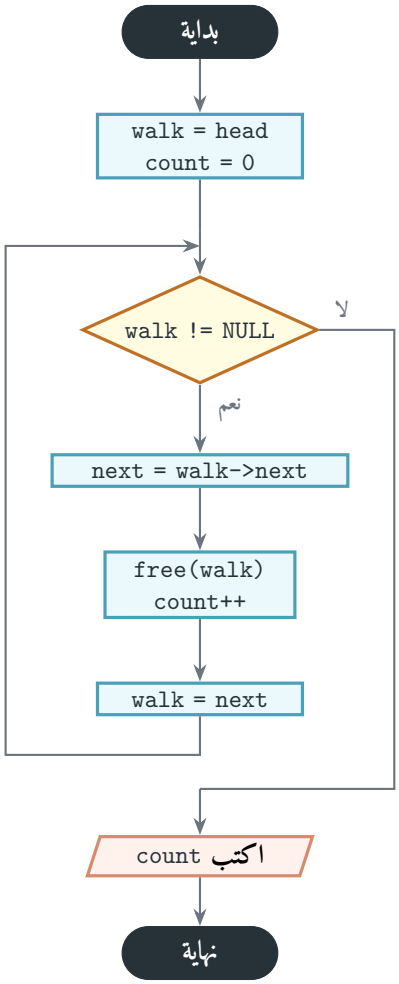
```c
        count++;
        walk = next;
    }
    return count;
}

int main(void) {
    Node *head = NULL, *node;
    int idx;
    for (idx = 0; idx < 7; idx++) {
        node = malloc(sizeof(Node));
        node->code = 100 + idx;
        node->next = head;
        head = node;
    }
    printf("Freed %d nodes\n", freeList(head));
    head = NULL;                          // القائمة أصبحت فارغة
    printf("List empty: %s\n", head == NULL ? "yes" : "no");
    return 0;
}
```

نتيجة التنفيذ

```
Freed 7 nodes
List empty: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 12.7 أغلى منتج في القائمة

تجارة إلكترونية

سهل

اكتب دالة `mostExpensive(head)` تعيد مؤشرًا على العقدة ذات أكبر سعر (أو `NULL` لقائمة فارغة)، واعرض اسمها وسعرها.

خطوات الحل

1) `best = head` ثم نقارن كل عقدة تالية.
2) نعيد العقدة لا السعر فقط، لنصل إلى كل حقولها.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char name[12]; double price; struct Node *next; } Node;

Node *pushFront(Node *head, const char *name, double price) {
    Node *node = malloc(sizeof(Node));
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
```

```
    node->price = price;
    node->next = head;
    return node;
}

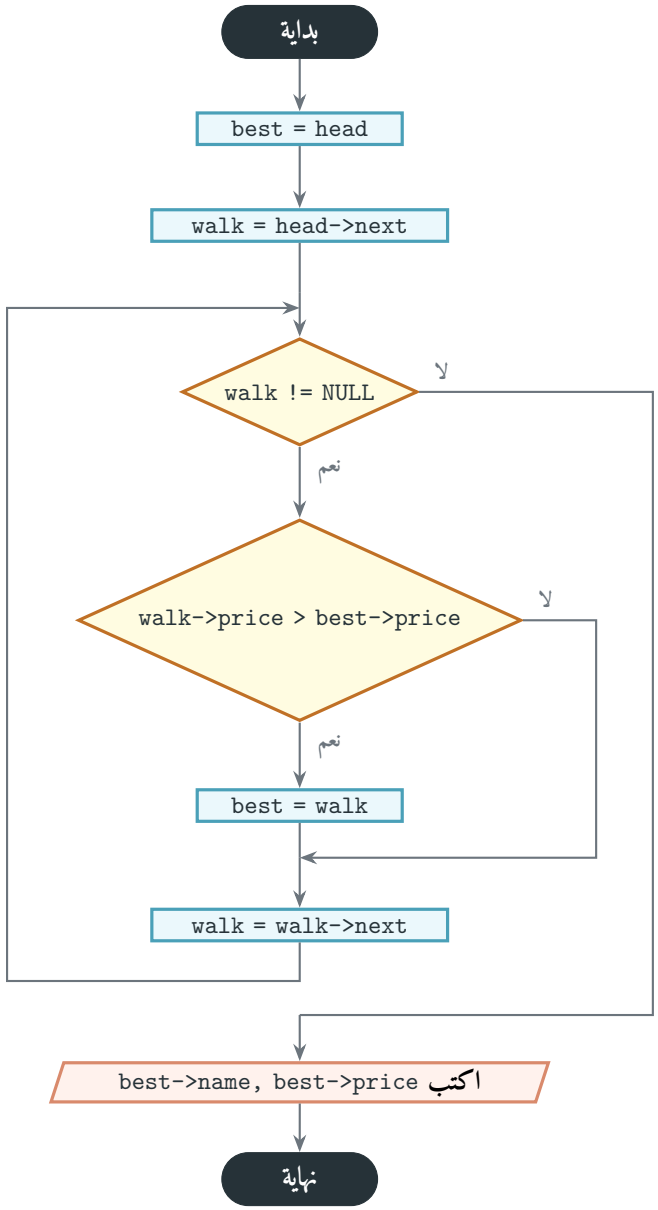
Node *mostExpensive(Node *head) {
    Node *best = head, *walk;
    if (head == NULL) return NULL;
    for (walk = head->next; walk != NULL; walk = walk->next)
        if (walk->price > best->price) best = walk;
    return best;
}

int main(void) {
    Node *head = NULL, *best, *next;
    head = pushFront(head, "Mouse", 1500);
    head = pushFront(head, "Laptop", 89000);
    head = pushFront(head, "Monitor", 32000);
    head = pushFront(head, "Cable", 450);
    best = mostExpensive(head);
    printf("Most expensive: %s (%.0f DA)\n", best->name, best->price);
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

نتيجة التنفيذ

```
Most expensive: Laptop (89000 DA)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.8 سحب الطلبية من رأس القائمة

تجارة إلكترونية

سهل

اكتب دالة `popFront(&head)` تحذف العقدة الأولى وتحررها وتعيد رقم الطلبية التي كانت فيها (أو -1 إذا كانت القائمة فارغة). عالج الطلبيات واحدة واحدة حتى تفرغ القائمة.

خطوات الحل

1) نحفظ العقدة الأولى، نجعل الرأس يشير إلى الثانية، ثم نحرر الأولى.
2) القائمة الفارغة حالة خاصة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>
```

```c

typedef struct Node { int number; struct Node *next; } Node;

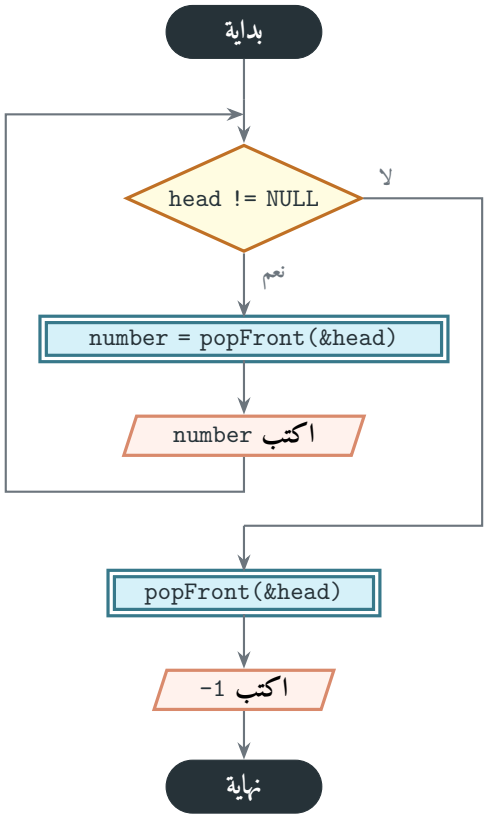
int popFront(Node **head) {
    Node *first = *head;
    int number;
    if (first == NULL) return -1;              // قائمة فارغة
    number = first->number;
    *head = first->next;                       // الرأس ينتقل إلى الثانية
    free(first);
    return number;
}

int main(void) {
    Node *head = NULL, *node;
    int idx;
    for (idx = 3; idx >= 0; idx--) {
        node = malloc(sizeof(Node));
        node->number = 7781 + idx;
        node->next = head;
        head = node;
    }
    while (head != NULL) printf("Processing order #%d\n", popFront(&head));
    printf("Pop on empty list: %d\n", popFront(&head));
    return 0;
}
```

نتيجة التنفيذ

```
Processing order #7781
Processing order #7782
Processing order #7783
Processing order #7784
Pop on empty list: -1
```

المخطط الانسيابي (Organigramme)



تمرين 12.9 عرض القائمة معكوسة عوديًا

علم البيانات
سهل

القائمة الأحادية لا تسمح بالرجوع إلى الخلف. اكتب دالة **عودية** `printReverse(node)` تعرض القائمة من آخرها إلى أولها دون تعديلها: نستدعي على العقدة التالية أولًا ثم نطبع.

خطوات الحل

1) الحالة الأساسية: `node == NULL`.
2) الطباعة بعد الاستدعاء تعكس الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int day; double sales; struct Node *next; } Node;

void printReverse(const Node *node) {
    if (node == NULL) return;
    printReverse(node->next);                // الباقي أولًا
    printf("d%d:%.0f ", node->day, node->sales);
}

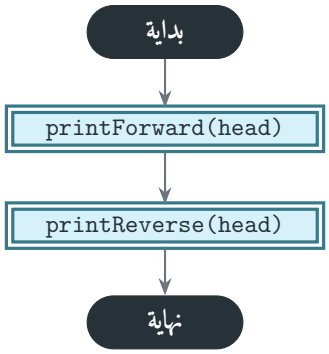
int main(void) {
    double sales[5] = {1200, 980, 1430, 1100, 1650};
    Node *head = NULL, *node, *walk, *next;
    int idx;
    for (idx = 4; idx >= 0; idx--) {
        node = malloc(sizeof(Node));
```

```
        node->day = idx + 1;
        node->sales = sales[idx];
        node->next = head;
        head = node;
    }
    printf("Forward: ");
    for (walk = head; walk != NULL; walk = walk->next) printf("d%d:%.0f ", walk->day, walk->sales);
    printf("\nReverse: ");
    printReverse(head);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
Forward: d1:1200 d2:980 d3:1430 d4:1100 d5:1650
Reverse: d5:1650 d4:1100 d3:1430 d2:980 d1:1200
```

المخطط الإنسيابي (Organigramme)



تمرين 12.10 بناء قائمة من جدول

تجارة إلكترونية سهل

اكتب دالة `fromArray(values, size)` تبني قائمة متصلة **بنفس ترتيب** الجدول (الإدراج من آخر عنصر إلى أوله في الرأس)، ودالة `printList` تعرضها بالأسهم.

خطوات الحل

1) الإدراج في الرأس يعكس الترتيب، لذلك نمر على الجدول من النهاية.
2) لا نحتاج إلى مؤشر ذيل بهذه الطريقة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *next; } Node;

Node *fromArray(const int *values, int size) {
    Node *head = NULL, *node;
    int idx;
    for (idx = size - 1; idx >= 0; idx--) {      // من النهاية للحفاظ على الترتيب
        node = malloc(sizeof(Node));
        node->code = values[idx];
        node->next = head;
        head = node;
```

```c
    }
    return head;
}

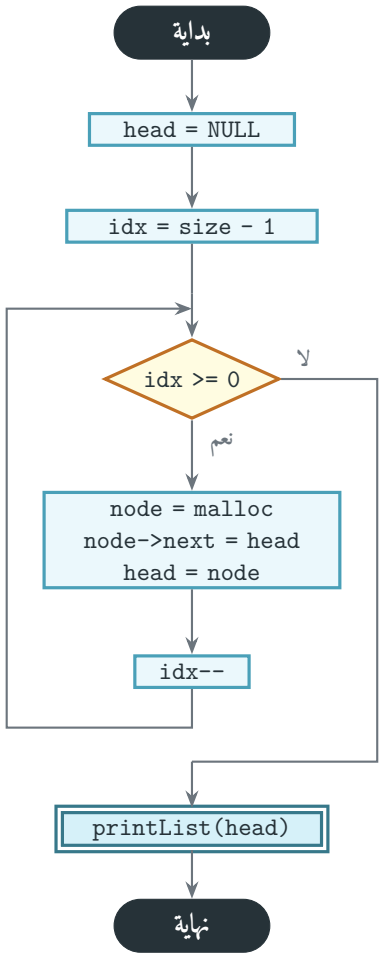
void printList(const Node *head) {
    for (; head != NULL; head = head->next) printf("%d -> ", head->code);
    printf("NULL\n");
}

int main(void) {
    int codes[6] = {101, 205, 330, 412, 500, 618};
    Node *head = fromArray(codes, 6), *next;
    printList(head);
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

نتيجة التنفيذ

```
101 -> 205 -> 330 -> 412 -> 500 -> 618 -> NULL
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 12.11 الإدراج المرتب حسب السعر

تجارة إلكترونية ★★ متوسط

اكتب دالة `insertSorted(head, name, price)` تدرج منتجًا في مكانه الصحيح لتبقى القائمة **مرتبة تصاعديًا** حسب السعر، وتعيد الرأس (قد يتغير). أدرج منتجات بترتيب عشوائي.

خطوات الحل

1) حالة خاصة: قائمة فارغة أو سعر أصغر من الرأس ← إدراج في الرأس.
2) وإلا تقدم بـ `walk` ما دام `walk->next->price < price`، ثم ندرج بعد `walk`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char name[12]; double price; struct Node *next; } Node;

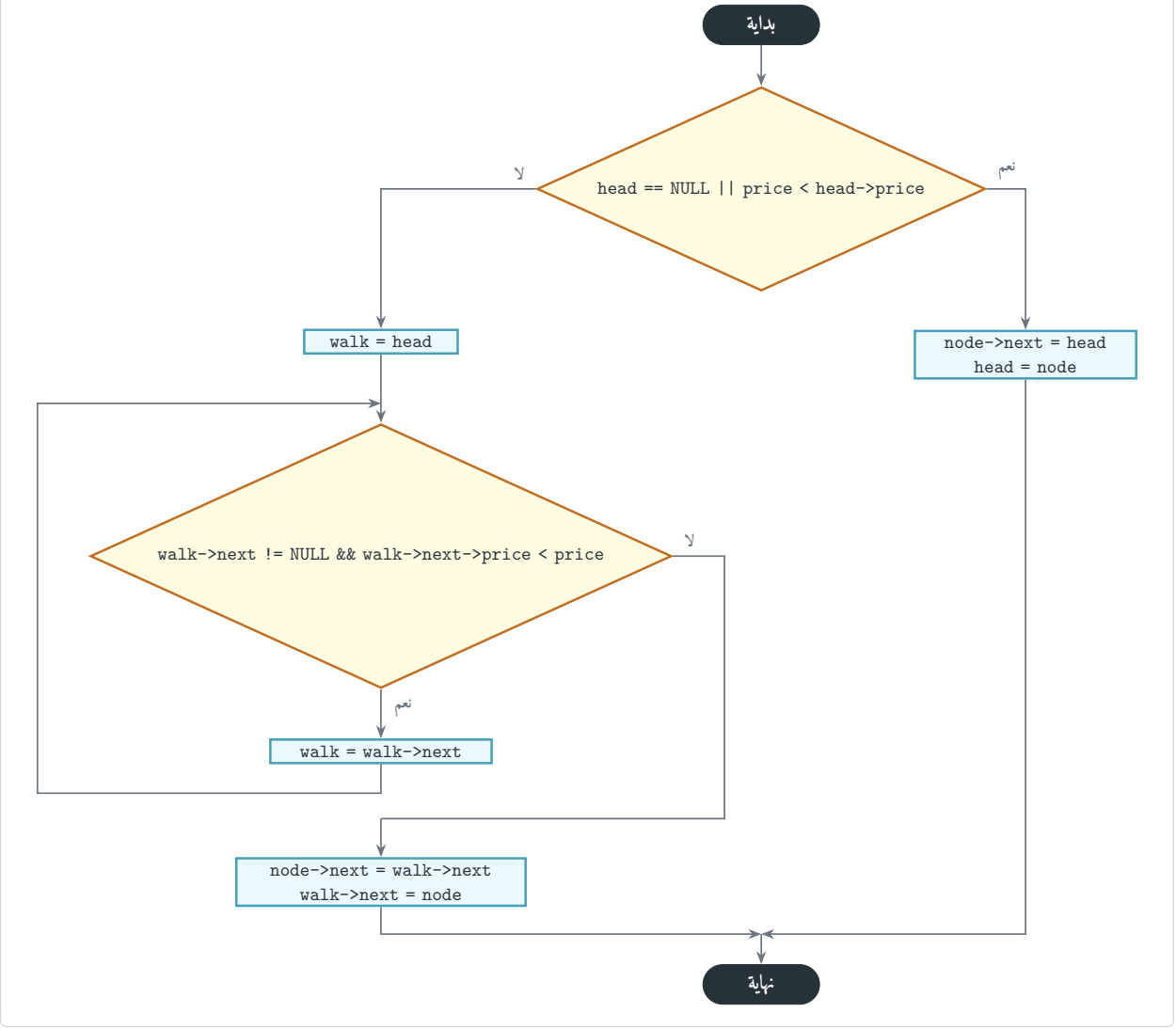
Node *insertSorted(Node *head, const char *name, double price) {
    Node *node = malloc(sizeof(Node)), *walk;
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
    node->price = price;
    if (head == NULL || price < head->price) {        // في الرأس
        node->next = head;
        return node;
    }
    walk = head;
    while (walk->next != NULL && walk->next->price < price) walk = walk->next;
    node->next = walk->next;                          // الربط بالتالي أولًا
    walk->next = node;
    return head;
}

int main(void) {
    const char *names[6] = {"Monitor", "Mouse", "Laptop", "Cable", "Keyboard", "Webcam"};
    double prices[6] = {32000, 1500, 89000, 450, 3200, 4100};
    Node *head = NULL, *walk, *next;
    int idx;
    for (idx = 0; idx < 6; idx++) head = insertSorted(head, names[idx], prices[idx]);
    for (walk = head; walk != NULL; walk = walk->next) printf("%s(%.0f) ", walk->name, walk->price);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
Cable(450) Mouse(1500) Keyboard(3200) Webcam(4100) Monitor(32000) Laptop(89000)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.12 حذف منتج برمزه

تجارة إلكترونية

متوسط

اكتب دالة `removeCode(head, code)` تحذف العقدة ذات الرمز المطلوب أينما كانت (رأس، وسط، نهاية) وتحررها، وتعيد الرأس الجديد. اختبر الحالات الثلاث وحالة الرمز غير الموجود.

خطوات الحل

1) نحتاج إلى العقدة السابقة `prev` لنربطها بعقدة ما بعد المحذوفة.

2) إذا كانت المحذوفة هي الرأس: الرأس الجديد `head->next`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *next; } Node;

Node *removeCode(Node *head, int code) {
    Node *prev = NULL, *walk = head;
    while (walk != NULL && walk->code != code) { prev = walk; walk = walk->next; }
```

```c
    if (walk == NULL) return head;              // غير موجود
    if (prev == NULL) head = walk->next;        // حذف الرأس
    else prev->next = walk->next;               // تخطي العقدة
    free(walk);
    return head;
}

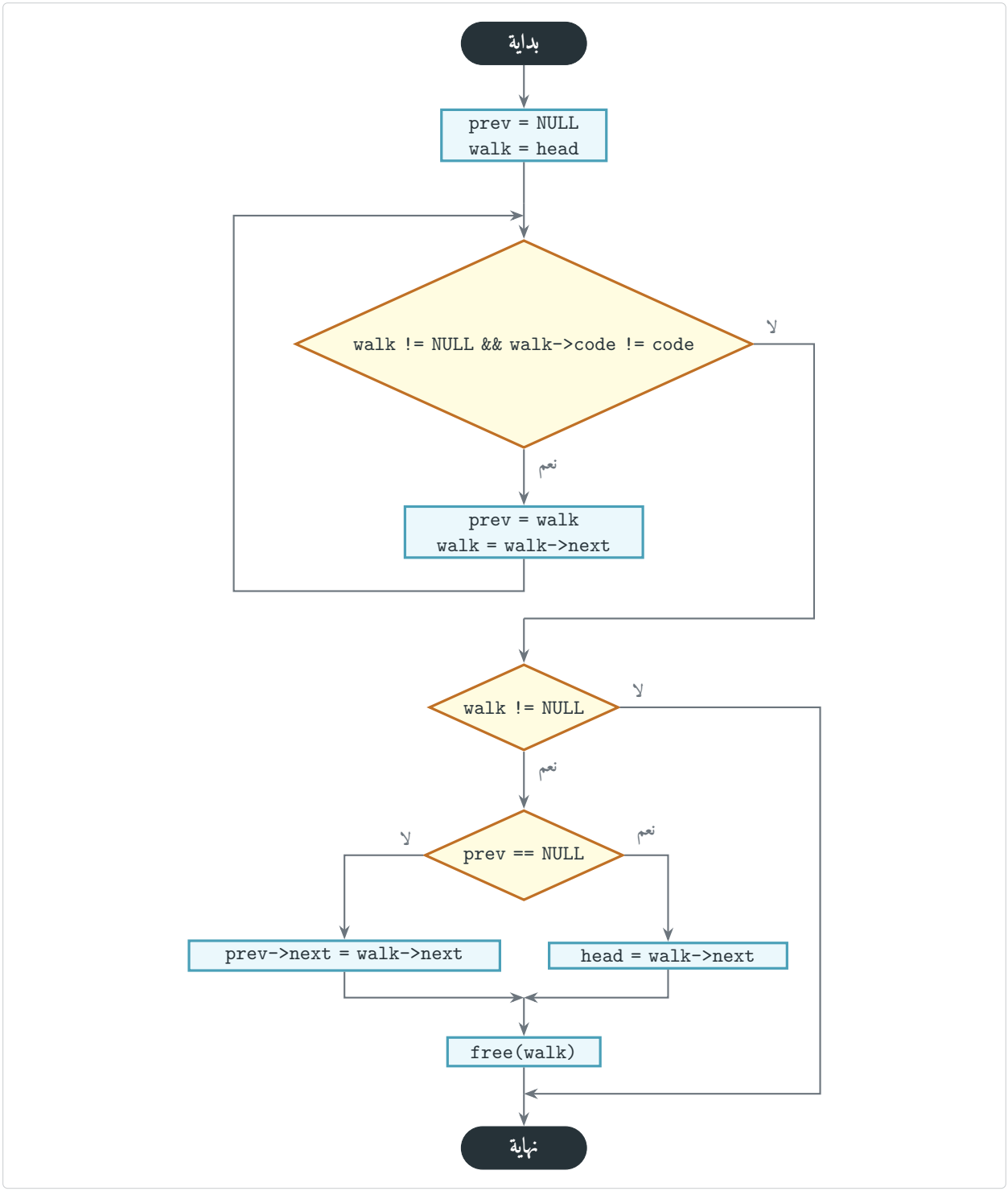
void printList(const char *label, const Node *head) {
    printf("%-12s", label);
    for (; head != NULL; head = head->next) printf("%d ", head->code);
    printf("\n");
}

int main(void) {
    int codes[5] = {101, 205, 330, 412, 500}, idx;
    Node *head = NULL, *node;
    for (idx = 4; idx >= 0; idx--) { node = malloc(sizeof(Node)); node->code = codes[idx]; node->next =
      ↪ head; head = node; }
    printList("start:", head);
    head = removeCode(head, 330); printList("remove 330:", head);
    head = removeCode(head, 101); printList("remove 101:", head);
    head = removeCode(head, 500); printList("remove 500:", head);
    head = removeCode(head, 999); printList("remove 999:", head);
    while (head != NULL) head = removeCode(head, head->code);
    return 0;
}
```

نتيجة التنفيذ

```
start:      101 205 330 412 500
remove 330: 101 205 412 500
remove 101: 205 412 500
remove 500: 205 412
remove 999: 205 412
```

المخطط الإنسيابي (Organigramme)



تمرين 12.13 قلب القائمة في مكانها

علم البيانات ★★ متوسط

اكتب دالة `reverse(head)` تقلب اتجاه كل الروابط **في مكانها** (دون حجز عقد جديدة) بثلاثة مؤشرات `prev` و `walk` و `next`، وتعيد الرأس الجديد. استعملها لعرض سجل الأسعار من الأحدث إلى الأقدم.

خطوات الحل

1) في كل خطوة: نحفظ `next = walk->next`، نقلب `walk->next = prev`، ثم نتقدم `prev = walk` و `walk = next`.
2) في النهاية `prev` هو الرأس الجديد.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int week; double price; struct Node *next; } Node;

Node *reverse(Node *head) {
    Node *prev = NULL, *walk = head, *next;
    while (walk != NULL) {
        next = walk->next;            // نحفظ الباقي
        walk->next = prev;            // نقلب الرابط
        prev = walk;
        walk = next;
    }
    return prev;
}

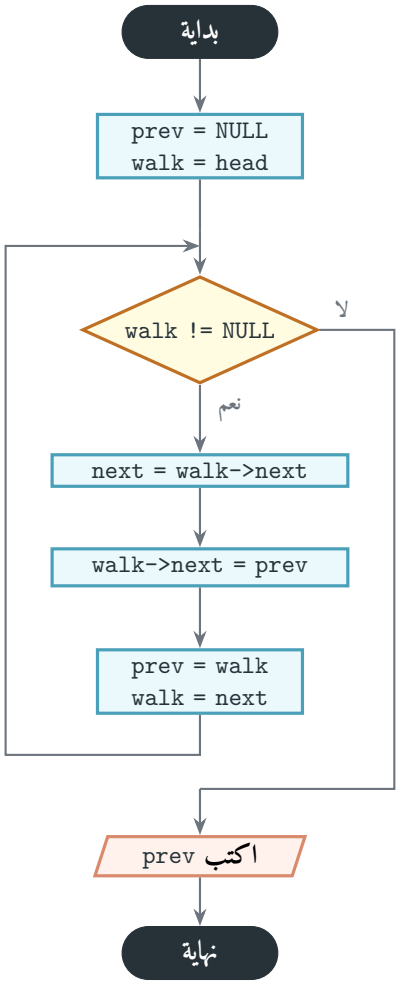
void printList(const Node *head) {
    for (; head != NULL; head = head->next) printf("w%d:%.0f ", head->week, head->price);
    printf("\n");
}

int main(void) {
    double prices[5] = {32000, 33500, 31000, 34500, 30000};
    Node *head = NULL, *node, *next;
    int idx;
    for (idx = 4; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->week = idx + 1; node->price = prices[idx];
        node->next = head; head = node;
    }
    printList(head);
    head = reverse(head);
    printList(head);
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

نتيجة التنفيذ

```
w1:32000 w2:33500 w3:31000 w4:34500 w5:30000
w5:30000 w4:34500 w3:31000 w2:33500 w1:32000
```

المخطط الإنسيابي (Organigramme)



تمرين 12.14 دمج قائمتين مرتبتين بالروابط

تجارة إلكترونية ★★ متوسط

قائمتا أسعار مرتبتان من موزّدين. اكتب `mergeLists(first, second)` تدمجهما في قائمة مرتبة واحدة **بإعادة ربط العقد** الموجودة (دون `malloc`)، مستعملة **عقدة حارسة** وهمية لتبسيط الكود.

خطوات الحل

1) `dummy` عقدة محلية، `tail` يشير إلى آخر عقدة في النتيجة.
2) نربط `tail->next` بالأصغر بين رأسي القائمتين ونتقدم فيها.
3) في النهاية نربط باقي القائمة غير المنتهية، النتيجة `dummy.next`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int price; struct Node *next; } Node;

Node *mergeLists(Node *first, Node *second) {
    Node dummy, *tail = &dummy;                 // عقدة حارسة
    while (first != NULL && second != NULL) {
        if (first->price <= second->price) { tail->next = first; first = first->next; }
```

```
        else { tail->next = second; second = second->next; }
        tail = tail->next;
    }
    tail->next = first != NULL ? first : second;   // الباقي كما هو
    return dummy.next;
}

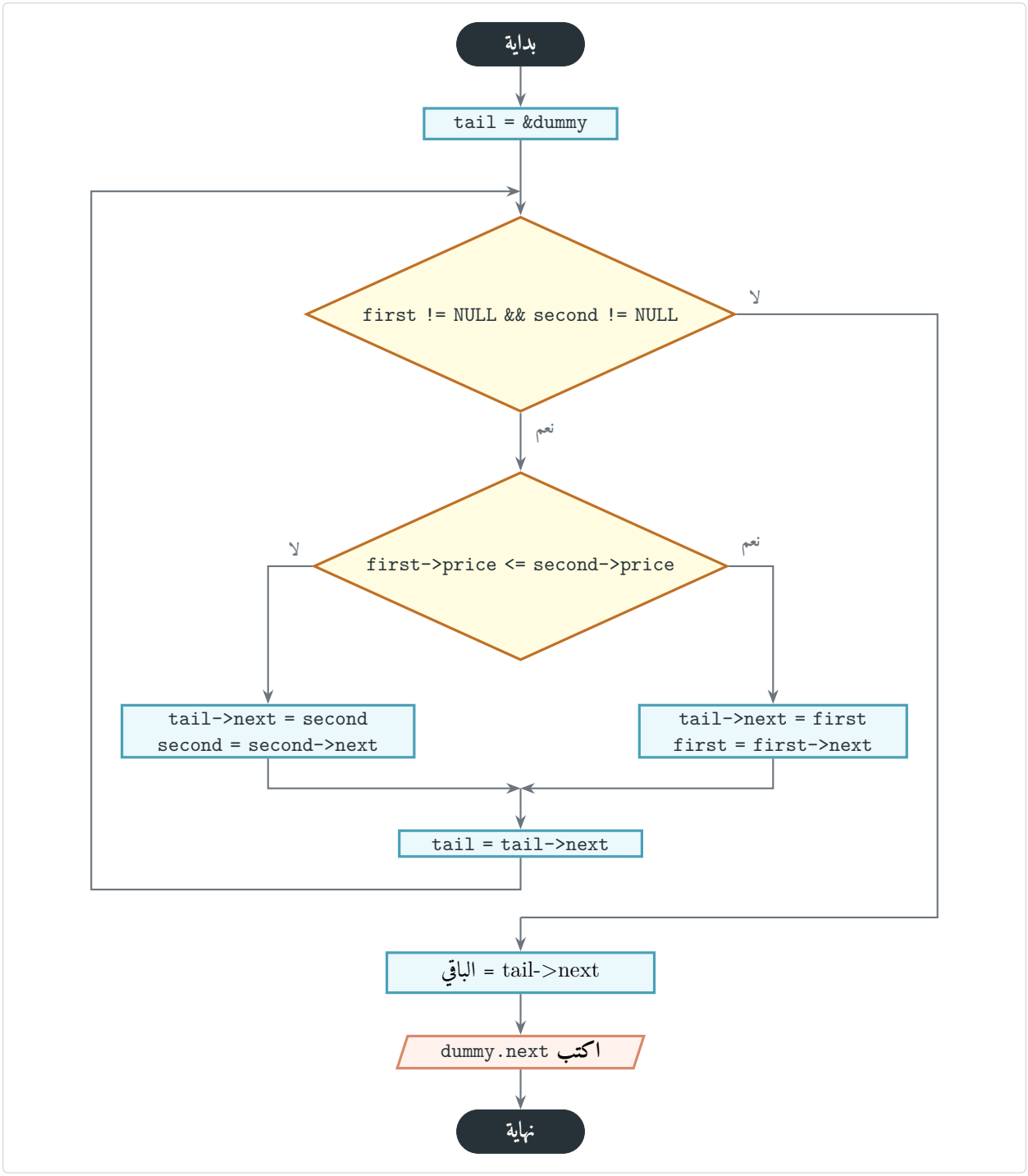
Node *fromArray(const int *values, int size) {
    Node *head = NULL, *node;
    while (size-- > 0) { node = malloc(sizeof(Node)); node->price = values[size]; node->next = head; head = node; }
    return head;
}

int main(void) {
    int supplierA[4] = {450, 1500, 3200, 7800}, supplierB[5] = {300, 899, 1200, 4100, 9900};
    Node *head = mergeLists(fromArray(supplierA, 4), fromArray(supplierB, 5)), *walk, *next;
    for (walk = head; walk != NULL; walk = walk->next) printf("%d ", walk->price);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
300 450 899 1200 1500 3200 4100 7800 9900
```

المخطط الإنسيابي (Organigramme)



تمرين 12.15 حذف المكررات من قائمة مرتبة

علم البيانات ★★ متوسط

قائمة معرّفات الزبائن الزائرين مرتبة وتحتوي على مكررات. اكتب `removeDuplicates(head)` تحذف العقد المكررة وتحررها، وتعيد عدد العقد المحذوفة.

خطوات الحل

1) بما أن القائمة مرتبة: المكررات متجاورة.
2) إذا كانت `walk->next` بنفس القيمة: تخطاها ونحررها، وإلا نتقدم.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int id; struct Node *next; } Node;

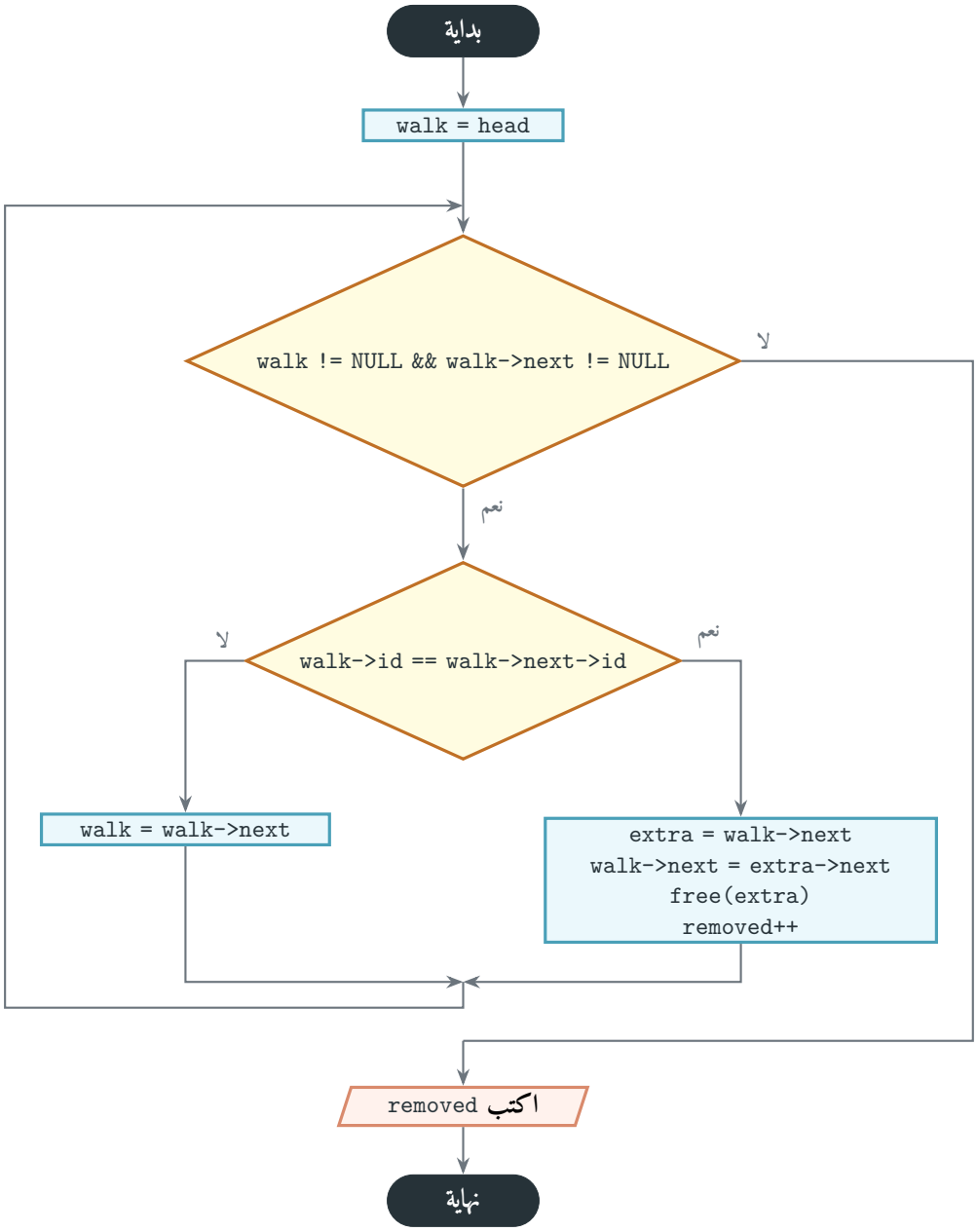
int removeDuplicates(Node *head) {
    Node *walk = head, *extra;
    int removed = 0;
    while (walk != NULL && walk->next != NULL) {
        if (walk->id == walk->next->id) {
            extra = walk->next;
            walk->next = extra->next;          // تخطى المكررة
            free(extra);
            removed++;
        } else {
            walk = walk->next;                 // تقدم فقط عند الاختلاف
        }
    }
    return removed;
}

int main(void) {
    int ids[12] = {501, 501, 502, 507, 507, 507, 510, 511, 511, 520, 520, 520};
    Node *head = NULL, *node, *walk, *next;
    int idx, removed;
    for (idx = 11; idx >= 0; idx--) { node = malloc(sizeof(Node)); node->id = ids[idx]; node->next = head;
      ↪ head = node; }
    removed = removeDuplicates(head);
    printf("Removed %d duplicates:", removed);
    for (walk = head; walk != NULL; walk = walk->next) printf(" %d", walk->id);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
Removed 6 duplicates: 501 502 507 510 511 520
```

المخطط الإنسيابي (Organigramme)



تمرين 12.16 العقدة الوسطى بمؤشرين

علم البيانات ★★ متوسط

لإيجاد وسيط قائمة طلبيات مرتبة دون معرفة طولها، استعمل مؤشرين: `slow` يتقدم بخطوة و `fast` بخطوتين. عندما يصل `fast` إلى النهاية يكون `slow` في المنتصف. جرّب على قائمة فردية وأخرى زوجية الطول.

خطوات الحل

1) `while (fast != NULL && fast->next != NULL)`:

```
slow = slow->next; fast = fast->next->next;
```

.

2) مرور واحد بدل مرورين (العد ثم التقدم إلى المنتصف).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int amount; struct Node *next; } Node;

Node *middle(Node *head) {
    Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;              // خطوة
        fast = fast->next->next;        // خطوتان
    }
    return slow;
}

Node *fromArray(const int *values, int size) {
    Node *head = NULL, *node;
    while (size-- > 0) { node = malloc(sizeof(Node)); node->amount = values[size]; node->next = head; head = node; }
    return head;
}

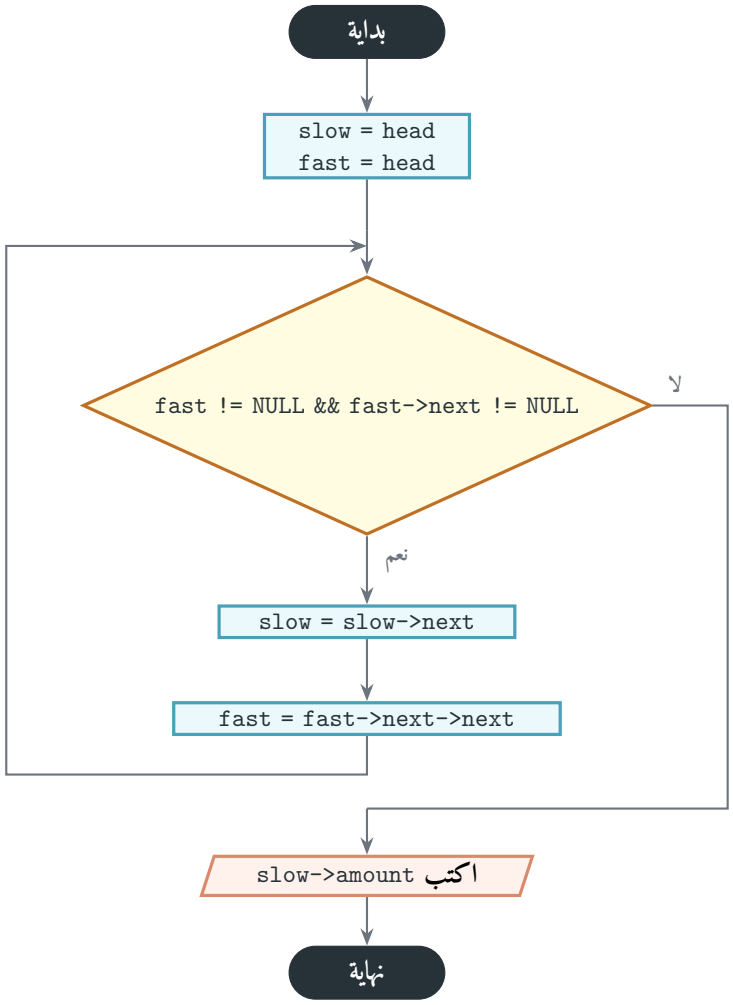
void freeList(Node *head) { Node *next; while (head != NULL) { next = head->next; free(head); head = next; } }

int main(void) {
    int oddList[7] = {450, 899, 1200, 1500, 3200, 4100, 7800};
    int evenList[6] = {300, 650, 990, 2200, 5400, 9900};
    Node *first = fromArray(oddList, 7), *second = fromArray(evenList, 6);
    printf("7 orders -> middle %d\n", middle(first)->amount);
    printf("6 orders -> middle (second of the two) %d\n", middle(second)->amount);
    freeList(first);
    freeList(second);
    return 0;
}
```

نتيجة التنفيذ

```
7 orders -> middle 1500
6 orders -> middle (second of the two) 2200
```

المخطط الانسيابي (Organigramme)



تمرين 12.17 كشف الحلقة في سلسلة التحويلات

علم البيانات ★★ متوسط

روابط إعادة التوجيه بين صفحات المنتجات تشكل قائمة؛ خطأ في الإعداد قد يصنع حلقة لا نهائية. اكتب `hasCycle(head)` بخوارزمية **فلويد** (السلحفاة والأرنب): إذا التقى المؤشران فهناك حلقة. احسب أيضًا طول الحلقة.

خطوات الحل

1) `slow` بخطوة و `fast` بخطوتين؛ إذا وصل `fast` إلى `NULL` فلا حلقة.
2) بعد اللقاء: نعد الخطوات حتى يعود أحدهما إلى نقطة اللقاء = طول الحلقة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int page; struct Node *next; } Node;

int cycleLength(Node *head) {
    Node *slow = head, *fast = head, *walk;
    int len;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
```

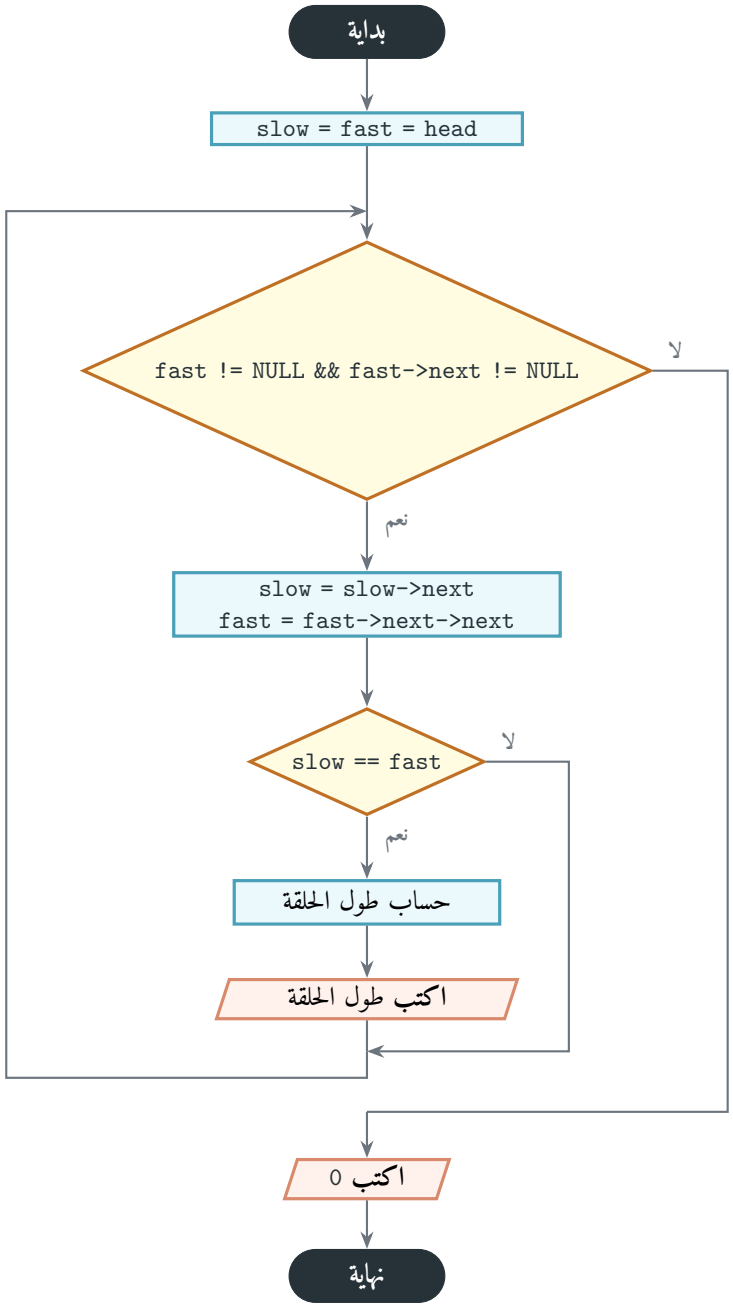
```c
        if (slow == fast) {                          // التقيا داخل الحلقة
            for (len = 1, walk = slow->next; walk != slow; walk = walk->next) len++;
            return len;
        }
    }
    return 0;                                        // وصلنا إلى NULL
}

int main(void) {
    Node nodes[7];
    int idx;
    for (idx = 0; idx < 7; idx++) { nodes[idx].page = 100 + idx; nodes[idx].next = idx < 6 ? &nodes[idx + 1]
      ↪ : NULL; }
    printf("Redirect chain OK: cycle length %d\n", cycleLength(&nodes[0]));
    nodes[6].next = &nodes[3];                       // خطأ: 106 -> 103
    printf("After misconfiguration: cycle length %d\n", cycleLength(&nodes[0]));
    return 0;
}
```

نتيجة التنفيذ

```
Redirect chain OK: cycle length 0
After misconfiguration: cycle length 4
```

المخطط الإنسيابي (Organigramme)



تمرين 12.18 العنصر رقم k من النهاية

تجارة إلكترونية

متوسط

اعرض الطلبية رقم k من النهاية في سجل اليوم بمرور واحد: نقدّم مؤشرًا أولًا k خطوات، ثم نحرك المؤشرين معًا حتى يصل الأول إلى النهاية.

خطوات الحل

1) إذا كانت القائمة أقصر من k: نعيد NULL.
2) المسافة الثابتة k بين المؤشرين تجعل الثاني على العنصر المطلوب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; struct Node *next; } Node;

Node *kthFromEnd(Node *head, int rank) {
    Node *lead = head, *follow = head;
    int step;
    for (step = 0; step < rank; step++) {
        if (lead == NULL) return NULL;          // القائمة أقصر من k
        lead = lead->next;
    }
    while (lead != NULL) { lead = lead->next; follow = follow->next; }   // نفس المسافة
    return follow;
}

int main(void) {
    Node *head = NULL, *node, *found, *next;
    int idx, rank;
    for (idx = 9; idx >= 0; idx--) { node = malloc(sizeof(Node)); node->number = 7781 + idx; node->next =
      ↪ head; head = node; }
    scanf("%d", &rank);
    found = kthFromEnd(head, rank);
    if (found != NULL) printf("Order %d from the end: #%d\n", rank, found->number);
    printf("Order 20 from the end: %s\n", kthFromEnd(head, 20) == NULL ? "none" : "?");
    while (head != NULL) { next = head->next; free(head); head = next; }
    return 0;
}
```

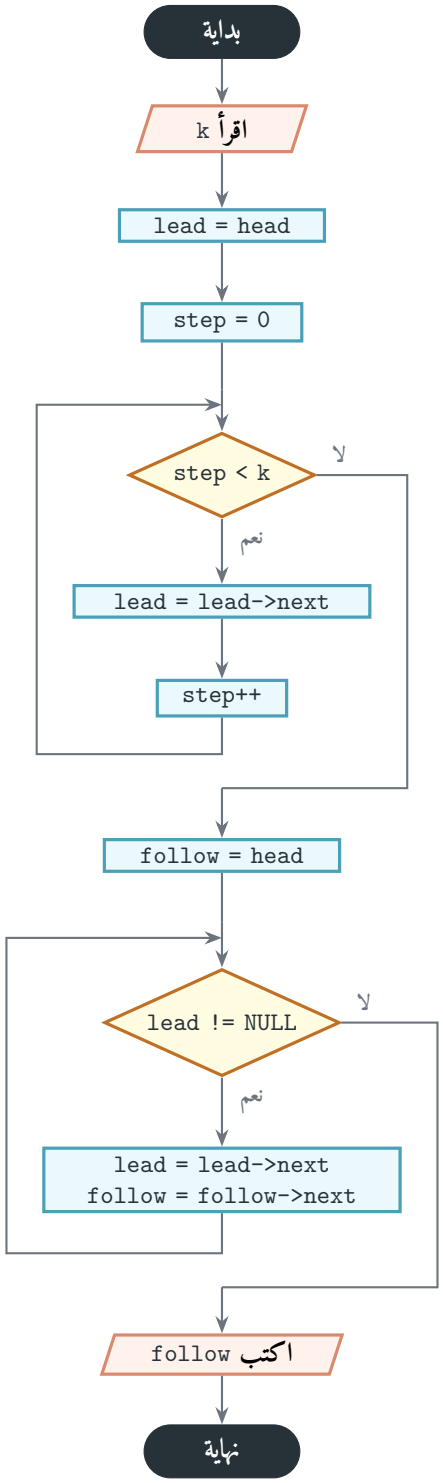
المدخلات

```
3
```

نتيجة التنفيذ

```
Order 3 from the end: #7788
Order 20 from the end: none
```

المخطط الإنسيابي (Organigramme)



تمرين 12.19 فصل الطلبيات حسب نوع التوصيل

تجارة إلكترونية ★★ متوسط

لدينا قائمة طلبيات واحدة، لكل طلبية نوع توصيل ('E' سريع، 'S' عادي). اكتب `splitByType(head, &express, &standard)` توزع العقد الموجودة على قائمتين مع **الحفاظ على الترتيب الأصلي**، دون حجز عقد جديدة.

خطوات الحل

1) عقدتان حارستان ومؤشرا ذيل لكل قائمة.
2) نفصل كل عقدة ونربطها في ذيل قائمتها.
3) في النهاية نضع NULL في ذيل كل قائمة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; char type; struct Node *next; } Node;

void splitByType(Node *head, Node **express, Node **standard) {
    Node dummyE, dummyS, *tailE = &dummyE, *tailS = &dummyS;
    while (head != NULL) {
        if (head->type == 'E') { tailE->next = head; tailE = head; }
        else { tailS->next = head; tailS = head; }
        head = head->next;
    }
    tailE->next = NULL;                          // نغلق القائمتين
    tailS->next = NULL;
    *express = dummyE.next;
    *standard = dummyS.next;
}

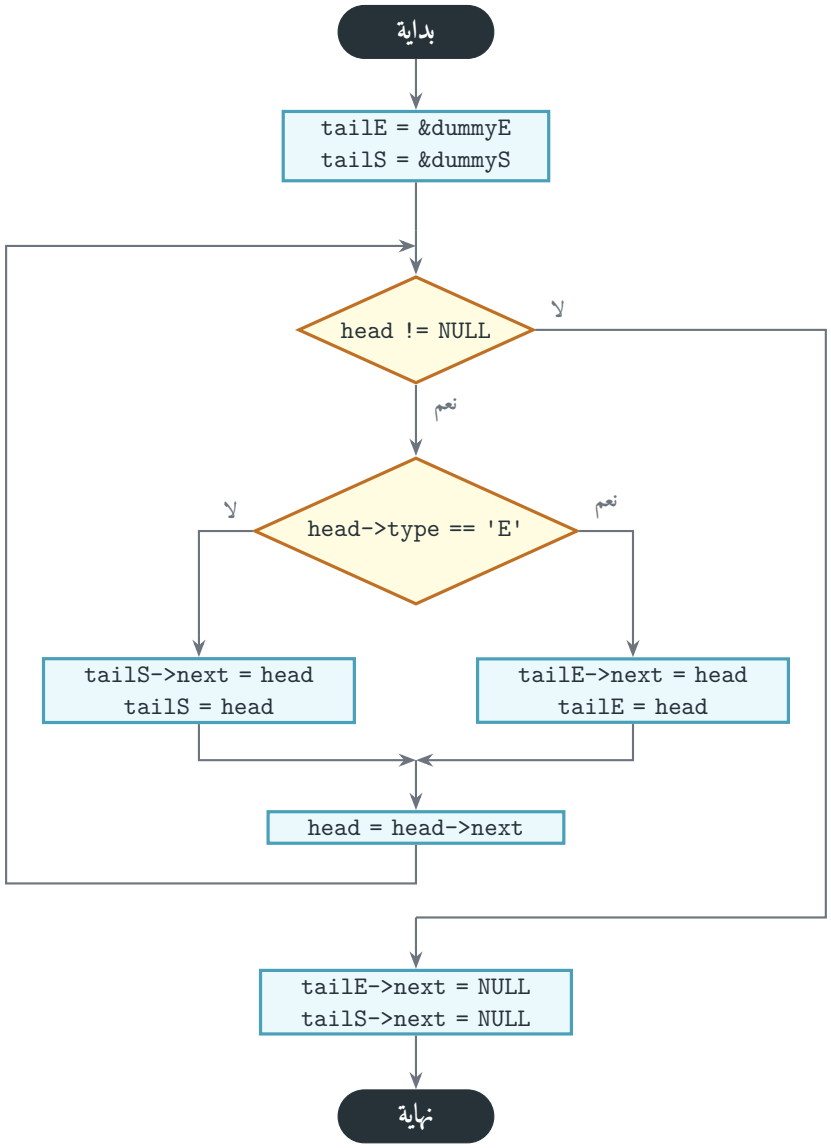
void printFree(const char *label, Node *head) {
    Node *next;
    printf("%s", label);
    for (; head != NULL; head = next) { printf(" #%d", head->number); next = head->next; free(head); }
    printf("\n");
}

int main(void) {
    char types[8] = {'S', 'E', 'S', 'S', 'E', 'E', 'S', 'E'};
    Node *head = NULL, *node, *express, *standard;
    int idx;
    for (idx = 7; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->number = 7781 + idx; node->type = types[idx];
        node->next = head; head = node;
    }
    splitByType(head, &express, &standard);
    printFree("Express: ", express);
    printFree("Standard:", standard);
    return 0;
}
```

نتيجة التنفيذ

```
Express:  #7782 #7785 #7786 #7788
Standard: #7781 #7783 #7784 #7787
```

المخطط الإنسيابي (Organigramme)



تمرين 12.20 سجل التصفح بقائمة مزدوجة

تجارة إلكترونية ★★ متوسط

زرّا رجوع وتقدم في متصفح المتجر: عرّف عقدة مزدوجة (prev و next)، وcurrent يشير إلى الصفحة الحالية. زيارة صفحة جديدة تحذف كل ما بعد current ثم تضيف الصفحة. نفّذ سلسلة أوامر.

خطوات الحل

1) v:page: نحرر العقد بعد current، ننشئ عقدة ونربطها في الاتجاهين.
2) b: current = current->prev إن وُجد؛ f: current = current->next إن وُجد.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Page { char name[16]; struct Page *prev, *next; } Page;

```

```c
int main(void) {
    char command[24];
    Page *current = NULL, *node, *walk, *next, *first = NULL;
    int pos;
    while (scanf("%23s", command) == 1) {
        if (command[0] == 'v') {
            if (current != NULL)                                    // نحذف تاريخ التقدم
                for (walk = current->next; walk != NULL; walk = next) { next = walk->next; free(walk); }
            node = malloc(sizeof(Page));
            for (pos = 0; (node->name[pos] = command[pos + 2]) != '\0'; pos++) {}
            node->prev = current;
            node->next = NULL;
            if (current != NULL) current->next = node; else first = node;
            current = node;
        } else if (command[0] == 'b' && current->prev != NULL) {
            current = current->prev;
        } else if (command[0] == 'f' && current->next != NULL) {
            current = current->next;
        }
        printf("%-10s -> now on %s\n", command, current->name);
    }
    for (walk = first; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

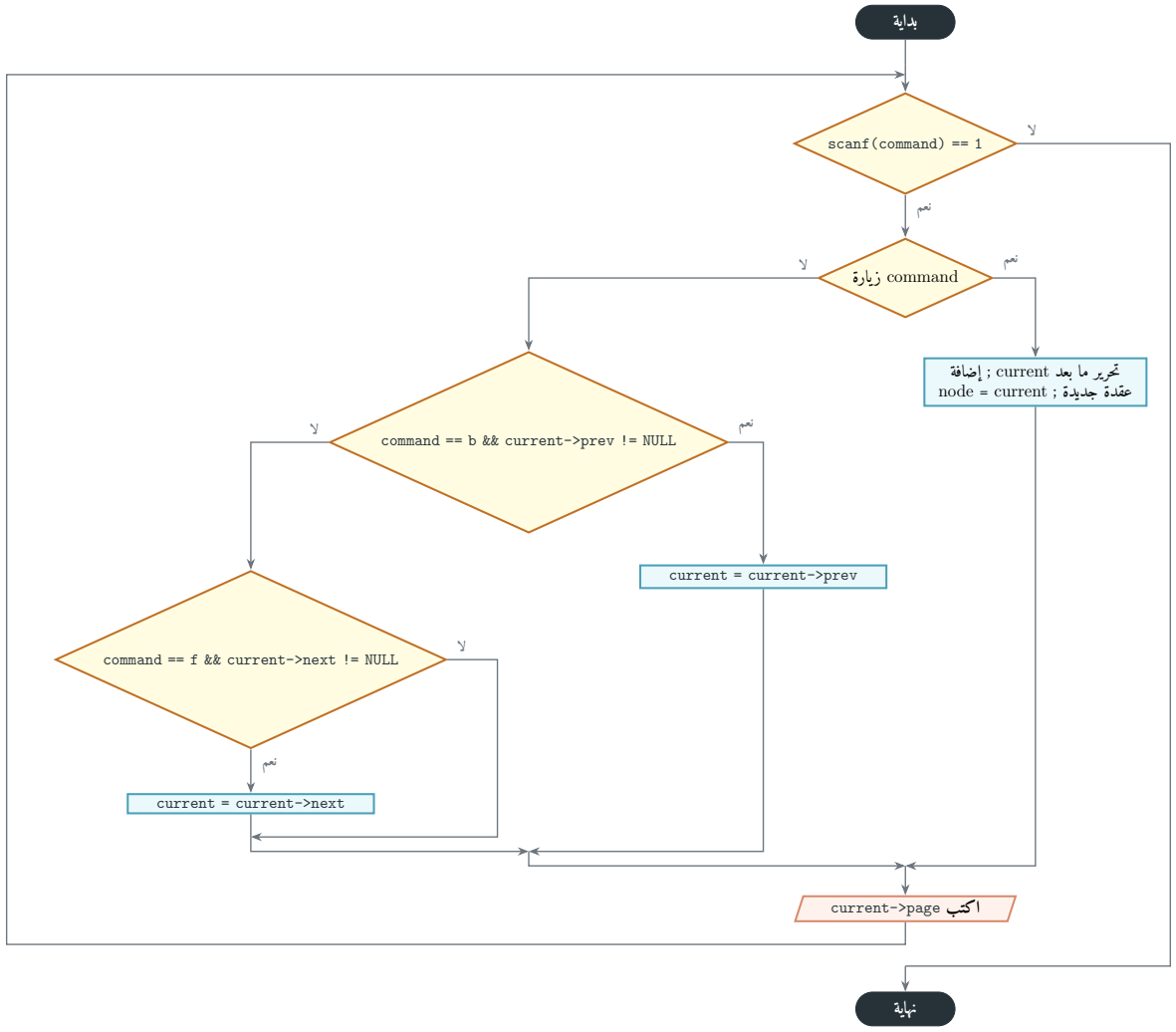
المدخلات

```
v:home v:phones v:galaxy b b v:laptops b f f
```

نتيجة التنفيذ

```
v:home     -> now on home
v:phones   -> now on phones
v:galaxy   -> now on galaxy
b          -> now on phones
b          -> now on home
v:laptops  -> now on laptops
b          -> now on home
f          -> now on laptops
f          -> now on laptops
```

المخطط الانسيابي (Organigramme)



تمرين 12.21 توزيع دائري للطلبيات على السائقين

تجارة إلكترونية ★★ متوسط

السائقون مرتبون في **قائمة دائرية** (آخر عقدة تشير إلى الأولى). تُسند كل طلبية جديدة إلى السائق التالي في الدور (Round Robin). اكتب `addDriver` (الإدراج بعد الذيل) و `nextDriver`، ثم وزّع 10 طلبيات وعدّ نصيب كل سائق.

خطوات الحل

1) نحتفظ بمؤشر `tail`؛ الرأس هو `tail->next`.
2) الإدراج:

```
node->next = tail->next; tail->next = node; tail = node
```

.
3) في القائمة الدائرية لا يوجد `NULL`: نتوقف عند العودة إلى البداية.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Driver { char name[10]; int load; struct Driver *next; } Driver;
```

```c

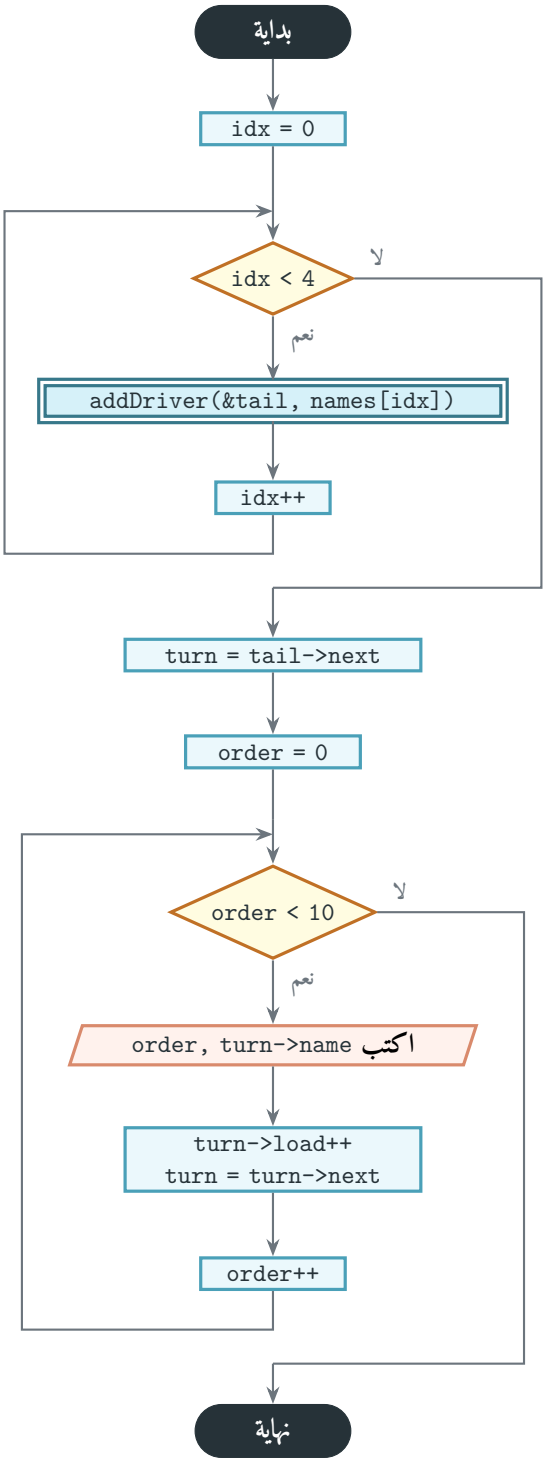
void addDriver(Driver **tail, const char *name) {
    Driver *node = malloc(sizeof(Driver));
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
    node->load = 0;
    if (*tail == NULL) node->next = node;               // سائق وحيد: يشير إلى نفسه
    else { node->next = (*tail)->next; (*tail)->next = node; }
    *tail = node;
}

int main(void) {
    const char *names[4] = {"Yacine", "Bilal", "Rania", "Hakim"};
    Driver *tail = NULL, *turn, *walk, *next;
    int idx, order;
    for (idx = 0; idx < 4; idx++) addDriver(&tail, names[idx]);
    turn = tail->next;                                  // الرأس
    for (order = 0; order < 10; order++) {
        printf("order #%d -> %s\n", 7781 + order, turn->name);
        turn->load++;
        turn = turn->next;                              // الدور التالي (دائري)
    }
    walk = tail->next;
    do { printf("%s: %d  ", walk->name, walk->load); walk = walk->next; } while (walk != tail->next);
    printf("\n");
    walk = tail->next;
    tail->next = NULL;                                  // نكسر الحلقة قبل التحرير
    for (; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
order #7781 -> Yacine
order #7782 -> Bilal
order #7783 -> Rania
order #7784 -> Hakim
order #7785 -> Yacine
order #7786 -> Bilal
order #7787 -> Rania
order #7788 -> Hakim
order #7789 -> Yacine
order #7790 -> Bilal
Yacine: 3  Bilal: 3  Rania: 2  Hakim: 2
```

المخطط الإنسيابي (Organigramme)



تمرين 12.22 مكدس التراجع بقائمة متصلة

تجارة إلكترونية

متوسط

محرر وصف المنتج يحفظ كل نسخة في **مكدس** مبني بقائمة متصلة (الإدراج والسحب من الرأس). اكتب `push(&top, price)` و `pop(&top)` و `peek(top)`، ونفّذ تعديلات على السعر ثم عمليات تراجع.

خطوات الحل

1) push = الإدراج في الرأس، pop = الحذف من الرأس: كلاهما في وقت ثابت.
2) القيمة الحالية هي قمة المكدس؛ التراجع يسحب القمة.
3) لا نسحب آخر نسخة (السعر الأصلي).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { double price; struct Node *next; } Node;

void push(Node **top, double price) {
    Node *node = malloc(sizeof(Node));
    node->price = price;
    node->next = *top;
    *top = node;
}

double pop(Node **top) {
    Node *old = *top;
    double price = old->price;
    *top = old->next;
    free(old);
    return price;
}

double peek(const Node *top) { return top->price; }

int main(void) {
    Node *top = NULL;
    char command[16];
    double value;
    int pos;
    push(&top, 3500);                                   // السعر الأصلي
    while (scanf("%15s", command) == 1) {
        if (command[0] == 'e') {
            for (value = 0, pos = 1; command[pos] != '\0'; pos++) value = value * 10 + (command[pos] - '0');
            push(&top, value);
        } else if (top->next != NULL) {
            pop(&top);                                  // تراجع
        } else {
            printf("%-6s nothing to undo\n", command);
            continue;
        }
        printf("%-6s price = %.0f\n", command, peek(top));
    }
    while (top != NULL) pop(&top);
    return 0;
}
```

المدخلات

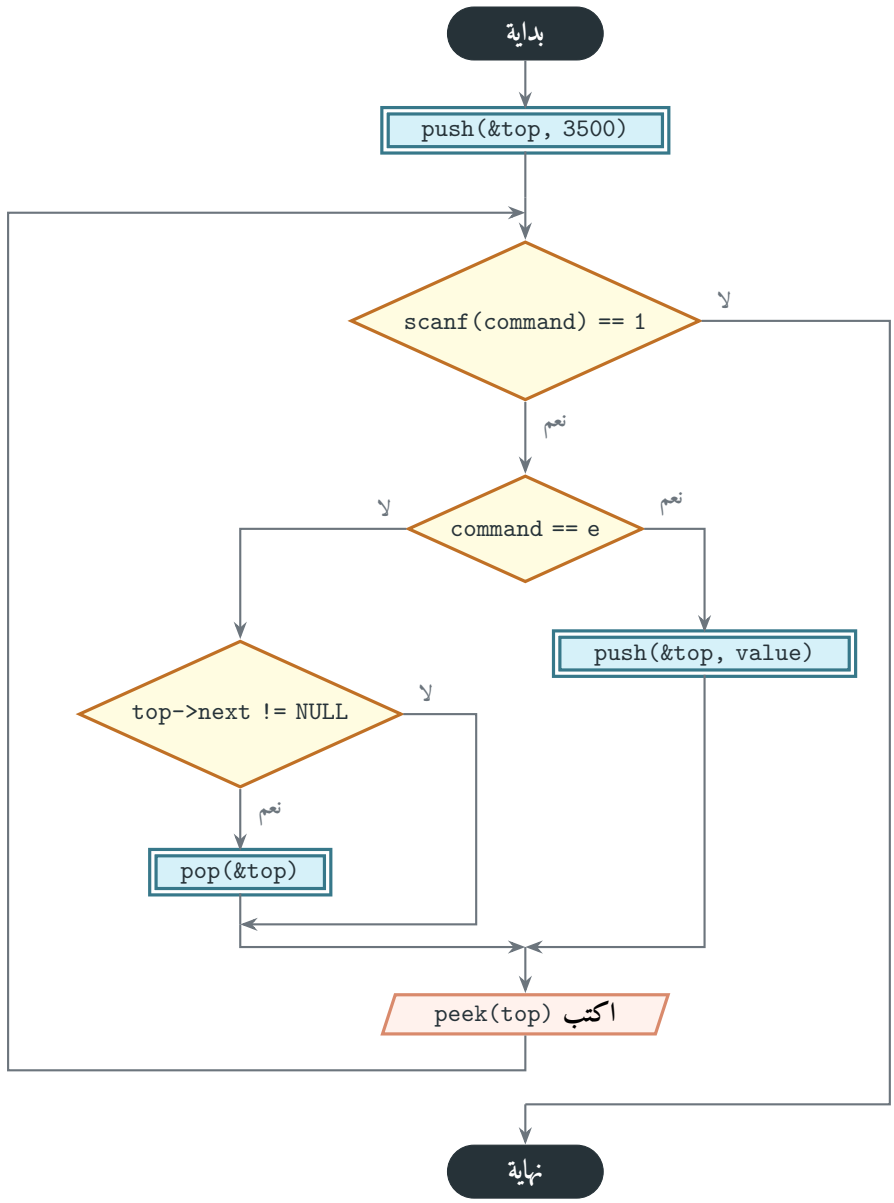
```
e3200 e2990 e2750 u e2800 u u u u
```

نتيجة التنفيذ

```
e3200  price = 3200
e2990  price = 2990
e2750  price = 2750
u      price = 2990
e2800  price = 2800
u      price = 2990
u      price = 3200
u      price = 3500
```

```
u       nothing to undo
```

المخطط الإنسيابي (Organigramme)



تمرين 12.23 طابور معالجة الطلبيات بقائمة

تجارة إلكترونية ★★ متوسط

اكتب طابورًا بقائمة متصلة مع مؤشري `front` و `rear` داخل هيكل `Queue`: `enqueue` في الذيل و `dequeue` من الرأس، كلاهما في وقت ثابت. حاكِ وصول ومعالجة الطلبيات واعرض حجم الطابور بعد كل حدث.

خطوات الحل

1) `enqueue`: إذا كان الطابور فارغًا فالعقدة هي `front` و `rear` معًا.
2) `dequeue`: إذا أصبح فارغًا نعيد `rear = NULL` أيضًا.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; struct Node *next; } Node;
typedef struct { Node *front, *rear; int size; } Queue;

void enqueue(Queue *queue, int number) {
    Node *node = malloc(sizeof(Node));
    node->number = number;
    node->next = NULL;
    if (queue->rear == NULL) queue->front = node;      // كان فارغًا
    else queue->rear->next = node;
    queue->rear = node;
    queue->size++;
}

int dequeue(Queue *queue) {
    Node *old = queue->front;
    int number;
    if (old == NULL) return -1;
    number = old->number;
    queue->front = old->next;
    if (queue->front == NULL) queue->rear = NULL;      // أصبح فارغًا
    free(old);
    queue->size--;
    return number;
}

int main(void) {
    Queue queue = {NULL, NULL, 0};
    char events[40];
    int idx, nextNumber = 7781;
    scanf("%39s", events);
    for (idx = 0; events[idx] != '\0'; idx++) {
        if (events[idx] == '+') {
            enqueue(&queue, nextNumber);
            printf("+ arrive #%d", nextNumber++);
        } else {
            printf("- serve  #%d", dequeue(&queue));
        }
        printf("   (waiting %d)\n", queue.size);
    }
    while (queue.size > 0) dequeue(&queue);
    return 0;
}
```

المدخلات

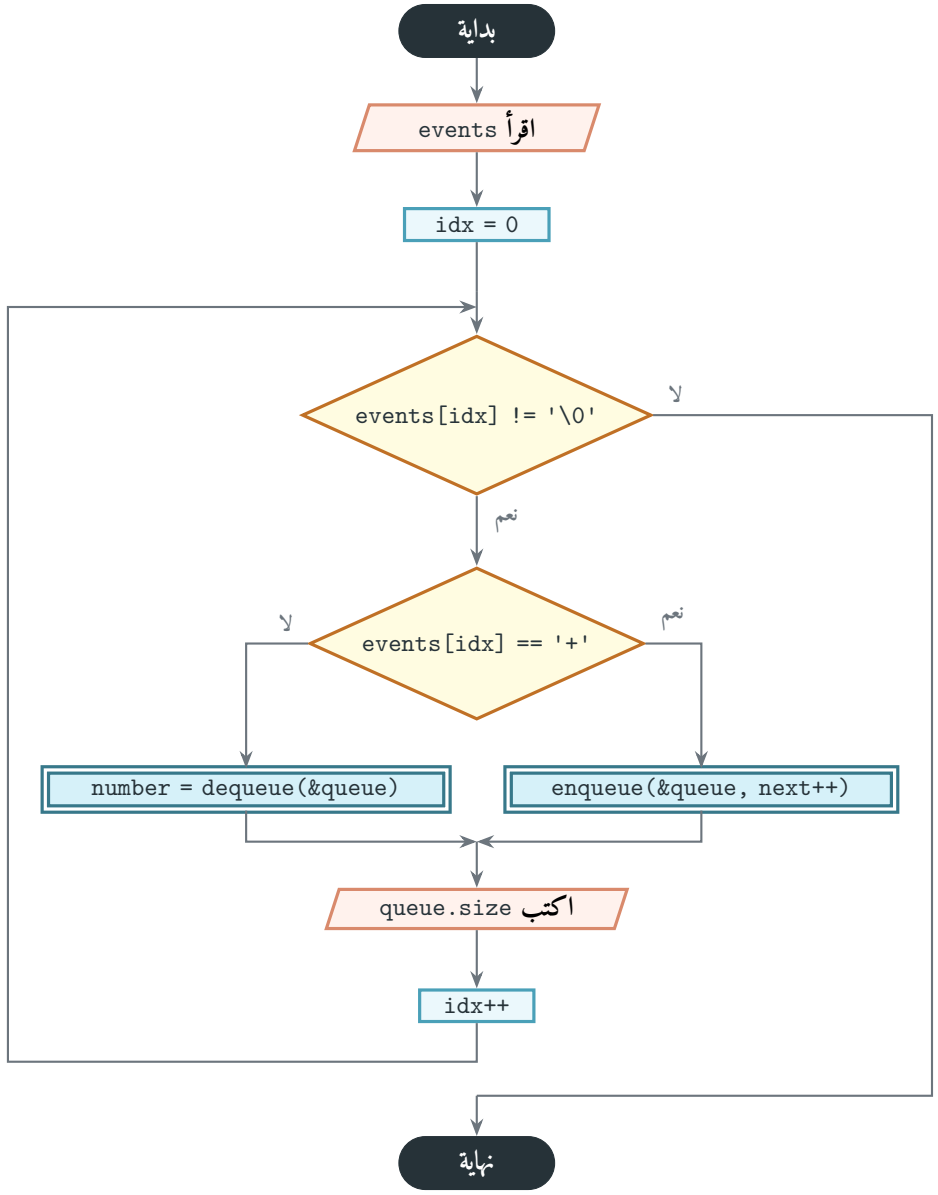
```
++-+++--+---
```

نتيجة التنفيذ

```
+ arrive #7781   (waiting 1)
+ arrive #7782   (waiting 2)
- serve  #7781   (waiting 1)
+ arrive #7783   (waiting 2)
+ arrive #7784   (waiting 3)
+ arrive #7785   (waiting 4)
```

```
- serve  #7782   (waiting 3)
- serve  #7783   (waiting 2)
+ arrive #7786   (waiting 3)
- serve  #7784   (waiting 2)
- serve  #7785   (waiting 1)
- serve  #7786   (waiting 0)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.24 جمع كثيرات حدود التكلفة بالقوائم

علم البيانات ★★ متوسط

تُمثَّل دالة التكلفة كثير حدود بقائمة حدود (المعامل، الأس) مرتبة تنازليًا حسب الأس، دون تخزين الحدود المعدومة. اكتب `addPoly(first, second)` تعيد قائمة جديدة لمجموع كثيري حدود (تكلفة الإنتاج + تكلفة التخزين).

خطوات الحل

1) مثل الدمج: نقارن أسّي الرأسين.
2) نفس الأس: نجمع المعاملين (ونتجاهل الحد إذا أصبح 0).
3) الأس الأكبر يُنسخ أولًا.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Term { double coef; int power; struct Term *next; } Term;

void append(Term **tail, double coef, int power) {
    Term *node = malloc(sizeof(Term));
    node->coef = coef; node->power = power; node->next = NULL;
    (*tail)->next = node;
    *tail = node;
}

Term *addPoly(const Term *first, const Term *second) {
    Term dummy, *tail = &dummy;
    dummy.next = NULL;
    while (first != NULL || second != NULL) {
        if (second == NULL || (first != NULL && first->power > second->power)) {
            append(&tail, first->coef, first->power); first = first->next;
        } else if (first == NULL || second->power > first->power) {
            append(&tail, second->coef, second->power); second = second->next;
        } else {                                                    // نفس الأس
            if (first->coef + second->coef != 0) append(&tail, first->coef + second->coef, first->power);
            first = first->next; second = second->next;
        }
    }
    return dummy.next;
}

Term *build(const double *coefs, const int *powers, int size) {
    Term dummy, *tail = &dummy;
    int idx;
    dummy.next = NULL;
    for (idx = 0; idx < size; idx++) append(&tail, coefs[idx], powers[idx]);
    return dummy.next;
}

void printPoly(const char *label, const Term *poly) {
    printf("%s", label);
    for (; poly != NULL; poly = poly->next) printf(" %+gq^%d", poly->coef, poly->power);
    printf("\n");
}

void freePoly(Term *poly) { Term *next; for (; poly != NULL; poly = next) { next = poly->next; free(poly); } }

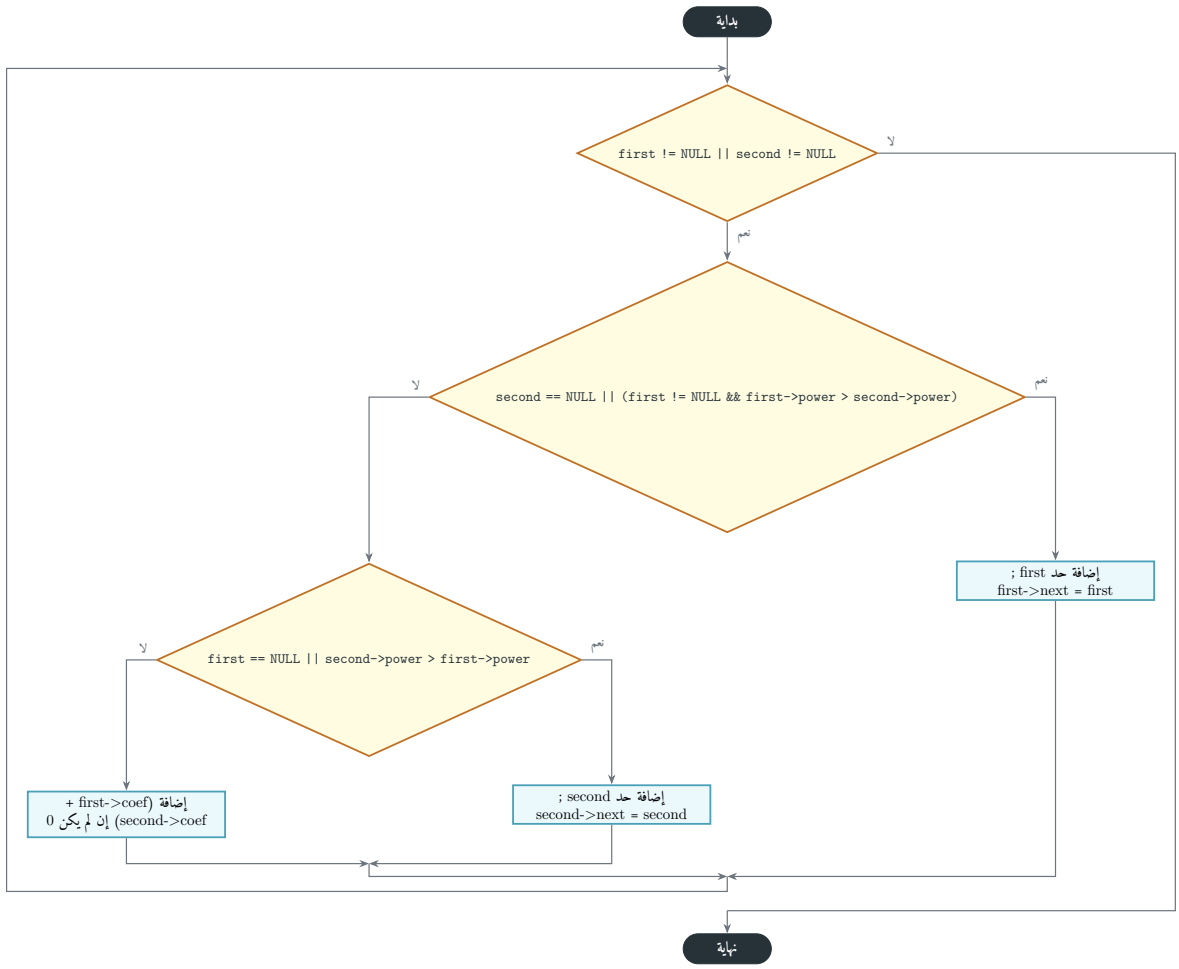
int main(void) {
    double coefA[4] = {0.002, -0.5, 60, 1500}, coefB[3] = {0.5, -20, 300};
    int powA[4] = {3, 2, 1, 0}, powB[3] = {2, 1, 0};
    Term *production = build(coefA, powA, 4), *storage = build(coefB, powB, 3);
    Term *total = addPoly(production, storage);
    printPoly("production:", production);
    printPoly("storage:   ", storage);
    printPoly("total:     ", total);
    freePoly(production); freePoly(storage); freePoly(total);
    return 0;
}
```

نتيجة التنفيذ

```
production: +0.002q^3 -0.5q^2 +60q^1 +1500q^0
storage:    +0.5q^2 -20q^1 +300q^0
```

```
total:      +0.002q^3 +40q^1 +1800q^0
```

المخطط الإنسيابي (Organigramme)



تمرين 12.25 جمع أعداد ضخمة بقوائم الأرقام

علم البيانات ★★ متوسط

رقم أعمال سلسلة متاجر يتجاوز سعة `long`. نمثل كل عدد بقائمة أرقام **معكوسة** (الآحاد في الرأس). اكتب `fromText` و `addNumbers(first, second)` (جمع بالحمل يبني قائمة جديدة) و `printNumber` (عودية لعرض الأرقام بالترتيب الصحيح).

خطوات الحل

1) القراءة: كل حرف يُدرج في الرأس، فيصبح الآحاد في الرأس تلقائيًا.
2) الجمع: رقم + رقم + حمل، نكتب `10 %` ونحمل `10 /`.
3) العرض عوديًا: الباقي أولًا ثم الرقم الحالي.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Digit { int value; struct Digit *next; } Digit;
```

```c

Digit *fromText(const char *text) {
    Digit *head = NULL, *node;
    for (; *text != '\0'; text++) {
        node = malloc(sizeof(Digit));
        node->value = *text - '0';
        node->next = head;                    // الآحاد تنتهي في الرأس
        head = node;
    }
    return head;
}

Digit *addNumbers(const Digit *first, const Digit *second) {
    Digit dummy, *tail = &dummy, *node;
    int carry = 0, digit;
    while (first != NULL || second != NULL || carry) {
        digit = carry + (first ? first->value : 0) + (second ? second->value : 0);
        node = malloc(sizeof(Digit));
        node->value = digit % 10;
        node->next = NULL;
        tail->next = node;
        tail = node;
        carry = digit / 10;                   // الحمل
        if (first) first = first->next;
        if (second) second = second->next;
    }
    return dummy.next;
}

void printNumber(const Digit *head) {
    if (head == NULL) return;
    printNumber(head->next);                  // الأرقام الأعلى أولًا
    printf("%d", head->value);
}

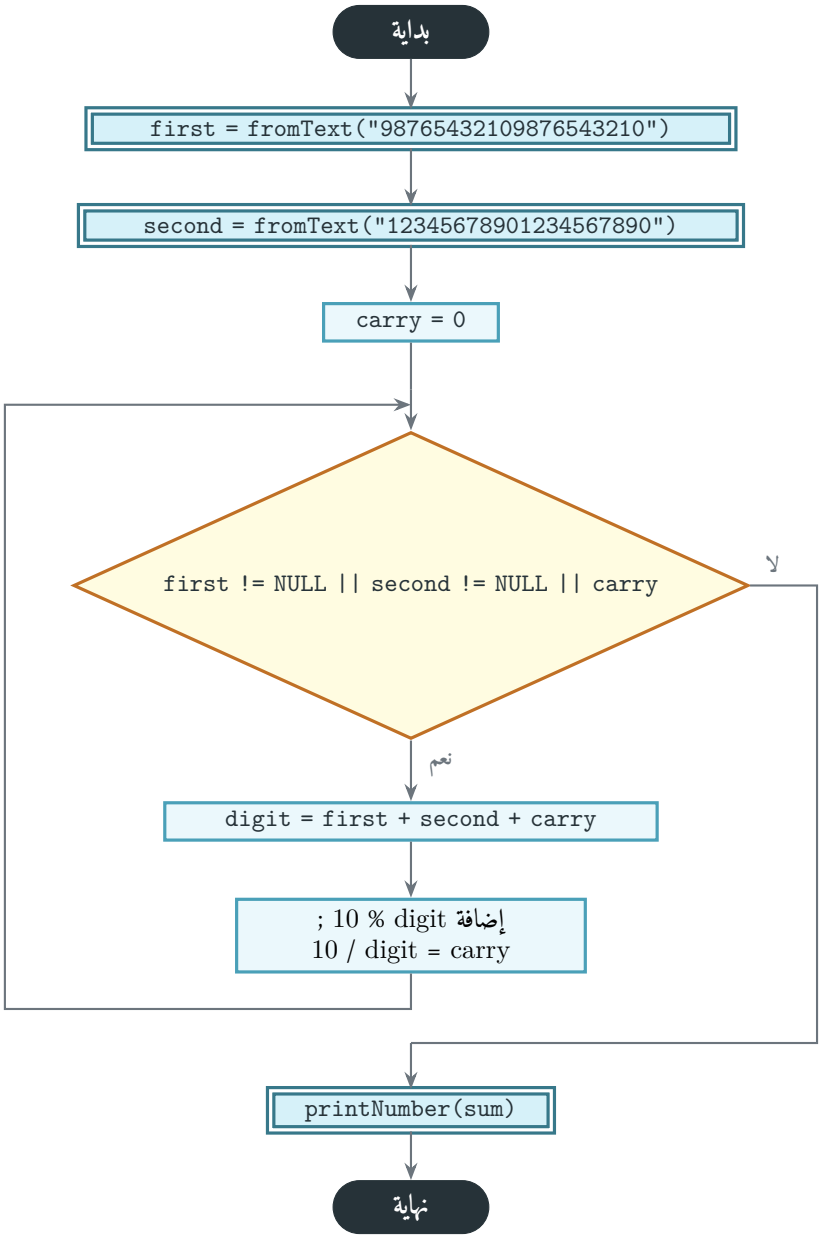
void freeDigits(Digit *head) { Digit *next; for (; head != NULL; head = next) { next = head->next;
    ↪ free(head); } }

int main(void) {
    Digit *first = fromText("98765432109876543210"), *second = fromText("12345678901234567890");
    Digit *sum = addNumbers(first, second);
    printf("  "); printNumber(first); printf("\n+ "); printNumber(second);
    printf("\n= "); printNumber(sum); printf("\n");
    freeDigits(first); freeDigits(second); freeDigits(sum);
    return 0;
}
```

نتيجة التنفيذ

```
  98765432109876543210
+ 12345678901234567890
= 111111111011111111100
```

المخطط الإنسيابي (Organigramme)



تمرين 12.26 الترتيب بالإدراج على قائمة متصلة

تجارة إلكترونية

متوسط

رتّب قائمة منتجات حسب التقييم تنازليًا بـ **الترتيب بالإدراج**: نأخذ العقد واحدة واحدة من القائمة الأصلية وندرجها في مكانها في قائمة جديدة مرتبة، بإعادة ربط العقد فقط.

خطوات الحل

1) `sorted` تبدأ فارغة، لكل عقدة `node` نفصلها عن الأصل.
2) ندرجها في `sorted` قبل أول عقدة تقييمها أقل منها.
3) لا `malloc` ولا `free`: نفس العقد.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char name[10]; double rating; struct Node *next; } Node;

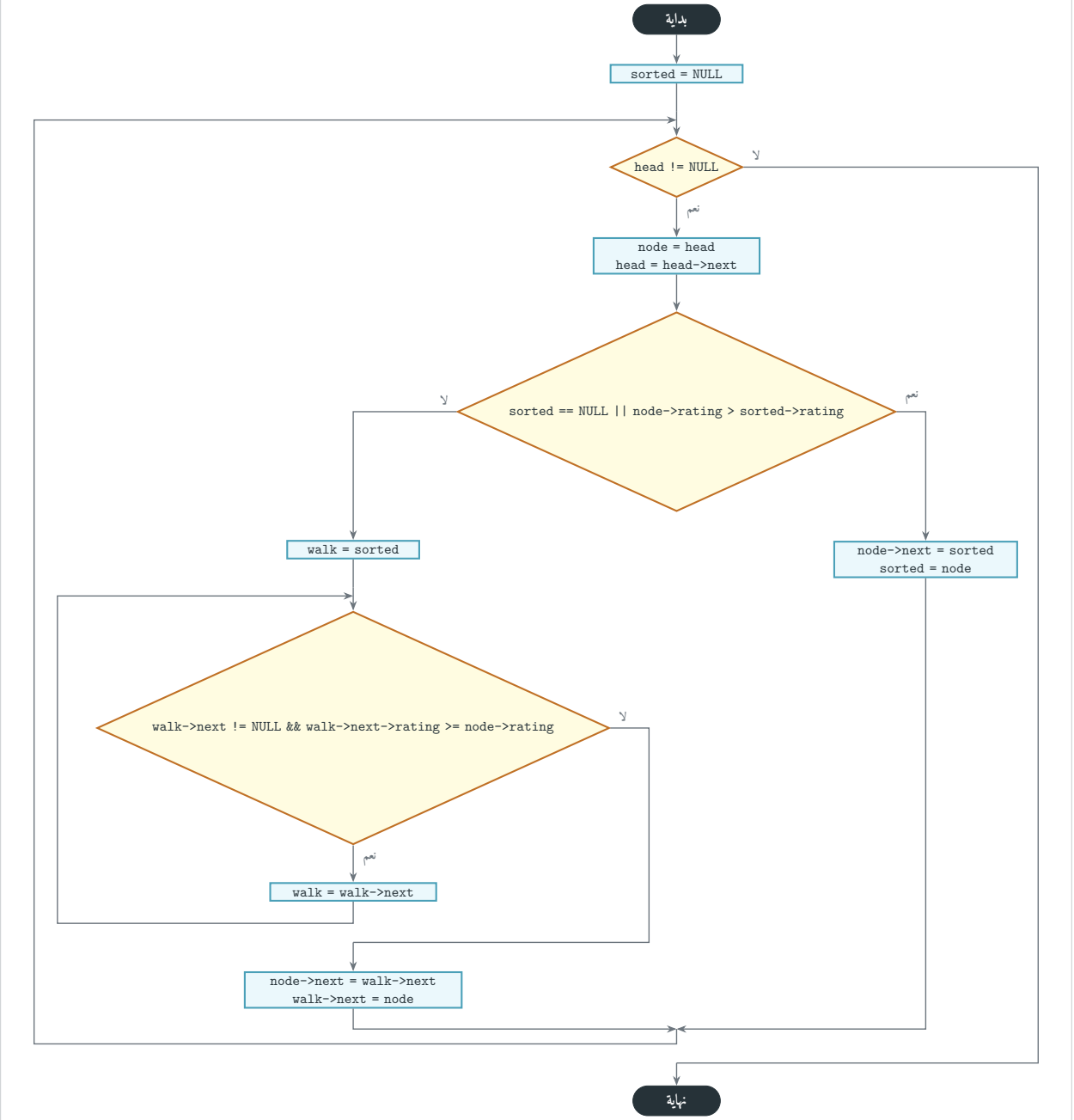
Node *insertionSort(Node *head) {
    Node *sorted = NULL, *node, *walk;
    while (head != NULL) {
        node = head;                          // نفصل العقدة الأولى
        head = head->next;
        if (sorted == NULL || node->rating > sorted->rating) {
            node->next = sorted;
            sorted = node;
        } else {
            walk = sorted;
            while (walk->next != NULL && walk->next->rating >= node->rating) walk = walk->next;
            node->next = walk->next;
            walk->next = node;
        }
    }
    return sorted;
}

int main(void) {
    const char *names[6] = {"Mouse", "Hub", "Laptop", "Cable", "Monitor", "Webcam"};
    double ratings[6] = {4.7, 4.3, 4.2, 3.6, 4.4, 3.9};
    Node *head = NULL, *node, *walk, *next;
    int idx, pos;
    for (idx = 5; idx >= 0; idx--) {
        node = malloc(sizeof(Node));
        for (pos = 0; (node->name[pos] = names[idx][pos]) != '\0'; pos++) {}
        node->rating = ratings[idx];
        node->next = head;
        head = node;
    }
    head = insertionSort(head);
    for (walk = head; walk != NULL; walk = walk->next) printf("%s(%.1f) ", walk->name, walk->rating);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
Mouse(4.7) Monitor(4.4) Hub(4.3) Laptop(4.2) Webcam(3.9) Cable(3.6)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.27 حذف المنتجات منخفضة التقييم

تجارة إلكترونية

متوسط

اكتب `removeBelow(head, minRating, &removed)` تحذف من القائمة **كل** المنتجات ذات التقييم الأقل من الحد (قد تكون متتالية أو في الرأس)، وتحررها، وتعيد الرأس الجديد. استعمل مؤشرًا على مؤشر `Node **link` لتوحيد حالة الرأس مع باقي العقد.

خطوات الحل

1) `link` يشير إلى الحقل الذي يحمل عنوان العقدة الحالية (`&head` أو `&prev->next`).
2) للحذف: `*link = node->next` ثم `free(node)`؛ وإلا `link = &node->next`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; double rating; struct Node *next; } Node;

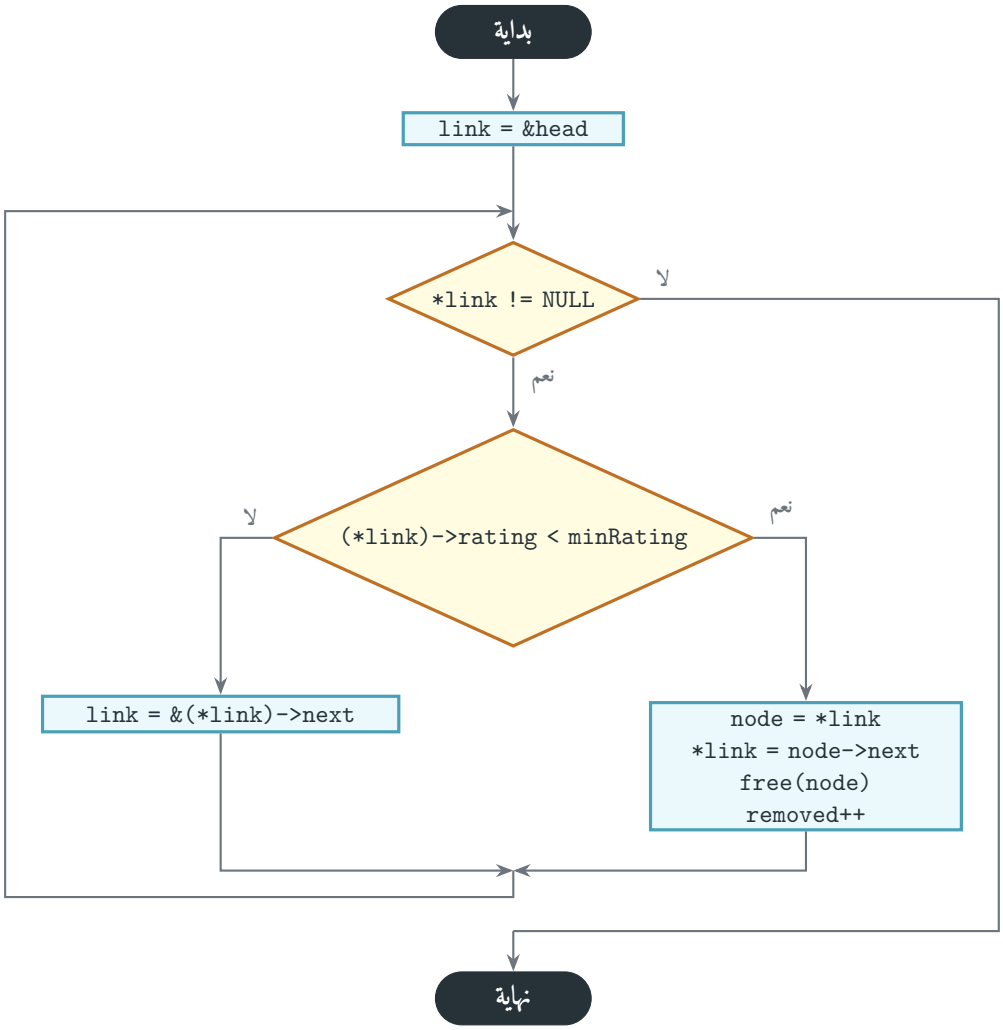
Node *removeBelow(Node *head, double minRating, int *removed) {
    Node **link = &head, *node;
    *removed = 0;
    while (*link != NULL) {
        if ((*link)->rating < minRating) {
            node = *link;
            *link = node->next;                // نفس الكود للرأس والوسط
            free(node);
            (*removed)++;
        } else {
            link = &(*link)->next;
        }
    }
    return head;
}

int main(void) {
    double ratings[8] = {2.9, 3.1, 4.5, 3.8, 2.5, 4.9, 3.0, 4.1};
    Node *head = NULL, *node, *walk, *next;
    int idx, removed;
    for (idx = 7; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->code = 101 + idx; node->rating = ratings[idx];
        node->next = head; head = node;
    }
    head = removeBelow(head, 3.5, &removed);
    printf("Removed %d products; kept:", removed);
    for (walk = head; walk != NULL; walk = walk->next) printf(" %d(%.1f)", walk->code, walk->rating);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
Removed 4 products; kept: 103(4.5) 104(3.8) 106(4.9) 108(4.1)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.28 الزبائن المشتركون بين حملتين

علم البيانات ★★ متوسط

قائمتان مرتبتان بمعرفات الزبائن الذين فتحوا رسالة الحملة الأولى والثانية. اكتب `intersection(first, second)` تبني **قائمة جديدة** بالزبائن المشتركين بمرور واحد متزامن، واحسب **معامل جاكار** `|A ∩ B| / |A ∪ B|`.

خطوات الحل

1) مؤشران يتقدمان: الأصغر يتقدم؛ عند التساوي نضيف ونتقدم بالاثنين.
2) `|A ∪ B| = |A| + |B| − |A ∩ B|`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int id; struct Node *next; } Node;

Node *fromArray(const int *values, int size) {
    Node *head = NULL, *node;
    while (size-- > 0) { node = malloc(sizeof(Node)); node->id = values[size]; node->next = head; head = node; }
    return head;
```

```c
}

int length(const Node *head) { int count = 0; for (; head != NULL; head = head->next) count++; return count; }
void freeList(Node *head) { Node *next; for (; head != NULL; head = next) { next = head->next; free(head); } }

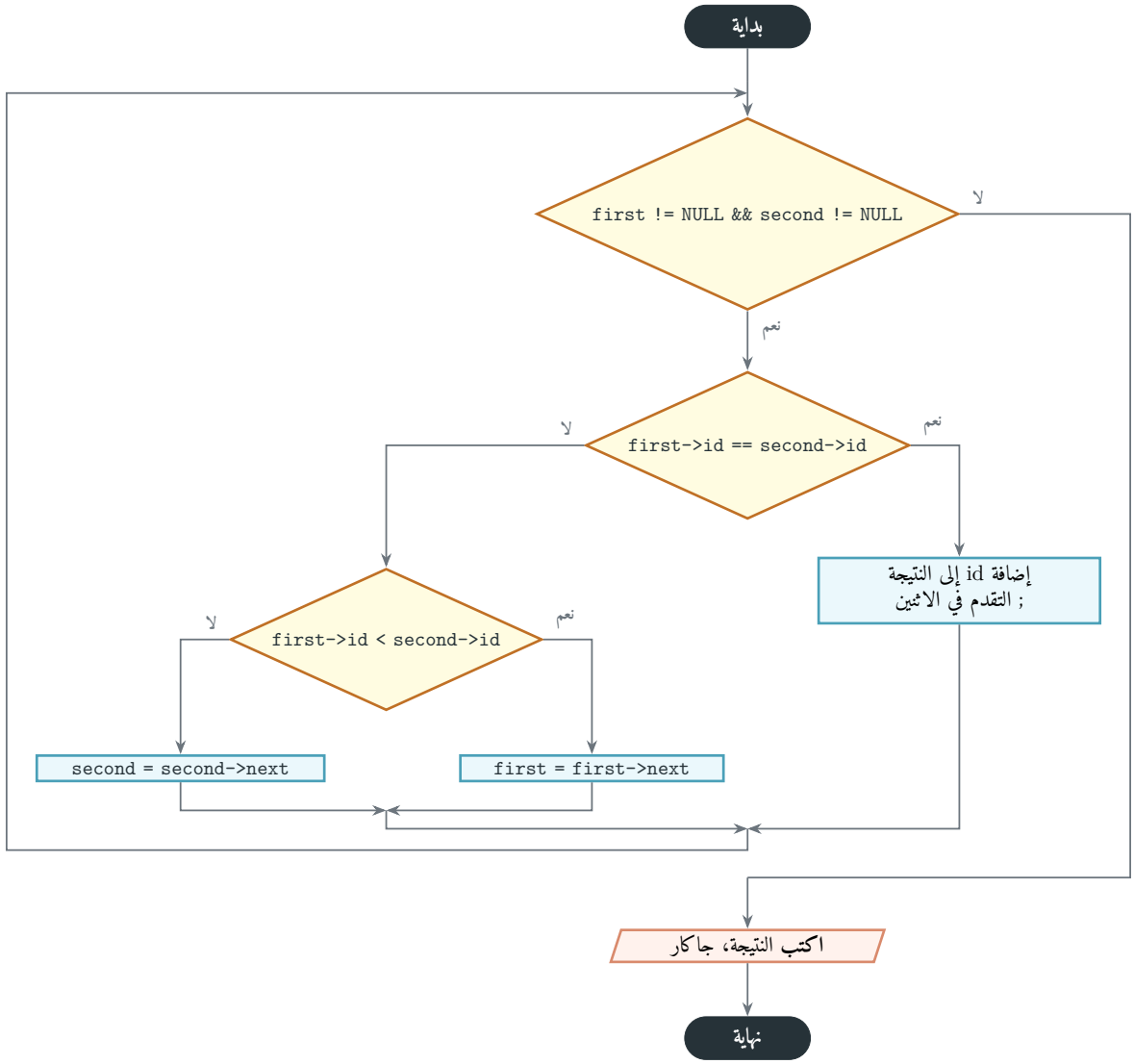
Node *intersection(const Node *first, const Node *second) {
    Node dummy, *tail = &dummy, *node;
    dummy.next = NULL;
    while (first != NULL && second != NULL) {
        if (first->id == second->id) {
            node = malloc(sizeof(Node));
            node->id = first->id;
            node->next = NULL;
            tail->next = node;
            tail = node;
            first = first->next;
            second = second->next;
        } else if (first->id < second->id) {
            first = first->next;                // الأصغر يتقدم
        } else {
            second = second->next;
        }
    }
    return dummy.next;
}

int main(void) {
    int campaignA[8] = {501, 503, 507, 510, 512, 518, 520, 525};
    int campaignB[7] = {502, 503, 510, 511, 518, 525, 530};
    Node *first = fromArray(campaignA, 8), *second = fromArray(campaignB, 7);
    Node *both = intersection(first, second), *walk;
    int common = length(both);
    printf("Opened both:");
    for (walk = both; walk != NULL; walk = walk->next) printf(" %d", walk->id);
    printf("\nJaccard = %d / %d = %.3f\n", common, 8 + 7 - common, (double)common / (8 + 7 - common));
    freeList(first); freeList(second); freeList(both);
    return 0;
}
```

نتيجة التنفيذ

```
Opened both: 503 510 518 525
Jaccard = 4 / 11 = 0.364
```

المخطط الإنسيابي (Organigramme)



تمرين 12.29 تقسيم القائمة حول عتبة المخزون

تجارة إلكترونية

متوسط

لتقرير التموين، أعد ترتيب قائمة المنتجات بحيث تأتي **كل** المنتجات ذات المخزون الأقل من 10 أولًا، ثم الباقي، **مع الحفاظ على** الترتيب النسبي داخل كل مجموعة. اكتب `partition(head, threshold)` بقائمتين حارستين.

خطوات الحل

1) قائمة `low` وقائمة `high` بعقدتين حارستين ومؤشري ذيل.
2) كل عقدة تُربط في ذيل قائمتها.
3) نربط ذيل `low` برأس `high`، وننهي ذيل `high` بـ `NULL`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code, stock; struct Node *next; } Node;
```

```c

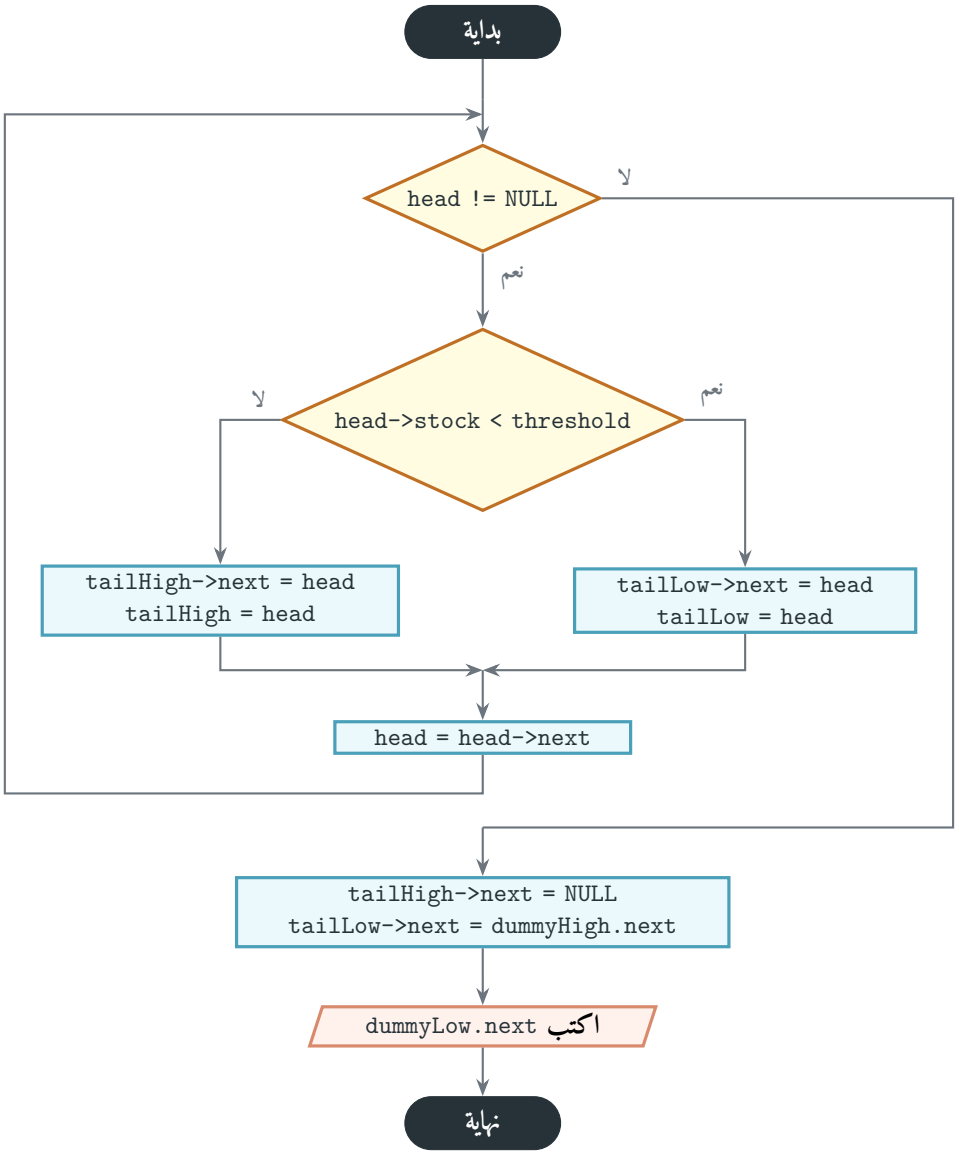
Node *partition(Node *head, int threshold) {
    Node dummyLow, dummyHigh, *tailLow = &dummyLow, *tailHigh = &dummyHigh;
    for (; head != NULL; head = head->next) {
        if (head->stock < threshold) { tailLow->next = head; tailLow = head; }
        else { tailHigh->next = head; tailHigh = head; }
    }
    tailHigh->next = NULL;
    tailLow->next = dummyHigh.next;          // الربط بين المجموعتين
    return dummyLow.next;
}

int main(void) {
    int stocks[8] = {25, 3, 40, 8, 12, 0, 9, 31};
    Node *head = NULL, *node, *walk, *next;
    int idx;
    for (idx = 7; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->code = 101 + idx; node->stock = stocks[idx];
        node->next = head; head = node;
    }
    head = partition(head, 10);
    for (walk = head; walk != NULL; walk = walk->next) printf("%d:%d ", walk->code, walk->stock);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
102:3 104:8 106:0 107:9 101:25 103:40 105:12 108:31
```

المخطط الإنسيابي (Organigramme)



تمرين 12.30 المنتجات المشاهدة مؤخرًا

تجارة إلكترونية ★★ متوسط

قسم "شوهد مؤخرًا" يعرض آخر 4 منتجات مختلفة. عند مشاهدة منتج: إذا كان في القائمة **نقله إلى الرأس**، وإلا ندرجه في الرأس ونحذف الأخير إذا تجاوز العدد 4. نفّذ سلسلة مشاهدات.

خطوات الحل

1) نبحث عن المنتج مع تذكر السابق؛ إن وُجد نفصله ونضعه في الرأس.
2) عند تجاوز السعة: نمر إلى العقدة قبل الأخيرة ونحرر الأخيرة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define CAP 4
```

```c

typedef struct Node { int code; struct Node *next; } Node;

int main(void) {
    Node *head = NULL, *prev, *walk, *node, *next;
    int code, count = 0;
    while (scanf("%d", &code) == 1) {
        for (prev = NULL, walk = head; walk != NULL && walk->code != code; prev = walk, walk = walk->next) {}
        if (walk != NULL) {
            if (prev != NULL) {                          // نقل إلى الرأس
                prev->next = walk->next;
                walk->next = head;
                head = walk;
            }
        } else {
            node = malloc(sizeof(Node));
            node->code = code;
            node->next = head;
            head = node;
            if (++count > CAP) {                         // حذف الأقدم (الأخير)
                for (walk = head; walk->next->next != NULL; walk = walk->next) {}
                free(walk->next);
                walk->next = NULL;
                count--;
            }
        }
        printf("view %d ->", code);
        for (walk = head; walk != NULL; walk = walk->next) printf(" %d", walk->code);
        printf("\n");
    }
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

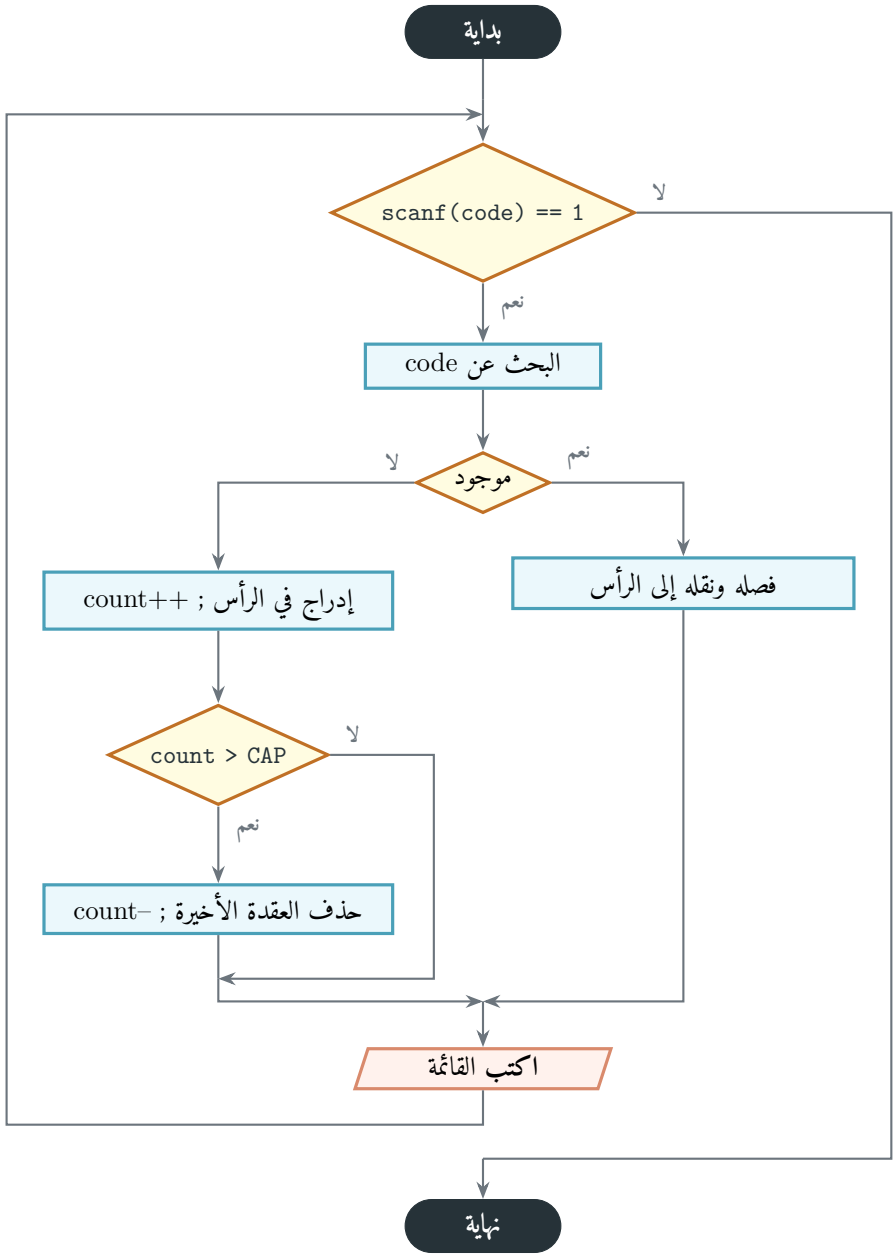
المدخلات

```
101 205 330 205 412 500 101 618
```

نتيجة التنفيذ

```
view 101 -> 101
view 205 -> 205 101
view 330 -> 330 205 101
view 205 -> 205 330 101
view 412 -> 412 205 330 101
view 500 -> 500 412 205 330
view 101 -> 101 500 412 205
view 618 -> 618 101 500 412
```

المخطط الإنسيابي (Organigramme)



تمرين 12.31 قائمة أقسام تحتوي على قوائم منتجات

تجارة إلكترونية ★★ متوسط

بنية من مستويين: قائمة **أقسام**، وكل قسم يحتوي على رأس قائمة **منتجاته**. اكتب `getCategory` (تبحث عن القسم أو تنشئه في النهاية) و

```
addProduct(categories, category, name, price)
```

، ثم اعرض كل قسم بمنتجاته ومجموعه، وحرر البنية كلها.

خطوات الحل

1) عقدة القسم: الاسم، رأس قائمة المنتجات، المؤشر على القسم التالي.
2) الإضافة: نجد القسم ثم ندرج المنتج في رأس قائمته.

3) التحرير: لكل قسم نحرر منتجاته ثم القسم نفسه.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Item { char name[12]; double price; struct Item *next; } Item;
typedef struct Category { char name[12]; Item *items; struct Category *next; } Category;

void copyText(char *target, const char *source) { while ((*target++ = *source++) != '\0') {} }

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

Category *getCategory(Category **list, const char *name) {
    Category **link = list;
    while (*link != NULL && !sameText((*link)->name, name)) link = &(*link)->next;
    if (*link == NULL) {                                    // قسم جديد في النهاية
        *link = malloc(sizeof(Category));
        copyText((*link)->name, name);
        (*link)->items = NULL;
        (*link)->next = NULL;
    }
    return *link;
}

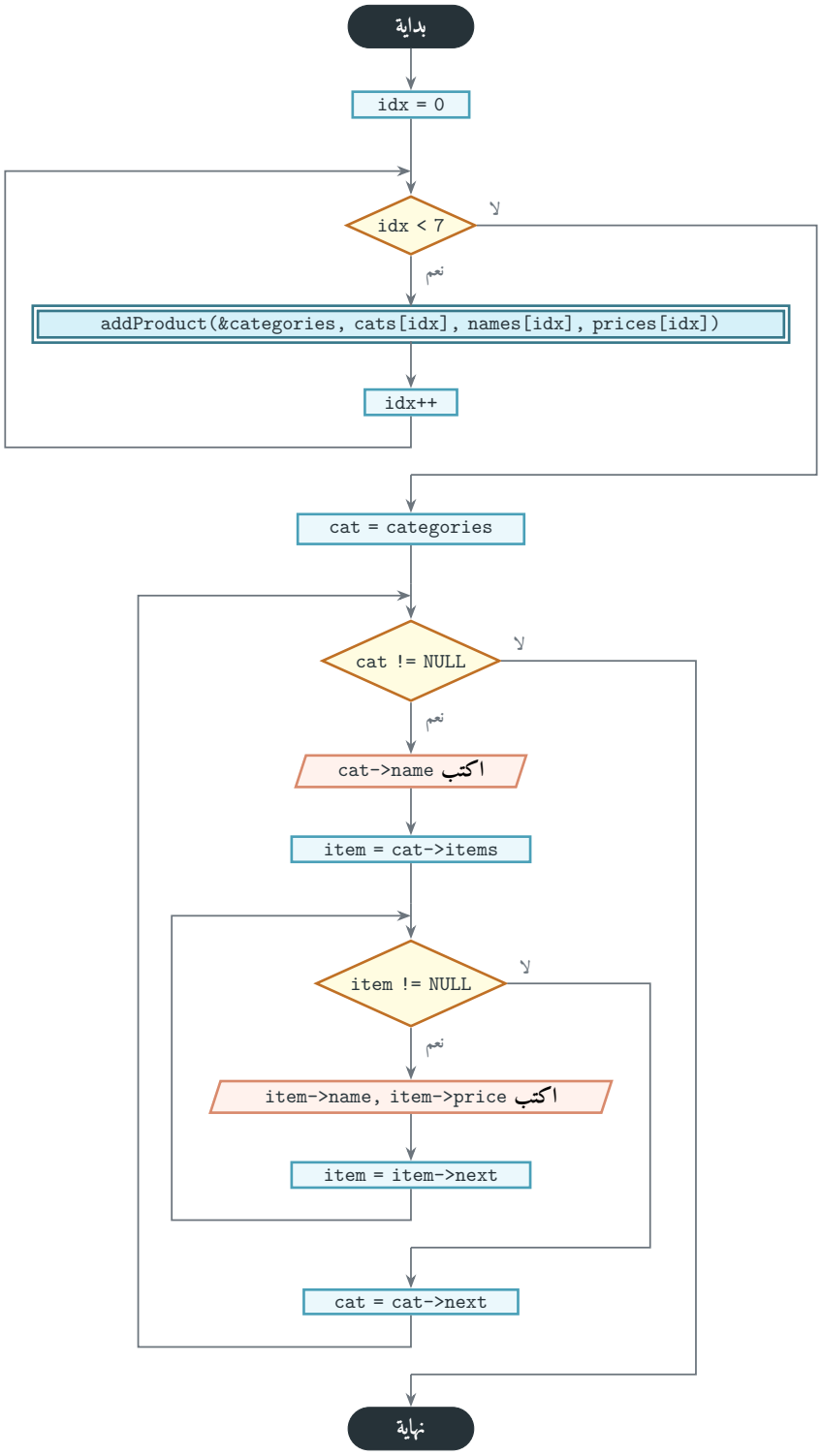
void addProduct(Category **list, const char *category, const char *name, double price) {
    Category *cat = getCategory(list, category);
    Item *item = malloc(sizeof(Item));
    copyText(item->name, name);
    item->price = price;
    item->next = cat->items;                                // في رأس قائمة القسم
    cat->items = item;
}

int main(void) {
    const char *cats[7] = {"Phones", "Audio", "Phones", "Laptops", "Audio", "Phones", "Laptops"};
    const char *names[7] = {"Galaxy A", "Earbuds", "Redmi 13", "ThinkPad", "Speaker", "iPhone", "IdeaPad"};
    double prices[7] = {38000, 5400, 27500, 95000, 6800, 165000, 72000}, total;
    Category *categories = NULL, *cat, *nextCat;
    Item *item, *nextItem;
    int idx;
    for (idx = 0; idx < 7; idx++) addProduct(&categories, cats[idx], names[idx], prices[idx]);
    for (cat = categories; cat != NULL; cat = cat->next) {
        printf("%s:", cat->name);
        for (total = 0, item = cat->items; item != NULL; item = item->next) {
            printf(" %s", item->name);
            total += item->price;
        }
        printf("  (total %.0f)\n", total);
    }
    for (cat = categories; cat != NULL; cat = nextCat) {
        for (item = cat->items; item != NULL; item = nextItem) { nextItem = item->next; free(item); }
        nextCat = cat->next;
        free(cat);
    }
    return 0;
}
```

نتيجة التنفيذ

```
Phones: iPhone Redmi 13 Galaxy A  (total 230500)
Audio: Speaker Earbuds  (total 12200)
Laptops: IdeaPad ThinkPad  (total 167000)
```

المخطط الإنسيابي (Organigramme)



تمرين 12.32 الجداء السلمي لمتجهات متناثرة

علم البيانات ★★ متوسط

اهتمامات كل زبون متجه من 10000 خانة (منتج)، معظمها 0. نخزن فقط الخانات غير المعدومة في قائمة مرتبة حسب الفهرس (الفهرس، القيمة). اكتب `dot(first, second)` تحسب الجداء السلمي بمرور متزامن، وتشابه **جيب التمام** بين زبونين.

خطوات الحل

1) مثل التقاطع: عند تساوي الفهرسين نضرب القيمتين ونجمع.
2) `norm(vec) = sqrt(dot(vec, vec))` (نيوتن).
3) `cos = dot(a, b) / (norm(a) * norm(b))`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Entry { int index; double value; struct Entry *next; } Entry;

Entry *build(const int *indexes, const double *values, int size) {
    Entry *head = NULL, *node;
    while (size-- > 0) {
        node = malloc(sizeof(Entry));
        node->index = indexes[size]; node->value = values[size];
        node->next = head; head = node;
    }
    return head;
}

double dot(const Entry *first, const Entry *second) {
    double sum = 0;
    while (first != NULL && second != NULL) {
        if (first->index == second->index) {
            sum += first->value * second->value;
            first = first->next; second = second->next;
        } else if (first->index < second->index) {
            first = first->next;
        } else {
            second = second->next;
        }
    }
    return sum;
}

double mySqrt(double value) {
    double guess = value > 1 ? value : 1;
    int iter;
    for (iter = 0; iter < 50; iter++) guess = (guess + value / guess) / 2;
    return guess;
}

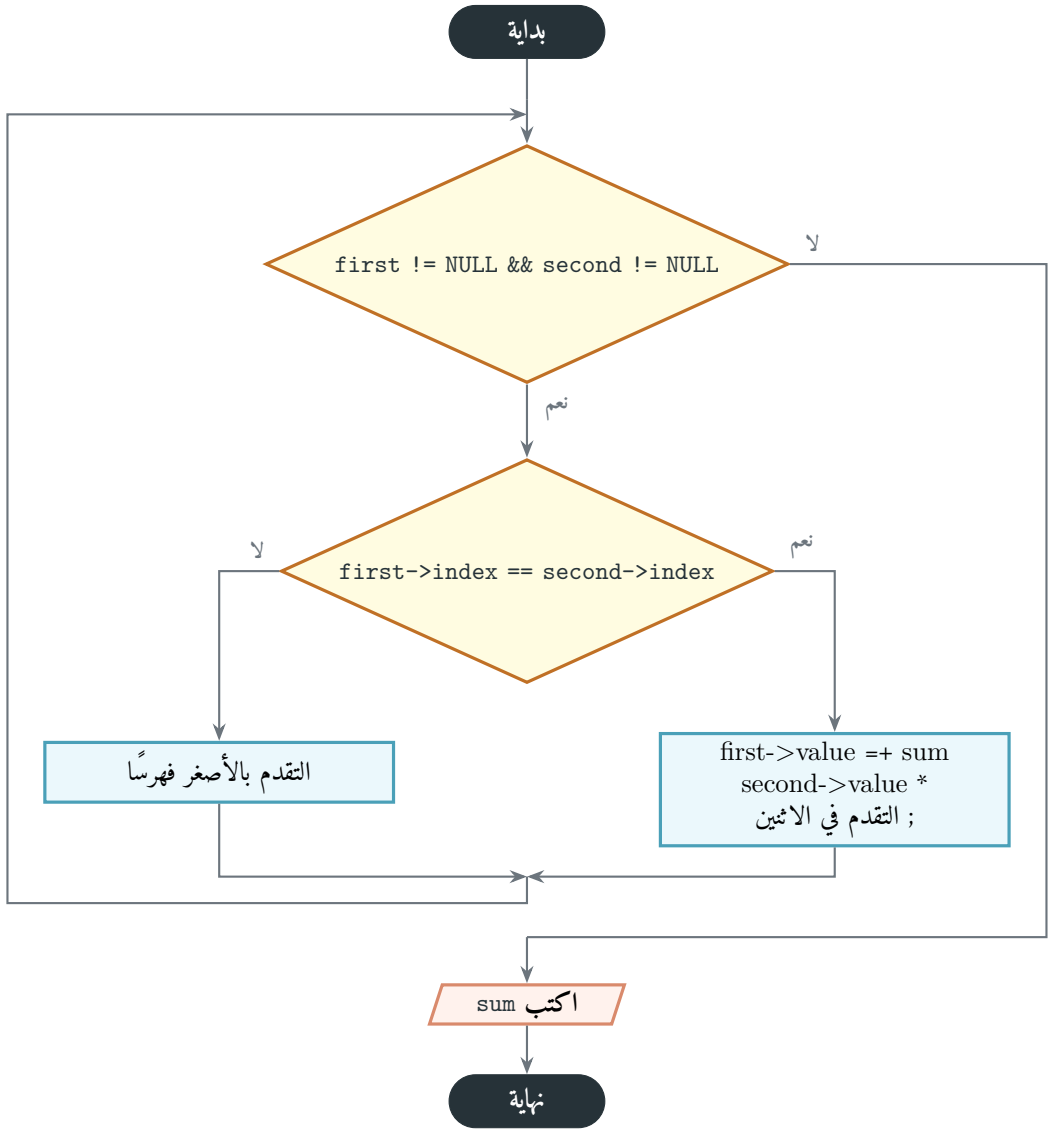
void freeVec(Entry *head) { Entry *next; for (; head != NULL; head = next) { next = head->next; free(head); } }

int main(void) {
    int idxA[5] = {12, 340, 2045, 5120, 9001}, idxB[4] = {12, 2045, 7781, 9001}, idxC[3] = {77, 4400, 8800};
    double valA[5] = {3, 1, 5, 2, 4}, valB[4] = {2, 4, 1, 5}, valC[3] = {5, 5, 1};
    Entry *first = build(idxA, valA, 5), *second = build(idxB, valB, 4), *third = build(idxC, valC, 3);
    printf("dot(A,B) = %.0f, cos(A,B) = %.3f\n", dot(first, second),
           dot(first, second) / (mySqrt(dot(first, first)) * mySqrt(dot(second, second))));
    printf("dot(A,C) = %.0f, cos(A,C) = %.3f\n", dot(first, third),
           dot(first, third) / (mySqrt(dot(first, first)) * mySqrt(dot(third, third))));
    printf("Stored %d values instead of 3 x 10000\n", 5 + 4 + 3);
    freeVec(first); freeVec(second); freeVec(third);
    return 0;
}
```

نتيجة التنفيذ

```
dot(A,B) = 46, cos(A,B) = 0.915
dot(A,C) = 0, cos(A,C) = 0.000
Stored 12 values instead of 3 x 10000
```

المخطط الإنسيابي (Organigramme)



تمرين 12.33 تدوير قائمة العروض

تجارة إلكترونية ★★ متوسط

لافتات الصفحة الرئيسية تدور: كل يوم تتقدم القائمة بـ `k` موضعًا. اكتب `rotate(head, k)` تجعل العقدة رقم `k` (من 0) هي الرأس الجديد، بربط الذيل بالرأس القديم ثم قطع الحلقة في المكان المناسب. عالج `k` أكبر من الطول بـ `k % length`.

خطوات الحل

1) نحسب الطول ونجد الذيل في مرور واحد.
2) `k = k % length`؛ إذا كان 0 لا تغيير.
3) نربط `tail->next = head` (حلقة)، نتقدم `k` خطوات، نجعل العقدة الموالية رأسًا ونقطع.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

```

```c
typedef struct Node { char name[10]; struct Node *next; } Node;

Node *rotate(Node *head, int shift) {
    Node *tail = head, *walk;
    int length = 1, step;
    if (head == NULL) return head;
    while (tail->next != NULL) { tail = tail->next; length++; }
    shift %= length;
    if (shift == 0) return head;
    tail->next = head;                                  // حلقة مؤقتة
    for (walk = head, step = 1; step < shift; step++) walk = walk->next;
    head = walk->next;                                  // الرأس الجديد
    walk->next = NULL;                                  // نقطع الحلقة
    return head;
}

int main(void) {
    const char *names[5] = {"Flash", "Phones", "Audio", "Gaming", "Bundle"};
    Node *head = NULL, *node, *walk, *next;
    int idx, pos, shift, query;
    for (idx = 4; idx >= 0; idx--) {
        node = malloc(sizeof(Node));
        for (pos = 0; (node->name[pos] = names[idx][pos]) != '\0'; pos++) {}
        node->next = head; head = node;
    }
    for (query = 0; query < 2; query++) {
        scanf("%d", &shift);
        head = rotate(head, shift);
        printf("rotate %d:", shift);
        for (walk = head; walk != NULL; walk = walk->next) printf(" %s", walk->name);
        printf("\n");
    }
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

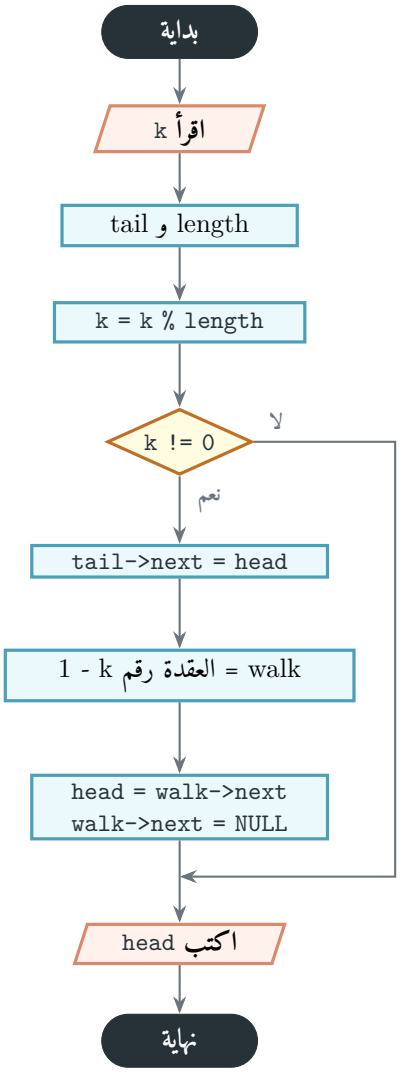
المدخلات

```
2 7
```

نتيجة التنفيذ

```
rotate 2: Audio Gaming Bundle Flash Phones
rotate 7: Bundle Flash Phones Audio Gaming
```

المخطط الإنسيابي (Organigramme)



تمرين 12.34 هل رمز المنتج متناظر في قائمة

علم البيانات ★★ متوسط

رمز المنتج مخزن كقائمة حروف. للتحقق من أنه متناظر دون ذاكرة إضافية: نجد المنتصف (مؤشران سريع وبطيء)، نقلب النصف الثاني، نقارن النصفين، ثم نعيد النصف الثاني كما كان.

خطوات الحل

1) `slow` و `fast` يجدان المنتصف.
2) `reverse(slow)` يقلب النصف الثاني.
3) المقارنة من الرأس ومن رأس النصف المقلوب.
4) القائمة يجب أن تعود كما كانت بعد الفحص.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char ch; struct Node *next; } Node;
```

```c

Node *reverse(Node *head) {
    Node *prev = NULL, *next;
    while (head != NULL) { next = head->next; head->next = prev; prev = head; head = next; }
    return prev;
}

int isPalindrome(Node *head) {
    Node *slow = head, *fast = head, *second, *left, *right;
    int same = 1;
    while (fast != NULL && fast->next != NULL) { slow = slow->next; fast = fast->next->next; }
    second = reverse(slow);                                  // نقلب النصف الثاني
    for (left = head, right = second; right != NULL; left = left->next, right = right->next)
        if (left->ch != right->ch) { same = 0; break; }
    reverse(second);                                         // نعيده كما كان
    return same;
}

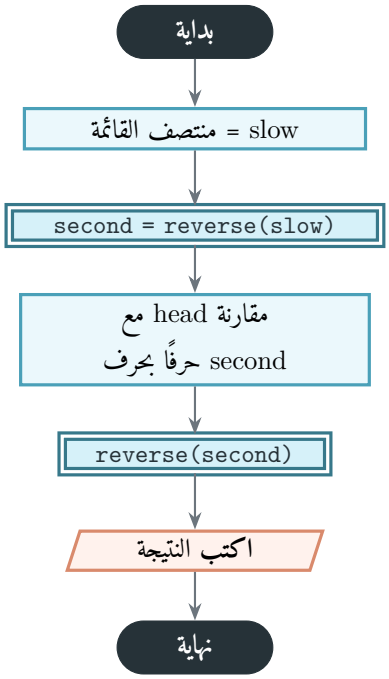
Node *fromText(const char *text) {
    Node dummy, *tail = &dummy;
    for (; *text != '\0'; text++) {
        tail->next = malloc(sizeof(Node));
        tail = tail->next;
        tail->ch = *text;
    }
    tail->next = NULL;
    return dummy.next;
}

int main(void) {
    const char *codes[3] = {"AB1221BA", "XY-7-YX", "SKU2026"};
    Node *head, *walk, *next;
    int idx;
    for (idx = 0; idx < 3; idx++) {
        head = fromText(codes[idx]);
        printf("%-9s %-14s list after: ", codes[idx], isPalindrome(head) ? "palindrome" : "not palindrome");
        for (walk = head; walk != NULL; walk = walk->next) putchar(walk->ch);
        printf("\n");
        for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    }
    return 0;
}
```

نتيجة التنفيذ

```
AB1221BA  palindrome     list after: AB1221BA
XY-7-YX   palindrome     list after: XY-7-YX
SKU2026   not palindrome list after: SKU2026
```

المخطط الإنسيابي (Organigramme)



تمرين 12.35 تبديل العقد زوجًا بزوج

متوسط ★★ تجارة إلكترونية

في قائمة ترتيب العرض، نريد تبديل كل منتجين متجاورين (1↔2، 3↔4، ...) **بإعادة ربط العقد** لا بتبديل البيانات. اكتب `swapPairs(head)` بمؤشر على مؤشر.

خطوات الحل

1) `link` يشير إلى الحقل الذي يحمل العقدة الأولى من الزوج.

2) `first = *link`, `second = first->next`:

```
first->next = second->next; second->next = first; *link = second;
```

.

3) ننتقل إلى `link = &first->next`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *next; } Node;

Node *swapPairs(Node *head) {
    Node **link = &head, *first, *second;
    while (*link != NULL && (*link)->next != NULL) {
        first = *link;
        second = first->next;
        first->next = second->next;          // إعادة الربط
        second->next = first;
        *link = second;
        link = &first->next;                 // الزوج التالي
    }
    return head;
}

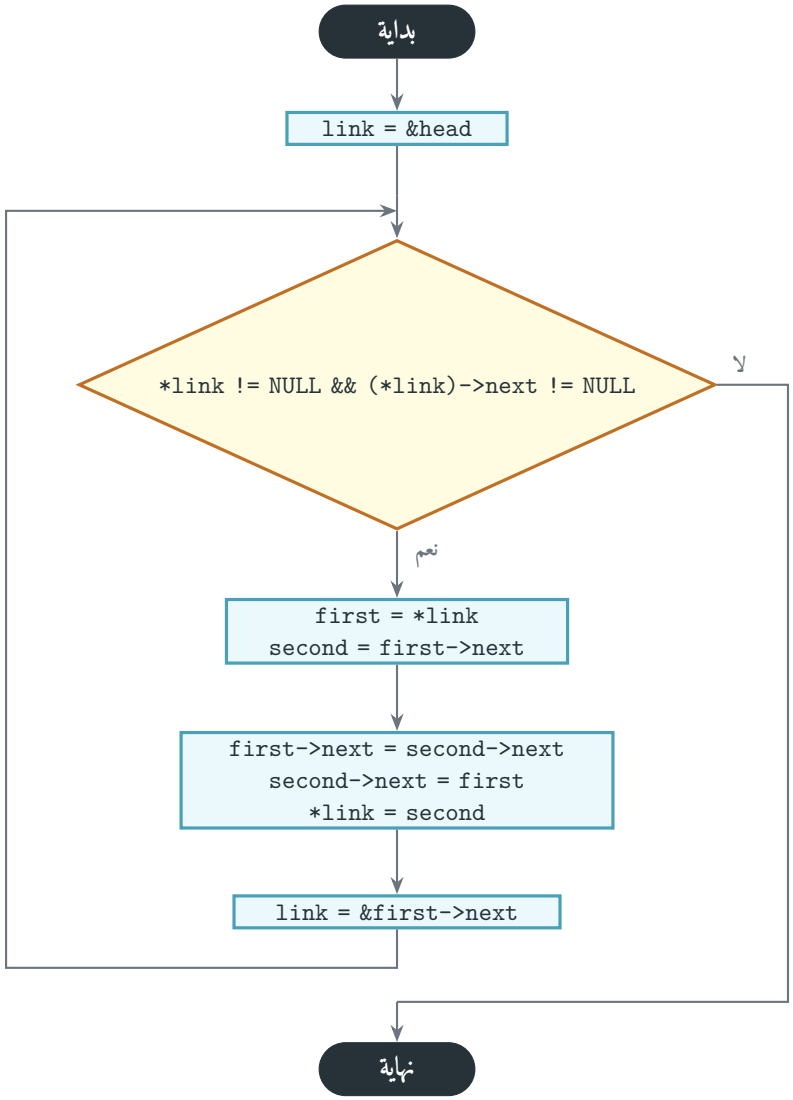
```

```c
int main(void) {
    Node *head = NULL, *node, *walk, *next;
    int idx;
    for (idx = 7; idx >= 1; idx--) { node = malloc(sizeof(Node)); node->code = idx; node->next = head; head
      ↪ = node; }
    head = swapPairs(head);
    for (walk = head; walk != NULL; walk = walk->next) printf("%d ", walk->code);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
2 1 4 3 6 5 7
```

المخطط الإنسيابي (Organigramme)



تمرين 12.36 نسخة مستقلة من قائمة الطلبيات

تجارة إلكترونية ★★ متوسط

اكتب `copyList(head)` تنشئ **نسخة عميقة** من قائمة الطلبيات (عقد جديدة بنفس البيانات ونفس الترتيب)، ثم عدّل النسخة (حذف وتغيير مبالغ) وبيّن أن الأصل لم يتغير.

خطوات الحل

1) عقدة حارسة ومؤشر ذيل للحفاظ على الترتيب.
2) لكل عقدة أصلية: `malloc` جديدة ونسخ البيانات.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; double amount; struct Node *next; } Node;

Node *copyList(const Node *head) {
    Node dummy, *tail = &dummy;
    for (; head != NULL; head = head->next) {
        tail->next = malloc(sizeof(Node));     // عقدة جديدة مستقلة
        tail = tail->next;
        tail->number = head->number;
        tail->amount = head->amount;
    }
    tail->next = NULL;
    return dummy.next;
}

void show(const char *label, const Node *head) {
    printf("%s", label);
    for (; head != NULL; head = head->next) printf(" #%d:%.0f", head->number, head->amount);
    printf("\n");
}

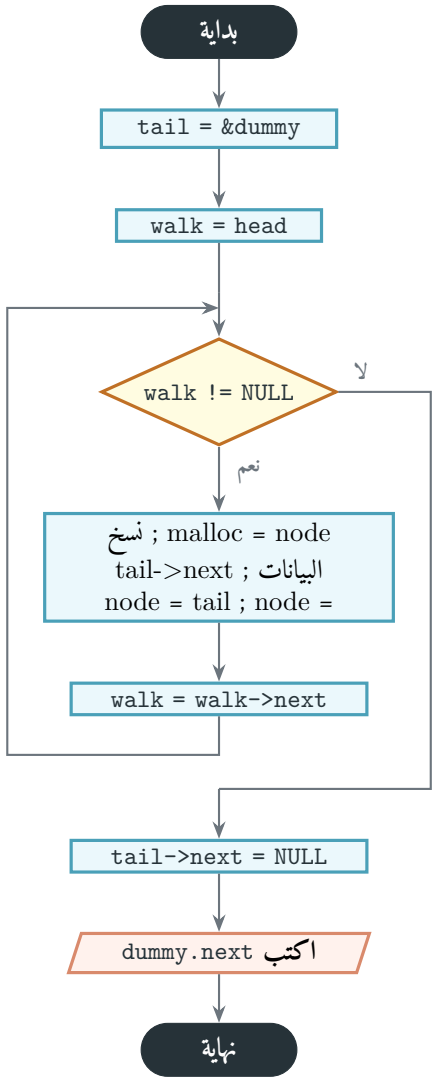
void freeList(Node *head) { Node *next; for (; head != NULL; head = next) { next = head->next; free(head); } }

int main(void) {
    double amounts[4] = {12500, 3200, 899, 45000};
    Node *original = NULL, *copy, *node, *walk;
    int idx;
    for (idx = 3; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->number = 7781 + idx; node->amount = amounts[idx];
        node->next = original; original = node;
    }
    copy = copyList(original);
    for (walk = copy; walk != NULL; walk = walk->next) walk->amount *= 0.9;   // خصم على النسخة
    node = copy->next;
    copy->next = node->next;                                                  // حذف الثانية من النسخة
    free(node);
    show("original:", original);
    show("copy:    ", copy);
    freeList(original);
    freeList(copy);
    return 0;
}
```

نتيجة التنفيذ

```
original: #7781:12500 #7782:3200 #7783:899 #7784:45000
copy:     #7781:11250 #7783:809 #7784:40500
```

المخطط الإنسيابي (Organigramme)



تمرين 12.37 تكرار الكلمات في قائمة مرتبة

علم البيانات

متوسط

ابنِ قائمة مرتبة أبجديًا من الكلمات الفريدة في وسوم المنتجات مع عداد لكل كلمة: عند كل كلمة نبحث عن موضعها، إن وُجدت نزيد عدادها، وإلا ندرجها في مكانها. اعرض القائمة النهائية.

خطوات الحل

1) نتوقف عند أول كلمة ≥ الكلمة الجديدة (مقارنة أبجدية يدوية).
2) المساواة: زيادة العداد؛ وإلا إدراج قبلها (مؤشر على مؤشر).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char word[12]; int count; struct Node *next; } Node;

int compareText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
```

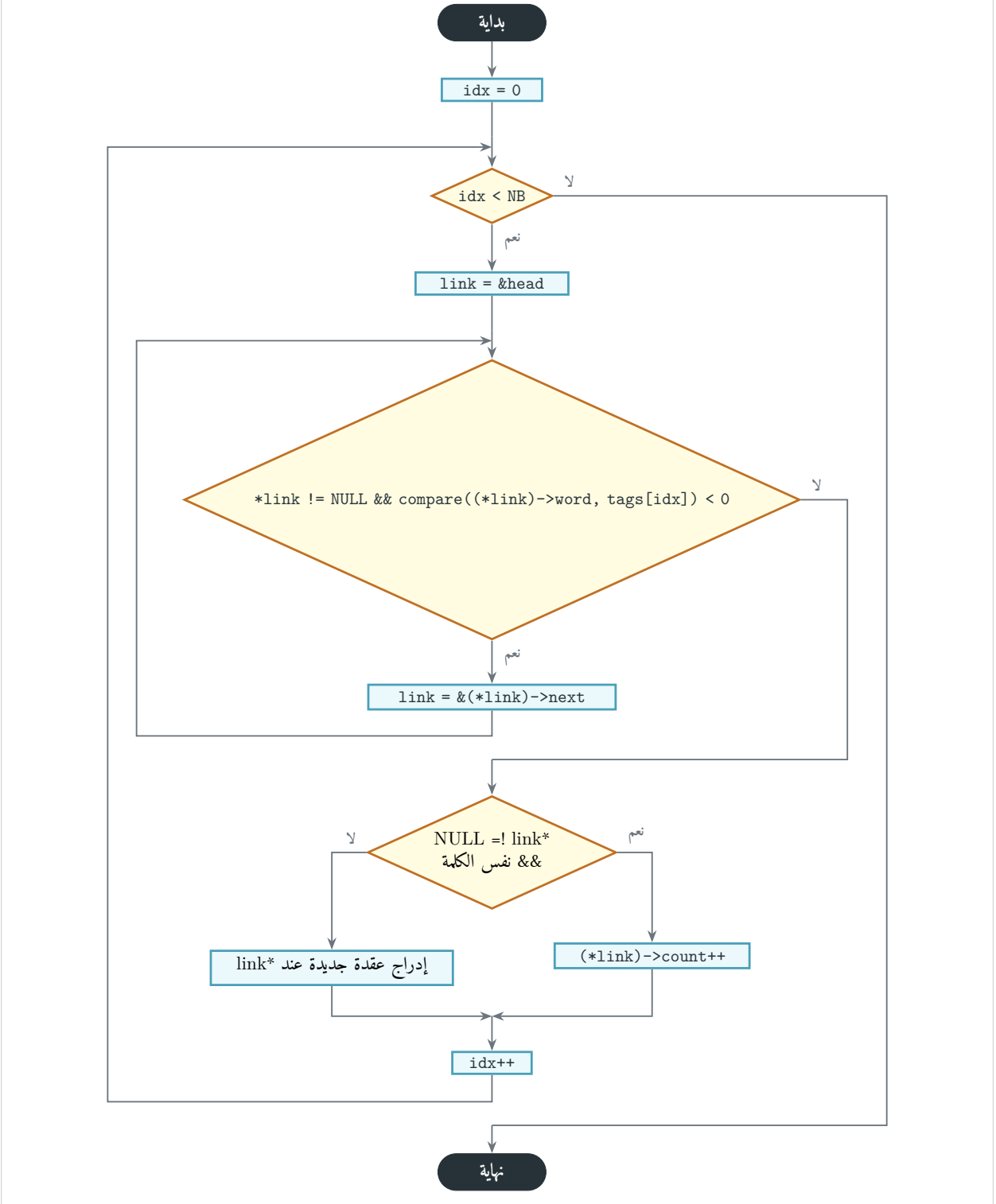
```c
    return *first - *second;
}

int main(void) {
    const char *tags[12] = {"wireless", "usb", "gaming", "usb", "rgb", "wireless", "fast",
                            "gaming", "usb", "audio", "rgb", "usb"};
    Node *head = NULL, **link, *node, *next;
    int idx, pos;
    for (idx = 0; idx < 12; idx++) {
        link = &head;
        while (*link != NULL && compareText((*link)->word, tags[idx]) < 0) link = &(*link)->next;
        if (*link != NULL && compareText((*link)->word, tags[idx]) == 0) {
            (*link)->count++;                                   // موجودة
        } else {
            node = malloc(sizeof(Node));                        // إدراج في مكانها
            for (pos = 0; (node->word[pos] = tags[idx][pos]) != '\0'; pos++) {}
            node->count = 1;
            node->next = *link;
            *link = node;
        }
    }
    for (node = head; node != NULL; node = node->next) printf("%-9s %d\n", node->word, node->count);
    for (node = head; node != NULL; node = next) { next = node->next; free(node); }
    return 0;
}
```

نتيجة التنفيذ

```
audio     1
fast      1
gaming    2
rgb       2
usb       4
wireless  2
```

المخطط الإنسيابي (Organigramme)



تمرين 12.38 حذف آخر ظهور لمنتج في السلة

تجارة إلكترونية

متوسط

السلة قائمة قد تحتوي على نفس المنتج عدة مرات. زر "إزالة واحد" يحذف **آخر ظهور** للمنتج. اكتب `removeLast(head, code)` بمرور واحد يتذكر الرابط المؤدي إلى آخر ظهور.

خطوات الحل

1) `lastLink` = عنوان الحقل الذي يشير إلى آخر عقدة مطابقة رأيناها.
2) بعد المرور: إن وُجد `lastLink` نحذف العقدة التي يشير إليها.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *next; } Node;

Node *removeLast(Node *head, int code) {
    Node **link = &head, **lastLink = NULL, *node;
    for (; *link != NULL; link = &(*link)->next)
        if ((*link)->code == code) lastLink = link;      // تذكّر آخر ظهور
    if (lastLink != NULL) {
        node = *lastLink;
        *lastLink = node->next;
        free(node);
    }
    return head;
}

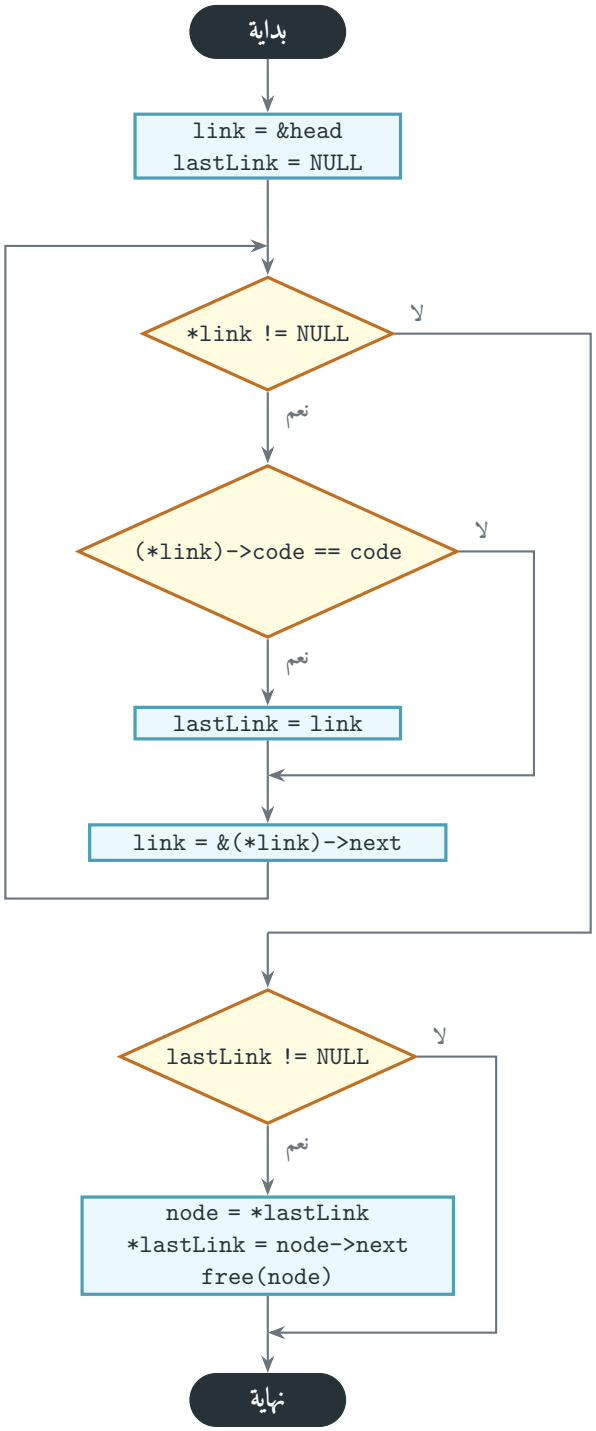
void show(const char *label, const Node *head) {
    printf("%-15s", label);
    for (; head != NULL; head = head->next) printf(" %d", head->code);
    printf("\n");
}

int main(void) {
    int cart[8] = {101, 205, 101, 330, 101, 205, 412, 205};
    Node *head = NULL, *node, *next;
    int idx;
    for (idx = 7; idx >= 0; idx--) { node = malloc(sizeof(Node)); node->code = cart[idx]; node->next = head;
      ↪ head = node; }
    show("cart:", head);
    head = removeLast(head, 101); show("remove one 101:", head);
    head = removeLast(head, 205); show("remove one 205:", head);
    head = removeLast(head, 999); show("remove one 999:", head);
    for (; head != NULL; head = next) { next = head->next; free(head); }
    return 0;
}
```

نتيجة التنفيذ

```
cart:           101 205 101 330 101 205 412 205
remove one 101: 101 205 101 330 205 412 205
remove one 205: 101 205 101 330 205 412
remove one 999: 101 205 101 330 205 412
```

المخطط الإنسيابي (Organigramme)



تمرين 12.39 اتحاد قائمتي مشتركي النشرة

علم البيانات

متوسط

قائمتا معرفات مشتركي النشرة البريدية (موقع وتطبيق) مرتبتان وقد تتداخلان. ابنِ قائمة **الاتحاد** دون تكرار بمرور متزامن واحد، واحسب عدد المشتركين المختلفين وعدد المسجلين في القناتين معًا.

خطوات الحل

1) الأصغر يُنسخ ويتقدم؛ عند التساوي ننسخ مرة واحدة ونتقدم بالاثنين (ونعد).
2) في النهاية ننسخ ما تبقى.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int id; struct Node *next; } Node;

Node *fromArray(const int *values, int size) {
    Node *head = NULL, *node;
    while (size-- > 0) { node = malloc(sizeof(Node)); node->id = values[size]; node->next = head; head = node; }
    return head;
}

void append(Node **tail, int id) {
    (*tail)->next = malloc(sizeof(Node));
    *tail = (*tail)->next;
    (*tail)->id = id;
    (*tail)->next = NULL;
}

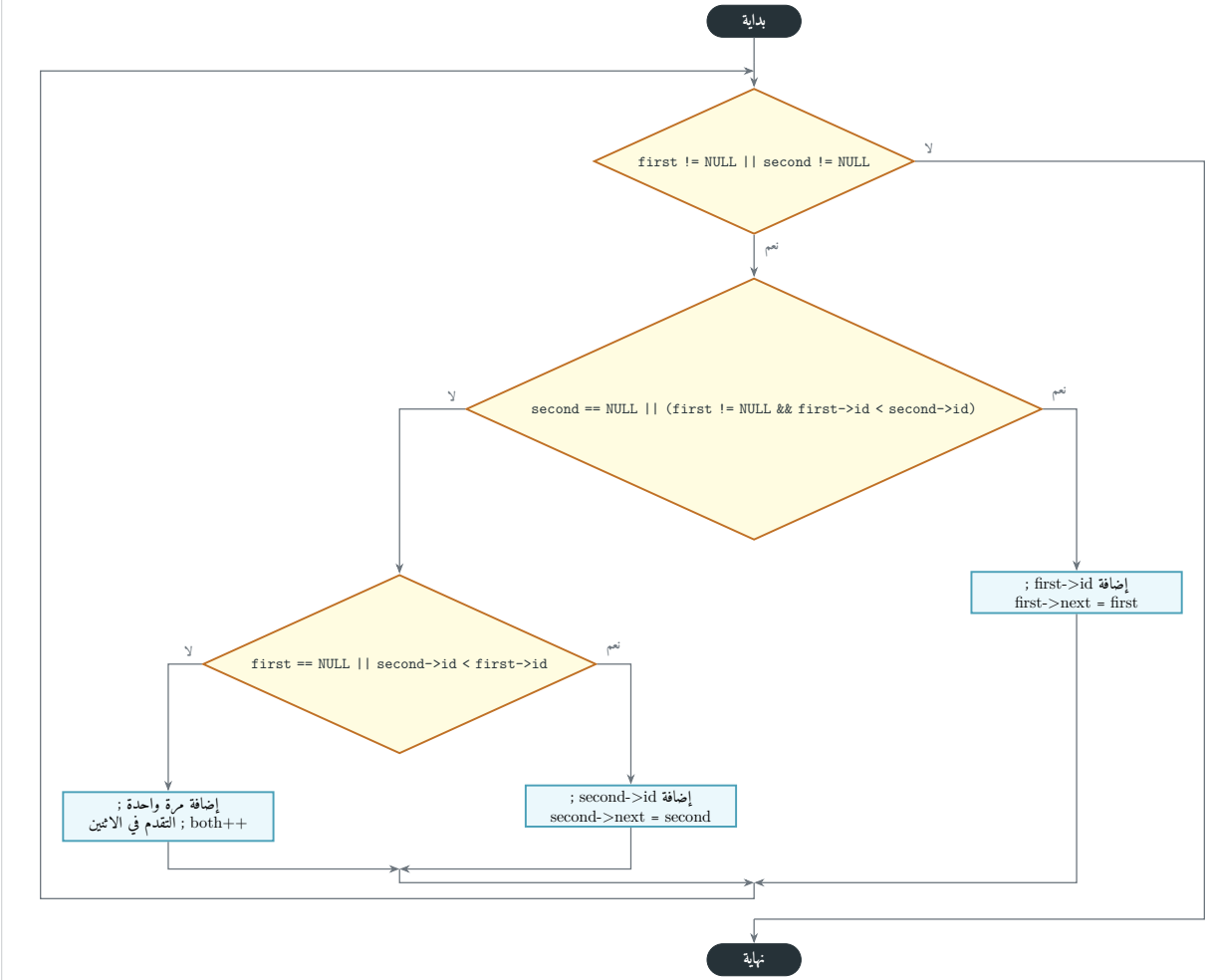
void freeList(Node *head) { Node *next; for (; head != NULL; head = next) { next = head->next; free(head); } }

int main(void) {
    int web[6] = {501, 503, 507, 512, 518, 520}, app[5] = {502, 503, 512, 519, 520};
    Node *first = fromArray(web, 6), *second = fromArray(app, 5), *walkA = first, *walkB = second;
    Node dummy, *tail = &dummy, *walk;
    int both = 0, total = 0;
    dummy.next = NULL;
    while (walkA != NULL || walkB != NULL) {
        if (walkB == NULL || (walkA != NULL && walkA->id < walkB->id)) { append(&tail, walkA->id); walkA = walkA->next; }
        else if (walkA == NULL || walkB->id < walkA->id) { append(&tail, walkB->id); walkB = walkB->next; }
        else { append(&tail, walkA->id); walkA = walkA->next; walkB = walkB->next; both++; }   // مرة واحدة
    }
    printf("Union:");
    for (walk = dummy.next; walk != NULL; walk = walk->next) { printf(" %d", walk->id); total++; }
    printf("\n%d distinct subscribers, %d on both channels\n", total, both);
    freeList(first); freeList(second); freeList(dummy.next);
    return 0;
}
```

نتيجة التنفيذ

```
Union: 501 502 503 507 512 518 519 520
8 distinct subscribers, 3 on both channels
```

المخطط الإنسيابي (Organigramme)



تمرين 12.40 الحذف من القائمة المزدوجة

تجارة إلكترونية متوسط

قائمة انتظار المنتجات النافذة **مزدوجة الربط** (رأس وذيل). اكتب `removeNode(list, node)` تحذف أي عقدة في وقت ثابت بتحديث جاريها (دون البحث عن السابقة)، و `printForward` و `printBackward` للتحقق من الاتجاهين بعد كل حذف.

خطوات الحل

1) إذا كان للعقدة سابق: `node->prev->next = node->next`، وإلا `list->head = node->next`.

2) إذا كان لها تالٍ: `node->next->prev = node->prev`، وإلا `list->tail = node->prev`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int code; struct Node *prev, *next; } Node;
typedef struct { Node *head, *tail; } List;

void pushBack(List *list, int code) {
    Node *node = malloc(sizeof(Node));
    node->code = code; node->next = NULL; node->prev = list->tail;
```

```c
    if (list->tail != NULL) list->tail->next = node; else list->head = node;
    list->tail = node;
}

void removeNode(List *list, Node *node) {
    if (node->prev != NULL) node->prev->next = node->next; else list->head = node->next;
    if (node->next != NULL) node->next->prev = node->prev; else list->tail = node->prev;
    free(node);                                          // دون بحث: وقت ثابت
}

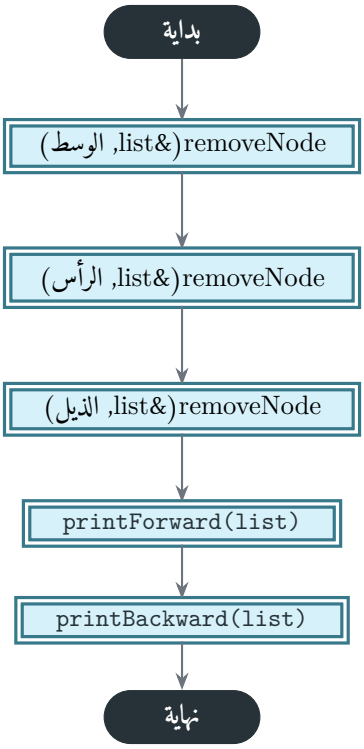
void printBoth(const char *label, const List *list) {
    const Node *walk;
    printf("%-14s fwd:", label);
    for (walk = list->head; walk != NULL; walk = walk->next) printf(" %d", walk->code);
    printf("  bwd:");
    for (walk = list->tail; walk != NULL; walk = walk->prev) printf(" %d", walk->code);
    printf("\n");
}

int main(void) {
    List list = {NULL, NULL};
    int idx;
    for (idx = 0; idx < 6; idx++) pushBack(&list, 101 + idx);
    printBoth("start", &list);
    removeNode(&list, list.head->next->next);  printBoth("remove 103", &list);
    removeNode(&list, list.head);              printBoth("remove head", &list);
    removeNode(&list, list.tail);              printBoth("remove tail", &list);
    while (list.head != NULL) removeNode(&list, list.head);
    return 0;
}
```

نتيجة التنفيذ

```
start          fwd: 101 102 103 104 105 106  bwd: 106 105 104 103 102 101
remove 103     fwd: 101 102 104 105 106  bwd: 106 105 104 102 101
remove head    fwd: 102 104 105 106  bwd: 106 105 104 102
remove tail    fwd: 102 104 105  bwd: 105 104 102
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 12.41 الترتيب بالدمج للقوائم المتصلة

تجارة إلكترونية ★★★ صعب

الترتيب بالدمج هو الأنسب للقوائم المتصلة (لا يحتاج إلى وصول عشوائي ولا ذاكرة إضافية). اكتب `mergeSort(head)` العودية: نقسم القائمة إلى نصفين بالمؤشرين السريع والبطيء، نرتب كل نصف، ثم ندمجهما بإعادة الربط. رتّب 12 طلبية حسب المبلغ.

خطوات الحل

1) الحالة الأساسية: قائمة فارغة أو بعقدة واحدة.
2) التقسيم: `slow` يتوقف قبل المنتصف؛ `second = slow->next; slow->next = NULL;`.
3) الدمج بعقدة حارسة كما في دمج قائمتين مرتبتين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int number; double amount; struct Node *next; } Node;

Node *merge(Node *first, Node *second) {
    Node dummy, *tail = &dummy;
    while (first != NULL && second != NULL) {
        if (first->amount <= second->amount) { tail->next = first; first = first->next; }
        else { tail->next = second; second = second->next; }
        tail = tail->next;
    }
    tail->next = first != NULL ? first : second;
    return dummy.next;
}

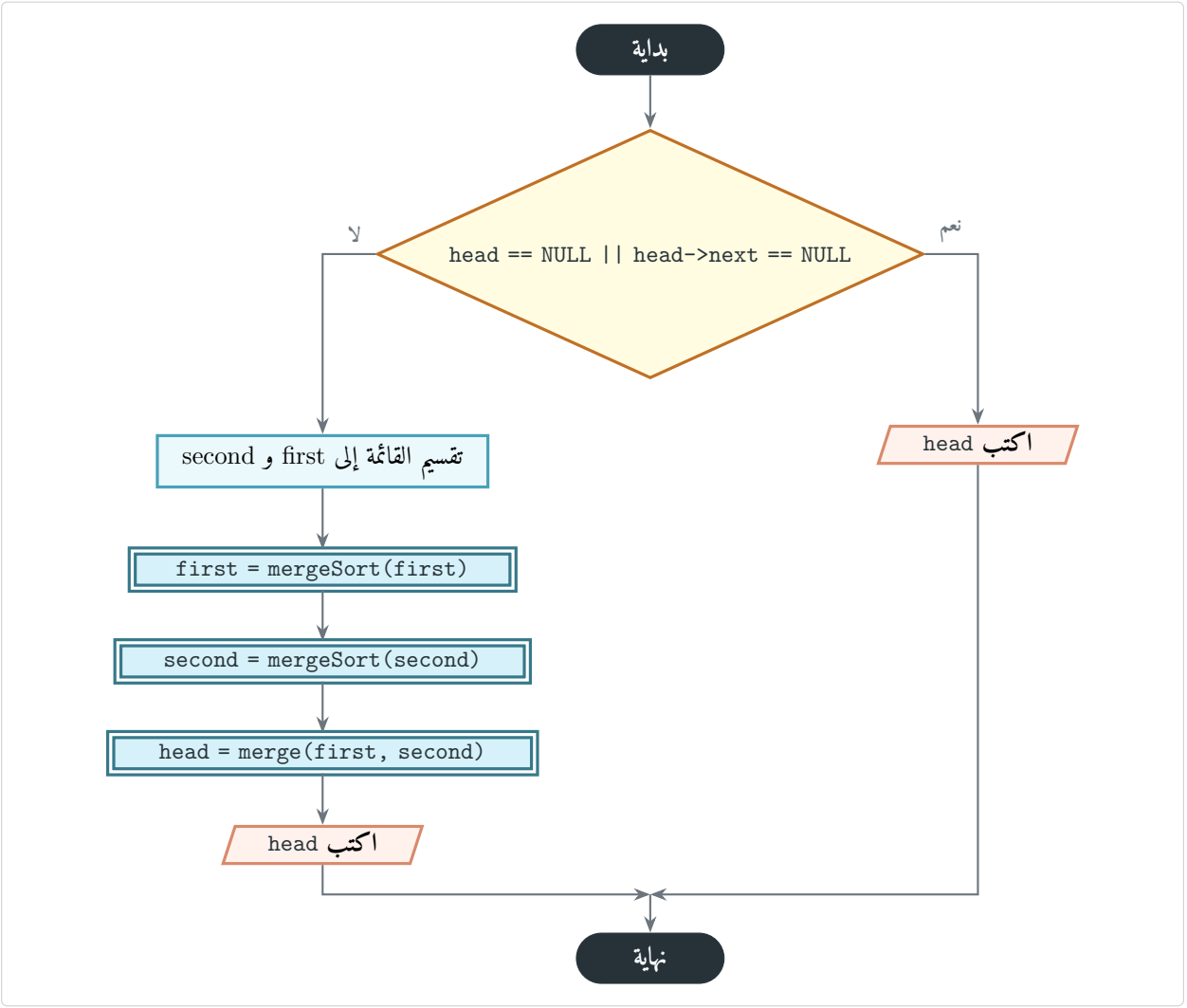
Node *mergeSort(Node *head) {
    Node *slow, *fast, *second;
    if (head == NULL || head->next == NULL) return head;
    slow = head;
    fast = head->next;
    while (fast != NULL && fast->next != NULL) { slow = slow->next; fast = fast->next->next; }
    second = slow->next;                        // النصف الثاني
    slow->next = NULL;                          // نقطع القائمة
    return merge(mergeSort(head), mergeSort(second));
}

int main(void) {
    double amounts[12] = {4500, 1200, 899, 27000, 650, 3200, 12500, 99, 7800, 2300, 560, 4100};
    Node *head = NULL, *node, *walk, *next;
    int idx;
    for (idx = 11; idx >= 0; idx--) {
        node = malloc(sizeof(Node)); node->number = 7781 + idx; node->amount = amounts[idx];
        node->next = head; head = node;
    }
    head = mergeSort(head);
    for (walk = head; walk != NULL; walk = walk->next) printf("#%d:%.0f ", walk->number, walk->amount);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
#7788:99 #7791:560 #7785:650 #7783:899 #7782:1200 #7790:2300 #7786:3200 #7792:4100
    #7781:4500 #7789:7800 #7787:12500 #7784:27000
```

المخطط الإنسيابي (Organigramme)



تمرين 12.42 ذاكرة تخزين مؤقت LRU لصفحات المنتجات

تجارة إلكترونية صعب

الخادم يحتفظ بآخر **3 صفحات منتجات** مولدة في ذاكرة مؤقتة. عند الطلب: إذا كانت الصفحة موجودة (**إصابة**) ننقلها إلى المقدمة، وإلا (**إخفاق**) نولدها ونضعها في المقدمة ونطرد **الأقل استعمالًا مؤخرًا** (الذيل). اكتب ذلك بقائمة **مزدوجة** + جدول فهرسة مباشر `slots[code]` يعطي العقدة في وقت ثابت.

خطوات الحل

1) `slots[code]` مؤشر على العقدة أو `NULL`: البحث في وقت ثابت.
2) القائمة المزدوجة تسمح بفصل أي عقدة ونقلها إلى الرأس في وقت ثابت.
3) عند الامتلاء نطرد `tail` ونمسح `slots[tail->code]`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define CAP 3
#define MAXCODE 100

typedef struct Node { int code; struct Node *prev, *next; } Node;

Node *head = NULL, *tail = NULL, *slots[MAXCODE];
int size = 0;

void detach(Node *node) {
    if (node->prev) node->prev->next = node->next; else head = node->next;
    if (node->next) node->next->prev = node->prev; else tail = node->prev;
}

void pushFront(Node *node) {
    node->prev = NULL;
    node->next = head;
    if (head) head->prev = node; else tail = node;
    head = node;
}

int main(void) {
    Node *node, *walk;
    int code, hits = 0, misses = 0;
    while (scanf("%d", &code) == 1) {
        node = slots[code];
        if (node != NULL) {                          // إصابة
            hits++;
            detach(node);
            pushFront(node);
            printf("get %d: HIT  |", code);
        } else {                                     // إخفاق
            misses++;
            node = malloc(sizeof(Node));
            node->code = code;
            pushFront(node);
            slots[code] = node;
            if (++size > CAP) {                      // طرد الأقل استعمالًا
                walk = tail;
                detach(walk);
                slots[walk->code] = NULL;
                printf("get %d: MISS evict %d |", code, walk->code);
                free(walk);
                size--;
            } else {
                printf("get %d: MISS |", code);
            }
        }
        for (walk = head; walk != NULL; walk = walk->next) printf(" %d", walk->code);
        printf("\n");
    }
    printf("hits %d, misses %d, hit rate %.0f%%\n", hits, misses, 100.0 * hits / (hits + misses));
    while (head != NULL) { node = head; detach(node); free(node); }
    return 0;
}
```

المدخلات

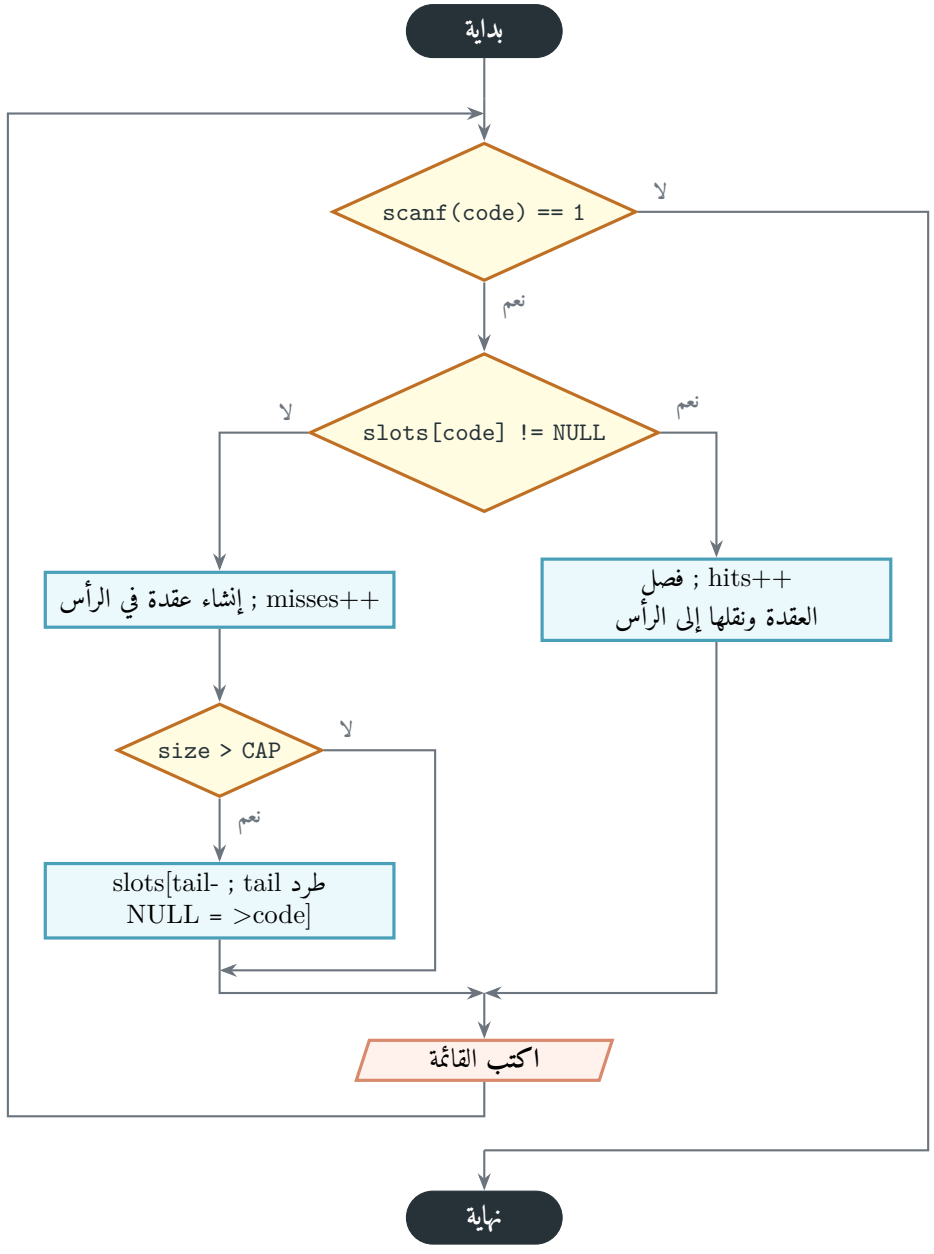
```
5 2 5 7 9 2 5 9 9 1
```

نتيجة التنفيذ

```
get 5: MISS | 5
get 2: MISS | 2 5
get 5: HIT  | 5 2
get 7: MISS | 7 5 2
get 9: MISS evict 2 | 9 7 5
get 2: MISS evict 5 | 2 9 7
```

```
get 5: MISS evict 7 | 5 2 9
get 9: HIT  | 9 5 2
get 9: HIT  | 9 5 2
get 1: MISS evict 2 | 1 9 5
hits 3, misses 7, hit rate 30%
```

المخطط الإنسيابي (Organigramme)



تمرين 12.43 سحب الفائز بقائمة دائرية

تجارة إلكترونية

صعب

في سحب ترويجي يجلس `count` مشاركين في دائرة، ونحذف كل `step`-ي مشارك حتى يبقى واحد هو الفائز (**مسألة يوسيفوس**). حاكِ ذلك بقائمة دائرية مع تحرير المحذوفين، ثم تحقق بالصيغة التكرارية `J(n) = (J(n-1) + step) % n`.

خطوات الحل

1) نبني القائمة الدائرية ونحتفظ بالعقدة السابقة للمشارك الحالي.
2) نتقدم `step - 1` خطوات، ثم نحذف `prev->next`.
3) نتوقف عندما تشير العقدة إلى نفسها.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int id; struct Node *next; } Node;

int main(void) {
    Node *head = NULL, *tail = NULL, *node, *prev, *out;
    int count, step, idx, survivor = 0, size;
    scanf("%d %d", &count, &step);
    for (idx = 1; idx <= count; idx++) {
        node = malloc(sizeof(Node));
        node->id = idx;
        node->next = NULL;
        if (head == NULL) head = node; else tail->next = node;
        tail = node;
    }
    tail->next = head;                              // نغلق الدائرة
    prev = tail;
    printf("Eliminated:");
    while (prev->next != prev) {
        for (idx = 1; idx < step; idx++) prev = prev->next;
        out = prev->next;                           // المشارك رقم step
        prev->next = out->next;
        printf(" %d", out->id);
        free(out);
    }
    printf("\nWinner: participant %d\n", prev->id);
    for (size = 2; size <= count; size++) survivor = (survivor + step) % size;   // صيغة يوسيفوس
    printf("Formula check: %d\n", survivor + 1);
    free(prev);
    return 0;
}
```

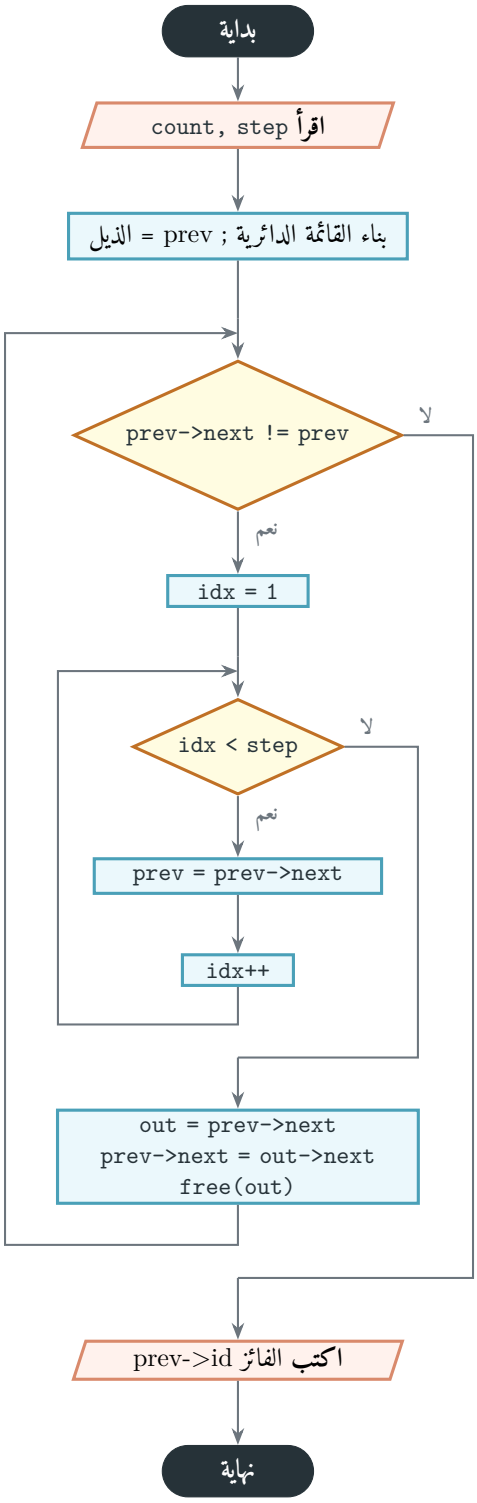
المدخلات

```
7 3
```

نتيجة التنفيذ

```
Eliminated: 3 6 2 7 5 1
Winner: participant 4
Formula check: 4
```

المخطط الإنسيابي (Organigramme)



تمرين 12.44 جداء كثيري حدود بالقوائم

علم البيانات صعب

الإيراد = السعر × الكمية، وكلاهما كثير حدود بدلالة الزمن. اكتب `multiplyPoly(first, second)`: لكل زوج من الحدود نضرب المعاملين ونجمع الأسين، وندرج الناتج في قائمة النتيجة مرتبة تنازليًا مع جمع الحدود المتشابهة وحذف المعدومة.

خطوات الحل

1) `addTerm(&result, coef, power)`: نبحث عن موضع الأس؛ إن وُجد نجمع المعاملين (ونحذف العقدة إذا أصبح 0)، وإلا ندرج عقدة جديدة.
2) حلقتان متداخلتان على حدود كثيري الحدود.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Term { double coef; int power; struct Term *next; } Term;

void addTerm(Term **poly, double coef, int power) {
    Term **link = poly, *node;
    while (*link != NULL && (*link)->power > power) link = &(*link)->next;
    if (*link != NULL && (*link)->power == power) {
        (*link)->coef += coef;                          // حد متشابه
        if ((*link)->coef == 0) { node = *link; *link = node->next; free(node); }
        return;
    }
    if (coef == 0) return;
    node = malloc(sizeof(Term));
    node->coef = coef;
    node->power = power;
    node->next = *link;                                 // إدراج مرتب
    *link = node;
}

Term *multiplyPoly(const Term *first, const Term *second) {
    Term *result = NULL;
    const Term *termA, *termB;
    for (termA = first; termA != NULL; termA = termA->next)
        for (termB = second; termB != NULL; termB = termB->next)
            addTerm(&result, termA->coef * termB->coef, termA->power + termB->power);
    return result;
}

void printPoly(const char *label, const Term *poly) {
    printf("%s", label);
    for (; poly != NULL; poly = poly->next) printf(" %+g t^%d", poly->coef, poly->power);
    printf("\n");
}

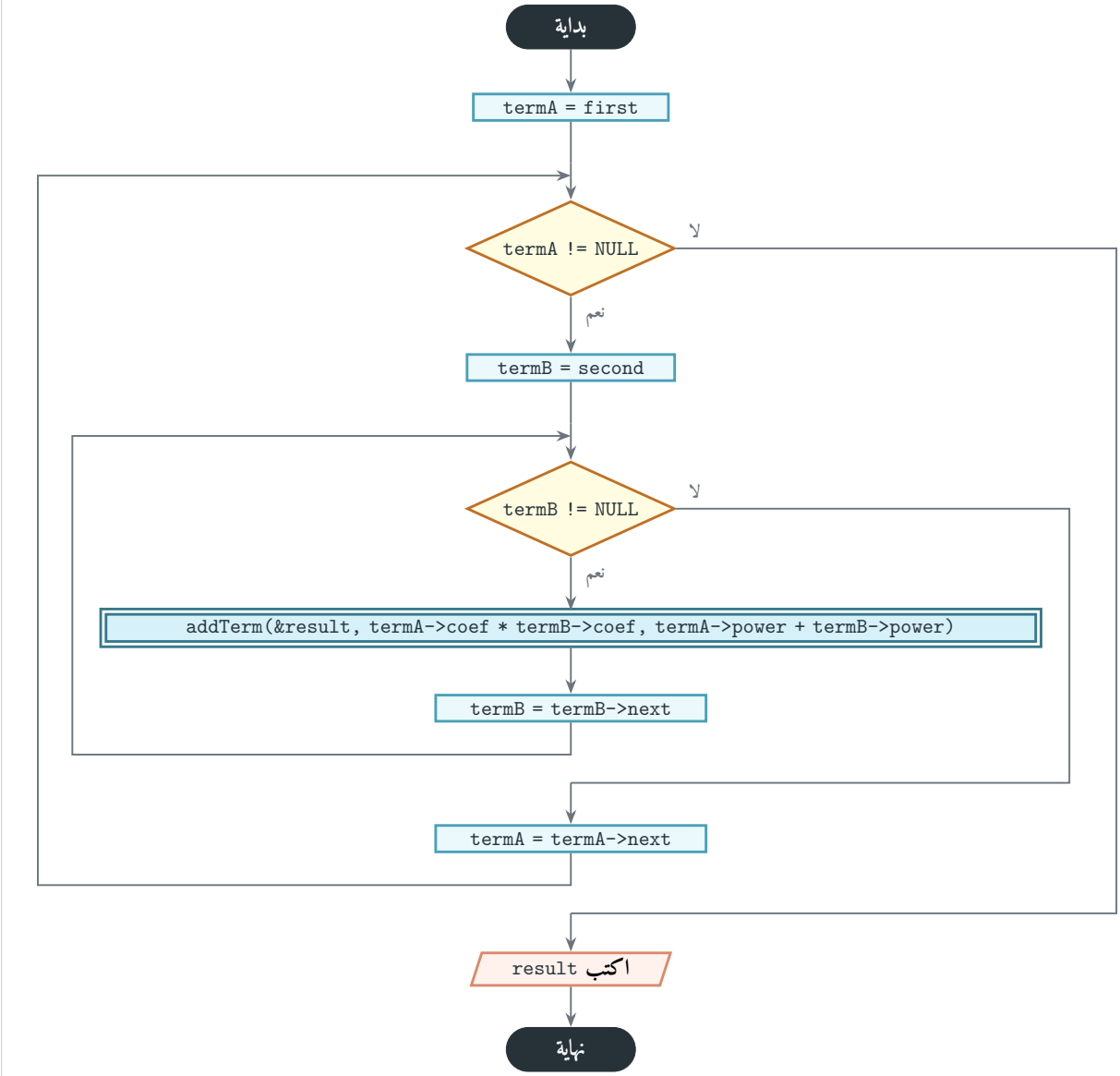
void freePoly(Term *poly) { Term *next; for (; poly != NULL; poly = next) { next = poly->next; free(poly); } }

int main(void) {
    Term *price = NULL, *quantity = NULL, *revenue;
    addTerm(&price, 3000, 0); addTerm(&price, -50, 1);                  // السعر يخفض
    addTerm(&quantity, 200, 0); addTerm(&quantity, 15, 1); addTerm(&quantity, -0.5, 2);
    revenue = multiplyPoly(price, quantity);
    printPoly("price(t)   =", price);
    printPoly("quantity(t)=", quantity);
    printPoly("revenue(t) =", revenue);
    freePoly(price); freePoly(quantity); freePoly(revenue);
    return 0;
}
```

نتيجة التنفيذ

```
price(t)   = -50 t^1 +3000 t^0
quantity(t)= -0.5 t^2 +15 t^1 +200 t^0
revenue(t) = +25 t^3 -2250 t^2 +35000 t^1 +600000 t^0
```

المخطط الإنسيابي (Organigramme)



تمرين 12.45 شبكة الإحالات بقوائم الجوار والبحث بالعرض

علم البيانات صعب ★★★★

كل زبون قد يُحيل زبائن آخرين (رابط إحالة في الاتجاهين = صداقة). نمثل الشبكة بـ **قوائم جوار**: لكل زبون قائمة متصلة بجيرانه. اكتب `addEdge` و **البحث بالعرض** (BFS) بطابور لحساب **درجة الانفصال** بين زبون مصدر وكل الآخرين، وعدد المكونات المتصلة.

خطوات الحل

1) `adjacency[node]` رأس قائمة الجيران؛ `addEdge(a, b)` تدرج b في قائمة a و a في قائمة b.
2) BFS: نبدأ من المصدر بمسافة 0، ونعالج الطابور: كل جار غير مزار يأخذ مسافة +1.
3) المكونات: نكرر BFS من كل عقدة غير مزارة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBN 9

typedef struct Edge { int to; struct Edge *next; } Edge;

Edge *adjacency[NBN];

void addEdge(int from, int to) {
    Edge *edge = malloc(sizeof(Edge));
    edge->to = to; edge->next = adjacency[from]; adjacency[from] = edge;
    edge = malloc(sizeof(Edge));
    edge->to = from; edge->next = adjacency[to]; adjacency[to] = edge;   // الاتجاهان
}

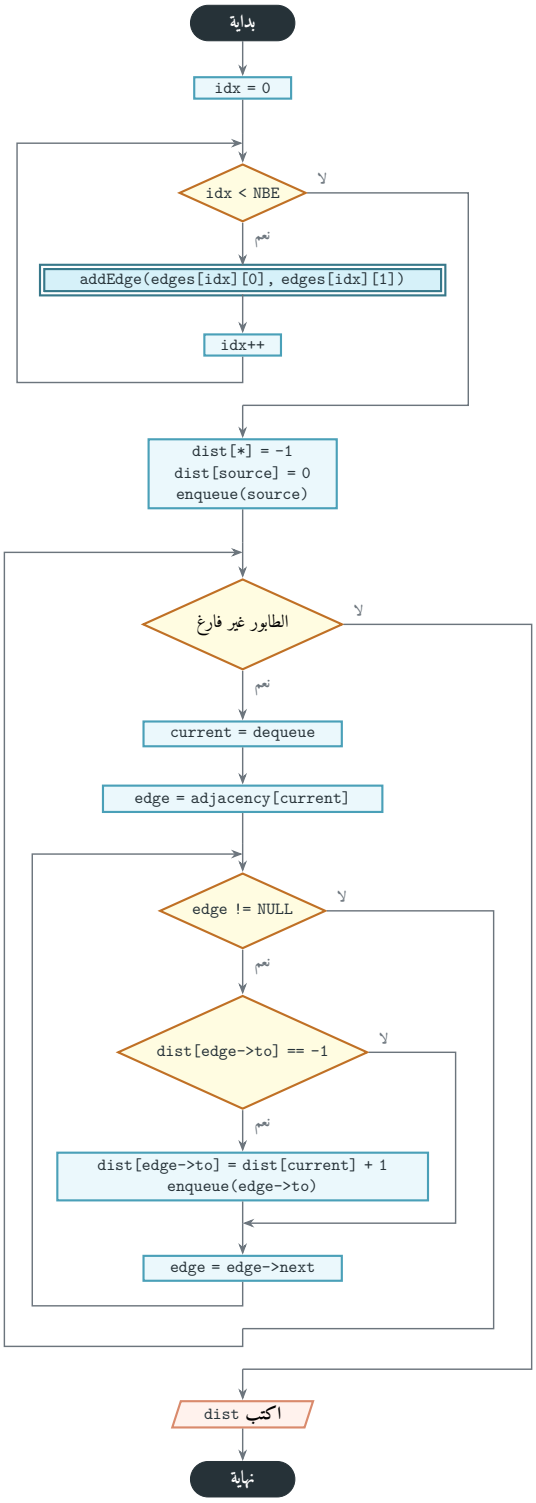
void bfs(int source, int dist[]) {
    int queue[NBN], front = 0, rear = 0, current;
    Edge *edge;
    dist[source] = 0;
    queue[rear++] = source;
    while (front < rear) {
        current = queue[front++];
        for (edge = adjacency[current]; edge != NULL; edge = edge->next)
            if (dist[edge->to] == -1) {
                dist[edge->to] = dist[current] + 1;   // مستوى إضافي
                queue[rear++] = edge->to;
            }
    }
}

int main(void) {
    int edges[9][2] = {{0, 1}, {0, 2}, {1, 3}, {2, 3}, {3, 4}, {4, 5}, {6, 7}, {1, 2}, {5, 2}};
    int dist[NBN], node, components = 0;
    Edge *edge, *next;
    for (node = 0; node < 9; node++) addEdge(edges[node][0], edges[node][1]);
    for (node = 0; node < NBN; node++) dist[node] = -1;
    bfs(0, dist);
    printf("Degrees of separation from customer 0:\n");
    for (node = 1; node < NBN; node++)
        if (dist[node] >= 0) printf("  customer %d: %d\n", node, dist[node]);
        else printf("  customer %d: unreachable\n", node);
    for (node = 0; node < NBN; node++) dist[node] = -1;
    for (node = 0; node < NBN; node++)
        if (dist[node] == -1) { bfs(node, dist); components++; }
    printf("Connected groups: %d\n", components);
    for (node = 0; node < NBN; node++)
        for (edge = adjacency[node]; edge != NULL; edge = next) { next = edge->next; free(edge); }
    return 0;
}
```

نتيجة التنفيذ

```
Degrees of separation from customer 0:
  customer 1: 1
  customer 2: 1
  customer 3: 2
  customer 4: 3
  customer 5: 2
  customer 6: unreachable
  customer 7: unreachable
  customer 8: unreachable
Connected groups: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 12.46 مجمع عقد يعيد استعمال الذاكرة

تجارة إلكترونية

صعب ★★★★

السلة تضيف وتحذف عناصر آلاف المرات؛ استدعاء `malloc`/`free` لكل عملية مكلف. نبني **مجمع عقد**: نحجز كتلة واحدة من 8 عقد ونربطها في **قائمة حرة**. `allocNode` تسحب من القائمة الحرة و `releaseNode` تعيد العقدة إليها. نفّذ سيناريو إضافة وحذف وعدّ استدعاءات `malloc`.

خطوات الحل

1) عند الإنشاء: `pool = malloc(CAP * sizeof(Node))` ثم ربط `pool[idx].next = &pool[idx + 1]`.
2) `allocNode`: `node = freeList; freeList = node->next;`.
3) `releaseNode`: `node->next = freeList; freeList = node;`.
4) نفس العقدة تُستعمل لعناصر مختلفة عبر الزمن.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define CAP 8

typedef struct Node { int code; struct Node *next; } Node;

Node *freeList = NULL;
int mallocCalls = 0;

void initPool(Node *pool) {
    int idx;
    for (idx = 0; idx < CAP - 1; idx++) pool[idx].next = &pool[idx + 1];   // سلسلة حرة
    pool[CAP - 1].next = NULL;
    freeList = pool;
}

Node *allocNode(void) {
    Node *node = freeList;
    if (node != NULL) freeList = node->next;
    return node;                                  // NULL إذا امتلأ المجمع
}

void releaseNode(Node *node) {
    node->next = freeList;                        // تعود إلى القائمة الحرة
    freeList = node;
}

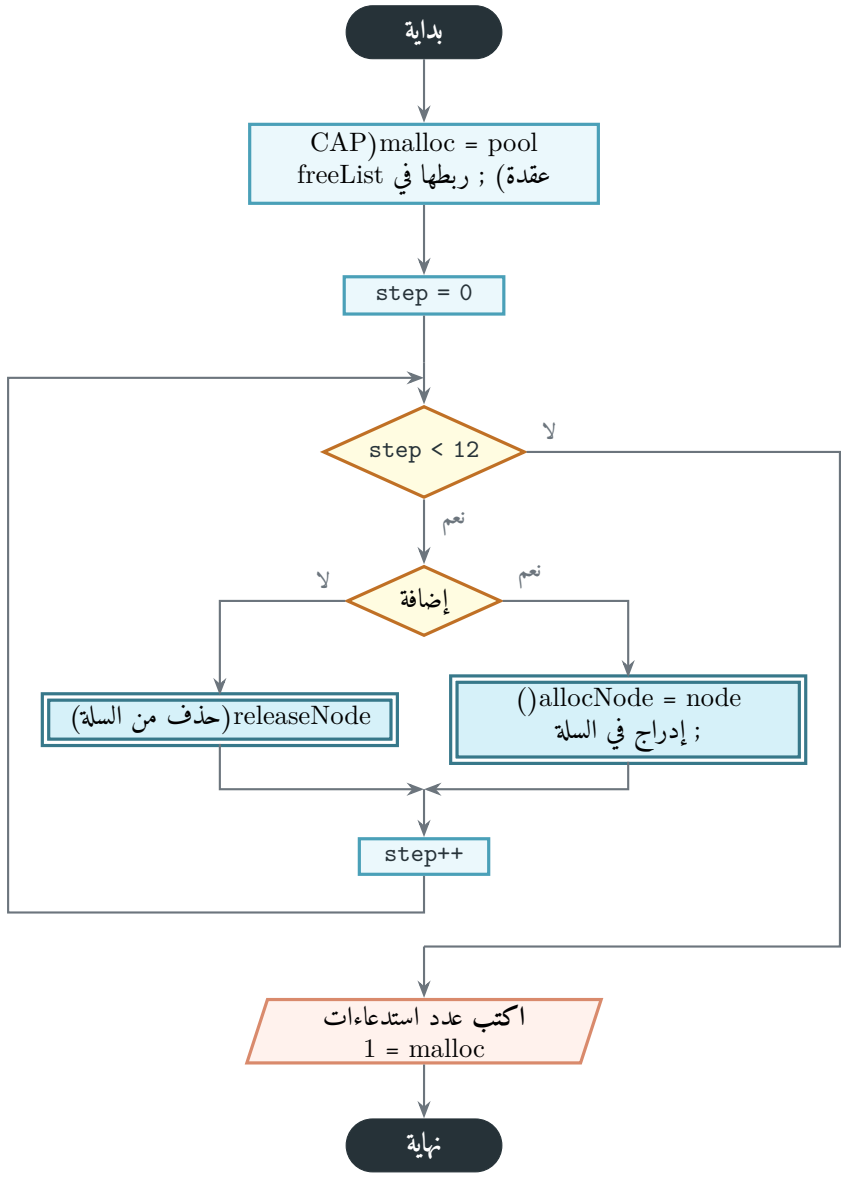
int freeCount(void) { int count = 0; Node *walk; for (walk = freeList; walk != NULL; walk = walk->next)
      ↪ count++; return count; }

int main(void) {
    Node *pool = malloc(CAP * sizeof(Node)), *cart = NULL, *node;
    int script[12] = {101, 205, 330, 0, 412, 0, 0, 500, 618, 701, 0, 802}, step;   // 0 = حذف الرأس
    mallocCalls++;
    initPool(pool);
    for (step = 0; step < 12; step++) {
        if (script[step] != 0) {
            node = allocNode();
            node->code = script[step];
            node->next = cart;
            cart = node;
            printf("add %d at slot %d", node->code, (int)(node - pool));
        } else {
            node = cart;
            cart = cart->next;
            printf("remove %d, slot %d freed", node->code, (int)(node - pool));
            releaseNode(node);
        }
        printf("   (free slots %d)\n", freeCount());
    }
    printf("malloc calls: %d for %d operations\n", mallocCalls, step);
    free(pool);
    return 0;
}
```

نتيجة التنفيذ

```
add 101 at slot 0   (free slots 7)
add 205 at slot 1   (free slots 6)
add 330 at slot 2   (free slots 5)
remove 330, slot 2 freed   (free slots 6)
add 412 at slot 2   (free slots 5)
remove 412, slot 2 freed   (free slots 6)
remove 205, slot 1 freed   (free slots 7)
add 500 at slot 1   (free slots 6)
add 618 at slot 2   (free slots 5)
add 701 at slot 3   (free slots 4)
remove 701, slot 3 freed   (free slots 5)
add 802 at slot 3   (free slots 4)
malloc calls: 1 for 12 operations
```

المخطط الإنسيابي (Organigramme)



تمرين 12.47 نسخ قائمة بروابط التوصية تجارة إلكترونية صعب ★★★★

كل منتج في قائمة العرض يحتوي، إضافة إلى `next`، على مؤشر `related` نحو منتج آخر في **نفس القائمة** (منتج موصى به) أو `NULL`. اكتب `cloneList(head)` تنسخ القائمة نسخة عميقة بحيث تشير روابط `related` في النسخة إلى عقد **النسخة**، بتقنية التشبيك: (1) ندرج كل نسخة بعد أصلها، (2) نضبط `related`، (3) نفصل القائمتين.

خطوات الحل

1) المرحلة 1: `A → A' → B → B' → ...`.

2) المرحلة 2:

```
copy->related = original->related ? original->related->next : NULL
```

.

3) المرحلة 3: نعيد ربط الأصل والنسخة كقائمتين منفصلتين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { char name[10]; struct Node *next, *related; } Node;

Node *cloneList(Node *head) {
    Node *walk, *copy, *cloneHead;
    int pos;
    for (walk = head; walk != NULL; walk = copy->next) {          // 1) تشبيك
        copy = malloc(sizeof(Node));
        for (pos = 0; (copy->name[pos] = walk->name[pos]) != '\0'; pos++) {}
        copy->next = walk->next;
        walk->next = copy;
    }
    for (walk = head; walk != NULL; walk = walk->next->next)       // 2) روابط التوصية
        walk->next->related = walk->related != NULL ? walk->related->next : NULL;
    cloneHead = head != NULL ? head->next : NULL;
    for (walk = head; walk != NULL; walk = walk->next) {           // 3) الفصل
        copy = walk->next;
        walk->next = copy->next;
        copy->next = copy->next != NULL ? copy->next->next : NULL;
    }
    return cloneHead;
}

void show(const char *label, Node *head, Node *other) {
    Node *walk;
    printf("%s", label);
    for (walk = head; walk != NULL; walk = walk->next) {
        printf(" %s->%s", walk->name, walk->related ? walk->related->name : "-");
        if (walk->related != NULL && other != NULL && walk->related == other) printf("(!)");
    }
    printf("\n");
}

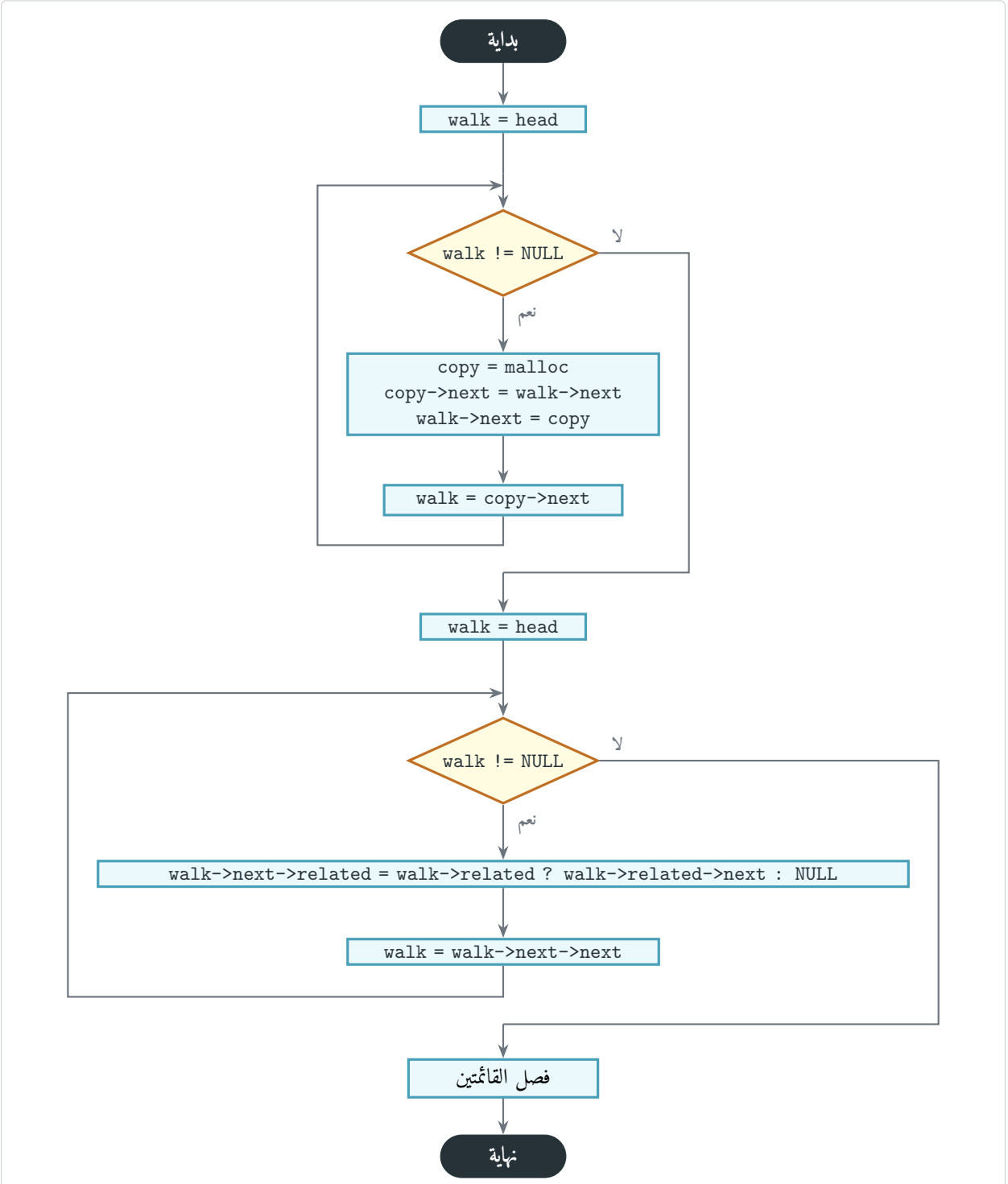
int main(void) {
    Node items[5] = {{"Phone", NULL, NULL}, {"Case", NULL, NULL}, {"Charger", NULL, NULL},
                     {"Laptop", NULL, NULL}, {"Mouse", NULL, NULL}};
    Node *clone, *walk, *next, *inside;
    int idx, independent = 1;
    for (idx = 0; idx < 4; idx++) items[idx].next = &items[idx + 1];
    items[0].related = &items[1]; items[1].related = &items[0];
    items[2].related = &items[0]; items[3].related = &items[4];
    clone = cloneList(&items[0]);
    show("original:", &items[0], NULL);
    show("clone:   ", clone, NULL);
    for (walk = clone; walk != NULL; walk = walk->next)            // هل تشير النسخة إلى الأصل؟
        for (inside = &items[0]; inside != NULL; inside = inside->next)
            if (walk->related == inside) independent = 0;
    printf("clone links stay inside clone: %s\n", independent ? "yes" : "no");
    for (walk = clone; walk != NULL; walk = next) { next = walk->next; free(walk); }
```

```
52      return 0;
53  }
```

نتيجة التنفيذ

```
original: Phone->Case Case->Phone Charger->Phone Laptop->Mouse Mouse->-
clone:    Phone->Case Case->Phone Charger->Phone Laptop->Mouse Mouse->-
clone links stay inside clone: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 12.48 تسطيح شجرة الأقسام متعددة المستويات

تجارة إلكترونية ★★★ صعب

قائمة الأقسام متعددة المستويات: كل عقدة لها `next` (القسم التالي في نفس المستوى) و `child` (قائمة الأقسام الفرعية). لإنشاء قائمة منسدلة في الموقع، اكتب `flatten(head)` تحول البنية إلى **قائمة واحدة** بترتيب العمق أولًا (القسم ثم فروعه ثم القسم التالي)، **في مكانها** ودون عودية، مع تسجيل عمق كل قسم للإزاحة.

خطوات الحل

1) نمر على القائمة؛ عند عقدة لها `child`: نجد ذيل قائمة الأبناء، نربطه بـ `node->next`، ثم `node->next = child` و `child = NULL`.
2) العمق: كل ابن عمقه عمق الأب + 1 (نضبطه قبل الربط).

البرنامج بلغة C

```c
#include <stdio.h>

typedef struct Cat { char name[14]; int depth; struct Cat *next, *child; } Cat;

void flatten(Cat *head) {
    Cat *walk, *tail;
    for (walk = head; walk != NULL; walk = walk->next) {
        if (walk->child == NULL) continue;
        for (tail = walk->child; ; tail = tail->next) {
            tail->depth = walk->depth + 1;              // عمق الأبناء
            if (tail->next == NULL) break;
        }
        tail->next = walk->next;                        // الأبناء قبل الإخوة
        walk->next = walk->child;
        walk->child = NULL;
    }
}

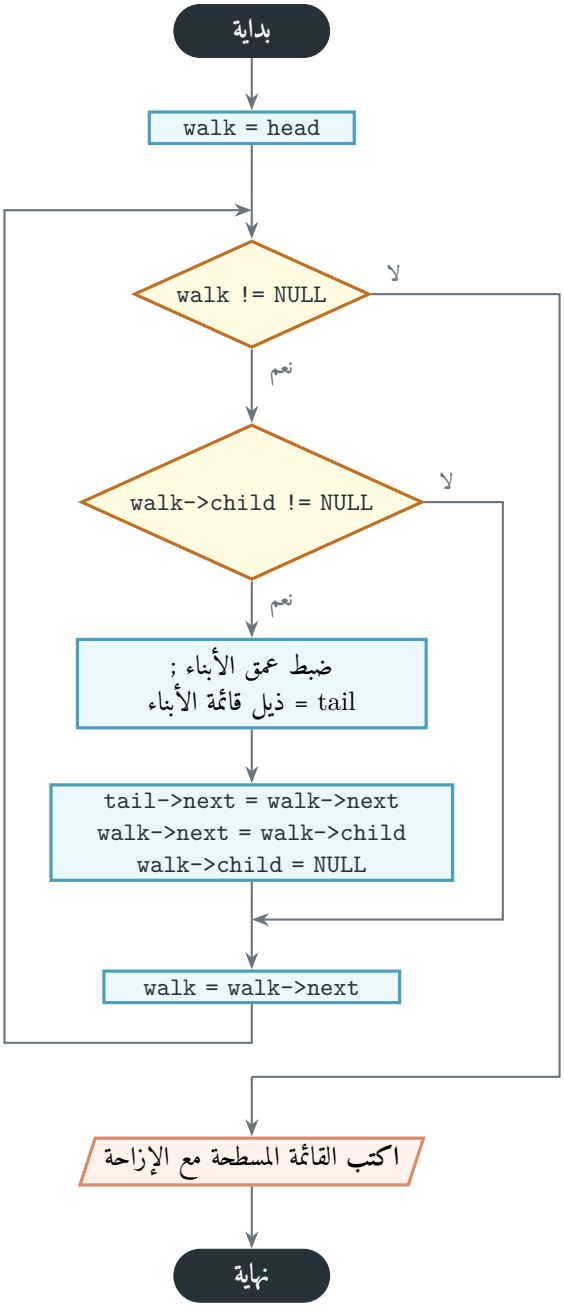
int main(void) {
    Cat nodes[11] = {{"Electronics", 0, 0, 0}, {"Phones", 0, 0, 0}, {"Android", 0, 0, 0},
                     {"iOS", 0, 0, 0}, {"Computers", 0, 0, 0}, {"Laptops", 0, 0, 0},
                     {"Gaming", 0, 0, 0}, {"Office", 0, 0, 0}, {"Home", 0, 0, 0},
                     {"Kitchen", 0, 0, 0}, {"Garden", 0, 0, 0}};
    Cat *walk;
    nodes[0].next = &nodes[8];                                  // المستوى 0
    nodes[0].child = &nodes[1]; nodes[1].next = &nodes[4];      // Electronics
    nodes[1].child = &nodes[2]; nodes[2].next = &nodes[3];      // Phones
    nodes[4].child = &nodes[5];                                 // Computers
    nodes[5].child = &nodes[6]; nodes[6].next = &nodes[7];      // Laptops
    nodes[8].child = &nodes[9]; nodes[9].next = &nodes[10];     // Home
    flatten(&nodes[0]);
    for (walk = &nodes[0]; walk != NULL; walk = walk->next)
        printf("%*s%s\n", 2 * walk->depth, "", walk->name);
    return 0;
}
```

نتيجة التنفيذ

```
Electronics
  Phones
    Android
    iOS
  Computers
```

```
    Laptops
      Gaming
      Office
Home
  Kitchen
  Garden
```

المخطط الإنسيابي (Organigramme)



تمرين 12.49 إعادة ترتيب العرض بالتناوب

تجارة إلكترونية صعب

القائمة مرتبة من الأكثر مبيعًا إلى الأقل L0, L1, ..., Ln. لتنويع الصفحة نعيد ترتيبها **بالتناوب** L0, Ln, L1, Ln-1, L2, ... في مكانها: نجد المنتصف، نقلب النصف الثاني، ثم ندمج النصفين بالتناوب.

خطوات الحل

1) المنتصف بالمؤشرين؛ نقطع بعده.
2) قلب النصف الثاني في مكانه.
3) الدمج:

```
first->next = second; second->next = nextFirst;
```

ونتقدم.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node { int rank; struct Node *next; } Node;

Node *reverse(Node *head) {
    Node *prev = NULL, *next;
    while (head != NULL) { next = head->next; head->next = prev; prev = head; head = next; }
    return prev;
}

void reorder(Node *head) {
    Node *slow = head, *fast = head, *first, *second, *nextFirst, *nextSecond;
    if (head == NULL) return;
    while (fast->next != NULL && fast->next->next != NULL) { slow = slow->next; fast = fast->next->next; }
    second = reverse(slow->next);                // النصف الثاني مقلوبًا
    slow->next = NULL;
    for (first = head; second != NULL; first = nextFirst, second = nextSecond) {
        nextFirst = first->next;
        nextSecond = second->next;
        first->next = second;                    // تناوب
        second->next = nextFirst;
    }
}

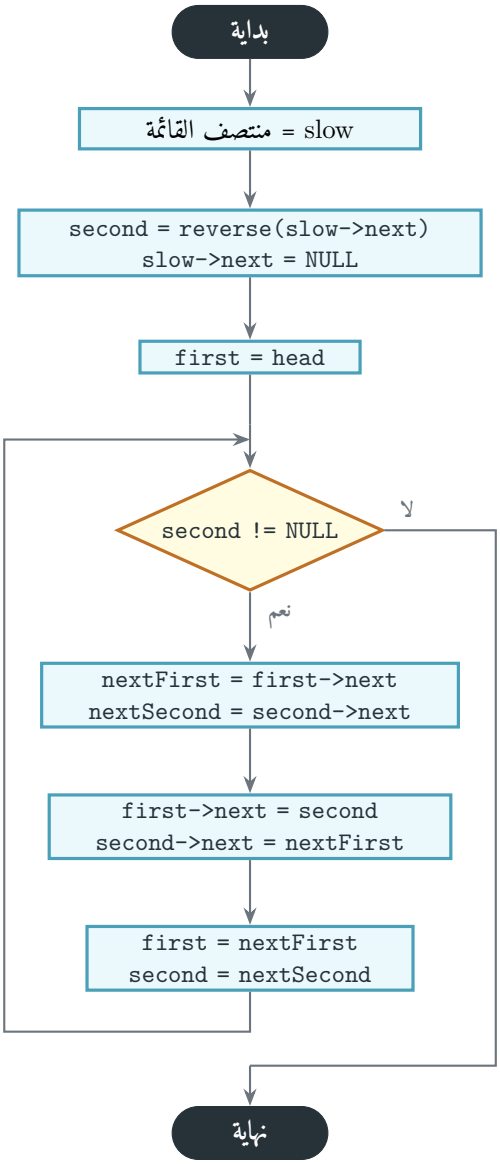
void runCase(int size) {
    Node *head = NULL, *node, *walk, *next;
    int idx;
    for (idx = size - 1; idx >= 0; idx--) { node = malloc(sizeof(Node)); node->rank = idx; node->next =
      ↪ head; head = node; }
    reorder(head);
    printf("%d items:", size);
    for (walk = head; walk != NULL; walk = walk->next) printf(" L%d", walk->rank);
    printf("\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->next; free(walk); }
}

int main(void) {
    runCase(8);
    runCase(7);
    runCase(2);
    return 0;
}
```

نتيجة التنفيذ

```
8 items: L0 L7 L1 L6 L2 L5 L3 L4
7 items: L0 L6 L1 L5 L2 L4 L3
2 items: L0 L1
```

المخطط الإنسيابي (Organigramme)



تمرين 12.50 ترتيب مهام تجهيز الطلبية طوبولوجيًا

تجارة إلكترونية

صعب

تجهيز الطلبية يتكون من مهام مترابطة (لا يمكن التغليف قبل الفحص، ولا الشحن قبل الفوترة والتغليف...). المهام عقد، والاعتماديات أسهم في **قوائم جوار**. اكتب **ترتيبًا طوبولوجيًا** بخوارزمية **كان**: نبدأ بالمهام التي لا تعتمد على شيء، ونزيل أسهمها تدريجيًا. اكشف أيضًا الاعتماديات الدائرية.

خطوات الحل

1) `inDegree[task]` = عدد المهام التي يجب أن تسبقها.
2) طابور المهام ذات `inDegree == 0`؛ عند إخراج مهمة ننقص `inDegree` لكل تابع لها.
3) إذا أخرجنا أقل من كل المهام فهناك حلقة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBT 7

typedef struct Arc { int to; struct Arc *next; } Arc;

const char *names[NBT] = {"receive", "check", "pick", "invoice", "pack", "label", "ship"};

int topoSort(Arc *adjacency[], int order[]) {
    int inDegree[NBT] = {0}, queue[NBT], front = 0, rear = 0, task, done = 0;
    Arc *arc;
    for (task = 0; task < NBT; task++)
        for (arc = adjacency[task]; arc != NULL; arc = arc->next) inDegree[arc->to]++;
    for (task = 0; task < NBT; task++) if (inDegree[task] == 0) queue[rear++] = task;
    while (front < rear) {
        task = queue[front++];
        order[done++] = task;
        for (arc = adjacency[task]; arc != NULL; arc = arc->next)
            if (--inDegree[arc->to] == 0) queue[rear++] = arc->to;    // كل ما قبلها أُنجز
    }
    return done;
}

void addArc(Arc *adjacency[], int from, int to) {
    Arc *arc = malloc(sizeof(Arc));
    arc->to = to; arc->next = adjacency[from]; adjacency[from] = arc;
}

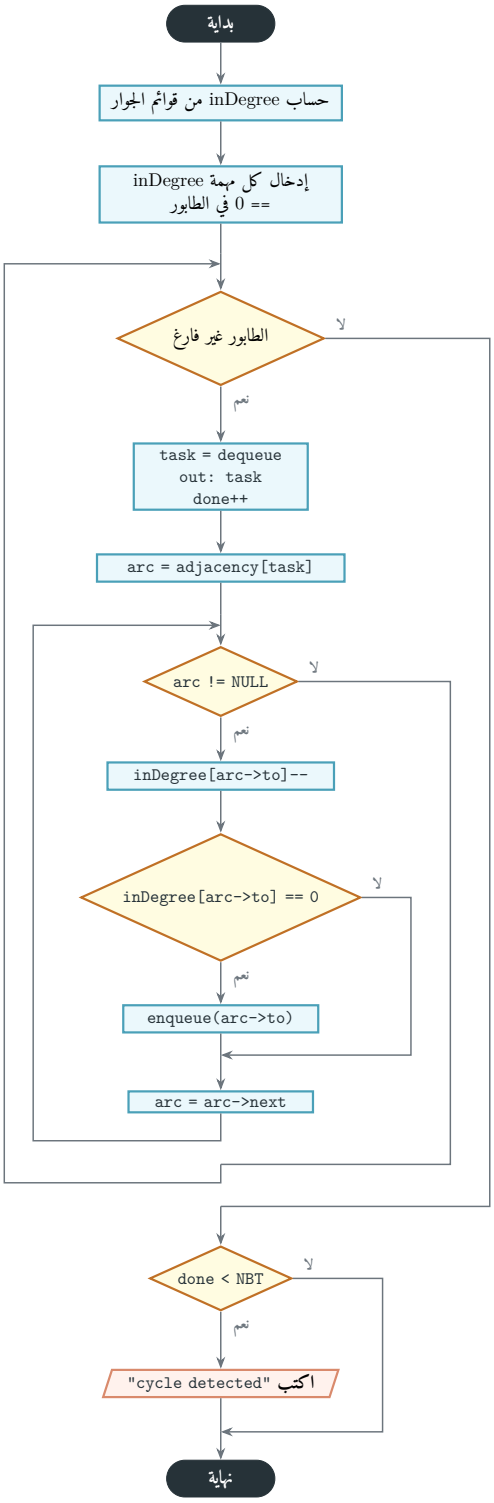
void runCase(int deps[][2], int nbDeps) {
    Arc *adjacency[NBT] = {NULL}, *arc, *next;
    int order[NBT], done, idx;
    for (idx = 0; idx < nbDeps; idx++) addArc(adjacency, deps[idx][0], deps[idx][1]);
    done = topoSort(adjacency, order);
    for (idx = 0; idx < done; idx++) printf("%s%s", names[order[idx]], idx < done - 1 ? " -> " : "\n");
    if (done < NBT) printf("cycle detected: %d task(s) blocked\n", NBT - done);
    for (idx = 0; idx < NBT; idx++)
        for (arc = adjacency[idx]; arc != NULL; arc = next) { next = arc->next; free(arc); }
}

int main(void) {
    int deps[8][2] = {{0, 1}, {1, 2}, {0, 3}, {2, 4}, {3, 5}, {4, 5}, {5, 6}, {3, 6}};
    int badDeps[8][2] = {{0, 1}, {1, 2}, {0, 3}, {2, 4}, {3, 5}, {4, 5}, {5, 6}, {6, 4}};
    runCase(deps, 8);
    runCase(badDeps, 8);
    return 0;
}
```

نتيجة التنفيذ

```
receive -> invoice -> check -> pick -> pack -> label -> ship
receive -> invoice -> check -> pick
cycle detected: 3 task(s) blocked
```

المخطط الإنسيابي (Organigramme)

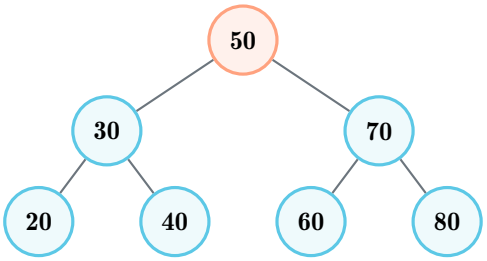


الفصل 13

الأشجار الثنائية

تعريفات

- **الشجرة الثنائية**: عقد مرتبطة، لكل عقدة **ابن أيسر** و**ابن أيمن** على الأكثر، العقدة العليا هي **الجذر** والعقد دون أبناء **أوراق**.
- **الارتفاع**: عدد العقد في أطول مسار من الجذر إلى ورقة.
- **شجرة البحث الثنائية** (BST): كل قيم الفرع الأيسر أصغر من العقدة، وكل قيم الفرع الأيمن أكبر.
- **طرق المرور**: قبلي (عقدة، يسار، يمين)، وسطي (يسار، عقدة، يمين)، بعدي (يسار، يمين، عقدة)، وبالعرض (مستوى بمستوى بطابور).



المرور الوسطي لشجرة البحث يعطي القيم مرتبة: 20 30 40 50 60 70 80

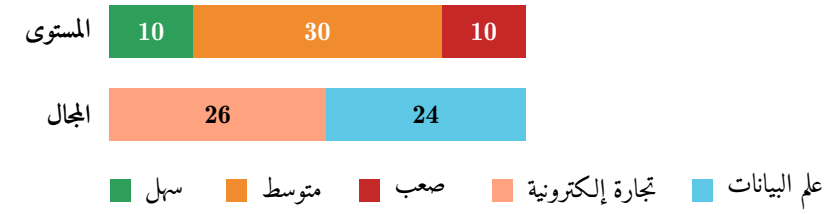
العملية في BST	التكلفة
بحث، إدراج، حذف	تتناسب مع الارتفاع: log n إذا كانت متوازنة، n إذا أصبحت سلسلة
الأصغر / الأكبر	النزول إلى أقصى اليسار / أقصى اليمين
عد أو جمع في مجال	تقليم الفروع التي لا يمكن أن تحتوي على قيم المجال
التوازن الذاتي (AVL)	دورانات بعد كل إدراج تبقي الارتفاع لوغاريتميًا

العودية طبيعية في الأشجار

كل عقدة هي جذر لشجرة فرعية، لذلك أغلب الدوال تتبع نفس النمط: حالة أساسية `node == NULL`، ثم معالجة العقدة واستدعاء الدالة على الفرعين. لتحرير الشجرة نستعمل المرور **البعدي**.

للاستزادة: [5, 11, 7, 8]

محتوى الفصل: 50 تمرينًا محلولًا



التمارين السهلة

تمرين 13.1 أول شجرة ثنائية للأقسام

تجارة إلكترونية سهل

عرّف عقدة شجرة ثنائية `Tree` (الاسم، ابن أيسر، ابن أيمن). ابنِ يدويًا شجرة أقسام صغيرة بـ `malloc`، ثم اعرضها بالترتيب القبلي (العقدة ثم اليسار ثم اليمين) مع إزاحة حسب العمق.

خطوات الحل

1) `newTree(name)` تحجز عقدة بابنين `NULL`.
2) `printPre(node, depth)`: نطبع ثم نستدعي على اليسار واليمين بعمق `+1`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char name[14]; struct Tree *left, *right; } Tree;

Tree *newTree(const char *name) {
    Tree *node = malloc(sizeof(Tree));
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
    node->left = node->right = NULL;
    return node;
}

void printPre(const Tree *node, int depth) {
    if (node == NULL) return;
    printf("%*s%s\n", 2 * depth, "", node->name);     // العقدة أولًا
    printPre(node->left, depth + 1);
    printPre(node->right, depth + 1);
}

void freeTree(Tree *node) {
    if (node == NULL) return;
    freeTree(node->left);
    freeTree(node->right);
    free(node);
}

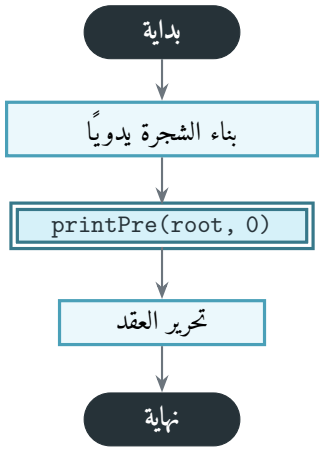
int main(void) {
    Tree *root = newTree("Store");
    root->left = newTree("Electronics");
    root->right = newTree("Home");
    root->left->left = newTree("Phones");
    root->left->right = newTree("Laptops");
    root->right->left = newTree("Kitchen");
    printPre(root, 0);
    freeTree(root);
    return 0;
```

```
38 }
```

نتيجة التنفيذ

```
Store
  Electronics
    Phones
    Laptops
  Home
    Kitchen
```

المخطط الإنسيابي (Organigramme)



تمرين 13.2 طرق المرور الثلاث

سهل علم البيانات

لنفس الشجرة، اكتب المرور **القبلي** (عقدة، يسار، يمين)، و**الوسطي** (يسار، عقدة، يمين)، و**البعدي** (يسار، يمين، عقدة)، واعرض النتائج الثلاث لتلاحظ الفرق.

خطوات الحل

1) الدوال الثلاث متطابقة ما عدا موضع `printf` بالنسبة إلى الاستدعاءين العوديين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

void preorder(const Tree *node)  { if (node) { printf("%d ", node->code); preorder(node->left);
      ↪ preorder(node->right); } }
void inorder(const Tree *node)   { if (node) { inorder(node->left); printf("%d ", node->code);
      ↪ inorder(node->right); } }
void postorder(const Tree *node) { if (node) { postorder(node->left); postorder(node->right); printf("%d ",
      ↪ node->code); } }

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

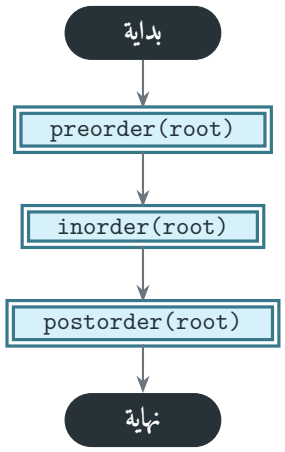
```

```c
int main(void) {
    Tree *root = newTree(50, newTree(30, newTree(20, NULL, NULL), newTree(40, NULL, NULL)),
                             newTree(70, newTree(60, NULL, NULL), newTree(80, NULL, NULL)));
    printf("preorder : "); preorder(root);  printf("\n");
    printf("inorder  : "); inorder(root);   printf("\n");
    printf("postorder: "); postorder(root); printf("\n");
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
preorder : 50 30 20 40 70 60 80
inorder  : 20 30 40 50 60 70 80
postorder: 20 40 30 60 80 70 50
```

المخطط الإنسيابي (Organigramme)



تمرين 13.3 عدد عقد الشجرة

علم البيانات سهل

اكتب دالة عودية `countNodes(node)` تعطي عدد عقد الشجرة: 0 للشجرة الفارغة، وإلا 1 + عقد اليسار + عقد اليمين. طبّقها على شجرة الأقسام وعلى كل فرع.

خطوات الحل

1) الحالة الأساسية `node == NULL`.
2) كل عقدة تُعد مرة واحدة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

int countNodes(const Tree *node) {
    if (node == NULL) return 0;
    return 1 + countNodes(node->left) + countNodes(node->right);
```

```c
}

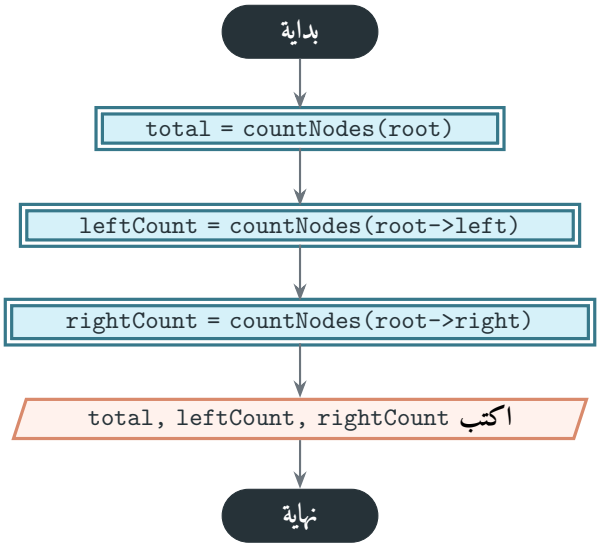
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(1, newTree(2, newTree(4, NULL, NULL), newTree(5, newTree(8, NULL, NULL), NULL)),
                            newTree(3, NULL, newTree(6, NULL, newTree(7, NULL, NULL))));
    printf("Total categories: %d\n", countNodes(root));
    printf("Left branch: %d, right branch: %d\n", countNodes(root->left), countNodes(root->right));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Total categories: 8
Left branch: 4, right branch: 3
```

المخطط الإنسيابي (Organigramme)



تمرين 13.4 ارتفاع شجرة الأقسام

علم البيانات سهل

ارتفاع الشجرة هو عدد العقد في أطول مسار من الجذر إلى ورقة (عدد مستويات القائمة المنسدلة). اكتب `height(node)` العودية = 1 + الأكبر بين ارتفاعي الفرعين.

خطوات الحل

1) الشجرة الفارغة ارتفاعها 0.
2) نحسب الارتفاعين ثم نأخذ الأكبر.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
```

```c
    return node;
}

int height(const Tree *node) {
    int leftH, rightH;
    if (node == NULL) return 0;
    leftH = height(node->left);
    rightH = height(node->right);
    return 1 + (leftH > rightH ? leftH : rightH);
}

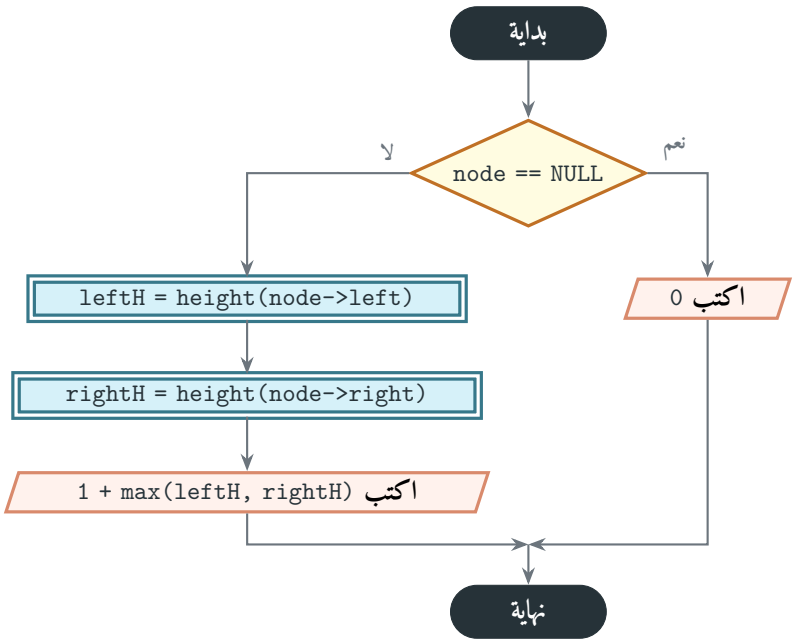
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(1, newTree(2, newTree(4, NULL, NULL), newTree(5, newTree(8, newTree(9, NULL, NULL),
      ↪ NULL), NULL)),
                           newTree(3, NULL, newTree(6, NULL, NULL)));
    printf("Menu depth (height): %d levels\n", height(root));
    printf("Left: %d, right: %d\n", height(root->left), height(root->right));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Menu depth (height): 5 levels
Left: 4, right: 2
```

المخطط الإنسيابي (Organigramme)



تمرين 13.5 مجموع الأسعار في الشجرة

تجارة إلكترونية سهل

كل عقدة في شجرة الكتالوج تحمل سعر منتج. اكتب `sumPrices(node)` تعطي مجموع أسعار كل العقد، و`maxPrice(node)` تعطي أكبر سعر (شجرة عادية، ليست شجرة بحث).

خطوات الحل

1) المجموع: سعر العقدة + مجموع اليسار + مجموع اليمين.

2) الأكبر: نقارن سعر العقدة بأكبر سعر في كل فرع.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { double price; struct Tree *left, *right; } Tree;

Tree *newTree(double price, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->price = price; node->left = left; node->right = right;
    return node;
}

double sumPrices(const Tree *node) {
    if (node == NULL) return 0;
    return node->price + sumPrices(node->left) + sumPrices(node->right);
}

double maxPrice(const Tree *node) {
    double best, side;
    if (node == NULL) return 0;
    best = node->price;
    side = maxPrice(node->left);  if (side > best) best = side;
    side = maxPrice(node->right); if (side > best) best = side;
    return best;
}

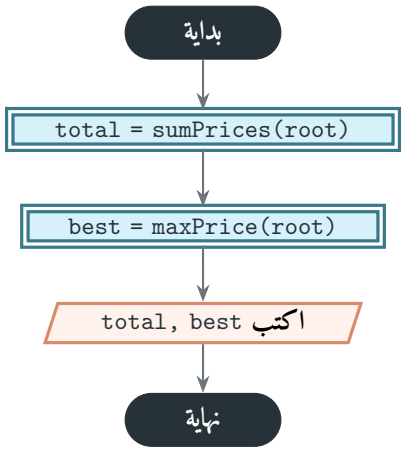
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(4500, newTree(1200, newTree(899, NULL, NULL), NULL),
                               newTree(32000, newTree(450, NULL, NULL), newTree(89000, NULL, NULL)));
    printf("Total: %.0f DA, most expensive: %.0f DA\n", sumPrices(root), maxPrice(root));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Total: 128049 DA, most expensive: 89000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 13.6 إدراج الأسعار في شجرة بحث ثنائية

تجارة إلكترونية سهل

في **شجرة البحث الثنائية** (BST): كل قيم الفرع الأيسر أصغر من العقدة، وكل قيم الأيمن أكبر. اكتب `insert(node, price)` العودية، أدرج أسعارًا غير مرتبة، ثم اعرضها **بالمرور الوسطي** لتظهر مرتبة.

خطوات الحل

1) شجرة فارغة: ننشئ العقدة ونعيدها.
2) السعر أصغر: ندرج في اليسار، وإلا في اليمين.
3) المرور الوسطي لشجرة بحث يعطي القيم مرتبة تصاعديًا.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { double price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, double price) {
    if (node == NULL) {
        node = malloc(sizeof(Tree));
        node->price = price;
        node->left = node->right = NULL;
    } else if (price < node->price) {
        node->left = insert(node->left, price);       // الأصغر يسارًا
    } else {
        node->right = insert(node->right, price);
    }
    return node;
}

void inorder(const Tree *node) {
    if (node == NULL) return;
    inorder(node->left);
    printf("%.0f ", node->price);
    inorder(node->right);
}

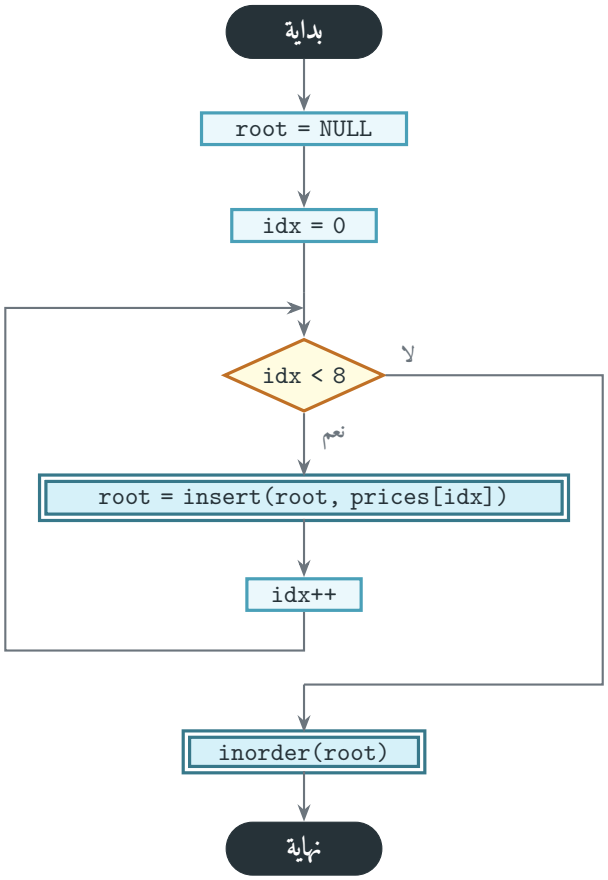
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    double prices[8] = {4500, 1200, 32000, 899, 3200, 12500, 450, 89000};
    Tree *root = NULL;
    int idx;
    for (idx = 0; idx < 8; idx++) root = insert(root, prices[idx]);
    printf("Sorted by BST: ");
    inorder(root);
    printf("\nRoot: %.0f\n", root->price);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Sorted by BST: 450 899 1200 3200 4500 12500 32000 89000
Root: 4500
```

المخطط الإنسيابي (Organigramme)



تمرين 13.7 البحث عن رمز في شجرة البحث

تجارة إلكترونية سهل

اكتب `search(node, code, &steps)` **تكرارية** (دون عودية) تبحث عن رمز منتج في شجرة بحث: في كل عقدة نذهب يساراً أو يميناً فقط. اعرض عدد العقد المزارة لكل بحث.

خطوات الحل

1) ما دام `node != NULL`: إذا ساوى الرمز نعيد العقدة، وإلا نختار الفرع.
2) عدد الخطوات لا يتجاوز ارتفاع الشجرة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int code) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->code = code; node->left = node->right = NULL; }
    else if (code < node->code) node->left = insert(node->left, code);
    else node->right = insert(node->right, code);
    return node;
}

Tree *search(Tree *node, int code, int *steps) {
    *steps = 0;
    while (node != NULL) {
        (*steps)++;
```

```c
        if (code == node->code) return node;
        node = code < node->code ? node->left : node->right;   // فرع واحد فقط
    }
    return NULL;
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int codes[9] = {5120, 2045, 7781, 1001, 3310, 6400, 9002, 1500, 4400}, idx, code, steps;
    Tree *root = NULL;
    for (idx = 0; idx < 9; idx++) root = insert(root, codes[idx]);
    for (idx = 0; idx < 2; idx++) {
        scanf("%d", &code);
        printf("Code %d: %s after %d node(s)\n", code,
               search(root, code, &steps) != NULL ? "found" : "not found", steps);
    }
    freeTree(root);
    return 0;
}
```

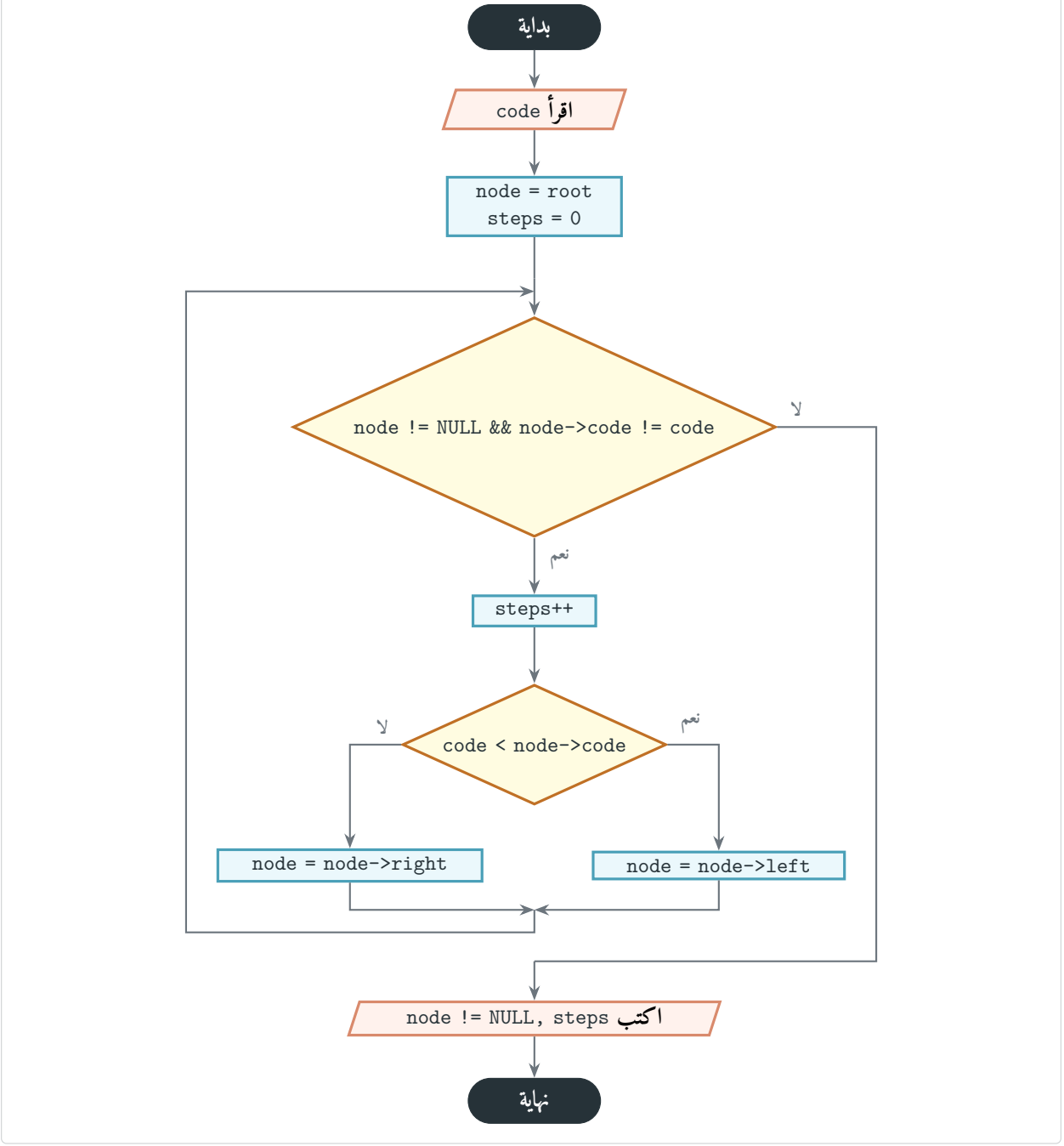
المدخلات

```
3310 7000
```

نتيجة التنفيذ

```
Code 3310: found after 2032 node(s)
Code 7000: not found after 3 node(s)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.8 أرخص وأغلى منتج في شجرة البحث

تجارة إلكترونية سهل

في شجرة البحث، أصغر قيمة هي أقصى عقدة على اليسار، وأكبر قيمة هي أقصى عقدة على اليمين. اكتب `minNode` و `maxNode` بحلقة بسيطة دون عودية.

خطوات الحل

1) `while (node->left != NULL) node = node->left;`

.

2) لا حاجة لزيارة باقي الشجرة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { double price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, double price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

Tree *minNode(Tree *node) { while (node->left != NULL) node = node->left; return node; }
Tree *maxNode(Tree *node) { while (node->right != NULL) node = node->right; return node; }

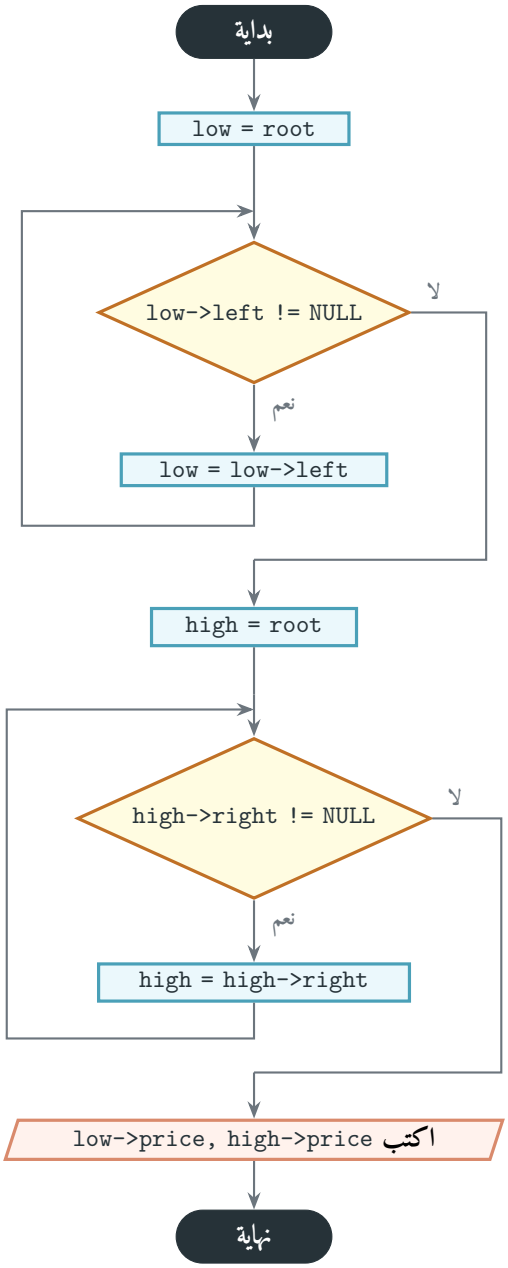
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    double prices[8] = {4500, 1200, 32000, 899, 3200, 12500, 450, 89000};
    Tree *root = NULL;
    int idx;
    for (idx = 0; idx < 8; idx++) root = insert(root, prices[idx]);
    printf("Cheapest: %.0f DA, most expensive: %.0f DA\n", minNode(root)->price, maxNode(root)->price);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Cheapest: 450 DA, most expensive: 89000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 13.9 عدد الأوراق في شجرة الأقسام

علم البيانات سهل

الورقة عقدة بلا أبناء (قسم نهائي يحتوي على منتجات). اكتب `countLeaves(node)` و `countInternal(node)` (العقد ذات ابن واحد على الأقل)، وتحقق أن مجموعهما يساوي عدد العقد.

خطوات الحل

1) ورقة: `left == NULL && right == NULL` ← 1.
2) داخلية: 1 + داخليات الفرعين، والورقة تعطي 0.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

int countLeaves(const Tree *node) {
    if (node == NULL) return 0;
    if (node->left == NULL && node->right == NULL) return 1;      // ورقة
    return countLeaves(node->left) + countLeaves(node->right);
}

int countInternal(const Tree *node) {
    if (node == NULL || (node->left == NULL && node->right == NULL)) return 0;
    return 1 + countInternal(node->left) + countInternal(node->right);
}

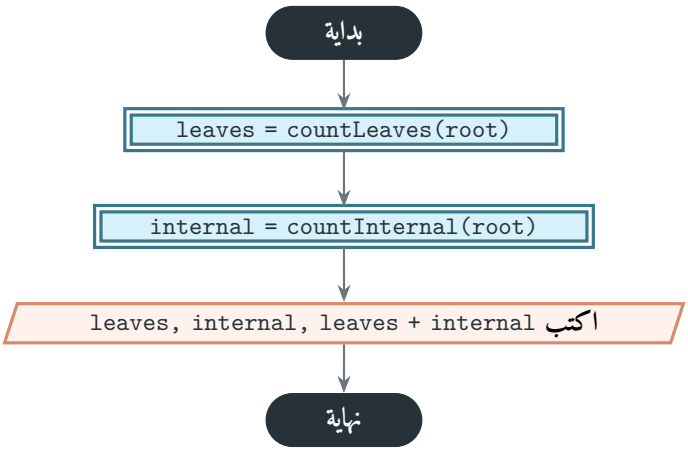
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(1, newTree(2, newTree(4, NULL, NULL), newTree(5, NULL, NULL)),
                            newTree(3, newTree(6, newTree(8, NULL, NULL), NULL), newTree(7, NULL, NULL)));
    int leaves = countLeaves(root), internal = countInternal(root);
    printf("Leaf categories: %d, internal: %d, total: %d\n", leaves, internal, leaves + internal);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Leaf categories: 4, internal: 4, total: 8
```

المخطط الإنسيابي (Organigramme)



تمرين 13.10 تحرير الشجرة بالمرور البعدي

تجارة إلكترونية سهل

اكتب `freeTree(node)` التي تحرر كل العقد وتعد المحررة. لماذا يجب أن يكون المرور **بعديًا** (الأبناء قبل الأب)؟

خطوات الحل

1) إذا حررنا الأب أولًا لا يمكننا الوصول بعدها إلى `node->left` و `node->right`.

2) المرور البعدي يحرر الأبناء ثم الأب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

int freed = 0;

Tree *insert(Tree *node, int code) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->code = code; node->left = node->right = NULL; }
    else if (code < node->code) node->left = insert(node->left, code);
    else node->right = insert(node->right, code);
    return node;
}

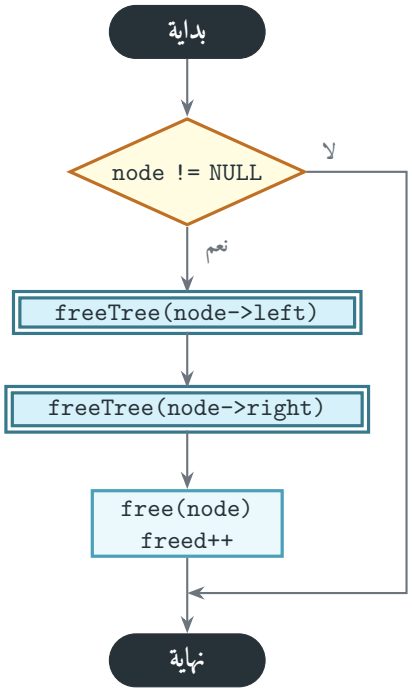
void freeTree(Tree *node) {
    if (node == NULL) return;
    freeTree(node->left);                // الأبناء أولًا
    freeTree(node->right);
    printf("free %d\n", node->code);
    free(node);                          // ثم الأب
    freed++;
}

int main(void) {
    int codes[6] = {50, 30, 70, 20, 40, 80}, idx;
    Tree *root = NULL;
    for (idx = 0; idx < 6; idx++) root = insert(root, codes[idx]);
    freeTree(root);
    root = NULL;
    printf("%d nodes freed\n", freed);
    return 0;
}
```

نتيجة التنفيذ

```
free 20
free 40
free 30
free 80
free 70
free 50
6 nodes freed
```

المخطط الإنسيابي (Organigramme)



التمارين المتوسطة

تمرين 13.11 حذف منتج من شجرة البحث

تجارة إلكترونية ★★ متوسط

اكتب `removeKey(node, price)` التي تحذف سعراً من شجرة البحث مع المحافظة على خاصيتها. ثلاث حالات: (1) ورقة: نحذفها؛ (2) ابن واحد: نعوضها بابنها؛ (3) ابنان: ننسخ فيها **خلفها** (أصغر قيمة في الفرع الأيمن) ثم نحذف الخلف من الفرع الأيمن.

خطوات الحل

1) البحث العودي حتى العقدة، ثم معالجة الحالات الثلاث.

2) الدالة تعيد جذر الفرع بعد الحذف ليُربط بالأب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

Tree *removeKey(Tree *node, int price) {
    Tree *child, *succ;
    if (node == NULL) return NULL;
    if (price < node->price) node->left = removeKey(node->left, price);
    else if (price > node->price) node->right = removeKey(node->right, price);
    else if (node->left == NULL || node->right == NULL) {      // 0 أو 1 ابن
        child = node->left != NULL ? node->left : node->right;
        free(node);
        return child;
    } else {                                                   // ابنان
        for (succ = node->right; succ->left != NULL; succ = succ->left) {}
        node->price = succ->price;                             // نسخ الخلف
        node->right = removeKey(node->right, succ->price);
    }
    return node;
}

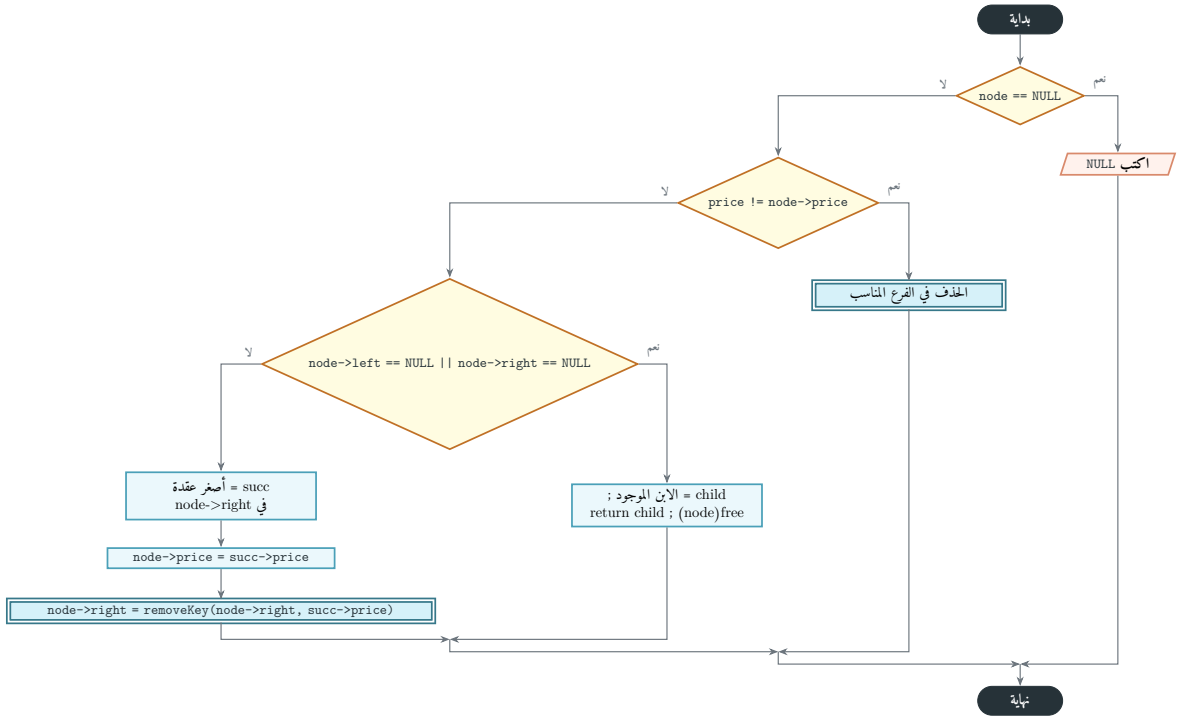
void inorder(const Tree *node) { if (node) { inorder(node->left); printf("%d ", node->price);
    ↪ inorder(node->right); } }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[9] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 2000, 15000}, idx;
    Tree *root = NULL;
    for (idx = 0; idx < 9; idx++) root = insert(root, prices[idx]);
    printf("start:          "); inorder(root); printf("\n");
    root = removeKey(root, 899);   printf("remove leaf 899: "); inorder(root); printf("\n");
    root = removeKey(root, 12500); printf("remove 1-child:  "); inorder(root); printf("\n");
    root = removeKey(root, 4500);  printf("remove root:     "); inorder(root); printf("\n");
    printf("new root: %d\n", root->price);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
start:          899 1200 2000 3200 4500 12500 15000 32000 89000
remove leaf 899: 1200 2000 3200 4500 12500 15000 32000 89000
remove 1-child:  1200 2000 3200 4500 15000 32000 89000
remove root:     1200 2000 3200 15000 32000 89000
new root: 15000
```

المخطط الإنسيابي (Organigramme)



تمرين 13.12 المرور بالعرض مستوى بمستوى

علم البيانات ★★ متوسط

اعرض شجرة الأقسام **مستوى بمستوى** (المرور بالعرض) باستعمال طابور من مؤشرات العقد، مع طباعة كل مستوى في سطر منفصل وعدد عقده.

خطوات الحل

1) نضع الجذر في الطابور.

2) لكل مستوى: نعرف عدد العقد الحالية في الطابور `levelSize`، ونخرجها كلها مع إدخال أبنائها.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char name[12]; struct Tree *left, *right; } Tree;

Tree *newTree(const char *name, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
    node->left = left; node->right = right;
    return node;
}
```

```c

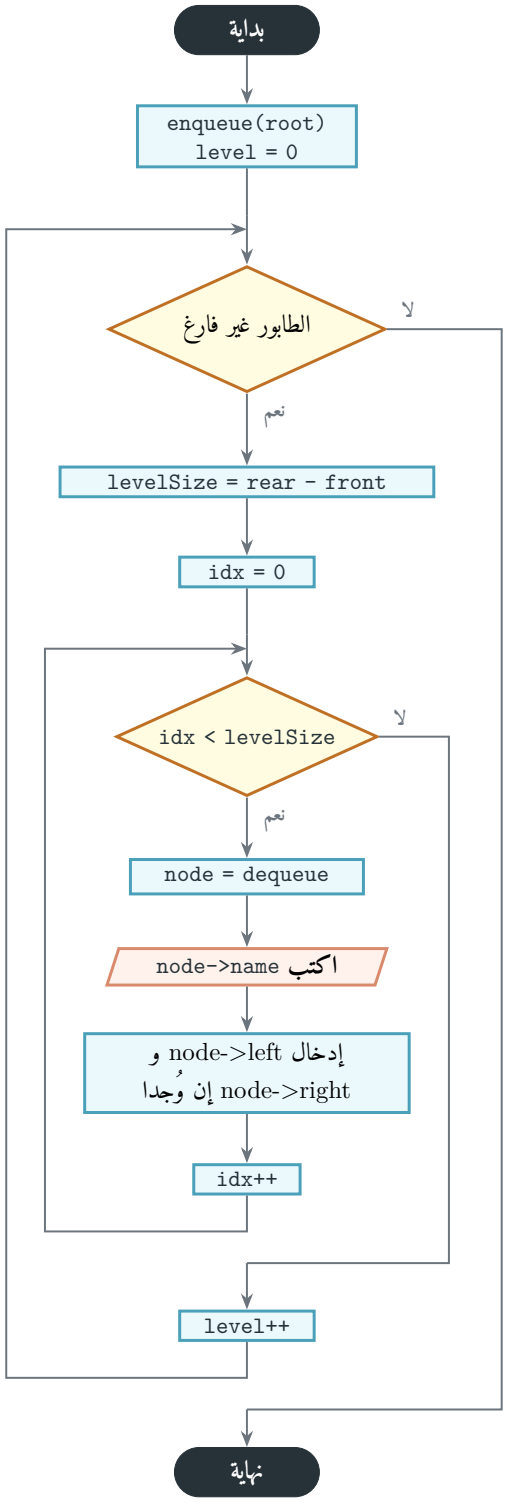
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree("Store",
        newTree("Tech", newTree("Phones", newTree("Android", NULL, NULL), NULL), newTree("Laptops", NULL, NULL)),
        newTree("Home", newTree("Kitchen", NULL, NULL), newTree("Garden", NULL, newTree("Tools", NULL, NULL))));
    Tree *queue[32], *node;
    int front = 0, rear = 0, levelSize, idx, level = 0;
    queue[rear++] = root;
    while (front < rear) {
        levelSize = rear - front;                    // عقد المستوى الحالي
        printf("level %d (%d):", level, levelSize);
        for (idx = 0; idx < levelSize; idx++) {
            node = queue[front++];
            printf(" %s", node->name);
            if (node->left) queue[rear++] = node->left;
            if (node->right) queue[rear++] = node->right;
        }
        printf("\n");
        level++;
    }
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
level 0 (1): Store
level 1 (2): Tech Home
level 2 (4): Phones Laptops Kitchen Garden
level 3 (2): Android Tools
```

المخطط الإنسيابي (Organigramme)



تمرين 13.13 رسم الشجرة أفقيًا

علم البيانات ★★ متوسط

اكتب `printSideways(node, depth)` تعرض شجرة البحث مدورة **90 درجة**: الفرع الأيمن في الأعلى ثم العقدة ثم الأيسر، مع إزاحة 6 فراغات لكل مستوى. هكذا نرى بنية شجرة أسعار مبنية من مدخلات.

خطوات الحل

1) مرور وسطي معكوس (يمين، عقدة، يسار).
2) الإزاحة `6 * depth` بـ `%*s`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

void printSideways(const Tree *node, int depth) {
    if (node == NULL) return;
    printSideways(node->right, depth + 1);            // الأكبر في الأعلى
    printf("%*s%d\n", 6 * depth, "", node->price);
    printSideways(node->left, depth + 1);
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = NULL;
    int count, idx, price;
    scanf("%d", &count);
    for (idx = 0; idx < count; idx++) { scanf("%d", &price); root = insert(root, price); }
    printSideways(root, 0);
    freeTree(root);
    return 0;
}
```

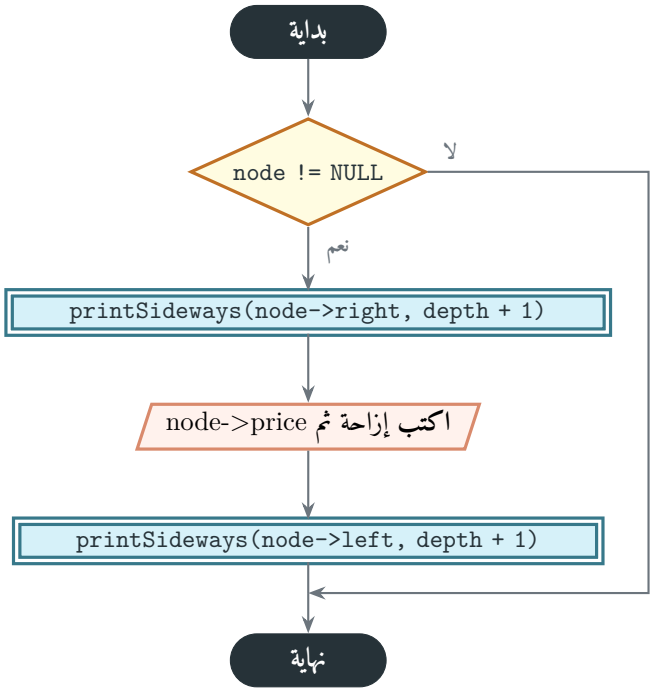
المدخلات

```
8
50 30 70 20 40 60 80 35
```

نتيجة التنفيذ

```
            80
        70
            60
50
            40
                35
        30
            20
```

المخطط الانسيابي (Organigramme)



تمرين 13.14 عدد المنتجات في مجال سعري

تجارة إلكترونية ★★ متوسط

مرشح السعر في الموقع: كم منتجًا سعره بين `low` و `high`؟ اكتب `countRange(node, low, high)` تستغل خاصية شجرة البحث لـ تقليم الفروع: لا ننزل يسارًا إذا كان سعر العقدة أصغر من `low`، ولا يمينًا إذا كان أكبر من `high`. عدّ العقد المزارة.

خطوات الحل

1) إذا `node->price < low`: كل الفرع الأيسر أصغر أيضًا، نذهب يمينًا فقط.
2) إذا `node->price > high`: نذهب يسارًا فقط.
3) وإلا: العقدة داخل المجال، ونستكشف الفرعين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

int visited = 0;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

int countRange(const Tree *node, int low, int high) {
    if (node == NULL) return 0;
    visited++;
    if (node->price < low) return countRange(node->right, low, high);    // تقليم اليسار
    if (node->price > high) return countRange(node->left, low, high);    // تقليم اليمين
```

```c
    return 1 + countRange(node->left, low, high) + countRange(node->right, low, high);
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[15] = {8000, 3000, 20000, 1200, 5000, 12000, 45000, 450, 2000, 4100, 6500,
                      9900, 15000, 32000, 89000};
    Tree *root = NULL;
    int idx, low, high, count;
    for (idx = 0; idx < 15; idx++) root = insert(root, prices[idx]);
    scanf("%d %d", &low, &high);
    count = countRange(root, low, high);
    printf("%d products between %d and %d DA (visited %d of 15 nodes)\n", count, low, high, visited);
    freeTree(root);
    return 0;
}
```

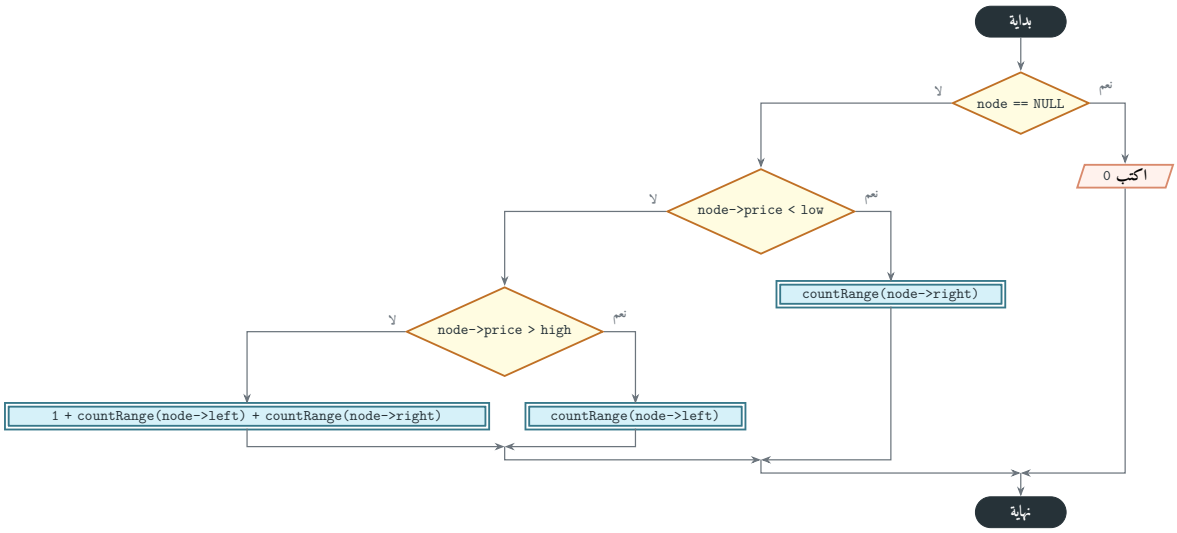
المدخلات

```
1000 10000
```

نتيجة التنفيذ

```
8 products between 1000 and 10000 DA (visited 11 of 15 nodes)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.15 السعر رقم k في الترتيب

علم البيانات ★★ متوسط

اكتب `kthSmallest(node, k)` تجد القيمة رقم `k` في الترتيب التصاعدي بالمرور الوسطي مع **عداد** يتوقف فور الوصول إلى `k` (دون زيارة باقي الشجرة). استعملها لإيجاد الوسيط والربيعين لأسعار السوق.

خطوات الحل

1) عداد عام يزداد عند زيارة كل عقدة في المرور الوسطي.
2) عندما يساوي `k` نحفظ القيمة ونوقف البحث (علم `found`).
3) الوسيط = العنصر رقم `(n + 1) / 2`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

int counter, found, result;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

void visit(const Tree *node, int rank) {
    if (node == NULL || found) return;
    visit(node->left, rank);
    if (found) return;
    if (++counter == rank) { result = node->price; found = 1; return; }   // توقف مبكر
    visit(node->right, rank);
}

int kthSmallest(const Tree *root, int rank) {
    counter = 0;
    found = 0;
    visit(root, rank);
    return result;
}

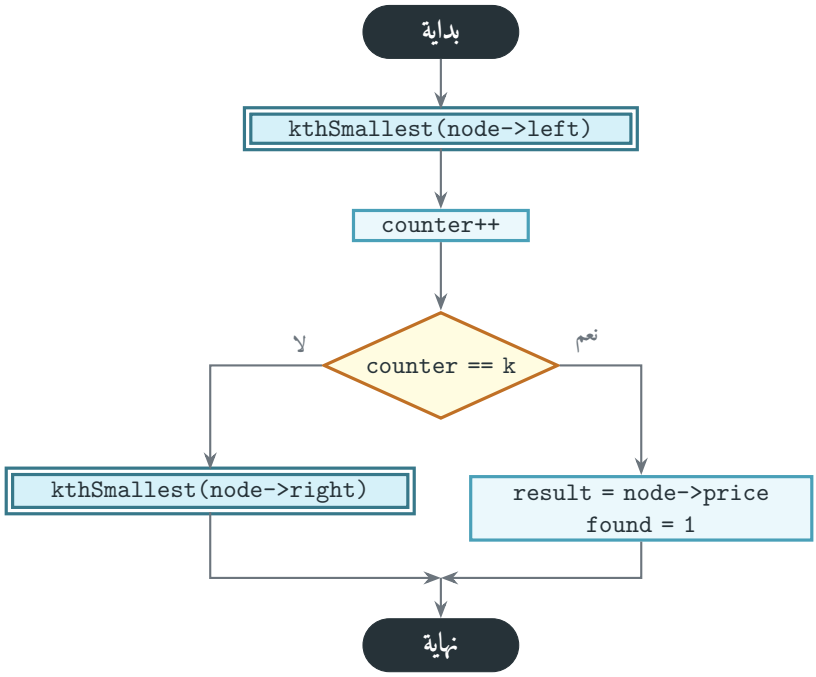
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[11] = {32000, 29500, 31000, 89000, 30500, 28900, 33000, 27500, 35000, 30000, 29900};
    Tree *root = NULL;
    int idx;
    for (idx = 0; idx < 11; idx++) root = insert(root, prices[idx]);
    printf("Q1 (3rd):     %d\n", kthSmallest(root, 3));
    printf("Median (6th): %d\n", kthSmallest(root, 6));
    printf("Q3 (9th):     %d\n", kthSmallest(root, 9));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Q1 (3rd):     29500
Median (6th): 30500
Q3 (9th):     33000
```

المخطط الإنسيابي (Organigramme)



تمرين 13.16 أغلى منتج ضمن الميزانية

تجارة إلكترونية ★★ متوسط

يعطي الزبون ميزانية، ونقترح **أغلى منتج لا يتجاوزها** (الأرضية) و**أرخص منتج يتجاوزها** (السقف). اكتب `floorPrice` و `ceilPrice` في شجرة البحث بحلقة واحدة لكل منهما.

خطوات الحل

1) الأرضية: إذا كان سعر العقدة ≤ الميزانية فهو مرشح ونذهب يمينًا (قد نجد أقرب)، وإلا يسارًا.
2) السقف: العكس.
3) -1 إذا لم يوجد.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

int floorPrice(const Tree *node, int budget) {
    int best = -1;
    while (node != NULL) {
        if (node->price <= budget) { best = node->price; node = node->right; }   // مرشح، نبحث عن أقرب
        else node = node->left;
    }
    return best;
}

```

```c
int ceilPrice(const Tree *node, int budget) {
    int best = -1;
    while (node != NULL) {
        if (node->price >= budget) { best = node->price; node = node->left; }
        else node = node->right;
    }
    return best;
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[9] = {4500, 1200, 32000, 899, 3200, 12500, 450, 89000, 6800}, idx, count, budget;
    Tree *root = NULL;
    for (idx = 0; idx < 9; idx++) root = insert(root, prices[idx]);
    scanf("%d", &count);
    for (idx = 0; idx < count; idx++) {
        scanf("%d", &budget);
        printf("budget %6d: best under %6d, next above %6d\n", budget,
               floorPrice(root, budget), ceilPrice(root, budget));
    }
    freeTree(root);
    return 0;
}
```

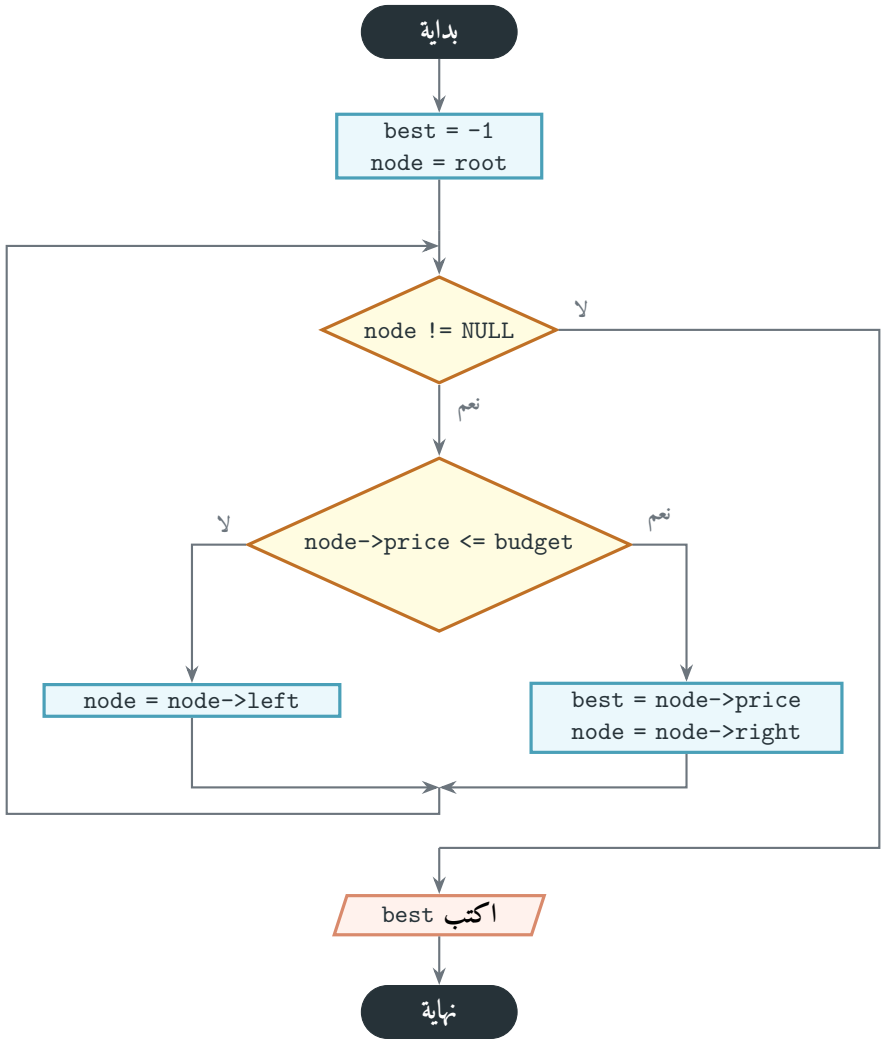
المدخلات

```
3 5000 450 100000
```

نتيجة التنفيذ

```
budget   5000: best under   4500, next above   6800
budget    450: best under    450, next above    450
budget 100000: best under  89000, next above     -1
```

المخطط الإنسيابي (Organigramme)



تمرين 13.17 التحقق من صحة شجرة البحث

علم البيانات ★★ متوسط

بعد تعديل يدوي على الفهرس، تحقق أن الشجرة ما زالت **شجرة بحث صحيحة**. الاختبار المحلي (الأب أكبر من ابنه الأيسر فقط) غير كافٍ. اكتب `isValid(node, low, high)` التي تمرر **المجال المسموح** لكل فرع.

خطوات الحل

1) الجذر مسموح في $(-\infty, +\infty)$.

2) الفرع الأيسر لعقدة قيمتها `v` مسموح في `(low, v)`، والأيمن في `(v, high)`.

3) مثال: في الشجرة الثانية العقدة 35 أكبر من أبيها 20 (فينجح الاختبار المحلي)، لكنها في الفرع الأيسر لـ 30 فيجب أن تكون أصغر من 30.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

```

```c
Tree *newTree(int price, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->price = price; node->left = left; node->right = right;
    return node;
}

int isValid(const Tree *node, long low, long high) {
    if (node == NULL) return 1;
    if (node->price <= low || node->price >= high) return 0;      // خارج المجال المسموح
    return isValid(node->left, low, node->price) && isValid(node->right, node->price, high);
}

int localCheck(const Tree *node) {                                // اختبار ناقص
    if (node == NULL) return 1;
    if (node->left && node->left->price > node->price) return 0;
    if (node->right && node->right->price < node->price) return 0;
    return localCheck(node->left) && localCheck(node->right);
}

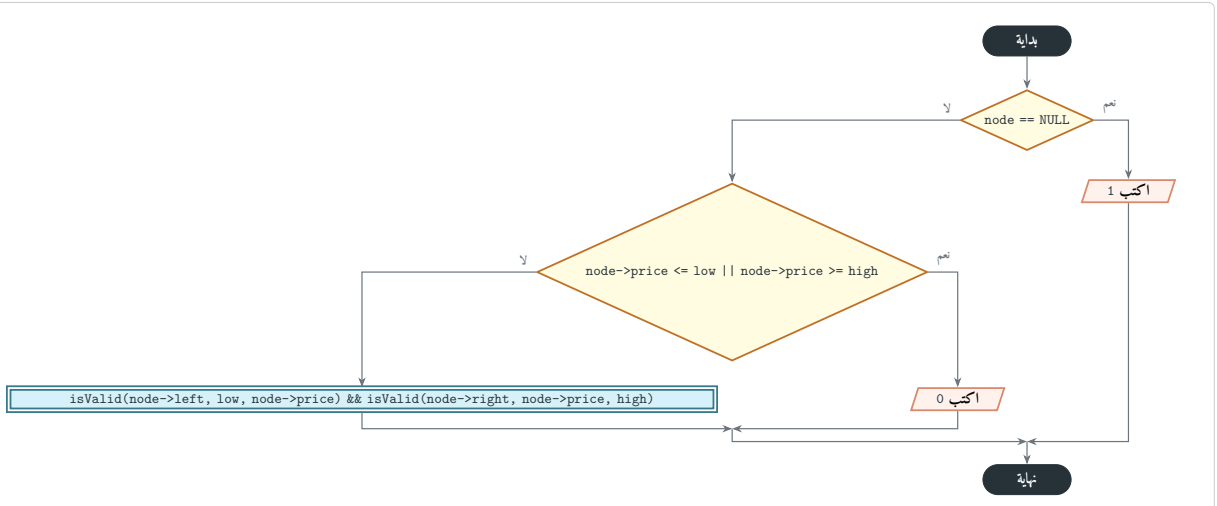
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *good = newTree(30, newTree(20, newTree(10, NULL, NULL), newTree(25, NULL, NULL)), newTree(40,
      ↪ NULL, NULL));
    Tree *bad = newTree(30, newTree(20, newTree(10, NULL, NULL), newTree(35, NULL, NULL)), newTree(40, NULL,
      ↪ NULL));
    printf("tree 1: local %s, full %s\n", localCheck(good) ? "ok" : "bad", isValid(good, -1000000, 1000000)
      ↪ ? "VALID" : "INVALID");
    printf("tree 2: local %s, full %s\n", localCheck(bad) ? "ok" : "bad", isValid(bad, -1000000, 1000000) ?
      ↪ "VALID" : "INVALID");
    freeTree(good);
    freeTree(bad);
    return 0;
}
```

نتيجة التنفيذ

```
tree 1: local ok, full VALID
tree 2: local ok, full INVALID
```

المخطط الإنسيابي (Organigramme)



تمرين 13.18 أقرب سلف مشترك لسحرين

علم البيانات ★★ متوسط

في شجرة البحث، **أقرب سلف مشترك** (LCA) لقيمتين هو أول عقدة تقع قيمتها بينهما عند النزول من الجذر. اكتب `lowestCommon(root, first, second)` بحلقة، واستعملها لمعرفة "نقطة الانقسام" بين منتجين في فهرس الأسعار،

مع المسافة بينهما (عدد الروابط).

خطوات الحل

1) إذا كانت القيمتان أصغر من العقدة: نذهب يسارًا، أكبر: يمينًا، وإلا فالعقدة هي الجواب.
2) المسافة = عمق الأولى + عمق الثانية - 2 × عمق السلف.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

const Tree *lowestCommon(const Tree *node, int first, int second) {
    while (node != NULL) {
        if (first < node->price && second < node->price) node = node->left;
        else if (first > node->price && second > node->price) node = node->right;
        else return node;                                   // تنفصل القيمتان هنا
    }
    return NULL;
}

int depthOf(const Tree *node, int price) {
    int depth = 0;
    while (node != NULL && node->price != price) { node = price < node->price ? node->left : node->right;
      ↪ depth++; }
    return depth;
}

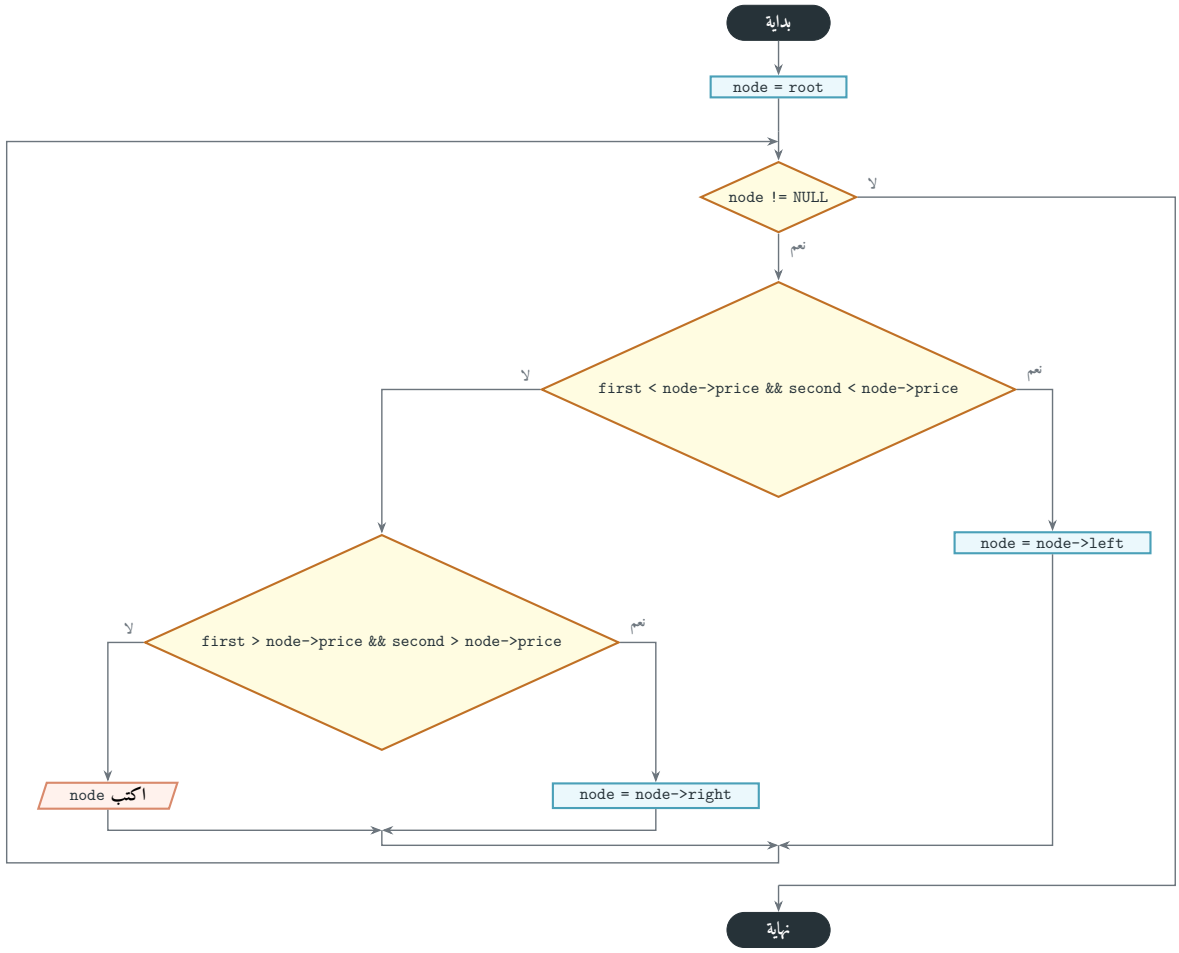
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[11] = {8000, 3000, 20000, 1200, 5000, 12000, 45000, 450, 2000, 4100, 15000};
    int pairs[3][2] = {{450, 2000}, {4100, 15000}, {12000, 15000}}, idx;
    Tree *root = NULL;
    const Tree *common;
    for (idx = 0; idx < 11; idx++) root = insert(root, prices[idx]);
    for (idx = 0; idx < 3; idx++) {
        common = lowestCommon(root, pairs[idx][0], pairs[idx][1]);
        printf("LCA(%d, %d) = %d, distance %d\n", pairs[idx][0], pairs[idx][1], common->price,
               depthOf(root, pairs[idx][0]) + depthOf(root, pairs[idx][1]) - 2 * depthOf(root,
      ↪ common->price));
    }
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
LCA(450, 2000) = 1200, distance 2
LCA(4100, 15000) = 8000, distance 6
LCA(12000, 15000) = 12000, distance 1
```

المخطط الانسيابي (Organigramme)



تمرين 13.19 الشجرة المرآة

علم البيانات ★★ متوسط

اكتب `mirror(node)` تقلب الشجرة في مكانها بتبديل الابنين في كل عقدة. طبّقها على شجرة بحث وبيّن أن المرور الوسطي يصبح تنازليًا، ثم أعدها كما كانت بنفس الدالة.

خطوات الحل

1) نبدل `left` و `right` ثم نستدعي على الفرعين.
2) المرآة مرتين تعيد الأصل.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

```

```c
void mirror(Tree *node) {
    Tree *tmp;
    if (node == NULL) return;
    tmp = node->left;                // تبديل الابنين
    node->left = node->right;
    node->right = tmp;
    mirror(node->left);
    mirror(node->right);
}

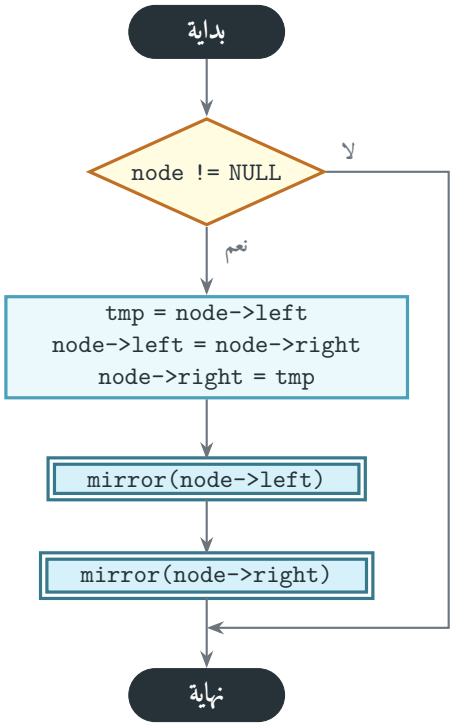
void inorder(const Tree *node) { if (node) { inorder(node->left); printf("%d ", node->price);
    ↪ inorder(node->right); } }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[7] = {4500, 1200, 32000, 899, 3200, 12500, 89000}, idx;
    Tree *root = NULL;
    for (idx = 0; idx < 7; idx++) root = insert(root, prices[idx]);
    printf("original: "); inorder(root); printf("\n");
    mirror(root);
    printf("mirror:   "); inorder(root); printf("\n");
    mirror(root);
    printf("back:     "); inorder(root); printf("\n");
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
original: 899 1200 3200 4500 12500 32000 89000
mirror:   89000 32000 12500 4500 3200 1200 899
back:     899 1200 3200 4500 12500 32000 89000
```

المخطط الإنسيابي (Organigramme)



تمرين 13.20 هل الشجرة متوازنة

علم البيانات ★★ متوسط

شجرة البحث المبنية من أسعار مدخلة مرتبة تصبح سلسلة طويلة وبطيئة. اكتب `checkBalance(node)` التي تعيد الارتفاع إذا كانت الشجرة متوازنة (فرق ارتفاعي الفرعين ∈ 1 في كل عقدة) و -1 وإلا، بمرور واحد. قارن شجرة من مدخلات مخلوطة وأخرى من مدخلات مرتبة.

خطوات الحل

1) نحسب ارتفاع الفرعين عوديًا، إذا كان أحدهما -1 ننشر -1.
2) إذا تجاوز الفرق 1 نعيد -1، وإلا `1 + max`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

int checkBalance(const Tree *node) {
    int leftH, rightH, diff;
    if (node == NULL) return 0;
    leftH = checkBalance(node->left);
    rightH = checkBalance(node->right);
    diff = leftH - rightH;
    if (leftH < 0 || rightH < 0 || diff > 1 || diff < -1) return -1;   // غير متوازنة
    return 1 + (leftH > rightH ? leftH : rightH);
}

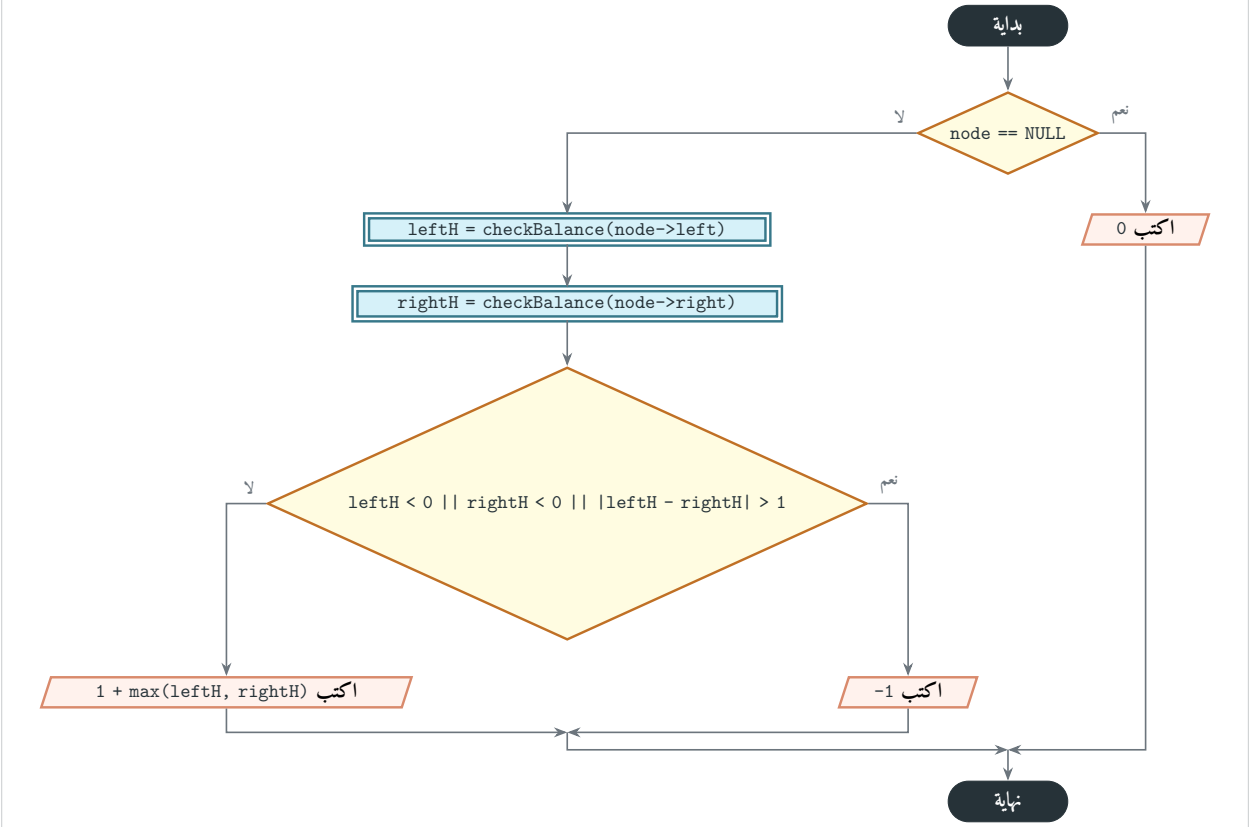
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int mixed[7] = {4000, 2000, 6000, 1000, 3000, 5000, 7000};
    int sorted[7] = {1000, 2000, 3000, 4000, 5000, 6000, 7000};
    Tree *first = NULL, *second = NULL;
    int idx, result;
    for (idx = 0; idx < 7; idx++) { first = insert(first, mixed[idx]); second = insert(second, sorted[idx]);
      ↪ }
    result = checkBalance(first);
    printf("mixed input:  %s (height %d)\n", result >= 0 ? "balanced" : "unbalanced", result);
    result = checkBalance(second);
    printf("sorted input: %s\n", result >= 0 ? "balanced" : "unbalanced (a chain of 7 nodes)");
    freeTree(first);
    freeTree(second);
    return 0;
}
```

نتيجة التنفيذ

```
mixed input:  balanced (height 3)
sorted input: unbalanced (a chain of 7 nodes)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.21 مسار حزمة بمجموع محدد

تجارة إلكترونية ★★ متوسط

شجرة اقتراحات المنتجات: من كل منتج يمكن الانتقال إلى أحد اقتراحين. هل يوجد **مسار من الجذر إلى ورقة** مجموع أسعاره يساوي ميزانية الحزمة بالضبط؟ اكتب `hasPathSum(node, remaining, path, depth)` التي تعرض المسار إن وُجد.

خطوات الحل

1) ننقص سعر العقدة من المتبقي ونحفظها في `path[depth]`.
2) في ورقة: نجاح إذا أصبح المتبقي 0.
3) نجرب اليسار ثم اليمين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *newTree(int price, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->price = price; node->left = left; node->right = right;
    return node;
}

int path[20];

int hasPathSum(const Tree *node, int remaining, int depth) {
    int idx;
```

```c
    if (node == NULL) return 0;
    path[depth] = node->price;
    remaining -= node->price;
    if (node->left == NULL && node->right == NULL) {           // ورقة
        if (remaining != 0) return 0;
        printf("Bundle:");
        for (idx = 0; idx <= depth; idx++) printf(" %d", path[idx]);
        printf("\n");
        return 1;
    }
    return hasPathSum(node->left, remaining, depth + 1) || hasPathSum(node->right, remaining, depth + 1);
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(5000, newTree(3200, newTree(1500, NULL, NULL), newTree(450, NULL, NULL)),
                               newTree(4100, newTree(600, NULL, NULL), newTree(2200, newTree(899, NULL,
      ↪ NULL), NULL)));
    int budget;
    scanf("%d", &budget);
    if (!hasPathSum(root, budget, 0)) printf("No bundle for %d DA\n", budget);
    if (!hasPathSum(root, 12000, 0)) printf("No bundle for 12000 DA\n");
    freeTree(root);
    return 0;
}
```

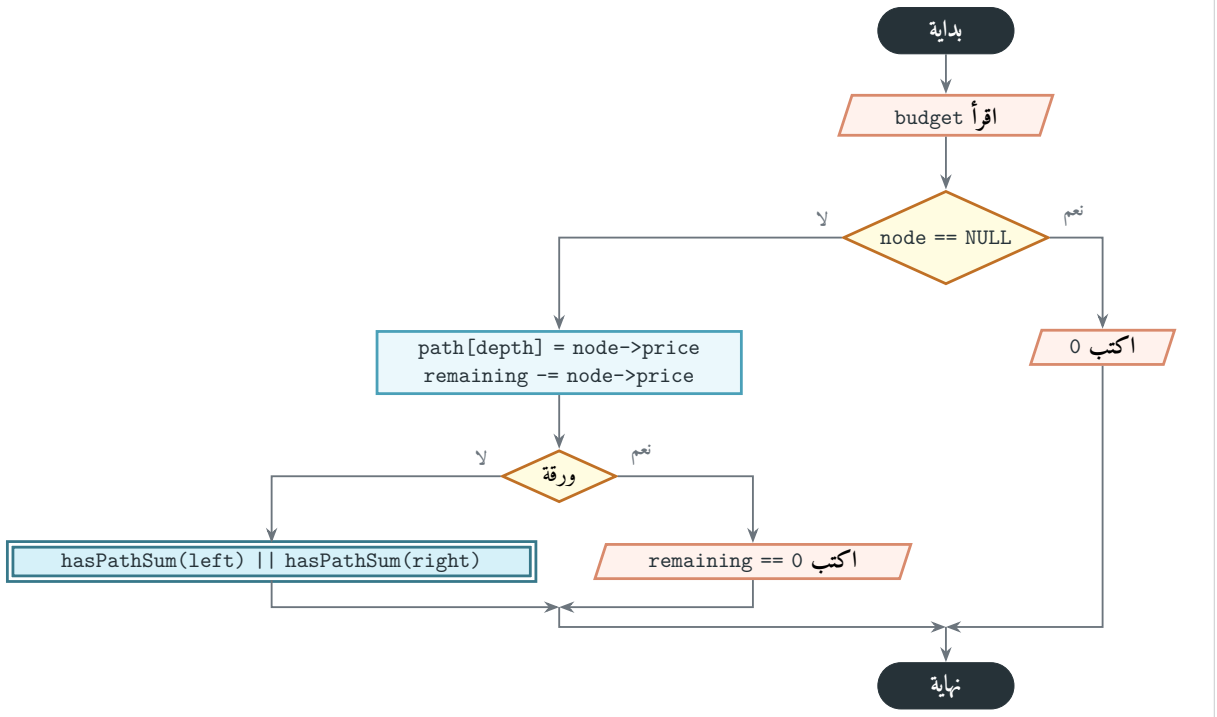
المدخلات

```
9700
```

نتيجة التنفيذ

```
Bundle: 5000 3200 1500
No bundle for 12000 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 13.22 كل قواعد شجرة القرار

علم البيانات
متوسط

شجرة قرار التخفيض: كل عقدة داخلية سؤال، وكل ورقة قرار. اكتب `printRules(node, path, depth)` التي تعرض **كل المسارات** من الجذر إلى الأوراق كقواعد "إذا ... و ... إذن ...". العقدة تحمل نصًا، والابن الأيسر = "نعم"، والأيمن = "لا".

خطوات الحل

1) نخزن في `path[depth]` نص السؤال مع الإجابة (نعم/لا) قبل النزول.
2) في الورقة نطبع كل الشروط ثم القرار.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { const char *text; struct Tree *left, *right; } Tree;

Tree *newTree(const char *text, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->text = text; node->left = left; node->right = right;
    return node;
}

const Tree *steps[10];
char answers[10];
int ruleCount = 0;

void printRules(const Tree *node, int depth) {
    int idx;
    if (node->left == NULL && node->right == NULL) {          // ورقة: قرار
        printf("R%d: IF", ++ruleCount);
        for (idx = 0; idx < depth; idx++)
            printf("%s %s%s", idx ? " AND" : "", answers[idx] == 'N' ? "NOT " : "", steps[idx]->text);
        printf(" THEN %s\n", node->text);
        return;
    }
    steps[depth] = node;
    answers[depth] = 'Y';
    printRules(node->left, depth + 1);
    answers[depth] = 'N';
    printRules(node->right, depth + 1);
}

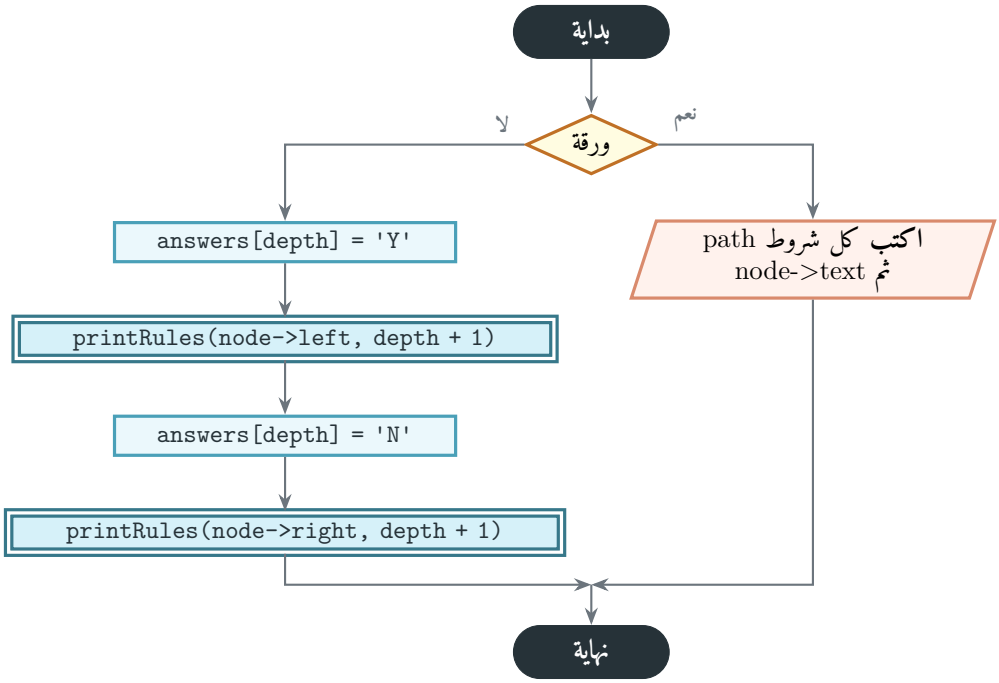
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree("loyal",
        newTree("cart>10000", newTree("15% off", NULL, NULL), newTree("5% off", NULL, NULL)),
        newTree("first_order", newTree("free shipping", NULL, NULL),
                newTree("cart>20000", newTree("5% off", NULL, NULL), newTree("no discount", NULL, NULL))));
    printRules(root, 0);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
R1: IF loyal AND cart>10000 THEN 15% off
R2: IF loyal AND NOT cart>10000 THEN 5% off
R3: IF NOT loyal AND first_order THEN free shipping
R4: IF NOT loyal AND NOT first_order AND cart>20000 THEN 5% off
R5: IF NOT loyal AND NOT first_order AND NOT cart>20000 THEN no discount
```

المخطط الإنسيابي (Organigramme)



تمرين 13.23 شجرة متوازنة من قائمة مرتبة

تجارة إلكترونية ★★ متوسط

لدينا كتالوج **مرتب** حسب الرمز. الإدراج المتتالي يعطي سلسلة. اكتب `buildBalanced(codes, low, high)` التي تجعل العنصر الأوسط جذرًا، وتبني الفرعين عوديًا من النصفين، فتعطي شجرة **بأقل ارتفاع ممكن**. قارن الارتفاع بالبناء بالإدراج.

خطوات الحل

1) `middle = (low + high) / 2` يصبح الجذر.
2) اليسار من `[low, middle-1]` واليمين من `[middle+1, high]`.
3) الارتفاع `⌊log2(n)⌋ + 1`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = node->right = NULL;
    return node;
}

Tree *buildBalanced(const int *codes, int low, int high) {
    Tree *node;
    int middle;
    if (low > high) return NULL;
    middle = (low + high) / 2;
    node = newTree(codes[middle]);                    // الأوسط جذرًا
    node->left = buildBalanced(codes, low, middle - 1);
    node->right = buildBalanced(codes, middle + 1, high);
    return node;
}
```

```c

Tree *insert(Tree *node, int code) {
    if (node == NULL) return newTree(code);
    if (code < node->code) node->left = insert(node->left, code);
    else node->right = insert(node->right, code);
    return node;
}

int height(const Tree *node) {
    int leftH, rightH;
    if (node == NULL) return 0;
    leftH = height(node->left); rightH = height(node->right);
    return 1 + (leftH > rightH ? leftH : rightH);
}

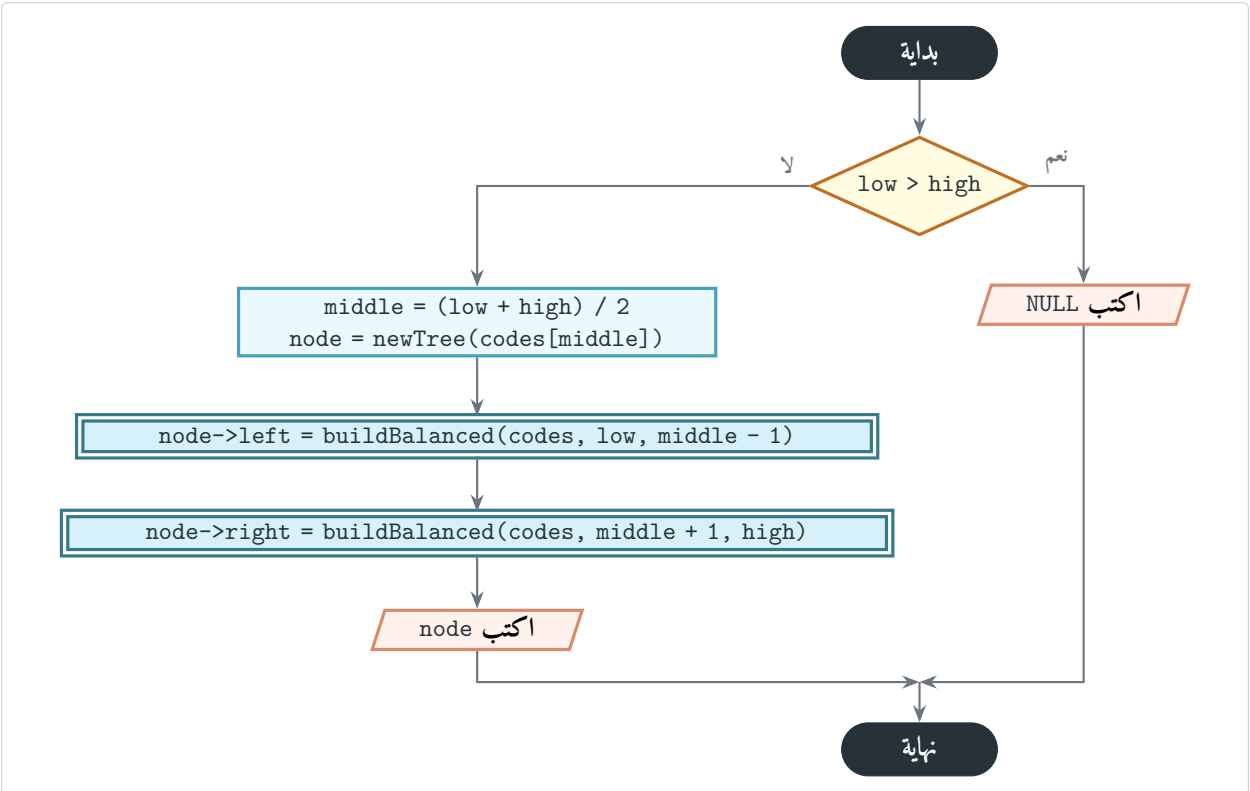
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int codes[15], idx;
    Tree *balanced, *chain = NULL;
    for (idx = 0; idx < 15; idx++) codes[idx] = 1001 + idx * 7;
    balanced = buildBalanced(codes, 0, 14);
    for (idx = 0; idx < 15; idx++) chain = insert(chain, codes[idx]);
    printf("15 sorted codes\nbalanced build: root %d, height %d\n", balanced->code, height(balanced));
    printf("insert one by one: root %d, height %d\n", chain->code, height(chain));
    freeTree(balanced);
    freeTree(chain);
    return 0;
}
```

نتيجة التنفيذ

```
15 sorted codes
balanced build: root 1050, height 4
insert one by one: root 1001, height 15
```

المخطط الإنسيابي (Organigramme)



تمرين 13.24 شجرة العبارة لحساب السعر

تجارة إلكترونية ★★ متوسط

صيغة السعر `(price - discount) * qty + shipping` تُمثل **بشجرة عبارة**: العقد الداخلية عمليات (+ - * /) والأوراق أعداد. اكتب `evaluate(node)` العودية، و `printInfix(node)` التي تعيد كتابة العبارة بالأقواس، ثم احسب السعر لعدة كميات بتغيير ورقة واحدة.

خطوات الحل

1) ورقة: نعيد قيمتها.
2) عقدة عملية: نحسب الفرعين ثم نطبق العملية.
3) `printInfix`: يسار عملية يمين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char op; double value; struct Tree *left, *right; } Tree;

Tree *leaf(double value) {
    Tree *node = malloc(sizeof(Tree));
    node->op = 0; node->value = value; node->left = node->right = NULL;
    return node;
}

Tree *operation(char op, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->op = op; node->value = 0; node->left = left; node->right = right;
    return node;
}

double evaluate(const Tree *node) {
    double left, right;
    if (node->op == 0) return node->value;                // ورقة
    left = evaluate(node->left);
    right = evaluate(node->right);
    switch (node->op) {
        case '+': return left + right;
        case '-': return left - right;
        case '*': return left * right;
        default:  return left / right;
    }
}

void printInfix(const Tree *node) {
    if (node->op == 0) { printf("%g", node->value); return; }
    printf("(");
    printInfix(node->left);
    printf(" %c ", node->op);
    printInfix(node->right);
    printf(")");
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

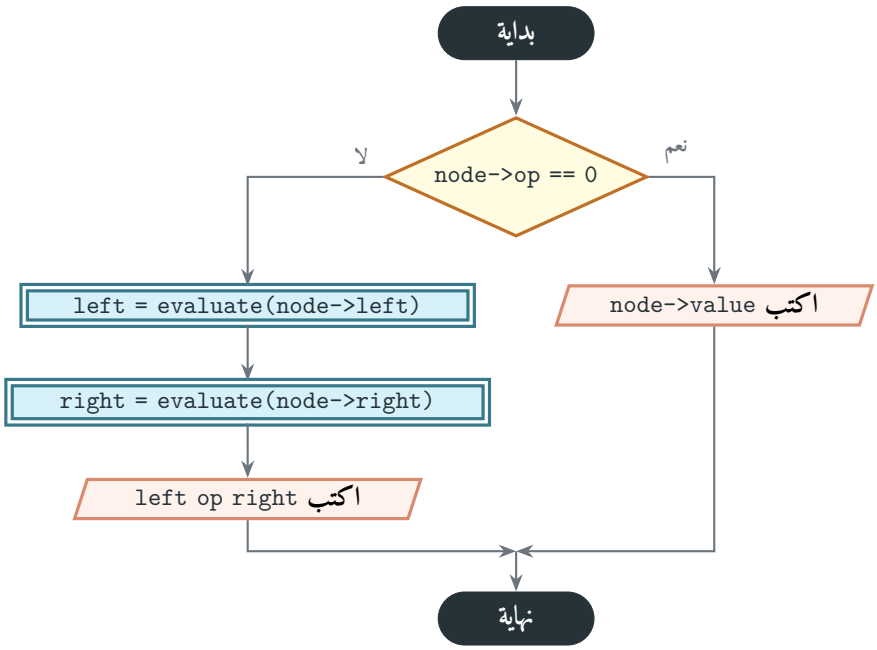
int main(void) {
    Tree *qty = leaf(1);
    Tree *root = operation('+', operation('*', operation('-', leaf(4500), leaf(500)), qty), leaf(400));
    int count;
    printInfix(root);
    printf("\n");
    for (count = 1; count <= 4; count++) {
        qty->value = count;                                // نغير الورقة فقط
        printf("qty %d -> %.0f DA\n", count, evaluate(root));
    }
```

```
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
(((4500 - 500) * 1) + 400)
qty 1 -> 4400 DA
qty 2 -> 8400 DA
qty 3 -> 12400 DA
qty 4 -> 16400 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 13.25 شجرة بحث لأسماء المنتجات

تجارة إلكترونية

متوسط

ابنِ شجرة بحث **مفاتيحها نصوص** (أسماء المنتجات) بمقارنة أبجدية يدوية، مع السعر في كل عقدة. اعرض الكتالوج أبجديًا، وابحث عن أسماء، واعرض كل الأسماء التي تبدأ **ببادئة** معينة (مع تقليم الفروع غير المعنية).

خطوات الحل

1) `compareText` تحدد الاتجاه في الإدراج والبحث.

2) البادئة: نزور عقدة إذا كان مجال اسمها يمكن أن يحتوي على البادئة؛ المرور الوسطي يعطي النتائج مرتبة.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char name[16]; double price; struct Tree *left, *right; } Tree;

int compareText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first - *second;
}
```

```c

int startsWith(const char *text, const char *prefix) {
    while (*prefix != '\0') if (*text++ != *prefix++) return 0;
    return 1;
}

Tree *insert(Tree *node, const char *name, double price) {
    int pos;
    if (node == NULL) {
        node = malloc(sizeof(Tree));
        for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
        node->price = price;
        node->left = node->right = NULL;
    } else if (compareText(name, node->name) < 0) node->left = insert(node->left, name, price);
    else node->right = insert(node->right, name, price);
    return node;
}

void inorder(const Tree *node) { if (node) { inorder(node->left); printf("%s ", node->name);
    ↪ inorder(node->right); } }

void printPrefix(const Tree *node, const char *prefix) {
    if (node == NULL) return;
    if (compareText(prefix, node->name) < 0) printPrefix(node->left, prefix);    // قد توجد يسارًا
    if (startsWith(node->name, prefix)) printf("  %s %.0f\n", node->name, node->price);
    if (startsWith(node->name, prefix) || compareText(node->name, prefix) < 0) printPrefix(node->right,
    ↪ prefix);
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    const char *names[8] = {"Monitor", "Cable", "Mouse", "Laptop", "Webcam", "Modem", "Hub", "Keyboard"};
    double prices[8] = {32000, 450, 1500, 89000, 4100, 6200, 3100, 3200};
    Tree *root = NULL;
    char prefix[16];
    int idx;
    for (idx = 0; idx < 8; idx++) root = insert(root, names[idx], prices[idx]);
    printf("A-Z: "); inorder(root); printf("\n");
    scanf("%15s", prefix);
    printf("Starting with \"%s\":\n", prefix);
    printPrefix(root, prefix);
    freeTree(root);
    return 0;
}
```

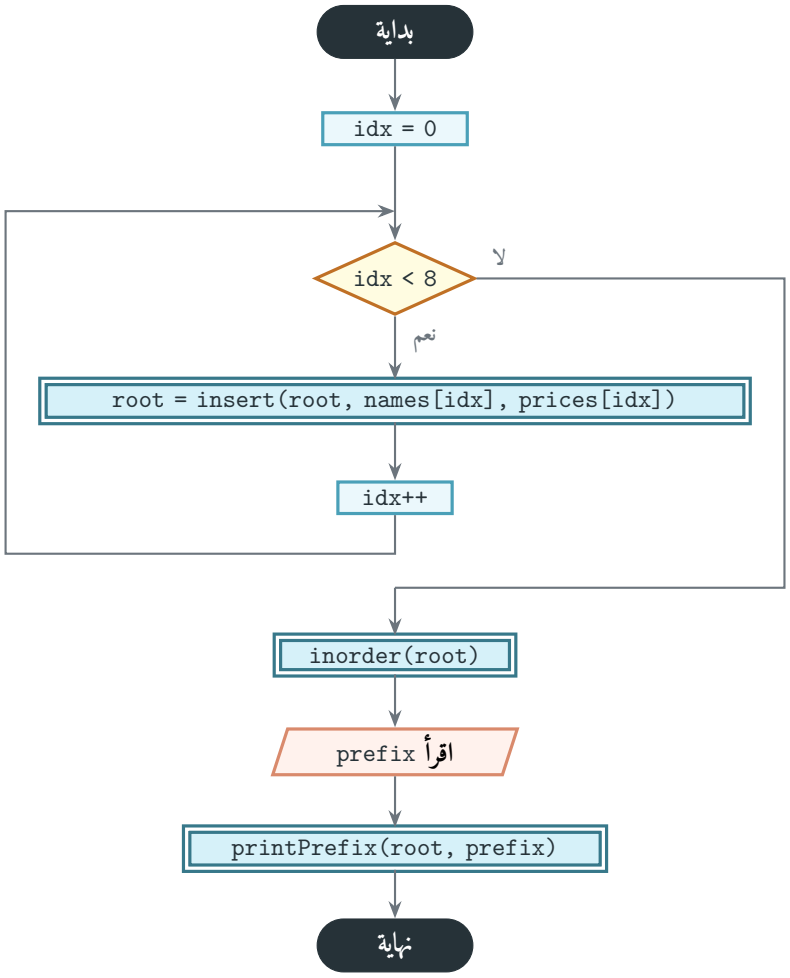
المدخلات

```
Mo
```

نتيجة التنفيذ

```
A-Z: Cable Hub Keyboard Laptop Modem Monitor Mouse Webcam
Starting with "Mo":
  Modem 6200
  Monitor 32000
  Mouse 1500
```

المخطط الإنسيابي (Organigramme)



تمرين 13.26 أعرض مستوى في شجرة الأقسام

علم البيانات متوسط

لتصميم القائمة المنسدلة، نحتاج إلى معرفة **أعرض مستوى** (أكبر عدد من الأقسام في نفس العمق). اكتب العودية التي تملأ جدول العدادات حسب العمق، ثم استخرج المستوى الأعرض. `countAtDepth(node, depth, counts)`

خطوات الحل

1) مرور قبلي مع تمرير العمق: `counts[depth]++`.
2) نحفظ أكبر عمق وصلنا إليه لنعرف عدد المستويات.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

int maxDepth = 0;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
```

```c
    return node;
}

void countAtDepth(const Tree *node, int depth, int counts[]) {
    if (node == NULL) return;
    counts[depth]++;
    if (depth > maxDepth) maxDepth = depth;
    countAtDepth(node->left, depth + 1, counts);
    countAtDepth(node->right, depth + 1, counts);
}

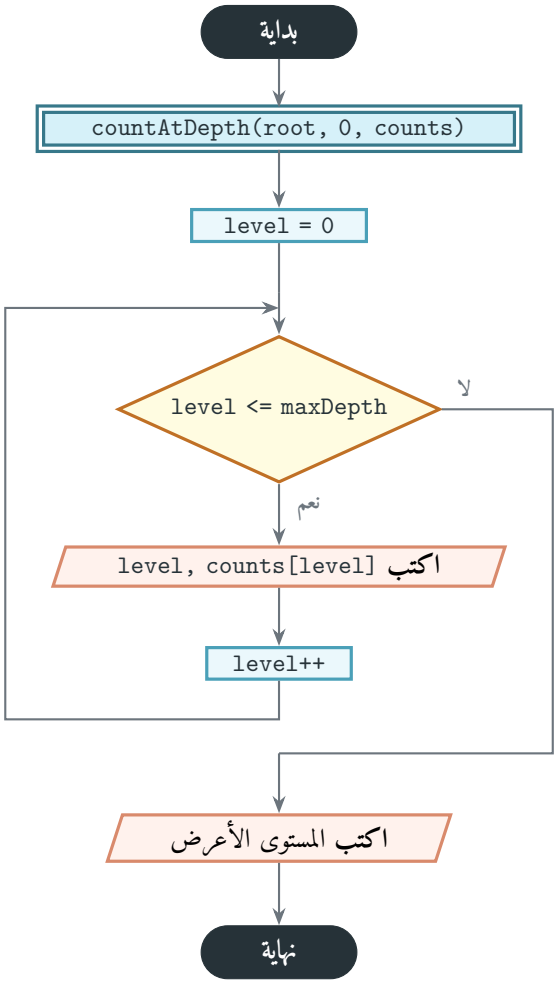
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(1, newTree(2, newTree(4, newTree(8, NULL, NULL), NULL), newTree(5, NULL, NULL)),
                            newTree(3, newTree(6, newTree(9, NULL, NULL), newTree(10, NULL, NULL)),
                                       newTree(7, NULL, newTree(11, NULL, NULL))));
    int counts[20] = {0}, level, widest = 0;
    countAtDepth(root, 0, counts);
    for (level = 0; level <= maxDepth; level++) {
        printf("level %d: %d categories\n", level, counts[level]);
        if (counts[level] > counts[widest]) widest = level;
    }
    printf("Widest level: %d (%d items)\n", widest, counts[widest]);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
level 0: 1 categories
level 1: 2 categories
level 2: 4 categories
level 3: 4 categories
Widest level: 2 (4 items)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.27 قطر شجرة شبكة التوزيع

تجارة إلكترونية ★★ متوسط

شبكة التوزيع شجرة (مستودعات ونقاط تسليم). **القطر** هو أطول مسار بين أي عقدتين (عدد الروابط)، أي أسوأ مسافة نقل. اكتب `depthAndDiameter(node, best)` التي تعيد ارتفاع الفرع وتحدّث القطر الأفضل في مرور واحد.

خطوات الحل

1) لكل عقدة: أطول مسار يمر بها = ارتفاع اليسار + ارتفاع اليمين.
2) نحدث `*best` إذا كان أكبر، ونعيد `1 + max` للأب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

```

```c
int depthAndDiameter(const Tree *node, int *best) {
    int leftH, rightH;
    if (node == NULL) return 0;
    leftH = depthAndDiameter(node->left, best);
    rightH = depthAndDiameter(node->right, best);
    if (leftH + rightH > *best) *best = leftH + rightH;      // مسار يمر بهذه العقدة
    return 1 + (leftH > rightH ? leftH : rightH);
}

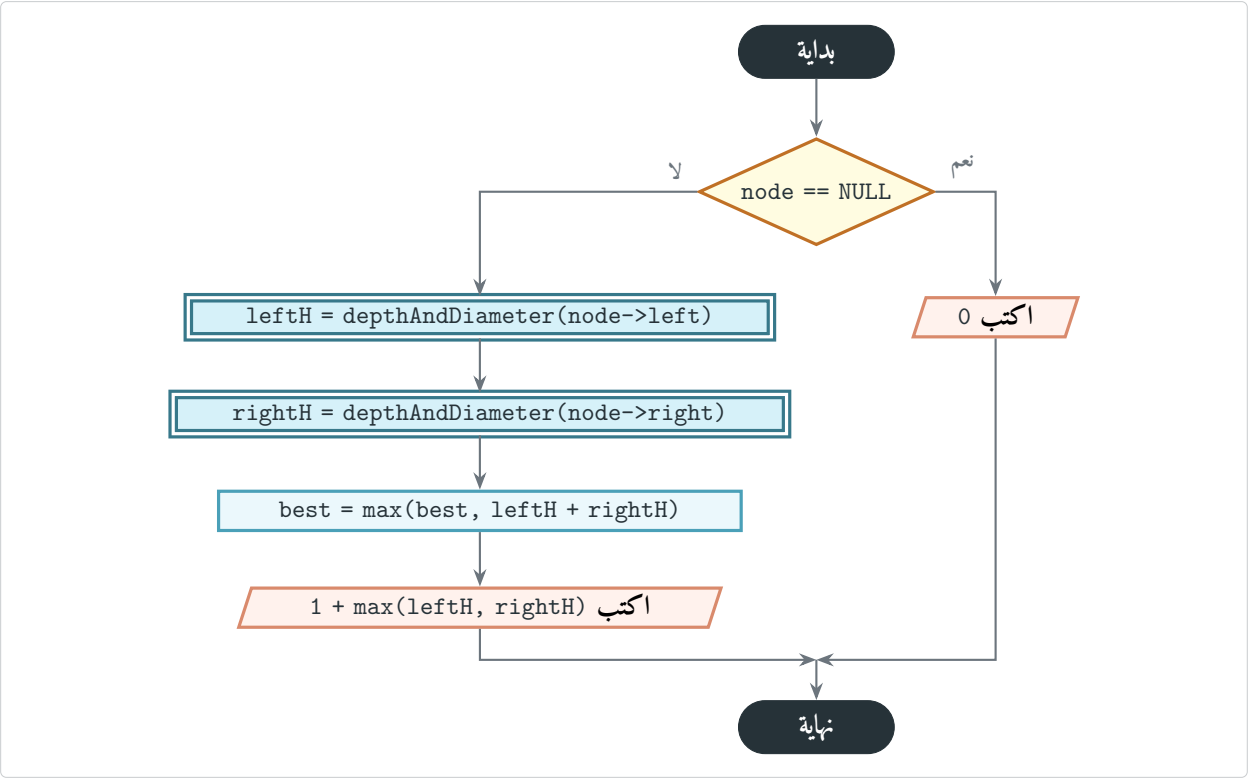
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(0,
        newTree(1, newTree(3, newTree(7, newTree(9, NULL, NULL), NULL), NULL), NULL), newTree(4, NULL, newTree(8,
      ↪ NULL, newTree(10, NULL, NULL)))),
        newTree(2, newTree(5, NULL, NULL), NULL));
    int best = 0, height = depthAndDiameter(root, &best);
    printf("Network height: %d levels\n", height);
    printf("Longest delivery chain: %d links\n", best);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Network height: 5 levels
Longest delivery chain: 6 links
```

المخطط الإنسيابي (Organigramme)



تمرين 13.28 مجموع المبيعات في كل مستوى

علم البيانات ★★ متوسط

شجرة الهيكل التنظيمي للمبيعات: المدير الجهوي، ثم المشرفون، ثم البائعون. كل عقدة تحمل مبيعات الشخص. احسب **مجموع** ومتوسط المبيعات **في كل مستوى** بمرور بالعرض باستعمال طابور، وحدد المستوى الأعلى متوسطًا.

خطوات الحل

1) طابور من المؤشرات، نعالج المستوى كاملًا في كل دورة (`levelSize`).
2) نجمع المبيعات ونعد العقد لكل مستوى.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { double sales; struct Tree *left, *right; } Tree;

Tree *newTree(double sales, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->sales = sales; node->left = left; node->right = right;
    return node;
}

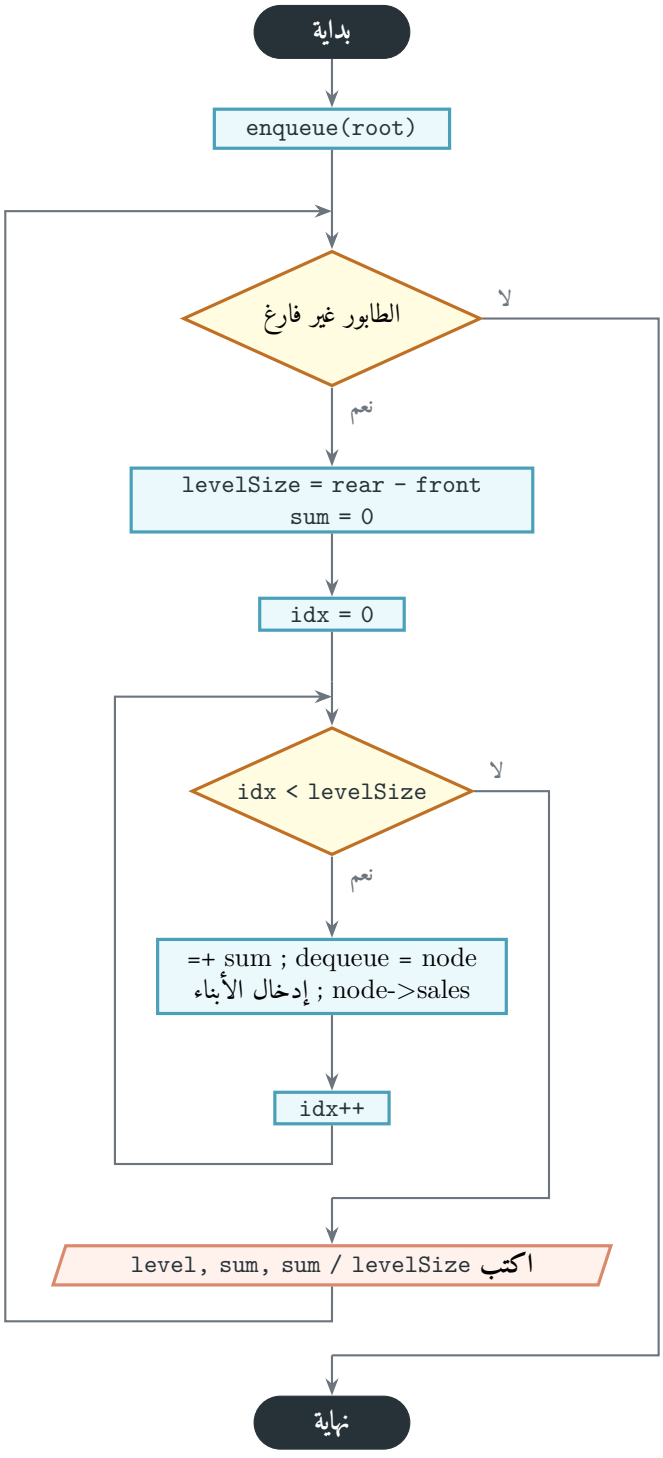
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(120000,
        newTree(310000, newTree(450000, NULL, NULL), newTree(380000, NULL, NULL)),
        newTree(290000, newTree(520000, NULL, NULL), newTree(410000, newTree(200000, NULL, NULL), NULL)));
    Tree *queue[32], *node;
    int front = 0, rear = 0, levelSize, idx, level = 0, bestLevel = 0;
    double sum, bestAvg = -1;
    queue[rear++] = root;
    while (front < rear) {
        levelSize = rear - front;
        for (sum = 0, idx = 0; idx < levelSize; idx++) {
            node = queue[front++];
            sum += node->sales;
            if (node->left) queue[rear++] = node->left;
            if (node->right) queue[rear++] = node->right;
        }
        printf("level %d: %d people, total %8.0f, average %8.0f\n", level, levelSize, sum, sum / levelSize);
        if (sum / levelSize > bestAvg) { bestAvg = sum / levelSize; bestLevel = level; }
        level++;
    }
    printf("Best average at level %d\n", bestLevel);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
level 0: 1 people, total   120000, average   120000
level 1: 2 people, total   600000, average   300000
level 2: 4 people, total  1760000, average   440000
level 3: 1 people, total   200000, average   200000
Best average at level 2
```

المخطط الإنسيابي (Organigramme)



تمرين 13.29 تكرار الكلمات بشجرة بحث

علم البيانات ★★ متوسط

لعدّ الكلمات في تعليقات الزبائن وعرضها مرتبة أبجديًا مباشرة، استعمل شجرة بحث مفاتيحها كلمات وتحمل عداداً: إذا وُجدت الكلمة تزيد عدادها، وإلا ننشئ عقدة. اعرض الكلمات بالمرور الوسطي مع تكرارها، وعدد الكلمات المختلفة.

خطوات الحل

1) نقسم النص إلى كلمات بحروف صغيرة.

2) `insertWord(node, word)`: مقارنة أبجدية؛ المساواة تزيد `count`.

3) المرور الوسطي يعطي الترتيب الأبجدي دون خوارزمية ترتيب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char word[16]; int count; struct Tree *left, *right; } Tree;

int distinct = 0;

int compareText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first - *second;
}

Tree *insertWord(Tree *node, const char *word) {
    int cmp, pos;
    if (node == NULL) {
        node = malloc(sizeof(Tree));
        for (pos = 0; (node->word[pos] = word[pos]) != '\0'; pos++) {}
        node->count = 1;
        node->left = node->right = NULL;
        distinct++;
        return node;
    }
    cmp = compareText(word, node->word);
    if (cmp == 0) node->count++;                          // كلمة موجودة
    else if (cmp < 0) node->left = insertWord(node->left, word);
    else node->right = insertWord(node->right, word);
    return node;
}

void inorder(const Tree *node) {
    if (node == NULL) return;
    inorder(node->left);
    printf("%s(%d) ", node->word, node->count);
    inorder(node->right);
}

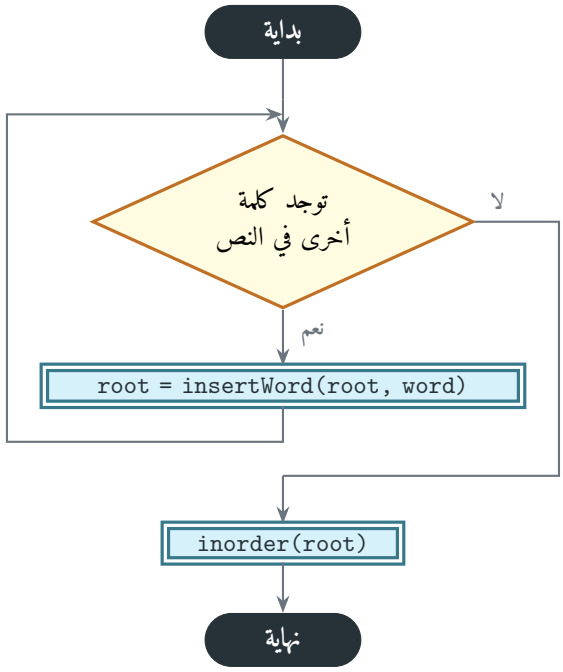
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    const char *text = "Good price good quality. Fast delivery, good packaging; delivery was fast. Price
      ↪ fair.";
    Tree *root = NULL;
    char word[16], ch;
    int pos = 0, len = 0;
    do {
        ch = text[pos++];
        if (ch >= 'A' && ch <= 'Z') ch = ch - 'A' + 'a';
        if (ch >= 'a' && ch <= 'z') { if (len < 15) word[len++] = ch; }
        else if (len > 0) { word[len] = '\0'; root = insertWord(root, word); len = 0; }
    } while (ch != '\0');
    inorder(root);
    printf("\n%d distinct words\n", distinct);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
delivery(2) fair(1) fast(2) good(3) packaging(1) price(2) quality(1) was(1)
8 distinct words
```

المخطط الانسيابي (Organigramme)



تمرين 13.30 حفظ الشجرة في نص واستعادتها

تجارة إلكترونية ★★ متوسط

لحفظ شجرة الأقسام في ملف إعدادات، نسلسلها بالمرور القبلي مع علامة # للفرع الفارغ: "10 5 # # 20 # #". اكتب `serialize(node)` و `deserialize(text, &pos)` التي تعيد بناء الشجرة نفسها، ثم تحقق بإعادة التسلسل.

خطوات الحل

1) التسلسل: قيمة العقدة ثم تسلسل اليسار ثم اليمين، `NULL` ← `#`.

2) الاستعادة: نقرأ الرمز التالي، `#` ← `NULL`، وإلا عقدة، ثم نبني اليسار ثم اليمين بنفس الترتيب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

void appendText(char *text, int *len, const char *piece) { while (*piece != '\0') text[(*len)++] = *piece++;
    ↪ text[*len] = '\0'; }

void appendNumber(char *text, int *len, int value) {
    char digits[12];
    int count = 0;
    do { digits[count++] = (char)('0' + value % 10); value /= 10; } while (value > 0);
    while (count > 0) text[(*len)++] = digits[--count];
    text[(*len)++] = ' ';
    text[*len] = '\0';
}

void serialize(const Tree *node, char *text, int *len) {
```

```c
    if (node == NULL) { appendText(text, len, "# "); return; }
    appendNumber(text, len, node->code);
    serialize(node->left, text, len);
    serialize(node->right, text, len);
}

Tree *deserialize(const char *text, int *pos) {
    Tree *node;
    int value = 0;
    while (text[*pos] == ' ') (*pos)++;
    if (text[*pos] == '#') { (*pos)++; return NULL; }       // فرع فارغ
    while (text[*pos] >= '0' && text[*pos] <= '9') value = value * 10 + (text[(*pos)++] - '0');
    node = newTree(value, NULL, NULL);
    node->left = deserialize(text, pos);                    // نفس ترتيب التسلسل
    node->right = deserialize(text, pos);
    return node;
}

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

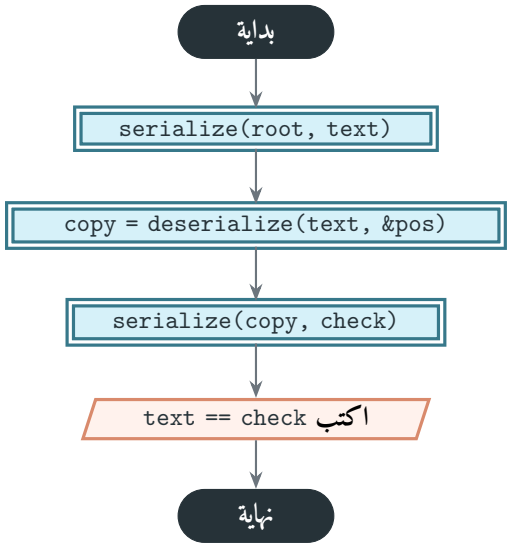
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(100, newTree(110, newTree(111, NULL, NULL), NULL),
                              newTree(120, newTree(121, NULL, NULL), newTree(122, NULL, newTree(1221, NULL,
      ↪ NULL))));
    Tree *copy;
    char text[200], check[200];
    int len = 0, pos = 0;
    serialize(root, text, &len);
    printf("saved:   %s\n", text);
    copy = deserialize(text, &pos);
    len = 0;
    serialize(copy, check, &len);
    printf("rebuilt: %s\n", check);
    printf("identical: %s\n", sameText(text, check) ? "yes" : "no");
    freeTree(root);
    freeTree(copy);
    return 0;
}
```

نتيجة التنفيذ

```
saved:   100 110 111 # # # 120 121 # # 122 # 1221 # #
rebuilt: 100 110 111 # # # 120 121 # # 122 # 1221 # #
identical: yes
```

المخطط الإنسيابي (Organigramme)



تمرين 13.31 مقارنة نسختين من شجرة الأقسام

تجارة إلكترونية ★★ متوسط

قبل نشر تحديث لشجرة الأقسام، نقارن النسخة الجديدة بالقديمة. اكتب `sameTree(first, second)` العودية التي تتحقق من تطابق **البنية والقيم**، و `countDiff` التي تعد العقد المختلفة (قيمة مختلفة أو عقدة موجودة في واحدة فقط).

خطوات الحل

1) كلاهما `NULL`: متطابقان. واحد فقط `NULL`: مختلفان.
2) وإلا: القيمتان متساويتان والفرعان متطابقان.
3) `countDiff`: إذا كانت إحدى العقدتين غائبة نعد كل عقد الأخرى.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

int sameTree(const Tree *first, const Tree *second) {
    if (first == NULL && second == NULL) return 1;
    if (first == NULL || second == NULL) return 0;
    return first->code == second->code && sameTree(first->left, second->left) && sameTree(first->right,
      ↪ second->right);
}

int countNodes(const Tree *node) { return node == NULL ? 0 : 1 + countNodes(node->left) +
      ↪ countNodes(node->right); }

int countDiff(const Tree *first, const Tree *second) {
    if (first == NULL) return countNodes(second);              // عقد مضافة
    if (second == NULL) return countNodes(first);              // عقد محذوفة
    return (first->code != second->code) + countDiff(first->left, second->left) + countDiff(first->right,
      ↪ second->right);
```

```c
}

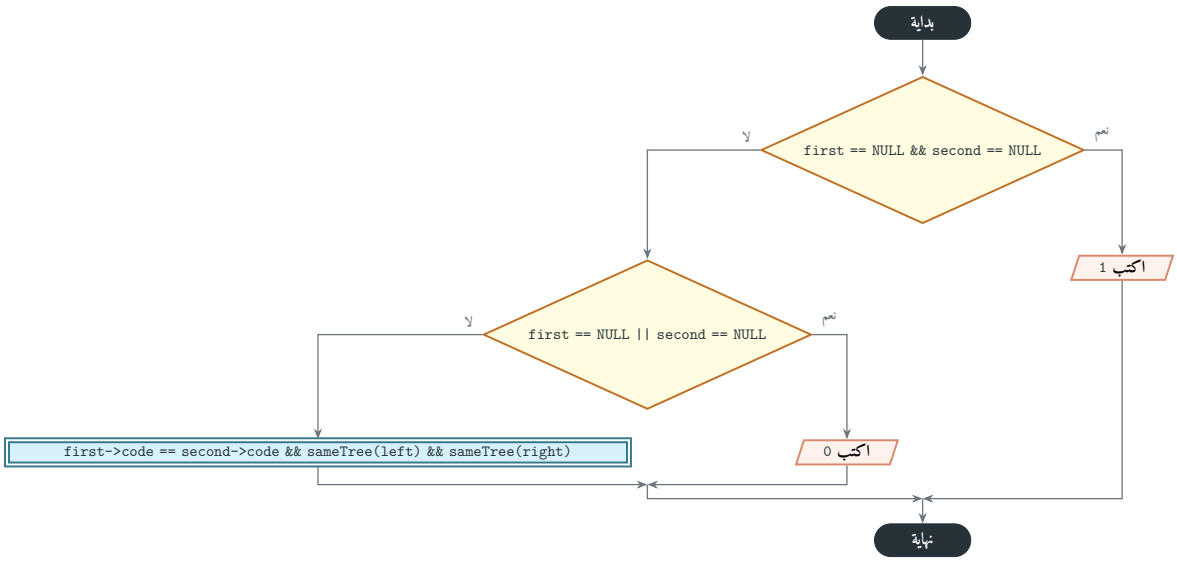
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *oldTree = newTree(1, newTree(2, newTree(4, NULL, NULL), NULL), newTree(3, newTree(5, NULL, NULL),
      ↪ newTree(6, NULL, NULL)));
    Tree *sameCopy = newTree(1, newTree(2, newTree(4, NULL, NULL), NULL), newTree(3, newTree(5, NULL, NULL),
      ↪ newTree(6, NULL, NULL)));
    Tree *newVersion = newTree(1, newTree(2, newTree(4, NULL, NULL), newTree(7, NULL, NULL)), newTree(9,
      ↪ newTree(5, NULL, NULL), NULL));
    printf("old vs copy: %s, %d differences\n", sameTree(oldTree, sameCopy) ? "same" : "different",
      ↪ countDiff(oldTree, sameCopy));
    printf("old vs new:  %s, %d differences\n", sameTree(oldTree, newVersion) ? "same" : "different",
      ↪ countDiff(oldTree, newVersion));
    freeTree(oldTree); freeTree(sameCopy); freeTree(newVersion);
    return 0;
}
```

نتيجة التنفيذ

```
old vs copy: same, 0 differences
old vs new:  different, 3 differences
```

المخطط الإنسيابي (Organigramme)



تمرين 13.32 السعر التالي في الترتيب

تجارة إلكترونية ★★ متوسط

زر "المنتج الأغلى التالي": اكتب `successor(root, price)` التي تجد أصغر سعر أكبر تمامًا من سعر معطى في شجرة البحث بحلقة واحدة من الجذر. إذا كان للعقدة فرع أيمن فالخلف أصغر عقدة فيه، وإلا فهو آخر سلف نزلنا منه يسارًا.

خطوات الحل

1) ننزل من الجذر: إذا كان سعر العقدة أكبر من القيمة فهو مرشح (`best`) ونذهب يسارًا، وإلا يمينًا.
2) هذه الطريقة تغطي الحالتين دون مؤشر على الأب.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

const Tree *successor(const Tree *node, int price) {
    const Tree *best = NULL;
    while (node != NULL) {
        if (node->price > price) { best = node; node = node->left; }   // مرشح أقرب
        else node = node->right;
    }
    return best;
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[9] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 6800, 15000}, idx, price;
    const Tree *next;
    Tree *root = NULL;
    for (idx = 0; idx < 9; idx++) root = insert(root, prices[idx]);
    scanf("%d", &price);
    printf("Walking up from %d:", price);
    for (next = successor(root, price); next != NULL; next = successor(root, next->price))
        printf(" -> %d", next->price);
    printf(" -> end\n");
    freeTree(root);
    return 0;
}
```

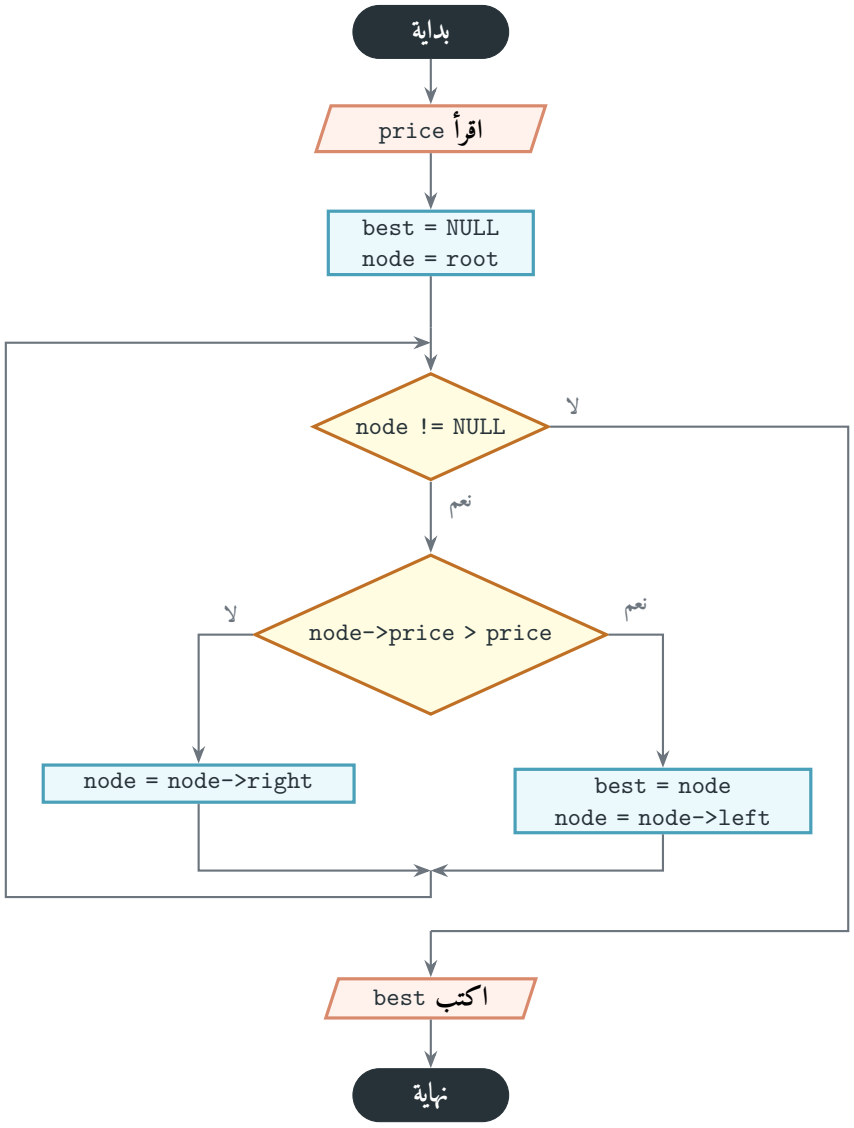
المدخلات

```
4500
```

نتيجة التنفيذ

```
Walking up from 4500: -> 6800 -> 12500 -> 15000 -> 32000 -> 89000 -> end
```

المخطط الإنسيابي (Organigramme)



تمرين 13.33 مجموع المبيعات في مجال تواريخ

علم البيانات ★★ متوسط

مفاتيح شجرة البحث أرقام أيام السنة، وكل عقدة تحمل مبيعات ذلك اليوم. اكتب `rangeSum(node, from, to)` التي تجمع المبيعات بين يومين مع **التقليم**، واستعملها لمقارنة مبيعات أسبوعين.

خطوات الحل

1) مثل عد المجال: نقلم الفروع التي لا يمكن أن تحتوي على مفاتيح داخل `[from, to]`.
2) نجمع مبيعات العقد الداخلة في المجال.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int day; double sales; struct Tree *left, *right; } Tree;

```

```c
Tree *insert(Tree *node, int day, double sales) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->day = day; node->sales = sales; node->left =
      ↪ node->right = NULL; }
    else if (day < node->day) node->left = insert(node->left, day, sales);
    else node->right = insert(node->right, day, sales);
    return node;
}

double rangeSum(const Tree *node, int from, int to) {
    if (node == NULL) return 0;
    if (node->day < from) return rangeSum(node->right, from, to);
    if (node->day > to) return rangeSum(node->left, from, to);
    return node->sales + rangeSum(node->left, from, to) + rangeSum(node->right, from, to);
}

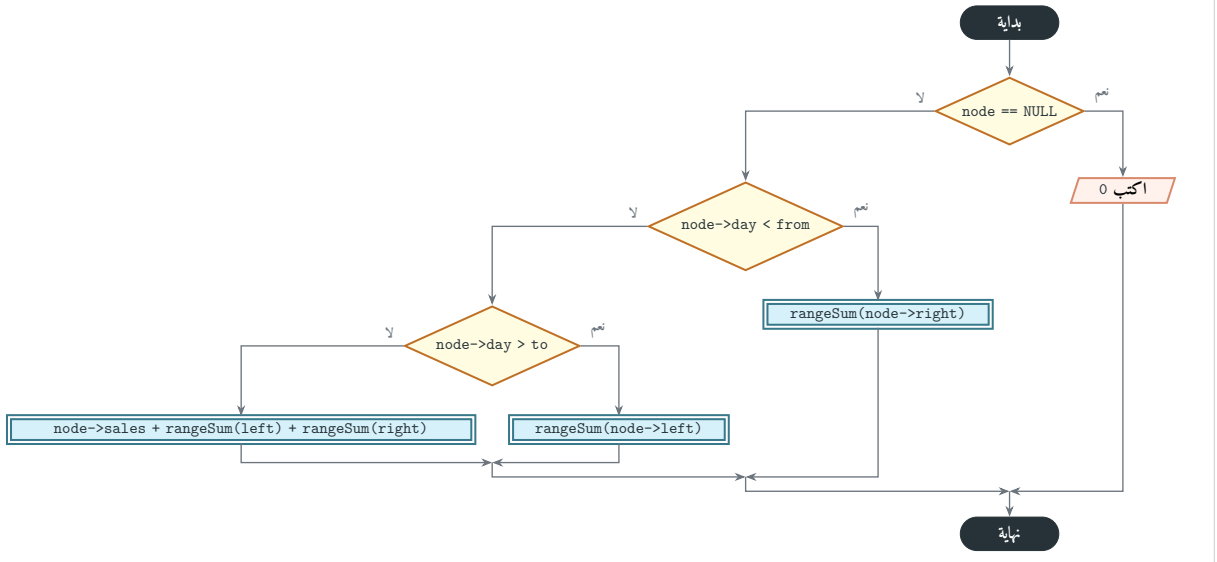
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = NULL;
    int day, order[14] = {7, 3, 11, 1, 5, 9, 13, 2, 4, 6, 8, 10, 12, 14}, idx;
    for (idx = 0; idx < 14; idx++) {
        day = order[idx];                                   // ترتيب إدراج متوازن
        root = insert(root, day, 1000 + (day * 173) % 600);
    }
    printf("Week 1 (days 1-7):  %.0f DA\n", rangeSum(root, 1, 7));
    printf("Week 2 (days 8-14): %.0f DA\n", rangeSum(root, 8, 14));
    printf("Weekend 6-7:        %.0f DA\n", rangeSum(root, 6, 7));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Week 1 (days 1-7):  8844 DA
Week 2 (days 8-14): 8921 DA
Weekend 6-7:        2449 DA
```

المخطط الإنسيابي (Organigramme)



تمرين 13.34 إعادة بناء الشجرة من مرورين

علم البيانات ★★ متوسط

فقدنا الشجرة لكن لدينا مرورها **القبلي** و**الوسطي**. أعد بناءها: أول عنصر في القبلي هو الجذر، موضعه في الوسطي يقسمه إلى الفرع الأيسر والأيمن، ونكرر عوديًا. تحقق بحساب المرور البعدي.

خطوات الحل

1) `build(preStart, inStart, size)`: الجذر `pre[preStart]`.
2) نبحث عن الجذر في `in` ← `leftSize`.
3) اليسار: `build(preStart + 1, inStart, leftSize)`؛ اليمين:
`build(preStart + 1 + leftSize, rootPos + 1, size - 1 - leftSize)`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char key; struct Tree *left, *right; } Tree;

const char pre[] = "ABDEGCFH", in[] = "DBGEACHF";

Tree *build(int preStart, int inStart, int size) {
    Tree *node;
    int rootPos, leftSize;
    if (size == 0) return NULL;
    node = malloc(sizeof(Tree));
    node->key = pre[preStart];                          // الجذر
    for (rootPos = inStart; in[rootPos] != node->key; rootPos++) {}
    leftSize = rootPos - inStart;
    node->left = build(preStart + 1, inStart, leftSize);
    node->right = build(preStart + 1 + leftSize, rootPos + 1, size - 1 - leftSize);
    return node;
}

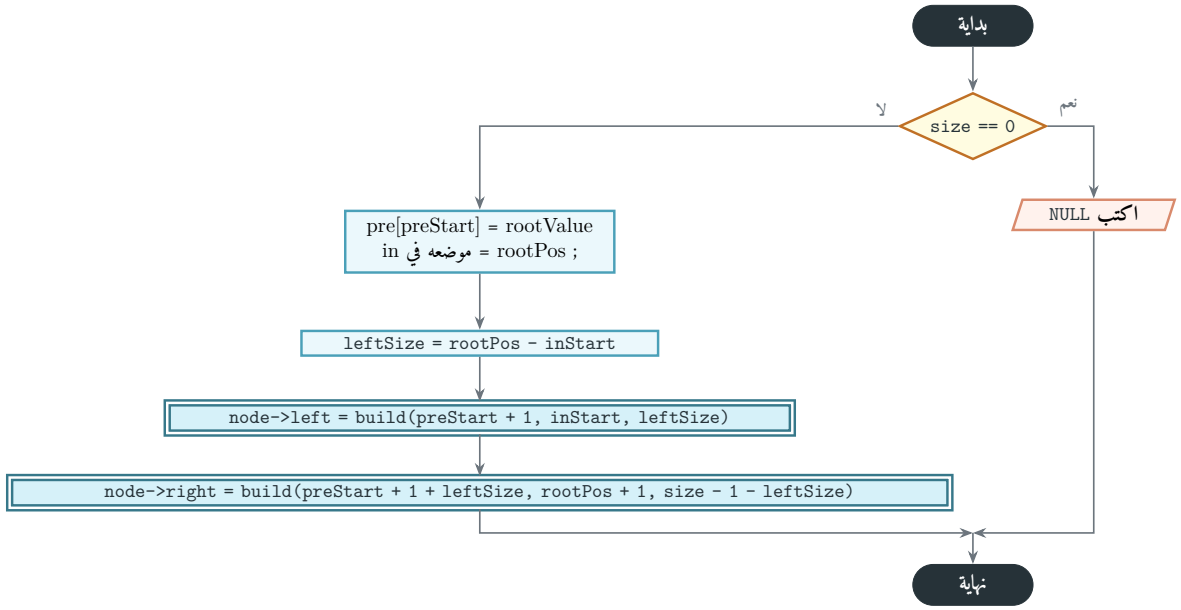
void postorder(const Tree *node) { if (node) { postorder(node->left); postorder(node->right);
    ↪ putchar(node->key); } }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = build(0, 0, 8);
    printf("preorder:  %s\ninorder:   %s\npostorder: ", pre, in);
    postorder(root);
    printf("\nroot %c, left child %c, right child %c\n", root->key, root->left->key, root->right->key);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
preorder:  ABDEGCFH
inorder:   DBGEACHF
postorder: DGEBHFCA
root A, left child B, right child C
```

المخطط الانسيابي (Organigramme)



تمرين 13.35 تصنيف زبون بشجرة قرار

علم البيانات ★★ متوسط

شجرة قرار مُدرَّبة مسبقًا: كل عقدة داخلية تختبر خاصية (feature) مقارنة بعتبة، والأوراق تحمل الصنف (يشتري / لا يشتري) واحتماله. اكتب `classify(node, features)` بحلقة تنزل من الجذر حتى ورقة، واعرض المسار المتبع لعدة زوار.

خطوات الحل

1) عقدة داخلية: إذا `features[node->feature] <= node->threshold` نذهب يسارًا، وإلا يمينًا.
2) الورقة: `feature == -1`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int feature; double threshold; int label; double probability; struct Tree *left,
    ↪ *right; } Tree;

const char *names[3] = {"pages", "minutes", "cart_items"};

Tree *test(int feature, double threshold, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->feature = feature; node->threshold = threshold; node->left = left; node->right = right;
    node->label = -1; node->probability = 0;
    return node;
}

Tree *leaf(int label, double probability) {
    Tree *node = test(-1, 0, NULL, NULL);
    node->label = label; node->probability = probability;
    return node;
}

const Tree *classify(const Tree *node, const double *features) {
    while (node->feature != -1) {
        printf(" %s=%.0f%s%.0f", names[node->feature], features[node->feature],
               features[node->feature] <= node->threshold ? "<=" : ">", node->threshold);
        node = features[node->feature] <= node->threshold ? node->left : node->right;
```

```c
    }
    return node;
}

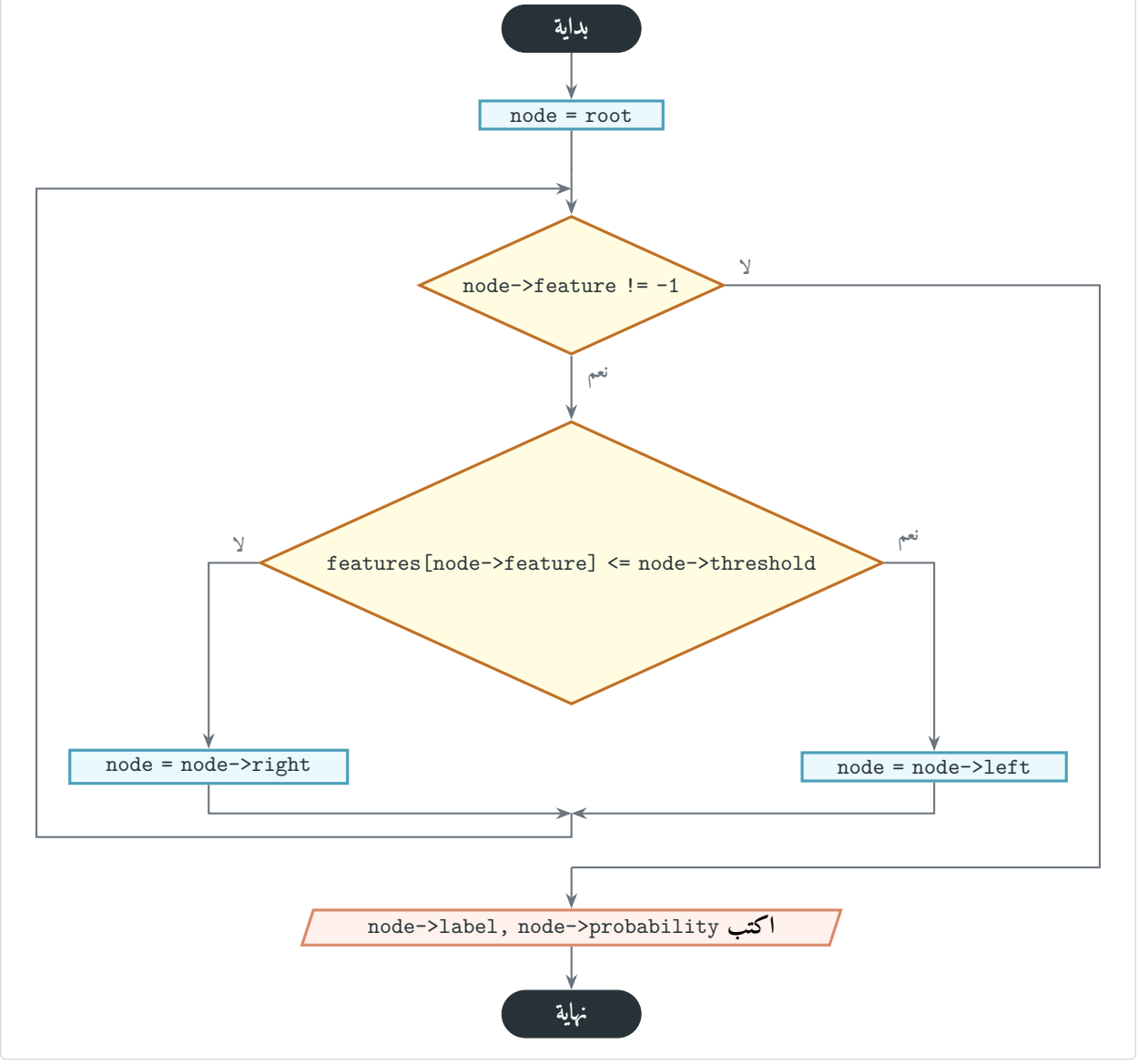
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = test(2, 0, test(1, 3, leaf(0, 0.95), test(0, 6, leaf(0, 0.70), leaf(1, 0.60))),
                            test(1, 2, leaf(0, 0.55), leaf(1, 0.85)));
    double visitors[4][3] = {{2, 1, 0}, {9, 12, 0}, {4, 1, 2}, {7, 9, 3}};
    const Tree *result;
    int idx;
    for (idx = 0; idx < 4; idx++) {
        printf("visitor %d:", idx + 1);
        result = classify(root, visitors[idx]);
        printf(" -> %s (%.0f%%)\n", result->label ? "BUY" : "no buy", 100 * result->probability);
    }
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
visitor 1: cart_items=0<=0 minutes=1<=3 -> no buy (95%)
visitor 2: cart_items=0<=0 minutes=12>3 pages=9>6 -> BUY (60%)
visitor 3: cart_items=2>0 minutes=1<=2 -> no buy (55%)
visitor 4: cart_items=3>0 minutes=9>2 -> BUY (85%)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.36 مسار التنقل من الجذر إلى القسم

تجارة إلكترونية ★★ متوسط

شريط التنقل (breadcrumb) في الموقع يعرض المسار من الجذر إلى القسم الحالي: Store > Electronics > Phones > Android. اكتب `findPath(node, target, path, depth)` العودية التي تملأ المسار إذا وُجد القسم في أحد الفرعين.

خطوات الحل

1) نضع العقدة الحالية في `path[depth]`.
2) إذا كانت الهدف: نجحنا (نعيد العمق + 1).
3) وإلا نبحث في اليسار ثم اليمين؛ إذا فشلا نعيد 0 (التراجع تلقائي بالكتابة فوق `path`).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char name[14]; struct Tree *left, *right; } Tree;

Tree *newTree(const char *name, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    int pos;
    for (pos = 0; (node->name[pos] = name[pos]) != '\0'; pos++) {}
    node->left = left; node->right = right;
    return node;
}

int sameText(const char *first, const char *second) {
    while (*first != '\0' && *first == *second) { first++; second++; }
    return *first == *second;
}

int findPath(const Tree *node, const char *target, const Tree *path[], int depth) {
    int found;
    if (node == NULL) return 0;
    path[depth] = node;
    if (sameText(node->name, target)) return depth + 1;
    found = findPath(node->left, target, path, depth + 1);
    if (found == 0) found = findPath(node->right, target, path, depth + 1);
    return found;
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree("Store",
        newTree("Electronics", newTree("Phones", newTree("Android", NULL, NULL), newTree("iOS", NULL,
      ↪ NULL)), newTree("Laptops", NULL, NULL)),
        newTree("Home", newTree("Kitchen", NULL, NULL), newTree("Garden", newTree("Tools", NULL, NULL),
      ↪ NULL)));
    const Tree *path[10];
    char target[14];
    int len, idx, query;
    for (query = 0; query < 3; query++) {
        scanf("%13s", target);
        len = findPath(root, target, path, 0);
        if (len == 0) { printf("%s: not found\n", target); continue; }
        for (idx = 0; idx < len; idx++) printf("%s%s", path[idx]->name, idx < len - 1 ? " > " : "\n");
    }
    freeTree(root);
    return 0;
}
```

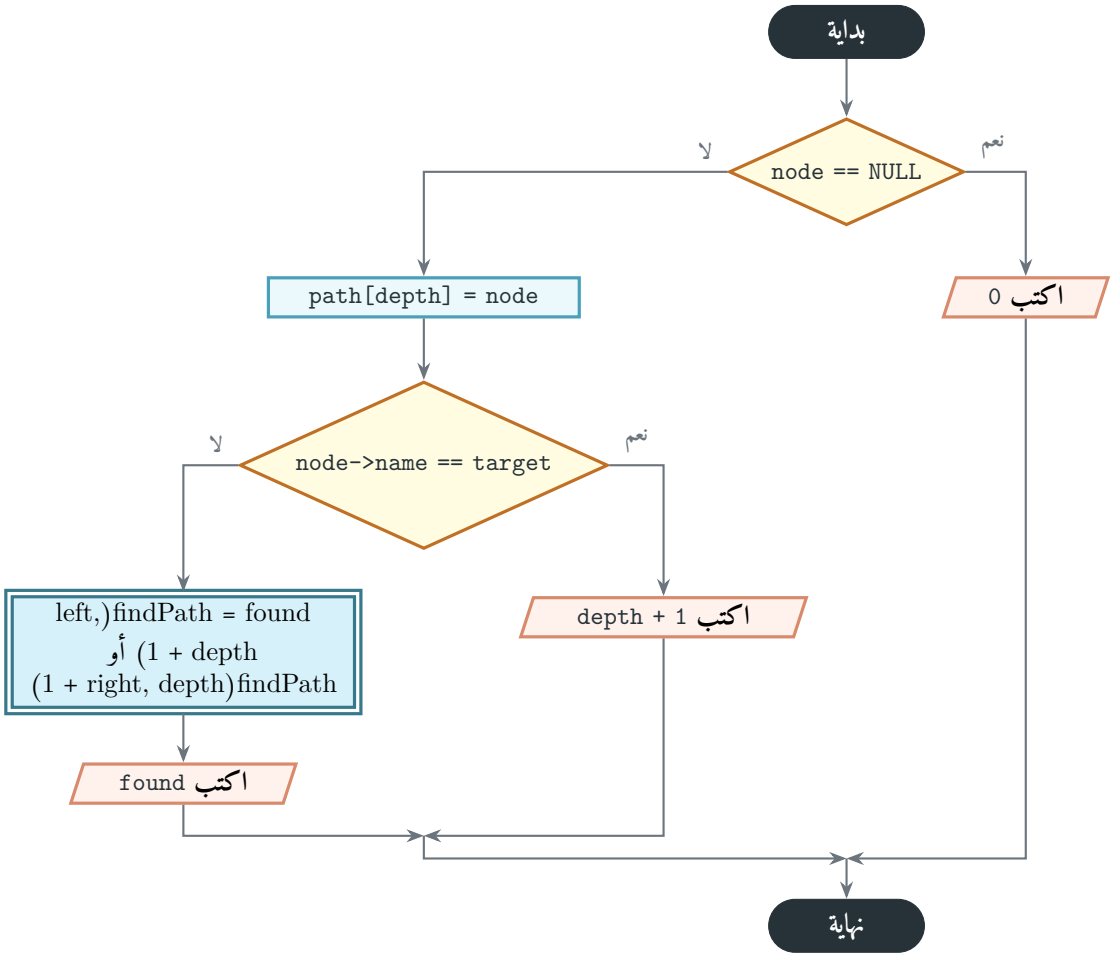
المدخلات

```
Android Garden Toys
```

نتيجة التنفيذ

```
Store > Electronics > Phones > Android
Store > Home > Garden
Toys: not found
```

المخطط الإنسيابي (Organigramme)



تمرين 13.37 تحويل شجرة البحث إلى قائمة مرتبة

تجارة إلكترونية
متوسط

لتصدير الكتالوج بالترتيب، حوّل شجرة البحث في مكانها إلى قائمة متصلة مرتبة، مستعملاً المؤشر `right` كـ `next` و `left = NULL`، دون حجز أي عقدة. نمر بالترتيب الوسطي ونربط كل عقدة بالعقدة السابقة.

خطوات الحل

1) متغير `prev` يحفظ آخر عقدة أضفناها إلى القائمة، و `head` أول عقدة.
2) في المرور الوسطي:

```
prev->right = node; node->left = NULL; prev = node
```

.

3) يجب حفظ `node->right` قبل التعديل لمواصلة المرور.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *head = NULL, *prev = NULL;

```

```c
Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

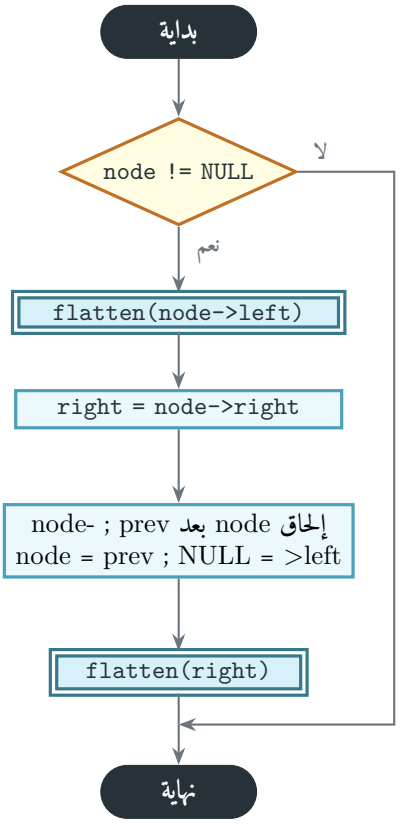
void flatten(Tree *node) {
    Tree *right;
    if (node == NULL) return;
    flatten(node->left);
    right = node->right;                        // نحفظه قبل الكتابة فوقه
    if (prev == NULL) head = node; else prev->right = node;
    node->left = NULL;
    prev = node;
    flatten(right);
}

int main(void) {
    int prices[8] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 6800}, idx;
    Tree *root = NULL, *walk, *next;
    for (idx = 0; idx < 8; idx++) root = insert(root, prices[idx]);
    flatten(root);
    prev->right = NULL;
    for (walk = head; walk != NULL; walk = walk->right) printf("%d -> ", walk->price);
    printf("NULL\n");
    for (walk = head; walk != NULL; walk = next) { next = walk->right; free(walk); }
    return 0;
}
```

نتيجة التنفيذ

```
899 -> 1200 -> 3200 -> 4500 -> 6800 -> 12500 -> 32000 -> 89000 -> NULL
```

المخطط الإنسيابي (Organigramme)



تمرين 13.38 دمج كتالوجين مرتبين من شجرتين

تجارة إلكترونية ★★ متوسط

بعد اندماج متجرين، لكل متجر شجرة بحث لأسعاره. اكتب `collect(node, out, &count)` (مرور وسطي يملأ جدولًا مرتبًا)، ثم ادمج الجدولين المرتبين في قائمة واحدة **دون تكرار**، ثم ابنِ منها شجرة متوازنة جديدة واعرض ارتفاعها.

خطوات الحل

1) المرور الوسطي لكل شجرة يعطي جدولًا مرتبًا.
2) دمج جدولين مرتبين مع تخطي المتساويين.
3) بناء متوازن من الجدول (العنصر الأوسط جذرًا).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *newTree(int price) { Tree *node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; return node; }

Tree *insert(Tree *node, int price) {
    if (node == NULL) return newTree(price);
    if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

void collect(const Tree *node, int *out, int *count) {
    if (node == NULL) return;
    collect(node->left, out, count);
    out[(*count)++] = node->price;            // بالترتيب
    collect(node->right, out, count);
}

Tree *buildBalanced(const int *values, int low, int high) {
    Tree *node;
    int middle;
    if (low > high) return NULL;
    middle = (low + high) / 2;
    node = newTree(values[middle]);
    node->left = buildBalanced(values, low, middle - 1);
    node->right = buildBalanced(values, middle + 1, high);
    return node;
}

int height(const Tree *node) { int lh, rh; if (!node) return 0; lh = height(node->left); rh = height(node->right); return 1 + (lh > rh ? lh : rh); }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

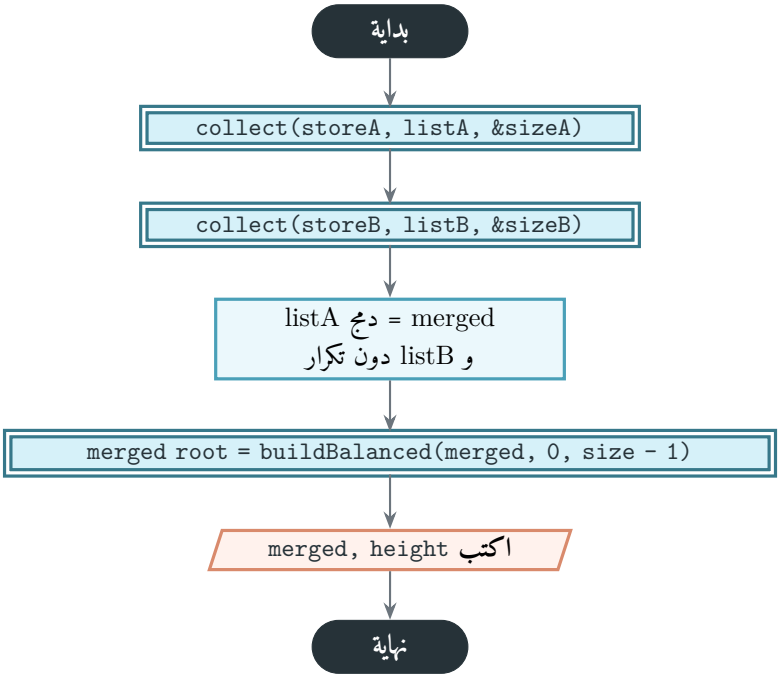
int main(void) {
    int pricesA[6] = {4500, 1200, 32000, 899, 3200, 12500}, pricesB[6] = {3200, 650, 15000, 899, 89000, 2200};
    int listA[10], listB[10], merged[20], sizeA = 0, sizeB = 0, size = 0, left = 0, right = 0, idx;
    Tree *storeA = NULL, *storeB = NULL, *combined;
    for (idx = 0; idx < 6; idx++) { storeA = insert(storeA, pricesA[idx]); storeB = insert(storeB, pricesB[idx]); }
    collect(storeA, listA, &sizeA);
    collect(storeB, listB, &sizeB);
    while (left < sizeA || right < sizeB) {
        if (right == sizeB || (left < sizeA && listA[left] < listB[right])) merged[size++] = listA[left++];
        else if (left == sizeA || listB[right] < listA[left]) merged[size++] = listB[right++];
        else { merged[size++] = listA[left++]; right++; }   // نفس السعر مرة واحدة
    }
    combined = buildBalanced(merged, 0, size - 1);
```

```
49      printf("merged (%d):", size);
50      for (idx = 0; idx < size; idx++) printf(" %d", merged[idx]);
51      printf("\nbalanced tree: root %d, height %d\n", combined->price, height(combined));
52      freeTree(storeA); freeTree(storeB); freeTree(combined);
53      return 0;
54  }
```

نتيجة التنفيذ

```
merged (10): 650 899 1200 2200 3200 4500 12500 15000 32000 89000
balanced tree: root 3200, height 4
```

المخطط الإنسيابي (Organigramme)



تمرين 13.39 المرور الوسطي التكراري بمكدس

علم البيانات متوسط

العودية تستعمل مكدس الاستدعاءات ضمنيًا، وقد يمتلئ في شجرة عميقة جدًا. اكتب المرور الوسطي **تكراريًا** بمكدس صريح من مؤشرات العقد: ننزل يسارًا مع تكديس العقد، ثم نسحب ونزور ونذهب يمينًا. اعرض أقصى حجم بلغه المكدس.

خطوات الحل

1) ما دام `node != NULL`: نكدس وننزل يسارًا.
2) ثم نسحب، نزور، و `node = node->right`.
3) نتوقف عندما يكون المكدس فارغًا و `node == NULL`.

البرنامج بلغة C

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Tree { int price; struct Tree *left, *right; } Tree;
5
6  Tree *insert(Tree *node, int price) {
```

```c
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

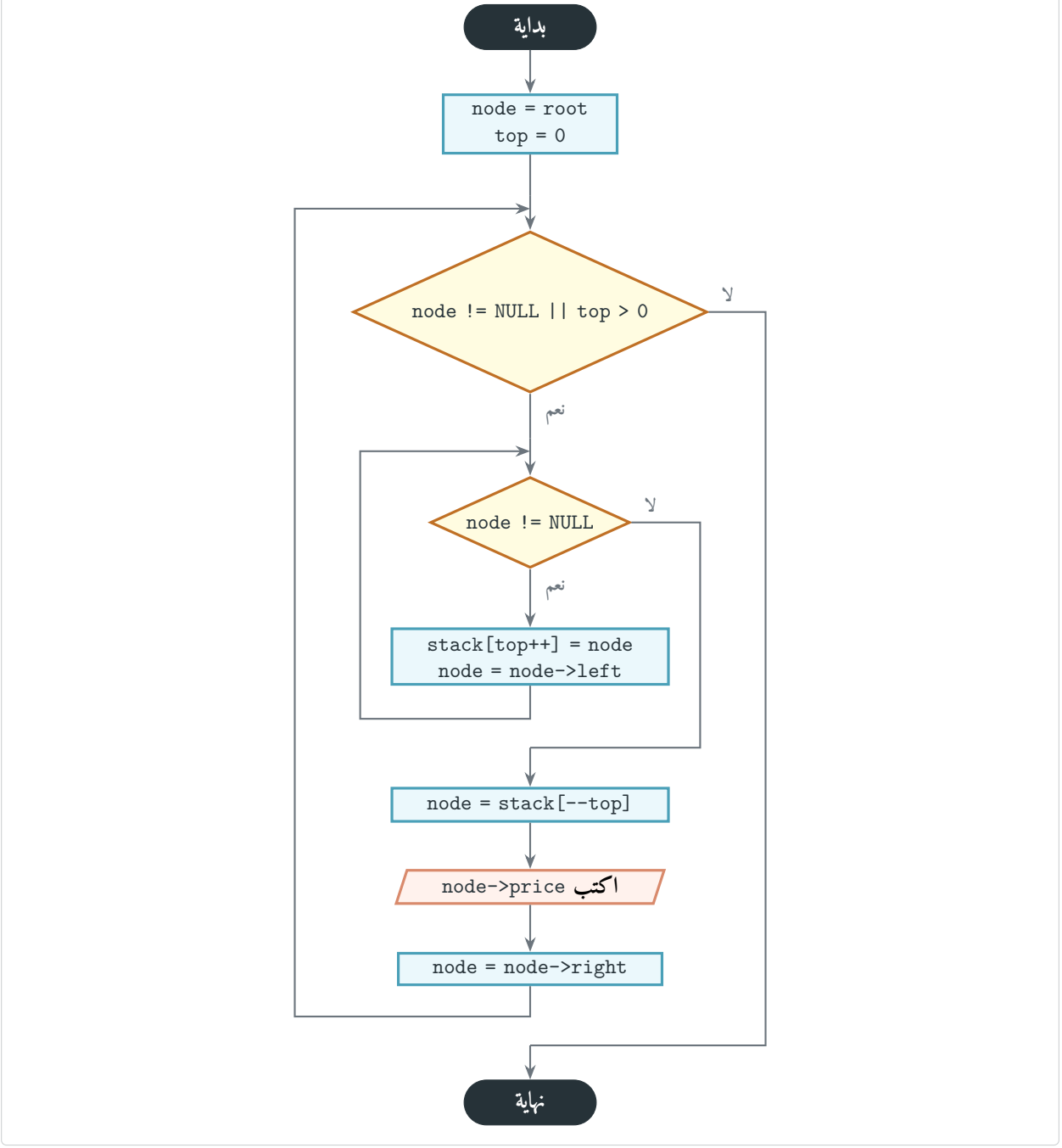
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[10] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 450, 2000, 6800}, idx, top = 0, peak = 0;
    Tree *root = NULL, *node, *stack[20];
    for (idx = 0; idx < 10; idx++) root = insert(root, prices[idx]);
    node = root;
    while (node != NULL || top > 0) {
        while (node != NULL) {                  // ننزل يسارًا
            stack[top++] = node;
            if (top > peak) peak = top;
            node = node->left;
        }
        node = stack[--top];                    // أصغر عقدة لم تُزر
        printf("%d ", node->price);
        node = node->right;
    }
    printf("\nmax stack size: %d\n", peak);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
450 899 1200 2000 3200 4500 6800 12500 32000 89000
max stack size: 4
```

المخطط الإنسيابي (Organigramme)



تمرين 13.40 أقرب قسم نهائي من الجذر

تجارة إلكترونية

متوسط

لتحسين تجربة التصفح نريد معرفة **أقل عدد من النقرات** للوصول من الصفحة الرئيسية إلى قسم نهائي (ورقة). اكتب `minDepth(node)` العودية، مع الانتباه: العقدة ذات ابن واحد **ليست** ورقة، فلا نأخذ الفرع الفارغ. قارن مع الارتفاع (أكبر عدد نقرات).

خطوات الحل

1) ورقة: 1.
2) ابن واحد فقط: `)  (  minDepth(` + 1.

3) ابنان: `1 + min(minDepth(left), minDepth(right))`.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int code; struct Tree *left, *right; } Tree;

Tree *newTree(int code, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->code = code; node->left = left; node->right = right;
    return node;
}

int minDepth(const Tree *node) {
    int leftD, rightD;
    if (node->left == NULL && node->right == NULL) return 1;       // ورقة
    if (node->left == NULL) return 1 + minDepth(node->right);      // لا نأخذ الفرع الفارغ
    if (node->right == NULL) return 1 + minDepth(node->left);
    leftD = minDepth(node->left);
    rightD = minDepth(node->right);
    return 1 + (leftD < rightD ? leftD : rightD);
}

int height(const Tree *node) {
    int leftH, rightH;
    if (node == NULL) return 0;
    leftH = height(node->left); rightH = height(node->right);
    return 1 + (leftH > rightH ? leftH : rightH);
}

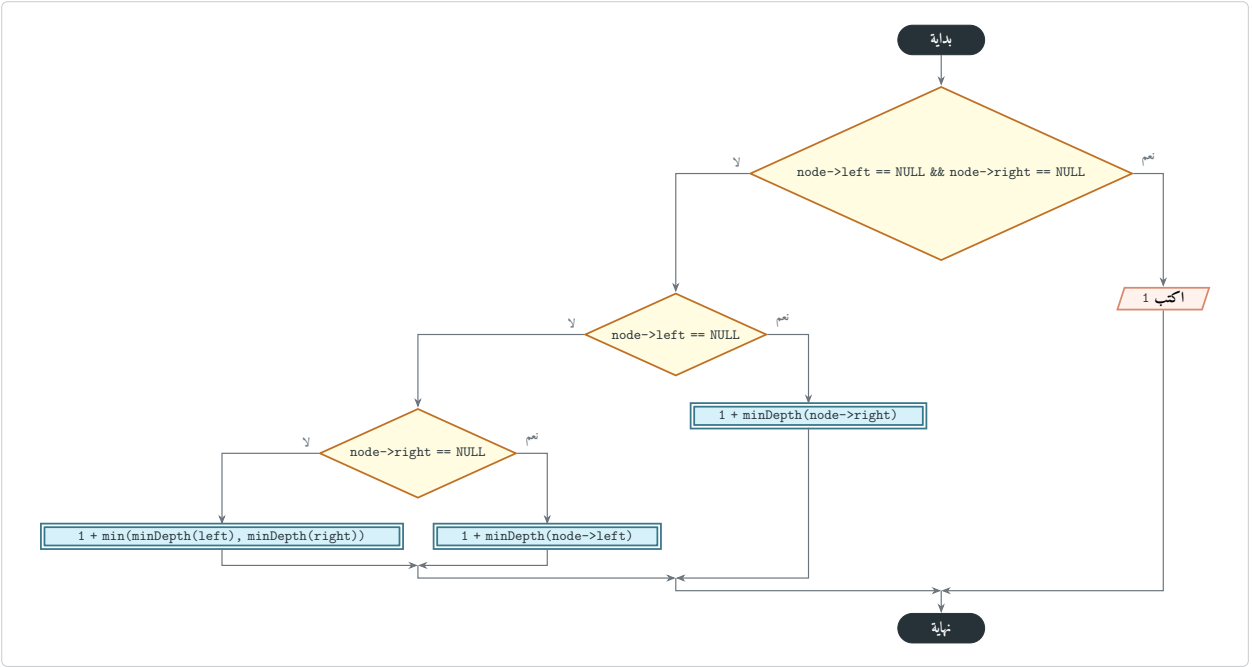
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *root = newTree(1, newTree(2, NULL, newTree(4, newTree(7, NULL, NULL), NULL)),
                            newTree(3, newTree(5, NULL, NULL), newTree(6, newTree(8, newTree(9, NULL, NULL),
      ↪ NULL), NULL)));
    printf("Fewest clicks to a final category: %d levels\n", minDepth(root));
    printf("Most clicks (height): %d levels\n", height(root));
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Fewest clicks to a final category: 3 levels
Most clicks (height): 5 levels
```

المخطط الإنسيابي (Organigramme)



التمارين الصعبة

تمرين 13.41 فهرس أسعار متوازن ذاتيًا AVL

تجارة إلكترونية صعب

الأسعار تصل غالبًا مرتبة (منتجات جديدة أغلى)، فتصبح شجرة البحث العادية سلسلة. اكتب شجرة **AVL**: كل عقدة تحفظ ارتفاعها، وبعد كل إدراج نحسب **معامل التوازن** ونطبّق **الدورانات** (يمين، يسار، يسار-يمين، يمين-يسار) عند الحاجة. أدرج 15 سعرًا مرتبًا وقارن الارتفاع بشجرة عادية.

خطوات الحل

1) `balance = height(left) - height(right)`؛ يجب أن يبقى بين -1 و 1.
2) يسار-يسار: دوران يمين. يمين-يمين: دوران يسار.
3) يسار-يمين: دوران يسار على الابن الأيسر ثم يمين. يمين-يسار: العكس.
4) بعد كل دوران نحدث ارتفاع العقدتين.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price, height; struct Tree *left, *right; } Tree;

int rotations = 0;

int heightOf(const Tree *node) { return node ? node->height : 0; }
void update(Tree *node) { int lh = heightOf(node->left), rh = heightOf(node->right); node->height = 1 + (lh
    ↪ > rh ? lh : rh); }

Tree *rotateRight(Tree *node) {
    Tree *pivot = node->left;
    node->left = pivot->right;
    pivot->right = node;
    update(node); update(pivot);
    rotations++;
    return pivot;
}

Tree *rotateLeft(Tree *node) {
    Tree *pivot = node->right;
    node->right = pivot->left;
    pivot->left = node;
    update(node); update(pivot);
    rotations++;
    return pivot;
}

Tree *insert(Tree *node, int price) {
    int balance;
    if (node == NULL) {
        node = malloc(sizeof(Tree));
        node->price = price; node->height = 1; node->left = node->right = NULL;
        return node;
    }
    if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    update(node);
    balance = heightOf(node->left) - heightOf(node->right);
    if (balance > 1) {                                  // ثقيلة يسارًا
```

```
        if (price > node->left->price) node->left = rotateLeft(node->left);
        return rotateRight(node);
    }
    if (balance < -1) {                                          // ثقيلة يمينًا
        if (price < node->right->price) node->right = rotateRight(node->right);
        return rotateLeft(node);
    }
    return node;
}

Tree *plainInsert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->height = 1; node->left =
      ↪ node->right = NULL; }
    else if (price < node->price) node->left = plainInsert(node->left, price);
    else node->right = plainInsert(node->right, price);
    return node;
}

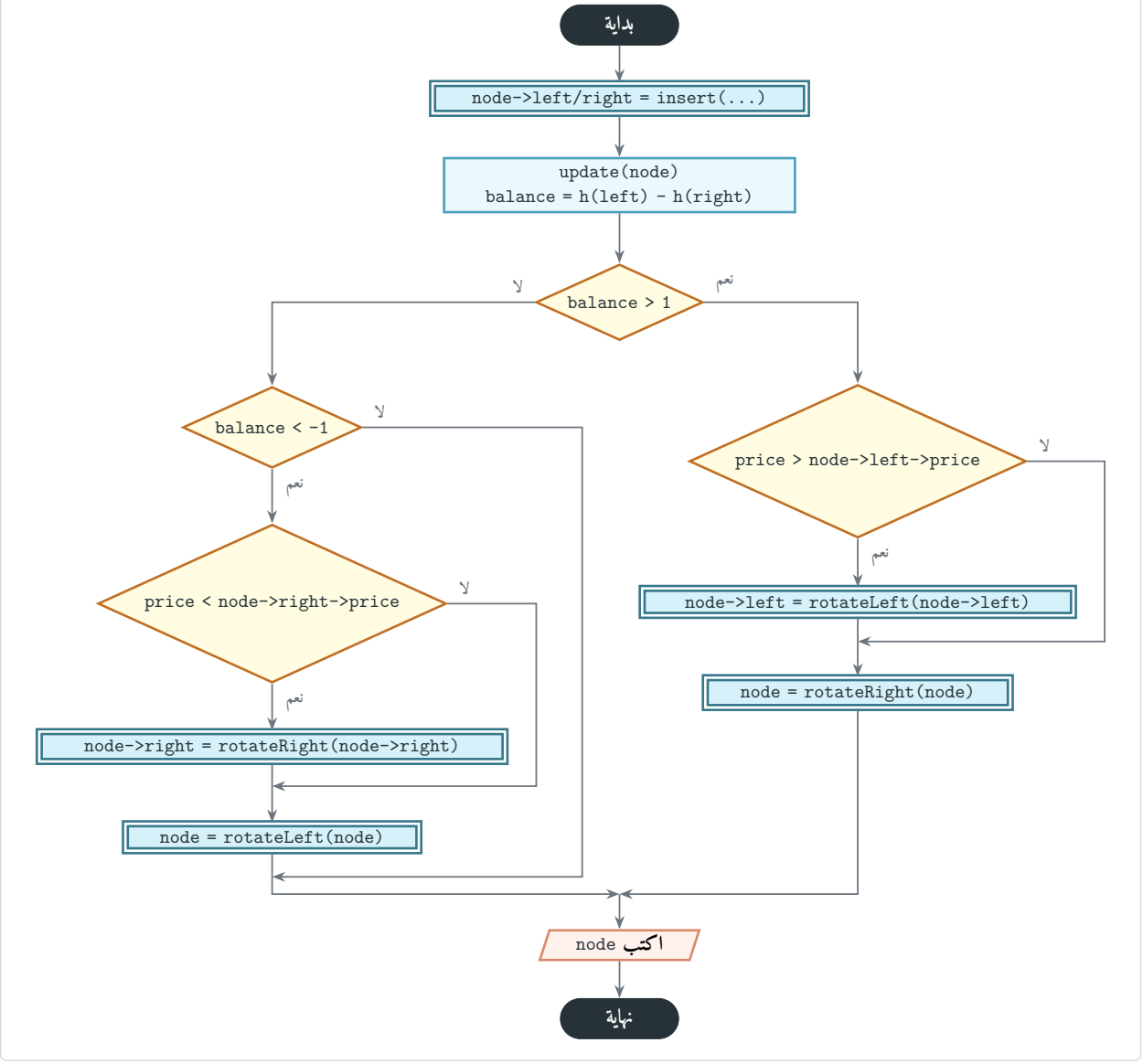
int realHeight(const Tree *node) { int lh, rh; if (!node) return 0; lh = realHeight(node->left); rh =
      ↪ realHeight(node->right); return 1 + (lh > rh ? lh : rh); }
void preorder(const Tree *node) { if (node) { printf("%d ", node->price); preorder(node->left);
      ↪ preorder(node->right); } }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    Tree *avl = NULL, *plain = NULL;
    int idx;
    for (idx = 1; idx <= 15; idx++) {
        avl = insert(avl, idx * 1000);
        plain = plainInsert(plain, idx * 1000);
    }
    printf("15 sorted prices inserted\n");
    printf("plain BST height: %d\n", realHeight(plain));
    printf("AVL height: %d after %d rotations, root %d\n", realHeight(avl), rotations, avl->price);
    printf("AVL preorder: ");
    preorder(avl);
    printf("\n");
    freeTree(avl);
    freeTree(plain);
    return 0;
}
```

نتيجة التنفيذ

```
15 sorted prices inserted
plain BST height: 15
AVL height: 4 after 11 rotations, root 8000
AVL preorder: 8000 4000 2000 1000 3000 6000 5000 7000 12000 10000 9000 11000 14000 13000
    15000
```

المخطط الإنسيابي (Organigramme)



تمرين 13.42 ترميز هوفمان لرموز الحالات

علم البيانات

صعب

سجلات الطلبيات تخزن حالة كل طلبية بحرف (D مسلّمة، S مشحونة، P مدفوعة، C ملغاة، R مرتجعة) بتكرارات مختلفة جدًا. ترميز هوفمان يعطي الحروف المتكررة رموزًا ثنائية أقصر. ابنِ شجرة هوفمان (دمج أصغر شجرتين وزنًا بشكل متكرر)، واستخرج الرموز، واحسب نسبة الضغط مقارنة بـ 3 بتات ثابتة.

خطوات الحل

1) في البداية كل حرف ورقة بوزن تكراره.
2) نكرر: نختار أصغر شجرتين، ننشئ عقدة أب وزنها مجموع وزنيهما، حتى تبقى شجرة واحدة.
3) رمز الحرف: المسار من الجذر (يسار = 0، يمين = 1).

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char symbol; long weight; struct Tree *left, *right; } Tree;

long totalBits = 0;

Tree *newTree(char symbol, long weight, Tree *left, Tree *right) {
    Tree *node = malloc(sizeof(Tree));
    node->symbol = symbol; node->weight = weight; node->left = left; node->right = right;
    return node;
}

void printCodes(const Tree *node, char *code, int depth) {
    if (node->left == NULL) {                               // ورقة
        code[depth] = '\0';
        printf("  %c  weight %5ld  code %-5s (%d bits)\n", node->symbol, node->weight, code, depth);
        totalBits += node->weight * depth;
        return;
    }
    code[depth] = '0'; printCodes(node->left, code, depth + 1);
    code[depth] = '1'; printCodes(node->right, code, depth + 1);
}

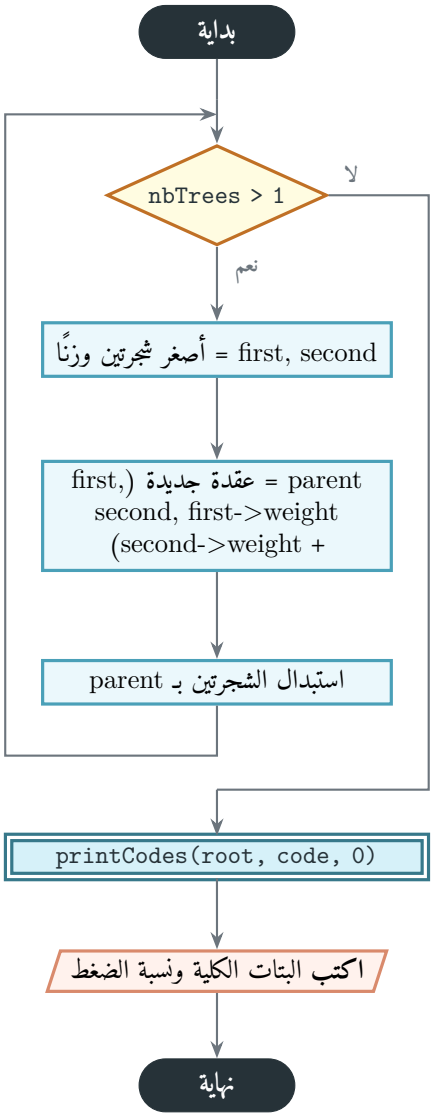
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    char symbols[5] = {'D', 'S', 'P', 'C', 'R'};
    long counts[5] = {6200, 1800, 1300, 450, 250}, total = 0;
    Tree *forest[5];
    char code[10];
    int nbTrees = 5, idx, first, second, tmp;
    for (idx = 0; idx < 5; idx++) { forest[idx] = newTree(symbols[idx], counts[idx], NULL, NULL); total +=
      ↪ counts[idx]; }
    while (nbTrees > 1) {
        first = 0; second = 1;
        if (forest[second]->weight < forest[first]->weight) { tmp = first; first = second; second = tmp; }
        for (idx = 2; idx < nbTrees; idx++) {               // أصغر وزنين
            if (forest[idx]->weight < forest[first]->weight) { second = first; first = idx; }
            else if (forest[idx]->weight < forest[second]->weight) second = idx;
        }
        forest[first] = newTree('*', forest[first]->weight + forest[second]->weight, forest[first],
      ↪ forest[second]);
        forest[second] = forest[--nbTrees];                 // نزيل الثانية
    }
    printCodes(forest[0], code, 0);
    printf("Huffman: %ld bits, fixed 3-bit: %ld bits\n", totalBits, total * 3);
    printf("Average %.3f bits/symbol, saving %.1f%%\n", (double)totalBits / total,
           100.0 * (1 - (double)totalBits / (total * 3)));
    freeTree(forest[0]);
    return 0;
}
```

نتيجة التنفيذ

```
  S  weight  1800  code 00    (2 bits)
  R  weight   250  code 0100  (4 bits)
  C  weight   450  code 0101  (4 bits)
  P  weight  1300  code 011   (3 bits)
  D  weight  6200  code 1     (1 bits)
Huffman: 16500 bits, fixed 3-bit: 30000 bits
Average 1.650 bits/symbol, saving 45.0%
```

المخطط الإنسيابي (Organigramme)



تمرين 13.43 الإكمال التلقائي لأسماء المنتجات بشجرة البادئات

تجارة إلكترونية صعب ★★★

مربع البحث يقترح الأسماء أثناء الكتابة. **شجرة البادئات** (Trie): كل عقدة تمثل حرفًا ولها حتى 26 ابنًا؛ المسار من الجذر يكوّن بادئة، والعقدة تحمل علامة "نهاية اسم" وشعبيته. اكتب `insertWord` و `suggest(prefix)` التي تعرض كل الأسماء المكملة للبادئة مرتبة أبجديًا، مع الأكثر شعبية.

خطوات الحل

1) الإدراج: لكل حرف ننزل إلى الابن `ch - 'a'` (وننشئه إن لم يوجد).
2) الاقتراح: ننزل حسب البادئة، ثم نمر عمقًا أولًا على كل الفرع ونطبع الكلمات المكتملة.
3) الترتيب الأبجدي تلقائي لأن الأبناء مرتبون حسب الحرف.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Trie { struct Trie *child[26]; int isWord, popularity; } Trie;

Trie *newTrie(void) {
    Trie *node = malloc(sizeof(Trie));
    int idx;
    for (idx = 0; idx < 26; idx++) node->child[idx] = NULL;
    node->isWord = 0;
    node->popularity = 0;
    return node;
}

void insertWord(Trie *root, const char *word, int popularity) {
    for (; *word != '\0'; word++) {
        if (root->child[*word - 'a'] == NULL) root->child[*word - 'a'] = newTrie();
        root = root->child[*word - 'a'];                // حرف واحد في كل مستوى //
    }
    root->isWord = 1;
    root->popularity = popularity;
}

char bestWord[32];
int bestScore;

void collect(const Trie *node, char *buffer, int depth) {
    int idx, pos;
    if (node->isWord) {
        buffer[depth] = '\0';
        printf(" %s", buffer);
        if (node->popularity > bestScore) {
            bestScore = node->popularity;
            for (pos = 0; (bestWord[pos] = buffer[pos]) != '\0'; pos++) {}
        }
    }
    for (idx = 0; idx < 26; idx++)
        if (node->child[idx] != NULL) {
            buffer[depth] = (char)('a' + idx);
            collect(node->child[idx], buffer, depth + 1);
        }
}

void freeTrie(Trie *node) { int idx; for (idx = 0; idx < 26; idx++) if (node->child[idx])
      ↪ freeTrie(node->child[idx]); free(node); }

int main(void) {
    const char *words[10] = {"mouse", "monitor", "modem", "mousepad", "charger", "cable",
                             "chair", "camera", "microphone", "motherboard"};
    int popularity[10] = {950, 720, 130, 410, 880, 990, 300, 560, 610, 240};
    Trie *root = newTrie(), *node;
    char prefix[32], buffer[32];
    int idx, nbQueries, query, depth;
    for (idx = 0; idx < 10; idx++) insertWord(root, words[idx], popularity[idx]);
    scanf("%d", &nbQueries);
    for (query = 0; query < nbQueries; query++) {
        scanf("%31s", prefix);
        for (node = root, depth = 0; node != NULL && prefix[depth] != '\0'; depth++) {
            buffer[depth] = prefix[depth];
            node = node->child[prefix[depth] - 'a'];
        }
        printf("\"%s\" ->", prefix);
        if (node == NULL) { printf(" no suggestion\n"); continue; }
        bestScore = -1;
        collect(node, buffer, depth);
        printf("\n   top: %s (%d)\n", bestWord, bestScore);
    }
    freeTrie(root);
    return 0;
}
```

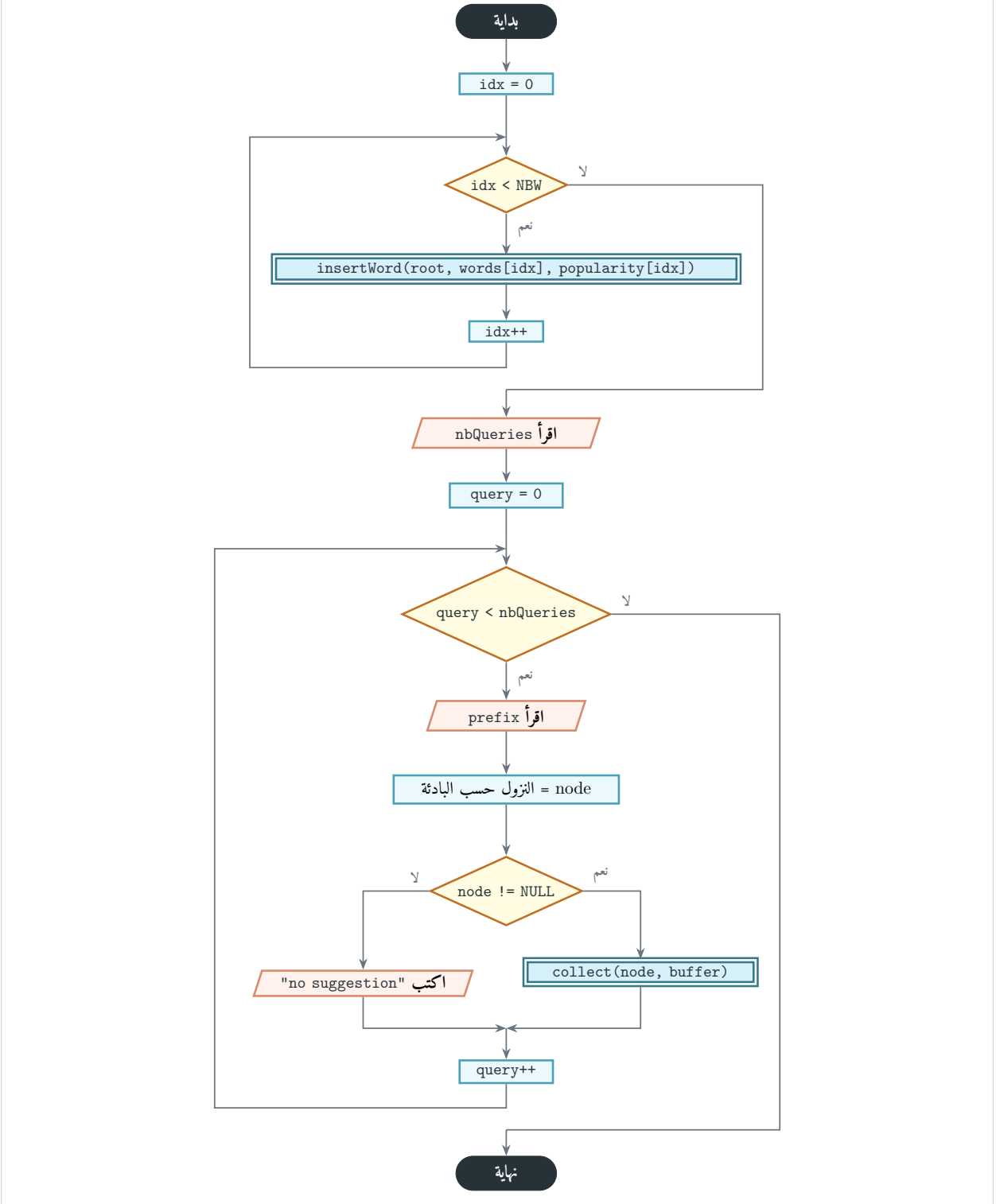
المدخلات

```
3 mo ch xy
```

نتيجة التنفيذ

```
"mo" -> modem monitor motherboard mouse mousepad
   top: mouse (950)
"ch" -> chair charger
   top: charger (880)
"xy" -> no suggestion
```

المخطط الإنسيابي (Organigramme)



تمرين 13.44 منتجان مجموع سعريهما يساوي القسيمة

تجارة إلكترونية صعب

قسيمة هدية بقيمة ثابتة؛ نبحث عن **منتجين** مجموع سعريهما يساويها بالضبط. أسعار الكتالوج في شجرة بحث. بدل نسخها إلى جدول، استعمل **مكررين** على الشجرة: أحدهما يعطي الأسعار تصاعديًا والآخر تنازليًا (كل منهما بمكدس)، وطبّق تقنية **المؤشرين** المتقاربين.

خطوات الحل

1) المكرر التصاعدي: مكدس يُملأ بالنزول يسارًا، `next` يسحب ثم ينزل يسار الفرع الأيمن.
2) التنازلي: العكس (النزول يمينًا).
3) إذا كان المجموع أصغر من الهدف نتقدم بالتصاعدي، أكبر: بالتنازلي، مساوٍ: وجدنا.

</> البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;
typedef struct { Tree *items[32]; int top, forward; } Iterator;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

void pushSide(Iterator *iter, Tree *node) {
    while (node != NULL) {
        iter->items[iter->top++] = node;
        node = iter->forward ? node->left : node->right;      // النزول إلى الطرف
    }
}

void startIter(Iterator *iter, Tree *root, int forward) { iter->top = 0; iter->forward = forward;
    ↪ pushSide(iter, root); }

int nextValue(Iterator *iter) {
    Tree *node = iter->items[--iter->top];
    pushSide(iter, iter->forward ? node->right : node->left);
    return node->price;
}

int main(void) {
    int prices[10] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 2900, 6800, 450};
    Tree *root = NULL;
    Iterator ascending, descending;
    int idx, nbQueries, target, low, high, steps, found;
    for (idx = 0; idx < 10; idx++) root = insert(root, prices[idx]);
    scanf("%d", &nbQueries);
    for (idx = 0; idx < nbQueries; idx++) {
        scanf("%d", &target);
        startIter(&ascending, root, 1);
        startIter(&descending, root, 0);
        low = nextValue(&ascending);
        high = nextValue(&descending);
        found = 0;
        steps = 0;
        while (low < high && !found) {
            steps++;
            if (low + high == target) found = 1;
            else if (low + high < target) low = nextValue(&ascending);   // نحتاج أكثر
            else high = nextValue(&descending);                          // نحتاج أقل
        }
        if (found) printf("%6d DA = %d + %d (%d steps)\n", target, low, high, steps);
        else printf("%6d DA: no pair\n", target);
    }
    while (root != NULL) {                                   // تحرير تكراري بسيط
        Tree *node = root;
        if (node->left) { root = node->left; node->left = root->right; root->right = node; }
        else { root = node->right; free(node); }
    }
    return 0;
}
```

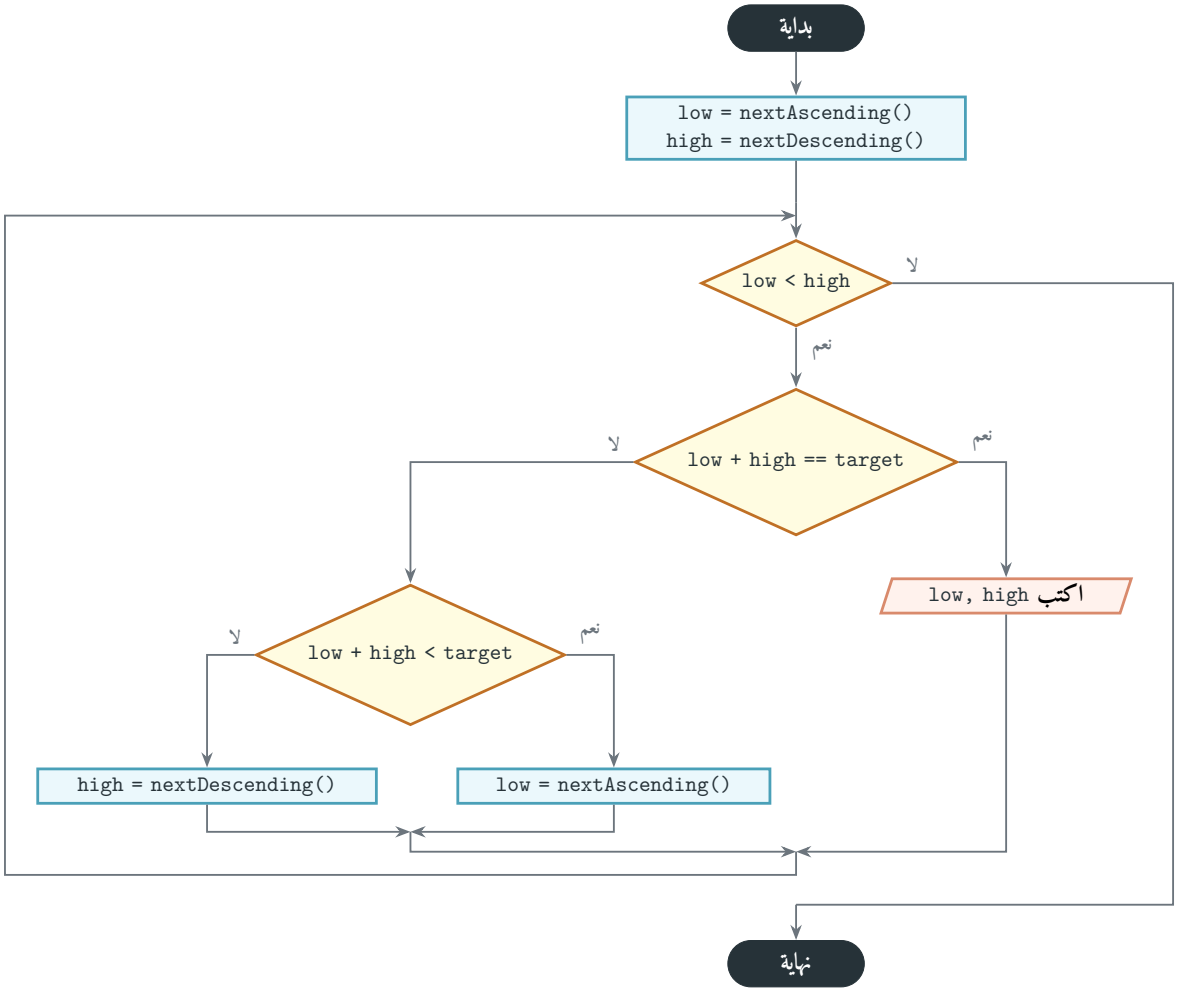
المدخلات

```
3 7400 33200 100
```

نتيجة التنفيذ

```
  7400 DA = 2900 + 4500 (8 steps)
 33200 DA = 1200 + 32000 (4 steps)
   100 DA: no pair
```

المخطط الإنسيابي (Organigramme)



تمرين 13.45 شجرة المقاطع لأدنى سعر في فترة

علم البيانات صعب

لدينا سعر منتج لكل يوم من 16 يومًا، ونريد الإجابة بسرعة عن "ما أدنى سعر بين اليوم a واليوم b؟" مع تحديث الأسعار أحيانًا. **شجرة المقاطع**: كل عقدة تحفظ أدنى قيمة في مجالها؛ الاستعلام والتحديث بتعقيد `log n`. نخزنها في جدول (node، أبناؤها 2node و 2node+1).

خطوات الحل

1) البناء: الأوراق = الأيام، وكل عقدة = أدنى ابنيها.
2) الاستعلام: إذا كان مجال العقدة داخل `[a, b]` نعيد قيمتها؛ خارجه نعيد ∞؛ وإلا ننزل في الابنين.

3) التحديث: نعدل الورقة ثم نعيد حساب أسلافها.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBD 16
#define INF 1000000000

int prices[NBD] = {320, 335, 310, 345, 300, 290, 315, 280, 295, 305, 330, 285, 300, 310, 325, 340};
int tree[4 * NBD], visits;

int minOf(int first, int second) { return first < second ? first : second; }

void build(int node, int low, int high) {
    int middle = (low + high) / 2;
    if (low == high) { tree[node] = prices[low]; return; }          // ورقة = يوم
    build(2 * node, low, middle);
    build(2 * node + 1, middle + 1, high);
    tree[node] = minOf(tree[2 * node], tree[2 * node + 1]);
}

int query(int node, int low, int high, int from, int to) {
    int middle = (low + high) / 2;
    visits++;
    if (to < low || high < from) return INF;                        // خارج المجال
    if (from <= low && high <= to) return tree[node];               // داخل المجال كليًا
    return minOf(query(2 * node, low, middle, from, to), query(2 * node + 1, middle + 1, high, from, to));
}

void update(int node, int low, int high, int day, int value) {
    int middle = (low + high) / 2;
    if (low == high) { tree[node] = value; return; }
    if (day <= middle) update(2 * node, low, middle, day, value);
    else update(2 * node + 1, middle + 1, high, day, value);
    tree[node] = minOf(tree[2 * node], tree[2 * node + 1]);         // نحدث الأسلاف
}

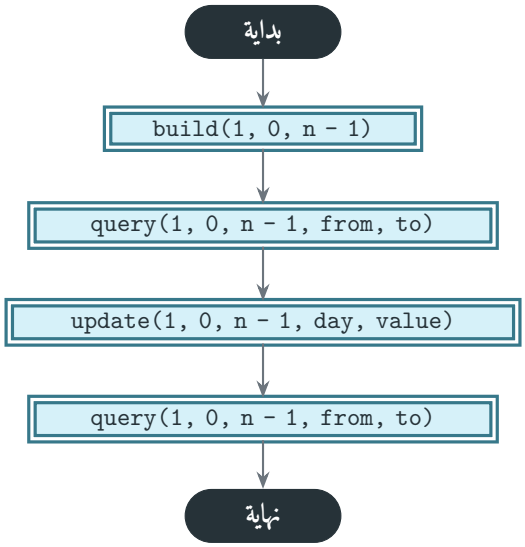
void ask(int from, int to) {
    int result;
    visits = 0;
    result = query(1, 0, NBD - 1, from, to);
    printf("min price days %2d-%2d: %d (visited %d nodes)\n", from + 1, to + 1, result, visits);
}

int main(void) {
    build(1, 0, NBD - 1);
    ask(0, 15);
    ask(0, 4);
    ask(8, 13);
    update(1, 0, NBD - 1, 10, 260);
    printf("flash sale: day 11 -> 260\n");
    ask(8, 13);
    ask(2, 9);
    return 0;
}
```

نتيجة التنفيذ

```
min price days  1-16: 280 (visited 1 nodes)
min price days  1- 5: 300 (visited 9 nodes)
min price days  9-14: 285 (visited 7 nodes)
flash sale: day 11 -> 260
min price days  9-14: 260 (visited 7 nodes)
min price days  3-10: 280 (visited 11 nodes)
```

المخطط الإنسيابي (Organigramme)



تمرين 13.46 تعلم شجرة قرار عوديًا

علم البيانات صعب

بدل كتابة شجرة القرار يدويًا، **تعلمها** من البيانات: لكل عقدة نختار الخاصية والعتبة اللتين تعطيان أقل **مؤشر جيني** مرجح، ثم نقسم البيانات ونبني الفرعين **عوديًا**، حتى تصبح العقدة نقية أو نبلغ العمق الأقصى 3. اعرض الشجرة المتعلمة ودقتها.

خطوات الحل

1) العقدة تستقبل جدول فهارس العينات التي تصلها.
2) `bestSplit`: كل خاصية × كل قيمة كعتبة (`<=` يسارًا).
3) التوقف: كل العينات من نفس الصنف، أو العمق = 3، أو لا يوجد تقسيم يحسن.
4) الورقة تحمل صنف الأغلبية.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBS 14
#define MAXD 3

typedef struct Tree { int feature, label; double threshold; struct Tree *left, *right; } Tree;

double data[NBS][2] = {{2, 0}, {3, 1}, {8, 0}, {9, 2}, {4, 3}, {10, 1}, {1, 1}, {7, 3},
                       {6, 0}, {5, 2}, {9, 0}, {3, 2}, {8, 3}, {2, 3}};      // (pages, cart)
int labels[NBS] = {0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1};
const char *names[2] = {"pages", "cart"};

double gini(const int *samples, int count) {
    int idx, buyers = 0;
    double ratio;
    if (count == 0) return 0;
    for (idx = 0; idx < count; idx++) buyers += labels[samples[idx]];
    ratio = (double)buyers / count;
    return 2 * ratio * (1 - ratio);
}

Tree *grow(const int *samples, int count, int depth) {
```

```c
    Tree *node = malloc(sizeof(Tree));
    int left[NBS], right[NBS], nLeft, nRight, feat, cand, idx, buyers = 0;
    double score, bestScore = gini(samples, count), threshold;
    for (idx = 0; idx < count; idx++) buyers += labels[samples[idx]];
    node->label = buyers * 2 >= count;                          // صنف الأغلبية
    node->feature = -1;
    node->left = node->right = NULL;
    if (buyers == 0 || buyers == count || depth == MAXD) return node;
    for (feat = 0; feat < 2; feat++)
        for (cand = 0; cand < count; cand++) {
            threshold = data[samples[cand]][feat];
            for (nLeft = nRight = 0, idx = 0; idx < count; idx++)
                if (data[samples[idx]][feat] <= threshold) left[nLeft++] = samples[idx]; else
      ↪ right[nRight++] = samples[idx];
            if (nLeft == 0 || nRight == 0) continue;
            score = (nLeft * gini(left, nLeft) + nRight * gini(right, nRight)) / count;
            if (score < bestScore - 1e-12) { bestScore = score; node->feature = feat; node->threshold =
      ↪ threshold; }
        }
    if (node->feature < 0) return node;                         // لا تحسين
    for (nLeft = nRight = 0, idx = 0; idx < count; idx++)
        if (data[samples[idx]][node->feature] <= node->threshold) left[nLeft++] = samples[idx]; else
      ↪ right[nRight++] = samples[idx];
    node->left = grow(left, nLeft, depth + 1);                  // بناء عودي
    node->right = grow(right, nRight, depth + 1);
    return node;
}

void show(const Tree *node, int depth) {
    if (node->feature < 0) { printf("%*s-> %s\n", 2 * depth, "", node->label ? "BUY" : "no buy"); return; }
    printf("%*sif %s <= %.0f:\n", 2 * depth, "", names[node->feature], node->threshold);
    show(node->left, depth + 1);
    printf("%*selse:\n", 2 * depth, "");
    show(node->right, depth + 1);
}

int predict(const Tree *node, const double *row) {
    while (node->feature >= 0) node = row[node->feature] <= node->threshold ? node->left : node->right;
    return node->label;
}

void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

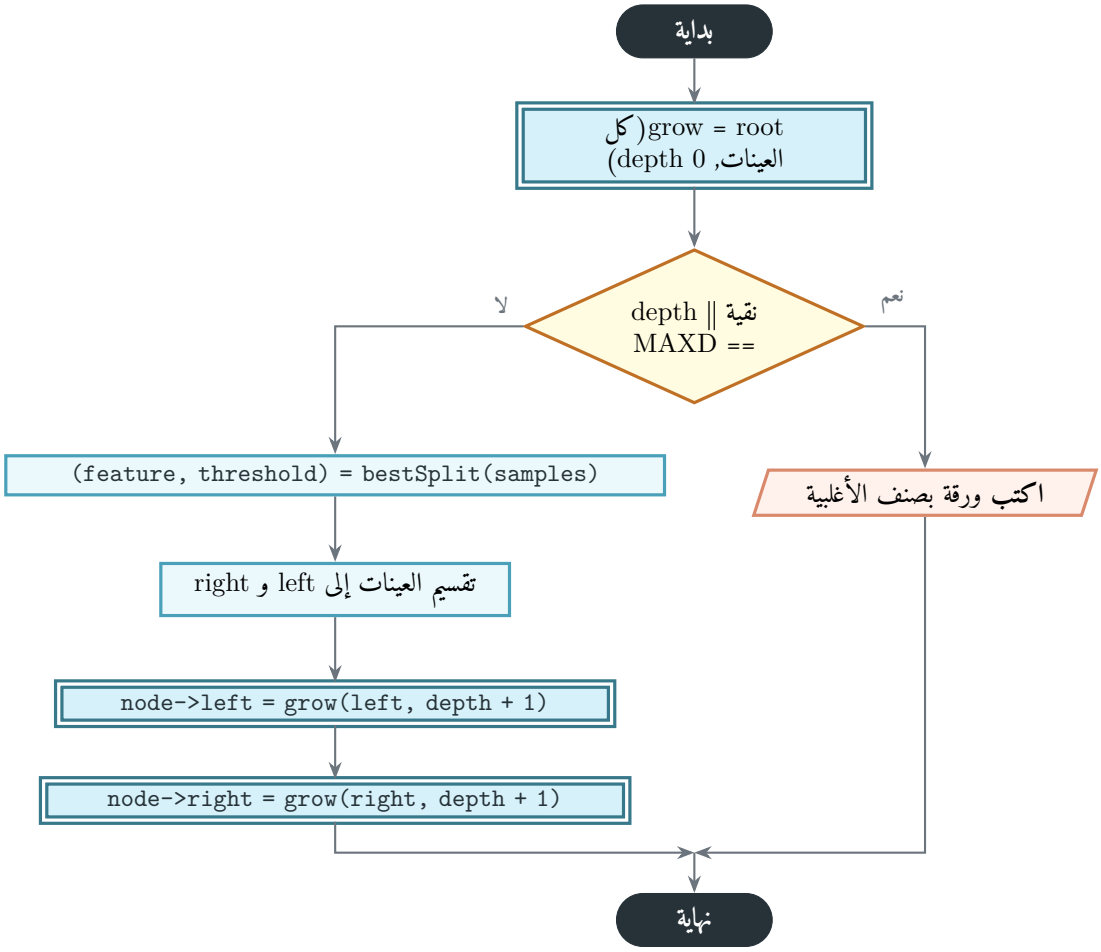
int main(void) {
    int samples[NBS], idx, correct = 0;
    Tree *root;
    for (idx = 0; idx < NBS; idx++) samples[idx] = idx;
    root = grow(samples, NBS, 0);
    show(root, 0);
    for (idx = 0; idx < NBS; idx++) correct += predict(root, data[idx]) == labels[idx];
    printf("training accuracy: %d/%d\n", correct, NBS);
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
if pages <= 3:
  if cart <= 2:
    -> no buy
  else:
    -> BUY
else:
  if pages <= 6:
    if pages <= 5:
      -> BUY
    else:
      -> no buy
  else:
```

```
    -> BUY
training accuracy: 14/14
```

المخطط الإنسيابي (Organigramme)



تمرين 13.47 أقرب مستودع بشجرة k-d

تجارة إلكترونية صعب

لكل طلبية نبحث عن **أقرب مستودع** من بين عدة مستودعات. **شجرة k-d** ثنائية البعد تقسم المستوى بالتناوب حسب x ثم y. اكتب `build` (الوسيط في كل مستوى) و `nearest` العودية مع **التقليم**: لا ننزل في الجانب الآخر إلا إذا كان خط القسمة أقرب من أفضل مسافة وجدناها. عدّ العقد المزارة.

خطوات الحل

1) البناء: نرتب النقاط حسب المحور `depth % 2` ونأخذ الوسيط جذرًا.
2) البحث: ننزل أولًا في الجانب الذي يحتوي على نقطة الطلب.
3) عند العودة: إذا `(diff)² < bestDist` نستكشف الجانب الآخر.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

#define NBW 10
```

```c

typedef struct KdNode { double point[2]; char name[6]; struct KdNode *left, *right; } KdNode;

KdNode warehouses[NBW] = {{{2, 3}, "W1", 0, 0}, {{5, 4}, "W2", 0, 0}, {{9, 6}, "W3", 0, 0}, {{4, 7}, "W4",
      ↪ 0, 0},
                        {{8, 1}, "W5", 0, 0}, {{7, 2}, "W6", 0, 0}, {{1, 9}, "W7", 0, 0}, {{6, 8}, "W8",
      ↪ 0, 0},
                        {{3, 1}, "W9", 0, 0}, {{10, 10}, "W10", 0, 0}};
KdNode *view[NBW];
const KdNode *best;
double bestDist;
int visited;

KdNode *build(int low, int high, int depth) {
    int axis = depth % 2, idx, back, middle;
    KdNode *key;
    if (low > high) return NULL;
    for (idx = low + 1; idx <= high; idx++) {                    // ترتيب حسب المحور
        key = view[idx];
        for (back = idx - 1; back >= low && view[back]->point[axis] > key->point[axis]; back--) view[back +
      ↪ 1] = view[back];
        view[back + 1] = key;
    }
    middle = (low + high) / 2;
    view[middle]->left = build(low, middle - 1, depth + 1);
    view[middle]->right = build(middle + 1, high, depth + 1);
    return view[middle];
}

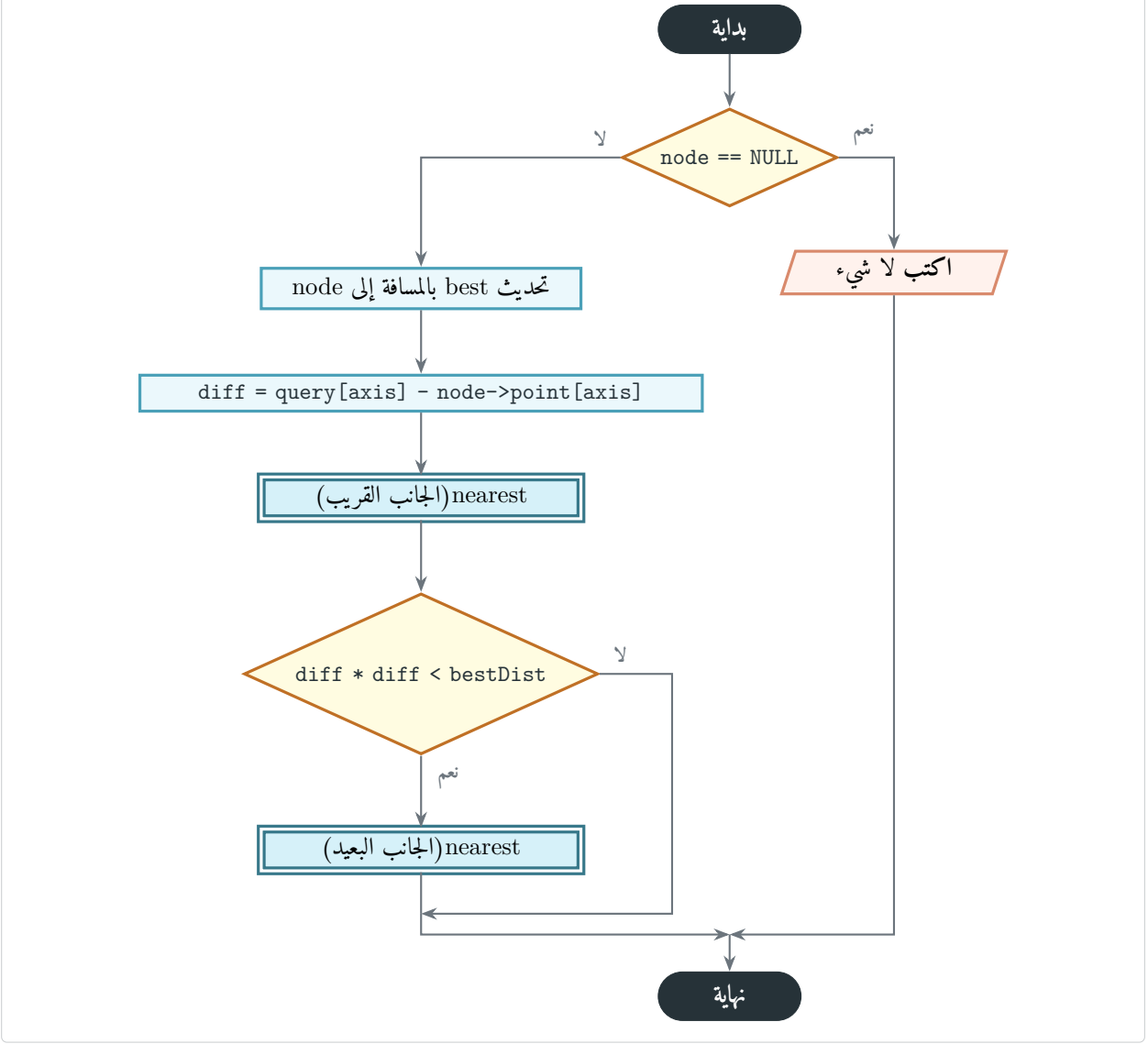
void nearest(const KdNode *node, const double *query, int depth) {
    double dx, dy, dist, diff;
    int axis = depth % 2;
    if (node == NULL) return;
    visited++;
    dx = node->point[0] - query[0];
    dy = node->point[1] - query[1];
    dist = dx * dx + dy * dy;
    if (dist < bestDist) { bestDist = dist; best = node; }
    diff = query[axis] - node->point[axis];
    nearest(diff <= 0 ? node->left : node->right, query, depth + 1);   // الجانب القريب
    if (diff * diff < bestDist)                                        // التقليم
        nearest(diff <= 0 ? node->right : node->left, query, depth + 1);
}

int main(void) {
    double orders[3][2] = {{6, 3}, {2, 8}, {9, 9}};
    KdNode *root;
    int idx;
    for (idx = 0; idx < NBW; idx++) view[idx] = &warehouses[idx];
    root = build(0, NBW - 1, 0);
    printf("root %s (%.0f,%.0f)\n", root->name, root->point[0], root->point[1]);
    for (idx = 0; idx < 3; idx++) {
        bestDist = 1e18;
        visited = 0;
        nearest(root, orders[idx], 0);
        printf("order at (%.0f,%.0f) -> %s, dist^2 %.0f, visited %d/%d\n", orders[idx][0], orders[idx][1],
               best->name, bestDist, visited, NBW);
    }
    return 0;
}
```

نتيجة التنفيذ

```
root W2 (5,4)
order at (6,3) -> W2, dist^2 2, visited 8/10
order at (2,8) -> W7, dist^2 2, visited 4/10
order at (9,9) -> W10, dist^2 2, visited 4/10
```

المخطط الإنسيابي (Organigramme)



تمرين 13.48 مرور موريس دون مكدس

علم البيانات صعب

على جهاز مدمج بذاكرة محدودة جدًا، نريد المرور الوسطي على شجرة الأسعار **دون عودية ودون مكدس.** طريقة **موريس** تنشئ روابط مؤقتة: لكل عقدة لها فرع أيسر، نجعل أقصى عقدة يمين ذلك الفرع (السلف الوسطي) تشير إليها، ثم نزيل الرابط عند العودة. تحقق أن الشجرة تعود كما كانت.

خطوات الحل

1) إذا لم يكن للعقدة ابن أيسر: نزورها ونذهب يمينًا.
2) وإلا نجد السلف `pred` (أقصى يمين الفرع الأيسر):
3) إذا كان `pred->right == NULL`: نضع الرابط المؤقت `pred->right = node` وننزل يسارًا.
4) إذا كان يشير إلى `node`: نزيل الرابط، نزور `node`، ونذهب يمينًا.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { int price; struct Tree *left, *right; } Tree;

Tree *insert(Tree *node, int price) {
    if (node == NULL) { node = malloc(sizeof(Tree)); node->price = price; node->left = node->right = NULL; }
    else if (price < node->price) node->left = insert(node->left, price);
    else node->right = insert(node->right, price);
    return node;
}

int countThreads = 0;

void morris(Tree *root) {
    Tree *node = root, *pred;
    while (node != NULL) {
        if (node->left == NULL) {
            printf("%d ", node->price);
            node = node->right;                          // قد يكون رابطًا مؤقتًا
        } else {
            for (pred = node->left; pred->right != NULL && pred->right != node; pred = pred->right) {}
            if (pred->right == NULL) {
                pred->right = node;                      // رابط مؤقت للعودة
                countThreads++;
                node = node->left;
            } else {
                pred->right = NULL;                      // نزيل الرابط
                printf("%d ", node->price);
                node = node->right;
            }
        }
    }
    printf("\n");
}

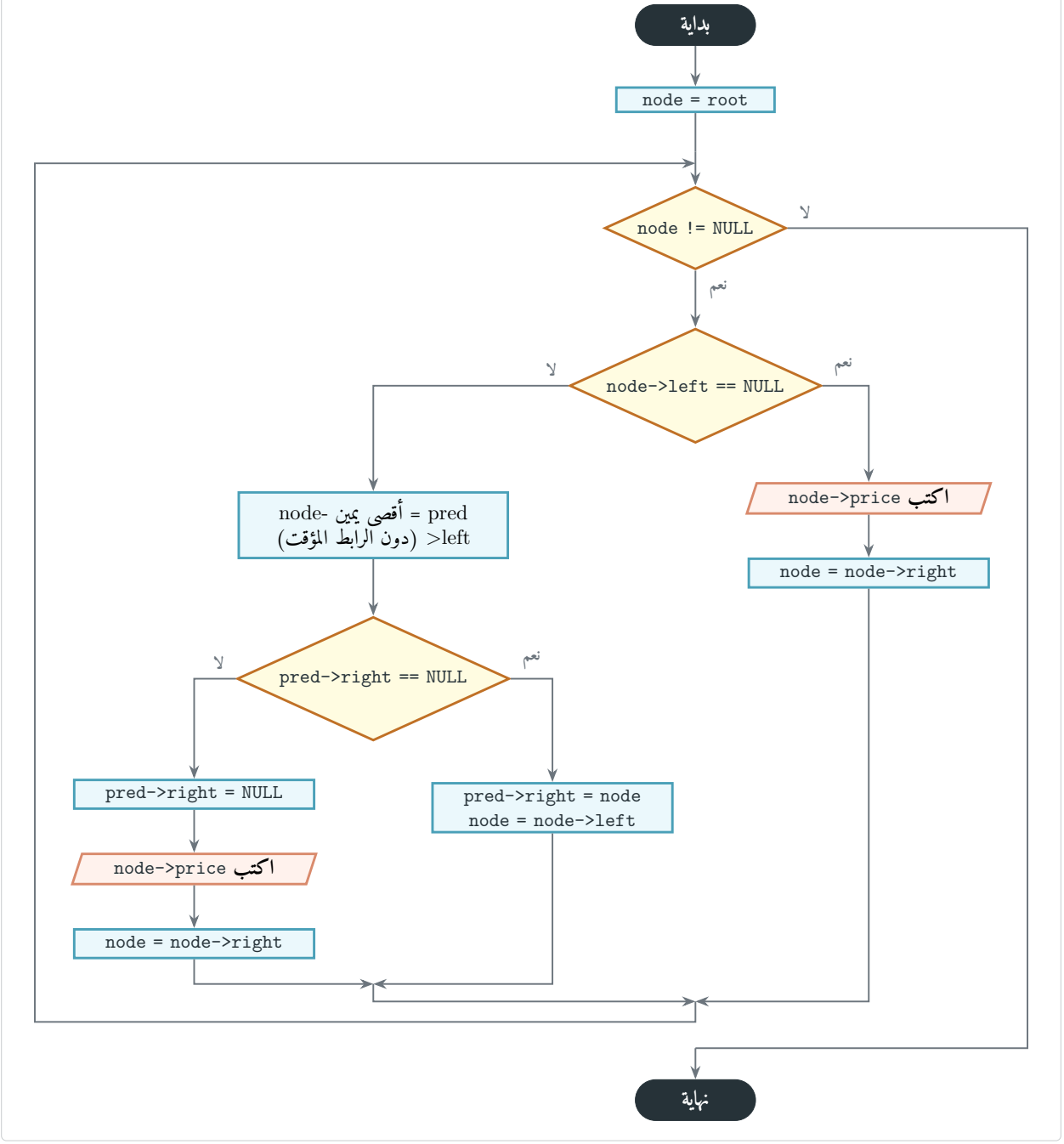
void inorder(const Tree *node) { if (node) { inorder(node->left); printf("%d ", node->price);
    ↪ inorder(node->right); } }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    int prices[10] = {4500, 1200, 32000, 899, 3200, 12500, 89000, 450, 2000, 6800}, idx;
    Tree *root = NULL;
    for (idx = 0; idx < 10; idx++) root = insert(root, prices[idx]);
    printf("Morris:    ");
    morris(root);
    printf("temporary links created and removed: %d\n", countThreads);
    printf("recursive: ");
    inorder(root);                                       // الشجرة سليمة
    printf("\n");
    freeTree(root);
    return 0;
}
```

نتيجة التنفيذ

```
Morris:    450 899 1200 2000 3200 4500 6800 12500 32000 89000
temporary links created and removed: 6
recursive: 450 899 1200 2000 3200 4500 6800 12500 32000 89000
```

المخطط الإنسيابي (Organigramme)



تمرين 13.49 شجرة فينويك للمبيعات التراكمية

علم البيانات صعب ★★★

لوحة القيادة تعرض المبيعات التراكمية حتى أي يوم، والمبيعات تُحدّث باستمرار (مرتجعات، طلبيات متأخرة). **شجرة فينويك** (Binary Indexed Tree) تجعل التحديث والاستعلام عن مجموع البادئة بتعقيد `log n`، بالاعتماد على **أدنى بت مرفوع** `idx & -idx`. قارن عدد العمليات بالطريقة البسيطة.

خطوات الحل

1) الخانة `tree[idx]` تجمع المجال `(idx - lowbit(idx), idx]` (الفهرسة من 1).

2) `add(idx, delta)`:

```
            while (idx <= n) { tree[idx] += delta; idx += idx & -idx; }
```

.

3) `prefix(idx)`:

```
            while (idx > 0) { sum += tree[idx]; idx -= idx & -idx; }
```

.

4) مجموع `[a, b] = prefix(b) - prefix(a - 1)`.

البرنامج بلغة C

```c
#include <stdio.h>

#define NBD 30

long tree[NBD + 1];
int operations = 0;

void add(int idx, long delta) {
    for (; idx <= NBD; idx += idx & -idx) { tree[idx] += delta; operations++; }   // نصعد
}

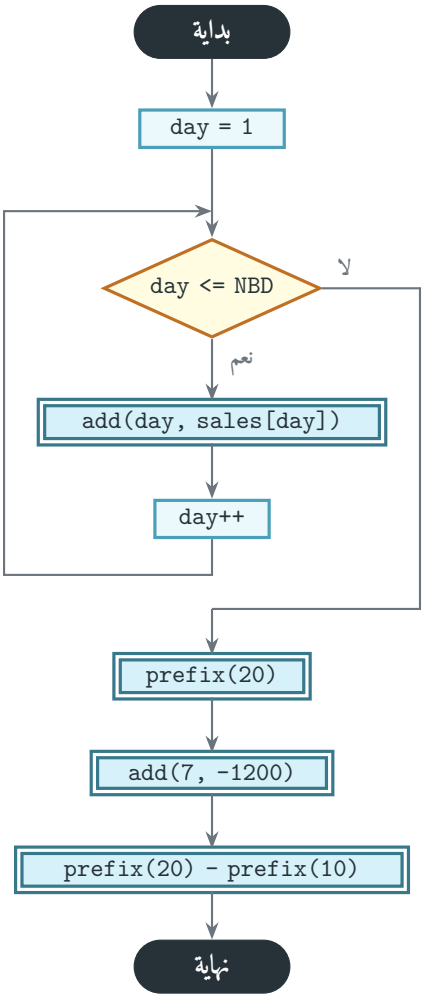
long prefix(int idx) {
    long sum = 0;
    for (; idx > 0; idx -= idx & -idx) { sum += tree[idx]; operations++; }         // نهبط
    return sum;
}

int main(void) {
    long sales[NBD + 1], total;
    int day;
    for (day = 1; day <= NBD; day++) {
        sales[day] = 1000 + (day * 389) % 900;
        add(day, sales[day]);
    }
    printf("build: %d operations\n", operations);
    operations = 0;
    total = prefix(20);                                  // نحسب قبل الطباعة
    printf("total days 1-20: %ld (%d ops, naive 20)\n", total, operations);
    operations = 0;
    add(7, -1200);                                       // مرتجع في اليوم 7
    printf("refund on day 7 applied (%d ops)\n", operations);
    operations = 0;
    total = prefix(20) - prefix(10);
    printf("days 11-20: %ld (%d ops, naive 10)\n", total, operations);
    printf("days 1-10 after refund: %ld\n", prefix(10));
    printf("tree[8] covers days 1-8: %ld\n", tree[8]);
    return 0;
}
```

نتيجة التنفيذ

```
build: 79 operations
total days 1-20: 28790 (2 ops, naive 20)
refund on day 7 applied (3 ops)
days 11-20: 14495 (4 ops, naive 10)
days 1-10 after refund: 13095
tree[8] covers days 1-8: 10004
```

المخطط الإنسيابي (Organigramme)



تمرين 13.50 بناء شجرة العبارة من صيغة نصية

تجارة إلكترونية صعب

حوّل صيغة سعر مكتوبة نصياً مثل `(4500-500)*3+400/2` إلى **شجرة عبارة** بخوارزمية **ساحة التحويل** (Shunting-yard) لدايكسترا: مكدس للعمليات ومكدس للشجيرات. ثم احسب قيمة الشجرة، واعرضها بالترميز **البعدي** (البولندي العكسي)، واعرض ارتفاعها.

خطوات الحل

1) عدد: ننشئ ورقة ونكدسها في مكدس الشجيرات.
2) عملية: ما دامت قمة مكدس العمليات أولويتها ≥ أولوية العملية الحالية: نطبقها (نسحب شجيرتين وننشئ عقدة).
3) `(` تُكدس، `)` تطبق العمليات حتى `(`.
4) في النهاية نطبق كل العمليات الباقية؛ الشجيرة الوحيدة هي الجذر.

البرنامج بلغة C

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Tree { char op; double value; struct Tree *left, *right; } Tree;

```

```c
Tree *nodes[50];
char ops[50];
int nodeTop = 0, opTop = 0;

int priority(char op) { return (op == '*' || op == '/') ? 2 : (op == '+' || op == '-') ? 1 : 0; }

void reduce(void) {
    Tree *node = malloc(sizeof(Tree));
    node->op = ops[--opTop];
    node->right = nodes[--nodeTop];                  // الترتيب مهم: الأيمن أولًا
    node->left = nodes[--nodeTop];
    node->value = 0;
    nodes[nodeTop++] = node;
}

double evaluate(const Tree *node) {
    double left, right;
    if (node->op == 0) return node->value;
    left = evaluate(node->left);
    right = evaluate(node->right);
    return node->op == '+' ? left + right : node->op == '-' ? left - right
         : node->op == '*' ? left * right : left / right;
}

void postfix(const Tree *node) {
    if (node->op == 0) { printf("%g ", node->value); return; }
    postfix(node->left);
    postfix(node->right);
    printf("%c ", node->op);
}

int height(const Tree *node) { int lh, rh; if (!node) return 0; lh = height(node->left); rh =
    ↪ height(node->right); return 1 + (lh > rh ? lh : rh); }
void freeTree(Tree *node) { if (node) { freeTree(node->left); freeTree(node->right); free(node); } }

int main(void) {
    char formula[100], ch;
    Tree *leaf;
    int pos;
    scanf("%99s", formula);
    for (pos = 0; formula[pos] != '\0'; pos++) {
        ch = formula[pos];
        if (ch >= '0' && ch <= '9') {
            leaf = malloc(sizeof(Tree));
            leaf->op = 0; leaf->left = leaf->right = NULL; leaf->value = 0;
            while (formula[pos] >= '0' && formula[pos] <= '9') leaf->value = leaf->value * 10 +
      ↪ (formula[pos++] - '0');
            pos--;
            nodes[nodeTop++] = leaf;
        } else if (ch == '(') {
            ops[opTop++] = ch;
        } else if (ch == ')') {
            while (ops[opTop - 1] != '(') reduce();
            opTop--;                                          // نحذف '('
        } else {
            while (opTop > 0 && priority(ops[opTop - 1]) >= priority(ch)) reduce();   // الأولوية
            ops[opTop++] = ch;
        }
    }
    while (opTop > 0) reduce();
    printf("formula: %s\npostfix: ", formula);
    postfix(nodes[0]);
    printf("\nvalue:   %.2f DA\ntree height: %d\n", evaluate(nodes[0]), height(nodes[0]));
    freeTree(nodes[0]);
    return 0;
}
```

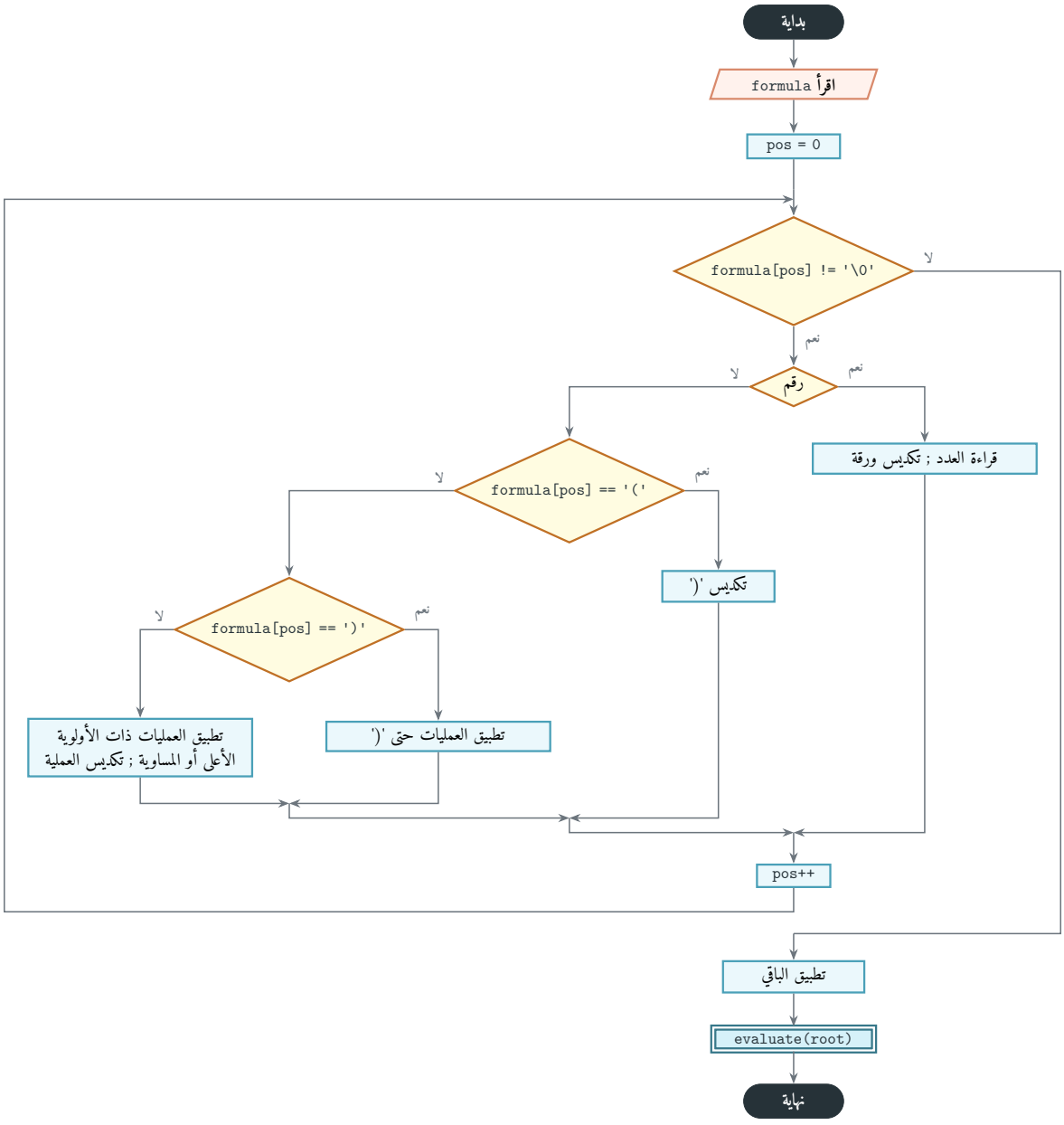
المدخلات

```
(4500-500)*3+400/2
```

نتيجة التنفيذ

```
formula: (4500-500)*3+400/2
postfix: 4500 500 - 3 * 400 2 / +
value:   12200.00 DA
tree height: 4
```

المخطط الإنسيابي (Organigramme)



المراجع

[1] B. W. Kernighan and D. M. Ritchie. *The C Programming Language*. 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988. ISBN 978-0-13-110362-7.

[2] K. N. King. *C Programming: A Modern Approach*. 2nd ed. New York: W. W. Norton, 2008. ISBN 978-0-393-97950-3.

[3] P. Deitel and H. Deitel. *C How to Program: With Case Studies in Applications and Systems Programming*. 9th ed. Hoboken, NJ: Pearson, 2022. ISBN 978-0-13-739839-3.

[4] International Organization for Standardization. *ISO/IEC 9899:1999 – Information technology – Programming languages – C*. Geneva: ISO, 1999.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*. 4th ed. Cambridge, MA: MIT Press, 2022. ISBN 978-0-262-04630-5.

[6] D. E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. 3rd ed. Reading, MA: Addison-Wesley, 1997. ISBN 978-0-201-89683-1.

[7] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd ed. Reading, MA: Addison-Wesley, 1998. ISBN 978-0-201-89685-5.

[8] R. Sedgewick. *Algorithms in C, Parts 1–4: Fundamentals, Data Structures, Sorting, Searching*. 3rd ed. Reading, MA: Addison-Wesley, 1998. ISBN 978-0-201-31452-6.

[9] N. Wirth. *Algorithms + Data Structures = Programs*. Englewood Cliffs, NJ: Prentice Hall, 1976. ISBN 978-0-13-022418-7.

[10] S. S. Skiena. *The Algorithm Design Manual*. 3rd ed. Texts in Computer Science. Cham: Springer, 2020. ISBN 978-3-030-54255-9.

[11] M. A. Weiss. *Data Structures and Algorithm Analysis in C*. 2nd ed. Menlo Park, CA: Addison-Wesley, 1997. ISBN 978-0-201-49840-0.

[12] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery. *Numerical Recipes in C: The Art of Scientific Computing*. 2nd ed. Cambridge: Cambridge University Press, 1992. ISBN 978-0-521-43108-8.

[13] G. James, D. Witten, T. Hastie and R. Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. 2nd ed. Springer Texts in Statistics. New York: Springer, 2021. ISBN 978-1-0716-1417-4.

تعلّم الخوارزميات بحلّ المسائل

يرافق هذا الكتاب طالب السنة الأولى في مقياس الخوارزميات ولغة C، من أول متغير إلى الأشجار الثنائية، عبر **650 تمرينًا محلولًا** مأخوذة من مسائل واقعية في **التجارة الإلكترونية** و**علم البيانات**.

- 13 فصلًا، في كل فصل 50 تمرينًا: 10 سهلة، 30 متوسطة، 10 صعبة.
- لكل تمرين: نص المسألة، خطوات الحل، برنامج C
  مشروح سطرًا بسطر، نتيجة التنفيذ الحقيقية، ومخطط انسيابي.
- كل الخوارزميات مكتوبة يدويًا، دون أي دالة جاهزة.
- برامج مترجمة ومختبرة بالمعيار C99.

د. بن عبد الرزاق يوسف
أستاذ مقياس الخوارزميات وهياكل المعطيات

ISBN .....